

Building Query Compilers
(Under Construction)
[expected time to completion: 5 years]

Guido Moerkotte

June 23, 2026

Contents

I	Basics	3
1	Introduction	5
1.1	General Remarks	5
1.2	DBMS Architecture	5
1.3	Interpretation versus Compilation	6
1.4	Requirements for a Query Compiler	9
1.5	Search Space	11
1.6	Generation versus Transformation	12
1.7	Focus	12
1.8	Organization of the Book	13
2	Textbook Query Optimization	15
2.1	Example Query and Outline	15
2.2	Algebra	16
2.3	Canonical Translation	17
2.4	Logical Query Optimization	20
2.5	Physical Query Optimization	24
2.6	Discussion	25
3	Join Ordering	31
3.1	Queries Considered	31
3.1.1	Query Graph	32
3.1.2	Join Tree	33
3.1.3	Simple Cost Functions	34
3.1.4	Classification of Join Ordering Problems	40
3.1.5	Search Space Sizes	41
3.1.6	Problem Complexity	45
3.2	Deterministic Algorithms	47
3.2.1	Heuristics	47
3.2.2	Determining the Optimal Join Order in Polynomial Time	49
3.2.3	The Maximum-Value-Precedence Algorithm	56
3.2.4	Dynamic Programming	61
3.2.5	Memoization	79
3.2.6	Join Ordering by Generating Permutations	79
3.2.7	A Dynamic Programming based Heuristics for Chain Queries	81
3.2.8	Transformation-Based Approaches	94

3.3	Probabilistic Algorithms	101
3.3.1	Generating Random Left-Deep Join Trees with Cross Products	101
3.3.2	Generating Random Join Trees with Cross Products	103
3.3.3	Generating Random Join Trees without Cross Products	107
3.3.4	Quick Pick	116
3.3.5	Iterative Improvement	116
3.3.6	Simulated Annealing	117
3.3.7	Tabu Search	118
3.3.8	Genetic Algorithms	119
3.4	Hybrid Algorithms	122
3.4.1	Two Phase Optimization	122
3.4.2	AB-Algorithm	122
3.4.3	Toured Simulated Annealing	122
3.4.4	GOO-II	123
3.4.5	Iterative Dynamic Programming	123
3.5	Ordering Order-Preserving Joins	123
3.6	Characterizing Search Spaces	131
3.6.1	Complexity Thresholds	131
3.7	Discussion	135
3.8	Bibliography	136
4	Database Items, Building Blocks, and Access Paths	137
4.1	Disk Drive	137
4.2	Database Buffer	146
4.3	Physical Database Organization	147
4.4	Slotted Page and Tuple Identifier (TID)	150
4.5	Physical Record Layouts	151
4.6	Physical Algebra (Iterator Concept)	152
4.7	Simple Scan	152
4.8	Scan and Attribute Access	153
4.9	Temporal Relations	155
4.10	Table Functions	155
4.11	Indexes	156
4.12	Single Index Access Path	158
4.12.1	Simple Key, No Data Attributes	158
4.12.2	Complex Keys and Data Attributes	163
4.13	Multi Index Access Path	165
4.14	Indexes and Joins	167
4.15	Remarks on Access Path Generation	172
4.16	Counting the Number of Accesses	172
4.16.1	Counting the Number of Direct Accesses	172
4.16.2	Counting the Number of Sequential Accesses	182
4.16.3	Pointers into the Literature	187
4.17	Disk Drive Costs for N Uniform Accesses	188
4.17.1	Number of Qualifying Cylinders, Tracks, and Sectors	188
4.17.2	Command Costs	189

4.17.3	Seek Costs	189
4.17.4	Settle Costs	191
4.17.5	Rotational Delay Costs	191
4.17.6	Head Switch Costs	193
4.17.7	Discussion	193
4.18	Concluding Remarks	194
4.19	Bibliography	194
II	Foundations	197
5	Logic, Null, and Boolean Expressions	199
5.1	Two-Valued Logic	199
5.2	Null Values	199
5.2.1	Functions and Operators	199
5.2.2	Comparison Operators	201
5.3	Three-Valued Logic	202
5.4	Preparation of Boolean Expressions	204
5.5	Equivalence Classes based on Equality	204
5.6	Nullability Inference	206
5.7	Bibliography	206
6	Functional Dependencies	207
6.1	Functional Dependencies	207
6.2	Functional Dependencies in the presence of NULL values	208
6.3	Deriving Functional Dependencies over algebraic operators	208
6.4	Bibliography	208
7	An Algebra for Sets, Bags, and Sequences	209
7.1	Sets, Bags, and Sequences	209
7.1.1	Sets	209
7.1.2	Duplicate Data: Bags	211
7.1.3	Explicit Duplicate Control	214
7.1.4	Ordered Data: Sequences	215
7.2	Aggregation Functions	216
7.3	Operators	220
7.3.1	Preliminaries	221
7.3.2	Signatures	223
7.3.3	Projection	225
7.3.4	Selection	226
7.3.5	Map	226
7.3.6	Unary Grouping	227
7.3.7	Unnest Operators	228
7.3.8	Flatten Operator	229
7.3.9	Join Operators	229
7.3.10	Groupjoin	230
7.3.11	Min/Max Operators	231

7.3.12	Other Dependent Operators	232
7.4	Linearity of Algebraic Operators	233
7.4.1	Linearity of Algebraic Operators	233
7.4.2	Exploiting Linearity	238
7.5	Representations	239
7.5.1	Three Different Representations	239
7.5.2	Conversion between Representations	241
7.5.3	Conversion between Bulk Types	241
7.5.4	Adjusting the Algebra	242
7.5.5	Partial Preaggregation	243
7.6	A Note on Equivalences	243
7.7	Simple Reorderability	244
7.7.1	Unary Operators	244
7.7.2	Push-Down/Pull-Up of Unary into/from Binary Operators	246
7.7.3	Binary Operators	248
7.8	Predicate Detachment and Attachment	253
7.9	Basic Equivalences for D-Join	255
7.10	Equivalences for Outerjoins	257
7.10.1	Outerjoin Simplification	264
7.10.2	Generalized Outerjoin	265
7.11	Equivalences for Unary Grouping	266
7.11.1	An Elementary Fact about Grouping	266
7.11.2	Join	267
7.11.3	Left Outerjoin	277
7.11.4	Left Outerjoin with Default	281
7.11.5	Full Outerjoin	282
7.11.6	D-Join	285
7.11.7	Groupjoin	286
7.11.8	Intersection and Difference	291
7.12	Eliminating Redundant Joins	292
7.13	Semijoin and Antijoin Reducer	293
7.14	Outerjoin Simplification	294
7.15	Correct and Complete Exploration of the Core Search Space . . .	294
7.15.1	The Core Search Space	294
7.15.2	Exploration	296
7.15.3	More Issues	304
7.16	Logical Algebra for Sequences	308
7.16.1	Introduction	308
7.16.2	Algebraic Operators	309
7.16.3	Equivalences	312
7.16.4	Bibliography	312
7.17	Literature	312
7.18	ToDo	313

8	Declarative Query Representation	315
8.1	Calculus Representations	315
8.2	Datalog	315
8.3	Tableaux Representation	315
8.4	Monoid Comprehension	315
8.5	Expressiveness	315
8.6	Bibliography	315
9	Translation and Lifting	317
9.1	Query Language to Calculus	317
9.2	Query Language to Algebra	317
9.3	Calculus to Algebra	317
9.4	Algebra to Calculus	317
9.5	Bibliography	317
10	Query Equivalence, Containment, Minimization, and Factorization	319
10.1	Set Semantics	320
10.1.1	Conjunctive Queries	320
10.1.2	... with Inequalities	322
10.1.3	... with Negation	323
10.1.4	... under Constraints	323
10.1.5	... with Aggregation	323
10.2	Bag Semantics	323
10.2.1	Conjunctive Queries	323
10.3	Sequences	324
10.3.1	Path Expressions	324
10.4	Minimization	325
10.5	Detecting common subexpressions	325
10.5.1	Simple Expressions	325
10.5.2	Algebraic Expressions	325
10.6	Bibliography	325
III	Rewrite Techniques	327
11	Simple Rewrites	329
11.1	Simple Adjustments	329
11.1.1	Rewriting Simple Expressions	329
11.1.2	Normal forms for queries with disjunction	331
11.2	Deriving new predicates	331
11.2.1	Collecting conjunctive predicates	331
11.2.2	Equality	331
11.2.3	Inequality	332
11.2.4	Aggregation	333
11.2.5	ToDo	335
11.3	Predicate Push-Down and Pull-Up	335

11.4	Eliminating Redundant Joins	335
11.5	Distinct Pull-Up and Push-Down	335
11.6	Set-Valued Attributes	335
11.6.1	Introduction	335
11.6.2	Preliminaries	336
11.6.3	Query Rewrite	337
11.7	Bibliography	338
12	View Merging	341
12.1	View Resolution	341
12.2	Simple View Merging	341
12.3	Predicate Move Around (Predicate pull-up and push-down) . . .	342
12.4	Complex View Merging	343
12.4.1	Views with Distinct	343
12.4.2	Views with Group-By and Aggregation	344
12.4.3	Views in IN predicates	345
12.4.4	Final Remarks	345
12.5	Bibliography	346
13	Quantifier treatment	347
13.1	Pseudo-Quantifiers	347
13.2	Existential quantifier	348
13.3	Universal quantifier	348
13.4	Bibliography	352
14	Unnesting Nested Queries	353
15	Optimizing Queries with Materialized Views	355
15.1	Conjunctive Views	355
15.2	Views with Grouping and Aggregation	355
15.3	Views with Disjunction	355
15.4	Bibliography	355
16	Semantic Query Rewrite	357
16.1	Constraints and their impact on query optimization	357
16.2	Semantic Query Rewrite	357
16.3	Exploiting Uniqueness in Query Optimization	358
16.4	Bibliography	358
IV	Plan Generation	359
17	Current Search Space and Its Limits	361
17.1	Plans with Outer Joins, Semijoins and Antijoins	361
17.2	Expensive Predicates and Functions	361
17.3	Techniques to Reduce the Search Space	361
17.4	Bibliography	361

18 Dynamic Programming-Based Plan Generation	363
18.1 Introduction	363
18.2 Hypergraphs	364
18.3 CCPs: Csg-Cmp-Pairs for Hypergraphs	365
18.4 Neighborhood	366
18.5 The CCP Enumerator BuEnumCppHyp	367
18.5.1 BuEnumCcpHyp	368
18.5.2 EnumerateCsgRec	369
18.5.3 EmitCsg	370
18.5.4 EnumerateCmpRec	371
18.5.5 EmitCsgCmp	371
18.5.6 Neighborhood Calculation	371
18.6 DPhyp	372
18.7 Adding Selections	372
18.8 Adding Maps	372
18.9 Adding Grouping	372
19 Optimizing Queries with Disjunctions	373
19.1 Introduction	373
19.2 Using Disjunctive or Conjunctive Normal Forms	374
19.3 Bypass Plans	374
19.4 Implementation remarks	376
19.5 Other plan generators/query optimizer	376
19.6 Bibliography	377
20 Generating Plans for the Full Algebra	379
21 Generating DAG-structured Plans	381
22 Simplifying the Query Graph	383
22.1 Introduction	383
22.2 On Optimizing Join Queries	384
22.3 Graph Simplification Algorithm	385
22.3.1 Simplifying the Query Graph	386
22.3.2 The Full Algorithm	388
22.3.3 Join Ordering Criterion	389
22.3.4 Theoretical Foundation	390
22.4 The Time/Quality Trade-Off	392
23 Deriving and Dealing with Interesting Orderings and Groupings	395
23.1 Introduction	395
23.2 Problem Definition	396
23.2.1 Ordering	396
23.2.2 Grouping	398
23.2.3 Functional Dependencies	399
23.2.4 Algebraic Operators	399

23.2.5	Plan Generation	400
23.3	Overview	401
23.4	Detailed Algorithm	404
23.4.1	Overview	404
23.4.2	Determining the Input	405
23.4.3	Constructing the NFSM	406
23.4.4	Constructing the DFSM	409
23.4.5	Precomputing Values	410
23.4.6	During Plan Generation	410
23.4.7	Reducing the Size of the NFSM	410
23.4.8	Complex Ordering Requirements	414
23.5	Experimental Results	415
23.6	Total Impact	415
23.7	Influence of Groupings	417
23.8	Annotated Bibliography	421
24	Cardinality and Cost Estimation	425
24.1	Introduction	425
24.2	A First Approach	428
24.2.1	Top-Most Cost Formula (Overall Costs)	428
24.2.2	Summation of Operator Costs	428
24.2.3	CPU Cost	429
24.2.4	Abbreviations	429
24.2.5	I/O Costs	429
24.2.6	Cardinality Estimates	431
24.3	The Simple Profile: A First Logical Profile and its Propagation	433
24.3.1	The Logical Profile	433
24.3.2	Assumptions	434
24.3.3	Profile Propagation for Selection	436
24.3.4	Profile Propagation for Join	442
24.3.5	Profile Propagation for Projection	443
24.3.6	Profile Propagation for Division	447
24.3.7	Remarks	448
24.4	Approximation of a Set of Values	449
24.4.1	Approximations and Error Metrics	449
24.4.2	Example Applications	450
24.5	Approximation with Linear Models	451
24.5.1	Linear Models	451
24.5.2	Example Applications	455
24.5.3	Linear Models Under l_2	462
24.5.4	Linear Models Under l_∞	467
24.5.5	Linear Models Under l_q	470
24.5.6	Non-Linear Models under l_q	477
24.5.7	Multidimensional Models under l_q	478
24.6	Traditional Histograms	479
24.6.1	Bucketization	480
24.6.2	Heuristics to Determine Bucket Boundaries	481

24.7	More on Q	482
24.7.1	Properties of the Q-Error	482
24.7.2	Properties of Estimation Functions	490
24.7.3	θ, q -Acceptability	491
24.7.4	Testing θ, q -Acceptability for Buckets	492
24.7.5	From Buckets To Histograms	495
24.7.6	Q-Compression	504
24.8	One Dimensional Synopses	507
24.8.1	Four Level Tree and Variants	507
24.8.2	Q-Histograms (Type I)	510
24.8.3	Q-Histogram (Type II)	510
24.9	Sketches For Counting The Number of Distinct Values	510
24.9.1	Linear Counting	512
24.9.2	DvByKMinVal	512
24.9.3	Logarithmic Counting	513
24.9.4	SuperLogLog Counting	514
24.9.5	HyperLogLog Counting	517
24.9.6	DvByMinAvg	517
24.9.7	DvByKMinAvg	518
24.9.8	Pointers to the Literature	519
24.10	Multidimensional Synopsis	519
24.10.1	Introductory Example	520
24.10.2	Solving the Introductory Problem without 2-Dimensional Synopsis	521
24.10.3	Statistical Views	522
24.10.4	Regular Partitioning: equi-width	523
24.10.5	Equi-Depth Histogram	523
24.10.6	2-Dimensional Synopsis based on SVD	523
24.10.7	PHASED	523
24.10.8	MHIST	523
24.10.9	GENHIST	523
24.10.10	HiRed	523
24.10.11	MI Histograms	523
24.10.12	Grid Trees	523
24.10.13	More	523
24.11	Iterative Selectivity Combination	524
24.12	Maximum Entropy	524
24.13	Selected Issues	525
24.13.1	Exploiting and Augmenting Existing DBMS Data Struc- tures	525
24.13.2	Sampling	528
24.13.3	Query Feedback	528
24.13.4	Combining Data Summaries with Sampling	528
24.13.5	Wavelets	528
24.13.6	Selectivity of String-Valued Attributes	528
24.14	Cost Functions	528
24.14.1	Disk-based Joins	528

24.14.2 Main Memory Joins	528
24.14.3 Additional Pointers to the Literature	528
V Implementation	531
25 Architecture of a Query Compiler	533
25.1 Compilation process	533
25.2 Architecture	533
25.3 Control Blocks	533
25.4 Memory Management	535
25.5 Tracing and Plan Visualization	535
25.6 Driver	535
25.7 Bibliography	535
26 Internal Representations	539
26.1 Requirements	539
26.2 Algebraic Representations	539
26.2.1 Graph Representations	540
26.2.2 Query Graph	540
26.2.3 Operator Graph	540
26.3 Query Graph Model (QGM)	540
26.4 Classification of Predicates	540
26.5 Treatment of Distinct	540
26.6 Query Analysis and Materialization of Analysis Results	540
26.7 Query and Plan Properties	541
26.8 Conversion to the Internal Representation	543
26.8.1 Preprocessing	543
26.8.2 Translation into the Internal Representation	543
26.9 Bibliography	543
27 Details on the Phases of Query Compilation	545
27.1 Parsing	545
27.2 Semantic Analysis, Normalization, Factorization, Constant Folding, and Translation	545
27.3 Normalization	547
27.4 Factorization	547
27.5 Constant Folding	548
27.6 Semantic analysis	548
27.7 Translation	550
27.8 Rewrite I	555
27.9 Plan Generation	555
27.10 Rewrite II	555
27.11 Code generation	555
27.12 Bibliography	556

28 Hard-Wired Algorithms	557
28.1 Hard-wired Dynamic Programming	557
28.1.1 Introduction	557
28.1.2 A plan generator for bushy trees	561
28.1.3 A plan generator for bushy trees and expensive selections	562
28.1.4 A plan generator for bushy trees, expensive selections and functions	562
28.2 Bibliography	562
29 Rule-Based Algorithms	565
29.1 Rule-based Dynamic Programming	565
29.2 Rule-based Memoization	565
29.3 Bibliography	565
30 Example Query Compiler	567
30.1 Research Prototypes	567
30.1.1 AQUA and COLA	567
30.1.2 Black Dahlia II	567
30.1.3 Epoq	567
30.1.4 Ereq	569
30.1.5 Exodus/Volcano/Cascade	570
30.1.6 Freytags regelbasierte System R-Emulation	572
30.1.7 Genesis	573
30.1.8 GOMbgo	575
30.1.9 Gral	578
30.1.10 Lambda-DB	581
30.1.11 Lancelotte in short	581
30.1.12 Opt++	582
30.1.13 Postgres	582
30.1.14 Sciore & Sieg	584
30.1.15 Secondo	584
30.1.16 Squiral	584
30.1.17 System R and System R*	586
30.1.18 Starburst and DB2	586
30.1.19 Der Optimierer von Straube	589
30.1.20 Other Query Optimizer	590
30.2 Commercial Query Compiler	592
30.2.1 The DB 2 Query Compiler	592
30.2.2 The Oracle Query Compiler	592
30.2.3 The SQL Server Query Compiler	596
VI Selected Topics	597
31 Generating Plans for Top-N-Queries?	599
31.1 Motivation and Introduction	599
31.2 Optimizing for the First Tuple	599

31.3 Optimizing for the First N Tuples	599
32 Recursive Queries	601
33 Issues Introduced by OQL	603
33.1 Type-Based Rewriting and Pointer Chasing Elimination	603
33.2 Class Hierarchies	605
33.3 Cardinalities and Cost Functions	607
34 Issues Introduced by XPath	609
34.1 A Naive XPath-Interpreter and its Problems	609
34.2 Dynamic Programming and Memoization	609
34.3 Naive Translation of XPath to Algebra	609
34.4 Pushing Duplicate Elimination	609
34.5 Avoiding Duplicate Work	609
34.6 Avoiding Duplicate Generation	609
34.7 Index Usage and Materialized Views	609
34.8 Cardinalities and Costs	609
34.9 Bibliography	609
35 Issues Introduced by XQuery	611
35.1 Reordering in Ordered Context	611
35.2 Result Construction	611
35.3 Unnesting Nested XQueries	611
35.4 Cardinalities and Cost Functions	611
35.5 Bibliography	611
36 Outlook	613
A Query Languages?	615
A.1 Designing a query language	615
A.2 SQL	615
A.3 OQL	615
A.4 XPath	615
A.5 XQuery	615
A.6 Datalog	615
B Query Execution Engine (?)	617
C Glossary of Rewrite and Optimization Techniques	619
D Useful Formulas	625
Bibliography	626
Index	695
E ToDo	697

List of Figures

1.1	DBMS architecture	6
1.2	Query interpreter	6
1.3	Simple query interpreter	7
1.4	Query compiler	7
1.5	Query compiler architecture	8
1.6	Execution plan	10
1.7	Potential and actual search space	12
1.8	Generation vs. transformation	13
2.1	Relational algebra	17
2.2	Equivalences for the relational algebra	18
2.3	(Simplified) Canonical translation of SQL to algebra	19
2.4	Text book query optimization	20
2.5	Logical query optimization	21
2.6	Different join trees	27
2.7	Plans for example query (Part I)	28
2.8	Plans for example query (Part II)	29
2.9	Physical query optimization	30
2.10	Plan for example query after physical query optimization	30
3.1	Query graph for example query of Section 2.1	32
3.2	Query graph shapes	33
3.3	Illustrations for the IKKBZ Algorithm	55
3.4	A query graph, its directed join graph, some spanning trees and join trees	56
3.5	A query graph, its directed join tree, a spanning tree and its problem	58
3.6	Search space with sharing under optimality principle	63
3.7	Algorithm DPsize	70
3.8	Algorithm DPsub	72
3.9	Size of the search space for different graph structures	74
3.10	Algorithm DPccp	75
3.11	Enumeration Example for DPccp	75
3.12	Sample graph to illustrate EnumerateCsgRec	77
3.13	Call sequence for Figure 3.12	77
3.14	Example of rule transformations (RS-1)	99

3.15	Encoding Trees	104
3.16	Paths	105
3.17	Tree-merge	108
3.18	Algorithm UnrankDecomposition	110
3.19	Leaf-insertion	110
3.20	A query graph, its tree, and its standard decomposition graph . .	111
3.21	Algorithm Adorn	114
3.22	A query graph, a join tree, and its encoding	121
3.23	Pseudo code for IDP-1	124
3.24	Pseudocode for IDP-2	125
3.25	Subroutine applicable-predicates	127
3.26	Subroutine construct-bushy-tree	128
3.27	Subroutine extract-plan and its subroutine	128
3.28	Impact of selectivity on the search space	134
3.29	Impact of relation sizes on the search space	134
3.30	Impact of parameters on the performance of heuristics	134
3.31	Impact of selectivities on probabilistic procedures	135
4.1	Disk drive assembly	138
4.2	Disk drive read request processing	139
4.3	Time to read 100 MB from disk (depending on the number of 8 KB blocks read at once)	142
4.4	Time needed to read n random pages	144
4.5	Break-even point in fraction of total pages depending on page size	145
4.6	Physical organization of a relational database	148
4.7	Slotted pages and TIDs	150
4.8	Various physical record layouts	151
4.9	Clustered vs. non-clustered index	157
4.10	Illustration of seek cost estimate	190
5.1	Truth tables for two-valued logic	199
5.2	Laws for two-valued logic	200
5.3	Comparison functions in the presence of NULL values	201
5.4	Truth tables for three-valued logic	202
5.5	True-/false-interpretation and Negation	203
7.1	Laws for Set Operations	210
7.2	Laws for Bag Operations	212
7.3	Decomposition of aggregate functions	219
7.4	Example for map and group operators	226
7.5	Three possible representations of a bag	240
7.6	Example for outerjoin reorderability (for strict q)	258
7.7	Example for outerjoin reorderability (for non-strict q')	259
7.8	Example for outerjoin reorderability (for partially non-strict q') .	259
7.9	Example for outerjoin associativity for strict q	260
7.10	Example for outerjoin associativity for non-strict q'	261
7.11	Example for outerjoin l-asscom for strict q	261

7.12	Example for grouping and join	269
7.13	Extended example for grouping and join	270
7.14	Example for Eqv. 7.118	274
7.15	Example relations	289
7.16	Join results	290
7.17	Left- and right-hand sides	290
7.18	Transformation rules for assoc, l-asscom, and r-asscom	294
7.19	Core search space example	295
7.20	The complete search space	296
7.21	Algorithm DPsube	297
7.22	Calculating TES for simple operator trees	300
7.23	Example showing the incompleteness of CD-A	301
7.24	Calculating conflict rules for simple operator trees	302
7.25	Example showing the incompleteness of CD-B	303
7.26	Conflict detection for unary and binary operators	305
7.27	Example for Map Operator	309
7.28	Examples for unary grouping and the groupjoin	311
11.1	Simplification rules for boolean expressions	332
11.2	Axioms for equality	332
11.3		339
11.4	Axioms for inequality	340
18.1	Sample hypergraph	364
18.2	Trace of algorithm for Figure ??	369
18.3	Pseudocode for calcNeighborhood	372
19.1	DNF plans	374
19.2	CNF plans	375
19.3	Bypass plans	375
22.1	Runtimes for Different Query Graphs	385
22.2	Exemplary Simplification Steps for a Star Query	386
22.3	Pseudo-Code for a Single Simplification Step	387
22.4	The Full Optimization Algorithm	389
22.5	The Effect of Simplification Steps for a Star Query with 20 Re- lations	393
22.6	The Effect of Simplification Steps for a Grid Query with 20 Re- lations	394
23.1	Propagation of orderings and groupings	400
23.2	Possible FSM for orderings	402
23.3	Possible FSM for groupings	403
23.4	Combined FSM for orderings and groupings	403
23.5	Possible DFSM for Figure 23.4	403
23.6	Preparation steps of the algorithm	405
23.7	Initial NFSM for sample query	407
23.8	NFSM after adding D_{FD} edges	408

23.9 NFSM after pruning artificial states	408
23.10Final NFSM	408
23.11Resulting DFSM	409
23.12 <i>contains</i> Matrix	409
23.13 <i>transition</i> Matrix	409
23.14Plan generation for different join graphs, Simmen's algorithm (left) vs. our algorithm (middle)	415
23.15Memory consumption in KB for Figure 23.14	417
23.16Time requirements for the preparation step	420
23.17Space requirements for the preparation step	421
24.1 Overview of operations for cardinality and cost estimations . . .	426
24.2 Sample for range query result estimation under CVA and ESA. .	437
24.3 Calculating the lower bound $D_G^\perp$	446
24.4 Calculating the estimate for D_G	447
24.5 Example frequency density and cumulated frequency	457
24.6 Cumulated frequency and its approximation	458
24.7 Q-error and plan optimality	462
24.8 Algorithm for best linear approximation under l_∞	471
24.9 Algorithm finding best linear approximation under l_q	476
24.10Sample data sets	479
24.11Q-compression, $\log_b$ -based	504
24.12Binary Q-compression	506
24.13FLT example 1	507
24.14FLT example 2	509
24.15Car database example	512
24.16Linear Counting	513
24.17Algorithm DvByKMinVal	513
24.18Algorithm <i>LogarithmicCounting</i>	514
24.19Algorithm PCSA	515
24.20Filling M for LogLogCounting, SuperLogLogCounting, and Hy- perLogLogCounting	516
24.21SuperLogLog Counting	516
24.22Calculation of $\tilde{a}$	517
24.23HyperLogLog Counting	518
24.24DvByMinAvg	519
24.25DvByKMinAvg	520
24.26Example for Equi-Depth Tree	523
24.27Sample B ⁺ -Tree	525
25.1 The compilation process	534
25.2 Class Architecture of the Query Compiler	536
25.3 Control Block Structure	537
27.1 Expression	546
27.2 Expression hierarchy	547
27.3 Expression	548

27.4 Query 1	549
27.5 Internal representation	551
27.6 An algebraic operator tree with a d-join	554
27.7 Algebra	555
28.1 A sample execution plan	558
28.2 Different join operator trees	559
28.3 Bottom up plan generation	561
28.4 A Dynamic Programming Optimization Algorithm	563
30.1 Beispiel einer Epoq-Architektur	568
30.2 Exodus Optimierer Generator	570
30.3 Organisation der Optimierung	573
30.4 Ablauf der Optimierung	576
30.5 Architektur von GOMrbo	577
30.6 a) Architektur des Gral-Optimierers; b) Operatorhierarchie nach Kosten	578
30.7 Die Squiralarchitektur	585
30.8 Starburst Optimierer	587
30.9 Der Optimierer von Straube	589
33.1 Algebraic representation of a query	603
33.2 A join replacing pointer chasing	605
33.3 A Sample Class Hierarchy	606
33.4 Implementation of Extents	607

Preface

Goals

Primary Goals:

- book covers many query languages (at least SQL, OQL, XQuery (XPath))
- techniques should be represented as query language independent as possible
- book covers all stages of the query compilation process
- book completely covers fundamental issues
- book gives implementation details and tricks

Secondary Goals:

- book is thin
- book is not totally unreadable
- book separates concepts from implementation techniques

Organizing the material is not easy: The same topic pops up

- at different stages of the query compilation process and
- at different query languages

Acknowledgements

Introducer to query optimization: Günther von Bültzingsloewen

Peter Lockemann

First paper coauthor: Stefan Karl,

Coworkers: Alfons Kemper, Klaus Peithner, Michael Steinbrunn, Donald

Kossmann, Carsten Gerlhof, Jens Claussen,

Sophie Cluet, Vassilis Christophides, Georg Gottlob, V.S. Subramanian,

Sven Helmer, Birgitta König-Ries, Wolfgang Scheufele, Carl-Christian Kanne,

Thomas Neumann, Norman May, Matthias Brantner

Robin Aly

Discussions: Umesh Dayal, Dave Maier, Gail Mitchell, Stan Zdonik, Tamer Özsu, Arne Rosenthal,

Don Chamberlin, Bruce Lindsay, Guy Lohman, Mike Carey, Bennet Vance, Laura Haas, Mohan, CM Park,

Yannis Ioannidis, Götz Graefe, Serge Abiteboul, Claude Delobel Patrick Valduriez, Dana Florescu, Jerome Simeon, Mary Fernandez, Christoph Koch, Adam Bosworth, Joe Hellerstein, Paul Larson, Hennie Steenhagen, Harald Schönig, Bernhard Seeger,

Encouragement: Anand Deshpande

Manuscript: Simone Seeger,

and many others to be inserted.

Part I

Basics

Chapter 1

Introduction

1.1 General Remarks

Query languages like SQL or OQL are declarative. That is, they specify what the user wants to retrieve and not how to retrieve it. It is the task of the query compiler to generate a *query evaluation plan* (*evaluation plan* for short, or *execution plan* or simply *plan*) for a given query. The query evaluation plan (QEP) essentially is an operator tree with physical algebraic operators as nodes. It is evaluated by the runtime system. Figure 1.6 shows a detailed execution plan ready to be interpreted by the runtime system. Figure 28.1 shows an abstraction of a query plan often used to explain algorithms or optimization techniques.

The book tries to demystify query optimization and query optimizers. By means of the multi-lingual query optimizer BD II, the most important aspects of query optimizers and their implementation are discussed. We concentrate not only on the *query optimizer* core (Rewrite I, Plan Generator, Rewrite II) of the query compilation process but touch on all issues from parsing to code generation and quality assurance.

We start by giving a two-module overview of a database management system. One of these modules covers the functionality of the query compiler. The query compiler itself involves several submodules. For each submodule we discuss the features relevant for query compilation.

1.2 DBMS Architecture

Any database management system (DBMS) can be divided into two major parts: the compile time system (CTS) and the runtime system (RTS). User commands enter the compile time system which translates them into executables which are then interpreted by the runtime system (Fig. 1.1).

The input to the CTS are statements of several kinds, for example connect to a database (starts a session), disconnect from a database, create a database, drop a database, add/drop a schema, perform schema changes (add relations, object types, constraints, ...), add/drop indexes, run statistics commands, manually modify the DBMS statistics, begin of a transaction, end of a transac-

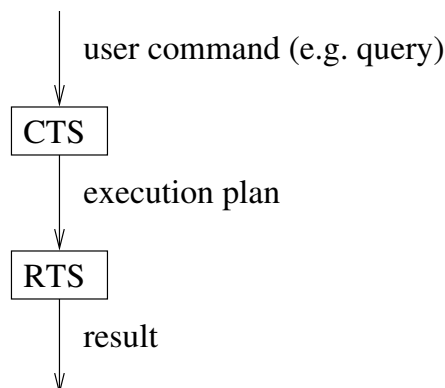


Figure 1.1: DBMS architecture

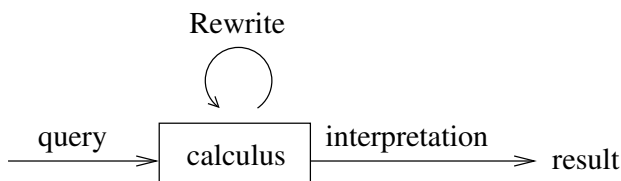


Figure 1.2: Query interpreter

tion, add/drop a view, update database items (e.g. tuples, relations, objects), change authorizations, and state a query. Within the book, we will only be concerned with the tiny last item.

1.3 Interpretation versus Compilation

There are two essential approaches to process a query: *interpretation* and *compilation*.

The path of a query through a query interpreter is illustrated in Figure 1.2. Query interpretation translates the query string into some internal representation that is mostly calculus-based. Optionally, some rewrite on this representation takes place. Typical steps during this rewrite phase are unnesting nested queries, pushing selections down, and introducing index structures. After that, the query is interpreted. A simple query interpreter is sketched in Figure 1.3. The first function, **interpret**, takes a simple SQL block and extracts the different clauses, initializes the result **R** and calls **eval**. Then, **eval** recursively evaluates the query by first producing the cross product of the entries in the **from** clause. After all of them have been processed, the predicate is applied and for those tuples where the **where** predicate evaluates to true, a result tuple is constructed and added to the result set **R**. Obviously, the sketched interpreter is far from being efficient. A much better approach has been described by Wong and Youssefi [931, 962].

Let us now discuss the compilation approach. The different steps are sum-

```

interprete(SQLBlock x) {

    /* possible rewrites go here */
    s := x.select();
    f := x.from();
    w := x.where();
    R := ∅; /* result */
    t := []; /* empty tuple */
    eval(s, f, w, t, R);
    return R;
}

eval(s, f, w, t, R) {

    if(f.empty())
        if(w(t))
            R += s(t);
    else
        foreach(t' ∈ first(f))
            eval(s, tail(f), w, t ∘ t', R);
}

```

Figure 1.3: Simple query interpreter

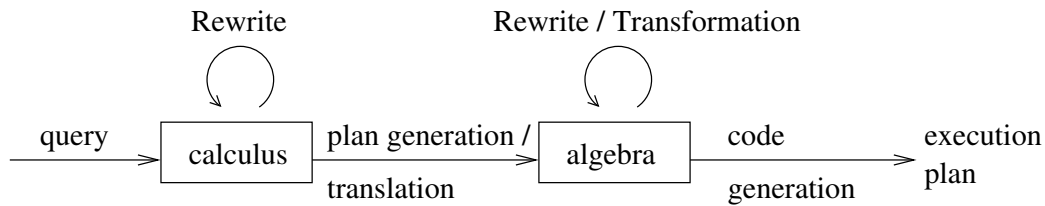


Figure 1.4: Query compiler

marized in Figure 1.4. First, the query is rewritten. Again, unnesting nested queries is a main technique for performance gains. Other rewrites will be discussed in Part ???. After the rewrite, the plan generation takes place. Here, an optimal plan is constructed. Whereas typically rewrite takes place on a calculus-based representation of the query, plan generation constructs an algebraic expression containing well-known operators like selection and join. Sometimes, after plan generation, the generated plan is refined: some polishing takes place. Then, code is generated, that can be interpreted by the runtime system. More specifically, the query execution engine—a part of the runtime system—interpretes the query execution plan. Let us illustrate this. The following query

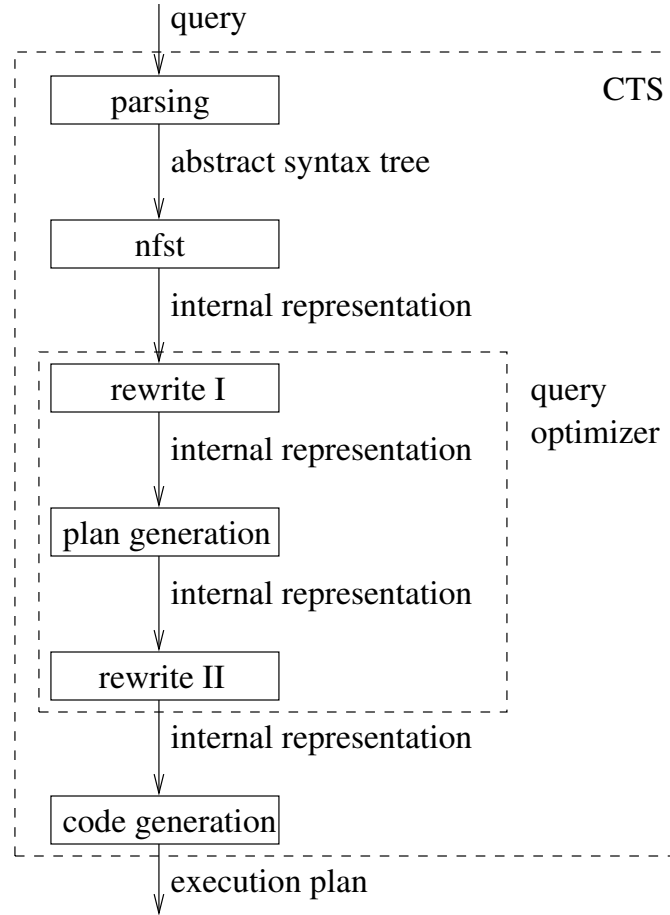


Figure 1.5: Query compiler architecture

is Query 1 of the now obsolete TPC-D benchmark [878].

```

SELECT    RETURNFLAG, LINESTATUS,
            SUM(QUANTITY) as SUM_QTY,
            SUM(EXTENDEDPRICE) as SUM_EXTPR,
            SUM(EXTENDEDPRICE * (1 - DISCOUNT)),
            SUM(EXTENDEDPRICE * (1 - DISCOUNT)*
              (1 + TAX)),
            AVG(QUANTITY),
            AVG(EXTENDEDPRICE),
            AVG(DISCOUNT),
            COUNT(*)
FROM      LINEITEM
WHERE      SHIPDDATE <= DATE '1998-12-01'
GROUP BY RETURNFLAG, LINESTATUS
ORDER BY RETURNFLAG, LINESTATUS
  
```

The CTS translates this query into a query execution plan. Part of the plan is shown in Fig. 1.6. One rarely sees a query execution plan. This is the reason why I included one. But note that the form of query execution plans differs from DBMS to DBMS since it is (unfortunately) not standardized the way SQL is. Most DBMSs can give the user abstract representations of query plans. It is worth the time to look at the plans generated by some commercial DBMSs.

I do not expect the reader to understand the plan in all details. Some of these details will become clear later. Anyway, this plan is given to the RTS which then interprets it. Part of the result of the interpretation might look like this:

RETURNFLAG	LINESTATUS	SUM_QTY	SUM_EXTPR	...
A	F	3773034	5319329289.68	...
N	F	100245	141459686.10	...
N	O	7464940	10518546073.98	...
R	F	3779140	5328886172.99	...

This should look familiar to you.

The above query plan is very simple. It contains only a few algebraic operators. Usually, more algebraic operators are present and the plan is given in a more abstract form that cannot be directly executed by the runtime system. Fig. 2.10 gives an example of an abstracted more complex operator tree. We will work with representations closer to this one.

A typical query compiler architecture is shown in Figure 1.5. The first component is the parser. It produces an abstract syntax tree. This is not always the case but this intermediate representation very much simplifies the task of following component. The NFST component performs several tasks. The first step is normalization. This mainly deals with introducing new variables for subexpressions. Factorization and semantic analysis are performed during NFST. Last, the abstract syntax tree is translated into the internal representation. All these steps can typically be performed during a single path through the query representation. Semantic analysis requires looking up schema definitions. This can be expensive and, hence, the number of lookups should be minimized. After NFST the core optimization steps rewrite I and plan generation take place. Rewrite II does some polishing before code generation. These modules directly correspond to the phases in Figure 1.4. They are typically further divided into submodules handling subphases. The most prominent example is the preparation phase that takes place just before the actual plan generation takes place. In our figures, we think of preparation as being part of the plan generation.

1.4 Requirements for a Query Compiler

Here are the main requirements for a query compiler:

1. Correctness
2. Completeness
3. Generate optimal plan (viz avoid the worst case)

```

(group
  (tbscan
    {segment 'lineitem.C4Kseg' 0 4096}
    {nalslottedpage 4096}
    {ctuple 'lineitem.cschema'}
    [ 20
      LOAD_PTR 1
      LOAD_SC1_C 8 1 2 // L_RETURNFLAG
      LOAD_SC1_C 9 1 3 // L_LINESTATUS
      LOAD_DAT_C 10 1 4 // L_SHIPDATE
      LEQ_DAT_ZC_C 4 '1998-02-09' 1
    ] 2 1 // number of help-registers and selection-register
  ) 10 22 // hash table size, number of registers
  [ // init
    MV_UI4_C_C 1 0 // COUNT(*) = 0
    LOAD_SF8_C 4 1 6 // L_QUANTITY
    LOAD_SF8_C 5 1 7 // L_EXTENDEDPRICE
    LOAD_SF8_C 6 1 8 // L_DISCOUNT
    LOAD_SF8_C 7 1 9 // L_TAX
    MV_SF8_Z_C 6 10 // SUM/AVG(L_QUANTITY)
    MV_SF8_Z_C 7 11 // SUM/AVG(L_EXTENDEDPRICE)
    MV_SF8_Z_C 8 12 // AVG(L_DISCOUNT)
    SUB_SF8_CZ_C 1.0 8 13 // 1 - L_DISCOUNT
    ADD_SF8_CZ_C 1.0 9 14 // 1 + L_TAX
    MUL_SF8_ZZ_C 7 13 15 // SUM(L_EXTDPRICE * (1 - L_DISC))
    MUL_SF8_ZZ_C 15 14 16 // SUM((...) * (1 + L_TAX))
  ] [ // advance
    INC_UI4 0 // inc COUNT(*)
    MV_PTR_Y 1 1
    LOAD_SF8_C 4 1 6 // L_QUANTITY
    LOAD_SF8_C 5 1 7 // L_EXTENDEDPRICE
    LOAD_SF8_C 6 1 8 // L_DISCOUNT
    LOAD_SF8_C 7 1 9 // L_TAX
    MV_SF8_Z_A 6 10 // SUM/AVG(L_QUANTITY)
    MV_SF8_Z_A 7 11 // SUM/AVG(L_EXTENDEDPRICE)
    MV_SF8_Z_A 8 12 // AVG(L_DISCOUNT)
    SUB_SF8_CZ_C 1.0 8 13 // 1 - L_DISCOUNT
    ADD_SF8_CZ_C 1.0 9 14 // 1 + L_TAX
    MUL_SF8_ZZ_B 7 13 17 15 // SUM(L_EXTDPRICE * (1 - L_DISC))
    MUL_SF8_ZZ_A 17 14 16 // SUM((...) * (1 + L_TAX))
  ] [ // finalize
    UIFC_C 0 18
    DIV_SF8_ZZ_C 10 18 19 // AVG(L_QUANTITY)
    DIV_SF8_ZZ_C 11 18 20 // AVG(L_EXTENDEDPRICE)
    DIV_SF8_ZZ_C 12 18 21 // AVG(L_DISCOUNT)
  ] [ // hash program
    HASH_SC1 2 HASH_SC1 3
  ] [ // compare program
    CMPA_SC1_ZY_C 2 2 0
    EXIT_NEQ 0
    CMPA_SC1_ZY_C 3 3 0
  ]
)

```

Figure 1.6: Execution plan

4. Efficiency, generate the plan fast, do not waste memory
5. Graceful degradation

6. Robustness

First of all, the query compiler must produce correct query evaluation plans. That is, the result of the query evaluation plan must be the result of the query as given by the specification of the query language. It must also cover the complete query language. The next issue is that an optimal query plan must (should) be generated. However, this is not always that easy. That is why some database researchers say that one must avoid the worst plan. Talking about the quality of a plan requires us to fix the optimization goal. Several goals are reasonable: We can optimize throughput, minimize response time, minimize resource consumption (both, memory and CPU), and so on. A good query compiler supports two optimization goals: minimize resource consumption and minimize the time to produce the first tuple. Obviously, both goals cannot be achieved at the same time. Hence, the query compiler must be instructed about the optimization goal.

Irrespective of the optimization goal, the query compiler should produce the query evaluation plan fast. It does not make sense to take 10 seconds to optimize a query whose execution time is below a second. This sounds reasonable but is not trivial to achieve. As we will see, the number of query execution plans that are equivalent to a given query, i.e. produce the same result as the query, can be very large. Sometimes, very large even means that not all plans can be considered. Taking the wrong approach to plan generation will result in no plan at all. This is the contrary of graceful degradation. Expressed positively, graceful degradation means that in case of limited resources, a plan is generated that may not be the optimal plan, but also not that far away from the optimal plan.

Last, typical software quality criteria should be met. We only mentioned robustness in our list, but others like maintainability must be met also.

1.5 Search Space

For a given query, there typically exists a high number of plans that are equivalent to the plan. Not all of these plans are accessible. Only those plans that can be generated by known optimization techniques (mainly algebraic equivalences) can potentially be generated. Since this number may still be too large, many query compilers restrict their search space further. We call the search space explored by a query optimizer the *actual search space*. The *potential search space* is the set of all plans that are known to be equivalent to the given query by applying the state of the art of query optimization techniques. The whole set of plans equivalent to a given query is typically unknown: we are not sure whether all optimization techniques have been discovered so far. Figure 1.7 illustrates the situation. Note that we run into problems if the actual search space is not a subset of the equivalent plans. Then the query compiler produces wrong results. As we will see in several chapters of this book, some optimization techniques have been proposed that produce plans that are not equivalent to the original query.

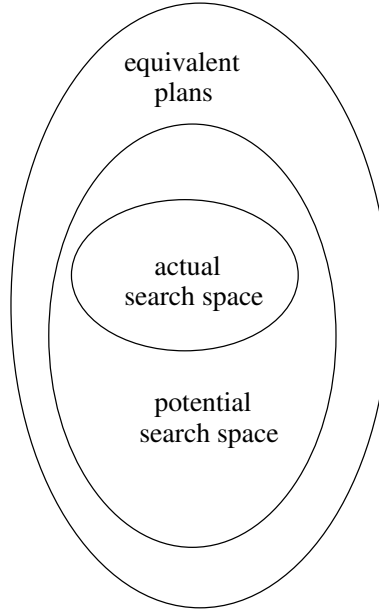


Figure 1.7: Potential and actual search space

1.6 Generation versus Transformation

Two different approaches to plan generation can be distinguished:

- The transformation-based approach transforms one query execution plan into another equivalent one. This can, for example, happen by applying an algebraic equivalence to a query execution plan in order to yield a better plan.
- The generic or synthetic approach produces a query execution plan by assembling building blocks and adding one algebraic operator after the other, until a complete query execution plan has been produced. Note that in this approach only when all building blocks and algebraic operators have been introduced the internal representation can be executed. Before that, no (complete) plan exists.

For an illustration see Figure 1.8.

A very important issue is how to explore the search space. Several well-known approaches exist: A*, Branch-and-bound, greedy algorithms, hill-climbing, dynamic programming, memoization, [209, 530, 531, 682]. These form the basis for most of the plan generation algorithms.

1.7 Focus

In this book, we consider only the compilation of queries. We leave out many special aspects like query optimization for multi-media database systems or

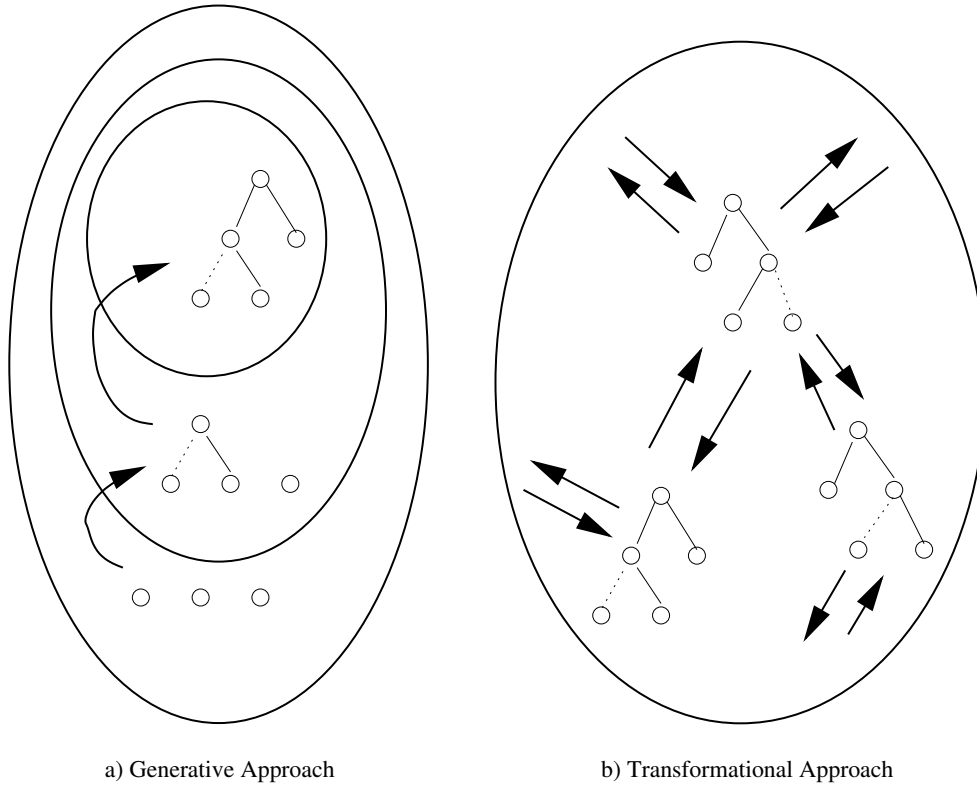


Figure 1.8: Generation vs. transformation

multidatabase systems. These are just two omissions. We further do not consider the translation of update statements which — especially in the presence of triggers — can become quite complex. Furthermore, we assume the reader to be familiar with the fundamentals of database systems [264, 484, 650, 709, 816] and their implementation [403, 316]. Especially, knowledge on query execution engines is required [347].

Last, the book presents a very personal view on query optimization. To see other views on the same topic, I strongly recommend to read the literature cited in this book and the references found therein. A good start are overview articles, PhD theses, and books, e.g. [902, 322, 445, 446, 468, 543] [610, 613, 662, 830, 850, 886, 887].

1.8 Organization of the Book

The first part of the book is an introduction to the topic. It should give an idea about the breadth and depth of query optimization. We first recapitulate query optimization the way it is described in numerous text books on database systems. There should be nothing really new here except for some pitfalls we will point out. The Chapter 3 is devoted to the join ordering problem. This has several reasons. First of all, at least one of the algorithms presented in

this chapter forms the core of every plan generator. The second reason is that this problem allows to discuss some issues like search space sizes and problem complexities. The third reason is that we do not have to delve into details. We can stick to very simple (you might call them unrealistic) cost functions, do not have to concern ourselves with details of the runtime system and the like. Expressed positively, we can concentrate on some algorithmic aspects of the problem. In Chapter 4 we do the opposite. The reader will not find any advanced algorithms in this chapter but plenty of details on disks and cost functions. The goal of the rest of the book is then to bring these issues together, broaden the scope of the chapters, and treat problems not even touched by them. The main issue not touched is query rewrite.

Chapter 2

Textbook Query Optimization

Almost every introductory textbook on database systems contains a section on query optimization (or at least query processing) [264, 484, 650, 709, 816]. Also, the two existing books on implementing database systems contain a section on query optimization [403, 316]. In this chapter we give an excerpt¹ of these sections and subsequently discuss the problems with the described approach. The bottom line will be that these descriptions of query optimization capture the essence of it but contain pitfalls that need to be pointed out and gaps to be filled.

2.1 Example Query and Outline

We use the following relations for our example query:

Student(SNo, SName, SAge, SYear)
Attend(ASNo, ALNo, AGrade)
Lecture(LNo, LTitle, LPNo)
Professor(PNo, PName)

Those attributes belonging to the key of the relations have been underlined.

With the following query we ask for all students attending a lecture by a Professor called “*Larson*”.

```
select  distinct s.SName
from    Student s, Attend a, Lecture l, Professor p
where   s.SNo = a.ASNo and a.ALNo = l.LNo
        and l.LPNo = p.PNo and p.PName = 'Larson'
```

The outline of the rest of the chapter is as follows. A query is typically translated into an algebraic expression. Hence, we first review the relational algebra and then discuss the translation process. Thereafter, we present the two phases of textbook query optimization: logical and physical query optimization. A brief discussion follows.

¹We do not claim to be fair to the above mentioned sections.

2.2 Algebra

Let us briefly recall the standard definition of the most important algebraic operators. Their inputs are relations, that is sets of tuples. Sets do not contain duplicates. The attributes of the tuples are assumed to be simple (non-decomposable) values. The most common algebraic operators are defined in Fig. 2.1. Although the common set operations union ($\cup$), intersection ($\cap$), and setdifference ($\setminus$) belong to the relational algebra, we did not list them. Remember that $\cup$ and $\cap$ are both commutative and associative. $\setminus$ is neither of them. Further, for $\cup$ and $\cap$, two distributivity laws hold. However, since these operations are not used in this section, we refer to Figure 7.1 in Section 7.1.1.

Before we can understand Figure 2.1, we must clarify some terms and notations. For us, a *tuple* is a mapping from a set of attribute names (or attributes for short) to their corresponding values. These values are taken from certain domains. An actual tuple is denoted embraced by brackets. They include a comma-separated list of the form attribute name, column and attribute value as in `[name: 'Anton', age: 2]`. If we have two tuples with different attribute names, they can be concatenated, i.e. we can take the union of their attributes. *Tuple concatenation* is denoted by $\circ$. For example `[name: 'Anton', age: 2]  $\circ$  [toy: 'digger']` results in `[name: 'Anton', age: 2, toy: 'digger']`. Let A and A' be two sets of attributes where $A' \subseteq A$ holds. Further let t a tuple with schema A . Then, we can project t on the attributes in A' (written as $t.A$). The resulting tuple contains only the attributes in A' ; others are discarded. For example, if t is the tuple `[name: 'Anton', age: 2, toy: 'digger']` and $A = \{\text{age}\}$, then $t.A$ is the tuple `[age: 2]`.

A relation is a set of tuples with the same attributes. The schema of a relation is the set of attributes. For a relation R this is sometimes denoted by $\text{sch}(R)$, the *schema* of R . We denote it by $\mathcal{A}(R)$ and extend it to any algebraic expression producing a set of tuples. That is, $\mathcal{A}(e)$ for any algebraic expression is the set of attributes the resulting relation defines. Consider the predicate `age = 2` where `age` is an attribute name. Then, `age` behaves like a free variable that must be bound to some value before the predicate can be evaluated. This motivates us to often use the terms *attribute* and *variable* synonymously. In the above predicate, we would call `age` a free variable. The set of free variables of an expression e is denoted by $\mathcal{F}(e)$.

Sometimes it is useful to work with sequences of attributes in comparison predicates. Let $A = \langle a_1, \dots, a_k \rangle$ and $B = \langle b_1, \dots, b_k \rangle$ be two attribute sequences. Then for any comparison operator $\theta \in \{=, \leq, <, \geq, >, \neq\}$, the expression $A \theta B$ abbreviates $a_1 \theta b_1 \wedge a_2 \theta b_2 \wedge \dots \wedge a_k \theta b_k$.

Often, a *natural join* is defined. Consider two relations R_1 and R_2 . Define $A_i := \mathcal{A}(R_i)$ for $i \in \{1, 2\}$, and $A := A_1 \cap A_2$. Assume that A is non-empty and $A = \langle a_1, \dots, a_n \rangle$. If A is non-empty, the natural join is defined as

$$R_1 \bowtie R_2 := \Pi_{A_1 \cup A_2}(R_1 \bowtie_p \rho_{A:A'}(R_2))$$

where $\rho_{A:A'}$ renames the attributes a_i in A to a'_i in A' and the predicate p has the form $A = A'$, i.e. $a_1 = a'_1 \wedge \dots \wedge a_n = a'_n$.

$$\begin{aligned}
\sigma_p(R) &:= \{r \mid r \in R, p(r)\} \\
\Pi_A(R) &:= \{r.A \mid r \in R\} \\
R_1 \times R_2 &:= \{r_1 \circ r_2 \mid r_1 \in R_1, r_2 \in R_2\} \\
R_1 \bowtie_p R_2 &:= \sigma_p(R_1 \times R_2)
\end{aligned}$$

Figure 2.1: Relational algebra

For our algebraic operators, several equivalences hold. They are given in Figure 2.2. For them to hold, we typically require that the relations involved have disjoint attribute sets. That is, we assume—even for the rest of the book—that attribute names are unique. This is often achieved by using the notation $R.a$ for a relation R or $v.a$ for a variable ranging over tuples with an attribute a . Another possibility is to use the renaming operator ρ .

Some equivalences are not always valid. Their validity depends on whether some condition(s) are satisfied or not. For example, Eqv. 2.4 requires $\mathcal{F}(p) \subseteq A$. That is, all attribute names occurring in p must be contained in the attribute set A the projection retains: otherwise, we could not evaluate p after the projection has been applied. Although all conditions in Fig. 2.2 are of this flavor, we will see throughout the course of the book that more complex conditions exist.

2.3 Canonical Translation

The next question is how to translate a given SQL query into the algebra. Since the relational algebra works on sets and not bags (multisets), we can only translate SQL queries that contain a **distinct**. Further, we restrict ourselves EX to flat queries not containing any subquery. Since negation, disjunction, aggregation, and quantifiers pose further problems, we neglect them. Further, we do not allow **group by**, **order by**, **union**, **intersection**, and **except** in our query. Last, we allow only attributes in the **select** clause and not more complex expressions.

Thus, the generic SQL query pattern we can translate into the algebra looks as follows:

```

select distinct  $a_1, a_2, \dots, a_m$ 
from            $R_1c_1, R_2c_2, \dots, R_nc_n$ 
where          $p$ 

```

Here, the R_i are relation names and the c_i are correlation names. The a_i in the **select** clause are attribute names (or expressions of the form $c_i.a_i$) taken from the relations in the **from** clause. The predicate p is assumed to be a conjunction of comparisons between attributes or attributes and constants.

The translation process then follows the procedure described in Figure 2.3. First, we construct an expression that produces the cross product of the entries

$$\sigma_{p_1 \wedge \dots \wedge p_k}(R) \equiv \sigma_{p_1}(\dots(\sigma_{p_k}(R))\dots) \quad (2.1)$$

$$\sigma_{p_1}(\sigma_{p_2}(R)) \equiv \sigma_{p_2}(\sigma_{p_1}(R)) \quad (2.2)$$

$$\begin{aligned} \Pi_{A_1}(\Pi_{A_2}(\dots(\Pi_{A_k}(R))\dots)) &\equiv \Pi_{A_1}(R) \\ &\text{if } A_i \subseteq A_j \text{ for } i < j \end{aligned} \quad (2.3)$$

$$\begin{aligned} \Pi_A(\sigma_p(R)) &\equiv \sigma_p(\Pi_A(R)) \\ &\text{if } \mathcal{F}(p) \subseteq A \end{aligned} \quad (2.4)$$

$$(R_1 \times R_2) \times R_3 \equiv R_1 \times (R_2 \times R_3) \quad (2.5)$$

$$\begin{aligned} (R_1 \bowtie_{p_{1,2}} R_2) \bowtie_{p_{2,3}} R_3 &\equiv R_1 \bowtie_{p_{1,2}} (R_2 \bowtie_{p_{2,3}} R_3) \\ &\text{if } \mathcal{F}(p_{1,2}) \subseteq \mathcal{A}(R_1) \cup \mathcal{A}(R_2) \\ &\text{and } \mathcal{F}(p_{2,3}) \subseteq \mathcal{A}(R_2) \cup \mathcal{A}(R_3) \end{aligned} \quad (2.6)$$

$$R_1 \times R_2 \equiv R_2 \times R_1 \quad (2.7)$$

$$R_1 \bowtie_p R_2 \equiv R_2 \bowtie_p R_1 \quad (2.8)$$

$$\begin{aligned} \sigma_p(R_1 \times R_2) &\equiv \sigma_p(R_1) \times R_2 \\ &\text{if } \mathcal{F}(p) \subseteq \mathcal{A}(R_1) \end{aligned} \quad (2.9)$$

$$\begin{aligned} \sigma_p(R_1 \bowtie_q R_2) &\equiv \sigma_p(R_1) \bowtie_q R_2 \\ &\text{if } \mathcal{F}(p) \subseteq \mathcal{A}(R_1) \end{aligned} \quad (2.10)$$

$$\begin{aligned} \Pi_A(R_1 \times R_2) &\equiv \Pi_{A_1}(R_1) \times \Pi_{A_2}(R_2) \\ &\text{if } A = A_1 \cup A_2, A_i \subseteq \mathcal{A}(R_i) \end{aligned} \quad (2.11)$$

$$\begin{aligned} \Pi_A(R_1 \bowtie_p R_2) &\equiv \Pi_{A_1}(R_1) \bowtie_p \Pi_{A_2}(R_2) \\ &\text{if } \mathcal{F}(p) \subseteq A, A = A_1 \cup A_2, \\ &\text{and } A_i \subseteq \mathcal{A}(R_i) \end{aligned} \quad (2.12)$$

$$\begin{aligned} \sigma_p(R_1 \theta R_2) &\equiv \sigma_p(R_1) \theta \sigma_p(R_2) \\ &\text{where } \theta \text{ is any of } \cup, \cap, \setminus \end{aligned} \quad (2.13)$$

$$\Pi_A(R_1 \cup R_2) \equiv \Pi_A(R_1) \cup \Pi_A(R_2) \quad (2.14)$$

$$\sigma_p(R_1 \times R_2) \equiv R_1 \bowtie_p R_2 \quad (2.15)$$

Figure 2.2: Equivalences for the relational algebra

found in the **from** clause. The result is

$$((\dots((R_1 \times R_2) \times R_3) \dots) \times R_n).$$

Next, we add a selection with the **where** predicate:

$$\sigma_p((\dots((R_1 \times R_2) \times R_3) \dots) \times R_n).$$

Last, we project on the attributes found in the **select** clause.

$$\Pi_{a_1, \dots, a_n}(\sigma_p((\dots((R_1 \times R_2) \times R_3) \dots) \times R_n)).$$

For our example query

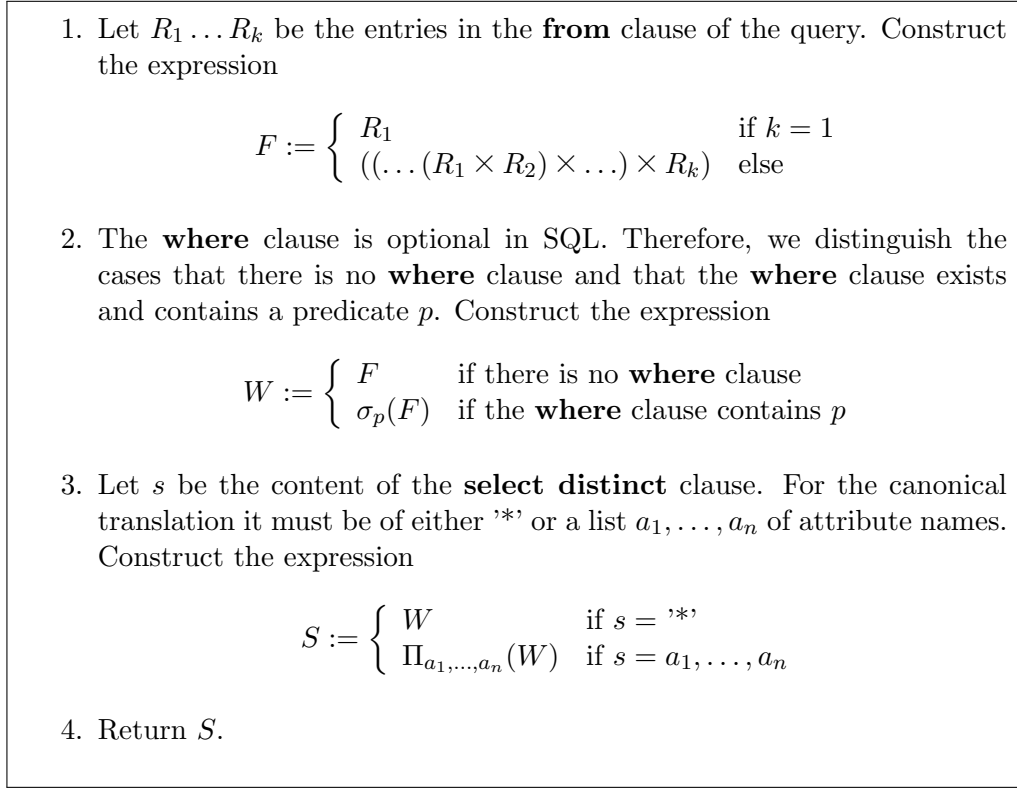


Figure 2.3: (Simplified) Canonical translation of SQL to algebra

```

select  distinct s.SName
from    Student s, Attend a, Lecture l, Professor p
where   s.SNo = a.ASNo and a.ALNo = l.LNo
          and l.LPNo = p.PNo and p.PName = 'Larson'

```

the result of the translation is

$$\Pi_{s.SName}(\sigma_p(((Student[s] \times Attend[a]) \times Lecture[l]) \times Professor[p]))$$

where p equals

$$s.SNo = a.ASNo \text{ **and** } a.ALNo = l.LNo \text{ **and** } l.LPNo = p.PNo \text{ **and** } p.PName = \text{'Larson'}.$$

Note that we used the notation $R[r]$ to say that a relation named R has the correlation name r . During the course of the book we will be more precise about the semantics of this notation and it will deviate from the one suggested here. We will take r as a variable successively bound to the elements (tuples) in R . However, for the purpose of this chapter it is sufficient to think of it as associating a correlation name with a relation. The query is represented graphically in Figure 2.7 (top).

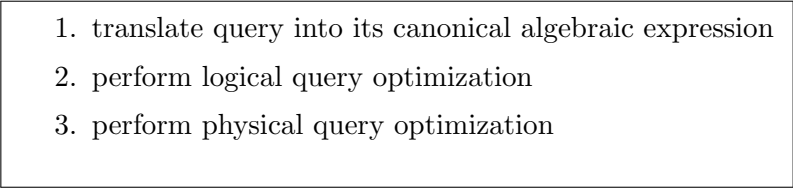
- 
1. translate query into its canonical algebraic expression
 2. perform logical query optimization
 3. perform physical query optimization

Figure 2.4: Text book query optimization

2.4 Logical Query Optimization

Textbook query optimization takes place in two separate phases. The first phase is called *logical query optimization* and the second *physical query optimization*. Figure 2.4 lists all these steps together with the translation step. In this section we discuss logical query optimization. The foundation for this step is formed by the set of algebraic equivalences (see Figure 2.2). The set of algebraic equivalences spans the potential search space for this step. Given an initial algebraic expression—resulting from the translation of the given query—the algebraic equivalences can be used to derive all algebraic expressions that are equivalent to the initial algebraic expression. This set of all equivalent algebraic expressions can be derived by applying the equivalences first to the initial expression and then to all derived expressions until no new expression is derivable. Thereby, the algebraic equivalences can be applied in both directions: from left to right and from right to left. Care has to be taken that the conditions attached to the equivalences are obeyed.

Of course, whenever we find a new algebraic equivalence that could not be derived from those already known, adding this equivalence increases our potential search space. On the one hand, this has the advantage that in a larger search space we may find better plans. On the other hand, it increases the already large search space which might cause problems for its exploration. Nevertheless, finding new equivalences is a well-established sport among database researchers.

One remark on *better* plans. Plans can only be compared if costs can be attached to them via some cost function. This is what happens in most industrial strength query optimizers. However, at the level of logical algebraic expressions adding precise costs is not possible: too many implementation details are missing. These are added to the plan during the next phase called *physical query optimization*. As a consequence, we are left with plans without costs. The only thing we can do is to *heuristically* judge the effectiveness of applying an equivalence from left to right or in the opposite direction. As always with heuristics, the hope is that they work for most queries. However, it is typically very easy to find counter examples where the heuristics do not result in the best plan possible. (Again, *best* with respect to some metrics.) This finding can be generalized: any query optimization that takes place in more than a single phase risks missing the best plan. This is an important observation and we will come back to this issue more than once.

After these words of warning let us continue to discuss textbook query

1. break up conjunctive selection predicates
(Eqv. 2.1: $\rightarrow$)
2. push down selections
(Eqv. 2.2: $\rightarrow$), (Eqv. 2.9: $\rightarrow$)
3. introduce joins
(Eqv. 2.15: $\rightarrow$)
4. determine join order
Eqv. 2.8, Eqv. 2.6, Eqv. 2.5, Eqv. 2.7
5. introduce and push down projections
(Eqv. 2.3: $\leftarrow$), (Eqv. 2.4: $\rightarrow$),
(Eqv. 2.11: $\rightarrow$), (Eqv. 2.12: $\rightarrow$)

Figure 2.5: Logical query optimization

optimization. Logical query optimization requires the organization of all equivalences into groups. Further, the equivalences are directed. That is, it is fixed whether they are applied in a left to right or right to left manner. A *directed* equivalence is called *rewrite rule*. The groups of rewrite rules are then successively applied to the initial algebraic expression. Figure 2.5 describes the different steps performed during logical query optimization. Associated with each step is a set of rewrite rules that are applied to the input expression to yield a result expression. The numbers correspond to the equivalences in Figure 2.2. A small arrow indicates the direction in which the equivalences are applied.

The first step breaks up conjunctive selection predicates. The motivation behind this step is that selections with simple predicates can be moved around easier. The rewrite rule used in this step is Equivalence 2.1 applied from left to right. For our example query Step 1 results in

$$\begin{aligned} &\Pi_{s.SName}(\sigma_{s.SNo=a.ASNo}(\sigma_{a.ALNo=l.LNo}(\sigma_{l.LPNo=p.PNo}(\sigma_{p.PName='Larson'}(\\ &\quad ((Student[s] \times Attend[a]) \times Lecture[l]) \times Professor[p]))))) \end{aligned}$$

The query is represented graphically in Figure 2.7 (middle).

Step 2 pushes selections down the operator tree. The motivation here is to reduce the number of tuples as early as possible such that subsequent (expensive) operators have smaller input. Applying this step to our example query yields:

$$\Pi_{s.SName}(\sigma_{s.SNo=a.ASNo}(\sigma_{a.ALNo=l.LNo}(\sigma_{l.LPNo=p.PNo}(\sigma_{p.PName='Larson'}($$

$$\begin{aligned}
&\sigma_{l.LPNo=p.PNo} (\\
&\quad \sigma_{a.ALNo=l.LNo} (\\
&\quad \quad \sigma_{s.SNo=a.ASNo} (Student[s] \times Attend[a]) \\
&\quad \quad \times Lecture[l]) \\
&\quad \times (\sigma_{p.PName='Larson'} (Professor[p])))
\end{aligned}$$

The query is represented graphically in Figure 2.7 (bottom).

Excursion In general, we might encounter problems when pushing down selections. It may be the case that the order of the cross products is not well-suited for pushing selections down. If this is the case, we must consider reordering cross products during this step (Eqv. 2.7 and 2.5). To illustrate this point consider the following example query.

```

select  distinct s.SName
from    Student s, Lecture l, Attend a
where   s.SNo = a.ASNo and a.ALNo = l.LNo
          and l.LTitle = 'Databases I'

```

After translation and Steps 1 and 2 the algebraic expression looks like

$$\begin{aligned}
&\Pi_{s.SName} (\\
&\quad \sigma_{s.SNo=a.ASNo} (\\
&\quad \quad \sigma_{a.ALNo=l.LNo} (\\
&\quad \quad \quad (Student[s] \times \sigma_{l.LTitle='Databases I'} (Lecture[l])) \times Attend[a]))).
\end{aligned}$$

Neither of $\sigma_{s.SNo=a.ASNo}$ and $\sigma_{a.ALNo=l.LNo}$ can be pushed down further. Only after reordering the cross products such as in

$$\begin{aligned}
&\Pi_{s.SName} (\\
&\quad \sigma_{s.SNo=a.ASNo} (\\
&\quad \quad \sigma_{a.ALNo=l.LNo} (\\
&\quad \quad \quad (Student[s] \times Attend[a]) \times \sigma_{l.LTitle='Databases I'} (Lecture[l])))
\end{aligned}$$

can $\sigma_{s.SNo=a.ASNo}$ be pushed down:

$$\begin{aligned}
&\Pi_{s.SName} (\\
&\quad \sigma_{a.ALNo=l.LNo} (\\
&\quad \quad \sigma_{s.SNo=a.ASNo} (Student[s] \times Attend[a]) \\
&\quad \quad \times \sigma_{l.LTitle='Databases I'} (Lecture[l]))
\end{aligned}$$

This is the reason why in some textbooks reorder cross products before selections are pushed down [264]. In this approach, reordering of cross products takes into account the selection predicates that can possibly be pushed down to the leaves and down to just prior a cross product. In any case, the Steps 2 and 4 are highly interdependent and there is no simple solution. $\square$

After this small excursion let us resume rewriting our main example query. The next step to be applied is converting cross products to join operations (Step 3). The motivation behind this step is that the evaluation of cross products is very expensive and results in huge intermediate results. For every tuple in

the left input an output tuple must be produced for every tuple in the right input. A join operation can be implemented much more efficiently. Applying Equivalence 2.15 from left to right to our example query results in

$$\begin{aligned} &\Pi_{s.SName}(\ \\ &\quad ((Student[s] \bowtie_{s.SNo=a.ASNo} Attend[a]) \\ &\quad \bowtie_{a.ALNo=l.LNo} Lecture[l]) \\ &\quad \bowtie_{l.LPNo=p.PNo} (\sigma_{p.PName='Larson'}(Professor[p]))) \end{aligned}$$

The query is represented graphically in Figure 2.8 (top).

The next step is really tricky and involved: we have to find an optimal order for evaluating the joins. The join's associativity and commutativity gives us plenty of alternative (equivalent) evaluation plans. For our rather simple query Figure 2.6 lists some of the possible join orders where we left out the join predicates and used the single letter correlation names to denote the relations to be joined. Only p abbreviates the more complex expression $\sigma_{p.PName='Larson'}(Professor[p])$. The edges show how plans can be derived from other plans by applying commutativity (c) or associativity (a).

Unfortunately, we cannot ignore the problem of finding a good join order. It has been shown that the order in which joins are evaluated has an enormous influence on the total evaluation cost of a query. Thus, it is an important problem. On the other hand, the problem is really tough. Most join ordering problems turn out to be NP-hard. As a consequence, many different heuristics and cost-based algorithms have been invented. They are discussed in depth in Chapter 3. There we will also find examples showing how important (in terms of costs) the right choice of the join order is.

To continue with our example query, we use a very simple heuristics: among all possible joins select the one first that produces the smallest intermediate result. This can be motivated as follows. In our current algebraic expression, the first join to be executed is

$$Student[s] \bowtie_{s.SNo=a.ASNo} Attend[a].$$

All students and their attendances to some lecture are considered. The result and hence the input to the next join will be very big. On the other hand, if there is only one professor named *Larson*, the output of $\sigma_{p.PName='Larson'}(Professor[p])$ is a single tuple. Joining this single tuple with the relation **Lecture** results in an output containing one tuple for every lecture taught by *Larson*. For a large university, this will be a small subset of all lectures. Continuing this line, we get the following algebraic expression:

$$\begin{aligned} &\Pi_{s.SName}(\ \\ &\quad ((\sigma_{p.PName='Larson'}(Professor[p]) \\ &\quad \bowtie_{p.PNo=l.LPNo} Lecture[l]) \\ &\quad \bowtie_{l.LNo=a.ALNo} Attend[a]) \\ &\quad \bowtie_{a.ASNo=s.SNo} Student[s]) \end{aligned}$$

The query is represented graphically in Figure 2.8 (middle).

The last step minimizes intermediate results by projecting out irrelevant attributes. An attribute is irrelevant, if it is not used further up the operator tree. When pushing down projections, we only apply them just before a pipeline breaker [347]. The reason is that for pipelined operators like selection, eliminating superfluous attributes does not gain much. The only pipeline breaker occurring in our plan is the join operator. Hence, before a join is applied, we project on the attributes that are further needed. The result is

$$\begin{aligned}
& \Pi_{s.SName} (\\
& \quad \Pi_{a.ASNo} (\\
& \quad \quad \Pi_{l.LNo} (\\
& \quad \quad \quad \Pi_{p.PNo} (\sigma_{p.PName='Larson'}(Professor[p])) \\
& \quad \quad \quad \bowtie_{p.PNo=l.LPNo} \\
& \quad \quad \quad \Pi_{l.LPNo,l.LNo}(Lecture[l])) \\
& \quad \quad \bowtie_{l.LNo=a.ALNo} \\
& \quad \quad \Pi_{a.ALNo,a.ASNo}(Attend[a])) \\
& \quad \bowtie_{a.ASNo=s.SNo} \\
& \quad \Pi_{s.SNo,s.SName}(Student[s]))
\end{aligned}$$

This expression is represented graphically in Figure 2.8 (bottom).

2.5 Physical Query Optimization

Physical query optimization adds more information to the logical query evaluation plan. First, there exist many different ways to access the data stored in a database. One possibility is to scan a relation to find the relevant tuples. Another alternative is to use an index to access only the relevant parts. If an unclustered index is used, it might be beneficial to sort the *tuple identifiers* (TIDs²) to turn otherwise random disk accesses into sequential accesses. Since there is a multitude of possibilities to access data, this topic is discussed in depth in Chapter 4. Second, the algebraic operators used in the logical plan may have different alternative implementations. The most prominent example is the join operator that has many different implementations: simple nested loop join, blockwise nested loop join, blockwise nested loop join with in-memory hash table, index nested loop join, hybrid hash join, sort merge join, bandwidth join, special spatial joins, set joins, and structural joins. Most of these join implementations can be applied only in certain situations. Most algorithms only implement equi-joins where the join predicate is a conjunction of simple equalities. Further, all the implementations differ in cost and robustness. But also other operators like grouping may have alternative implementations. Typically, for these operators exist sort-based and hash-based alternatives. Third, some operators require certain *properties* for their input streams. For example, a sort merge join requires its input to be sorted on the join attributes occurring in the equalities of the join predicate. These attributes are called *join attributes*. The sortedness property can be *enforced* by a sort operator. The sort operator

²Sometimes TIDs are called RIDs (Row Identifiers).

is thus an *enforcer* since it makes sure that the required property holds. As we will see, properties and enforcers play a crucial role during plan generation.

If common subexpressions are detected at the algebraic level, it might be beneficial to compute them only once and store the result. To do so, a *tmp* operator must be introduced. Later on, we will see more of these operators that materialize (partial) intermediate results in order to avoid the same computation to be performed more than once. An alternative is to allow QEPs which are DAGs and not merely trees (see Section ??).

Physical query optimization is concerned with all the issues mentioned above. The outline of it is given in Figure 2.9. Let us demonstrate this for our small example query. Let us assume that there exists an index on the name of the professors. Then, instead of scanning the whole professor relation, it is beneficial to use the index to retrieve only those professors named *Larson*. Further, since a sort merge join is very robust and not the slowest alternative, we choose it as an implementation for all our join operations. This requires that we sort the inputs to the join operator on the join attributes. Since sorting is a pipeline breaker, we introduce it between the projections and the joins. The resulting plan is

$$\begin{aligned}
 & \Pi_{s.SName} (\\
 & \quad \text{Sort}_{a.ASNo} (\Pi_{a.ASNo} (\\
 & \quad \quad \text{Sort}_{l.LNo} (\Pi_{l.LNo} (\\
 & \quad \quad \quad \text{Sort}_{p.PNo} (\Pi_{p.PNo} (\text{IdxScan}_{p.PName='Larson'} (\text{Professor}[p]))) \\
 & \quad \quad \quad \bowtie_{p.PNo=l.LPNo}^{smj} \\
 & \quad \quad \quad \text{Sort}_{l.LPNo} (\Pi_{l.LPNo,l.LNo} (\text{Lecture}[l]))) \\
 & \quad \quad \quad \bowtie_{l.LNo=a.ALNo}^{smj} \\
 & \quad \quad \quad \text{Sort}_{a.ALNo} (\Pi_{a.ALNo,a.ASNo} (\text{Attend}[a])))) \\
 & \quad \quad \quad \bowtie_{a.ASNo=s.SNo}^{smj} \\
 & \quad \text{Sort}_{s.SNo} (\Pi_{s.SNo,s.SName} (\text{Student}[s]))))
 \end{aligned}$$

where we annotated the joins with *smj* to indicate that they are sort merge joins. The *sort* operator has the attributes on which to sort as a subscript. We cheated a little bit with the notation of the index scan. The index is a physical entity stored in the database. An index scan typically allows to retrieve the TIDs of the tuples qualifying the predicate. If this is the case, another access to the relation itself is necessary to fetch the relevant attributes (*p.PNo* in our case) from the qualifying tuples of the relation. This issue is rectified in Chapter 4. The plan is shown as an operator graph in Figure 2.10.

2.6 Discussion

This chapter left open many interesting issues. We took it for granted that the presentation of a query is an algebraic expression or operator tree. Is this really true? We have been very vague about ordering joins and cross products. We only considered queries of the form **select distinct**. How can we assure correct duplicate treatment for **select all**? We separated query optimization into two distinct phases: logical and physical query optimization. Any separation into

different phases results in the danger of not producing an optimal plan. Logical query optimization turned out to be a little difficult: pushing selections down and reordering joins are mutually interdependent. How can we integrate these steps into a single one and thereby avoid the problem mentioned? Further, our logical query optimization was not cost based and cannot be: too much information is still missing from the plan to associate precise costs with a logical algebraic expression. How can we integrate the phases? How can we determine the costs of a plan? We covered only a small fraction of SQL. We did not discuss disjunction, negation, union, intersection, except, aggregate functions, group-by, order-by, quantifiers, outer joins, and nested queries. Furthermore, how about other query languages like OQL, XPath, XQuery? Further, enhancements like materialized views exist nowadays in many commercial systems. How can we exploit them beneficially? Can we exploit semantic information? Is our exploitation of index structures complete? What happens if we encounter NULL-values? Many questions and open issues remain. The rest of the book is about filling these gaps.

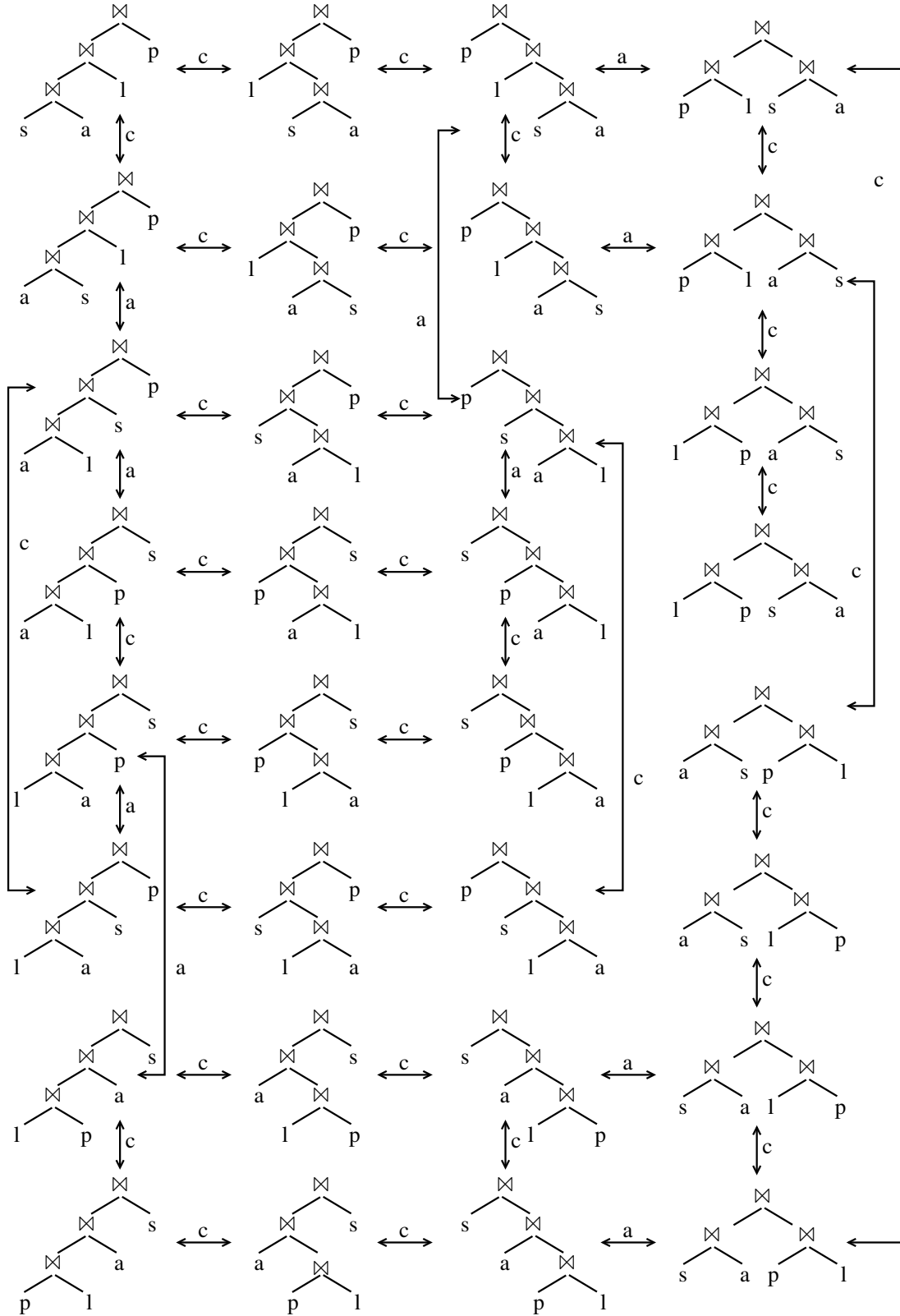


Figure 2.6: Different join trees

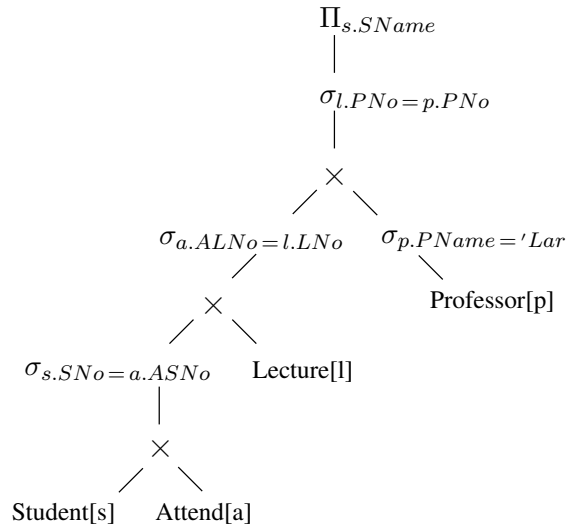
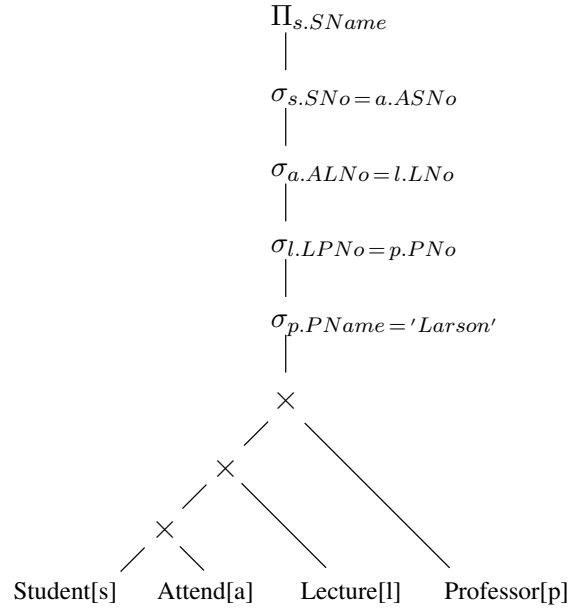
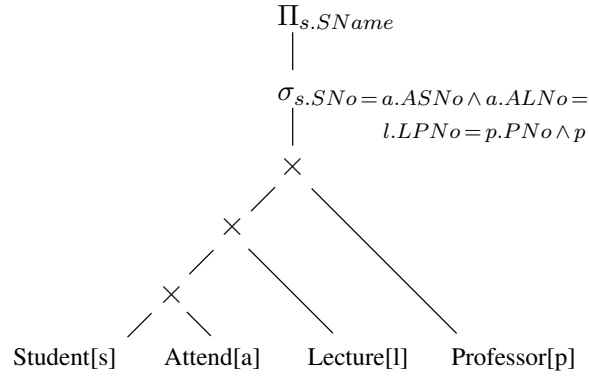


Figure 2.7: Plans for example query (Part I)

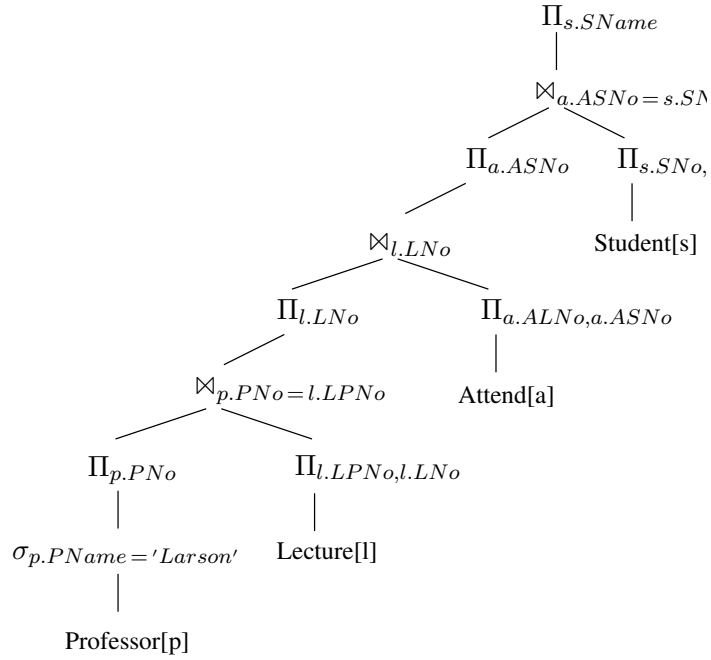
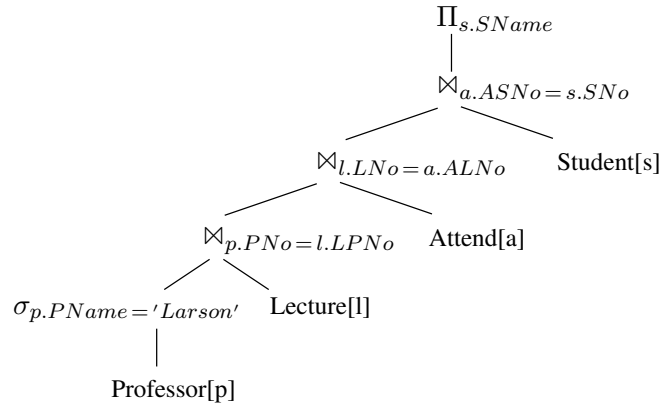
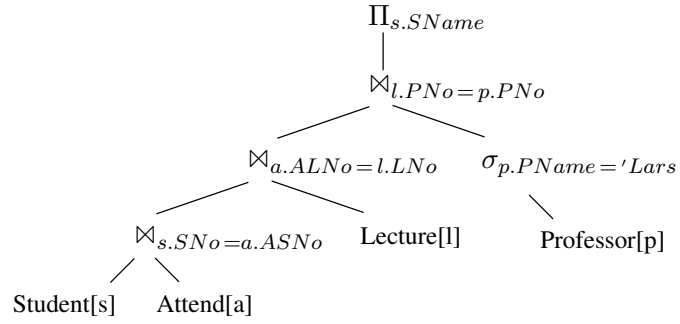


Figure 2.8: Plans for example query (Part II)

1. introduce index accesses
2. choose implementations for algebraic operators
3. introduce physical operators (sort, tmp)

Figure 2.9: Physical query optimization

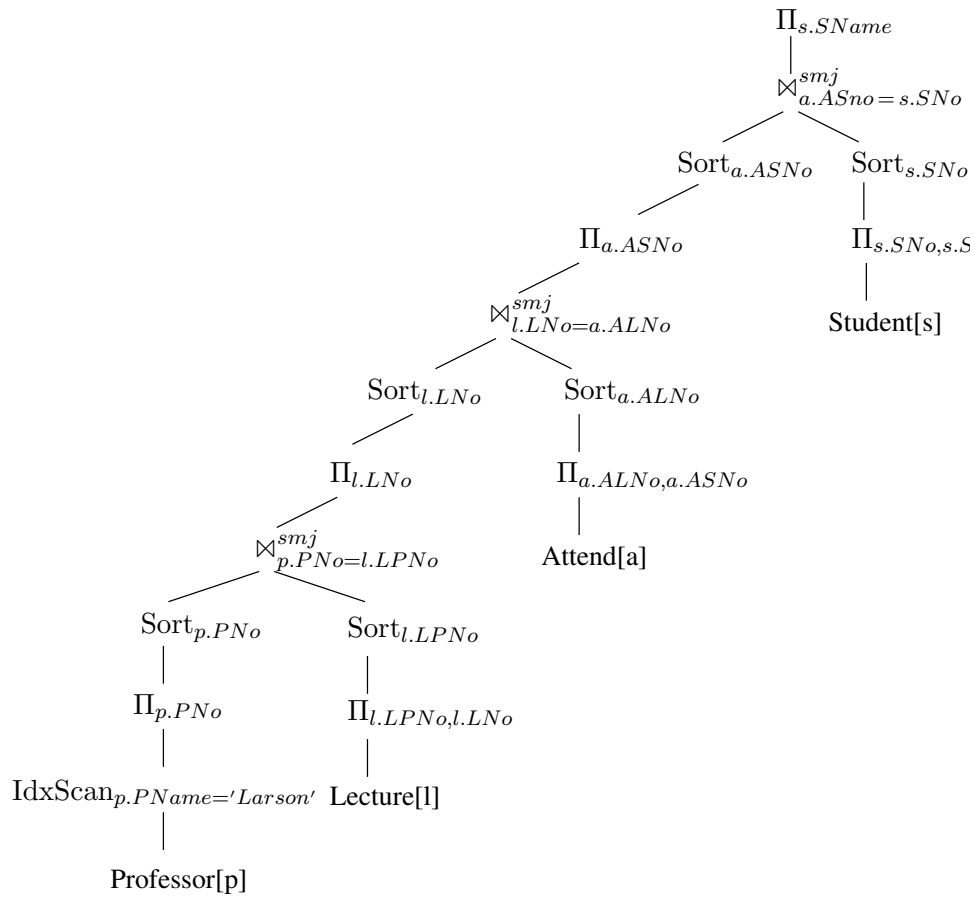


Figure 2.10: Plan for example query after physical query optimization

Chapter 3

Join Ordering

The problem of *join ordering* is a very restricted and — at the same time — a very complex one. We have touched this issue while discussing logical query optimization in Chapter 2. Join ordering is performed in Step 4 of Figure 2.5. In this chapter, we simplify the problem of join ordering by not considering duplicates, disjunctions, quantifiers, grouping, aggregation, or nested queries. Expressed positively, we concentrate on conjunctive queries with simple and cheap join predicates. What this exactly means will become clear in the next section. Subsequent sections discuss different algorithms for solving the join ordering problem. Finally, we take a look at the structure of the search space. This is important if different join ordering algorithms are compared via benchmarks. If the wrong parameters are chosen, benchmark results can be misleading.

The algorithms of this chapter form the core of every plan generator.

3.1 Queries Considered

A *conjunctive query* is one whose **where** clause contains a (complex) predicate which in turn is a conjunction of (simple) predicates. Hence, a conjunctive query involves only **and** and no *or* or *not* operations. A *simple predicate* is of the form $e_1 \theta e_2$ where $\theta \in \{=, \neq, <, >, \leq, \geq\}$ is a comparison operator and the e_i are simple expressions in attribute names possibly containing some simple and cheap arithmetic operators. By cheap we mean that it is not worth applying extra optimization techniques. In this chapter, we restrict simple predicates even further to the form $A = B$ for attributes A and B . A and B must also belong to different relations such that every simple predicate in this chapter is a join predicate. There are two reasons for this restriction. First, the most efficient join algorithms rely on the fact that the join predicate is of the form $A = B$. Such joins are called *equi-joins*. Any other join is called a *non-equijoin*. Second, in relational systems joins on foreign key attributes of one relation and key attributes of the other relation are very common. Other joins are rare.

A *base relation* is a relation that is stored (explicitly) in the database. For the rest of the chapter, let R_i ($1 \leq i \leq n$) be n relations. These relations can be base relations but do not necessarily have to be. They could also be base relations to which predicates have already been supplied, e.g. as a result

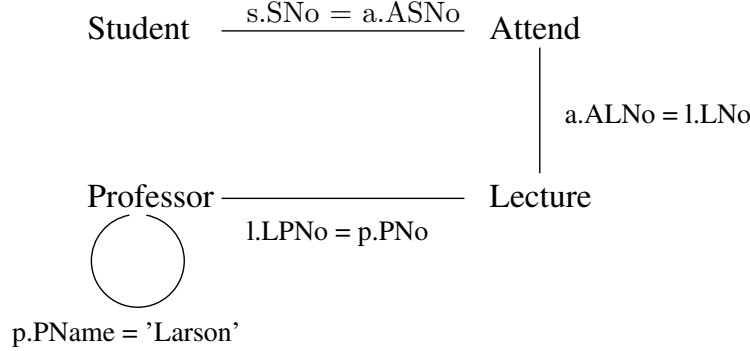


Figure 3.1: Query graph for example query of Section 2.1

of applying the first three steps of logical query optimization.

Summarizing, the queries we consider can be expressed in SQL as

```

select distinct *
from            $R_1, \dots, R_n$ 
where          $p$ 

```

where p is a conjunction of simple join predicates with attributes from exactly two relations. The latter restriction is not really necessary for the algorithms presented in this chapter but simplifies the exposition.

3.1.1 Query Graph

A query graph is a convenient representation of a query. It is an undirected graph with nodes $R_1, \dots, R_n$. For every simple predicate in the conjunction P whose attributes belong to the relations R_i and R_j , we add an edge between R_i and R_j . This edge is labeled by the simple predicate. From now on, we denote the join predicate connecting R_i and R_j by $p_{i,j}$. In general, $p_{i,j}$ can be a conjunction of simple join predicates connecting R_i and R_j .

If query graphs are used for more than join ordering, selections need to be represented. This is done by self-edges from the relation to which the selection applies to itself. For the example query of Chapter 2.6, Figure 3.1 contains the according query graph.

Query graphs can have many different shapes. The shapes that play a certain role in query optimization and the evaluation of join ordering algorithms are shown in Fig. 3.2. The query graph classes relevant for this chapter are chain queries, star queries, tree queries, cyclic queries and clique queries. Note that these classes are not disjoint and that some classes are subsets of other classes.

EX

In this chapter, we only treat connected query graphs. These can be evaluated without cross products.

Excursion In general, the query graph is a hypergraph [888] as the following example shows.

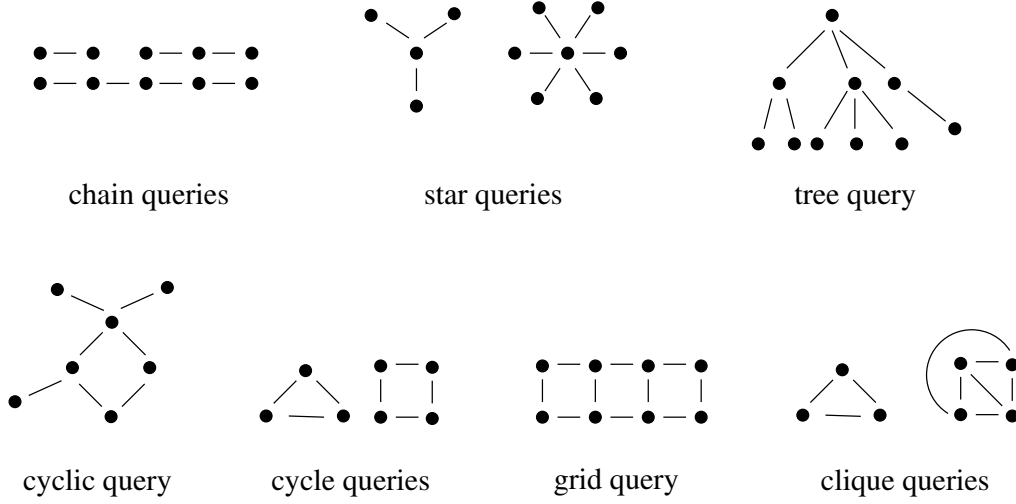


Figure 3.2: Query graph shapes

```

select *
from R1, R2, R3, R4
where f(R1.a, R2.a, R3.a) = g(R2.b, R3.b, R4.b)

```

3.1.2 Join Tree

A join tree is an algebraic expression in relation names and join operators. Sometimes, cross products are allowed, too. A cross product is the same as a join operator with *true* as its join predicate. A join tree has its name from its graph representation. There, a join tree is a binary tree whose leaf nodes are the relations and whose inner nodes are joins (and possibly cross products). The edges represent the input/output relationship. Examples of join trees have been shown in Figure 2.6.

Join trees fall into different classes. The most important classes are left-deep trees, right-deep trees, zig-zag trees, and bushy trees. *Left-deep trees* are join trees where every join has one of the relations R_i as its right input. *Right-deep trees* are defined analogously. In *zig-zag trees* at least one input of every join is a relation R_i . The class of zig-zag trees contains both left-deep and right-deep trees. For *bushy trees* no restriction applies. Hence, the class of bushy trees contains all of the above three classes. The roots of these notions date back to the paper by Selinger et al. [784], where the search space of the query optimizer was restricted to left-deep trees. There are two main reasons for this restriction. First, only one intermediate result is generated at any time during query evaluation. Second, the number of left-deep trees is far less than the number of e.g. bushy trees. The other classes were then added later by other researchers whenever they found better join trees in them. The different classes are illustrated in Figure 2.6. From left to right, the columns contain left-deep, zig-zag, right-deep, and bushy trees.

Left-deep trees directly correspond to an ordering (i.e. a permutation) of the relations. For example, the left-deep tree

$$((((R_2 \bowtie R_3) \bowtie R_1) \bowtie R_4) \bowtie R_5)$$

directly corresponds to the permutation R_2, R_3, R_1, R_4, R_5 . It should be clear that there is a one-to-one correspondence between permutations and left-deep join trees. We will also use the term *sequence of relations* synonymously. The notion of *join ordering* goes back to the times where only left-deep trees were considered and, hence, producing an optimal join tree was equivalent to optimally ordering the joins, i.e. determining a permutation with lowest cost.

Left-deep, right-deep, and zig-zag trees can be classed under the general term *linear trees*. Sometimes, the term linear trees is used synonymously for left-deep trees. We will not do so. Join trees are sometimes called *operator trees* or *query evaluation plans*. Although this is not totally wrong, these terms have a slightly different connotation. Operator trees typically contain more than only join operators. Query evaluation plans (QEPs or plans for short) typically have more information from physical query optimization associated with them.

3.1.3 Simple Cost Functions

In order to judge the quality of join trees, we need a cost function that associates a certain positive cost with each join tree. Then, the *task of join ordering* is to find among all equivalent join trees the join tree with lowest associated costs.

One part of any cost function are cardinality estimates. They are based on the cardinalities of the relations, i.e. the number of tuples contained in them. For a given relation R_i , we denote its cardinality by $|R_i|$.

Then, the cardinality of intermediate results must be estimated. This is done by introducing the notion of join selectivity. Let $p_{i,j}$ be a join predicate between relations R_i and R_j . The *selectivity* $f_{i,j}$ of $p_{i,j}$ is then defined as

$$f_{i,j} = \frac{|R_i \bowtie_{p_{i,j}} R_j|}{|R_i| * |R_j|}$$

This is the number of tuples in the join's result divided by the number of tuples in the Cartesian Product between R_i and R_j . If $f_{i,j}$ is 0.1, then only 10% of all tuples in the Cartesian Product survive the predicate $p_{i,j}$. Note that the selectivity is always a number between 0 and 1 and that $f_{i,j} = f_{j,i}$. We use an f and not an s , since the selectivity of a predicate is often called *filter factor*.

Besides the relation's cardinalities, the selectivities of the join predicates $p_{i,j}$ are assumed to be given as input to the join ordering algorithm. Therefore, we can compute the output cardinality of a join $R_i \bowtie_{p_{i,j}} R_j$, as

$$|R_i \bowtie_{p_{i,j}} R_j| = f_{i,j} |R_i| |R_j|$$

From this it becomes clear that if there is no join predicate for two relations R_i and R_j , we can assume a join predicate *true* and associate a selectivity of 1 with it. The output cardinality is then the cardinality of the cross product

between R_i and R_j . We also define $f_{i,i} = 1$ for all $1 \leq i \leq n$. This allows us to keep subsequent formulas simple.

We now need to extend our cardinality estimation to join trees. This can be done by recursively applying the above formula. Consider a join tree T joining two join trees T_1 and T_2 , i.e. $T = T_1 \bowtie T_2$. Then, the result cardinality $|T|$ can be calculated as follows. If T is a leaf R_i , then $|T| := |R_i|$. Otherwise,

$$|T| = \left(\prod_{R_i \in T_1, R_j \in T_2} f_{i,j} \right) |T_1| |T_2|.$$

Note that this formula assumes that the selectivities are independent of each other. Assuming independence is common but may be very misleading. More on this issue can be found in Chapter ???. Nevertheless, we assume independence and stick to the above formula.

For sequences of joins we can give a simple cardinality definition. Let $s = R_1, \dots, R_n$ be a sequence of relations. Then

$$|s| = \prod_{k=1}^n |R_k| \left(\prod_{i=1}^k f_{i,k} \right).$$

Given the above, a query graph alone is not really sufficient for the specification of a join ordering problem: cardinalities and selectivities are missing. On the other hand, from a complete list of cardinalities and selectivities we can derive the query graph. Obviously, the following defines a chain query with query graph $R_1 - - - R_2 - - - R_3$:

$$\begin{aligned} |R_1| &= 10 \\ |R_2| &= 100 \\ |R_3| &= 1000 \\ f_{1,2} &= 0.1 \\ f_{2,3} &= 0.2 \end{aligned}$$

In all examples, we assume for all i and j for which $f_{i,j}$ is not given that there is no join predicate and hence $f_{i,j} = 1$.

We now come to cost functions. The first cost function we consider is called C_{out} . For a join tree T , $C_{\text{out}}(T)$ is the sum of all output cardinalities of all joins in T . Recursively, we can define C_{out} as

$$C_{\text{out}}(T) = \begin{cases} 0 & \text{if } T \text{ is a single relation} \\ |T| + C_{\text{out}}(T_1) + C_{\text{out}}(T_2) & \text{if } T = T_1 \bowtie T_2 \end{cases}$$

From a theoretical point of view, C_{out} has many interesting properties: it is symmetric, it has the ASI property, and it can be applied to an expression of the logical algebra. From a practical point of view, however, it is rarely applied (yet).

In real cost functions, the cardinalities only serve as input to more complex formulas capturing the costs of a join implementation. Since real cost functions

are too complex for this section, we stick to simple cost functions proposed by Krishnamurthy, Boral, and Zaniolo [520]. They argue that these cost functions are appropriate for main memory database systems. For the three different join implementations nested loop join (nlj), hash join (hj), and sort merge join (smj), they give the following cost functions:

$$\begin{aligned} C_{\text{nlj}}(e_1 \bowtie_p e_2) &= |e_1||e_2| \\ C_{\text{hj}}(e_1 \bowtie_p e_2) &= h|e_1| \\ C_{\text{smj}}(e_1 \bowtie_p e_2) &= |e_1|\log(|e_1|) + |e_2|\log(|e_2|) \end{aligned}$$

where e_i are join trees and h is the average length of the collision chain in the hash table. We will assume $h = 1.2$. All these cost functions are defined for a single join operator. The cost of a join tree is defined as the sum of the costs of all joins it contains. We use the symbols C_x to also denote the costs of not only a single join but the costs of the whole tree. Hence, for sequences s of relations, we have

$$\begin{aligned} C_{\text{nlj}}(s) &= \sum_{i=2}^n |s_1, \dots, s_{i-1}| * |s_i| \\ C_{\text{hj}}(s) &= \sum_{i=2}^n 1.2 |s_1, \dots, s_{i-1}| \\ C_{\text{smj}}(s) &= \sum_{i=2}^n |s_1, \dots, s_{i-1}| \log(|s_1, \dots, s_{i-1}|) + \sum_{i=2}^n |s_i| \log(|s_i|) \end{aligned}$$

Some notes on the cost functions are in order. First, note that these cost functions are even for main memory a little incomplete. For example, constant factors are missing. Second, the cost functions are mainly devised for left-deep trees. This becomes apparent when looking at the costs of hash joins. It is assumed that the right input is already stored in an appropriate hash table. Obviously, this can only hold for base relations, giving rise to left-deep trees. Third, C_{hj} and C_{smj} do not work for cross products. However, we can extend these cost functions by defining the cost of a cross product to be equal to its output cardinality, which happens to be the cost of C_{nlj} . Fourth, in reality, more complex cost functions are used and other parameters like the width of the tuples—i.e. the number of bytes needed to store them—also play an important role. Fifth, the above cost functions assume that the same join algorithm is chosen throughout the whole plan. In practice, this will not be true.

For the above chain query, we compute the costs of different join trees. The last join tree contains a cross product.

	C_{out}	C_{nlj}	C_{hj}	C_{smj}
$R_1 \bowtie R_2$	100	1000	12	697.61
$R_2 \bowtie R_3$	20000	100000	120	10630.26
$R_1 \times R_3$	10000	10000	10000	10000.00
$(R_1 \bowtie R_2) \bowtie R_3$	20100	101000	132	11327.86
$(R_2 \bowtie R_3) \bowtie R_1$	40000	300000	24120	32595.00
$(R_1 \times R_3) \bowtie R_2$	30000	1010000	22000	143542.00

For the calculation of C_{out} note that $|R_1 \bowtie R_2 \bowtie R_3| = 20000$ is included in all of the last three lines of its column. For the nested loop cost function, the costs are calculated as follows:

$$\begin{aligned} C_{\text{nlj}}((R_1 \bowtie R_2) \bowtie R_3) &= 1000 + 100 * 1000 = 101000 \\ C_{\text{nlj}}((R_2 \bowtie R_3) \bowtie R_1) &= 100000 + 20000 * 10 = 300000 \\ C_{\text{nlj}}((R_1 \times R_3) \bowtie R_2) &= 10000 + 10000 * 100 = 1010000 \end{aligned}$$

The reader should verify the other costs.

Several observations can be made from the above numbers:

- The costs of different join trees differ vastly under every cost function. Hence, it is worth spending some time to find a cheap join order.
- The costs of the same join tree differ under the different cost functions.
- The cheapest join tree is $(R_1 \bowtie R_2) \bowtie R_3$ under all four cost functions.
- Join trees with cross products are expensive.
Thus, a heuristics often used is not to consider join trees that contain unnecessary cross products. (If the query graph consists of several unconnected components, then and only then cross products are necessary. In other words: if the query graph is connected, no cross products are necessary.).
- The join order matters even for join trees without cross products.

We would like to emphasize that the join order is also relevant under other cost functions.

Avoiding cross products is not always beneficial, as the following query specification shows:

$$\begin{aligned} |R_1| &= 1000 \\ |R_2| &= 2 \\ |R_3| &= 2 \\ f_{1,2} &= 0.1 \\ f_{1,3} &= 0.1 \end{aligned}$$

For C_{out} we have costs

Join Tree	C_{out}
$R_1 \bowtie R_2$	200
$R_2 \times R_3$	4
$R_1 \bowtie R_3$	200
$(R_1 \bowtie R_2) \bowtie R_3$	240
$(R_2 \times R_3) \bowtie R_1$	44
$(R_1 \bowtie R_3) \bowtie R_2$	240

Note that although the absolute numbers are quite small, the ratio of the best and the second best join tree is quite large. The reader is advised to find more examples and to apply other cost functions.

The following example illustrates that a bushy tree can be superior to any linear tree. Let us use the following query specification:

$$\begin{aligned}
 |R_1| &= 10 \\
 |R_2| &= 20 \\
 |R_3| &= 20 \\
 |R_4| &= 10 \\
 f_{1,2} &= 0.01 \\
 f_{2,3} &= 0.5 \\
 f_{3,4} &= 0.01
 \end{aligned}$$

If we do not consider cross products, we have for the symmetric (see below) cost function C_{out} the following join trees and costs:

Join Tree	C_{out}
$R_1 \bowtie R_2$	2
$R_2 \bowtie R_3$	200
$R_3 \bowtie R_4$	2
$((R_1 \bowtie R_2) \bowtie R_3) \bowtie R_4$	24
$((R_2 \bowtie R_3) \bowtie R_1) \bowtie R_4$	222
$(R_1 \bowtie R_2) \bowtie (R_3 \bowtie R_4)$	6

Note that all other linear join trees fall into one of these classes, due to the symmetry of the cost function and the join ordering problem. Again, the reader is advised to find more examples and to apply other cost functions.

If we want to annotate a join operator by its implementation—which is necessary for the correct computation of costs—we write $\bowtie^{\text{impl}}$ for an implementation `impl`. For example, $\bowtie^{\text{smj}}$ is a sort-merge join, and the according cost function C_{smj} is used to compute its costs.

Two properties of cost functions have some impact on the join ordering problem. The first is symmetry. A cost function C_{impl} is called *symmetric* if $C_{\text{impl}}(R_1 \bowtie^{\text{impl}} R_2) = C_{\text{impl}}(R_2 \bowtie^{\text{impl}} R_1)$ for all relations R_1 and R_2 . For symmetric cost functions, it does not make sense to consider commutativity. Hence, it suffices to consider left-deep trees only if we want to restrict ourselves to linear join trees. Note that C_{out} , C_{nlj} , C_{smj} are symmetric while C_{hj} is not.

The other property is the *adjacent sequence interchange* (ASI) property. Informally, the ASI property states that there exists a rank function such that the order of two subsequences is optimal if they are ordered according to the rank function. The ASI property is formally defined in Section 3.2.2. Only for tree queries and cost functions with the ASI property, a polynomial algorithm to find an optimal join order is known. Our cost functions C_{out} and C_{hj} have the ASI property, C_{smj} does not. Summarizing the properties of our cost functions, we see that the classification is orthogonal:

	ASI	$\neg$ ASI
symmetric	$C_{\text{out}}, C_{\text{nlj}}$	C_{smj}
$\neg$ symmetric	C_{hj}	(see text)

For the missing case of a non-symmetric cost function not having the ASI property, we can use the cost function of the hybrid hash join [238, 677].

We turn to another not really well-researched topic. The goal is to cut down the number of cost functions which have to be considered for optimization and to possibly allow for simpler cost functions, which saves time during plan generation. Unfortunately, we have to restrict ourselves to left-deep join trees. Let s denote a sequence or permutation of a given set of joins. We define an equivalence relation on cost functions.

Definition 3.1.1 *Let C and C' be two cost functions. Then*

$$C \equiv C' :\prec\succ (\forall s \ C(s) \text{ minimal} \prec\succ C'(s) \text{ minimal})$$

Here, s is a join sequence.

Obviously, $\equiv$ is an equivalence relation.

Now we can define the ΣIR property.

Definition 3.1.2 *A cost function C is ΣIR : $\prec\succ C \equiv C_{\text{out}}$.*

That is, ΣIR is the set of all cost functions that are equivalent to C_{out} .

Let us consider a very simple example. The last element of the sum in C_{out} is the size of the final join (all relations are joined). This is not the case for the following cost function:

$$C'_{\text{out}}(s) := \sum_{i=2}^{n-1} |s_1, \dots, s_i|$$

Obviously, we have C'_{out} is ΣIR . The next observation shows that we can construct quite complex ΣIR cost functions:

Observation 3.1.3 *Let C_1 and C_2 be two ΣIR cost functions. For non-decreasing functions $f_1 : R \rightarrow R$ and $f_2 : R \times R \rightarrow R$ and constants $c \in R$ and $d \in R^+$, we have that*

EX

$$\begin{aligned} &C_1 + c \\ &C_1 * d \\ &f_1 \circ C_1 \\ &f_2 \circ (C_1, C_2) \end{aligned}$$

are ΣIR . Here, $\circ$ denotes function composition and $(\cdot, \cdot)$ function pairing.

There are of course many more possibilities of constructing ΣIR functions. For the cost functions C_{hj} , C_{smj} , and C_{nlj} , we now investigate which of them have the ΣIR property.

Let us consider C_{hj} first. From

$$\begin{aligned} C_{hj}(s) &= \sum_{i=2}^n 1.2|s_1, \dots, s_{i-1}| \\ &= 1.2|s_1| + 1.2 \sum_{i=2}^{n-1} |s_1, \dots, s_i| \\ &= 1.2|s_1| + 1.2C'_{out}(s) \end{aligned}$$

and observation 3.1.3, we conclude that C_{hj} is ΣIR for a fixed relation to be joined first. If we can optimize C_{out} in polynomial time, then we can optimize C_{out} for a fixed starting relation. Indeed, by trying each relation as a starting relation, we can find the optimal join tree in polynomial time. An algorithm that computes the optimal solution for an arbitrary relation to be joined first can be found in Section 3.2.2.

Now, consider C_{smj} . Since

$$\sum_{i=2}^n |s_1, \dots, s_{i-1}| \log(|s_1, \dots, s_{i-1}|)$$

is minimal if and only if

$$\sum_{i=2}^n |s_1, \dots, s_{i-1}|$$

is minimal and $\sum_{i=2}^n |s_i| \log(|s_i|)$ is independent of the order of the relations within s — that is constant — we conclude that C_{smj} is ΣIR .

Last, we have that C_{nlj} is not ΣIR . To see this, consider the following counter example with three relations R_1 , R_2 , and R_3 of sizes 10, 10, and 100, resp. The selectivities are $f_{1,2} = \frac{9}{10}$, $f_{2,3} = \frac{1}{10}$, and $f_{1,3} = \frac{1}{10}$. Now,

$$\begin{aligned} |R_1 R_2| &= 90 \\ |R_1 R_3| &= 100 \\ |R_2 R_3| &= 100 \end{aligned}$$

and

$$\begin{aligned} C_{nl}(R_1 R_2 R_3) &= 10 * 10 + 90 * 100 = 9100 \\ C_{nl}(R_1 R_3 R_2) &= 10 * 100 + 100 * 10 = 2000 \\ C_{nl}(R_2 R_3 R_1) &= 10 * 100 + 100 * 10 = 2000 \end{aligned}$$

We see that $R_1 R_2 R_3$ has the smallest sum of intermediate result sizes but produces the highest cost. Hence, C_{nlj} is not ΣIR .

3.1.4 Classification of Join Ordering Problems

After having discussed the different classes of query graphs, join trees and cost functions, we can classify join ordering problems. To define a certain join ordering problem, we have to pick one entry from every class:

EX

Query Graph Classes $\times$ Possible Join Tree Classes $\times$ Cost Function
Classes

The query graph classes considered are *chain*, *star*, *tree*, and *cyclic*. For the join tree classes we distinguish between the different join tree shapes, i.e. whether they are left-deep, zig-zag, or bushy trees. We left out the right-deep trees, since they do not differ in their behavior from left-deep trees. We also have to take into account whether cross products are considered or not. For cost functions, we use a simple classification: we only distinguish between those that have the ASI property and those that do not. This leaves us with $4 \times 3 \times 2 \times 2 = 48$ different join ordering problems. For these, we will first review search space sizes and complexity. Then, we discuss several algorithms for join ordering. Last, we give some insight into cost distributions over the search space and how this might influence the benchmarking of different join ordering algorithms.

3.1.5 Search Space Sizes

Since search space sizes are easier to count if cross products are allowed, we consider them first. Then we turn to search spaces where cross products are not considered.

Join Trees with Cross Products We consider the number of join trees for a query graph with n relations. When cross products are allowed, the number of left-deep and right-deep join trees is $n!$. By allowing cross products, the query graph does not restrict the search space in any way. Hence, any of the $n!$ permutations of the n relations corresponds to a valid left-deep join tree. This is true independent of the query graph.

Similarly, the number of zig-zag trees can be estimated independently of the query graph. First note that for joining n relations, we need $n - 1$ join operators. From any left-deep tree, we derive zig-zag trees by using the join's commutativity and exchange the left and right inputs. Hence, from any left-deep tree for n relations, we can derive 2^{n-2} zig-zag trees. We have to subtract another one, since the bottommost joins' arguments are exchanged in different left-deep trees. Thus, there exists a total of $2^{n-2}n!$ zig-zag trees. Again, this number is independent of the query graph.

The number of bushy trees can be estimated as follows. First, we need the number of binary trees. For n leaf nodes, the number of binary trees is given by $\mathcal{C}(n - 1)$, where $\mathcal{C}(n)$ is defined by the recurrence

$$\mathcal{C}(n) = \sum_{k=0}^{n-1} \mathcal{C}(k)\mathcal{C}(n - k - 1)$$

with $\mathcal{C}(0) = 1$. The numbers $\mathcal{C}(n)$ are called the *Catalan Numbers* (see [209]). They can also be computed by the following formula:

$$\mathcal{C}(n) = \frac{1}{n + 1} \binom{2n}{n}.$$

The Catalan Numbers grow in the order of $\Theta(4^n/n^{3/2})$.

After we know the number of binary trees with n leaves, we now have to attach the n relations to the leaves in all possible ways. For a given binary tree, this can be done in $n!$ ways. Hence, the total number of bushy trees is $n!\mathcal{C}(n-1)$. This can be simplified as follows (see also [307, 532, 867]):

$$\begin{aligned}
 n!\mathcal{C}(n-1) &= n! \frac{1}{n} \binom{2(n-1)}{n-1} \\
 &= n! \frac{1}{n} \frac{(2n-2)!}{(n-1)!((2n-2)-(n-1))!} \\
 &= \frac{(2n-2)!}{(n-1)!}
 \end{aligned}$$

Chain Queries, Left-Deep Join Trees, No Cartesian Product We now derive the function that calculates the number of left-deep join trees with no cross products for a chain query of n relations. That is, the query graph is $R_1 - R_2 - \dots - R_{n-1} - R_n$. Let us denote the number of join trees by $f(n)$. Obviously, for $n = 0$ there is only one (the empty) join tree. For $n = 1$, there is also only one join tree (no join). For larger n : Consider the join trees for $R_1 - \dots - R_{n-1}$ where relation R_{n-1} is the k -th relation from the bottom where k ranges from 1 to $n-1$. From such a join tree we can derive join trees for all n relations by adding relation R_n at any position following R_{n-1} . There are $n-k$ such join trees. Only for $k = 1$, we can also add R_n below R_{n-1} . Hence, for $k = 1$ we have n join trees. How many join trees with R_{n-1} at position k are there? For $k = 1$, R_{n-1} must be the first relation to be joined. Since we do not consider cross products, it must be joined with R_{n-2} . The next relation must be R_{n-3} , and so on. Hence, there is only one such join tree. For $k = 2$, the first relation must be R_{n-2} , which is then joined with R_{n-1} . Then $R_{n-3}, \dots, R_1$ must follow in this order. Again, there is only one such join tree. For higher k , for R_{n-1} to occur safely at position k (no cross products) the $k-1$ relations $R_{n-2}, \dots, R_{n-k}$ must occur before R_{n-1} . There are exactly $f(k-1)$ join trees for the $k-1$ relations. On each such join tree we just have to add R_{n-1} on top of it to yield a join tree with R_{n-1} at position k .

Now we can compute the $f(n)$ as $n + \sum_{k=2}^{n-1} f(k-1) * (n-k)$ for $n > 1$. Solving this recurrence gives us $f(n) = 2^{n-1}$. The proof is by induction. The case $n = 1$ is trivial.

The induction step for $n > 1$ provided by Thomas Neumann goes as follows:

$$\begin{aligned}
f(n) &= n + \sum_{k=2}^{n-1} f(k-1) * (n-k) \\
&= n + \sum_{k=0}^{n-3} f(k+1) * (n-k-2) \\
&= n + \sum_{k=0}^{n-3} 2^k * (n-k-2) \\
&= n + \sum_{k=1}^{n-2} k 2^{n-k-2} \\
&= n + \sum_{k=1}^{n-2} 2^{n-k-2} + \sum_{k=2}^{n-2} (k-1) 2^{n-k-2} \\
&= n + \sum_{i=1}^{n-2} \sum_{j=i}^{n-2} 2^{n-j-2} \\
&= n + \sum_{i=1}^{n-2} \sum_{j=0}^{n-i-2} 2^j \\
&= n + \sum_{i=1}^{n-2} (2^{n-i-1} - 1) \\
&= n + \sum_{i=1}^{n-2} 2^i - (n-2) \\
&= n + (2^{n-1} - 2) - (n-2) \\
&= 2^{n-1}
\end{aligned}$$

Chain Queries, Zig-Zag Join Trees, No Cartesian Product All possible zig-zag trees can be derived from a left-deep tree by exchanging the left and right arguments of a subset of the joins. Since for the first join these alternatives are already considered within the set of left-deep trees, we are left with $n-2$ joins. Hence, the number of zig-zag trees for n relations in a chain query is $2^{n-2} * 2^{n-1} = 2^{2n-3}$.

Chain Queries, Bushy Join Trees, No Cartesian Product We can compute the number of bushy trees with no cross products for a chain query in the following way. Let us denote this number by $f(n)$. Again, let us assume that the chain query has the form $R_1 - R_2 - \dots - R_{n-1} - R_n$. For $n = 0$, we only have the empty join tree. For $n = 1$ we have one join tree. For $n = 2$ we have two join trees. For more relations, every subtree of the join tree must contain a subchain in order to avoid cross products. Further, the subchain can occur

as the left or right argument of the join. Hence, we can compute $f(n)$ as

$$\sum_{k=1}^{n-1} 2 f(k) f(n-k)$$

This is equal to

$$2^{n-1} \mathcal{C}(n-1)$$

EX

where $\mathcal{C}(n)$ are the Catalan Numbers.

Star Queries, No Cartesian Product The first join has to connect the center relation R_0 with any of the other relations. The other relations can follow in any order. Since R_0 can be the left or the right input of the first join, there are $2 * (n-1)!$ possible left-deep join trees for Star Queries with no Cartesian Product.

The number of zig-zag join trees is derived by exchanging the arguments of all but the first join in any left-deep join tree. We cannot consider the first join since we did so in counting left-deep join trees. Hence, the total number of zig-zag join trees is $2 * (n-1)! * 2^{n-2} = 2^{n-1} * (n-1)!$.

Constructing bushy join trees with no Cartesian Product from a Star Query other than zig-zag join trees is not possible.

Remarks The numbers for star queries are also upper bounds for tree queries. For clique queries, no join tree containing a cross product is possible. Hence, all join trees are valid join trees and the search space size is the same as the corresponding search space for join trees with cross products.

To give the reader a feeling for the numbers, the following tables contain the potential search space sizes for some n .

	Join trees without cross products				
	chain query			star query	
	left-deep	zig-zag	bushy	left-deep	zig-zag/bushy
n	2^{n-1}	2^{2n-3}	$2^{n-1}\mathcal{C}(n-1)$	$2 * (n-1)!$	$2^{n-1}(n-1)!$
1	1	1	1	1	1
2	2	2	2	2	2
3	4	8	8	4	8
4	8	32	40	12	48
5	16	128	224	48	384
6	32	512	1344	240	3840
7	64	2048	8448	1440	46080
8	128	8192	54912	10080	645120
9	256	32768	366080	80640	10321920
10	512	131072	2489344	725760	185794560

	With cross products/clique		
	left-deep	zig-zag	bushy
n	$n!$	$2^{n-2} * n!$	$n!C(n-1)$
1	1	1	1
2	2	2	2
3	6	12	12
4	24	96	120
5	120	960	1680
6	720	11520	30240
7	5040	161280	665280
8	40320	2580480	17297280
9	362880	46448640	518918400
10	3628800	928972800	17643225600

Note that in Figure 2.6 only 32 join trees are listed, whereas the number of bushy trees for chain queries with four relations is 40. The missing eight cases are those zig-zag trees which are symmetric (i.e. derived by applying commutativity to all occurring joins) to the ones contained in the second column.

From these numbers, it becomes immediately clear why historically the search space of query optimizers was restricted to left-deep trees and cross products for connected query graphs were not considered.

3.1.6 Problem Complexity

The complexity of the join ordering problem depends on several parameters. These are the shape of the query graph, the class of join trees to be considered, whether cross products are considered or not, and whether the cost function has the ASI property or not. Not for all the combinations complexity results are known. What is known is summarized in the following table.

Query graph	Join tree	Cross products	Cost function	Complexity
general tree/star/chain star	left-deep	no	ASI	NP-hard
	left-deep	no	one join method (ASI)	P
	left-deep	no	two join methods (NLJ+SMJ)	NP-hard
general/tree/star chain	left-deep	yes	ASI	NP-hard
	left-deep	yes	—	open
general tree star	bushy	no	ASI	NP-hard
	bushy	no	—	open
	bushy	no	ASI	P
chain	bushy	no	any	P
general tree/star/chain	bushy	yes	ASI	NP-hard
	bushy	yes	ASI	NP-hard

Ibaraki and Kameda were the first who showed that the problem of deriving optimal left-deep trees for cyclic queries is NP-hard for a cost function for an n -way nested loop join implementation [438]. The proof was repeated for the cost function C_{out} which has the ASI property [194, 877]. In both proofs, the

clique problem was used for the reduction [320]. C_{out} was also used in the other proofs of NP-hardness results. The next line goes back to the same paper. Ibaraki and Kameda also described an algorithm to solve the join ordering problem for tree queries producing optimal left-deep trees for a special cost function for a nested-loop n-way join algorithm. Their algorithm was based on the observation that their cost function has the ASI property. For this case, they could derive an algorithm from an algorithm for a sequencing problem for job scheduling designed by Monma and Sidney [627]. They, in turn, used an earlier result by Lawler [537]. The algorithm of Ibaraki and Kameda was slightly generalized by Krishnamurthy, Boral, and Zaniolo, who were also able to sketch a more efficient algorithm. It improves the time bounds from $O(n^2 \log n)$ to $O(n^2)$. The disadvantage of both approaches is that with every relation, a fixed (i.e. join-tree independent) join implementation must be associated before the optimization starts. Hence, it only produces optimal trees if there is only one join implementation available or one is able to guess the optimal join method before hand. This might not be the case. The polynomial algorithm which we term IKKBZ is described in Section 3.2.2.

For star queries, Ganguly investigated the problem of generating optimal left-deep trees if no cross products but two different cost functions (one for nested loop join, the other for sort merge join) are allowed. It turned out that this problem is NP-hard [312].

The next line is due to Cluet and Moerkotte [194]. They showed by reduction from 3DM that taking into account cross products results in an NP-hard problem even for star queries. Remember that star queries are tree queries and general graphs.

The problem for general bushy trees follows from a result by Scheufele and Moerkotte [768]. They showed that building optimal bushy trees for cross products only (i.e. all selectivities equal one) is already NP-hard. This result also explains the last two lines.

By noting that for star queries, all bushy trees that do not contain a cross product are left-deep trees, the problem can be solved by the IKKBZ algorithm for left-deep trees. Ono and Lohman showed that for chain queries dynamic programming considers only a polynomial number of bushy trees if no cross products are considered [653]. This is discussed in Section 3.2.4.

The table is rather incomplete. Many open problems exist. For example, if we have chain queries and consider cross products: is the problem NP-hard or in P? Some results for this problem have been presented [768], but it is still an open problem (see Section 3.2.7). Open is also the case where we produce optimal bushy trees with no cross products for tree queries. Yet another example of an open problem is whether we could drop the ASI property and are still able to derive a polynomial algorithm for a tree query. This is especially important, since the cost function for a sort-merge algorithm does not have the ASI property.

Good summaries of complexity results for different join ordering problems can be found in the theses of Scheufele [766] and Hamalainen [394].

Given that join ordering is an inherently complex problem with no polynomial algorithm in sight, one might wonder whether there exists good polynomial

approximation algorithms. Chances are that even this is not the case. Chatterji, Evani, Ganguly, and Yemmanuru showed that three different optimization problems — all asking for linear join trees — are not approximable [145].

3.2 Deterministic Algorithms

3.2.1 Heuristics

We now present some simple heuristic solutions to the problem of join ordering. These heuristics only produce left-deep trees. Since left-deep trees are equivalent with permutations, these heuristics order the joins according to some criterion.

The core algorithm for the heuristics discussed here is the *greedy algorithm* (for an introduction see [209]). In greedy algorithms, a *weight* is associated with each entity. In our case, weights are associated with each relation. A typical weight function is the cardinality of the relation ($|R|$). Given a weight function *weight*, a greedy join ordering algorithm works as follows:

```
GreedyJoinOrdering-1( $\{R_1, \dots, R_n\}$ , (*weight)(Relation))
Input: a set of relations to be joined and a weight function
Output: a join order
 $S = \epsilon$ ; // initialize  $S$  to the empty sequence
 $R = \{R_1, \dots, R_n\}$ ; // let  $R$  be the set of all relations
while(!empty( $R$ )) {
  Let  $k$  be such that:  $\text{weight}(R_k) = \min_{R_i \in R}(\text{weight}(R_i))$ ;
   $R \setminus = R_k$ ; // eliminate  $R_k$  from  $R$ 
   $S \circ = R_k$ ; // append  $R_k$  to  $S$ 
}
return  $S$ 
```

This algorithm takes cross products into account. If we are only interested in left-deep join trees with no cross products, we have to require that R_k is connected to some of the relations contained in S in case $S \neq \epsilon$. Note that a more efficient implementation would sort the relations according to their weight.

Not all heuristics can be implemented with a greedy algorithm as simple as above. An often-used heuristics is to take the relation next that produces the smallest (next) intermediate result. This cannot be determined by the relation alone. One must take into account the sequence S already processed, since only then the selectivities of all predicates connecting relations in S and the new relation are deducible. And we must take the product of these selectivities and the cardinality of the new relation in order to get an estimate of the intermediate result's cardinality. As a consequence, the weights become *relative* to S . In other words, the weight function now has two parameters: a sequence of relations already joined and the relation whose relative weight is to be computed. Here is the next algorithm:

```

GreedyJoinOrdering-2( $\{R_1, \dots, R_n\}$ ,
    (*weight)(Sequence of Relations, Relation))
Input: a set of relations to be joined and a weight function
Output: a join order
 $S = \epsilon$ ; // initialize  $S$  to the empty sequence
 $R = \{R_1, \dots, R_n\}$ ; // let  $R$  be the set of all relations
while(!empty( $R$ )) {
    Let  $k$  be such that:  $\text{weight}(S, R_k) = \min_{R_i \in R}(\text{weight}(S, R_i))$ ;
     $R \setminus = R_k$ ; // eliminate  $R_k$  from  $R$ 
     $S \circ = R_k$ ; // append  $R_k$  to  $S$ 
}
return  $S$ 

```

Note that for this algorithm, sorting is not possible. GreedyJoinOrdering-2 can be improved by taking every relation as the starting one.

```

GreedyJoinOrdering-3( $\{R_1, \dots, R_n\}$ , (*weight)(Sequence of Relations, Relation))
Input: a set of relations to be joined and a weight function
Output: a join order
Solutions =  $\emptyset$ ;
for ( $i = 1$ ;  $i \leq n$ ;  $++i$ ) {
     $S = R_i$ ; // initialize  $S$  to a singleton sequence
     $R = \{R_1, \dots, R_n\} \setminus \{R_i\}$ ; // let  $R$  be the set of all relations
    while(!empty( $R$ )) {
        Let  $k$  be such that:  $\text{weight}(S, R_k) = \min_{R_i \in R}(\text{weight}(S, R_i))$ ;
         $R \setminus = R_k$ ; // eliminate  $R_k$  from  $R$ 
         $S \circ = R_k$ ; // append  $R_k$  to  $S$ 
    }
    Solutions +=  $S$ ;
}
return cheapest in Solutions

```

In addition to the relative weight function mentioned before, another often used relative weight function is the product of the selectivities connecting relations in S with the new relation. This heuristic is sometimes called *MinSel*.

The above two algorithms generate linear join trees. Lohman and Fegaras independently proposed heuristics (named Greedy Operator Ordering (GOO)) to generate bushy join trees [276, 277, 566, 805]. The idea is as follows. A set of join trees **Trees** is initialized such that it contains all the relations to be joined. It then investigates all pairs of trees contained in **Trees**. Among all of these, the algorithm joins the two trees that result either in the smallest cost of performing this join (Lohman [566], Shekita, Tan, Young [805]) or in the smallest intermediate result (Fegaras [276, 277]). The two trees are then eliminated from **Trees** and the new join tree joining them is added to it. The algorithm then looks as follows:

```

GOO( $\{R_1, \dots, R_n\}$ ,  $(\text{*weight})(T_1, T_2)$ )
Input: a set of relations to be joined
Output: join tree
Trees :=  $\{R_1, \dots, R_n\}$ 
while ( $|\text{Trees}| \neq 1$ ) {
    find  $T_i, T_j \in \text{Trees}$  such that  $i \neq j$ ,  $\text{weight}(T_i, T_j)$  is minimal
        among all pairs of trees in Trees
    Trees - =  $T_i$ ;
    Trees - =  $T_j$ ;
    Trees + =  $T_i \bowtie T_j$ ;
}
return the tree contained in Trees;

```

Our GOO variant differs slightly from the one proposed by Fegaras. He uses arrays, explicitly handles the forming of the join predicates, and materializes intermediate result sizes. Hence, his algorithm is a little more elaborated, but we assume that the reader is able to fill in the gaps. Further, Fegaras proposes the weight function $\text{weight}(T_1, T_2) = |T_i \bowtie T_j|$ whereas Lohman proposes to use the weight function $\text{weight}(T_1, T_2) = \text{cost}(T_i \bowtie T_j)$.

None of our algorithms so far considers different join implementations. An explicit consideration of commutativity for non-symmetric cost functions could also help to produce better join trees. The reader is asked to work out the details of these extensions. In general, the heuristics do not produce the optimal plan. EX
The reader is advised to find examples where the heuristics are far off the best possible plan. EX

3.2.2 Determining the Optimal Join Order in Polynomial Time

Since the general problem of join ordering is NP-hard, we cannot expect to find a polynomial solution for it. However, for special cases, we can expect to find solutions that work in polynomial time. These solutions can also be used as heuristics for the general case, either to find a not-that-bad join tree or to determine an upper bound for the costs that is then fed into a search procedure to prune the search space.

The most general case for which a polynomial solution is known is characterized by the following features:

- the query graph must be acyclic
- no cross products are considered
- the search space is restricted to left-deep trees
- the cost function must have the ASI property

The algorithm was presented by Ibaraki and Kameda [438]. Later Krishnamurthy, Boral, and Zaniolo presented it again for some other cost functions (still having the ASI property) [520]. They also observed that the upper bound

$O(n^2 \log n)$ of the original algorithm could be improved to $O(n^2)$. In any case, the algorithm is based on an algorithm discovered by Monma and Sidney for job scheduling [537, 627]. Let us call the (unimproved) algorithm IKKBZ-Algorithm.

The IKKBZ-Algorithm considers only join operations that have a cost function of the form:

$$\text{cost}(R_i \bowtie R_j) = |R_i| * h_j(|R_j|)$$

where each R_j can have its own cost function h_j . We denote the set of h_j by H and parameterize cost functions with it. Example instantiations are

- $h_j \equiv 1.2$ for main memory hash-based joins
- $h_j \equiv \text{id}$ for nested-loop joins

where id is the identity function. Let us denote by n_i the cardinality of the relation R_i ($n_i := |R_i|$). Then, the $h_i(n_i)$ represent the costs per input tuple to be joined with R_i .

The algorithm works as follows. For every relation R_k it computes the optimal join order under the assumption that R_k is the first relation in the join sequence. The resulting subproblems then resemble a job-scheduling problem that can be solved by the Monma-Sidney-Algorithm [627].

In order to present this algorithm, we need the notion of a *precedence graph*. A *precedence graph* is formed by taking a node in the (undirected) query graph and making this node a root node of a (directed) tree where the edges point away from the selected root node. Hence, for acyclic, connected query graphs—those we consider in this section—a precedence graph is a tree. We construct the precedence graph of a query graph $G = (V, E)$ as follows:

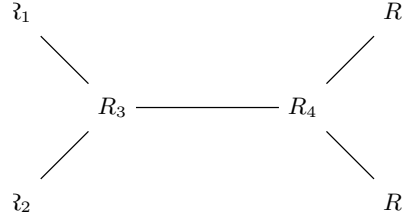
- Make some relation $R_k \in V$ the root node of the precedence graph.
- As long as not all relations are included in the precedence graph: Choose a relation $R_i \in V$, such that $(R_j, R_i) \in E$ is an edge in the query graph and R_j is already contained in the (partial) precedence graph constructed so far and R_i is not. Add R_j and the edge $R_j \rightarrow R_i$ to the precedence graph.

A sequence $S = v_1, \dots, v_k$ of nodes conforms to a precedence graph $G = (V, E)$ if the following conditions are satisfied:

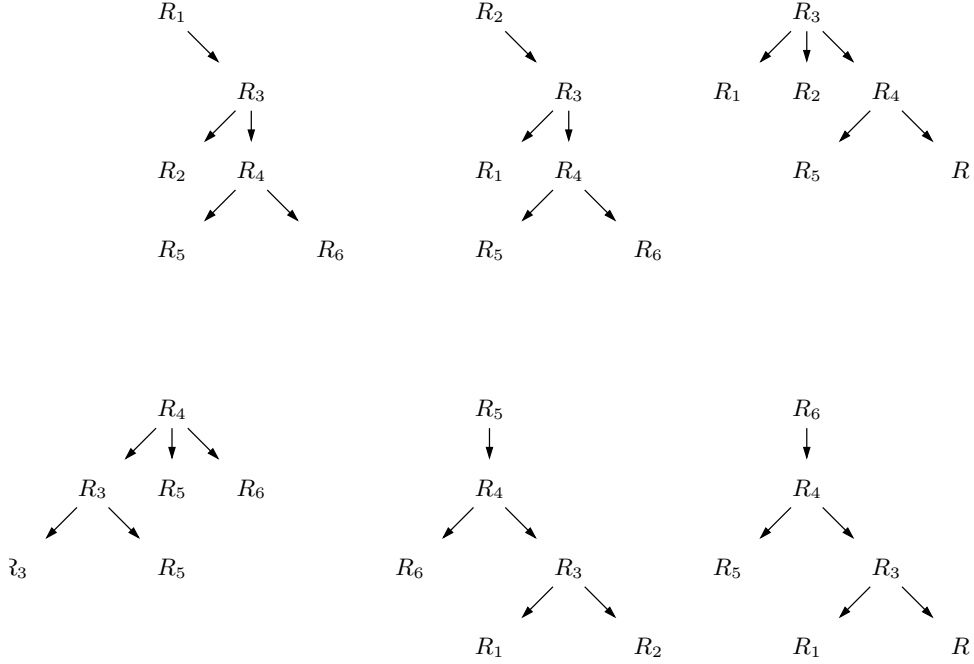
1. for all i ($2 \leq i \leq k$) there exists a j ($1 \leq j < i$) with $(v_j, v_i) \in E$ and
2. there is no edge $(v_i, v_j) \in E$ for $i > j$.

For non-empty sequences U and V in a precedence graph, we write $U \rightarrow V$ if, according to the precedence graph, U must occur before V . This requires U and V to be disjoint. More precisely, there can only be paths from nodes in U to nodes in V and at least one such path exists.

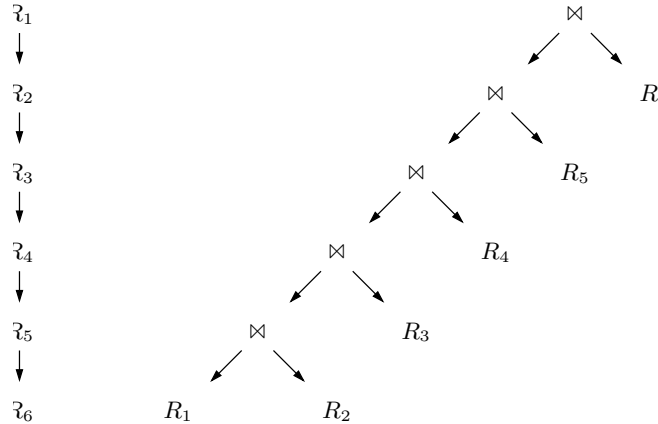
Consider the following query graph:



For this query graph, we can derive the following precedence graphs:



The IKKBZ-Algorithm takes a single precedence graph and produces a new one that is totally ordered. From this total order it is very easy to construct a corresponding join graph: the following figure contains a totally ordered precedence graph (left-hand side) as generated by the IKKBZ-Algorithm and the corresponding join graph on the right-hand side.



Define

$$\begin{aligned} R_{1,2,\dots,k} &:= R_1 \bowtie R_2 \bowtie \dots \bowtie R_k \\ n_{1,2,\dots,k} &:= |R_{1,2,\dots,k}| \end{aligned}$$

For a given precedence graph, let R_i be a relation and $\mathcal{R}_i$ be the set of relations from which there exists a path to R_i . Then, in any join tree adhering to the precedence graph, all relations in $\mathcal{R}_i$ and only those will be joined before R_i . Hence, we can define $s_i = \prod_{R_j \in \mathcal{R}_i} f_{i,j}$ for $i > 1$. Note that for any i only one j with $f_{i,j} \neq 1$ exists in the product. If the precedence graph is a chain, then the following holds:

$$n_{1,2,\dots,k+1} = n_{1,2,\dots,k} * s_{k+1} * n_{k+1}$$

We define $s_1 = 1$. Then we have

$$n_{1,2} = s_2 * (n_1 * n_2) = (s_1 * s_2) * (n_1 * n_2)$$

and, in general,

$$n_{1,2,\dots,k} = \prod_{i=1}^k (s_i * n_i).$$

We call the s_i selectivities, although they depend on the precedence graph.

The costs for a totally ordered precedence graph G can thus be computed as follows:

$$\begin{aligned} Cost_H(G) &= \sum_{i=2}^n [n_{1,2,\dots,i-1} * h_i(n_i)] \\ &= \sum_{i=2}^n [(\prod_{j=1}^{i-1} s_j * n_j) * h_i(n_i)] \end{aligned}$$

If we define $h_i(n_i) = s_i n_i$, then $Cost_H \equiv C_{out}$. The factor $s_i n_i$ determines by how much the input relation to be joined with R_i changes its cardinality after the join has been performed. If $s_i n_i$ is less than one, we call the join *decreasing*, if it is larger than one, we call the join *increasing*. This distinction plays an important role in the heuristic discussed in Section 3.2.3.

The cost function can also be defined recursively.

Definition 3.2.1 Define the cost function C_H as follows:

$$\begin{aligned} C_H(\epsilon) &= 0 \\ C_H(R_j) &= 0 \quad \text{if } R_j \text{ is the root} \\ C_H(R_j) &= h_j(n_j) \quad \text{else} \\ C_H(S_1 S_2) &= C_H(S_1) + T(S_1) * C_H(S_2) \end{aligned}$$

where

$$\begin{aligned} T(\epsilon) &= 1 \\ T(S) &= \prod_{R_i \in S} (s_i * n_i) \end{aligned}$$

It is easy to prove by induction that C_H is well-defined and that $C_H(G) = \text{Cost}_H(G)$. EX

Definition 3.2.2 *Let A and B be two sequences and V and U two non-empty sequences. We say that a cost function C has the adjacent sequence interchange property (ASI property) if and only if there exists a function T and a rank function defined for sequences S as*

$$\text{rank}(S) = \frac{T(S) - 1}{C(S)}$$

such that for non-empty sequences $S = AUVB$ the following holds

$$C(AUVB) \leq C(AVUB) \quad \prec \succ \quad \text{rank}(U) \leq \text{rank}(V) \quad (3.1)$$

if $AUVB$ and $AVUB$ satisfy the precedence constraints imposed by a given precedence graph.

Lemma 3.2.3 *The cost function C_H defined in Definition 3.2.1 has the ASI property.*

The proof is very simple. Using the definition of C_H , we have

$$\begin{aligned} C_H(AUVB) &= C_H(A) \\ &\quad + T(A)C_H(U) \\ &\quad + T(A)T(U)C_H(V) \\ &\quad + T(A)T(U)T(V)C_H(B) \end{aligned}$$

and, hence,

$$\begin{aligned} C_H(AUVB) - C_H(AVUB) &= T(A)[C_H(V)(T(U) - 1) - C_H(U)(T(V) - 1)] \\ &= T(A)C_H(U)C_H(V)[\text{rank}(U) - \text{rank}(V)] \end{aligned}$$

The proposition follows. □

Definition 3.2.4 *Let $M = \{A_1, \dots, A_n\}$ be a set of node sequences in a given precedence graph. Then, M is called a module if for all sequences B that do not overlap with the sequences in M one of the following conditions holds:*

- $B \rightarrow A_i, \forall 1 \leq i \leq n$
- $A_i \rightarrow B, \forall 1 \leq i \leq n$
- $B \not\rightarrow A_i$ and $A_i \not\rightarrow B, \forall 1 \leq i \leq n$

Lemma 3.2.5 *Let C be any cost function with the ASI property and $\{A, B\}$ a module. If $A \rightarrow B$ and additionally $\text{rank}(B) \leq \text{rank}(A)$, then we find an optimal sequence among those in which B directly follows A .*

Proof Every optimal permutation must have the form (U, A, V, B, W) , since $A \rightarrow B$. Assumption: $V \neq \epsilon$. If $\text{rank}(V) \leq \text{rank}(A)$, then we can exchange V and A without increasing the costs. If $\text{rank}(A) \leq \text{rank}(V)$, we have $\text{rank}(B) \leq \text{rank}(V)$ due to the transitivity of $\leq$. Hence, we can exchange B and V without increasing the costs. Both exchanges produce legal sequences obeying the precedence graph, since $\{A, B\}$ is a module. $\square$

If the precedence graph demands $A \rightarrow B$ but $\text{rank}(B) \leq \text{rank}(A)$, we speak of *contradictory sequences* A and B . Since the lemma shows that no non-empty subsequence can occur between A and B , we will combine A and B into a new single node replacing A and B . This node represents a *compound relation* comprising all relations in A and B . Its cardinality is computed by multiplying the cardinalities of all relations occurring in A and B , and its selectivity s is the product of all the selectivities s_i of the relations R_i contained in A and B . The continued process of this step until no more contradictory sequence exists is called *normalization*. The opposite step, replacing a compound node by the sequence of relations it was derived from, is called *denormalization*.

We can now present the algorithm IKKBZ.

IKKBZ(G)

Input: an acyclic query graph G for relations $R_1, \dots, R_n$

Output: the best left-deep tree

$R = \emptyset$;

for ($i = 1$; $i \leq n$; $++i$) {

 Let G_i be the precedence graph derived from G and rooted at R_i ;

$T = \text{IKKBZ-Sub}(G_i)$;

$R+ = T$;

}

return best of R ;

IKKBZ-Sub(G_i)

Input: a precedence graph G_i for relations $R_1, \dots, R_n$ rooted at some R_i

Output: the optimal left-deep tree under G_i

while (G_i is not a chain) {

 let r be the root of a subtree in G_i whose subtrees are chains;

 IKKBZ-Normalize(r);

 merge the chains under r according to the rank function

 in ascending order;

}

IKKBZ-Denormalize(G_i);

return G_i ;

IKKBZ-Normalize(r)

Input: the root r of a subtree T of a precedence graph $G = (V, E)$

Output: a normalized subchain

while ($\exists r', c \in V, r \rightarrow^* r', (r', c) \in E: \text{rank}(r') > \text{rank}(c)$) {

 replace r' by a compound relation r'' that represents $r'c$;

};

We do not give the details of IKKBZ-Denormalize, as it is trivial.

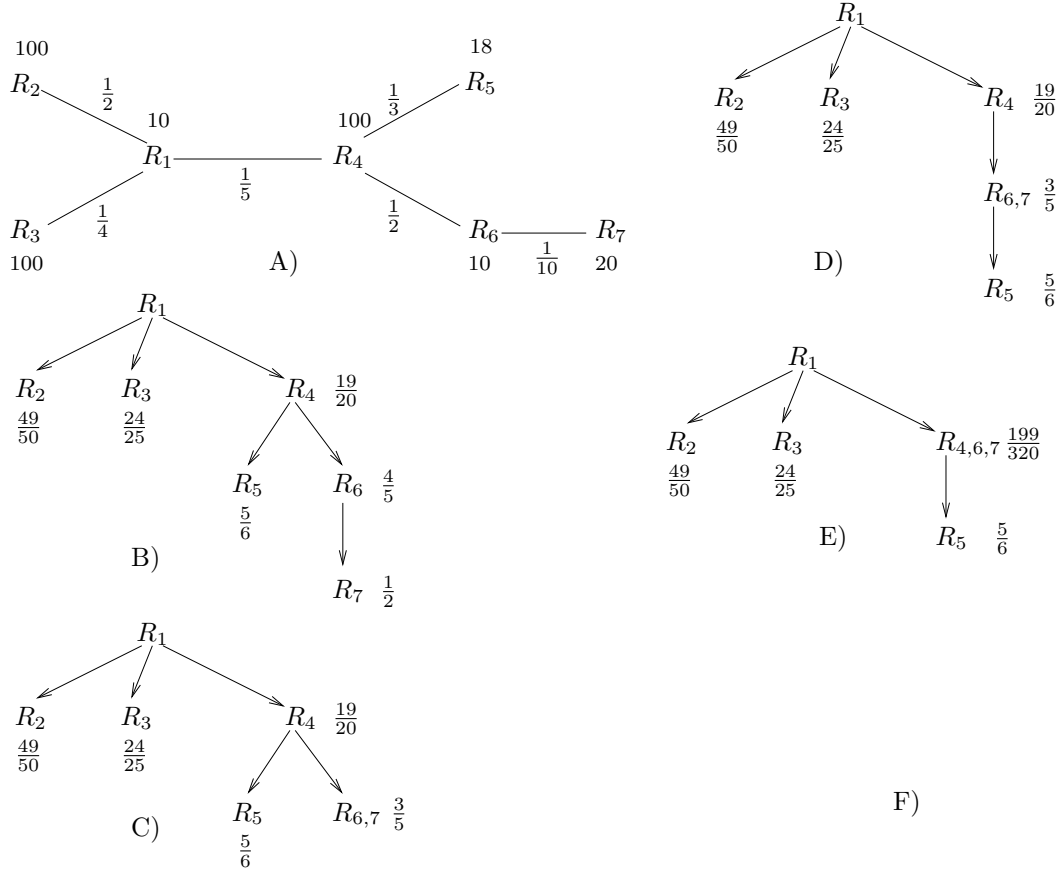


Figure 3.3: Illustrations for the IKKBZ Algorithm

Let us illustrate the algorithm IKKBZ-Sub by a simple example. We use the cost function C_{out} . Figure 3.3 A) shows a query graph. The relations are annotated with their sizes and the edges with the join selectivities. Choosing R_1 as the root of the precedence graph results in B). There, the nodes are annotated by the ranks of the relations. R_4 is the root of a subtree all of whose subtrees are chains. Hence, we normalize it. For R_5 , there is nothing to do. The ranks of R_6 and R_7 are contradictory. We form a compound relation $R_{6,7}$, calculate its cardinality, selectivity, and rank. The latter is shown in C). Merging the two subchains under R_4 results in D). Now R_1 is the root of a subtree with only chains underneath. Normalization detects that the ranks for R_4 and R_5 are contradictory. E) shows the tree after introducing the compound relation $R_{4,5}$. Now $R_{4,5}$ and $R_{6,7}$ have contradictory ranks, and we replace them by the compound relation $R_{4,5,6,7}$ as shown in F). Merging the chains under R_1 gives G). Since this is a chain, we leave the loop and denormalize. The final result is shown in H).

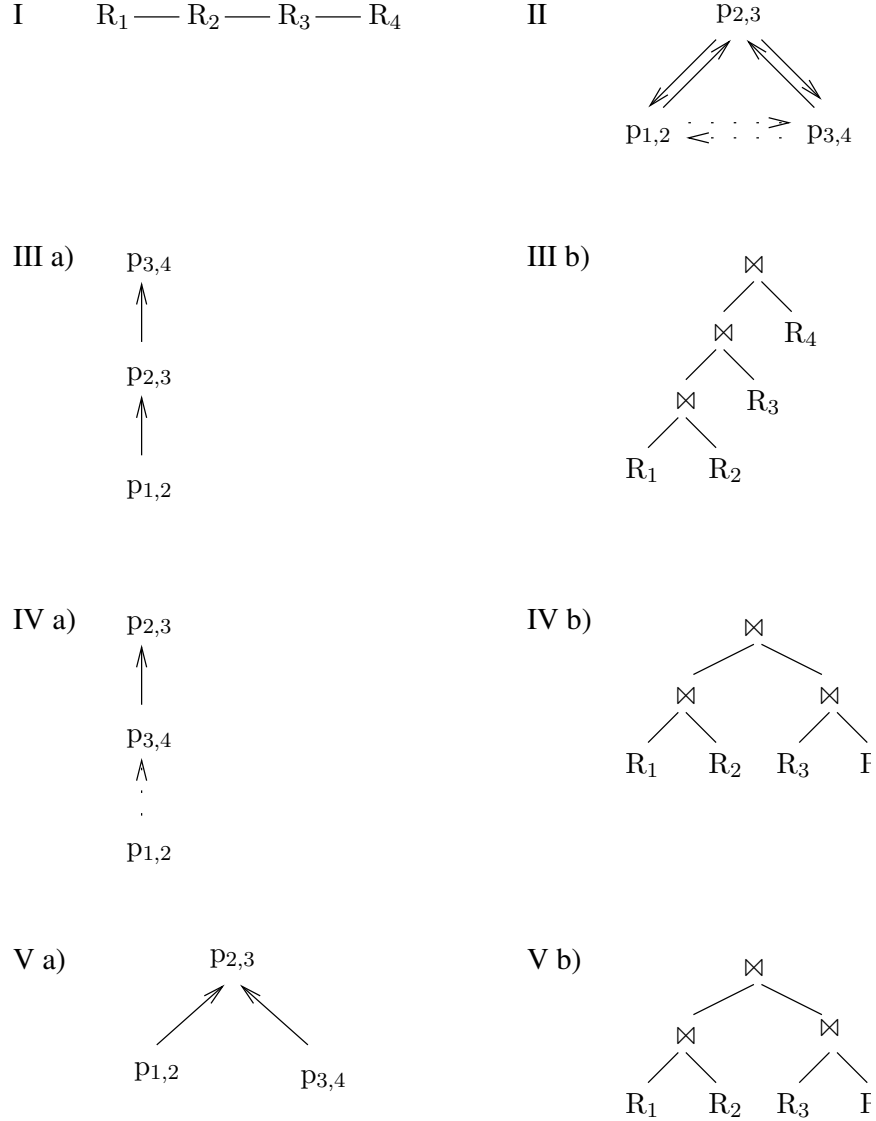


Figure 3.4: A query graph, its directed join graph, some spanning trees and join trees

We can use the IKKBZ-Algorithm to derive a heuristics also for cyclic queries, i.e. for general query graphs. In a first step, we determine a minimal spanning tree of the query graph. It is then used as the input query graph for the IKKBZ-Algorithm. Let us call this the *IKKBZ-based Heuristics*.

3.2.3 The Maximum-Value-Precedence Algorithm

Lee, Shih, and Chen proposed a very interesting heuristics for the join ordering problem [539]. They use a *weighted directed join graph* (WDJG) to represent queries. Within this graph, every join tree corresponds to a spanning tree.

Given a conjunctive query with join predicates P . For a join predicate $p \in P$, we denote by $\mathcal{R}(p)$ the relations whose attributes are mentioned in p .

Definition 3.2.6 *The directed join graph of a conjunctive query with join predicates P is a triple $G = (V, E_p, E_v)$, where V is the set of nodes and E_p and E_v are sets of directed edges defined as follows. For any two nodes $u, v \in V$, if $\mathcal{R}(u) \cap \mathcal{R}(v) \neq \emptyset$ then $(u, v) \in E_p$ and $(v, u) \in E_p$. If $\mathcal{R}(u) \cap \mathcal{R}(v) = \emptyset$, then $(u, v) \in E_v$ and $(v, u) \in E_v$. The edges in E_p are called physical edges, those in E_v virtual edges.*

Note that in G for every two nodes u, v , there is an edge (u, v) that is either physical or virtual. Hence, G is a clique.

Let us see how we can derive a join tree from a spanning tree of a directed join graph. Figure 3.4 I) gives a simple query graph Q corresponding to a chain and Part II) presents Q 's directed join graph. Physical edges are drawn by solid arrows, virtual edges by dotted arrows. Let us first consider the spanning tree shown in Part III a). It says that we first execute $R_1 \bowtie_{p_{1,2}} R_2$. The next join predicate to evaluate is $p_{2,3}$. Obviously, it does not make much sense to execute $R_2 \bowtie_{p_{2,3}} R_3$, since R_1 and R_2 have already been joined. Hence, we replace R_2 in the second join by the result of the first join. This results in the join tree $(R_1 \bowtie_{p_{1,2}} R_2) \bowtie_{p_{2,3}} R_3$. For the same reason, we proceed by joining this result with R_4 . The final join tree is shown in Part III b). Part IV a) shows another spanning tree. The two joins $R_1 \bowtie_{p_{1,2}} R_2$ and $R_3 \bowtie_{p_{3,4}} R_4$ can be executed independently and do not influence each other. Next, we have to consider $p_{2,3}$. Both R_2 and R_3 have already been joined. Hence, the last join processes both intermediate results. The final join tree is shown in Part IV b). The spanning tree shown in Part V a) results in the same join tree shown in Part V b). Hence, two different spanning trees can result in the same join tree. However, the spanning tree in Part IV a) is more specific in that it demands $R_1 \bowtie_{p_{1,2}} R_2$ to be executed before $R_3 \bowtie_{p_{3,4}} R_4$.

Next, take a look at Figure 3.5. Part I), II), and III a) show a query graph, its directed join tree and a spanning tree. To build a join tree from the spanning tree we proceed as follows. We have to execute $R_2 \bowtie_{p_{2,3}} R_3$ and $R_3 \bowtie R_4$ first. In which way we do so is not really fixed by the spanning tree. So let us do both in parallel. Next is $p_{1,2}$. The only dependency the spanning tree gives us is that it should be executed after $p_{3,4}$. Since there is no common relation between those two, we perform $R_1 \bowtie_{p_{1,2}} R_2$. Last is $p_{4,5}$. Since we find $p_{3,4}$ below it, we use the intermediate result produced by it as a replacement for R_4 . The result is shown in Part III b). It has three loose ends. Additional joins are required to tie the partial results together. Obviously, this is not what we want. A spanning tree that avoids this problem of additional joins is called *effective*. It can be shown that a spanning tree $T = (V, E)$ is effective if it satisfies the following conditions [539]:

1. T is a binary tree,
2. for all inner nodes v and node u with $(u, v) \in E$ it holds that $\mathcal{R}^*(T(u)) \cap \mathcal{R}(v) \neq \emptyset$, and

$$\text{I} \quad R_1 \text{ --- } R_2 \text{ --- } R_3 \text{ --- } R_4 \text{ --- } R_5$$

$$\text{II} \quad p_{1,2} \rightleftarrows p_{2,3} \rightleftarrows p_{3,4} \rightleftarrows p_{4,5}$$

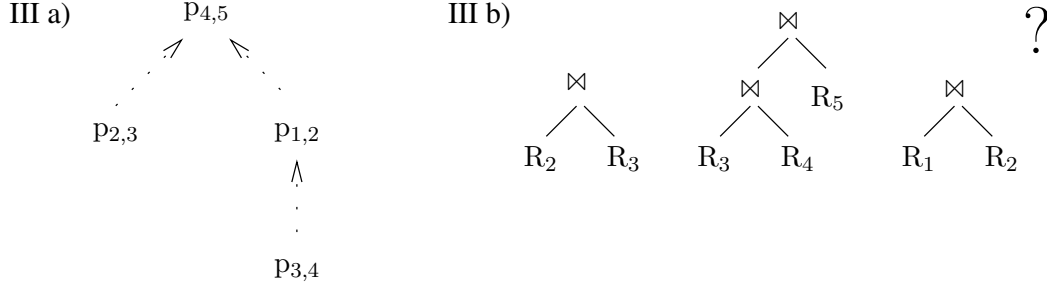


Figure 3.5: A query graph, its directed join tree, a spanning tree and its problem

3. for all nodes v, u_1, u_2 with $u_1 \neq u_2$, $(u_1, v) \in E$, and $(u_2, v) \in E$ one of the following two conditions holds:

- (a) $((\mathcal{R}^*(T(u_1)) \cap \mathcal{R}(v)) \cap (\mathcal{R}^*(T(u_2)) \cap \mathcal{R}(v))) = \emptyset$ or
- (b) $(\mathcal{R}^*(T(u_1)) \cap \mathcal{R}(v) = \mathcal{R}(v)) \vee (\mathcal{R}^*(T(u_2)) \cap \mathcal{R}(v) = \mathcal{R}(v))$.

Thereby, we denote by $T(v)$ the partial tree rooted at v and by $\mathcal{R}^*(T') = \cup_{v \in T'} \mathcal{R}(v)$ the set of all relations in subtree T' .

We see that the spanning tree in Figure 3.5 III a) is ineffective since, for example, $\mathcal{R}(p_{2,3}) \cap \mathcal{R}(p_{4,5}) = \emptyset$. The spanning tree in Figure 3.4 IV a) is also ineffective. During the algorithm we will take care—by checking the above conditions—that only effective spanning trees are generated.

We now assign weights to the edges of the directed join graph. For two nodes $v, u \in V$ define $u \sqcap v := \mathcal{R}(u) \cap \mathcal{R}(v)$. For simplicity, we assume that every predicate involves exactly two relations. Then for all $u, v \in V$, $u \sqcap v$ contains a single relation. Let $v \in V$ be a node with $\mathcal{R}(v) = \{R_i, R_j\}$. We abbreviate $R_i \bowtie_v R_j$ by $\bowtie_v$. Using these notations, we can attach weights to the edges to define the *weighted directed join graph*.

Definition 3.2.7 Let $G = (V, E_p, E_v)$ be a directed join graph for a conjunctive query with join predicates P . The weighted directed join graph is derived from G by attaching a weight to each edge as follows:

- Let $(u, v) \in E_p$ be a physical edge. The weight $w_{u,v}$ of (u, v) is defined as

$$w_{u,v} = \frac{|\bowtie_u|}{|u \sqcap v|}.$$

- For virtual edges $(u, v) \in E_v$, we define $w_{u,v} = 1$.

(Lee, Shih, and Chen actually attach two weights to each edge: one additional weight for the size of the tuples (in bytes) [539].)

The weights of physical edges are equal to the s_i of the dependency graph used in the IKKBZ-Algorithm (Section 3.2.2). To see this, assume $\mathcal{R}(u) = \{R_1, R_2\}$, $\mathcal{R}(v) = \{R_2, R_3\}$. Then

$$\begin{aligned} w_{u,v} &= \frac{|\bowtie_u|}{|u \sqcap v|} \\ &= \frac{|R_1 \bowtie_u R_2|}{|R_2|} \\ &= \frac{f_{1,2} |R_1| |R_2|}{|R_2|} \\ &= f_{1,2} |R_1| \end{aligned}$$

Hence, if the join $R_1 \bowtie_u R_2$ is executed before the join $R_2 \bowtie_v R_3$, the input size to the latter join changes by a factor $w_{u,v}$. This way, the influence of a join on another join is captured by the weights. Since those nodes connected by a virtual edge do not influence each other, a weight of 1 is appropriate.

Additionally, we assign weights to the nodes of the directed join graph. The weight of a node reflects the change in cardinality to be expected when certain other joins have been executed before. They are specified by a (partial) spanning tree S . Given S , we denote by $\bowtie_{p_{i,j}}^S$ the result of the join $\bowtie_{p_{i,j}}$ if all joins preceding $p_{i,j}$ in S have been executed. Then the weight attached to node $p_{i,j}$ is defined as

$$w(p_{i,j}, S) = \frac{|\bowtie_{p_{i,j}}^S|}{|R_i \bowtie_{p_{i,j}} R_j|}.$$

For empty sequences ϵ , we define $w(p_{i,j}, \epsilon) = |R_i \bowtie_{p_{i,j}} R_j|$. Similarly, we define the cost of a node $p_{i,j}$ depending on other joins preceding it in some given spanning tree S . We denote this by $\text{cost}(p_{i,j}, S)$. The actual cost function can be one we have introduced so far or any other one. In fact, if we have a choice of several join implementations, we can take the minimum over all their cost functions. This then chooses the most effective join implementation.

The maximum value precedence algorithm works in two phases. In the first phase, it searches for edges with a weight smaller than one. Among these, the one with the biggest impact is chosen. This one is then added to the spanning tree. In other words, in this phase, the costs of expensive joins are minimized by making sure that *(size) decreasing joins* are executed first. The second phase adds edges such that the intermediate result sizes increase as little as possible.

MVP(G)

Input: a weighted directed join graph $G = (V, E_p, E_v)$

Output: an effective spanning tree

```

 $Q_1.\text{insert}(V);$  /* priority queue with largest node weights  $w(\cdot)$  first */
 $Q_2 = \emptyset;$  /* priority queue with smallest node weights  $w(\cdot)$  first */
 $G' = (V', E')$  with  $V' = V$  and  $E' = E_p;$  /* working graph */

```



```

 $S = (V_S, E_S)$  with  $V_S = V$  and  $E_S = \emptyset$ ; /* resulting effective spanning tree */
while (! $Q_1.empty()$  &&  $|E_S| < |V| - 1$ ) { /* Phase I */
   $v = Q_1.head()$ ;
  among all  $(u, v) \in E'$ ,  $w_{u,v} < 1$  such that
     $S' = (V, E'_S)$  with  $E'_S = E_S \cup \{(u, v)\}$  is acyclic and effective
    select one that maximizes  $cost(\bowtie_v, S) - cost(\bowtie_v, S')$ ;
  if (no such edge exists) {
     $Q_1.remove(v)$ ;
     $Q_2.insert(v)$ ;
    continue;
  }
  MvpUpdate( $(u, v)$ );
  recompute  $w(\cdot)$  for  $v$  and its ancestors; /* rearranges  $Q_1$  */
}
while (! $Q_2.empty()$  &&  $|E_S| < |V| - 1$ ) { /* Phase II */
   $v = Q_2.head()$ ;
  among all  $(u, v), (v, u) \in E'$  denoted by  $(x, y)$  henceforth
    such that
     $S' = (V, E'_S)$  with  $E'_S = E_S \cup \{(x, y)\}$  is acyclic and effective
    select the one that minimizes  $cost(\bowtie_v, S') - cost(\bowtie_v, S)$ ;
  MvpUpdate( $(x, y)$ );
  recompute  $w(\cdot)$  for  $y$  and its ancestors; /* rearranges  $Q_2$  */
}
return  $S$ ;

MvpUpdate( $(u, v)$ )
Input: an edge to be added to  $S$ 
Output: side-effects on  $S$ ,  $G'$ ,
   $E_S \cup = \{(u, v)\}$ ;
   $E' \setminus = \{(u, v), (v, u)\}$ ;
   $E' \setminus = \{(u, w) | (u, w) \in E'\}$ ; /* (1) */
   $E' \cup = \{(v, w) | (u, w) \in E_p, (v, w) \in E_v\}$ ; /* (3) */
  if ( $v$  has two inflowing edges in  $S$ ) { /* (2) */
     $E' \setminus = \{(w, v) | (w, v) \in E'\}$ ;
  }
  if ( $v$  has one outflowing edge in  $S$ ) { /* (1) in paper but not needed */
     $E' \setminus = \{(v, w) | (v, w) \in E'\}$ ;
  }
}

```

Note that in order to test for the effectiveness of a spanning tree in the algorithm, we just have to check the conditions for the node the selected edge leads to.

MvpUpdate first adds the selected edge to the spanning tree. It then eliminates edges that need not to be considered for building an effective spanning tree. Since (u, v) has been added, both (u, v) and (v, u) do not have to be considered any longer. Also, since effective spanning trees are binary trees, (1)

every node must have only one parent node and (2) at most two child nodes. The edges leading to a violation are eliminated by `MvpUpdate` in the lines commented with the corresponding numbers. For the line commented (3) we have the situation that $u \rightarrow v \dashrightarrow w$ and $u \rightarrow w$ in G . This means that u and w have common relations, but v and w do not. Hence, the result of performing v on the result of u will have a common relation with w . Thus, we add a (physical) edge $v \rightarrow w$.

3.2.4 Dynamic Programming

Algorithms

Consider the two join trees

$$(((R_1 \bowtie R_2) \bowtie R_3) \bowtie R_4) \bowtie R_5$$

and

$$(((R_3 \bowtie R_1) \bowtie R_2) \bowtie R_4) \bowtie R_5.$$

If we know that $((R_1 \bowtie R_2) \bowtie R_3)$ is cheaper than $((R_3 \bowtie R_1) \bowtie R_2)$, we know that the first join tree is cheaper than the second. Hence, we could avoid generating the second alternative and still won't miss the optimal join tree. The general principle behind this is the *optimality principle* (see [208]). For the join ordering problem, it can be stated as follows.¹

Let T be an optimal join tree for relations $R_1, \dots, R_n$. Then, every subtree S of T must be an optimal join tree for the relations it contains.

To see why this holds, assume that the optimal join tree T for relations $R_1, \dots, R_n$ contains a subtree S which is not optimal. That is, there exists another join tree S' for the relations contained in S with strictly lower costs. Denote by T' the join tree derived by replacing S in T by S' . Since S' contains the same relations as S , T' is a join tree for the relations $R_1, \dots, R_n$. The costs of the join operators in T and T' that are not contained in S and S' are the same. Then, since the total cost of a join tree is the sum of the costs of the join operators and S' has lower costs than S , T' has lower costs than T . This contradicts the optimality of T .

The idea of dynamic programming applied to the generation of optimal join trees now is to generate optimal join trees for subsets of $R_1, \dots, R_n$ in a bottom-up fashion. First, optimal join trees for subsets of size one, i.e. single relations, are generated. From these, optimal join trees of size two, three and so on until n are generated.

Let us first consider generating optimal left-deep trees. There, join trees for subsets of size k are generated from subsets of size $k - 1$ by adding a new join operator whose left argument is a join tree for $k - 1$ relations and whose right argument is a single relation. Exchanging left and right gives us the procedure for generating right-deep trees. If we want to generate zig-zag trees since our

¹The optimality principle does not hold in the presence of properties.

cost function is asymmetric, we have to consider both alternatives and take the cheapest one. We capture this in a procedure `CreateJoinTree` that takes two join trees as arguments and generates the above-mentioned alternatives. In case we want to consider different implementations for the join, we have to perform the above steps for all of them and return the cheapest alternative. Summarizing, the pseudo-code for `CreateJoinTree` looks as follows:

```

CreateJoinTree( $T_1$ ,  $T_2$ )
Input: two (optimal) join trees  $T_1$  and  $T_2$ .
           for linear trees, we assume that  $T_2$  is a single relation
Output: an (optimal) join tree for joining  $T_1$  and  $T_2$ .
BestTree = NULL;
for all implementations impl do {
    if(!RightDeepOnly) {
        Tree =  $T_1 \bowtie^{impl} T_2$ 
        if (BestTree == NULL || cost(BestTree) > cost(Tree)) {
            BestTree = Tree;
        }
    }
    if(!LeftDeepOnly) {
        Tree =  $T_2 \bowtie^{impl} T_1$ 
        if (BestTree == NULL || cost(BestTree) > cost(Tree)) {
            BestTree = Tree;
        }
    }
}
return BestTree;

```

The boolean variables `RightDeepOnly` and `LeftDeepOnly` are used to restrict the search space to right-deep trees and left-deep trees. If both are false, zig-zag trees are generated. However, `CreateJoinTree` also generates bushy trees, if none of the input trees is a single relation.

In case of linear trees, T_2 will be the single relation in all of our algorithms. `CreateJoinTree` should not copy T_1 or T_2 . Instead, the newly generated join trees should share T_1 and T_2 by using pointers. Further, the join trees generated do not really need to be generated except for the final (best) join tree: the cost functions should be implemented such that they can be evaluated if they are given the left and right argument of the join.

Using `CreateJoinTree`, we are now ready to present our first dynamic programming algorithm in pseudo-code.

```

DP-Linear-1( $\{R_1, \dots, R_n\}$ )

```

Input: a set of relations to be joined

Output: an optimal left-deep (right-deep, zig-zag) join tree

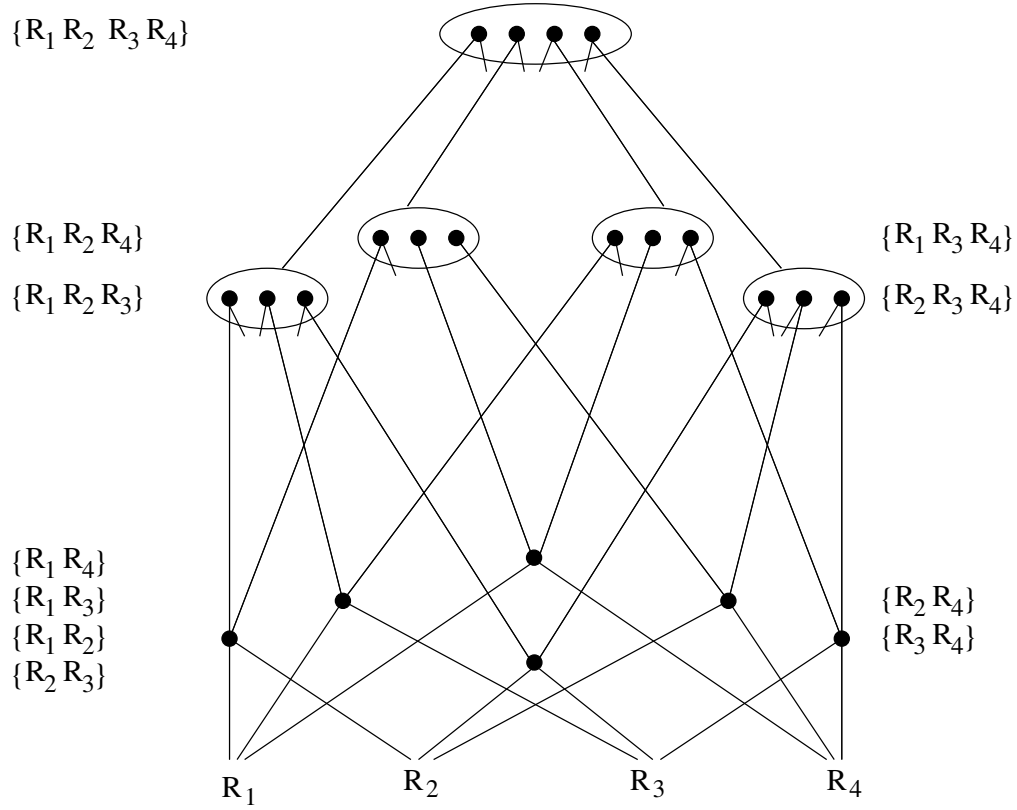


Figure 3.6: Search space with sharing under optimality principle

```

for (i = 1; i <= n; ++i) {
    BestTree({Ri}) = Ri;
}
for (i = 1; i < n; ++i) {
    for all  $S \subseteq \{R_1, \dots, R_n\}, |S| = i$  do {
        for all  $R_j \in \{R_1, \dots, R_n\}, R_j \notin S$  do {
            if (NoCrossProducts && !connected({Rj}, S)) {
                continue;
            }
            CurrTree = CreateJoinTree(BestTree(S), Rj);
            S' = S ∪ {Rj};
            if (BestTree(S') == NULL || cost(BestTree(S')) > cost(CurrTree)) {
                BestTree(S') = CurrTree;
            }
        }
    }
}
return BestTree({R1, ..., Rn});

```

NoCrossProducts is a boolean variable indicating whether cross products should be investigated. Of course, if the join graph is not connected, there must be

XC search
space size
difference
problem

a cross product, but for **DP-Linear-1** and subsequent algorithms we assume that it is connected. The boolean function **connected** returns true, if there is a join predicate between one of the relations in its first argument and one of the relations in its second. The variable **BestTree** keeps track of the best join trees generated for every subset of the relations $\{R_1, \dots, R_n\}$. How this is done may depend on several parameters. The approaches are to use a hash table or an array of size $2^n(-1)$. Another issue is how to represent the sets of relations. Typically, bitvector representations are used. Then, testing for membership, computing a set's complement, adding elements and unioning is cheap. Yet another issue is the order in which join trees are generated. The procedure **DP-Linear-1** takes the approach to generate the join trees for subsets of size $1, 2, \dots, n$. To do so, it must be able to access the subsets of $\{R_1, \dots, R_n\}$ or their respective join trees by their size. One possibility is to chain all the join trees for subsets of a given size k ($1 \leq k \leq n$) and to use an array of size n to keep pointers to the start of the lists. In this case, to every join tree the set of relations it contains is attached, in order to be able to perform the test $R_i \notin S$. One way to do this is to embed a bitvector into each join tree node.

Figure 3.6 illustrates how the procedure **DP-Linear-1** works. In its first loop, it initializes the bottom row of join trees of size one. Then it computes the join trees joining exactly two relations. This is indicated by the next group of join trees. Since the figure leaves out commutativity, only one alternative join tree for every subset of size two is generated. This changes for subsets of size three. There, three alternative join trees are generated. Only the best join tree is retained. This is indicated by the ovals that encircle three join trees. Only this best join tree of size three is used to generate the final best join tree.

The short clarification after the algorithm already adumbrated that the order in which join trees are generated is not compulsory. The only necessary condition is the following.

Let S be a subset of $\{R_1, \dots, R_n\}$. Then, before a join tree for S can be generated, the join trees for all relevant subsets of S must already be available.

EX

Note that this formulation is general enough to also capture the generation of bushy trees. It is, however, a little vague due to its reference to “relevance”. For the different join tree classes, this term can be given a precise semantics.

Let us take a look at an alternative order to join tree generation. Assume that sets of relations are represented as bitvectors. A bitvector is nothing more than a base two integer. Successive increments of an integer/bitvector lead to different subsets. Further, the above condition is satisfied. We illustrate this by a small example. Assume that we have three relations R_1, R_2, R_3 . The i -th bit from the right in a three-bit integer indicates the presence of R_i for $1 \leq i \leq 3$.

000	$\{\}$
001	$\{R_1\}$
010	$\{R_2\}$
011	$\{R_1, R_2\}$
100	$\{R_3\}$
101	$\{R_1, R_3\}$
110	$\{R_2, R_3\}$
111	$\{R_1, R_2, R_3\}$

This observation leads to another formulation of our dynamic programming algorithm. For this algorithm, it is very convenient to use an array of size 2^n to represent $\text{BestTree}(S)$ for subsets S of $\{R_1, \dots, R_n\}$.

DP-Linear-2($\{R_1, \dots, R_n\}$)

Input: a set of relations to be joined

Output: an optimal left-deep (right-deep, zig-zag) join tree

```

for (i = 1; i <= n; ++i) {
    BestTree(1 << i - 1) =  $R_i$ ;
}
for (S = 1; S <  $2^n$ ; ++S) {
    if (BestTree(S) != NULL) continue;
    for all  $i \in S$  do {
         $S' = S \setminus \{i\}$ ;
        CurrTree = CreateJoinTree(BestTree( $S'$ ),  $R_i$ );
        if (BestTree(S) == NULL || cost(BestTree(S)) > cost(CurrTree)) {
            BestTree(S) = CurrTree;
        }
    }
}
return BestTree( $2^n - 1$ );

```

DP-Linear-2 differs from DP-Linear-1 not only in the order in which join trees are generated. Another difference is that it takes cross products into account.

From DP-Linear-2, it is easy to derive an algorithm that explores the space of bushy trees.

DP-Bushy($\{R_1, \dots, R_n\}$)

Input: a set of relations to be joined

Output: an optimal bushy join tree

```

for (i = 1; i <= n; ++i) {
    BestTree(1 << i - 1) =  $R_i$ ;
}
for (S = 1; S <  $2^n$ ; ++S) {
    if (BestTree(S) != NULL) continue;
    for all  $S_1 \subset S$ ,  $S_1 \neq \emptyset$  do {

```

```

    S2 = S \ S1;
    CurrTree = CreateJoinTree(BestTree(S1), BestTree(S2));
    if (BestTree(S) == NULL || cost(BestTree(S)) > cost(CurrTree)) {
        BestTree(S) = CurrTree;
    }
}
}
return BestTree(2n - 1);

```

This algorithm also takes cross products into account. The critical part is the generation of all subsets of S . Fortunately, Vance and Maier [898] provide a code fragment with which subset bitvector representations can be generated very efficiently. In C, this fragment looks as follows:

```

S1 = S & - S;
do {
    /* do something with subset S1 */
    S1 = S & (S1 - S);
} while (S1 != S);

```

S represents the input set. S_1 iterates through all subsets of S where S itself and the empty set are not considered. Analogously, all supersets can be generated as follows:

```

S1 = ~S & - ~S;
/* do something with first superset S1 */
while (S1) {
    S1 = ~S & (S1 - ~S)
    /* do something with superset S1 */
}

```

S represents the input set. S_1 iterates through all supersets of S including S itself.

XC ToDo	Excursion Problem: exploiting orderings devastates the optimality principle. Example: ...
XC ToDo	Excursion Pruning ...

Number of Entries to be stored in the dynamic programming table

If dynamic programming uses a static hash table, determining its size in advance is necessary as the search space sizes differ vastly for different query graphs. In general, for every connected subgraph of the query graph one entry must

exist. Chains require far fewer entries than cliques. It would be helpful to have a small routine solving the following problem: given a query graph, how many connected subgraph are there? Unfortunately, this problem is #P-hard as Sutner, Satyanarayana, and Suffel showed [856]. They build on results by Valiant [896] and Lichtenstein [555]. (For a definition of #P-hard see the book by Lewis and Papadimitriou [553] or the original paper by Valiant [895].)

However, for specific cases, these numbers can be given. If cross products are considered, the number of join trees stored in the dynamic programming table is

$$2^n - 1$$

which is one for each non-empty subset of relations.

If we do not consider cross products, the number of entries in the dynamic programming table corresponds to the number of connected subgraphs of the query graph. For connected query graphs, we denote this by $\#csg$. For chains, cycles, stars, and cliques with n nodes, we have

$$\#csg^{\text{chain}}(n) = \frac{n(n+1)}{2} \quad (3.2)$$

$$\#csg^{\text{cycle}}(n) = n^2 - n + 1 \quad (3.3)$$

$$\#csg^{\text{star}}(n) = 2^{n-1} + n - 1 \quad (3.4)$$

$$\#csg^{\text{clique}}(n) = 2^n - 1 \quad (3.5)$$

These equations can be derived from the following by summing over $k > 1$ where k gives the size of the connected subset:

$$\begin{aligned} \#csg^{\text{chain}}(n, k) &= (n - k + 1) \\ \#csg^{\text{cycle}}(n, k) &= \begin{cases} 1 & n = k \\ n & \text{else} \end{cases} \\ \#csg^{\text{star}}(n, k) &= \begin{cases} n & k = 1 \\ \binom{n-1}{k-1} & k > 1 \end{cases} \\ \#csg^{\text{clique}}(n, k) &= \binom{n}{k} \end{aligned}$$

Number of Join Trees Investigated

The number of join trees investigated by dynamic programming was extensively studied by Ono and Lohman [652, 653]. In order to estimate these numbers, we assume that `CreateJoinTree` produces a single join tree and hence counts as one although it may evaluate the costs for several join alternatives. We further do not count the initial join trees containing only a single relation.

Join Trees With Cartesian Product For the analysis of dynamic programming variants that do consider cross products, the notion of *join-pair* is helpful. Let S_1 and S_2 be subsets of the nodes (relations) of the query graph. We say (S_1, S_2) is a *join-pair*, if and only if

1. S_1 and S_2 are disjoint

If (S_1, S_2) is a join-pair, then (S_2, S_1) is a join pair. Further, if T_1 is a join tree for the relations in S_1 and T_2 is one for those in S_2 , then we can construct two valid join trees $T_1 \bowtie T_2$ and $T_2 \bowtie T_1$ where the joins may be cross products. Hence, the number of join-pairs coincides with the search space a dynamic programming algorithm explores. In fact, the number of join-pairs is the minimum number of join trees any dynamic programming algorithm that considers cross products has to investigate.

If **CreateJoinTree** considers commutativity of joins, the number of calls to it is precisely expressed by the count of non-symmetric join-pairs. In other implementations **CreateJoinTree** might be called for all join-pairs and, thus, may not consider commutativity. The two formulas below only count non-symmetric join pairs.

The numbers of linear and bushy join trees with cartesian product is easiest to determine. They are independent of the query graph. For linear join trees, the number of join trees investigated by dynamic programming is equal to the number of non-symmetric join-pairs which is

$$n2^{n-1} - \frac{n(n+1)}{2}$$

Dynamic programming investigates the following number of bushy trees if cross products are considered.

$$\frac{(3^n - 2^{n+1} + 1)}{2}$$

This is equal to the number of non-symmetric join-pairs.

Join Trees without Cross Products In this paragraph, we assume that the query graph is connected. For the analysis of dynamic programming variants that do not consider cross products, it is helpful to have the notion of a *csg-cmp-pair*. Let S_1 and S_2 be subsets of the nodes (relations) of the query graph. We say (S_1, S_2) is a *csg-cmp-pair*, if and only if

1. S_1 induces a connected subgraph of the query graph,
2. S_2 induces a connected subgraph of the query graph,
3. S_1 and S_2 are disjoint, and
4. there exists at least one edge connected a node in S_1 to a node in S_2 .

If (S_1, S_2) is a csg-cmp-pair, then (S_2, S_1) is a valid csg-cmp-pair. Further, if T_1 is a join tree for the relations in S_1 and T_2 is one for those in S_2 , then we can construct two valid join trees $T_1 \bowtie T_2$ and $T_2 \bowtie T_1$. Hence, the number of csg-cmp-pairs coincides with the search space a dynamic programming algorithm explores. In fact, the number of csg-cmp-pairs is the minimum number of join trees any dynamic programming algorithm that does not consider cross products has to investigate.

If **CreateJoinTree** considers commutativity of joins, the number of calls to it is precisely expressed by the count of non-symmetric csg-cmp-pairs. In other implementations **CreateJoinTree** might be called for all csg-cmp-pairs and, thus, may not consider commutativity.

Let us denote the number of non-symmetric csg-cmp-pairs by $\#ccp$. Then

$$\begin{aligned}\#ccp^{\text{chain}}(n) &= \frac{1}{6}(n^3 - 3n^2 + 2n) \\ \#ccp^{\text{cycle}}(n) &= (n^3 - 2n^2 + n)/2 \\ \#ccp^{\text{star}}(n) &= (n-1)2^{n-2} \\ \#ccp^{\text{clique}}(n) &= (3^n - 2^{n+1} + 1)/2\end{aligned}$$

These numbers have to be multiplied by two if we want to count all csg-cmp-pairs.

If we do not consider composite inners, that is we restrict ourselves to left-deep join trees, then dynamic programming makes the following number of calls to **CreateJoinTree** for chain queries [653]:

$$(n-1)^2$$

The following table presents some results for the above formulas.

	without cross products			with cross products	
	chain		star	any query graph	
	linear	bushy	linear	linear	bushy
n	$(n-1)^2$	$(n^3 - n)/6$	$(n-1)2^{n-2}$	$n2^{n-1} - n(n+1)/2$	$(3^n - 2^{n+1} + 1)/2$
2	1	1	1	1	1
3	4	4	4	6	6
4	9	10	12	22	25
5	16	20	32	65	90
6	25	35	80	171	301
7	36	56	192	420	966
8	49	84	448	988	3025
9	64	120	1024	2259	9330
10	81	165	2304	5065	28501

Compare this table with the actual sizes of the search spaces in Section 3.1.5.

The dynamic programming algorithms can be implemented very efficiently and often form the core of commercial plan generators. However, they have the disadvantage that no plan is generated if they run out of time or space since the search space they have to explore is too big. One possible remedy goes as follows. Assume that a dynamic programming algorithm is stopped in the middle of its way through its actual search space. Further assume that the largest plans generated so far involve k relations. Then the cheapest of the plans with k relations is completed by applying any heuristics (e.g. MinSel). The completed plan is then returned. In Section 3.4.5, we will see two alternative solutions. Another solution is presented in [488].

```

DPsize
Input: a connected query graph with relations  $R = \{R_0, \dots, R_{n-1}\}$ 
Output: an optimal bushy join tree without cross products
for all  $R_i \in R$  {
    BestPlan( $\{R_i\}$ ) =  $R_i$ ;
}
for all  $1 < s \leq n$  ascending // size of plan
for all  $1 \leq s_1 \leq s/2$  { // size of left/right subplan
     $s_2 = s - s_1$ ; // size of right/left subplan
    for all  $S_1 \subset R$  in BestPlan with  $|S_1| = s_1$ 
         $S_2 \subset R$  in BestPlan with  $|S_2| = s_2$  {
            ++InnerCounter;
            if ( $\emptyset \neq S_1 \cap S_2$ ) continue;
            if not ( $S_1$  connected to  $S_2$ ) continue;
            ++CsgCmpPairCounter;
             $p_1 = \text{BestPlan}(S_1)$ ;
             $p_2 = \text{BestPlan}(S_2)$ ;
            CurrPlan = CreateJoinTree( $p_1, p_2$ );
            if ( $\text{cost}(\text{BestPlan}(S_1 \cup S_2)) > \text{cost}(\text{CurrPlan})$ ) {
                BestPlan( $S_1 \cup S_2$ ) = CurrPlan;
            }
        }
    }
}
OnoLohmanCounter = CsgCmpPairCounter / 2;
return BestPlan( $\{R_0, \dots, R_{n-1}\}$ );

```

Figure 3.7: Algorithm DPsize

Generating Bushy Trees without Cross Products

We now discuss dynamic programming algorithms to generate bushy trees without cross products. For this section, we assume that the query graph is connected. We will present three algorithms. The first algorithm (DPsize) generates its plans in increasing size of subplans and, hence, is a generalization of DP-Linear-1. The second algorithm (DPsub) generates its plans by considering plans subsets as does DP-Linear-2. An analysis of these two algorithms reveals that both are far away from the lower bound presented in the previous sections. Thus, a third algorithm (DPccp) which reaches this lower bound is presented. The results of this section are taken from [618, 616].

Size-based enumeration: DPsize In general, dynamic programming generates solutions for a larger problem in a bottom-up fashion by combining solutions for smaller problems. Taking this description literally, we can construct optimal plans of size n by joining plans p_1 and p_2 of size k and $n - k$. We just have to take care that (1) the sets of relations contained in p_1 and p_2 do not overlap, and (2) there is a join predicate connecting a relation p_1 with a relation in p_2 . After this remark, we are prepared to understand the pseudocode

for algorithm **DPsize** (see Fig. 3.7). A table **BestPlan** associates with each set of relations the best plan found so far. The algorithm starts by initializing this table with plans of size one, i.e. single relations. After that, it constructs plans of increasing size (loop over s). Thereby, the first size considered is two, since plans of size one have already been constructed. Every plan joining n relations can be constructed by joining a plan containing s_1 relations with a plan containing s_2 relations. Thereby, $s_i > 0$ and $s_1 + s_2 = n$ must hold. Thus, the pseudocode loops over s_1 and sets s_2 accordingly. Since for every possible size there exist many plans, two more loops are necessary in order to loop over the plans of sizes s_1 and s_2 . (This is best implemented by keeping list heads for every possible plan size pointing to a first plan of this size and chaining plans of equal size via some **next**-pointer.) Then, conditions (1) and (2) from above are tested. Only if their outcome is positive, we consider joining the plans p_1 and p_2 . The result is a plan **CurrPlan**. Let S be the relations contained in **CurrPlan**. If **BestPlan** does not contain a plan for the relations in S or the one it contains is more expensive than **CurrPlan**, we register **CurrPlan** with **BestPlan**.

The algorithm **DPsize** can be made more efficient in case of $s_1 = s_2$. The algorithm as stated cycles through all plans p_1 joining s_1 relations. For each such plan, all plans p_2 of size s_2 are tested. Assume that plans of equal size are represented as a linked list. If $s_1 = s_2$, then it is possible to iterate through the list for retrieving all plans p_1 . For p_2 we consider the plans succeeding p_1 in the list. Thus, the complexity can be decreased from $P(s_1) * P(s_2)$ to $P(s_1) * P(s_2)/2$, where $P(s_i)$ denotes the number of plans of size s_i . The following formulas are valid only for the variant of **DPsize** where this optimization has been incorporated (see [616] for details).

If the counter **InnerCounter** is initialized with zero at the beginning of the algorithm **DPsize**, then we are able to derive analytically its value after **DPsize** terminates. Since this value of the inner counter depends on the query graph, we have to distinguish several cases. For chain, cycle, star, and clique queries, we denote by $I_{\text{DPsize}}^{\text{chain}}$, $I_{\text{DPsize}}^{\text{cycle}}$, $I_{\text{DPsize}}^{\text{star}}$, and $I_{\text{DPsize}}^{\text{clique}}$ the value of **InnerCounter** after termination of algorithm **DPsize**.

For chain queries, we then have: $I_{\text{DPsize}}^{\text{chain}}(n) =$

$$\begin{cases} 1/48(5n^4 + 6n^3 - 14n^2 - 12n) & n \text{ even} \\ 1/48(5n^4 + 6n^3 - 14n^2 - 6n + 11) & n \text{ odd} \end{cases}$$

For cycle queries, we have: $I_{\text{DPsize}}^{\text{cycle}}(n) =$

$$\begin{cases} \frac{1}{4}(n^4 - n^3 - n^2) & n \text{ even} \\ \frac{1}{4}(n^4 - n^3 - n^2 + n) & n \text{ odd} \end{cases}$$

For star queries, we have: $I_{\text{DPsize}}^{\text{star}}(n) =$

$$\begin{cases} 2^{2n-4} - 1/4 \binom{2(n-1)}{n-1} + q(n) & n \text{ even} \\ 2^{2n-4} - 1/4 \binom{2(n-1)}{n-1} + 1/4 \binom{n-1}{(n-1)/2} + q(n) & n \text{ odd} \end{cases}$$

DPsub

Input: a connected query graph with relations $R = \{R_0, \dots, R_{n-1}\}$

Output: an optimal bushy join tree

```

for all  $R_i \in R$  {
    BestPlan( $\{R_i\}$ ) =  $R_i$ ;
}
for  $1 \leq i < 2^n - 1$  ascending {
     $S = \{R_j \in R \mid \lfloor i/2^j \rfloor \bmod 2 = 1\}$ 
    if not (connected  $S$ ) continue;    // *
    for all  $S_1 \subset S, S_1 \neq \emptyset$  do {
        ++InnerCounter;
         $S_2 = S \setminus S_1$ ;
        if ( $S_2 = \emptyset$ ) continue;
        if not (connected  $S_1$ ) continue;
        if not (connected  $S_2$ ) continue;
        if not ( $S_1$  connected to  $S_2$ ) continue;
        ++CsgCmpPairCounter;
         $p_1 = \text{BestPlan}(S_1)$ ;
         $p_2 = \text{BestPlan}(S_2)$ ;
        CurrPlan = CreateJoinTree( $p_1, p_2$ );
        if ( $\text{cost}(\text{BestPlan}(S)) > \text{cost}(\text{CurrPlan})$ ) {
            BestPlan( $S$ ) = CurrPlan;
        }
    }
}
OnoLohmanCounter = CsgCmpPairCounter / 2;
return BestPlan( $\{R_0, \dots, R_{n-1}\}$ );

```

Figure 3.8: Algorithm DPsub

with $q(n) = n2^{n-1} - 5 * 2^{n-3} + 1/2(n^2 - 5n + 4)$. For clique queries, we have:
 $I_{\text{DPsize}}^{\text{clique}}(n) =$

$$\begin{cases} 2^{2n-2} - 5 * 2^{n-2} + 1/4 \binom{2n}{n} - 1/4 \binom{n}{n/2} + 1 & n \text{ even} \\ 2^{2n-2} - 5 * 2^{n-2} + 1/4 \binom{2n}{n} + 1 & n \text{ odd} \end{cases}$$

Note that $\binom{2n}{n}$ is in the order of $\Theta(4^n/\sqrt{n})$.

Proofs of the above formulas as well as implementation details for the algorithm DPsize can be found in [616].

Subset-Driven Enumeration: DPsub Fig. 3.8 presents the pseudocode for the algorithm DPsub. The algorithm first initializes the table **BestPlan** with all possible plans containing a single relation. Then, the main loop starts. It iterates over all possible non-empty subsets of $\{R_0, \dots, R_{n-1}\}$ and constructs the best possible plan for each of them. The enumeration makes use of a bitvector representation of sets: The integer i induces the current subset S with its binary representation. Taken as bitvectors, the integers in the range from 1

to $2^n - 1$ exactly represent the set of all non-empty subsets of $\{R_0, \dots, R_{n-1}\}$, including the set itself. Further, by starting with 1 and incrementing by 1, the enumeration order is valid for dynamic programming: for every subset, all its subsets are generated before the subset itself.

This enumeration is very fast, since increment by one is a very fast operation. However, the relations contained in S may not induce a connected subgraph of the query graph. Therefore, we must test for connectedness. The goal of the next loop over all subsets of S is to find the best plan joining all the relations in S . Therefore, S_1 ranges over all non-empty, strict subsets of S . This can be done very efficiently by applying the code snippet of Vance and Maier [897, 898]. Then, the subset of relations contained in S but not in S_1 is assigned to S_2 . Clearly, S_1 and S_2 are disjoint. Hence, only connectedness tests have to be performed. Since we want to avoid cross products, S_1 and S_2 both must induce connected subgraphs of the query graph, and there must be a join predicate between a relation in S_1 and one in S_2 . If these conditions are fulfilled, we can construct a plan **CurrPlan** by joining the plans associated with S_1 and S_2 . If **BestPlan** does not contain a plan for the relations in S or the one it contains is more expensive than **CurrPlan**, we register **CurrPlan** with **BestPlan**.

For chain, cycle, star, and clique queries, we denote by $I_{\text{DPsub}}^{\text{chain}}$, $I_{\text{DPsub}}^{\text{cycle}}$, $I_{\text{DPsub}}^{\text{star}}$, and $I_{\text{DPsub}}^{\text{clique}}$ the value of **InnerCounter** after termination of algorithm **DPsub**.

For chains, we have

$$I_{\text{DPsub}}^{\text{chain}}(n) = 2^{n+2} - n^n - 3n - 4 \quad (3.6)$$

For cycles, we have

$$I_{\text{DPsub}}^{\text{cycle}}(n) = n2^n + 2^n - 2n^2 - 2 \quad (3.7)$$

For stars, we have

$$I_{\text{DPsub}}^{\text{star}}(n) = 2 * 3^{n-1} - 2^n \quad (3.8)$$

For cliques, we have

$$I_{\text{DPsub}}^{\text{clique}}(n) = 3^n - 2^{n+1} + 1 \quad (3.9)$$

The number of failures for the additional check can easily be calculated as $2^n - \#\text{csg}(n) - 1$.

Sample numbers Fig. 3.9 contains tables with values produced by our formulas for input query graph sizes between 2 and 20. For different kinds of query graphs, it shows the number of csg-cmp-pairs ($\#\text{ccp}$), and the values for the inner counter after termination of **DPsize** and **DPsub** applied to the different query graphs.

Looking at these numbers, we observe the following:

- For chain and cycle queries, the **DPsize** soon becomes much faster than **DPsub**.

	Chain			Cycle		
n	#ccp/2	DPsub	DPsize	#ccp/2	DPsub	DPsize
2	1	2	1	1	2	1
5	20	84	73	40	140	120
10	165	3962	1135	405	11062	2225
15	560	130798	5628	1470	523836	11760
20	1330	4193840	17545	3610	22019294	37900
	Star			Clique		
n	#ccp/2	DPsub	DPsize	#ccp/2	DPsub	DPsize
2	1	2	1	1	2	1
5	32	130	110	90	180	280
10	2304	38342	57888	28501	57002	306991
15	114688	9533170	57305929	7141686	14283372	307173877
20	4980736	2323474358	59892991338	1742343625	3484687250	309338182241

Figure 3.9: Size of the search space for different graph structures

- For star and clique queries, the **DPsub** soon becomes much faster than **DPsize**.
- Except for clique queries, the number of csg-cmp-pairs is orders of magnitude less than the value of *InnerCounter* for all DP-variants.

From the latter observation we can conclude that in almost all cases the tests performed by both algorithms in their innermost loop fail. Both algorithms are far away from the theoretical lower bound given by #ccp. This conclusion motivates us to derive a new algorithm whose *InnerCounter* value is equal to the number of csg-cmp-pairs.

Csg-cmp-pair enumeration-based algorithm: DPccp The algorithm **DPsub** solves the join ordering problem for a given subset S of relations by considering all pairs of disjoint subproblems which were already solved. Since the enumeration of subsets is very fast, this is a very efficient strategy if the search space is dense, e.g. for clique queries. However, if the search space is sparse, e.g. for chain queries, the **DPsub** algorithm considers many subproblems which are not connected and, therefore, are not relevant for the solution, i.e. the tests in the innermost loop fail for the majority of cases. The main idea of our algorithm **DPccp** is that it only considers pairs of connected subproblems. More precisely, the algorithm considers exactly the csg-cmp-pairs of a graph.

Thus, our goal is to efficiently enumerate all csg-cmp-pairs (S_1, S_2) . Clearly, we want to enumerate every pair once and only once. Further, the enumeration must be performed in an order valid for dynamic programming. That is, whenever a pair (S_1, S_2) is generated, all non-empty subsets of S_1 and S_2 must have been generated before as a component of a pair. The last requirement is that the overhead for generating a single csg-cmp-pair must be constant or at most linear. This condition is necessary in order to beat **DPsize** and **DPsub**.

DPccp

Input: a connected query graph with relations $R = \{R_0, \dots, R_{n-1}\}$

Output: an optimal bushy join tree

```

for all  $R_i \in R$  {
    BestPlan( $\{R_i\}$ ) =  $R_i$ ;
}
for all csg-cmp-pairs  $(S_1, S_2)$ ,  $S = S_1 \cup S_2$  {
    ++InnerCounter;
    ++OnoLohmanCounter;
     $p_1$  = BestPlan( $S_1$ );
     $p_2$  = BestPlan( $S_2$ );
    CurrPlan = CreateJoinTree( $p_1, p_2$ );
    if (cost(BestPlan( $S$ )) > cost(CurrPlan)) {
        BestPlan( $S$ ) = CurrPlan;
    }
    CurrPlan = CreateJoinTree( $p_2, p_1$ );
    if (cost(BestPlan( $S$ )) > cost(CurrPlan)) {
        BestPlan( $S$ ) = CurrPlan;
    }
}
CsgCmpPairCounter = 2 * OnoLohmanCounter;
return BestPlan( $\{R_0, \dots, R_{n-1}\}$ );

```

Figure 3.10: Algorithm DPccp

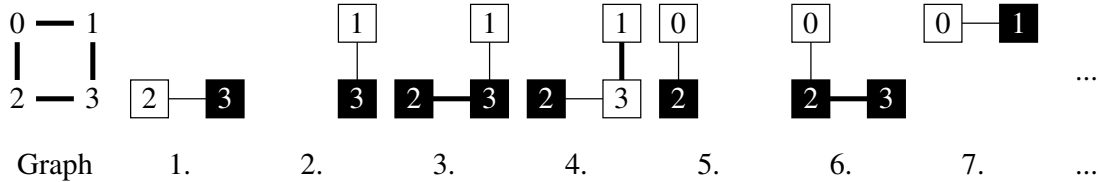


Figure 3.11: Enumeration Example for DPccp

If we meet all these requirements, the algorithm DPccp is easily specified: iterate over all csg-cmp-pairs (S_1, S_2) and consider joining the best plans associated with them. Figure 3.10 shows the pseudocode. The first steps of an example enumeration are shown in Figure 3.11. Thick lines mark the connected subsets while thin lines mark possible join edges. Note that the algorithm explicitly exploits join commutativity. This is due to our enumeration algorithm developed below. If (S_1, S_2) is a csg-cmp-pair, then either (S_1, S_2) or (S_2, S_1) will be generated, but never both of them. An alternative is to modify **CreateJoinTree** to take care of commutativity.

We proceed as follows. Next we discuss an algorithm enumerating non-empty connected subsets S_1 of $\{R_0, \dots, R_{n-1}\}$. Then, we show how to enumerate the complements S_2 such that (S_1, S_2) is a csg-cmp-pair.

Let us start the exposition by fixing some notations. Let $G = (V, E)$ be an undirected graph. For a node $v \in V$ define the *neighborhood* $N(v)$ of v as $N(v) := \{v' | (v, v') \in E\}$. For a subset $S \subseteq V$ of V we define the *neighborhood* of S as $N(S) := \cup_{v \in S} N(v) \setminus S$. The neighborhood of a set of nodes thus consists of all nodes reachable by a single edge. Note that for all $S, S' \subset V$ we have $N(S \cup S') = (N(S) \cup N(S')) \setminus (S \cup S')$. This allows for an efficient bottom-up calculation of neighborhoods.

The following statement gives a hint on how to construct an enumeration procedure for connected subsets. Let S be a connected subset of an undirected graph G and S' be any subset of $N(S)$. Then $S \cup S'$ is connected. As a consequence, a connected subset can be enlarged by adding any subset of its neighborhood.

We could generate all connected subsets as follows. For every node $v_i \in V$ we perform the following enumeration steps: First, we emit $\{v_i\}$ as a connected subset. Then, we expand $\{v_i\}$ by calling a routine that extends a given connected set to bigger connected sets. Let the routine be called with some connected set S . It then calculates the neighborhood $N(S)$. For every non-empty subset $N \subseteq N(S)$, it emits $S' = S \cup N$ as a further connected subset and recursively calls itself with S' . The problem with this routine is that it produces duplicates.

This is the point where the breadth-first numbering comes into play. Let $V = \{v_0, \dots, v_{n-1}\}$, where the indices are consistent with a breadth-first numbering produced by a breadth-first search starting at node v_0 [209]. The idea is to use the numbering to define an enumeration order: In order to avoid duplicates, the algorithm enumerates connected subgraphs for every node v_i , but restricts them to contain no v_j with $j < i$. Using the definition $\mathcal{B}_i = \{v_j | j \leq i\}$, the pseudocode looks as follows:

EnumerateCsg

Input: a connected query graph $G = (V, E)$

Precondition: nodes in V are numbered according to a breadth-first search

Output: emits all subsets of V inducing a connected subgraph of G

for all $i \in [n - 1, \dots, 0]$ **descending** {

emit $\{v_i\}$;

 EnumerateCsgRec($G, \{v_i\}, \mathcal{B}_i$);

}

EnumerateCsgRec(G, S, X)

$N = N(S) \setminus X$;

for all $S' \subseteq N, S' \neq \emptyset$, enumerate subsets first {

emit $(S \cup S')$;

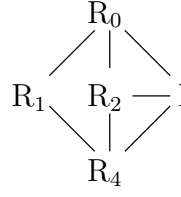
}

for all $S' \subseteq N, S' \neq \emptyset$, enumerate subsets first {

 EnumerateCsgRec($G, (S \cup S'), (X \cup N)$);

}

Let us consider an example. Figure 3.12 contains a query graph whose nodes are numbered in a breadth-first fashion. The calls to **EnumerateCsgRec**

Figure 3.12: Sample graph to illustrate **EnumerateCsgRec**

EnumerateCsgRec			
S	X	N	emit/ S
{4}	{0, 1, 2, 3, 4}	$\emptyset$	
{3}	{0, 1, 2, 3}	{4}	
			{3, 4}
{2}	{0, 1, 2}	{3, 4}	
			{2, 3}
			{2, 4}
			{2, 3, 4}
{1}	{0, 1}	{4}	
			{1, 4}
$\rightarrow$ {1, 4}	{0, 1, 4}	{2, 3}	
			{1, 2, 4}
			{1, 3, 4}
			{1, 2, 3, 4}
{0}	{0}	{1, 2, 3}	
			{0, 1}
			{0, 2}
			{0, 3}
			{0, 1, 2}
			{0, 1, 3}
			{0, 2, 3}
			{0, 1, 2, 3}
$\rightarrow$ {0, 1}	{0, 1, 2, 3}	{4}	
			{0, 1, 4}
$\rightarrow$ {0, 2}	{0, 1, 2, 3}	{4}	
			{0, 2, 4}

Figure 3.13: Call sequence for Figure 3.12

are contained in the table in Figure 3.13. In this table, S and X are the arguments of **EnumerateCsgRec**. N is the local variable after its initialization. The column **emit**/ S contains the connected subset emitted, which then becomes the argument of the recursive call to **EnumerateCsgRec** (labelled by $\rightarrow$). Since listing all calls is too lengthy, only a subset of the calls is listed.

Generating the connected subsets is an important first step but clearly not

sufficient: we have to generate all csg-cmp-pairs. The basic idea to do so is as follows. Algorithm **EnumerateCsg** is used to create the first component S_1 of every csg-cmp-pair. Then, for each such S_1 , we generate all its complement components S_2 . This can be done by calling **EnumerateCsgRec** with the correct parameters. Remember that we have to generate every csg-cmp-pair once and only once.

To achieve this, we use a similar technique as for connected subsets, using the breadth-first numbering to define an enumeration order: we consider only sets S_2 in the complement of S_1 (with (S_1, S_2) being a csg-cmp-pair) such that S_2 contains only v_j with j larger than any i with $v_i \in S_1$. This avoids the generation of duplicates.

We need some definitions to state the actual algorithm. Let $S_1 \subseteq V$ be a non-empty subset of V . Then, we define $\min(S_1) := \min(\{i | v_i \in S_1\})$. This is used to extract the starting node from which S_1 was constructed (see Lemma ??). Let $W \subset V$ be a non-empty subset of V . Then, we define $\mathcal{B}_i(W) := \{v_j | v_j \in W, j \leq i\}$. Using this notation, the algorithm to construct all S_2 for a given S_1 such that (S_1, S_2) is a csg-cmp-pair looks as follows:

EnumerateCmp

Input: a connected query graph $G = (V, E)$, a connected subset S_1

Precondition: nodes in V are numbered according to a breadth-first search

Output: emits all complements S_2 for S_1 such that (S_1, S_2) is a csg-cmp-pair

$X = \mathcal{B}_{\min(S_1)} \cup S_1$;

$N = \mathcal{N}(S_1) \setminus X$;

for all $(v_i \in N \text{ by descending } i) \{$

emit $\{v_i\}$;

EnumerateCsgRec $(G, \{v_i\}, X \cup (\mathcal{B}_i \cap N))$;

$\}$

Algorithm **EnumerateCmp** considers all neighbors of S_1 . First, they are used to determine those S_2 that contain only a single node. Then, for each neighbor of S_1 , it recursively calls **EnumerateCsgRec** to create those S_2 that contain more than a single node. Note that here both nodes concerning the enumeration of S_1 ($\mathcal{B}_{\min(S_1)} \cup S_1$) and nodes concerning the enumeration of S_2 (N) have to be considered in order to guarantee a correct enumeration. Otherwise the combined algorithm would emit (commutative) duplicates.

Let us consider an example for algorithm **EnumerateCmp**. The underlying graph is again the one shown in Fig. 3.12. Assume **EnumerateCmp** is called with $S_1 = \{R_1\}$. In the first statement, the set $\{R_0, R_1\}$ is assigned to X . Then, the neighborhood is calculated. This results in

$$N = \{R_0, R_4\} \setminus \{R_0, R_1\} = \{R_4\}.$$

Hence, $\{R_4\}$ is emitted and together with $\{R_1\}$, it forms the csg-cmp-pair $(\{R_1\}, \{R_4\})$. Then, the recursive call to **EnumerateCsgRec** follows with arguments G , $\{R_4\}$, and $\{R_0, R_1, R_4\}$. Subsequent **EnumerateCsgRec** generates the connected sets $\{R_2, R_4\}$, $\{R_3, R_4\}$, and $\{R_2, R_3, R_4\}$, giving three more csg-cmp-pairs.

3.2.5 Memoization

Whereas dynamic programming constructs the join trees iteratively from small trees to larger trees, i.e. works bottom up, memoization works recursively. For a given set of relations S , it produces the best join tree for S by recursively calling itself for every subset S_1 of S and considering all join trees between S_1 and its complement S_2 . The best alternative is memoized (hence the name). The reason is that two (even different) (sub-) sets of all relations may very well have the common subsets. For example, $\{R_1, R_2, R_3, R_4, R_5\}$ and $\{R_2, R_3, R_4, R_5, R_6\}$ have the common subset $\{R_2, R_3, R_4, R_5\}$. In order to avoid duplicate work, memoization is essential.

In the following variant of memoization, we explore the search space of all bushy trees and consider cross products. We split the functionality across two EX functions. The first one initializes the **BestTree** data structure with single relation join trees for R_i and then calls the second one. The second one is the core memoization procedure which calls itself recursively.

MemoizationJoinOrdering(R)

Input: a set of relations R

Output: an optimal join tree for R

```
for (i = 1; i <= n; ++i) {
    BestTree( $\{R_i\}$ ) =  $R_i$ ;
}
return MemoizationJoinOrderingSub( $R$ );
```

MemoizationJoinOrderingSub(S)

Input: a (sub-) set of relations S

Output: an optimal join tree for S

```
if(NULL == BestTree( $S$ )) {
    for all  $S_1 \subset S$  do {
         $S_2 = S \setminus S_1$ ;
        CurrTree = CreateJoinTree(MemoizationJoinOrderingSub( $S_1$ ), MemoizationJoinOrderingSub( $S_2$ ));
        if (BestTree( $S$ ) == NULL || cost(BestTree( $S$ )) > cost(CurrTree)) {
            BestTree( $S$ ) = CurrTree;
        }
    }
}
return BestTree( $S$ );
```

Again, pruning techniques can help to speed up plan generation [798].

ToDo?

3.2.6 Join Ordering by Generating Permutations

For any set of cost functions, we can directly generate permutations. Generating all permutations is clearly too expensive for more than a couple of relations. However, we can safely neglect some of them. Consider the join

sequence $R_1R_2R_3R_4$. If we know that $R_1R_3R_2$ is cheaper than $R_1R_2R_3$, we do not have to consider $R_1R_2R_3R_4$. The idea of the following algorithm is to construct permutations by successively adding relations. Thereby, an extended sequence is only explored if exchanging the last two relations does not result in a cheaper sequence.

```

ConstructPermutations(Query Specification)
Input: query specification for relations  $\{R_1, \dots, R_n\}$ 
Output: optimal left-deep tree
BestPermutation = NULL;
Prefix =  $\epsilon$ ;
Rest =  $\{R_1, \dots, R_n\}$ ;
ConstructPermutationsSub(Prefix, Rest);
return BestPermutation

ConstructPermutationsSub(Prefix, Rest)
Input: a prefix of a permutation and the relations to be added (Rest)
Output: none, side-effect on BestPermutation
if (Rest ==  $\emptyset$ ) {
    if (BestPermutation == NULL || cost(Prefix) < cost(BestPermutation)) {
        BestPermutation = Prefix;
    }
    return
}
foreach ( $R_i, R_j \in \text{Rest}$ ) {
    if (cost(Prefix  $\circ \langle R_i, R_j \rangle$ )  $\leq$  cost(Prefix  $\circ \langle R_j, R_i \rangle$ )) {
        ConstructPermutationsSub(Prefix  $\circ \langle R_i \rangle$ , Rest  $\setminus \{R_i\}$ );
    }
    if (cost(Prefix  $\circ \langle R_j, R_i \rangle$ )  $\leq$  cost(Prefix  $\circ \langle R_i, R_j \rangle$ )) {
        ConstructPermutationsSub(Prefix  $\circ \langle R_j \rangle$ , Rest  $\setminus \{R_j\}$ );
    }
}
return

```

The algorithm can be made more efficient, if the **foreach** loop considers only a single relation and performs the swap test with this relation and the last relation occurring in **Prefix**.

The algorithm has two main advantages over dynamic programming and memoization. The first advantage is that it needs only linear space opposed to exponential space for the two mentioned alternatives. The other main advantage over dynamic programming is that it generates join trees early, whereas with dynamic programming we only generate a plan after the whole search space has been explored. Thus, if the query contains too many joins—that is, the search space cannot be fully explored in reasonable time and space—dynamic programming will not generate any plan at all. If stopped,

ConstructPermutations will not necessarily compute the best plan, but still some plans have been investigated. This allows us to stop it after some time limit has exceeded. The time limit itself can be fixed, like 100 ms, or variable, like 5% of the execution time of the best plan found so far.

The predicates in the **if** statement can be made more efficient if a (local) ranking function is available. Further speed-up of the algorithm can be achieved if additionally the idea of memoization is applied (of course, this jeopardizes the small memory footprint).

The following variant might be interesting if one is willing to go from linear space consumption to quadratic space consumption. The original algorithm is then started n times, once for each relation as a starting relation. The n different instantiations then have to run interleaved. This variant reduces the dependency on the starting relation.

Worst Case Analysis

ToDo/EX

Pruning/memoization/propagation

ToDo/EX

3.2.7 A Dynamic Programming based Heuristics for Chain Queries

In Section 3.1.6, we saw that the complexity of producing optimal left-deep trees possibly containing cross products for chain queries is an open problem. However, the case does not seem to be hopeless. In fact, Scheufele and Mörkotte present two algorithms [766, 768] for this problem. For one algorithm, it can be proven that it has polynomial runtime, for the other, it can be proven that it produces the optimal join tree. However, for none of them both could be proven so far.

Basic Definitions and Lemmata

An instance of the *join-ordering problem for chain queries* (or a *chain query* for short) is fully described by the following parameters. First, n relations $R_1, \dots, R_n$ are given. The size of relation R_i ($1 \leq i \leq n$) is denoted by $|R_i|$ or n_{R_i} . Second, the query graph G on the set of relations $R_1, \dots, R_n$ must be a chain. That is, its edges are $\{(R_i, R_{i+1}) \mid 1 \leq i < n\}$:

$$R_1 - R_2 - \dots - R_n$$

For every edge (R_i, R_{i+1}) , there is an associated selectivity $f_{i,i+1} = |R_i \bowtie R_{i+1}| / |R_i \times R_{i+1}|$. We define all other selectivities $f_{i,j} = 1$ for $|i - j| \neq 1$. They correspond to cross products.

In this section we consider only left-deep processing trees. However, we allow them to contain cross products. Hence, any permutation is a valid join tree. There is a unique correspondence not only between left-deep join trees but also between consecutive parts of a permutation and segments of a left-deep tree. Furthermore, if a segment of a left-deep tree does not contain cross products, it uniquely corresponds to a consecutive part of the chain in the query graph. In this case, we also speak of (sub)chains or connected (sub)sequences. We say that two relations R_i and R_j are *connected* if they are adjacent in G ; more generally, two sequences s and t are connected if there exist relations R_i

in s and R_j in t such that R_i and R_j are connected. A sequence of relations s is connected if for all subsequences s_1 and s_2 satisfying $s = s_1 s_2$ it holds that s_1 is connected to s_2 .

Given a chain query, we ask for a permutation $s = r_1 \dots r_n$ of the n relations (i.e. there is a permutation π such that $r_i = R_{\pi(i)}$ for $1 \leq i \leq n$) that produces minimal costs under the cost function C_{out} .

Remember that the dynamic programming approach considers $n2^{n-1} - n(n+1)/2$ alternatives for left-deep processing trees with cross products—independently of the query graph and the cost function. The question arises whether it is possible to lower the complexity in case of simple chain queries.

The IKKBZ algorithm solves the join ordering problem for tree queries by decomposing the problem into polynomially many subproblems which are subject to tree-like precedence constraints. The precedence constraints ensure that the cost functions of the subproblems now have the ASI property. The remaining problem is to optimize the constrained subproblems under the simpler cost function. Unfortunately, this approach does not work in our case, since no such decomposition seems to exist.

Let us introduce some notions used for the algorithms. We have to generalize the rank used in the IKKBZ algorithm to *relativized ranks*. We start by relativizing the cost function. The costs of a sequence s *relative* to a sequence u are defined as

$$\begin{aligned} C_u(\epsilon) &:= 0 \\ C_u(R_i) &:= 0 \quad \text{if } u = \epsilon \\ C_u(R_i) &:= \left(\prod_{R_j <_u R_i} f_{j,i} \right) n_i \quad \text{if } u \neq \epsilon \\ C_u(s_1 s_2) &:= C_u(s_1) + T_u(s_1) * C_{us_1}(s_2) \end{aligned}$$

with

$$\begin{aligned} T_u(\epsilon) &:= 1 \\ T_u(s) &:= \prod_{R_i \in s} \left(\prod_{R_j <_{us} R_i} f_{j,i} \right) * n_i \end{aligned}$$

Here, $R_i <_s R_j$ is true if and only if R_i appears before R_j in s . As usual, empty products evaluate to 1. Several things should be noted. First, $C_{us}(t) = C_u(t)$ holds if there is no connection between relations in s and t . Second, $T_\epsilon(R_i) = |R_i|$ and $T_\epsilon(s) = |s|$. That is, T_u generalizes the size of a single relation or of a sequence of relations. Third, note that $C_u(\epsilon) = 0$ for all u but $C_\epsilon(s) = 0$ only if s does not contain more than one relation. The special case that $C_\epsilon(R) = 0$ for a single relation R causes some problems in the homogeneity of definitions and proofs. Hence, we abandon this case from all definitions and lemmata of this section. This will not be repeated in every definition and lemma, but will implicitly be assumed. Further, the two algorithms will be presented in two versions. The first version is simpler and relies on a modified cost function C' ,

and only the second version will apply to the original cost function C . As we will see, C' differs from C in exactly the problematic case in which it is defined as $C'_u(R_i) := |R_i|$. Now, $C'_\epsilon(s) = 0$ holds if and only if $s = \epsilon$ holds. Within subsequent definitions and lemmata, C can also be replaced by C' without changing their validity. Last, we abbreviate C_ϵ by C for convenience.

Example 1: Consider a chain query involving the relations R_1, R_2, R_3 . The parameters are $|R_1| = 1, |R_2| = 100, |R_3| = 10$ and $f_{1,2} = f_{2,3} = 0.9$. The expected size of the query result is independent of the ordering of the relations. Hence, we have

$$T(R_1 R_2 R_3) = \dots = T(R_3 R_2 R_1) = 100 * 10 * 1 * .9 * .9 = 810.$$

There are 6 possible orderings of the relations with the following costs:

$$\begin{aligned} C(R_1 R_2 R_3) &= 1 * 100 * 0.9 + 1 * 100 * 10 * 0.9 * 0.9 = 900 \\ C(R_1 R_3 R_2) &= 1 * 10 + 1 * 10 * 100 * 0.9 * 0.9 = 820 \\ C(R_2 R_3 R_1) &= 100 * 10 * 0.9 + 100 * 10 * 1 * 0.9 * 0.9 = 1710 \\ C(R_2 R_1 R_3) &= C(R_1 R_2 R_3) \\ C(R_3 R_1 R_2) &= C(R_1 R_3 R_2) \\ C(R_3 R_2 R_1) &= C(R_2 R_3 R_1) \end{aligned}$$

Note that the cost function is invariant with respect to the order of the first two relations. The minimum over all costs is 820, and the corresponding optimal join ordering is $R_1 R_3 R_2$. □

Using the relativized cost function, we can define the relativized rank.

Definition 3.2.8 (rank) *The rank of a sequence s relative to a non-empty sequence u is given by*

$$rank_u(s) := \frac{T_u(s) - 1}{C_u(s)}$$

In the special case that s consists of a single relation R_i , the intuition behind the *rank* function becomes transparent. Let f_i be the product of the selectivities between relations in u and R_i . Then $rank_u(R_i) = \frac{f_i |R_i| - 1}{f_i |R_i|}$. Hence, the *rank* becomes a function of the form $f(x) = \frac{x-1}{x}$. This function is monotonously increasing in x for $x > 0$. The argument to the function $f(x)$ is (for the computation of the size of a single relation R_i) $f_i |R_i|$. But this is the factor by which the next intermediate result will increase (or decrease). Since we sum up intermediate results, this is an essential number. Furthermore, it follows from the monotonicity of $f(x)$ that $rank_u(R_i) \leq rank_u(R_j)$ if and only if $f_i |R_i| \leq f_j |R_j|$ where f_j is the product of all selectivities between R_j and relations in u .

Example 1 (cont'd): Supposing the query given in Example 1, the optimal sequence $R_1 R_3 R_2$ gives rise to the following ranks.

$$\begin{aligned} rank_{R_1}(R_2) &= \frac{T_{R_1}(R_2) - 1}{C_{R_1}(R_2)} = \frac{100 * 0.9 - 1}{100 * 0.9} \approx 0.9888 \\ rank_{R_1}(R_3) &= \frac{T_{R_1}(R_3) - 1}{C_{R_1}(R_3)} = \frac{10 * 1.0 - 1}{10 * 1.0} = 0.9 \\ rank_{R_1 R_3}(R_2) &= \frac{T_{R_1 R_3}(R_2) - 1}{C_{R_1 R_3}(R_2)} = \frac{100 * 0.9 * 0.9 - 1}{100 * 0.9 * 0.9} \approx 0.9877 \end{aligned}$$

Hence, within the optimal sequence, the relation with the smallest rank (here R_3 , since $\text{rank}_{R_1}(R_3) < \text{rank}_{R_1}(R_2)$) is preferred. As the next lemma will show, this is no accident.

□

Using the rank function, the following lemma can be proved.

Lemma 3.2.9 *For sequences*

$$\begin{aligned} S &= r_1 \cdots r_{k-1} r_k r_{k+1} r_{k+2} \cdots r_n \\ S' &= r_1 \cdots r_{k-1} r_{k+1} r_k r_{k+2} \cdots r_n \end{aligned}$$

the following holds:

$$C(S) \leq C(S') \Leftrightarrow \text{rank}_u(r_k) \leq \text{rank}_u(r_{k+1})$$

where $u = r_1 \cdots r_{k-1}$. Equality only holds if it holds on both sides.

Example 1 (cont'd): Since the ranks of the relations in Example 1 are ordered with ascending ranks, Lemma 3.2.9 states that, whenever we exchange two adjacent relations, the costs cannot decrease. In fact, we observe that $C(R_1 R_3 R_2) \leq C(R_1 R_2 R_3)$. □

An analogous lemma still holds for two unconnected subchains:

Lemma 3.2.10 *Let u, x and y be three subchains where x and y are not interconnected. Then we have:*

$$C(uxy) \leq C(uyx) \Leftrightarrow \text{rank}_u(x) \leq \text{rank}_u(y)$$

Equality only holds if it holds on both sides.

Next, we define the notion of a *contradictory chain*, which will be essential to the algorithms. The subsequent lemmata will allow us to cut down the search space to be explored by any optimization algorithm.

Definition 3.2.11 (contradictory pair of subchains) *Let u, x, y be nonempty sequences. We call (x, y) a contradictory pair of subchains if and only if*

$$C_u(xy) \leq C_u(yx) \quad \wedge \quad \text{rank}_u(x) > \text{rank}_{ux}(y)$$

A special case occurs when x and y are single relations. Then the above condition simplifies to

$$\text{rank}_{ux}(y) < \text{rank}_u(x) \leq \text{rank}_u(y)$$

To explain the intuition behind the definition of contradictory subchains, we need another example.

Example 2: Suppose a chain query involving R_1, R_2, R_3 is given. The relation sizes are $|R_1| = 1, |R_2| = |R_3| = 10$ and the selectivities are $f_{1,2} = 0.5, f_{2,3} = 0.2$. Consider the sequences $R_1R_2R_3$ and $R_1R_3R_2$, which differ in the order of the last two relations. We have

$$\begin{aligned} \text{rank}_{R_1}(R_2) &= 0.8 \\ \text{rank}_{R_1R_2}(R_3) &= 0.0 \\ \text{rank}_{R_1}(R_3) &= 0.9 \\ \text{rank}_{R_1R_3}(R_2) &= 0.5 \end{aligned}$$

and

$$\begin{aligned} C(R_1R_2R_3) &= 15 \\ C(R_1R_3R_2) &= 20 \end{aligned}$$

Hence,

$$\begin{aligned} \text{rank}_{R_1}(R_2) &> \text{rank}_{R_1R_2}(R_3) \\ \text{rank}_{R_1}(R_3) &> \text{rank}_{R_1R_3}(R_2) \\ C(R_1R_2R_3) &< C(R_1R_3R_2) \end{aligned}$$

and (R_2, R_3) is a contradictory pair within $R_1R_2R_3$. Now the use of the term *contradictory* becomes clear: the costs do not behave as could be expected from the ranks. $\square$

The next (obvious) lemma states that contradictory chains are necessarily connected.

Lemma 3.2.12 *If there is no connection between two subchains x and y , then they cannot build a contradictory pair (x, y) .*

Now we present the fact that between a contradictory pair of relations, there cannot be any other relation not connected to them without increasing cost.

Lemma 3.2.13 *Let $S = usvtw$ be a sequence. If there is no connection between relations in s and v and relations in v and t , and $\text{rank}_u(s) \geq \text{rank}_{us}(t)$, then there exists a sequence S' not having higher costs, where s immediately precedes t .*

Example 3: Consider five relations $R_1, \dots, R_5$. The relation sizes are $|R_1| = 1, |R_2| = |R_3| = |R_4| = 8$, and $|R_5| = 2$. The selectivities are $f_{1,2} = \frac{1}{2}, f_{2,3} = \frac{1}{4}, f_{3,4} = \frac{1}{8}$, and $f_{4,5} = \frac{1}{2}$. Relation R_5 is not connected to relations R_2 and R_3 . Further, within the sequence $R_1R_2R_5R_3R_4$ relations R_2 and R_3 have contradictory ranks: $\text{rank}_{R_1}(R_2) = \frac{4-1}{4} = \frac{3}{4}$ and $\text{rank}_{R_1R_2R_5}(R_3) = \frac{2-1}{2} = \frac{1}{2}$. Hence, at least one of $R_1R_5R_2R_3R_4$ and $R_1R_2R_3R_5R_4$ must be of no greater cost than $R_1R_2R_5R_3R_4$. This is indeed the case:

$$\begin{aligned} C(R_1R_2R_3R_5R_4) &= 4 + 8 + 16 + 8 = 36 \\ C(R_1R_2R_5R_3R_4) &= 4 + 8 + 16 + 8 = 36 \\ C(R_1R_5R_2R_3R_4) &= 2 + 8 + 16 + 8 = 34 \end{aligned}$$

□

The next lemma shows that, if there exist two sequences of single rank-sorted relations, then their costs as well as their ranks are necessarily equal.

Lemma 3.2.14 *Let $S = x_1 \cdots x_n$ and $S' = y_1 \cdots y_n$ be two different rank-sorted chains containing exactly the relations $R_1, \dots, R_n$, i.e.*

$$\begin{aligned} \text{rank}_{x_1 \dots x_{i-1}}(x_i) &\leq \text{rank}_{x_1 \dots x_i}(x_{i+1}) \text{ for all } 1 \leq i \leq n, \\ \text{rank}_{y_1 \dots y_{i-1}}(y_i) &\leq \text{rank}_{y_1 \dots y_i}(y_{i+1}) \text{ for all } 1 \leq i \leq n, \end{aligned}$$

then S and S' have equal costs and, furthermore,

$$\text{rank}_{x_1 \dots x_{i-1}}(x_i) = \text{rank}_{y_1 \dots y_{i-1}}(y_i) \text{ for all } 1 < i \leq n$$

One could conjecture that the following generalization of Lemma 3.2.14 is true, although no one has proved it so far.

Conjecture 3.2.1 *Let $S = x_1 \cdots x_n$ and $S' = y_1 \cdots y_m$ be two different rank-sorted chains for the relations $R_1, \dots, R_n$ where the x'_i s and y'_i s are subsequences such that*

$$\begin{aligned} \text{rank}_{x_1 \dots x_{i-1}}(x_i) &\leq \text{rank}_{x_1 \dots x_i}(x_{i+1}) \text{ for all } 1 \leq i < n, \\ \text{rank}_{y_1 \dots y_{i-1}}(y_i) &\leq \text{rank}_{y_1 \dots y_i}(y_{i+1}) \text{ for all } 1 \leq i < m, \end{aligned}$$

and the subsequences x_i and y_j are all optimal (with respect to the fixed prefixes $x_1 \dots x_{i-1}$ and $y_1 \dots y_{j-1}$), then S and S' have equal costs.

Consider the problem of merging two optimal unconnected chains. If we knew that the ranks of relations in an optimal chain are always sorted in ascending order, we could use the classical merge procedure to combine the two chains. The resulting chain would also be rank-sorted in ascending order and, according to Lemma 3.2.14, it would be optimal. Unfortunately, this does not work, since there are optimal chains whose ranks are not sorted in ascending order: those containing sequences with contradictory ranks.

Now, as shown in Lemma 3.2.13, between contradictory pairs of relations there cannot be any other relation not connected to them. Hence, in the merging process, we have to take care that we do not merge a contradictory pair of relations with a relation not connected to the pair. In order to achieve this, we apply the same trick as in the IKKBZ algorithm: we tie the relations of a contradictory subchain together by building a *compound relation*. Assume that we tie together the relations $r_1, \dots, r_n$ to a new relation $r_{1, \dots, n}$. Then we define the size of $r_{1, \dots, n}$ as $|r_{1, \dots, n}| = |r_1 \bowtie \dots \bowtie r_n|$. Further, if some r_i ($1 \leq i \leq n$) does have a connection to some $r_k \notin \{r_1, \dots, r_n\}$ then we define the selectivity factor $f_{r_{1, \dots, n}, r_k}$ between r_k and $r_{1, \dots, n}$ as $f_{r_{1, \dots, n}, r_k} = f_{i, k}$.

If we tie together contradictory pairs, the resulting chain of compound relations still does not have to be rank-sorted with respect to the compound relations. To overcome this, we iterate the process of tying contradictory pairs of compound relations together until the sequence of compound relations is rank-sorted, which will eventually be the case. That is, we apply the *normalization* as used in the IKKBZ algorithm. However, we have to reformulate it for relativized costs and ranks:

```

Normalize( $p, s$ )
  while (there exist subsequences  $u, v$  ( $u \neq \epsilon$ ) and
    compound relations  $x, y$  such that  $s = uxyv$ 
    and  $C_{pu}(xy) \leq C_{pu}(yx)$ 
    and  $\text{rank}_{pu}(x) > \text{rank}_{pux}(y)$ ) {
    replace  $xy$  by a compound relation  $(x, y)$ ;
  }
  return ( $p, s$ );

```

The compound relations in the result of the procedure **Normalize** are called *contradictory chains*. A *maximal contradictory subchain* is a contradictory subchain that cannot be made longer by further tying steps. Resolving the tyings introduced in the procedure **normalize** is called *de-normalization*. It works the same way as in the IKKBZ algorithm. The cost, size and rank functions can now be extended to sequences containing compound relations in a straightforward way. We define the cost of a sequence containing compound relations to be identical with the cost of the corresponding de-normalized sequence. The size and rank functions are defined analogously.

The following simple observation is central to the algorithms: every chain can be decomposed into a sequence of adjacent maximal contradictory subchains. For convenience, we often speak of chains instead of subchains and of contradictory chains instead of maximal contradictory subchains. The meaning should be clear from the context. Further, we note that the decomposition into adjacent maximal contradictory subchains is not unique. For example, consider an optimal subchain $r_1 r_2 r_3$ and a sequence u of preceding relations. If $\text{rank}_u(r_1) > \text{rank}_{ur_1}(r_2) > \text{rank}_{ur_1 r_2}(r_3)$ one can easily show that both $(r_1, (r_2, r_3))$ and $((r_1, r_2), r_3)$ are contradictory subchains. Nevertheless, this ambiguity is not important since in the following we are only interested in contradictory subchains which are *optimal*. In this case, the condition $C_u(xy) \leq C_u(yx)$ is certainly true and can therefore be neglected. One can show that for the case of optimal subchains the indeterministically defined normalization process is well-defined, that is, if S is optimal, **normalize**(P, S) will always terminate with a unique “flat” decomposition of S into maximal contradictory subchains (flat means that we remove all but the outermost parenthesis, e.g. $(R_1 R_2) (((R_5 R_4) R_3) R_6)$ becomes $(R_1 R_2) (R_5 R_4 R_3 R_6)$).

The next two lemmata and the conjecture show a possible way to overcome the problem that if we consider cross products, we have an unconstrained ordering problem and the idea of Monma and Sidney as exploited in the IKKBZ algorithm is no longer applicable. The next lemma is a direct consequence of the normalization procedure.

Lemma 3.2.15 *Let $S = s_1 \dots s_m$ be an optimal chain consisting of the maximal contradictory subchains $s_1, \dots, s_m$ (as determined by the function **normalize**). Then*

$$\text{rank}(s_1) \leq \text{rank}_{s_1}(s_2) \leq \text{rank}_{s_1 s_2}(s_3)$$

$$\leq \dots \leq \text{rank}_{s_1 \dots s_{m-1}}(s_m),$$

in other words, the (maximal) contradictory subchains in an optimal chain are always sorted by ascending ranks.

The next result shows how to build an optimal sequence from two optimal non-interconnected sequences.

Lemma 3.2.16 *Let x and y be two optimal sequences of relations where x and y are not interconnected. Then the sequence obtained by merging the maximal contradictory subchains in x and y (as obtained by **normalize**) according to their ascending rank is optimal.*

Merging two sequences in the way described in Lemma 3.2.16 is a fundamental process. We henceforth refer to it by simply saying that we *merge by the ranks*.

We strongly conjecture that the following generalization of Lemma 3.2.14 is true, although it is yet unproven. It uses the notion of *optimal recursive decomposable subchains* defined in the next subsection.

Conjecture 3.2.2 *Consider two sequences S and T containing exactly the relations $R_1, \dots, R_n$. Let $S = s_1 \dots s_k$ and $T = t_1 \dots t_l$ be such that each of the maximal contradictory subchains $s_i, i = 1, \dots, k$ and $t_j, j = 1, \dots, l$ are optimal recursively decomposable. Then S and T have equal costs.*

The first algorithm

We first use a slightly modified cost function C' , which additionally respects the size of the first relation in the sequence, i.e. C and C' relate via

$$C'_u(s) = \begin{cases} C(s) + |n_R|, & \text{if } u = \epsilon \text{ and } s = Rs' \\ C_u(s), & \text{otherwise} \end{cases}$$

This cost function can be treated in a more elegant way than C . The new rank function is now defined as $\text{rank}_u(s) := (T_u(s) - 1)/C'_u(s)$. Note that the rank function is now defined even if $u = \epsilon$ and s is a single relation. The size function remains unchanged. At the end of this subsection, we describe how our results can be adapted to the original cost function C .

The rank of a contradictory chain depends on the relative position of the relations that are directly connected to it. For example, the rank of the contradictory subchain $(R_5 R_3 R_4 R_2)$ depends on the position of the neighbouring relations R_1 and R_6 relative to $(R_5 R_3 R_4 R_2)$. That is, whether they appear before or after the sequence $(R_5 R_3 R_4 R_2)$. Therefore, we introduce the following fundamental definitions:

Definition 3.2.17 (neighbourhood) *We call the set of relations that are directly connected to a subchain (with respect to the query graph G) the complete neighbourhood of that subchain. A neighbourhood is a subset of the complete neighbourhood. The complement of a neighbourhood u of a subchain s is defined as $v \setminus u$, where v denotes the complete neighbourhood of s .*

Note that the neighbourhood of a subchain s within a larger chain us is uniquely determined by the subsequence u of relations preceding it. For convenience, we will often use sequences of preceding relations to specify neighbourhoods. We henceforth denote a pair consisting of a connected sequence s and a neighbourhood u by $[s]_u$.

Definition 3.2.18 (contradictory subchain, extent) *A contradictory subchain $[s]_u$ is inductively defined as follows.*

1. *For a single relation s , $[s]_u$ is a contradictory subchain.*
2. *There is a decomposition $s = vw$ such that (v, w) is a contradictory pair with respect to the preceding subsequence u and both $[v]_u$ and $[w]_{uv}$ are contradictory subchains themselves.*

The extent of a contradictory chain $[s]_u$ is defined as the pair consisting of the neighbourhood u and the set of relations occurring in s . Since contradictory subchains are connected, the set of occurring relations has always the form $\{R_i, R_{i+1}, \dots, R_{i+l}\}$ for some $1 \leq i \leq n$, $0 \leq l \leq n - i$. An optimal contradictory subchain to a given extent is a contradictory subchain with lowest cost among all contradictory subchains of the same extent.

The number of different extents of contradictory subchains for a chain query of n relations is $2n^2 - 2n + 1$. Each contradictory chain can be completely recursively decomposed into adjacent pairs of connected subchains. Subchains with this property are defined next (similar types of decompositions occur in [435, 799]).

Definition 3.2.19 ((optimal) recursively decomposable subchain) *A recursively decomposable subchain $[s]_u$ is inductively defined as follows.*

1. *If s is a single relation, then $[s]_u$ is recursively decomposable.*
2. *There is a decomposition $s = vw$ such that v is connected to w and both $[v]_u$ and $[w]_{uv}$ are recursively decomposable subchains.*

The extent of a recursively decomposable chain is defined in the same way as for contradictory chains. Note that every contradictory subchain is recursively decomposable. Consequently, the set of all contradictory subchains for a certain extent is a subset of all recursively decomposable subchains of the same extent.

Example 4: Consider the sequence of relations

$$s = R_2 R_4 R_3 R_6 R_5 R_1.$$

Using parentheses to indicate the recursive decompositions, we have the following two possibilities

$$\begin{aligned} &(((R_2(R_4 R_3))(R_6 R_5))R_1) \\ &((R_2((R_4 R_3)(R_6 R_5)))R_1) \end{aligned}$$

The extent of the recursively decomposable subchain $R_4R_3R_6R_5$ of s is $(\{R_2\}, \{R_3, R_4, R_5, R_6\})$. $\square$

The number of different recursively decomposable chains involving the relations $R_1, \dots, R_n$ is r_n , where r_n denotes the n -th Schröder number [799]. Hence, the total number of recursively decomposable chains is $r_n + 2(n-1)r_{n-1} + 4 \sum_{i=1}^{n-2} \binom{n-2}{i} r_i$. It can be shown that

$$r_n \approx \frac{C(2 + \sqrt{8})^n}{n^{3/2}}$$

where $C = 1/2\sqrt{\frac{2\sqrt{2}-4}{\pi}}$. Using Stirling's formula for $n!$ it is easy to show that $\lim_{n \rightarrow \infty} \frac{r_n}{n!} = 0$. Thus, the probability of a random permutation to be recursively decomposable strives to zero for large n .

An *optimal recursively decomposable subchain* to a given extent is a recursively decomposable subchain with lowest cost among all recursively decomposable subchains of the same extent. There is an obvious dynamic programming algorithm to compute optimal recursive decomposable subchains. It is not hard to see that *Bellman's optimality principle* [608, 209] holds and every optimal recursively decomposable subchain can be decomposed into smaller optimal recursively decomposable subchains.

Example 5: In order to compute an optimal recursively decomposable subchain for the extent

$$(\{R_2, R_7\}, \{R_3, R_4, R_5, R_6\})$$

the algorithm makes use of optimal recursively decomposable subchains for the extents

$$\begin{array}{ll} (\{R_2\}, \{R_3\}) & (\{R_7, R_3\}, \{R_4, R_5, R_6\}) \\ (\{R_2\}, \{R_3, R_4\}) & (\{R_7, R_4\}, \{R_5, R_6\}) \\ (\{R_2\}, \{R_3, R_4, R_5\}) & (\{R_5, R_7\}, \{R_6\}) \\ (\{R_7\}, \{R_4, R_5, R_6\}) & (\{R_2, R_4\}, \{R_3\}) \\ (\{R_7\}, \{R_5, R_6\}) & (\{R_2, R_5\}, \{R_3, R_4\}) \\ (\{R_7\}, \{R_6\}) & (\{R_2, R_6\}, \{R_3, R_4, R_5\}) \end{array}$$

which have been computed in earlier steps². A similar dynamic programming algorithm can be used to determine optimal contradictory subchains. $\square$

Let E be the set of all possible extents. We define the following partial order $\mathcal{P} = (E, \prec)$ on E . For all extents $e_1, e_2 \in E$, we have $e_1 \prec e_2$ if and only if e_1 can be obtained by splitting the extent e_2 . For example, $(\{R_7\}, \{R_5, R_6\}) \prec (\{R_2, R_7\}, \{R_3, R_4, R_5, R_6\})$. The set of maximal extents M then corresponds to a set of incomparable elements (antichain) in $\mathcal{P}$ such that for all extents e enumerated so far, there is an extent $e' \in M$ with $e \prec e'$.

Now, since every optimal join sequence has a representation as a sequence of contradictory subchains, we only have to determine this representation. Consider a contradictory subchain c in an optimal join sequence s . What can we say

²The splitting of extents induces a partial order on the set of extents.

about c ? Obviously, c has to be optimal with respect to the neighbourhood defined by the relations preceding c in s . Unfortunately, identifying contradictory subchains that are optimal sequences seems to be as hard as the whole problem of optimizing chain queries. Therefore, we content ourselves with the following weaker condition which may lead to multiple representations. Nevertheless, it seems to be the strongest condition for which all subchains satisfying the condition can be computed in polynomial time. The condition says that s should be optimal both with respect to all contradictory chains of the same extent as s and with respect to all recursively decomposable subchains of the same extent. So far it is not clear whether these conditions lead to multiple representations. Therefore, we have no choice but to enumerate all possible representations and select the one with minimal costs. Next we describe the first algorithm.

Algorithm Chain-I’:

1. Use dynamic programming to determine all optimal contradictory subchains.
This step can be made faster by keeping track of the set M of all maximal extents (with respect to the partial order induced by splitting extents).
2. Determine all optimal recursively decomposable subchains for all extents included in some maximal extent in M .
3. Compare the results from steps 1 and 2 and retain only matching subchains.
4. Sort the contradictory subchains according to their ranks.
5. Eliminate contradictory subchains that cannot be part of a solution.
6. Use backtracking to enumerate all sequences of rank-ordered optimal contradictory subchains and keep track of the sequence with lowest cost.

In step 5 of the algorithm, we eliminate contradictory subchains that do not contribute to a solution. Note that the contradictory subchains in an optimal sequence are characterized by the following two conditions.

1. The extents of all contradictory subchains in the representation build a partition of the set of all relations.
2. The neighbourhoods of all contradictory subchains are consistent with the relations occurring at earlier and later positions in the sequence.

Note that any contradictory subchain occurring in the optimal sequence (except at the first and last positions) necessarily has matching contradictory subchains preceding and succeeding it in the list. In fact, every contradictory subchain X occurring in the optimal join sequence must satisfy the following two conditions.

1. For every relation R in the neighbourhood of X , there exists a contradictory subchain Y at an earlier position in the list which itself meets condition 1, such that R occurs in Y , and Y can be followed by X .
2. For every relation R in the complementary neighbourhood of X , there exists a contradictory subchain Y at a later position in the list which itself meets condition 2, such that R occurs in the neighbourhood of Y , and X can be followed by Y .

Using these two conditions, we can eliminate “useless” contradictory chains from the rank-ordered list by performing a reachability algorithm for each of the DAGs defined by the conditions 1 and 2. In the last step of our algorithm, backtracking is used to enumerate all representations. Suppose that at some step of the algorithm we have determined an initial sequence of contradictory subchains and have a rank-sorted list of the remaining possible contradictory subchains. In addition to the two conditions mentioned above, another reachability algorithm can be applied to determine the set of reachable relations from the list (with respect to the given prefix). With the use of this information, all branches that do not lead to a complete join sequence can be pruned.

Let us analyze the worst case time complexity of the algorithm. The two dynamic programming steps both iterate over $O(n^2)$ different extents, and each extent gives rise to $O(n)$ splittings. Moreover, for each extent one normalization is necessary, which requires linear time (cost, size and rank can be computed in constant time using recurrences). Therefore, the complexity of the two dynamic programming steps is $O(n^4)$. Sorting $O(n^2)$ contradictory chains can be done in time $O(n^2 \log n)$. The step where all “useless” contradictory subchains are eliminated, consists of two stages of a reachability algorithm which has complexity $O(n^4)$. If conjecture 3.2.2 is true, the backtracking step requires linear time, and the total complexity of the algorithm is $O(n^4)$. Otherwise, if conjecture 3.2.2 is false, the algorithm might exhibit exponential worst case time complexity.

We now describe how to reduce the problem for our original cost function C to the problem for the modified cost function C' . One difficulty with the original cost function is that the ranks are defined only for subsequences of *at least two* relations. Hence, for determining the first relation in our solution we do not have sufficient information. An obvious solution to this problem is to try every relation as starting relation, process each of the two resulting chain queries separately and choose the chain with minimum costs. The new complexity will increase by about a factor of n . This first approach is not very efficient, since the dynamic programming computations overlap considerably, e.g. if we perform dynamic programming on the two overlapping chains $R_1 R_2 R_3 R_4 R_5 R_6$ and $R_2 R_3 R_4 R_5 R_6 R_7$, for the intersecting chain $R_2 R_3 R_4 R_5 R_6$ everything is computed twice. The cue is that we can perform the dynamic programming calculations before we consider a particular starting relation. Hence, the final algorithm can be sketched as follows:

Algorithm CHAIN-I:

1. Compute all optimal contradictory chains by dynamic programming (corresponds to the steps 1-4 of Algorithm I')
2. For each starting relation R_i , perform the following steps:
 - (a) Let L_1 be the result of applying steps 5 and 6 of Algorithm I' to all contradictory subchains whose extent (N, M) satisfies $R_i \in N$ and $M \subseteq \{R_1, \dots, R_i\}$.
 - (b) Let L_2 be the result of applying steps 5 and 6 of Algorithm I' to all contradictory subchains whose extent (N, M) satisfies $R_i \in N$ and

$$M \subseteq \{R_i, \dots, R_n\}.$$

- (c) For all $(l_1, l_2) \in L_1 \times L_2$, perform the following steps:
- i. Let L be the result of merging l_1 and l_2 according to their ranks.
 - ii. Use $R_i L$ to update the current-best join ordering.

Suppose that conjecture 3.2.2 is true, and we can replace the backtracking part by a search for the first solution. Then the complexity of the step 1 is $O(n^4)$, whereas the complexity of step 2 amounts to $\sum_{i=1}^n (O(i^2) + O(n-i)^2 + O(n)) = O(n^3)$. Hence, the total complexity would be $O(n^4)$ in the worst case. Of course, if our conjecture is false, the necessary backtracking step might lead to an exponential worst case complexity.

The second algorithm

The second algorithm is much simpler than the first one but proves to be less efficient in practice. Since the new algorithm is very similar to some parts of the old one, we just point out the differences between both algorithms. The new version of the algorithm works as follows.

Algorithm CHAIN-II':

1. Use dynamic programming to compute an optimal recursive decomposable chain for the whole set of relations $\{R_1, \dots, R_n\}$.
2. Normalize the resulting chain.
3. Reorder the contradictory subchains according to their ranks.
4. De-normalize the sequence.

Step 1 is identical to step 2 of our first algorithm. Note that Lemma 3.2.15 cannot be applied to the sequence in Step 2, since an optimal recursive decomposable chain is not necessarily an optimal chain. Therefore, the question arises whether Step 3 really makes sense. One can show that the partial order defined by the precedence relation among the contradictory subchains has the property that all elements along paths in the partial order are sorted by rank. By computing a greedy topological ordering (greedy with respect to the ranks), we obtain a sequence as requested in step 3.

Let us briefly analyze the worst case time complexity of the second algorithm. The first step requires time $O(n^4)$, whereas the second step requires time $O(n^2)$. The third step has complexity $O(n \log n)$. Hence, the total complexity is $O(n^4)$.

Algorithm II' is based on the cost function C' . We can now modify the algorithm for the original cost function C as follows.

Algorithm CHAIN-II:

1. Compute all optimal recursive decomposable chains by dynamic programming (corresponds to step 1 of Algorithm II')
2. For each starting relation R_i , perform the following steps:

- (a) Let L_1 be the result of applying the steps 2 and 3 of Algorithm II' to all optimal recursive decomposable subchains whose extent (N, M) satisfies $R_i \in N$ and $M \subseteq \{R_1, \dots, R_i\}$.
- (b) Let L_2 be the result of applying the steps 2 and 3 of Algorithm II' to all optimal recursive decomposable subchains whose extent (N, M) satisfies $R_i \in N$ and $M \subseteq \{R_i, \dots, R_n\}$.
- (c) Let L be the result of merging L_1 and L_2 according to their ranks.
- (d) De-normalize L .
- (e) Use $R_i L$ to update the current-best join ordering.

The complexity of Step 1 is $O(n^4)$, whereas the complexity of Step 2 amounts to $\sum_{i=1}^n (O(i^2) + O(n-i)^2 + O(n)) = O(n^3)$. Hence, the time complexity of Algorithm II is $O(n^4)$.

Summarizing, we are now left with one algorithm that produces the optimal result but whose worst-case runtime behavior is unknown and one algorithm with polynomial runtime but producing a result which has not been proven to be optimal. Due to this lack of hard facts, Moerkotte and Scheufele ran about 700,000 experiments with random queries of sizes up to 30 relations and fewer experiments for random queries with up to 300 relations to compare the results of our algorithms. For $n \leq 15$, they additionally compared the results with a standard dynamic programming algorithm. Their findings can be summarized as follows.

- All algorithms yielded identical results.
- Backtracking always led to exactly one sequence of contradictory chains.
- In the overwhelming majority of cases the first algorithm proved to be faster than the second.

Whereas the run time of the second algorithm is mainly determined by the number of relations in the query, the run time of the first also heavily depends on the number of existing optimal contradictory subchains. In the worst case, the first algorithm is slightly inferior to the second. Additionally, Hamalainen reports on an independent implementation of the second algorithm [394]. He could not find an example where the second algorithm did not produce the optimal result either. We encourage the reader to prove that it produces the optimal result.

EX

3.2.8 Transformation-Based Approaches

The idea of transformation-based algorithms can be described as follows. Starting from an arbitrary join tree, equivalences (such as commutativity and associativity) are applied to it to derive a set of new join trees. For each of the join trees, the equivalences are again applied to derive even more join trees. This procedure is repeated until no new join tree can be derived. This procedure exhaustively enumerates the set of all bushy trees. Furthermore, before an

equivalence is applied, it is difficult to see whether the resulting join tree has already been produced or not (see also Figure 2.6). Thus, this procedure is highly inefficient. Hence, it does not play any role in practice. Nevertheless, we give the pseudo-code for it, since it forms the basis for several of the following algorithms. We split the exhaustive transformation approach into two algorithms. One that applies all equivalences to a given join tree (**ApplyTransformations**) and another that does the loop (**ExhaustiveTransformation**). A transformation is applied in a directed way. Thus, we reformulate commutativity and associativity as rewrite rules using $\rightsquigarrow$ to indicate the direction.

The following table summarizes all rules commonly used in transformation-based and randomized join ordering algorithms. The first three are directly derived from the commutativity and associativity laws for the join. The other rules are shortcuts used under special circumstances. For example, left associativity may turn a left-deep tree into a bushy tree. When only left-deep trees are to be considered, we need a replacement for left associativity. This replacement is called left join exchange.

$R_1 \bowtie R_2$	$\rightsquigarrow$	$R_2 \bowtie R_1$	Commutativity
$(R_1 \bowtie R_2) \bowtie R_3$	$\rightsquigarrow$	$R_1 \bowtie (R_2 \bowtie R_3)$	Right Associativity
$R_1 \bowtie (R_2 \bowtie R_3)$	$\rightsquigarrow$	$(R_1 \bowtie R_2) \bowtie R_3$	Left Associativity
$(R_1 \bowtie R_2) \bowtie R_3$	$\rightsquigarrow$	$(R_1 \bowtie R_3) \bowtie R_2$	Left Join Exchange
$R_1 \bowtie (R_2 \bowtie R_3)$	$\rightsquigarrow$	$R_2 \bowtie (R_1 \bowtie R_3)$	Right Join Exchange

Two more rules are often used to transform left-deep trees. The first operation (*swap*) exchanges two arbitrary relations in a left-deep tree. The second operation (*3Cycle*) performs a cyclic rotation of three arbitrary relations in a left-deep tree. To account for different join methods, a rule called *join method exchange* is introduced.

The first rule set (*RS-0*) we are using contains the commutativity rule and both associativity rules. Applying associativity can lead to cross products. RS-0 If we do not want to consider cross products, we only apply any of the two associativity rules if the resulting expression does not contain a cross product. It is easy to extend **ApplyTransformations** to cover this by extending the if conditions with

and (**ConsiderCrossProducts** || **connected(.)**)

where the argument of **connected** is the result of applying a transformation.

ExhaustiveTransformation($\{R_1, \dots, R_n\}$)

Input: a set of relations

Output: an optimal join tree

Let T be an arbitrary join tree for all relations

Done = $\emptyset$; // contains all trees processed

ToDo = $\{T\}$; // contains all trees to be processed

while (!empty(ToDo)) {

Let T be an arbitrary tree in ToDo

```

    ToDo \ = T;
    Done ∪ = T;
    Trees = ApplyTransformations(T);
    for all T ∈ Trees do {
        if (T ∉ ToDo ∪ Done) {
            ToDo += T;
        }
    }
}
return cheapest tree found in Done;

ApplyTransformations(T)
Input: join tree
Output: all trees derivable by associativity and commutativity
Trees = ∅;
Subtrees = all subtrees of T rooted at inner nodes
for all S ∈ Subtrees do {
    if (S is of the form S1 ⋈ S2) {
        Trees += S2 ⋈ S1;
    }
    if (S is of the form (S1 ⋈ S2) ⋈ S3) {
        Trees += S1 ⋈ (S2 ⋈ S3);
    }
    if (S is of the form S1 ⋈ (S2 ⋈ S3)) {
        Trees += (S1 ⋈ S2) ⋈ S3;
    }
}
return Trees;

```

Besides the problems mentioned above, this algorithm also has the problem that the sharing of subtrees is a non-trivial task. In fact, we assume that **ApplyTransformations** produces modified copies of T . To see how **ExhaustiveTransformation** works, consider again Figure 2.6. Assume that the top-left join tree is the initial join tree. Then, from this join tree **ApplyTransformations** produces all trees reachable by some edge. All of these are then added to **ToDo**. The next call to **ApplyTransformations** with any to the produced join trees will have the initial join tree contained in **Trees**. The complete set of visited join trees after this step is determined from the initial join tree by following at most two edges.

Let us reformulate the algorithm such that it uses a data structure similar to dynamic programming or memoization in order to avoid duplicate work. For any subset of relations, dynamic programming remembers the best join tree. This does not quite suffice for the transformation-based approach. Instead, we have to keep all join trees generated so far including those differing in the order of the arguments or a join operator. However, subtrees can be shared. This is done by keeping pointers into the data structure (see below). So, the difference between dynamic programming and the transformation-based approach becomes smaller. The main remaining difference is that dynamic programming

only considers these join trees while with the transformation-based approach we have to keep the considered join trees since other join trees (more beneficial) might be generatable from them.

The data structure used for remembering trees is often called the MEMO structure. For every subset of relations to be joined (except the empty set), a *class* exists in the MEMO structure. Each class contains all the join trees that join exactly the relations describing the class. Here is an example for join trees containing three relations.

$\{R_1, R_2, R_3\}$	$\{R_1, R_2\} \bowtie R_3, R_3 \bowtie \{R_1, R_2\},$ $\{R_1, R_3\} \bowtie R_2, R_2 \bowtie \{R_1, R_3\},$ $\{R_2, R_3\} \bowtie R_1, R_1 \bowtie \{R_2, R_3\}$
$\{R_2, R_3\}$	$\{R_2\} \bowtie \{R_3\}, \{R_3\} \bowtie \{R_2\}$
$\{R_1, R_3\}$	$\{R_1\} \bowtie \{R_3\}, \{R_3\} \bowtie \{R_1\}$
$\{R_1, R_2\}$	$\{R_1\} \bowtie \{R_2\}, \{R_2\} \bowtie \{R_1\}$
$\{R_3\}$	R_3
$\{R_2\}$	R_2
$\{R_1\}$	R_1

Here, we used the set notation $\{\dots\}$ as an argument to a join to denote a reference to the class of join trees joining the relations contained in it.

We reformulate our transformation-based algorithm such that it fills in and uses the MEMO structure [683]. In a first step, the MEMO structure is initialized by creating an arbitrary join tree for the class $\{R_1, \dots, R_n\}$ and then going down this join tree and creating an entry for every join encountered. Then, we call **ExploreClass** on the root class comprising all relations to be joined. **ExploreClass** then applies **ApplyTransformations2** to every member of the class it is called upon. **ApplyTransformations2** then applies all rules to generate alternatives.

ExhaustiveTransformation2(Query Graph G)

Input: a query specification for relations $\{R_1, \dots, R_n\}$.

Output: an optimal join tree

```

    initialize MEMO structure
    ExploreClass( $\{R_1, \dots, R_n\}$ )
    return best of class  $\{R_1, \dots, R_n\}$ 

```

ExploreClass(C)

Input: a class $C \subseteq \{R_1, \dots, R_n\}$

Output: none, but has side-effect on MEMO-structure

```

    while (not all join trees in  $C$  have been explored) {
        choose an unexplored join tree  $T$  in  $C$ 
        ApplyTransformation2( $T$ )
        mark  $T$  as explored
    }

```

```

}
return

```

ApplyTransformations2(T)

Input: a join tree of a class $\mathcal{C}$

Output: none, but has side-effect on MEMO-structure

```

ExploreClass(left-child( $T$ ));
ExploreClass(right-child( $T$ ));
foreach transformation  $\mathcal{T}$  and class member of child classes {
  foreach  $T'$  resulting from applying  $\mathcal{T}$  to  $T$  {
    if  $T'$  not in MEMO structure {
      add  $T'$  to class  $\mathcal{C}$  of MEMO structure
    }
  }
}
return

```

ApplyTransformations2 uses a set of transformations to be applied. We discuss now the effect of different transformation sets on the complexity of the algorithm. Applying **ExhaustiveTransformation2** with a rule set consisting of Commutativity and Left and Right Associativity generates $4^n - 3^{n+1} + 2^{n+2} - n - 2$ duplicates for n relations. Contrast this with the number of join trees contained in a completely filled MEMO structure³: $3^n - 2^{n+1} + n + 1$. This clearly shows the problem.

The problem of generating the same join tree several times was considered by Pellenkoff, Galindo-Legaria, and Kersten [683, 684, 685]. The solution lies in parameterizing **ExhaustiveTransformation2** by an appropriate set of transformations. The basic idea is to remember for every join operator which rules are applicable to it. For example, after applying commutativity to a join operator, we disable commutativity for it.

For acyclic queries, the following rule set guarantees that all bushy join trees are generated, but no duplicates [685]. Thereby, cross products are not considered. That is, a rule is only applicable if it does not result in a cross product. This restricts the applicability of the above algorithm to connected queries. We use C_i to denote some class of the MEMO structure. We call the following rule set RS-1:

RS-1

T_1 : Commutativity $C_1 \bowtie_0 C_2 \rightsquigarrow C_2 \bowtie_1 C_1$

Disable all transformations T_1 , T_2 , and T_3 for $\bowtie_1$.

T_2 : Right Associativity $(C_1 \bowtie_0 C_2) \bowtie_1 C_3 \rightsquigarrow C_1 \bowtie_2 (C_2 \bowtie_3 C_3)$

Disable transformations T_2 and T_3 for $\bowtie_2$ and enable all rules for $\bowtie_3$.

T_3 : Left associativity $C_1 \bowtie_0 (C_2 \bowtie_1 C_3) \rightsquigarrow (C_1 \bowtie_2 C_2) \bowtie_3 C_3$

Disable transformations T_2 and T_3 for $\bowtie_3$ and enable all rules for $\bowtie_2$.

³The difference to the according number for dynamic programming is due to the fact that we have to keep alternatives generated by commutativity and that join trees for single relations are counted.

Class	Initialization	Transformation	Step
$\{R_1, R_2, R_3, R_4\}$	$\{R_1, R_2\} \bowtie_{111} \{R_3, R_4\}$	$\{R_3, R_4\} \bowtie_{000} \{R_1, R_2\}$	3
		$R_1 \bowtie_{100} \{R_2, R_3, R_4\}$	4
		$\{R_1, R_2, R_3\} \bowtie_{100} R_4$	5
		$\{R_2, R_3, R_4\} \bowtie_{000} R_1$	8
		$R_4 \bowtie_{000} \{R_1, R_2, R_3\}$	10
$\{R_2, R_3, R_4\}$		$R_2 \bowtie_{111} \{R_3, R_4\}$	4
		$\{R_3, R_4\} \bowtie_{000} R_2$	6
		$\{R_2, R_3\} \bowtie_{100} R_4$	6
		$R_4 \bowtie_{000} \{R_2, R_3\}$	7
$\{R_1, R_3, R_4\}$		$\{R_1, R_2\} \bowtie_{111} R_3$	5
$\{R_1, R_2, R_4\}$		$R_3 \bowtie_{000} \{R_1, R_2\}$	9
$\{R_1, R_2, R_3\}$		$R_1 \bowtie_{100} \{R_2, R_3\}$	9
		$\{R_2, R_3\} \bowtie_{000} R_1$	9
$\{R_3, R_4\}$	$R_3 \bowtie_{111} R_4$	$R_4 \bowtie_{000} R_3$	2
$\{R_2, R_4\}$			
$\{R_2, R_3\}$			
$\{R_1, R_4\}$			
$\{R_1, R_3\}$			
$\{R_1, R_2\}$			
	$R_1 \bowtie_{111} R_2$	$R_2 \bowtie_{000} R_1$	1

Figure 3.14: Example of rule transformations (RS-1)

In order to be able to follow these rules, the procedure **ApplyTransformations2** has to be enhanced such that it is able to keep track of the application history of the rules for every join operator. The additional memory requirement is neglectible, since a single bit for each rules suffices.

As an example, let us consider the chain query $R_1 - R_2 - R_3 - R_4$. Figure 3.14 shows the MEMO structure. The first column gives the sets of the relations identifying each class. We leave out the single relation classes assuming that $\{R_i\}$ has R_i as its only join tree which is marked as explored.

The second column shows the initialization with an arbitrarily chosen join tree. The third column is the one filled by the **Apply Transformation2** procedure. We apply the rule set RS-1, which consists of three transformations. Each join is annotated with three bits, where the i -th bit indicates whether T_i is applicable (1) or not (0). After initializing the MEMO structure, **ExhaustiveTransformation2** calls **ExploreClass** for $\{R_1, R_2, R_3, R_4\}$. The only (unexplored) join tree is $\{R_1, R_2\} \bowtie_{111} \{R_3, R_4\}$, which will become the argument of **ApplyTransformations2**. Next, **ExploreClass** is called on $\{R_1, R_2\}$ and $\{R_3, R_4\}$. In both cases, T_1 is the only applicable rule, and the result is shown in the third column under steps 1 and 2. Now we have to apply all transformations on $\{R_1, R_2\} \bowtie_{111} \{R_3, R_4\}$.

Commutativity T_1 gives us $\{R_3, R_4\} \bowtie_{000} \{R_1, R_2\}$ (Step 3). For right associativity, we have two elements in class $\{R_1, R_2\}$. Substituting them and applying T_2 gives

1. $(R_1 \bowtie R_2) \bowtie \{R_3, R_4\} \rightsquigarrow R_1 \bowtie_{100} (R_2 \bowtie_{111} \{R_3, R_4\})$
2. $(R_2 \bowtie R_1) \bowtie \{R_3, R_4\} \rightsquigarrow R_2 \bowtie_{111} (R_1 \times \{R_3, R_4\})$

The latter contains a cross product. This leaves us with the former as the result of Step 4. The right argument of the top most join is $R_2 \bowtie_{111} \{R_3, R_4\}$. Since we do not find it in class $\{R_2, R_3, R_4\}$, we add it (4).

T_3 is next.

1. $\{R_1, R_2\} \bowtie (R_3 \bowtie R_4) \rightsquigarrow (\{R_1, R_2\} \bowtie_{111} R_3) \bowtie_{100} R_4$
2. $\{R_1, R_2\} \bowtie (R_4 \bowtie R_3) \rightsquigarrow (\{R_1, R_2\} \times R_4) \bowtie_{100} R_3$

The latter contains a cross product. This leaves us with the former as the result of Step 5. We also add $\{R_1, R_2\} \bowtie_{111} R_3$. Now that $\{R_1, R_2\} \bowtie_{111} \{R_3, R_4\}$ is completely explored, we turn to $\{R_3, R_4\} \bowtie_{000} \{R_1, R_2\}$, but all transformations are disabled here.

$R_1 \bowtie_{100} \{R_2, R_3, R_4\}$ is next. First, $\{R_2, R_3, R_4\}$ has to be explored. The only entry is $R_2 \bowtie_{111} \{R_3, R_4\}$. Remember that $\{R_3, R_4\}$ is already explored. T_2 is not applicable. The other two transformations give us

$$T_1 \quad \{R_3, R_4\} \bowtie_{000} R_2$$

$$T_3 \quad (R_2 \bowtie_{000} R_3) \bowtie_{100} R_4 \text{ and } (R_2 \times R_4) \bowtie_{100} R_3$$

Those join trees not exhibiting a cross product are added to the MEMO structure under 6. Applying commutativity to $\{R_2, R_4\} \bowtie_{100} R_3$ gives 7. Commutativity is the only rule enabled for $R_1 \bowtie_{100} \{R_2, R_3, R_4\}$. Its application results in 8.

$\{R_1, R_2, R_3\} \bowtie_{100} R_4$ is next. It is simple to explore the class $\{R_1, R_2, R_3\}$ with its only entry $\{R_1, R_2\} \bowtie_{111} R_3$:

$$T_1 \quad R_3 \bowtie_{000} \{R_1, R_2\}$$

$$T_2 \quad R_1 \bowtie_{100} (R_2 \bowtie_{111} R_3) \text{ and } R_2 \bowtie_{100} (R_1 \times R_3)$$

Commutativity can still be applied to $R_1 \bowtie_{100} (R_2 \bowtie_{111} R_3)$. All the new entries are numbered 9. Commutativity is the only rule enabled for $\{R_1, R_2, R_3\} \bowtie_{100} R_4$. Its application results in 10.

□

The next two sets of transformations were originally intended for generating all bushy/left-deep trees for a clique query [684]. They can, however, also be used to generate all bushy trees when cross products are considered. The rule set RS-2 for bushy trees is

T_1 : Commutativity $C_1 \bowtie_0 C_2 \rightsquigarrow C_2 \bowtie_1 C_1$

Disable all transformations T_1 , T_2 , T_3 , and T_4 for $\bowtie_1$.

T_2 : Right Associativity $(C_1 \bowtie_0 C_2) \bowtie_1 C_3 \rightsquigarrow C_1 \bowtie_2 (C_2 \bowtie_3 C_3)$

Disable transformations T_2 , T_3 , and T_4 for $\bowtie_2$.

T_3 : Left Associativity $C_1 \bowtie_0 (C_2 \bowtie_1 C_3) \rightsquigarrow (C_1 \bowtie_2 C_2) \bowtie_3 C_3$

Disable transformations T_2 , T_3 and T_4 for $\bowtie_3$.

T_4 : Exchange $(C_1 \bowtie_0 C_2) \bowtie_1 (C_3 \bowtie_2 C_4) \rightsquigarrow (C_1 \bowtie_3 C_3) \bowtie_4 (C_2 \bowtie_5 C_4)$

Disable all transformations T_1 , T_2 , T_3 , and T_4 for $\bowtie_4$.

If we initialize the MEMO structure with left-deep trees, we can strip down the above rule set to Commutativity and Left Associativity. The reason is an observation made by Shapiro et al.: from a left-deep join tree we can generate all bushy trees with only these two rules [798].

If we want to consider only left-deep trees, the following rule set RS-3 is appropriate:

T_1 Commutativity $R_1 \bowtie_0 R_2 \rightsquigarrow R_2 \bowtie_1 R_1$

Here, the R_i are restricted to classes with exactly one relation. T_1 is disabled for $\bowtie_1$.

T_2 Right Join Exchange $(C_1 \bowtie_0 C_2) \bowtie_1 C_3 \rightsquigarrow (C_1 \bowtie_2 C_3) \bowtie_3 C_2$

Disable T_2 for $\bowtie_3$.

3.3 Probabilistic Algorithms

3.3.1 Generating Random Left-Deep Join Trees with Cross Products

The basic idea of the algorithms in this section and the following sections is to generate a set of randomly chosen join trees, evaluate their costs, and return the best one. The problem with this approach lies in the random generation of join trees: every join tree has to be generated with equal probability. Although there are some advocates of the pure random approach [303, 304, 306, 302], typically a random join tree or a set of random join trees is used in subsequent algorithms like iterative improvement and simulated annealing.

Obviously, if we do not consider cross products the problem is really hard, since the query graph plays an important role. So let us start with the simplest case where random join trees are generated that might contain cross products even for connected query graphs. Then, any join tree is a valid join tree.

The general idea behind all algorithms is the following. Assume that the number of join trees in the considered search space is known to be N . Then, instead of generating a random join tree directly, a bijective mapping from the interval of non-negative integers $[0, N[$ to a join tree in the search space is established. Then, a random join tree can be generated by (1) generating a random number in $[0, N[$ and (2) mapping the number to the join tree. The problem of bijectively mapping an interval of non-negative integers to elements of a set is usually called *unranking*. The opposite mapping is called *ranking*. Obviously, the crux in our case is the efficiency of the unranking problem.

We start with generating random left-deep join trees for n relations. This problem is identical to generating random permutations. That is, we look for a fast unranking algorithm that maps the non-negative integers in $[0, n[$ to permutations. Let us consider permutations of the numbers $\{0, \dots, n-1\}$. A mapping between these numbers and relations is established easily, e.g. via an array. The traditional approach to ranking/unranking of permutations is to first define an ordering on the permutations and then find a ranking and unranking algorithm relative to that ordering. For the lexicographic order, algorithms require $O(n^2)$ time [556, 725]. More sophisticated algorithms separate the ranking/unranking algorithms into two phases. For ranking, first the *inversion vector* of the permutation is established. Then, ranking takes place for the inversion vector. Unranking works in the opposite direction. The *inversion vector* of a permutation $\pi = \pi_0, \dots, \pi_{n-1}$ is defined to be the sequence $v = v_0, \dots, v_{n-1}$, where v_i is equal to the number of entries π_j with $\pi_j > \pi_i$ and $j < i$. Inversion vectors uniquely determine a permutation [876]. However, naive algorithms of this approach again require $O(n^2)$ time. Better algorithms require $O(n \log n)$. Using an elaborated data structure, Dietz' algorithm requires $O((n \log n)/(\log \log n))$ [242]. Other orders like the Steinhaus-Johnson-Trotter order have been exploited for ranking/unranking but do not yield any run-time advantage over the above mentioned algorithms (see [519, 725]).

Since it is not important for our problem that any order constraints are satisfied for the ranking/unranking functions, we use the fastest possible algorithm established by Myrvold and Ruskey [636]. It runs in $O(n)$ which is also easily seen to be a lower bound.

The algorithm is based on the standard algorithm to generate random permutations [224, 251, 630]. An array π is initialized such that $\pi[i] = i$ for $0 \leq i \leq n-1$. Then, the loop

```
for ( $k = n - 1$ ;  $k \geq 0$ ;  $--k$ ) swap( $\pi[k], \pi[\text{random}(k)]$ );
```

is executed where **swap** exchanges two elements and **random**(k) generates a random number in $[0, k]$. This algorithm randomly picks any of the possible permutations. Assume the random elements produced by the algorithm are $r_{n-1}, \dots, r_0$ where $0 \leq r_i \leq i$. Obviously, there are exactly $n(n-1)(n-2) \dots 1 = n!$ such sequences and there is a one-to-one correspondence between these sequences and the set of all permutations. We can thus unrank $r \in [0, n[$ by turning it into a unique sequence of values $r_{n-1}, \dots, r_0$. Note that after executing the swap with r_{n-1} , every value in $[0, n[$ is possible at position $\pi[n-1]$. Further, $\pi[n-1]$ is never touched again. Hence, we can unrank r as follows. We first set $r_{n-1} = r \bmod n$ and perform the swap. Then, we define $r' = \lfloor r/n \rfloor$ and iteratively unrank r' to construct a permutation of $n-1$ elements. The following algorithm realizes this idea.

Unrank(n, r) {

Input: the number n of elements to be permuted

and the rank r of the permutation to be constructed

```

Output: a permutation  $\pi$ 
  for ( $i = 0$ ;  $i < n$ ;  $++i$ )  $\pi[i] = i$ ;
  Unrank-Sub( $n, r, \pi$ );
  return  $\pi$ ;
}

Unrank-Sub( $n, r, \pi$ ) {
  for ( $i = n$ ;  $i > 0$ ;  $--i$ ) {
    swap( $\pi[i - 1]$ ,  $\pi[r \bmod i]$ );
     $r = \lfloor r/i \rfloor$ ;
  }
}

```

3.3.2 Generating Random Join Trees with Cross Products

Next, we want to randomly construct bushy plans possibly containing cross products. This is done in several steps:

1. Generate a random number b in $[0, C(n-1)[$.
2. Unrank b to obtain a bushy tree with $n-1$ inner nodes.
3. Generate a random number p in $[0, n![$.
4. Unrank p to obtain a permutation.
5. Attach the relations in order p from left to right as leaf nodes to the binary tree obtained in Step 2.

The only step that we still have to discuss is Step 2. It is a little involved and we can only try to bring across the general idea. For details, the reader is referred to the literature [556, 557, 558].

Consider Figure 3.15. It contains all 14 possible trees with four inner nodes. The trees are ordered according to the rank we will consider. The bottom-most number below any tree is its rank in $[0, 14[$. While unranking, we do not generate the trees directly, but an encoding of the tree instead. This encoding works as follows. Any binary tree corresponds to a word in a Dyck language with one pair of parenthesis. The alphabet hence consists of $\Sigma = \{ '(', ') '\}$. For join trees with n inner nodes, we use Dyck words of length $2n$ whose parenthesization is correct. That is, for every $'('$, we have a subsequent $)'$. From a given join tree, we obtain the Dyck word by a preorder traversal. Whenever we encounter an inner node, we encode this with a $'('$. All but the last leaf nodes are encoded by a $)'$. Appending all these $2n$ encodings gives us a Dyck word of length $2n$. Figure 3.15 shows directly below each tree its corresponding Dyck word. In the line below, we simply changed the representation by substituting every $'('$ by a $'1'$ and every $)'$ by a $'0'$. The encoding that will be generated by the unranking algorithm is shown in the third line below each tree: we remember the places (index in the bit-string) where we find a $'1'$.

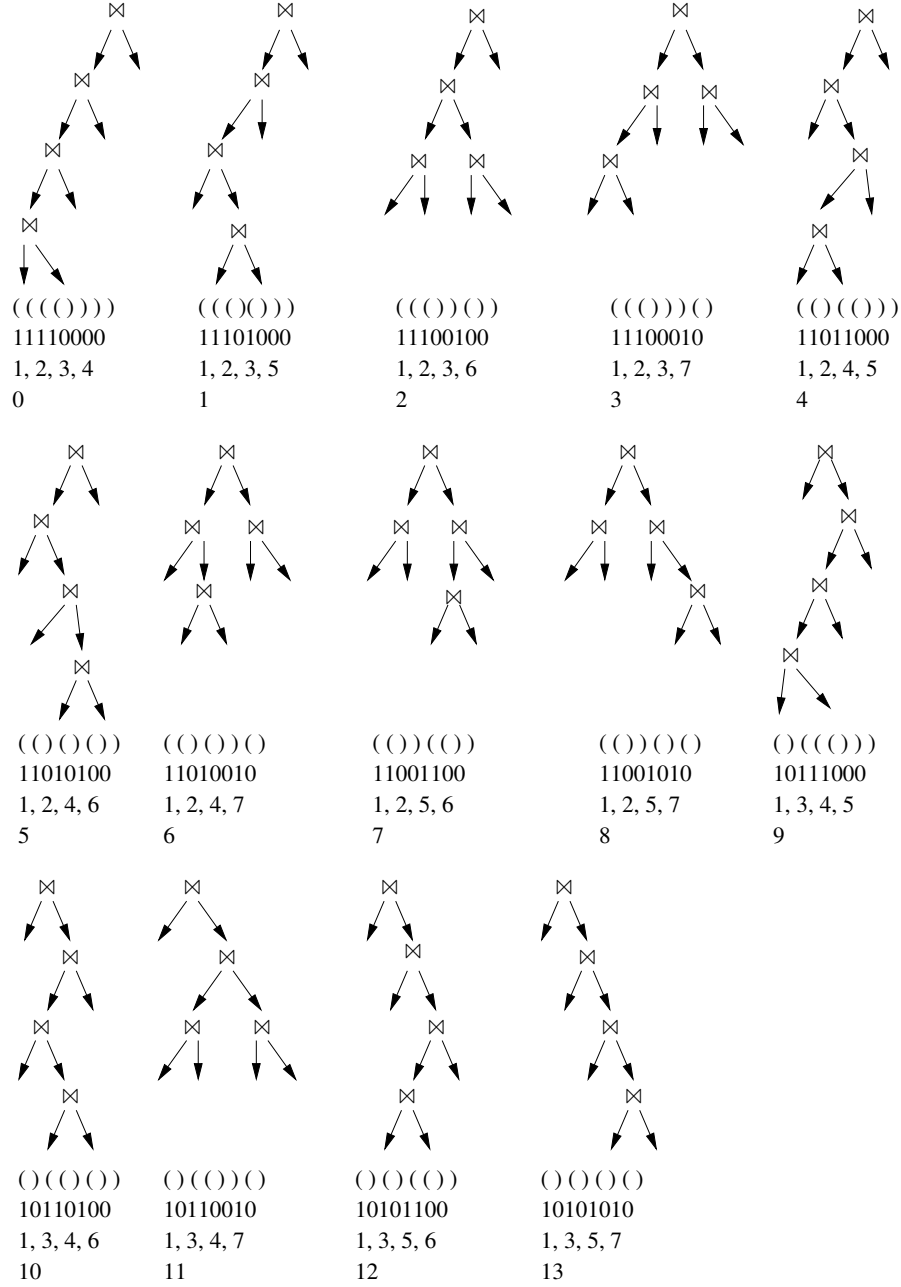


Figure 3.15: Encoding Trees

In order to do the unranking, we need to do some counting. Therefore, we map Dyck words to paths in a triangular grid. For $n = 4$ this grid is shown in Figure 3.16. We always start at $(0,0)$ which means that we have not opened a parenthesis. When we are at (i,j) , opening a parenthesis corresponds to going to $(i+1, j+1)$ and closing a parenthesis to going to $(i+1, j-1)$. We have thus established a bijective mapping between Dyck words and paths in the grid. Thus counting Dyck words corresponds to counting paths.

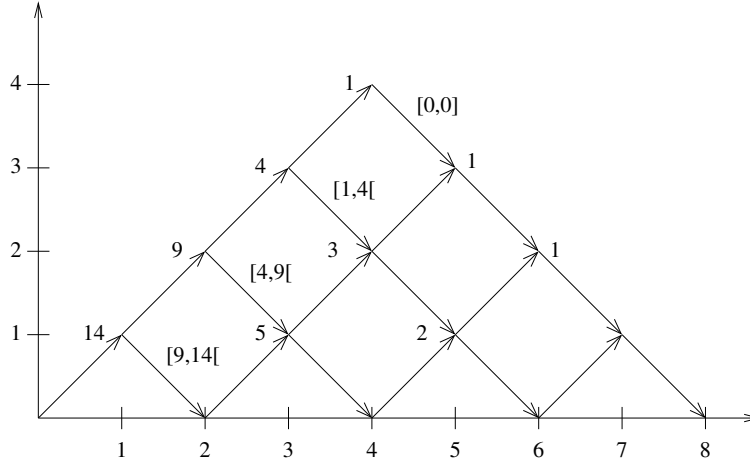


Figure 3.16: Paths

The number of different paths from $(0, 0)$ to (i, j) can be computed by

$$p(i, j) = \frac{j+1}{i+1} \binom{i+1}{\frac{1}{2}(i+j)+1}$$

These numbers are called the *Ballot numbers* [131]. The number of paths from (i, j) to $(2n, 0)$ can thus be computed as (see [557, 558]):

$$q(i, j) = p(2n - i, j)$$

Note the special case $q(0, 0) = p(2n, 0) = C(n)$. In Figure 3.16, we annotated nodes (i, j) by $p(i, j)$. These numbers can be used to assign (sub-) intervals to paths (Dyck words, trees). For example, if we are at $(4, 4)$, there exists only a single path to $(2n, 0)$. Hence, the path that travels the edge $(4, 4) \rightarrow (5, 3)$ has rank 0. From $(3, 3)$ there are four paths to $(2n, 0)$, one of which we already considered. This leaves us with three paths that travel the edge $(3, 3) \rightarrow (4, 2)$. The paths in this part as assigned ranks in the interval $[1, 4[$. Figure 3.16 shows the intervals near the edges. For unranking, we can now proceed as follows. Assume we have a rank r . We consider opening a parenthesis (go from (i, j) to $(i+1, j+1)$) as long as the number of paths from that point does no longer exceed our rank r . If it does, we close a parenthesis instead (go from (i, j) to $(i-1, j+1)$). Assume, that we went upwards to (i, j) and then had to go down to $(i-1, j+1)$. We subtract the number of paths from $(i+1, j+1)$ from our rank r and proceed iteratively from $(i-1, j+1)$ by going up as long as possible and going down again. Remembering the number of parenthesis opened and closed along our way results in the required encoding. The following algorithm finalizes these ideas.

UnrankTree(n, r)

Input: a number of inner nodes n and a rank $r \in [0, C(n-1)]$

Output: encoding of the inner leaves of a tree

```

lNoParOpen = 0;
lNoParClose = 0;
i = 1; // current encoding
j = 0; // current position in encoding array
while (j < n) {
    k = q(lNoParOpen + lNoParClose + 1, lNoParOpen - lNoParClose + 1);
    if (k ≤ r) {
        r -= k;
        ++lNoParClose;
    } else {
        aTreeEncoding[j++] = i;
        ++lNoParOpen;
    }
    ++i;
}

```

Given an array with the encoding of a tree, it is easy to construct the tree from it. The following procedure does that.

```

TreeEncoding2Tree(n, aEncoding) {
Input: the number of internal nodes of the tree n
Output: root node of the result tree
    root = new Node; /* root of the result tree */
    curr = root; /* curr: current internal node whose subtrees are to be created */
    i = 1; /* pointer to entry in encoding */
    child = 0; /* 0 = left, 1 = right: next child whose subtree is to be created */
    while (i < n) {
        lDiff = aEncoding[i] - aEncoding[i - 1];
        for (k = 1; k < lDiff; ++k) {
            if (child == 0) {
                curr->addLeftLeaf();
                child = 1;
            } else {
                curr->addRightLeaf();
                while (curr->right() != 0) {
                    curr = curr->parent();
                }
                child = 1;
            }
        }
    }
    if (child == 0) {
        curr->left(new Node(curr)); // curr becomes parent of new node
        curr = curr->left();
        ++i;
    }
}

```

```

    child = 0;
  } else {
    curr->right(new Node(curr));
    curr = curr->right();
    ++i;
    child = 0;
  }
}
while (curr != 0) {
  curr->addLeftLeaf(); // addLeftLeaf adds leaf if no left-child exists
  curr->addRightLeaf(); // analogous
  curr = curr->parent();
}
return root;
}

```

3.3.3 Generating Random Join Trees without Cross Products

A general solution for randomly generating join trees without cross products is not known. However, if we restrict ourselves to acyclic queries, we can apply an algorithm developed by Galindo-Legaria, Pellenkoff, and Kersten [304, 303, 306]. For this algorithm to work, we have to assume that the query graph is connected and acyclic.

For the rest of this section, we assume that $G = (V, E)$ is the query graph and $|V| = n$. That is, n relations are to be joined. No join tree contains a cross product. With every node in a join tree, we associate a *level*. The root has level 0. Its children have level 1, and so on. We further use lower-case letters for relations.

For a given query graph G , we denote by $\mathcal{T}_G$ the set of join trees for G . Let $\mathcal{T}_G^{v(k)} \subseteq \mathcal{T}_G$ be the subset of join trees where the leaf node (i.e. relation) v occurs at level k . Some trivial observations follow. If the query graph consists of a single node ($n = 1$), then $|\mathcal{T}_G| = |\mathcal{T}_G^{v(0)}| = 1$. If $n > 1$, the top node in the join tree is a join and not a relation. Hence, $|\mathcal{T}_G^{v(0)}| = 0$. Obviously, the maximum level that can occur in any join tree is $n - 1$. Hence, $|\mathcal{T}_G^{v(k)}| = 0$ if $k \geq n$. Since the level at which a leaf node v occurs in some join tree is unique, we have $\mathcal{T}_G = \cup_{k=0}^n \mathcal{T}_G^{v(k)}$ and $\mathcal{T}_G^{v(i)} \cap \mathcal{T}_G^{v(j)} = \emptyset$ for $i \neq j$. This gives us $|\mathcal{T}_G| = \sum_{k=0}^n |\mathcal{T}_G^{v(k)}|$.

The algorithm generates an unordered tree with n leaf nodes. If we wish to have a random ordered tree, we have to pick one of the 2^{n-1} possibilities to order the $(n - 1)$ joins within the tree. We proceed as follows. We start with some notation for lists, discuss how two lists can be merged, describe how a specific merge can be specified, and count the number of possible merges. This is important, since join trees will be described as lists of trees. Given a leaf node v , we simply traverse the path from the root to v . Thereby, subtrees that branch off can be collected into a list of trees. After these remarks, we start developing the algorithm in several steps. First, we consider two operations

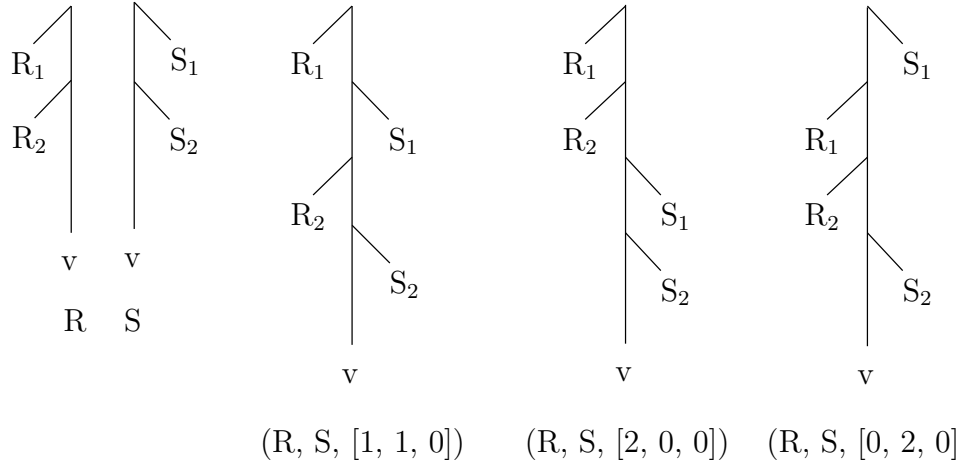


Figure 3.17: Tree-merge

with which we can construct new join trees: *leaf-insertion* introduces a new leaf node into a given tree and *tree-merging* merges two join trees. Since we do not want to generate cross products in this section, we have to apply these operations carefully. Therefor, we need a description of how to generate *all* valid join trees for a given query graph. The central data structure for this purpose is the *standard decomposition graph* (SDG). Hence, in the second step, we define SDGs and introduce an algorithm that derives an SDG from a given query graph. In the third step, we start counting. The fourth and final step consists of the unranking algorithm. We do not discuss the ranking algorithm. It can be found in [306].

We use the Prolog notation $|$ to separate the first element of a list from its tail. For example, the list $\langle a|t \rangle$ has a as its first element and a tail t . Assume that P is a property of elements. A list l' is the *projection* of a list L on P , if L' contains all elements of L satisfying the property P . Thereby, the order is retained. A list L is a *merge* of two disjoint lists L_1 and L_2 if L contains all elements from L_1 and L_2 and both are projections of L .

A merge of a list L_1 with a list L_2 whose respective lengths are l_1 and l_2 can be described by an array $\alpha = [\alpha_0, \dots, \alpha_{l_2}]$ of non-negative integers whose sum is equal to l_1 . The non-negative integer α_{i-1} gives the number of elements of L_1 which precede the i -th element of L_2 in the merged list. We obtain the merged list L by first taking α_0 elements from L_1 . Then, an element from L_2 follows. Then α_1 elements from L_1 and the next element of L_2 follow and so on. Finally follow the last α_{l_2} elements of L_1 . Figure 3.17 illustrates possible merges.

Compare list merges to the problem of non-negative (weak) integer composition [?]. There, we ask for the number of compositions of a non-negative integer n into k non-negative integers α_i with $\sum_{i=1}^k \alpha_k = n$. The answer is $\binom{n+k-1}{k-1}$ [829]. Since we have to decompose l_1 into $l_2 + 1$ non-negative integers, the number of possible merges is $M(l_1, l_2) = \binom{l_1+l_2}{l_2}$. The observation

$M(l_1, l_2) = M(l_1 - 1, l_2) + M(l_1, l_2 - 1)$ allows us to construct an array of size $n * n$ in $O(n^2)$ that materializes the values for M . This array will allow us to rank list merges in $O(l_1 + l_2)$.

The idea for establishing a bijection between $[1, M(l_1, l_2)]$ and the possible α s is a general one and used for all subsequent algorithms of this section. Assume that we want to rank the elements of some set S and $S = \cup_{i=0}^n S_i$ is partitioned into disjoint S_i . If we want to rank $x \in S_k$, we first find the *local rank* of $x \in S_k$. The rank of x is then defined as

$$\sum_{i=0}^{k-1} |S_i| + \text{local-rank}(x, S_k)$$

To unrank some number $r \in [1, N]$, we first find k such that

$$k = \min_j (r \leq \sum_{i=0}^j |S_i|)$$

Then, we proceed by unranking with the new local rank

$$r' = r - \sum_{i=0}^{k-1} |S_i|$$

within S_k .

Accordingly, we partition the set of all possible merges into subsets. Each subset is determined by α_0 . For example, the set of possible merges of two lists L_1 and L_2 with length $l_1 = l_2 = 4$ is partitioned into subsets with $\alpha_0 = j$ for $0 \leq j \leq 4$. In each partition, we have $M(j, l_2 - 1)$ elements. To unrank a number $r \in [1, M(l_1, l_2)]$, we first determine the partition by computing $k = \min_j r \leq \sum_{i=0}^j M(j, l_2 - 1)$. Then, $\alpha_0 = l_1 - k$. With the new rank $r' = r - \sum_{i=0}^k M(j, l_2 - 1)$, we start iterating all over. The following table gives the numbers for our example and can be used to understand the unranking algorithm. The algorithm itself can be found in Figure 3.18.

k	α_0	$(k, l_2 - 1)$	$M(k, l_2 - 1)$	rank intervals
0	4	(0, 3)	1	[1, 1]
1	3	(1, 3)	4	[2, 5]
2	2	(2, 3)	10	[6, 15]
3	1	(3, 3)	20	[16, 35]
4	0	(4, 3)	35	[36, 70]

We now turn to the *anchored list representation* of join trees.

Definition 3.3.1 *Let T be a join tree and v be a leaf of T . The anchored list representation L of T is constructed as follows:*

- *If T consists of the single leaf node v , then $L = \langle \rangle$.*
- *If $T = (T_1 \bowtie T_2)$ and without loss of generality v occurs in T_2 , then $L = \langle T_1 | L_2 \rangle$, where L_2 is the anchored list representation of T_2 .*

```

UnrankDecomposition( $r, l_1, l_2$ )
Input: a rank  $r$ , two list sizes  $l_1$  and  $l_2$ 
Output: a merge specification  $\alpha$ .
for ( $i = 0$ ;  $i \leq l_2$ ;  $++i$ ) {
     $\alpha[i] = 0$ ;
}
 $i = k = 0$ ;
while ( $l_1 > 0 \ \&\& \ l_2 > 0$ ) {
     $m = M(k, l_2 - 1)$ ;
    if ( $r \leq m$ ) {
         $\alpha[i++] = l_1 - k$ ;
         $l_1 = k$ ;
         $k = 0$ ;
         $--l_2$ ;
    } else {
         $r -= m$ ;
         $++k$ ;
    }
}
 $\alpha[i] = l_1$ ;
return  $\alpha$ ;

```

Figure 3.18: Algorithm UnrankDecomposition

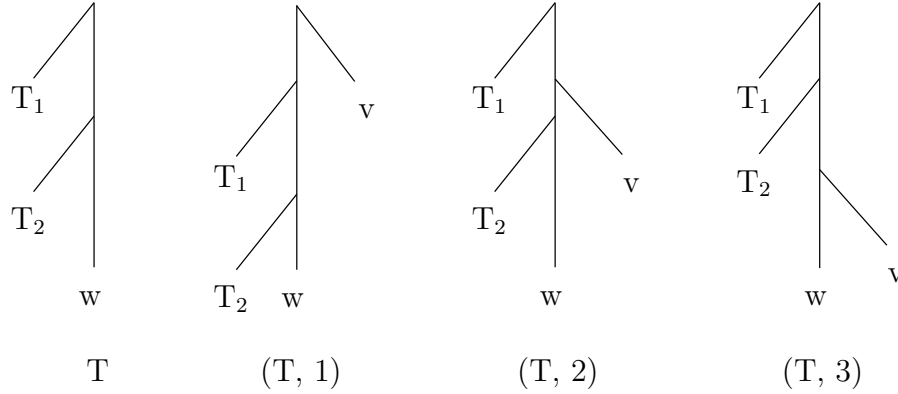


Figure 3.19: Leaf-insertion

We then write $T = (L, v)$.

Observe that if $T = (L, v) \in \mathcal{T}_G$, then $T \in \mathcal{T}_G^{v(k)} \prec \succ |L| = k$.

The operation *leaf-insertion* is illustrated in Figure 3.19. A new leaf v is inserted into the tree at level k . Formally, it is defined as follows.

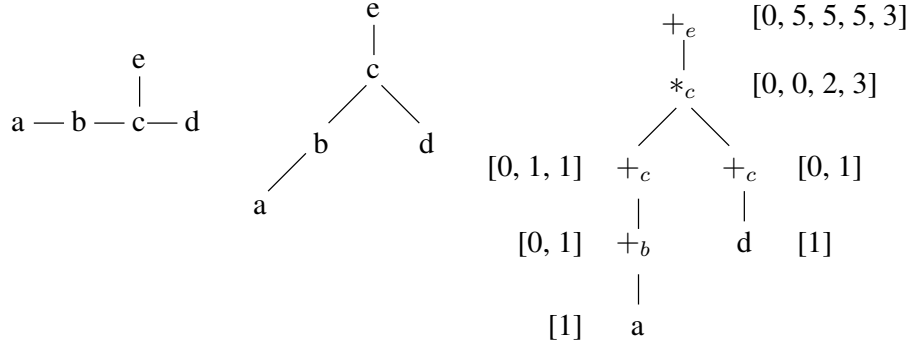


Figure 3.20: A query graph, its tree, and its standard decomposition graph

Definition 3.3.2 Let $G = (V, E)$ be a query graph, T a join tree of G . $v \in V$ be such that $G' = G|_{V \setminus \{v\}}$ is connected, $(v, w) \in E$, $1 \leq k < n$, and

$$T = (\langle T_1, \dots, T_{k-1}, v, T_{k+1}, \dots, T_n \rangle, w) \quad (3.10)$$

$$T' = (\langle T_1, \dots, T_{k-1}, T_{k+1}, \dots, T_n \rangle, w). \quad (3.11)$$

Then we call (T', k) an insertion pair on v and say that T is decomposed into (or constructed from) the pair (T', k) on v .

Observe that leaf-insertion defines a bijective mapping between $\mathcal{T}_G^{v(k)}$ and insertion pairs (T', k) on v , where T' is an element of the disjoint union $\bigcup_{i=k-1}^{n-2} \mathcal{T}_{G'}^{w(i)}$.

The operation *tree-merging* is illustrated in Figure 3.17. Two trees $R = (L_R, w)$ and $S = (L_S, w)$ on a common leaf w are merged by merging their anchored list representations.

Definition 3.3.3 Let $G = (V, E)$ be a query graph, $w \in V$, $T = (L, w)$ a join tree of G , $V_1, V_2 \subseteq V$ such that $G_1 = G|_{V_1}$ and $G_2 = G|_{V_2}$ are connected, $V_1 \cup V_2 = V$, and $V_1 \cap V_2 = \{w\}$. For $i = 1, 2$:

- Define the property P_i to be “every leaf of the subtree is in V_i ”.
- Let L_i be the projection of L on P_i .
- $T_i = (L_i, w)$.

Let α be the integer composition such that L is the result of merging L_1 and L_2 on α . Then we call (T_1, T_2, α) a merge triplet. We say that T is decomposed into (constructed from) (T_1, T_2, α) on V_1 and V_2 .

Observe that the *tree-merging* operation defines a bijective mapping between $\mathcal{T}_G^{w(k)}$ and merge triplets (T_1, T_2, α) , where $T_1 \in \mathcal{T}_{G_1}^{w(i)}$, $T_2 \in \mathcal{T}_{G_2}^{w(k-i)}$, and α specifies a merge of two lists of sizes i and $k-i$. Further, the number of these merges (i.e. the number of possibilities for α) is $\binom{i+(k-i)}{k-i} = \binom{k}{i}$.

A *standard decomposition graph* of a query graph describes the possible constructions of join trees. It is not unique (for $n > 1$) but anyone can be used

to construct all possible unordered join trees. For each of our two operations it has one kind of inner nodes. A unary node labeled $+_v$ stands for leaf-insertion of v . A binary node labeled $*_w$ stands for tree-merging its subtrees whose only common leaf is w .

The standard decomposition graph of a query graph $G = (V, E)$ is constructed in three steps:

1. pick an arbitrary node $r \in V$ as its root node;
2. transform G into a tree G' by directing all edges away from r ;
3. call $\text{QG2SDG}(G', r)$

with

```

QG2SDG( $G', r$ )
Input:  a query tree  $G' = (V, E)$  and its root  $r$ 
Output: a standard query decomposition tree of  $G'$ 
Let  $\{w_1, \dots, w_n\}$  be the children of  $v$ ;
switch ( $n$ ) {
  case 0:  label  $v$  with " $v$ ";
  case 1:
    label  $v$  as " $+_v$ ";
    QG2SDG( $G', w_1$ );
  otherwise:
    label  $v$  as " $*_v$ ";
    create new nodes  $l, r$  with label  $+_v$ ;
     $E \setminus = \{(v, w_i) | 1 \leq i \leq n\}$ ;
     $E \cup = \{(v, l), (v, r), (l, w_1)\} \cup \{(r, w_i) | 2 \leq i \leq n\}$ ;
    QG2SDG( $G', l$ );
    QG2SDG( $G', r$ );
}
return  $G'$ ;

```

Note that **QG2SDG** transforms the original graph G' into its SDG by side-effects. Thereby, the n -ary tree is transformed into a binary tree similar to the procedure described by Knuth [504, Chap 2.3.2]. Figure 3.20 shows a query graph G , its tree G' rooted at e , and its standard decomposition tree.

For an efficient access to the number of join trees in some partition $\mathcal{T}_G^{v(k)}$ in the unranking algorithm, we materialize these numbers. This is done in the **count** array. The semantics of a **count** array $[c_0, c_1, \dots, c_n]$ of a node u with label $\circ_v$ ($\circ \in \{+, *\}$) of the SDG is that u can construct c_i different trees in which leaf v is at level i . Then, the total number of trees for a query can be computed by summing up all the c_i in the **count** array of the root node of the decomposition tree.

To compute the **count** and an additional **summand** adornment of a node labeled $+_v$, we use the following lemma.

Lemma 3.3.4 *Let $G = (V, E)$ be a query graph with n nodes, $v \in V$ such that $G' = G|_{V \setminus v}$ is connected, $(v, w) \in E$, and $1 \leq k < n$. Then*

$$|\mathcal{T}_G^{v(k)}| = \sum_{i \geq k-1} |\mathcal{T}_{G'}^{w(i)}|$$

This lemma follows from the observation made after the definition of the leaf-insertion operation.

The sets $\mathcal{T}_{G'}^{w(i)}$ used in the summands of Lemma 3.3.4 directly correspond to subsets $\mathcal{T}_G^{v(k),i}$ ($k-1 \leq i \leq n-2$) defined such that $T \in \mathcal{T}_G^{v(k),i}$ if

1. $T \in \mathcal{T}_G^{v(k)}$,
2. the insertion pair on v of T is (T', k) , and
3. $T' \in \mathcal{T}_{G'}^{w(i)}$.

Further, $|\mathcal{T}_G^{v(k),i}| = |\mathcal{T}_{G'}^{w(i)}|$. For efficiency, we materialize the summands in an array of arrays **summands**.

To compute the **count** and **summand** adornment of a node labeled $*_v$, we use the following lemma.

Lemma 3.3.5 *Let $G = (V, E)$ be a query graph, $w \in V$, $T = (L, w)$ a join tree of G , $V_1, V_2 \subseteq V$ such that $G_1 = G|_{V_1}$ and $G_2 = G|_{V_2}$ are connected, $V_1 \cup V_2 = V$, and $V_1 \cap V_2 = \{v\}$. Then*

$$|\mathcal{T}_G^{v(k)}| = \sum_i \binom{k}{i} |\mathcal{T}_{G_1}^{v(i)}| |\mathcal{T}_{G_2}^{v(k-i)}|$$

This lemma follows from the observation made after the definition of the tree-merge operation.

The sets $\mathcal{T}_{G'}^{w(i)}$ used in the summands of Lemma 3.3.5 directly correspond to subsets $\mathcal{T}_G^{v(k),i}$ ($0 \leq i \leq k$) defined such that $T \in \mathcal{T}_G^{v(k),i}$ if

1. $T \in \mathcal{T}_G^{v(k)}$,
2. the merge triplet on V_1 and V_2 of T is (T_1, T_2, α) , and
3. $T_1 \in \mathcal{T}_{G_1}^{v(i)}$.

Further, $|\mathcal{T}_G^{v(k),i}| = \binom{k}{i} |\mathcal{T}_{G_1}^{v(i)}| |\mathcal{T}_{G_2}^{v(k-i)}|$.

Before we come to the algorithm for computing the adornments **count** and **summands**, let us make one observation that follows directly from the above two lemmata. Assume a node v whose **count** array is $[c_1, \dots, c_m]$ and whose **summands** is $s = [s^0, \dots, s^n]$ with $s_i = [s_0^i, \dots, s_m^i]$, then $c_i = \sum_{j=0}^m s_j^i$ holds. Figure 3.21 contains the algorithm to adorn SDG's nodes with **count** and **summands**. It has worst-case complexity $O(n^3)$. Figure 3.20 shows the **count** adornment for the SDG. Looking at the **count** array of the root node, we see that the total number of join trees for our example query graph is 18.

The algorithm **UnrankLocalTreeNoCross** called by **UnrankTreeNoCross** adorns the standard decomposition graph with **insert-at** and **merge-using** annotations. These can then be used to extract the join tree.

```

Adorn( $v$ )
Input: a node  $v$  of the SDG
Output:  $v$  and nodes below are adorned by count and summands
Let  $\{w_1, \dots, w_n\}$  be the children of  $v$ ;
switch ( $n$ ) {
  case 0: count( $v$ ) := [1]; // no summands for  $v$ 
  case 1:
    Adorn( $w_1$ );
    assume count( $w_1$ ) =  $[c_0^1, \dots, c_{m_1}^1]$ ;
    count( $v$ ) =  $[0, c_1, \dots, c_{m_1+1}]$  where  $c_k = \sum_{i=k-1}^{m_1} c_i^1$ ;
    summands( $v$ ) =  $[s^0, \dots, s^{m_1+1}]$  where  $s^k = [s_0^k, \dots, s_{m_1+1}^k]$  and
     $s_i^k = \begin{cases} c_i^1 & \text{if } 0 < k \text{ and } k-1 \leq i \\ 0 & \text{else} \end{cases}$ 
  case 2:
    Adorn( $w_1$ );
    Adorn( $w_2$ );
    assume count( $w_1$ ) =  $[c_0^1, \dots, c_{m_1}^1]$ ;
    assume count( $w_2$ ) =  $[c_0^2, \dots, c_{m_2}^2]$ ;
    count( $v$ ) =  $[c_0, \dots, c_{m_1+m_2}]$  where
       $c_k = \sum_{i=0}^{m_1} \binom{k}{i} c_i^1 c_{k-i}^2$ ; //  $c_i^2 = 0$  for  $i \notin \{0, \dots, m_2\}$ 
    summands( $v$ ) =  $[s^0, \dots, s^{m_1+m_2}]$  where  $s^k = [s_0^k, \dots, s_{m_1}^k]$  and
     $s_i^k = \begin{cases} \binom{k}{i} c_i^1 c_{k-i}^2 & \text{if } 0 \leq k-i \leq m_2 \\ 0 & \text{else} \end{cases}$ 
}

```

Figure 3.21: Algorithm Adorn

```

UnrankTreeNoCross( $r, v$ )
Input: a rank  $r$  and the root  $v$  of the SDG
Output: adorned SDG
let count( $v$ ) =  $[x_0, \dots, x_m]$ ;
 $k := \min_j r \leq \sum_{i=0}^j x_i$ ; // efficiency: binary search on materialized sums.
 $r' := r - \sum_{i=0}^{k-1} x_i$ ;
UnrankLocalTreeNoCross( $v, r', k$ );

```

The following table shows the intervals associated with the partitions $\mathcal{T}_G^{e(k)}$ for the standard decomposition graph in Figure 3.20:

Partition	Interval
$\mathcal{T}_G^{e(1)}$	[1, 5]
$\mathcal{T}_G^{e(2)}$	[6, 10]
$\mathcal{T}_G^{e(3)}$	[11, 15]
$\mathcal{T}_G^{e(4)}$	[16, 18]

The unranking procedure makes use of unranking decompositions and unranking triplets. For the latter and a given X, Y, Z , we need to assign each member in

$$\{(x, y, z) | 1 \leq x \leq X, 1 \leq y \leq Y, 1 \leq z \leq Z\}$$

a unique number in $[1, XYZ]$ and base an unranking algorithm on this assignment. We leave this as a simple exercise to the reader and call the function $\text{UnrankTriplet}(r, X, Y, Z)$. Here, r is the rank and X, Y , and Z are the upper bounds for the numbers in the triplets. The code for unranking looks as follows:

$\text{UnrankingTreeNoCrossLocal}(v, r, k)$

Input: an SDG node v , a rank r , a number k identifying a partition

Output: adornments of the SDG as a side-effect

Let $\{w_1, \dots, w_n\}$ be the children of v

switch (n) {

case 0:

assert ($r = 1 \ \&\& \ k = 0$);

 // no additional adornment for v

case 1:

 let $\text{count}(v) = [c_0, \dots, c_n]$;

 let $\text{summands}(v) = [s^0, \dots, s^n]$;

assert ($k \leq n \ \&\& \ r \leq c_k$);

$k_1 = \min_j r \leq \sum_{i=0}^j s_i^k$;

$r_1 = r - \sum_{i=0}^{k_1-1} s_i^k$;

insert-at (v) = k ;

$\text{UnrankingTreeNoCrossLocal}(w_1, r_1, k_1)$;

case 2:

 let $\text{count}(v) = [c_0, \dots, c_n]$;

 let $\text{summands}(v) = [s^0, \dots, s^n]$;

 let $\text{count}(w_1) = [c_0^1, \dots, c_{n_1}^1]$;

 let $\text{count}(w_2) = [c_0^2, \dots, c_{n_2}^2]$;

assert ($k \leq n \ \&\& \ r \leq c_k$);

$k_1 = \min_j r \leq \sum_{i=0}^j s_i^k$;

$q = r - \sum_{i=0}^{k_1-1} s_i^k$;

$k_2 = k - k_1$;

$(r_1, r_2, a) = \text{UnrankTriplet}(q, c_{k_1}^1, c_{k_2}^2, \binom{k}{i})$;

$\alpha = \text{UnrankDecomposition}(a)$;

merge-using (v) = α ;

$\text{UnrankingTreeNoCrossLocal}(w_1, r_1, k_1)$;

$\text{UnrankingTreeNoCrossLocal}(w_2, r_2, k_2)$;

}

3.3.4 Quick Pick

The QuickPick algorithm of Waas and Pellenkoff [904, 905] does not generate random join trees in the strong sense but comes close to it and is far easier to implement and more broadly applicable. The idea is to randomly select an edge in the query graph and to construct a join tree corresponding to this edge.

```

QuickPick(Query Graph  $G$ )
Input: a query graph  $G = (\{R_1, \dots, R_n\}, E)$ 
Output: a bushy join tree
BestTreeFound = any join tree
while stopping criterion not fulfilled {
     $E' = E$ ;
    Trees =  $\{R_1, \dots, R_n\}$ ;
    while ( $|Trees| > 1$ ) {
        choose  $e \in E'$ ;
         $E' - = e$ ;
        if ( $e$  connects two relations in different subtrees  $T_1, T_2 \in Trees$ ) {
            Trees  $- = T_1$ ;
            Trees  $- = T_2$ ;
            Trees  $+ = CreateJoinTree(T_1, T_2)$ ;
        }
    }
    Tree = single tree contained in Trees;
    if ( $cost(Tree) < cost(BestTreeFound)$ ) {
        BestTreeFound = Tree;
    }
}
return BestTreeFound

```

3.3.5 Iterative Improvement

Swami and Gupta [860], Swami [859] and Ioannidis and Kang [452] applied the idea of iterative improvement to join ordering [452]. The idea is to start from a random plan and then to apply randomly selected transformations from a rule set if they improve the current join tree, until not further improvement is possible.

```

IterativeImprovementBase(Query Graph  $G$ )
Input: a query graph  $G = (\{R_1, \dots, R_n\}, E)$ 
Output: a join tree
do {
    JoinTree = random tree
    JoinTree = IterativeImprovement(JoinTree)
}

```

```

    if (cost(JoinTree) < cost(BestTree)) {
        BestTree = JoinTree;
    }
} while (time limit not exceeded)
return BestTree

IterativeImprovement(JoinTree)
Input:  a join tree
Output: improved join tree
do {
    JoinTree' = randomly apply a transformation to JoinTree;
    if (cost(JoinTree') < cost(JoinTree)) {
        JoinTree = JoinTree';
    }
} while (local minimum not reached)
return JoinTree

```

The number of variants of iterative improvements is large. The first parameter is the used rule set. To restrict search to left-deep trees, a rule set consisting of *swap* and *3cycle* is appropriate [860]. If we consider bushy trees, a complete set consisting of commutativity, associativity, left join exchange and right join exchange makes sense. This rule set (proposed by Ioannidis and Kang) is appropriate to explore the whole space of bushy join trees. A second parameter is how to determine whether the local minimum has been reached. Considering all possible neighbor states of a join tree is expensive. Therefore, a subset of size k is sometimes considered. Then, for example, k can be limited to the number of edges in the query graph [860].

3.3.6 Simulated Annealing

Iterative Improvement suffers from the drawback that it only applies a move if it improves the current plan. This leads to the problem that one is often stuck in a local minimum. Simulated annealing tries to avoid this problem by allowing moves that result in more expensive plans [457, 452, 860]. However, instead of considering every plan, only those whose cost increase does not exceed a certain limit are considered. During time, this limit decreases. This general idea is cast into the notion temperatures and probabilities of performing a selected transformation. A generic formulation of simulated annealing could look as follows:

```

SimulatedAnnealing(Query Graph  $G$ )
Input:  a query graph  $G = (\{R_1, \dots, R_n\}, E)$ 
Output: a join tree
BestTreeSoFar = random tree;
Tree = BestTreeSoFar;

```

```

do {
  do {
    Tree' = apply random transformation to Tree;
    if (cost(Tree') < cost(Tree)) {
      Tree = Tree';
    } else {
      with probability  $e^{-(cost(Tree')-cost(Tree))/temperature}$ 
        Tree = Tree';
    }
    if (cost(Tree) < cost(BestTreeSoFar)) {
      BestTreeSoFar = Tree';
    }
  } while (equilibrium not reached)
reduce temperature;
} while (not frozen)
return BestTreeSoFar

```

Besides the rule set used, the initial temperature, the temperature reduction, and the definitions of equilibrium and frozen determine the algorithm's behavior. For each of them several alternatives have been proposed in the literature. The starting temperature can be calculated as follows: determine the standard deviation σ of costs by sampling and multiply it with a constant value ([860] use 20). An alternative is to set the starting temperature twice the cost of the first randomly selected join tree [452] or to determine the starting temperature such that at least 40% of all possible transformations are accepted [834].

For temperature reduction, we can apply the formula $temp* = 0.975$ [452] or $\max(0.5, e^{-\frac{\lambda t}{\sigma}})$ [860].

The equilibrium is defined to be reached if for example the cost distribution of the generated solutions is sufficiently stable [860], the number of iterations is sixteen times the number of relations in the query [452], or number of iterations is the same as the number of relations in the query [834].

We can establish frozenness if the difference between the maximum and minimum costs among all accepted join trees at the current temperature equals the maximum change in cost in any accepted move at the current temperature [860], the current solution could not be improved in four outer loop iterations and the temperature has been fallen below one [452], or the current solution could not be improved in five outer loop iterations and less than two percent of the generated moves were accepted [834].

Considering databases are used in mission critical applications. Would you bet your business on these numbers?

3.3.7 Tabu Search

Morzy, Matyasiak and Salza applied Tabu Search to join ordering [629]. The general idea is that among all neighbors reachable via the transformations, only

the cheapest is considered even if its cost are higher than the costs of the current join tree. In order to avoid running into cycles, a tabu set is maintained. It contains the last join trees generated, and the algorithm is not allowed to visit them again. This way, it can escape local minima, since eventually all nodes in the valley of a local minimum will be in the tabu set. The stopping conditions could be that there ws no improvement over the current best solution found during the last given number of iterations or if the set neighbors minus the tabu set is empty (in line (*)).

Tabu Search looks as follows:

```

TabuSearch(Query Graph)
Input:  a query graph  $G = (\{R_1, \dots, R_n\}, E)$ 
Output: a join tree
Tree = random join tree;
BestTreeSoFar = Tree;
TabuSet =  $\emptyset$ ;
do {
    Neighbors = all trees generated by applying a transformation to Tree;
    Tree = cheapest in Neighbors  $\setminus$  TabuSet; (*)
    if (cost(Tree) < cost(BestTreeSoFar)) {
        BestTreeSoFar = Tree;
    }
    if(|TabuSet| > limit) remove oldest tree from TabuSet;
    TabuSet += Tree;
} while (not stopping condition satisfied);
return BestTreeSoFar;

```

3.3.8 Genetic Algorithms

Genetic algorithms are inspired by evolution: only the fittest survives [333]. They work with a population that evolves from generation to generation. Successors are generated by crossover and mutation. Further, a subset of the current population (the fittest) are propagated to the next generation (selection). The first generation is generated by a random generation process.

The problem is how to represent each individual in a population. The following analogies are used:

- Chromosome $\longleftrightarrow$ string
- Gene $\longleftrightarrow$ character

In order to solve an optimization problem with genetic algorithms, an encoding is needed as well as a specification for selection, crossover, and mutation.

Genetic algorithms for join ordering have been considered in [74, 834]. We first introduce alternative encodings, then come to the selection process, and finally discuss crossover and mutation.

Encodings We distinguish *ordered list* and *ordinal number* encodings. Both encodings are used for left-deep and bushy trees. In all cases we assume that the relations $R_1, \dots, R_n$ are to be joined and use the index i to denote R_i .

1. Ordered List Encoding

(a) left-deep trees

A left-deep join tree is encoded by a permutation of $1, \dots, n$. For instance, $((R_1 \bowtie R_4) \bowtie R_2) \bowtie R_3$ is encoded as “1423”.

(b) bushy trees

Bennet, Ferris, and Ioannidis proposed the following encoding scheme [74, 75]. A bushy join-tree without cartesian products is encoded as an ordered list of the edges in the join graph. Therefore, we number the edges in the join graph. Then the join tree is encoded in a bottom-up, left-to-right manner. See Figure 3.22 for an example.

2. Ordinal Number Encoding

(a) left-deep trees

A join tree is encoded by using a list of relations that is shortened whenever a join has been encoded. We start with the list $L = \langle R_1, \dots, R_n \rangle$. Then within L we find the index of first relation to be joined. Let this relation be R_i . R_i is the i -th relation in L . Hence, the first character in the chromosome string is i . We eliminate R_i from L . For every subsequent relation joined, we again determine its index in L , remove it from L and append the index to the chromosome string. For instance, starting with $\langle R_1, R_2, R_3, R_4 \rangle$, the left-deep join tree $((R_1 \bowtie R_4) \bowtie R_2) \bowtie R_3$ is encoded as “1311”.

(b) bushy trees

Again, we start with the list $L = \langle R_1, \dots, R_n \rangle$ and encode a bushy join tree in a bottom-up, left-to-right manner. Let $R_i \bowtie R_j$ be the first join in the join tree under this ordering. Then we look up their positions in L and add them to the encoding. Next we eliminate R_i and R_j from L and push $R_{i,j}$ to the front of it. We then proceed for the other joins by again selecting the next join which now can be between relations and/or subtrees. We determine their position within L , add these positions to the encoding, remove them from L , and insert a composite relation into L such that the new composite relation directly follows those already present. For instance, starting with the list $\langle R_1, R_2, R_3, R_4 \rangle$, the bushy join tree $((R_1 \bowtie R_2) \bowtie (R_3 \bowtie R_4))$ is encoded as “12 23 12”.

The encoding is completed by adding join methods.

Crossover A crossover generates a new solution from two individuals. Therefore, two partial solutions are combined. Obviously, its definition depends on the encoding. Two kinds of crossovers are distinguished: the subsequence and the subset exchange.

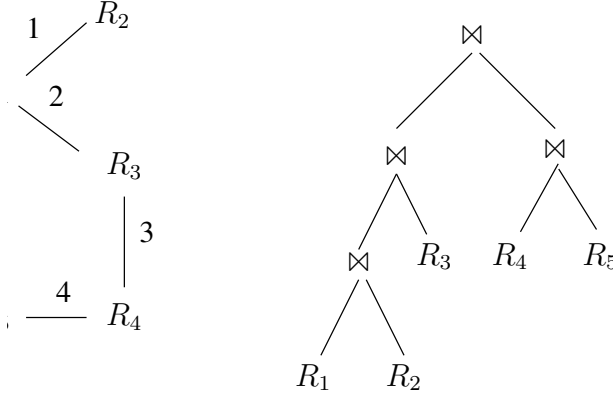


Figure 3.22: A query graph, a join tree, and its encoding

The subsequence exchange for the ordered list encoding works as follows. Assume two individuals with chromosomes $u_1v_1w_1$ and $u_2v_2w_2$. From these we generate $u_1v'_1w_1$ and $u_2v'_2w_2$, where v'_i is a permutation of the relations in v_i such that the order of their appearance is the same as in $u_{3-i}v_{3-i}w_{3-i}$. In order to adapt the subsequence exchange operator to the ordinal number encoding, we have to require that the v_i are of equal length ($|v_1| = |v_2|$) and occur at the same offset ($|u_1| = |u_2|$). We then simply swap the v_i . That is, we generate $u_1v_2w_1$ and $u_2v_1w_2$.

The subset exchange is defined only for the ordered list encoding. Within the two chromosomes, we find two subsequences of equal length comprising the same set of relations. These sequences are then simply exchanged.

Mutation A mutation randomly alters a character in the encoding. If duplicates must not occur — as in the ordered list encoding — swapping two characters is a perfect mutation.

Selection The probability of a join tree's survival is determined by its rank in the population. That is, we calculate the costs of the join trees encoded for each member of the population. Then we sort the population according to their associated costs and assign probabilities to each individual such that the best solution in the population has the highest probability to survive and so on. After probabilities have been assigned, we randomly select members of the population taking these probabilities into account. That is, the higher the probability of a member, the higher is its chance to survive.

Algorithm The genetic algorithm then works as follows. First, we create a random population of a given size (say 128). We apply crossover and mutation with a given rate, for example such that 65% of all members of a population participate in crossover, and 5% of all members of a population are subject to random mutation. Then we apply selection until we again have a population

of a given size. We stop after we have not seen an improvement within the population for a fixed number of iterations (say 30).

3.4 Hybrid Algorithms

All the algorithms we have seen so far can be combined to result in new approaches to join ordering. Some of the numerous possibilities have been described in the literature. We present them.

3.4.1 Two Phase Optimization

Two phase optimization combines Iterative Improvement with Simulated Annealing [452]. For a number of randomly generated initial trees, Iterative Improvement is used to find a local minimum. Then Simulated Annealing is started to find a better plan in the neighborhood of the local minima. The initial temperature of Simulated Annealing can be lower as is its original variants.

3.4.2 AB-Algorithm

The AB-Algorithm was developed by Swami and Iyer [861, 862]. It builds on the IKKBZ-Algorithm by resolving its limitations. First, if the query graph is cyclic, a spanning tree is selected. Second, two different cost functions for joins (join methods) are supported by the AB-Algorithm: nested loop join and sort merge join. In order to make the sort merge join's cost model fit the ASI property, it is simplified. Third, join methods are assigned randomly before IKKBZ is called. Afterwards, an iterative improvement phase follows. The algorithm can be formulated as follows:

```

AB(Query Graph  $G$ )
Input: a query graph  $G = (\{R_1, \dots, R_n\}, E)$ 
Output: a left-deep join tree
while (number of iterations  $\leq n^2$ ) {
  if  $G$  is cyclic take spanning tree of  $G$ 
  randomly attach a join method to each relation
  JoinTree = result of IKKBZ
  while (number of iterations  $\leq n^2$ ) {
    apply Iterative Improvement to JoinTree
  }
}
return best tree found

```

3.4.3 Toured Simulated Annealing

Lanzelotte, Valduriez, and Zäit introduced *toured simulated annealing* as a search strategy useful in distributed databases where the search space is even

larger than in centralized systems [532]. The basic idea is that simulated annealing is called n times with different initial join trees, if n is the number of relations to be joined. Each join sequence in the set `Solutions` produced by `GreedyJoinOrdering-3` is used to start an independent run of simulated annealing. As a result, the starting temperature can be decreased to 0.1 times the cost of the initial plan.

3.4.4 GOO-II

GOO-II appends an Iterative Improvement step to the GOO-Algorithm.

3.4.5 Iterative Dynamic Programming

Iterative Dynamic Programming combines heuristics with dynamic programming in order to overcome the deficiencies of both. It comes in two variants [515, 806]. The first variant, `IDP-1` (see Figure 3.23), first creates all join trees which contain up to k relations where k is a parameter of the algorithm. After this step, it selects the cheapest join tree comprising k relations, replaces it by a new compound relation and starts all over again. The iteration stops, when only one compound relation representing a join tree for all relations remains in the `ToDo` list.

The second variant, `IDP-2` (see Figure 3.24), works the other way round. It first applies a greedy heuristics to build join trees of size up to k . To the larger subtree it applies dynamic programming to improve it. The result of the optimized outcome of the greedy algorithm is then encapsulated in a new compound relation which replaces its constituent relations in the `ToDo` list. The algorithm then iterates until only one entry remains in the `ToDo` list.

Obviously, from these two basic variants several others can be derived. A systematic investigation of the basic algorithms and their variants is given by Kossmann and Stocker [515]. It turns out that the most promising variants exist for `IDP-1`.

3.5 Ordering Order-Preserving Joins

This section covers an algorithm for ordering order-preserving joins [615]. This is important for XQuery and other languages that require order-preservation. XQuery specifies that the result of a query is a sequence. If no `unordered` or `order by` instruction is given, the order of the output sequence is determined by the order of the input sequences given in the `for` clauses of the query. If there are several entries in a `for` clause or several `for` clauses, order-preserving join operators [183] can be a natural component for the evaluation of such a query.

The order-preserving join operator is used in several algebras in the context of

- semi-structured data and XML (e.g. SAL [70], XAL [293]),
- OLAP [820], and


```

IDP-1( $\{R_1, \dots, R_n\}$ ,  $k$ )
Input: a set of relations to be joined, maximum block size  $k$ 
Output: a join tree
for ( $i = 1$ ;  $i \leq n$ ;  $++i$ ) {
     $\text{BestTree}(\{R_i\}) = R_i$ ;
}
 $\text{ToDo} = \{R_1, \dots, R_n\}$ ;
while ( $|\text{ToDo}| > 1$ ) {
     $k = \min(k, |\text{ToDo}|)$ ;
    for ( $i = 2$ ;  $i < k$ ;  $++i$ ) {
        for all  $S \subseteq \text{ToDo}$ ,  $|S| = i$  do {
            for all  $O \subset S$  do {
                 $\text{BestTree}(S) = \text{CreateJoinTree}(\text{BestTree}(S \setminus O), \text{BestTree}(O))$ ;
            }
        }
    }
    find  $V \subset \text{ToDo}$ ,  $|V| = k$ 
    with  $\text{cost}(\text{BestTree}(V)) = \min\{\text{cost}(\text{BestTree}(W)) \mid W \subset \text{ToDo}, |W| = k\}$ ;
    generate new symbol  $T$ ;
     $\text{BestTree}(\{T\}) = \text{BestTree}(V)$ ;
     $\text{ToDo} = (\text{ToDo} \setminus V) \cup \{T\}$ ;
    for all  $O \subset V$  do  $\text{delete}(\text{BestTree}(O))$ ;
}
return  $\text{BestTree}(\{R_1, \dots, R_n\})$ ;

```

Figure 3.23: Pseudo code for IDP-1

- time series data [544].

We give a polynomial algorithm that produces bushy trees for a sequence of order-preserving joins and selections. These trees may contain cross products even if the join graph is connected. However, we apply selections as early as possible. The algorithm then produces the optimal plan among those who push selections down. The cost function is a parameter of the algorithm, and we do not need to restrict ourselves to those having the ASI property. Further, we need no restriction on the join graph, i.e. the algorithm produces the optimal plan even if the join graph is cyclic.

Before defining the order-preserving join, we need some preliminaries. The above algebras work on sequences of sets of variable bindings, i.e. sequences of unordered tuples where every attribute corresponds to a variable. (See Chapter 7.16 for a general discussion.) Single tuples are constructed using the standard $[\cdot]$ brackets. Concatenation of tuples and functions is denoted by $\circ$. The set of attributes defined for an expression e is defined as $\mathcal{A}(e)$. The set of free variables of an expression e is defined as $\mathcal{F}(e)$. For sequences e , we use

```

IDP-2( $\{R_1, \dots, R_n\}, k$ )
Input: a set of relations to be joined, maximum block size  $k$ 
Output: a join tree
for ( $i = 1; i \leq n; ++i$ ) {
    BestTree( $\{R_i\}$ ) =  $R_i$ ;
}
ToDo =  $\{R_1, \dots, R_n\}$ ;
while ( $|\text{ToDo}| > 1$ ) {
    // apply greedy algorithm to select a good building block
     $B = \emptyset$ ;
    for all  $v \in \text{ToDo}$ , do {
         $B += \text{BestTree}(\{v\})$ ;
    }
    do {
        find  $L, R \in B$ 
        with  $\text{cost}(\text{CreateJoinTree}(L, R))$ 
            =  $\min\{\text{cost}(\text{CreateJoinTree}(L', R')) \mid L', R' \in B\}$ ;
         $P = \text{CreateJoinTree}(L, R)$ ;
         $B = (B \setminus \{L, R\}) \cup \{P\}$ ;
    } while ( $P$  involves no more than  $k$  relations and  $|B| > 1$ );
    // reoptimize the bigger of  $L$  and  $R$ ,
    // selected in the last iteration of the greedy loop
    if ( $L$  involves more tables than  $R$ ) {
        ReOpRels = relations involved in  $L$ ;
    } else {
        ReOpRels = relations involved in  $R$ ;
    }
     $P = \text{DP-Bushy}(\text{ReOpRels})$ ;
    generate new symbol  $T$ ;
    BestTree( $\{T\}$ ) =  $P$ ;
    ToDo =  $(\text{ToDo} \setminus \text{ReOpRels}) \cup \{T\}$ ;
    for all  $O \subset V$  do delete(BestTree( $O$ ));
}
return BestTree( $\{R_1, \dots, R_n\}$ );

```

Figure 3.24: Pseudocode for IDP-2

$\alpha(e)$ to denote the first element of a sequence. We identify single element sequences with elements. The function τ retrieves the tail of a sequence, and $\oplus$ concatenates two sequences. We denote the empty sequence by ϵ .

We define the algebraic operators recursively on their input sequences. The order-preserving join operator is defined as the concatenation of an order-preserving selection and an order-preserving cross product. For unary operators, if the input sequence is empty, the output sequence is also empty. For

binary operators, the output sequence is empty whenever the left operand represents an empty sequence.

The order-preserving join operator is based on the definition of an order-preserving cross product operator defined as

$$e_1 \hat{\times} e_2 := (\alpha(e_1) \hat{\times} e_2) \oplus (\tau(e_1) \hat{\times} e_2)$$

where

$$e_1 \hat{\times} e_2 := \begin{cases} \epsilon & \text{if } e_2 = \epsilon \\ (e_1 \circ \alpha(e_2)) \oplus (e_1 \hat{\times} \tau(e_2)) & \text{else} \end{cases}$$

We are now prepared to define the join operation on ordered sequences:

$$e_1 \hat{\bowtie}_p e_2 := \hat{\sigma}_p(e_1 \hat{\times} e_2)$$

where the order-preserving selection is defined as

$$\hat{\sigma}_p(e) := \begin{cases} \epsilon & \text{if } e = \epsilon \\ \alpha(e) \oplus \hat{\sigma}_p(\tau(e)) & \text{if } p(\alpha(e)) \\ \hat{\sigma}_p(\tau(e)) & \text{else} \end{cases}$$

As usual, selections can be reordered and pushed inside order-preserving joins. Besides, the latter are associative. The following equivalences formalize this.

$$\begin{aligned} \hat{\sigma}_{p_1}(\hat{\sigma}_{p_2}(e)) &= \hat{\sigma}_{p_2}(\hat{\sigma}_{p_1}(e)) \\ \hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2} e_2) &= \hat{\sigma}_{p_1}(e_1) \hat{\bowtie}_{p_2} e_2 & \text{if } \mathcal{F}(p_1) \subseteq \mathcal{A}(e_1) \\ \hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2} e_2) &= e_1 \hat{\bowtie}_{p_2} \hat{\sigma}_{p_1}(e_2) & \text{if } \mathcal{F}(p_1) \subseteq \mathcal{A}(e_2) \\ e_1 \hat{\bowtie}_{p_1}(e_2 \hat{\bowtie}_{p_2} e_3) &= (e_1 \hat{\bowtie}_{p_1} e_2) \hat{\bowtie}_{p_2} e_3 & \text{if } \mathcal{F}(p_i) \subseteq \mathcal{A}(e_i) \cup \mathcal{A}(e_{i+1}) \end{aligned}$$

While being associative, the order-preserving join is not commutative, as the following example illustrates. Given two tuple sequences $R_1 = \langle [a : 1], [a : 2] \rangle$ and $R_2 = \langle [b : 1], [b : 2] \rangle$, we have

$$\begin{aligned} R_1 \hat{\bowtie}_{true} R_2 &= \langle [a : 1, b : 1], [a : 1, b : 2], [a : 2, b : 1], [a : 2, b : 2] \rangle \\ R_2 \hat{\bowtie}_{true} R_1 &= \langle [a : 1, b : 1], [a : 2, b : 1], [a : 1, b : 2], [a : 2, b : 2] \rangle \end{aligned}$$

Before introducing the algorithm, let us have a look at the size of the search space. Since the order-preserving join is associative but not commutative, the input to the algorithm must be a sequence of join operators or, likewise, a sequence of relations to be joined. The output is then a fully parenthesized expression. Given a sequence of n binary associative but not commutative operators, the number of fully parenthesized expressions is (see [208])

$$P(n) = \begin{cases} 1 & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n > 1 \end{cases}$$

We have that $P(n) = C(n-1)$, where $C(n)$ are the Catalan numbers defined as $C(n) = \frac{1}{n+1} \binom{2n}{n}$. Since $C(n) = \Omega(\frac{4^n}{n^{3/2}})$, the search space is exponential in size.

```
applicable-predicates( $\mathcal{R}$ ,  $\mathcal{P}$ )
```

```
01   $\mathcal{B} = \emptyset$ 
02  foreach  $p \in \mathcal{P}$ 
03      IF ( $\mathcal{F}(p) \subseteq \mathcal{A}(\mathcal{R})$ )
04           $\mathcal{B}+ = p$ 
05  return  $\mathcal{B}$ 
```

Figure 3.25: Subroutine `applicable-predicates`

The algorithm is inspired by the dynamic programming algorithm for finding optimal parenthesized expressions for matrix-chain multiplication [208]. The differences are that we have to encapsulate the cost function and deal with selections. We give a detailed example application of the algorithm below. This example illustrates (1) the optimization potential, (2) that cross products can be favorable, (3) how to plug in a cost function into the algorithm, and (4) the algorithm itself.

The algorithm itself is broken up into several subroutines. The first is `applicable-predicates` (see Fig. 3.25). Given a sequence of relations $R_i, \dots, R_j$ and a set of predicates, it retrieves those predicates applicable to the result of the join of the relations. Since joins and selections can be reordered freely, the only condition for a predicate to be applicable is that all its free variables are bound by the given relations.

The second subroutine is the most important and intrigued. It fills several arrays with values in a bottom-up manner. The third subroutine then builds the query evaluation plan using the data in the arrays.

The subroutine `construct-bushy-tree` takes as input a sequence $R_1, \dots, R_n$ of relations to be joined and a set $\mathcal{P}$ of predicates to be applied. For every possible subsequence $R_i, \dots, R_j$, the algorithm finds the best plan to join these relations. Therefore, it determines some k such that the cheapest plan joins the intermediate results for $R_i, \dots, R_k$ and $R_{k+1}, \dots, R_j$ by its topmost join. For this it is assumed that for all k the best plans for joining $R_i, \dots, R_k$ and $R_{k+1}, \dots, R_j$ are known. Instead of directly storing the best plan, we remember (1) the costs of the best plan for $R_i, \dots, R_j$ for all $1 \leq i \leq j \leq n$ and (2) the k where the split takes place. More specifically, the array $c[i, j]$ contains the costs of the best plan for joining $R_i, \dots, R_j$, and the array $t[i, j]$ contains the k such that this best plan joins $R_i, \dots, R_k$ and $R_{k+1}, \dots, R_j$ with its topmost join. For every sequence $R_i, \dots, R_j$, we also remember the set of predicates that can be applied to it, excluding those that have been applied earlier. These applicable predicates are contained in $p[i, j]$. Still, we are not done. All cost functions we know use some kind of statistics on the argument relation(s) in order to compute the costs of some operation. Since we want to be generic with respect to the cost function, we encapsulate the computation of statistics and costs within functions S_0 , C_0 , S_1 , and C_1 . The function S_0 retrieves statistics for base relations. The function C_0 computes the costs of retrieving (part of) a base relation. Both functions take a set of applicable predicates as an additional

```

construct-bushy-tree( $\mathcal{R}$ ,  $\mathcal{P}$ )

01   $n = |\mathcal{R}|$ 
02  for  $i = 1$  to  $n$ 
03       $\mathcal{B} = \text{applicable-predicates}(R_i, \mathcal{P})$ 
04       $\mathcal{P} = \mathcal{P} \setminus \mathcal{B}$ 
05       $p[i, i] = \mathcal{B}$ 
06       $s[i, i] = S_0(R_i, \mathcal{B})$ 
07       $c[i, i] = C_0(R_i, \mathcal{B})$ 
08  for  $l = 2$  to  $n$ 
09      for  $i = 1$  to  $n - l + 1$ 
10           $j = i + l - 1$ 
11           $\mathcal{B} = \text{applicable-predicates}(R_{i\dots j}, \mathcal{P})$ 
12           $\mathcal{P} = \mathcal{P} \setminus \mathcal{B}$ 
13           $p[i, j] = \mathcal{B}$ 
14           $s[i, j] = S_1(s[i, j-1], s[j, j], \mathcal{B})$ 
15           $c[i, j] = \infty$ 
16          for  $k = i$  to  $j - 1$ 
17               $q = c[i, k] + c[k+1, j] + C_1(s[i, k], s[k+1, j], \mathcal{B})$ 
18              IF ( $q < c[i, j]$ )
19                   $c[i, j] = q$ 
20                   $t[i, j] = k$ 

```

Figure 3.26: Subroutine `construct-bushy-tree`

```

extract-plan( $\mathcal{R}, t, p$ )

01  return extract-subplan( $\mathcal{R}, t, p, 1, |\mathcal{R}|$ )

extract-subplan( $\mathcal{R}, t, p, i, j$ )

01  IF ( $j > i$ )
02       $X = \text{extract-subplan}(\mathcal{R}, t, p, i, t[i, j])$ 
03       $Y = \text{extract-subplan}(\mathcal{R}, t, p, t[i, j] + 1, j)$ 
04      return  $X \bowtie_{p[i, j]} Y$ 
05  else
06      return  $\hat{\sigma}_{p[i, i]}(R_i)$ 

```

Figure 3.27: Subroutine `extract-plan` and its subroutine

argument. The function S_1 computes the statistics for intermediate relations. Since the result of joining some relations $R_i, \dots, R_j$ may occur in many different plans, we compute it only once and store it in the array s . C_1 computes the costs of joining two relations and applying a set of predicates. Below, we show how concrete (simple) cost and statistics functions can look like.

Given the above, the algorithm (see Fig. 3.26) fills the arrays in a bottom-up manner by first computing for every base relation the applicable predicates, the

statistics of the result of applying the predicates to the base relation and the costs for computing these intermediate results, i.e. for retrieving the relevant part of the base relation and applying the predicates (lines 02-07). Note that this is not really trivial if there are several index structures that can be applied. Then computing C_0 involves considering different access paths. Since this is an issue orthogonal to join ordering, we do not detail on it.

After we have the costs and statistics for sequences of length one, we compute the same information for sequences of length two, three, and so on until n (loop starting at line 08). For every length, we iterate over all subsequences of that length (loop starting at line 09). We compute the applicable predicates and the statistics. In order to determine the minimal costs, we have to consider every possible split point. This is done by iterating the split point k from i to $j - 1$ (line 16). For every k , we compute the cost and remember the k that resulted in the lowest costs (lines 17-20).

The last subroutine takes the relations, the split points (t), and the applicable predicates (p) as its input and extracts the plan. The whole plan is extracted by calling **extract-plan**. This is done by instructing **extract-subplan** to retrieve the plan for all relations. This subroutine first determines whether the plan for a base relation or that of an intermediate result is to be constructed. In both cases, we did a little cheating here to keep things simple. The plan we construct for base relations does not take the above-mentioned index structures into account but simply applies a selection to a base relation instead. Obviously, this can easily be corrected. We also give the join operator the whole set of predicates that can be applied. That is, we do not distinguish between join predicates and other predicates that are better suited for a selection subsequently applied to a join. Again, this can easily be corrected.

Let us have a quick look at the complexity of the algorithm. Given n relations with m attributes in total and p predicates, we can implement **applicable-predicates** in $O(pm)$ by using a bit vector representation for attributes and free variables and computing the attributes for each sequence $R_i, \dots, R_j$ once upfront. The latter takes $O(n^2m)$.

The complexity of the routine **construct-bushy-tree** is determined by the three nested loops. We assume that S_1 and C_1 can be computed in $O(p)$, which is quite reasonable. Then, we have $O(n^3p)$ for the innermost loop, $O(n^2)$ calls to **applicable-predicates**, which amounts to $O(n^2pm)$, and $O(n^2p)$ for calls of S_1 . Extracting the plan is linear in n . Hence, the total runtime of the algorithm is $O(n^2(n + m)p)$.

In order to illustrate the algorithm, we need to fix the functions S_0 , S_1 , C_0 and C_1 . We use the simple cost function C_{out} . As a consequence, the array s simply stores cardinalities, and S_0 has to extract the cardinality of a given base relation and multiply it by the selectivities of the applicable predicates. S_1 multiplies the input cardinalities with the selectivities of the applicable predicates. We set C_0 to zero and C_1 to S_1 . The former is justified by the fact that every relation must be accessed exactly once and hence, the access costs are equal in

all plans. Summarizing, we define

$$\begin{aligned}
S_0(R, \mathcal{B}) &:= |R| \prod_{p \in \mathcal{B}} f(p) \\
S_1(x, y, \mathcal{B}) &:= xy \prod_{p \in \mathcal{B}} f(p) \\
C_0(R, \mathcal{B}) &:= 0 \\
C_1(x, y, \mathcal{B}) &:= S_1(x, y, \mathcal{B})
\end{aligned}$$

where $\mathcal{B}$ is a set of applicable predicates and for a single predicate p , $f(p)$ returns its selectivity.

We illustrate the algorithm by an example consisting of four relations $R_1, \dots, R_4$ with cardinalities $|R_1| = 200$, $|R_2| = 1$, $|R_3| = 1$, $|R_4| = 20$. Besides, we have three predicates $p_{i,j}$ with $\mathcal{F}(p_{i,j}) \subseteq \mathcal{A}(R_i) \cup \mathcal{A}(R_j)$. They are $p_{1,2}$, $p_{3,4}$, and $p_{1,4}$ with selectivities $1/2$, $1/10$, $1/5$.

Let us first consider an example plan and its costs. The plan

$$((R_1 \hat{\bowtie}_{p_{1,2}} R_2) \hat{\bowtie}_{true} R_3) \hat{\bowtie}_{p_{1,4} \wedge p_{3,4}} R_4$$

has the costs $240 = 100 + 100 + 40$.

For our simple cost function, the algorithm **construct-bushy-tree** will fill the array s with the initial values:

s			
200			
	1		
		1	
			20

After initialization, the array c has 0 everywhere in its diagonal and the array p empty sets.

For $l = 2$, the algorithm produces the following values:

l	i	j	k	s[i,j]	q	current c[i,j]	current t[i,j]
2	1	2	1	100	100	100	1
2	2	3	2	1	1	1	2
2	3	4	3	2	2	2	3

For $l = 3$, the algorithm produces the following values:

l	i	j	k	s[i,j]	q	current c[i,j]	current t[i,j]
3	1	3	1	200	101	101	1
3	1	3	2	200	200	101	1
3	2	4	2	2	4	4	2
3	2	4	3	2	3	3	3

For $l = 4$, the algorithm produces the following values:

l	i	j	k	s[1,4]	q	current c[1,4]	current t[1,4]
4	1	4	1	40	43	43	1
4	1	4	2	40	142	43	1
4	1	4	3	40	141	43	1

where for each k the value of q (in the following table denoted by q_k) is determined as follows:

$$\begin{aligned}
q_1 &= c[1,1] + c[2,4] + 40 = 0 + 3 + 40 = 43 \\
q_2 &= c[1,2] + c[3,4] + 40 = 100 + 2 + 40 = 142 \\
q_3 &= c[1,3] + c[4,4] + 40 = 101 + 0 + 40 = 141
\end{aligned}$$

Collecting all the above $t[i, j]$ values leaves us with the following array as input for **extract-plan**:

$i \setminus j$	1	2	3	4
1		1	1	1
2			2	3
3				3
4				

The function **extract-plan** merely calls **extract-subplan**. For the latter, we give the call hierarchy and the result produced:

```

000  extract-plan(..., 1, 4)
100    extract-plan(..., 1, 1)
200    extract-plan(..., 2, 4)
210      extract-plan(..., 2, 3)
211        extract-plan(..., 2, 2)
212        extract-plan(..., 3, 3)
210      return  $(R_2 \hat{\bowtie}_{\text{true}} R_3)$ 
220    extract-plan(..., 4, 4)
200  return  $((R_2 \hat{\bowtie}_{\text{true}} R_3) \hat{\bowtie}_{p_{3,4}} R_4)$ 
000  return  $(R_1 \hat{\bowtie}_{p_{1,2} \wedge p_{1,4}} ((R_2 \hat{\bowtie}_{\text{true}} R_3) \hat{\bowtie}_{p_{3,4}} R_4))$ 

```

The total cost of this plan is $c[1, 4] = 43$.

3.6 Characterizing Search Spaces

3.6.1 Complexity Thresholds

The complexity results presented in Section 3.1.6 show that most classes of join ordering problems are NP-hard. However, it is quite clear that some instances of the join ordering problem are simpler than others. For example, consider a query graph which is a clique in n relations $R_1, \dots, R_n$. Further assume that each R_i has cardinality 2^i and all join selectivities are $1/2$ (i.e. $f_{i,j} = 1/2$ for all $1 \leq i, j \leq n$, $i \neq j$). Obviously, this problem is easy to optimize although the query graph is clique. In this section we present some ideas on how the

complexity of instances of the join ordering problem is influenced by certain parameters.

How can we judge the complexity of a single instance of a join ordering problem? Using standard complexity theory, for single problem instances we easily derive an algorithm that works in $\Theta(1)$. Hence, we must define other complexity measures. Consider our introductory join ordering problem. A simple greedy algorithm that orders relations according to their cardinality produces an optimal solution for it. Hence, one possibility to define the problem complexity would be how far a solution produced by typical heuristics for join ordering differ from the optimal solution. Another possibility is to use randomized algorithms like iterative improvement of simulated annealing and see how far the plans generated by them deviate from the optimal plan. These approaches have the problem that the results may depend on the chosen algorithm. This can be avoided by using the following approach. For each join ordering problem instance, we compute the fraction of good plans compared to all plans. Therefore, we need a measure of “good”. Typical examples thereof would be to say a plan is “good” if it does not deviate more than 10% or a factor of two from the optimal plan.

If these investigations were readily available, there are certain obvious benefits [511]:

1. The designer of an optimizer can classify queries such that heuristics are applied where they guarantee success; cases where they are bound to fail can be avoided. Furthermore, taking into account the vastly different run time of the different join ordering heuristics and probabilistic optimization procedures, the designer of an optimizer can choose the method that achieves a satisfactory result with the least effort.
2. The developer of search procedures and heuristics can use this knowledge to design methods solving hard problems (as exemplified for graph coloring problems [430]).
3. The investigator of different join ordering techniques is able to (1) consciously design challenging benchmarks and (2) evaluate existing benchmarks according to their degree of challenge.

The kind of investigation presented in this section first started in the context of artificial intelligence where a paper by Cheeseman, Kanefsky, and Taylor [162] spurred a whole new branch of research where the measures to judge the complexity of problem instances was investigated for many different NP-complete problems like satisfiability [162, 212, 326, 609], graph coloring [162], Hamiltonian circuits [162], traveling salesman [162], and constraint satisfaction [928].

We only present a small fraction of all possible investigations. The restrictions are that we do not consider all parameters that possibly influence the problem complexity, we only consider left-deep trees, and we restrict ourselves to the cost function C_{hj} . The join graphs are randomly generated. Starting with a circle, we randomly added edges until a clique is reached. The reader is advised to carry out his or her own experiments. Therefore, the following

pointer into the literature might be useful. Lanzelotte and Valduriez provide an object-oriented design for search strategies [530]. This allows easy modification and even the exchange of the plan generator's search strategy.

Search Space Analysis

The goal of this section is to determine the influence of the parameters on the search space of left-deep join trees. More specifically, we are interested in how a variation of the parameters changes the percentage of good solutions among all solutions. The quality of a solution is measured by the factor its cost deviates from the optimal permutation. For this, all permutations have to be generated and evaluated. The results of this experiment are shown in Figures 3.28 and 3.29. Each single curve accumulates the percentage of all permutations deviating less than a certain factor (given as the label) from the optimum. The accumulated percentages are given at the y-axes, the connectivity at the x-axes. The connectivity is given by the number of edges in the join graph. The curves within the figures are organized as follows. Figure 3.28 (3.29) shows varying mean selectivity values (relation sizes) and variances where the mean selectivity values (relation sizes) increase from top to bottom and the variances increase from left to right.

Note that the more curves are visible and the lower their y-values, the harder is the problem. We observe the following:

- all curves exhibit a minimum value at a certain connectivity
- which moves with increasing mean values to the right;
- increasing variances does not have an impact on the *minimum connectivity*,
- problems become less difficult with increasing mean values.

These findings can be explained as follows. With increasing connectivity, the join ordering problem becomes more complex up to a certain point and then less complex again. To see this, consider the following special though illustrative case. Assume an almost equal distribution of the costs of all alternatives between the worst case and optimal costs, equal relation sizes, and equal selectivities. Then the optimization potential *worst case/optimum* is 1 for connectivity 0 and cliques. In between, there exists a connectivity exhibiting the maximum optimization potential. This connectivity corresponds to the minimum connectivity of Figures 3.28 and 3.29.

There is another factor which influences the complexity of a single problem instance. Consider joining n relations. The problem becomes less complex if after joining $i < n$ relations the intermediate result becomes so small that the accumulated costs of the subsequent $n - i$ joins are small compared to the costs of joining the first i relations. Hence, the ordering of the remaining $n - i$ relations does not have a big influence on the total costs. This is the case for very small relations, small selectivities, or high connectivities. The greater selectivities and relation sizes are, the more relations have to be joined to reach this critical size of the intermediate result. If the connectivity is enlarged, this

critical size is reached earlier. Since the number of selectivities involved in the first few joins is small regardless of the connectivity, there is a lower limit to the number of joined relations required to arrive at the critical intermediate result size. If the connectivity is larger, this point is reached earlier, but there exists a lower limit on the connectivity where this point is reached. The reason for this lower limit is that the number of selectivities involved in the joins remains small for the first couple of relations, independent of their connectivity. These lines of argument explain subsequent findings, too.

The reader should be aware of the fact that the number of relations joined is quite small (10) in our experiments. Further, as observed by several researchers, if the number of joins increases, the number of “good” plans decreases [302, 858]. That is, increasing the number of relations makes the join ordering problem more difficult.

Figure 3.28: Impact of selectivity on the search space

Figure 3.29: Impact of relation sizes on the search space

Heuristics

For analyzing the influence of the parameters on the performance of heuristics, we give the figures for four different heuristics. The first two are very simple. The *minSel* heuristic selects those relations first of which incident join edges exhibit the minimal selectivity. The *recMinRel* heuristic chooses those relations first which result in the smallest intermediate relation.

We also analyzed the two advanced heuristics *IKKBZ* and *RDC*. The *IKKBZ* heuristic [520] is based on an optimal join ordering procedure [438, 520] which is applied to the minimal spanning tree of the join graph where the edges are labeled by the selectivities. The family of *RDC* heuristics is based on the relational difference calculus as developed in [418]. Since our goal is not to benchmark different heuristics in order to determine the best one, we have chosen the simplest variant of the family of *RDC* based heuristics. Here, the relations are ordered according to a certain weight whose actual computation is—for the purpose of this section—of no interest. The results of the experiments are presented in Figure 3.30.

On a first glance, these figures look less regular than those presented so far. This might be due to the non-stable behavior of the heuristics. Nevertheless, we can extract the following observations. Many curves exhibit a peak at a certain connectivity. Here, the heuristics perform worst. The peak connectivity is dependent on the selectivity size but not as regular as in the previous curves. Further, higher selectivities flatten the curves, that is, heuristics perform better at higher selectivities.

Figure 3.30: Impact of parameters on the performance of heuristics

Probabilistic Optimization Procedures

Figure 3.31 shows four pictures corresponding to simulated annealing (SA), iterative improvement (II), iterative improvement applied to the outcome of the IKKBZ heuristic (IKKBZ/II) and the RDC heuristic (RDC/II) [418]. The patterns shown in Figure 3.31 are very regular. All curves exhibit a peak at a certain connectivity. The peak connectivities typically coincide with the minimum connectivity of the search space analysis. Higher selectivities result in flatter curves; the probabilistic procedures perform better. These findings are absolutely coherent with the search space analysis. This is not surprising, since the probabilistic procedures investigate systematically —although with some random influence— a certain part of the search space.

Given a join ordering problem, we can describe its potential search space as a graph. The set of nodes consists of the set of join trees. For every two join trees a and b , we add an edge (a, b) if b can be reached from a by one of the transformation rules used in the probabilistic procedure. Further, with every node we can associate the cost its corresponding join tree.

Having in mind that the probabilistic algorithms are always in danger of being stuck in a local minima, the following two properties of the search space are of interest:

1. the cost distribution of local minima, and
2. the connection cost of low local minima.

Of course, if all local minima are of about the same cost, we do not have to worry, otherwise we do. It would be very interesting to know the percentage of local minima that are close to the global minima.

Concerning the second property, we first have to define the connection cost. Let a and b be two nodes and P be the set of all paths from a to b . The *connection cost* of a and b is then defined as $\min_{p \in P} \max_{s \in p} \{cost(s) | s \neq a, s \neq b\}$. Now, if the connection costs are high, we know that if we have to travel from one local minima to another, there is at least one node we have to pass which has high costs. Obviously, this is bad for our probabilistic procedures. Ioannidis and Kang [453] call a search graph that is favorable with respect to the two properties a *well*. Unfortunately, investigating these two properties of real search spaces is rather difficult. However, Ioannidis and Kang, later supported by Zhang, succeeded in characterizing cost wells in random graphs [453, 454]. They also conclude that the search space comprising bushy trees is better w.r.t. our two properties than the one for left-deep trees.

Figure 3.31: Impact of selectivities on probabilistic procedures

3.7 Discussion

Choose one of dynamic programming, memoization, permutations as the core of your plan generation algorithm and extend it with the rest of book.

ToDo

3.8 Bibliography

ToDo: Oezsu, Meechan [663, 664]

Chapter 4

Database Items, Building Blocks, and Access Paths

In this chapter we go down to the storage layer and discuss leaf nodes of query execution plans and plan fragments. We briefly recap some notions, but reading a book on database implementation might be helpful [403, 316]. Although alternative storage technologies exist and are being developed [764], databases are mostly stored on disks. Thus, we start out by introducing a simple disk model to capture I/O costs. Then, we say some words about database buffers, physical data organization, slotted pages and tuple identifiers (TIDs), physical record layout, physical algebra, and the iterator concept. These are the basic notions in order to start with the main purpose of this section: giving an overview over the possibilities available to structure the low level parts of a physical query evaluation plan. In order to calculate the I/O costs of these plan fragments, a more sophisticated cost model for several kinds of disk accesses is introduced.

4.1 Disk Drive

Figure 4.1 shows a top and a side view of a typical disk. A disk consists of several platters that rotate around the spindle at a fixed speed. The platters are coated with a magnetic material on at least one of their surfaces. All coated sides are organized into the same pattern of concentric circles. One concentric circle is called a track. All the tracks residing exactly underneath and above each other form a cylinder. We assume that there is only one read/write head for every coated surface.¹ All tracks of a cylinder can be accessed with only minor adjustments at the same time by their respective heads. By moving the arm around the arm pivot, other cylinders can be accessed. Each track is partitioned into sectors. Sectors have a disk specific (almost) fixed capacity of 512 B. The read and write granularity is a sector. Read and write accesses take place while the sector passes under the head.

The top view of Figure 4.1 shows that the outer sectors are longer than the

¹This assumption is valid for most but not all disks.

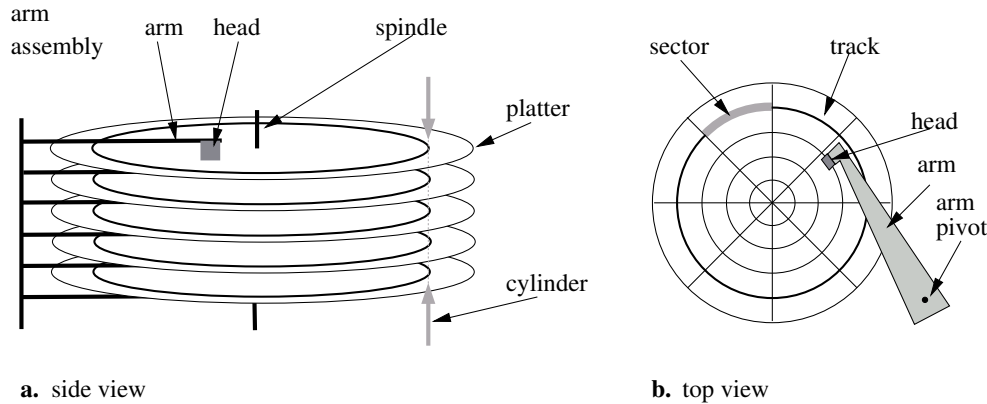


Figure 4.1: Disk drive assembly

inner sectors. The highest density (e.g. in bits per centimeter) at which bits can be separated is fixed for a given disk. For storing 512 B, this results in a minimum sector length which is used for the tracks of the innermost cylinder. Thus, since sectors on outer tracks are longer, storage capacity is wasted there. To overcome this problem, disks have a varying number of sectors per track. (This is where the picture lies.) Therefore, the cylinders are organized into zones. Every zone contains a fixed number of consecutive cylinders, each having a fixed number of sectors per track. Between zones, the number of sectors per track varies. Outer zones have more sectors per track than inner zones. Since the platters rotate with a fixed angular speed, sectors of outer cylinders can be read faster than sectors of inner cylinders. As a consequence, the throughput for reading and writing outer cylinders is higher than for inner cylinders.

Assume that we sequentially read all the sectors of all tracks of some consecutive cylinders. After reading all sectors of some track, we must proceed to the next track. If it is contained in the same cylinder, then we must (simply) use another head: a *head switch* occurs. Due to calibration, this takes some time. Thus, if all sectors start at the same angular position, we come too late to read the first sector of the next track and have to wait. To avoid this, the angular start positions of the sectors of tracks in the same cylinder are skewed such that this *track skew* compensates for the head switch time. If the next track is contained in another cylinder, the heads have to switch to the next cylinder. Again, this takes time and we miss the first sector if all sectors of a surface start at the same angular positions. *Cylinder skew* is used such that the time needed for this switch does not make us miss to start reading the next sector. In general, skewing works in only one direction.

A sector can be addressed by a triple containing its cylinder, head (surface), and sector number. This triple is called the physical address of a sector. However, disks are accessed using logical addresses. These are called *logical block numbers* (LBN) and are consecutive numbers starting with zero. The disk internally maps LBNs to physical addresses. This mapping is captured in the following table:

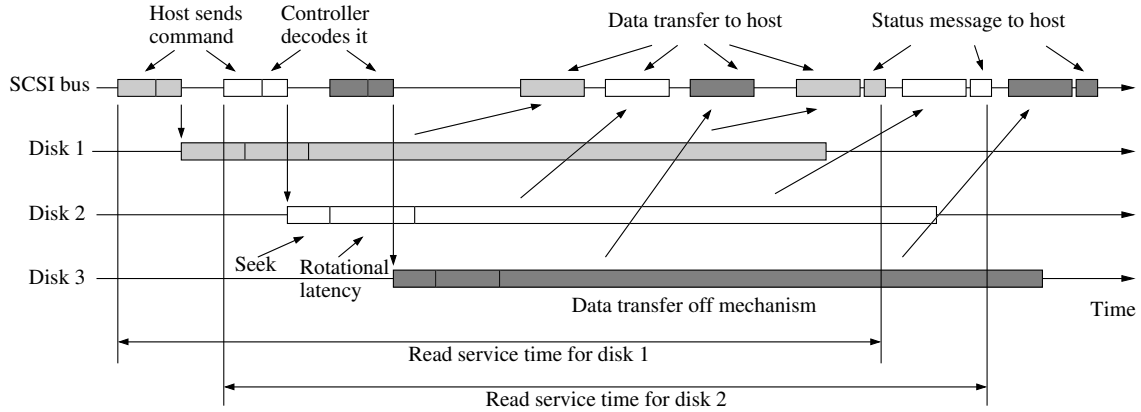


Figure 4.2: Disk drive read request processing

cylinder	track	LBN	number of sectors per track
0	0	0	573
	1	573	573
...	...	...	...
	5	2865	573
1	0	3438	573
...	...	...	...
15041	0	35841845	253
...	...	...	...

However, this ideal view is disturbed by the phenomenon of bad blocks. A *bad block* is one with a defect and it cannot be read or written. After a block with a certain LBN is detected to be bad, it is assigned to another sector. The above mapping changes. In order to be able redirect LBNs, extra space on the disk must exist. Hence, some cylinders, tracks, and sectors are reserved for this purpose. They may be scattered all over the platters. Redirected blocks cause hiccups during sequential read.

Building (see e.g. [646]) and modeling (see e.g. [586, 750, 811, 812, 880, 926]) disk drives is challenging. Whereas the former is not really important when building query compiler, the latter is, as we have to attach costs to query evaluation plans. These costs reflect the amount of time we occupy the resource disk. Since disks are relatively slow, they may become the bottleneck of a database server. Modeling and minimizing disk access (time) is thus an important topic. Consider the case where we want to read a block from a SCSI disk. Simplified, the following actions take place and take their time (see also Fig. 4.2):

1. The host sends the SCSI command.
2. The disk controller decodes the command and calculates the physical address.

3. During the seek the disk drive's arm is positioned such that the according head is correctly placed over the cylinder where the requested block resides. This step consists of several phases.
 - (a) The disk controller accelerates the arm.
 - (b) For long seeks, the arm moves with maximum velocity (coast).
 - (c) The disk controller slows down the arm.
 - (d) The disk arm settles for the desired location. The settle times differ for read and write requests. For reads, an aggressive strategy is used. If, after all, it turns out that the block could not be read correctly, we can just discard it. For writing, a more conservative strategy is in order.
4. The disk has to wait until the sector where the requested block resides comes under the head (rotation latency).
5. The disk reads the sector and transfers data to the host.
6. Finally, it sends a status message.

Note that the transfers for different read requests are interleaved. This is possible since the capacity of the SCSI bus is higher than the read throughput of the disk. Also note that we did not mention the operating system delay and congestions on the SCSI bus.

Disk drives apply several strategies to accelerate the above-mentioned round-trip time and access patterns like sequential read. Among them are caching, read-ahead, and command queuing. (discuss interleaving?)

ToDo

The seek and rotation latency times highly depend on the head's position on the platter surface. Let us consider seek time. A good approximation of the seek time where d cylinders have to be travelled is given by

$$seektime(d) = \begin{cases} c_1 + c_2\sqrt{d} & d \leq c_0 \\ c_3 + c_4d & d > c_0 \end{cases}$$

where the constants c_i are disk-specific. The constant c_0 indicates the maximum number of cylinders where no coast takes place: seeking over a distance of more than c_0 cylinders results in a phase where the disk arm moves with maximum velocity.

For disk accesses, the database system must be able to estimate the time they take to be executed. First of all, we need the parameters of the disk. It is not too easy to get hold of them, but we can make use of several tools to extract them from a given disk [243, 311, 866, 771, 938, 939]. However, then we have a big problem: when calculating I/O costs, the query compiler has no idea where the head will be when the query evaluation plan emits a certain read (or write) command. Thus, we have to find another solution. In the following, we will discuss a rather simplistic cost model that will serve us to get a feeling for disk behavior. Later, we develop a more realistic model (Section 4.17).

The solution is rather trivial: we sum up all command sending and interpreting times as well the times for positioning (seek and rotation latency) which

form by far the major part. Let us call the result *latency time*. Then, we assume an average latency time. This, of course, may result in large errors for a single request. However, on average, the error can be as “low” as 35% [750]. The next parameter is the *sustained read rate*. The disk is assumed to be able to deliver a certain amount of bytes per second while reading data stored consecutively. Of course, considering multi-zone disks, we know that this is oversimplified, but we are still in our simplistic model. Analogously, we have a sustained write rate. For simplicity, we will assume that this is the same as the sustained read rate. Last, the capacity is of some interest. A hypothetical disk (inspired by disks available in 2004) then has the following parameters:

Model 2004		
Parameter	Value	Abbreviated Name
capacity	180 GB	D_{cap}
average latency time	5 ms	D_{lat}
sustained read rate	100 MB/s	D_{srr}
sustained write rate	100 MB/s	D_{swr}

The time a disk needs to read and transfer n bytes is then approximated by $D_{\text{lat}} + n/D_{\text{srr}}$. Again, this is overly simplistic: (1) due to head switches and cylinder switches, long reads have lower throughput than short reads and (2) multiple zones are not modelled correctly. However, let us use this very simplistic model to get some feeling for disk costs.

Database management system developers distinguish between *sequential* I/O and *random* I/O. For sequential I/O, there is only one positioning at the beginning and then, we can assume that data is read with the sustained read rate. For random I/O, one positioning for every unit of transfer—typically a page of say 8 KB—is assumed. Let us illustrate the effect of positioning by a small example. Assume that we want to read 100 MB of data stored consecutively on a disk. Sequential read takes 5 ms plus 1 s. If we read in blocks of 8 KB where each block requires positioning then reading 100 MB takes 65 s.

Assume that we have a relation of about 100 MB in size, stored on a disk, and we want to read it. Does it take 1 s or 65 s? If the blocks on which it is stored are randomly scattered on disk and we access them in a random order, 65 s is a good approximation. So let us assume that it is stored on consecutive blocks. Assume that we read in chunks of 8 KB. Then,

- other applications,
- other transactions, and
- other read operations of the same query evaluation plan

could move the head away from our reading position. (Congestion on the SCSI bus may also be problem.) Again, we could be left with 65 s. Reading the whole relation with one read request is a possibility but may pose problems to the buffer manager. Fortunately, we can read in chunks much smaller than 100 MB. Consider Figure 4.3. If we read in chunks of 100 8 KB blocks we are already pretty close to one second (within a factor of two).

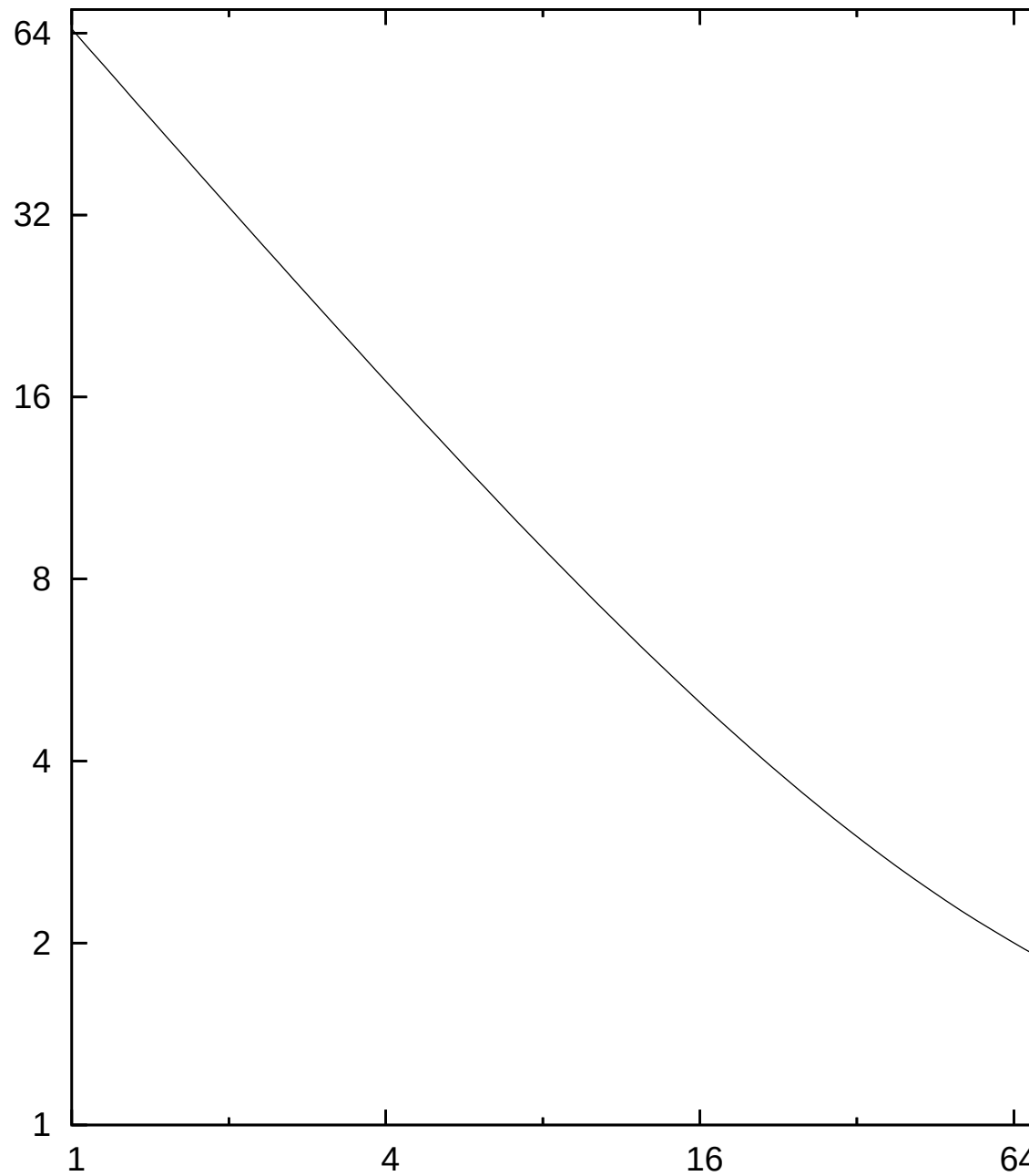


Figure 4.3: Time to read 100 MB from disk (depending on the number of 8 KB blocks read at once)

Note that the interleaving of actions does not necessarily mean a negative impact. This depends on the point of view, i.e. what we want to optimize. If we want to optimize response time for a single query, then obviously the impact of concurrent actions is negative. If, however, we want to optimize resource (here: disk) usage, concurrent actions might help.

ToDo?

There are two important things to learn here. First, sequential read is much faster than random read. Second, the runtime system should secure sequential read. The latter point can be generalized: the runtime system of a database management system has, as far as query execution is concerned, two equally important tasks:

- allow for efficient query evaluation plans and
- allow for smooth, simple, and robust cost functions.

Typical measures on the database side are

- carefully chosen physical layout on disk
(e.g. cylinder or track-aligned extents [772, 773, 770], clustering),
- disk scheduling, multi-page requests
[228, 458, 781, 782, 789, 807, 838, 930, 937],
- (asynchronous) prefetching,
- piggy-back scans,
- buffering (e.g. multiple buffers, replacement strategy from [71] to [600]),
and last but not least
- efficient and robust algorithms for algebraic operators [347].

Let us take yet another look at it. 100 MB can be stored on 12800 8 KB pages. Figure 4.4 shows the time to read n random pages. In our simplistic cost model, reading 200 pages randomly costs about the same as reading 100 MB sequentially. That is, reading 1/64th of 100 MB randomly takes as long as reading the 100 MB sequentially. Let us denote by a the positioning time, s the sustained read rate, p the page size, and d some amount of consecutively stored bytes. Let us calculate the break-even point

$$\begin{aligned} n * (a + p/s) &= a + d/s \\ n &= (a + d/s)/(a + p/s) \\ &= (as + d)/(as + p) \end{aligned}$$

a and s are disk parameters and, hence, fixed. For a fixed d , the break-even point depends on the page size. This is illustrated in Figure 4.5. The x-axis is the page size p in multiples of 1 K and the y-axis is $(d/p)/n$ for $d = 100$ MB.

For sequential reads, the page size does not matter. (Be aware that our simplistic model heavily underestimates sequential reads.) For random reads,

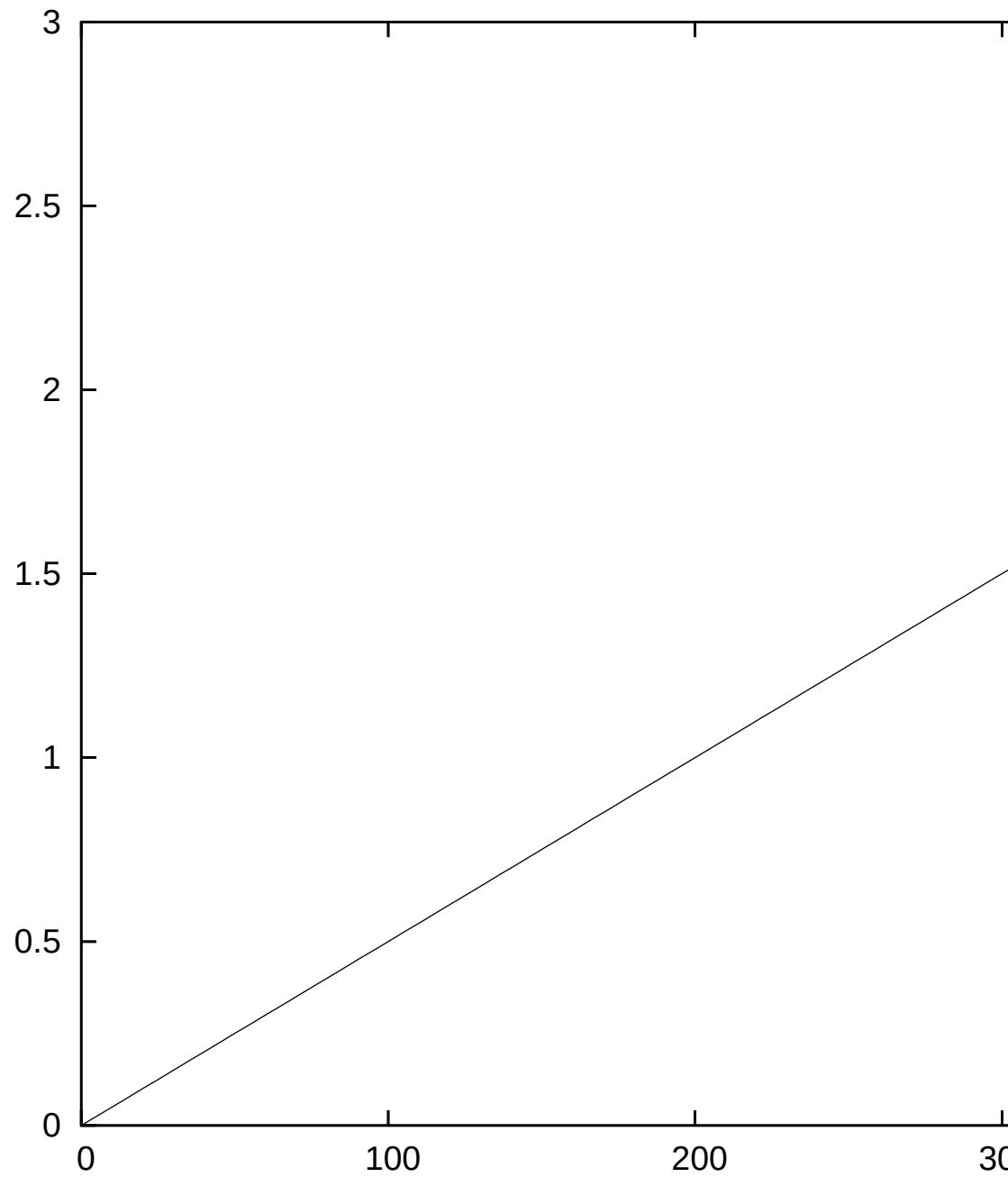


Figure 4.4: Time needed to read n random pages

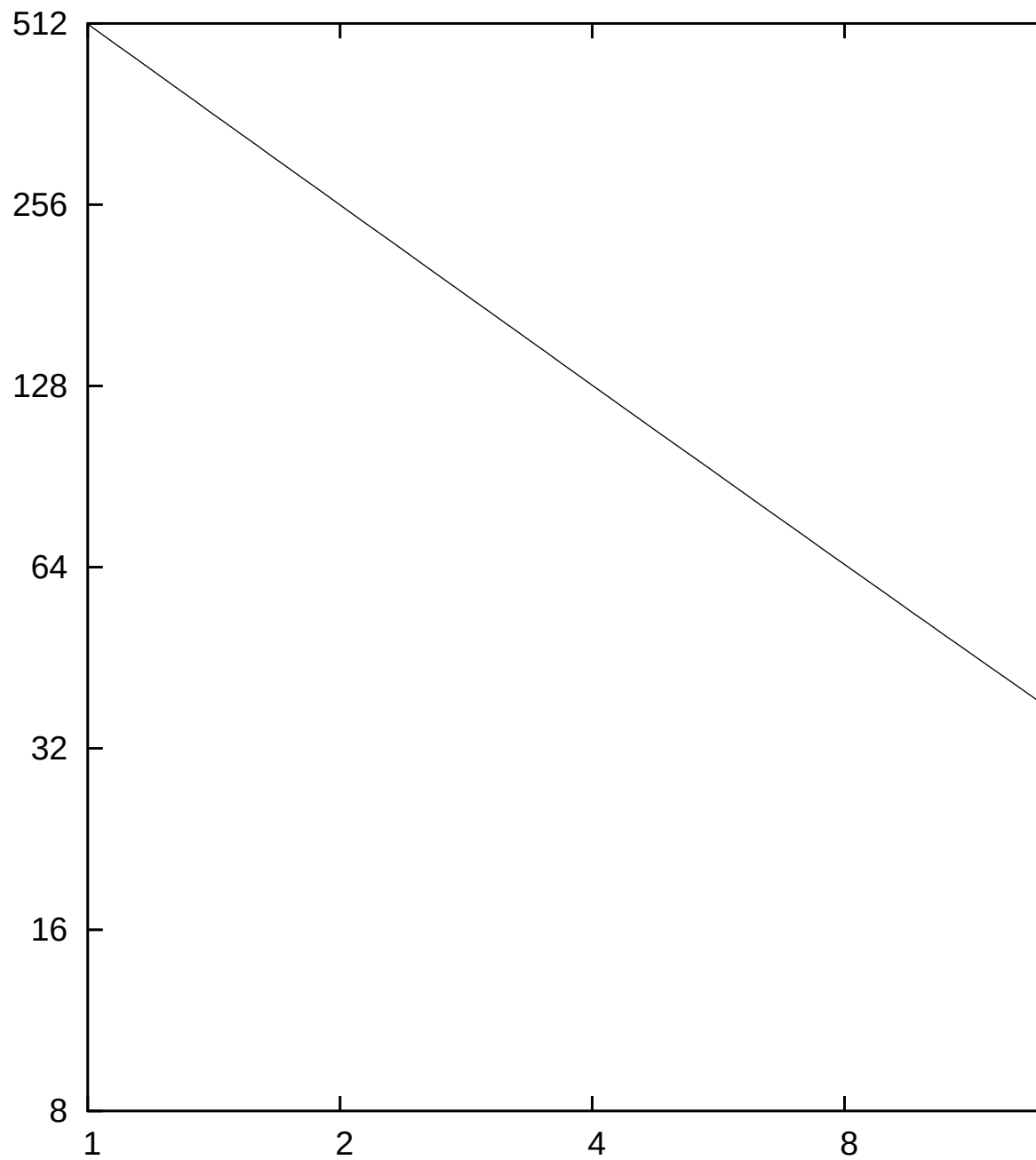


Figure 4.5: Break-even point in fraction of total pages depending on page size

as long as a single page is read, it matters neither: reading a single page of 1 KB lasts 5.0097656 ms, for an 8 KB page the number is 5.0781250 ms. From all this, we could draw the conclusion that the larger the page the better. However, this is only true for the disk, not, e.g., for the buffer or the SCSI bus. If we need to access only 500 B of a page, then the larger the page the higher the fraction that is wasted. This is not as severe as it sounds. Other queries or transactions might need other parts of the page during a single stay in the buffer. Let us call the fraction of the page that is read by some transaction during a stay in the buffer by utilization. Obviously, the higher the utilization the better is our usage of the main memory in which the buffer resides. For smaller pages, the utilization is typically higher than for larger pages. The frequency by which pages are used is another factor. [367, 368].

Excursion. Consider the root page of a B-tree. It is accessed quite frequently and most of its parts will be used, no matter how large it is. Hence, utilization is always good. Thus, the larger the root page of a B-tree the better. On the other hand, consider a leaf page of a B-tree that is much bigger than main memory. During a single stay of it, only a small fraction of the page will be used. That is, smaller leaf pages are typically better. By converting everything to money instead of time, Gray and Graefe [367] as well as Lomet [568] come to the conclusion that a page size between 8 and 16 KB was a good choice at the end of the last century.

For the less simplistic model of disk access costs developed in Section 4.17, we need to describe a disk drive by a set of parameters. These parameters are summarized in Table 4.1.

Let us close this section by giving upper bounds on seek time and rotational latency. Qyang proved the following theorem which gives a tight upper bound of disk seek time if several cylinders of a consecutive range of cylinders have to be visited [705].

Theorem 4.1.1 (Qyang) *If the disk arm has to travel over a region of C cylinders, it is positioned on the first of the C cylinders and has to stop at $s - 1$ of them, then $sD_{seek}(C/s)$ is an upper bound for the seek time.*

The time required for s consecutive sectors in a track of zone i to pass by the head is

$$D_{\text{rot}}(s, i) = sD_{\text{Zscan}}(i) = s \frac{D_{\text{rot}}}{D_{\text{Zspt}}(i)} \quad (4.1)$$

A trivial upper bound for the rotational delay is a full rotation.

4.2 Database Buffer

The database buffer

1. is a finite piece of memory,
2. typically supports a limited number of different page sizes (mostly one or two),

D_{cyl}	total number of cylinders
D_{track}	total number of tracks
D_{sector}	total number of sectors
D_{tpc}	number of tracks per cylinder (= number of surfaces)
D_{cmd}	command interpretation time
D_{rot}	time for a full rotation
D_{rdsettle}	time for settle for read
D_{wrsettle}	time for settle for write
D_{hdswitch}	time for head switch
D_{Zone}	total number of zones
$D_{\text{Zcyl}}(i)$	number of cylinders in zone i
$D_{\text{Zspt}}(i)$	number of sectors per track in zone i
$D_{\text{Zspc}}(i)$	number of sectors per cylinder in zone i ($= D_{\text{tpc}} D_{\text{Zspt}}(i)$)
$D_{\text{Zscan}}(i)$	time to scan a sector in zone i ($= D_{\text{rot}} / D_{\text{Zspt}}(i)$)
D_{avgseek}	average seek costs
D_{c_0}	parameter for seek cost function
D_{c_1}	parameter for seek cost function
D_{c_2}	parameter for seek cost function
D_{c_3}	parameter for seek cost function
D_{c_4}	parameter for seek cost function
$D_{\text{seek}}(d)$	cost of a seek of d cylinders
	$D_{\text{seek}}(d) = \begin{cases} D_{c_1} + D_{c_2} \sqrt{d} & \text{if } d \leq D_{c_0} \\ D_{c_3} + D_{c_4} d & \text{if } d > D_{c_0} \end{cases}$
$D_{\text{rot}}(s, i)$	rotation cost for s sectors of zone i ($= s D_{\text{Zscan}}(i)$)

Table 4.1: Disk drive parameters and elementary cost functions

3. is often fragmented into several buffer pools,
4. each having a replacement strategy (typically enhanced by hints).

Given the page identifier, the buffer frame is found by a hashtable lookup. Accesses to the hash table and the buffer frame need to be synchronized. Before accessing a page in the buffer, it must be fixed. These points account for the fact that the costs of accessing a page in the buffer are, therefore, greater than zero.

4.3 Physical Database Organization

We call everything that is stored in the database and relevant for answering queries a *database item*. Let us exclude meta data. In a relational system, a database item can be a relation, a fragment of a relation (if the relation is horizontally or vertically fragmented), a segment, an index, a materialized view,

or an index on a materialized view. In object-oriented databases, a database item can be the extent of a class, a named object, an index and so forth. In XML databases, a database item can be a named document, a collection of documents, or an index. Access operations to database items form the leaves of query evaluation plans.

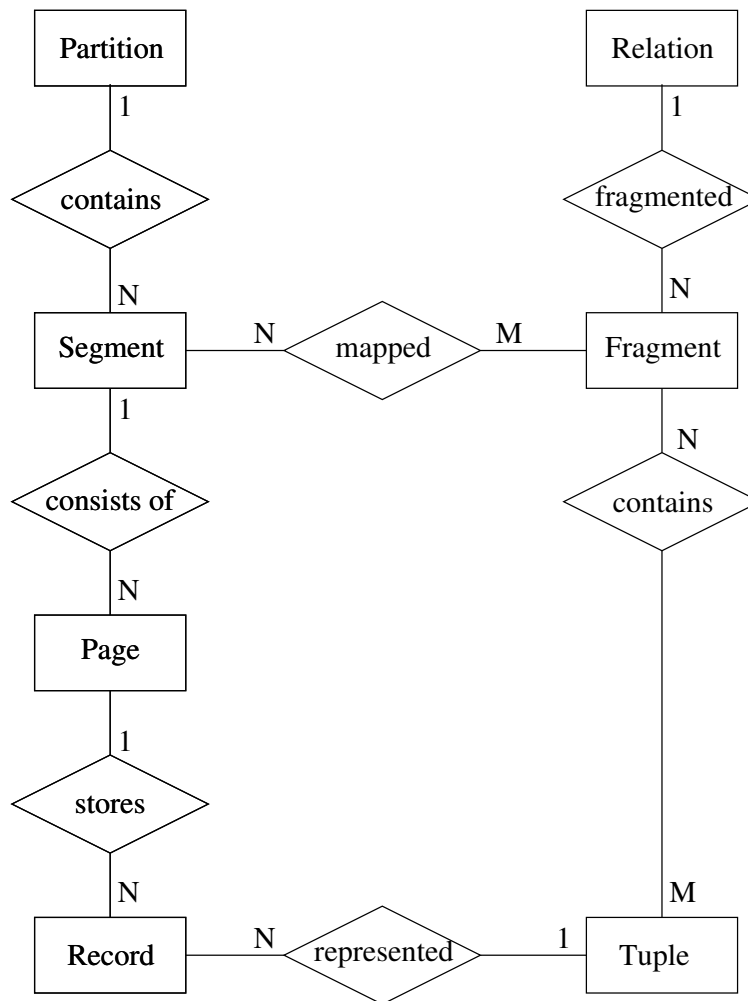


Figure 4.6: Physical organization of a relational database

The physical algebra implemented in the query execution engine of some runtime systems allow to access database items. Since most database items consist of several data items (tuples, objects, documents), these access operations produce a *stream* of data items. This kind of collection-valued access operation is called a *scan*. Consider the simple query

```

select  *
from    Student
  
```

This query is valid only if the database item (relation) **Student** exists. It could

be accessible via a **relation scan** operation `rscan(Student)`. However, in reality we have to consider the physical organization of the database.

Figure 4.6 gives an overview of how relations can be stored in a relational database system. Physical database items can be found on the left-hand side, logical database items on the right-hand side. A fraction of a physical disk is a partition. It can be an operating system file or a raw partition. A partition is organized into several segments. A segment consists of several pages. The pages within a segment are typically accessible by a non-negative integer in $[0, n[$, where n is the number of pages of the segment². Iterative access to all pages of a segment is typically possible. The access is called a *scan*. As there are several types of segments (e.g. data segments, index segments), several kinds of scans exist. Within a page, *physical records* are stored. Each physical record represents a (part of a) tuple of a fragment of a relation.

Fragments are mapped to segments and relations are partitioned into fragments. In the simplest and most common organization, every relation has only one fragment with a one-to-one mapping to segments, and for every tuple there exists exactly one record representing only this tuple. Hence, both of relationships **mapped** and **represented** are one-to-one. However, this organization does not scale well. A relation could be larger than a disk. Even if a large relation, say 180 GB fits on a disk, scanning it takes half an hour (Model 2004). Horizontal partitioning and allocation of the fragments on several disks reduces the scan time by allowing for parallelism. Vertical partitioning is another means of reducing I/O [206]. Here, a tuple is represented by several physical records, each one containing a subset of the tuple's attributes. Since the relationship *mapped* is N:M, tuples from different relations can be stored in the same segment. Furthermore, in distributed database systems some fragments might be stored redundantly at different locations to improve access times [136, 514, 706, 665]. Some systems support clustering of tuples of different relations. For example, department tuples can be clustered with employee tuples such that those employees belonging to the department are close together and close to their department tuple. Such an organization speeds up join processing.

To estimate costs, we need a model of a segment. We assume an extent-based implementation. That is, a segment consists of several extents³. Each extent occupies consecutive sectors on disk. For simplicity, we assume that whole cylinders belong to a segment. Then, we can model segments as follows. Each segment consists of a sequence of *extents*. Each extent is stored on *consecutive cylinders*. Cylinders are exclusively assigned to a segment. We then describe each extent j as a pair (F_j, L_j) where F_j is the first and L_j the last cylinder of a consecutive sequence of cylinders. A segment can then be described by a sequence of such pairs. We assume that these pairs are sorted in ascending order. In such a description, an extent may include a zone boundary. Since cost functions are dependent on the zone, we break up cylinder ranges that are not contained in a single zone. The result can be described by a sequence of triples

²This might not be true. Alternatively, the pages of a partition can be consecutively numbered.

³Extents are not shown in Fig. 4.6. They can be included between Partitions and Segments.

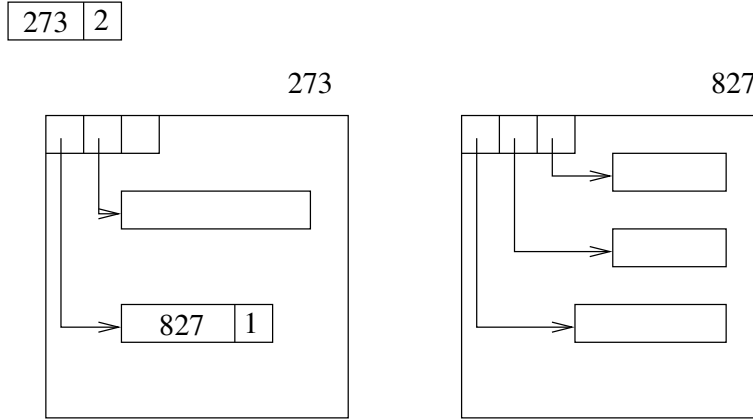


Figure 4.7: Slotted pages and TIDs

(F_i, L_i, z_i) where F_i and L_i mark a range of consecutive cylinders in a zone z_i . Although the z_i 's can be inferred from the cylinder numbers, we include them for clarity. Also of interest are the total number of sectors in a segment and the number of cylinders $S_{\text{cpe}}(i)$ in an extent i . Summarizing, we describe a segment by the parameter given in Table 4.2.

S_{ext}	number of extents in the segment
S_{sec}	total number of sectors in the segment $(= \sum_{i=1}^{S_{\text{ext}}} S_{\text{cpe}}(i) D_{\text{Zspc}}(S_{\text{zone}}(i)))$
$S_{\text{first}}(i)$	first cylinder in extent i
$S_{\text{last}}(i)$	last cylinder in extent i
$S_{\text{cpe}}(i)$	number of cylinders in extent i $(= S_{\text{last}}(i) - S_{\text{first}}(i) + 1)$
$S_{\text{zone}}(i)$	zone of extent i

Table 4.2: Segment parameters

4.4 Slotted Page and Tuple Identifier (TID)

Let us briefly review *slotted pages* and the concept of *tuple identifiers (TIDs)* (see Figure 4.7) [42, 41, 569, 845]. Sometimes, *record identifier* or *row identifier* (RID) is used in the literature. A TID consists of (at least) two parts. The first part identifies a page, the second part a slot on a *slotted page*. The slot contains—among other things, e.g. the record's size—a (relative) pointer to the actual record. This way, the record can be moved within the page without invalidating its TID. When a record grows beyond the available space, it is moved to another page and leaves a forward pointer (again consisting of a page and a slot identifier) in its original position. This happened to the TID [273, 1] in Figure 4.7. If the record has to be moved again, the forward pointer is adjusted. This way, at most two page accesses are needed to retrieve a record, given its TID. For evaluating the costs of record accesses, we will assume that

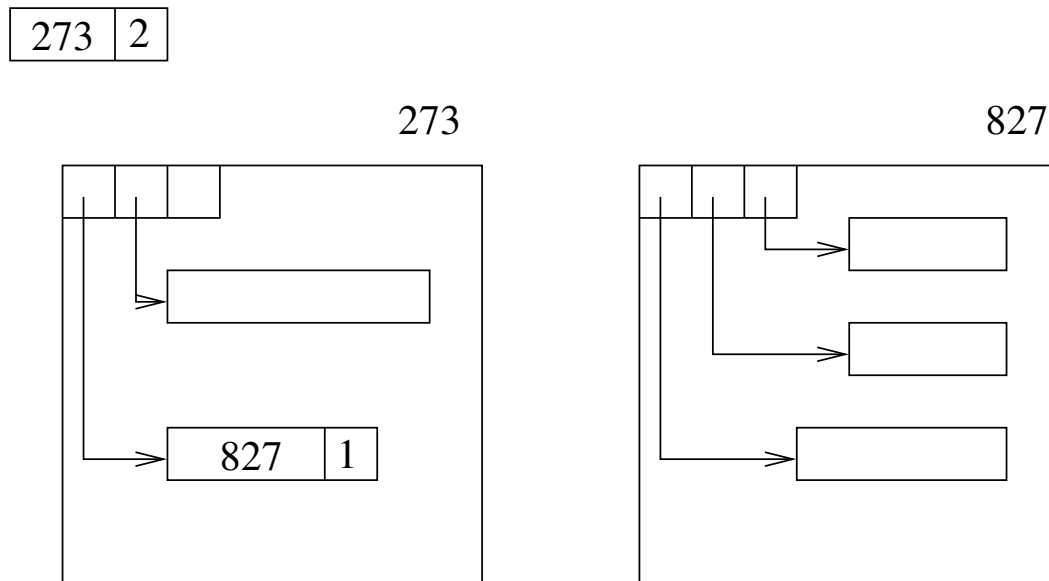


Figure 4.8: Various physical record layouts

the fraction of moved records is known.

4.5 Physical Record Layouts

A physical record represents a tuple, object, or some other logical entity or fraction thereof. In case it represents a tuple, it consists of several fields, each representing the value of an attribute. These values can be integers, floating point numbers, or strings. In case of object-oriented or object-relational systems, the values can also be of a complex type. Tuple identifiers are also possible as attribute values [731]. This can, for example, speed up join processing.

In any case, we can distinguish between types whose values all exhibit the same fixed length and those whose values may vary in length. In a physical record, the values of fixed-length attributes are concatenated and the offset from the beginning of the record to the value of some selected attribute can be inferred from the types of the values preceding it. This differs for values of varying length. Here, several encodings are possible. Some simple ones are depicted in Figure 4.8. The topmost record encodes varying length values as a sequence of pairs of the form $[size, value]$. This encoding has the disadvantage that access to an attribute of varying length is linear in the number of those preceding it. This disadvantage is avoided in the solution presented in the middle. Instead of storing the sizes of the individual values, there is an array containing relative offsets into the physical record. They point to the start of the values. The length of the values can be inferred from these offsets and, in case of the last value, from the total length of the physical record, which is typically stored in its slot. Access to a value of varying size is now simplified to an indirect memory access plus some length calculations. Although this might

be cheaper than the first solution, there is still a non-negligible cost associated with an attribute access.

The third physical record layout can be used to represent compressed attribute values and even compressed length information for parts of varying size. Note that if fixed size fields are compressed, their length becomes varying. Access to an attribute now means decompressing length/offset information and decompressing the value itself. The former is quite cheap: it boils down to an indirect memory access with some offset taken from an array [921]. The cost of the latter depends on the compression scheme used. It should be clear that accessing an attribute value now is even more expensive. To make the costs of an attribute access explicit was the sole purpose of this small section.

Remark Westmann et al. discuss an efficient implementation of compression and evaluate its performance [921]. Yiannis and Zobel report on experiments with several compression techniques used to speed up the sort operator. For some of them, the CPU usage is twice as large [959].

4.6 Physical Algebra (Iterator Concept)

Physical algebraic operators are mostly implemented as *iterators*. This means that they support the the interface operations **open**, **next**, and **close**. With **open**, the stream of items (e.g. tuples) is initialized. With **next**, the next item on the stream is fetched. When no more items are available, e.g. **next** returns false, **close** can be called to clean up things. The iterator concept is explained in many text books (e.g. [316, 403, 484]) and the query processing survey by Graefe [347]. This basic iterator concept has been extended to better cope with nested evaluation by Westmann in his thesis [919], Westmann et al. [921], and Graefe [351]. The two main issues are separation of storage allocation and initialization, and batched processing. The former splits **open** into resource allocation, initialization of the operator, and initialization of the iterator.

4.7 Simple Scan

Let us come back to the scan operations. A logical operation for scanning relations (which could be called **rscan**) is rarely supported by relational database management systems. Instead, they provide (physical) scans on segments. Since a (data) segment is sometimes called *file*, the correct plan for the above query is often denoted by **fscan(Student)**. Several assumptions must hold: the **Student** relation is not fragmented, it is stored in a single segment, the name of this segment is the same as the relation name, and no tuples from other relations are stored in this segment. Until otherwise stated, we will assume that relations are not partitioned, are stored in a single segment and that the segment can be inferred from the relation's name. Instead of **fscan(Student)**, we could then simply use **Student** to denote leaf nodes in a query execution plan. If we want to use a variable that is bound subsequently to each tuple in a relation, the query

```
select *
```

from Student

can be expressed as **Student**[s] instead of **Student**. In this notation, the output stream contains tuples having a single attribute s bound to a tuple. Physically, s will not hold the whole tuple but, for example, a pointer into the buffer where the tuple can be found. An alternative is a pointer to a slot of a slotted page contained in the buffer.

A simple scan is an example for a *building block*. In general, a building block is something that is used as a bottommost operator in a query evaluation plan. Hence, every leaf node of a query evaluation plan is a building block or a part thereof. This is not really a sharp definition, but is sometimes useful to describe the behavior of a query compiler: after their determination, it will leave building blocks untouched even if reorderings are hypothetically possible. Although a building block can be more than a leaf node (scan) of a query evaluation plan, it will never include more than a single database item. As soon as more database items are involved, we use the notion of *access path*, a term which will become more precise later on when we discuss index usage.

The disk access costs for a simple scan can be derived from the considerations in Section 4.1 and Section 4.17.

4.8 Scan and Attribute Access

Strictly speaking, a plan like $\sigma_{\text{age} > 30}(\text{Student}[s])$ is invalid, since the tuple stream produced by **Student**[s] contains tuples with a single attribute s . We have a choice. Either we assume that attribute access takes place implicitly, or we make it explicit. Whether this makes sense or not depends on the database management system for which we generate plans. Let us discuss the advantages of explicit attribute retrieval. Assume **s.age** retrieves the age of a student. Then we can write $\sigma_{\text{s.age} > 30}(\text{Student}[s])$, where there is some non-neglectable cost for **s.age**. The expression $\sigma_{\text{s.age} > 30 \wedge \text{s.age} < 40}(\text{Student}[s])$ executes **s.age** twice. This is a bad idea. Instead, we would like to retrieve it once and reuse it later.

This purpose is well-served by the *map* operator (χ). It adds new attributes to a given tuple and is defined as

$$\chi_{a_1:e_1, \dots, a_n:e_n}(e) := \{t \circ [a_1 : c_1, \dots, a_n : c_n] \mid t \in e, c_i = e_i(t) \ \forall (1 \leq i \leq n)\}$$

where $\circ$ denotes tuple concatenation and the a_i must not be in $\mathcal{A}(e)$. (Remember that $\mathcal{A}(e)$ is the set of attributes produced by e .) Every input tuple t is extended by new attributes a_i , whose values are computed by evaluating the expression e_i , in which free variables (attributes) are bound to the attributes (variables) provided by t .

The above problem can now be solved by

$$\sigma_{\text{age} > 30 \wedge \text{age} < 40}(\chi_{\text{age:s.age}}(\text{Student}[s])).$$

In general, it is beneficial to load attributes as late as possible. The latest point at which all attributes must be read from the page is typically just before a pipeline breaker⁴.

To see why this is useful, consider the simple query

```
select  name
from    Student
where   age > 30
```

The plan

$$\Pi_n(\chi_{n:s.name}(\sigma_{a>30}(\chi_{a:s.age}(\mathbf{Student}[s]))))$$

makes use of this feature, while

$$\Pi_n(\sigma_{a>30}(\chi_{n:s.name,a:s.age}(\mathbf{Student}[s])))$$

does not. In the first plan the **name** attribute is only accessed for those students with age over 30. Hence, it should be cheaper to evaluate. If the database management system does not support this selective access mechanism, we often find the scan enhanced by a list of attributes that is projected and included in the resulting tuple stream.

In order to avoid copying attributes from their storage representation to some main memory representation, some database management systems apply another mechanism. They support the evaluation of some predicates directly on the storage representation. These are boolean expressions consisting of simple predicates of the form $A\theta c$ for attributes A , comparison operators θ , and constants c . Instead of a constant, c could also be the value of some attribute or expression thereof given that it can be evaluated before the access to A .

Predicates evaluable on the disk representation are called *SARGable* where *SARG* is an acronym for *search argument*. Note that SARGable predicates may also be good for index lookups. Then they are called *index SARGable*. In case they can not be evaluated by an index, they are called *data SARGable* [784, 863, 322].

Since relation or segment scans can evaluate predicates, we have to extend our notation for scans. Let I be a database item like a relation or segment. Then, $I[v;p]$ scans I , binds each item in I successively to v and returns only those items for which p holds. $I[v;p]$ is equivalent to $\sigma_p(I[v])$, but cheaper to evaluate. If p is a conjunction of predicates, the conjuncts should be ordered such that the attribute access cost reductions described above are reflected (for details see Chapter ??). Syntactically, we express this by separating the predicates by a comma as in `Student[s; age > 30, name like '%m%']`. If we want to make a distinction between SARGable and non-SARGable predicates, we write $I[v;p_s;p_r]$, with p_s being the SARGable predicate and p_r a non-SARGable predicate. Additional extensions like a projection list are also possible.

⁴The page on which the physical record resides must be fixed until all attributes are loaded. Hence, an earlier point in time might be preferable.

4.9 Temporal Relations

Scanning a temporal relation or segment also makes sense. Whenever the result of some (partial) query evaluation plan is used more than once, it might be worthwhile to materialize it in some temporary relation. For this purpose, a **tmp** operator evaluates its argument expression and stores the result relation in a temporary segment. Consider the following example query.

```
select  e.name, d.name
from    Emp e, Dept d
where   e.age > 30 and e.age < 40 and e.dno = d.dno
```

It can be evaluated by

$$\text{Dept}[d] \bowtie_{e.dno=d.dno}^{n1} \sigma_{e.age>30 \wedge e.age<40}(\text{Emp}[d]).$$

Since the inner (right) argument of the nested-loop join is evaluated several times (once for each department), materialization may pay off. The plan then looks like

$$\text{Dept}[d] \bowtie_{e.dno=d.dno}^{n1} \text{Tmp}(\sigma_{e.age>30 \wedge e.age<40}(\text{Emp}[d])).$$

If we choose to factorize and materialize a common subexpression, the query evaluation plan becomes a DAG. Alternatively, we could write a small “program” that has some statements materializing some expressions which are then used later on. The last expression in a program determines its result. For our example, the program looks as follows.

1. $R_{\text{tmp}} = \sigma_{e.age>30 \wedge e.age<40}(\text{Emp}[d]);$
2. $\text{Dept}[d] \bowtie_{e.dno=d.dno}^{n1} R_{\text{tmp}}[e]$

The disk costs of writing and reading temporary relations can be calculated using the considerations of Section 4.1.

4.10 Table Functions

A *table function* is a function that returns a relation [576]. An example is **Primes(int from, int to)**, which returns all primes between **from** and **to**, e.g. via a sieve-method. It can be used in any place where a relation name can occur. The query

```
select  *
from    TABLE(Primes(1,100)) as p
```

returns all primes between 1 and 100. The attribute names of the resulting relation are specified in the declaration of the table function. Let us assume that for **Primes** a single attribute **prime** is specified. Note that table functions may take parameters. This does not pose any problems, as long as we

know that `Primes` is a table function and we translate the above query into `Primes(1, 100)[p]`. Although this looks exactly like a table scan, the implementation and cost calculations are different.

Consider the following query where we extract the years in which we expect a special celebration of Anton's birthday.

```
select *
from Friends f,
      TABLE(Primes(
              CURRENT_YEAR, EXTRACT(YEAR FROM f.birthday) + 100)) as p
where f.name = 'Anton'
```

The result of the table function depends on our friend Anton. Hence, a join is no solution. Instead, we have to introduce a new kind of join, the *d-join* where the *d* stands for dependent. It is defined as

$$R < S > = \{t \circ s \mid t \in T, s \in S(t)\}.$$

The above query can now be evaluated as

$\chi_{b:EXTRACT_YEAR(f.birthday)+100}(\sigma_{f.name='Anton'}(Friends[f])) < Primes(c, b)[p] >$

where we assume that some global entity *c* holds the value of `CURRENT_YEAR`.

Let us do the above query for all friends. We just have to drop the `where` clause. Obviously, this results in many redundant computations of primes. At the SQL level, using the birthday of the youngest friend is beneficial:

```
select *
from Friends f,
      TABLE(Primes(
              CURRENT_YEAR, (select max(birthday) from Friends) + 100)) as p
where p.prime ≥ f.birthday
```

At the algebraic level, this kind of optimizations will be considered in Section ??.

Things can get even more involved if table functions can consume and produce relations, i.e. arguments and results can be relations.

ToDo?

Little can be said about the disk costs of table functions. They can be zero if the function is implemented such that it does not access any disks (files stored there), but it can also be very expensive if large files are scanned each time it is called. One possibility is to let the database administrator specify the numbers the query optimizer needs. However, since parameters are involved, this is not really an easy task. Another possibility is to measure the table function's behavior whenever it is executed, and learn about its resource consumption.

4.11 Indexes

There exists a plethora of different index structures. In the context of relational database management systems, the most versatile and robust index is the B-tree or variants/improvements thereof (e.g. [?]). It is implemented in almost

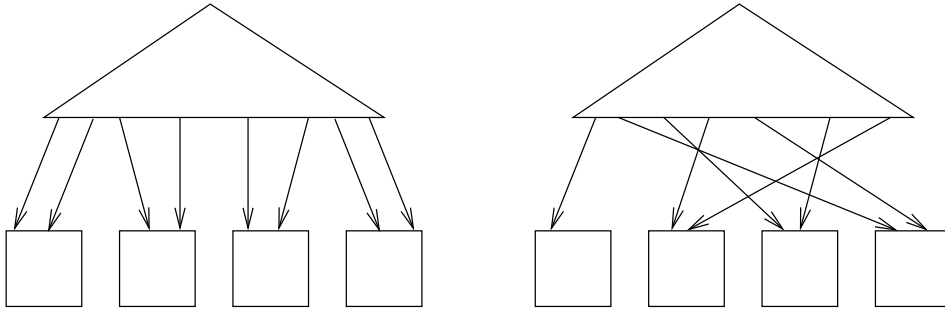


Figure 4.9: Clustered vs. non-clustered index

every commercial database management system. Some support hash-indexes (e.g. [?]). Other data models or specific applications need specialized indexes. There exist special index structures for indexing path expressions in object-oriented databases (e.g. [?]) and XML databases (e.g. [?]). Special purpose indexes include join indexes (e.g. [401, 892]) multi-dimensional indexes (e.g. [?]), variant (projection) indexes [651], small materialized aggregates [614], bitmap indexes [?], and temporal indexes (e.g. [?]). We cannot discuss all indexes and their exploitations for efficient query evaluation. This fills more than a single book. Instead, we concentrate on B-tree indexes. In general, a B-tree can be used to index several relations. We only discuss cases where B-trees index a single relation.

The *search key* (or *key* for short) of an index is the sequence of attributes of the *indexed relation* over which the index is defined. A key is a *simple key* if it consists of a single attribute. Otherwise, it is a *complex key*. Each entry in the B-tree's leaf page consists of pairs containing the key values and a sequence of tuple identifiers (typically sorted by increasing page number). Every tuple with a TID in this list satisfies the condition that its indexed attribute's values are equal to the key values. If for every sequence of key values there is at most one such tuple, we have a *unique index*, otherwise a *non-unique index*.

The leaf entries may contain values from additional (non-key) attributes. Then we call the index *attribute data added* and the additional attributes *data attributes*. If the index contains all attributes of the indexed relation—in its key or data attributes—storing the relation is no longer necessary. The result is an *index-only relation*. In this case, the concept of tuple identifiers is normally no longer used since tuples can now be moved frequently, e.g. due to a leaf page split. This has two consequences. First, the data part does not longer contain the TID. Second, other indexes on the index-only relation cannot have tuple identifiers in their data part either. Instead, they use the key of the index-only relation to uniquely reference a tuple. For this to work, we must have a unique index.

B-trees can be either *clustered* or *non-clustered* indexes. In a clustered index, the tuple identifiers in the list of leaf pages are ordered according to their page

numbers. Otherwise, it is a *non-clustered* index⁵. Figure 4.9 illustrates this. Range queries result in sequential access for clustered indexes and in random access for non-clustered indexes.

4.12 Single Index Access Path

4.12.1 Simple Key, No Data Attributes

Consider the *exact match query*

```
select  name
from    Emp
where   eno = 1077
```

If there exists a unique index on the key attribute **eno**, we can first access the index to retrieve the TID of the employee tuple satisfying **eno** = 1077. Another page access yields the tuple itself which constitutes the result of the query. Let Emp_{eno} be the index on **eno**, then we can descend the B-tree, using 1077 as the search key. A predicate that can be used to descend the B-tree or, in general, governing search within an index structure, is called an *index sargable predicate*.

For the example query, the index scan, denoted as $\text{Emp}_{\text{eno}}[x; \text{eno} = 1077]$, retrieves a single leaf node entry with attributes **eno** and TID. Similar to the regular scan, we assume x to be a variable holding a pointer to this index entry. We use the notations $x.\text{eno}$ and $x.\text{TID}$ to access these attributes. To dereference the TID, we use the map (χ) operator and a dereference function **deref** (or $*$ for short). It turns a TID into a pointer in the buffer area. This of course requires the page to be loaded, if it is not in the buffer yet. The complete plan for the query is

$$\Pi_{\text{name}}(\chi_{e:*(x.\text{TID}),\text{name}:e.\text{name}}(\text{Emp}_{\text{eno}}[x; \text{eno} = 1077]))$$

where we computed several new attributes with one χ operator. Note that they are dependent on previously computed attributes and, hence, the order of evaluation does matter.

We can make the dependency of the map operator more explicit by applying a d-join. Denote by $\square$ an operator that returns a single empty tuple. Then

$$\Pi_{\text{name}}(\text{Emp}_{\text{eno}}[x; \text{eno} = 1077] < \chi_{e:*(x.\text{TID}),\text{name}:e.\text{name}}(\square) >)$$

is equivalent to the former plan. Joins and indexes will be discussed in Section 4.14.

A *range query* like

```
select  name
from    Emp
where   age ≥ 25 and age ≤ 35
```

⁵Of course, any degree of clusteredness may occur and has to be taken into account in cost calculations.

specifies a range for the indexed attribute. It is evaluated by an index scan with *start* and *stop* conditions. In our case, the start condition is **age** ≥ 25 , and the stop condition is **age** ≤ 35 . The start condition is used to retrieve the first tuple satisfying it by searching within the B-tree. In our case, 25 is used to descend from the root to the leaf page containing the key 25. Then, all records with keys larger than 25 within the page are searched. Since entries in B-tree pages are sorted on key values, this is very efficient. If we are done with the leaf page that contains 25 and the stop key has not been found yet, we proceed to the next leaf page. This is possible since leaf pages of B-trees tend to be chained. Then all records of the next leaf page are scanned and so on until we find the stop key. The complete plan then is

$$\Pi_{\text{name}}(\chi_{e:*(x.\text{TID}),\text{name}:e.\text{name}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35]))$$

If the index on **age** is non-clustered, this plan results in random I/O. We can turn random I/O into sequential I/O by sorting the result of the index scan on its TID attribute before dereferencing it⁶. This results in the following plan:

$$\Pi_{\text{name}}(\chi_{e:*(\text{TID}),\text{name}:e.\text{name}}(\text{Sort}_{\text{TID}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35; \text{TID}])))$$

Here, we explicitly included the TID attribute of the index into the projection list.

Consider a similar query which demands the output to be sorted:

```
select    name, age
from      Emp
where     age  $\geq$  25 and age  $\leq$  35
order by  age
```

Since an index scan on a B-tree outputs its result ordered on the indexed attribute, the following plan produces the perfect result:

$$\Pi_{\text{name},\text{age}}(\chi_{e:*(x.\text{TID}),\text{name}:e.\text{name}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35]))$$

On a clustered index this is most probably the best plan. On a non-clustered index, random I/O disturbs the picture. We avoid that by sorting the result of the index scan on the TID attribute and, after accessing the tuples, restore the order on **age** as in the following plan:

$$\Pi_{\text{name},\text{age}}(\text{Sort}_{\text{age}}(\chi_{e:*(\text{TID}),\text{name}:e.\text{name}}(\text{Sort}_{\text{TID}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35; \text{TID}]))))$$

An alternative to this plan is not to sort on the original indexed attribute (**age** in our example), but to introduce a new attribute that holds the rank in the sequence derived from the index scan. This leads to the plan

$$\Pi_{\text{name},\text{age}}(\text{Sort}_{\text{rank}}(\chi_{e:*(\text{TID}),\text{name}:e.\text{name}}(\text{Sort}_{\text{TID}}(\chi_{\text{rank:counter}++}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35; \text{TID}]))))))$$

⁶This might not be necessary, if **Emp** fits main memory. Then, preferably asynchronous I/O should be used.

This alternative might turn out to be more efficient since sorting on an attribute with a dense domain can be implemented efficiently. (We admit that in the above example this is not worth considering.) There is another important application of this technique: XQuery often demands output in document order. If this order is destroyed during processing, it must at the latest be restored when the output is produced [592]. Depending on the implementation (i.e. the representation of document nodes or their identifiers), this might turn out to be a very expensive operation.

The fact that index scans on B-trees return their result ordered on the indexed attributes is also very useful if a merge-join on the same attributes (or a prefix thereof, see Chapter 23 for further details) occurs. An example follows later on.

Some *predicates* are not index SARGable, but can still be evaluated with the index as in the following query

```
select  name
from    Emp
where   age ≥ 25 and age ≤ 35 and age ≠ 30
```

The predicate `age ≠ 30` is an example of a *residual predicate*. We can once more extend the index scan and compile the query into

$$\Pi_{\text{name}}(\chi_{t:x.\text{TID}, e:*t, \text{name}:e.\text{name}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35; \text{age} \neq 30]))$$

Some index scan implementations allow exclusive bounds for start and stop conditions. With them, the query

```
select  name
from    Emp
where   age > 25 and age < 35
```

can be evaluated using

$$\Pi_{\text{name}}(\chi_{t:x.\text{TID}, e:*t, \text{name}:e.\text{name}}(\text{Emp}_{\text{age}}[x; 25 < \text{age}; \text{age} < 35]))$$

If this is not the case, two residual predicates must be used as in

$$\Pi_{\text{name}}(\chi_{t:x.\text{TID}, e:*t, \text{name}:e.\text{name}}(\text{Emp}_{\text{age}}[x; 25 \leq \text{age}; \text{age} \leq 35; \text{age} \neq 25, \text{age} \neq 35]))$$

Especially for predicates on strings, this might be expensive.

Start and stop conditions are optional. To evaluate

```
select  name
from    Emp
where   age ≥ 60
```

we use `age ≥ 60` as the start condition to find the leaf page containing the key 60. From there on, we scan all the leaf pages “to the right”.

If we have no start condition, as in

```

select  name
from    Emp
where   age  $\leq$  20

```

we descend the B-tree to the “leftmost” page, i.e. the page containing the smallest key value, and then proceed scanning leaf pages until we encounter the key 20.

Having neither a start nor stop condition is also quite useful. The query

```

select  count(*)
from    Emp

```

can be evaluated by counting the entries in the leaf pages of a B-tree. Since a B-tree typically occupies far fewer pages than the original relation, we have a viable alternative to a relation scan. The same applies to the aggregate functions **sum** and **avg**. The other aggregate functions **min** and **max** can be evaluated much more efficiently by descending to the leftmost or rightmost leaf page of a B-tree. This can be used to answer queries like

```

select  min/max(salary)
from    Emp

```

much more efficiently than by a relation scan. Consider the query

```

select  name
from    Emp
where   salary = (select max(salary)
                  from    Emp)

```

It can be evaluated by first computing the maximum salary and then retrieving the employees earning this salary. This requires two descendants into the B-tree, while obviously one is sufficient. Depending on the implementation of the index (scan), we might be able to perform this optimization.

Further, the result of an index scan, whether it uses start and/or stop conditions or not, is always sorted on the key. This property can be useful for queries with no predicates. If we have neither a start nor a stop condition, the resulting scan is called *full index scan*. As an example consider the query

```

select   salary
from     Emp
order by salary

```

which is perfectly answered by the following full index scan:

$\text{Emp}_{\text{salary}}$

So far, we have only seen indexes on numerical attributes.

```

select  name, salary
from    Emp
where   name  $\geq$  'Maaa'

```

gives rise to a start condition ' $\text{Maaa} \leq \text{name}$ '. From the query

```

select  name, salary
from    Emp
where   name like 'M%'

```

we can deduce the start condition ' $\text{M} \leq \text{name}$ '.

To express all the different alternatives of index usage, we need a powerful (and runtime system dependent) index scan expression. Let us first summarize what we can specify for an index scan:

1. the name of the variable for index entries (or pointers to them),
2. the start condition,
3. the stop condition,
4. a residual predicate, and
5. a projection list.

A projection list has entries of the form $a : x.b$ for attribute names a and b and x being the name of the variable for the index entry. $a : x.a$ is also allowed and often abbreviated as a . We also often summarize start and stop conditions into a single expression like in $25 \leq \text{age} \leq 35$.

For a full index specification, we list all items in the subscript of the index name separated by a semicolon. Still, we need some extensions to express the queries with aggregation. Let a and b be attribute names, then we allow entries of the form $b : \text{aggr}(a)$ in the projection list and start/stop conditions of the form $\text{min}/\text{max}(a)$. The latter tells us to minimize/maximize the value of the indexed attribute a . Only a complete enumeration gives us the full details. On the other hand, extracting start and stop conditions and residual predicates from a given boolean expression is rather simple. Hence, we often summarize these three under a single predicate. This is especially useful when talking about index scans in general. If we have a full index scan, we leave out the predicate. We use a star '*' as an abbreviated projection list that projects all attributes of the index. (So far, these are the key attribute and the TID.) If the projection list is empty, we assume that only the variable/attribute holding the pointer to the index entry is projected.

Using this notation, we can express some plan fragments. These fragments are complete plans for the above queries, except that the final projection is not present. As an example, consider the following fragment:

$$\chi_{e:*\text{TID},\text{name}:e.\text{name}}(\text{Emp}_{\text{salary}}[x; \text{TID}, \text{salary}])$$

All the plan fragments seen so far are examples of access paths. An *access path* is a plan fragment with building blocks concerning a single database item.

Hence, every building block is an access path. The above plans touch two database items: a relation and an index on some attribute of that relation. If we say that an index concerns the relation it indexes, such a fragment is an access path. For relational systems, the most general case of an access path uses several indexes to retrieve the tuples of a single relation. We will see examples of these more complex access paths in the following section. An access to the original relation is not always necessary. A query that can be answered solely by accessing indexes is called an *index only query*.

A query with **in** like

```
select  name
from    Emp
where   age in {28, 29, 31, 32}
```

can be evaluated by taking the minimum and the maximum found in the left-hand side of **in** as the start and stop conditions. We further need to construct a residual predicate. The residual predicate can be represented either as $\text{age} = 28 \vee \text{age} = 29 \vee \text{age} = 31 \vee \text{age} = 32$ or as $\text{age} \neq 30$.

An alternative is to use a d-join. Consider the example query

```
select  name
from    Emp
where   salary in {1111, 11111, 111111}
```

Here, the numbers are far apart and separate index accesses might make sense. Therefore, let us create a temporary relation **Sal** equal to $\{[s : 1111], [s : 11111], [s : 111111]\}$. When using it, the access path becomes

$$\text{Sal}[S] < \chi_{e:*TID, \text{name}:e.\text{name}}(\text{Emp}_{\text{salary}}[x; \text{salary} = S.s; TID]) >$$

Some B-tree implementations allow efficient searches for multiple ranges and implement *gap skipping* [34, 35, 170, 322, 323, 476, 545]. *Gap skipping*, sometimes also called *zig-zag skipping*, continues the search for keys in a new key range from the latest position visited. The implementation details vary but the main idea of it is that after one range has been completely scanned, the current (leaf) page is checked for its highest key. If it is not smaller than the lower bound of the next range, the search continues in the current page. If it is smaller than the lower bound of the next range, alternative implementations are described in the literature. The simplest is to start a new search from the root for the lower bound. Another alternative uses parent pointers to go up a page as long as the highest key of the current page is smaller than the lower bound of the next range. If this is no longer the case, the search continues downwards again.

Gap skipping gives even more opportunities for index scans and allows efficient implementations of various index nested loop join strategies.

4.12.2 Complex Keys and Data Attributes

In general, an index can have a complex key comprised of the key attributes $k_1, \dots, k_n$ and the data attributes $d_1, \dots, d_m$. One possibility is to use a full

index scan on such an index. Having more attributes in the index makes it more probable that queries are index-only.

Besides a full index scan, the index can be descended to directly search for the desired tuple(s). Let us take a closer look at this possibility.

If the search predicate is of the form

$$k_1 = c_1 \wedge k_2 = c_2 \wedge \dots \wedge k_j = c_j$$

for some constants c_i and some $j \leq n$, we can generate the start and stop condition

$$k_1 = c_1 \wedge \dots \wedge k_j = c_j.$$

This simple approach is only possible if the search predicates define values for all search key attributes, starting from the first search key and then for all keys up to the j -th search key with no key attribute unspecified in between. Predicates concerning the other key attributes after the first non-specified key attribute and the additional data attributes only allow for residual predicates. This condition is often not necessary for multi-dimensional index structures, whose discussion is beyond the book.

With ranges things become more complex and highly dependent on the implementation of the facilities of the B-tree. Consider a query predicate restricting key values as follows

$$k_1 = c_1 \wedge k_2 \geq c_2 \wedge k_3 = c_3$$

Obviously, we can generate the start condition $k_1 = c_1 \wedge k_2 \geq c_2$ and the stop condition $k_1 = c_1$. Here, we neglected the condition on k_3 which becomes a residual predicate. However, with some care we can extend the start condition to $k_1 = c_1 \wedge k_2 \geq c_2 \wedge k_3 = c_3$: we only have to keep $k_3 = c_3$ as a residual predicate, since for k_2 values larger than c_2 , values different from c_3 can occur for k_3 .

If closed ranges are specified for a prefix of the key attributes as in

$$a_1 \leq k_1 \leq b_1 \wedge \dots \wedge a_j \leq k_j \leq b_j$$

we can generate the start key $k_1 = a_1 \wedge \dots \wedge k_j = a_j$, the stop key $k_1 = b_1 \wedge \dots \wedge k_j = b_j$, and

$$a_2 \leq k_2 \leq b_2 \wedge \dots \wedge a_j \leq k_j \leq b_j$$

as the residual predicate. If for some search key attribute k_j the lower bound a_j is not specified, the start condition cannot contain k_j and any k_{j+i} . If for some search key attribute k_j the upper bound b_j is not specified, the stop condition cannot contain k_j and any k_{j+i} .

Two further enhancements of the B-tree functionality possibly allow for alternative start/stop conditions:

- The B-tree implementation allows to specify the order (ascending or descending) for each key attribute individually.

- The B-tree implementation implements forward and backward scans (e.g. implemented in Rdb [34]).

So far, we are only able to exploit query predicates which specify value ranges for a prefix of all key attributes. Consider querying a person on his/her height and his/her hair color: **haircolor = 'blond' and height between 180 and 190**. If we have an index on **sex, haircolor, height**, this index cannot be used by means of the techniques described so far. However, since there are only the two values **male** and **female** available for **sex**, we can rewrite the query predicate to **(sex = 'm' and haircolor = 'blond' and height between 180 and 190) or (sex = 'f' and haircolor = 'blond' and height between 180 and 190)** and use two accesses to the index. This approach works fine for attributes with a small domain and is described by Antoshenkov [35]. (See also the previous section for gap skipping.) Since the possible values for key attributes may not be known to the query optimizer, Antoshenkov goes one step further and shifts the construction of search ranges to index scan time. Therefore, the index can be provided with a complex boolean expression which is then refined (rewritten) as soon as search key values become known. Search ranges are then generated dynamically, and gap skipping is applied to skip the intervals between the qualifying ranges during the index scan.

4.13 Multi Index Access Path

We wish to buy a used digital camera and state the following query:

```
select *
from   Camera
where  megapixel > 5 and distortion < 0.05
      and noise < 0.01
      and zoomMin < 35 and zoomMax > 105
```

We assume that on every attribute used in the **where** clause there exists an index. Since the predicates are conjunctively connected, we can use a technique called *index and-ing*. Every index scan returns a set (list) of tuple identifiers. These sets/lists are then intersected. This operation is also called *And merge* [562]. Using index and-ing, a possible plan is

```
((((
  Cameramegapixel[c; megapixel > 5; TID]
  ∩
  Cameradistortion[c; distortion < 0.05; TID])
  ∩
  Cameranoise[c; noise < 0.01; TID])
  ∩
  CamerazoomMin[c; zoomMin < 35; TID])
  ∩
  CamerazoomMax[c; zoomMax > 105; TID])
```

This results in a set of tuple identifiers that only needs to be dereferenced to access the according **Camera** tuples and produce the final result.

Since the costs of the expression clearly depend on the costs of the index scans and the size of the intermediate TID sets, two questions arise:

- In which order do we intersect the TID sets resulting from the index scans?
- Do we really apply all indexes before dereferencing the tuple identifiers?

The answer to the latter question is clearly “no”, if the next index scan is more expensive than accessing the records in the current TID list. It can be shown that the indexes in the cascade of intersections are ordered on increasing $(f_i - 1)/c_i$ terms, where f_i is the selectivity of the index and c_i its access cost. Further, we can stop as soon as accessing the original tuples in the base relation becomes cheaper than intersecting with another index and subsequently accessing the base relation.

Index or-ing is used to process disjunctive predicates. Here, we take the *union* of the TID sets to produce a set of TIDs containing references to all qualifying tuples. Note that duplicates must be eliminated during the processing of the union. This operation is also called *Or merge* [562]. Consider the query

```
select *
from Emp
where yearsOfEmployment ≥ 30
      or age ≥ 65
```

This query can be answered by constructing a TID set using the expression

$\text{Emp}_{\text{yearsOfEmployment}}[c; \text{yearsOfEmployment} \geq 30; \text{TID}] \cup \text{Emp}_{\text{age}}[c; \text{age} \geq 65; \text{TID}]$

and then dereferencing the list of tuple identifiers. Again, the index accessing can be ordered for better performance. Given a general boolean expression in **and** and **or**, constructing the optimal access path using index and-ing and or-ing is a challenging task that will be discussed in Chapter ???. This task is even more challenging, if some simple predicates occur more than once in the complex boolean expression and factorization has to be taken into account. This issue was first discussed by Chaudhuri, Ganesan and Saragawi [149]. We will come back to this in Chapter ??.

The names *index and-ing* and *or-ing* become clear if bitmap indexes are considered. Then the bitwise **and** and **or** operations can be used to efficiently compute the intersection and union.

Excursion on bitmap indexes. □

There are even more possibilities to work with TID sets. Consider the query

```
select *
from Emp
where yearsOfEmployment ≠ 10
      and age ≥ 65
```

This query can be evaluated by scanning the index on `age` and then eliminating all employees with `yearsOfEmployment = 10`:

$\text{Emp}_{\text{age}}[c; \text{age} \geq 65; \text{TID}] \setminus \text{Emp}_{\text{yearsOfEmployment}}[c; \text{yearsOfEmployment} \neq 10; \text{TID}]$

Let us call the application of set difference on index scan results *index differencing*.

Some predicates might not be very restrictive in the sense that more than half the index has to be scanned. By negating these predicates and using index differencing, we can make sure that at most half of the index needs to be scanned. As an example consider the query

```
select *
from   Emp
where  yearsOfEmployment ≤ 5
      and age ≤ 65
```

Assume that most of our employees' age is below 65. Then

$\text{Emp}_{\text{yearsOfEmployment}}[c; \text{yearsOfEmployment} \leq 5; \text{TID}] \setminus \text{Emp}_{\text{age}}[c; \text{age} > 65; \text{TID}]$

could be more efficient than

$\text{Emp}_{\text{yearsOfEmployment}}[c; \text{yearsOfEmployment} \leq 5; \text{TID}] \cap \text{Emp}_{\text{age}}[c; \text{age} \leq 65; \text{TID}]$

4.14 Indexes and Joins

There are two issues when discussing indexes and joins. The first is that indexes can be used to speed up join processing. The second is that index accesses can be expressed as joins. We discuss both of these issues, starting with the latter.

In our examples, we used the map operation to (implicitly) access the relation by dereferencing the tuple identifiers. We can make the implicit access explicit by exchanging the map operator by a d-join or even a join. Then, for example,

$\chi_{e:*TID, \text{name}:e.\text{name}}(\text{Emp}_{\text{salary}}[x; 25 \leq \text{age} \leq 35; \text{TID}])$

becomes

$\text{Emp}_{\text{salary}}[x; 25 \leq \text{age} \leq 35; \text{TID}] \lt \chi_{e:*TID, \text{name}:e.\text{name}}(\square) \gt$

where $\square$ returns a single empty tuple. Assume that every tuple contains an attribute `TID` containing its TID. This attribute does not have to be stored explicitly but can be derived. Then, we have the following alternative access path for the join (ignoring projections):

$\text{Emp}_{\text{salary}}[x; 25 \leq \text{age} \leq 35] \bowtie_{x.\text{TID}=e.\text{TID}} \text{Emp}[e]$

For the join operator, the *pointer-based join* implementation developed in the context of object-oriented databases may be the most efficient way to evaluate

ToDo

the access path [804]. Obviously, sorting the result of the index scan on the tuple identifiers can speed up processing since it turns random into sequential I/O. However, this destroys the order on the key which might itself be useful later on during query processing or required by the query⁷. Sorting the tuple identifiers was proposed by, e.g., Yao [957], Makinouchi, Tezuka, Kitakami, and Adachi in the context of RDB/V1 [578]. The different variants (whether or not and where to sort, join order) can now be transparently determined by the plan generator: no special treatment is necessary. Further, the join predicates can not only be on the tuple identifiers but also on key attributes. This often allows to join with other than the indexed relations (or their indexes) before accessing the relation.

Rosenthal and Reiner proposed to use joins to represent access paths with indexes [739]. This approach is very elegant since no special treatment for index processing is required. However, if there are many relations and indexes, the search space might become very large, as every index increases the number of joins to be performed. This is why Mohan, Haderle, Wang, and Cheng abandoned this approach and sketched a heuristics which determines an access path in case multiple indexes on a single table exist [625].

The query

```
select  name, age
from    Person
where   name like 'R%' and age between 40 and 50
```

is an index only query (assuming indexes on **name** and **age**) and can be translated to

$$\Pi_{\text{name, age}}(\text{Emp}_{\text{age}}[a; 40 \leq \text{age} \leq 50; \text{TIDa, age}] \bowtie_{\text{TIDa}=\text{TIDn}} \text{Emp}_{\text{name}}[n; \text{name} \geq 'R'; \text{name} \leq 'R'; \text{TIDn, name}])$$

Let us now discuss the former of the two issues mentioned in the section's introduction. The query

```
select  *
from    Emp e, Dept d
where   e.name = 'Maier' and e.dno = d.dno
```

can be directly translated to

$$\sigma_{e.\text{name}='Maier'}(\text{Emp}[e]) \bowtie_{e.\text{dno}=d.\text{dno}} \text{Dept}[d]$$

If there are indexes on **Emp.name** and **Dept.dno**, we can replace $\sigma_{e.\text{name}='Maier'}(\text{Emp}[e])$ by an index scan as we have seen previously:

$$\chi_{e:*(x.\text{TID}), A(\text{Emp}):e.*}(\text{Emp}_{\text{name}}[x; \text{name} = 'Maier'])$$

⁷Restoring the order may be cheaper than typical sorting since tuples can be numbered before the first sort on tuple identifiers, and this dense numbering leads to efficient sort algorithms.

Here, $\mathcal{A}(\text{Emp}) : t.*$ abbreviates access to all **Emp** attributes. This especially includes **dno:t.dno**. (Strictly speaking, we do not have to access the **name** attribute, since its value is already known.)

As we have also seen, an alternative is to use a d-join instead:

$$\text{Emp}_{\text{name}}[x; \text{name} = \text{'Maier'}] < \chi_{t:*(x.TID), \mathcal{A}(e)t.*}(\square) >$$

Let us abbreviate $\text{Emp}_{\text{name}}[x; \text{name} = \text{'Maier'}]$ by E_i and $\chi_{t:*(x.TID), \mathcal{A}(e)t.*}(\square)$ by E_a .

Now, for any **e.dno**, we can use the index on **Dept.dno** to access the according department tuple:

$$E_i < E_a > < \text{Dept}_{\text{dno}}[y; y.dno = dno] >$$

Note that the inner expression $\text{Dept}_{\text{dno}}[y; y.dno = dno]$ contains the free variable **dno**, which is bound by E_a . Dereferencing the TID of the department results in the following algebraic modelling which models a complete *index nested loop join*:

$$E_i < E_a > < \text{Dept}_{\text{dno}}[y; y.dno = \text{dno}; \text{dTID} : y.TID] > < \chi_{u:*\text{dTID}, \mathcal{A}(\text{Dept})u.*}(\square) >$$

Let us abbreviate $\text{Dept}_{\text{dno}}[y; y.dno = \text{dno}; \text{dTID} : y.TID]$ by D_i and $\chi_{u:*\text{dTID}, \mathcal{A}(\text{Dept})u.*}(\square)$ by D_a . Fully abbreviated, the expression then becomes

$$E_i < E_a > < D_i > < D_a >$$

Several optimizations can possibly be applied to this expression. Sorting the *outer* of a d-join is useful under several circumstances since it may

- turn random I/O into sequential I/O and/or
- avoid reading the same page twice.

In our example expression,

- we can sort the result of expression E_i on **TID** in order to turn random I/O into sequential I/O, if there are many employees named “Maier”.
- we can sort the result of the expression $E_i < E_a >$ on **dno** for two reasons:
 - If there are duplicates for **dno**, i.e. there are many employees named “Maier” in each department, then this guarantees that no index page (of the index **Dept.dno**) has to be read more than once.
 - If additionally **Dept.dno** is a clustered index or **Dept** is an index-only table contained in **Dept.dno**, then large parts of the random I/O can be turned into sequential I/O.
 - If the result of the inner is materialized (see below), then only one result needs to be stored. Note that sorting is not necessary, but grouping would suffice to avoid duplicate work.

- We can sort the result of the expression $E_i < E_a > < D_i >$ on dTID for the same reasons as mentioned above for sorting the result of E_i on TID.

The reader is advised to explicitly write down the alternatives. Another exercise EX is to give plan alternatives for the different cases of DB2's Hybrid Join [322] which can now be decomposed into primitives like relation scan, index scan, d-join, sorting, TID dereferencing, and access to a unique index (see below).

Let us take a closer look at materializing the result of the inner of the d-join. IBM's DB2 for MVS considers temping (i.e. creating a temporary relation) the inner if it is an index access [322]. Graefe provides a general discussion on the subject [351]. Let us start with the above example. Typically, many employees will work in a single department and possibly several of them are called "Maier". For everyone of them, we can be sure that there exists at most one department. Let us assume that referential integrity has been specified. Then, there exists exactly one department for every employee. We have to find a way to rewrite the expression

$$E_i < E_a > < \text{Dept}_{\text{dno}}[y; y.\text{dno} = \text{dno}; \text{dTID} : y.\text{TID}] >$$

such that the mapping $\text{dno} \longrightarrow \text{dTID}$ is explicitly materialized (or, as one could also say, *cached*). For this purpose, Hellerstein and Naughton introduced a modified version of the map operator that materializes its result [414]. Let us denote this operator by χ^{mat} . The advantage of using this operator is that it is quite general and can be used for different purposes (see e.g. [103], Chap. ??, Chap. ??). Since the map operator extends a given input tuple by some attribute values, which must be computed by an expression, we need one to express the access to a unique index. For our example, we write

$$\text{IdxAcc}_{\text{dno}}^{\text{Dept}}[y; y.\text{dno} = \text{dno}]$$

to express the lookup of a single (unique) entry in the index on Dept.dno . We assume that the result is a (pointer to the) tuple containing the key attributes and all data attributes including the TID of some tuple. Then, we have to perform a further attribute access (dereferenciation) if we are interested in only one of the attributes.

Now, we can rewrite the above expression to

$$E_i < E_a > < \chi_{\text{dTID}:(\text{IdxAcc}_{\text{dno}}^{\text{Dept}}[y; y.\text{dno}=\text{dno}])\text{TID}}^{\text{mat}}(\square) >$$

If we further assume that the outer $(E_i < E_a >)$ is sorted on dno , then it suffices to remember only the TID for the latest dno . We define the map operator $\chi^{\text{mat},1}$ to do exactly this. A more efficient plan could thus be

$$\text{Sort}_{\text{dno}}(E_i < E_a >) < \chi_{\text{dTID}:(\text{IdxAcc}_{\text{dno}}^{\text{Dept}}[y; y.\text{dno}=\text{dno}])\text{TID}}^{\text{mat},1}(\square) >$$

where, strictly speaking, sorting is not necessary: grouping would suffice.

Consider a general expression of the form $e_1 < e_2 >$. The free variables used in e_2 must be a subset of the variables (attributes) produced by e_1 , i.e.

$\mathcal{F}(e_2) \subseteq \mathcal{A}(e_1)$. Even if e_1 does not contain duplicates, the projection of e_1 on $\mathcal{F}(e_2)$ may contain duplicates. If so, materialization could pay off. However, in general, for every binding of the variables $\mathcal{F}(e_2)$, the expression e_2 may produce several tuples. This means that using χ^{mat} is not sufficient. Consider the query

```
select  *
from    Emp e, Wine w
where   e.yearOfBirth = w.year
```

If we have no indexes, we can answer this query by a simple join where we only have to decide the join method and which of the relations becomes the outer and which the inner. Assume we have only wines from a few years. (Alternatively, some selection could have been applied.) Then it might make sense to consider the following alternative:

$$\text{Wine}[w] < \sigma_{e.\text{yearOfBirth}=w.\text{year}}(\text{Emp}[e]) >$$

However, the relation **Emp** is scanned once for each **Wine** tuple. Hence, it might make sense to materialize the result of the inner for every **year** value of **Wine** if we have only a few **year** values. In other words, if we have many duplicates for the **year** attribute of **Wine**, materialization may pay off since then we have to scan **Emp** only once for each **year** value of **Wine**. To achieve caching of the inner, in case every binding of its free variables possibly results in many tuples, requires a new operator. Let us call this operator *memox* and denote it by $\mathfrak{M}$ [351, 103]. For the free variables of its only argument, it remembers the set of result tuples produced by its argument expression and does not evaluate it again if it is already cached. Using memox, the above plan becomes

$$\text{Wine}[w] < \mathfrak{M}(\sigma_{e.\text{yearOfBirth}=w.\text{year}}(\text{Emp}[e])) >$$

It should be clear that for more complex inner, the memox operator can be applied at all branches, giving rise to numerous caching strategies. Analogously to the materializing map operator, we are able to restrict the materialization to the results for a single binding for the free variables if the outer is sorted (or grouped) on the free variables:

$$\text{Sort}_{w.\text{yearOfBirth}}(\text{Wine}[w]) < \mathfrak{M}^1(\sigma_{e.\text{yearOfBirth}=w.\text{year}}(\text{Emp}[e])) >$$

Things can become even more efficient if there is an index on **Emp.yearOfBirth**:

$$\begin{aligned} & \text{Sort}_{w.\text{yearOfBirth}}(\text{Wine}[w]) \\ & < \mathfrak{M}^1(\text{Emp}_{\text{yearOfBirth}}[x; x.\text{yearOfBirth} = w.\text{year}] < \chi_{e:*(x.\text{TID}), \mathcal{A}(\text{Emp}):*e}(\square) >) > \end{aligned}$$

So far we have seen different operators which materialize values: **Tmp**, $\mathfrak{M}$, and χ_{mat} . The latter in two variants. As an exercise, the reader is advised to discuss the differences between them. EX

Assume, we have indexes on both **Emp.yearOfBirth** and **Wine.year**. Besides the possibilities to use either **Emp** or **Wine** as the outer, we now also have the possibility to perform a join on the indexes before accessing the actual **Emp**

and `Wine` tuples. Since the index scan produces its output ordered on the key attributes, a simple merge join suffices (and we are back at the latter):

$$\text{Emp}_{\text{yearOfBirth}}[x] \bowtie_{x.\text{yearOfBirth}=y.\text{year}}^{\text{merge}} \text{Wine}_{\text{year}}[y]$$

This example makes clear that the order provided by an index scan can be used to speed up join processing. After evaluating this plan fragment, we have to access the actual `Emp` and `Wine` tuples. We can consider zero, one, or two sorts on their respective tuple identifiers. If the join is sufficiently selective, one of these alternatives may prove more sufficient than the ones we have considered so far.

EX

4.15 Remarks on Access Path Generation

A last kind of optimization we briefly want to mention is *sideways information passing*. Consider a simple join between two relations: $R \bowtie_{R.a=S.b} S$. If we decide to perform a sort merge join or a hash join, we can implement it by first sorting/partitioning R before looking at S . While doing so, we can remember the minimum and maximum value of $R.a$ and use these as a restriction on S such that fewer tuples of S have to be sorted/partitioned. In case we perform a blockwise nested loop join, after the first scan of S we know the minimum and maximum value of $S.b$ and can use these to restrict R .

If the number of distinct values of $R.a$ is small, we could also decide to remember all these values and evaluate perform a semi-join before the actual join. Algebraically, this could be expressed as

$$R \bowtie_{R.a=S.b} (S \ltimes_{S.b=R.a} \Pi_{R.a}(R))$$

An alternative is to use a bitmap to represent the projection of R on a .

The semi-join technique should be well-known from distributed database systems. In deductive database systems, this kind of optimization often carries the attribute *magic*. We will more deeply discuss this issue in Chapter ??.

The following problem is not discussed in the book. Assume that we have fully partitioned a relation vertically into a set of files which are chronologically ordered. Then, the attribute a_i of the j -th tuple can be found at the j -th position of the i -th file. This organization is called *partitioned transposed file* [56]. (Compare this with variant (projection) indexes [651] and small materialized aggregates [614].) The problem is to find an access strategy to all the attribute required by the query given a collection of restriction on some of the relation's attributes. This problem has been discussed in depth by Batory [56]. Full vertical partitioning is also used as the organizing principle of Monet [?]. Lately, it also gained some interest in the US [?].

4.16 Counting the Number of Accesses

4.16.1 Counting the Number of Direct Accesses

After the index scan, we have a set of (distinct) tuple identifiers for which we have to access the original tuples. The question we would like to answer is:

How many pages do we have to read?

Let R be the relation for which we have to retrieve the tuples. Then we use the following abbreviations

N	$ R $	number of tuples in the relation R
m	$ R $	number of pages on which tuples of R are stored
B	N/m	number of tuples per page (<i>blocking factor</i>)
k		number of (distinct) TIDs for which tuples have to be retrieved

We assume that the tuples are uniformly distributed among the m pages. Then, each page stores $B = N/m$ tuples. B is called *blocking factor*.

Let us consider some borderline cases. If $k > N - N/m$ or $m = 1$, then all pages are accessed. If $k = 1$ then exactly one page is accessed. The answer to the general question will be expressed in terms of *buckets* (pages in the above case) and *items* contained therein (tuples in the above case). Later on, we will also use extents, cylinders, or tracks as buckets and tracks or sectors/blocks as items.

We assume that a bucket contains items. The total number of items will be N and the number of requested items will be k . The above question can then be reformulated to how many buckets contain at least one of the k requested items, i.e. how many qualifying buckets exist. We start out by investigating the case where the items are uniformly distributed among the buckets. Two subcases will be distinguished:

1. k distinct items are requested
2. k non-distinct items are requested.

We then discuss the case where the items are non-uniformly distributed.

In any case, the underlying access model is random access. For example, given a tuple identifier, we can directly access the page storing the tuple. Other access models are possible. The one we will subsequently investigate is sequential access where the buckets have to be scanned sequentially in order to find the requested items. After that, we are prepared to develop a model for disk access costs.

Throughout this section, we will further assume that the probability that we request a set with k items is $\frac{1}{\binom{N}{k}}$ for all of the $\binom{N}{k}$ possibilities to select a k -set.⁸ We often make use of established equalities for binomial coefficients. For convenience, the most frequently used equalities are listed in Appendix D.

Selecting k distinct items

Our first theorem was discovered independently by Waters [913] and Yao [954]. We formulate it in terms of buckets containing items. We say a bucket *qualifies* if it contains at least one of the k items we are looking for.

⁸A k -set is a set with cardinality k .

Theorem 4.16.1 (Waters/Yao) Consider m buckets with n items each. Then there is a total of $N = nm$ items. If we randomly select k distinct items from all items, then the number of qualifying buckets is

$$\bar{\mathcal{Y}}_n^{N,m}(k) = m * \mathcal{Y}_n^N(k) \quad (4.2)$$

where $\mathcal{Y}_n^N(k)$ is the probability that a bucket contains at least one item. This probability is equal to

$$\mathcal{Y}_n^N(k) = \begin{cases} [1 - p] & k \leq N - n \\ 1 & k > N - n \end{cases}$$

where p is the probability that a bucket contains none of the k items. The following alternative expressions can be used to calculate p :

$$p = \frac{\binom{N-n}{k}}{\binom{N}{k}} \quad (4.3)$$

$$= \prod_{i=0}^{k-1} \frac{N - n - i}{N - i} \quad (4.4)$$

$$= \prod_{i=0}^{n-1} \frac{N - k - i}{N - i} \quad (4.5)$$

The second expression (4.4) is due to Yao, the third (4.5) is due to Waters. Palvia and March proved both formulas to be equal [669] (see also [39]). The fraction $m = N/n$ may not be an integer. For these cases, it is advisable to have a Gamma-function based implementation of binomial coefficients at hand (see [702] for details).

Depending on k and n , either the expression of Yao or the one of Waters is faster to compute. After the proof of the above formulas and the discussion of some special cases, we will give several approximations for p .

Proof The total number of possibilities to pick the k items from all N items is $\binom{N}{k}$. The number of possibilities to pick k items from all items not contained in a fixed single bucket is $\binom{N-n}{k}$. Hence, the probability p that a bucket does not qualify is $p = \binom{N-n}{k} / \binom{N}{k}$. Using this result, we can do the following calculation

$$\begin{aligned} p &= \frac{\binom{N-n}{k}}{\binom{N}{k}} \\ &= \frac{(N-n)! \, k! (N-k)!}{k! ((N-n)-k)! \, N!} \\ &= \prod_{i=0}^{k-1} \frac{N-n-i}{N-i} \end{aligned}$$

which proves the second expression. The third follows from

$$\begin{aligned}
 p &= \frac{\binom{N-n}{k}}{\binom{N}{k}} \\
 &= \frac{(N-n)! \, k!(N-k)!}{k!((N-n)-k)! \, N!} \\
 &= \frac{(N-n)! \, (N-k)!}{N! \, ((N-k)-n)!} \\
 &= \prod_{i=0}^{n-1} \frac{N-k-i}{N-i}
 \end{aligned}$$

□

Let us list some special cases:

If	then $\mathcal{Y}_m^N(k) =$
$n = 1$	k/N
$n = N$	1
$k = 0$	0
$k = 1$	$B/N = (N/m)N = 1/m$
$k = N$	1

We examine a slight generalization of the first case in more detail. Let N items be distributed over N buckets such that every bucket contains exactly one item. Further let us be interested in a subset of m buckets ($1 \leq m \leq N$). If we pick k items, then the number of buckets within the subset of size m that qualify is

$$m\mathcal{Y}_1^N(k) = m \frac{k}{N} \quad (4.6)$$

In order to see that the two sides are equal, we perform the following calculation:

$$\begin{aligned}
 \mathcal{Y}_1^N(k) &= \left(1 - \frac{\binom{N-1}{k}}{\binom{N}{k}}\right) \\
 &= \left(1 - \frac{\frac{(N-1)!}{k!((N-1)-k)!}}{\frac{N!}{k!(N-k)!}}\right) \\
 &= \left(1 - \frac{(N-1)!k!(N-k)!}{N!k!((N-1)-k)!}\right) \\
 &= \left(1 - \frac{N-k}{N}\right) \\
 &= \left(\frac{N}{N} - \frac{N-k}{N}\right) \\
 &= \frac{N - N + k}{N} \\
 &= \frac{k}{N}
 \end{aligned}$$

Since the computation of $\mathcal{Y}_n^N(k)$ can be quite expensive, several approximations have been developed. The first one was given by Waters [912, 913]:

$$p \approx (1 - k/N)^n$$

This approximation (also described elsewhere [317, 669]) turns out to be pretty good. However, below we will see even better approximations.

For $\overline{\mathcal{Y}}_n^{N,m}(k)$ Whang, Wiederhold, and Sagalowicz gave the following approximation for faster calculation [925]:

$$m * [(1 - (1 - 1/m)^k) + (1/(m^2 n) * k(k-1)/2 * (1 - 1/m)^{k-1}) + (1.5/(m^3 n^4) * k(k-1)(2k-1)/6 * (1 - 1/m)^{k-1})]$$

A rough estimate is presented by Bernstein, Goodman, Wong, Reeve, and Rothnie [78]:

$$\overline{\mathcal{Y}}_n^{N,m}(k) \approx \begin{cases} k & \text{if } k < \frac{m}{2} \\ \frac{k+m}{2} & \text{if } \frac{m}{2} \leq k < 2m \\ m & \text{if } 2m \leq k \end{cases}$$

An interesting and useful result was derived by Dihr and Saharia [241]. They give two formulas and show that they are lower and upper bounds to Water and Yao's formula. The upper and lower bounds for p are

$$\begin{aligned} p_{\text{lower}} &= \left(1 - \frac{k}{N - \frac{n-1}{2}}\right)^n \\ p_{\text{upper}} &= \left(\left(1 - \frac{k}{N}\right) * \left(1 - \frac{k}{N - n + 1}\right)\right)^{n/2} \end{aligned}$$

for $n = N/m$. Dihr and Saharia claim that the maximal difference resulting from the use of the lower and the upper bound to compute the number of page accesses is 0.224—far less than a single page access.

Selecting k non-distinct items

So far, we assumed that we retrieve k *distinct* items. We could ask the same question for k *non-distinct* items. This question demands a different urn model. In urn model terminology, the former case is an urn model with a *non-replacement* assumption, while the latter case is one with a *replacement* assumption. (Deeper insight into urn models is given by Drmota, Gardy, and Gittenberger [248].)

Before presenting a theorem discovered by Cheung [174], we repeat a theorem from basic combinatorics. We know that the number of subsets of size k of a set with N elements is $\binom{N}{k}$. The following lemma gives us the number of k -multisets⁹ (see, e.g. [829]). The number of k -multisets taken from a set S with $|S|$ elements is denoted by $\left(\left(\begin{smallmatrix} N \\ k \end{smallmatrix}\right)\right)$.

⁹A k -multiset is a multiset with k elements.

Lemma 4.16.2 *Let S be a set with $|S| = N$ elements. Then, the number of multisets with cardinality k containing only elements from S is*

$$\left(\binom{N+k-1}{k} \right) = \binom{N+k-1}{k}$$

For a proof we just note that there is a bijection between the k -multisets and the k -subsets of a $N+k-1$ -set. We can go from a multiset to a set by f with $f(\{x_1 \leq \dots \leq x_k\}) = \{x_1+0 < x_2+1 < \dots < x_k+(k-1)\}$ and from a set to a multiset via g with $g(\{x_1 < \dots < x_k\}) = \{x_1-0 < x_2-1 < \dots < x_k-(k-1)\}$.

Theorem 4.16.3 (Cheung) *Consider m buckets with n items each. Then there is a total of $N = nm$ items. If we randomly select k not necessarily distinct items from all items, then the number of qualifying buckets is*

$$\overline{Cheung}_n^{N,m}(k) = m * Cheung_n^N(k) \quad (4.7)$$

where

$$Cheung_n^N(k) = [1 - \tilde{p}] \quad (4.8)$$

with the following equivalent expressions for $\tilde{p}$:

$$\tilde{p} = \frac{\binom{N-n+k-1}{k}}{\binom{N+k-1}{k}} \quad (4.9)$$

$$= \prod_{i=0}^{k-1} \frac{N-n+i}{N+i} \quad (4.10)$$

$$= \prod_{i=0}^{n-1} \frac{N-1-i}{N-1+k-i} \quad (4.11)$$

Eq. 4.9 follows from the observation that the probability that some bucket does not contain any of the k possibly duplicate items is $\frac{\binom{N-n+k-1}{k}}{\binom{N+k-1}{k}}$. Eq. 4.10 follows from

$$\begin{aligned} \tilde{p} &= \frac{\binom{N-n+k-1}{k}}{\binom{N+k-1}{k}} \\ &= \frac{(N-n+k-1)!}{k!} \frac{k!((N+k-1)-k)!}{((N-n+k-1)-k)! (N+k-1)!} \\ &= \frac{(N-n-1+k)! (N-1)!}{(N-n-1)! (N-1+k)!} \\ &= \prod_{i=0}^{k-1} \frac{N-n+i}{N+i} \end{aligned}$$

Eq. 4.11 follows from

$$\begin{aligned}
\tilde{p} &= \frac{\binom{N-n+k-1}{k}}{\binom{N+k-1}{k}} \\
&= \frac{(N-n+k-1)! \cdot k!((N+k-1)-k)!}{k!((N-n+k-1)-k)! \cdot (N+k-1)!} \\
&= \frac{(N+k-1-n)! \cdot (N-1)!}{(N+k-1)! \cdot (N-1-n)!} \\
&= \prod_{i=0}^{n-1} \frac{N-n+i}{N+k-n+i} \\
&= \prod_{i=0}^{n-1} \frac{N-1-i}{N-1+k-i}
\end{aligned}$$

□

Cardenas discovered a formula that can be used to approximate $\tilde{p}$ [125]:

$$(1 - n/N)^k$$

As Cheung pointed out, we can use the theorem to derive the number of distinct items accessed contained in a k -multiset.

Corollary 4.16.4 *Let S be a k -multiset containing elements from an N -set T . Then the number of distinct items contained in S is*

$$\mathcal{D}(N, k) = \frac{Nk}{N+k-1} \quad (4.12)$$

if the elements in T occur with the same probability in S .

We apply the theorem for the special case where every bucket contains exactly one item ($n = 1$). In this case, $\prod_{i=0}^0 \frac{N-1-i}{N-1+k-i} = \frac{N-1}{N-1+k}$. And the number of qualifying buckets is $N(1 - \frac{N-1}{N-1+k}) = N(\frac{N-1+k-N+1}{N-1+k}) = N\frac{k}{N+k-1}$. □

Another way to achieve this formula is the following. There are $\binom{N}{l}$ possibilities to pick l different elements out of the N elements in T . In order to build a k -multiset with l different elements, we must additionally choose $n-l$ elements from the l elements. Thus, we have $\binom{N}{l} \left(\binom{l}{n-l} \right)$ possibilities to build a k -multiset. The total number of multisets is $\left(\binom{N}{l} \right)$. Thus we may conclude that

$$\mathcal{D}(N, k) = l \sum_{l=1}^{\min(N, k)} \frac{\binom{N}{l} \left(\binom{l}{n-l} \right)}{\left(\binom{N}{l} \right)}$$

which can be simplified to the above.

A useful application of this formula is to calculate the size of a projection [174]. Another use is that calculating the number of distinct values contained in a multiset allows us to shift from the model with replacement to a model without replacement. However, there is a difference between

$$\overline{\mathcal{Y}}_n^{N,m}(\text{Distinct}(N, k)) \approx \overline{\text{Cheung}}_n^{N,m}(k)$$

even when computing $\overline{\mathcal{Y}}$ with Eq. 4.5. Nonetheless, for $n \geq 5$, the error is less than two percent. One of the problems when calculating the result of the left-hand side is that the number of distinct items is not necessarily an integer. To solve this problem, we can implement all our formulas using the Gamma-function. But even then a small difference remains.

The approximation given in Theorem 4.16.3 is not too accurate. A better approximation can be calculated from the probability distribution. Denote by $p(\mathcal{D}(N, k) = j)$ the probability that the number of distinct values if we randomly select k items with replacement from N given items equals j . Then

$$p(\mathcal{D}(N, k) = j) = \binom{N}{j} \sum_{l=0}^j j(-1)^k \binom{j}{l} ((j-l)/N)^k$$

and thus

$$\mathcal{D}(N, k) = \sum_{j=1}^{\min(N,k)} j \binom{N}{j} \sum_{l=0}^j j(-1)^k \binom{j}{l} ((j-l)/N)^k$$

This formula is quite intense to calculate. We can derive a very good approximation by the following reasoning. We draw k elements from the set T with $|T| = N$ elements. Every element from T can be drawn at most k times. We produce N buckets, one for each element of T . In each bucket, we insert k copies of the according element from t . Then, a sequence of draws from T with duplicates can be represented by a sequence of draws without duplicate by mapping them to different copies. Thus, the first occurrence is mapped to the first element in the according bucket, the second one to the second copy and so on. Then, we can apply formula by Waters and Yao to calculate the number of buckets (and hence elements of T) hit:

$$\mathcal{D}(N, k) = \overline{\mathcal{Y}}_N^{Nk,k}(k)$$

Since the approximation is quite accurate and we already know how to efficiently calculate this formula, this is our method of choice.

Non-Uniform Distribution of Items

In the previous sections, we assumed that

1. every page contains the same number of records, and
2. every record is accessed with the same probability.

We now turn to relax the first assumption. Christodoulakis models the distribution by m numbers n_i (for $1 \leq i \leq m$) if there are m buckets. Each n_i equals the number of records in some bucket i [177]. Luk proposes Zipfian record distribution [571]. However, Ijbema and Blanken say that Water and Yao's formula is still better, as Luk's formula results in too low values [440]. They all come up with the same general formula presented below. Vander Zander, Taylor, and Bitton [968] discuss the problem of correlated attributes which results in some clusteredness. Zahorjan, Bell, and Sevcik discuss the problem where every item is assigned its own access probability [967]. That is, they relax the second assumption. We will come back to these issues in Section ??.

We still assume that every item is accessed with the same probability. However, we relax the first assumption. The following formula derived by Christodoulakis [177], Luk [571], and Ijbema and Blanken [440] is a simple application of Waters's and Yao's formula to a more general case.

Theorem 4.16.5 (Yao/Waters/Christodoulakis) *Assume a set of m buckets. Each bucket contains $n_j > 0$ items ($1 \leq j \leq m$). The total number of items is $N = \sum_{j=1}^m n_j$. If we look up k distinct items, then the probability that bucket j qualifies is*

$$\mathcal{W}_{n_j}^N(k, j) = \left[1 - \frac{\binom{N-n_j}{k}}{\binom{N}{k}}\right] \quad (= \mathcal{Y}_{n_j}^N(k)) \quad (4.13)$$

and the expected number of qualifying buckets is

$$\overline{\mathcal{W}}_{n_j}^{N,m}(k) := \sum_{j=1}^m \mathcal{W}_{n_j}^N(k, j) \quad (4.14)$$

Note that the product formulation in Eq. 4.5 of Theorem 4.16.1 results in a more efficient computation. We make a note of this in the following corollary.

Corollary 4.16.6 *Assume a set of m buckets. Each bucket contains $n_j > 0$ items ($1 \leq j \leq m$). The total number of items is $N = \sum_{j=1}^m n_j$. If we look up k distinct items, then the expected number of qualifying buckets is*

$$\overline{\mathcal{W}}_{n_j}^{N,m}(k) = \sum_{j=1}^m (1 - p_j) \quad (4.15)$$

with

$$p_j = \begin{cases} \prod_{i=0}^{n_j-1} \frac{N-k-i}{N-i} & k \leq n_j \\ 0 & N - n_j < k \leq N \end{cases} \quad (4.16)$$

If we compute the p_j after we have sorted the n_j in ascending order, we can use the fact that

$$p_{j+1} = p_j * \prod_{i=n_j}^{n_{j+1}-1} \frac{N-k-i}{N-i}.$$

We can also use the theorem to calculate the number of qualifying buckets in case the distribution is given by a histogram.

Corollary 4.16.7 *For $1 \leq i \leq L$ let there be l_i buckets containing n_i items. Then the total number of buckets is $m = \sum_{i=1}^L l_i$, and the total number of items in all buckets is $N = \sum_{i=1}^L l_i n_i$. For k randomly selected items, the number of qualifying buckets is*

$$\overline{W}_{n_j}^{N,m}(k) = \sum_{i=1}^L l_i \mathcal{Y}_{n_j}^N(k) \quad (4.17)$$

Last in this section, let us calculate the probability distribution for the number of qualifying items within a bucket. The probability that $x \leq n_j$ items in a bucket j qualify can be calculated as follows. The number of possibilities to select x items in bucket n_j is $\binom{n_j}{x}$. The number of possibilities to draw the remaining $k - x$ items from the other buckets is $\binom{N-n_j}{k-x}$. The total number of possibilities to distribute k items over the buckets is $\binom{N}{k}$. This shows the following:

Theorem 4.16.8 *Assume a set of m buckets. Each bucket contains $n_j > 0$ items ($1 \leq j \leq m$). The total number of items is $N = \sum_{j=1}^m n_j$. If we look up k distinct items, the probability that x items in bucket j qualify is*

$$\mathcal{X}_{n_j}^N(k, x) = \frac{\binom{n_j}{x} \binom{N-n_j}{k-x}}{\binom{N}{k}} \quad (4.18)$$

Further, the expected number of qualifying items in bucket j is

$$\overline{\mathcal{X}}_{n_j}^{N,m}(k) = \sum_{x=0}^{\min(k, n_j)} x \mathcal{X}_{n_j}^N(k, x) \quad (4.19)$$

In standard statistics books the probability distribution $\mathcal{X}_{n_j}^N(k, x)$ is called *hypergeometric distribution*.

Let us consider the case where all n_j are equal to n . Then we can calculate the average number of qualifying items in a bucket. With $y := \min(k, n)$ we

have

$$\begin{aligned}
\overline{\mathcal{X}}_{n_j}^{N,m}(k) &= \sum_{x=0}^{\min(k,n)} x \mathcal{X}_n^N(k, x) \\
&= \sum_{x=1}^{\min(k,n)} x \mathcal{X}_n^N(k, x) \\
&= \frac{1}{\binom{N}{k}} \sum_{x=1}^y x \binom{n}{x} \binom{N-n}{k-x} \\
&= \frac{1}{\binom{N}{k}} \sum_{x=1}^y \binom{x}{1} \binom{n}{x} \binom{N-n}{k-x} \\
&= \frac{1}{\binom{N}{k}} \sum_{x=1}^y \binom{n}{1} \binom{n-1}{x-1} \binom{N-n}{k-x} \\
&= \frac{\binom{n}{1}}{\binom{N}{k}} \sum_{x=0}^{y-1} \binom{n-1}{0+x} \binom{N-n}{(k-1)-x} \\
&= \frac{\binom{n}{1}}{\binom{N}{k}} \binom{n-1+N-n}{0+k-1} \\
&= \frac{\binom{n}{1}}{\binom{N}{k}} \binom{N-1}{k-1} \\
&= n \frac{k}{N} = \frac{k}{m}
\end{aligned}$$

Let us consider the even more special case where every bucket contains a single item. That is, $N = m$ and $n_i = 1$. The probability that a bucket contains a qualifying item reduces to

$$\begin{aligned}
\mathcal{X}_1^N(k, x) &= \frac{\binom{1}{x} \binom{N-1}{k-1}}{\binom{N}{k}} \\
&= \frac{\binom{N-1}{k-1}}{\binom{N}{k}} \\
&= \frac{k}{N} \quad (= \frac{k}{m})
\end{aligned}$$

Since x can then only be zero or one, the average number of qualifying items a bucket contains is also $\frac{k}{N}$.

The formulas presented in this section can be used to estimate the number of block/page accesses in case of random direct accesses. As we will see next, other kinds of accesses occur and need different estimates.

4.16.2 Counting the Number of Sequential Accesses

Vector of Bits

When estimating seek costs, we need to calculate the probability distribution for the distance between two subsequent qualifying cylinders. We model the

situation as a bitvector of length B with b bits set to 1. Then B corresponds to the number of cylinders and a 1 indicates that a cylinder qualifies.

Theorem 4.16.9 *Assume a bitvector of length B . Within it b ones are uniformly distributed. The remaining $B - b$ bits are zero. Then the probability distribution of the number j of zeros*

1. *between two consecutive ones,*

2. *before the first one, and*

3. *after the last one*

is given by

$$\mathcal{B}_b^B(j) = \frac{\binom{B-j-1}{b-1}}{\binom{B}{b}} \quad (4.20)$$

A more general theorem (see Theorem 4.16.13) was first presented by Yao [955]. The above formulation is due to Christodoulakis [180].

To see why the formula holds, consider the total number of bitvectors having a one in position i followed by j zeros followed by a one. This number is $\binom{B-j-2}{b-2}$. We can choose $B - j - 1$ positions for i . The total number of bitvectors is $\binom{B}{b}$ and each bitvector has $b - 1$ sequences of the form that a one is followed by a sequence of zeros is followed by a one. Hence,

$$\begin{aligned} \mathcal{B}_b^B(j) &= \frac{(B-j-1)\binom{B-j-2}{b-2}}{(b-1)\binom{B}{b}} \\ &= \frac{\binom{B-j-1}{b-1}}{\binom{B}{b}} \end{aligned}$$

Part 1. of the theorem follows. To prove part 2., we count the number of bitvectors that start with j zeros before the first one. There are $B - j - 1$ positions left for the remaining $b - 1$ ones. Hence, the number of these bitvectors is $\binom{B-j-1}{b-1}$ and part 2 follows. Part 3 follows by symmetry.

We can derive a less expensive way to evaluate the formula for $\mathcal{B}_b^B(j)$ as

follows. For $j = 0$, we have $\mathcal{B}_b^B(0) = \frac{b}{B}$. If $j > 0$, then

$$\begin{aligned}
 \mathcal{B}_b^B(j) &= \frac{\binom{B-j-1}{b-1}}{\binom{B}{b}} \\
 &= \frac{\frac{(B-j-1)!}{(b-1)!((B-j-1)-(b-1))!}}{\frac{B!}{b!(B-b)!}} \\
 &= \frac{(B-j-1)! \, b!(B-b)!}{(b-1)!((B-j-1)-(b-1))! \, B!} \\
 &= b \frac{(B-j-1)! \, (B-b)!}{((B-j-1)-(b-1))! \, B!} \\
 &= b \frac{(B-j-1)! \, (B-b)!}{(B-j-b)! \, B!} \\
 &= \frac{b}{B-j} \frac{(B-j)! \, (B-b)!}{(B-b-j)! \, B!} \\
 &= \frac{b}{B-j} \prod_{i=0}^{j-1} \left(1 - \frac{b}{B-i}\right)
 \end{aligned}$$

This formula is useful when $\mathcal{B}_b^B(j)$ occurs in sums over j because we can compute the product incrementally.

Corollary 4.16.10 *Using the terminology of Theorem 4.16.9, the expected value for the number of zeros*

1. *before the first one,*
2. *between two successive ones, and*
3. *after the last one*

is

$$\bar{\mathcal{B}}_b^B = \sum_{j=0}^{B-b} j \mathcal{B}_b^B(j) = \frac{B-b}{b+1} \quad (4.21)$$

Let us calculate:

$$\begin{aligned}
\sum_{j=0}^{B-b} j \binom{B-j-1}{b-1} &= \sum_{j=0}^{B-b} (B - (B-j)) \binom{B-j-1}{b-1} \\
&= B \sum_{j=0}^{B-b} \binom{B-j-1}{b-1} - \sum_{j=0}^{B-b} (B-j) \binom{B-j-1}{b-1} \\
&= B \sum_{j=0}^{B-b} \binom{b-1+j}{b-1} - b \sum_{j=0}^{B-b} \binom{B-j}{b} \\
&= B \sum_{j=0}^{B-b} \binom{b-1+j}{j} - b \sum_{j=0}^{B-b} \binom{b+j}{b} \\
&= B \binom{(b-1) + (B-b) + 1}{(b-1) + 1} - b \binom{b + (B-b) + 1}{b+1} \\
&= B \binom{B}{b} - b \binom{B+1}{b+1} \\
&= (B - b \frac{B+1}{b+1}) \binom{B}{b}
\end{aligned}$$

With

$$\begin{aligned}
B - b \frac{B+1}{b+1} &= \frac{B(b+1) - (Bb+b)}{b+1} \\
&= \frac{B-b}{b+1}
\end{aligned}$$

the claim follows.

Corollary 4.16.11 *Using the terminology of Theorem 4.16.9, the expected total number of bits from the first bit to the last one, both included, is*

$$\bar{\mathcal{B}}_{tot}(B, b) = \frac{Bb+b}{b+1} \quad (4.22)$$

To see this, we subtract from B the average expected number of zeros between the last one and the last bit:

$$\begin{aligned}
B - \frac{B-b}{b+1} &= \frac{B(b+1)}{b+1} - \frac{B-b}{b+1} \\
&= \frac{Bb + B - B + b}{b+1} \\
&= \frac{Bb+b}{b+1}
\end{aligned}$$

An early approximation of this formula was discovered by Kollias [507].

Corollary 4.16.12 *Using the terminology of Theorem 4.16.9, the number of bits from the first one and the last one, both included, is*

$$\overline{\mathcal{B}}_{1\text{-span}}(B, b) = \frac{Bb - B + 2b}{b + 1} \quad (4.23)$$

We have two possibilities to argue here. The first subtracts from B the number of zeros at the beginning and the end:

$$\begin{aligned} \overline{\mathcal{B}}_{1\text{-span}}(B, b) &= B - 2\frac{B - b}{b + 1} \\ &= \frac{Bb + B - 2B + 2b}{b + 1} \\ &= \frac{Bb - B + 2b}{b + 1} \end{aligned}$$

The other possibility is to add the number of zeros between the first and the last one and the number of ones:

$$\begin{aligned} \overline{\mathcal{B}}_{1\text{-span}}(B, b) &= (b - 1)\overline{\mathcal{B}}_b^B + b \\ &= (b - 1)\frac{B - b}{b + 1} + \frac{b(b + 1)}{b + 1} \\ &= \frac{Bb - b^2 - B + b + b^2 + b}{b + 1} \\ &= \frac{Bb - B + 2b}{b + 1} \end{aligned}$$

EX or Cor? The number of bits from the first bit to the last one including both ... The distance between the first and the last one ...

Let us have a look at some possible applications of these formulas. If we look up one record in an array of B records and we search sequentially, how many array entries do we have to examine on average if the search is successful?

In [584] we find these formulas used for the following scenario. Let a file consist of B consecutive cylinders. We search for k different keys, all of which occur in the file. These k keys are distributed over b different cylinders. Of course, we can stop as soon as we have found the last key. What is the expected total distance the disk head has to travel if it is placed on the first cylinder of the file at the beginning of the search?

Another interpretation of these formulas can be found in [429, 585]. Assume we have an array consisting of B different entries. We sequentially go through all entries of the array until we have found all the records for b different keys. We assume that the B entries in the array and the b keys are sorted. Further, all b keys occur in the array. On the average, how many comparisons do we need to find all keys?

Vector of Buckets

A more general scenario is as follows. Consider a sequence of m buckets containing n_i items each. Yao [955] developed the following theorem.

Theorem 4.16.13 (Yao) *Consider a sequence of m buckets. For $1 \leq i \leq m$, let n_i be the number of items in a bucket i . Then there is a total of $N = \sum_{i=1}^m n_i$ items. Let $t_i = \sum_{l=0}^i n_l$ be the number of items in the first i buckets. If the buckets are searched sequentially, then the number of buckets that have to be examined until k distinct items have been found is*

$$\mathcal{C}_{n_i}^{N,m}(k, j) = \frac{\binom{t_j}{k} - \binom{t_{j-1}}{k}}{\binom{N}{k}} \quad (4.24)$$

Thus, the expected number of buckets that need to be examined in order to retrieve k distinct items is

$$\bar{\mathcal{C}}_{n_i}^{N,m}(k) = \sum_{j=1}^m j \mathcal{C}_{n_i}^{N,m}(k, j) = m - \frac{\sum_{j=1}^m \binom{t_{j-1}}{k}}{\binom{N}{k}} \quad (4.25)$$

Applications of this formula can be found in [177, 180, 584, 586, 880]. Manolopoulos and Kollias describe the analogue for the replacement model [584].

Lang, Driscoll, and Jou discovered a general theorem which allows us to estimate the expected number of block accesses for sequential search.

Theorem 4.16.14 (Lang/Driscoll/Jou) *Consider a sequence of N items. For a batched search of k items, the expected number of accessed items is*

$$A(N, k) = N - \sum_{i=1}^{N-1} \text{Prob}[Y \leq i] \quad (4.26)$$

where Y is a random variable for the last item in the sequence that occurs among the k items searched.

proof?

?

With the help of this theorem, it is quite easy to derive many average sequential accesses for different models.

Cor or EX?

4.16.3 Pointers into the Literature

Segments containing records can be organized differently. Records can be placed randomly in the segment, they can be ordered according to some key, or the segment is organized as a tree. Accordingly, the segment is called random, sequential, or tree-structure. From a segment, records are to be retrieved for a given bag of k keys. The general question then is: how many pages do we have to access? The answer depends on whether we assume the replacement or non-replacement model. Six cases occur. For sequential and tree-structured segments, it also makes sense to distinguish between successful, partially (un-) successful, and (totally) unsuccessful searches. These notions capture the different possibilities where for all, some, none of the k keys records are found. The following table provides some entry points into the literature. It is roughly organized around the above categories. (Remember that we discussed the random file organization at length in Section 4.16.1.)

	non-replacement	replacement
random	[174, 177, 571, 687, 925, 954]	[125, 177, 669, 687]
sequential	[63, 177, 527, 586, 669, 668, 810, 955]	[177, 527, 586, 810]
tree-structured	[527, 526, 586, 668, 692]	[527, 526, 586, 810]

4.17 Disk Drive Costs for N Uniform Accesses

The goal of this section is to derive estimates for the costs (time) for retrieving N cache-missed sectors of a segment S from disk. We assume that the N sectors are read in their physical order on disk. This can be enforced by the DBMS, by the operating system's disk scheduling policy (SCAN policy), or by the disk drive controller.

Remembering the description of disk drives, the total costs can be described as

$$\mathcal{C}_{\text{disk}} = \mathcal{C}_{\text{cmd}} + \mathcal{C}_{\text{seek}} + \mathcal{C}_{\text{settle}} + \mathcal{C}_{\text{rot}} + \mathcal{C}_{\text{headswitch}} \quad (4.27)$$

For brevity, we omitted the parameter N and the parameters describing the segment and the disk drive on which the segment resides. Subsequently, we devote a (sometimes tiny) section to each summand. Before that, we have to calculate the number of qualifying cylinders, tracks, and sectors. These numbers will be used later on.

4.17.1 Number of Qualifying Cylinders, Tracks, and Sectors

If N sectors are to be retrieved, we have to find the number of cylinders qualifying in an extent i . Let S_{sec} denote the total number of sectors our segment contains and $S_{\text{cpe}}(i) = L_i - F_i + 1$ be the number of cylinders of the extent. If the N sectors we want to retrieve are uniformly distributed among the S_{sec} sectors of the segment, the number of cylinders that qualifies in (F_i, L_i, z_i) is $S_{\text{cpe}}(i)$ times 1 minus the probability that a cylinder does not qualify. The probability that a cylinder does not qualify can be computed by deviding the total number of possibilities to chose the N sectors from sectors outside the cylinder by the total number of possibilities to chose N sectors from all S_{sec} sectors of the segment. Hence, the number of qualifying cylinders in the considered extent is:

$$Q_c(i) = S_{\text{cpe}}(i) \mathcal{Y}_{D_{\text{Zspc}}(i)}^{S_{\text{sec}}}(N) = S_{\text{cpe}}(i) \left(1 - \frac{\binom{S_{\text{sec}} - D_{\text{Zspc}}(i)}{N}}{\binom{S_{\text{sec}}}{N}} \right) \quad (4.28)$$

We could also have used Theorem 4.16.13.

Let us also calculate the number of qualifying tracks in a partion i . It can be calculated by $S_{\text{cpe}}(i) D_{\text{tpc}} (1 - \text{Prob}(\mathbf{a \ track \ does \ not \ qualify}))$. The probability that a track does not qualify can be computed by dividing the number of ways to pick N sectors from sectors not belonging to a track divided by the number of possible ways to pick N sectors from all sectors.

$$Q_t(i) = S_{\text{cpe}}(i) D_{\text{tpc}} \mathcal{Y}_{D_{\text{Zspt}}(i)}^{S_{\text{sec}}}(N) = S_{\text{cpe}}(i) D_{\text{tpc}} \left(1 - \frac{\binom{S_{\text{sec}} - D_{\text{Zspt}}(i)}{N}}{\binom{S_{\text{sec}}}{N}} \right) \quad (4.29)$$

Just for fun, we calculate the number of qualifying sectors of an extent in zone i . It can be approximated by

$$Q_s(i) = S_{\text{cpe}}(i) D_{\text{Zspc}}(i) \frac{N}{S_{\text{sec}}} \quad (4.30)$$

Since all $S_{\text{cpe}}(i)$ cylinders are in the same zone, they have the same number of sectors per track, and we could also use Waters/Yao to approximate the number of qualifying cylinders by

$$Q_c(i) = \overline{Y}_{D_{\text{Zspc}}(S_{\text{zone}}(i))}^{S_{\text{cpe}}(i) D_{\text{Zspc}}(S_{\text{zone}}(i)), S_{\text{cpe}}(i)}(Q_s(i)) \quad (4.31)$$

This is a good approximation, as long as $Q_s(i)$ is not too small (e.g. > 4).

4.17.2 Command Costs

The command costs $\mathcal{C}_{\text{cmd}}$ are easy to compute. Every read of a sector requires the execution of a command. Hence

$$\mathcal{C}_{\text{cmd}} = N D_{\text{cmd}}$$

estimates the total command costs.

4.17.3 Seek Costs

We give different alternative possibilities to estimate seek costs. We start with an upper bound by exploring Theorem 4.1.1. The first cylinder we have to visit requires a random seek with cost D_{avgseek} . (Well this does not really give us an upper bound. For a true upper bound we should use $D_{\text{seek}}(D_{\text{cyl}} - 1)$.) After that, we have to visit the remaining $Q_c(i) - 1$ qualifying cylinders. The segment spans a total of $S_{\text{last}}(S_{\text{ext}}) - S_{\text{first}}(1) + 1$ cylinders. Let us assume that the first qualifying cylinder is the first cylinder and the last qualifying cylinder is the last cylinder of the segment. Then applying Theorem 4.1.1 gives us the upper bound

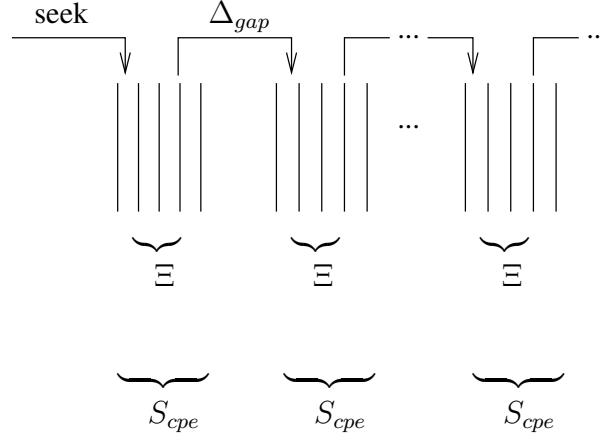
$$\mathcal{C}_{\text{seek}}(i) \leq (Q_c(i) - 1) D_{\text{seek}} \left(\frac{S_{\text{last}}(S_{\text{ext}}) - S_{\text{first}}(1) + 1}{Q_c(i) - 1} \right)$$

after we have found the first qualifying cylinder.

We can be a little more precise by splitting the seek costs into two components. The first component $\mathcal{C}_{\text{seekgap}}$ expresses the costs of finding the first qualifying cylinder and jumping from the last qualifying cylinder of extent i to the first qualifying cylinder of extent $i + 1$. The second component $\mathcal{C}_{\text{seekext}}(i)$ calculates the seek costs within an extent i . Figure 4.10 illustrates the situation. The total seek costs then are

$$\mathcal{C}_{\text{seek}}(i) = \mathcal{C}_{\text{seekgap}} + \sum_{i=1}^{S_{\text{ext}}} \mathcal{C}_{\text{seekext}}(i)$$

Since there is no estimate in the literature for $\mathcal{C}_{\text{seekgap}}$, we have to calculate it ourselves. After we have done so, we present several alternatives to calculate $\mathcal{C}_{\text{seekext}}(i)$.



The upper path illustrates $\mathcal{C}_{\text{seekgap}}$, the lower braces indicate those parts for which $\mathcal{C}_{\text{seekext}}$ is responsible.

Figure 4.10: Illustration of seek cost estimate

The average seek cost for reaching the first qualifying cylinder is D_{avgseek} . How far are we now within the first extent? We use Corollary 4.16.10 to derive that the number of non-qualifying cylinders preceding the first qualifying one in some extent i is

$$\bar{B}_{Q_c(i)}^{S_{\text{cpe}}(i)} = \frac{S_{\text{cpe}}(i) - Q_c(i)}{Q_c(i) + 1}.$$

The same is found for the number of non-qualifying cylinders following the last qualifying cylinder. Hence, for every gap between the last and the first qualifying cylinder of two extents i and $i + 1$, the disk arm has to travel the distance

$$\Delta_{\text{gap}}(i) := \bar{B}_{Q_c(i)}^{S_{\text{cpe}}(i)} + S_{\text{first}}(i + 1) - S_{\text{last}}(i) - 1 + \bar{B}_{Q_c(i+1)}^{S_{\text{cpe}}(i+1)}$$

Using this, we get

$$\mathcal{C}_{\text{seekgap}} = D_{\text{avgseek}} + \sum_{i=1}^{S_{\text{ext}}-1} D_{\text{seek}}(\Delta_{\text{gap}}(i))$$

Let us turn to $\mathcal{C}_{\text{seekext}}(i)$. We first need the number of cylinders between the first and the last qualifying cylinder, both included, in extent i . It can be calculated using Corollary 4.16.12:

$$\Xi_{\text{ext}}(i) = \bar{B}_{1\text{-span}}(S_{\text{cpe}}(i), Q_c(i))$$

Hence, $\Xi(i)$ is the minimal span of an extent that contains all qualifying cylinders.

Using $\Xi(i)$ and Theorem 4.1.1, we can derive an upper bound for $\mathcal{C}_{\text{seekext}}(i)$:

$$\mathcal{C}_{\text{seekext}}(i) \leq (Q_c(i) - 1) D_{\text{seek}}\left(\frac{\Xi(i)}{Q_c(i) - 1}\right) \quad (4.32)$$

Alternatively, we could formulate this as

$$\mathcal{C}_{\text{seekext}}(i) \leq (Q_c(i) - 1)D_{\text{seek}}(\overline{\mathcal{B}}_{Q_c(i)}^{S_{\text{cpe}}(i)}) \quad (4.33)$$

by applying Corollary 4.16.10.

A seemingly more precise estimate for the expected seek cost within the qualifying cylinders of an extent is derived by using Theorem 4.16.9:

$$\mathcal{C}_{\text{seekext}}(i) = Q_c(i) \sum_{j=0}^{S_{\text{cpe}}(i) - Q_c(i)} D_{\text{seek}}(j+1) \mathcal{B}_{Q_c(i)}^{S_{\text{cpe}}(i)}(j) \quad (4.34)$$

There are many more estimates for seek times. Older ones rely on a linear disk model but also consider different disk scan policies. A good entry point is the work by Theorey and Pinkerton [871, 872].

4.17.4 Settle Costs

The average settle cost is easy to calculate. For every qualifying cylinder, one head settlement takes place:

$$\mathcal{C}_{\text{settle}}(i) = Q_c(i)D_{\text{rdsettle}} \quad (4.35)$$

4.17.5 Rotational Delay Costs

Let us turn to the rotational delay. For some given track in zone i , we want to read the $Q_t(i)$ qualifying sectors contained in it. On average, we would expect that the read head is ready to start reading in the middle of some sector of a track. If so, we have to wait for $\frac{1}{2}D_{\text{Zscan}}(S_{\text{zone}}(i))$ before the first whole sector occurs under the read head. However, due to track and cylinder skew, this event does not occur after a head switch or a cylinder switch. Instead of being overly precise here, we ignore this half sector pass by time and assume that we are always at the beginning of a sector. This is also justified by the fact that we model the head switch time explicitly. EX

Assume that the head is ready to read at the beginning of some sector of some track. Then, in front of us is a — cyclic, which does not matter — bitvector of qualifying and non-qualifying sectors. We can use Corollary 4.16.11 to estimate the total number of qualifying and non-qualifying sectors that have to pass under the head until all qualifying sectors have been seen. The total rotational delay for the tracks of zone i is

$$\mathcal{C}_{\text{rot}}(i) = Q_t(i) D_{\text{Zscan}}(S_{\text{zone}}(i)) \overline{\mathcal{B}}_{\text{tot}}(D_{\text{Zspt}}(S_{\text{zone}}(i)), Q_{\text{spt}}(i)) \quad (4.36)$$

where $Q_{\text{spt}}(i) = \overline{W}_1^{S_{\text{sec}}, D_{\text{Zspt}}(S_{\text{zone}}(i))}(N) = D_{\text{Zspt}}(S_{\text{zone}}(i)) \frac{N}{S_{\text{sec}}}$ is the expected number of qualifying sectors per track in extent i . In case $Q_{\text{spt}}(i) < 1$, we set $Q_{\text{spt}}(i) := 1$.

A more precise model is derived as follows. We sum up for all j the product of (1) the probability that j sectors in a track qualify and (2) the average number of sectors that have to be read if j sectors qualify. This gives us the number of

sectors that have to pass the head in order to read all qualifying sectors. We only need to multiply this number by the time to scan a single sector and the number of qualifying tracks. We can estimate (1) using Theorem 4.16.8. For (2) we again use Corollary 4.16.11.

$$\begin{aligned} \mathcal{C}_{\text{rot}}(i) &= S_{\text{cpe}}(i) D_{\text{tpc}} D_{\text{Zscan}}(S_{\text{zone}}(i)) \\ &\quad \cdot \sum_{j=1}^{\min(N, D_{\text{Zspt}}(S_{\text{zone}}(i)))} \mathcal{X}_{D_{\text{Zspt}}(S_{\text{zone}}(i))}^{S_{\text{sec}}}(N, j) \bar{\mathcal{B}}_{\text{tot}}(D_{\text{Zspt}}(S_{\text{zone}}(i)), j) \end{aligned} \quad (4.37)$$

Another approach is taken by Triantafillou, Christodoulakis, and Georgiadis [880]. They split the total rotational delay into two components. The first component ($\mathcal{C}_{\text{rotpass}}$) measures the time needed to skip unqualifying sectors and the second ($\mathcal{C}_{\text{rotread}}$) that for (scanning and transferring) the qualifying sectors to the host.

Let us deal with the first component. Assume that j sectors of a track in extent i qualify. The expected position on a track where the head is ready to read is the middle between two qualifying sectors. Since the expected number of sectors between two qualifying sectors is $D_{\text{Zspt}}(S_{\text{zone}}(i))/j$, the expected number of sectors scanned before the first qualifying sector comes under the head is $\frac{D_{\text{Zspt}}(S_{\text{zone}}(i))}{2j}$. The expected positions of j qualifying sectors on the same track is such that the number of non-qualifying sectors between two successively qualifying sectors is the same. Hence, after having read a qualifying sector, $\frac{D_{\text{Zspt}}(S_{\text{zone}}(i))}{j}$ unqualifying sectors must pass by until the next qualifying sector shows up. The total number of unqualifying sectors to be passed if j sectors qualify in a track of zone i is

$$N_s(j, i) = \frac{D_{\text{Zspt}}(S_{\text{zone}}(i))}{2j} + (j-1) \frac{D_{\text{Zspt}}(S_{\text{zone}}(i)) - j}{j} \quad (4.38)$$

Using again Theorem 4.16.8, the expected rotational delay for the unqualifying sectors then is

$$\begin{aligned} \mathcal{C}_{\text{rotpass}}(i) &= S_{\text{cpe}}(i) D_{\text{tpc}} D_{\text{Zscan}}(S_{\text{zone}}(i)) \\ &\quad \cdot \sum_{j=1}^{\min(N, D_{\text{Zspt}}(S_{\text{zone}}(i)))} \mathcal{X}_{D_{\text{Zspt}}(S_{\text{zone}}(i))}^{S_{\text{sec}}}(N, j) N_s(j, i) \end{aligned} \quad (4.39)$$

We have to sum up this number for all extents and then add the time needed to scan the N sectors. Hence

$$\mathcal{C}_{\text{rot}} = \sum_{i=1}^{S_{\text{ext}}} \mathcal{C}_{\text{rotpass}}(i) + \mathcal{C}_{\text{rotread}}(i)$$

where the total transfer cost for the qualifying sectors of an extent can be estimated as

$$\mathcal{C}_{\text{rotread}}(i) = Q_s(i) D_{\text{Zscan}}(S_{\text{zone}}(i))$$

4.17.6 Head Switch Costs

The average head switch cost is equal to the average number of head switches that occur times the average head switch cost. The average number of head switch is equal to the number of tracks that qualify minus the number of cylinders that qualify since a head switch does not occur for the first track of each cylinder. Summarizing

$$C_{\text{headswitch}} = \sum_{i=1}^{S_{\text{ext}}} (Q_t(i) - Q_c(i)) D_{\text{hdswitch}} \quad (4.40)$$

where Q_t is the average number of tracks qualifying in an extent.

4.17.7 Discussion

The disk drive cost model derived depends on many parameters. The first bunch of parameters concerns the disk drive itself. These parameters can (and must be) extracted from disk drives by using (micro-) benchmarking techniques [311, 866, 605, 771]. The second bunch of parameters concerns the layout of a segment on disk. The database system is responsible for providing these parameters. The closer it is to the disk, the easier these parameters are extracted. Building a runtime system atop the operating system's file system is obviously a bad idea from the cost model perspective. If instead the storage manager of the runtime system implements cylinder aligned extents (or at least track aligned extents) using a raw I/O interface, the cost model will be easier to develop and much more precise. Again, providing reliable cost models is one of the most important tasks of the runtime system.

We have neglected many problems in our disk access model: partially filled cylinders, pages larger than a block, disk drive's cache, remapping of bad blocks, non-uniformly distributed accesses, clusteredness, and so on. Whereas the first two items are easy to fix, the rest is not so easy. In general, database systems ignore the disk drive cache. The justifying argument is that the database buffer is much larger than the disk drive's cache and, hence, it is very unlikely that we read a page that is not in the database buffer but in the disk cache. However, this argument falls short for non-random accesses. Nevertheless, we will ignore the issue in this book. The interested reader is referred to Shriver's thesis for disk cache modeling [811].

Remapping of bad sectors to other sectors really prevents the development of a precise cost model for disk accesses. Modelling disk drives becomes already a nightmare since a nice partitioning of the disk into zones is no longer possible since some sectors, tracks and even cylinders are reserved for the remapping. So even if no remapping takes place (which is very unlikely), having homogeneous zones of hundreds of cylinders is a dream that will never come true. The result is that we do not have dozens of homogeneous zones but hundreds (if not thousands) of zones of medium homogeneity. These should be reduced to a model of dozens of homogeneous zones such that the error does not become too large. The remaining issues will be discussed later in the book.

EX

There is even more to say about our cost model. A very practical issue arises if the number of qualifying cylinders is small. Then for some extent i , the expected number of qualifying cylinders could be $Q_c(i) = 0.38$. For some of our formulas this is a big problem. As a consequence, special cases for small N , small Q_c , small Q_t have to be developed and implemented.

Another issue is the performance of the cost model itself. The query compiler might evaluate the cost model's formulas thousands or millions of times. Hence, they should be fast to evaluate.

So far, we can adequately model the costs of N disk accesses. Some questions remain. For example, how do we derive the number N of pages we have to access? Do we really need to fetch all N pages from disk or will we find some of them in the buffer? If yes, how many? Further, CPU costs are also an important issue. Deriving a cost model for CPU costs is even more tedious than modelling disk drive costs. The only choice available is to benchmark all parts of a system and then derive a cost model using the extracted parameters. To give examples of parameters to be extracted: we need the CPU costs for accessing a page present in the buffer, for accessing a page absent in the buffer, for a next call of an algebraic operator, for executing an integer addition, and so on. Again, this cannot be done without tools [47, 244, 412, 463, 686].

The bottom line is that a cost model does not have to be accurate, but must lead to correct decisions. In that sense, it must be accurate at the break even points between plan alternatives. Let us illustrate this point by means of our motivating example. If we know that the index returns a single tuple, it is quite likely that the sequential scan is much more expensive. The same might be true for 2, 3, 4, and 5 tuples. Hence, an accurate model for small N is not really necessary. However, as we come close to the costs of a sequential scan, both the cost model for the sequential scan and the one for the index-based access must be correct since the product of their errors is the factor a bad choice is off the best choice. This is a crucial point, since it is easy to underestimate sequential access costs by a factor of 2-3 and overestimate random access cost by a factor of 2-5.

4.18 Concluding Remarks

Learned:

- Open Cost: I/O costs: non-uniform stuff, CPU costs: nothing done
- Wrong cardinality estimates: Open, leads to dynamic qo

4.19 Bibliography

ToDo:

- CPU Costs for B-tree search within inner and leaf pages [523]
- Index/Relations: only joins between building blocks [739]

- RDB/V1: predicate push down (views), 2 phase optimization (local: traditional, global: sharing of tables), five categories for predicates, nested loops evaluation for nested correlated subqueries, use of transitivity of equality, conjunctive normal form, use of min/max value of join column to reduce join cardinality by adding another selection to the other relation ($\min(a) \leq b \leq \max(a)$) for join predicate $a=b$.
- K accesses to unique index: how many page faults if buffer has size b? [752]
- buffer mgmt: [257]
- buffer mgmt: [809]
- buffer mgmt: [528]
- buffer mgmt: [753, 754]
- buffer mgmt: [109]
- buffer mgmt: [176]
- structured, semi-structured, unstructured data: [337] cited in Dono76
- B-trees and their improvements [213]
- Vertical partitioning: [261, 587, 56]
- Horizontal and vertical partitioning: [135]
- set oriented disk access to large complex objects [918, 917], assembly operator: [483],
- large objects: [89, 126, 542]

Part II

Foundations

Chapter 5

Logic, Null, and Boolean Expressions

5.1 Two-Valued Logic

The Boolean algebra with its operations *not* ($\neg$), *and* ($\wedge$), and *or* ($\vee$) is well-known. The truth tables for these operations is given in Figure 5.1. Figure 5.2 summarizes well-known laws for two-valued logic.

5.2 Null Values

Many database management systems (in particular all SQL-based relational systems but also object-oriented databases) support a special *NULL* value. It is used to express semantic concepts like *undefined*, *unknown* or *not applicable*. Although there exist proposals for supporting different NULL values for these different semantic concepts [?, ?], database management systems only support one NULL value.

This NULL value is a special value, distinguishable from all other values in a domain. That is, all the domains are extended by this very special value. This necessitates the definition of operations, functions, and comparison operators in case some argument is NULL.

5.2.1 Functions and Operators

If any of the arguments of a function or an operator is NULL, the result of the operator is typically also NULL. For example, in SQL every arithmetic operator

$\neg$	$true$	$false$	$\vee$	$true$	$false$	$\wedge$	$true$	$false$
	$false$	$true$	$true$	$true$	$true$	$true$	$true$	$false$
			$false$	$true$	$false$	$false$	$false$	$false$

Figure 5.1: Truth tables for two-valued logic

Commutativity

$$\begin{array}{llll}
p_1 \vee p_2 & \equiv & p_2 \vee p_1 & p_1 \wedge p_2 & \equiv & p_2 \wedge p_1 \\
\exists e_1 \exists e_2 p & \equiv & \exists e_2 \exists e_1 p & \forall e_1 \forall e_2 p & \equiv & \forall e_2 \forall e_1 p
\end{array}$$

Associativity

$$(p_1 \vee p_2) \vee p_3 \equiv p_1 \vee (p_2 \vee p_3) \quad (p_1 \wedge p_2) \wedge p_3 \equiv p_1 \wedge (p_2 \wedge p_3)$$

Distributivity

$$\begin{array}{llll}
p_1 \vee (p_2 \wedge p_3) & \equiv & (p_1 \vee p_2) \wedge (p_1 \vee p_3) & p_1 \wedge (p_2 \vee p_3) & \equiv & (p_1 \wedge p_2) \vee (p_1 \wedge p_3) \\
\exists e (p_1 \vee p_2) & \equiv & (\exists e p_1) \vee (\exists e p_2) & \forall e (p_1 \wedge p_2) & \equiv & (\forall e p_1) \wedge (\forall e p_2)
\end{array}$$

Idempotency

$$\begin{array}{llll}
p \vee p & \equiv & p & p \wedge p & \equiv & p \\
p \vee \neg p & \equiv & \text{true} (*) & p \wedge \neg p & \equiv & \text{false} (*) \\
p_1 \vee (p_1 \wedge p_2) & \equiv & p_1 & p_1 \wedge (p_1 \vee p_2) & \equiv & p_1 \\
p \vee \text{false} & \equiv & p & p \wedge \text{true} & \equiv & p \\
p \vee \text{true} & \equiv & \text{true} & p \wedge \text{false} & \equiv & \text{false}
\end{array}$$

De Morgan

$$\neg(p_1 \vee p_2) \equiv \neg(p_1) \wedge \neg(p_2) \quad \neg(p_1 \wedge p_2) \equiv \neg(p_1) \vee \neg(p_2)$$

Negation of Quantifiers

$$\neg(\forall e p) \equiv \exists e(\neg p) \quad \neg(\exists e p) \equiv \forall e(\neg p)$$

Elimination of Negation

$$\neg(\neg(p)) \equiv p \quad \neg t_1 \theta t_2 \equiv t_1 \bar{\theta} t_2$$

Conditioned Distributivity ($\mathcal{F}(p_1) \cap \mathcal{A}(e) = \emptyset$)

$$\begin{array}{llll}
p_1 \vee (\forall e p_2) & \equiv & \forall e (p_1 \vee p_2) & p_1 \wedge (\exists e p_2) & \equiv & \exists e (p_1 \wedge p_2) \\
p_1 \vee (\exists e p_2) & \equiv & \begin{cases} \exists e (p_1 \vee p_2) & \text{if } e \neq \{\} \\ p_1 & \text{if } e = \{\} \end{cases} & & & \\
p_1 \wedge (\forall e p_2) & \equiv & \begin{cases} \forall e (p_1 \wedge p_2) & \text{if } e \neq \{\} \\ p_1 & \text{if } e = \{\} \end{cases} & & &
\end{array}$$

Figure 5.2: Laws for two-valued logic

and function is defined that way. Thus, we have for example $0 * NULL = NULL$, although 0 could also be reasonable.

$x = y$			$x \doteq y$		
	x is null	x not null		x is null	x not null
y is null	$\perp$	$\perp$	y is null	<i>true</i>	<i>false</i>
y not null	$\perp$	$x = y$	y not null	<i>false</i>	$x = y$

Figure 5.3: Comparison functions in the presence of NULL values

5.2.2 Comparison Operators

NULL values stored directly in base tables thus typically propagate up through operators and function calls. Thus, comparison operators must deal with NULL values as input. Without NULL values as input, all comparison operators yield either *true* or *false* as output. In the presence of NULL values, a third value called *unknown* ($\perp$) is possible. How these *unknown* values are handled by standard logical operators is the topic of the next section. Here, we concentrate on the definition of comparison operators. Since there will be many of them with different semantics, we need some specific notation. First, we assume that $=^v$ is the standard value-based comparison operator for a given domain (e.g., integer or varchar) that cannot have NULL values as argument. The standard comparison operator $=^v$ can be extended in two ways to handle NULL values as input. The first is denoted by $=$ and has the same semantics as the equality in SQL. It returns any of *true*, *false*, $\perp$. It is defined in Figure 5.3. Note that it always evaluates to $\perp$, if at least one of its arguments is NULL. If none of the arguments are NULL, it behaves like a regular value comparison operator $=^v$. The second operator is $\doteq$. It is also defined in Figure 5.3. If both inputs are NULL, it returns *true*. If only one input is NULL, it returns *false*. If no input is NULL, it returns the result of the regular value comparison operator. Note, that $\doteq$ never returns $\perp$. In SQL predicates, $\doteq$ is called **is not distinct from**. Also in SQL, $\doteq$ is used for grouping and duplicate elimination.

Using $\neg$, we can define abbreviations for inequality:

$$\begin{aligned} x \neq y &:= \neg(a = b) \\ x \not\doteq y &:= \neg(a \doteq b) \end{aligned}$$

For other comparison operators $\theta \in \{\leq, \geq, <, >\}$, we treat their negation by defining an operator $\bar{\theta}$ that converts θ into its negated counterpart:

$$\begin{aligned} \bar{\leq} &:= > \\ \bar{\geq} &:= < \\ \bar{<} &:= \geq \\ \bar{>} &:= \leq \end{aligned}$$

Their semantics is defined analogously to that of equality, i.e., if at least one of their arguments is NULL, they return $\perp$, otherwise they return the result of their regular value comparison counterparts.

$\neg$	<i>true</i>	<i>false</i>	$\perp$
	<i>false</i>	<i>true</i>	$\perp$

$\vee$	<i>true</i>	<i>false</i>	$\perp$
<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>false</i>	$\perp$
$\perp$	<i>true</i>	$\perp$	$\perp$

$\wedge$	<i>true</i>	<i>false</i>	$\perp$
<i>true</i>	<i>true</i>	<i>false</i>	$\perp$
<i>false</i>	<i>false</i>	<i>false</i>	<i>false</i>
$\perp$	$\perp$	<i>false</i>	$\perp$

Figure 5.4: Truth tables for three-valued logic

5.3 Three-Valued Logic

As we have seen, comparison operators can have three possible outcomes (*true*, *false*, and $\perp$). A logic dealing with these three values is called *three-valued logic*. In this section, we review the operators of three-valued logic and give some useful laws. Figure 5.4 shows the extended truth-tables for the standard operators $\neg$, $\wedge$, and $\vee$. For completeness, we also define *implication* and *exclusive or*:

$$a \implies b := \neg a \vee b$$

$$a \dot{\vee} b := (a \vee b) \wedge \neg(a \wedge b)$$

While three-valued logic correctly captures the uncertainty caused by NULL values, and a result of $\perp$ can be reported back to the user as a boolean NULL, it is often necessary to convert a three-valued result into a two-valued one. Obviously, this can be done by converting $\perp$ to either *true* or *false*. This is called *true-interpreted* or *false-interpreted* $\perp$. Two operators $[\cdot]_{\perp}$ and $[\cdot]_{\perp}$ perform this conversion:

x	$[x]_{\perp}$	$[x]_{\perp}$
<i>true</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>false</i>	<i>false</i>
$\perp$	<i>true</i>	<i>false</i>

An example for false-interpreted $\perp$ values are **where** clauses in SQL: a given tuple only qualifies, if the predicate evaluates to *true*. An example for true-interpreted $\perp$ values are SQL **check** conditions: a constraint violation only occurs if the predicate in the **check** condition returns *false*.

In the following, we need to say that two expressions in three-valued logic are equivalent ($\equiv$). This is defined to be true if and only if for all assignments of variables found on the left-hand side and the right-hand side to constants/null-values/truth values including $\perp$, the evaluation of the right-hand side yields the same value (*true*, *false*, $\perp$) as the evaluation of the left-hand side.

Any database management system has a choice. Either

- work all the way with NULL values and three-valued logic or
- convert expressions in three-valued logic to two-valued logic.

For the former see the exercises. For the latter, we need to push down $[\cdot]_{\perp}$ and $[\cdot]_{\perp}$ and prepare at the bottom of our expressions. Pushing $[\cdot]_{\perp}$ and $[\cdot]_{\perp}$

x	$\neg x$	$\lfloor x \rfloor_{\perp}$	$\neg \lfloor x \rfloor_{\perp}$	$\lfloor \neg x \rfloor_{\perp}$	$\neg \lfloor \neg x \rfloor_{\perp}$	$\lceil x \rceil_{\perp}$	$\neg \lceil x \rceil_{\perp}$	$\lceil \neg x \rceil_{\perp}$	$\neg \lceil \neg x \rceil_{\perp}$
<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
$\perp$	$\perp$	<i>false</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>false</i>

Figure 5.5: True-/false-interpretation and Negation

down $\wedge$ and $\vee$ is rather easy:

$$\lfloor p_1 \wedge p_2 \rfloor_{\perp} \equiv \lfloor p_1 \rfloor_{\perp} \wedge \lfloor p_2 \rfloor_{\perp} \quad (5.1)$$

$$\lceil p_1 \wedge p_2 \rceil_{\perp} \equiv \lceil p_1 \rceil_{\perp} \wedge \lceil p_2 \rceil_{\perp} \quad (5.2)$$

$$\lfloor p_1 \vee p_2 \rfloor_{\perp} \equiv \lfloor p_1 \rfloor_{\perp} \vee \lfloor p_2 \rfloor_{\perp} \quad (5.3)$$

$$\lceil p_1 \vee p_2 \rceil_{\perp} \equiv \lceil p_1 \rceil_{\perp} \vee \lceil p_2 \rceil_{\perp} \quad (5.4)$$

However, we must be very careful with negation. A complete account of the situation is given in Fig. 5.5. From there, we see that

$$\lceil \neg x \rceil_{\perp} \equiv \neg \lfloor x \rfloor_{\perp} \quad (5.5)$$

$$\lfloor \neg x \rfloor_{\perp} \equiv \neg \lceil x \rceil_{\perp} \quad (5.6)$$

$$\lceil x \rceil_{\perp} \equiv \neg \lfloor \neg x \rfloor_{\perp} \quad (5.7)$$

$$\lfloor x \rfloor_{\perp} \equiv \neg \lceil \neg x \rceil_{\perp} \quad (5.8)$$

Using these equivalences, we can push down $\lfloor \cdot \rfloor_{\perp}$ and $\lceil \cdot \rceil_{\perp}$ until we meet some built-in predicate or boolean function. For built-in comparison operators, we can combine the false/true-interpretation with the operator yielding two additional comparison operators. As an example let us consider the equality operator. For it, we define two new equality operators, each combining $=$ with one possible interpretation of *unknown*:

$$e_1 =^- e_2 \quad := \quad \lfloor e_1 = e_2 \rfloor_{\perp}$$

$$e_1 =^+ e_2 \quad := \quad \lceil e_1 = e_2 \rceil_{\perp}$$

Analogously, we define for any comparison operator $\theta \in \{\leq, \geq, <, >, \neq\}$ two operators θ^- and θ^+ . For operators that do not yield *unknown*, we can eliminate the interpretation. These operators include for example **exists**, **match**, **is distinct from**, and **is null**. Thus, if we have an expression b that is guaranteed to evaluate only in *true* and *false* (and not $\perp$), then we have

$$\lceil b \rceil_{\perp} = b$$

$$\lfloor b \rfloor_{\perp} = b$$

If we take a careful look at Figure 5.2, we see that all equivalences except those marked by '*' hold for three-valued logic.

Exercise 1. Find a 2-bit encoding of the values *true*, *false*, $\perp$ which requires only one machine instruction to implement each of $\wedge$, $\vee$, and $\neg$.

Exercise 2. Manually build a truth table for $x_1 \implies x_2$ and $x_1 \dot{\vee} x_2$ in three-valued logic. Then check, whether the right-hand side of their definitions is equivalent with your truth table.

Exercise 3. Look for ways to move $\lceil \cdot \rceil_\perp$ and $\lfloor \cdot \rfloor_\perp$ down $x_1 \implies x_2$ and $x_1 \dot{\vee} x_2$.

5.4 Preparation of Boolean Expressions

Before any further optimization can take place, boolean expressions need to be preprocessed. The most important steps that are required to take place are **pushnot** and **pushunk**. Before we come to these steps, let us consider another trick that is commonly found. It is *partial evaluation*:

pareval If one term of a conjunction yields *false* then the other (not yet) evaluated terms are not evaluated. This is typically represented by cascading select operators, which can then be pushed down independently. Partial evaluation can also be applied to disjunctions if one factor evaluates to true.

Now consider $\neg(a \wedge b)$. Due to the negation, we cannot apply **pareval** without any precaution. Further, as we saw previously, negation swaps $\lfloor \cdot \rfloor_\perp$ and $\lceil \cdot \rceil_\perp$. Thus, if not handled very carefully, negation leads to all kinds of problems. Thus, the first step in the preparation of a predicate is to push negation down. After this step, we can push $\lfloor \cdot \rfloor_\perp$ and $\lceil \cdot \rceil_\perp$ down to convert three-valued logic expressions to two-valued logic expressions. Thus, we perform in this order:

pushnot push negation down

pushunk push $\lfloor \cdot \rfloor_\perp$ and $\lceil \cdot \rceil_\perp$ down

More on simple rewrites can be found in Chapter ??.

5.5 Equivalence Classes based on Equality

The traditional equality operator $=^v$ is reflexive, symmetric, and transitive. Hence, it induces equivalence classes of expressions which, in the context of predicate evaluations must evaluate to the same value. It is good practice in database systems, to collect equivalence classes. As a prerequisite, we need the notion of some expression e' occurring conjunctively in some predicate (boolean expression) e . A substitution is a partial mapping from subexpressions of some expression e to other expressions. Here, we are interested in ground substitutions where expressions are mapped to constants. A substitution is denoted by

$$[e_1/e'_1, \dots, e_k/e'_k]$$

where each e_i is mapped to e'_i . A substitution can be applied to some other expression and replaces each occurrence of e_i by e'_i . If p is some boolean expression, then application of a substitution to p is denoted by $p[e_1/e'_1, \dots, e_k/e'_k]$.

We define that e occurs conjunctively in p if and only if $p[e/false]$ is equivalent to $false$. This can be checked by applying the simplification rules (under idempotency in Fig. 5.2). If $p[e/false]$ simplifies to $false$, e occurs conjunctively in p . Now we can build equivalence classes induced by p by finding all expressions of the form $e_1 = e_2$ which occur conjunctively in p . Then, if for example two attributes A and B are in the same equivalence class, we can replace an occurrence of A by B .

To see an example, why this procedure might be very helpful consider two relations $R(A, B)$ and $S(C)$ and assume that we have to evaluate the following algebraic expression:

$$R \bowtie_{A=C \vee B=C} S$$

In this case, any efficient implementation of the join operator, like hash joins, does not work. In order to avoid a nested-loop evaluation of this expression, we rewrite it as follows:

$$(R \bowtie_{A=C} S) \cup (R \bowtie_{B=C \wedge A \neq C} S)$$

Note that the two arguments of the union operator are disjoint. Thus, we do not have a problem with duplicates. Now, we can apply a hash-join on either side. However, we have to evaluate $A \neq B$ as a residual predicate after the join. Noting that $B = C$ occurs conjunctively in $B = C \wedge A \neq C$, we can replace C by B in $A \neq C$. Then, we have

$$B = C \wedge A \neq C \equiv B = C \wedge A \neq B \quad (5.9)$$

Applying this equivalence to the above algebraic expression yields

$$(R \bowtie_{A=C} S) \cup (R \bowtie_{B=C \wedge A \neq B} S)$$

Now, $A \neq B$ is a selection predicate (sometimes called *restriction*, as it involves two attributes from the same relation) and we can rewrite our expression to

$$(R \bowtie_{A=C} S) \cup (\sigma_{A \neq B}(R) \bowtie_{B=C} S),$$

which is more efficient since we eliminate the R tuples before the join.

Now let us see what happens in the presence of NULL values and three-valued logic. At the core of our reasoning was Equivalence 5.9. For illustrating purposes, we assume that each of the attributes A , B , and C can have values 3, 7, and NULL. The following table contains all cases, where it *does not hold*. (The other cases are left to the reader.)

A	B	C	LHS	RHS
3	3	NULL	$\perp$	<i>false</i>
7	7	NULL	$\perp$	<i>false</i>
NULL	3	3	<i>false</i>	$\perp$
NULL	7	7	<i>false</i>	$\perp$

Thus, the equivalence, valid in two-valued logic does not hold in the presence of NULL values and three-valued logic. Predicates are typically true- or false-interpreted. If the predicate is true-interpreted, than the left-hand side and the

right-hand side of 5.9 clearly differ. However, if the predicate is false-interpreted we have that

$$\lfloor B = C \wedge A \neq C \rfloor_{\perp} \equiv \lfloor B = C \wedge A \neq B \rfloor_{\perp} \quad (5.10)$$

or, after pushing $\lfloor \cdot \rfloor_{\perp}$ down:

$$B =^{\neg} C \wedge A \neq^{\neg} C \equiv B =^{\neg} C \wedge A \neq^{\neg} B \quad (5.11)$$

To perform this kind of optimizations, we must be able to build equivalence classes in the presence of NULL values. First note, that $=$ is neither symmetric, reflexive nor transitive, as it might return $\perp$. Thus, the only way to produce equivalence classes is by exploiting conjunctively occurring $=^{\neg}$ expressions. We define that an expressions e occurs conjunctively in p if

1. $p[e/false] \equiv false$ and
2. $p[e/\perp] \equiv false$

Then, we can collect all conjunctively occurring expressions in $=^{\neg}$ and build equivalence classes based on them. Note that this requires that we pushed down $\lceil \cdot \rceil_{\perp}$ and $\lfloor \cdot \rfloor_{\perp}$ operations as otherwise there are no $=^{\neg}$ expressions.

5.6 Nullability Inference

5.7 Bibliography

NULL-values: [90, 546, 547, 747, 748]

Chapter 6

Functional Dependencies

In many query results attribute values are not independent of each other but have certain dependencies. Keeping track of these dependencies is very useful for many optimizations, for example in the following query

```
select  c.id, n.name
from    customers c, nations n
where   c.nid=n.id
order by c.id, n.name
```

the *order by* clause can be simplified to *c.id* without affecting the result: *c.id* is the key of *customers*, and thus determines *c.nid*. *c.nid* is joined with *n.id*, which is the key of *nations* and determines *n.name*, thus transitively *c.id* determines *n.name*.

These *functional dependencies* between attributes have been studied primarily in the context of database design, but many optimization steps like order optimization (Chapter 23) and query unnesting (Chapter 14) profit greatly from known functional dependencies. In the following we first study functional dependencies when all attributes are not NULL, then extend this to attributes with NULL values, and finally discuss how functional dependencies are effected by relational operators.

6.1 Functional Dependencies

As illustrated by the previous example, a functional dependency describes how attribute values depend on other attribute values. More formally, a relation R (with $A_1, A_2 \subseteq \mathcal{A}(R)$) satisfies a functional dependency

$$f : A_1 \rightarrow A_2$$

if and only if the following condition holds:

$$\forall t_1, t_2 ([t_1 \in R \wedge t_2 \in R \wedge t_1.A_1 = t_2.A_1] \Rightarrow [t_1.A_2 = t_2.A_2]).$$

For base relations functional dependencies can be derived from the schema, in particular key constraints and *check* conditions [680]. For intermediate results

additional function dependencies can be induced by algebraic operators, as we will see below.

Once some functional dependencies are known to hold, further functional dependencies can be derived by using Armstrong's axioms [?] (assuming $A_1, A_2, A_3 \subseteq \mathcal{A}(R)$):

1. $A_2 \subseteq A_1 \Rightarrow A_1 \rightarrow A_2$
2. $A_1 \rightarrow A_2 \Rightarrow (A_1 \cup A_3) \rightarrow (A_2 \cup A_3)$
3. $A_1 \rightarrow A_2 \wedge A_2 \rightarrow A_3 \Rightarrow A_1 \rightarrow A_3$

The Armstrong axioms are sound and complete, i.e., it is possible to derive all valid functional dependencies by applying these three axioms. For practical purposes it is often convenient to include three additional rules which can be derived from the original axioms:

4. $A_1 \rightarrow A_2 \wedge A_1 \rightarrow A_3 \Rightarrow A_1 \rightarrow (A_2 \cup A_3)$
5. $A_1 \rightarrow (A_2 \cup A_3) \Rightarrow A_1 \rightarrow A_2 \wedge A_1 \rightarrow A_3$
6. $A_1 \rightarrow A_2 \wedge (A_2 \cup A_4) \rightarrow A_3 \Rightarrow (A_1 \cup A_4) \rightarrow A_3$

Given a set of functional dependencies $\mathcal{F}$, we denote with $\mathcal{F}^+$ the *closure* of $\mathcal{F}$, i.e., the set of all functional dependencies that can be derived from $\mathcal{F}$ by using the inference rules shown above.

Closely related to the concept of functional dependencies is the concept of *keys*: Given a relation R and an attribute set $A \subseteq \mathcal{A}(R)$, A is a *super key* of R if $A \rightarrow \mathcal{A}(R)$ holds in R . Further A is a *key* of R if the following condition holds:

$$\forall A(A' \subset A \Rightarrow \neg(A' \rightarrow \mathcal{A}(R))).$$

6.2 Functional Dependencies in the presence of NULL values

In the presence of NULL values, a relation R (with $A_1, A_2 \subseteq \mathcal{A}(R)$) satisfies a functional dependency

$$f : A_1 \rightarrow A_2$$

if and only if the following condition holds:

$$\forall t_1, t_2 ([t_1 \in R \wedge t_2 \in R \wedge t_1 \cdot A_1 \doteq t_2 \cdot A_1] \Rightarrow [t_1 \cdot A_2 \doteq t_2 \cdot A_2]).$$

XXX explain why, discuss lax dependencies

6.3 Deriving Functional Dependencies over algebraic operators

XXX dependency graphs

6.4 Bibliography

Chapter 7

An Algebra for Sets, Bags, and Sequences

This section summarizes a logical algebra that is sufficient to express queries written in SQL, OQL and XPath/XQuery. The algebra is based upon substantial work by many people [68, 70, 189, 191, 196, 487, 488, 544, 820]. The most prominent features of the algebra are:

- All operators are polymorphic and can deal with (almost) any kind of complex arguments.
- The operators take arbitrary complex expressions as subscripts. This includes algebraic expressions. The advantage is that nested queries can directly be expressed as nested algebraic expressions and unnesting possibilities can be represented at the algebraic level, which allows rigorous correctness proofs.
- The algebra is redundant, since some special cases of the operators can be implemented more efficiently.

This chapter is organized as follows. First, we prepare some background material by discussing sets, bags, and sequences, as well as aggregation functions. Then we are ready to present the algebraic operators. This is done in two steps. First, we introduce their signatures and then their semantics.

ToDo

7.1 Sets, Bags, and Sequences

7.1.1 Sets

A set contains elements drawn from some domain $\mathcal{D}$. In our case, the domain will often be tuples and we only consider finite sets. The set operations we are interested in are union ($\cup_s$), intersection ($\cap_s$), and difference ($\setminus_s$). If the domain consists of tuples, we assume that both arguments have the same schema. That is, the attributes and their domains are the same in both arguments. Otherwise, the expression is not well-typed. In any case, set union and intersection are commutative and associative. Set difference is neither of them. Expressions

$X \cup_s \emptyset$	$= X$	
$X \cup_s X$	$= X$	(idempotency)
$X \cup_s Y$	$= Y \cup_s X$	(commutativity)
$(X \cup_s Y) \cup_s Z$	$= X \cup_s (Y \cup_s Z)$	(associativity)
$X \cap_s \emptyset$	$= \emptyset$	
$X \cap_s X$	$= X$	(idempotency)
$X \cap_s Y$	$= Y \cap_s X$	(commutativity)
$(X \cap_s Y) \cap_s Z$	$= X \cap_s (Y \cap_s Z)$	(associativity)
$X \setminus_s \emptyset$	$= X$	
$\emptyset \setminus_s X$	$= \emptyset$	
$X \setminus_s X$	$= \emptyset$	
$X \setminus_s Y$	$\neq Y \setminus_s X$	(wrong)
$(X \setminus_s Y) \setminus_s Z$	$\neq X \setminus_s (Y \setminus_s Z)$	(wrong)
$X \cap_s Y$	$= X \setminus_s (X \setminus_s Y)$	
$X \cup_s (Y \cap_s Z)$	$= (X \cup_s Y) \cap_s (X \cup_s Z)$	(distributivity)
$X \cap_s (Y \cup_s Z)$	$= (X \cap_s Y) \cup_s (X \cap_s Z)$	(distributivity)
$(X \cup_s Y) \setminus_s Z$	$= (X \setminus_s Z) \cup_s (Y \setminus_s Z)$	(distributivity)
$(X \cap_s Y) \setminus_s Z$	$= (X \setminus_s Z) \cap_s (Y \setminus_s Z)$	(distributivity)
$X \setminus_s (Y \cup_s Z)$	$= (X \setminus_s Y) \cap_s (X \setminus_s Z)$	
$X \setminus_s (Y \cap_s Z)$	$= (X \setminus_s Y) \cup_s (X \setminus_s Z)$	

Figure 7.1: Laws for Set Operations

containing the empty set can be simplified. Last but not least, some distributivity laws hold. These and other laws for set operation (see Fig. 7.1) should be well-known.

A set of elements from a domain $\mathcal{D}$ can be seen as a function from $\mathcal{D}$ to $\{0, 1\}$. For a given set S , this function is called the *characteristic function* of S . It can be defined as $\chi_S(s) = \begin{cases} 0 & \text{if } s \notin S \\ 1 & \text{if } s \in S \end{cases}$. Obviously, there is a bijection between characteristic functions and sets. That is, sets can be characterized by their characteristic functions, and the set operations can be expressed in terms of operations on characteristic functions.

EXC

In the presence of null values, we have to be a little careful to evaluate an expression like $x \in S$. Assume x is null and S contains some element y which is also null. Then, we would like to have that $x \in S$ and x is equal to y . Thus, we must use ‘ $\doteq$ ’. Set equality can be expressed as equality of characteristic functions. The subset relationship $A \subseteq B$ can be expressed as $\chi_A(x) \leq \chi_B(x)$ for all x . The *cardinality* $|S|$ for a set S is defined as $\sum_x \chi_S(x)$. Because we deal with finite sets only, cardinality is well-defined. A *singleton set* is a set containing only one element, i.e., a set whose cardinality equals 1.

As we have seen in Chapter 2, algebraic equivalences that reorder algebraic operators form the fundamental basis for query optimization. One could discuss the reorderability of each pair of operators resulting in n^2 investigations if the number of operators in the algebra is n . In order to simplify this tedious task, we introduce a general argument covering most of the cases. The observation

will be that *set-linearity* of set operators implies their reorderability easily.

A unary function f from sets to sets is called *set-linear* (or *homomorph*), if and only if the following two conditions hold for all sets X and Y :

$$\begin{aligned} f(\emptyset) &= \emptyset, \\ f(X \cup_s Y) &= f(X) \cup_s f(Y). \end{aligned}$$

An n -ary mapping from sets to a set is called *set-linear in its i -th argument*, if and only if for all sets $X_1, \dots, X_n$ and X'_i the following conditions hold:

$$\begin{aligned} f(X_1, \dots, X_{i-1}, \emptyset, X_{i+1}, \dots, X_n) &= \emptyset, \\ f(X_1, \dots, X_{i-1}, X_i \cup X'_i, X_{i+1}, \dots, X_n) &= f(X_1, \dots, X_{i-1}, X_i, X_{i+1}, \dots, X_n) \\ &\quad \cup_s f(X_1, \dots, X_{i-1}, X'_i, X_{i+1}, \dots, X_n). \end{aligned}$$

It is called *set-linear*, if it is set-linear in all its arguments. For a binary function or operator where we can distinguish between the left and the right argument, we call it left (right) set-linear if it is set-linear in its first (second) argument. Note that if an equivalence with linear mappings on both sides has to be proven, it suffices to proof it for singleton sets, i.e. sets with one element only.

Using the commutativity of set union and set intersection as well as the observations above, we see that for a non-empty set X

$$\begin{aligned} (\emptyset \cup_s X) &\neq \emptyset, \\ (\emptyset \cap_s X) &= \emptyset, \\ (\emptyset \setminus_s X) &= \emptyset, \\ (X \setminus_s \emptyset) &\neq \emptyset, \end{aligned}$$

and for arbitrary sets X , Y , and Z

$$\begin{aligned} (X \cup_s Y) \cup_s Z &= (X \cup_s Z) \cup_s (Y \cup_s Z), \\ (X \cup_s Y) \cap_s Z &= (X \cap_s Z) \cup_s (Y \cap_s Z), \\ (X \cup_s Y) \setminus_s Z &= (X \setminus_s Z) \cup_s (Y \setminus_s Z), \\ X \setminus_s (Y \cup_s Z) &\neq (X \setminus_s Y) \cup_s (X \setminus_s Z). \end{aligned}$$

We can conclude that set union is neither left nor right set-linear, set intersection is set-linear, and set difference is left set-linear but not right set-linear.

7.1.2 Duplicate Data: Bags

A *bag* or *multiset* can contain every element more than once. It cannot contain an element less than zero times. A typical bag is $\{a, b, b\}_b$, for which we also write $\{a^1, b^2\}_b$. Another example is $\{a, b\}_b$. The latter bag does not contain any duplicates. Hence, it could also be considered a set. We will only consider finite bags.

For a given bag B , the characteristic function for bags maps every element of a domain D to the set of non-negative integers $\mathbb{N}_0$. The characteristic function gives the number of occurrences of each element in the bag. The number

$X \cup_b \emptyset_b$	$= X$	
$X \cup_b X$	$\neq X$	(wrong)
$X \cup_b Y$	$= Y \cup_b X$	(commutativity)
$(X \cup_b Y) \cup_b Z$	$= X \cup_b (Y \cup_b Z)$	(associativity)
$X \cap_b \emptyset_b$	$= \emptyset_b$	
$X \cap_b X$	$= X$	(idempotency)
$X \cap_b Y$	$= Y \cap_b X$	(commutativity)
$(X \cap_b Y) \cap_b Z$	$= X \cap_b (Y \cap_b Z)$	(associativity)
$X \setminus_b \emptyset_b$	$= X$	
$\emptyset_b \setminus_b X$	$= \emptyset_b$	
$X \setminus_b X$	$= \emptyset_b$	
$X \setminus_b Y$	$\neq Y \setminus_b X$	(wrong)
$(X \setminus_b Y) \setminus_b Z$	$\neq X \setminus_b (Y \setminus_b Z)$	(wrong)
$X \cap_b Y$	$= X \setminus_b (X \setminus_b Y)$	
$X \cup_b (Y \cap_b Z)$	$= (X \cup_b Y) \cap_b (X \cup_b Z)$	(distributivity)
$X \cap_b (Y \cup_b Z)$	$\neq (X \cap_b Y) \cup_b (X \cap_b Z)$	(wrong)
$(X \cup_b Y) \setminus_b Z$	$\neq (X \setminus_b Z) \cup_b (Y \setminus_b Z)$	(wrong)
$(X \cap_b Y) \setminus_b Z$	$= (X \setminus_b Z) \cap_b (Y \setminus_b Z)$	(distributivity)
$X \setminus_b (Y \cup_b Z)$	$\neq (X \setminus_b Y) \cap_b (X \setminus_b Z)$	(wrong)
$X \setminus_b (Y \cap_b Z)$	$\neq (X \setminus_b Y) \cup_b (X \setminus_b Z)$	(wrong)

Figure 7.2: Laws for Bag Operations

of occurrences of some element x in a bag B is $\chi_B(x)$, and we call this the *multiplicity* of x . We often denote the multiplicity of an element by a superscript as in $\{x^{77}\}_b$, where the element x has multiplicity 77. Again, there is a bijection between bags and their characteristic functions. We use $\in$ to denote bag membership. Given a bag B and its characteristic function χ_B , we have $x \in B \prec \succ \chi_B(x) > 0$. If we use $\in$ within a bag constructor, as in $\{x|x \in B\}_b$, x iterates over all elements in B . This means, that if some element has multiplicity m , then x iterates over m duplicates of this element. In order to determine it for a given bag, we must have an equality defined on the items in the bag. Here, we have to use $\doteq$, which reflects the semantics of SQL. Thus, in $\{\text{null}, \text{null}, \text{null}\}_b$ the multiplicity of null is 3. It would be bad to have three nulls with multiplicity 1 each.

Equality on bags is defined as equality of their characteristic functions. Sub-bag relationships can be defined using the characteristic function. For example, $A \subseteq B$ can be defined as $\chi_A(x) \leq \chi_B(x)$ for all x . The cardinality $|B|$ for a bag B is defined as $\sum_x \chi_B(x)$. Because we deal with finite bags only, cardinality is well-defined. A *bag* B *containing a single element* is one whose characteristic function equals 0 for all but one element x . A *singleton bag* is one whose cardinality equals 1.

The bag union $X \cup_b Y$ of two bags is defined such that the number of occurrences of some element in the union is the sum of its occurrences in X and Y . The number of occurrences of some element in the bag intersection $X \cap_b Y$

is the minimum of the number of its occurrences in X and Y . In the bag difference $X \setminus_b Y$, the number of occurrences of some element is the difference ($\dot{-}$) of its occurrences in X and Y , where $a \dot{-} b$ is defined as $\max(0, a - b)$. Using characteristic functions, we can define

$$\begin{aligned}\chi_{X \cup_b Y}(z) &= \chi_X(z) + \chi_Y(z) \\ \chi_{X \cap_b Y}(z) &= \min(\chi_X(z), \chi_Y(z)) \\ \chi_{X \setminus_b Y}(z) &= \chi_X(z) \dot{-} \chi_Y(z)\end{aligned}$$

The laws for sets do not necessarily hold for bags (see Figure 7.2). We have that bag union and bag intersection are both commutative and associative. Bag difference is neither of them. Let us take a closer look at the different distributivity laws. Therefore, denote by LHS the left-hand side of an equivalence and by RHS its right-hand side. Let us first prove

$$X \cup_b (Y \cap_b Z) = (X \cup_b Y) \cap_b (X \cup_b Z).$$

Since for all x we have

$$\begin{aligned}\chi_{\text{LHS}}(x) &= \chi_X(x) + \min(\chi_Y(x), \chi_Z(x)) \\ &= \min(\chi_X(x) + \chi_Y(x), \chi_X(x) + \chi_Z(x)) \\ &= \chi_{\text{RHS}}(x),\end{aligned}$$

the claim follows.

For the bags $X = \{1^5\}_b$, $Y = \{1^3\}_b$, and $Z = \{1^3\}_b$, we get

$$X \cap_b (Y \cup_b Z) = \{1^5\}_b \cap_b \{1^6\}_b = \{1^5\}_b,$$

but

$$(X \cap_b Y) \cup_b (X \cap_b Z) = \{1^3\}_b \cup_b \{1^3\}_b = \{1^6\}_b.$$

For the bags $X = \{1^5\}_b$, $Y = \{1^3\}_b$, and $Z = \{1^2\}_b$, we calculate

$$(X \cup_b Y) \setminus_b Z = \{1^8\}_b \setminus_b \{1^2\}_b = \{1^6\}_b,$$

but

$$(X \setminus_b Z) \cup_b (Y \setminus_b Z) = \{1^3\}_b \cup_b \{1^1\}_b = \{1^4\}_b.$$

Consider

$$(X \cap_b Y) \setminus_b Z = (X \setminus_b Z) \cap_b (Y \setminus_b Z).$$

This holds, since

$$\begin{aligned}\chi_{\text{LHS}}(x) &= \min(\chi_X(x), \chi_Y(x)) \dot{-} \chi_Z(x) \\ &= \min(\chi_X(x) \dot{-} \chi_Z(x), \chi_Y(x) \dot{-} \chi_Z(x)) \\ &= \chi_{\text{RHS}}(x).\end{aligned}$$

For the bags $X = \{1^2\}_b$, $Y = \{1^1\}_b$, and $Z = \{1^1\}_b$, we calculate

$$X \setminus_b (Y \cup_b Z) = \{1^2\}_b \setminus_b \{1^2\}_b = \emptyset_b,$$

but

$$(X \setminus_b Y) \cap_b (X \setminus_b Z) = \{1^1\}_b \cap_b \{1^1\}_b = \{1^1\}_b,$$

and

$$X \setminus_b (Y \cap_b Z) = \{1^2\}_b \setminus_b \{1^1\}_b = \{1^1\}_b,$$

but

$$(X \setminus_b Y) \cup_b (X \setminus_b Z) = \{1^1\}_b \cup_b \{1^1\}_b = \{1^2\}_b.$$

Remark. Our definition of bag union is not the usual definition. The standard set theoretic definition of the bag union operator $\cup_{\max}$ is defined such that

$$\chi_{X \cup_{\max} Y}(x) = \max(\chi_X(x), \chi_Y(x))$$

holds [21, 221]. With this definition, the laws for sets carry over to bags. We decided to use the non-standard definition, since this is the semantics of bag union in SQL and other query languages. Dayal, Goodman, and Katz [221] and Albert [21] also investigate the non-standard bag union in their papers, although under a different name. For example, Albert calls it *bag concatenation*. As a side remark, it is interesting to note that Albert showed that bag concatenation can not be expressed using $\cup_{\max}$, $\cap_b$, $\setminus_b$ [21]. Thus, any query language featuring $\cup_b$ is strictly more expressive, since $\cup_{\max}$ can be expressed using $\setminus_b$ and $\cup_b$ because the equivalence

$$X \cup_{\max} Y \equiv (X \setminus_b Y) \cup_b Y$$

holds. Two other laws involving $\cup_{\max}$ are

$$\begin{aligned} X \cup_{\max} Y &\equiv (X \cup_b Y) \setminus_b (X \cap_b Y), \\ X \cap_b Y &\equiv (X \cup_b Y) \setminus_b (X \cup_{\max} Y). \end{aligned}$$

We introduce linearity for bags in Sec. 7.4.

7.1.3 Explicit Duplicate Control

Having every operation twice, once for bags and once for sets is quite inconvenient. Fortunately, for some operations we only need the one for bags. We can get rid of some set operations as follows. Every set can be seen as a bag whose characteristic function never exceeds one. Let $\bar{I}(S)$ turn a set S into a bag with identical characteristic function. The partial function $\bar{I}^{-1}(B)$ turns a bag into a set if the bag's characteristic function does not exceed one. Otherwise let $\bar{I}^{-1}$ be undefined. Let X and Y be two sets. For the intersection function, we then have

$$\bar{I}^{-1}(\bar{I}(X) \cap_b \bar{I}(Y)) = X \cap_s Y.$$

That is, for any two sets X and Y bag intersection and set intersection are the same. Thus, we only need one intersection operation, which is defined on bags and which we will denote by $\cap$.

The above observation gives rise to the notion of *set-faithfulness*. We call a unary function on sets f *set-faithful* if and only if

$$\bar{I}^{-1}(f(\bar{I}(X))) = f(X)$$

holds for all sets X . Analogously, we call binary functions g *set-faithful* if and only if

$$\bar{I}^{-1}(g(\bar{I}(X), \bar{I}(Y))) = g(X, Y)$$

holds for all sets X and Y .

$\setminus_b$ and $\cap_b$ are set-faithful. Hence, we can (and often will) simply use $\setminus$ and $\cap$ to denote bag difference and intersection. If the arguments happen to be sets, the resulting bag will not contain any duplicates, i.e., it is a set.

Note that $\cup_b$ is not set-faithful. One possibility is to carefully distinguish between $\cup_b$ and $\cup_s$. However, this does not solve our problem for query processing. A relation can be a set (e.g. if a primary key is defined) or a bag. Assume we have two relations (or intermediate results) R_1 , which is a set, and R_2 , which is a bag. Obviously, $R_1 \cup_s R_2$ is not valid since R_2 is a bag. By treating sets as special bags, $R_1 \cup_b R_2$ is valid. However, we cannot control duplicates in the result as demanded by SQL, where there is a fundamental difference between **union all** and **union distinct**. We could thus use two different union operators. Both take bags as input but one preserves duplicates, as does the bag union, and the other eliminates duplicates. Let us denote the former by $\cup$ and the latter by $\cup^d$.

To go from a bag to a set, we have to eliminate duplicates. Let us denote by Π^D the duplicate elimination operation. For a given bag B , we then have $\chi_{\Pi^D(B)}(z) = \min(1, \chi_B(z))$. Using Π^D , we can define $\cup^d$ as

$$R_1 \cup^d R_2 := \Pi^D(R_1 \cup R_2).$$

However, the right-hand side is our preferred way to take care of duplicate handling: we will always use the bag operator, denoted by $\cup$ and then, if necessary, eliminate duplicates explicitly.

Summarizing, instead of working with sets and bags, we can work with bags only by identifying every set S with the bag $\bar{I}(S)$. To keep track of (possible) duplicates, we can annotate all bags with a property indicating whether it contains duplicates or not. If at some place a set is required and we cannot infer that the bag in that place is duplicate free, we can use Π^D as an enforcer of the set property. Note that for every set S we have $\Pi^D(S) = S$. Hence, Π^D does not do any harm except for the resources it takes. The reasoning whether a given expression produces duplicates or not is very important. Below, we will indicate on the fly how reasoning about duplicates can be performed.

7.1.4 Ordered Data: Sequences

A sequence is ordered and may contain duplicates. An example sequence is $\langle a, b, b, c, b \rangle$. The length of the sequence is the number of elements it contains. For any sequence S , the length of the sequence is denoted by $|S|$. The above sequence has length five. The empty sequence (ϵ) contains zero elements and has length zero.

As we consider only finite sequences, a sequence of length $n \geq 0$ has a characteristic function χ from an interval $[0, n[$ to a domain D . Outside $[0, n[$, χ is undefined ($\perp$). Let S be a sequence. Then $\alpha(S)$ gives us the first element of

the sequence, i.e., $\alpha(S) = \chi_S(0)$. For our example sequence, $\alpha(\langle a, b, b, c, b \rangle) = a$. The rest or tail of a sequence S of length n is denoted by $\tau(S)$ and contains all but the first element in the sequence. That is $\chi_{\tau(S)}(i) = \chi_S(i + 1)$. For our example sequence, $\tau(\langle a, b, b, c, b \rangle) = \langle b, b, c, b \rangle$.

Concatenation of two sequences is denoted by $\oplus$. The characteristic function of the concatenation of two sequences S and T is

$$\chi_{S \oplus T}(i) = \begin{cases} \chi_S(i) & \text{if } i < |S|, \\ \chi_T(i - |S|) & \text{if } i \geq |S|. \end{cases}$$

As an example, $\langle a, b, b, c, b \rangle \oplus \langle a, b, c \rangle = \langle a, b, b, c, b, a, b, c \rangle$.

We can easily go from a sequence to a bag by just forgetting the order. To convert a bag into a sequence, we typically have to apply a **Sort** operator. In reality, however, bags are often represented as (ordered) streams, i.e., they are sequences. This is due to the fact that most physical algebras are implemented using the iterator concept introduced in Section 4.6.

Analogously to set and bag linearity, we can introduce *sequence linearity* of unary and n-ary functions on sequences. In the definition, we only have to exchange the set union operator by concatenation. A unary function f from sequences to sequences is called *sequence-linear*, if and only if the following two conditions hold for all sequences X and Y :

$$\begin{aligned} f(\epsilon) &= \epsilon, \\ f(X \oplus Y) &= f(X) \oplus f(Y). \end{aligned}$$

An n -ary mapping from sequences to a sequence is called *sequence-linear in its i -th argument* if and only if for all sequences $X_1, \dots, X_n$ and X'_i the following conditions hold:

$$\begin{aligned} f(X_1, \dots, X_{i-1}, \epsilon, X_{i+1}, \dots, X_n) &= \epsilon \\ f(X_1, \dots, X_{i-1}, X_i \oplus X'_i, X_{i+1}, \dots, X_n) &= f(X_1, \dots, X_{i-1}, X_i, X_{i+1}, \dots, X_n) \\ &\quad \oplus f(X_1, \dots, X_{i-1}, X'_i, X_{i+1}, \dots, X_n) \end{aligned}$$

It is called *sequence-linear*, if it is sequence-linear in all its arguments. For a binary function or operator where we can distinguish between the left and the right argument, we call it left (right) sequence-linear if it is sequence-linear in its first (second) argument. Note that if an equivalence with linear mappings on both sides has to be proven, it suffices to proof it for singleton sequences, i.e. sequences with one element only.

7.2 Aggregation Functions

SQL and other query languages support at least five aggregation functions. These are *min*, *max*, *count*, *sum*, and *avg*. In addition, SQL allows to qualify whether duplicates are removed before computing the aggregate or whether they are also considered by the aggregation function. For example, we may specify **sum(distinct a)** or **sum(all a)** for some attribute a . The term **sum(a)** is

equivalent to **sum(all a)**. From this follows that aggregation functions can be applied to sets or bags. Other query languages (OQL and XQuery) also allow lists as arguments to aggregation functions. Additionally, OQL allows arrays. Hence, aggregation functions should be defined for any bulk type.

Most query languages provide a special null value. In SQL it is called NULL. Initially, OQL did not have a special null value. Fortunately, it was introduced in version 3.0. There, the null value is called UNKNOWN. So far, XQuery has no null value. Instead, the inventors of XQuery tried hard to let the empty sequence play a dual role: that of an empty sequence and that of a null value. Of course, this leads to awkward complications. We will use '⊥', ⊥, or NULL to represent a null value. From this variance, the reader can already imagine its importance.

Typically, aggregation functions can safely ignore null values. The only exception is **count(*)**, where all input elements are counted. If for some attribute a , we want to count only values of a with $a \neq \perp$, then we often use $\text{count}^{\text{NN}}(a)$ to emphasize this fact. The corresponding SQL function is $\text{count}(a)$.

Let x be a single value and $\{x\}$ a bag containing x only once. Since

$$\begin{aligned}\min(\{x\}) &= x, \\ \max(\{x\}) &= x, \\ \text{sum}(\{x\}) &= x, \\ \text{avg}(\{x\}) &= x,\end{aligned}$$

these aggregation functions behave like identity if we identify single elements with singleton bags.

If we identify a single value with a bag containing this single value once, we see that

$$\begin{aligned}\min(\min(X)) &= \min(X), \\ \max(\max(X)) &= \max(X), \\ \text{sum}(\text{sum}(X)) &= \text{sum}(X), \\ \text{avg}(\text{avg}(X)) &= \text{avg}(X),\end{aligned}$$

that is, these aggregation functions are idempotent.

Let $\mathcal{N}$ denote either a numeral data type (e.g. *integer* or *float*) or a tuple $[a_1 : \tau_1, \dots, a_n : \tau_n]$ where each type τ_n is a numeral data type. Further, let $\mathcal{N}$ contain the null value.

A *scalar* aggregation function agg is a function with signature

$$\text{agg} : \{\tau\}_b \rightarrow \mathcal{N}.$$

A scalar aggregation function $\text{agg} : \{\tau\}_b \rightarrow \mathcal{N}$ is called *decomposable* if there exist functions

$$\begin{aligned}\text{agg}^1 : \{\tau\}_b &\rightarrow \mathcal{N}', \\ \text{agg}^2 : \{\mathcal{N}'\}_b &\rightarrow \mathcal{N},\end{aligned}$$

with

$$\text{agg}(Z) = \text{agg}^2(\{\text{agg}^1(X), \text{agg}^1(Y)\}_b)$$

for all X and Y (not empty) with $Z = X \cup Y$. This condition assures that $\text{agg}(Z)$ can be computed on arbitrary subsets (-lists, -bags) of Z independently and the (partial) results can be aggregated to yield the correct total result. If the condition holds, we say that agg is *decomposable* with *inner* agg^1 and *outer* agg^2 .

A decomposable scalar aggregation function $\text{agg} : \{\tau\}_b \rightarrow \mathcal{N}$ is called *reversible* if for agg^O there exists a function $(\text{agg}^O)^{-1} : \mathcal{N}', \mathcal{N}' \rightarrow \mathcal{N}'$ with

$$\text{agg}(X) = \gamma((\text{agg}^O)^{-1}(\text{agg}^I(Z), \text{agg}^I(Y)))$$

for all X , Y , and Z with $Z = X \cup Y$. This condition assures that we can compute $\text{agg}(X)$ for a subset (-list, -bag) X of Z by “subtracting” its aggregated complement Y from the “total” $\text{agg}^O(\text{agg}^I(Z))$ by using $(\text{agg}^O)^{-1}$.

The fact that scalar aggregation functions can be decomposable and reversible is the basic observation upon which builds the efficient evaluation of aggregation functions.

As an example, consider the scalar aggregation $\text{avg} : \{[a : \text{float}]\}_b \rightarrow \text{float}$ averaging the values of the attributes a of a bag of tuples with a single attribute a . It is reversible with

$$\begin{aligned} \text{agg}^I : \{[a : \text{float}]\} &\rightarrow [\text{sum} : \text{float}, \text{count} : \text{float}], \\ \text{agg}^O : [\text{sum} : \text{float}, \text{count} : \text{float}], [\text{sum} : \text{float}, \text{count} : \text{float}] &\rightarrow [\text{sum} : \text{float}, \text{count} : \text{float}], \\ (\text{agg}^O)^{-1} : [\text{sum} : \text{float}, \text{count} : \text{float}], [\text{sum} : \text{float}, \text{count} : \text{float}] &\rightarrow [\text{sum} : \text{float}, \text{count} : \text{float}], \\ \gamma : [\text{sum} : \text{float}, \text{count} : \text{float}] &\rightarrow \text{float}, \end{aligned}$$

where

$$\begin{aligned} \text{agg}^I(X) &= [\text{sum} : \text{sum}(X.a), \text{count} : |X|], \\ \text{agg}^O([\text{sum} : s_1, \text{count} : c_1], [\text{sum} : s_2, \text{count} : c_2]) &= [\text{sum} : s_1 + s_2, \text{count} : c_1 + c_2], \\ (\text{agg}^O)^{-1}([\text{sum} : s_1, \text{count} : c_1], [\text{sum} : s_2, \text{count} : c_2]) &= [\text{sum} : s_1 - s_2, \text{count} : c_1 - c_2], \\ \gamma([\text{sum} : s, \text{count} : c]) &= s/c. \end{aligned}$$

Here, $\text{sum}(X.a)$ denotes the sum of all values of attribute a of the tuples in X , and $|X|$ denotes the cardinality of X . Note that $\text{agg}^I(\emptyset) = [\text{sum} : 0, \text{count} : 0]$, and $\gamma([\text{sum} : 0, \text{count} : 0])$ is undefined as is $\text{avg}(\emptyset)$.

In statistics, the variance of a bag of numbers is often calculated. For a bag B , it is defined as $s^2 = \frac{1}{n-1} \sum_{x \in B} (x - \bar{x})^2$, where $\bar{x}$ is the average of the values in B , i.e., $\bar{x} = \frac{1}{n} \sum_{x \in B} x$. As an exercise, the reader should show that variance is decomposable and reversible.

Not all aggregation functions are decomposable and reversible. For instance, *min* and *max* are decomposable but not reversible. If an aggregation function is applied to a bag that has to be converted to a set, then decomposability is jeopardized for *sum* and *count*. That is, in SQL **sum(distinct)** and **count(distinct)** are not decomposable.

Let us look at the decomposition of our five aggregation functions. We can decompose them as follows:

$$\begin{aligned}\min(X \cup Y) &= \min(\min(X), \min(Y)), \\ \max(X \cup Y) &= \max(\max(X), \max(Y)), \\ \text{count}(X \cup Y) &= \text{sum}(\text{count}(X), \text{count}(Y)), \\ \text{sum}(X \cup Y) &= \text{sum}(\text{sum}(X), \text{sum}(Y)).\end{aligned}$$

The treatment of avg is slightly more complicated, as we have already seen above. In the presence of null values, avg is defined as $\text{avg}(X) = \text{sum}(X) / \text{count}^{\text{NN}}(X)$. Hence, we can decompose it on the basis of

$$\text{avg}(X \cup Y) = \text{sum}(\text{sum}(X), \text{sum}(Y)) / (\text{count}^{\text{NN}}(X) + \text{count}^{\text{NN}}(Y))$$

In a typical query compiler, every occurrence of $\text{avg}(e)$ is replaced by $\text{sum}(e) / \text{count}^{\text{NN}}(e)$ during the NFST phase. Thus, during subsequent phases of the query compiler, we can safely ignore the intricacies of average¹.

Table 7.3 summarizes these findings.

agg	agg ¹	agg ²
min	min	min
max	max	max
count(*)	count(*)	sum
count(a)	count(a)	sum
sum	sum	sum
avg	sum, count ^{NN}	sum, sum

Figure 7.3: Decomposition of aggregate functions

We now extend the notion of decomposability to aggregation vectors. An aggregation vector is an expression of the form

$$(b_1 : \underset{1}{\text{agg}}(a_1), \dots, b_k : \underset{k}{\text{agg}}(a_k)),$$

where the a_i and b_i are attribute names and the agg_i are aggregation functions. Often, we will leave out the enclosing parenthesis and simply write

$$b_1 : \underset{1}{\text{agg}}(a_1), \dots, b_k : \underset{k}{\text{agg}}(a_k).$$

We use $\circ$ to denote the concatenation of two aggregation vectors.

Let $F = (b_1 : \underset{1}{\text{agg}}(a_1), \dots, b_k : \underset{k}{\text{agg}}(a_k))$ be an aggregation vector and all aggregates agg_i be decomposable into agg_i^1 and agg_i^2 . Then, we say that F is decomposable into F^1 and F^2 where

$$\begin{aligned}F^1 &:= (b'_1 : \underset{1}{\text{agg}}^1(a_1), \dots, b'_k : \underset{k}{\text{agg}}^1(a_k)), \\ F^2 &:= (b_1 : \underset{1}{\text{agg}}^2(b'_1), \dots, b_k : \underset{k}{\text{agg}}^2(b'_k)).\end{aligned}$$

¹These are nicely described in a book by Savage [762]

Note that in all cases, we have that if F is decomposable into F^1 and F^2 , then F^1 is decomposable into $F^{1,1}$ and $F^{1,2}$, and F^2 is decomposable into $F^{2,1}$ and $F^{2,2}$. Further, we have

$$\begin{aligned} F^{1,1} &= F^1, \\ F^{1,2} &= F^2, \\ F^{2,1} &= F^2, \\ F^{2,2} &= F^2. \end{aligned}$$

Let e_1 and e_2 be arbitrary expressions. We say that an aggregation vector F is splittable into F_1 and F_2 with respect to e_1 and e_2 if $F = F_1 \circ F_2$, $\mathcal{F}(F_1) \cap \mathcal{A}(e_2) = \emptyset$ and $\mathcal{F}(F_2) \cap \mathcal{A}(e_1) = \emptyset$. Assume that F contains an aggregation function agg_i applied to some attribute a_i . If $a \in \mathcal{A}(e_1)$, then clearly $\text{agg}_i(a_i)$ belongs to F_1 , if $a \in \mathcal{A}(e_2)$ then $\text{agg}_i(a_i)$ belongs to F_2 . There are other cases where F is splittable. Consider, for example, $\text{sum}(a_1 + a_2)$ for $a_i \in \mathcal{A}(e_i)$. Since $\text{sum}(a_1 + a_2) = \text{sum}(a_1) + \text{sum}(a_2)$, this does not hinder splittability. The same holds for subtraction.

The correct handling of duplicates, i.e., bags, is essential for the correctness of the query compiler and requires some care. We will therefore classify our aggregation functions into those which are sensitive to duplicates and those which are not. An aggregation function is called *duplicate agnostic* if the multiplicity of the elements in the bag does not influence its result. It is called *duplicate sensitive* otherwise. For our aggregation functions we have

- min , max , $\text{sum}(\text{distinct})$, $\text{count}(\text{distinct})$, $\text{avg}(\text{distinct})$ are duplicate agnostic and
- sum , count , avg are duplicate sensitive.

Yan and Larson used the term Class C aggregation function for duplicate sensitive aggregation functions and Class D for duplicate agnostic aggregation functions [946].

Finally, note that for all aggregate functions except $\text{count}(\ast)$, we have $\text{agg}(\{a\}) = a$ for arbitrary elements a . Thus, if we are sure that we deal with only one tuple, we can apply the following rewrite. Let a_i and b_i be attributes. Then, if $F = (b_1 : \text{agg}_1(a_1), \dots, b_m : \text{agg}_m(a_m))$, we define $\hat{F} = (b_1 : a_1, \dots, b_m : a_m)$.

7.3 Operators

The bag operators as well as other typical operators like selection and join are well-known. As we will see, the only difference in the definitions used here is that they are extended to express nested queries. In order to enable this, we allow the subscripts (predicates, expressions) of these operators to contain algebraic expressions.

In this section, we define all our operators on bags. Besides duplicate elimination, only projection will have explicit control over duplicates.

Sometimes, the left outerjoin needs some additional tuning. The standard definition of the left outerjoin demands that if some tuple from its left argument does not have a join partner in its right argument, the attributes from the right argument are given null values. We extend the left outerjoin such that values other than null can be given to attributes of the right hand side. Similarly, the full outerjoin will be extended to carry two superscripts for this kind of defaults.

The d-join operation is used for performing a join between two bag valued items, where the second one is dependent on the first one. One use is to express queries with table functions (see Sec. 4.10). Another is to access index structures (see Sec. 4.14). The d-join can also be used to unnest nested queries. It is often equivalent to a join between two bags with a membership predicate [803]. In some cases, it corresponds to an unnest operation.

The map operator χ ([487]) is well-known from the functional programming language context. A special case of it, where it adds derived information in form of an added attribute with an according value (e.g. by object-base lookup or by method calls) to each tuple of a bag has been proposed in [486, 487]. Later, this special case was given the name *materialization operator* [94].

The unnest operator is known from NF² [765, 746]. It will come in two different flavors allowing us to perform unnesting not only on nested relations but also on attributes whose value is a bag of elements which are not tuples. The reverse operator is the nest operator, which can be generalized to a grouping operator. In our algebra, there exist two grouping operators: one unary grouping operator and one binary grouping operator (called *groupjoin*). The unary grouping operator groups one bag of tuples according to a grouping condition. Further, it can apply an arbitrary expression to the newly formed group. The groupjoin adds a group to each element in the first argument bag. This group is formed from the second argument. The groupjoin will exploit the fact that in the object-oriented context objects can have bag-valued attributes. As we will see, this is useful for both, unnesting nested queries and producing nested results. We will even use nesting as a useful tool for processing SQL queries.

7.3.1 Preliminaries

As already mentioned, our algebraic operators not only deal with standard relations but are polymorphic in the general sense. In order to fix the domain of the operators, we need some technical abbreviations and notations. Let us introduce these first.

Since our operators are polymorphic, we need variables for types. We use τ possibly with a subscript to denote types. To express that a certain expression is of type e , we write $e :: \tau$. Starting from concrete names for types and type variables, we can build type expressions the standard way by using type constructors to build tuple types $([\cdot])$, set types $\{\cdot\}_s$, bag types $\{\cdot\}_b$ and sequence types $\langle \cdot \rangle$. Having two type expressions t_1 and t_2 , we denote by $t_1 \leq t_2$ that t_1 is a subtype of t_2 . It is important to note that this subtype relationship is not based on the sub-/superclass hierarchy found in most object-oriented models. Instead, it simply denotes substitutability. That is, type t_1 provides at least all

the attributes and member functions that t_2 provides [124].

Most of our algebraic operators are tuned to work on bags of tuples. The most important information here is the set of attributes $\mathcal{A}(e)$ it provides or produces. For some expression e , the function $\mathcal{A}(e)$ is defined as follows. $\mathcal{A}(e) = \{a_1, \dots, a_n\}$ if $e :: \{[a_1 : \tau_1, \dots, a_n : \tau_n]\}_s$, $e :: \{[a_1 : \tau_1, \dots, a_n : \tau_n]\}_b$, $e :: \langle [a_1 : \tau_1, \dots, a_n : \tau_n] \rangle$, or $e :: [a_1 : \tau_1, \dots, a_n : \tau_n]$. Given a set of attributes A , we are sometimes interested in the attributes provided by an expression e which are not in A . For this complement we use the notation $\overline{\mathcal{A}}(e)$, which is defined as $\mathcal{A}(e) \setminus A$.

Often, we are not only interested in the set of attributes an expression provides, but also in the set of free variables occurring in an expression e . We use $\mathcal{F}(e)$ to denote the set of all free variables (attributes) of e .

Since the subscripts of our algebraic operators can contain arbitrary expressions, they may contain variables or even free variables. Then there is a need to get bindings for these variables before the subscript expression can be evaluated. These bindings are taken from the argument(s) of the operator. In order to do so, we need a specified binding mechanism. The λ -notation is such a mechanism and can be used, e.g., in case of ambiguities. For our purpose, it suffices if we stick to the following convention.

- For an expression e with free variables $\mathcal{F}(e) = \{a_1, \dots, a_n\}$ and a tuple t with $\mathcal{F}(e) \subseteq \mathcal{A}(t)$ we define $e(t) := e[a_1 \leftarrow t.a_1, \dots, a_n \leftarrow t.a_n]$.² Similarly, we define $e(t_1, \dots, t_n)$ for more than a single tuple. This way, we can use an expressions as a function. Note that the attribute names of the t_i have to be distinct to avoid name conflicts.
- For an expression e with only one free variable x , we define $e(t) = e[x \leftarrow t]$.

The mechanism is very much like the standard binding for the relational algebra. Consider for example a select operation $\sigma_{a=3}(R)$. Then we assume that a , the free variable of the subscript expression $a = 3$, is bound to the value of the attribute a of the tuples of the relation R . To express this binding explicitly, we would write for a tuple $t \in R$ $(a = 3)(t)$. Since a is an attribute of R and hence of t , by our convention a is replaced by $t.a$, the value of attribute a of tuple t . Since we want to avoid name conflicts right away, we assume that all variable/attribute names used in a query are distinct. This can be achieved in a renaming step. Typically, renaming takes place during the NFST phase.

Application of a function f to arguments e_i is denoted by either regular (e.g., $f(e_1, \dots, e_n)$) or dot (e.g., $e_1.f(e_2, \dots, e_n)$) notation. The dot notation is used for type-associated methods occurring in the object-oriented context.

Last, we introduce the heavily overloaded symbol $\circ$. It denotes function concatenation and (as a special case) tuple concatenation as well as the concatenation of tuple types to yield a tuple type containing the union of the attributes of the two argument tuple types.

² $e[v_1 \leftarrow e_1, \dots, v_n \leftarrow e_n]$ denotes a substitution of the variables v_i by the expressions e_i within an expression e .

Sometimes it is useful to be able to produce a bag containing only a single tuple with no attributes. This is done by the *singleton scan* operator denoted by $\square$. Thus, $\square \equiv \{\square\}_b$.

Very often, we are given some database item which is a bag of other items. Binding these to variables or, equivalently, embedding the items into a tuple, we use the notation $e[x]$ for an expression e and a variable/attribute name x . For bag-valued expressions e , $e[x]$ is defined as $e[x] = \{[x : y] | y \in e\}$. For sequence-valued expressions e , we define $e[a] = \epsilon$ if e is empty and $e[a] = \langle [a : \alpha(e)] \rangle \oplus \tau(e)[a]$ otherwise.

By id we denote the identity function.

7.3.2 Signatures

We are now ready to define the signatures of the operators of our algebra. Their semantics is defined in a subsequent step. Remember that we consider all operators as being polymorphic. Hence, their signatures are polymorphic and contain type variables, denoted by τ , often with an index. As mentioned before, we define all operators on bags. Let us start by typing our bag operators

$$\begin{aligned} \cup & : \{\tau\}_b, \{\tau\}_b \rightarrow \{\tau\}_b, \\ \cap & : \{\tau\}_b, \{\tau\}_b \rightarrow \{\tau\}_b, \\ \setminus & : \{\tau\}_b, \{\tau\}_b \rightarrow \{\tau\}_b, \\ \Pi^D & : \{\tau\}_b \rightarrow \{\tau\}_b. \end{aligned}$$

The unary operators we use have the following signatures, where $\mathcal{B}$ denotes

the type *boolean*:

$$\begin{aligned}
\Pi_A &: \{\tau\}_b \rightarrow \{\tau'\}_b \\
&\quad \text{if } \tau \leq \tau' = [a_1 : \tau_1, \dots, a_n : \tau_n], A = \{a_1, \dots, a_n\}, \\
\Pi_A^D &: \{\tau\}_b \rightarrow \{\tau'\}_b \\
&\quad \text{if } \tau \leq \tau' = [a_1 : \tau_1, \dots, a_n : \tau_n], A = \{a_1, \dots, a_n\}, \\
\sigma_p &: \{\tau\}_b \rightarrow \{\tau\}_b \\
&\quad \text{if } p : \tau \rightarrow \mathcal{B}, \\
\chi_f &: \{\tau_1\}_b \rightarrow \{\tau_2\}_b \\
&\quad \text{if } f : \tau_1 \rightarrow \tau_2, \\
\chi_{a:f} &: \{\tau_1\}_b \rightarrow \{\tau_1 \circ [a : \tau_2]\}_b \\
&\quad \text{if } f : \tau_1 \rightarrow \tau_2, \\
\Gamma_{\theta G;g:f} &: \{\tau_1 \circ \tau_2\}_b \rightarrow \{\tau_1 \circ [g : \tau']\}_b \\
&\quad \text{if } \tau_i \leq [], f : \{\tau_2\}_b \rightarrow \tau', G = \mathcal{A}(\tau_1), \\
\nu_{G;g} &: \{\tau_1 \circ \tau_2\}_b \rightarrow \{\tau_1 \circ [g : \{\tau_2\}_b]\}_b \\
&\quad \text{if } \tau_i \leq [], G = \mathcal{A}(\tau_1), \\
\mu_g &: \{\tau\}_b \rightarrow \{\tau'\}_b \\
&\quad \text{if } \tau = [a_1 : \tau_1, \dots, a_n : \tau_n, g : \{\tau_0\}_b], \\
&\quad \tau_0 \leq [], \\
&\quad \tau' = [a_1 : \tau_1, \dots, a_n : \tau_n] \circ \tau_0, \\
\mu_{g;c} &: \{\tau\}_b \rightarrow \{\tau'\}_b \\
&\quad \text{if } \tau = [a_1 : \tau_1, \dots, a_n : \tau_n, g : \{\tau_0\}_b], \\
&\quad \tau' = [a_1 : \tau_1, \dots, a_n : \tau_n] \circ [c : \tau_0].
\end{aligned}$$

One special operator is needed to translate OQL, which exhibits an explicit **flatten** operator to unnest bags of bags. An according algebraic operator is defined easily:

$$\mathbf{flatten} : \{\{\tau\}_b\}_b \rightarrow \{\tau\}_b.$$

The following is a list of signatures of some binary operators.

$$\begin{aligned}
\times & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1 \circ \tau_2\}_b, \\
\bowtie_q & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1 \circ \tau_2\}_b \\
& \quad \text{if } \tau_i \leq \square, q : \tau_1, \tau_2 \rightarrow \mathcal{B}, \\
\bowtie_q & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1\}_b \\
& \quad \text{if } \tau_i \leq \square, q : \tau_1, \tau_2 \rightarrow \mathcal{B}, \\
\triangleright_q & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1\}_b \\
& \quad \text{if } \tau_i \leq \square, q : \tau_1, \tau_2 \rightarrow \mathcal{B}, \\
\bowtie_q & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1^+ \circ \tau_2\}_b \\
& \quad \text{if } \tau_i < \square, q : \tau_1, \tau_2 \rightarrow \mathcal{B}, \\
\bowtie_q & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1^+ \circ \tau_2^+\}_b \\
& \quad \text{if } \tau_i < \square, q : \tau_1, \tau_2 \rightarrow \mathcal{B}, \\
\blacktriangleright & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1 \circ \tau_2\}_b \\
& \quad \text{if } \tau_i \leq \square, \\
\bowtie_{A_1 \theta A_2; g; f} & : \{\tau_1\}_b, \{\tau_2\}_b \rightarrow \{\tau_1 \circ [g : \tau']\}_b \\
& \quad \text{if } \tau_1 \leq \square, f : \{\tau_2\}_b \rightarrow \tau', A_i \subseteq \mathcal{A}(\tau_i) \text{ for } i = 1, 2.
\end{aligned}$$

Using some special min/max operators to retrieve the element(s) whose value becomes minimal/maximal often results in more efficient plans:

$$\begin{aligned}
max_{g; m; f} & : \{\tau\}_b \rightarrow [m : \tau_a, g : \tau_f] \\
& \quad \text{if } \tau \leq [a : \tau_a], f : \{\tau_a\}_b \rightarrow \tau_f.
\end{aligned}$$

7.3.3 Projection

Let $A \{a_1, \dots, a_n\}$ be a set of attributes. We define the two projection operators

$$\begin{aligned}
\Pi_A(e) & := \{[a_1 : x.a_1, \dots, a_n : x.a_n] \mid x \in e\}_b, \\
\Pi_A^D(e) & := \Pi^D(\Pi_A(e)).
\end{aligned}$$

The result of Π^D is always duplicate-free. In concordance with the characteristic function of bags, we use $\doteq$ to determine whether two elements are equal or not. Thus, Π_A^D is defined such that the characteristic function of $\Pi_A^D(e)$ yields $\max(1, \chi_{\Pi_A(e)}(x))$ for all $x \in \Pi_A(e)$.

Thus, it is set-faithful. Typically, the result of Π_A is not duplicate-free, even if its input is duplicate-free. Thus, we need explicit duplicate control here. One exception occurs in the presence of functional dependencies. If $A \rightarrow \mathcal{A}(e)$ and e is duplicate-free, then $\Pi_A(e)$ is duplicate-free.

Sometimes, we want to eliminate a single attribute or a set of attributes. This is denoted by

$$\begin{aligned}
\Pi_{\overline{A}}(e) & := \Pi_{\mathcal{A}(e) \setminus A}(e) \\
\Pi_{\overline{A}}^D(e) & := \Pi_{\mathcal{A}(e) \setminus A}^D(e)
\end{aligned}$$

$e_1 := R_1$ <table><tr><th>a_1</th></tr><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>		a_1	1	2	3	$e_2 := R_2$ <table><tr><th>a_2</th><th>b</th></tr><tr><td>1</td><td>2</td></tr><tr><td>1</td><td>3</td></tr><tr><td>2</td><td>4</td></tr><tr><td>2</td><td>5</td></tr></table>		a_2	b	1	2	1	3	2	4	2	5						
a_1																							
1																							
2																							
3																							
a_2	b																						
1	2																						
1	3																						
2	4																						
2	5																						
$e_3 := \Gamma_{a_2;g;\text{id}}(e_2)$ <table><tr><th>a_2</th><th>g</th></tr><tr><td>1</td><td>$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$</td></tr><tr><td>2</td><td>$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$</td></tr></table>		a_2	g	1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$	2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$	$e_4 := \chi_{g:\sigma_{a_1=a_2}}(e_2)(e_1)$ <table><tr><th>a_1</th><th>g</th></tr><tr><td>1</td><td>$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$</td></tr><tr><td>2</td><td>$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$</td></tr><tr><td>3</td><td>$\emptyset_b$</td></tr></table>		a_1	g	1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$	2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$	3	$\emptyset_b$						
a_2	g																						
1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$																						
2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$																						
a_1	g																						
1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$																						
2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$																						
3	$\emptyset_b$																						
$e_5 := e_1 \bowtie_{a_1=a_2;g;\text{id}} e_2$ <table><tr><th>a_1</th><th>g</th></tr><tr><td>1</td><td>$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$</td></tr><tr><td>2</td><td>$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$</td></tr><tr><td>3</td><td>$\emptyset_b$</td></tr></table>		a_1	g	1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$	2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$	3	$\emptyset_b$	$e_6 := e_1 \bowtie_{a_1=a_2} e_3$ <table><tr><th>a_1</th><th>a_2</th><th>g</th></tr><tr><td>1</td><td>1</td><td>$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$</td></tr><tr><td>2</td><td>2</td><td>$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$</td></tr><tr><td>3</td><td>-</td><td>-</td></tr></table>		a_1	a_2	g	1	1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$	2	2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$	3	-	-
a_1	g																						
1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$																						
2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$																						
3	$\emptyset_b$																						
a_1	a_2	g																					
1	1	$\{[a_2 : 1, b_2 : 2], [a_2 : 1, b_2 : 3]\}_b$																					
2	2	$\{[a_2 : 2, b_2 : 4], [a_2 : 2, b_2 : 5]\}_b$																					
3	-	-																					

Figure 7.4: Example for map and group operators

7.3.4 Selection

Note that in the following definition there is no restriction on the selection predicate. It may contain path expressions, method calls, nested algebraic operators, etc.:

$$\sigma_p(e) := \{x \mid x \in e, p(x)\}_b.$$

The output of the selection is duplicate-free if its input is duplicate-free. As selection is set-faithful, we do not need any additional set-selection. An example of a selection together with a map operator (discussed next) can be found in Fig. 7.4.

7.3.5 Map

The map operator is of fundamental importance to the algebra. It comes in two flavors. The first one extends a given input tuple by an attribute and assigns a value to this new attribute. This variant is also called *materialize operator* [486, 94]. The second one produces for each input element an output element by applying a function to it. This corresponds to the standard map as defined in, e.g., [487]. The latter is able to express the former. The two variants of the map operator are defined as follows:

$$\begin{aligned} \chi_{a:e_2}(e_1) &:= \{y \circ [a : e_2(y)] \mid y \in e_1\}_b, \\ \chi_{e_2}(e_1) &:= \{e_2(x) \mid x \in e_1\}_b. \end{aligned}$$

We can generalize the last variant to calculate values for many attributes. Given an *attribute assignment vector* $a_1 : e_1, \dots, a_k : e_k$, we define

$$\chi_{a_1:e_1, \dots, a_k:e_k}(e) := \chi_{a_k:e_k}(\dots \chi_{a_1:e_1}(e) \dots).$$

If we demand that $a_i \notin \mathcal{A}(e)$, then the a_i are new attributes. Then, the materialize operator and its special single-attribute case $\chi_{a:e}$ are called *extending*, because it extends a given input tuple with new attributes while it does not modify the values of the input attributes. Many equivalences only hold for this specialization of the map operator, which, at the same time, is the predominant variant used. In fact, it is sufficient for SQL. An example of an extending map operator can be found in Fig. 7.4.

Note that the map operator for the object-oriented and object-relational context obviates the need of a relational projection. Sometimes the map operator is equivalent to a renaming. In this case, we will use ρ instead of χ . Let $A = \{a_1, \dots, a_n\}$ and $B = \{b_1, \dots, b_n\}$ be two sets with n attributes each. We then define

$$\rho_{A \leftarrow B}(e) := \Pi_{\bar{A}}(\chi_{b_1:a_1, \dots, b_n:a_n}(e))$$

The result of the extending variant of the map operator is duplicate-free if and only if its input is. Thus, the extending map operator is set-faithful.

7.3.6 Unary Grouping

Two grouping operators are contained in our algebra. The first one, discussed here and called (*unary*) *grouping*, is defined on a bag and its subscript indicates the (i) grouping criterions and (ii) a new attribute name as well as a function which is used to calculate its value.

$$\Gamma_{\theta G;g:f}(e) := \{y \circ [g : x] \mid y \in \Pi_G^D(e), x = f(\{z \mid z \in e, z.G \theta y.G\}_b)\}_s$$

for some set of attributes G , an attribute g and a function f . The comparison operator θ must be a null-extended comparison operator like ' $\doteq$ '. Note that the result is a set, but f is applied to a bag. An example for the grouping operator can be found in Fig. 7.4.

The grouping criterion may be defined on several attributes. Then, G and θ represent sequences of attributes and comparators. In case all θ equal ' $\doteq$ ', we abbreviate $\Gamma_{\doteq G;g:f}$ by $\Gamma_{G;g:f}$.

We can extend the above definition to calculate several new attribute values by defining

$$\Gamma_{\theta G;b_1:f_1, \dots, b_k:f_k}(e) := \{y \circ [b_1 : x_1, \dots, b_k : x_k] \mid y \in \Pi_G^D(e), x_i = f_i(\{z \mid z \in e, z.G \theta y.G\}_b)\}_s.$$

We also introduce two variants of the grouping operator, which can be used to abbreviate small expressions. Let $F = b_1 : e_1, \dots, b_k : e_k$ and $\mathcal{F}(e_i) = \{g\}$ for all $i = 1, \dots, k$. Then we define

$$\Gamma_{G;F}(e) := \Pi_{\bar{g}}(\chi_F(\Gamma_{G;g:\text{id}}(e))).$$

Here, the free attribute g is implicit. If we wish to make it explicit, we write $\Gamma_{G;g;F}$ instead of simply $\Gamma_{G;F}$. Note that g plays the same role as *partition* in OQL ([133, p. 114]).

Let us also introduce a SQL-notation based variant. Let F be an aggregation vector of the form

$$F = b_1 : \underset{1}{\text{agg}}(a_1), \dots, b_k : \underset{k}{\text{agg}}(a_k)$$

for attributes a_i . Then we define F_g as

$$F_g = b_1 : \underset{1}{\text{agg}}(g.a_1), \dots, b_k : \underset{k}{\text{agg}}(g.a_k)$$

and introduce the following abbreviation:

$$\Gamma_{G;F}(e) := \Gamma_{G;g;F_g}(e).$$

This is the version we have to use for SQL.

The traditional nest operator ν [765], which nests a relation R given a set of attributes $G \subset \mathcal{A}(e)$, can be defined as an abbreviation of the grouping operator:

$$\nu_{G;g}(e) := \Gamma_{G;g;\Pi_{\overline{G}}}(e),$$

where $\overline{G}$ abbreviates $\mathcal{A}(e) \setminus G$.

The results of Γ and ν are always duplicate-free. Thus, these operators are set-faithful.

7.3.7 Unnest Operators

The unnest operator comes in two different flavors. The first one is responsible for unnesting a set of tuples on an attribute being a set/bag/sequence of tuples itself. The second one unnests sets of tuples on an attribute not being a bulk of tuples but a set of something else, e.g., integers. The according definitions are

$$\begin{aligned} \mu_g(e) &:= \{y.[\mathcal{A}(y) \setminus \{g\}] \circ x | y \in e, x \in y.g\}_b, \\ \mu_{a:g}(e) &:= \{y.[\mathcal{A}(y) \setminus \{g\}] \circ [a : x] | y \in e, x \in y.g\}_b. \end{aligned}$$

If the bag-valued attribute is not stored explicitly but derived by the evaluation of an expression, we use the *unnest map* operator to unnest it:

$$\begin{aligned} \Upsilon_{e_2}(e_1) &:= \Pi_{\overline{g}}(\mu_g(\chi_{g:e_2}(e_1))), \\ \Upsilon_{a:e_2}(e_1) &:= \Pi_{\overline{g}}(\mu_{a:g}(\chi_{g:e_2}(e_1))). \end{aligned}$$

The motivation for the unnest map operator is that it saves the explicit materialization of the result of the evaluation of the expression e_2 .

The results of $\mu_g(e)$ and $\mu_{a:g}$ are duplicate-free, if and only if the following two conditions hold.

1. The input e is duplicate-free.
2. For each tuple $t \in e$ we have that $t.g$ is duplicate-free.

Hence, explicit duplicate control for the unnest operator is in order. The same holds for the unnest map operator.

7.3.8 Flatten Operator

The flatten operator flattens a bag of bags by unioning the elements of the bags contained in the outer bag.

$$\text{flatten}(e) = \{y \mid x \in e, y \in x\}_b$$

The flatten operator's result is duplicate-free if and only if the bags it contains are duplicate-free and they have a pairwise empty intersection. Thus, explicit duplicate control is very much in order.

7.3.9 Join Operators

The algebra features many different join operators. The first five, namely join, semijoin, antijoin, left outerjoin, and full outerjoin – are rather standard:

$$\begin{aligned} e_1 \times e_2 &:= \{y \circ x \mid y \in e_1, x \in e_2\}_b, \\ e_1 \bowtie_p e_2 &:= \{y \circ x \mid y \in e_1, x \in e_2, p(y, x)\}_b, \\ e_1 \ltimes_p e_2 &:= \{y \mid y \in e_1, \exists x \in e_2, p(y, x)\}_b, \\ e_1 \triangleright_p e_2 &:= \{y \mid y \in e_1, \neg \exists x \in e_2, p(y, x)\}_b, \\ e_1 \bowtie_p e_2 &:= (e_1 \bowtie_p e_2) \cup ((e_1 \triangleright_p e_2) \times \{\perp_{\mathcal{A}(e_2)}\}), \\ e_1 \bowtie_p e_2 &:= (e_1 \bowtie_p e_2) \\ &\quad \cup ((e_1 \triangleright_p e_2) \times \{\perp_{\mathcal{A}(e_2)}\}) \\ &\quad \cup (\{\perp_{\mathcal{A}(e_1)}\} \times (e_2 \triangleright_p e_1)). \end{aligned}$$

An example for the left outerjoin can be found in Fig. 7.4. More examples for join, left outerjoin, and full outerjoin can be found in Fig. 7.6 for the predicate $q_{ij} := (b_i = b_j)$ and in Fig. 7.7 for the predicate $q'_{ij} := (b_i \doteq b_j)$.

Regular joins were already present in Codd's original proposal of a relational algebra [196]. Outerjoins were invented by Lacroix and Pirotte [524].

The next join operator to come is called *dependency join*, or *d-join*, and is denoted by $\bowtie$. It is a join between two bags, where the evaluation of the second bag may depend on the first bag. The filled triangle thus shows the direction into which information has to flow in order to evaluate the d-join. It is used to translate **from** clauses containing table functions with parameters (see Sec. 4.10 for an example) and lateral derived tables into the algebra. Whenever possible, d-joins will be rewritten into standard joins. The definition of the d-join is

$$e_1 \bowtie e_2 := \{y \circ x \mid y \in e_1, x \in e_2(y)\}_b.$$

The result of a d-join is duplicate-free if e_1 is duplicate-free and if for each $t_1 \in e_1$ we have that $e_2(t_1)$ is duplicate-free. Example applications of the d-join can be found in Sec. 4.10 and Sec. 4.14.

For the left outerjoin and the full outerjoin, we need a variant which allows us to set some attribute values to constants other than null for tuples with no

join partner. Let $D_i = d_1^i : c_1^i, \dots, d_k^i : c_k^i$ ($i = 1, 2$) be two vectors assigning constants c_j^i to attributes d_j^i . We then define

$$\begin{aligned} e_1 \bowtie_p^{D^2} e_2 &:= (e_1 \bowtie_p e_2) \\ &\quad \cup ((e_1 \bowtie_p e_2) \times \{\perp_{\mathcal{A}(e_2) \setminus \mathcal{A}(D_2)} \circ [D_2]\}), \\ e_1 \bowtie_p^{D^1; D^2} e_2 &:= (e_1 \bowtie_p e_2) \\ &\quad \cup ((e_1 \bowtie_p e_2) \times \{\perp_{\mathcal{A}(e_2) \setminus \mathcal{A}(D_2)} \circ [D_2]\}), \\ &\quad \cup ((e_2 \bowtie_p e_1) \times \{\perp_{\mathcal{A}(e_1) \setminus \mathcal{A}(D_1)} \circ [D_1]\}), \end{aligned}$$

If one of D^1 or D^2 is empty, we use $-$ to denote this.

As can already be seen from the definitions, this set of join operators is highly redundant. As is well-known, the (regular) join can be expressed as a sequence of selection and cross product:

$$e_1 \bowtie_q e_2 \equiv \sigma_q(e_1 \times e_2).$$

For an expression e_2 and a predicate q , define the predicate p as $p = (\sigma_q(e_2) \neq \emptyset)$. Therewith, the semijoin can be expressed as a selection:

$$e_1 \bowtie_q e_2 \equiv \sigma_p(e_1).$$

If we define p as $p = (\sigma_q(e_2) = \emptyset)$, then the antijoin can be expressed as

$$e_1 \bowtie_q e_2 \equiv \sigma_q(e_1).$$

The outerjoins were already defined using these three operators, which in turn can be expressed using only selection and cross product.

We observe that:

- The results of cross product, (regular) join, left outerjoin and full outerjoin are duplicate-free if and only if both of their inputs are duplicate-free.
- The results of a semi- and an antijoin are duplicate-free if and only if their left-input is duplicate-free.

Thus, it follows that these operators are set-faithful.

7.3.10 Groupjoin

The second grouping operator — called *groupjoin* or *binary grouping* — is defined on two input bags. It is more than 20 years old, but there is still no common name for it. It was first introduced by von Bülzingsloewen [901, 902] under the name of *outer aggregation*. Nakano calls the same operator *general aggregate formation* [637], since unary grouping is called *aggregate formation* by Klug [502]. Steenhagen, Apers, Blanken, and de By call a variant of the groupjoin *nest-join* [831]. The groupjoin is quite versatile and we strongly believe that no DBMS can do without it. For example, it has been

successfully applied to the problem of unnesting nested queries in the context of SQL [92, 104, 105, 637, 901, 902], OQL [189, 190, 191], and XQuery [597]. Chatziantoniou, Akinde, Johnson, and Kim apply the groupjoin to efficiently evaluate data warehouse queries which feature a **cube-by** or **group-by grouping sets** clause [146]. They call the groupjoin *MD-Join*.

The groupjoin is defined as follows:

$$e_1 \bowtie_{A_1\theta A_2;g:f} e_2 := \{y \circ [g : G] \mid y \in e_1, G = f(\{x \mid x \in e_2, y.A_1\theta x.A_2\})_b\}_b.$$

Thus, each tuple t_1 in e_1 is extended by a new attribute g , whose value is the result of applying a function f to a bag. This bag contains all tuples from e_2 which join on $A_1\theta A_2$ with e_1 . An example for the groupjoin can be found in Fig. 7.4. In fact, we do not have to rely on a comparison-based predicate. We can generalize the groupjoin to any join predicate:

$$e_1 \bowtie_{q;g:f} e_2 := \{y \circ [g : G] \mid y \in e_1, G = f(\{x \mid x \in e_2, q(x, y)\})_b\}_b.$$

Similar to unary grouping, we will use $\bowtie_{q;g:F}$ to abbreviate $\Pi_{\bar{g}}(\chi_F(e_1 \bowtie_{q;g:\text{id}} e_2))$, and $\bowtie_{q;F}$ to abbreviate $\bowtie_{A;g;F}$. In both cases, F must be an aggregation vector with $\mathcal{F}(F) = \{g\}$. An SQL notation variant of the groupjoin is defined as $e_1 \bowtie_{q;F} e_2 := e_1 \bowtie_{q;F_g} e_2$, where the requirements for F and F_g are the same as for unary grouping.

Since the reader is most likely not familiar with groupjoin, let us give some remarks and pointers on its implementation. Obviously, implementation techniques for the equijoin and the nest operator can be used if θ stands for equality. For the other cases, implementations based on sorting seem promising. One could also consider implementation techniques for non-equi joins, e.g., those developed for the band-width join [239]. An alternative is to use θ -tables, which were developed for efficient aggregate processing [192]. Implementation techniques for groupjoin have also been discussed in [146, 598].

Note that the groupjoin produces a duplicate-free result if and only if its left input is duplicate-free. It is thus set-faithful.

7.3.11 Min/Max Operators

The max operator has a very specific use that will be explained in the sequel. The following definition is a generalization of the *Max* operator as defined in [189]. Defining a min operator is left to the reader.

$$\text{Max}_{m;g;a:f}(e) := [m : \max(\{x.a \mid x \in e\})_b, g : f(\{x \mid x \in e, x.a = m\})_b]$$

The max operator successively performs three tasks. First, it calculates the maximum (m) of all elements contained in $e.a$ for some attribute $a \in \mathcal{A}(e)$. Second, it uses this maximum (m) to select exactly those elements t from e such that $t.a = m$, i.e., their a value is maximal. Third, these maximizing elements t from e are collected into a bag and the result of applying the function f to it is stored as the value for the attribute g . In a real implementation, at least the first two phases will be merged. Thus, max requires only a single scan over e .

The sole purpose of these two operators is to efficiently evaluate expressions which demand to select a maximizing or minimizing element:

$$f(\sigma_{a=\text{agg}(\chi_a(e_2))}(e_1)) \equiv \text{agg}_{m;g;a;f}(e_1).g \quad (7.1)$$

$$\begin{aligned} \chi_{g:f(\sigma_{a=m}(e_2))}(\chi_{m:\text{agg}}(e_1)(e)) &\equiv \chi_{\text{agg}_{m;g;a;f}(e_1)}(e) \\ &\text{if } \Pi_a(e_1) = \rho_{b \leftarrow a}(\Pi_b(e_2)), \end{aligned} \quad (7.2)$$

where agg can stand for $\min$ or $\max$. Clearly, in case $e_1 = e_2$ the conditions are fulfilled.

ToDo

This can be very useful also in the nested case.

7.3.12 Other Dependent Operators

Similar to the d-join, we can introduce a d-semijoin, d-antijoin, and so forth. Before we introduce them, let us first make an important observation on the d-join. Therefore, let e_1 and e_2 be two expressions and let $J = \mathcal{F}(e_2) \cap \mathcal{A}(e_1)$. Then

$$e_1 \bowtie e_2 \equiv e_1 \bowtie (\rho_{J \leftarrow J'}(\Pi_J^D(e_1) \bowtie e_2)). \quad (7.3)$$

Thus, we can evaluate the d-join by first evaluating it for all distinct attribute combinations contained in $\Pi_J^D(e_1)$ and then joining the result with e_1 . This saves redundant evaluations of expression e_2 for the same attribute combination. The motivation for the memox operator ($\mathfrak{M}$) followed exactly this reasoning (see Sec. 4.14). The expression $(\rho_{J \leftarrow J'}(\Pi_J^D(e_1) \bowtie e_2))$ will be used quite frequently. Thus, we abbreviate it by $\widehat{e_2}(e_1)$ and even by $\widehat{e_2}$ if e_1 is clear from the context. So far, the d-join had no selection predicate. We can simply add one, by defining

$$e_1 \bowtie_q e_2 := e_1 \bowtie \sigma_q(e_2).$$

Now, we can give alternative expressions for the d-join:

$$\begin{aligned} e_1 \bowtie_q e_2 &\equiv e_1 \bowtie \sigma_q(e_2), \\ e_1 \bowtie_q e_2 &\equiv e_1 \bowtie_{J=J'} \widehat{\sigma_q(e_2)}, \\ e_1 \bowtie_q e_2 &\equiv e_1 \bowtie_{J=J'} (\rho_{J \leftarrow J'}(\Pi_J^D(e_1) \bowtie_q e_2)), \\ e_1 \bowtie_q e_2 &\equiv e_1 \bowtie_{J=J' \wedge q} \widehat{e_2}, \\ e_1 \bowtie_q e_2 &\equiv e_1 \bowtie_{\hat{q}} \widehat{e_2}, \end{aligned}$$

where $J = \mathcal{F}(e_2) \cap \mathcal{A}(e_1)$ and $\hat{q} = ((J = J') \wedge q)$.

As for the d-join, the filled triangle points into the direction of the information flow for all subsequently defined dependent operators. Let us start with the d-semijoin and d-antijoin. They can be defined using the selection:

$$\begin{aligned} e_1 \ltimes_q e_2 &:= \sigma_{\sigma_q(e_2) \neq \emptyset}(e_1), \\ e_1 \triangleright_q e_2 &:= \sigma_{\sigma_q(e_2) = \emptyset}(e_1). \end{aligned}$$

We observe that

$$\begin{aligned} e_1 \ltimes_q e_2 &\equiv e_1 \ltimes_{\hat{q}} \widehat{e_2}, \\ e_1 \triangleright_q e_2 &\equiv e_1 \triangleright_{\hat{q}} \widehat{e_2}, \end{aligned}$$

where $J = \mathcal{F}(e_2) \cap \mathcal{A}(e_1)$ and $\hat{q} = ((J = J') \wedge q)$. The results of the d-semijoin and d-antijoin are duplicate-free if and only if their left argument is.

We define the left outer d-join analogously to the left outerjoin:

$$e_1 \bowtie_q e_2 := (e_1 \bowtie_q e_2) \cup ((e_1 \blacktriangleright_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}).$$

Let us expand this definition. With $E_{\perp 2} = \{\perp_{\mathcal{A}(e_2)}\}$, $J = \mathcal{F}(e_2) \cap \mathcal{A}(e_1)$, and $\hat{q} = ((J = J') \wedge q)$, we then have

$$\begin{aligned} e_1 \bowtie_q e_2 &\equiv (e_1 \bowtie_q e_2) \cup ((e_1 \blacktriangleright_q e_2) \times E_{\perp 2}) \\ &\equiv (e_1 \bowtie_{\hat{q}} \hat{e}_2) \cup ((e_1 \blacktriangleright_{\hat{q}} \hat{e}_2) \times E_{\perp 2}) \\ &\equiv e_1 \bowtie_{\hat{q}} \hat{e}_2. \end{aligned}$$

The result of a left outer d-join is duplicate-free if its left input and $\hat{e}_2$ are.

Defining a full outer d-join does not make much sense. The third part of the expression

$$(e_1 \bowtie_q e_2) \cup ((e_1 \blacktriangleright_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}) \cup ((e_2 \blacktriangleright_q e_1) \times \{\perp_{\mathcal{A}(e_1)}\})$$

is not even evaluable, since e_2 can only be evaluated in the context of bindings derived from e_1 . One might be tempted to use $\hat{e}_2$ such that the problematic part becomes

$$(\hat{e}_2 \blacktriangleright_q e_1) \times \{\perp_{\mathcal{A}(e_1)}\}.$$

However, we abandon this possibility.

The situation is less complicated for the dependent groupjoin. We can define it as

$$e_1 \bowtie_{q;g:f} e_2 := e_1 \bowtie_{\hat{q};g:f} \hat{e}_2. \quad (7.4)$$

We leave it as an exercise to the reader to show that

$$e_1 \bowtie_{q;g:f} e_2 \equiv e_1 \bowtie_{J=J'}^{g:f(\emptyset)} \Gamma_{q;g:f}(\hat{e}_2), \quad (7.5)$$

where $J = \mathcal{F}(e_2) \cap \mathcal{A}(e_1)$. The result of a dependent groupjoin is duplicate-free if and only if its left input is duplicate-free.

7.4 Linearity of Algebraic Operators

7.4.1 Linearity of Algebraic Operators

The notion of linearity was first used by von Bülzingsloewen to simplify proofs of algebraic equivalences [903]. Since it saves a lot of work, we loosely follow his approach. Let us carry over the definition of linearity as defined for sets to bags. A unary function f from bags to bags is called *strongly linear* if and only if the following two conditions hold for all bags X and Y :

$$\begin{aligned} f(\emptyset_b) &= \emptyset_b, \\ f(X \cup_b Y) &= f(X) \cup_b f(Y). \end{aligned}$$

An n -ary mapping from bags to a bag is called *strongly linear in its i -th argument* if and only if for all bags $X_1, \dots, X_n$ and X'_i the following conditions hold:

$$\begin{aligned} f(X_1, \dots, X_{i-1}, \emptyset_b, X_{i+1}, \dots, X_n) &= \emptyset_b \\ f(X_1, \dots, X_{i-1}, X_i \cup_b X'_i, X_{i+1}, \dots, X_n) &= f(X_1, \dots, X_{i-1}, X_i, X_{i+1}, \dots, X_n) \\ &\quad \cup_b f(X_1, \dots, X_{i-1}, X'_i, X_{i+1}, \dots, X_n) \end{aligned}$$

It is called *strongly linear* if it is strongly linear in all its arguments. For a binary function or operator where we can distinguish between the left and the right argument, we call it strongly left (right) linear if it is strongly linear in its first (second) argument.

Using the commutativity of bag union and bag intersection as well as the observations that in general

$$\begin{aligned} (\emptyset_b \cup_b X) &\neq \emptyset_b, \\ (\emptyset_b \cap_b X) &= \emptyset_b, \\ (\emptyset_b \setminus_b X) &= \emptyset_b, \\ (X \setminus_b \emptyset_b) &\neq \emptyset_b \end{aligned}$$

and

$$\begin{aligned} (X \cup_b Y) \cup_b Z &\neq (X \cup_b Z) \cup_b (Y \cup_b Z), \\ (X \cup_b Y) \cap_b Z &\neq (X \cap_b Z) \cup_b (Y \cap_b Z), \\ (X \cup_b Y) \setminus_b Z &\neq (X \setminus_b Z) \cup_b (Y \setminus_b Z), \\ X \setminus_b (Y \cup_b Z) &\neq (X \setminus_b Y) \cup_b (X \setminus_b Z), \end{aligned}$$

we can conclude that bag union is neither strongly left nor strongly right linear, bag intersection is neither strongly left nor strongly right bag-linear, and bag difference is neither strongly left nor strongly right linear.

We can relax the definition of strongly linear by the additional assumption that the intersection of the two unioned bags is empty. A unary function f from bags to bags is called *weakly linear* if and only if the following two conditions hold for all bags X and Y with $X \cap_b Y = \emptyset_b$:

$$\begin{aligned} f(\emptyset_b) &= \emptyset_b, \\ f(X \cup_b Y) &= f(X) \cup_b f(Y). \end{aligned}$$

An n -ary mapping from bags to a bag is called *weakly linear in its i -th argument* if and only if for all bags $X_1, \dots, X_n$ and X'_i with $X_i \cap_b X'_i = \emptyset_b$ the following conditions hold:

$$\begin{aligned} f(X_1, \dots, X_{i-1}, \emptyset_b, X_{i+1}, \dots, X_n) &= \emptyset_b, \\ f(X_1, \dots, X_{i-1}, X_i \cup_b X'_i, X_{i+1}, \dots, X_n) &= f(X_1, \dots, X_{i-1}, X_i, X_{i+1}, \dots, X_n) \\ &\quad \cup_b f(X_1, \dots, X_{i-1}, X'_i, X_{i+1}, \dots, X_n). \end{aligned}$$

It is called *weakly linear*, if it is weakly linear in all its arguments. For a binary function or operator where we can distinguish between the left and the right

unary		binary		
operator	linear	operator	left lin.	right lin.
Π^D	o	$\cup$	-	-
Π_A	+	$\cap$	o	o
Π_A^D	o	$\setminus$	o	-
σ_p	+	$\times$	+	+
$\chi_{a:e}$	+	$\boxtimes_p$	+	+
χ_f	+	$\boxtimes_p$	+	-
$\Gamma_{\theta G;F}$	-	$\triangleright_p$	+	-
$\nu_{G;g}$	-	$\boxtimes_p$	+	-
μ_g	+	$\boxtimes_p$	-	-
$\mu_{a:g}$	+	$\boxtimes_{p;F}$	+	-
Υ_f	+	$\blacktriangleleft$	+	does not apply
$\Upsilon_{a:f}$	+			
flatten	+			

Table 7.1: Linearity of algebraic operators

argument, we call it weakly left (right) linear if it is weakly linear in its first (second) argument.

Using the commutativity of bag union and bag intersection as well as the observations that in general

$$\begin{aligned}
(\emptyset_b \cup_b X) &\neq \emptyset_b, \\
(\emptyset_b \cap_b X) &= \emptyset_b, \\
(\emptyset_b \setminus_b X) &= \emptyset_b, \\
(X \setminus_b \emptyset_b) &\neq \emptyset_b
\end{aligned}$$

and

$$\begin{aligned}
(X \cup_b Y) \cup_b Z &\neq (X \cup_b Z) \cup_b (Y \cup_b Z), \\
(X \cup_b Y) \cap_b Z &= (X \cap_b Z) \cup_b (Y \cap_b Z), \\
(X \cup_b Y) \setminus_b Z &= (X \setminus_b Z) \cup_b (Y \setminus_b Z), \\
Z \setminus_b (X \cup_b Y) &\neq (Z \setminus_b X) \cup_b (Z \setminus_b Y)
\end{aligned}$$

for $X \cap_b Y = \emptyset_b$, we can conclude that bag union is neither weakly left nor weakly right linear, bag intersection is weakly linear, and bag difference is weakly left but not weakly right linear.

For the whole algebra, Table 7.1 summarizes the linearity properties for all of our algebraic operators. Thereby, a '+' denotes strong linearity, 'o' denotes weak linearity, and '-' denotes neither of them.

Let us take a closer look at the gap between weak and strong linearity. For some bag B , define the unary function f on bags such that

$$\chi_{f(B)}(x) = \begin{cases} 3 & \text{if } x \in B, \\ 0 & \text{if } x \notin B \end{cases}$$

holds. Then, f is weakly linear but not strongly linear. The problem is that f manipulates the multiplicity of the elements. We can make the difference between weakly and strongly linear explicit. Therefore, we remember that the only difference in the definition was the disjointness we required for weak linearity. Consequently, we consider now the special case of bags containing a single element multiple times. We say that a unary function f is *duplicate faithful* if and only if for all x

$$f(\{x^m\}_b) = \cup_{i=1}^m f(\{x\}_b)$$

holds. Then, a unary function is strongly bag linear if and only if it is weakly bag linear and duplicate faithful. The same holds for n -ary functions if we extend the property duplicate faithful to multiple arguments.

To see that the left semijoin is not even weakly right linear, consider the following example:

$$\begin{aligned} \{[a : 1]\}_b &= \{[a : 1]\}_b \bowtie_{a=b} \{[b : 1, c : 1], [b : 1, c : 2]\}_b \\ &= \{[a : 1]\}_b \bowtie_{a=b} (\{[b : 1, c : 1]\}_b \cup \{[b : 1, c : 2]\}_b) \\ &\neq (\{[a : 1]\}_b \bowtie_{a=b} \{[b : 1, c : 1]\}_b) \cup (\{[a : 1]\}_b \bowtie_{a=b} \{[b : 1, c : 2]\}_b) \\ &= \{[a : 1]^2\}_b. \end{aligned}$$

This is the reason why some equivalences valid for sets do not hold for bags anymore. For example, $\Pi_{\mathcal{A}(e_1)}(e_1 \bowtie_{q_{12}} e_2) \equiv e_1 \bowtie_{q_{12}} e_2$ holds for sets but not for bags. If we eliminate duplicates explicitly, we still have

$$\Pi_{\mathcal{A}(e_1)}^D(e_1 \bowtie_{q_{12}} e_2) \equiv \Pi_{\mathcal{A}(e_1)}^D(e_1 \bowtie_{q_{12}} e_2). \quad (7.6)$$

Similiarly, we have

$$\Pi_{\mathcal{A}(e_1)}^D(e_1 \bowtie_{q_{12}} e_2) \equiv \Pi_{\mathcal{A}(e_1)}^D(e_1), \quad (7.7)$$

$$\Pi_{\mathcal{A}(e_1)}^D(e_1 \bowtie_{q_{12}} e_2) \equiv \Pi_{\mathcal{A}(e_1)}^D(e_1). \quad (7.8)$$

Let us now present some sample proofs of linearity. All proofs are by induction on the number of distinct elements contained in the argument bags.

χ_f is strongly linear.

$$\begin{aligned} \chi_f(\emptyset_b) &= \emptyset_b \\ \chi_f(\{x^m\}_b) &= \cup_{i=1}^m f(\{x\}_b) \\ \chi_f(e_1 \cup e_2) &= \{f(x) | x \in e_1 \cup e_2\}_b \\ &= \{f(x) | x \in e_1\}_b \cup \{f(x) | x \in e_2\}_b \\ &= \chi_f(e_1) \cup \chi_f(e_2) \end{aligned}$$

$\bowtie$ is strongly left linear.

$$\begin{aligned} \emptyset_b \bowtie_q e_2 &= \emptyset_b \\ \{x^m\}_b \bowtie_q e_2 &= \cup_{i=1}^m (\{x\}_b \bowtie_q e_2) \\ (e'_1 \cup e''_1) \bowtie_q e_2 &= ((e'_1 \cup e''_1) \bowtie_q e_2) \cup (((e'_1 \cup e''_1) \triangleright_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}_b) \\ &= (e'_1 \bowtie_q e_2) \cup (e''_1 \bowtie_q e_2) \cup ((e'_1 \triangleright_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}_b) \cup ((e''_1 \triangleright_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}_b) \\ &= (e_1 \bowtie_q e) \cup (e_2 \bowtie_q e) \end{aligned}$$

Here, we exploited the linearity of join and antijoin. Since $e_1 \bowtie_q \emptyset_b = \emptyset_b$ if and only if $e_1 = \emptyset_b$, $\bowtie$ is not even weakly right linear.

$\bowtie$ is strongly left linear.

$$\begin{aligned}
\emptyset_b \bowtie e_2 &= \emptyset_b \\
\{x^m\}_b \bowtie e_2 &= \cup_{i=1}^m (\{x\}_b \bowtie e_2) \\
(e'_1 \cup e''_1) \bowtie e_2 &= \{y \circ x | y \in e'_1 \cup e''_1, x \in e_2(y)\}_b \\
&= \{y \circ x | y \in e'_1, x \in e_2(y)\}_b \cup \{y \circ x | y \in e''_1, x \in e_2(y)\}_b \\
&= (e'_1 \bowtie e_2) \cup (e''_1 \bowtie e_2)
\end{aligned}$$

Note that the notion of linearity cannot be applied to the second (inner) argument of the d-join, since, in general, it cannot be evaluated independently of the first argument.

$\Gamma_{G::f}$ is not linear.

Consider the following counterexample:

$$\begin{aligned}
&\Gamma_{a;g:\text{id}}(\{[a : 1, b : 1], [a : 1, b : 2]\}_b) \\
&= \{[a : 1, g : \{[a : 1, b : 1], [a : 1, b : 2]\}_b]\}_b \\
&\neq \{[a : 1, g : \{[a : 1, b : 1]\}_b]\}_b \cup \{[a : 1, g : \{[a : 1, b : 2]\}_b]\}_b \\
&= \Gamma_{a;g:\text{id}}(\{[a : 1, b : 1]\}_b) \cup \Gamma_{a;g:\text{id}}(\{[a : 1, b : 2]\}_b).
\end{aligned}$$

μ_g is strongly linear.

$$\begin{aligned}
\mu_g(\emptyset_b) &= \emptyset_b \\
\mu_g(\{x^m\}_b) &= \cup_{i=1}^m (\mu_g(\{x\}_b)) \\
\mu_g(e_1 \cup e_2) &= \{x.[\bar{g}] \circ y | x \in e_1 \cup e_2, y \in x.g\}_b \\
&= \{x.[\bar{g}] \circ y | x \in e_1, y \in x.g\}_b \cup \{x.[\bar{g}] \circ y | x \in e_2, y \in x.g\}_b \\
&= \mu_g(e_1) \cup \mu_g(e_2)
\end{aligned}$$

$\mu_{a:g}$ is also linear. This is shown analogously to the linearity of μ_g .

flatten is strongly linear.

$$\begin{aligned}
\text{flatten}(\emptyset_b) &= \emptyset_b \\
\text{flatten}(\{x^m\}_b) &= \cup_{i=1}^m (\text{flatten}(x)) \\
\text{flatten}(e_1 \cup e_2) &= \{x | y \in e_1 \cup e_2, x \in y\}_b \\
&= \{x | y \in e_1, x \in y\}_b \cup \{x | y \in e_2, x \in y\}_b \\
&= \text{flatten}(e_1) \cup \text{flatten}(e_2)
\end{aligned}$$

Note that the notion of linearity does not apply to the *max* operator, since it does not return a bag.

7.4.2 Exploiting Linearity

The concatenation of two weakly (strongly) linear mappings is again a weakly (strongly) linear mapping. Assume f and g to be weakly (strongly) linear mappings. Then

$$\begin{aligned} f(g(\emptyset_b)) &= \emptyset_b, \\ f(g(\{x^m\}_b)) &= \cup_{i=1}^m f(g(\{x\}_b)), \\ f(g(X \cup Y)) &= f(g(X) \cup g(Y)) \\ &= f(g(X)) \cup f(g(Y)), \end{aligned}$$

where the second line only applies for strongly linear mappings, and the bags X and Y are disjoint in case of weakly linear mappings.

From the linearity considerations of the previous subsection, it is easy to derive reorderability laws.

Let $f : \{\tau_1^f\} \rightarrow \{\tau_2^f\}$ and $g : \{\tau_1^g\} \rightarrow \{\tau_2^g\}$ be two strongly linear mappings. If $f(g(\{x\}_b)) = g(f(\{x\}_b))$ for all bags $\{x\}_b$ containing a single element, then

$$f(g(e)) = g(f(e)) \quad (7.9)$$

This is proven by induction on the number of distinct elements contained in a bag e . If e is empty, the statement follows directly from the linearity of f and g . For the induction, let $e = e_1 \cup e_2$. Then

$$\begin{aligned} f(g(e)) &= f(g(e_1 \cup e_2)) \\ &= f(g(e_1) \cup g(e_2)) \\ &\stackrel{I.H.}{=} g(f(e_1)) \cup g(f(e_2)) \\ &= g(f(e_1 \cup e_2)) \\ &= g(f(e)). \end{aligned}$$

□

For strongly linear algebraic operators working on bags of tuples, we can replace the semantic condition $f(g(\{x\}_b)) = g(f(\{x\}_b))$ by a syntactic criterion. The main issue here is to formalize that two operations do not interfere in their consumer/producer/modifier relationship on attributes. Let us first get rid of modifications. There are only a few algebraic operators which are capable of modifying an attribute's value. One of them is the map operator. By a proper renaming of attributes, we can assume without loss of generality that no operator modifies an existing attribute value. For the map operator (say $\chi_{a:e_2}(e_1)$), it would mean that it only introduces new attributes (thus $a \notin \mathcal{A}(e_1)$). This renaming step is an essential part of the NFST phase.

This leaves us with checking the consumer/producer relationships. Consider, for example, the algebraic equivalence

$$\sigma_p(e_1 \bowtie_{q_{12}} e_2) \equiv (\sigma_p(e_1)) \bowtie_{q_{12}} e_2.$$

It is well-typed if and only if the predicate p does not access any attributes from e_2 , i.e., $\mathcal{F}(p) \cap \mathcal{A}(e_2) = \emptyset$.

Now, for most of our operators we can be sure that $f(g(e)) = g(f(e))$ for singleton bags e . This holds if and only if the following two conditions hold:

unary			binary		
operator	produced	deleted	operator	produced	deleted
Π^D	$\emptyset$	$\emptyset$	$\cup$	$\emptyset$	$\emptyset$
Π_A	$\emptyset$	$\overline{A}$	$\cap$	$\emptyset$	$\emptyset$
Π_A^D	$\emptyset$	$\overline{A}$	$\setminus$	$\emptyset$	$\mathcal{A}(e_2)$
σ_p	$\emptyset$	$\emptyset$	$\times$	$\emptyset$	$\emptyset$
$\chi_{a:e}$	$\{a\}$	$\emptyset$	$\bowtie_q$	$\emptyset$	$\emptyset$
$\Gamma_{\theta G;F}$	$\mathcal{A}(F)$	$\overline{G}$	$\bowtie_q$	$\emptyset$	$\mathcal{A}(e_2)$
$\nu_{G;g}$	$\{g\}$	$\overline{G}$	$\triangleright_q$	$\emptyset$	$\mathcal{A}(e_2)$
μ_g	$\mathcal{A}(g)$	$\{g\}$	$\bowtie_q$	$\mathcal{A}(e_2)$	$\emptyset$
$\mu_{a:g}$	$\{a\}$	$\{g\}$	$\bowtie_q$	$\mathcal{A}(e_1) \cup \mathcal{A}(e_2)$	$\emptyset$
$\Upsilon_{a:f}$	$\{a\}$	$\emptyset$	$\bowtie_{q;g:F}$	$\{g\}$	$\emptyset$

Table 7.2: Produced and deleted attributes of algebraic operators

1. g does not access attributes produced by f , and
2. f does not access attributes produced by g .

We can formalize this as follows. We denote by $\mathcal{P}$ the set of *produced attributes* and by $\mathcal{D}$ the set of *destroyed attributes* (e.g., projected away). Given a unary operator f and an expression e , and a binary operator $\circ$ and expressions e_1 and e_2 , we can define

$$\begin{aligned}
\mathcal{P}(f) &:= \mathcal{A}(f(e)) \setminus \mathcal{A}(e), \\
\mathcal{D}(f) &:= \mathcal{A}(e) \setminus \mathcal{A}(f(e)), \\
\mathcal{P}(\circ) &:= \mathcal{A}(e_1 \circ e_2) \setminus (\mathcal{A}(e_1) \cup \mathcal{A}(e_2)), \\
\mathcal{D}(\circ) &:=^* (\mathcal{A}(e_1) \cup \mathcal{A}(e_2)) \setminus \mathcal{A}(e_1 \circ e_2).
\end{aligned}$$

A special case concerns the produced attributes of outerjoins. Since attribute values are assigned in case of null-padding a tuple with no join partner, we have to add the attributes of the preserved side(s) to the set of produced attributes. Table 7.2 shows the sets of produced and deleted attributes for some selected algebraic operators.

Using this notation, the condition $f(g(\{x\}_b)) = g(f(\{x\}_b))$ is satisfied if the two conditions

$$\begin{aligned}
\mathcal{F}(f) \cap \mathcal{P}(g) &= \mathcal{P}(f) \cap \mathcal{F}(g) = \emptyset_b, \\
\mathcal{F}(f) \cap \mathcal{D}(g) &= \mathcal{D}(f) \cap \mathcal{F}(g) = \emptyset_b
\end{aligned}$$

hold and if f and g are any of our unary operators. This statement is valid because we excluded attribute modifications.

7.5 Representations

7.5.1 Three Different Representations

In this section, we discuss different representations for sets, bags, and sequences. Let us start with bags. Fig. 7.5 shows three different representations for bags.

R		R			R		
A	B	A	B	$\mathfrak{m}$	A	B	i
1	1	1	1	2	1	1	1
1	1	2	2	1	1	1	2
2	2				2	2	3

Figure 7.5: Three possible representations of a bag

The first representation (left) is the usual representation. Thus, we call it *standard representation*. Here, duplicates are represented by duplicating tuples. The order of appearance of the tuples in a bag is immaterial. Thus, the tuples could be represented in any order and still yield the same bag. The second representation (middle) contains every tuple only once, and the multiplicity of a tuple is given explicitly in the special attribute $\mathfrak{m}$. Hence, we call it *multiplicity-based representation*. Again, the order of tuples is immaterial. The third representation (right) adds a surrogate or tuple identifier i to each tuple. Again, the attribute i is special and not visible to the user. Klausner and Goodman say that i is a *hidden attribute* [497, 498]. In the example, it is a non-negative integer. We call this representation *tid-based*. The order of tuples is again immaterial if we use this representation for bags. The only property we need is the uniqueness of the TID-attribute i .

Observe that although the bag represented contains duplicates, the multiplicity-based and the tid-based representations are duplicate-free, i.e., they are sets. For a bag e in a multiplicity-based representation, we even assume that $\Pi_{\mathfrak{m}}(e)$ is duplicate-free. This assumption will be relaxed later on. Note that if there are a lot of duplicates, then the multiplicity-based representation requires far less storage than the other representations. Otherwise, the storage overhead for the multiplicity-based and tid-based representations is negligible if the original tuples are not too small.

For a set, all three representations are valid. Further, all of them are sets. Note that in the multiplicity-based representation $\mathfrak{m} = 1$ for all tuples.

Sequences are a little trickier. Here, the order is important. We must thus assume that all representations imply some implicit order. This implicit order can be caused, for example, by a certain storage order, a list representation, or a tuple stream. For the rest of this section, we assume that all representations of bulk types are based on streams of tuples. Thus, they have an implicit order, which in case of sets or bags might not be relevant. A typical example of a relevant implicit order is a document scan in XQuery, where the resulting nodes in the stream are in document order.

Consider now the tid-based representation. We require that i reflects the order of tuples in the sequence. For any two tuples t_1 and t_2 , we must have that t_1 occurs before t_2 in the sequence if and only if $t_1.i < t_2.i$. Consider the sequence

$$\langle [a : 1], [a : 2], [a : 1] \rangle.$$

Clearly, the multiplicity-based representation $\langle [a : 1, \mathfrak{m} : 2], [a : 2, \mathfrak{m} : 1] \rangle$ loses

the order. One way to remedy this situation is to keep not only the multiplicity of an element but its positions. This results in

$$\langle [a : 1]^{1,3}, [a : 2]^2 \rangle$$

or in

$$\langle [a : 1, \mathbf{p} : \{1, 3\}_s], [a : 2, \mathbf{p} : \{2\}_s] \rangle$$

if we represent the set of positions at which a tuple occurs in an extra attribute $\mathbf{p}$. We call this a *position-based representation*. It is duplicate free and in case of multiple duplicates in the original sequence, it saves some memory.

7.5.2 Conversion between Representations

Assume that the bag e is given in standard representation, and we wish to convert it to a multiplicity-based representation. Then $\Gamma_{\mathcal{A}(e); \mathbf{p}; \mathbf{m}; |\mathbf{p}|}(e)$ does the job. To go into the other direction, we need a special unnest operator, which, for a given attribute $\mathbf{m}$ in non-negative integers, produces $\mathbf{m}$ copies of a tuple. Since it looks like a special unnest operator, we use $\mu_{\mathbf{m}}$ to denote it. It is defined as

$$\mu_{\mathbf{m}}(e) := \Pi_{\overline{\mathbf{m}}}(\{t^{\mathbf{m}} | t \in e\}_b)$$

for attributes $\mathbf{m}$ in non-negative integers. Then, $\mu_{\mathbf{m}}(e)$ converts a bag e from multiplicity-based to standard representation.

To go from a standard representation of a bag to a tid-based representation, we apply a special *tid-operator* TID_i , which produces distinct TIDs (numbers) for every input tuple and stores this number in the attribute i . Assume some global variable c is initialized with 0, then $\text{TID}_i(e)$ could be defined as $\chi_{i; ++c}(e)$, mixing algebra and C++ code. The reverse direction is easily specified by $\Pi_{\overline{i}}$.

Let us now turn to sets. Converting a set e in standard notation to a multiplicity-based representation is performed by $\chi_{\mathbf{m}; 1}(e)$. Converting a set e in multiplicity-based representation to one in standard representation can be simplified to $\Pi_{\overline{\mathbf{m}}}(e)$. The other conversions for sets are the same as for bags.

Given a sequence e in standard representation with implicit ordering, we can apply the above TID-operator to convert e into a tid-based representation with $\text{TID}_i(e)$. The reverse conversion is not simply $\Pi_{\overline{i}}(e)$, except if we are sure that e is already sorted on i . Otherwise, since sorting on i will restore the order, we can convert a sequence e in tid-based representation with $\Pi_{\overline{i}}(\text{Sort}_i(e))$, where we again must assume an implicit ordering.

We can construct a position-based representation of a sequence given in a tid-based representation with $\Gamma_{\mathcal{A}(e) \setminus \{i\}; g; \Pi_i}(e)$. The opposite direction is specified by $\text{Sort}_i(\mu_g(e))$. As an exercise, the reader should design an algorithm which allows to calculate the combination $\text{Sort}_i \circ \mu_g$ efficiently. EXC

7.5.3 Conversion between Bulk Types

Given a set e in some representation, the same representation is a valid bag representation. Hence, this conversion is a no-op.

The opposite direction is also simple. Given a bag e in standard representation, Π^D produces a set in standard representation. This is expensive. Given a bag e in tid-based representation, $\Pi_{\mathcal{A}(e) \setminus \{i\}}^D(e)$ produces a set in standard representation. This is expensive, too. Given a bag e in multiplicity-based representation, $\Pi_{\overline{\mathbf{m}}}(e)$ produces a set in standard representation. This is cheap.

Going from a sequence to a bag is simple. If the sequence is given in standard representation with implicit order or in tid-based representation, the conversion to a bag is the identity function. If a sequence e is given in the position-based representation with attribute p containing the set of positions, then $\Pi_{\overline{p}}(\chi_{\mathbf{m}:|p|}(e))$ converts it to a bag in multiplicity-based representation. From there, we can go anywhere. Obviously, going from a bag to a sequence requires explicit sorting.

7.5.4 Adjusting the Algebra

Implicitly, we have defined our algebra on the standard representation. This is not coercive. Consider two bags e_1 and e_2 in a multiplicity-based representation. We can define a special *counting cross product* by

$$e_1 \times^{\mathbf{m}_{12}:\mathbf{m}_1*\mathbf{m}_2} e_2 := \Pi_{\overline{\mathbf{m}_1}, \overline{\mathbf{m}_2}}(\{t_1 \circ t_2 \circ [\mathbf{m} : t_1.\mathbf{m}_1 * t_2.\mathbf{m}_2] \mid t_i \in e_i\})$$

Doing the same exercise with the regular join operator results in the so-called *counting join* [944].

Luckily, it is not necessary to introduce a special counting cross product, as can be seen from

$$e_1 \times^{\mathbf{m}_{12}:\mathbf{m}_1*\mathbf{m}_2} e_2 \equiv \Pi_{\overline{\mathbf{m}_1}, \overline{\mathbf{m}_2}}(\chi_{\mathbf{m}_{12}:\mathbf{m}_1*\mathbf{m}_2}(e_1 \times e_2)),$$

which can be generalized to

$$\Pi_{\overline{\{\mathbf{m}_i\}}}(\chi_{\mathbf{m}:\prod_i \mathbf{m}_i}(e_1 \times \dots \times e_n))$$

if we want to take the counting cross product of n bags e_i .

Similarly, we can handle a cross product of sequences in a position-based representation, which is left as an exercise to the reader.

Let us turn to projection. Consider the bag

$$\{[a : 1, b : 2, \mathbf{m} : 3], [a : 2, b : 3, \mathbf{m} : 4], [a : 1, b : 4, \mathbf{m} : 2]\}_b$$

in the multiplicity-based representation. Applying Π_a carelessly results in

$$\{[a : 1, \mathbf{m} : 3], [a : 2, \mathbf{m} : 4], [a : 1, \mathbf{m} : 2]\}_b,$$

which is no longer a multiplicity-based representation, as it is not a set anymore. More specifically, Π_a of the above bag contains duplicates. We use this representation as an alternative fourth representation. We call it *multiplicity-based representation with duplicates*. First, note that in terms of conversions and algebraic operators like join, the duplicates do not imply problems. We just lose some compression of the data. Second, observe that performing a $\Gamma_{a:\mathbf{m}:sum(\mathbf{m})}$ fixes this problem, i.e., it turns a bag in multiplicity-based representation with duplicates into the regular, duplicate-free multiplicity-based representation.

7.5.5 Partial Preaggregation

To illustrate *partial preaggregation* (or *partial pregrouping*) we use a hash-based implementation of the grouping operator as an example. Assume that it has limited buffer space and can keep k groups. If the buffer overflows, some group is ejected from the buffer to produce new free space for a new group. We denote such a special implementation of the grouping operator by $\Gamma^{\text{pre}(k)}$ and call it *partial preaggregation* or *partial pregrouping*.

We illustrate it by applying it in the form of $\Gamma_{a;g;\mathbf{m};|g|}^{\text{pre}(1)}$ to the following bag

$$B := \{[a : 1], [a : 1], [a : 2], [a : 1], [a : 1]\}_b.$$

The result is

$$B_{\text{pre}} := \{[a : 1, \mathbf{m} : 2], [a : 2, \mathbf{m} : 1], [a : 1, \mathbf{m} : 2]\}_b,$$

where we assumed that the implicit order in which the pregrouping operator sees the tuples is from left to right. Calculating $\Gamma_{a;g;\mathbf{m};\text{sum}(\mathbf{m})}$ gives with

$$B_m := \{[a : 1, \mathbf{m} : 4], [a : 2, \mathbf{m} : 1]\}_b$$

the regular duplicate-free multiplicity-based representation. This observation also holds for sets of attributes, as in

$$\Gamma_{G;\mathbf{m};\text{count}(\ast)}(e) \equiv \Gamma_{G;\mathbf{m};\text{sum}(\mathbf{m}')}(\Gamma_{G;\mathbf{m}';\text{count}(\ast)}^{\text{pre}(k)}(e)) \quad (7.10)$$

for any $k \geq 0$, where we used the SQL-notation based variant of grouping.

Recall that aggregation functions and vectors can be decomposable. Then it is easy to generalize the above equivalence. Let F be an aggregation vector decomposable into F^1 and F^2 . Then

$$\Gamma_{G;F}(e) \equiv \Gamma_{G;F^2}(\Gamma_{G;F^1}^{\text{pre}(k)}(e)) \quad (7.11)$$

holds. If the grouping operator is pushed into a join or any other binary operator and still some outer grouping is present (see Sec. 7.11), then the inner grouping can be replaced by a pregrouping. General partial pregrouping or preaggregation is discussed in several papers [425, 534]. They also discuss the expected resulting number of tuples of partial pregrouping.

7.6 A Note on Equivalences

We have already seen expressions like $E_1 \equiv E_2$, where the expressions E_i contain algebraic operators and other symbols. Typically, they contain variables e_i to denote base tables or other database items or even algebraic expressions. They also contain p or q for predicates. These are essentially variables for predicates. Also attributes names a or b can be contained in the E_i . Again, these are essentially place holders for ‘real’ attribute names. Sometimes constants c are used. Let us collectively call all these underspecified symbols variables. We will then say that $E_1 \equiv E_2$ if and only if the following condition holds: For all

bindings for all variables in E_1 and E_2 , if E'_1 and E'_2 , which result from applying these bindings to them, are well-typed, then the evaluation of E'_1 and E'_2 yields the same result.

Equivalence of relational expressions is discussed, e.g., by Aho, Saviv, and Ullman [17, 18]. In fact, they discuss weak and strong equivalence. We use strong equivalence. Weak equivalence is defined on universal relations and is not sufficient for our purpose.

7.7 Simple Reorderability

7.7.1 Unary Operators

The simplest property we are interested in is the *idempotency* of unary operators. A unary operator f is called *idempotent*, if $f(f(x)) = f(x)$ for all x . Since $\Pi^D(x) = \Pi^D(\Pi^D(x))$, $\Pi_A(x) = \Pi_A(\Pi_A(x))$, $\Pi_A^D(x) = \Pi_A^D(\Pi_A^D(x))$, and $\sigma_p(x) = \sigma_p(\sigma_p(x))$, these operators are idempotent. Our other operators are not. For projections, we also have some generalized idempotency. Let A and B be two attribute sets with $A \subseteq B$. Then $\Pi_A(x) = \Pi_A(\Pi_B(x))$, $\Pi_A^D(x) = \Pi_A^D(\Pi_B^D(x))$, and $\Pi_A^D(x) = \Pi_A^D(\Pi_B(x))$.

As can be seen from the definition of the grouping operator, it is a generalization of duplicate elimination. If we apply a grouping with an empty aggregation vector, then it is equivalent to a duplicate elimination. In other words,

$$\Pi_A^D(e) \equiv \Gamma_{A;()}(e) \quad (7.12)$$

holds for any set of attributes A with $A \subseteq \mathcal{A}(e)$. As a consequence, we have to ask ourselves whether there exists a property generalizing idempotency that holds for grouping. Indeed there is one. Let F be an aggregation vector which is decomposable into F^1 and F^2 , and G and G^+ be two sets of grouping attributes with $G \subseteq G^+$. Then

$$\Gamma_{G;F}(e) \equiv \Gamma_{G;F^2}(\Gamma_{G^+;F^1}(e)) \quad (7.13)$$

holds, since we can first group at a finer granularity and then combine finer groups to the groups derived from the grouping by G . We can even go a step further in the presence of functional dependencies. Assume the functional dependency $G \rightarrow G'$ holds for two sets of grouping attributes G and G' . Then, the equivalence

$$\Gamma_{G;F}(e) \equiv \Pi_{G \cup \mathcal{A}(F)}(\Gamma_{G \cup G';F}(e)) \quad (7.14)$$

holds, since the groups and their contents are the same in both cases. This equivalence can also be found under the name *simplify group-by* in a paper by Tsois and Sellis [882]. A slightly more general version for any function f also holds:

$$\Gamma_{G;g:f}(e) \equiv \Pi_{G \cup \{g\}}(\Gamma_{G \cup G';g:f}(e)). \quad (7.15)$$

Eqv. 7.14 can be simplified if, in addition to $G \rightarrow G'$, $G \subseteq G'$ holds:

$$\Gamma_{G;F}(e) \equiv \Pi_{G \cup \mathcal{A}(F)}(\Gamma_{G';F}(e)). \quad (7.16)$$

	Π^D	Π_A	Π_A^D	σ	χ	Γ	ν	μ	Υ
Π^D	+	-	-	+	+	-	-	(-)	(-)
Π_A	-	-	-	+	+	-	-	+	+
Π_A^D	-	-	-	+	+	-	-	(-)	(-)
σ	+	+	+	+	+	o	+	+	+
χ	+	+	+	+	+	-	-	+	+
Γ	-	-	-	o	-	-	-	-	-
ν	-	-	-	+	-	-	-	-	-
μ	(-)	+	(-)	+	+	-	-	+	+
Υ	(-)	+	(-)	+	+	-	-	+	+

Table 7.3: Reorderability of unary operators

While introducing linearity, we have already discussed the usefulness of linearity for reordering unary operators. Table 7.3 shows a '+' sign if $f(g(x)) \equiv g(f(x))$ for two unary operators f and g . If this does not hold, the according entry contains a '-'. Thereby, we have to take care that the consumer/producer relationship is not disturbed. For example, $\chi_{a:e}$ and σ_p can only be reordered if $a \notin \mathcal{F}(p)$. Since this should be clear by now, we will not always mention it explicitly anymore.

The cases marked by '(-)' involve the unnest or unnest map operator, and they need an additional condition to be reorderable. If we want to reorder μ_g with a duplicate eliminating projection on some input e , we must require that $t.g$ is duplicate-free for all $t \in e$. For Υ_f , we must require that $f(t)$ is duplicate-free for all $t \in e$.

In general, reordering a selection with a grouping $\Gamma_{\theta G;g:f}$ is wrong. If, however, all comparison operators are based on equality, we can reorder it with a selection σ_p as in

$$\sigma_p(\Gamma_{G;g:f}(e)) \equiv \Gamma_{G;g:f}(\sigma_p(e)). \quad (7.17)$$

Of course, this requires that $g \notin \mathcal{F}(p)$ and $(\mathcal{F}(p) \cap \mathcal{A}(e)) \subseteq G$.

A real annoyance is the fact that we cannot reorder a map with a grouping operator. But this situation can be remedied. Then, the map as well as the selection will become reorderable with all operators needed in the context of SQL. Consider the expression $\chi_{a:e_2}(\Gamma_{G;g;F}(e_1))$. It is valid only if $\mathcal{F}(e_2) \subseteq G \cup \mathcal{A}(F)$. If $\mathcal{F}(e_2) \cap \mathcal{A}(F) \neq \emptyset$, there is no hope to change the order of the map and grouping operators. Thus, assume that $\mathcal{F}(e_2) \subseteq G$. The expression $\Gamma_{G;g;F}(\chi_{a:e_2}(e_1))$ typically does not contain attribute a . However, since $\mathcal{F}(e_2) \subseteq G$, we observe that $G \rightarrow a$ if e_2 contains only deterministic functions, which we assume. Thus, we have

$$\chi_{a:e_2}(\Gamma_{G;g;F}(e_1)) = \Gamma_{G \cup \{a\};g;F}(\chi_{a:e_2}(e_1)), \quad (7.18)$$

and

$$\chi_{a:e_2}(\Gamma_{G;F}(e_1)) = \Gamma_{G \cup \{a\};F}(\chi_{a:e_2}(e_1)) \quad (7.19)$$

if $\mathcal{F}(e_2) \subseteq G$. Whereas the expression e_2 is evaluated once per group on the left-hand side, it is evaluated once per item in e_1 on the right-hand side. This

	$\cup$	$\cap$	$\setminus$	$\times$	$\bowtie$	$\ltimes$	$\rhd$	$\blacktriangleright$	$\bowtie$	$\ltimes$	$\blacktriangleright$
Π^D	$\circ/\circ$	$+/+$	$+/\circ$	$\circ/\circ$	$\circ/\circ$	$+/-$	$+/-$	$\circ/-$	$\circ/-$	$\circ/\circ$	$+/-$
Π_A	$-/-$	$-/-$	$-/-$	$-/-$	$-/-$	$+/-$	$+/-$	$-/-$	$-/-$	$-/-$	$-/-$
Π_A^D	$-/-$	$-/-$	$-/-$	$-/-$	$-/-$	$+/-$	$+/-$	$-/-$	$-/-$	$-/-$	$-/-$
σ	$-/-$	$+/+$	$+/-$	$+/+$	$+/+$	$+/-$	$+/-$	$+/+$	$+/-$	$-/-$	$+/-$
χ	$-/-$	$-/-$	$-/-$	$+/+$	$+/+$	$+/-$	$+/-$	$+/+$	$+/+$	$\circ/\circ$	$+/-$
Γ	$-/-$	$-/-$	$-/-$	$-/-$	$-/-$	$+/-$	$+/-$	$-/-$	$-/-$	$-/-$	$-/-$
ν	$-/-$	$-/-$	$-/-$	$-/-$	$-/-$	$+/-$	$+/-$	$-/-$	$-/-$	$-/-$	$-/-$
μ	$-/-$	$-/-$	$-/-$	$+/+$	$+/+$	$+/-$	$+/-$	$+/+$	$+/-$	$-/-$	$+/-$
Υ	$-/-$	$-/-$	$-/-$	$+/+$	$+/+$	$+/-$	$+/-$	$+/+$	$+/-$	$-/-$	$+/-$

Table 7.4: Left/right push-down

is unfortunate if we want to apply the equation from left to right. Remember that $\mathcal{A}(g) = \mathcal{A}(e_1)$ in $\Gamma_{G;g;F}(e_1)$. Thus, $\mathcal{A}(e_2) \subseteq G \subseteq \mathcal{A}(g)$, and we can add the calculation of e_2 to F . Therefore, we need the function *pick*, which picks an arbitrary element out of a bag. This is deterministic, since all tuples in a group g have the same values for all attributes contained in G . Then, we have

$$\chi_{a:e_2}(\Gamma_{G;g;F}(e_1)) = \Gamma_{G;g;F \circ (a:(e_2(\text{pick}(g))))}(e_1) \quad (7.20)$$

if $\mathcal{F}(e_2) \subseteq G$. In our SQL-notation variant of Γ , this reads like

$$\chi_{a:e_2}(\Gamma_{G;F}(e_1)) = \Gamma_{G;g;F \circ (a:e_2(\text{pick}(g)))}(e_1) \quad (7.21)$$

if $\mathcal{F}(e_2) \subseteq G$.

7.7.2 Push-Down/Pull-Up of Unary into/from Binary Operators

In this section, we consider pushing down (pulling up) unary operators into (from) the arguments of binary operators. Thus, we are interested in equivalences of the form $f(e_1 \circ e_2) \equiv f(e_1) \circ e_2$ and $f(e_1 \circ e_2) \equiv e_1 \circ f(e_2)$. First, let us see how linearity helps in this context. Let f be a unary, strongly linear mapping and $\circ$ a binary mapping that is strongly linear in its left argument. If for all expressions e_1 and e_2 and for all $x_i \in e_i$ we have $f(\{x_1\}_b \circ \{x_2\}_b) = (f(\{x_1\}_b) \circ \{x_2\}_b)$, then

$$f(e_1 \circ e_2) \equiv f(e_1) \circ e_2.$$

If e_1 is empty, $f(e_1 \circ e_2)$ and $f(e_1) \circ e_2$ are also empty. If e_1 is a singleton bag, the claim follows from the prerequisite. For the induction step, we observe that

$$\begin{aligned}
f(e_1 \circ e_2) &= f((e'_1 \circ e_2) \cup (e''_1 \circ e_2)) \\
&= f(e'_1 \circ e_2) \cup f(e''_1 \circ e_2) \\
&\stackrel{I.H.}{=} (f(e'_1) \circ e_2) \cup (f(e''_1) \circ e_2) \\
&= f(e_1) \circ e_2
\end{aligned}$$

if $e_1 = e'_1 \cup e''_1$. Our prerequisite required $f(\{x_1\}_b \circ \{x_2\}_b) = (f(\{x_1\}_b) \circ \{x_2\}_b)$. If instead the stronger $f(\{x_1^m\}_b \circ \{x_2^n\}_b) = (f(\{x_1^m\}_b) \circ \{x_2^n\}_b)$ holds and $\circ$ is weakly left linear, then it suffices to push f down into the left argument of $\circ$. This follows from the above prove and the additional condition $e'_1 \cap e''_1 = \emptyset$. The induction is on the number of distinct elements in e_1 .

Table 7.4 summarizes the validity of pushing a unary operator down into the left or right argument of a binary operator. Again, some restrictions apply. First, we restrict ourselves to the map operator in its extending form $\chi_{a:e}$. Other critical cases are marked by $\circ$. They include duplicate elimination and outerjoins. We open our discussion with duplicate elimination. Since duplicate elimination is weakly but not strongly linear, it is not surprising that we need additional conditions to push it down a binary operator. We have

$$\begin{array}{lll}
\Pi^D(e_1 \cup e_2) & \equiv & \Pi^D(e_1) \cup e_2 & \text{if } \text{dupfree}(e_2) \wedge (e_1 \cap e_2) = \emptyset_b, \\
\Pi^D(e_1 \times e_2) & \equiv & \Pi^D(e_1) \times e_2 & \text{if } \text{dupfree}(e_2), \\
\Pi^D(e_1 \bowtie_{q_{12}} e_2) & \equiv & \Pi^D(e_1) \bowtie_{q_{12}} e_2 & \text{if } \text{dupfree}(e_2), \\
\Pi^D(e_1 \Join e_2) & \equiv & \Pi^D(e_1) \Join e_2 & \text{if } \forall t_1 \in e_1 \text{ dupfree}(e_2(t_1)), \\
\Pi^D(e_1 \Join_{q_{12}} e_2) & \equiv & \Pi^D(e_1) \Join_{q_{12}} e_2 & \text{if } \text{dupfree}(e_2), \\
\Pi^D(e_1 \Join_{q_{12}} e_2) & \equiv & \Pi^D(e_1) \Join_{q_{12}} e_2 & \text{if } \text{dupfree}(e_2),
\end{array}$$

where $\text{dupfree}(e)$ denotes the fact that e is duplicate-free.

Let us now take a closer look at the case where we try to push down a map operator into the right-hand side of a left outerjoin. Consider the expression $\chi_{a_2:f_2}(e_1 \Join_{q_{12}} e_2)$, where $\mathcal{F}(f_2) \cap \mathcal{A}(e_1) = \emptyset$. The question is to which value $f(\perp_{\mathcal{A}(e_2)})$ evaluates. If it evaluates to null, then we do not have any problem, since outerjoins append nulls. If it does not, the value for a will differ in $e_1 \Join_{q_{12}} \chi_{a_2:f_2}(e_2)$. We thus say that an expression or function f *rejects null values* on a set of attributes A if $f(A) = \text{null}$. With conditions attached, the equivalences read as follows:

$$\begin{array}{ll}
\chi_{a_2:f_2}(e_1 \Join_{q_{12}} e_2) & \equiv e_1 \Join_{q_{12}} \chi_{a_2:f_2}(e_2) \\
& \text{if } f_2 \text{ rejects null values on } \mathcal{A}(e_2), \\
\chi_{a_1:f_1}(e_1 \Join_{q_{12}} e_2) & \equiv \chi_{a_1:f_1}(e_1) \Join_{q_{12}} e_2 \\
& \text{if } f_1 \text{ rejects null values on } \mathcal{A}(e_1), \\
\chi_{a_2:f_2}(e_1 \Join_{q_{12}} e_2) & \equiv e_1 \Join_{q_{12}} \chi_{a_2:f_2}(e_2) \\
& \text{if } f_2 \text{ rejects null values on } \mathcal{A}(e_2).
\end{array}$$

The reorderability properties of the grouping operator and its special case, the nest operator, are of some concern because they are not linear. However, reordering is not hopeless. Let us consider the semijoin first. Let p be a selection predicate and G be a set of grouping attributes. If for some expression e the condition $\mathcal{F}(p) \cap \mathcal{A}(e) \subseteq G$ holds, we know that we can exchange the order of grouping and selection: $\Gamma_{G;F}(\sigma_p(e)) \equiv \sigma_p(\Gamma_{G;F}(e))$. From the definition of the semijoin and the above equivalence with $p = (\sigma_q(e_2) \neq \emptyset)$ it follows that

$$\begin{aligned}
\Gamma_{G;F}(e_1 \Join_q e_2) & \equiv \Gamma_{G;F}(\sigma_{\sigma_q(e_2) \neq \emptyset}(e_1)) \\
& \equiv \sigma_{\sigma_q(e_2) \neq \emptyset}(\Gamma_{G;F}(e_1)) \\
& \equiv \Gamma_{G;F}(e_1) \Join_q e_2
\end{aligned} \tag{7.22}$$

	Π^D	Π_A	Π_A^D	σ	χ	Γ	ν	μ	Υ
$\cup$	-	+	-	+	+	-	-	+	+
$\cap$	+	-	-	+	+	-	-	+	+
$\setminus$	+	-	-	+	+	-	-	+	+

Table 7.5: Simultaneous push-down

if $(\mathcal{F}(q) \cap \mathcal{A}(e_1)) \subseteq G$. Analogously, we can derive

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F}(e_1) \bowtie_q e_2 \quad (7.23)$$

if $(\mathcal{F}(q) \cap \mathcal{A}(e_1)) \subseteq G$. Pushing down a grouping into one of the arguments of a join, outerjoin, d-join, or groupjoin is a little more complex. Thus, we devoted a whole section to this problem (see Sec. 7.11).

Since the bag operators $\cup$, $\cap$, $\setminus$ require the same schema for both arguments, it is quite natural to ask for a simultaneous push-down into both arguments. Thus, we are interested in equivalences of the form $f(e_1 \circ e_2) \equiv f(e_1) \circ f(e_2)$ for the bag operators above. Table 7.5 summarizes the valid instances of this equivalence pattern. Again, we must restrict the map operator to its extending form $\chi_{a:e}$.

And again, problems occur for duplicate elimination and grouping as well as its special case nest. Consider duplicate elimination first. Evaluating the expression $\Pi^D(e_1 \cup e_2)$ will never result in any duplicates. $\Pi^D(e_1) \cup \Pi^D(e_2)$, however, might contain duplicates, e.g., if $e_1 = e_2 = \{[a : 1]\}_b$, the result is $\{[a : 1]^2\}_b$. Since this is the only problem, we immediately conclude that

$$\Pi^D(e_1 \cup e_2) \equiv \Pi^D(\Pi^D(e_1) \cup \Pi^D(e_2)), \quad (7.24)$$

$$\Pi_A^D(e_1 \cup e_2) \equiv \Pi_A^D(\Pi_A^D(e_1) \cup \Pi_A^D(e_2)). \quad (7.25)$$

We now turn our attention to the grouping operator. Let e_1 and e_2 be two expressions with $\mathcal{A}(e_1) = \mathcal{A}(e_2)$. Further, let $G \subseteq \mathcal{A}(e_1)$ be a set of grouping attributes and F an aggregation vector. If $(\Pi_G(e_1) \cap \Pi_G(e_2)) = \emptyset$, then

$$\Gamma_{G;F}(e_1 \cup e_2) \equiv \Gamma_{G;F}(e_1) \cup \Gamma_{G;F}(e_2). \quad (7.26)$$

If $(\Pi_G(e_1) \cap \Pi_G(e_2)) \neq \emptyset$, and F is decomposable into F^1 and F^2 , then

$$\Gamma_{G;F}(e_1 \cup e_2) \equiv \Gamma_{G;F^2}(\Gamma_{G;F^1}(e_1) \cup \Gamma_{G;F^1}(e_2)). \quad (7.27)$$

Of course, this equivalence also holds if $(\Pi_G(e_1) \cap \Pi_G(e_2)) = \emptyset$.

The cases of pushing grouping down an intersection or difference are discussed in Sec. 7.11.8.

7.7.3 Binary Operators

Reordering binary operators is the core operation of any plan generator. We have already devoted a whole chapter to the problem of finding the optimal join order (Chap. 3). The search space for join ordering is huge since the join is commutative and associative. Thus, there was no restriction on the valid

join trees besides syntactic constraints resulting from the consumer/producer relationship. In this section, we investigate the commutativity and associativity of our binary operators.

Commutativity is the easiest. It is obvious that $\cup$, $\cap$, $\times$, $\bowtie$, and $\bowtie$ are commutative while the other binary operators are not. Let us denote the fact that a binary operator $\circ$ is commutative by $\text{comm}(\circ)$.

In traditional mathematics, a binary operator $\circ$ is called associative if $(a \circ b) \circ c = a \circ (b \circ c)$. Since we have to reorder many different operators, which possibly contain subscripts, we consider equivalences of the form

$$(e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 \equiv e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3)$$

for not necessarily distinct operators $\circ^a$ and $\circ^b$. The subscripts in this equivalence have the following meaning. For operators not carrying a predicate or other expressions, it is immaterial and can be ignored. If an operator has an expression e as a subscript, then ij (for $1 \leq i, j \leq 3$, $i \neq j$) indicates that $\mathcal{F}(e) \cap e_k = \emptyset$ for $1 \leq k \leq 3$ and $k \notin \{i, j\}$. This ensures that the equivalence is correctly typed on both sides of the equivalence sign. If for two operators $\circ^a$ and $\circ^b$ the above equivalence holds, then we denote this by $\text{assoc}(\circ^a, \circ^b)$. As we will see, assoc is not symmetric. Thus, we have to be very careful about the order of the operators, which is tight to the syntactic pattern of the equivalence above. In order not to make a mistake, one has to remember two things. First, the operators appear in assoc in the same order as on the left-hand side of the equivalence. Second, the equivalence has left associativity on its left-hand side and, consequently, right associativity on its right-hand side.

If both operators are commutative, then the assoc property is symmetric, i.e.,

$$\begin{aligned} \text{assoc}(\circ^a, \circ^b), \text{comm}(\circ^a), \text{comm}(\circ^b) &\succ \text{assoc}(\circ^b, \circ^a), \\ \text{assoc}(\circ^b, \circ^a), \text{comm}(\circ^a), \text{comm}(\circ^b) &\succ \text{assoc}(\circ^a, \circ^b) \end{aligned}$$

as can be seen from

$$\begin{aligned} (e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 &\equiv e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3) && \text{assoc}(\circ^a, \circ^b) \\ &\equiv (e_2 \circ_{23}^b e_3) \circ_{12}^a e_1 && \text{comm}(\circ^a) \\ &\equiv (e_3 \circ_{23}^b e_2) \circ_{12}^a e_1 && \text{comm}(\circ^b) \\ &\equiv e_3 \circ_{23}^b (e_2 \circ_{12}^a e_1) && \text{assoc}(\circ^b, \circ^a) \\ &\equiv (e_2 \circ_{12}^a e_1) \circ_{23}^b e_3 && \text{comm}(\circ^b) \\ &\equiv (e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 && \text{comm}(\circ^a). \end{aligned}$$

Assume we wish to prove associativity for two binary operators $\circ^a$ and $\circ^b$, where $\circ^a$ is strongly right linear and $\circ^b$ is strongly left linear. Further assume that for all elements $t_1 \in e_1$, $t_2 \in e_2$, and $t_3 \in e_3$

$$\{t_1\}_b \circ_{12}^a (\{t_2\}_b \circ_{23}^b \{t_3\}_b) = (\{t_1\}_b \circ_{12}^a \{t_2\}_b) \circ_{23}^b \{t_3\}_b$$

holds, where the subscript ij in $\circ_{ij}$ indicates that any subscript in $\circ_{ij}$ does not access attributes from e_k if $k \neq i$ and $k \neq j$. Then, we can easily prove

	$\cup$	$\cap$	$\setminus$	$\times$	$\bowtie$	$\ltimes$	$\triangleright$	$\blacktriangleright$	$\bowtie$	$\ltimes$	$\triangleright$
$\cup$	+	-	-	-	-	-	-	-	-	-	-
$\cap$	-	+	-	-	-	+	+	-	-	-	-
$\setminus$	-	-	-	-	-	-	-	-	-	-	-
$\times$	-	-	-	+	+	+	+	+	+	-	+
$\bowtie$	-	-	-	+	+	+	+	+	+	-	+
$\ltimes$	-	-	-	-	-	-	-	-	-	-	-
$\triangleright$	-	-	-	-	-	-	-	-	-	-	-
$\blacktriangleright$	-	-	-	+	+	+	+	+	+	-	+
$\bowtie$	-	-	-	-	-	-	-	-	o	-	-
$\ltimes$	-	-	-	-	-	-	-	-	o	o	-
$\triangleright$	-	-	-	-	-	-	-	-	-	-	-

Table 7.6: The assoc-property for binary operators

associativity by induction on the number of elements in the bag e_2 . If e_2 is empty, then $(e_1 \circ_{12}^a e_2) \circ_{23}^b e_3$ and $e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3)$ are also empty. For singleton bags, we apply the prerequisite above, and for $e_2 = e'_2 \cup e''_2$,

$$\begin{aligned}
(e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 &\equiv (e_1 \circ_{12}^a (e'_2 \cup e''_2)) \circ_{23}^b e_3 \\
&\equiv ((e_1 \circ_{12}^a e'_2) \cup (e_1 \circ_{12}^a e''_2)) \circ_{23}^b e_3 \\
&\equiv ((e_1 \circ_{12}^a e'_2) \circ_{23}^b e_3) \cup ((e_1 \circ_{12}^a e''_2) \circ_{23}^b e_3) \\
&\equiv^{I.H.} (e_1 \circ_{12}^a (e'_2 \circ_{23}^b e_3)) \cup (e_1 \circ_{12}^a (e''_2 \circ_{23}^b e_3)) \\
&\equiv (e_1 \circ_{12}^a ((e'_2 \circ_{23}^b e_3) \cup (e''_2 \circ_{23}^b e_3))) \\
&\equiv e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3)
\end{aligned}$$

provides the induction step.

Table 7.6 summarizes the associativities that hold. Be careful to determine $\text{assoc}(\circ^a, \circ^b)$ from this table by looking up the row with $\circ^a$ and the column with $\circ^b$. Almost all '+' entries' proofs benefit from the strong linearity of both operators. Some of the exceptions benefit from the fact that semi- and antijoin can be expressed as selections and we already know how to push down/pull up selections. The final set of exceptions deals with outerjoins. Here, we also find most of the asymmetries. Since the reader might not be familiar with the d-join, reordering the d-join is discussed in Sec. 7.9. Since reordering outerjoins is complicated, the discussion is deferred to Sec. 7.10.

Now, imagine the operators $\circ^a$ and $\circ^b$ access other attributes than in the associativity pattern above. For example, let $\circ^b$ possibly access e_1 and e_3 but not e_2 . Then, the associativity pattern becomes

$$(e_1 \circ_{12}^a e_2) \circ_{13}^b e_3 \equiv e_1 \circ_{12}^a (e_2 \circ_{13}^b e_3).$$

Obviously, the right-hand side is ill-typed. However, we could rewrite the pattern to

$$(e_1 \circ_{12}^a e_2) \circ_{13}^b e_3 \equiv (e_1 \circ_{13}^b e_3) \circ_{12}^a e_2$$

because then both sides are well-typed. Let us call instances of this pattern *left asscom property* and denote by $\text{l-asscom}(\circ^a, \circ^b)$ the fact that the accord-

ing equivalence holds. Analogously, we can define a *right asscom property* (r-asscom):

$$e_1 \circ_{13}^a (e_2 \circ_{23}^b e_3) \equiv e_2 \circ_{23}^b (e_1 \circ_{13}^a e_3).$$

First note that l-asscom and r-asscom are symmetric properties, i.e.,

$$\begin{aligned} \text{l-asscom}(\circ^a, \circ^b) &\prec\succ \text{l-asscom}(\circ^b, \circ^a), \\ \text{r-asscom}(\circ^a, \circ^b) &\prec\succ \text{r-asscom}(\circ^b, \circ^a). \end{aligned}$$

Then, the calculation

$$\begin{aligned} (e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 &\equiv (e_2 \circ_{12}^a e_1) \circ_{23}^b e_3 && \text{if comm}(\circ_{12}^a) \\ &\equiv (e_2 \circ_{23}^b e_3) \circ_{12}^a e_1 && \text{if l-asscom}(\circ_{12}^a, \circ_{23}^b) \\ &\equiv e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3) && \text{if comm}(\circ_{12}^a) \\ &\equiv (e_1 \circ_{12}^a e_2) \circ_{23}^b e_3 && \text{if assoc}(\circ_{12}^a, \circ_{23}^b) \end{aligned}$$

implies that

$$\begin{aligned} \text{comm}(\circ_{12}^a), \text{assoc}(\circ_{12}^a, \circ_{23}^b) &\succ \text{l-asscom}(\circ_{12}^a, \circ_{23}^b), \\ \text{comm}(\circ_{12}^a), \text{l-asscom}(\circ_{12}^a, \circ_{23}^b) &\succ \text{assoc}(\circ_{12}^a, \circ_{23}^b). \end{aligned}$$

Thus, the l-asscom property is implied by associativity and commutativity, which explains its name. We leave it to the reader to show that

$$\begin{aligned} \text{comm}(\circ_{23}^b), \text{assoc}(\circ_{12}^a, \circ_{23}^b) &\succ \text{r-asscom}(\circ_{12}^a, \circ_{23}^b), \\ \text{comm}(\circ_{23}^b), \text{r-asscom}(\circ_{12}^a, \circ_{23}^b) &\succ \text{assoc}(\circ_{12}^a, \circ_{23}^b). \end{aligned}$$

The important question is whether there are instances of l/r-asscom which do not follow from the commutativity and associativity properties. The answer is yes, as the following investigation shows. Assume $e \circ^a e'$ can be expressed as a selection $\sigma_{p(\circ^a, e')}(e)$, then

$$\begin{aligned} (e_1 \circ_{12}^a e_2) \circ_{13}^b e_3 &\equiv \sigma_{p(\circ_{12}^a, e_2)}(e_1) \circ_{13}^b e_3 && \text{assumption} \\ &\equiv \sigma_{p(\circ_{12}^a, e_2)}(e_1 \circ_{13}^b e_3) && \text{l-pushable}(\sigma, \circ_{13}^b) \\ &\equiv (e_1 \circ_{13}^b e_3) \circ_{12}^a e_2 && \text{assumption} \end{aligned}$$

Thus,

$$\text{isLikeSelection}(\circ^a, \text{l-pushable}(\circ^a, \circ^b)) \succ \text{l-asscom}(\circ^a, \circ^b).$$

We leave the symmetric case for r-asscom to the reader.

Another important exception is l-asscom($\bowtie$, $\bowtie$), which follows from the fact that both operators are strongly left linear. Assume that

$$(\{t_1\}_b \circ_{12}^a \{t_2\}_b) \circ_{13}^b \{t_3\}_b \equiv (\{t_1\}_b \circ_{13}^b \{t_3\}_b) \circ_{12}^a \{t_2\}_b$$

for all t_i and that $\circ^a$ and $\circ^b$ are strongly left linear. Then l-asscom($\circ^a, \circ^b$) holds. The proof is by induction on the number of elements contained in e_1 . First observe that if e_1 is empty, then $(e_1 \circ_{12}^a e_2) \circ_{13}^b e_3$ and $(e_1 \circ_{13}^b e_3) \circ_{12}^a e_2$ are

	$\cup$	$\cap$	$\setminus$	$\times$	$\bowtie$	$\ltimes$	$\rhd$	$\blacktriangleright$	$\bowtie$	$\bowtie$	$\bowtie'$
$\cup$	+/+	-/-	-/-	-/-	-/-	-/-	-/-	-/-	-/-	-/-	-/-
$\cap$	-/-	+/+	-/-	-/-	-/-	+/-	+/-	-/-	-/-	-/-	-/-
$\setminus$	-/-	-/-	-/-	-/-	-/-	+/-	+/-	-/-	-/-	-/-	-/-
$\times$	-/-	-/-	-/-	+/+	+/+	+/-	+/-	+/+	+/-	-/-	+/-
$\bowtie$	-/-	-/-	-/-	+/+	+/+	+/-	+/-	+/+	+/-	-/-	+/-
$\ltimes$	-/-	+/-	+/-	+/-	+/-	+/-	+/-	+/-	+/-	-/-	+/-
$\rhd$	-/-	+/-	+/-	+/-	+/-	+/-	+/-	+/-	+/-	-/-	+/-
$\blacktriangleright$	-/-	-/-	-/-	+/+	+/+	+/-	+/-	+/+	+/-	-/-	+/-
$\bowtie$	-/-	-/-	-/-	+/-	+/-	+/-	+/-	+/-	+/-	o/-	+/-
$\bowtie$	-/-	-/-	-/-	-/-	-/-	-/-	-/-	-/-	o/-	o/o	-/-
$\bowtie'$	-/-	-/-	-/-	+/-	+/-	+/-	+/-	+/-	+/-	-/-	+/-

Table 7.7: The l/r-asscom property for binary operators

	$\cup$	$\cap$	$\setminus$	$\times$	$\bowtie$	$\ltimes$	$\rhd$	$\blacktriangleright$	$\bowtie$	$\bowtie$	$\bowtie'$
$\cup$	-/-	-/-	-/-	+/+	+/+	-/+	-/+	+/+	-/+	-/-	-/+
$\cap$	+/+	+/+	-/+	+/+	+/+	-/+	-/+	+/+	-/+	-/-	-/+
$\setminus$	-/-	-/-	-/-	+/+	+/+	-/+	-/+	+/+	-/+	-/-	-/+

Table 7.8: The l/r-dist property for binary operators

also empty. Let e'_1 and e''_1 be two bags such that $e_1 = e'_1 \cup e''_1$. The induction step looks like this:

$$\begin{aligned}
(e_1 \circ_{12}^a e_2) \circ_{13}^b e_3 &\equiv ((e'_1 \cup e''_1) \circ_{12}^a e_2) \circ_{13}^b e_3 \\
&\equiv ((e'_1 \circ_{12}^a e_2) \cup (e''_1 \circ_{12}^a e_2)) \circ_{13}^b e_3 \\
&\equiv ((e'_1 \circ_{12}^a e_2) \circ_{13}^b e_3) \cup ((e''_1 \circ_{12}^a e_2) \circ_{13}^b e_3) \\
&\equiv^{I.H.} ((e'_1 \circ_{13}^b e_3) \circ_{12}^a e_2) \cup ((e''_1 \circ_{13}^b e_3) \circ_{12}^a e_2) \\
&\equiv ((e'_1 \circ_{13}^b e_3) \cup (e''_1 \circ_{13}^b e_3)) \circ_{12}^a e_2 \\
&\equiv ((e'_1 \cup e''_1) \circ_{13}^b e_3) \circ_{12}^a e_2 \\
&\equiv (e_1 \circ_{13}^b e_3) \circ_{12}^a e_2.
\end{aligned}$$

Table 7.7 summarizes the l/r-asscom properties for all pairs of operators. Most of the entries follow from the abovementioned. Some equivalences for the d-join and the groupjoin, especially in conjunction with outerjoins, need dedicated proofs. This is a good sign, since, thanks to l/r-asscom, reorderings become possible which were not possible with commutativity and associativity alone.

Distributivity laws play a minor role in query compilers, but are very useful to prove equivalences. We consider right and left distributivity (l/r-dist):

$$\begin{aligned}
e_1 \circ^b (e_2 \circ^a e_3) &\equiv (e_1 \circ^b e_2) \circ^a (e_1 \circ^b e_3) \text{ l-dist,} \\
(e_1 \circ^a e_2) \circ^b e_3 &\equiv (e_1 \circ^b e_3) \circ^a (e_2 \circ^b e_3) \text{ r-dist.}
\end{aligned}$$

With these definitions, it is easy to show that

$$\begin{aligned} \text{comm}(\circ^b), \text{r-dist}(\circ^a, \circ^b) &\succ \text{l-dist}(\circ^a, \circ^b), \\ \text{comm}(\circ^b), \text{l-dist}(\circ^a, \circ^b) &\succ \text{r-dist}(\circ^a, \circ^b). \end{aligned}$$

Table 7.8 summarizes the distributivity laws for $\circ^a \in \{\cup, \cap, \setminus\}$.

7.8 Predicate Detachment and Attachment

In some cases, an operator with a conjunctive selection predicate allows to move a part of it to a newly introduced selection operator:

$$e_1 \circ_{p \wedge p'} e_2 \equiv \sigma_p(e_1 \circ_{p'} e_2) \quad (7.28)$$

$$e_1 \circ_{p_1 \wedge p} e_2 \equiv \sigma_{p_1}(e_1) \circ_p e_2 \quad (7.29)$$

$$e_1 \circ_{p_2 \wedge p} e_2 \equiv e_1 \circ_p \sigma_{p_2}(e_2) \quad (7.30)$$

We call these properties up-/left-/right-*detachable*. The syntactic constraints are $\mathcal{F}(p), \mathcal{F}(p_1), \mathcal{F}(p_2) \subseteq \mathcal{A}(e_1) \cup \mathcal{A}(e_2)$, $\mathcal{F}(p) \subseteq \mathcal{A}(e_1 \circ_{p'} e_2)$, $\mathcal{F}(p_1) \subseteq \mathcal{A}(e_1)$, and $\mathcal{F}(p_2) \subseteq \mathcal{A}(e_2)$. We call this process *predicate detachment*. *Predicate attachment* denotes the opposite rewrite. Detachability should not be confused with the left- and right- pushdown of selection predicates as in

$$\sigma_{p_1}(e_1 \circ e_2) \equiv \sigma_{p_1}(e_1) \circ e_2 \quad (7.31)$$

$$\sigma_{p_2}(e_1 \circ e_2) \equiv e_1 \circ \sigma_{p_2}(e_2) \quad (7.32)$$

where $\mathcal{F}(p_i) \cap \mathcal{A}(e_{3-i}) = \emptyset$.

Let q, q_i be join predicates and p and p_i be selection predicates. Further, we require that $\mathcal{F}(p_i) \cap \mathcal{A}(e_{3-i}) = \emptyset$ for $i = 1, 2$. Then, the following equivalences hold

$$\sigma_{q \wedge p}(e) \equiv \sigma_q(\sigma_p(e)), \quad (7.33)$$

$$e_1 \bowtie_{q \wedge p_1} e_2 \equiv \sigma_{p_1}(e_1) \bowtie_q e_2, \quad (7.34)$$

$$e_1 \bowtie_{q \wedge p_2} e_2 \equiv e_1 \bowtie_q \sigma_{p_2}(e_2), \quad (7.35)$$

$$e_1 \ltimes_{q \wedge p_1} e_2 \equiv \sigma_{p_1}(e_1) \ltimes_q e_2, \quad (7.36)$$

$$e_1 \ltimes_{q \wedge p_2} e_2 \equiv e_1 \ltimes_q \sigma_{p_2}(e_2), \quad (7.37)$$

$$e_1 \triangleright_{q \wedge p_2} e_2 \equiv e_1 \triangleright_q \sigma_{p_2}(e_2), \quad (7.38)$$

$$e_1 \bowtie_{q \wedge p_2} e_2 \equiv e_1 \bowtie_q \sigma_{p_2}(e_2), \quad (7.39)$$

$$e_1 \bowtie_{q \wedge p_2; g; f} e_2 \equiv e_1 \bowtie_{q; g; f} \sigma_{p_2}(e_2). \quad (7.40)$$

There is no possibility to move a part of a conjunctive predicate, which only accesses attributes from one side, into or out of a full outerjoin.

In case of a disjunction, we have a nice equivalence for the antijoin.

$$e_1 \triangleright_{q_1 \vee q_2} e_2 \equiv (e_1 \triangleright_{q_1} e_2) \triangleright_{q_2} e_2 \quad (7.41)$$

Further, the equivalence

$$e_1 \triangleright_{q \vee p_1} e_2 \equiv \sigma_{\neg p_1}(e_1) \triangleright_q e_2 \quad (7.42)$$

holds if $e_2 \neq \emptyset$.

Assume that the whole predicate of a binary operator references only attributes from its left or its right argument. Then, some simplifications/rewrites are possible:

$$e_1 \bowtie_{p_1} e_2 \equiv \sigma_{p_1}(e_1) \times e_2, \quad (7.43)$$

$$e_1 \bowtie_{p_1} e_2 \equiv \sigma_{p_1}(e_1) \bowtie_{true} e_2 \equiv \sigma_{e_2 \neq \emptyset}(\sigma_{p_1}(e_1)), \quad (7.44)$$

$$e_1 \bowtie_{p_2} e_2 \equiv e_1 \bowtie_{true} \sigma_{p_2}(e_2) \equiv \sigma_{\sigma_{p_2}(e_2) \neq \emptyset}(e_1), \quad (7.45)$$

$$e_1 \bowtie_{p_1} e_2 \equiv \sigma_{(e_2 = \emptyset) \vee (\neg p_1)}(e_1), \quad (7.46)$$

$$e_1 \bowtie_{p_2} e_2 \equiv e_1 \bowtie_{true} \sigma_{p_2}(e_2) \equiv \sigma_{\sigma_{p_2}(e_2) = \emptyset}(e_1), \quad (7.47)$$

$$e_1 \bowtie_{p_1} e_2 \equiv (\sigma_{p_1}(e_1) \times e_2) \cup ((\sigma_{(e_2 = \emptyset) \vee (\neg p_1)}(e_1)) \times \{\perp_{\mathcal{A}(e_2)}\}), \quad (7.48)$$

$$e_1 \bowtie_{p_2} e_2 \equiv e_1 \bowtie_{true} \sigma_{p_2}(e_2) \quad (7.49)$$

$$\equiv (e_1 \times \sigma_{p_2}(e_2)) \cup ((\sigma_{\sigma_{p_2}(e_2) = \emptyset}(e_1)) \times \{\perp_{\mathcal{A}(e_2)}\}) \quad (7.50)$$

$$e_1 \bowtie_{p_1} e_2 \equiv (\sigma_{p_1}(e_1) \times e_2) \cup (\sigma_{e_2 = \emptyset \vee \neg p_1}(e_1) \times \{\perp_{\mathcal{A}(e_2)}\}) \quad (7.51)$$

$$\cup (\sigma_{\sigma_{p_1}(e_1) = \emptyset}(e_1) \times \{\perp_{\mathcal{A}(e_1)}\}) \quad (7.52)$$

$$e_1 \bowtie_{p_1;g:f} e_2 \equiv e_1 \bowtie_{true;g:f} \sigma_{p_1}(e_2) \quad (7.53)$$

$$\begin{aligned} e_1 \bowtie_{p_2;g:f} e_2 &\equiv e_1 \bowtie_{true;g:f \circ \sigma_{p_2}} e_2 \\ &\equiv e_1 \bowtie_{true;g:f} \sigma_{p_2} e_2 \\ &\equiv \chi_{g:f(\sigma_{p_2}(e_2))}(e_1) \\ &\equiv e_1 \times \{[g : f(\sigma_{p_2}(e_2))]\}, \end{aligned} \quad (7.54)$$

where we left out symmetric cases, which are possible due to commutativity.

Let us consider the semijoin. If $\mathcal{F}(p_1) \cap \mathcal{A}(e_2) = \emptyset$, then

$$e_1 \bowtie_{p_1} e_2 = \begin{cases} \sigma_{p_1}(e_1) & \text{if } e_2 \neq \emptyset, \\ \emptyset & \text{if } e_2 = \emptyset, \end{cases}$$

which can be summarized to $\sigma_{e_2 \neq \emptyset}(\sigma_{p_1}(e_1))$. If $\mathcal{F}(p_2) \cap \mathcal{A}(e_1) = \emptyset$, then

$$e_1 \bowtie_{p_2} e_2 = \begin{cases} e_1 & \text{if } \sigma_{p_2}(e_2) \neq \emptyset, \\ \emptyset & \text{if } \sigma_{p_2}(e_2) = \emptyset, \end{cases}$$

which can be summarized to $\sigma_{\sigma_{p_2}(e_2) \neq \emptyset}(e_1)$.

Let us consider the antijoin. If $\mathcal{F}(p_1) \cap \mathcal{A}(e_2) = \emptyset$, then

$$e_1 \bowtie_{p_1} e_2 = \begin{cases} e_1 & \text{if } e_2 = \emptyset, \\ \sigma_{\neg p_1}(e_1) & \text{if } e_2 \neq \emptyset, \end{cases}$$

which can be summarized to $\sigma_{e_2 = \emptyset \vee \neg p_1}(e_1)$. If $\mathcal{F}(p_2) \cap \mathcal{A}(e_1) = \emptyset$, then

$$e_1 \bowtie_{p_2} e_2 = \begin{cases} e_1 & \text{if } \sigma_{p_2}(e_2) = \emptyset \\ \emptyset & \text{if } \sigma_{p_2}(e_2) \neq \emptyset, \end{cases}$$

which can be summarized to $\sigma_{\sigma_{p_2}(e_2) = \emptyset}(e_1)$.

For the semi- and the antijoin, we have the expression e_2 in the subscript of some operator on the left-hand side of the equivalences. This could mean

nested-loop evaluation. However, since the evaluation of e_2 is independent of e_1 , we can easily apply unnesting techniques, and evaluate e_2 only once and only as far as necessary to evaluate the expressions for the semi- and antijoin.

To consider the different cases for the left and full outerjoin, it is convenient to define $E_{\perp i} = \{\perp_{\mathcal{A}(e_i)}\}$ for $i = 1, 2$ and given expressions e_i . If $\mathcal{F}(p_1) \cap \mathcal{A}(e_2) = \emptyset$, we can reason as follows for the left outerjoin:

$$\begin{aligned} e_1 \bowtie_{p_1} e_2 &\equiv (e_1 \bowtie_{p_1} e_2) \cup ((e_1 \triangleright_{p_1} e_2) \times E_{\perp 2}) \\ &\equiv (\sigma_{p_1}(e_1) \times e_2) \cup ((\sigma_{(e_2=\emptyset) \vee (\neg p_1)}(e_1)) \times E_{\perp 2}). \end{aligned}$$

If $\mathcal{F}(p_2) \cap \mathcal{A}(e_1) = \emptyset$, we have

$$\begin{aligned} e_1 \bowtie_{p_2} e_2 &\equiv (e_1 \bowtie_{p_2} e_2) \cup ((e_1 \triangleright_{p_2} e_2) \times E_{\perp 2}) \\ &\equiv (e_1 \times \sigma_{p_2}(e_2)) \cup ((e_1 \triangleright_{true} \sigma_{p_2}(e_2)) \times E_{\perp 2}) \\ &\equiv (e_1 \bowtie_{true} \sigma_{p_2}(e_2)) \end{aligned}$$

or, alternatively,

$$\begin{aligned} e_1 \bowtie_{p_2} e_2 &\equiv (e_1 \bowtie_{p_2} e_2) \cup ((e_1 \triangleright_{p_2} e_2) \times E_{\perp 2}) \\ &\equiv (e_1 \times \sigma_{p_2}(e_2)) \cup ((\sigma_{\sigma_{p_2}(e_2)=\emptyset}(e_1)) \times E_{\perp 2}). \end{aligned}$$

Next, we consider the full outerjoin. Assume $\mathcal{F}(p_1) \cap \mathcal{A}(e_2) = \emptyset$, then

$$\begin{aligned} e_1 \bowtie_{p_1} e_2 &\equiv (e_1 \bowtie_{p_1} e_2) \cup ((e_1 \triangleright_{p_1} e_2) \times E_{\perp 2}) \cup ((e_2 \triangleright_{p_1} e_1) \times E_{\perp 1}) \\ &\equiv (\sigma_{p_1}(e_1) \times e_2) \cup (\sigma_{(e_2=\emptyset) \vee (\neg p_1)}(e_1) \times E_{\perp 2}) \cup ((\sigma_{\sigma_{p_1}(e_1)=\emptyset}(e_2)) \times E_{\perp 1}). \end{aligned}$$

Finally, let us consider groupjoin. If $\mathcal{F}(q) \cap \mathcal{A}(e_2) = \emptyset$, there is not much we can do. If $\mathcal{F}(q) \cap \mathcal{A}(e_1) = \emptyset$, the expression can be considerably simplified. This is due to the fact that now every item in e_1 has the same members of e_1 in its group. In other words, in the result of $e_1 \bowtie_{true;g:id} e_2$ all tuples have the same value for g .

For convenience, we summarize whether predicates can be pushed down or are detachable for some binary operators in Table 7.9.

7.9 Basic Equivalences for D-Join

From the definition of the d-join follows that

$$e_1 \bowtie e_2 \equiv \cup_{t_1 \in e_1} (\{t_1\}_b \times e_2(t_1)). \quad (7.55)$$

Assume e_2 depends on some attributes provided by a tuple t_1 . Then

$$e_2(t_1) \bowtie e_3 \equiv (e_2 \bowtie e_3)(t_1). \quad (7.56)$$

op	pushable		detachable		
	l-push	r-push	up-det	l-det	r-det
$\times$	+	+	-	-	-
$\bowtie$	+	+	+	+	+
$\ltimes$	+	-	+	+	+
$\triangleright$	+	-	-	-	+
$\Join$	+	-	-	-	+
$\Join$	-	-	-	-	-
$\Join^\top$	+	-	-	-	+

Table 7.9: Predicate push/pull and detachability

That is, it is immaterial where we feed in the bindings contained in t_1 . Of course, this only holds as long as $\mathcal{F}(e_3) \cap \mathcal{A}(t_1) = \emptyset$.

Let us prove that $\Join$ is associative:

$$\begin{aligned}
(e_1 \Join e_2) \Join e_3 &\equiv (\cup_{t_1 \in e_1} (\{t_1\}_b \times e_2(t_1))) \Join e_3 \\
&\equiv \cup_{t_1 \in e_1} ((\{t_1\}_b \times e_2(t_1)) \Join e_3) \\
&\equiv \cup_{t_1 \in e_1} (\{t_1\}_b \times (e_2(t_1) \Join e_3)) \\
&\equiv \cup_{t_1 \in e_1} (\{t_1\}_b \times (e_2 \Join e_3)(t_1)) \\
&\equiv e_1 \Join (e_2 \Join e_3)
\end{aligned}$$

In a sense, the d-join is strongly right linear. If for all $t_1 \in e_1$ $e_2(t_1) = \emptyset$, then $e_1 \Join e_2 = \emptyset$. Assume e_2 has the form $e'_2 \cup e''_2$. Then

$$\begin{aligned}
e_1 \Join e_2 &= e_1 \Join (e'_2 \cup e''_2) \\
&= e_1 \Join_{J=J'} (\widehat{e'_2} \cup \widehat{e''_2}) \\
&= (e_1 \Join_{J=J'} \widehat{e'_2}) \cup (e_1 \Join_{J=J} \widehat{e''_2}) \\
&= (e_1 \Join e'_2) \cup (e_1 \Join e''_2)
\end{aligned}$$

for $J = \mathcal{A}(e_1) \cap \mathcal{F}(e_2)$.

The d-join and the unnest operators are closely related:

$$e_1 \Join e_2 \equiv \mu_g(\chi_{g:e_2}(e_1)). \quad (7.57)$$

Between *flatten* and the d-join, there also exists a correspondence:

$$\text{flatten}(\chi_{e_2}(e_1)) \equiv \Pi_{\mathcal{A}(e_2)}(e_1 \Join e_2). \quad (7.58)$$

Sometimes a d-join can be expressed as a cross product or a join:

$$e_1 \Join e_2 \equiv e_1 \times e_2 \quad (7.59)$$

$$\text{if } \mathcal{F}(e_2) \cap \mathcal{A}(e_1) = \emptyset,$$

$$e_1 \Join \sigma_q(e_2) \equiv e_1 \Join_q e_2 \quad (7.60)$$

$$\text{if } \mathcal{F}(e_2) \cap \mathcal{A}(e_1) = \emptyset.$$

Denote by $e \downarrow$ the fact that some expression e is defined, i.e., returns a valid result. Then, we call a function f *extending*, if and only if

$$\forall x, y : (f(x) \circ y) \downarrow \implies f(x \circ y) = f(x) \circ y$$

and we call it *restricting*, if and only if

$$\forall x, y : f(x) \downarrow, (x \circ y) \downarrow \implies f(x \circ y) = f(x)$$

Let us give an example of a function that is neither extending nor restricting. It returns a tuple with a single attribute c , whose value is bound to the number of attributes of its input tuple.

Unnesting of operations buried in the d-join can be performed by applying the following equivalence:

$$e \bowtie f(\sigma_{A=A'}(\rho_{A \leftarrow A'}(e))) \equiv \mu_g(\Gamma_{A;g:f}(e)) \quad (7.61)$$

if $A \subseteq \mathcal{A}(e)$,

$$e_1 \bowtie (e_2 \bowtie e_3) \equiv (e_1 \bowtie e_2) \bowtie e_3 \quad (7.62)$$

if $\mathcal{F}(e_3) \cap \mathcal{A}(e_1) = \emptyset$,

$$e_1 \bowtie \chi_f(e_2) \equiv \chi_f(e_1 \bowtie e_2) \quad (7.63)$$

if f extending,

$$\Pi_A(e_1 \bowtie \chi_f(e_2)) \equiv \Pi_A(\chi_f(e_1 \bowtie e_2)) \quad (7.64)$$

if $A \subseteq \mathcal{A}(\chi_f(e_2))$, and f restricting.

We have to be careful if we exchange a d-join with a join. The dependency can move. In the following equivalences we provide the dependencies explicitly in parenthesis.

$$e_1 \bowtie_{p_{12}} (e_2(e_1) \bowtie_{p_{23}} e_3(e_1)) \equiv (e_1 \bowtie_{p_{12}} e_2(e_1)) \bowtie_{p_{23}} e_3(e_1) \quad (7.65)$$

$$e_1 \bowtie_{p_{12}} (e_2 \bowtie_{p_{23}} e_3(e_1)) \equiv (e_1 \bowtie_{p_{12}} e_2) \bowtie_{p_{23}} e_3(e_1) \quad (7.66)$$

In the first equivalence, the join between e_2 and e_3 on the left-hand side must be turned into a dependent join on the right-hand side. In the second equivalence, the first dependent join between e_1 and e_2 becomes a regular join between e_1 and e_2 on the right-hand side and the regular join between e_2 and e_3 on the left-hand side becomes a dependent join on the right-hand side.

7.10 Equivalences for Outerjoins

Outerjoins are a little brittle. Long papers have already been written on this subject. In this section, we summarize the most important findings, which are useful in the context of query optimization. For a full account on outerjoins the reader is referred to the literature [736, 298, 308]. The occurrence of an outerjoin can have several reasons. First, outerjoins are part of the SQL 2 specification. Second, outerjoins can be introduced during query rewrite. For example, unnesting nested queries or hierarchical views may result in outerjoins.

$e_1 := R_1$	$e_2 := R_2$	$e_3 := R_3$
$\begin{array}{cc} a_1 & b_1 \\ 1 & 1 \\ 2 & 4 \\ 3 & 5 \\ 4 & - \end{array}$	$\begin{array}{cc} a_2 & b_2 \\ 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & - \end{array}$	$\begin{array}{cc} a_3 & b_3 \\ 1 & 3 \\ 2 & 5 \\ 3 & 6 \\ 4 & - \end{array}$
$e_{12}^j := e_1 \bowtie_{q_{12}} e_2$	$e_{13}^j := e_1 \bowtie_{q_{13}} e_3$	$e_{23}^j := e_2 \bowtie_{q_{23}} e_3$
$\begin{array}{cccc} a_1 & b_1 & a_2 & b_2 \\ 2 & 4 & 2 & 4 \end{array}$	$\begin{array}{cccc} a_1 & b_1 & a_3 & b_3 \\ 3 & 5 & 2 & 5 \end{array}$	$\begin{array}{cccc} a_2 & b_2 & a_3 & b_3 \\ 3 & 6 & 3 & 6 \end{array}$
$e_{12}^{lo} := e_1 \bowtie_{q_{12}} e_2$	$e_{13}^{lo} := e_1 \bowtie_{q_{13}} e_3$	$e_{23}^{lo} := e_2 \bowtie_{q_{23}} e_3$
$\begin{array}{cccc} a_1 & b_1 & a_2 & b_2 \\ 1 & 1 & - & - \\ 2 & 4 & 2 & 4 \\ 3 & 5 & - & - \\ 4 & - & - & - \end{array}$	$\begin{array}{cccc} a_1 & b_1 & a_3 & b_3 \\ 1 & 1 & - & - \\ 2 & 4 & - & - \\ 3 & 5 & 2 & 5 \\ 4 & - & - & - \end{array}$	$\begin{array}{cccc} a_2 & b_2 & a_3 & b_3 \\ 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & - & - \end{array}$
$e_{12}^{fo} := e_1 \bowtie_{q_{12}} e_2$	$e_{13}^{fo} := e_1 \bowtie_{q_{13}} e_3$	$e_{23}^{fo} := e_2 \bowtie_{q_{23}} e_3$
$\begin{array}{cccc} a_1 & b_1 & a_2 & b_2 \\ 1 & 1 & - & - \\ 2 & 4 & 2 & 4 \\ 3 & 5 & - & - \\ 4 & - & - & - \\ - & - & 1 & 2 \\ - & - & 3 & 6 \\ - & - & 4 & - \end{array}$	$\begin{array}{cccc} a_1 & b_1 & a_3 & b_3 \\ 1 & 1 & - & - \\ 2 & 4 & - & - \\ 3 & 5 & 2 & 5 \\ 4 & - & - & - \\ - & - & 1 & 3 \\ - & - & 3 & 6 \\ - & - & 4 & - \end{array}$	$\begin{array}{cccc} a_2 & b_2 & a_3 & b_3 \\ 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & - & - \\ - & - & 1 & 3 \\ - & - & 2 & 5 \\ - & - & 4 & - \end{array}$

Figure 7.6: Example for outerjoin reorderability (for strict q)

Sometimes, it is also possible to rewrite universal quantifiers to outerjoins [886, 219].

Before reading any further, the reader should get acquainted to outerjoins by checking whether there is a mistake in Figs. 7.6, 7.7, or 7.8. There, we calculated for three relations R_i their joins, left outerjoins, and full outerjoins for three different sets of predicates. The first set of predicates does not apply any special comparisons with respect to null values. All predicates in this set are denoted by q_{ij} ($1 \leq i, j \leq 3$) and defined as $q_{ij} := (b_i = b_j)$. The second set of predicates uses the special comparison ‘ $\doteq$ ’. Remember that this dotted equality returns true in the additional case that both arguments are null. The predicates of the second set are denoted by q'_{ij} and defined as $q'_{ij} := (b_i \doteq b_j)$. The third set of predicates consists of $q_{12'} := b_1 = b_2 \vee b_2 \doteq \text{null}$ and $q_{2'3} := b_2 = b_3 \vee b_2 \doteq \text{null}$. Note that in Fig. 7.8 there is no difference between $e_2 \bowtie_{q_{2',3}} e_3$ and $e_2 \bowtie_{q_{2',3}} e_3$. Why?

The main purpose of this section is to derive equivalences among expressions containing outerjoins. Let us start with the observation that the full outerjoin is commutative, but the left outerjoin is not. Less simple is the next item on our list: associativity. As a simple start, consider

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \not\equiv e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)$$

$e_1 := R_1$	$e_2 := R_2$	$e_3 := R_3$
$\begin{array}{c c} a_1 & b_1 \\ \hline 1 & 1 \\ 2 & 4 \\ 3 & 5 \\ 4 & - \end{array}$	$\begin{array}{c c} a_2 & b_2 \\ \hline 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & - \end{array}$	$\begin{array}{c c} a_3 & b_3 \\ \hline 1 & 3 \\ 2 & 5 \\ 3 & 6 \\ 4 & - \end{array}$
$e_{12}^{j'} := e_1 \bowtie_{q'_{12}} e_2$	$e_{13}^{j'} := e_1 \bowtie_{q'_{13}} e_3$	$e_{23}^{j'} := e_2 \bowtie_{q'_{23}} e_3$
$\begin{array}{c cccc} a_1 & b_1 & a_2 & b_2 \\ \hline 2 & 4 & 2 & 4 \\ 4 & - & 4 & - \end{array}$	$\begin{array}{c cccc} a_1 & b_1 & a_3 & b_3 \\ \hline 3 & 5 & 2 & 5 \\ 4 & - & 4 & - \end{array}$	$\begin{array}{c cccc} a_2 & b_2 & a_3 & b_3 \\ \hline 3 & 6 & 3 & 6 \\ 4 & - & 4 & - \end{array}$
$e_{12}^{lo'} := e_1 \bowtie_{q'_{12}} e_2$	$e_{13}^{lo'} := e_1 \bowtie_{q'_{13}} e_3$	$e_{23}^{lo'} := e_2 \bowtie_{q'_{23}} e_3$
$\begin{array}{c cccc} a_1 & b_1 & a_2 & b_2 \\ \hline 1 & 1 & - & - \\ 2 & 4 & 2 & 4 \\ 3 & 5 & - & - \\ 4 & - & 4 & - \end{array}$	$\begin{array}{c cccc} a_1 & b_1 & a_3 & b_3 \\ \hline 1 & 1 & - & - \\ 2 & 4 & - & - \\ 3 & 5 & 2 & 5 \\ 4 & - & 4 & - \end{array}$	$\begin{array}{c cccc} a_2 & b_2 & a_3 & b_3 \\ \hline 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & 4 & - \end{array}$
$e_{12}^{fo'} := e_1 \bowtie_{q'_{12}} e_2$	$e_{13}^{fo'} := e_1 \bowtie_{q'_{13}} e_3$	$e_{23}^{fo'} := e_2 \bowtie_{q'_{23}} e_3$
$\begin{array}{c cccc} a_1 & b_1 & a_2 & b_2 \\ \hline 1 & 1 & - & - \\ 2 & 4 & 2 & 4 \\ 3 & 5 & - & - \\ 4 & - & 4 & - \\ - & - & 1 & 2 \\ - & - & 3 & 6 \end{array}$	$\begin{array}{c cccc} a_1 & b_1 & a_3 & b_3 \\ \hline 1 & 1 & - & - \\ 2 & 4 & - & - \\ 3 & 5 & 2 & 5 \\ 4 & - & 4 & - \\ - & - & 1 & 3 \\ - & - & 3 & 6 \end{array}$	$\begin{array}{c cccc} a_2 & b_2 & a_3 & b_3 \\ \hline 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & 4 & - \\ - & - & 1 & 3 \\ - & - & 2 & 5 \end{array}$

Figure 7.7: Example for outerjoin reorderability (for non-strict q')

$e_{12}^{fo2'} := e_1 \bowtie_{q'_{12}} e_2$	$e_{23}^{lo2'} := e_2 \bowtie_{q'_{23}} e_3$	$e_{23}^{fo2'} := e_2 \bowtie_{q'_{23}} e_3$
$\begin{array}{c cccc} a_1 & b_1 & a_2 & b_2 \\ \hline 1 & 1 & 4 & - \\ 2 & 4 & 2 & 4 \\ 2 & 4 & 4 & - \\ 3 & 5 & 4 & - \\ 4 & - & 4 & - \\ - & - & 1 & 2 \\ - & - & 3 & 6 \end{array}$	$\begin{array}{c cccc} a_2 & b_2 & a_3 & b_3 \\ \hline 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & 1 & 3 \\ 4 & - & 2 & 5 \\ 4 & - & 3 & 6 \\ 4 & - & 4 & - \end{array}$	$\begin{array}{c cccc} a_2 & b_2 & a_3 & b_3 \\ \hline 1 & 2 & - & - \\ 2 & 4 & - & - \\ 3 & 6 & 3 & 6 \\ 4 & - & 1 & 3 \\ 4 & - & 2 & 5 \\ 4 & - & 3 & 6 \\ 4 & - & 4 & - \end{array}$

Figure 7.8: Example for outerjoin reorderability (for partially non-strict q')

If we let e_2 and e_3 be empty bags, then the right-hand side evaluates to the empty bag but the left-hand side simplifies to $e_1 \times \{\perp_{\mathcal{A}(e_2) \cup \mathcal{A}(e_3)}\}$. Thus, $\neg\text{assoc}(\bowtie, \bowtie)$. By taking a look at

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \neq e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3),$$

with e_2 and e_3 yielding the empty bag, we see that $\neg\text{assoc}(\bowtie, \bowtie)$. Imagine e_1

$e_{12}^{lo} \bowtie_{q_{23}} e_3$							$e_1 \bowtie_{q_{12}} e_{23}^j$						
a_1	b_1	a_2	b_2	a_3	b_3		a_1	b_1	a_2	b_2	a_3	b_3	
							1	1	-	-	-	-	
							2	4	-	-	-	-	
							3	5	-	-	-	-	
							4	-	-	-	-	-	
$e_{12}^{fo} \bowtie_{q_{23}} e_3$							$e_1 \bowtie_{q_{12}} e_{23}^j$						
a_1	b_1	a_2	b_2	a_3	b_3		a_1	b_1	a_2	b_2	a_3	b_3	
							1	1	-	-	-	-	
							2	4	-	-	-	-	
							3	5	-	-	-	-	
							4	-	-	-	-	-	
-	-	3	6	3	6		-	-	3	6	3	6	
$e_{12}^j \bowtie_{q_{23}} e_3$							$e_1 \bowtie_{q_{12}} e_{23}^{fo}$						
a_1	b_1	a_2	b_2	a_3	b_3		a_1	b_1	a_2	b_2	a_3	b_3	
2	4	2	4	-	-		2	4	2	4	-	-	
-	-	-	-	1	3								
-	-	-	-	2	5								
-	-	-	-	3	6								
-	-	-	-	4	-								
$e_{12}^{lo} \bowtie_{q_{23}} e_3$							$e_1 \bowtie_{q_{12}} e_{23}^{fo}$						
a_1	b_1	a_2	b_2	a_3	b_3		a_1	b_1	a_2	b_2	a_3	b_3	
1	1	-	-	-	-		1	1	-	-	-	-	
2	4	2	4	-	-		2	4	2	4	-	-	
3	5	-	-	-	-		3	5	-	-	-	-	
4	-	-	-	-	-		4	-	-	-	-	-	
-	-	-	-	1	3								
-	-	-	-	2	5								
-	-	-	-	3	6								
-	-	-	-	4	-								

Figure 7.9: Example for outerjoin associativity for strict q

and e_2 yield empty bags. The left-hand side of

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \neq e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)$$

then evaluates to $\{\perp_{\mathcal{A}(e_1) \cup \mathcal{A}(e_2)}\} \times e_3$. Since the right-hand side gives the empty bag, we have $\neg \text{assoc}(\bowtie, \bowtie)$. Last in this sequence, we consider

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \neq e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3).$$

Assume again, that e_1 and e_2 evaluate to the empty bag. Then, the right-hand side does the same, whereas the left-hand side results in the familiar $\{\perp_{\mathcal{A}(e_1) \cup \mathcal{A}(e_2)}\} \times e_3$. Consequently, $\neg \text{assoc}(\bowtie, \bowtie)$. Summarizing, we have $\neg \text{assoc}(\bowtie, \bowtie)$, $\neg \text{assoc}(\bowtie, \bowtie)$, $\neg \text{assoc}(\bowtie, \bowtie)$, and $\neg \text{assoc}(\bowtie, \bowtie)$. These negative results are also confirmed by our example (see Fig. 7.9). This leaves us

$e_{100} := e_{12}^{lo} \bowtie_{q'_{23}} e_3$						$e_{100} := e_1 \bowtie_{q_{12}} e_{23}^{lo'}$					
a_1	b_1	a_2	b_2	a_3	b_3	a_1	b_1	a_2	b_2	a_3	b_3
1	1	-	-	4	-	1	1	-	-	-	-
2	4	2	4	-	-	2	4	2	4	-	-
3	5	-	-	4	-	3	5	-	-	-	-
4	-	-	-	4	-	4	-	-	-	-	-

$e_{100} := e_{12}^{fo} \bowtie_{q'_{23}} e_3$						$e_{100} := e_1 \bowtie_{q_{12}} e_{23}^{lo'}$					
a_1	b_1	a_2	b_2	a_3	b_3	a_1	b_1	a_2	b_2	a_3	b_3
1	1	-	-	4	-	1	1	-	-	-	-
2	4	2	4	-	-	2	4	2	4	-	-
3	5	-	-	4	-	3	5	-	-	-	-
4	-	-	-	4	-	4	-	-	-	-	-
-	-	1	2	-	-	-	-	1	2	-	-
-	-	3	6	3	6	-	-	3	6	3	6
-	-	4	-	4	-	-	-	4	-	4	-

$e_{100} := e_{12}^{fo} \bowtie_{q'_{23}} e_3$						$e_{100} := e_1 \bowtie_{q_{12}} e_{23}^{fo'}$					
a_1	b_1	a_2	b_2	a_3	b_3	a_1	b_1	a_2	b_2	a_3	b_3
1	1	-	-	4	-	1	1	-	-	-	-
2	4	2	4	-	-	2	4	2	4	-	-
3	5	-	-	4	-	3	5	-	-	-	-
4	-	-	-	4	-	4	-	-	-	-	-
-	-	1	2	-	-	-	-	1	2	-	-
-	-	3	6	3	6	-	-	3	6	3	6
-	-	4	-	4	-	-	-	4	-	4	-
-	-	-	-	1	3	-	-	-	-	1	3
-	-	-	-	2	5	-	-	-	-	2	5

Figure 7.10: Example for outerjoin associativity for non-strict q'

$e_{100} := e_{12}^{fo} \bowtie_{q_{13}} e_3$						$e_{100} := e_{13}^j \bowtie_{q_{12}} e_2$					
a_1	b_1	a_2	b_2	a_3	b_3	a_1	b_1	a_2	b_2	a_3	b_3
3	5	-	-	2	5	3	5	-	-	2	5
-	-	1	2	-	-	-	-	1	2	-	-
-	-	2	4	-	-	-	-	2	4	-	-
-	-	3	6	-	-	-	-	3	6	-	-
-	-	4	-	-	-	-	-	4	-	-	-

Figure 7.11: Example for outerjoin l-asscom for strict q

to check each of $\text{assoc}(\bowtie, \bowtie)$, $\text{assoc}(\bowtie, \bowtie)$, $\text{assoc}(\bowtie, \bowtie)$, and $\text{assoc}(\bowtie, \bowtie)$, apart from the already known $\text{assoc}(\bowtie, \bowtie)$. Fig. 7.9 shows that for this particular example all four properties hold.

Let us start with $\text{assoc}(\bowtie, \bowtie)$. To illustrate one problem which occurs in the context of associativity, consider the following three relations:

$\frac{e_1 := R_1}{a}$	$\frac{e_2 := R_2}{\begin{array}{c c} b & c \\ \hline b & - \end{array}}$	$\frac{e_3 := R_3}{d}$
------------------------	---	------------------------

The results of different left outerjoin applications are

$\frac{e_1 \bowtie_{a=b} e_2}{\begin{array}{c c c} a & b & c \\ \hline a & - & - \end{array}}$	$\frac{e_2 \bowtie_{c=d \vee c \doteq \text{null}} e_3}{\begin{array}{c c c} b & c & d \\ \hline b & - & c \end{array}}$
$\frac{(e_1 \bowtie_{a=b} e_2) \bowtie_{c=d \vee c \doteq \text{null}} e_3}{\begin{array}{c c c c} a & b & c & d \\ \hline a & - & - & c \end{array}}$	$\frac{e_1 \bowtie_{a=b} (e_2 \bowtie_{c=d \vee c \doteq \text{null}} e_3)}{\begin{array}{c c c c} a & b & c & d \\ \hline a & - & - & - \end{array}}$

Hence, in general $(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \neq e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)$. The problem is that the predicate q_{23} does *not reject null values*, where a predicate *rejects null values* for a set of attributes A if it evaluates to *false* or *undefined* on every tuple in which all attributes in A are *null*. That is, q rejects null values if and only if $q(\perp_A) \neq \text{true}$. We also say that a predicate is *strict* or *strong* if it rejects null values. For our example predicates, the following holds. All q_{ij} reject null values on any $\mathcal{A}(e_i)$. The predicates $q_{12'}$ and $q_{2'3}$ do not reject null values on $\mathcal{A}(e_2)$ but on $\mathcal{A}(e_1)$ or $\mathcal{A}(e_3)$, respectively. The predicates q'_{ij} neither reject null values on $\mathcal{A}(e_i)$ nor on $\mathcal{A}(e_j)$.

In order to understand why this is the core of the problem, let us investigate this more thoroughly. Define $E_{\perp i} := \{\perp_{\mathcal{A}(e_i)}\}$ for $i = 1, 2, 3$ and $E_{\perp ij} := \{\perp_{\mathcal{A}(e_i) \cup \mathcal{A}(e_j)}\}$ for $i, j = 1, 2, 3$. Further, let q_{12} and q_{23} be join predicates such that $\mathcal{F}(q_{12}) \cap \mathcal{A}(e_3) = \emptyset$ and $\mathcal{F}(q_{23}) \cap \mathcal{A}(e_1) = \emptyset$. For the left-hand side of associativity, we have

$$\begin{aligned}
(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 &\equiv ((e_1 \bowtie_{q_{12}} e_2) \cup ((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 2})) \bowtie_{q_{23}} e_3 \\
&\equiv (((e_1 \bowtie_{q_{12}} e_2) \cup ((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 2})) \bowtie_{q_{23}} e_3) \\
&\quad \cup (((e_1 \bowtie_{q_{12}} e_2) \cup ((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 2})) \bowtie_{q_{23}} e_3) \times E_{\perp 3}) \\
&\equiv ((e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3) \\
&\quad \cup (((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 2}) \bowtie_{q_{23}} e_3) \\
&\quad \cup (((e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3) \times E_{\perp 3}) \\
&\quad \cup (((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 2}) \bowtie_{q_{23}} e_3) \times E_{\perp 3}) \\
&\equiv (e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)) \\
&\quad \cup ((e_1 \bowtie_{q_{12}} e_2) \times (E_{\perp 2} \bowtie_{q_{23}} e_3)) \\
&\quad \cup (e_1 \bowtie_{q_{12}} ((e_2 \bowtie_{q_{23}} e_3) \times E_{\perp 3})) \\
&\quad \cup ((e_1 \bowtie_{q_{12}} e_2) \times (E_{\perp 2} \bowtie_{q_{23}} e_3) \times E_{\perp 3}) \\
&\equiv (e_1 \bowtie_{q_{12}} ((e_2 \bowtie_{q_{23}} e_3) \cup ((e_2 \bowtie_{q_{23}} e_3) \times E_{\perp 3}))) \\
&\quad \cup ((e_1 \bowtie_{q_{12}} e_2) \times ((E_{\perp 2} \bowtie_{q_{23}} e_3) \cup (E_{\perp 2} \bowtie_{q_{23}} e_3) \times E_{\perp 3})) \\
&\equiv (e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)) \\
&\quad \cup ((e_1 \bowtie_{q_{12}} e_2) \times (E_{\perp 2} \bowtie_{q_{23}} e_3)).
\end{aligned}$$

The right part of the cross product on the right-hand side of the union, $(E_{\perp 2} \bowtie_{q_{23}} e_3)$, does look suspicious. Note that if q_{23} rejects nulls on $\mathcal{A}(e_2)$, this part simplifies to $E_{\perp 23}$. To confirm our suspicion, we take a look at the other side of associativity:

$$\begin{aligned} e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) &\equiv (e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)) \\ &\quad \cup ((e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)) \times E_{\perp 23}) \\ &\equiv (e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)) \\ &\quad \cup ((e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 23}). \end{aligned}$$

The last step is true, since $e_2 \bowtie_{q_{23}} e_3$ preserves e_2 and $\mathcal{F}(q_{12}) \cap \mathcal{A}(e_3) = \emptyset$. Thus, the left outerjoin is associative if and only if

$$((e_1 \bowtie_{q_{12}} e_2) \times (E_{\perp 2} \bowtie_{q_{23}} e_3)) \equiv (e_1 \bowtie_{q_{12}} e_2) \times E_{\perp 23}.$$

But this holds if q_{23} rejects nulls on $\mathcal{A}(e_2)$. Thus, without any effort we have just proven the second of the following equivalences:

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \equiv e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3), \quad (7.67)$$

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \equiv e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) \quad (7.68)$$

if q_{23} rejects nulls on $\mathcal{A}(e_2)$,

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \equiv e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) \quad (7.69)$$

if q_{23} rejects nulls on $\mathcal{A}(e_2)$,

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}} e_3 \equiv e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) \quad (7.70)$$

if q_{12} and q_{23} reject nulls on $\mathcal{A}(e_2)$.

As an exercise, the reader should prove the remaining equivalences. This is necessary since the proofs of Galindo-Legaria [297] are valid for sets only.

Let us now come to l-asscom. Fig. 7.11 shows that for our example l-asscom($\bowtie, \bowtie$) does not hold. Bearing the symmetry of l-asscom in mind, the equivalences

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{13}} e_3 \equiv (e_1 \bowtie_{q_{13}} e_3) \bowtie_{q_{12}} e_2, \quad (7.71)$$

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{13}} e_3 \equiv (e_1 \bowtie_{q_{13}} e_3) \bowtie_{q_{12}} e_2, \quad (7.72)$$

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{13}} e_3 \equiv (e_1 \bowtie_{q_{13}} e_3) \bowtie_{q_{12}} e_2 \quad (7.73)$$

if q_{12} rejects nulls on $\mathcal{A}(e_1)$,

$$(e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{13}} e_3 \equiv (e_1 \bowtie_{q_{13}} e_3) \bowtie_{q_{12}} e_2 \quad (7.74)$$

if q_{12} and q_{13} reject nulls on $\mathcal{A}(e_1)$

cover all combinations of $\bowtie, \bowtie$, and $\bowtie$ except the well-known case for regular joins. Since $\text{comm}(\bowtie)$ and $\text{assoc}(\bowtie, \bowtie)$ hold without restrictions, Eqv. 7.71 holds without restrictions. Since $\bowtie$ is strongly left linear and the consumer/producer relationship is not disturbed because q_{12} does not access attributes from e_3 and q_{13} does not access attributes from e_2 , Eqv. 7.72 holds without restrictions. From Eqv. 7.69 and the fact that the full outerjoin is commutative, Eqv. 7.73 follows. Some care is just needed to see how the necessary restriction for associativity carries over to the l-asscom equivalence. Similarly, Eqv. 7.74 follows

from Eqv.7.70. In all cases, the necessity of the restrictions is due to the fact that commutativity and l-asscom imply associativity.

The r-asscom property is handled quickly. The only valid equivalence we have is

$$e_1 \bowtie_{q_{13}} (e_2 \bowtie_{q_{23}} e_3) \equiv e_2 \bowtie_{q_{23}} (e_1 \bowtie_{q_{13}} e_3), \quad (7.75)$$

which follows directly from $\text{comm}(\bowtie)$ and $\text{assoc}(\bowtie, \bowtie)$ if q_{13} and q_{23} are both strict on $\mathcal{A}(e_3)$.

7.10.1 Outerjoin Simplification

Sometimes, a left or full outerjoin can be turned into a regular or one-sided outerjoin if it is followed a unary or binary algebraic operator with a strict predicate. These simplifications are important and should be applied before plan generation. We first present equivalences showing that selections are left-pushable and right-detachable for the left outerjoin:

$$\sigma_{p_1}(e_1 \bowtie_q e_2) \equiv \sigma_{p_1}(e_1) \bowtie_q e_2 \quad (7.76)$$

$$e_1 \bowtie_{q \wedge p_2} e_2 \equiv e_1 \bowtie_q \sigma_{p_2}(e_2) \quad (7.77)$$

This possibly leads to more applications of the following equivalences.

We start with the simple equivalences where a selection removes the null-padded outer join tuples:

$$\sigma_p(e_1 \bowtie_q e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } p \text{ rejects nulls on } \mathcal{A}(e_2), \quad (7.78)$$

$$\sigma_p(e_1 \bowtie_q e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } p \text{ rejects nulls on } \mathcal{A}(e_1), \quad (7.79)$$

$$\sigma_p(e_1 \bowtie_q e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } p \text{ rejects nulls on } \mathcal{A}(e_2). \quad (7.80)$$

We can extend the last two equivalences to outerjoins with default values:

$$\sigma_p(e_1 \bowtie_q^{D_2} e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } \neg p(D_2), \quad (7.81)$$

$$\sigma_p(e_1 \bowtie_q^{D_1, D_2} e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } \neg p(D_1), \quad (7.82)$$

$$\sigma_p(e_1 \bowtie_q^{D_1, D_2} e_2) \equiv \sigma_p(e_1 \bowtie_q e_2) \quad \text{if } \neg p(D_2). \quad (7.83)$$

Using right detachability of implied **not null** predicates, we can easily prove the following equivalences

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.84)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.85)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.86)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.87)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.88)$$

These equivalences hold under the condition that p rejects nulls on $\mathcal{A}(e_3)$. In

[308, 297], we find the first two of them. Further, the equivalences

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.89)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.90)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.91)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.92)$$

$$e_1 \bowtie_p (e_2 \bowtie_q e_3) \equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \quad (7.93)$$

hold if p rejects nulls on $\mathcal{A}(e_2)$. Commutativity of the full outerjoin gives symmetric equivalences if p rejects nulls on $\mathcal{A}(e_3)$.

7.10.2 Generalized Outerjoin

As pointed out by Galindo-Legaria and Rosenthal [736, 298, 308], the different outerjoins can be defined using the *outer union* operator, which in turn was introduced by Codd [200]. Let e_1 and e_2 be two relations and A_1 and A_2 their corresponding attributes. The *outer union* is then defined by padding the union of the relations with *null* values to the schema $A_1 \cup A_2$:

$$e_1 \overset{+}{\cup} e_2 := (e_1 \times \{\perp_{A_2 \setminus A_1}\}) \cup (e_2 \times \{\perp_{A_1 \setminus A_2}\}). \quad (7.94)$$

Given this definition of the *outer union* operator, we can define the outerjoin operations as follows:

$$e_1 \bowtie_q e_2 := e_1 \bowtie_q e_2 \overset{+}{\cup} (e_1 \setminus \Pi_{A_1}(e_1 \bowtie_q e_2)), \quad (7.95)$$

$$e_1 \bowtie_q e_2 := e_1 \bowtie_q e_2 \overset{+}{\cup} (e_1 \setminus \Pi_{A_1}(e_1 \bowtie_q e_2)) \overset{+}{\cup} (e_2 \setminus \Pi_{A_2}(e_1 \bowtie_q e_2)). \quad (7.96)$$

The expression $e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3)$ cannot be reordered given the equivalences so far. In order to allow reorderability of this expression, the *generalized outerjoin* was introduced by Dayal [220]. Here, we follow Rosenthal and Galindo-Legaria [736]. The *generalized left outerjoin* preserves attributes for a subset $A \subseteq \mathcal{A}(e_1)$ only. It is defined as

$$e_1 \bowtie_q^A e_2 := (e_1 \bowtie_q e_2) \overset{+}{\cup} (\Pi_A(e_1) \setminus \Pi_A(e_1 \bowtie_q e_2)). \quad (7.97)$$

However, we prefer a slightly different definition based on the antijoin:

$$e_1 \bowtie_q^A e_2 := (e_1 \bowtie_q e_2) \cup (\Pi_A(e_1 \bowtie_q e_2) \times \{\perp_{A \cup \mathcal{A}(e_2)}\}_b). \quad (7.98)$$

This definition is equivalent to the one above.

The generalized left outerjoin allows us to reorder left outerjoins and joins as well as full outerjoins and joins, but only in the context of sets. The equivalences

$$e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) \equiv (e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}}^{A(e_1)} e_3 \quad (7.99)$$

if q_{23} rejects nulls on $\mathcal{A}(e_2)$,

$$e_1 \bowtie_{q_{12}} (e_2 \bowtie_{q_{23}} e_3) \equiv (e_1 \bowtie_{q_{12}} e_2) \bowtie_{q_{23}}^{A(e_1)} e_3 \quad (7.100)$$

if q_{23} rejects nulls on $\mathcal{A}(e_2)$.

only hold for sets.

The following is a counterexample for bags. Define $R_1 := \{[a_1 : 1, b_1 : 1]\}_b$, $R_2 := \{[a_2 : 1, b_2 : 1], [a_2 : 2, b_2 : 1]\}_b$, and $R_3 := \emptyset_b$ with schema $\{[a_3 : \text{int}, b_3 : \text{int}]\}_b$. Evaluating

$$R_1 \bowtie_{b_1=b_2} (R_2 \bowtie_{b_2=b_3} R_3)$$

then yields

$$\{[a_1 : 1, b_1 : 1, a_2 : -, b_2 : -, a_3 : -, b_3 : -]\}_b.$$

Evaluating

$$(R_1 \bowtie_{b_1=b_2} R_2)$$

yields

$$\{[a_1 : 1, b_1 : 1, a_2 : 1, b_2 : 1], [a_1 : 1, b_1 : 1, a_2 : 2, b_2 : 1]\}_b.$$

Thus,

$$(R_1 \bowtie_{q_{12}} R_2) \bowtie_{q_{23}}^{A(R_1)} R_3$$

evaluates to

$$\{[a_1 : 1, b_1 : 1, a_2 : -, b_2 : -, a_3 : -, b_3 : -]^2\}_b.$$

We only discussed the basic equivalences for reordering algebraic expressions containing outerjoins. General frameworks for dealing with these expressions in toto are presented in [85, 86, 297, 298, 308]. Especially, the generalized left outerjoin can be generalized to preserve disjoint sets of attributes in order to derive more equivalences [297, 308], which also only hold in the context of sets. Bhargava, Goel, and Iyer propose the modified generalized outer join (MGOJ), a variant of the generalized outerjoin which correctly deals with bags [85].

ToDo

7.11 Equivalences for Unary Grouping

7.11.1 An Elementary Fact about Grouping

Let us first resume the discussion of properties of the grouping operator, which we started in Sec. 7.7.1. Assume that the functional dependencies $H \rightarrow G$ and $G \rightarrow H$ hold. Then, it should not make any difference whether we group by H or G . The only problem we have to solve is that H might not contain all attributes of G (or vice versa). However, since $H \rightarrow G$, any attribute $g \in (G \setminus H)$ has only one possible value per group if we group by H . Thus, we can simply copy this value. We do so by adding a new aggregation function $\text{cpf}(g)$, which copies the value g of the first tuple seen for a group. This is deterministic, since all tuples in a group have the same value for g (as $H \rightarrow G$). Thus, to make sure that all values of G are extracted if we group according to H , we extend a given aggregation vector F as follows. Assume $(G \setminus H) = \{g_1, \dots, g_k\}$. Then, we define $F \circ (G \setminus H)$ as

$$F \circ (g_1 : \text{cpf}(g_1), \dots, g_k : \text{cpf}(g_k)).$$

Using this definition, we can state the equivalence

$$\Gamma_{G;F}(e) \equiv \Pi_C(\Gamma_{H;F \circ (G \setminus H)}(e)), \quad (7.101)$$

which holds if $H \rightarrow G$ and $C = G \cup \mathcal{A}(F)$. This equivalence allows to determine some set H with $H \rightarrow G$ such that grouping on H might become cheaper compared to grouping on G . Especially, it allows to minimize the number of grouping attributes. This trick can be applied to all equivalences in this section.

Let G be a set of grouping attributes and assume that $G \rightarrow \text{TID}(e)$. Then, every group consists of only one tuple, i.e., $\Pi_G^D(e) = \Pi_G(e)$, and we can replace a grouping by a map:

$$\Gamma_{G;F}(e) \equiv \Pi_C(\chi_{\hat{F}}(e)) \quad (7.102)$$

if $F = (b_1 : \text{agg}_1(a_1), \dots, b_m : \text{agg}_m(a_m))$, $\hat{F} = (b_1 : a_1, \dots, b_m : a_m)$, and $C = G \cup \mathcal{A}(F)$. Note that using F instead of $\hat{F}$ also works. Tsois and Sellis call this equivalence *remove-group-by* [882].

7.11.2 Join

Let us now come to some more complex cases concerning the reorganization of expressions containing grouping and join. Traditionally, the grouping operator, if specified in some SQL query, is performed after the evaluation of all join operations. However, pushing down grouping can substantially reduce the input size of the joins and, thus, can be highly beneficial. Before we give some equivalences, let us look at some example relations, their joins and the result of applying some grouping operators. Fig 7.12 presents two relations R_1 and R_2 and the result of their join (e_3) in the top row. The next row shows the result of applying a grouping operator to each of these items (e_4 to e_6). The last row contains the results of joining a grouped result with one original relation and the result of joining the two grouped results given in e_4 and e_5 . Let us assume that our original expression

$$\Gamma_{g_1, g_2; c: \text{count}(*), b_1: \text{sum}(a_1), b_2: \text{sum}(a_2)}(e_1 \bowtie_{j_1=j_2} R_2)$$

is equivalent to the original query we have to evaluate. The result of this expression is given in expression e_6 of Fig 7.12. The question is whether any of e_7 , e_8 , or e_9 can be used to provide the same result. Let us start with e_7 . We have $\text{sum}(c_1) = 4$ and $\text{sum}(b'_1) = 16$, which is perfect. However, $\text{sum}(a_2) = 14$, but according to e_6 we should have 22. How can we fix that? Note that the difference is 8, which is exactly the value we see in the last row of e_7 . Further, the count in c_1 is 2. It indicates that for $g_1 = 1$ and $j_1 = 2$ two tuples were grouped. Each of these tuples joins with the tuple $(1, 2, 8)$ of R_2 . This is what we missed. This point is illustrated in the example in Fig. 7.13, where we added one more tuple to e_2 , which is marked by a *. All tuples to which this tuple contributes are also marked by a * later on. The reader should carefully follow the stars.

Let us return to e_7 in Fig. 7.12. We have to apply some correction for the fact that $(1, 2, 8)$ of R_2 has two ($= c_1$) join partners. Thus, we calculate $\text{sum}(c_1 * a_2)$ in e_7 instead of the plain $\text{sum}(a_2)$ and get the correct result 22. Let us turn to e_8 in Fig. 7.12. There, we calculate $\text{sum}(a_1) = 16$, $\text{sum}(b'_2) = 22$, and $\text{sum}(c_2) = 4$, which are all correct results. However, in e_8 of Fig. 7.13, the tuples $(1, 2, 4)$ and $(1, 2, 8)$ of R_1 now find two join partners in R_2 , since

we added the tuple marked by the asterisk. We should expect problems! And there are some, since $\text{sum}(a_1)$ in e_8 gives 14 but should result in 28, according to e_6 . Again, c_2 holds the number of join partners the R_1 tuples find in R_2 . Thus calculating $\text{sum}(a_2 * c_2) = 28$ solves the problem.

Turning our attention to e_9 of Fig. 7.12, we see that $\text{sum}(b'_1 * c_2) = 16$ and $\text{sum}(b'_2 * c_1) = 22$ give the correct results (compare these values to b_1 and b_2 of e_6). However, $\text{sum}(c_1) = \text{sum}(c_2) = 3$ but c of e_6 indicates that 4 is the correct result. The reader might guess that we have to take $\text{sum}(c_1 * c_2) = 4$, which indeed is what has to be calculated.

These observations give rise to the following definition. Let $F = (b_1 : \text{agg}_1(a_1), \dots, b_m : \text{agg}_m(a_m))$ be an aggregation vector. We define $F \otimes c$ for some attribute c , which will typically contain the result of some $\text{count}(*)$, as $F \otimes c = (b_1 : \text{agg}'_1(e_1), \dots, b_m : \text{agg}'_m(e_m))$ with

$$\text{agg}'_i(e_i) = \begin{cases} \text{agg}_i(e_i) & \text{if } \text{agg}_i \text{ is duplicate agnostic,} \\ \text{agg}_i(e_i * c) & \text{if } \text{agg}_i \text{ is the duplicate sensitive sum,} \\ \text{sum}(c) & \text{if } \text{agg}_i(e_i) = \text{count}(*), \\ \text{sum}(e_i = \text{NULL} ? 0 : c) & \text{if } \text{agg}_i(e_i) = \text{count}(e_i), e_i \neq '*'. \end{cases}$$

The goal now is to exploit the decomposability of aggregation functions and this definition to push a grouping down a join.

Let us start with an equivalence, which Rosenthal, Rich, and Scholl [742, 729] used to speed up join processing. They noted that an ordinary join can be calculated by unnesting the result of a join of two nested relations. Let q be a join predicate and e_1 and e_2 be two expressions to be joined. Denote by $J_i = \mathcal{F}(q) \cap \mathcal{A}(e_i)$ the join attributes from e_i for $i = 1, 2$. Then

$$e_1 \bowtie_q e_2 \equiv \mu_{g_2}(\mu_{g_1}(\Gamma_{J_1;g_1:\Pi_{\mathcal{A}(e_1)\setminus J_1}}(e_1) \bowtie_q \Gamma_{J_2;g_2:\Pi_{\mathcal{A}(e_2)\setminus J_2}}(e_2))). \quad (7.103)$$

We need the projections $\Pi_{\mathcal{A}(e_i)\setminus J_i}$ to prevent the duplication of attributes in the result.

J_1 and J_2 are the minimal sets of grouping attributes we can use, but nothing hinders us from using larger grouping sets. Let G_1^+ and G_2^+ be two sets of attributes with $J_i \subseteq G_i^+$ for $i = 1, 2$. Then

$$e_1 \bowtie_q e_2 \equiv \mu_{g_2}(\mu_{g_1}(\Gamma_{G_1^+;g_1:\Pi_{\mathcal{A}(e_1)\setminus J_1}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2:\Pi_{\mathcal{A}(e_2)\setminus J_2}}(e_2))) \quad (7.104)$$

holds. Unnesting two nested attributes g_1 and g_2 in a row, as done in the above equivalences, is like generating the cross product of the items contained in g_1 and g_2 . Under the above assumptions, we can thus state the following two equivalences

$$e_1 \bowtie_q e_2 \equiv \Pi_{\overline{g_1, g_2}}(\mu_g(\chi_{g:g_1 \times g_2}(\Gamma_{J_1;g_1:\Pi_{\mathcal{A}(e_1)\setminus J_1}}(e_1) \bowtie_q \Gamma_{J_2;g_2:\Pi_{\mathcal{A}(e_2)\setminus J_2}}(e_2))))), \quad (7.105)$$

$$e_1 \bowtie_q e_2 \equiv \Pi_{\overline{g_1, g_2}}(\mu_g(\chi_{g:g_1 \times g_2}(\Gamma_{G_1^+;g_1:\Pi_{\mathcal{A}(e_1)\setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2:\Pi_{\mathcal{A}(e_2)\setminus G_2^+}}(e_2))))). \quad (7.106)$$

$e_1 := R_1$		
g_1	j_1	a_1
1	1	2
1	2	4
1	2	8

$e_2 := R_2$		
g_2	j_2	a_2
1	1	2
1	1	4
1	2	8

$e_3 := R_1 \bowtie_{j_1=j_2} R_2$					
g_1	j_1	a_1	g_2	j_2	a_2
1	1	2	1	1	2
1	1	2	1	1	4
1	2	4	1	2	8
1	2	8	1	2	8

$e_4 := \Gamma_{g_1, j_1; F_1}(e_1)$			
g_1	j_1	c_1	b'_1
1	1	1	2
1	2	2	12

$e_5 := \Gamma_{g_2, j_2; F_2}(e_2)$			
g_2	j_2	c_2	b'_2
1	1	2	6
1	2	1	8

$e_6 := \Gamma_{g_1, g_2; F}(e_3)$			
g_1	c	b_1	b_2
1	4	16	22

$e_7 := e_4 \bowtie_{j_1=j_2} e_2$						
g_1	j_1	c_1	b'_1	g_2	j_2	a_2
1	1	1	2	1	1	2
1	1	1	2	1	1	4
1	2	2	12	1	2	8

$e_8 := e_1 \bowtie_{j_1=j_2} e_5$						
g_1	j_1	a_1	g_2	j_2	c_2	b'_2
1	1	2	1	1	2	6
1	2	4	1	2	1	8
1	2	8	1	2	1	8

$e_9 := e_4 \bowtie_{j_1=j_2} e_5$							
g_1	j_1	c_1	b'_1	g_2	j_2	c_2	b'_2
1	1	1	2	1	1	2	6
1	2	2	12	1	2	1	8

where

$$\begin{aligned}
F &= c : \text{count}(*), b_1 : \text{sum}(a_1), b_2 : \text{sum}(a_2) \\
F_1 &= c_1 : \text{count}(*), b'_1 : \text{sum}(a_1) \\
F_2 &= c_2 : \text{count}(*), b'_2 : \text{sum}(a_2)
\end{aligned}$$

Figure 7.12: Example for grouping and join

In the next step, we want to get rid of g . Assume we apply some aggregation function $\text{agg}(g.a_i)$ to g of the latter equivalence, where a_i is an attribute of g_1 , i.e., $a_i \in \mathcal{A}(g_1)$ or, by definition, $a_i \in \mathcal{A}(e_1) \setminus G_1^+$. It should be clear that

$$\text{agg}(g.a_i) = \begin{cases} \text{sum}(g_1.a_i) * |g_2| & \text{if agg} = \text{sum}, \\ \text{count}(g_1.a_i) * |g_2| & \text{if agg} = \text{count}, \\ \min(g_1.a_i) & \text{if agg} = \text{min}, \\ \max(g_1.a_i) & \text{if agg} = \text{max}, \end{cases}$$

and $|g| = |g_1| * |g_2|$. Analogously, we can exchange 1 and 2 in case $a_i \in \mathcal{A}(g_2)$.

Now, we are prepared to add an additional grouping operator $\Gamma_{G^+, g; F}$ to both sides of Eqv. 7.106. Therefore, we assume that $J_1 \subseteq G^+$ and $J_2 \subseteq G^+$. Further, we define G_i^+ as $G^+ \cap \mathcal{A}(e_i)$ for $i = 1, 2$. This results in

$$\begin{aligned}
\Gamma_{G^+, g; F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G^+, g; F}(\Pi_{\overline{g_1, g_2}}(\mu_g(\chi_{g: g_1 \times g_2} (\\
&\quad \Gamma_{G_1^+, g_1; \Pi_{\mathcal{A}(e_1) \setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+, g_2; \Pi_{\mathcal{A}(e_2) \setminus G_2^+}}(e_2))))).
\end{aligned}$$

We know that g , g_1 , and g_2 cannot be part of the left-hand side. This means that they cannot occur in G or $\mathcal{A}(F)$. Thus, we can eliminate the projection,

$e_1 := R_1$		
g_1	j_1	a_1
1	1	2
1	2	4
1	2	8

$e_2 := R_2$			
g_2	j_2	a_2	
1	1	2	
1	1	4	
1	2	8	
1	2	16	*

$e_3 := e_1 \bowtie_{j_1=j_2} e_2$						
g_1	j_1	a_1	g_2	j_2	a_2	
1	1	2	1	1	2	
1	1	2	1	1	4	
1	2	4	1	2	8	
1	2	4	1	2	16	*
1	2	8	1	2	8	
1	2	8	1	2	16	*

$e_4 := \Gamma_{g_1, j_1; F_1}(e_1)$			
g_1	j_1	c_1	b'_1
1	1	1	2
1	2	2	12

$e_5 := \Gamma_{g_2, j_2; F_2}(e_2)$				
g_2	j_2	c_2	b'_2	
1	1	2	6	
1	2	2	24	*

$e_6 := \Gamma_{g_1, g_2; F}(e_3)$				
g_1	c	b_1	b_2	
1	6	28	54	*

$e_7 := e_4 \bowtie_{j_1=j_2} e_5$							
g_1	j_1	c_1	b'_1	g_2	j_2	a_2	
1	1	1	2	1	1	2	
1	1	1	2	1	1	4	
1	2	2	12	1	2	8	
1	2	2	12	1	2	16	*

$e_8 := e_1 \bowtie_{j_1=j_2} e_5$							
g_1	j_1	a_1	g_2	j_2	c_2	b'_2	
1	1	2	1	1	2	6	
1	2	4	1	2	2	24	*
1	2	8	1	2	2	24	*

$e_9 := e_4 \bowtie_{j_1=j_2} e_5$								
g_1	j_1	c_1	b'_1	g_2	j_2	c_2	b'_2	
1	1	1	2	1	1	2	6	
1	2	2	12	1	2	2	24	*

where

where

$$\begin{aligned}
F &= c : \text{count}(*), b_1 : \text{sum}(a_1), b_2 : \text{sum}(a_2) \\
F_1 &= c_1 : \text{count}(*), b'_1 : \text{sum}(a_1) \\
F_2 &= c_2 : \text{count}(*), b'_2 : \text{sum}(a_2)
\end{aligned}$$

Figure 7.13: Extended example for grouping and join

which gives us

$$\Gamma_{G^+;g;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G^+;g;F}(\mu_g(\chi_{g:g_1 \times g_2}(\Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1) \setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2) \setminus G_2^+}}(e_2))))$$

Now, note that the outer grouping on the right-hand side undoes the unnesting which immediately proceeds it. We could be tempted to rewrite the right-hand side to something like

$$\chi_F(\chi_{g:g_1 \times g_2}(\Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1) \setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2) \setminus G_2^+}}(e_2))).$$

In order to verify this, we have to take a close look at

$$E := (\Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1) \setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2) \setminus G_2^+}}(e_2)).$$

We must make sure that E produces only a single tuple for each group constructed by $\Gamma_{G^+;g;F}$. From the definition of Γ , we see that neither $\Pi_{G_1^+}(\Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1) \setminus G_1^+}}(e_1))$

nor $\Pi_{G_2^+}(\Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2)\setminus G_2^+}}(e_2))$ contain duplicates. Since $G^+ = G_1^+ \cup G_2^+$, the claim follows.

In the next step, we eliminate $\chi_{g:g_1 \times g_2}$. As we have seen, we might need the cardinalities of g_1 and g_2 if we have to deal with duplicate sensitive aggregation functions. We can calculate them using a map operator. Let us further assume that F can be split into F_1 and F_2 such that $F = F_1 \circ F_2$ and the only free variable of F_i is g_i . Then we can rewrite the equivalence to

$$\begin{aligned} \Gamma_{G^+;g;F}(e_1 \bowtie_q e_2) \equiv & \chi_{F_2 \otimes c_1} (\\ & \chi_{F_1 \otimes c_2} (\\ & \chi_{c_1:|g_1|} (\\ & \chi_{c_2:|g_2|} (\\ & \Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1)\setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2)\setminus G_2^+}}(e_2))))). \end{aligned}$$

Next, we should start moving the different map operators inwards. The only problem occurs since $F_1 \otimes c_2$ and $F_2 \otimes c_1$ need elements of both parts of the join. Let F_1 be decomposable into F_1^1 and F_1^2 and F_2 be decomposable into F_2^1 and F_2^2 . Then, we have

$$\begin{aligned} \Gamma_{G^+;g;F}(e_1 \bowtie_q e_2) \equiv & \chi_{F_2^2 \otimes c_1} (\\ & \chi_{F_1^2 \otimes c_2} (\\ & \chi_{F_1^1} (\\ & \chi_{F_2^1} (\\ & \chi_{c_1:|g_1|} (\\ & \chi_{c_2:|g_2|} (\\ & \Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1)\setminus G_1^+}}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2)\setminus G_2^+}}(e_2)))))). \end{aligned}$$

Pushing down the last four χ operators yields

$$\begin{aligned} \Gamma_{G^+;g;F}(e_1 \bowtie_q e_2) \equiv & \chi_{F_2^2 \otimes c_1} (\\ & \chi_{F_1^2 \otimes c_2} (\\ & \chi_{F_1^1} (\chi_{c_1:|g_1|}(\Gamma_{G_1^+;g_1;\Pi_{\mathcal{A}(e_1)\setminus G_1^+}}(e_1))) \\ & \bowtie_q \\ & \chi_{F_2^1} (\chi_{c_2:|g_2|}(\Gamma_{G_2^+;g_2;\Pi_{\mathcal{A}(e_2)\setminus G_2^+}}(e_2))))), \end{aligned}$$

which can now easily be rewritten to the following equivalence by observing that g_1 and g_2 are not needed outside the join:

$$\begin{aligned} \Gamma_{G^+;g;F}(e_1 \bowtie_q e_2) \equiv & \chi_{F_1^2 \otimes c_2, F_2^2 \otimes c_1} (\\ & \Gamma_{G_1^+;g_1;F_1^1 \circ (c_1:|g_1|)}(e_1) \bowtie_q \Gamma_{G_2^+;g_2;F_2^1 \circ (c_2:|g_2|)}(e_2)). \end{aligned} \quad (7.107)$$

In our SQL-notation based variant of Γ , this equivalence reads

$$\begin{aligned} \Gamma_{G^+;F}(e_1 \bowtie_q e_2) \equiv & \chi_{F_1^2 \otimes c_2, F_2^2 \otimes c_1} (\\ & \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)) \end{aligned} \quad (7.108)$$

and it holds if F is splittable into F_1 and F_2 such that $\mathcal{F}(F_i) \subseteq \mathcal{A}(e_i)$ and $F = F_1 \circ F_2$, and F_i is splittable and decomposable into F_i^1 and F_i^2 .

Consider the expression $\Gamma_{G;g;F}(e_1 \bowtie_q e_2)$. We denote the set of join attributes of q from e_i as $J_i = \mathcal{F}(q) \cap \mathcal{A}(e_i)$ for $i = 1, 2$, and the set of all join attributes by $J = J_1 \cup J_2$. If $J \subseteq G$, we have the above case. Assume $J \not\subseteq G$. Define $G^+ = G \cup J$, $G_i = G \cap \mathcal{A}(e_i)$ and $G_i^+ = G_i \cup J_i$ for $i = 1, 2$. Let F be an aggregation vector splittable into F_1 and F_2 such that $\mathcal{F}(F_i) \subseteq \mathcal{A}(e_i)$ and $F = F_1 \circ F_2$. Further, let F_i be decomposable into F_i^1 and F_i^2 . Then Eqvs. 7.13 and 7.108, together with the properties of aggregation functions and vectors discussed in Sec. 7.2, give us the following

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;F_1^2, F_2^2}(\Gamma_{G^+;F_1^1, F_2^1}(e_1 \bowtie_q e_2)), \\ &\equiv \Gamma_{G;F_1^2, F_2^2}(\chi_{F_1^{1,2} \otimes c_2, F_2^{1,2} \otimes c_1} (\\ &\quad \Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2))), \\ &\equiv \Gamma_{G;F_1^2 \otimes c_2, F_2^2 \otimes c_1} (\\ &\quad \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)), \end{aligned}$$

where $C = G \cup \mathcal{A}(F)$. It should be obvious that the expression on the right-hand side can be cleaned up in case it contains several $\text{count}(*)$, or no $\text{count}(*)$ is needed because all aggregation functions in some F_i are duplicate agnostic.

By now, the reader should be prepared to understand and prove the equivalences provided by Yan and Larson, which we present next. Let e_1 and e_2 be two expressions and q a join predicate for them. Define for $i = 1, 2$ the following sets of join attributes $J_i = \mathcal{F}(q) \cap \mathcal{A}(e_i)$. Let $F_1 = (b_1 : \text{agg}_1(a_1), \dots, b_k : \text{agg}_k(a_k))$ and $F_2 = (b_{k+1} : \text{agg}_{k+1}(a_{k+1}), \dots, b_m : \text{agg}_m(a_m))$ be two aggregation vectors. Define $A_1 = \{a_1, \dots, a_k\}$, $A_2 = \{a_{k+1}, \dots, a_m\}$, $F = F_1 \circ F_2$, $A = A_1 \cup A_2$, and $B = \{b_1, \dots, b_m\}$. Let G be a set of grouping attributes and define $G_i \cap \mathcal{A}(e_i)$. We denote by G_i^+ the union of the grouping and join attributes of e_i , that is, $G_i^+ = G_i \cup J_i$.

Eager/Lazy Groupby-Count

The following equivalence corresponds to the main theorem of Yan and Larson [947]. It states that

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \quad (7.109)$$

holds if F is splittable and F_1 is decomposable into F_1^1 and F_1^2 . The proof of it can be found in [947].

From Eqv. 7.109 several other equivalences can be derived easily. First, since the join is commutative,

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)) \quad (7.110)$$

holds if F is splittable and F_2 is decomposable into F_2^1 and F_2^2 .

Eager/Lazy Group-by

If F_2 is empty, that is $F_2 = ()$, then Eqv. 7.109 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_1^2}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q e_2). \quad (7.111)$$

This equivalence holds if F_1 is decomposable into F_1^1 and F_1^2 .

If F_1 is empty, then Eqv. 7.110 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1}(e_2)). \quad (7.112)$$

This equivalence holds if F_2 is decomposable into F_2^1 and F_2^2 .

Eager/Lazy Count

If $F_1 = ()$, then Eqv. 7.109 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1)}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q e_2). \quad (7.113)$$

If $F_2 = ()$, then Eqv. 7.110 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2)}(e_1 \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)). \quad (7.114)$$

Double Eager/Lazy

For the next equivalence, assume $F_2 = ()$. Then

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\stackrel{\text{Eqv. 7.111}}{=} \Gamma_{G;F_1^2}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q e_2) \\ &\stackrel{\text{Eqv. 7.114}}{=} \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)). \end{aligned}$$

Thus,

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)) \quad (7.115)$$

if F_1 is decomposable into F_1^1 and F_1^2 .

If F_1 is empty, then, due to the commutativity of the join, the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1}(e_2)) \quad (7.116)$$

holds if F_2 is decomposable into F_2^1 and F_2^2 .

Eager/Lazy Split

Applying Eqv. 7.109 and then Eqv. 7.110 results in the equivalence

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;(F_1^2 \otimes c_2) \circ (F_2^2 \otimes c_1)}(\\ &\quad \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)), \end{aligned} \quad (7.117)$$

which holds if F is splittable into F_1 and F_2 , F_1 is decomposable into F_1^1 and F_1^2 , and F_2 is decomposable into F_2^1 and F_2^2 .

R_1			R_2^{good}		R_2^{bad}		R_2^{ugly}		
g_1	j_1	a_1	g_2	j_2	g_2	j_2	g_2	j_2	k_2
1	1	2	1	1	1	1	1	1	1
1	2	4	2	2	1	2	2	1	2
1	2	8	2	2	1	2	2	1	3

$e_{12}^g := R_1 \bowtie_{j_1=j_2} R_2^{\text{good}}$					$E_1^g := \Gamma_{g_1, g_2; F}(e_{12}^g)$			
g_1	j_1	a_1	g_2	j_2	g_1	g_2	c_1	b_1
1	1	2	1	1	1	1	1	2
1	2	4	2	2	1	1	1	2
1	2	8	2	2	1	2	2	12

$e_{12}^b := R_1 \bowtie_{j_1=j_2} R_2^{\text{bad}}$					$E_1^b := \Gamma_{g_1, g_2; F}(e_{12}^b)$			
g_1	j_1	a_1	g_2	j_2	g_1	g_2	c_1	b_1
1	1	2	1	1	1	1	1	2
1	2	4	1	2	1	1	3	14
1	2	8	1	2				

$e_{12}^u := R_1 \bowtie_{j_1=j_2} R_2^{\text{ugly}}$						$E_1^u := \Gamma_{g_1, g_2; F}(e_{12}^u)$			
g_1	j_1	a_1	g_2	j_2	k_2	g_1	g_2	c_1	b_1
1	1	2	1	1	1	1	1	1	2
1	1	2	2	1	2	1	1	1	2
1	1	2	2	1	3	1	2	2	4

$G_1 := \Gamma_{g_1, j_1; F}(R_1)$				$E_2^g := G_1 \bowtie_{j_1=j_2} R_2^{\text{good}}$					
g_1	j_1	c_1	b_1	g_1	j_1	c_1	b_1	g_2	j_2
1	1	1	2	1	1	1	2	1	1
1	2	2	12	1	2	2	12	2	2

$G_1 := \Gamma_{g_1, j_1; F}(R_1)$				$E_2^b := G_1 \bowtie_{j_1=j_2} R_2^{\text{bad}}$					
g_1	j_1	c_1	b_1	g_1	j_1	c_1	b_1	g_2	j_2
1	1	1	2	1	1	1	2	1	1
1	2	2	12	1	2	2	12	1	2

$G_1 := \Gamma_{g_1, j_1; F}(R_1)$				$E_2^u := G_1 \bowtie_{j_1=j_2} R_2^{\text{ugly}}$						
g_1	j_1	c_1	b_1	g_1	j_1	c_1	b_1	g_2	j_2	k_2
1	1	1	2	1	1	1	2	1	1	1
1	2	2	12	1	1	1	2	2	1	2
				1	1	1	2	2	1	3

where $F = [c_1 : \text{count}(*), b_1 : \text{sum}(a_1)]$.

Figure 7.14: Example for Eqv. 7.118

Eliminating the top grouping

Historically, the first equivalence that reordered grouping and join was derived by Yan and Larson [946]. Opposed to the equivalences above, it has no final grouping on the right-hand side. Grouping is simply pushed down into the right path. Before we present the equivalence, we need some specialization of our notation. Let G be a set of grouping attributes, $F = (b_1 : \text{agg}_1(a_1), \dots, b_k : \text{agg}_k(a_k))$ an aggregation vector, $A = \{a_1, \dots, a_k\}$ the set of aggregated attributes, $B = \{b_1, \dots, b_k\}$ the set of attributes containing the results of the

aggregations, $G = G_1 \cup G_2$ the grouping attributes, $G_i = G \cap \mathcal{A}(e_i)$, q a join predicate, $J_i = \mathcal{F}(q) \cap \mathcal{A}(e_i)$, and $G_1^+ = G_1 \cup J_1$. The following equivalence demands that aggregation functions are only applied to the attributes of e_1 . That is, $A \cap \mathcal{F}(e_2) = \emptyset$. The equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_{G,B}(\Gamma_{G_1^+;F}(e_1) \bowtie_q \Pi_{G_2^+}(e_2)) \quad (7.118)$$

holds if and only if the following two functional dependencies hold in $e_1 \bowtie_q e_2$:

FD 1 $(G_1, G_2) \rightarrow G_1^+$ and

FD 2 $(G_1^+, G_2) \rightarrow \text{TID}(e_2)$.

For FD 2, we use an artificial attribute TID for expression e_2 . It can be present explicitly or just in the mind of the query compiler. Its purpose is to uniquely identify a tuple in e_2 . A consequence of FD 2 is that e_2 cannot contain duplicates (without the TID attribute!). This point is illustrated in the example below. Further, since it does not contain duplicates (again, if TID is ignored), we might assume that e_2 has a key (either an artificially generated one using the tid operator or, in case it is a base relation, a user-specified primary key). Then, we can replace the TID on the right-hand side by the key.

Further note that the functional dependencies can be simplified. We did not do so since we wanted to state them in the same way as Yan and Larson did. As an exercise, the reader should perform the simplification.

The purpose of the functional dependencies can be sketched as follows. FD 1 ensures that each group on the left-hand side corresponds to one group on the right-hand side. That is, the grouping by G_1^+ is not finer grained than the grouping by G . FD 2 ensures that each row in the left argument of the join on the right-hand side contributes at most one row to the overall result of the right-hand side. This is illustrated by the following examples.

Fig 7.14 contains a relation R_1 , which we use for expression e_1 , and three relations R_2^{good} , R_2^{bad} , and R_2^{ugly} , which we use for expression e_2 . All of them are depicted in the top row of Fig. 7.14. The next three rows contain the evaluations of the left-hand side of Eqv. 7.118, divided into two steps. The first step (left column) calculates the join between R_1 and each of the possibilities for e_2 . The second step groups the result of the join (right column). The last three columns evaluate the right-hand side of Eqv. 7.118. Again, the calculation is separated into two steps. The first step does the grouping, the second step the join. We leave the execution of the final projection to the reader.

For this example, the functional dependencies read as follows:

FD 1 $(g_1, g_2) \rightarrow g_1, j_1$ and

FD 2 $(g_1, j_1, g_2) \rightarrow \text{tid}(e_2)$.

In case of R_2^{good} , we observe that both functional dependencies hold. We further observe that the left-hand side and the right-hand side of Eqv. 7.118 give the same result. In case of R_2^{bad} , we observe that FD 1 is violated and FD 2 is satisfied. The results of the left-hand side and the right-hand side of Eqv. 7.118

differ. In case of R_2^{ugly} , we added an explicit key column (k_2), which can serve as its TID. We observe that FD 1 is satisfied, but FD 2 is violated. Again, the results of the left-hand side and the right-hand side of Eqv. 7.118 differ. As an exercise, the reader should apply Eqvs. 7.109 to 7.115 to the examples.

Yan and Larson [946] also give two extended equivalences, which allow an additional projection, either duplicate preserving or eliminating on top of the right-hand side of Eqv. 7.118. Obviously, this transformation is valid for all equivalences. With $C \subseteq G \cup \mathcal{A}(F)$, we therefore get

$$\Pi_C(\Gamma_{G;F}(e_1 \bowtie_q e_2)) \equiv \Pi_C(\Gamma_{G_1^+;F}(e_1) \bowtie_q e_2), \quad (7.119)$$

$$\Pi_C^D(\Gamma_{G;F}(e_1 \bowtie_q e_2)) \equiv \Pi_C^D(\Gamma_{G_1^+;F}(e_1) \bowtie_q e_2), \quad (7.120)$$

which hold if FD 1 and FD 2 hold.

After having seen the problems which can occur if we skip the top-level grouping, let us now prove the following equivalences, which result from Eqvs. 7.109 to 7.117 by eliminating the top-level grouping. Let $C = G \cup \mathcal{A}(F)$. Without the necessary conditions, which will be discussed afterwards, we have the following equivalences:

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{(\widehat{F_2 \otimes c_1}) \circ \widehat{F_1^2}}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2)), \quad (7.121)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{(\widehat{F_1 \otimes c_2}) \circ \widehat{F_2^2}}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))), \quad (7.122)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\Gamma_{G_1^+;F}(e_1) \bowtie_q e_2) \quad (7.123)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(e_1 \bowtie_q \Gamma_{G_2^+;F}(e_2)) \quad (7.124)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_2 \otimes c_1}}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q e_2)), \quad (7.125)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1 \otimes c_2}}(e_1 \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2))), \quad (7.126)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1^2 \otimes c_2}}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2))), \quad (7.127)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_2^2 \otimes c_1}}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1}(e_2))), \quad (7.128)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1^2 \otimes c_2} \circ \widehat{F_2^2 \otimes c_1}}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))) \quad (7.129)$$

We can prove Eqv. 7.122 by eliminating the top-most grouping operator on the right-hand side of Eqv. 7.110 via an application of Eqv. 7.16 followed by an application of remove-group-by (Eqv. 7.102):

$$\begin{aligned} & \Gamma_{G;F}(e_1 \bowtie_q e_2) \\ \equiv^{7.110} & \Gamma_{G;(\widehat{F_1 \otimes c_2}) \circ \widehat{F_2^2}}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)) \\ \equiv^{7.16} & \Pi_C(\Gamma_{G_1, G_2^+;(\widehat{F_1 \otimes c_2}) \circ \widehat{F_2^2}}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))) \\ \equiv^{7.102} & \Pi_C(\chi_{(\widehat{F_1 \otimes c_2}) \circ \widehat{F_2^2}}(\\ & e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))) \end{aligned}$$

Let us now come to the conditions attached to the equivalences. For our discussion, we denote by I the join with its (grouped) arguments, i.e.,

$$I = e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2).$$

The precondition of Eqv. 7.16 requires $G \rightarrow G_1, G_2^+$ to hold. Thus, a grouping by G_1, G_2^+ is not finer grained than a grouping by G . We still have to make sure that the precondition required by Eqv. 7.102 holds. In our context, the precondition is that $G_1, G_2^+ \rightarrow \text{TID}(I)$ holds in I , or, equivalently, $\Pi_{G_1, G_2^+}(I)$ is duplicate-free. Clearly, $\Pi_{G_2^+}(\Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2))$ is duplicate-free. It follows that $\Pi_{G_1, G_2^+}(I)$ is duplicate-free if $G_1, G_2^+ \rightarrow \text{TID}(e_1)$ holds in I . Summarizing, Eqv. 7.122 holds if $G \rightarrow G^+$ and $G_1, G_2^+ \rightarrow \text{TID}(e_1)$ both hold in I .

Eqv. 7.122 can be further simplified. If F_1 is empty, some simplification yields Eqv. 7.124:

$$\begin{aligned} & \Gamma_{G; F}(e_1 \bowtie_q e_2) \\ \equiv^{7.122} & \Pi_C(\chi_{(\widehat{F_1 \otimes c_2}) \circ F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2))) \\ \equiv & \Pi_C(e_1 \bowtie_q \Gamma_{G_2^+; F}(e_2)). \end{aligned}$$

By symmetry, Eqvs. 7.121 and 7.123 hold. Since Eqvs. 7.125 and 7.126 are also simplifications of Eqvs. 7.121 and 7.122, they can be proven similarly.

Let us turn to Eqv. 7.129. Since

$$\Pi_{G_1^+}(\Gamma_{G_1^+; F_1^1 \circ (c_1: \text{count}(*))}(e_1))$$

and

$$\Pi_{G_2^+}(\Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2))$$

are duplicate-free, Eqv. 7.129 holds if $G \rightarrow G^+$. Eqvs. 7.127 and 7.128 follow by simplifications from Eqv. 7.129, if F_1 or F_2 is empty.

7.11.3 Left Outerjoin

Consider a left outerjoin followed by a grouping operator. The goal is to push down the grouping operator into the arguments of the outerjoin. In order to do so, we will first mirror (and apply) Eqv. 7.109. The definition of the left outerjoin gives us

$$\Gamma_{G; F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G; F}((e_1 \bowtie_q e_2) \cup ((e_1 \bowtie_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\})).$$

As before, we define $J_i = \mathcal{F}(q) \cap \mathcal{A}(e_i)$, $J = J_1 \cup J_2$, $G_i = G \cap \mathcal{A}(e_i)$, $G_i^+ = G_i \cup J_i$, $G^+ = G \cup J$. We further demand that F is a splittable and decomposable aggregation vector. Define $C = G \cup \mathcal{A}(F)$. We also define abbreviations for some subexpressions:

$$\begin{aligned} E_j &= (e_1 \bowtie_q e_2), \\ E_a &= ((e_1 \bowtie_q e_2) \times \{\perp_{\mathcal{A}(e_2)}\}), \\ E_\perp &= \{\perp_{\mathcal{A}(e_2)}\}. \end{aligned}$$

We could consider two cases to push down the grouping operator into the arguments of the outerjoin. Case 1 requires $\Pi_G(E_j) \cap \Pi_G(E_a) = \emptyset$, and case 2 requires $\Pi_G(E_j) \cap \Pi_G(E_a) \neq \emptyset$. The former condition is fulfilled, e.g., if $G \rightarrow J_1$.

Then, we can apply Eqv. 7.26 in case 1 and Eqv. 7.27 in case 2. Since Eqv. 7.27 also holds if $\Pi_G(E_j) \cap \Pi_G(E_a) = \emptyset$, it suffices to apply it. As an exercise, the reader should consider case 1 explicitly.

Eqv. 7.27 gives us

$$\begin{aligned} & \Gamma_{G;F_1,F_2}((e_1 \bowtie_q e_2) \cup ((e_1 \triangleright_q e_2) \times E_\perp)) \\ \equiv & \Gamma_{G;F_1^2,F_2^2}(\Gamma_{G;F_1^1,F_2^1}(e_1 \bowtie_q e_2) \cup \Gamma_{G;F_1^1,F_2^1}((e_1 \triangleright_q e_2) \times E_\perp)) \end{aligned}$$

where we expanded F to F_1, F_2 . Applying Eqv. 7.109 to the left branch of the union gives us

$$\Gamma_{G;F_1^1,F_2^1}(e_1 \bowtie_q e_2) \equiv \Pi_C(\Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2))$$

Applying Eqv. 7.109 and then Eqv. 7.23 to the right branch of the union gives us:

$$\begin{aligned} & \Gamma_{G;F_1^1,F_2^1}((e_1 \triangleright_q e_2) \times E_\perp) \\ \equiv & \Pi_C(\Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1 \triangleright_q e_2) \times E_\perp)) \\ \equiv & \Pi_C(\Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}((\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \triangleright_q e_2) \times E_\perp)) \end{aligned}$$

Putting these two things together yields

$$\begin{aligned} & \Gamma_{G;F}(e_1 \bowtie_q e_2) \\ \equiv & \Gamma_{G;F_1^2,F_2^2}(\Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \\ & \cup \\ & \Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}((\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \triangleright_q e_2) \times E_\perp)) \\ \equiv & \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \\ & \cup \\ & ((\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \triangleright_q e_2) \times E_\perp)) \\ \equiv & \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \end{aligned}$$

where in the first step we could omit the Π_C due to the subsequent grouping. The second step pulls the two $\Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}$ operators out of the two union branches and merges them with the outer $\Gamma_{G;F_1^2,F_2^2}$. This is possible due to the properties of the aggregation vectors involved and the fact that both group on the same set G of grouping attributes.

Eager/Lazy Groupby-Count

Summarizing, we have the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2), \quad (7.130)$$

which holds if F is splittable into F_1 and F_2 with respect to e_1 and e_2 , and F_i is decomposable into F_i^1 and F_i^2 .

The companion of Eqv. 7.110 is

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q^{F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)). \quad (7.131)$$

To prove it, we start with

$$\begin{aligned}\Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;F}((e_1 \bowtie_q e_2) \cup ((e_1 \triangleright_q e_2) \times E_\perp)) \\ &\equiv \Gamma_{G;F_1^2, F_2^2}(\Gamma_{G;F_1^1, F_2^1}(e_1 \bowtie_q e_2) \cup \Gamma_{G;F_1^1, F_2^1}((e_1 \triangleright_q e_2) \times E_\perp)),\end{aligned}$$

where $E_\perp = \{\perp_{\mathcal{A}(e_2)}\}$. Applying Eqv. 7.110 to the left argument of the union results in

$$\Gamma_{G;F_1^1, F_2^1}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2)).$$

Applying Eqv. 7.110 to the right argument of the union yields

$$\begin{aligned}\Gamma_{G;F_1^1, F_2^1}((e_1 \triangleright_q e_2) \times E_\perp) &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^2}((e_1 \triangleright_q e_2) \times \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(E_\perp)) \\ &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^2}((e_1 \triangleright_q \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2)) \\ &\quad \times \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(E_\perp)) \\ &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^2}((e_1 \triangleright_q \Gamma_{G_2^+; F_2^1 \circ (c_2: \text{count}(*))}(e_2)) \\ &\quad \times \Pi_{G_2^+ \cup \mathcal{A}(F) \cup \{c_2\}}(\chi_{F_2^1(\emptyset), c_2:1}(E_\perp))),\end{aligned}$$

and the claim follows.

Eager/Lazy Group-by

If $F_2 = ()$, Eqv. 7.130 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_1^2}(\Gamma_{G_1^+; F_1^1}(e_1) \bowtie_q e_2) \quad (7.132)$$

This equivalence holds if F_1 is decomposable into F_1^1 and F_1^2 .

If $F_1 = ()$, Eqv. 7.131 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_2^2}(e_1 \bowtie_q^{F_2^1(\emptyset)} \Gamma_{G_2^+; F_2^1}(e_2)), \quad (7.133)$$

which holds if F_2 is decomposable.

Eager/Lazy Count

If $F_1 = ()$, Eqv. 7.130 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1)}(\Gamma_{G_1^+; (c_1: \text{count}(*))}(e_1) \bowtie_q e_2). \quad (7.134)$$

This equivalence holds if F_2 is decomposable into F_2^1 and F_2^2 .

If $F_2 = ()$, Eqv. 7.131 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2)}(e_1 \bowtie_q^{c_2:1} \Gamma_{G_2^+; c_2: \text{count}(*)}(e_2)). \quad (7.135)$$

Double Eager/Lazy

For the next equivalence assume $F_2 = ()$. We would like to derive an equivalence similar to Eqv. 7.115. Here it is:

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{c_2:1} \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)), \quad (7.136)$$

which holds if F_1 is decomposable into F_1^1 and F_1^2 .

If $F_1 = ()$ and F_2 is decomposable into F_2^1 and F_2^2 , the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q^{F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2)) \quad (7.137)$$

holds.

Eager/Lazy Split

The companion of Eqv. 7.117 for the left outerjoin is

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;(F_1^2 \otimes c_2) \circ (F_2^2 \otimes c_1)}(\\ &\quad \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)), \end{aligned} \quad (7.138)$$

which holds if F_1 is decomposable into F_1^1 and F_1^2 , and F_2 is decomposable into F_2^1 and F_2^2 .

Eliminating the top grouping

We can eliminate the top grouping in the above equivalences for the outerjoin by the same arguments as for the join. The resulting equivalences are

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{(F_2 \otimes c_1) \circ F_1^2}}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2)), \quad (7.139)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{(F_1 \otimes c_2) \circ F_2^2}}(e_1 \bowtie_q^{F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))), \quad (7.140)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\Gamma_{G_1^+;F}(e_1) \bowtie_q e_2), \quad (7.141)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(e_1 \bowtie_q^{F(\emptyset)} \Gamma_{G_2^+;F}(e_2)), \quad (7.142)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_2 \otimes c_1}}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q e_2)), \quad (7.143)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1 \otimes c_2}}(e_1 \bowtie_q^{c_2:1} \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2))), \quad (7.144)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1^2 \otimes c_2}}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{c_2:1} \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2))), \quad (7.145)$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_2^2 \otimes c_1}}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q^{F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2))), \quad (7.146)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Pi_C(\chi_{\widehat{G;F_1^2 \otimes c_2 \circ F_2^2 \otimes c_1}}(\\ &\quad \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))). \end{aligned} \quad (7.147)$$

These equivalences hold if in addition to the according conditions concerning splittability, decomposability, and emptyness, the following functional dependencies hold:

$$\begin{aligned}
\text{Eqv. 7.139} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.140} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.141} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.142} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.143} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.144} \quad & G \rightarrow G_1, G_2^+, G_1, G_2^+ \rightarrow \text{TID}(e_1), \\
\text{Eqv. 7.145} \quad & G \rightarrow G_1, G_2^+, \\
\text{Eqv. 7.146} \quad & G \rightarrow G_1, G_2^+, \\
\text{Eqv. 7.147} \quad & G \rightarrow G_1, G_2^+.
\end{aligned}$$

7.11.4 Left Outerjoin with Default

Main Equivalences

Let us next consider the outerjoin with default. For a set of attributes $\{d_1, \dots, d_l\} \subseteq \mathcal{A}(e_2)$ of e_2 , constants $c_1, \dots, c_l$ and a vector

$$D = d_1 : c_1, \dots, d_l : c_l,$$

we now consider the expression

$$\Gamma_{G;F}(e_1 \bowtie_q^D e_2).$$

If we take a close look at the proof of Eqv. 7.130 and think of $E_\perp$ as being defined as

$$E_\perp := (\perp_{\mathcal{A}(e_2) \setminus \mathcal{A}(D)} \times \{D\}),$$

we see that the proof remains valid. Thus, we have the following equivalences:

$$\Gamma_{G;F}(e_1 \bowtie_q^D e_2) \equiv \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1) \bowtie_q e_2), \quad (7.148)$$

$$\Gamma_{G;F}(e_1 \bowtie_q^D e_2) \equiv \Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q^{D, F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+; F_2^1 \circ (c_2 : \text{count}(*))}(e_2)), \quad (7.149)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;F_1^2}(\Gamma_{G_1^+; F_1^1}(e_1 \bowtie_q^D e_2)) \\ &\quad \text{if } F_2 \text{ is empty,} \end{aligned} \quad (7.150)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;F_2^2}(e_1 \bowtie_q^{D, F_2^1(\emptyset)} \Gamma_{G_2^+; F_2^1}(e_2)) \\ &\quad \text{if } F_1 \text{ is empty,} \end{aligned} \quad (7.151)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;(F_2 \otimes c_1)}(\Gamma_{G_1^+; (c_1 : \text{count}(*))}(e_1) \bowtie_q^D e_2) \\ &\quad \text{if } F_1 \text{ is empty,} \end{aligned} \quad (7.152)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;(F_1 \otimes c_2)}(e_1 \bowtie_q^{D, c_2:1} \Gamma_{G_2^+; c_2 : \text{count}(*)}(e_2)) \\ &\quad \text{if } F_2 \text{ is empty,} \end{aligned} \quad (7.153)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;(F_1 \otimes c_2)}(\Gamma_{G_1^+; F_1^1}(e_1) \bowtie_q^{D, c_2:1} \Gamma_{G_2^+; c_2 : \text{count}(*)}(e_2)) \\ &\quad \text{if } F_2 \text{ is empty,} \end{aligned} \quad (7.154)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+; c_1 : \text{count}(*)}(e_1) \bowtie_q^{D, F_2^1(\emptyset)} \Gamma_{G_2^+; F_2^1}(e_2)) \\ &\quad \text{if } F_1 \text{ is empty,} \end{aligned} \quad (7.155)$$

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q^D e_2) &\equiv \Gamma_{G;(F_1 \otimes c_2) \circ (F_2 \otimes c_1)}(\\ &\quad \Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1) \bowtie_q^{D, F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+; F_2^1 \circ (c_2 : \text{count}(*))}(e_2)). \end{aligned} \quad (7.156)$$

These equivalences hold under the same conditions as their corresponding equivalences for the outerjoin with no default.

Eliminating the top grouping

This can be performed analogously to the left outerjoin without default.

7.11.5 Full Outerjoin

The next expression we consider is

$$\Gamma_{G;F}(e_1 \bowtie_q e_2).$$

In order to deal with this expression, we will need the full outerjoin with defaults for both sides. Define $E_{1\perp} = \{\perp_{\mathcal{A}(e_1)}\}$ and let us start by observing

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;F}((e_1 \bowtie_q e_2) \cup ((e_2 \bowtie_q e_1) \times E_{1\perp})) \\ &\equiv \Gamma_{G;F^2}(\Gamma_{G;F^1}(e_1 \bowtie_q e_2) \cup \Gamma_{G;F^1}((e_2 \bowtie_q e_1) \times E_{1\perp})). \end{aligned}$$

Applying Eqv. 7.130 to the left-hand side of the union results in

$$\begin{aligned} \Gamma_{G;F^1}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;(F_2^1 \otimes c_1) \circ F_1^{1,2}}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \\ &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^{1,2}}(e_1 \bowtie_q^{F_2^{1,1}(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)). \end{aligned}$$

Applying Eqvs. 7.109 and 7.23 to the right-hand side of the union yields

$$\begin{aligned} \Gamma_{G;F^1}((e_2 \bowtie_q e_1) \times E_{1\perp}) &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^{1,2}}(\Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2 \bowtie_q e_1) \times E_{1\perp}) \\ &\equiv \Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^{1,2}}((\Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2) \bowtie_q e_1) \times E_{1\perp}). \end{aligned}$$

Putting these things together, we have

$$\begin{aligned} &\Gamma_{G;F}(e_1 \bowtie_q e_2) \\ \equiv &\Gamma_{G;F^2}(\Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^{1,2}}(e_1 \bowtie_q^{F_2^{1,1}(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)) \\ &\cup \\ &(\Gamma_{G;(F_1^1 \otimes c_2) \circ F_2^{1,2}}((\Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2) \bowtie_q e_1) \times E_{1\perp}))) \\ \equiv &\Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(\Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2) \bowtie_q^{F_2^{1,1}(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)) \\ &\cup \\ &((\Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2) \bowtie_q e_1) \times E_{1\perp})) \\ \equiv &\Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q^{-;F_2^{1,1}(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)). \end{aligned}$$

Eager/Lazy Groupby-Count

Due to the commutativity of the full outerjoin, we thus have

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_1^1(\emptyset), c_1:1;-} e_2) \quad (7.157)$$

if F is splittable and F_1 is decomposable into F_1^1 and F_1^2 .

If F is splittable and F_2 is decomposable into F_2^1 and F_2^2 ,

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q^{-;F_2^1(\emptyset), c_2:1} \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)) \quad (7.158)$$

holds.

Eager/Lazy Group-by

If F_2 is empty, then Eqv. 7.157 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_1^2}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{F_1^1(\emptyset);-} e_2). \quad (7.159)$$

This equivalence holds if F_1 is decomposable into F_1^1 and F_1^2 . If F_1 is empty, then Eqv. 7.158 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_2^2}(e_1 \bowtie_q^{-;F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2)). \quad (7.160)$$

This equivalence holds if F_2 is decomposable into F_2^1 and F_2^2 .

Eager/Lazy Count

If F_1 is empty, then Eqv. 7.157 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1)}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q^{c_1:1;-} e_2). \quad (7.161)$$

If F_2 is empty, then Eqv. 7.158 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2)}(e_1 \bowtie_q^{-;c_2:1} \Gamma_{G_2^+;(c_2:\text{count}(*))}(e_2)). \quad (7.162)$$

Double Eager/Lazy

If F_2 is empty, the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{F_1^1(\emptyset);c_2:1} \Gamma_{G_2^+;(c_2:\text{count}(*))}(e_2)) \quad (7.163)$$

holds if F_1 is decomposable into F_1^1 and F_1^2 . If F_1 is empty, the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q^{c_1:1;F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2)) \quad (7.164)$$

holds if F_2 is decomposable into F_2^1 and F_2^2 .

Proof: If F_2 is empty, then

$$\begin{aligned}\Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv_{\text{Eqv. 7.159}} \Gamma_{G;F_1^2}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{F_1^1(\emptyset);-} e_2) \\ &\equiv_{\text{Eqv. 7.162}} \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{F_1^1(\emptyset);c_2:1} \Gamma_{G_2^+;(c_2:\text{count}(*))}(e_2)).\end{aligned}$$

If F_1 is empty, then

$$\begin{aligned}\Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv_{\text{Eqv. 7.160}} \Gamma_{G;F_2^2}(e_1 \bowtie_q^{-;F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2)) \\ &\equiv_{\text{Eqv. 7.161}} \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q^{c_1:1;F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1}(e_2)).\end{aligned}$$

□

Eager/Lazy Split

If F is splittable and decomposable, then

$$\begin{aligned}\Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;(F_1^2 \otimes c_2) \circ (F_2^2 \otimes c_1)}(\\ &\quad \Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_1^{1,1}(\emptyset),c_1:1;F_2^{1,1}(\emptyset),c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2)).\end{aligned}\tag{7.165}$$

Proof:

$$\begin{aligned}&\Gamma_{G;F}(e_1 \bowtie_q e_2) \\ &\equiv_{\text{Eqv. 7.157}} \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_1^1(\emptyset),c_1:1;-} e_2)) \\ &\equiv_{\text{Eqv. 7.158}} \Gamma_{G;(F_1^2 \otimes c_2) \circ (F_2^2 \otimes c_1)}(\Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_1^1(\emptyset),c_1:1;F_2^1(\emptyset),c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2))\end{aligned}$$

□

Eliminating the top grouping

Under the same conditions under which their counterparts are valid, the following equivalences hold for the full outerjoin:

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{(F_2 \otimes c_1)} \circ F_1^2}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q^{F_1^1(\emptyset),c_1:1;-} e_2)),\tag{7.166}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{(F_1 \otimes c_2)} \circ F_2^2}(e_1 \bowtie_q^{-;F_2^1(\emptyset),c_2:1} \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2))),\tag{7.167}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\Gamma_{G_1^+;F}(e_1) \bowtie_q^{F(\emptyset);-} e_2),\tag{7.168}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(e_1 \bowtie_q^{-;F(\emptyset)} \Gamma_{G_2^+;F}(e_2)),\tag{7.169}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_2 \otimes c_1}}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q^{c_1:1;-} e_2)),\tag{7.170}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1 \otimes c_2}}(e_1 \bowtie_q^{-;c_2:1} \Gamma_{G_2^+;(c_2:\text{count}(*))}(e_2))),\tag{7.171}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{F_1^2 \otimes c_2}}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q^{F_1^1(\emptyset);c_2:1} \Gamma_{G_2^+;(c_2:\text{count}(*))}(e_2))),\tag{7.172}$$

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Pi_C(\chi_{\widehat{G;F_2^2 \otimes c_1}}(\Gamma_{G_1^+;(c_1:\text{count}(*))}(e_1) \bowtie_q^{c_1:1;F_2^1(\emptyset)} \Gamma_{G_2^+;F_2^1})),\tag{7.173}$$

$$\begin{aligned}\Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Pi_C(\chi_{\widehat{G;F_1^2 \otimes c_2} \circ \widehat{F_2^2 \otimes c_1}}(\\ &\quad \Gamma_{G_1^+;F_1^{1,1} \circ (c_1:\text{count}(*))}(e_1) \\ &\quad \bowtie_q^{F_1^{1,1}(\emptyset),c_1:1;F_2^{1,1}(\emptyset),c_2:1} \Gamma_{G_2^+;F_2^{1,1} \circ (c_2:\text{count}(*))}(e_2))).\end{aligned}\tag{7.174}$$

7.11.6 D-Join

Next, let us turn to the d-join. The outline of this subsection mirrors the one for regular joins. Indeed, all equivalences that hold for regular joins will also hold for d-joins.

Eager/Lazy Groupby-Count

The equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q e_2) \quad (7.175)$$

holds if F_1 is splittable into F_1 and F_2 , and F_1 is decomposable into F_1^1 and F_1^2 .

The equivalence

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2) \circ F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)) \quad (7.176)$$

holds if F_2 is splittable into F_1 and F_2 , and F_2 is decomposable into F_2^1 and F_2^2 .

Eager/Lazy Group-by

If F_2 is empty, that is $F_2 = ()$, Eqv. 7.175 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_1^2}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q e_2). \quad (7.177)$$

This equivalence holds if F_1 is splittable and decomposable into F_1^1 and F_1^2 .

If F_1 is empty, Eqv. 7.176 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;F_2^2}(e_1 \bowtie_q \Gamma_{G_2^+;F_2^1}(e_2)). \quad (7.178)$$

This equivalence holds if F_2 is splittable and decomposable into F_2^1 and F_2^2 .

Eager/Lazy Count

If F_1 is empty, Eqv. 7.175 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2 \otimes c_1)}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q e_2). \quad (7.179)$$

If F_2 is empty, then Eqv. 7.176 simplifies to

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1 \otimes c_2)}(e_1 \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)). \quad (7.180)$$

Double Eager/Lazy

If F_2 is empty

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_1^2 \otimes c_2)}(\Gamma_{G_1^+;F_1^1}(e_1) \bowtie_q \Gamma_{G_2^+;c_2:\text{count}(*)}(e_2)), \quad (7.181)$$

if F_1 is splittable and decomposable into F_1^1 and F_1^2 .

If F_1 is empty

$$\Gamma_{G;F}(e_1 \bowtie_q e_2) \equiv \Gamma_{G;(F_2^2 \otimes c_1)}(\Gamma_{G_1^+;c_1:\text{count}(*)}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1}(e_2)) \quad (7.182)$$

holds if F_2 is splittable decomposable into F_2^1 and F_2^2 .

Eager/Lazy Split

Applying Eqv. 7.175 and then Eqv. 7.176 results in the equivalence

$$\begin{aligned} \Gamma_{G;F}(e_1 \bowtie_q e_2) &\equiv \Gamma_{G;(F_1^2 \otimes c_2) \circ (F_2^2 \otimes c_1)}(\\ &\quad \Gamma_{G_1^+;F_1^1 \circ (c_1:\text{count}(*))}(e_1) \bowtie_q \Gamma_{G_2^+;F_2^1 \circ (c_2:\text{count}(*))}(e_2)), \end{aligned} \quad (7.183)$$

which holds if F is splittable into F_1 and F_2 , F_1 is decomposable into F_1^1 and F_1^2 , and F_2 is decomposable into F_2^1 and F_2^2 .

Eliminating the top grouping

The top grouping can be eliminated under the conditions for the regular joins.

7.11.7 Groupjoin**Simple Facts about the Groupjoin**

Last in this section, we consider the groupjoin and thus the expressions of the form

$$\Gamma_{G;F}(e_1 \bowtie_{q;\hat{F}} e_2).$$

Before we start, we discuss some equivalences for the groupjoin. Since σ and χ are linear and $\bowtie$ is linear in its left argument, it is easy to show that

$$\sigma_p(e_1 \bowtie_{G_1\theta G_2;g:f} e_2) \equiv \sigma_p(e_1) \bowtie_{G_1\theta G_2;g:f} e_2, \quad (7.184)$$

$$\chi_{a:e}(e_1 \bowtie_{G_1\theta G_2;g:f} e_2) \equiv \chi_{a:e}(e_1) \bowtie_{G_1\theta G_2;g:f} e_2. \quad (7.185)$$

Then, we note that unary grouping can be expressed with the help of the groupjoin.

$$\begin{aligned} \Gamma_{\theta G;f}(e) &\equiv \Pi_C(\rho_{\mathcal{A}(e_1)' \leftarrow \mathcal{A}(e_1)}(\rho_{\mathcal{A}(e_1) \leftarrow \mathcal{A}(e_1)' }(\Pi_G^D(e)) \bowtie_{G'\theta G;f} e)), \\ \Gamma_{\theta G;g;F}(e) &\equiv \Pi_C(\rho_{\mathcal{A}(e_1)' \leftarrow \mathcal{A}(e_1)}(\rho_{\mathcal{A}(e_1) \leftarrow \mathcal{A}(e_1)' }(\Pi_G^D(e)) \bowtie_{G'\theta G;g;F} e)), \\ \Gamma_{\theta G;F}(e) &\equiv \Pi_C(\rho_{\mathcal{A}(e_1)' \leftarrow \mathcal{A}(e_1)}(\rho_{\mathcal{A}(e_1) \leftarrow \mathcal{A}(e_1)' }(\Pi_G^D(e)) \bowtie_{G'\theta G;F} e)), \end{aligned}$$

where C on the right-hand side of an equivalence contains all attributes provided in the result of the left-hand side of the equivalence.

The groupjoin itself can be expressed with the help of unary grouping and a left outerjoin:

$$e_1 \bowtie_{G_1\theta G_2;f} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{f(\emptyset)} \Gamma_{\theta G_2;f}(e_2)), \quad (7.186)$$

$$e_1 \bowtie_{G_1\theta G_2;g;F} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{F(\emptyset)} \Gamma_{\theta G_2;g;F}(e_2)), \quad (7.187)$$

$$e_1 \bowtie_{G_1\theta G_2;F} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{F(\emptyset)} \Gamma_{\theta G_2;F}(e_2)), \quad (7.188)$$

where $C = G \cup \mathcal{A}(F)$. We need to attach a small correction to these equivalences. Consider for example Eqv. 7.188. It only holds if $F(\emptyset) = F(\{\perp_{\mathcal{A}(e_2)}\})$. This is true in SQL-92 for min, max, sum, count(a), but not count(*). More precisely, count(*) yields 0 if the input is the empty set, and 1 if it is applied to some null-tuple. Thus, the right-hand side yields 0 for empty groups, whereas it should

produce 1. Obviously, this problem can easily be fixed in the left outerjoin by using the correct default value of 1 for all attributes containing the result of a count^* . Hence, we define $\text{count}^*(\emptyset) := 1$ in the context of default values for outerjoins. Thus, the above equivalences now read

$$e_1 \bowtie_{G_1 \theta G_2; f} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{f(\{\perp_{\mathcal{A}(e_2)}\})} \Gamma_{\theta G_2; f}(e_2)), \quad (7.189)$$

$$e_1 \bowtie_{G_1 \theta G_2; g; F} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{F(\{\perp_{\mathcal{A}(e_2)}\})} \Gamma_{\theta G_2; g; F}(e_2)), \quad (7.190)$$

$$e_1 \bowtie_{G_1 \theta G_2; F} e_2 \equiv \Pi_C(e_1 \bowtie_{G_1=G_2}^{F(\{\perp_{\mathcal{A}(e_2)}\})} \Gamma_{\theta G_2; F}(e_2)). \quad (7.191)$$

Apart from this detail, these equivalences follow directly from the definition of the groupjoin.

For the regular join, we can apply a selection to get rid of tuples not finding a join partner by counting the number of join partners. This leads to the following equivalences:

$$\begin{aligned} \Pi_C(e_1 \bowtie_{G_1=G_2} \Gamma_{\theta G_2; g; F}(e_2)) &\equiv \sigma_{c_2 > 0}(e_1 \bowtie_{G_1 \theta G_2; g; F \circ (c_2:|g|)} e_2), \\ \Pi_C(e_1 \bowtie_{G_1=G_2} \Gamma_{\theta G_2; F}(e_2)) &\equiv \sigma_{c_2 > 0}(e_1 \bowtie_{G_1 \theta G_2; F \circ (c_2:\text{count}^*)} e_2), \\ \Pi_C(e_1 \bowtie_{G_1=G_2} \Gamma_{G_2; g; F}(e_2)) &\equiv \sigma_{c_2 > 0}(e_1 \bowtie_{G_1=G_2; g; F \circ (c_2:|g|)} e_2), \\ \Pi_C(e_1 \bowtie_{G_1=G_2} \Gamma_{G_2; F}(e_2)) &\equiv \sigma_{c_2 > 0}(e_1 \bowtie_{G_1=G_2; F \circ (c_2:\text{count}^*)} e_2). \end{aligned}$$

Pushing Grouping into the Groupjoin

The general assumptions for the next three equivalences are as follows. Let F and $\bar{F}$ be two aggregation vectors. Let G be a set of grouping attributes such that $G \subseteq \mathcal{A}(e_1) \cup \mathcal{A}(F)$. Let J_1 and J_2 be non-empty sets of attributes with $J_1 \subseteq \mathcal{A}(e_1)$ and $J_2 \subseteq \mathcal{A}(F)$. Define $G_1 = G \cap \mathcal{A}(e_1)$ and $G_1^+ = G_1 \cup J_1$.

Assume F is splittable into F_1 and F_2 , and F_1 is decomposable into F_1^1 and F_1^2 . Then

$$\Gamma_{G; F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Gamma_{G; (F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+; F_1^1 \circ (c_1:\text{count}^*)}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2). \quad (7.192)$$

Note that F_2 can only use attributes from $\bar{F}$. Before we state the proof, note that two simplifications are derivable: If F_2 is empty, then

$$\Gamma_{G; F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Gamma_{G; F_1^2}(\Gamma_{G_1^+; F_1^1}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \quad (7.193)$$

holds if F_1 is decomposable into F_1^1 and F_1^2 . If F_1 is empty, then

$$\Gamma_{G; F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Gamma_{G; (F_2 \otimes c_1)}(\Gamma_{G_1^+; (c_1:\text{count}^*)}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \quad (7.194)$$

holds if F_2 is decomposable into F_2^1 and F_2^2 .

Trying to push the outer unary grouping into the right argument of the groupjoin does not make sense, since the right-hand side of a groupjoin will already be grouped by the groupjoin itself and a double grouping is not beneficial. However, it could be done.

Proof of Eqv. 7.192:

$$\begin{aligned}
\Gamma_{G;F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) &\stackrel{7.188}{=} \Gamma_{G;F}(e_1 \bowtie_{J_1=J_2}^{\bar{F}(\emptyset)} \Gamma_{\theta J_2, \bar{F}}(e_2)) \\
&\stackrel{7.148}{=} \Gamma_{G; (F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1) \bowtie_{J_1=J_2}^{\bar{F}(\emptyset)} \Gamma_{\theta J_2, \bar{F}}(e_2))) \\
&\stackrel{7.188}{=} \Gamma_{G; (F_2 \otimes c_1) \circ F_1^2}(\Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2))
\end{aligned}$$

□

Eliminating the top grouping

Since $\Pi_{G_1^+}(\Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1))$ is duplicate-free, we can apply Eqv. 7.102 to Eqv. 7.192 if $G \rightarrow G^+$ holds. With $C = G \cup \mathcal{A}(F)$, this gives us

$$\Gamma_{G;F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Pi_C(\chi_{\widehat{F_2 \otimes c_1 \circ F_1^2}}(\Gamma_{G_1^+; F_1^1 \circ (c_1 : \text{count}(*))}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2)). \quad (7.195)$$

Simplifications result in the following equivalences, which also hold if $G \rightarrow G^+$ holds:

$$\Gamma_{G;F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Pi_C(\Gamma_{G_1^+; F}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2), \quad (7.196)$$

$$\Gamma_{G;F}(e_1 \bowtie_{J_1 \theta J_2; \bar{F}} e_2) \equiv \Pi_C(\chi_{\widehat{F_2 \otimes c_1}}(\Gamma_{G_1^+; (c_1 : \text{count}(*))}(e_1) \bowtie_{J_1 \theta J_2; \bar{F}} e_2)). \quad (7.197)$$

The first equivalence additionally needs that F_2 is empty, the second that F_1 is empty.

Important Operator Conversions

We now introduce two equivalences which allow us to replace a sequence of a grouping operator and a left-outerjoin/join by a single groupjoin [621].

For $i = 1, 2$, let e_i be algebraic expressions and $J_1 = J_2$ be a join predicate, such that for the join attributes $J_i \subseteq \mathcal{A}(e_i)$ holds. For a set of grouping attributes G , define $G_i = G \cap \mathcal{A}(e_i)$ and $G_i^+ = G_i \cup J_i$. Further, let F be a splittable and decomposable aggregation vector with $\mathcal{F}(F) \subseteq \mathcal{A}(e_2)$. We denote by C the set of attributes occurring in the result, i.e., $C = G \cup \mathcal{A}(F)$. Then, the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_{J_1=J_2} e_2) \equiv \Pi_C(e_1 \bowtie_{J_1=J_2; F} e_2) \quad (7.198)$$

holds under the conditions that

1. $G \rightarrow G_2^+$ and $G_1, G_2^+ \rightarrow \text{TID}(e_1)$ hold in $e_1 \bowtie_{J_1=J_2} e_2$,
2. $J_2 \rightarrow G_2^+$ holds in e_2 ,
3. $\mathcal{F}(F) \subseteq \mathcal{A}(e_2)$, and
4. $F(\emptyset) = F(\{\perp_{\mathcal{A}(e_2)}\})$.

R_1	R_2	R_3		S		
a	a	a	b	c	d	e
1	1	1	1	1	8	1
	1	1	2	1	9	2

Figure 7.15: Example relations

We discuss these conditions to provide the intuition behind them. The two conditions under 1. stem from the main theorem of Yan and Larson in [946]. They assure that a grouping can be pushed into a regular join. In our context, the condition $G_1, G_2^+ \rightarrow \text{TID}(e_1)$ assures that no two tuples from e_1 belong to the same group. This is necessary since the groupjoin on the right-hand side provides exactly one output tuple for each input tuple of e_1 . The condition $G \rightarrow G_2^+$ implies that grouping by G_2^+ is not finer grained than grouping by G , which would lead to problems.

In case the second condition ($J_2 \rightarrow G_2^+$) is not fulfilled, we would have more groups on the left-hand side than on the right-hand side of our equivalence, which would violate it. This is easy to see if we add to G an evil attribute from e_2 , which is not functionally determined by J_2 . The importance of the functional dependencies is illustrated in the examples below.

The third condition ($\mathcal{F}(F) \subseteq \mathcal{A}(e_2)$) can actually be relaxed if we maintain a final map operator (see Eqvs. 7.122 and 7.140). The fourth condition follows from the discussion of Eqv. 7.188.

Eqv. 7.198 is important since it allows us to replace a unary grouping and a left outerjoin by a groupjoin. This is very beneficial in several scenarios. Consider just the one where all these operators have a hash-based implementation in a main-memory setting. Then, the left-hand side requires to build two hash tables, whereas the right-hand side requires to build only one. Further, no intermediate result tuples for the outerjoin have to be built.

The second equivalence replaces a sequence of a join and a grouping by a groupjoin. Given the notations of the previous subsection, the equivalence

$$\Gamma_{G;F}(e_1 \bowtie_{J_1=J_2} e_2) \equiv \Pi_C(\sigma_{c_2>0}(e_1 \bowtie_{J_1=J_2;F \circ (c_2:\text{count}(*))} e_2)) \quad (7.199)$$

holds under the conditions that

1. $G \rightarrow G_2^+$ and $G_1, G_2^+ \rightarrow \text{TID}(e_1)$ hold in $e_1 \bowtie_{J_1=J_2} e_2$
2. $J_2 \rightarrow G_2^+$ holds in e_2 , and
3. $\mathcal{F}(F) \subseteq \mathcal{A}(e_2)$.

The intuition behind these conditions is the same as for the previous equivalence. The fourth condition could be omitted, since empty groups are eliminated by the selection $\sigma_{c_2>0}$. Eqv. 7.199 is beneficial under similar circumstances as Eqv. 7.198.

Before we come to the proofs, let us have a look at some examples. Fig. 7.15 contains some relations. The results of some outerjoins ($R_i \bowtie_q S$) with two

$m_1 : R_1 \bowtie_{a=c} S$				$m_2 : R_2 \bowtie_{a=c} S$				$m_3 : R_3 \bowtie_{b=e} S$				
a	c	d	e	a	c	d	e	a	b	c	d	e
1	1	8	1	1	1	9	2	1	1	1	8	1
1	1	9	2	1	1	8	1	1	2	1	9	2
				1	1	9	2					

Figure 7.16: Join results

$l_1 : \Gamma_{a;\text{sum}(d)}(R_1 \bowtie_{a=c} S)$		
a	sum(d)	
1	17	
$l_2 : \Gamma_{a,e;\text{sum}(d)}(R_1 \bowtie_{a=c} S)$		
a	e	sum(d)
1	1	8
1	2	9
$l_3 : \Gamma_{a;\text{sum}(d)}(R_2 \bowtie_{a=c} S)$		
a	sum(d)	
1	34	
$l_4 : \Gamma_{a;\text{sum}(d)}(R_3 \bowtie_{b=e} S)$		
a	sum(d)	
1	17	

$r_1 : R_1 \bowtie_{a=c;\text{sum}(d)} S$		
a	sum(d)	
1	17	
$r_2 : R_1 \bowtie_{a=c;\text{sum}(d)} S$		
a	sum(d)	
1	17	
$r_3 : R_2 \bowtie_{a=c;\text{sum}(d)} S$		
a	sum(d)	
1	17	
1	17	
$r_4 : R_3 \bowtie_{b=e;\text{sum}(d)} S$		
a	b	sum(d)
1	1	8
1	2	9

Figure 7.17: Left- and right-hand sides

different join predicates are given in Fig. 7.16. Since all tuples in some R_i always find a join partner, the results of the outerjoins are the same as the corresponding join results. We are now interested in the functional dependencies occurring in the conditions of our main equivalences. Therefore, we discuss four example instances of Eqv. 7.199, where at most one of the functional dependencies is violated:

	$G \rightarrow G_2^+$	$G_1, G_2^+ \rightarrow \text{TID}(e_1)$	$J_2 \rightarrow G_2^+$
1	+	+	+
2	+	+	-
3	+	-	+
4	-	+	+

The according instances of the left-hand and right-hand side of Eqv. 7.199 are:

	LHS	RHS
1	$\Gamma_{a;\text{sum}(d)}(R_1 \bowtie_{a=c} S)$	$R_1 \bowtie_{a=c;\text{sum}(d)} S$
2	$\Gamma_{a,e;\text{sum}(d)}(R_1 \bowtie_{a=c} S)$	$R_1 \bowtie_{a=c;\text{sum}(d)} S$
3	$\Gamma_{a;\text{sum}(d)}(R_2 \bowtie_{a=c} S)$	$R_2 \bowtie_{a=c;\text{sum}(d)} S$
3	$\Gamma_{a;\text{sum}(d)}(R_3 \bowtie_{b=e} S)$	$R_3 \bowtie_{b=e;\text{sum}(d)} S$

The functional dependencies have to be checked on the join results given in Fig. 7.16. In order to help the reader to check the functional dependencies, we provide the following table holding the main attribute sets occurring in our main equivalences:

	G	G_1	G_2	J_2	G_2^+
1	$\{a\}$	$\{a\}$	$\emptyset$	$\{c\}$	$\{c\}$
2	$\{a, e\}$	$\{a\}$	$\{e\}$	$\{c\}$	$\{c, e\}$
3	$\{a\}$	$\{a\}$	$\emptyset$	$\{c\}$	$\{c\}$
4	$\{a\}$	$\{a\}$	$\emptyset$	$\{e\}$	$\{e\}$

Taking a look at Fig. 7.17, we see that both sides of the equivalence give the same result only if none of the functional dependencies is violated.

Proof of Eqv. 7.198 We now give the proof of Eqv. 7.198. We start with the right-hand side and transform it until we get the left-hand side:

$$\begin{aligned}
& \Pi_C(e_1 \bowtie_{J_1=J_2;F} e_2) \\
& \equiv^{7.188} \Pi_C(e_1 \bowtie_{J_1=J_2}^{F(\emptyset)} \Gamma_{J_2;F}(e_2)) \\
& \equiv^{7.14} \Pi_C(e_1 \bowtie_{J_1=J_2}^{F(\emptyset)} \Gamma_{G_2^+;F}(e_2)) \\
& \equiv^{7.142} \Gamma_{G;F}(e_1 \bowtie_{J_1=J_2} e_2).
\end{aligned}$$

The preconditions follow from collecting the preconditions of the different equivalences applied. $\square$

Proof of Eqv. 7.199 Eqv. 7.199 follows directly from Eqv. 7.198. An alternative is to modify the above proof by using Eqv. 7.192 instead of Eqv. 7.188 and Eqv. 7.122 instead of Eqv. 7.142.

Remark Often, we encounter expression of the form $\Gamma_{G;F}(e_1) \bowtie_{J_1=J_2} e_2$. If $G = J_1$, the hash table for the grouping can be reused by the groupjoin. Similarly, if $G \supseteq J_1$, any sorting produced to perform a sort-based grouping can be reused for a a sort-based groupjoin.

7.11.8 Intersection and Difference

There is not much we can do in terms of pushing a unary grouping operator down an intersection or set difference. We can only change an explicit bag representation into a multiplicity-based bag representation. This gives us the following two equivalences:

$$\Gamma_{G;F}(e_1 \cap e_2) \equiv \Gamma_{G;(F \otimes m)}(\chi_{m:\min(c_1, c_2)}($$

$$E_1 \bowtie_{\mathcal{A}(e_1)=\mathcal{A}(e_2)'} \rho_{\mathcal{A}(e_2) \leftarrow \mathcal{A}(e_2)'}(E_2))), \quad (7.200)$$

$$\Gamma_{G;F}(e_1 \setminus e_2) \equiv \Gamma_{G;(F \otimes m)}(\chi_{m:c_1 \dot{-} c_2}($$

$$E_1 \bowtie_{\mathcal{A}(e_1)=\mathcal{A}(e_2)'}^{c_2:0} \rho_{\mathcal{A}(e_2) \leftarrow \mathcal{A}(e_2)'}(E_2))), \quad (7.201)$$

where E_i is defined as

$$\Gamma_{\mathcal{A}(e_i);c_i:\text{count}(*)}(e_i)$$

for $i = 1, 2$.

7.12 Eliminating Redundant Joins

Since the join and outerjoin operations are very expensive, it makes sense to investigate possibilities to eliminate redundant joins and outerjoins. These often occur if queries use views [87, 658, 659], if queries are generated by front-end tools [327], if data is integrated from different source [165], or if objects are instantiated using relational views [88, 538]. One possibility studied intensively is to use tableaux techniques to detect and eliminate redundant joins [17, 18, ?, 470, 471, 577, 759]. Chandra and Merlin show that finding the minimal conjunctive query for a given one is NP-hard [144].

The techniques discussed in the above papers apply to set semantics and not to bags. This deficiency was later remedied [159, 160, 221, 497]. Chaudhuri and Vardi as well as Ioannidis and Ramakrishnan showed independently that the NP-hardness result still holds under bag semantics [159, 160, 449, 450].

Other work on join elimination occurs in the context of *semantic query optimization*. Early work here is by King [496]. The implementation of inner join elimination techniques in DBMSs is described by Cheng et al. for DB2 [172] and by Ghazal, Bhashyam, and Crolotte for Teradata [327]. They also describe the left-outer join elimination technique implemented in Teradata [328].

In this brief subsection, we present some simple algebraic equivalences that allow us to remove redundant joins and outerjoins. To derive an algorithm to eliminate all unnecessary joins is rather complicated and builds upon query containment and query equivalence. These will be discussed in depth in Chapter. 10.

Clearly, it is beneficial to eliminate joins with relations, whose attributes are not needed to evaluate the query. Consider a simple algebraic expression containing a single join and another expression where the join has been eliminated:

$$\Pi_{\mathcal{A}(e_1)}(e_1 \bowtie_{A_1=A_2} e_2) \equiv e_1$$

where $A_i \subseteq \mathcal{A}(e_i)$. This equivalence only holds if

1. for every tuple in e_1 at most one join partner in e_2 exists and
2. for every tuple in e_1 at least one join partner in e_2 exists.

The first condition is easily satisfied if A_2 is a (super-) key of e_2 . The second condition demands that $\Pi_{A_1}(e_1) \subseteq \rho_{A_2 \leftarrow A_1}(e_2)$. However, this does not truly suffice. Additionally, we must have that all attributes in A_1 are not null.

If there is a referential integrity constraint $e_1.A_1 \rightarrow e_2.A_2$ and the foreign-key attributes A_1 are nullable, the join can still be eliminated if we add a not-null predicate on the foreign-key attributes [172]:

$$\Pi_{\mathcal{A}(e_1)}(e_1 \bowtie_{A_1=A_2} e_2) \equiv \sigma_{\neg(A_1 \doteq \perp)}(e_1)$$

If we work with sets, things simplify a lot. Then,

$$\Pi_{\mathcal{A}(e_1)}^D(e_1 \bowtie_{A_1=A_2} e_2) \equiv e_1$$

holds whenever the second condition is fulfilled.

Outerjoins are also easier. The equivalence

$$\Pi_{\mathcal{A}(e_1)}(e_1 \Join_{A_1=A_2} e_2) \equiv e_1$$

holds if the first condition holds.

Note that the above equivalences do not demand e_1 and e_2 to be different. Thus they can also be used to eliminate redundant self-(outer)-joins.

Other pointers to the literature on join elimination are [164, 466, 855].

7.13 Semijoin and Antijoin Reducer

In the context of distributed and cluster-based database systems, it is important to reduce the amount of data shipped around [136, 264, 514, 666]. Introducing *semijoin reducer* is a common technique to achieve this. The according equivalences are:

$$e_1 \bowtie_q e_2 \equiv e_1 \bowtie_q (e_2 \bowtie_q e_1) \quad (7.202)$$

$$e_1 \bowtie_q e_2 \equiv e_1 \bowtie_q (e_2 \bowtie_q e_1) \quad (7.203)$$

$$e_1 \bowtie_q e_2 \equiv e_1 \bowtie_q (e_2 \bowtie_q e_1) \quad (7.204)$$

$$e_1 \bowtie_q e_2 \equiv e_1 \bowtie_q (e_2 \bowtie_q e_1) \quad (7.205)$$

$$e_1 \bowtie_{q;g:e} e_2 \equiv e_1 \bowtie_{q;g:e} (e_2 \bowtie_q e_1) \quad (7.206)$$

Assume we are given two relations R_1 and R_2 on two computers (stations) C_1 and C_2 and wish to calculate the join $R_1 \bowtie_{p_{12}} R_2$. Let $J_i := \mathcal{A}(R_i) \cap \mathcal{F}(p_{12})$ be the join attributes of the relations R_i . And define $e_{J_2} := \Pi_{J_2}^D(R_2)$. Semijoin reduction then works by sending the projection of the join attributes, i.e., e_{J_2} of relation R_2 to the computer C_1 where the relation R_1 resides. There, the semijoin $R_1 \bowtie_{p_{12}} e_{J_2}$ is calculated. Then, we could send over the result to computer C_2 and perform the join. However, sometimes it is beneficial to also reduce relation R_2 , e.g., if the join is calculated at some computer C_3 . Then, we can send the result of $e_{J_{21}} := \Pi_{J_1}^D(R_1 \bowtie_{p_{12}} e_{J_2})$ to computer C_2 and use it to reduce R_2 . However, if less than half of the values qualify, it is better to send the antijoin's result, i.e., $e_{A_{21}} := \Pi_{J_1}^D(R_1 \bowtie_{p_{12}} e_{J_2})$ and use $R_2 \bowtie_{p_{12}} e_{A_{21}}$ to reduce R_2 . Integrating semijoin reducers into old-style plan generators has been described by Stocker, Kossmann, Braumandl, and Kemper [836].

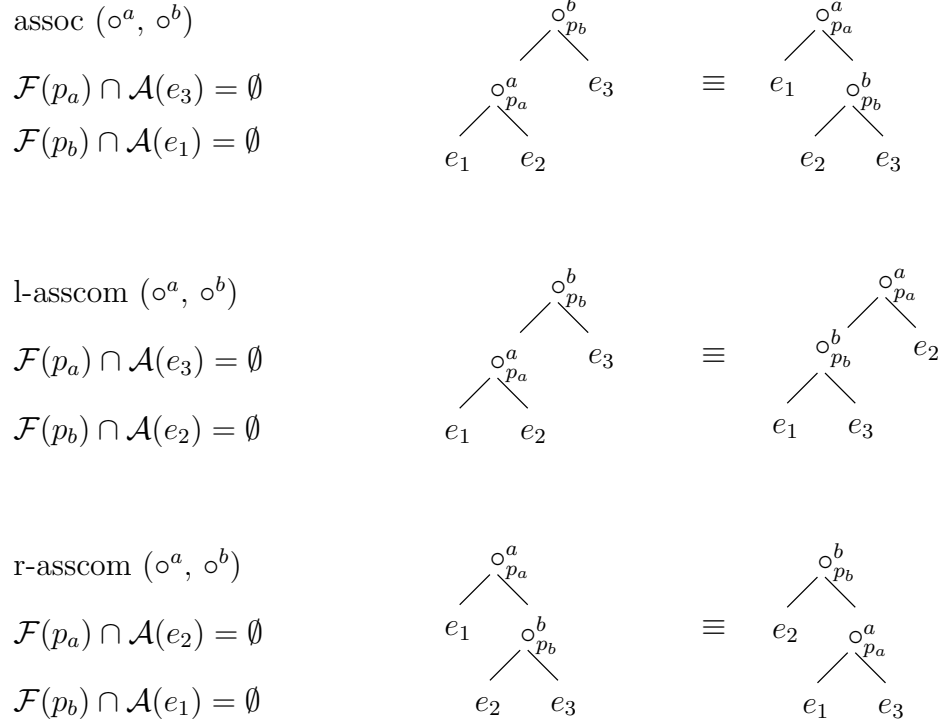


Figure 7.18: Transformation rules for assoc, l-asscom, and r-asscom

7.14 Outerjoin Simplification

7.15 Correct and Complete Exploration of the Core Search Space

7.15.1 The Core Search Space

The *core search space* for a given operator tree, normally derived from the input query, is spanned by the transformations derived from the commutativity, associativity, l-asscom and r-asscom properties of the operators occurring in the input tree. Except for commutativity, these transformations are shown in Fig. 7.18 for some arbitrary binary operators $\circ^a$ and $\circ^b$ with their according predicates. Note the syntactic constraints on the left and remember that commutativity does not have these syntactic constraints. These syntactic constraints have one interesting consequence. Let us call a predicate p of some binary operator $\circ$ *degenerate*, if it does not reference relations from at least one argument side of $\circ$. Then, we can observe that the syntactic constraints for non-degenerate predicates imply that either associativity or l-asscom can be applied for left-nesting but not both, and either associativity or r-asscom can be applied for right-nesting but not both.

Fig. 7.19 shows an example of the search space for an expression $(e_1 \circ_{12}^a e_2) \circ_{13}^b e_3$, where the subscripts of the operators indicate which arguments are referenced in their predicate. We observe that any expression in this search space

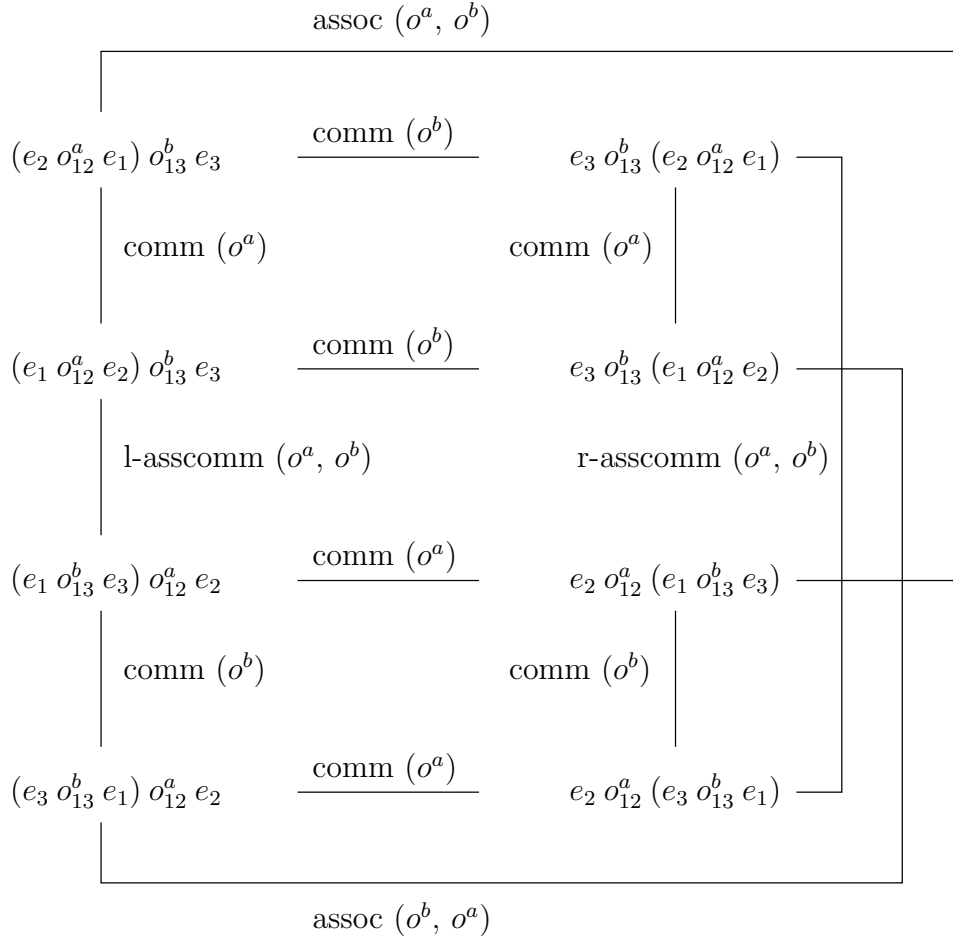


Figure 7.19: Core search space example

can be reached by a sequence of at most two applications of commutativity, at most one application of associativity, l-asscom, or r-asscom, finally followed by at most two applications of commutativity. The total number of applications of commutativity can be restricted to 2. The case $(e_1 o_{12}^a e_2) o_{23}^b e_3$ is left to the reader.

The last observation only holds if there are no degenerate predicates and no cross products in the original plan. Fig. 7.20 shows all possible plans for two binary operators $\circ^a$ and $\circ^b$. One can think of them as cross products. The plans are generated by applying assoc, l-asscom, r-asscom, and commutativity rewrites. Assume that the initial plan is the one in row 1 and column 3. The other plans in the first row are generated by using all rewrites but commutativity. The second row shows the plans derived from the plan above them by applying commutativity to the lower operator. The third row applies commutativity to the top operator of the plan above it in the first row. The fourth row applies commutativity to both operators. Thus, all plans in a column below a plan in the first row can be generated by at most two applications of commutativity. Of course, there are more possibilities to transform one plan into

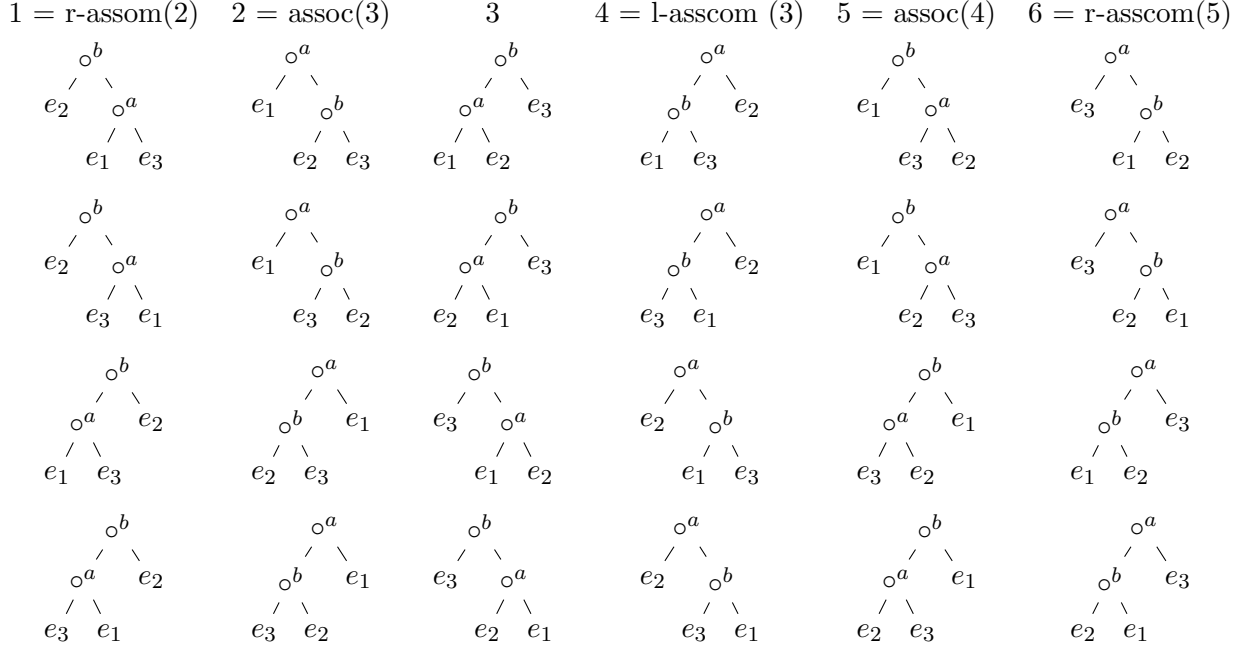


Figure 7.20: The complete search space

another. In order to indicate them, let us denote the matrix of plans by P . The application of transformations other than commutativity gives:

$$\begin{aligned}
 P[2, i] &\longleftrightarrow P[3, i + 1] \\
 P[3, i] &\longleftrightarrow P[2, i + 1] \\
 P[4, i] &\longleftrightarrow P[4, i + 1] \\
 P[1, 1] &\longleftrightarrow P[4, 6] \\
 P[2, 1] &\longleftrightarrow P[3, 6] \\
 P[3, 1] &\longleftrightarrow P[3, 6]
 \end{aligned}$$

It is easy to see, that we need more than one of assoc, l-asscom, or r-asscom to get from $P[1, 3]$ to, e.g., $P[1, 1]$.

7.15.2 Exploration

How does the plan generator explore this search space? Remember the join ordering algorithms from Chapter 3, especially **DPsub**, **DPsize**, and **DPccp**, which are all based on dynamic programming. We extend the simple algorithm **DPsub** to one called **DPsube**. The resulting code is shown in Fig. ???. As input it takes the set of n relations $R = \{R_0, \dots, R_{n-1}\}$ and the set of operators O containing $n - 1$ operators which **DPsube** has to use in order to build a plan. First, it constructs plan for single relations. Then, it enumerates all subsets S of relations by decoding an integer, which represents a bitvector. For each set of relations S , **DPsube** then enumerates all subsets S_1 of S and their complements S_2 . Both of them must be non-empty. For each pair (S_1, S_2) , all operators $\circ$

Algorithm **DPsube**

Input: a set of relations $R = \{R_0, \dots, R_{n-1}\}$
a set of operators O with associated conflict descriptors

Output: an optimal bushy operator tree

```

for all  $R_i \in R$ 
   $\text{BestPlan}(\{R_i\}) = R_i$ ;
for  $1 \leq i < 2^n - 1$  ascending
   $S = \{R_j \in R \mid \lfloor i/2^j \rfloor \bmod 2 = 1\}$ 
  if  $(|S| = 1)$  continue
  for all  $S_1 \subset S, S_1 \neq \emptyset$  do
     $S_2 = S \setminus S_1$ ;
    for all  $\circ \in O$  do
      if  $(\text{applicable}(\circ, S_1, S_2))$ 
        build and handle the plans  $\text{BestPlan}(S_1) \circ \text{BestPlan}(S_2)$ 
        if  $(\circ \text{ is commutative})$ 
          build and handle the plans  $\text{BestPlan}(S_2) \circ \text{BestPlan}(S_1)$ 
return  $\text{BestPlan}(R)$ ;

```

Figure 7.21: Algorithm **DPsube**

in O are then tested for applicability via a call to *applicable*. If the operator is applicable, then the best plans p_1 for S_1 and p_2 for S_2 are recalled from the dynamic programming table *BestPlan* and combined into the plan $p_1 \circ p_2$ for S . The costs of this plan are then calculated and it is possibly added to the DP-table. Since this piece of code is straight forward, we did not detail on it. Note that only if an operator is applicable then **DPsube** also considers $p_2 \circ p_1$ if $\circ$ is commutative. The rest of the section deals with different implementations of *applicable*.

Two implementations of *applicable* are described in the literature. Each of them uses a set of relations as a short-hand representation of possible reordering conflicts. The first set is called EEL, and is presented by Rao, Lindsay, Lohman, Pirahesh, and Simmen [717, 716]. The second set is called TES, and is presented by Moerkotte and Neumann [620, 619]. The first approach is limited to $\bowtie$, $\triangleright$, and $\bowtie$. Both approaches generate invalid plans, i.e., plans which are not equivalent to the input operator tree. Thus, we will present an alternative test. The main properties are that it will be correct and complete. An implementation of *applicable* is *correct*, if only valid plans are generated. It is *complete*, if all valid plans are generated.

Preliminaries

In order to open our approach for new algebraic operators, we use a table driven approach. We use four tables which contain the properties of the algebraic operators. These contain the information of Tables 7.6 and 7.7 together with

the information about the commutativity of the operators. Thus, extending our approach only requires to extend these tables.

We develop our final approach in three steps. At each step, we present a complete bundle consisting of three components:

1. a representation for conflicts
2. a conflict detection (CD) algorithm, which detects the conflicts from an initial operator tree and produces a conflict representation for this operator, and
3. the implementation of *applicable*, which uses the conflict representation for an operator and then determines whether the operator can be applied in a given context.

Each of the subsequently discussed bundles is correct, but only the last one is complete.

The main idea in the following (and in the literature cited above) is to extend the consumer/producer constraints. Therefore, we first introduce *syntactic eligibility sets* (SES for short), which are attached to operators and contain the set of relations that must be present before the operator can be applied. Sometimes, SES is called NEL.

For every operator $\circ$, $\text{SES}(\circ)$ is thus a set of relations. Then, a plan of the form $\text{plan}(S_1) \circ \text{plan}(S_2)$ is only considered if the test $\text{SES}(\circ) \subseteq S_1 \cup S_2$ succeeds. Hence, SES checks for a consumer/producer relationships.

Some operators like the groupjoin or map operator introduce new attributes. These are treated as if they belong to a new artificial relation. This new relation is present in the set of accessible relations after the groupjoin or map operator has been applied.

We assume that an initial operator tree is given and refer to it as *the operator tree*. We need some notation. For a set of attributes A , we denote by $\text{REL}(A)$ the set of relations to which these attributes belong. We abbreviate $\text{REL}(\mathcal{F}(e))$ by $\mathcal{F}_T(e)$. Let $\circ$ be an operator in the initial operator tree. We denote by $\text{left}(\circ)$ ($\text{right}(\circ)$) its left (right) descendants. $\text{STO}(\circ)$ denotes the operators contained in the operator subtree rooted at $\circ$. $\text{REL}(\circ)$ denotes the set of relations contained in the subtree rooted at $\circ$.

The *syntactic eligibility set* (SES) is used to express the syntactic constraints: all referenced attributes/relations must be present before an expression can be evaluated. First of all, it contains the relations referenced by a predicate. Further, as we also deal with table functions and dependent join operators as well as groupjoins, we need the following extensions. Let R be a relation, T a table-valued function call, $\circ_p$ any of our binary or unary operators except a

groupjoin, and $gj \in \{\bowtie, \Join\}$. Then, we define:

$$\begin{aligned}
 \text{SES}(R) &= \{R\} \\
 \text{SES}(T) &= \{T\} \\
 \text{SES}(\circ_p) &= \bigcup_{R \in \mathcal{F}_T(p)} \text{SES}(R) \cap \text{REL}(\circ_p) \\
 \text{SES}(gj_{p;a_1:e_1, \dots, a_n:e_n}) &= \bigcup_{R \in \mathcal{F}_T(p) \cup \mathcal{F}_T(e_i)} \text{SES}(R) \cap \text{REL}(gj)
 \end{aligned}$$

All conflict representations have a component **TES** which contains a set of tables. We always initialize **TES** with **SES** as calculated above. Further, we assume that our conflict representation has two accessors *tesl* and *tesr* which return

$$\begin{aligned}
 \text{tesl}(\circ) &:= \text{TES}(\circ) \cap \text{REL}(\text{left}(\circ)) \\
 \text{tesr}(\circ) &:= \text{TES}(\circ) \cap \text{REL}(\text{right}(\circ))
 \end{aligned}$$

This distinction is necessary, since we want to consider commutativity explicitly and prevent in those cases where commutativity does not hold, that operators which occurred on the left-hand side of an operator move to its right-hand side or vice versa. All our implementations of **applicable** conjunctively include the tests

$$\begin{aligned}
 \text{tesl} &\subseteq S_1, \text{ and} \\
 \text{tesr} &\subseteq S_2.
 \end{aligned}$$

Approach CD-A

Let us first consider a simple operator tree with only two operators. Take a look at the upper half of Fig. 7.22. There, it illustrates the application of associativity and l-asscom to some plan. In the case that associativity does not hold, we add $\text{REL}(e_1)$ to $\text{TES}(\circ^b)$. This prevents the plan on the right-hand side of the arrow marked with *assoc*. It does not, however, prevent the plan on the right-hand side of the arrow marked with *l-asscom*. Similarly, adding $\text{REL}(e_2)$ to $\text{TES}(\circ^b)$ does prevent the plan resulting from *l-asscom* but not the plan resulting from applying associativity. The lower part of Fig. 7.22 shows the actions needed if an operator is nested in the right argument. Again, we can precisely prevent the invalid plans.

The only problem we now have to solve is that a conflicting operator is deeper down the tree. This is possible since in general the e_i are trees themselves. Some reordering could possibly move a conflicting operator up to the top of an argument subtree. We thus have to calculate the total eligibility sets bottom-up. In a first step, for every operator $\circ$ in a given operator tree $\text{SES}(\circ)$ is calculated. Then, $\text{TES}(\circ)$ is initialized to $\text{SES}(\circ)$. After that, the following procedure is applied bottom-up to every operator $\circ_{p_a}^a$ in the operator tree:

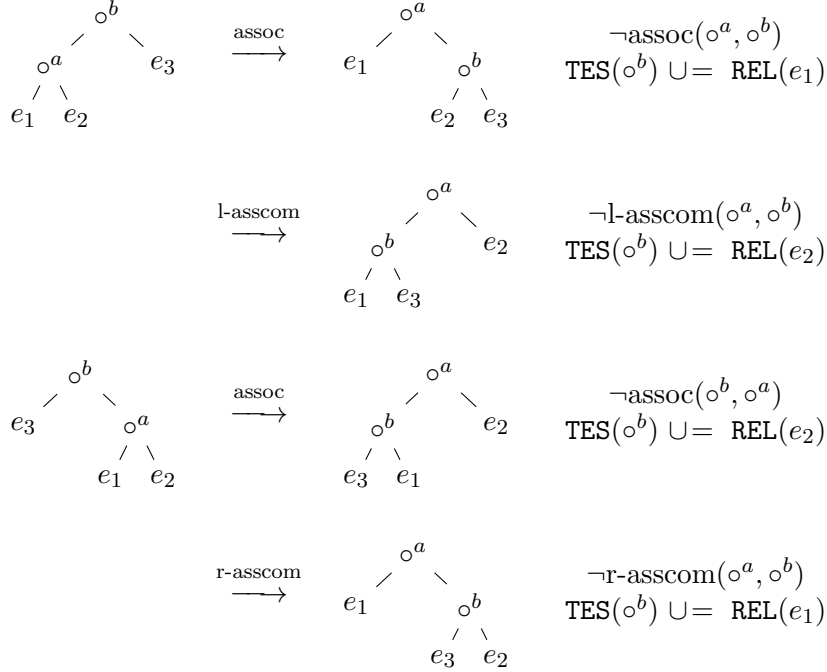


Figure 7.22: Calculating TES for simple operator trees

CD-A($\circ_{p_b}^b$) // operator $\circ^b$ and its predicate p_b

```

for  $\forall \circ^a \in \text{ST0}(\text{left}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^a, \circ^b)$  then  $\text{TES}(\circ^b) \cup \text{REL}(\text{left}(\circ^a))$ 
  if  $\neg \text{l-asscom}(\circ^a, \circ^b)$  then  $\text{TES}(\circ^b) \cup \text{REL}(\text{right}(\circ^a))$ 
for  $\forall \circ^a \in \text{ST0}(\text{right}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^b, \circ^a)$  then  $\text{TES}(\circ^b) \cup \text{REL}(\text{right}(\circ^a))$ 
  if  $\neg \text{r-asscom}(\circ^a, \circ^b)$  then  $\text{TES}(\circ^b) \cup \text{REL}(\text{left}(\circ^a))$ 

```

If we do not have degenerate predicates and cross products among the operators in the initial operator tree, we can safely use TES instead of REL.

The conflict representation comprises the TES for every operator. The definition of *applicable* is

$$\text{applicable}(\circ, S_1, S_2) := \text{tesl}(\circ) \subseteq S_1 \wedge \text{tesr}(\circ) \subseteq S_2.$$

Let us now see why *applicable* is correct. We have to show that it prevents the generation of bad plans. Take the $\neg \text{assoc}$ case with nesting on the left. Let the original operator tree contain $(e_1 \circ_{12}^a e_2) \circ_{23}^b e_3$. Define the set of tables $R_2 := \mathcal{F}_T(\circ_{23}^b) \cap \text{REL}(\text{left}(\circ_{23}^b))$ and $R_3 := \mathcal{F}_T(\circ_{23}^b) \cap \text{REL}(\text{right}(\circ_{23}^b))$. Then $\text{SES}(\circ_{23}^b) = R_2 \cup R_3$. Further, since $\neg \text{assoc}(\circ_{12}^a, \circ_{23}^b)$, we have

$$\text{TES}(\circ_{23}^b) \supseteq \text{SES}(\circ_{23}^b) \cup \text{REL}(e_1).$$

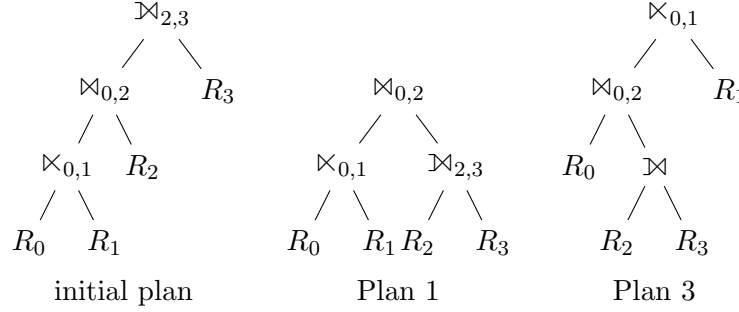


Figure 7.23: Example showing the incompleteness of CD-A

Note that we used $\supseteq$ and not equality since due to other conflicts, $\text{TES}(\circ^b)$ could be larger. Next, we observe that

$$\begin{aligned} \text{tesl}(\circ_{23}^b) &\supseteq (\text{SES}(\circ_{23}^b) \cup \text{REL}(e_1)) \cap \text{REL}(\text{left}(\circ_{23}^b)) = \text{REL}(e_1) \cup R_2 \\ \text{tesr}(\circ_{23}^b) &\supseteq (\text{SES}(\circ_{23}^b) \cup \text{REL}(e_1)) \cap \text{REL}(\text{right}(\circ_{23}^b)) = R_3 \end{aligned}$$

Let S_1, S_2 be a pair of two arbitrary subsets of relations generated by DPsube . Then, the call $\text{applicable}(\circ^b, S_1, S_2)$ checks

$$\begin{aligned} \text{tesl}(\circ_{23}^b) &\subseteq S_1 \text{ and} \\ \text{tesr}(\circ_{23}^b) &\subseteq S_2, \end{aligned}$$

and fails if $S_1 \not\supseteq \text{REL}(e_1)$. Thus, neither $e_2 \circ_{23}^b e_3$ nor $e_3 \circ_{23}^b e_2$ will be generated and, hence, $e_1 \circ_{12}^a (e_2 \circ_{23}^b e_3)$ will not be generated. Similarly, if $\neg \text{l-asscom}(\circ^a, \circ^b)$, $\text{tesl}(\circ^b)$ will contain $\text{REL}(e_2)$ and the test prevents the generation of $e_1 \circ^b e_3$. The remaining two cases can be checked analogously.

From this discussion, it follows that DPsube generates only valid plans. However, it does not generate all valid plans. It is thus incomplete, as we can see from the example shown in Fig. 7.23. Since $\neg \text{assoc}(\ltimes, \ltimes)$, $\text{TES}(\ltimes)$ contains R_1 . Thus, neither Plan 1 nor Plan 3 or any of those derived from applying join commutativity to them will be generated.

Approach CD-B

In order to avoid this problem, we need the more flexible mechanism of *conflict rules*. A *conflict rule* is simply a pair of sets of tables denoted by $T_1 \rightarrow T_2$. With every operator node $\circ$ in the operator tree, we associate a set of conflict rules. Thus, our conflict representation now associates with every operator a TES and a set of conflict rules.

Before we introduce their construction, let us illustrate their role in *applicable*(S_1, S_2). A conflict rule $T_1 \rightarrow T_2$ is *obeyed* for S_1 and S_2 , if with $S = S_1 \cup S_2$ the following condition holds:

$$T_1 \cap S \neq \emptyset \implies T_2 \subseteq S.$$

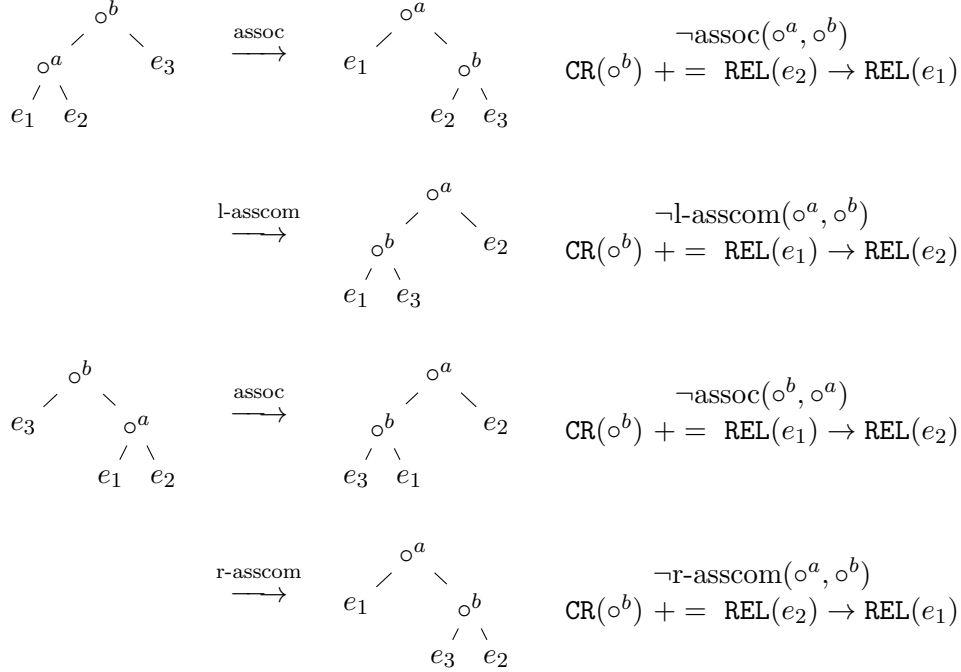


Figure 7.24: Calculating conflict rules for simple operator trees

Thus, if T_1 contains a single relation from S , then S must contain all relations in T_2 . Keeping this in mind, it is easy to see that the invalid plans are indeed prevented by the rules shown in Fig. 7.24 if they are obeyed.

As before, we just need to generalize it to arbitrary trees:

CD-B($\circ^b_{p_b}$) // operator $\circ^b$ and its predicate p_b

```

for  $\forall \circ^a \in \text{STO}(\text{left}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^a, \circ^b)$  then  $\text{CR}(\circ^b) += \text{REL}(\text{right}(\circ^a)) \rightarrow \text{REL}(\text{left}(\circ^a))$ 
  if  $\neg \text{l-asscom}(\circ^a, \circ^b)$  then  $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a))$ 
for  $\forall \circ^a \in \text{STO}(\text{right}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^b, \circ^a)$  then  $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a))$ 
  if  $\neg \text{r-asscom}(\circ^a, \circ^b)$  then  $\text{CR}(\circ^b) += \text{REL}(\text{right}(\circ^a)) \rightarrow \text{REL}(\text{left}(\circ^a))$ 

```

The test $\text{applicable}(\circ, S_1, S_2)$ checks two conditions:

1. $\text{tesl} \subseteq S_1 \wedge \text{tesr} \subseteq S_2$ must hold, and
2. all rules in the rule set of $\circ$ must be obeyed.

Again, this implementation of applicable is correct but not complete, as the example in Fig. 7.25 shows. Since $\text{assoc}(\bowtie, \bowtie)$ and $\text{l-asscom}(\bowtie, \bowtie)$, the only conflict occurs due to $\text{r-asscom}(\bowtie, \bowtie)$. Thus,

$$\text{REL}(\{R_3\}) \rightarrow \text{REL}(\{R_1, R_2\}) \in \text{CR}(\bowtie_{0,1})$$

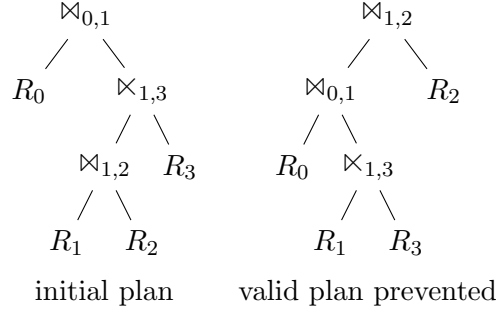


Figure 7.25: Example showing the incompleteness of CD-B

The latter rule prevents the plan on the right-hand side of Fig. 7.25. Note that it is overly careful since $R_2 \notin \mathcal{F}_T(\Join_{1,3})$. In fact, r-asscom would never be applied in this example, since $\Join_{0,1}$ accesses table R_1 and applying r-asscom would thus destroy the consumer/producer relationship already checked by $\text{SES}(\Join_{0,1})$.

Approach CD-C

The approach CD-C differs from CD-B only by the calculation of the conflict rules. The conflict representation and the procedure for *applicable* remain the same. The idea is now to learn from the above example and include only those relations under operator $\circ^a$, which occur in the predicate. However, we have to be careful to include special cases for degenerate predicates and cross products.

CD-C($\circ_{p_b}^b$) // operator $\circ^b$ and its predicate p_b

```

for  $\forall \circ^a \in \text{STO}(\text{left}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^a, \circ^b)$  then
    if  $\text{REL}(\text{left}(\circ^a)) \cap \mathcal{F}_T(\circ^a) \neq \emptyset$  then
       $\text{CR}(\circ^b) += \text{REL}(\text{right}(\circ^a)) \rightarrow \text{REL}(\text{left}(\circ^a)) \cap \mathcal{F}_T(\circ^a)$ 
    else
       $\text{CR}(\circ^b) += \text{REL}(\text{right}(\circ^a)) \rightarrow \text{REL}(\text{left}(\circ^a))$ 
  if  $\neg \text{l-asscom}(\circ^a, \circ^b)$  then
    if  $\text{REL}(\text{right}(\circ^a)) \cap \mathcal{F}_T(\circ^a) \neq \emptyset$  then
       $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a)) \cap \mathcal{F}_T(\circ^a)$ 
    else
       $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a))$ 
for  $\forall \circ^a \in \text{STO}(\text{right}(\circ^b))$ 
  if  $\neg \text{assoc}(\circ^b, \circ^a)$  then
    if  $\text{REL}(\text{right}(\circ^a)) \cap \mathcal{F}_T(\circ^a) \neq \emptyset$  then
       $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a)) \cap \mathcal{F}_T(\circ^a)$ 
    else
       $\text{CR}(\circ^b) += \text{REL}(\text{left}(\circ^a)) \rightarrow \text{REL}(\text{right}(\circ^a))$ 
  if  $\neg \text{r-asscom}(\circ^a, \circ^b)$  then
    if  $\text{REL}(\text{left}(\circ^a)) \cap \mathcal{F}_T(\circ^a) \neq \emptyset$  then
       $\text{CR}(\circ^b) += \text{REL}(\text{right}(\circ^a)) \rightarrow \text{REL}(\text{left}(\circ^a)) \cap \mathcal{F}_T(\circ^a)$ 

```

```

else
  CR( $\circ^b$ ) += REL(right( $\circ^a$ )) → REL(left( $\circ^a$ ))

```

Rule Simplification

Large TES make the search space to be explored by the plan generator smaller and, thus, lead to more efficiency, at least if an advanced plan generator like DPhy is used. Further, reducing the number of rules slightly decreases plan generation time. Thus, applying laws like

$$\begin{aligned}
 R_1 \rightarrow R_2, R_1 \rightarrow R_3 &\equiv R_1 \rightarrow R_2 \cup R_3 \\
 R_1 \rightarrow R_2, R_3 \rightarrow R_2 &\equiv R_1 \cup R_3 \rightarrow R_2
 \end{aligned}$$

can be used to rearrange the rule set for efficient evaluation. However, we are much more interested in eliminating rules altogether by adding their right-hand side to the TES. For some operator $\circ$, consider a conflict rule $R_1 \rightarrow R_2$. If $R_1 \cap \text{TES}(\circ) \neq \emptyset$, then we can add R_2 to TES due to the existential quantifier on the left-hand side of a rule in the definition of *obey*. Further, if $R_2 \subseteq \text{TES}(\circ)$, we can safely eliminate the rule. Applying these rearrangements is often possible since both $\text{REL}(\text{left}(\circ^a)) \cap \mathcal{F}_T(\circ)$ and $\text{REL}(\text{right}(\circ^a)) \cap \mathcal{F}_T(\circ)$ will be non-empty.

7.15.3 More Issues

Unary Operators

Not all unary operators are freely reorderable (see Table 7.3). Fortunately, handling conflicts for unary operators is quite simple. We associate a new artificial relation with every unary operator (that needs one). For a given unary operator $\circ^a$, denote this relation by $\text{AREL}(\circ^a)$. Then, whenever a conflict with a unary operator $\circ^b$ above $\circ^a$ occurs, we add $\text{AREL}(\circ^a)$ to $\text{TES}(\circ^b)$. The result is that they will never be reordered. This is captured in the following code fragment:

```

CD-C

```

```

  for each unary operator ( $\circ^b$ )
    for each unary operator  $\circ^a$  in  $\text{STO}(\circ^b)$ 
      if  $\neg \text{reorderable}(\circ^a, \circ^b)$ 
        TES( $\circ^b$ ) += AREL( $\circ^a$ )

```

Mixing Unary and Binary Operators

A unary operator maybe left- or right-pushable into some binary operator (see Table 7.4). In the given original operator tree, a unary operator may occur above or below a binary operator. Accordingly, we extend CD-C by two more cases.

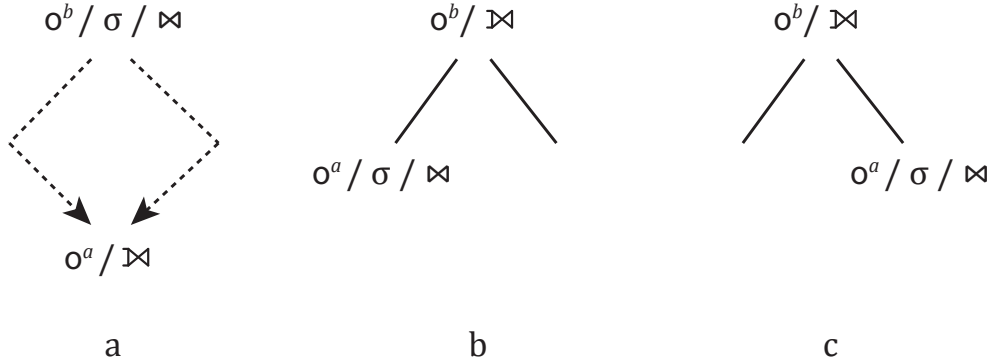


Figure 7.26: Conflict detection for unary and binary operators

Let us first consider the case where a binary operator $\circ^a$ can be found somewhere below a unary operator $\circ^b$. This is illustrated in Fig. 7.26 a. (Don't be confused by the two dotted lines, they will be used later on. Just image a single line connecting $\circ^b$ with $\circ^a$.) If $\circ^b$ is left- and right-pushable into $\circ^a$, we do not have any conflict. If $\circ^b$ is neither left- nor right-pushable into $\circ^a$, any valid plan must contain $\circ^b$ above $\circ^a$. This is achieved by extending the TES of $\circ^b$ by all relations below $\circ^a$. Consider the case where $\circ^b$ is not right-pushable. Then, we must prevent any plan where $\circ^b$ occurs in the right subtree of $\circ^a$. Adding a conflict rule $\circ^a$ which says that if any relation from $\circ^a$'s right subtree occurs in the current plan to which we want to add $\circ^b$, then it must contain all relations from its left subtree. The other case is symmetric. We summarize these ideas in the following extension to CD-C:

CD-C

```

for all unary operators  $\circ^b$  in the original operator tree
  for all binary operators  $\circ^a \in STO(\circ^b)$ 
    if  $\neg \text{left-pushable}(\circ^b, \circ^a) \wedge \text{right-pushable}(\circ^b, \circ^a)$ 
       $CR(\circ^b)+ = REL(\text{left}(\circ^a)) \rightarrow REL(\text{right}(\circ^a))$ 
    if  $\text{left-pushable}(\circ^b, \circ^a) \wedge \neg \text{right-pushable}(\circ^b, \circ^a)$ 
       $CR(\circ^b)+ = REL(\text{right}(\circ^a)) \rightarrow REL(\text{left}(\circ^a))$ 
    if  $\neg \text{left-pushable}(\circ^b, \circ^a) \wedge \neg \text{right-pushable}(\circ^b, \circ^a)$ 
       $TES(\circ^b)+ = REL(\circ^a)$ 

```

Now, we consider the case where a unary operator $\circ^a$ can be found somewhere below a binary operator $\circ^b$ (see Fig. 7.26 b,c). In this case, if it cannot be pulled up, we prevent this by adding the artificial relation AREL of $\circ^b$ to the TES of $\circ^a$:

CD-C

```

for all binary operators  $\circ^b$  in the original operator tree

```

```

for all unary operators  $\circ^a \in STO(left(\circ^b))$ 
  if  $\neg left\text{-pushable}(\circ^a, \circ^b)$ 
     $TES(\circ^b)+ = AREL(\circ^b)$ 
for all unary operators  $\circ^a \in STO(right(\circ^b))$ 
  if  $\neg right\text{-pushable}(\circ^a, \circ^b)$ 
     $TES(\circ^b)+ = AREL(\circ^b)$ 

```

A selection operator can be changed into a join if its predicate references two or more relations. In this case, a conflict between the resulting join and some other binary operator might occur. We can handle these potential conflicts as follows. Consider Fig. 7.26 a. By $\circ^b/\sigma/\bowtie$ we denote our selection that can be turned into a join. By $\circ^a/\bowtie$ we denote a binary operator below our selection. The case that it might be a left outerjoin is used in a subsequent example. The Figure shows the trick we perform. We assume that a selection that can be turned into a join has two arguments, a left and a right subtree. Both of which point to the (only) child node of the selection. Thus, the left outerjoin is once the left child of the selection/join and once the right one. Then, the usual CD-C procedure can be run in addition to the above conflict handling. Let us do so for the example. In case we treat the left outerjoin as the left child of the outerjoin, we derive from $\neg assoc(\bowtie, \bowtie)$

$$CR(\circ^b)+ = REL(right(\circ^a)) \rightarrow REL(left(\circ^a))$$

possibly with $\cap \mathcal{F}_T(\circ^a)$ on the right-hand side. In the other case, we get due to the fact that $\neg r\text{-asscom}(\bowtie, \bowtie)$

$$CR(\circ^b)+ = REL(right(\circ^a)) \rightarrow REL(left(\circ^a)),$$

possibly with $\cap \mathcal{F}_T(\circ^a)$ on the right-hand side. In any case, both conflicts result in the same conflict rule. Further, both are subsumed by the above conflict handling for the unary/binary operator mix. The reader should validate that this is the case for all the operators in our algebra. However, since we want to be purely table driven, we simply add these (redundant) conflict rules and rely on rule simplification.

Cross Products and Degenerate Predicates

Cross products and degenerate predicates require more care as the comparison between Fig. 7.19 and Fig. 7.20 shows. They lack the syntactic constraints due to attribute accesses, which highly restrict the number of syntactically valid plans. Consider an example like

$$(R_1 \times R_2) \bowtie_{1,3} (R_3 \bowtie_{3,4} R_4).$$

So far, nothing prevents **DPsube** to consider invalid plans like

$$R_1 \bowtie_{1,3} (R_3 \bowtie_{3,4} (R_2 \times R_4)).$$

Note that in order to prevent this plan, we would have to detect conflicts on the “other side” of the plan. In our example, we need to consider conflicts between operators in the left and the right subtree of $\bowtie_{1,3}$. Since cross products and degenerate predicates should be rare in real queries, it suffices to produce correct plans. We have no ambition to explore the complete search space. Thus, we just want to make sure that in these abnormal cases the plan generator still produces a correct plan. In order to do so, we proceed as follows. We extend the conflict representation by two bitvectors representing the left and the right relations of an operator. Let us call them *relLeft* and *relRight*. Then, we extend the *applicable* test and check that at least one relation from *relLeft* occurs in the left subplan, and at least one relation from *relRight* occurs in the right subplan. That is, in the test for *applicable*($\circ$, S_1 , S_2), we conjunctively check

$$(\text{relLeft} \cap S_1 \neq \emptyset) \wedge (\text{relRight} \cap S_2 \neq \emptyset).$$

This results in a correct test, but, as experiments have shown, about a third of the valid search space will not be explored if cross products are present in the initial operator tree. However, note that if the initial plan does not contain cross products and degenerate predicates, this test will always succeed such that in this case still the whole core search space is explored. Further, still a larger portion of the core search space is explored when comparing this approach to the one by Rao et al. [716, 717]. There, two separate runs of the plan generator for the arguments of a cross product hinders any reordering of operators with cross products.

There is a second issue concerning cross products. In some rare cases, they might be beneficially introduced, even if the initial plan does not demand them. In this case, we can proceed as proposed by Rao et al. [716, 717]. For each relation R , a *companion set* is calculated which contains all relations, that are connected to R only by inner join predicates. Within a companion set, all join orders and introductions of cross products are valid.

Other Plan Generators

It is rather simple to incorporate our test into other algorithms than **DPsub** (e.g., **DPsize**, **DPccp**, **TDxxx**). However, the result is not necessarily too efficient. An efficient approach is discussed in Chapter ??, where we generalize **DPccp** to cover hypergraphs. To see why this is appropriate, observe that $(\text{tesl}(\circ), \text{tesr}(\circ))$ is a hyperedge.

Beyond the Core Search Space

At the beginning, we talked about the core search space. Why core? Because there are more equivalences which we would like to be considered by the plan generator. First of all, early grouping can significantly improve performance. Then, some equivalences with operator conversions (e.g., Eqvs. ??, 7.99, 7.100, 7.199), and 7.198) are also important. These cases require some special treatment. This is discussed in Chapter ??.

7.16 Logical Algebra for Sequences

7.16.1 Introduction

The algebra (NAL) we use here extends the SAL-Algebra [70] developed by Beeri and Tzaban. SAL is the order-preserving counterpart of the algebra used in [189, 191] and in this book.

SAL and NAL work on sequences of sets of variable bindings, i.e., sequences of unordered tuples where every attribute corresponds to a variable. We allow nested tuples, i.e. the value of an attribute may be a sequence of tuples. Single tuples are constructed by using the standard $[\cdot]$ brackets. The concatenation of tuples and functions is denoted by $\circ$. The set of attributes defined for an expression e is defined as $\mathcal{A}(e)$. The set of free variables of an expression e is defined as $\mathcal{F}(e)$.

The projection of a tuple on a set of attributes A is denoted by $|_A$. For an expression e_1 possibly containing free variables, and a tuple e_2 , we denote by $e_1(e_2)$ the result of evaluating e_1 where bindings of free variables are taken from variable bindings provided by e_2 . Of course this requires $\mathcal{F}(e_1) \subseteq \mathcal{A}(e_2)$. For a set of attributes A , we define the tuple constructor $\perp_A$ such that it returns a tuple with attributes in A initialized to NULL.

For sequences e we use $\alpha(e)$ to denote the first element of a sequence. We identify single element sequences and elements. The function τ retrieves the tail of a sequence, and $\oplus$ concatenates two sequences. We denote the empty sequence by ϵ . As a first application, we construct from a sequence of non-tuple values e a sequence of tuples denoted by $e[a]$. It is empty if e is empty. Otherwise, $e[a] = [a : \alpha(e)] \oplus \tau(e)[a]$.

By *id* we denote the identity function. In order to avoid special cases during the translation of XQuery into the algebra, we use the special algebraic operator ($\Box$) that returns a singleton sequence consisting of the empty tuple, i.e., a tuple with no attributes.

We will only define order-preserving algebraic operators. For the unordered counterparts see [191]. Typically, when translating a more complex XQuery into our algebra, a mixture of order-preserving and not order-preserving operators will occur. In order to keep the section readable, we only employ the order-preserving operators and use the same notation for them that has been used in [189, 191], SAL [70], and this book.

Again, our algebra will allow nesting of algebraic expressions. For example, within a selection predicate of a select operator we allow the occurrence of further nested algebraic expressions. Hence, a join within a selection predicate is possible. This simplifies the translation procedure of nested XQuery expressions into the algebra. However, nested algebraic expressions force a nested loop evaluation strategy. Thus, the goal of the paper will be to remove nested algebraic expressions. As a result, we perform unnesting of nested queries not at the source level, but at the algebraic level. This approach is more versatile and less error-prone.

$e_1 := R_1$	$e_2 := R_2$		$e_3 := \hat{\chi}_{a:\hat{\sigma}_{a_1=a_2}}(e_1)$
a_1	a_2	b	a
1	1	2	$\langle [1, 2], [1, 3] \rangle$
2	1	3	$\langle [2, 4], [2, 5] \rangle$
3	2	4	$\langle \rangle$
	2	5	$\langle \rangle$

Figure 7.27: Example for Map Operator

7.16.2 Algebraic Operators

We define the algebraic operators recursively on their input sequences. For unary operators, if the input sequence is empty, the output sequence is also empty. For binary operators, the output sequence is empty whenever the left operand represents an empty sequence.

The order-preserving selection operator is defined as

$$\hat{\sigma}_p(e) := \begin{cases} \epsilon & \text{if } e = \epsilon, \\ \alpha(e) \oplus \hat{\sigma}_p(\tau(e)) & \text{if } p(\alpha(e)), \\ \hat{\sigma}_p(\tau(e)) & \text{else.} \end{cases}$$

For a list of attribute names A , we define the projection operator as

$$\hat{\Pi}_A(e) := \begin{cases} \epsilon & \text{if } e = \epsilon, \\ \alpha(e)|_A \oplus \hat{\Pi}_A(\tau(e)) & \text{else.} \end{cases}$$

We also define a duplicate-eliminating projection $\hat{\Pi}_A^D$. Besides the projection, its semantics is similar to that of the **distinct-values** function of XQuery: it does not preserve order. However, we require it to be deterministic and idempotent. Sometimes we just want to eliminate some attributes. When we want to eliminate the set of attributes A , we denote this by $\hat{\Pi}_{\bar{A}}$. We use $\hat{\Pi}$ also for renaming attributes. Then we write $\hat{\Pi}_{A':A}$. The attributes in A are renamed to those in A' . Attributes other than those in A remain untouched.

The map operator is defined as follows:

$$\hat{\chi}_{a:e_2}(e_1) := \begin{cases} \epsilon & \text{if } e_1 = \epsilon, \\ \alpha(e_1) \circ [a : e_2(\alpha(e_1))] \oplus \hat{\chi}_{a:e_2}(\tau(e_1)) & \text{else.} \end{cases}$$

It extends a given input tuple $t_1 \in e_1$ by a new attribute a whose value is computed by evaluating $e_2(t_1)$. For an example see Figure 7.27.

We define the cross product of two tuple sequences as

$$e_1 \hat{\times} e_2 := \begin{cases} \epsilon & \text{if } e_1 = \epsilon, \\ (\alpha(e_1) \hat{\times} e_2) \oplus (\tau(e_1) \hat{\times} e_2) & \text{else.} \end{cases}$$

where

$$e_1 \hat{\times} e_2 := \begin{cases} \epsilon & \text{if } e_2 = \epsilon, \\ (e_1 \circ \alpha(e_2)) \oplus (e_1 \hat{\times} \tau(e_2)) & \text{else.} \end{cases}$$

We are now prepared to define the join operation on ordered sequences:

$$e_1 \hat{\bowtie}_p e_2 := \hat{\sigma}_p(e_1 \hat{\times} e_2).$$

We define the semijoin as

$$e_1 \hat{\ltimes}_p e_2 := \begin{cases} \alpha(e_1) \oplus (\tau(e_1) \hat{\ltimes}_p e_2) & \text{if } \exists x \in e_2 \ p(\alpha(e_1) \circ x), \\ \tau(e_1) \hat{\ltimes}_p e_2 & \text{else.} \end{cases}$$

and the anti-join as

$$e_1 \hat{\hat{\ltimes}}_p e_2 := \begin{cases} \alpha(e_1) \oplus (\tau(e_1) \hat{\hat{\ltimes}}_p e_2) & \text{if } \nexists x \in e_2 \ p(\alpha(e_1) \circ x), \\ (\tau(e_1) \hat{\hat{\ltimes}}_p e_2) & \text{else.} \end{cases}$$

The left outer join, which will play an essential role in unnesting, is defined as $e_1 \hat{\bowtie}_p^{g:e} e_2 :=$

$$\begin{cases} (\alpha(e_1) \hat{\ltimes}_p e_2) \oplus (\tau(e_1) \hat{\bowtie}_p^{g:e} e_2) & \text{if } (\alpha(e_1) \hat{\ltimes}_p e_2) \neq \epsilon, \\ (\alpha(e_1) \circ \perp_{\mathcal{A}(e_2) \setminus \{g\}} \circ [g : e]) \oplus (\tau(e_1) \hat{\bowtie}_p^{g:e} e_2) & \text{else.} \end{cases}$$

where $g \in \mathcal{A}(e_2)$. Our definition deviates slightly from the standard left outer join operator, as we want to use it in conjunction with grouping and (aggregate) functions. Consider the relations R_1 and R_2 in Figure 7.28. If we want to join R_1 (via a left outerjoin) with e_3 , which is grouped on a_2 , we need to be able to handle empty groups (as for the tuple with $a_1 = 3$ in e_1 in the example). In the definition of the left outerjoin with default, the expression e then defines the value given to the attribute g for all those elements in e_1 that do not find a join partner in e_2 . In our example, it would specify $\hat{\bowtie}^{g=0}$.

We define the dependency join (d-join for short) as

$$e_1 \hat{\ltimes}_{\theta} e_2 := \begin{cases} \epsilon & \text{if } e_1 = \epsilon \\ \alpha(e_1) \hat{\times}_{\theta} e_2(e_1) \oplus \tau(e_1) \hat{\ltimes}_{\theta} e_2 & \text{else} \end{cases}$$

Let $\theta \in \{=, \leq, \geq, <, >, \neq\}$ be a comparison operator on atomic values. The grouping operator which produces a sequence-valued new attribute containing “the group” is defined by using a groupjoin.

$$\hat{\Gamma}_{\theta A;g:f}(e) := \hat{\Pi}_{A:A'}(\hat{\Pi}_{A':A}^D(\hat{\Pi}_A(e)) \hat{\bowtie}_{A'\theta A;g:f} e)$$

where the groupjoin operator (sometimes called nest-join [831]) is defined as

$$e_1 \hat{\bowtie}_{A_1\theta A_2;e:f} e_2 := \begin{cases} \epsilon & \text{if } e_1 = \epsilon \\ \alpha(e_1) \circ [g : G(\alpha(e_1))] \oplus (\tau(e_1) \hat{\bowtie}_{A_1\theta A_2;g:f} e_2) & \text{else} \end{cases}$$

Here, $G(x) := f(\hat{\sigma}_{x|_{A_1}\theta A_2}(e_2))$, and function f assigns a meaningful value to empty groups. See also Figure 7.28 for an example. The unary grouping operator processes a single relation and obviously groups only on those values that are present. The groupjoin works on two relations and uses the left-hand one to determine the groups. This will become important for the correctness of the unnesting procedure.

$e_1 := R_1 \quad e_2 := R_2$			$e_3 := \hat{\Gamma}_{a_2;g:count}(R_2)$		$e_4 := \hat{\Gamma}_{=a_2;g:id}(R_2)$	
a_1	a_2	b	a_2	g	a_2	g
1	1	2	1	2	1	$\langle [1, 2], [1, 3] \rangle$
2	1	3	2	2	2	$\langle [2, 4], [2, 5] \rangle$
3	2	4				
	2	5				
$e_5 := R_1 \hat{\bowtie}_{a_1=a_2;g:id}(R_2)$						
a_1	g					
1	$\langle [1, 2], [1, 3] \rangle$					
2	$\langle [2, 4], [2, 5] \rangle$					
3	$\langle \rangle$					

Figure 7.28: Examples for unary grouping and the groupjoin

Given a tuple with a sequence-valued attribute, we can unnest it using the unnest operator defined as

$$\hat{\mu}_g(e) := \begin{cases} \epsilon & \text{if } e = \epsilon, \\ (\alpha(e)|_{\overline{\{g\}}} \hat{\times} \alpha(e).g) \oplus \hat{\mu}_g(\tau(e)) & \text{else.} \end{cases}$$

where $e.g$ retrieves the sequence of tuples of attribute g . In case that g is empty, it returns the tuple $\perp_{\mathcal{A}(e.g)}$. (In our example in Figure 7.28, $\hat{\mu}_g(e_4) = e_2$.)

We define the unnest map operator as follows:

$$\hat{\Upsilon}_{a:e_2}(e_1) := \hat{\mu}_g(\hat{\chi}_{g:e_2[a]}(e_1)).$$

This operator is mainly used for evaluating XPath expressions. Since this is a very complex issue [338, 340, 416], we do not delve into optimizing XPath evaluation, but instead take an XPath expression occurring in a query as it is and use it in the place of e_2 . Optimized translation of XPath is orthogonal to our unnesting approach and not covered in this paper. The interested reader is referred to [416, 417].

7.16.3 Equivalences

To acquaint the reader with ordered sequences, we state some familiar equivalences that still hold.

$$\hat{\sigma}_{p_1}(\hat{\sigma}_{p_2}(e)) = \hat{\sigma}_{p_2}(\hat{\sigma}_{p_1}(e)), \quad (7.207)$$

$$\hat{\sigma}_p(e_1 \hat{\times} e_2) = \hat{\sigma}_p(e_1) \hat{\times} e_2, \quad (7.208)$$

$$\hat{\sigma}_p(e_1 \hat{\times} e_2) = e_1 \hat{\times} \hat{\sigma}_p(e_2), \quad (7.209)$$

$$\hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2} e_2) = \hat{\sigma}_{p_1}(e_1) \hat{\bowtie}_{p_2} e_2, \quad (7.210)$$

$$\hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2} e_2) = e_1 \hat{\bowtie}_{p_2} \hat{\sigma}_{p_1}(e_2), \quad (7.211)$$

$$\hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2}^{g:e} e_2) = \hat{\sigma}_{p_1}(e_1) \hat{\bowtie}_{p_2}^{g:e} e_2, \quad (7.212)$$

$$\hat{\sigma}_{p_1}(e_1 \hat{\bowtie}_{p_2}^{g:e} e_2) = \hat{\sigma}_{p_1}(e_1) \hat{\bowtie}_{p_2}^{g:e} e_2, \quad (7.213)$$

$$e_1 \hat{\times} (e_2 \hat{\times} e_3) = (e_1 \hat{\times} e_2) \hat{\times} e_3, \quad (7.214)$$

$$e_1 \hat{\bowtie}_{p_1}(e_2 \hat{\bowtie}_{p_2} e_3) = (e_1 \hat{\bowtie}_{p_1} e_2) \hat{\bowtie}_{p_2} e_3, \quad (7.215)$$

$$\hat{\sigma}_p(e_1 \hat{\times} e_2) = e_1 \hat{\bowtie}_p e_2, \quad (7.216)$$

$$e_1 \hat{<} e_2 \hat{>} = e_1 \hat{\times} e_2, \quad (7.217)$$

$$\hat{\Upsilon}_{a:f(\hat{\chi}_b(e))}(\Box) = \hat{\Pi}_{a:b}(f(e)), \quad (7.218)$$

$$\hat{\Upsilon}_{a:e_2}(e_1) = e_1 \hat{\times} \hat{\Upsilon}_{a:e_2}(\Box). \quad (7.219)$$

$$(7.220)$$

Of course, in the above equivalences the usual restrictions hold. For example, if we want to push a selection predicate into the left part of a join, it may not reference attributes of the join's right argument. In other words, $\mathcal{F}(p_1) \cap \mathcal{A}(e_2) = \emptyset$ is required. As another example, Eqv. 7.219 only holds if $\mathcal{F}(e_1) \cap \mathcal{A}(e_1) = \emptyset$. In Eqv. 7.218, the function f may not alter the schema, and b must be an attribute name. Please note that cross product and join are still associative in the ordered context. However, neither of them is commutative. Further, pushing selections into the second argument of a left-outer join is (in general) not possible. For strict predicates we can do better, but this is beyond the scope of the book.

7.16.4 Bibliography

Zaniolo [536]

7.17 Literature

- Bags, Sets, boolean algebras: [626]
- NF²: [4, 204, 431, 746, 747, 546, 748, 547, 778]
- HAS: [130]
- Aggregates: [492, 502]
- SQL to Algebra: [903, 134]

- Calculus to Algebra: [903, 637] Nakano: without duplicates
- BAGs: [21], [221]
- OO Algebra of Steenhagen et al. [831, 832, 830, 833]
- OO Algebra of Cluet and Moerkotte [189, 191].
- OO Algebra [173]
- OO Algebra [188]
- OO Algebra [380]
- OO Algebra [561]
- OO Algebra [760]
- OO Algebra [802, 801, 800, 803]
- OO Algebra [899, 900]
- OO Algebra [964]
- SAL [70]: works on lists. Intended for semistructured data. SAL can be thought of as the order-preserving counterpart of the algebra presented in [189, 191] extended to handle semistructured data. These extensions are similar to those proposed in [5, 181]
- TAX [462]: The underlying algebra's data model is based on sets of ordered labeled trees. Intended for XML.
- XML: Construction: [281, 282]
- no-name [820]: order preserving algebra for OLAP-style queries
- [369]
- Document Processing Algebras: [182, 384]
- Geo: [385]
- Mumick, Pirahesh, and Ramakrishnan introduce a variant of Datalog with bag semantics [633]. Further, they show that the magic set transformation retains the correct number of duplicates.

7.18 **ToDo**

Grouping Mapping and Commutativity Comment on $\Pi_A(R_1 \cap R_2) \not\equiv \Pi_A(R_1) \cap \Pi_A(R_2)$ ToDo
 $\Pi_A(R_1 \setminus R_2) \not\equiv \Pi_A(R_1) \setminus \Pi_A(R_2)$ ToDo

[717]

pregrouping Tsois, Sellis: [882]

bulktypes: Albert: [21]

bulktypes: Dayal: [221]

Chapter 8

Declarative Query Representation

8.1 Calculus Representations

relational calculus originally introduced by Codd: [198, 197].

Variant for embedding in Pascal/R: [776]

calculus for complex objects: [49]

8.2 Datalog

8.3 Tableaux Representation

Tableaus have been introduced by [19, 17, 18] Tableaus are able to represent a particular kind of queries, the so called *conjunctive queries* ([143], [733]).

Expressions containing disjunction (set union) and negation (set difference) can be represented by sets of tableaux ([759],[472]).

query processing with tables: [660]

8.4 Monoid Comprehension

[119, 278, 279]

8.5 Expressiveness

transitivity: [691]. aggregates: [502]. complex object and nested relations: [3].

8.6 Bibliography

Chapter 9

Translation and Lifting

9.1 Query Language to Calculus

9.2 Query Language to Algebra

9.3 Calculus to Algebra

9.4 Algebra to Calculus

9.5 Bibliography

Chapter 10

Query Equivalence, Containment, Minimization, and Factorization

This chapter is devoted to three related problems. Let q be a query and d be a database. We denote by $q(d)$ the result of evaluating the query on the database. Two queries q_1 and q_2 are by definition *equivalent* ($q_1 \equiv q_2$), if $q_1(d) = q_2(d)$ for all databases d . The problem of deciding whether two queries are equivalent is the *query equivalence* problem. We define $q_1 \subseteq q_2$ if $q_1(d) \subseteq q_2(d)$ for all databases d . The problem of deciding whether for two $q_1 \subseteq q_2$ holds is the *query containment* problem. Obviously, $q_1 \equiv q_2 \iff q_1 \subseteq q_2 \wedge q_2 \subseteq q_1$.

The third problem is query minimization. Consider the following conjunctive query written as a rule in datalog:

$$q(X, Y) : -p(X, Y), p(X, Y).$$

Obviously, this query is equivalent to

$$q(X, Y) : -p(X, Y).$$

under set semantics. The latter query now contains fewer body literals. Query minimization now asks for an equivalent query with the least possible number of body literals. One possible approach is to successively delete a body literal until no more body literal can be deleted without violating equivalence to the original query.

The above example is also illustrative since it shows that query equivalence (and thus query containment) differs under different semantics: whereas the above two queries are equivalent under set semantics, they are not under bag semantics. To see this, consider the extensional database $\{p(a, b), p(a, b)\}$. The result of the first query contains $p(a, b)$ four times whereas the last query contains it only 2 times.

10.1 Set Semantics

10.1.1 Conjunctive Queries

A conjunctive query may contain literals where the predicate is built-in. For example, it applies comparison operators $=, <, >, \leq, \geq, \neq$. Any explicit use of equality can be eliminated as follows. For any literal of the form $X = c$, any occurrence of X is replaced by c and the equality literal is dropped from the query clause. For any literal of the form $X = Y$, any occurrence of Y is replaced by X and the equality literal is dropped from the query clause. This procedure is not possible for the other comparison operators $<, >, \leq, \geq, \neq$ which we call *inequality operators*. An *inequality* is any literal using an inequality operator.

Containment and minimization for conjunctive queries without inequalities are NP-complete Problems. First note that a tableau directly corresponds to a conjunctive query with all body literals having a common predicate. From that and the NP-completeness results for tableau containment which in turn follows immediately from first order subsumption [65, 319], it follows immediately that containment of conjunctive queries is NP-complete. Chandra and Merlin proved that minimization is NP-complete [143]. The complexity of checking for equivalence of conjunctive queries is related to graph isomorphism.

EX

The procedure for checking query containment builds upon mappings from queries to queries¹. These mappings have two different names: *homomorphism* and *containment mapping*.

Let q_1 and q_2 be the two queries for which we want to check containment. Assume the q_i are of the form

$$\begin{aligned} q_1 : r_1 & :- l_1, \dots, l_k \\ q_2 : r_2 & :- l'_1, \dots, l'_m \end{aligned}$$

Let $\mathcal{V}(q_i)$ be the set of variables occurring in q_i , and $\mathcal{C}(q_i)$ be the set of constants occurring in q_i . Further, let h be a substitution $h : \mathcal{V}(q_2) \rightarrow (\mathcal{V}(q_1) \cup \mathcal{C}(q_1))$. We call h a *containment mapping* from q_2 to q_1 , if and only if the following conditions are fulfilled:

1. $h(r_2) = r_1$ for the head literals, and
2. for all i ($1 \leq i \leq m$) there exists a j ($1 \leq j \leq k$) such that $h(l'_i) = l_j$.

The latter condition states that for each body literal l'_i in q_2 there is a body literal l_j in q_1 such that $h(l'_i) = l_j$. Note that this does not imply that h is injective or surjective.

The following theorem connects containment mappings with the containment problem:

Theorem 10.1.1 *Let q_1 and q_2 be two conjunctive queries. Then $q_1 \subseteq q_2$ if and only if there is a containment mapping h mapping q_2 to q_1 .*

¹In fact, Chandra and Merlin mapped *natural models* which is essentially the same.

Consider the following example:

$$\begin{aligned} q_1 : p(X_1, X_2) & : - q(X_2, X_1), q(X_1, X_3) \\ q_2 : p(Y_1, Y_2) & : - q(Y_2, Y_1), q(Y_3, Y_1), q(Y_1, Y_4) \end{aligned}$$

Consider h with $h(Y_1) = X_1$, $h(Y_2) = X_2$, $h(Y_3) = X_2$, and $h(Y_4) = X_3$. Then

$$\begin{array}{l|lll} l : & p(Y_1, Y_2) & q(Y_2, Y_1) & q(Y_3, Y_1) & q(Y_1, Y_4) \\ h(l) : & p(X_1, X_2) & q(X_2, X_1) & q(X_2, X_1) & q(X_1, X_3) \end{array}$$

and, hence, $q_1 \subseteq q_2$.

A query q is minimal, if it contains the minimal possible number of body literals. More formally, q is *minimal*, if for any query q' with $q \equiv q'$ the number of body literals in q' greater than or equal to the number of body literals in q . The following theorem shows that our initial thoughts on minimization are correct for conjunctive queries.

Theorem 10.1.2 *Let q be a conjunctive query. Then there is a minimal query q' equivalent to q such that q' results from q by deleting zero or more body literals.*

This suggests a simple procedure for minimizing a given query q . For every body literal check whether some containment mapping h exists such that it is subsumed by some other body literal. Note that this containment mapping must not rename head variables.

Let q and q' be two conjunctive queries. If q can be derived from q' solely by reordering body literals and renaming variables, then q and q' are called *isomorphic*. Minimal queries are unique up to some isomorphism. Obviously, minimizing conjunctive queries is also NP-complete.

Let us now come to unions of conjunctive queries. Let $Q = Q_1 \cup \dots \cup Q_k$ and $Q' = Q'_1 \cup \dots \cup Q'_l$ be two unions of conjunctive queries Q_i and Q'_j with a common head predicate. A containment mapping h from Q to Q' maps each Q_i to some Q'_j such that $h(Q_i) \subseteq Q'_j$. Sagiv and Yannakakis showed the following theorem [759].

Theorem 10.1.3 *Let $Q = Q_1 \cup \dots \cup Q_k$ and $Q' = Q'_1 \cup \dots \cup Q'_l$ be two unions of conjunctive queries Q_i and Q'_j with a common head predicate. Then $Q \subseteq Q'$ if and only if there is a containment mapping from Q to Q' .*

This theorem gives us a corollary which allows us minimizing unions of conjunctive queries by a pairwise checking of containment [759] (see also [887]).

Corollary 10.1.4 *Let $Q = Q_1 \cup \dots \cup Q_k$ be a union of conjunctive queries Q_i with common head predicate. Then there exists a subset R of Q such that*

1. $R \equiv Q$
2. $\neg \exists R' \subset R \ R' \equiv Q$
3. If Q_m is any equivalent to Q , then there is a containment mapping from Q_m to R but none from Q_m to any proper subset R' of R .

This corollary implies that we can minimize a query that is a union of conjunctive queries by eliminating those conjunctive queries Q_i from it that are contained in some Q_j .

For conjunctive queries the problems of containment, equivalence, and minimization are

The problems of containment, equivalence, and minimization of conjunctive queries are most difficult if all body literals have a common predicate p . This is quite an unrealistic assumption as typical conjunctive queries will not only self-join the same relation. A first question is thus whether there exist special cases where there are polynomial algorithms for containment checking. Another strain of work is devoted to more complex queries. As it turns out, the results become less nice and more restricted.

10.1.2 ... with Inequalities

We now turn to conjunctive queries with inequalities in their body. For this section, we assume that the domain is totally ordered and dense. That is, for all x and y with $x < y$, there exists a z with $x < z < y$. In this context, we have the following theorem:

Theorem 10.1.5 *Assume the two conjunctive queries q_1 and q_2 are of the form*

$$\begin{aligned} q_1 : p_1 & : - l_1, \dots, l_k, e_1, \dots, e_l \\ q_2 : p_2 & : - l'_1, \dots, l'_m, e'_1, \dots, e'_n \end{aligned}$$

where p_i are the head literals, l_i and l'_i are ordinary subgoals and e_i and e'_i are inequalities. Let h be a containment mapping from q_2 to q_1 where both are restricted to their ordinary literals. If additionally for all $i = 1, \dots, n$ we have

$$e_1, \dots, e_l \implies h(e'_i)$$

then $q_1 \subseteq q_2$.

This result is due to Klug [503] who used the following procedure to reason about inequalities using comparison operators in $\{=, <, \leq\}$. Given a set of inequalities L , an directed graph G is defined whose nodes are the variables and constants in L . Whenever for all $x < y$ or $x \leq y$ in L , the edge (x, y) is added to G . For all constants c and c' in L , if $c < c'$ then we add an edge (c, c') . Edges are labeled with the according comparison operator. For equality predicates, an edge in both direction is added. Given the graph G , we conclude that $x \leq y$ if there is a path from x to y and $x < y$ only if additionally at least one edge is labelled by $<$. An alternative is to use the procedure presented in Section 11.2.3 to solve the inequality inference problem. It also allows for the comparison operator $\neq$.

To see why a dense domain is important consider the domain of integers. From $1 < x < 3$ we can easily conclude that $x = 2$, a fact we can derive neither from the procedure above nor from the axioms and inference procedure presented in Section 11.2.3.

Unfortunately, the other direction of Theorem ?? is wrong as the following example shows:

$$\begin{aligned} q_1 : p(X_1, X_2) & : - q(X_1, X_2), r(X_3, X_4), r(X_4, X_3) \\ q_2 : p(Y_1, Y_2) & : - q(Y_1, Y_2), r(Y_3, Y_4), Y_3 \leq Y_4 \end{aligned}$$

Obviously, $Y_3 \leq Y_4$ cannot be implied by any non-existing inequalities from q_1 . However, for q_1 to be non-empty, we must have $r(a, b)$ and $r(b, a)$ for some a and b . We also have $a \leq b$ or $b \leq a$. In the former case, we can chose $Y_3 = a$ and $Y_4 = b$ and in the latter $Y_3 = b$ and $Y_4 = a$ to satisfy $r(Y_3, Y_4)$ and $Y_3 \leq Y_4$.

Klug provides an alternative method to solve the containment problem. It builds upon *canonical models*. He then shows that if the containment test succeeds for all canonical models then and only then query containment holds [503]. Klug does not give an explicit algorithm for constructing these canonical models but these can be found in the literature [?]. He also gives two simple subclasses of inequality queries, where constants are allowed only on the left- or only on the right-hand side. For these subclasses if and only if holds in the above Theorem ??.

Although the theorem is stated in terms of conjunctive queries with inequalities, it holds for any predicate p . Assume two queries of the following form:

$$\begin{aligned} q_1 : p_1 & : - l_1, \dots, l_k, P \\ q_2 : p_2 & : - l'_1, \dots, l'_m, P' \end{aligned}$$

where P and P' are arbitrary formulas. If there is a containment mapping from q_2 to q_1 where both are restricted to their ordinary literals and $P \implies h(P')$, then $q_1 \subseteq q_2$.

10.1.3 ... with Negation

The first incarnation of negation we consider is set difference. Here, Sagiv and Yannakakis were the first to derive some results [759].

[201]

10.1.4 ... under Constraints

constraints: [470, 471, 439]

negation+constraints: [270, 271, 272, 273]

10.1.5 ... with Aggregation

[202, 203]

10.2 Bag Semantics

10.2.1 Conjunctive Queries

- definition bag-containment, bag-equivalence [221, 497]
 - characterizations [159, 450, 449] (no proofs in [160])

- complexity results [159]
- definition bag-set containment, bag-set equivalence [159]

10.3 Sequences

10.3.1 Path Expressions

XPath constructs and their short-hands to denote XPath sublanguages.

- branching ('[]')
- wild cards ('*')
- descendant axis ('//')
- disjunction : only binary *or* branching, ('—') (or-branching)

Otherwise XPath only contains the child axis and node name tests. These sublanguages are represented as tree patterns.

Query containment for certain subclasses:

- $XP^{\square,*,//}$ is coNP-complete [606]

Consider we have to answer $p \subseteq p'$. Then

- if $p \in P^{\square,//}$ and $p \in P^{\square,*,//}$ then query containment is coNP-complete.
- in PTIME if number of '/' is restricted by d which then gives the degree of the polynomial describing the time complexity
- remains coNP-complete if p contains no '*' p' contains at most two '*'_S
- remains coNP-complete if p contains at most 5 branches and p' contains at most 3 branches.
- $P^{\square,*}$ is in PTIME (follows from work on conjunctive acyclic queries [951], also noted by Wood [934])
- $P^{\square,//}$ is in PTIME [28]
- $P^{*,//}$ is in PTIME (these are related to a fragment of regular expressions [607])
- P^{or} is in PTIME
- $P^{\square,or}$ is coNP-complete
- $P^{\square}$ is coNP-complete [606]
[645] showed that $P^{\square,*,//,\square}$ is coNP-complete for infinite alphabets and in PSPACE for finite alphabets.

- $P^{//,|}$ is PSPACE-complete
- $P^{\square,*,//}$ with variable binding and equality tests is Π_2^P -hard [235]

A PTIME algorithm for the fragment $P^{//}$ can be found in [118]

Florescu, Levy, and Suciu showed that for a language quite similar to $P^{\square,//}$ containment is NP-complete if evaluated on a graph-based data model instead of a tree-based one [290].

Calvanese et al. also consider a graph-based data model and more expressive queries [122].

[645] also contains work on languages with variable bindings with different semantics.

More result: [235]

query containment with DTDs: [935]

Schwentick gives a very good overview over complexity results for containment checking [779]. We should repeat his table here.

10.4 Minimization

minimization: [522]

10.5 Detecting common subexpressions

[283, 393, 391]

10.5.1 Simple Expressions

Simple Non-Expensive Expressions

Simple Expensive Expressions

10.5.2 Algebraic Expressions

10.6 Bibliography

In a pair of papers Aho, Sagiv, and Ullmann [17, 18] study equivalence, containment, and minimization problems for tableaux. More specifically, they introduce a restricted variant of relational expressions containing projection, natural join, and selection with predicates that only compare attributes with constants. They further assume the existence of a universal relation. That is, every relation R is the projection of the universal relation on $\mathcal{A}(R)$. Now, these restricted conjunctive queries can be expressed with tableaux. The authors tableaux equivalence, containment, and minimization problems also in the presence of functional dependences. The investigated problems are all NP-complete. Since the practical usefulness is limited we do not give the concrete results of this pair of papers.

[158, 161] contains (complexity) results for deciding query equivalence in the case of recursive and nonrecursive datalog.

View selection problem (just pointers): ...

[468, 464, 465]

[167]

- conjunctive queries: equivalence and minimization are NP-complete [143, 18]
In [18] tableaux are used.
- polynomial algorithms for equivalence and minimization for simple tableaux: [18, 17]
- union of elementary differences: Π_2^P complete: remark in [759] and a pointer to the thesis of Sagiv [756].
- acyclic conjunctive queries: PTIME [951]
- equivalence $(\sigma, \bowtie, \pi, \cup)$, equivalence $(\sigma, \bowtie, \pi, \cup, \setminus)$: Π_2^P -complete [759]
- Recursive Datalog: [100]

Part III

Rewrite Techniques

Chapter 11

Simple Rewrites

11.1 Simple Adjustments

11.1.1 Rewriting Simple Expressions

Constant Folding

Constant subexpressions are evaluated and the result replaces the subexpression. For example an expression $1/100$ is replaced by 0.01 . Other expressions like $a - 10 = 50$ can be rewritten to $a = 60$. However, the latter kind of rewrite is rarely performed by commercial systems.

Eliminate BETWEEN

A predicate of the form $Y \text{ BETWEEN } X \text{ AND } Z$ is replaced by $X \leq Y \text{ AND } Y \leq Z$. This step not only eliminates syntactic sugar but also enables transitivity reasoning to derive new predicates (see).

Eliminate IN

A predicate of the form $x \text{ IN } (c_1, \dots, c_n)$ is rewritten to $x = c_1 \text{ OR } \dots \text{OR } x = c_n$. This eliminates on form of the IN predicate and enables multikey index access.

Another possibility is to use a table function that produces a table with one column whose values are exactly those in the IN-list. From thereon, regular optimization takes place. This possibility is also investigated when several comparisons of a column with a constants are disjunctively connected.

Eliminating LIKE

A predicate of the form $a \text{ LIKE 'Guy'}$ can only be rewritten to $a = \text{'Guy'}$ if a is of type varchar. This is due to the different white space padding rules for LIKE and =.

Start and Stop conditions derived from LIKE predicates

A predicate of the form $a \text{ LIKE 'bla\%'}$ gives rise to a start condition $a \geq \text{'bla'}$. Which can enable subsequent index usage. A stop predicate of the form $a < \text{'blb'}$ can also be derived. completing a range predicate for an index scan. Start and stop conditions can only be derived if there is no leading '%' in the pattern.

Pushing NOT operations down and eliminating them

NOT operations need to be pushed downwards for correctness reasons. Attention has to be paid to the IS NOT NULL and IS NULL predicates. XXX complete set of rules go into some table.

Merge AND, OR, and other associative operations

While parsing, AND and OR operations are binary. For simpler processing they are often n-ary in the internal representation. Therefore $(p \text{ AND } (q \text{ AND } r))$ is rewritten to $(\text{AND } p \ q \ r)$.

In general, associative nested operations should be merged. Examples of other associative operations are $+$ and $*$.

Normalized Argument Order for Commutative Operations

ToDo

enabling factorization, constant folding: move constants to the left Speed up evaluation of *equal*.

Eliminate - and /

$$(x - y) \leadsto x + (-y) \quad x/y \leadsto x * (1/y)$$

Adjust join predicates

$A = B + C$ becomes $A - C = B$ if A and B are from one relation and C is from another.

Simplifying boolean expressions

The usual simplification rules for boolean expressions can be applied. For example, if a contradiction can be derived.

Eliminating ANY, SOME, and ALL

ANY and SOME operators in conjunction with a comparison operator are rewritten into disjunction of comparison predicates. For example $a > \text{ANY}(c_1, c_2)$ is rewritten to $a > c_1 \text{ OR } a > c_2$. Correspondingly, an ALL operator with a constant list is rewritten into a conjunction of comparisons. For example, $a > \text{ALL}(c_1, c_2)$ is rewritten to $a > c_1 \text{ AND } a > c_2$.

If a subquery occurs, then the ANY or SOME expression is rewritten to a correlated subquery in an EXIST predicate. Consider the query $a > \text{ANY}$

(SELECT b FROM ...WHERE p). It is rewritten to EXISTS(SELECT ...FROM ...WHERE p AND a > b).

Correspondingly, a subquery within an ALL operator is rewritten into a NOT EXISTS subquery. For example, a > (SELECT b FROM ...WHERE p) is rewritten into NOT EXISTS (SELECT b FROM ...WHERE p and a <= b)

- CASE j==i UNION

11.1.2 Normal forms for queries with disjunction

Another step of the NFST component or the first step of the rewriting component can be the transformation of boolean expressions found in *where* clauses in order to account for NULL values. Pushing **not** operators inside the boolean expression allows to use *two-valued logic* instead of *three-valued logic*. If we miss this step, we can get wrong results.

Another possible step is the subsequent transformation of the boolean expressions in **where** clauses into disjunctive normal form (DNF) or conjunctive normal form (CNF). This step is not always necessary and really depends on which plan generation approach is taken. Hence, this step could take place as late as in a preparatory step for plan generation. It is (obviously) only necessary if the query contains disjunctions. We discuss plan generation for queries with disjunctions in Section ??.

11.2 Deriving new predicates

Given a set of conjunctive predicates, it is often possible to derive new predicates which might be helpful during query plan generation.

This section discusses ways to infer new predicates.

11.2.1 Collecting conjunctive predicates

A query predicate may not only contain the **and** connector, but also **or** or **not**.

For the inference rules in this section we need base predicates that *occur conjunctively*.

We say that a (base) predicate q occurs conjunctively in a (complex) predicate p if $p [q \leftarrow false]$ can be simplified to *false*. That is, if we replace every occurrence of q by *true*, the simplification rules in Figure 11.1 (Fig. ??) simplify $p [q \leftarrow true]$ to *false*.

These simplification rules can be used to implement a simple member function *occursConjunctively* to determine whether a predicate occurs conjunctively in a predicate or not. Together with a member function or visitor *Collect-BasePredicates*, we can compute the set of conjunctively occurring predicates. This set will form the basis for the next subsections.

11.2.2 Equality

Equality is a reflexive, symmetric and transitive binary relationship (see Fig. 11.2). Such a relation is called an *equivalence relation*. Hence, a set of conjunctively

$$\begin{aligned}
NOT\ true &\rightarrow false \\
NOT\ false &\rightarrow true \\
p\ AND\ true &\rightarrow p \\
p\ AND\ false &\rightarrow false \\
p\ OR\ true &\rightarrow true \\
p\ OR\ false &\rightarrow p
\end{aligned}$$

Figure 11.1: Simplification rules for boolean expressions

$$\begin{aligned}
&x = x \\
x = y &\implies y = x \\
x = y \wedge y = z &\implies x = z
\end{aligned}$$

Figure 11.2: Axioms for equality

occurring equality predicates implicitly partitions the set of composed terms (IUs) into disjunctive equivalence classes.

Constants: Let X be an equivalence class of equal expressions. Let Y be the set of all equality expressions that contributed to X . Then, in the query predicate we replace all expressions $x = y$ by $x = c$ and $y = c$ and subsequently eliminate redundant expressions.

$$\sigma_{x=c}(e_1 \bowtie_{x=y} e_2) \equiv \sigma_{x=c}(e_1) \times \sigma_{y=c}(e_2)$$

replace all predicates by $IU=C.IU$'s equivalent to a constant In [208] an abstract data structure is presented that helps computing the equivalence classes fast and also allows for a fast check whether two terms (IUs) are in the same equivalence class. Since we are often interested in whether a given IU is equal to a constant - or, more specifically, equal to another IU bound to a constant -, we have to modify these algorithms such that the IU bound to a constant, if it exists, becomes the representative of its equivalence class.

For the member functions *addEqualityPredicate*, *getEqualityRepresentative* and *isInSameEqualityClass* we need an attribute *_equalityRepresentative* in class IU that is initialized such that it points to itself. Another member *_equalityClassRank* is initialized to 0. The code for the two member functions is given in Figure 11.3.

By calling *addEqualityPredicate* for all conjunctively occurring equality predicates we can build the equivalence classes.

11.2.3 Inequality

Table 11.1 gives a set of axioms used to derive new predicates from a set of conjunctively occurring inequalities S (see [887], see Fig. 11.4).

These axioms have to be applied until no more predicates can be derived. The following algorithm [887] performs this task efficiently:

1. Convert each $X < Y$ into $X \neq Y$ and $X \leq Y$.
2. Compute the transitive closure of $\leq$.
3. Apply axiom A8 until no more new predicates can be derived.
4. Reconstruct $<$ by using axiom A4.

Step 3 can be performed as follows. For any true IUs X and Y we find these IUs Z with $X \leq Z \leq Y$.

Then we check whether any two such Z 's are related by $\neq$. Here, it is sufficient to check the original $\neq$ pairs in S and these derived in 1.

A1	:	$X \leq X$
A2	:	$X < Y \Rightarrow X \leq Y$
A3	:	$X < Y \Rightarrow X \neq Y$
A4	:	$X \leq Y \wedge X \neq Y \Rightarrow X < Y$
A5	:	$X \neq Y \Rightarrow Y \neq X$
A6	:	$X < Y \wedge Y < Z \Rightarrow X < Z$
A7	:	$X \leq Y \wedge Y \leq Z \Rightarrow X \leq Z$
A8	:	$X \leq Z \wedge Z \leq Y \wedge X \leq W \wedge W \leq Y \wedge W \neq Z \Rightarrow X \neq Y$

Table 11.1: Axioms for inequality

11.2.4 Aggregation

Let $R_1, \dots, R_n$ be relations or views, $A_1, \dots, A_m$ attributes thereof, p_w and p_h predicates, and $a_1, \dots, a_l$ expressions of the form $f_j(B_j)$ for aggregate functions f_j and attributes B_j . For a query block of the form

```

select   $A_1, \dots, A_k, a_1, \dots, a_l$ 
from     $R_1, \dots, R_n$ 
where    $p_w$ 
group by  $A_1, \dots, A_m$ 
having   $p_h$ 

```

we consider the derivation of new predicates [551]. Obviously, the following predicates are true:

$$\begin{aligned}\min(B) &\leq B \\ \max(B) &\geq B \\ \max(B) &\geq \min(B) \\ \min(B) &\leq \text{avg}(B) \\ \text{avg}(B) &\leq \max(B)\end{aligned}$$

If p_w contains conjunctively a predicate $B\theta c$ for some constant c , we can fur-

ther infer $\min(B) \theta c$ if $\theta \in \{>, \geq\}$
 $\max(B) \theta c$ if $\theta \in \{<, \leq\}$ These predicates can then
 $\text{avg}(B) \theta c$ if $\theta \in \{<, \leq, >, \geq\}$

be used to derive further predicates. The original and the derived predicates are useful when the query block is embedded in another query block since we are allowed to add them to the embedding query block conjunctively (see Section 12.3).

If we know restrictions on the aggregates from some embedding query block, we might be able to add predicates to p_w . The following table contains the restrictions on an aggregate we know in the left column and the predicates we can

	$\max(B) \geq c \rightsquigarrow B \geq c$	if no other aggregation occurs
infer in the right column:	$\max(B) > c \rightsquigarrow B > c$	if no other aggregation occurs
	$\min(B) \leq c \rightsquigarrow B \leq c$	if no other aggregation occurs
	$\min(B) < c \rightsquigarrow B < c$	if no other aggregation occurs

Note that the aggregation occurring in the left column must be the only aggregation found in the query block. That is, $l = 1$ and p_h contains no aggregation other than a_1 . To see why this is necessary, consider the following query

```
select deptNo, max(salary), min(salary)
from Employee
group by deptNo
```

Even if we know that $\max(\text{salary}) > 100.000$, the above query block is not equivalent to

```
select deptNo, max(salary), min(salary)
from Employee
where salary > 100.000
group by deptNo
```

Neither is

```
select deptNo, max(salary)
from Employee
group by deptNo
having avg(salary) > 50.000
```

equivalent to

```
select deptNo, max(salary)
from Employee
where salary < 100.000
group by deptNo
having avg(salary) < 50.000
```

even if we know that `max(salary) > 100.000`.

11.2.5 ToDo

[579]

11.3 Predicate Push-Down and Pull-Up

11.4 Eliminating Redundant Joins

11.5 Distinct Pull-Up and Push-Down

11.6 Set-Valued Attributes

In this section, we investigate the effect of query rewriting on joins involving set-valued attributes in object-relational database management systems. We show that by unnesting set-valued attributes (that are stored in an internal nested representation) prior to the actual set containment or intersection join we can improve the performance of query evaluation by an order of magnitude. By giving example query evaluation plans we show the increased possibilities for the query optimizer. This section is based on [423].

11.6.1 Introduction

The growing importance of object-relational database systems (ORDBMS) [841] has kindled a renewed interest in the efficient processing of set-valued attributes. One particular problem in this area is the joining of two relations on set-valued attributes [315, 420, 713]. Recent studies have shown that finding optimal join algorithms with set-containment predicates is very hard [121]. Nevertheless, a certain level of efficiency for joins on set-valued attributes is indispensable in practice.

Obviously, brute force evaluation via a nested-loop join is not going to be very efficient. An alternative is the introduction of special operators on the physical level of a DBMS [420, 713]. Integration of new algorithms and data structures on the physical level is problematic, however. On one hand this approach will surely result in tremendous speed-ups, but on the other hand this efficiency is purchased dearly. It is very costly to implement and integrate new algorithms robustly and reliably.

We consider an alternative approach to support set-containment and non-empty intersection join queries by compiling these join predicates away. The main idea is to unnest the set-valued attributes prior to the join. Thereby, we assume a nested internal representation [712]. This is also the underlying representation for the specific join algorithms proposed so far [420, 713]. Whereas [713] concentrates on set-containment joins, we also consider joins based on non-empty intersections. Ramasamy et al. also present a query rewrite for containment queries in [713], but on an unnested external representation, which (as shown there) exhibits very poor performance. Further, the special case of empty sets was not dealt with.

The goal of our paper is to show that by rewriting queries we can compile away the original set-containment or intersection join. As our experiments with DB2 show, our rewrite results in speed-up factors that grow linearly in the size of the input relations as compared to quadratic growth for brute-force nested-loop evaluation. The advantage of this approach—as compared to [420, 713]—is that no new join algorithms have to be added to the database system.

11.6.2 Preliminaries

In this section we give an overview of the definition of the set type. Due to the deferral of set types to SQL-4 [291], we use a syntax similar to that of Informix¹. A possible example declaration of a table with a set-valued attribute is:

```
create table ngrams (
    setID    integer not null primary key,
    content  set<char(3)>
);
```

setID is the key of the relation, whereas **content** stores the actual set. The components of a set can be any built-in or user-defined type. In our case we used **set<char(3)>**, because we wanted to store 3-grams (see also Section ??). We further assume that on set-valued attributes the standard set operations and comparison operators are available.

Our rewriting method is based on unnesting the internal nested representation. The following view defining the unnested version of the above table keeps our representation more concise:

```
create view view_ngrams(setID, d, card) as (
    (select ngrams.setID, d.value, count(ngrams.content)
     from ngrams, table(unnest<char(3)>(ngrams.content)) d)
    union all
    (select ngrams.setID, NULL, 0)
     from ngrams
     where count(ngrams.content) = 0)
);
```

¹<http://www.informix.com/documentation/>

where `setID` identifies the corresponding set, `d` takes on the different values in `content` and `card` is the cardinality of the set. We also need `unnest<char(3)>`, a table function that returns a set in the form of a relation. As `unnest<char(3)>` returns an empty relation for an empty set, we have to consider this special case in the second subquery of the union statement, inserting a tuple containing a dummy value.

11.6.3 Query Rewrite

We are now ready to describe the queries we used to compare the nested and unnested approach. We concentrate on joins based on subset-equal and non-empty intersection predicates, because these are the difficult cases as shown in [121]. We have skipped joins involving predicates based on equality, because the efficient evaluation of these predicates is much simpler and can be done in a straightforward fashion (see [420]).

Checking Subset Equal Relation

Here is a query template for a join based on a subset-equal predicate:

```
select n_1.setID, n_2.setID
from   ngrams n_1, ngrams n_2
where  is_subseteq(n_1.content, n_2.content) <> 0;
```

(The comparison with 0 is only needed for DB2, which does not understand the type bool.)

This query can be rewritten as follows. The basic idea is to join the unnested version of the table based on the set elements, group the tuples by their set identifiers, count the number of elements for every set identifier and compare this number with the original counts. The filter predicate `vn1.card <= vn2.card` discards some sets that cannot be in the result of the set-containment join. We also consider the case of empty sets in the second part of the query. Summarizing the rewritten query we get

```
(select vn1.setID, vn2.setID
 from   view_ngrams vn1, view_ngrams vn2
 where  vn1.d = vn2.d
 and    vn1.card <= vn2.card
 group by vn1.setID, vn1.card, vn2.setID, vn2.card
 having count(*) = vn1.card)
union all
(select vn1.setID, vn2.setID
 from   view_ngrams vn1, view_ngrams vn2
 where  vn1.card = 0);
```

Checking Non-empty Intersection

Our query template for joins based on non-empty intersections looks as follows.

```
select n_1.setID, n_2.setID
from   ngrams n_1, ngrams n_2
where  intersects(n_1.content, n_2.content) <> 0;
```

The formulation of the unnested query is much simpler than the unnested query in Section 11.6.3. Due to our view definition, not much rewriting is necessary. We just have to take care of empty sets again, although this time in a different, simpler way.

```
select distinct vn1.setID, vn2.setID
from   view_ngrams vn1, view_ngrams vn2
where  vn1.d = vn2.d
and    vn1.card > 0;
```

11.7 Bibliography

This section is based on the investigations by Helmer and Moerkotte [423]. There, we also find a performance evaluation indicating that the rewrites depending on the relation sizes result in speed-up factors between 5 and 50 even for moderately sized relations. Nevertheless, it is argued there, that support for set-valued attributes must be build into the DBMS. A viable alternative to the rewrites presented here is the usage of special join algorithms for join predicates involving set-valued attributes [315, 419, 420, 581, 601, 602, 713]. Nevertheless, as has been shown by Cai, Chakaravarthy, Kaushik, and Naughton, dealing with set-valued attributes in joins theoretically (and of course practical) difficult issue [121]. Last, to efficiently support simple selection predicates on set-valued attributes, special index structures should be incorporated into the DBMS [421, 422, 424].

```

IU::addEqualityClassUnderThis(IU* lIU){
    IU*lRepresentativeThis = this -> getEqualityRepresentativeIU;
    IU*lRepresentativeArg = aIU -> getEqualityRepresentativeIU;

    lRepresentativeArg -> _equalityRepresentative =
    lRepresentativeThis;
    if(lRepresentativeArg -> _equalityClassRank >=
        lRepresentativeThis -> _equalityClassRank){
        lRepresentativeThis -> _equalityClassRank =
        lRepresentativeArg -> _equalityClassRank + 1;
    }
}

IU::addEqualityPredicate(Compositing* p){
    IU*lLeft = p -> leftIU;
    IU*lRight = p -> rightIU;
    if (p -> isEqualityPredicateIU &&
        lLeft -> getEqualityRepresentativeIU ==
        lRight -> getEqualityRepresentativeIU){
        if(lLeft -> isBoundToConstantIU) {
            lLeft -> addEqualityClassUnderThis(lRight);
        }else
        if(lRight -> isBoundToConstantIU){
            lRight -> addEqualityClassUnderThis(lLeft),
        }else
        if (lLeft -> _equalityClassRank > lRight ->
            _equalityClassRank){
            lLeft -> addEqualityClassUnderThis(lRight)
        }else{
            lright -> addEqualityClassUnderThis(lLeft)
        }
    }
}

IU* IU:: getEqualityRepresentativeIU(){
    if (this == _equalityRepresentative){
        _equalityRepresentative = _equalityRepresentative ->
        getEqualityRepresentativeIU;
    }
    return _equalityRepresentative;
}

```

Figure 11.3:

A1		$X \leq X$
A2	$X < Y \Rightarrow$	$X \leq Y$
A3	$X < Y \Rightarrow$	$X \neq Y$
A4	$X \leq Y \wedge X \neq Y \Rightarrow$	$X < Y$
A5	$X \neq Y \Rightarrow$	$Y \neq X$
A6	$X < Y \wedge Y < Z \Rightarrow$	$X < Z$
A7	$X \leq Y \wedge Y \leq Z \Rightarrow$	$X \leq Z$
A8	$X \leq Z \wedge Z \leq Y \wedge X \leq W \wedge W \leq Y \wedge W \neq Z \Rightarrow$	$X \neq Y$

Figure 11.4: Axioms for inequality

Chapter 12

View Merging

12.1 View Resolution

View merging can be as simple as replacing the view name in the **from** clause of a query by the view definition. We would like to call this step *view resolution*. This then results in a query with nesting in the **from** clause that can subsequently be unnested (see ??). Consider the following example: XXX Example Other examples are given below. One must be careful not to produce variable clashes. Especially if a view is referenced several times, variables must be renamed.

12.2 Simple View Merging

Of course, these two steps can be merged into one step. The overall effect is then that the view name is replaced by all the entries in the **from** clause of the view definition and the predicate contained in the **where** clause of the view definition is conjunctively added to **where** clause of the query block whose **from** clause contained the view name. Consider the following view definition

create view

which is referenced in the following query:

View merging results in

However, there are a few pitfalls. This simple version of view merging can only be applied to simple select-project-join queries not containing duplicate elimination, set operations, grouping or aggregation. In these cases, complex view merging must be applied.

12.3 Predicate Move Around (Predicate pull-up and push-down)

If unnesting is not implemented or not possible, several techniques like predicate move around, semi-join techniques and magic rewriting allow the copying of predicates from one block into another block in order to reduce the number of qualifying tuples [551, 631, 632, 633, 791].

Let us briefly illustrate the main idea by means of a simple example query

```

select  e.name
from    Employee e,
        (select  d.name, d.dno
         from    Department d
         where   d.dno = e.dno and
                d.boss.name = e.name and
                d.boss.name like '%S') as D(dname,ddno)
where   e.dno between 1 and 10

```

which can be rewritten by predicate move around to

```

select  e.name
from    Employee e,
        (select  d.name, d.dno
         from    Department d
         where   d.dno = e.dno and
                d.boss.name = e.name and
                d.dno between 1 and 10 and
                d.boss.name like '%S') as D(dname,dd no)
where   e.dno between 1 and 10 and
        e.name like '%S'

```

Predicate push-down and pull-up often occurs in conjunction with views. Let us therefore consider some examples. The following view that cannot be simply merged because it contains a **union** operator. Consider the case where there are two different employee tables that are unioned in a view.

```

create view Emp(eno, name, salary, dno) as
select  e1.eno, e1.name, e1.salary, e1.dno
from    Emp1[e1]
union all
select  e2.eno, e2.name, e2.salary, e2.dno
from    Emp2[e2]

```

Simple view merging cannot be applied to the query

```

select  e.eno, e.name
from    Emp[e]
where   e.salary > 150000

```

but view resolution with a subsequent push-down of the predicate `e.salary > 150.000` will result in

```

select  e.eno, e.name
from    ( select   e1.eno, e1.name, e1.salary, e1.dno
           from     Emp1[e1]
           where    e1.salary > 150000)
union all select   e2.eno, e2.name, e2.salary, e2.dno
           from     Emp2[e2]
           where    e2.salary > 150000)

```

Note that we did not eliminate unneeded columns/attributes. Further note that we can now exploit possible indexes on `Emp1.salary` and `Emp2.salary`. In case **union** would have been used in the view definition, the rewritten query would also contain **union** requiring a duplicate elimination.

Here is another example where pushing a predicate down results in much more efficient plans. Given the view

```

define view EmpStat as
select  e.dno, min(e.salary) minSal, max(e.salary) maxSal, avg(e.salary) avgSal
from    Emp[e]
group by e.dno

```

the query

```

select  *
from    EmpStat[e]
where    e.dno = 10

```

can be rewritten to

```

select  e.dno, min(e.salary) minSal, max(e.salary) maxSal, avg(e.salary) avgSal
from    Emp[e]
where    e.dno = 10
group by e.dno

```

which can be further simplified to

```

select  e.dno, min(e.salary) minSal, max(e.salary) maxSal, avg(e.salary) avgSal
from    Emp[e]
where    e.dno = 10

```

12.4 Complex View Merging

12.4.1 Views with Distinct

XXX TODO views with distinct

12.4.2 Views with Group-By and Aggregation

Consider the following view with a group-by clause and aggregation:

```
create view AvgSalary as
select  e.dno, avg(e.salary) as avgSalary
from    Emp[e]
group by e.dno
```

The following query uses this view:

```
select  d.name, s.avgSalary)
from    Dept[d], AvgSalary[s]
where   d.location = 'Paris' and
        d.dno = s.dno
```

Using the view definition, this query can be rewritten to

```
select  d.name, avg(e.salary) as avgSalary
from    Dept[d], Emp[e]
where   d.location = 'Paris' and
        d.dno = e.dno
group by d.ROWID, d.name
```

where `d.ROWID` is either a key-attribute like `d.dno` or a unique row identifier of the tuples in `Dept`. Of course, this transformation is not valid in general. The primary condition here is that we have a key-foreign key join. More specifically, `d.dno` must be the key of the `Dept` table or it must be a unique attribute.

Applying simple view resolution results in:

```
select  d.name, s.avgSalary)
from    Dept[d], (select  e.dno, avg(salary) as avgSalary
                  from    Emp[e]
                  group by e.dno) [s]
where   d.location = 'Paris' and
        d.dno = s.dno
```

This query can then be unnested using the techniques of Section ??.

Sometimes strange results occur. Consider for example the view

```
define view EmpStat as
select  e.dno, min(e.salary) minSal, max(e.salary) maxSal, avg(e.salary) avgSal
from    Emp[e]
group by e.dno
```

If the user issues the query

```

select avg(minSal), avg(maxSal), avg(avgSal)
from EmpStat

```

view merging results in

```

select  avg(min(e.salary)), avg(max(e.salary)), avg(avg(e.salary))
from    Emp[e]
group by e.dno

```

This is perfectly o.k. You just need to think twice about it. The resulting plan will contain two group operations: XXX Plan

12.4.3 Views in IN predicates

Consider a view that contains the minimum salary for each department

```

create view MinSalary as
select  e.dno, min(e.salary) as minSalary
from    Emp[e]
group by e.dno

```

and a query asking for all those employees together with their salaries in Parisian departments earning the minimum salary:

```

select  e.name, e.salary
from    Emp[e], Dept[d]
where   e.dno = d.dno and
          d.location = 'Paris' and
          (e.dno, e.sal) in MinSalary

```

This query can be rewritten to:

```

select  e.name, e.salary
from    Emp[e], Dept[d], Emp[e2]
where   e.dno = d.dno and
          d.location = 'Paris' and
          e.dno = e2.dno
group by e.ROWID, d.ROWID, e.name, e.salary
having  e.salary = min(e2.sal)

```

Note that the employee relation occurs twice. Avoiding to scan the employee relation twice can be done as follows:

12.4.4 Final Remarks

Not all views can be merged. If for example a `rownum` function that numbers rows in a table is used in a view definition for a result column, then the view cannot be merged. Unmerged views will remain as nested subqueries with

two alternative evaluation strategies: Either they will be evaluated as nested queries, that is for every row produced by some outer producer the view is evaluated, or the view will be materialized into a temporary table. Whatever is more efficient must be chosen by the plan generator. However, techniques for deriving additional predicates and subsequent techniques such as predicate move around (predicate pull-down, push-down) are still applicable.

12.5 Bibliography

Chapter 13

Quantifier treatment

13.1 Pseudo-Quantifiers

Again, the clue to rewrite subqueries with a **ANY** or **ALL** predicate is to apply aggregate functions [314]. A predicate of the form

```
< ANY (select  ...
        from    ...
        where   ...)
```

can be transformed into the equivalent predicate

```
< (select  max(...)
    from    ...
    where   ...)
```

Analogously, a predicate of the form

```
< ALL (select  ...
        from    ...
        where   ...)
```

can be transformed into the equivalent predicate

```
< (select  min(...)
    from    ...
    where   ...)
```

In the above rewrite rules, the predicate $<$ can be replaced by $=$, $\leq$, etc. If the predicate is $>$ or $\geq$ then the above rules are flipped. For example, a predicate of the form $>\mathbf{ANY}$ becomes $>\mathbf{select\ min}$ and $>\mathbf{ALL}$ becomes $>\mathbf{select\ max}$.

After the rewrites have been applied, the Type A or Type JA unnesting techniques can be applied, depending on the details of the inner query block.

13.2 Existential quantifier

Existential quantifiers can be seen as special aggregate functions and query blocks exhibiting an existential quantifier can be unnested accordingly [220]. For example, an independent existential subquery can be treated the same way as a Type A query. Nested existential quantifiers with a correlation predicate can be unnested using a semi-join. Other approaches rewrite (existential) quantifiers using the aggregate function *count* [314]. Consider the partial query pattern

```
...
where exists (select ...
from          ...
where          ...)
```

It is equivalent to

```
...
where 0 > (select  count(...)
from          ...
where          ...)
```

A **not exists** like in

```
...
where not exists (select ...
from          ...
where          ...)
```

is equivalent to

```
...
where 0 = (select  count(...)
from          ...
where          ...)
```

After these rewrites have been applied, the Type A or Type JA unnesting techniques can be applied, depending on the details of the inner query block.

13.3 Universal quantifier

Universal quantification is a little more complex. An overview is provided in [184]. Here is the prototypical OQL query pattern upon which our discussion

Case-No.	1	2	3	4	5	6	7	8
	$p()$ $q()$	$p()$ $q(e_1)$	$p()$ $q(e_2)$	$p()$ $q(e_1, e_2)$	$p(e_1)$ $q()$	$p(e_1)$ $q(e_1)$	$p(e_1)$ $q(e_2)$	$p(e_1)$ $q(e_1, e_2)$
Case-No.	9	10	11	12	13	14	15	16
	$p(e_2)$ $q()$	$p(e_2)$ $q(e_1)$	$p(e_2)$ $q(e_2)$	$p(e_2)$ $q(e_1, e_2)$	$p(e_1, e_2)$ $q()$	$p(e_1, e_2)$ $q(e_1)$	$p(e_1, e_2)$ $q(e_2)$	$p(e_1, e_2)$ $q(e_1, e_2)$

Table 13.1: Classification Scheme According to the Variable Bindings

of universal quantifiers nested within a query block is based:

$$\begin{aligned}
 Q \equiv & \text{select } e_1 \\
 & \text{from } e_1 \text{ in } E_1 \\
 & \text{where for all } e_2 \text{ in select } e_2 \\
 & \quad \text{from } e_2 \text{ in } E_2 \\
 & \quad \text{where p:} \\
 & \quad \quad q
 \end{aligned}$$

where p (called the *range predicate*) and q (called the *quantifier predicate*) are predicates in a subset of the variables $\{e_1, e_2\}$. This query pattern is denoted by Q .

In order to emphasize the (non-)occurrence of variables in a predicate p , we write $p(e_1, \dots, e_n)$ if p depends on the variables $e_1, \dots, e_n$. Using this convention, we can list all the possible cases of variable occurrence. Since both e_1 and e_2 may or may not occur in p or q , we have to consider 16 cases (see Table 13.1). All cases but 12, 15, and 16 are rather trivial. Class 12 queries can be unnested by replacing the universal quantifier by a division, set difference, anti-semijoin, or counting. Class 15 queries are treated by set difference, anti-semijoin or grouping with count aggregation. For Class 16 queries, the alternatives are set difference, anti-semijoin, and grouping with count aggregation. In all cases, special care has to be taken regarding NULL values. For details see [184].

Class 12 Let us first consider an example of a Class 12 query.

```

select  al.name
from    al in Airline
where  for all ap in (select  ap
                        from    ap in Airport
                        where   apctry = 'USA'):
      ap in al.lounges

```

Define $U \equiv \pi_{ap}(\sigma_{apctry='USA'}(Airport[ap, apctry]))$. Then the three alternative algebraic expressions equivalent to this query are

- plan with division:
if $U = \emptyset$

then $\text{Airline}[\text{name}]$
else $\mu_{ap:\text{lounges}}(\text{Airline}[\text{name}, \text{lounges}]) \div U$

- plan with set difference:

$$\text{Airline}[\text{name}] \setminus (\pi_{\text{name}}(U \bowtie_{ap \notin \text{lounges}} \text{Airline}[\text{name}, \text{lounges}]))$$

- plan with anti-semijoin:

$$\pi_{\text{name}}(U \Join_{ap \notin \text{lounges}} \text{Airline}[\text{name}, \text{lounges}])$$

This plan is only valid, if the projected attributes of *Airline* form a superkey.

The plan with the anti-semijoin is typically the most efficient.

In general, the plan with division is [637, 355]:

$$if_{\sigma_{p(e_2)}(E_2[e_2]) \neq \emptyset}(((E_1[e_1] \bowtie_{q(e_1, e_2)} E_2[e_2]) \div \sigma_{p(e_2)}(E_2[e_2])), E_1[e_1])$$

In case the selection $\sigma_{p(e_2)}(E_2[e_2])$ yields at least a one tuple or object, we can apply the predicate p to the dividend, as in

$$if_{\sigma_{p(e_2)}(E_2[e_2]) \neq \emptyset}(((E_1[e_1] \bowtie_{q(e_1, e_2)} \sigma_{p(e_2)}(E_2[e_2])) \div \sigma_{p(e_2)}(E_2[e_2])), E_1[e_1]).$$

If the quantifier predicate $q(e_1, e_2)$ is of the form $e_2 \in e_1.\text{SetAttribute}$, then the join can be replaced by an unnest operator:

$$if_{\sigma_{p(e_2)}(E_2[e_2]) \neq \emptyset}((\mu_{e_2:\text{SetAttribute}}(E_1[e_1], \text{SetAttribute}]) \div \sigma_{p(e_2)}(E_2[e_2])), E_1[e_1])$$

Using set difference, the translation is

$$E_1[e_1] \setminus \pi_{e_1}((E_1[e_1] \times \sigma_{p(e_2)}(E_2[e_2])) \setminus (E_1[e_1] \bowtie_{q(e_1, e_2)} \sigma_{p(e_2)}(E_2[e_2])))$$

which can be optimized to

$$E_1[e_1] \setminus E_1[e_1] \bowtie_{\neg q(e_1, e_2)} \sigma_{p(e_2)}(E_2[e_2])$$

This plan is mentioned in [830], however using a regular join instead of a semi-join.

The anti-semijoin can be employed to eliminate the set difference yielding the following plan:

$$E_1[e_1] \Join_{\neg q(e_1, e_2)} \sigma_{p(e_2)}(E_2[e_2])$$

This plan is in many cases the most efficient plan. However, the correctness of this plan depends on the uniqueness of e_1 , i.e., the attribute(s) e_1 must be a (super) key of E_1 . This is especially fulfilled in the object-oriented context if e_1 consists of or contains the object identifier.

We do not present the plans based group and count operations (see [184]).

Class 15 Here is an example query of Class 15:

```

select al.name
from al in Airline
where for all f in (
    select f
    from f in Flight
    where al = f.carrier):
    f.to.apctry != "Libya"

```

The quantifier's range formula $\sigma_{p(e_1, e_2)}(E_2[e_2])$ is obviously not closed. It contains the free variable e_1 . According to the reduction algorithm of Codd [199], the division plan is

$$(E_1[e_1] \bowtie_{\neg p(e_1, e_2) \vee q(e_2)} E_2[e_2]) \div E_2[e_2].$$

The plan with set difference is

$$E_1[e_1] \setminus \pi_{e_1}((E_1[e_1] \bowtie_{p(e_1, e_2)} E_2[e_2]) \setminus (E_1[e_1] \bowtie_{p(e_1, e_2)} \sigma_{q(e_2)}(E_2[e_2])))$$

and the most efficient plan using the antijoin is

$$E_1[e_1] \bowtie_{p(e_1, e_2)} \sigma_{\neg q(e_2)}(E_2[e_2]).$$

Class 16 Here is an example Class 16 query:

```

select al.name
from al in Airline
where for all ap in (
    select ap
    from ap in Airport
    where apctry = alctry):
    ap in al.lounges

```

The range predicate again depends on the outer level variable e_1 . A valid division plan looks similar to the one for Class 15. A plan with set difference is

$$E_1[e_1] \setminus \pi_{e_1}((E_1[e_1] \bowtie_{p(e_1, e_2)} E_2[e_2]) \setminus (E_1[e_1] \bowtie_{p(e_1, e_2) \wedge q(e_1, e_2)} E_2[e_2])).$$

This plan can first be refined by replacing the set difference of the two join expression by a semijoin result in

$$E_1[e_1] \setminus (E_1[e_1] \bowtie_{p(e_1, e_2) \wedge \neg q(e_1, e_2)} E_2[e_2])$$

Finally, the remaining set difference is transformed into an anti-semijoin which also covers the semijoin:

$$E_1[e_1] \bowtie_{p(e_1, e_2) \wedge \neg q(e_1, e_2)} E_2[e_2].$$

Again, the uniqueness constraint on $E_2[e_2]$ is required for this most efficient plan to be valid.

For all discussed classes, problems with NULL values might occur. In that case, the plans have to be refined [184].

13.4 Bibliography

[467] [220] [184] [715, 708]

Chapter 14

Unnesting Nested Queries

Chapter 15

Optimizing Queries with Materialized Views

15.1 Conjunctive Views

15.2 Views with Grouping and Aggregation

15.3 Views with Disjunction

15.4 Bibliography

materialized view with aggregates: [828],
materialized view with disjunction: [11],
SQL Server: [334]
other: [12, 151, 152, 163, 549, 843, 881, 948] [139, 143, 166, 154, 283, 482, 535, 676, 707, 785]
some more including maintenance etc: [10, 15, 53, 95, 151, 157, 205, 383, 398] [434, 481, 548, 704, 749, 266, 828] [843, 852, 851, 973, 949] [6, 252, 253, 405]
Overview: [390]
[550]
performance eval: [93]
Stacked views: [225]
recursion: [255]
with patterns (integration): [707], [254, 256], [234]

Chapter 16

Semantic Query Rewrite

16.1 Constraints and their impact on query optimization

Using Constraints: [332, 374]

16.2 Semantic Query Rewrite

Semantic query rewrite exploits knowledge (semantic information) about the content of the object base. This knowledge is typically specified by the user. We already saw one example of user-supplied information: *inverse relationships*. As we already saw, inverse relationships can be exploited for more efficient query evaluation.

Another important piece of information is knowledge about keys. In conjunction with type inference, this information can be used during query rewrite to speed up query execution. A typical example is the following query

```
select distinct   *  
from             Professor p1, Professor p2  
where            p1.university.name = p2.university.name
```

By type inference, we can conclude that the expressions *p1.university* and *p2.university* are of type University. If we further knew that the name of universities are unique, that is the name is a candidate key for universities, then the query could be simplified to

```
select distinct   *  
from             Professor p1, Professor p2  
where            p1.university = p2.university
```

Evaluating this query does no longer necessitate accessing the universities to retrieve their *name*.

Some systems consider even more general knowledge in form of equivalences holding over user-defined functions [1, 289]. These equivalences are then used to rewrite the query. Thereby, alternatives are generated all of which are subsequently optimized.

Semantic Query Optimization: [139]

16.3 Exploiting Uniqueness in Query Optimization

[681]

16.4 Bibliography

[82] [73] [943] Foreign functions semantic rules rewrite: [154] Conjunctive Queries,
Branch Minimization: [743]

Part IV

Plan Generation

Chapter 17

Current Search Space and Its Limits

17.1 Plans with Outer Joins, Semijoins and Anti-joins

outer join reordering [299, 298, 736, 308], outer join/antijoin plan generation [717], semijoin reducer [836],

17.2 Expensive Predicates and Functions

17.3 Techniques to Reduce the Search Space

- join single row tables first
- push down SARGable predicates
- For large join queries do not apply transitivity of equality to derive new predicates and disable cross products and possibly bushy trees.

17.4 Bibliography

Chapter 18

Dynamic Programming-Based Plan Generation

18.1 Introduction

So far, we treated predicates that reference a single relation as selection predicates and predicates that reference two relations as join predicates. In general, a predicate can reference more than two relations. In this case, it can be treated as a join predicate. Consider for example the query

```
select * from R, S, T, where R.A = S.B AND S.C = T.D and R.E
+ S.F = T.G
```

A query graph as defined in Section ?? does not suffice to capture these predicates. What is needed are hypergraphs.

There exists a second reason why hypergraphs are needed. In Section 7.15, we introduced several conflict handling mechanisms, which allow for the correct enumeration of the core search space. Every operator $\circ$ within some operator tree has a set of relations **TES** associated with it. This set of relations was a subset of all the relations in the leaf nodes below the operator subtree rooted at $\circ$. Hence, some relations occurred on $\circ$'s left side, others on its right side. Thus, we splitted **TES** into **TES_{left}** and **TES_{right}**. Then, the pair (**TES_{left}**, **TES_{right}**) is a hyperedge.

Algorithm **DPsube** was used to calculate the best plan bottom-up. The applicability test, among other things, assured that only connected components and connected complements thereof were formed. The test often fails. This is similar to the manner the tests of **DPsub** and **DPsize** (see Section ??) failed for regular graphs. There, this fact lead us to the development of **DPccp**, which enumerates CCPs for regular graphs quite efficiently.

The first goal of this section is to build an equally efficient enumerator for CCPs for hypergraphs. Then, this basic algorithm is extended such that it is able to deal with more operators than those handled in the core search space.

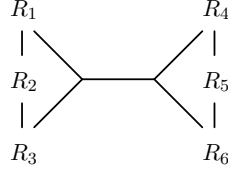


Figure 18.1: Sample hypergraph

18.2 Hypergraphs

Let us start with the definition of hypergraphs.

Definition 18.2.1 (hypergraph) A hypergraph is a pair $H = (V, E)$ such that

1. V is a non-empty set of nodes and
2. E is a set of hyperedges, where a hyperedge is an unordered pair (u, v) of non-empty subsets of V ($u \subset V$ and $v \subset V$) with the additional condition that $u \cap v = \emptyset$.

We call any non-empty subset of V a hypernode. We assume that the nodes in V are totally ordered via an (arbitrary) relation $\prec$. The ordering on nodes is important for our algorithm.

A hyperedge (u, v) is simple if $|u| = |v| = 1$. A hypergraph is simple if all its hyperedges are simple.

Note that a simple hypergraph is the same as an ordinary undirected graph. In our context, the nodes of hypergraphs are relations and the edges are abstractions of join predicates. Consider, for example, a join predicate of the form $R_1.a + R_2.b + R_3.c = R_4.d + R_5.e + R_6.f$. This predicate will result in a hyperedge $(\{R_1, R_2, R_3\}, \{R_4, R_5, R_6\})$. Fig. ?? contains an example of a hypergraph. The set V of nodes is $V = \{R_1, \dots, R_6\}$. Concerning the node ordering, we assume that $R_i \prec R_j \iff i < j$. There are the simple edges $(\{R_1\}, \{R_2\})$, $(\{R_2\}, \{R_3\})$, $(\{R_4\}, \{R_5\})$, and $(\{R_5\}, \{R_6\})$. The hyperedge from above is the only true hyperedge in the hypergraph.

Note that it is possible to rewrite the above complex join predicate. For example, it is equivalent to $R_1.a + R_2.b = R_4.d + R_5.e + R_6.f - R_3.c$. This leads to a hyperedge

$(\{R_1, R_2\}, \{R_3, R_4, R_5, R_6\})$. If the query optimizer is capable of performing this kind of algebraic transformations, all derived hyperedges are added to the hypergraph, at least conceptually. We will come back to this issue in Section ??.

To decompose a join ordering problem represented as a hypergraph into smaller problems, we need the notion of subgraph. More specifically, we only deal with node-induced subgraphs.

Definition 18.2.2 (subgraph) Let $H = (V, E)$ be a hypergraph and $V' \subseteq V$ a subset of nodes. The node induced subgraph $G|_{V'}$ of G is defined as $G|_{V'} =$

(V', E') with $E' = \{(u, v) \mid (u, v) \in E, u \subseteq V', v \subseteq V'\}$. The node ordering on V' is the restriction of the node ordering of V .

As we are interested in connected subgraphs, we give

Definition 18.2.3 (connected) Let $H = (V, E)$ be a hypergraph. H is connected if $|V| = 1$ or if there exists a partitioning V', V'' of V and a hyperedge $(u, v) \in E$ such that $u \subseteq V', v \subseteq V''$, and both $G|_{V'}$ and $G|_{V''}$ are connected.

If $H = (V, E)$ is a hypergraph and $V' \subseteq V$ is a subset of the nodes such that the node-induced subgraph $G|_{V'}$ is connected, then we call V' a *connected subgraph* or *csg* for short. The number of connected subgraphs is important for dynamic programming: it directly corresponds to the number of entries in the dynamic programming table. If a node set $V'' \subseteq (V \setminus V')$ induces a connected subgraph $G|_{V''}$, we call V'' a *connected complement* of V' or *cmp* for short.

For the purpose of this chapter, we assume that all hypergraphs are connected. This way, we can make sure that no (additional) cross products are needed. This condition can easily be assured by adding according hyperedges: for every pair of connected components, we can add a hyperedge whose hypernodes contain exactly the relations of the connected components. By considering these hyperedges as $\times$ operators. As we saw in Section 7.15, cross products can be handled by our conflict detectors can be handled

18.3 CCPs: Csg-Cmp-Pairs for Hypergraphs

With these notations, we can move closer to the heart of dynamic programming by defining a *csg-cmp-pair*, or *ccp* for short.

Definition 18.3.1 (csg-cmp-pair, ccp) Let $H = (V, E)$ be a hypergraph and S_1, S_2 two subsets of V such that $S_1 \subseteq V$ and $S_2 \subseteq (V \setminus S_1)$ are a *connected subgraph* and a *connected complement*. If there further exists a hyperedge $(u, v) \in E$ such that $u \subseteq S_1$ and $v \subseteq S_2$, we call (S_1, S_2) a *csg-cmp-pair*.

Note that if (S_1, S_2) is a csg-cmp-pair, then (S_2, S_1) is one as well. Out of these two possibilities, only one will be enumerated by our subsequent algorithm. More specifically, we will restrict the enumeration of csg-cmp-pairs to those (S_1, S_2) which satisfy the condition that $\min(S_1) \prec \min(S_2)$, where $\min(S) = s$ such that $s \in S$ and $\forall s' \in S : s \neq s' \implies s \prec s'$. Since this restriction will hold for all csg-cmp-pairs enumerated by our procedure, we are sure that no duplicate csg-cmp-pairs are calculated. As a consequence, we have to take some care in order to ensure that our dynamic programming procedure is complete: if the binary operator we apply is commutative, the procedure to build a plan for $S_1 \cup S_2$ from plans for S_1 and S_2 has to take commutativity into account. However, this is not really a challenge.

Obviously, in order to be correct, any dynamic programming algorithm has to consider all csg-cmp-pairs [618]. Further, only these have to be considered. Thus, the minimal number of cost function calls of any dynamic programming algorithm is exactly the number of csg-cmp-pairs for a given hypergraph. Note

that the number of connected subgraphs is far smaller than the number of csg-cmp-pairs. The problem now is to enumerate the csg-cmp-pairs efficiently and in an order acceptable for dynamic programming. The latter can be expressed more specifically. Before enumerating a csg-cmp-pair (S_1, S_2) , all csg-cmp-pairs (S'_1, S'_2) with $S'_1 \subseteq S_1$ and $S'_2 \subseteq S_2$ have to be enumerated.

18.4 Neighborhood

The main idea to generate csg-cmp-pairs is to incrementally expand connected subgraphs by considering new nodes in the *neighborhood* of a subgraph. Informally, the neighborhood $N(S)$ under an exclusion set X consists of all nodes reachable from S that are not in X . We derive an exact definition below.

When choosing subsets of the neighborhood for inclusion, we have to treat a hypernode as a single instance: either all of its nodes are inside an enumerated subset or none of them. Since we want to use the fast subset enumeration procedure introduced by Vance and Maier [898], we must have a single bit representing a hypernode and also single bits for relations occurring in simple edges. Since these may overlap, we are constrained to choose one unique representative of every hypernode occurring in a hyperedge. We choose the node that is minimal with respect to $\prec$. Accordingly, we define:

$$\min(S) = \{s | s \in S, \forall s' \in S \ s \neq s' \implies s \prec s'\}$$

Note that if S is empty, then $\min(S)$ is also empty. Otherwise, it contains a single element. Hence, if S is a singleton set, then $\min(S)$ equals the only element contained in S . For our hypergraph in Fig. ?? and with $S = \{R_4, R_5, R_6\}$, we have $\min(S) = \{R_4\}$.

Let S be a current set, which we want to expand by adding further relations. Consider a hyperedge (u, v) with $u \subseteq S$. Then, we will add $\min(v)$ to the neighborhood of S . However, we have to make sure that the missing elements of v , i.e. $v \setminus \min(v)$, are also contained in any set emitted. We thus define

$$\overline{\min}(S) = S \setminus \min(S)$$

For our hypergraph in Fig. ?? and with $S = \{R_4, R_5, R_6\}$, we have $\overline{\min}(S) = \{R_5, R_6\}$.

We define the set of non-subsumed hyperedges as the minimal subset $E \downarrow$ of E such that for all $(u, v) \in E$ there exists a hyperedge $(u', v') \in E \downarrow$ with $u' \subseteq u$ and $v' \subseteq v$. Additionally, we make sure that none of the nodes of a hypernode are contained in a set X , which is to be excluded from neighborhood considerations. We thus define a set containing the *interesting hypernodes* for given sets S and X . We do so in two steps. First, we collect the potentially interesting hypernodes into a set $E \downarrow' (S, X)$ and then minimize this set to eliminate subsumed hypernodes. This step then results in $E \downarrow (S, X)$, with which the algorithm will work.

$$E \downarrow' (S, X) = \{v | (u, v) \in E, u \subseteq S, v \cap S = \emptyset, v \cap X = \emptyset\}$$

Define $E \downarrow (S, X)$ to be the minimal set of hypernodes such that for all $v \in E \downarrow (S, X)$ there exists a hypernode v' in $E \downarrow (S, X)$ such that $v' \subseteq v$. Note that apart from the connectedness, we test exactly the conditions given in Def. 18.3.1. For our hypergraph in Fig. ?? and with $X = S = \{R_1, R_2, R_3\}$, we have $E \downarrow (S, X) = \{\{R_4, R_5, R_6\}\}$.

We are now ready to define the neighborhood of a hypernode S , given a set of excluded nodes X .

$$N(S, X) = \bigcup_{v \in E \downarrow (S, X)} \min(v) \quad (18.1)$$

For our hypergraph in Fig. ?? and with $X = S = \{R_1, R_2, R_3\}$, we have $N(S, X) = \{R_4\}$. Assuming a bit vector representation of sets, the neighborhood can be efficiently calculated bottom-up.

18.5 The CCP Enumerator BuEnumCppHyp

Before starting with the algorithm description we give a high-level overview of the general principles used in the algorithm:

1. The algorithm constructs ccps by enumerating connected subgraphs from an increasing part of the query graph;
2. both the primary connected subgraphs and its connected complement are created by recursive graph traversals;
3. during traversal, some nodes are *forbidden* to avoid creating duplicates. More precisely, when a function performs a recursive call it forbids all nodes it will investigate itself;
4. connected subgraphs are increased by following edges to neighboring nodes. For this purpose hyperedges are interpreted as $n : 1$ edges, leading from n of one side to one (specific) canonical node of the other side (cmp. Eq. 18.1).

Summarizing the above, the algorithm traverses the graph in a fixed order and recursively produces larger connected subgraphs. The main challenge relative to DPccp is the traversal of hyperedges: First, the "starting" side of the edge can require multiple nodes, which complicates neighborhood computation. In particular the neighborhood can no longer be computed as a simple bottom-up union of local neighborhoods. Second, the "ending" side of the edge can lead to multiple nodes at once, which disrupts the recursive growth of components. Consider a set S_1 , which we want to extend by a hyperedge (u, w) . Even if $u \subseteq S_1$, there is no guarantee that $S_1 \cup w$ will be connected.

To overcome these problems, the algorithm picks a *representative end node*. In our example, it picks the 1 in the $n : 1$ of item 4 (see also Eq. 18.1). With it, it starts the recursive growth and exploits the DP table to check if a valid constellation has been reached, i.e., the constructed hypernode induces a connected subgraph. This exploitation builds on the fact that our DP strategies

enumerate subsets before supersets. We are now prepared to discuss the details of the algorithm.

We give the implementation of our join ordering algorithm for hypergraphs by means of the pseudocode for member functions of a class `BuEnumCcpHyp`. This allows us to minimize the number of parameters by assuming that this class contains references to the query hypergraph ($G = (V, E)$) and to the dynamic programming table (`DpTable`).

The whole algorithm is distributed over five subroutines. The top-level routine `BuEnumCcpHyp` initializes the `DpTable` with access plans for single relations and then calls `EmitCsg` and `EnumerateCsgRec` for each set containing exactly one relation. In a real implementation, the `DpTable` should be initialized before calling `BuEnumCcpHyp`.

The member function `EnumerateCsgRec` is responsible for enumerating connected subgraphs. It does so by calculating the neighborhood and iterating over each of its subset. For each such subset S_1 , it calls `EmitCsg`. This member function is responsible for finding suitable complements. It does so by calling `EnumerateCmpRec`, which recursively enumerates the complements S_2 for the connected subgraph S_1 found before. The pair (S_1, S_2) is a csg-cmp-pair. For every such pair, `EmitCsgCmp` is called. Its main responsibility is to consider a plan built up from the plans for S_1 and S_2 . The following subsections discuss these five member functions in detail. We illustrate them with the example hypergraph shown in Fig. ???. The corresponding traversal steps are shown in Fig. 18.2, we will illustrate them during the description of the algorithm.

18.5.1 BuEnumCcpHyp

The pseudocode for `BuEnumCcpHyp` looks as follows:

```

BuEnumCcpHyp()
for each  $v \in V$  // initialize DpTable
    DpTable[{ $v$ }] = plan for  $v$ 
for each  $v \in V$  descending according to  $\prec$ 
    EmitCsg({ $v$ }) // process singleton sets
    EnumerateCsgRec({ $v$ },  $\mathbf{B}_v$ ) // expand singleton sets
return DpTable[V]
```

In the first loop, it initializes the dynamic programming table with plans for single relations. In the second loop, it calls for every node in the query graph, in decreasing order (according to $\prec$) the two subroutines `EmitCsg` and `EnumerateCsgRec`. In Fig. 18.2, we find the call stack of our algorithm. The calls generated by `BuEnumCcpHyp` correspond to those with stack-depth zero, where the stack-depth is indicated in the second column from the left. For convenience, we not only give the parameters, but also the neighborhood $\mathcal{N}$. The algorithm calls `EmitCsg({ $v$ })` for single nodes $v \in V$ to generate all csg-cmp-pairs $(\{v\}, S_2)$ via calls to `EnumerateCsgCmp` and `EmitCsgCmp`, where $v \prec \min(S_2)$ holds. This condition implies that every csg-cmp-pair is generated only once, and no symmetric pairs are generated. In Fig. 18.2, this corresponds to single vertex graphs, e.g. step 1 and 2. The calls to `EnumerateCsgRec` extend the initial set $\{v\}$ to larger

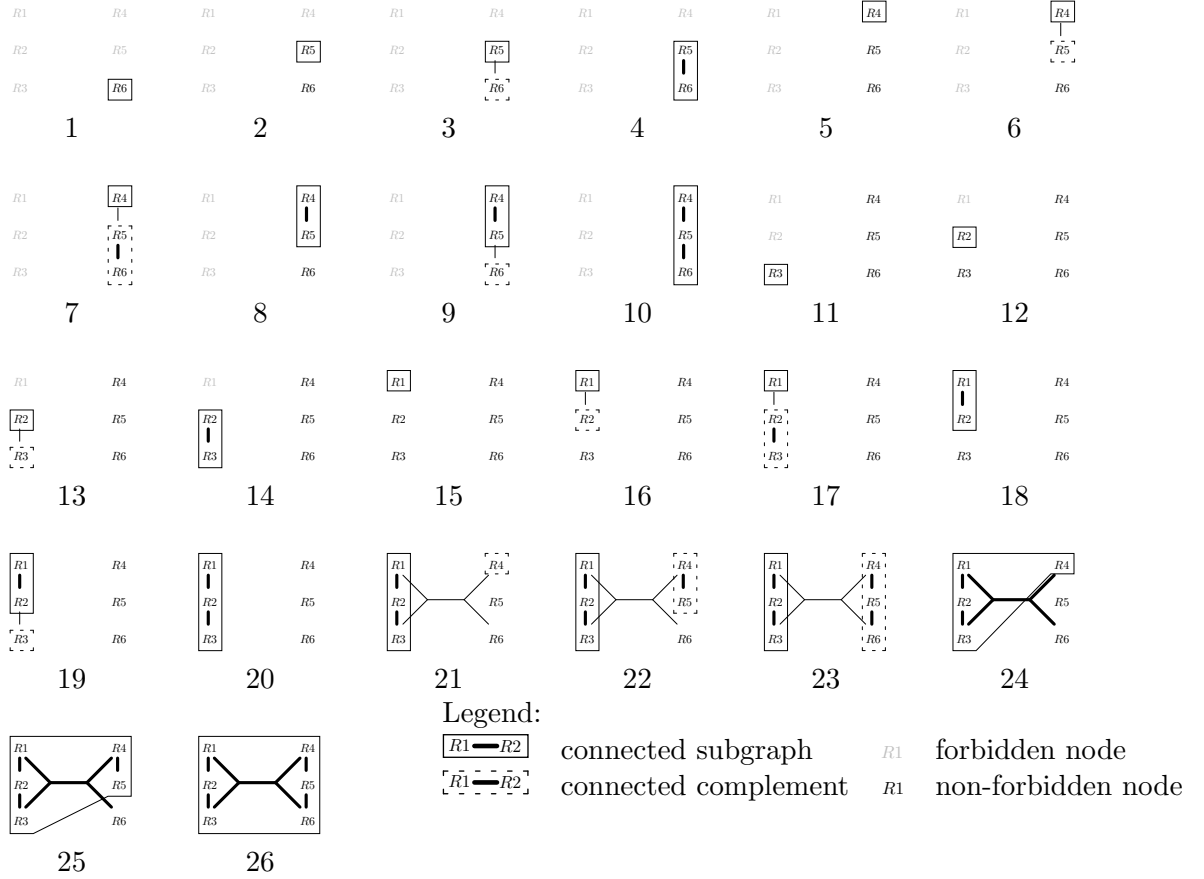


Figure 18.2: Trace of algorithm for Figure ??

sets S_1 , for which then connected subsets of its complement S_2 are found such that (S_1, S_2) results in a csg-cmp-pair. In Fig. 18.2, this is shown in step 2, for example, where **EnumerateCsgRec** starts with R_5 and expands it to $\{R_5, R_6\}$ in step 4 (step 3 being the construction of the complement). To avoid duplicates during enumerations, all nodes that are ordered before v according to $\prec$ are prohibited during the recursive expansion [618]. Formally, we define this set as $B_v = \{w | w \prec v\} \cup \{v\}$.

18.5.2 EnumerateCsgRec

The general purpose of **EnumerateCsgRec** is to extend a given set S_1 , which induces a connected subgraph of G to a larger set with the same property. It does so by considering each non-empty, proper subset of the neighborhood of S_1 . For each of these subsets N , it checks whether $S_1 \cup N$ is a connected component. This is done by a lookup into the **DpTable**. If this test succeeds, a new connected component has been found and is further processed by a call **EmitCsg**($S_1 \cup N$). Then, in a second step, for all these subsets N of the neighborhood, we call **EnumerateCsgRec** such that $S_1 \cup N$ can be further extended recursively. The

reason why we first call **EmitCsg** and then **EnumerateCsgRec** is that in order to have an enumeration sequence valid for dynamic programming, smaller sets must be generated first. Summarizing, the code looks as follows:

```

EnumerateCsgRec( $S_1, X$ )
for each  $N \subseteq \mathcal{N}(S_1, X)$ :  $N \neq \emptyset$ 
    if  $\text{DpTable}[S_1 \cup N] \neq \emptyset$ 
        EmitCsg( $S_1 \cup N$ )
    for each  $N \subseteq \mathcal{N}(S_1, X)$ :  $N \neq \emptyset$ 
        EnumerateCsgRec( $S_1 \cup N, X \cup \mathcal{N}(S_1, X)$ )

```

Take a look at step 12. This call was generated by **BuEnumCcpHyp** on $S_1 = \{R_2\}$. The neighborhood consists only of $\{R_3\}$, since R_1 is in X (R_4, R_5, R_6 are not in X either, but not reachable). **EnumerateCsgRec** first calls **EmitCsg**, which will create the joinable complement (step 13). It then tests $\{R_2, R_3\}$ for connectedness. The according **DpTable** entry was generated in step 13. Hence, this test succeeds, and $\{R_2, R_3\}$ is further processed by a recursive call to **EnumerateCsgRec** (step 14). Now the expansion stops, since the neighborhood of $\{R_2, R_3\}$ is empty, because $R_1 \in X$.

18.5.3 EmitCsg

EmitCsg takes as an argument a non-empty, proper subset S_1 of V , which induces a connected subgraph. It is then responsible to generate the seeds for all S_2 such that (S_1, S_2) becomes a csg-cmp-pair. Not surprisingly, the seeds are taken from the neighborhood of S_1 . All nodes that have ordered before the smallest element in S_1 (captured by the set $\mathbf{B}_{\min(S_1)}$) are removed from the neighborhood to avoid duplicate enumerations [618]. Since the neighborhood also contains $\min(v)$ for hyperedges (u, v) with $|v| > 1$, it is not guaranteed that S_1 is connected to v . To avoid the generation of false csg-cmp-pairs, **EmitCsg** checks for connectedness. However, each single neighbor might be extended to a valid complement S_2 of S_1 . Hence, no such test is necessary before calling **EnumerateCmpRec**, which performs this extension. The pseudocode looks as follows:

```

EmitCsg( $S_1$ )
 $X = S_1 \cup \mathbf{B}_{\min(S_1)}$ 
 $N = \mathcal{N}(S_1, X)$ 
for each  $v \in N$  descending according to  $\prec$ 
     $S_2 = \{v\}$ 
    if  $\exists (u, v) \in E : u \subseteq S_1 \wedge v \subseteq S_2$ 
        EmitCsgCmp( $S_1, S_2$ )
    EnumerateCmpRec( $S_1, S_2, X \cup B_v(N)$ )

```

where $B_v(W) = \{w | w \in W, w \leq v\}$ is defined as in Section 3.2.4 for **DPccp**.

Take a look at step 20. The current set S_1 is $S_1 = \{R_1, R_2, R_3\}$, and the neighborhood is $\mathcal{N} = \{R_4\}$. As there is no hyperedge connecting these two

sets, there is no call to **EmitCsgCmp**. However, the set $\{R_4\}$ can be extended to a valid complement, namely $\{R_4, R_5, R_6\}$. Properly extending the seeds of complements is the task of the call to **EnumerateCmpRec** in step 21.

18.5.4 EnumerateCmpRec

EnumerateCsgRec has three parameters. The first parameter S_1 is only used to pass it to **EmitCsgCmp**. The second parameter is a set S_2 which is connected and must be extended until a valid csg-cmp-pair is reached. Therefore, it considers the neighborhood of S_2 . For every non-empty, proper subset N of the neighborhood, it checks whether $S_2 \cup N$ induces a connected subset and is connected to S_1 . If so, we have a valid csg-cmp-pair (S_1, S_2) and can start plan construction (done in **EmitCsgCmp**). Irrespective of the outcome of the test, we recursively try to extend S_2 such that this test becomes successful. Overall, the **EnumerateCmpRec** behaves very much like **EnumerateCsgRec**. Its pseudocode looks as follows:

```

EnumerateCmpRec( $S_1, S_2, X$ )
for each  $N \subseteq \mathcal{N}(S_2, X)$ :  $N \neq \emptyset$ 
    if  $\text{DpTable}[S_2 \cup N] \neq \emptyset \wedge$ 
         $\exists (u, v) \in E : u \subseteq S_1 \wedge v \subseteq S_2 \cup N$ 
        EmitCsgCmp( $S_1, S_2 \cup N$ )
 $X = X \cup \mathcal{N}(S_2, X)$ 
for each  $N \subseteq \mathcal{N}(S_2, X)$ :  $N \neq \emptyset$ 
    EnumerateCmpRec( $S_1, S_2 \cup N, X$ )

```

Take a look at step 21 again. The parameters are $S_1 = \{R_1, R_2, R_3\}$ and $S_2 = \{R_4\}$. The neighborhood consists of the single relation R_5 . The set $\{R_4, R_5\}$ induces a connected subgraph. It was inserted into **DpTable** in step 6. However, there is no hyperedge connecting it to S_1 . Hence, there is no call to **EmitCsgCmp**. Next is the recursive call in step 22 with S_2 changed to $\{R_4, R_5\}$. Its neighborhood is $\{R_6\}$. The set $\{R_4, R_5, R_6\}$ induces a connected subgraph. The according test via a lookup into **DpTable** succeeds, since the according entry was generated in step 7. The second part of the test also succeeds, as our only true hyperedge connects this set with S_1 . Hence, the call to **EmitCsgCmp** in step 23 takes place and generates the plans containing all relations.

18.5.5 EmitCsgCmp

The procedure **EmitCsgCmp**(S_1, S_2) is called for every S_1 and S_2 such that (S_1, S_2) forms a csg-cmp-pair. It is the (call back) interface for **BuEnumCcpHyp**. Its only task is to call **BuildPlan**, which then builds the optimal plan(s) for (S_1, S_2) .

18.5.6 Neighborhood Calculation

The formulation of neighborhood we used, is only one possibility. In fact, any neighborhood satisfying the following condition will do. Let $G = (V, E)$ be a


```

calcNeighborhood( $S, X$ )
 $N := \emptyset$ 
if isConnected( $S$ )
     $N = \text{simpleNeighborhood}(S) \setminus X$ 
else
    foreach  $s \in S$ 
         $N \cup= \text{simpleNeighborhood}(s)$ 
 $F = (S \cup X \cup N) \text{ // forbidden since in } X \text{ or already handled}$ 
foreach  $(u, v) \in E$ 
    if  $u \subseteq S$ 
        if  $v \cap F = \emptyset$ 
             $N += \min(v)$ 
             $F \cup= N$ 
    if  $v \subseteq S$ 
        if  $u \cap F = \emptyset$ 
             $N += \min(u)$ 
             $F \cup= N$ 

```

Figure 18.3: Pseudocode for `calcNeighborhood`

hypergraph not containing any subsumed edges. For some set S , for which we want to calculate the neighborhood, define the set of reachable hypernodes as

$$W(S, X) := \{w | (u, w) \in E, u \subseteq S, w \cap (S \cup X) = \emptyset\},$$

where X contains the forbidden nodes. Then, any set of nodes N such that for every hypernode in $W(S, X)$ exactly one element is contained in N can serve as the neighborhood.

Further, in order to make `BuEnumCcpHyp` as efficient as `DPccp` for simple graphs, it is convenient to materialize the simple neighborhood for every plan class contained in the `DpTable` and calculate it bottom-up. Figure 18.3 contains one possible implementation of the neighborhood calculation.

18.6 DPhyp

18.7 Adding Selections

18.8 Adding Maps

18.9 Adding Grouping

Chapter 19

Optimizing Queries with Disjunctions

19.1 Introduction

Simple rewrites as indicated in Section ?? for IN and OR predicates that boil down to comparisons of a column with a set of constants can eliminate disjunction from the plan or push it into a multirange index access.

Another possibility that can be used for disjunctions on single columns is to use DISJOINT UNION of plans. This is a special form of UNION where conditions ensure that no phantom duplicates are produced. The DISJOINT UNION operator merely concatenates the result tables without any further overhead like duplicate elimination.

For example a predicate of the form $x = c_1$ or $y = c_2$ where x and y are columns of the same table results in two predicates

1. $x = c_1$
2. $x \neq c_1$ AND $y = c_2$

Obviously, no row can satisfy both conditions. Hence, the query `select * from R where  $x = c_1$  or  $y = c_2$` can be safely rewritten to

```
(select * from R where  $x = c_1$ ) DISJOINT UNION (select * from
R where  $x \neq c_1$  AND  $y = c_2$ )
```

In case there are indexes on x and y efficient plans do exist. If they don't the table R needs to be scanned twice. This problem is avoided by using bypass plans.

DISJOIN UNIONS can also be used for join predicates. Consider the following example query: `select * from R, S where  $R.a = S.a$  OR  $R.b = S.a$` This query can be rewritten to `(select * from R, S where  $R.a = S.a$ ) DISJOINT UNION (select * from R, S where  $R.a \neq S.a$  and  $R.b = S.a$ )` The general condition here is that all equality predicates have one side identical. Note that both tables are scanned and joined twice. Bypass plans will eliminate this problem.

Let us consider a more complex example: `select * from R, S where R.a = S.a AND h.b IN (c1,c2).`
 XXX

19.2 Using Disjunctive or Conjunctive Normal Forms

19.3 Bypass Plans

All the above approaches rely on conjunctive normal forms. However, in the presence of disjunctions, this does not necessarily yield good plans. Using a disjunctive normal form does not always solve the problem either and this approach has its own problems with duplicates. This is why bypass plans were developed [489, 835, 186]. The idea is to provide selection and join operators with two different output streams: one for qualifying tuples and one for the not qualifying tuples. We cannot go into the details of this approach and only illustrate it by means of examples. Let us first consider a query with no join and a selection predicate of the form $a \wedge (b \vee c)$. This selection predicate is already in conjunctive normal form. The disjunctive normal form is $(a \wedge b) \vee (a \wedge c)$. We first consider some DNF-based plans (Fig. 19.1). These plans generate duplicates, if a tuple qualifies for both paths. Hence, some duplicate elimination procedure is needed. Note that these duplicates have nothing to do with the duplicates generated by queries. Even if the query does not specify **distinct**, the duplicates generated must be eliminated. If there are duplicates, which is quite likely, then the condition a is evaluated twice for those tuples qualifying for both conjuncts (Plan a and b). Figure 19.2 presents two CNF plans.

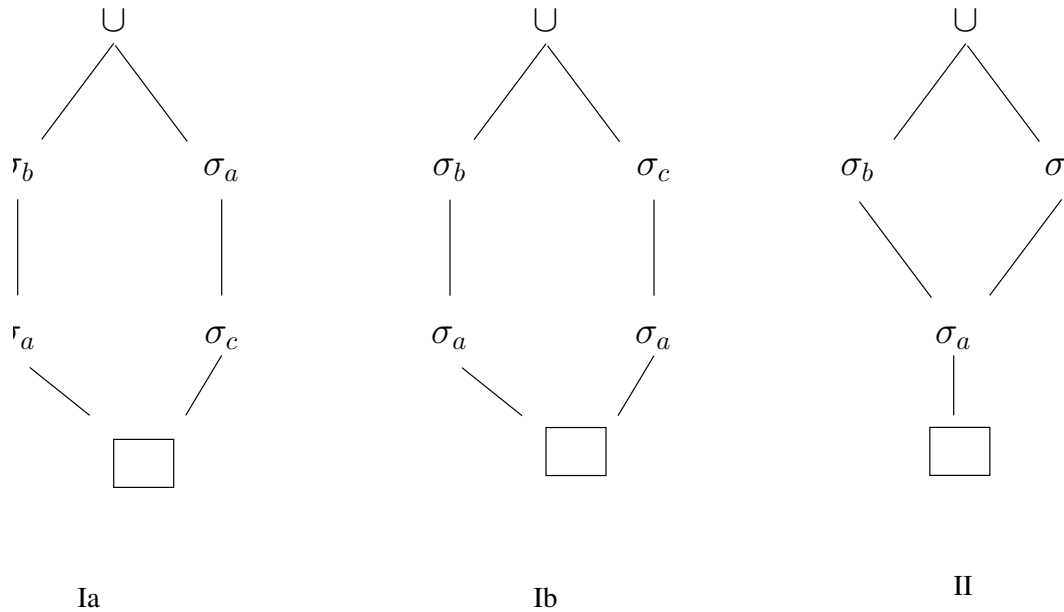


Figure 19.1: DNF plans

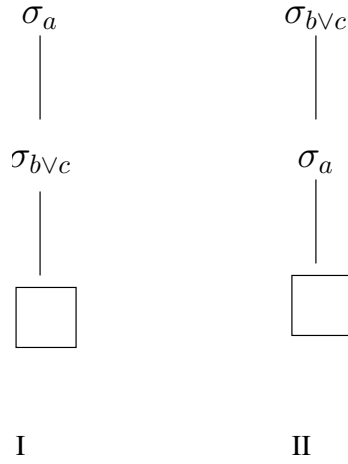


Figure 19.2: CNF plans

CNF plans never produce duplicates. The evaluation of the boolean factors can stop as soon as some predicate evaluates to *true*. Again, some (expensive) predicates might be evaluated more than once in CNF plans. Figure 19.3 shows some bypass plans. Note the different output streams. It should be obvious, that a bypass plan can be more efficient than both a CNF or DNF plan. It

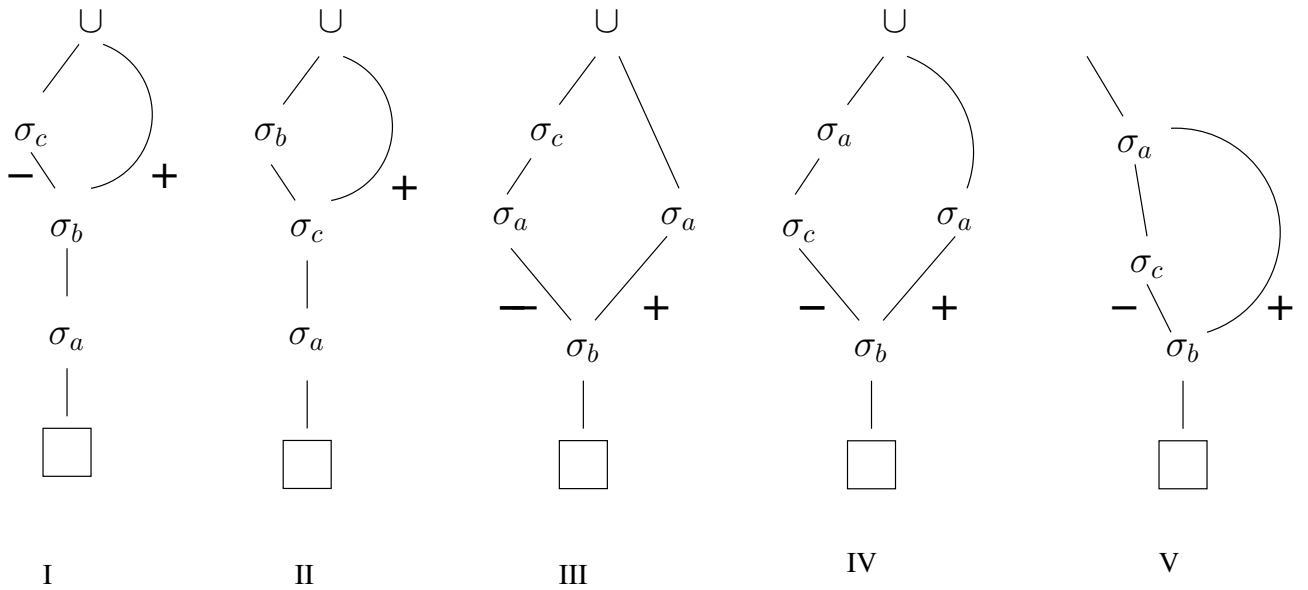


Figure 19.3: Bypass plans

is possible to extend the idea of bypass plans to join operators. However, this and the algorithm to generate bypass plans is beyond the scope of the current paper (see [489, 835, 186]).

19.4 Implementation remarks

The internal representation of execution plans during plan generation typically differs from that used in Rewrite I. The reason is that many plans have to be generated and space efficiency is a major concern. As in the query representation discussed earlier, the physical algebraic operators can be organized into a hierarchy. Besides their arguments, they possibly contain backpointers to the original query representation (e.g. for predicates). Sharing is a must for plan generation. Hence, subplans are heavily shared. The plan nodes are enhanced by so-called property vectors. These contain information about the plan:

- logical information
 - the set of relations joined
 - the set of predicates applied so far
 - the set of IUs computed so far
 - order information
- physical information
 - costs
 - cardinality information

For fast processing, the first three set-valued items in the logical information block are represented as bit-vectors. However, the problem is that an upper bound on the size of these bitvectors is not reasonable. Hence, they are of variant size. It is recommendable, to have a plan node factory that generates plan nodes of different length such that the bit-vectors are included in the plan node. A special interpreter class then knows the offsets and lengths of the different bitvectors and supplies the operations needed to deal with them. This bit-vector interpreter can be attached to the plan generator's control block as indicated in Fig. 25.3.

19.5 Other plan generators/query optimizer

There are plenty of other query optimizers described in the literature. Some of my personal favorites not mentioned so far are the Blackboard query optimizer [488], the Epoq optimizer [612, 611], the Genesis optimizer [57, 62], the Gral query optimizer [67], the Lanzelotte query optimizer [529, 530, 531], the Orion optimizer [50, 51, 493], the Postgres optimizer [469, 400, 398, 399], the Prima optimizer [404, 402], the Probe optimizer [223, 222, 655], the Straube optimizer [850, 889, 847, 848, 846, 849]. Highly recommended is a description of the DB2 query optimizer(s) [322].

Also interesting to read is the first proposal for a rule-based query optimizer called Squirrel [823] and other proposals for rule-based query optimizers [295, 780, 486, 485, 580].

19.6 Bibliography

Disjunctive queries: P. Ciaccia and M. Scalas: Optimization Strategy for Relational Queries. *IEEE Transaction on Software Engineering* 15 (10), pp 1217-1235, 1989.

Kristofer Vorwerk, G. N. Paulley: On Implicate Discovery and Query Optimization. *International Database Engineering and Applications Symposium (IDEAS'02)*

Jack Minker, Rita G. Minker: Optimization of Boolean Expressions-Historical Developments. *IEEE Annals of the History of Computing* 2 (3), pp 227-238, 1980.

Chaudhuri: SIGMOD 03: [149]

Conjunctive Queries, Branch Minimization: [743]

Also Boolean Difference Calculus (?): [824]

Chapter 20

Generating Plans for the Full Algebra

Chapter 21

Generating DAG-structured Plans

```
@misc{ roy-optimization,  
  author = "Prasan Roy",  
  title = "Optimization of DAG-Structured Query Evaluation Plans",  
  url = "citeseer.nj.nec.com/roy98optimization.html" }
```


Chapter 22

Simplifying the Query Graph

[This chapter was written by Thomas Neumann]

22.1 Introduction

As we have seen in Chapter 3, computing the the optimal join for large queries is a very hard problem. Most hand-written queries join just a few (<15) relations, but in general join queries can become quite large: Some systems like SAP R/3 store their data in thousands of relations, and subsequently generate large join queries. Other examples include data warehousing, where a fact table is joined with a large number of dimension tables, forming a star join, and databases that make heavy use of views to simplify query formulation (where the views then implicitly add joins). Existing database management systems have difficulties optimizing very large join queries, falling back to heuristics when they cannot solve them exactly anymore. This is unfortunate, as it does not offer a smooth transition. Ideally, one would optimize a query as much as possible under given time constraints.

When optimizing join queries, the optimal join order is usually determined using some variant of dynamic programming (DP). However finding the optimal join is NP-hard in general, which means that large join queries become intractable at some point. On the other hand, the complexity of the problem depends heavily upon the structure of the query (see Chapter 3), where some queries can be optimized exactly even for a large number of relations while other queries quickly become too difficult. As computing the optimal join order becomes intractable at some point, the standard technique of handling large join queries resorts to some heuristics. Some commercial database systems first try to solve the problem exactly using DP, and then fall back to greedy heuristics when they run out memory. As we have seen in Chapter 3, a wide range of heuristics has been proposed in the literature. Most of them integrate some kind of greedy processing in the optimization process, greedily building execution plan fragments that seem plausible. The inherent problem of this approach is that it is quite likely to greedily make a decision that one would regret having more information about the complete execution plan. For example greedily deciding which two relations should be joined first is very hard, as it depends

on all other joins involved in the query.

Here, we follow a different approach presented in [639]: If a query is too complex to optimize exactly, we *simplify* it using a greedy heuristic until it becomes tractable using DP. The simplification step does not build execution plans but modifies the join graph of the query to make it more restrictive, ruling out join orders that it considers unlikely. In a way this is the opposite of the standard greedy plan building techniques: *Instead of greedily choosing joins (which is very hard), we choose joins that must be avoided.* The great advantage of this approach is that we can start with the 'easy' decisions (i.e., the relatively obvious ones) using the heuristic and then leave the hard execution plan building to the DP algorithm once the problem is simplified enough. The resulting optimization algorithm adapts naturally to the query complexity and the given time budget, simplifying the query just as much as needed and then optimizing the simplified query exactly.

22.2 On Optimizing Join Queries

Optimizing the join order is one of the most important steps of query optimization, as changes in the join order can affect the query execution times by orders of magnitudes. Unfortunately computing the optimal join order is NP-hard in general, and the standard technique of using dynamic programming fails if the query is large enough.

Still, there are large differences in problem complexity even for queries of the same size. When disregarding cross products, the join predicates included in the query induce a query graph, and the structure of that query graph determines the complexity of the problem. Clique queries for example, where there is a join predicate between any two relations involved in the query, are the worst-case scenario for join ordering. Here any combination of relations is joinable, all joins affect each other through redundant join edges, and both the space complexity and the runtime complexity of the best known algorithm increases exponentially (in the order of $O(2^n)$ and $O(4^n)$, where n is the number of relations). For clique queries there is little hope of ever finding a good algorithm, but fortunately large clique queries never occur in practice. Chain queries on the other hand, where relations are joined sequentially, are quite common in practice and much easier to optimize: Any join tree without cross products must only consist of relations that are neighboring in the chain, i.e., that form a subchain. As there are less than n^2 subchains of a chain of length n , and we can join a subchain only with less than n other (neighboring) subchains, we get a space complexity of $O(n^2)$ and a time complexity of $O(n^3)$. Other graph structures are between these two extremes. Star queries, which are common in data warehouse applications where dimension tables are joined to a central fact table, have a space complexity of $O(2^n)$ and a time complexity of $O(n2^n)$.

The practical impact of these complexity differences can be seen in Figure 22.1. It shows the optimization time using DPhyp and the setup discussed in Section [639]. One observation here is that while small queries (<10 relations) can be optimized quickly regardless of the graph structure, larger queries

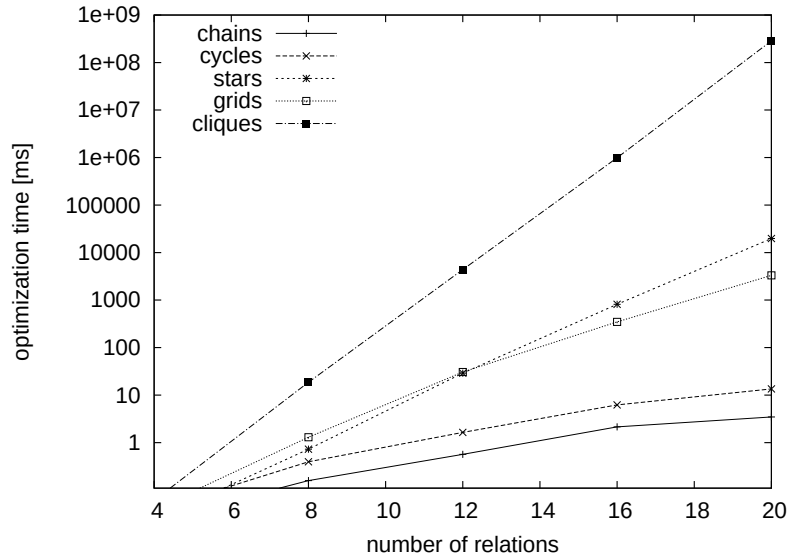


Figure 22.1: Runtimes for Different Query Graphs

soon become too expensive for everything except chain and cycle queries. Clique queries are particular bad, of course, but even the data warehousing star queries are too complex relatively soon. For really large queries (e.g., 50 relations), finding the optimal solution using DP is out of question for most query types.

Now the basic idea of graph simplification stems from the fact that some graph easier to solve than others: If the problem is too difficult to solve exactly, we change the query graph to make it easier to solve. We will look at this simplification strategy in the next section.

22.3 Graph Simplification Algorithm

After examining the impact of the query graph structure on optimization time, we now study an algorithm to simplify the query graph as much as needed to allow for a dynamic programming solution. We first discuss the simplification itself, then how this can be used to simplify a query graph as much as needed, and then one edge selection heuristic (which is orthogonal to the main simplification algorithm). Finally we show that the approach is plausible by proving optimality for star queries and certain classes of cost functions.

During this section we assume that the query has been brought into proper query (hyper-)graph form. In particular we assume that all non-inner joins have been expressed as hyperedges, as suggested in [619]. This allows us to reason about graph structures alone without violating proper query semantics.

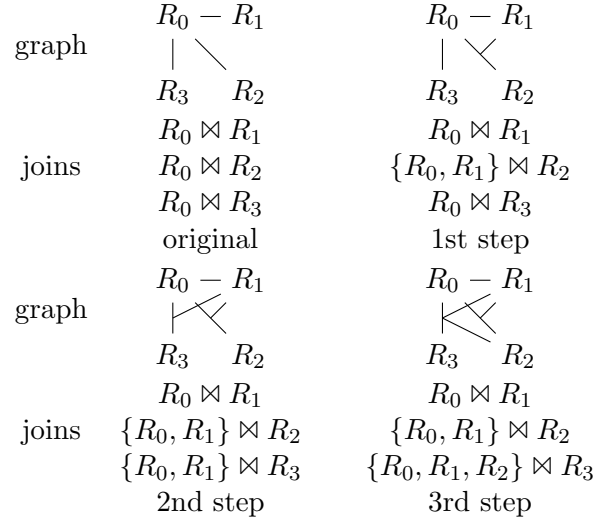


Figure 22.2: Exemplary Simplification Steps for a Star Query

22.3.1 Simplifying the Query Graph

When a query graph is too complex to solve exactly, we perform a *simplification step* to reduce its complexity. Note that with *simplification* we mean a simplification of the underlying optimization problem. The graph itself becomes more complex, at least for a human. This is illustrated in Figure 22.2. The original query is a star query with three satellite relations. The number of possible join trees (ignoring commutativity) is $3! = 6$, as any linear join order is valid. To reduce the search space we look for decisions that are relatively easy. For example if $R_0 \bowtie R_1$ is very selective and $R_0 \bowtie R_2$ greatly increases the cardinality, it is probably a good idea to join R_1 first (for the real criterion see Section 22.3.3). We thus modify the join $R_0 \bowtie R_2$ into $\{R_0, R_1\} \bowtie R_2$. This describes that we can join a join tree containing R_0 and R_1 with a join tree contain R_2 , and forms a hyperedge in the query graph. The search space shrinks to 3 possible trees, as now R_1 is required to come before R_2 . R_3 can still be joined arbitrary, either before R_1 , between R_1 and R_2 , or after R_2 . We can reduce the search space to two trees by requiring R_1 to be joined before R_3 (2. step), and finally to just one valid tree by ordering the join with R_2 before the join with R_3 (3. step). At this point the optimization problem is trivial to solve, but the solution could be poor due to the heuristical join ordering. In the actual algorithm we therefore simplify just as much as needed to be able to solve the optimization problem, and we perform these simplification first where we are most certain about the correct join ordering.

The pseudo-code for a single simplification step is shown in Figure 22.3. It examines all pairs of joins, and checks if they are *neighboring* in the query graph, i.e., they touch via common relations. The condition is somewhat complex, as the query graph contains hyperedges and not just regular join edges. It checks if $\bowtie_2$ could occur in a subtree of $\bowtie_1$ and if $\bowtie_2$ need not come before $\bowtie_1$ (otherwise ordering has no effect). If they are neighboring, we compute the

```

SimplifyGraph( $G = (V, E)$ )
 $j_1 = \emptyset, j_2 = \emptyset, M = -\infty$ 
// Find the most beneficial simplification
for each  $S_1^L \bowtie_1 S_1^R \in E$ 
  for each  $S_2^L \bowtie_2 S_2^R \in E, \bowtie_1 \neq \bowtie_2$ 
    // Does  $\bowtie_1$  neighbor  $\bowtie_2$ ?
    if  $((S_2^L \subseteq S_1^L \vee S_2^R \subseteq S_1^R) \wedge (S_2^L \cup S_2^R \not\subseteq S_1^L)) \vee$ 
       $((S_2^L \subseteq S_1^R \vee S_2^R \subseteq S_1^L) \wedge (S_2^L \cup S_2^R \not\subseteq S_1^R))$ 
       $b = \text{orderingBenefit}(S_1^L \bowtie_1 S_1^R, S_2^L \bowtie_2 S_2^R)$ 
      if  $b > M \wedge (\bowtie_2 \text{ could be ordered before } \bowtie_1)$ 
         $j_1 = S_1^L \bowtie_1 S_1^R, j_2 = S_2^L \bowtie_2 S_2^R, M = b$ 
// No further simplification possible?
if  $j_1 = \emptyset$  return  $G$ 
// Order  $j_2 = S_2^L \bowtie_2 S_2^R$  before  $j_1 = S_1^L \bowtie_1 S_1^R$ 
if  $(S_2^L \subseteq S_1^L \vee S_2^R \subseteq S_1^R) \wedge (S_2^L \cup S_2^R \not\subseteq S_1^L)$ 
  return  $(V, E \setminus \{j_1\} \cup \{(S_1^L \cup S_2^L \cup S_2^R) \bowtie_1 S_1^R\})$ 
else
  return  $(V, E \setminus \{j_1\} \cup \{S_1^L \bowtie_1 (S_1^R \cup S_2^L \cup S_2^R)\})$ 

```

Figure 22.3: Pseudo-Code for a Single Simplification Step

expected benefit of ordering $\bowtie_2$ before $\bowtie_1$. The implementation of *orderingBenefit* is orthogonal to the simplification itself, it should predict how likely it is that $\bowtie_2$ must come before $\bowtie_1$ (see Section 22.3.3). We restrict ourselves to ordering neighboring joins as it is hard to make useful predictions about arbitrary unrelated joins. Note that through a series of simplification steps the join neighborhoods increase, such that the algorithm can ultimately order all joins if needed. The algorithm remembers the join pair (j_1, j_2) with the maximum estimated benefit, and modifies the query graph such that j_2 must come before j_1 . This creates an hyperedge in the query graph, as now j_1 'requires' all relations involved in j_2 to guarantee the ordering, effectively shrinking the search space.

A detail of the pseudo-code not discussed yet is the condition ' $\bowtie_2$ could be ordered before $\bowtie_1$ ' in the first loop. So far we have assumed that it is indeed possible to order $\bowtie_2$ before $\bowtie_1$, but this might not be the case: First, the query might contain non-inner joins, which are not freely reorderable. Second, if the query is cyclic, a series of simplification steps could lead to a contradiction, demanding (transitively) that $\bowtie_1$ must come before $\bowtie_2$ and $\bowtie_2$ before $\bowtie_1$. To avoid this, we build a partial ordering of joins as a directed graph, deriving the initial one from the original query hypergraph and then ordering the joins as indicated by the simplification step. The condition ' $\bowtie_2$ could be ordered before $\bowtie_1$ ' is effectively a check if an edge $\bowtie_2 \rightarrow \bowtie_1$ would create a cycle in our graph.

In general the performance of *SimplifyGraph* can be improved significantly by maintaining proper data structures. As we will see in the next section, the algorithm is applied repeatedly to simplify a query graph, thus it pays off to remember already computed results. We therefore materialize all neighbors of a

join, and update the neighbors when a join is modified. Further, we remember the estimated benefit for each neighbor, and keep all joins in a priority queue ordered by the maximum benefit they can get from ordering a neighbor. This eliminates the two nested loops in the algorithm, and greatly improves runtime.

22.3.2 The Full Algorithm

The full algorithm works in two phases: First, it repeatedly performs simplification steps until the problem complexity has decreased enough, and then it runs a dynamic programming algorithm to find the optimal solution for the simplified graph. To check if the graph has been simplified enough, we can reuse the DPccp algorithm [618], where the complexity of the dynamic programming algorithm depends on the number of connected subgraphs in the query graph. More precisely, the number of connected subgraphs is identical to the size of the DP table after the optimization is finished. We can therefore simplify the query graph until the number of connected subgraphs decreased sufficiently.

Counting the number of connected subgraphs is not that trivial, but an algorithm follows naturally from graph-based query optimization: The DPhyp algorithm [619] solves the join ordering problem by enumerating all connected subgraphs of the query graph and joining them with all connected subgraphs that are disjoint from but connected to the first subgraph. By simply eliminating the enumeration of the second subgraph we get an algorithm that produces all connected subgraphs. Note that the algorithm does not have to fill a DP table, as we are only interested in the number of connected subgraphs, and we can stop as soon as we have enumerated more than our maximum number of connected subgraphs. However, enumerating 10,000 subgraphs in a query graph with 100 relations takes roughly 5ms. This means that while checking the problem complexity is not that expensive, we cannot afford to check it after each simplifications step, as there may be thousands of simplification steps.

The full algorithm therefore operates as depicted in Figure 22.4. It is invoked by giving a query graph G and a maximum complexity budget b . It first generates all possible simplifications $\bar{G}$ by applying the *SimplifyGraph* step repeatedly. The complexity of these graphs decreases monotonically, as each simplification step adds more restrictions. Then, it performs a binary search over the list of graphs, and computes the complexity just for the currently examined graph. The graph with the least number of simplification steps that has a complexity $\leq b$ is stored in G_b . Note that G_b could be equal to G , i.e., the original problem, if the graph is simple enough. After the binary search, the optimal solution for G_b is computed by using DPhyp [619].

Again the pseudo-code is simplified. In particular it is not advisable to really materialize all query graphs in $\bar{G}$, as this becomes noticeably expensive for queries with more than 50 relations. Instead, we just remember the two joins (j_1, j_2) selected for merging by the *SimplifyGraph* step. Then we materialize the graphs examined by the binary search by replaying the merge steps based upon these (j_1, j_2) values relative to the last materialized graph. Using these techniques, the full algorithm (including the final *DPhyp* call) takes less than one second for a star query with 50 relations and a complexity budget of 10,000 in

```

GraphSimplificationOptimizer( $G = (V, E), b$ )
// Input: A query graph  $G$  and a complexity budget  $b$ 
// Output: The best plan found under the budget  $b$ 

// Compute all possible simplification steps
 $\bar{G}$  = a list of query graphs,  $G' = G$ 
do
    append  $G'$  to  $\bar{G}$ 
     $G = G', G' = \text{SimplifyGraph}(G)$ 
while  $G \neq G'$ 
// Use binary search to find the necessary simplifications
 $l = 0, r = |\bar{G}|, v = r, G_b = \bar{G}[r - 1]$ 
while  $l < r$ 
     $m = \lfloor \frac{l+r}{2} \rfloor$ 
     $c = \# \text{connected subgraphs in } \bar{G}[m] \text{ (count at most } b + 1)$ 
    if  $c > b$ 
         $l = c + 1$ 
    else
         $r = c$ 
        if  $c < v$ 
             $v = c, G_b = \bar{G}[m]$ 
// Solve the simplified graph
return  $DPhyp(G_b)$ 

```

Figure 22.4: The Full Optimization Algorithm

experiments [639]. Note that we can even avoid generating all possible merge steps: By using search techniques for unbounded search (e.g., [76]) we can generate merging steps as required by the search strategy. This does not change the asymptotic complexity, but it is more efficient if most queries require few or no simplification steps (which is probably the case in practice).

22.3.3 Join Ordering Criterion

So far we have assumed that the simplification algorithm can somehow estimate the benefit of ordering $\bowtie_2$ before $\bowtie_1$. In principle this is orthogonal to the simplification algorithm, and different kinds of ordering criterion could be used. The experiments in [639] used the following estimation function, which compares the relative costs of the join orders, and gave very good results:

$$\text{orderingBenefit}(X \bowtie_1 R_1, X \bowtie_2 R_2) = \frac{C((X \bowtie_1 R_1) \bowtie_2 R_2)}{C((X \bowtie_2 R_2) \bowtie_1 R_1)}$$

The rational here is that if joining first R_2 and then R_1 is orders of magnitude cheaper than first joining R_1 and then R_2 , it is very likely that the join with R_2 will come before the join with R_1 in the optimal solution, regardless of the other relations involved. As the simplification algorithms orders the highest

expected benefit first, it first enforces orderings where the cost differences are particularly large (and thus safe).

Note that the criterion shown above is oversimplified. First, computing the cost function C is not trivial, as we are only comparing joins and do not have complete execution plans yet. In particular information about physical properties of the input is missing, which is required by some cost functions. One way to avoid this is to use the C_{out} cost functions for the benefit estimation. The advantage of C_{out} is that it can be used without physical information, and further the optimizations based upon C_{out} are usually not that bad, as minimizing intermediate results is a plausible goal. Using the real cost function would be more attractive, but for some cost functions we can only use the real cost function in the final DP phase, as then physical information is available.

The second problem is that we are not really comparing $X \bowtie_1 R_1$ with $X \bowtie_2 R_2$, but $S_1^L \bowtie_1 S_1^R$ with $S_2^L \bowtie_2 S_2^R$, where $\bowtie_1$ and $\bowtie_2$ are neighboring hyperedges in the query graph. There are multiple cases that can occur, here we assume that $S_2^L \subseteq S_1^L$, the other cases are analogous. We define $|S|_{\bowtie}$ as the output cardinality of joining the relations in S :

$$|S|_{\bowtie} = (\Pi_{R \in S} |R|) * (\Pi_{\bowtie_i \in V_{|S}} sel(\bowtie_i)).$$

Then the joins $S_1^L \bowtie_1 S_1^R$ and $S_2^L \bowtie_2 S_2^R$ can be interpreted as $X \bowtie_1 R_1$ and $X \bowtie_2 R_2$ with $|X| = \max(|S_1^L|_{\bowtie}, |S_2^L|_{\bowtie})$, $|R_1| = |S_1^R|_{\bowtie}$, and $|R_2| = |S_2^R|_{\bowtie}$. Note that we do not have to compute the costs of joining the relations in S_i , as we are only interested in comparing the relative performance of $\bowtie_1$ and $\bowtie_2$. Note further that the accuracy of the prediction will increase over time, as the S_i grow and at some point contain all relations that will come before a join. Therefore it is important to make the 'safe' orderings early, when the uncertainty is higher, and perform the more unclear orderings later when more is known about the input.

22.3.4 Theoretical Foundation

The join ordering criterion presented in the previous section is a heuristic, and can lead to suboptimal execution plans. However, in some cases, in particular for star queries with certain cost functions, we can guarantee the optimality of the reduction.

We define that cost function C is *relative order preserving* if the following condition holds for arbitrary relations $R_0, \dots, R_3$ and arbitrary joins $\bowtie_{1,2,3}$ with independent join predicates:

$$\begin{aligned} C(R_0 \bowtie_1 R_1 \bowtie_2 R_2) &\geq C(R_0 \bowtie_2 R_2 \bowtie_1 R_1) \\ \Rightarrow C(R_0 \bowtie_3 R_3 \bowtie_1 R_1 \bowtie_2 R_2) &\geq C(R_0 \bowtie_3 R_3 \bowtie_2 R_2 \bowtie_1 R_1) \end{aligned}$$

Or, in other words, the optimal relative ordering of $\bowtie_1$ and $\bowtie_2$ remains unchanged by changing the cardinality of R_0 by a factor of α . This is closely related to the known ASI property of cost functions [438]. as it can be shown easily that every ASI cost function is relative order preserving. But relative order preserving is more general than ASI, for example a simple sort-merge-join

cost function ($C_{SM}(R_1 \bowtie R_2) = C(R_1) + C(R_2) + |R_1| \log |R_1| + |R_2| \log |R_2|$) does not satisfy the ASI property, but is relative order preserving.

As queries we consider star queries of the form $Q = (V = \{R_0, \dots, R_n\}, E = \{R_0 \bowtie_1 R_1, \dots, R_0 \bowtie_n R_n\})$ (can be guaranteed by renaming relations), and require independence between join predicates and a relative order preserving cost function C . W.l.o.g. we assume that the cost function is symmetric, as we can always construct a symmetric cost function by using $\min(C(R_i \bowtie R_j), C(R_j \bowtie R_i))$. Then, star queries have two distinct properties: First, all query plans are linear, with R_0 involved in the first join. Thus, as our cost function is symmetric, we can restrict ourselves to plans of the form $(R_0 \bowtie R_{\pi(1)}) \dots \bowtie R_{\pi(n)}$, where $\pi(i)$ defines a permutation of $[1, n]$. Second, given a non-empty join tree T and a relation $R_i \notin T$, $T' = T \bowtie R_i$ is a valid join tree and $|T'| = |T| |R_i| \frac{|R_0 \bowtie R_i|}{|R_0| |R_i|}$. Thus any (new) relation can be joined to an existing join tree and the selectivity of the join is unaffected by the relations already contained in the tree (due to the independence of join predicates). Note that while this holds for star queries, it does not hold in general. For example, clique queries also allow for an arbitrary join order, but the selectivities are affected by previously joined relations.

Using these observations, we now show the optimality for star queries:

Lemma 22.3.1 *Given a query $Q = (V, E)$, a relative order preserving cost function C and four relations $R_0, R_i, R_j, R_k \in V$ ($i \neq j \neq k \neq 0$). Then $C(R_0 \bowtie_i R_i \bowtie_j R_j) \geq C(R_0 \bowtie_j R_j \bowtie_i R_i)$ implies $C(R_0 \bowtie_i R_i \bowtie_j R_j \bowtie_k R_k) \geq C(R_0 \bowtie_j R_j \bowtie_i R_i \bowtie_k R_k)$.*

Theorem 22.3.2 *Follows directly from the fact that $(R_0 \bowtie_i R_i \bowtie_j R_j) \equiv (R_0 \bowtie_j R_j \bowtie_i R_i)$. The join $\bowtie_k$ gets the same input in both cases, and thus causes the same costs. This lemma holds even for non-star queries and arbitrary (monotonic) cost functions.*

Lemma 22.3.3 *Given a query $Q = (V, E)$, a relative order preserving cost function C and four relations $R_0, R_i, R_j, R_k \in V$ ($i \neq j \neq k \neq 0$). Then $C(R_0 \bowtie_i R_i \bowtie_j R_j) \geq C(R_0 \bowtie_j R_j \bowtie_i R_i)$ implies $C(R_0 \bowtie_k R_k \bowtie_i R_i \bowtie_j R_j) \geq C(R_0 \bowtie_k R_k \bowtie_j R_j \bowtie_i R_i)$.*

Theorem 22.3.4 *Follows from the definition of relative order preserving cost functions.*

Corollary 1 *Given a query $Q = (V, E)$, a relative order preserving cost function C , three relations $R_0, R_i, R_j \in V$ ($i \neq j \neq 0$), and two join sequences S_1, S_2 of relations in V such that $R_0 S_1 \bowtie_i R_i \bowtie_j R_j S_2$ forms a valid join tree. Then $C(R_0 \bowtie_i R_i \bowtie_j R_j) \geq C(R_0 \bowtie_j R_j \bowtie_i R_i)$ implies $C(R_0 S_1 \bowtie_i R_i \bowtie_j R_j S_2) \geq C(R_0 S_1 \bowtie_j R_j \bowtie_i R_i S_2)$.*

Theorem 22.3.5 *Follows from Lemma 22.3.1 and 22.3.3. Both assume nothing about R_k except independence, thus $\bowtie_k R_k$ could be a sequence of joins.*

Theorem 1 *Given a star query $Q = (V, E)$ and a relative order preserving cost function C . Then for any optimal join tree T and pairs of relations R_i, R_j neighbored in T (i.e., T has the form $R_0 S_1 \bowtie_i R_i \bowtie_j R_j S_2$) the following condition holds: Either $C(R_0 \bowtie_i R_i \bowtie_j R_j) \leq C(R_0 \bowtie_j R_j \bowtie_i R_i)$ or $T' = R_0 S_1 \bowtie_j R_j \bowtie_i R_i S_2$ is optimal, too.*

Theorem 22.3.6 *By contradiction. We assume that $C(R_0 \bowtie_i R_i \bowtie_j R_j) > C(R_0 \bowtie_j R_j \bowtie_i R_i)$ and T' is not optimal. By Corollary 1 we can deduce that $C(R_0 \bowtie_i R_i \bowtie_j R_j) > C(R_0 \bowtie_j R_j \bowtie_i R_i) \Rightarrow C(T') = C(R_0 S_1 \bowtie_j R_j \bowtie_i R_i S_2) \leq C(R_0 S_1 \bowtie_i R_i \bowtie_j R_j S_2) = C(T)$. This is a contradiction to the assumption that T' is not optimal*

This theorem is a strong indication that our simplification algorithm is plausible, as we know that one of the optimal solutions will satisfy the ordering constraints used by the algorithm. Unfortunately the authors of [639] were only able to prove the optimality by restricting the cost function some more (perhaps unnecessarily): A cost function C is *fully relative order preserving* if it is relative order preserving and the following condition holds for arbitrary relations $R_0, \dots, R_3$ and arbitrary joins $\bowtie_{1,2,3}$ with independent join predicates: $C(R_0 \bowtie_1 R_1 \bowtie_2 R_2) \geq C(R_0 \bowtie_2 R_2 \bowtie_1 R_1) \Rightarrow C(R_0 \bowtie_1 R_1 \bowtie_3 R_3 \bowtie_2 R_2) \geq C(R_0 \bowtie_2 R_2 \bowtie_3 R_3 \bowtie_1 R_1)$. Again, this property is satisfied by all ASI cost functions. Using this definition, we can show the optimality as follows.

Lemma 22.3.7 *Given a query $Q = (V, E)$, a fully relative order preserving cost function C , three relations $R_0, R_i, R_j \in V$ ($i \neq j \neq 0$), and three join sequences S_1, S_2, S_3 of relations in V such that $R_0 S_1 \bowtie_i R_i S_2 \bowtie_j R_j S_3$ forms a valid join tree. Then $C(R_0 \bowtie_i R_i \bowtie_j R_j) \geq C(R_0 \bowtie_j R_j \bowtie_i R_i)$ implies $C(R_0 S_1 \bowtie_i R_i S_2 \bowtie_j R_j S_3) \geq C(R_0 S_1 \bowtie_j R_j S_2 \bowtie_i R_i S_3)$.*

Theorem 22.3.8 *Follows from Corollary 1 and the definition of fully relative order preserving cost functions.*

Theorem 2 *Given a star query $Q = (V, E)$ and a fully relative order preserving cost function C . Applying the GraphSimplificationOptimizer algorithm repeatedly leads to the optimal execution plan.*

Theorem 22.3.9 *As Q is a star query, any linear join order is valid, thus join ordering is done purely based upon costs. The algorithm repeatedly orders the two joins with the largest quotient, which is guaranteed to be ≥ 1 due to the lack of join ordering constraints. Lemma 22.3.7 shows joins can be ordered relative to each other regardless of other relations, thus if the algorithm orders $\bowtie_i$ before $\bowtie_j$ there exists an optimal solution with $\bowtie_i$ before $\bowtie_j$ (analogue to Theorem 1). The algorithm simplified the graph until the joins are in a total order, which uniquely describes one optimal execution plan.*

22.4 The Time/Quality Trade-Off

One particular interesting property of the simplification algorithm is that it offers a direct trade-off between time and result quality. We therefore repeat some experimental results from [639] here that illustrate this trade-off.

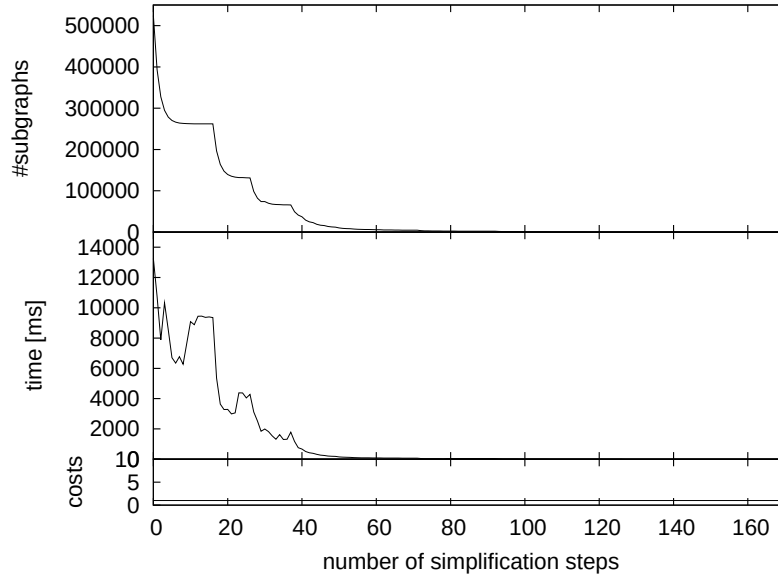


Figure 22.5: The Effect of Simplification Steps for a Star Query with 20 Relations

Clearly, each simplification step decreases the search space, i.e., the number of connected subgraphs. Ideally the optimization time goes down analogously, and, unfortunately, the costs will go up if the heuristic makes mistakes. Figure 22.5 shows how the number of connected subgraphs, the optimization time, and the scaled costs (relative to the optimal solution) change during simplification of a star query with 20 relations. As predicted, the search space shrinks monotonically with simplification. It does not shrink strictly monotonically, as the simplification algorithm sometimes adds restrictions that are already implied through other restrictions, but this is not an issue for the full algorithm due to the binary search. The optimization time follows the search space size, although there are some local peaks. Apparently they are caused by the higher costs of hyperedges for the DPhyp algorithm relative to normal edges. The scaled costs are constantly 1 here, i.e., the algorithm produces the optimal solution regardless of the number of simplification steps. This is due to the theoretical properties of the ordering heuristic (see Section 22.3.4), which in this case is optimal.

For grid queries the general situation is similar as shown in Figure 22.6. Search space and optimization time decrease similar to star queries, the costs however increase over time. Initially the heuristic performs only the relatively safe orderings, which do not cause any increases in costs, but at some point it makes a mistake in ordering and causes the costs to increase step-wise. Fortunately this happens when the search space has already been reduced a lot, which means that for simpler queries there is a reasonable hope that the heuristic will

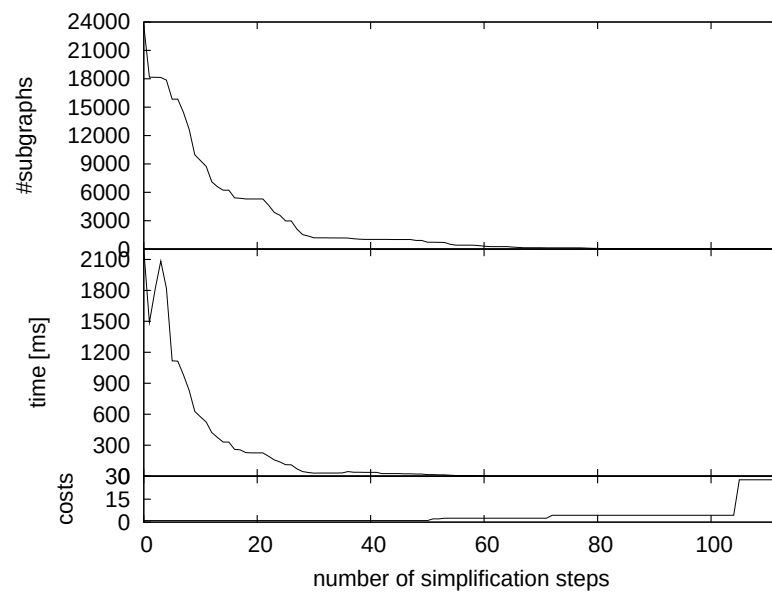


Figure 22.6: The Effect of Simplification Steps for a Grid Query with 20 Relations

never reach the point where it starts making mistakes.

Chapter 23

Deriving and Dealing with Interesting Orderings and Groupings

[This chapter was written by Thomas Neumann and Guido Moerkotte]

23.1 Introduction

The most expensive operations (e.g. join, grouping, duplicate elimination) during query evaluation can be performed more efficiently if the input is ordered or grouped in a certain way. Therefore, it is crucial for query optimization to recognize cases where the input of an operator satisfies the ordering or grouping requirements needed for a more efficient evaluation. Since a plan generator typically considers millions of different plans – and, hence, operators –, this recognition easily becomes a performance bottleneck for plan generation, often leading to heuristic solutions.

The importance of exploiting available orderings has already been recognized in the seminal work of Selinger et al [784]. They presented the concept of interesting orderings and showed how redundant sort operations could be avoided by reusing available orderings, rendering sort-based operators like sort-merge join much more interesting.

Along these lines, it is beneficial to reuse available grouping properties, for example for hash-based operators. While heuristic techniques to avoid redundant group-by operators have been given [155], for a long time groupings have not been treated as thoroughly as orderings. One reason might be that while orderings and groupings are related (every ordering is also a grouping), groupings behave somewhat differently. For example, a tuple stream grouped on the attributes $\{a, b\}$ need not be grouped on the attribute $\{a\}$. This is different from orderings, where a tuple stream ordered on the attributes (a, b) is also ordered on the attribute (a) . Since no simple prefix (or subset) test exists for groupings, optimizing groupings even in a heuristic way is much more difficult than optimizing orderings. Still, it is desirable to combine order optimization and the optimization of groupings, as the problems are related and treated sim-

ilarly during plan generation. Recently, some work in this direction has been published [911]. However, this only covers a special case of grouping. Instead, in this chapter we follow the approach presented by Neumann and Moerkotte [644, 643]

Other existing frameworks usually consider only order optimization, and experimental results have shown that the costs for order optimization can have a large impact on the total costs of query optimization [644]. Therefore, some care is needed when adding groupings to order optimization, as a slowdown of plan generation would be unacceptable.

In this chapter, we present a framework to efficiently reason about orderings and groupings. It can be used for the plan generator described in Chapter ??, but is actually an independent component that could be used in any kind of plan generator. Experimental results show that it efficiently handles orderings and groupings at the same time, with no additional costs during plan generation and only modest one time costs. Actually, the operations needed for both ordering and grouping optimization during plan generation can be performed in $O(1)$, basically allowing to exploit groupings for free.

23.2 Problem Definition

The order manager component used by the plan generator combines order optimization and the handling of grouping in one consistent set of algorithms and data structures. In this section, we give a more formal definition of the problem and the scope of the framework. First, we define the operations of ordering and grouping (Section 23.2.1 and 23.2.2). Then, we briefly discuss functional dependencies (Section 23.2.3) and how they interact with algebraic operators (Section 23.2.4). Finally, we explain how the component is actually used during plan generation (Section 23.2.5).

23.2.1 Ordering

During plan generation, many operators require or produce certain orderings. To avoid redundant sorting, it is required to keep track of the orderings a certain plan satisfies. The orderings that are relevant for query optimization are called *interesting orders* [784]. The set of *interesting orders* for a given query consists of

1. all orderings required by an operator of the physical algebra that may be used in a query execution plan for the given query, and
2. all orderings produced by an operator of the physical algebra that may be used in a query execution plan for the given query.

This includes the final ordering requested by the given query, if this is specified.

The interesting orders are *logical orderings*. This means that they specify a condition a tuple stream must meet to satisfy the given ordering. In contrast, the *physical ordering* of a tuple stream is the actual succession of tuples in the stream. Note that while a tuple stream has only one physical ordering,

it can satisfy multiple logical orderings. For example, the stream of tuples $((1, 1), (2, 2))$ with schema (a, b) has one physical ordering (the actual stream), but satisfies the logical orderings a, b, ab and ba .

Some operators, like **sort**, actually influence the physical ordering of a tuple stream. Others, like **select**, only influence the logical ordering. For example, a **sort**[**a**] produces a tuple stream satisfying the ordering (a) by actually changing the physical order of tuples. After applying **select**[**a=b**] to this tuple stream, the result satisfies the logical orderings $(a), (b), (a, b), (b, a)$, although the physical ordering did not change. Deduction of logical orderings can be described by using the well-known notion of *functional dependency* (FD) [818]. In general, the influence of a given algebraic operator on a set of logical orderings can be described by a set of functional dependencies.

We now formalize the problem. Let $R = (t_1, \dots, t_r)$ be a stream (ordered sequence) of tuples in attributes $A_1, \dots, A_n$. Then R satisfies the logical ordering $o = (A_{o_1}, \dots, A_{o_m})$ ($1 \leq o_i \leq n$) if and only if for all $1 \leq i < j \leq r$ the following condition holds:

$$\begin{aligned} & (t_i.A_{o_1} \leq t_j.A_{o_1}) \\ \wedge \quad & \forall 1 < k \leq m \quad ((\exists 1 \leq l < k (t_i.A_{o_l} < t_j.A_{o_l})) \vee \\ & ((t_i.A_{o_{k-1}} = t_j.A_{o_{k-1}}) \wedge \\ & (t_i.A_{o_k} \leq t_j.A_{o_k}))) \end{aligned}$$

Next, we need to define the inference mechanism. Given a logical ordering $o = (A_{o_1}, \dots, A_{o_m})$ of a tuple stream R , then R obviously satisfies any logical ordering that is a prefix of o including o itself.

Let R be a tuple stream satisfying both the logical ordering $o = (A_1, \dots, A_n)$ and the functional dependency $f = B_1, \dots, B_k \rightarrow B_{k+1}$ ¹ with $B_i \in \{A_1 \dots A_n\}$. Then R also satisfies any logical ordering derived from o as follows: add B_{k+1} to o at any position such that all of $B_1, \dots, B_k$ occurred before this position in o . For example, consider a tuple stream satisfying the ordering (a, b) ; after inducing the functional dependency $a, b \rightarrow c$, the tuple stream also satisfies the ordering (a, b, c) , but not the ordering (a, c, b) . Let O' be the set of all logical orderings that can be constructed this way from o and f after prefix closure. Then, we use the following notation: $o \vdash_f O'$. Let e be the equation $A_i = A_j$. Then, $o \vdash_e O'$, where O' is the prefix closure of the union of the following three sets. The first set is O_1 defined as $o \vdash_{A_i \rightarrow A_j} O_1$, the second is O_2 defined as $o \vdash_{A_j \rightarrow A_i} O_2$, and the third is the set of logical orderings derived from o where a possible occurrence of A_i is replaced by A_j or vice versa. For example, consider a tuple stream satisfying the ordering (a) ; after inducing the equation $a = b$, the tuple stream also satisfies the orderings $(a, b), (b)$ and (b, a) . Let e be an equation of the form $A = \text{const}$. Then $O' (o \vdash_e O')$ is derived from o by inserting A at any position in o . This is equivalent to $o \vdash_{\emptyset \rightarrow A} O'$. For example, consider a tuple stream satisfying the ordering (a, b) ; after inducing the equation $c = \text{const}$ the tuple stream also satisfies the orderings $(c, a, b), (a, c, b)$ and (a, b, c) .

¹Any functional dependency which is not in this form can be normalized into a set of FDs of this form.

Let O be a set of logical orderings and F a set of functional dependencies (and possibly equations). We define the sets of inferred logical orderings $\Omega_i(O, F)$ as follows:

$$\begin{aligned}\Omega_0(O, F) &:= O \\ \Omega_i(O, F) &:= \Omega_{i-1}(O, F) \cup \\ &\quad \bigcup_{f \in F, o \in \Omega_{i-1}(O, F)} O' \text{ with } o \vdash_f O'\end{aligned}$$

Let $\Omega(O, F)$ be the prefix closure of $\bigcup_{i=0}^{\infty} \Omega_i(O, F)$. We write $o \vdash_F o'$ if and only if $o' \in \Omega(O, F)$.

23.2.2 Grouping

It was shown in [911] that, similar to order optimization, it is beneficial to keep track of the groupings satisfied by a certain plan. Traditionally, group-by operators are either applied after the rest of the query has been processed or are scheduled using some heuristics [155]. However, the plan generator could take advantage of grouping properties produced e.g. by avoiding re-hashing if such information was easily available.

Analogous to order optimization, we call this *grouping optimization* and define that the set of *interesting groupings* for a given query consists of

1. all groupings required by an operator of the physical algebra that may be used in a query execution plan for the given query
2. all groupings produced by an operator of the physical algebra that may be used in a query execution plan for the given query.

This includes the grouping specified by the group-by clause of the query, if any exists.

These groupings are similar to logical orderings, as they specify a condition a tuple stream must meet to satisfy a given grouping. Likewise, functional dependencies can be used to infer new groupings.

More formally, a tuple stream $R = (t_1, \dots, t_r)$ in attributes $A_1, \dots, A_n$ satisfies the grouping $g = \{A_{g_1}, \dots, A_{g_m}\}$ ($1 \leq g_i \leq n$) if and only if for all $1 \leq i < j < k \leq r$ the following condition holds:

$$\begin{aligned}\forall 1 \leq l \leq m \quad t_i.A_{g_l} &= t_k.A_{g_l} \\ \Rightarrow \forall 1 \leq l \leq m \quad t_i.A_{g_l} &= t_j.A_{g_l}\end{aligned}$$

Two remarks are in order here. First, note that a grouping is a set of attributes and not – as orderings – a sequence of attributes. Second, note that given two groupings g and $g' \subset g$ and a tuple stream R satisfying the grouping g , R need not satisfy the grouping g' . For example, the tuple stream $((1, 2), (2, 3), (1, 4))$ with the schema (a, b) is grouped by $\{a, b\}$, but not by $\{a\}$. This is different from orderings, where a tuple stream satisfying an ordering o also satisfies all orderings that are a prefix of o .

New groupings can be inferred by functional dependencies as follows: Let R be a tuple stream satisfying both the grouping $g = \{A_1, \dots, A_n\}$ and the functional dependency $f = B_1, \dots, B_k \rightarrow B_{k+1}$ with $\{B_1, \dots, B_k\} \subseteq \{A_1, \dots, A_n\}$. Then R also satisfies the grouping $g' = \{A_1, \dots, A_n\} \cup \{B_{k+1}\}$. Let G' be the set of all groupings that can be constructed this way from g and f . Then we use the following notation: $g \vdash_f G'$. For example $\{a, b\} \vdash_{a,b \rightarrow c} \{a, b, c\}$. Let e be the equation $A_i = A_j$. Then $g \vdash_e G'$ where G' is the union of the following three sets. The first set is G_1 defined as $g \vdash_{A_i \rightarrow A_j} G_1$, the second is G_2 defined as $g \vdash_{A_j \rightarrow A_i} G_2$, and the third is the set of groupings derived from g where a possible occurrence of A_i is replaced by A_j or vice versa. For example, $\{a, b\} \vdash_{b=c} \{a, c\}$. Let e be an equation of the form $A = \text{const}$. Then $g \vdash_e G'$ is defined as $g \vdash_{\emptyset \rightarrow A} G'$. For example, $\{a, b\} \vdash_{c=\text{const}} \{a, b, c\}$.

Let G be a set of groupings and F be a set of functional dependencies (and possibly equations). We define the set of inferred groupings $\Omega_i(G, F)$ as follows:

$$\begin{aligned} \Omega_0(G, F) &:= G \\ \Omega_i(G, F) &:= \Omega_{i-1}(G, F) \cup \\ &\quad \bigcup_{f \in F, g \in \Omega_{i-1}(G, F)} G' \text{ with } g \vdash_f G' \end{aligned}$$

Let $\Omega(G, F)$ be $\bigcup_{i=0}^{\infty} \Omega_i(G, F)$. We write $g \vdash_F g'$ if and only if $g' \in \Omega(G, F)$.

23.2.3 Functional Dependencies

The reasoning about orderings and groupings assumes that the set of functional dependencies is known. The process of gathering the relevant functional dependencies is described in detail in [818, 819]. Predominantly, there are three sources of functional dependencies:

1. key constraints
2. join predicates [references constraints]
3. filter predicates
4. simple expressions

However, the algorithm makes no assumption about the functional dependencies. If for some reason an operator induces another kind of functional dependency (e.g., when using TID-based optimizations [588]), this can be handled the same way. The only important fact is that we provide the set of functional dependencies as input to the algorithm.

23.2.4 Algebraic Operators

To illustrate the propagation of orderings and groupings during query optimization, we give some rules for concrete (physical) operators in Figure 23.1. As a

operator	requires	produces
$\text{scan}(R)$	-	$O(R)$
$\text{indexscan}(Idx)$	-	$O(Idx)$
$\text{map}(S, a = f(b))$	-	$\Omega(O(S), b \rightarrow a)$
$\text{select}(S, a = b)$	-	$\Omega(O(S), a = b)$
$\text{bnl-join}(S_1, S_2)$	-	$O(S_1)$
$\text{indexnl-join}(S_1, S_2)$	-	$O(S_1)$
$\text{djoin}(S_1, S_2)$	-	$O(S_1)$
$\text{sort}(S, a_1, \dots, a_n)$	-	$(a_1, \dots, a_n)$
$\text{group-by}(S, a_1, \dots, a_n)$	-	$\{a_1, \dots, a_n\}$
$\text{hash}(S, a_1, \dots, a_n)$	-	$\{a_1, \dots, a_n\}$
$\text{sort-merge}(S_1, S_2, \vec{a} = \vec{b})$	$\vec{a} \in O(S_1) \wedge \vec{b} \in O(S_2)$	$\Omega(O(S_1), \vec{a} = \vec{b})$
$\text{hash-join}(S_1, S_2, \vec{a} = \vec{b})$	$\vec{a} \downarrow \in O(S_1) \wedge \vec{b} \downarrow \in O(S_2)$	$\Omega(O(S_1), \vec{a} = \vec{b})$

Figure 23.1: Propagation of orderings and groupings

shorthand, we use the following notation:

$O(R)$ set of logical orderings and groupings satisfied by the physical ordering of the relation R

$O(S)$ inferred set of logical orderings and groupings satisfied by the tuple stream S

$x \downarrow \quad \{y | y \in x\}$

Note that these rules somewhat depend on the actual implementation of the operators, e.g. a blockwise nested loop join might actually destroy the ordering if the blocks are stored in hash tables. The rules are also simplified: For example, a group-by will probably compute some aggregate functions, inducing new functional dependencies. Furthermore, additional information can be derived from schema information: If the right-hand side of a dependent join (index nested loop joins are similar) produces at most one tuple, and the left-hand side is grouped on the free attributes of the right-hand side (e.g. if they do not contain duplicates) the output is also grouped on the attributes of the right-hand side. This situation is common, especially for index nested loop joins, and is detected automatically if the corresponding functional dependencies are considered. Therefore, it is important that all operators consider all functional dependencies they induce.

EXC

23.2.5 Plan Generation

To exploit available logical orderings and groupings, the plan generator needs access to the combined order optimization and grouping component, which we describe as an *abstract data type* (ADT). An instance of this abstract data type **OrderingGrouping** represents a set of logical orderings and groupings, and wherever necessary, an instance is embedded into a plan node. The main operations the abstract data type **OrderingGrouping** must provide are

1. a constructor for a given logical ordering or grouping,

2. a membership test (called `containsOrdering(LogicalOrdering)`) which tests whether the set contains the logical ordering given as parameter,
3. a membership test (called `containsGrouping(Grouping)`) which tests whether the set contains the grouping given as parameter, and
4. an inference operation (called `infer(set<FD>)`). Given a set of functional dependencies and equations, it computes a new set of logical orderings and groupings a tuple stream satisfies.

These operations can be implemented by using the formalism described before: `containsOrdering` tests for $o \in O$, `containsGrouping` tests for $o \in G$ and `infer(F)` calculates $\Omega(O, F)$ respectively $\Omega(G, F)$. Note that the intuitive approach to explicitly maintain the set of all logical orderings and groupings is not useful in practice. For example, if a sort operator sorts a tuple stream on (a, b) , the result is compatible with logical orderings $\{(a, b), (a)\}$. After a selection operator with selection predicate $x = \text{const}$ is applied, the set of logical orderings changes to $\{(x, a, b), (a, x, b), (a, b, x), (x, a), (a, x), (x)\}$. Since the size of the set increases quadratically with every additional selection predicate of the form $v = \text{const}$, a naive representation as a set of logical orderings is problematic. This led Simmen et al. to introduce a more concise representation, which is discussed in the related work section. Note that Simmen's technique is not easily applicable to groupings, and no algorithm was proposed to efficiently maintain the set of available groupings. The order optimization component described here closes this gap by supporting both orderings and groupings. The problem of quadratic growth is avoided by only implicitly representing the set.

23.3 Overview

As we have seen, explicit maintenance of the set of logical orderings and groupings can be very expensive. However, the ADT `OrderingGrouping` required for plan generation does not need to offer access to this set: It only allows to test if a given interesting order or grouping is in the set and changes the set according to new functional dependencies. Hence, it is *not* required to explicitly represent this set; an implicit representation is sufficient as long as the ADT operations can be implemented atop of it. In other words, we need not be able to reconstruct the set of logical orderings and groupings from the state of the ADT. This gives us room for optimizations.

The initial idea (see [644]) was to represent sets of logical orderings as *states* of a *finite state machine* (FSM). Roughly, a state of the FSM represents a current physical ordering and the set of logical orderings that can be inferred from it given a set of functional dependencies. The edges (transitions) in the FSM are labeled by sets of functional dependencies. They lead from one state to another, if the target state of the edge represents the set of logical orderings that can be derived from the orderings the edge's source node represents by applying the set of functional dependencies the edge is labeled with. We have

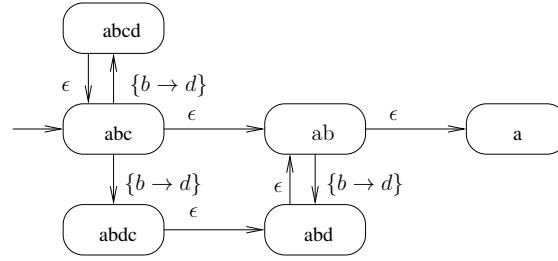


Figure 23.2: Possible FSM for orderings

to use sets of functional dependencies, since a single algebraic operator may introduce more than one functional dependency.

Let us illustrate the idea by a simple example and then discuss some problems. In Figure 23.2, an FSM for the interesting order (a, b, c) and its prefixes (remember that we need prefix closure) and the set of functional dependencies $\{b \rightarrow d\}$ is given. When a physical ordering satisfies (a, b, c) , it also satisfies its prefixes (a, b) and (a) . This is indicated by the ϵ transitions. The functional dependency $b \rightarrow d$ allows to derive the *logical* orderings (a, b, c, d) and (a, b, d, c) . This is handled by assuming that the *physical* ordering changes to either (a, b, c, d) or (a, b, d, c) . Hence, these states have to be added to the FSM. We further add the transitions induced by $\{b \rightarrow d\}$. Note that the resulting FSM is a *non-deterministic finite state machine* (NFSM).

Assume we have an NFSM as above. Then (while ignoring groupings) the state of the ADT is a state of the NFSM and the operations of the ADT can easily be mapped to the FSM. Testing for a logical ordering can be performed by checking if the node with the ordering is reachable from the current state by following ϵ edges. If the set must be changed because of a functional dependency the state is changed by following the edge labeled with the functional dependency. Of course, the non-determinism is in our way.

While remembering only the active state of the NFSM avoids the problem of maintaining a set of orderings, the NFSM is not really useful from a practical point of view, since the transitions are non-deterministic. Nevertheless, the NFSM can be considered as a special *non-deterministic finite automaton* (NFA), which consumes the functional dependencies and "recognizes" the possible physical orderings. Further, an NFA can be converted into a *deterministic finite automaton* (DFA), which can be handled efficiently. Remember that the construction is based on the power set of the NFA's states. That is, the states of the DFA are sets of states of the NFA [553]. We do not take the deviation over the finite automaton but instead lift the construction of deterministic finite automata from non-deterministic ones to finite state machines. Since this is not a traditional conversion, we give a proof of this step in Section ??.

Yet another problem is that the conversion from an NFSM to a *deterministic FSM* (DFSM) can be expensive for large NFSMs. Therefore, reducing the size of the NFSM is another problem we look at. We introduce techniques for reducing the set of functional dependencies that have to be considered and

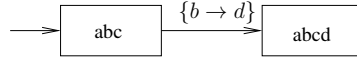


Figure 23.3: Possible FSM for groupings

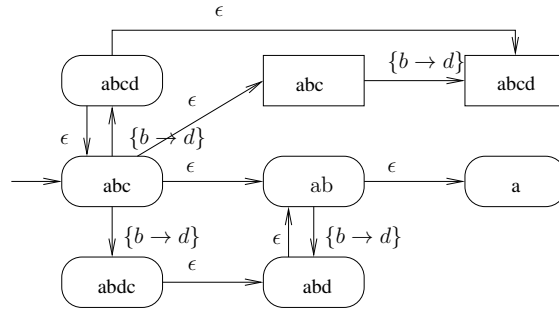


Figure 23.4: Combined FSM for orderings and groupings

further techniques to prune the NFSM in Section 23.4.7.

The idea of a combined framework for orderings and groupings was presented in [643]. Here, the main point is to construct a similar FSM for groupings and integrate it into the FSM for orderings, thus handling orderings and groupings at the same time. An example of this is shown in Figure 23.3. Here, the FSM for the grouping $\{a, b, c\}$ and the functional dependency $b \rightarrow c$ is shown. We represent states for orderings as rounded boxes and states for groupings as rectangles. Note that although the FSM for groupings has a start node similar to the FSM for orderings, it is much smaller. This is due to the fact that groupings are only compatible with themselves, no nodes for prefixes are required. However, the FSM is still non-deterministic: given the functional dependency $b \rightarrow c$, the grouping $\{a, b, c, d\}$ is compatible with $\{a, b, c, d\}$ itself and with $\{a, b, c\}$; therefore, there exists an (implicit) edge from each grouping to itself.

The FSM for groupings is integrated into the FSM for orderings by adding ϵ edges from each ordering to the grouping with the same attributes; this is due to the fact that every ordering is also a grouping. Note that although the ordering (a, b, c, d) also implies the grouping $\{a, b, c\}$, no edge is required for this, since there exists an ϵ edge to (a, b, c) and from there to $\{a, b, c\}$.

After constructing a combined FSM as described above, the full ADT supporting both orderings and groupings can easily be mapped to the FSM: The

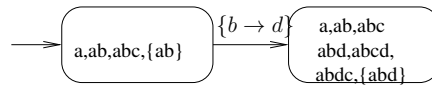


Figure 23.5: Possible DFSM for Figure 23.4

state of the ADT is a state of the FSM and testing for a logical ordering or grouping can be performed by checking if the node with the ordering or grouping is reachable from the current state by following ϵ edges (as we will see, this can be precomputed to yield the $O(1)$ time bound for the ADT operations). If the state of the ADT must be changed because of functional dependencies, the state in the FSM is changed by following the edge labeled with the functional dependency.

However, the non-determinism of this transition is a problem. Therefore, for practical purposes the NFSM must be converted into a DFSM. The resulting DFSM is shown in Figure 23.5. Note that although in this simple example the DFSM is very small, the conversion could lead to exponential growth. Therefore, additional pruning techniques for groupings are presented in Section 23.4.7. However, the inclusion of groupings is not critical for the conversion, as the grouping part of the NFSM is nearly independent of the ordering part. In Section 23.5 we look at the size increase due to groupings. The memory consumption usually increases by a factor of two, which is the minimum expected increase, since every ordering is a grouping.

Some operators, like *sort*, change the physical ordering. In the NFSM, this is handled by changing the state to the node corresponding to the new physical ordering. Implied by its construction, in the DFSM this new physical ordering typically occurs in several nodes. For example, (a, b, c) occurs in both nodes of the DFSM in Figure 23.5. It is, therefore, not obvious which node to choose. We will take care of this problem during the construction of the NFSM (see Section 23.4.3).

23.4 Detailed Algorithm

23.4.1 Overview

Our approach consists of two phases. The first phase is the preparation step taking place before the actual plan generation starts. The output of this phase are the precomputed values used to implement the ADT. Then the ADT is used during the second phase where the actual plan generation takes place. The first phase is performed exactly once and is quite involved. Most of this section covers the first phase. Only Section 23.4.6 deals with the ADT implementation.

Figure 23.6 gives an overview of the preparation phase. It is divided into four major steps, which are discussed in the following subsections. Subsection 23.4.2 briefly reviews how the input to the first phase is determined and, more importantly, what it looks like. Section 23.4.3 describes in detail the construction of the NFSM from the input. The conversion from the NFSM to the DFSM is only briefly sketched in Section 23.4.4, for details see [553]. From the DFSM some values are precomputed which are then used for the efficient implementation of the ADT. The precomputation is described in Section 23.4.5, while their utilization and the ADT implementation are the topic of Section 23.4.6. Section 23.4.7 contains some important techniques to reduce the size of the NFSM. They are applied in Steps 2 (b), 2 (c) and 2 (e). During the discussion, we illustrate the different steps by a simple running example. More complex

1. Determine the input
 - (a) Determine interesting orders
 - (b) Determine interesting groupings
 - (c) Determine set of functional dependencies
2. Construct the NFSM
 - (a) Construct states of the NFSM
 - (b) Filter functional dependencies
 - (c) Build filters for orderings and groupings
 - (d) Add edges to the NFSM
 - (e) Prune the NFSM
 - (f) Add artificial start state and edges
3. Convert the NFSM into a DFSM
4. Precompute values
 - (a) Precompute the compatibility matrix
 - (b) Precompute the transition table

Figure 23.6: Preparation steps of the algorithm

examples can be found in Section 23.5.

23.4.2 Determining the Input

Since the preparation step is performed immediately before plan generation, it is assumed that the query optimizer has already determined which indices are applicable and which algebraic operators can possibly be used to construct the query execution plan.

Before constructing the NFSM, the set of interesting orders, the set of interesting groupings and the sets of functional dependencies for each algebraic operator are determined. We denote the set of sets of functional dependencies by $\mathcal{F}$. It is important for the correctness of our algorithms that we note which of the interesting orders are (1) produced by some algebraic operator or (2) only tested for. Note that the interesting orders which satisfy (1) may additionally be tested for as well. We denote those orderings under (1) by O_P , those under (2) by O_T . The total set of interesting orders is defined as $O_I = O_P \cup O_T$. The orders produced are treated slightly differently in the following steps. The groupings are classified similarly to the orderings: We denote the grouping produced by some algebraic operator by G_P , and those just tested for by G_T . The total set of interesting groupings is defined as $G_I = G_P \cup G_T$. More information on how to extract interesting groupings can be found in [911]. Furthermore, for a sample query the extraction of both interesting orders and groupings is

ToDo: details on determining interesting orders?

illustrated in Section 23.5.

To illustrate subsequent steps, we assume that the set of sets of functional dependencies

$$\mathcal{F} = \{\{b \rightarrow c\}, \{b \rightarrow d\}\},$$

the interesting groupings

$$G_I = \{\{b\}\} \cup \{\{b, c\}\}$$

and the interesting orders

$$O_I = \{(b), (a, b)\} \cup \{(a, b, c)\}$$

have been extracted from the query. We assume that those in $O_T = \{(a, b, c)\}$ and $G_T = \{\{b, c\}\}$ are tested for but not produced by any operator, whereas those in $O_P = \{(b), (a, b)\}$ and $G_P = \{\{b\}\}$ may be produced by some algebraic operators.

23.4.3 Constructing the NFSM

An NFSM consists of a tuple (Σ, Q, D, q_o) , where

- Σ is the input alphabet,
- Q is the set of possible states,
- $D \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation, and
- q_o is the initial state.

Coarsely, Σ consists of the functional dependencies, Q of the relevant orderings and groupings, and D describes how the orderings or groupings change under a given functional dependency. Some refinements are needed to provide efficient ADT operations. The details of the construction are described now.

For the order optimization part the states are partitioned in $Q = Q_I \cup Q_A \cup \{q_o\}$, where q_o is an artificial state to initialize the ADT, Q_I is the set of states corresponding to interesting orderings and Q_A is a set of artificial states only required for the algorithm itself. Q_A is described later. Furthermore, the set Q_I is partitioned in Q_I^P and Q_I^T , representing the orderings in O_P and O_T , respectively. To support groupings, we add to Q_I^P states corresponding to the groupings in G_P and to Q_I^T states corresponding to the groupings in G_T .

The initial NFSM contains the states Q_I of interesting groupings and orderings. For the example, this initial construction not including the start state q_o is shown in Figure 23.7. The states representing groupings are drawn as rectangles and the states representing orderings are drawn with rounded corners.

When considering functional dependencies, additional groupings and orderings can occur. These are not directly relevant for the query, but have to be represented by states to handle transitive changes. Since they have no direct connection to the query, these states are called artificial states. Starting with

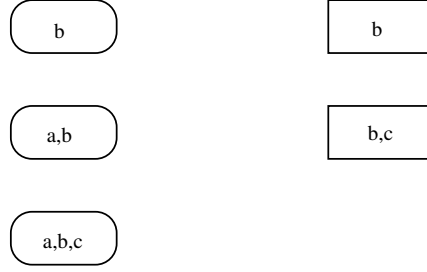


Figure 23.7: Initial NFSM for sample query

the initial states Q_I , artificial states are constructed by considering functional dependencies

$$Q_A = (\Omega(O_I, \mathcal{F}) \setminus O_I) \cup (\Omega(G_I, \mathcal{F}) \setminus G_I).$$

In our example, this creates the states (b, c) and (a) , as (b, c) can be inferred from (b) when considering $\{b \rightarrow c\}$ and (a) can be inferred from (a, b) , since (a) is a prefix of (a, b) . The result is shown in Figure 23.8 (ignore the edges).

Sometimes the ADT has to be explicitly initialized with a certain ordering or grouping (e.g. after a `sort`). To support this, artificial edges are added later on. These point to the requested ordering or grouping (states in Q_I^P) and are labeled with the state that they lead to. Therefore, the input alphabet Σ consists of the sets of functional dependencies and produced orderings and groupings:

$$\Sigma = \mathcal{F} \cup Q_I^P \cup \{\epsilon\}.$$

In our example, $\Sigma = \{\{b \rightarrow c\}, \{b \rightarrow d\}, (b), (a, b), \{b\}\}$.

Accordingly, the domain of the transition relation D is

$$\begin{aligned} D \subseteq & ((Q \setminus \{q_0\}) \times (\mathcal{F} \cup \{\epsilon\}) \times (Q \setminus \{q_0\})) \\ & \cup (\{q_0\} \times Q_I^P \times Q_I^P). \end{aligned}$$

The edges are formed by the functional dependencies and the artificial edges. Furthermore, ϵ edges exist between orderings and the corresponding groupings, as orderings are a special case of grouping:

$$\begin{aligned} D_{FD} &= \{(q, f, q') \mid q \in Q, f \in \mathcal{F} \cup \{\epsilon\}, q' \in Q, q \vdash_f q'\} \\ D_A &= \{(q_0, q, q) \mid q \in Q_I^P\} \\ D_{OG} &= \{(o, \epsilon, g) \mid o \in \Omega(O_I, \mathcal{F}), g \in \Omega(G_I, \mathcal{F}), o \equiv g\} \\ D &= D_{FD} \cup D_A \cup D_{OG} \end{aligned}$$

First, the edges corresponding to functional dependencies are added (D_{FD}). In our example, this results in the NFSM shown in Figure 23.8.

Note that the functional dependency $b \rightarrow d$ has been pruned, since d does not occur in any interesting order or grouping. The NFSM can be further simplified by pruning the artificial state (b, c) , which cannot lead to a new interesting order. The result is shown in Figure 23.9. A detailed description of these pruning techniques can be found in Section 23.4.7.

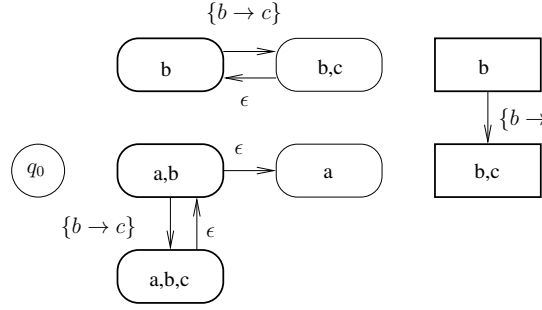


Figure 23.8: NFSM after adding D_{FD} edges

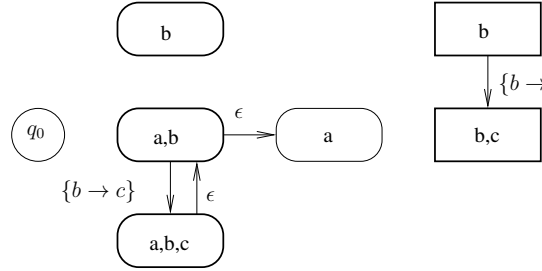


Figure 23.9: NFSM after pruning artificial states

The artificial start state q_0 has emanating edges incident to all states representing interesting orders in O_I^P and interesting groupings in G_I^P (D_A). Also, the states representing orderings have edges to their corresponding grouping states (D_{OG}), as every ordering is also a grouping. The final NFSM for the example is shown in Figure 23.10. Note that the states representing (a, b, c) and $\{b, c\}$ are not linked by an artificial edge since it is only tested for, as they are in Q_I^T .

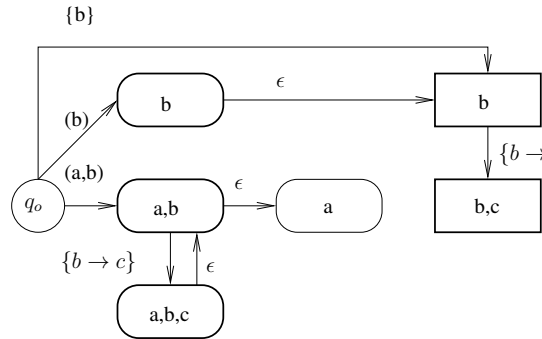


Figure 23.10: Final NFSM

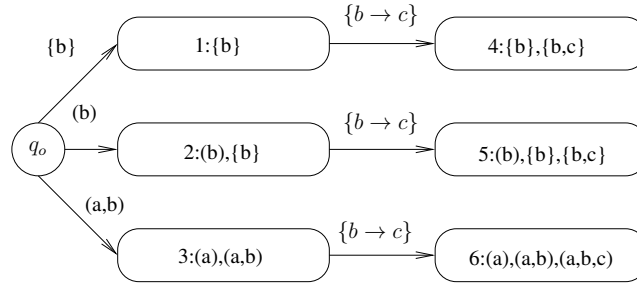


Figure 23.11: Resulting DFSM

state	1: (a)	2: (a,b)	3: (a,b,c)	4: (b)	5: {b}	6: {b,c}
1	0	0	0	0	1	0
2	0	0	0	1	1	0
3	1	1	0	0	0	0
4	0	0	0	0	1	1
5	0	0	0	1	1	1
6	1	1	1	0	0	0

Figure 23.12: *contains* Matrix

23.4.4 Constructing the DFSM

The construction of the DFSM from the NFSM follows the standard power set construction that is used to translate an NFA into a DFA [553]. A formal description and a proof of correctness is given in Section ?? . It is important to note that this construction preserves the start state and the artificial edges. The resulting DFSM for the example is shown in Figure 23.11.

state	1: $\{b \rightarrow c\}$	2: (a, b)	3: (b)	4: {b}
q_o	-	3	2	1
1	4	-	-	-
2	5	-	-	-
3	6	-	-	-
4	4	-	-	-
5	5	-	-	-
6	6	-	-	-

Figure 23.13: *transition* Matrix

23.4.5 Precomputing Values

To allow for an efficient precomputation of values, every occurrence of an interesting order, interesting grouping or set of functional dependencies is replaced by integers. This allows comparisons in constant time (equivalent entries are mapped to same integer). Further, the DFSM is represented by an adjacency matrix.

The precomputation step itself computes two matrices. The first matrix denotes whether an NFSM state in Q_I is active, i.e. an interesting order or an interesting grouping, is contained in a specific DFSM state. This matrix can be represented as a compact bit vector, allowing tests in $O(1)$. For our running example, it is given (in a more readable form) in Figure 23.12. The second matrix contains the transition table for the DFSM relation D . Using it, edges in the DFSM can be followed in $O(1)$. For the example, the transition matrix is given in Figure 23.13.

23.4.6 During Plan Generation

During plan generation, larger plans are constructed by adding algebraic operators to existing (sub-)plans. Each subplan contains the available orderings and groupings in the form of the corresponding DFSM state. Hence, the state of the DFSM, a simple integer, is the state of our ADT `OrderingGrouping`.

When applying an operator to subplans, the ordering and grouping requirements are tested by checking whether the DFSM state of the subplan contains the required ordering or grouping of the operator. This is done by a simple lookup in the *contains* matrix.

If the operator introduces a new set of functional dependencies, the new state of the ADT is computed by following the according edge in the DFSM. This is performed by a quick lookup in the *transition* matrix.

For “atomic” subplans like table or index scans, the ordering and grouping is determined explicitly by the operator. The state of the DFSM is determined by a lookup in the transition matrix with start state q_o and the edge annotated by the produced ordering or grouping. For sort and group-by operators the state of the DFSM is determined as before by following the artificial edge for the produced ordering or grouping and then reapplying the set of functional dependencies that currently hold.

In the example, a sort on (b) results in a subplan with ordering/grouping state 2 (the state 2 is active in the DFSM), which satisfies the ordering (b) and the grouping $\{b\}$. After applying an operator which induces $b \rightarrow c$, the ordering/grouping changes to state 5 which also satisfies $\{b, c\}$.

23.4.7 Reducing the Size of the NFSM

Reducing the size of the NFSM is important for two reasons: First, it reduces the amount of work needed during the preparation step, especially the conversion from NFSM to DFSM. Even more important is that a reduced NFSM results in a smaller DFSM. This is crucial for plan generation, since it reduces

the search space: Plans can only be compared and pruned if they have comparable ordering and a comparable set of functional dependencies (see [818, 819] for details). Reducing the size of the DFSM removes information that is not relevant for plan generation and, therefore, allows a more aggressive pruning of plans.

At first, the functional dependencies are pruned. Here, functional dependencies which can never lead to a new interesting order or grouping are removed. For convenience, we extend the definition of $\Omega(O, F)$ and define

$$\Omega(O, \epsilon) := \Omega(O, \emptyset).$$

Then the set of prunable functional dependencies F_P can be described by

$$\begin{aligned} \Omega_N(o, f) &:= \Omega(\{o\}, \{f\}) \setminus \Omega(\{o\}, \epsilon) \\ F_P &:= \{f \mid f \in F \wedge \forall o \in O_I \cup G_I : \\ &\quad (\Omega(\Omega_N(o, f), F) \setminus \Omega(\{o\}, \epsilon)) \cap (O_I \cup G_I) = \emptyset\}. \end{aligned}$$

Pruning functional dependencies is especially useful, since it also prunes artificial states that would be created because of the dependencies. In the example, this removed the functional dependency $b \rightarrow d$, since d does not appear in any interesting order or grouping. This step also removes the artificial states containing d .

The artificial states are required to build the NFSM, but they are not visible outside the NFSM. Therefore, they can be pruned and merged without affecting plan generation. Two heuristics are used to reduce the set of artificial states:

1. All artificial nodes that behave exactly the same (that is, their edges lead to the same states given the same input) are merged and
2. all edges to artificial states that can reach states in Q_I only through ϵ edges are replaced with corresponding edges to the states in Q_I .

More formally, the following pairs of states can be merged:

$$\begin{aligned} \{(o_1, o_2) \mid & o_1 \in Q_A, o_2 \in Q_A \wedge \forall f \in F : \\ & (\Omega(\{o_1\}, \{f\}) \setminus \Omega(\{o_1\}, \epsilon)) = \\ & (\Omega(\{o_2\}, \{f\}) \setminus \Omega(\{o_2\}, \epsilon))\}. \end{aligned}$$

The following states can be replaced with the next state reachable by an ϵ edge:

$$\begin{aligned} \{o \mid & o \in Q_A \wedge \forall f \in F : \\ & \Omega(\Omega(\{o\}, \epsilon), \{f\}) \setminus \{o\} = \\ & \Omega(\Omega(\{o\}, \epsilon) \setminus \{o\}, \{f\})\}. \end{aligned}$$

In the example, this removed the state (b, c) , which was artificial and only led to the state (b) .

These techniques reduce the size of the NFSM, but still most states are artificial states, i.e. they are only created because they can be reached by considering functional dependencies when a certain ordering or grouping is available. But many of these states are not relevant for the actual query processing.

For example, given a set of interesting orders which consists only of a single ordering (a) and a set of functional dependencies which consists only of $a \rightarrow b$, the NFSM will contain (among others) two states: (a) and (a, b) . The state (a, b) is created since it can be reached from (a) by considering the functional dependency, however, it is irrelevant for the plan generation, since (a, b) is not an interesting order and is never created nor tested for. Actually, in the example above, the whole functional dependency would be pruned (since b never occurs in an interesting order), but the problem remains for combinations of interesting orders: Given the interesting orders (a) , (b) and (c) and the functional dependencies $\{a \rightarrow b, b \rightarrow a, b \rightarrow c, c \rightarrow b\}$, the NFSM will contain states for all permutations of a , b and c . But these states are completely useless, since all interesting orders consist only of a single attribute and, therefore, only the first entry of an ordering is ever tested.

Ideally, the NFSM should only contain states which are relevant for the query; since this is difficult to ensure, a heuristic can be used which greatly reduces the size of the NFSM and still guarantees that all relevant states are available: When considering a functional dependency of the form $a \rightarrow b$ and an ordering $o_1, o_2, \dots, o_n$ with $o_i = a$ for some i ($1 \leq i \leq n$), the b can be inserted at any position j with $i < j \leq n+1$ (for the special case of a condition $a = b$, $i = j$ is also possible). So, an entry of an ordering can only affect entries on the right of its own position. This means that it is unnecessary to consider those parts of an ordering which are behind the length of the longest interesting order; since that part cannot influence any entries relevant for plan generation, it can be omitted. Therefore, the orderings created by functional dependencies can be cut off after the maximum length of interesting orders, which results in less possible combinations and a smaller NFSM.

The space of possible orderings can be limited further by taking into account the prefix of the ordering: before inserting an entry b in an ordering $o_1, o_2, \dots, o_n$ at the position i , check if there is actually an interesting order with the prefix $o_1, o_2, \dots, o_{i-1}, b$ and stop inserting if no interesting order is found. Also limit the new ordering to the length of the longest matching interesting order; further attributes will never be used. If functional dependencies of the form $a = b$ occur, they might influence the prefix of the ordering and the simple test described above is not sufficient. Therefore, a representative is chosen for each equivalence class created by these dependencies, and for the prefix test the attributes are replaced with their representatives. Since the set of interesting orders with a prefix of $o_1, \dots, o_n$ is a superset of the set for the prefix $o_1, \dots, o_n, o_{n+1}$, this heuristic can be implemented very efficiently by iterating over i and reducing the set as needed.

Additional techniques can be used to avoid creating superfluous artificial states for groupings: First, in Step 2.3 (see Figure 23.6) the set of attributes occurring in interesting groupings is determined:

$$A_G = \{a \mid \exists g \in G_I : a \in g\}$$

Now, for every attribute a occurring on the right-hand side of a functional

dependency the set of potentially reachable relevant attributes is determined:

$$\begin{aligned}
 r(a, 0) &= \{a\} \\
 r(a, n) &= r(a, n-1) \cup \\
 &\quad \{a' \mid \exists (a_1 \dots a_m \rightarrow a') \in \mathcal{F} : \\
 &\quad \quad \{a_1 \dots a_m\} \cap r(a, n-1) \neq \emptyset\} \\
 r(a) &= r(a, |\mathcal{F}|) \cap A_G
 \end{aligned}$$

This can be used to determine if a functional dependency actually adds useful attributes. Given a functional dependency $a_1 \dots a_n \rightarrow a$ and a grouping g with $\{a_1 \dots a_n\} \subseteq g$, a should only be added to g if $r(a) \not\subseteq g$, i.e. the attribute might actually lead to a new interesting grouping. For example, given the interesting groupings $\{a\}, \{a, b\}$ and the functional dependencies $a \rightarrow c, a \rightarrow d, d = b$. When considering the grouping $\{a\}$, the functional dependency $a \rightarrow c$ can be ignored, as it can only produce the attribute c , which does not occur in an interesting grouping. However, the functional dependency $a \rightarrow d$ should be added, since transitively the attribute b can be produced, which does occur in an interesting grouping.

Since there are no ϵ edges between groupings, i.e. groupings are not compatible with each other, a grouping can only be relevant for the query if it is a subset of an interesting ordering (as further attributes could be added by functional dependencies). However, a simple subset test is not sufficient, as equations of the form $a = b$ are also supported; these can effectively rename attributes, resulting in a slightly more complicated test:

In Step 2.3 (see Figure 23.6) the equivalence classes induced by the equations in $\mathcal{F}$ are determined and for each class a representative is chosen (a and $a_1 \dots a_n$ are attributes occurring in the G_I):

$$\begin{aligned}
 E(a, 0) &= \{a\} \\
 E(a, n) &= E(a, n-1) \cup \\
 &\quad \{a' \mid ((a = a') \in \mathcal{F}) \vee ((a' = a) \in \mathcal{F})\} \\
 E(a) &= E(a, |\mathcal{F}|) \\
 e(a) &= \text{a representative chosen from } E(a) \\
 e(\{a_1 \dots a_n\}) &= \{e(a_1) \dots e(a_n)\}.
 \end{aligned}$$

Using these equivalence classes, a mapped set of interesting groupings is produced that will be used to test if a grouping is relevant:

$$G_I^E = \{e(g) \mid g \in G_I\}$$

Now a grouping g can be pruned if $\exists g' \in G_I^E : e(g) \subseteq g'$. For example, given the interesting grouping $\{a\}$ and the equations $a = b, b = c$, the grouping $\{d\}$ can be pruned, as it will never lead to an interesting grouping; however, the groupings $\{b\}$ and $\{c\}$ have to be kept, as they could change to an interesting grouping later on.

Note that although they appear to test similar conditions, the first pruning technique (using $r(a)$) is not dominated by the second one (using $e(a)$). Consider e.g. the interesting grouping $\{a\}$, the equation $a = b$ and the functional dependency $a \rightarrow b$. Using only the second technique, the grouping $\{a, b\}$ would be created, although it is not relevant.

23.4.8 Complex Ordering Requirements

Specifying the ordering requirements of an operator can be surprisingly difficult. Consider the following SQL query:

```
select *
from   S s, R r
where  r.a=s.a and r.b=s.b and
       r.c=s.c and r.d=s.d
```

When answering this query using a sort-merge join, the operator has to request a certain ordering. But there are many orderings that could be used: The intuitive ordering would be $abcd$, but $adcb$ or any other permutation could have been used as well. This is problematic, as checking for an exponential number of possibilities is not acceptable in general. Note that this problem is not specific to our approach, the same is true, e.g., for Simmen's approach.

The problem can be solved by defining a total ordering between the attributes, such that a canonical ordering can be constructed. We give some rules how to derive such an ordering below, but it can happen that such an ordering is unavailable (or rather the construction rules are ambiguous). Given, for example, two indices, one on $abcd$ and one on $adcb$, both orderings would be a reasonable choice. If this happens, the operators have two choices: Either they accept all reasonable orderings (which could still be an exponential number, but most likely only a few orderings remaining) or they limit themselves to one ordering, which could induce unnecessary sort operators. Probably the second choice is preferable, as the ambiguous case should be rare and does not justify the complex logic of the first solution.

The attribute ordering can be derived by using the following heuristical rules:

1. Only attributes that occur in sets without natural ordering (i.e. complex join predicates or grouping attributes) have to be ordered.
2. Orderings that are given (e.g., indices, user-requested orderings etc.) order some attributes.
3. Small orderings should be considered first. If an operator requires an ordering with the attributes abc , and another operator requires an ordering with the attributes bc , the attributes b and c should come before a .
4. The attributes should be ordered according to equivalence classes. If a is ordered before b , all orderings in $E(a)$ should be ordered before all orderings in $E(b)$.

n	#Edges	t (ms)	#Plans	t/plan	t (ms)	#Plans	t/plan	% t	% #Plans	%. t/plan
5	n-1	2	1541	1.29	1	1274	0.78	2.00	1.21	1.65
6	n-1	9	7692	1.17	2	5994	0.33	4.50	1.28	3.55
7	n-1	45	36195	1.24	12	26980	0.44	3.75	1.34	2.82
8	n-1	289	164192	1.76	74	116562	0.63	3.91	1.41	2.79
9	n-1	1741	734092	2.37	390	493594	0.79	4.46	1.49	3.00
10	n-1	11920	3284381	3.62	1984	2071035	0.95	6.01	1.59	3.81
5	n	4	3060	1.30	1	2051	0.48	4.00	1.49	2.71
6	n	21	14733	1.42	4	9213	0.43	5.25	1.60	3.30
7	n	98	64686	1.51	20	39734	0.50	4.90	1.63	3.02
8	n	583	272101	2.14	95	149451	0.63	6.14	1.82	3.40
9	n	4132	1204958	3.42	504	666087	0.75	8.20	1.81	4.56
10	n	26764	4928984	5.42	2024	2465646	0.82	13.22	2.00	6.61
5	n+1	12	5974	2.00	1	3016	0.33	12.00	1.98	6.06
6	n+1	69	26819	2.57	6	12759	0.47	11.50	2.10	5.47
7	n+1	370	119358	3.09	28	54121	0.51	13.21	2.21	6.06
8	n+1	2613	509895	5.12	145	208351	0.69	18.02	2.45	7.42
9	n+1	27765	2097842	13.23	631	827910	0.76	44.00	2.53	17.41
10	n+1	202832	7779662	26.07	3021	3400945	0.88	67.14	2.29	29.62

Figure 23.14: Plan generation for different join graphs, Simmen’s algorithm (left) vs. our algorithm (middle)

5. Attributes should be ordered according to the functional dependencies, i.e. if $a \rightarrow b$, a should come before b . Note that $a = b$ suggests no ordering between a and b .
6. The remaining unordered attributes can be ordered in an arbitrary way.

The rules must check if they create contradictions. If this happens, the contradicting ordering must be omitted, resulting in potentially superfluous sort operators. Note that in some cases these sort operators are simply unavoidable: If for the example query one index on R exists with the ordering $abcd$ and one index on S with the ordering $dcba$, the heuristical rules detect a contradiction and choose one of the orderings. This results in a sort operator before the (sort-merge) join, but this sort could not have been avoided anyway.

23.5 Experimental Results

The framework described in this chapter solves two problems: First, it provides an efficient representation for reasoning about orderings and second, it allows keeping track of orderings and groupings at the same time. Since these topics are treated separately in the related work, the experimental results are split in two sections: In Section 23.6 the framework is compared to another published framework while only considering orderings, and in Section 23.7 the influence of groupings is evaluated.

23.6 Total Impact

We now consider how order processing influences the time needed for plan generation. Therefore, we implemented both our algorithm and the algorithm proposed by Simmen et al. [818, 819] and integrated them into a bottom-up plan generator based on [567].

To get a fair comparison, we tuned Simmen’s algorithm as much as possible. The most important measure was to cache results in order to eliminate repeated

calls to the very expensive *reduce* operation. Second, since Simmen’s algorithm requires dynamic memory, we implemented a specially tailored memory management. This alone gave us a speed up by a factor of three. We further tuned the algorithm by thoroughly profiling it until no more improvements were possible. For each order optimization framework the plan generator was recompiled to allow for as many compiler optimizations as possible. We also carefully observed that in all cases both order optimization algorithms produced the same optimal plan.

We first measured the plan generation times and memory usage for TPC-R Query 8. A detailed discussion of this query follows in Section 23.7, here we ignored the grouping properties to compare it with Simmen’s algorithm. The result of this experiment is summarized in the following table. Since order optimization is tightly integrated with plan generation, it is impossible to exactly measure the time spent just for order optimization during plan generation. Hence, we decided to measure the impact of order optimization on the total plan generation time. This has the advantage that we can also (for the first time) measure the impact order optimization has on plan generation time. This is important since one could argue that we are optimizing a problem with no significant impact on plan generation time, hence solving a non-problem. As we will see, this is definitely not the case.

In subsequent tables, we denote by $t(ms)$ the total execution time for plan generation measured in milliseconds, by $\#Plans$ the total number of subplans generated, by $t/plan$ the average time (in microseconds) needed to introduce one plan operator, i.e. the time to produce a single subplan, and by *Memory* the total memory (in KB) consumed by the order optimization algorithms.

	Simmen	Our algorithm
t (ms)	262	52
$\#Plans$	200536	123954
$t/plan$ (μs)	1.31	0.42
Memory (KB)	329	136

From these numbers, it becomes obvious that order optimization has a significant influence on total plan generation time. It may come as a surprise that fewer plans need to be generated by our approach. This is due to the fact that the (reduced) FSM only contains the information relevant to the query, resulting in fewer states. With Simmen’s approach, the plan generator can only discard plans if the ordering is the same and the set of functional dependencies is equal (respectively a subset). It does not recognize that the additional information is not relevant for the query.

In order to show the influence of the query on the possible gains of our algorithm, we generated queries with 5-10 relations and a varying number of join predicates —that is, edges in the join graph. We always started from a chain query and then randomly added some edges. For small queries we averaged the results of 100 queries and averaged 10 queries for large queries. The results of the experiment can be found in Fig. 23.14. In the second column, we denote the number of edges in terms of the number of relations (n) given in the first column. The next six columns contain (1) the total time needed for

n	#Edges	Simmen	Our Algorithm	DFSM
5	n-1	14	10	2
6	n-1	44	28	2
7	n-1	123	77	2
8	n-1	383	241	3
9	n-1	1092	668	3
10	n-1	3307	1972	4
5	n	27	12	2
6	n	68	36	2
7	n	238	98	3
8	n	688	317	3
9	n	1854	855	4
10	n	5294	2266	4
5	n+1	53	15	2
6	n+1	146	49	3
7	n+1	404	118	3
8	n+1	1247	346	4
9	n+1	2641	1051	4
10	n+1	8736	3003	5

Figure 23.15: Memory consumption in KB for Figure 23.14

plan generation (in ms), (2) the number of (sub-) plans generated, and (3) the time needed to generate a subplan (in μ s), i.e. to add a single plan operator, for (a) Simmen's algorithm (columns 3-5) and our algorithm (columns 6-8). The total plan generation time includes building the DFSM when our algorithm is used. The last three columns contain the improvement factors for these three measures achieved by our algorithm. More specifically, column $\% x$ contains the result of dividing the x column of Simmen's algorithm by the corresponding x column entry of our algorithm.

Note that we are able to keep the plan generation time below one second in most cases and three seconds in the worst case, whereas when Simmen's algorithm is applied, plan generation time can be as high as 200 seconds. This observation leads to two important conclusions:

1. Order optimization has a significant impact on total plan generation time.
2. By using our algorithm, significant performance gains are possible.

For completeness, we also give the memory consumption during plan generation for the two order optimization algorithms (see Fig. 23.15). For our approach, we also give the sizes of the DFSM which are included in the total memory consumption. All memory sizes are in KB. As one can see, our approach consumes about half as much memory as Simmen's algorithm.

23.7 Influence of Groupings

Integrating groupings in the order optimization framework allows the plan generator to easily exploit groupings and, thus, produce better plans. However, order optimization itself might become prohibitively expensive by considering

groupings. Therefore, we evaluated the costs of including groupings for different queries.

Since adding support for groupings has no effect on the runtime behavior of the plan generator (all operations are still one table lookup), we measured the runtime and the memory consumption of the preparation step both with and without considering groupings. When considering groupings, we treated each interesting ordering also as an interesting grouping, i.e. we assumed that a grouping-based (e.g. hash-based) operator was always available as an alternative. Since this is the worst-case scenario, it should give an upper bound for the additional costs. All experiments were performed on a 2.4 GHz Pentium IV, using the gcc 3.3.1.

To examine the impact for real queries, we choose a more complex query from the well-known TPC-R benchmark ([879], Query 8):

```
select
  o_year,
  sum(case when nation = '[NATION]'
    then volume
    else 0
  end) / sum(volume) as mkt_share
from
  (select
    extract(year from o_orderdate) as o_year,
    l_extendedprice * (1-l_discount) as volume,
    n2.n_name as nation
  from part,supplier,lineitem,orders,customer,
    nation n1,nation n2,region
  where
    p_partkey = l_partkey and
    s_suppkey = l_suppkey and
    l_orderkey = o_orderkey and
    o_custkey = c_custkey and
    c_nationkey = n1.n_nationkey and
    n1.n_regionkey = r_regionkey and
    r_name = '[REGION]' and
    s_nationkey = n2.n_nationkey and
    o_orderdate between date '1995-01-01' and
      date '1996-12-31' and
    p_type = '[TYPE]'
  ) as all_nations
group by o_year
order by o_year;
```

When considering this query, all attributes used in joins, group-by and order-by clauses are added to the set of interesting orders. Since hash-based solutions are possible, they are also added to the set of interesting groupings.

This results in the sets

$$\begin{aligned}
O_I^P &= \{(o_year), (o_partkey), (p_partkey), \\
&\quad (l_partkey), (l_suppkey), (l_orderkey), \\
&\quad (o_orderkey), (o_custkey), (c_custkey), \\
&\quad (c_nationkey), (n1.n_nationkey), \\
&\quad (n2.n_nationkey), (n_regionkey), \\
&\quad (r_regionkey), (s_suppkey), (s_nationkey)\} \\
O_I^T &= \emptyset \\
G_I^P &= \{\{o_year\}, \{o_partkey\}, \{p_partkey\}, \\
&\quad \{l_partkey\}, \{l_suppkey\}, \{l_orderkey\}, \\
&\quad \{o_orderkey\}, \{o_custkey\}, \{c_custkey\}, \\
&\quad \{c_nationkey\}, \{n1.n_nationkey\}, \\
&\quad \{n2.n_nationkey\}, \{n_regionkey\}, \\
&\quad \{r_regionkey\}, \{s_suppkey\}, \{s_nationkey\}\} \\
G_I^T &= \emptyset
\end{aligned}$$

Note that here O_I^T and G_I^T are empty, as we assumed that each ordering and grouping would be produced if beneficial. For example, we might assume that it makes no sense to intentionally group by o_year : If a tuple stream is already grouped by o_year it makes sense to exploit this, however, instead of just grouping by o_year it could make sense to sort by o_year , as this is required anyway (although here it only makes sense if the sort operator performs early aggregation). In this case, $\{o_year\}$ would move from G_I^P to G_I^T , as it would be only tested for, but not produced.

The set of functional dependencies (and equations) contains all join conditions and constant conditions:

$$\begin{aligned}
\mathcal{F} &= \{\{p_partkey = l_partkey\}, \{\emptyset \rightarrow p_type\}, \\
&\quad \{o_custkey = c_custkey\}, \{\emptyset \rightarrow r_name\}, \\
&\quad \{c_nationkey = n1.n_nationkey\}, \\
&\quad \{s_nationkey = n2.n_nationkey\}, \\
&\quad \{l_orderkey = o_orderkey\}, \\
&\quad \{s_suppkey = l_suppkey\}, \\
&\quad \{n1.n_regionkey = r_regionkey\}\}
\end{aligned}$$

To measure the influence of groupings, the preparation step was executed twice: Once with the data as given above and once with $G_I^P = \emptyset$ (i.e. groupings were ignored). The space and time requirements are shown below:

	With Groups	Without Groups
Duration [ms]	0.6ms	0.3ms
DFSM [nodes]	63	32
Memory [KB]	5	2

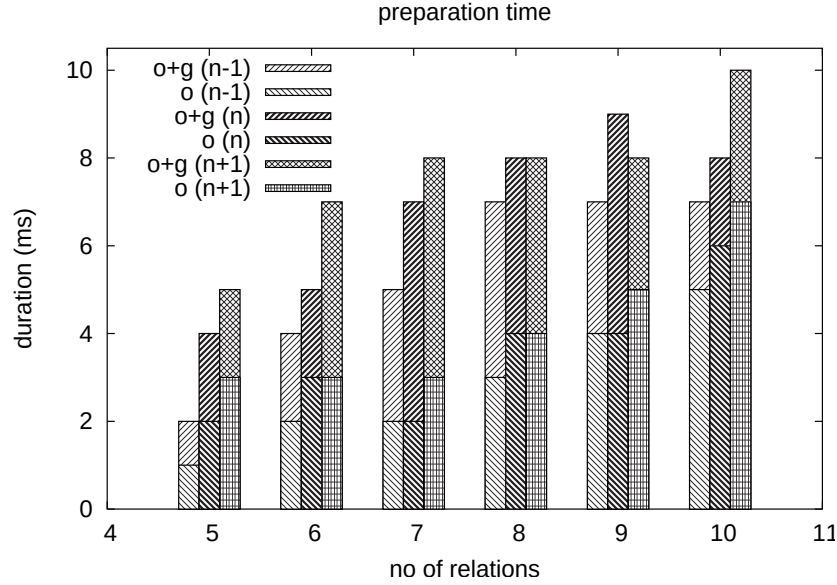


Figure 23.16: Time requirements for the preparation step

Here time and space requirements both increase by a factor of two. Since all interesting orderings are also treated as interesting groupings, a factor of about two was expected.

While Query 8 is one of the more complex TPC-R queries, it is not overly complex when looking at order optimization. It contains 16 interesting orderings/groupings and 8 functional dependencies, but they cannot be combined in many reasonable ways, resulting in a comparatively small DFSM. In order to get more complex examples, we produced randomized queries with 5 – 10 relations and a varying number of join predicates. We always started from a chain query and then randomly added additional edges to the join graph. The results are shown for $n - 1$, n and $n + 1$ additional edges. In the case of 10 relations, this means that the join graph consisted of 18, 19 and 20 edges, respectively.

The time and space requirements for the preparation step are shown in Figure 23.16 and Figure 23.17, respectively. For each number of relations, the requirements for the combined framework ($o+g$) and the framework ignoring groupings (o) are shown. The numbers in parentheses ($n - 1$, n and $n + 1$) are the number of additional edges in the join graph.

As with Query 8, the time and space requirements roughly increase by a factor of two when adding groupings. This is a very positive result, given that a factor of two can be estimated as a lower bound (since every interesting ordering is also an interesting grouping here). Furthermore, the absolute time and space requirements are very low (a few ms and a few KB), encouraging the inclusion of groupings in the order optimization framework.

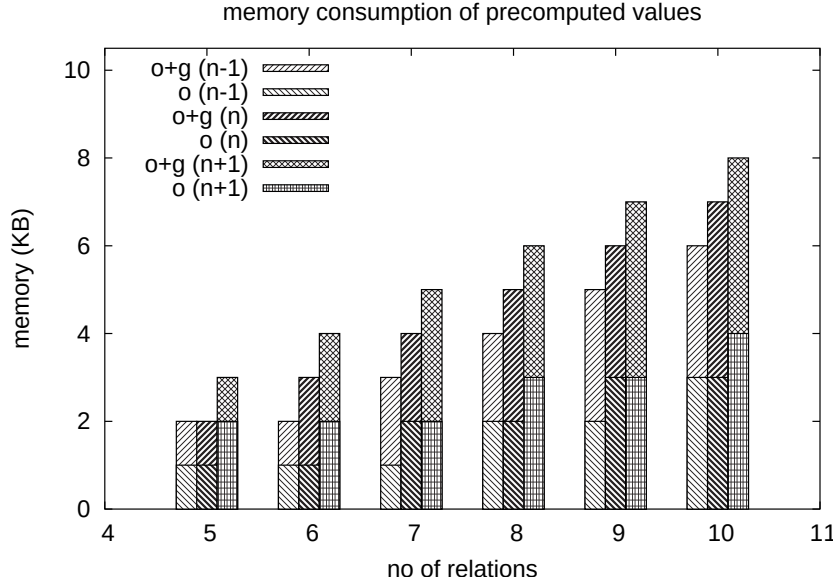


Figure 23.17: Space requirements for the preparation step

23.8 Annotated Bibliography

Very few papers exist on order optimization. While the problem of optimizing interesting orders was already introduced by Selinger et al. [784], later papers usually concentrated on exploiting, pushing down or combining orders, not on the abstract handling of orders during query optimization.

Papers by Simmen, Shekita, and Malkemus [818, 819] introduced a framework based on functional dependencies for reasoning about orderings. Since this is the only paper which really concentrates on the abstract handling orders and our approach is similar in the usage of functional dependencies, we will describe their approach in some more detail.

For a plan node they keep just a single (physical) ordering. Additionally, they associate all the applicable functional dependencies with a plan node. Hence, the lower-bound space requirement for this representation is essentially $\Omega(n)$, where n is the number of functional dependencies derived from the query. Note that the set of functional dependencies is still (typically) much smaller than the set of all logical orderings. In order to compute the function `containsOrdering`, Simmen et al. apply a *reduction algorithm* on both the ordering associated with a plan node and the ordering given as an argument to `containsOrdering`. Their reduction roughly does the opposite of deducing more orderings using functional dependencies. Let us briefly illustrate the reduction by an example. Assume the physical ordering a tuple stream satisfies is (a) , and the required ordering is (a, b, c) . Further assume that there are two functional dependencies available: $a \rightarrow b$ and $a, b \rightarrow c$. The reduction algorithm is performed on both orderings. Since (a) is already minimal, nothing changes. Let us now reduce (a, b, c) . We apply the second functional dependency first. Using $a, b \rightarrow c$, the reduction algorithm yields (a, b) because c appears in (a, b, c)

after a and b . Hence, c is removed. In general, every occurrence of an attribute on the right-hand side of a functional dependency is removed if all attributes of the left-hand side of the functional dependency precede the occurrence. Reduction of (a, b) by $a \rightarrow b$ yields (a) . After both orderings are reduced, the algorithm tests whether the reduced required ordering is a prefix of the reduced physical ordering. Note that if we applied $a \rightarrow b$ first, then (a, b, c) would reduce to (a, c) and no further reduction would be possible. Hence, the rewrite system induced by their reduction process is not confluent. This problem is not mentioned by Simmen et al., but can have the effect that `containsOrdering` returns *false* whereas it should return *true*. The result is that some orderings remain unexploited; this could be avoided by maintaining a minimal set of functional dependencies, but the computation costs would probably be prohibitive. This problem does not occur with our approach. On the complexity side, every functional dependency has to be considered by the reduction algorithm at least once. Hence, the lower time bound is $\Omega(n)$.

In case all functional dependencies are introduced by a single plan node and all of them have to be inserted into the set of functional dependencies associated with that plan node, the lower bound for `inferNewLogicalOrderings` is also $\Omega(n)$.

Overall, Simmen et al. proposed the important framework for order optimization utilizing functional dependencies and nice algorithms to handle orderings during plan generation, but the space and time requirements are unfortunate since plan generation might generate millions of subplans. Also note that the reduction algorithm is not applicable for groupings (which, of course, was never intended by Simmen): Given the grouping $\{a, b, c\}$ and the functional dependencies $a \rightarrow b$ and $b \rightarrow c$, the grouping would be reduced to $\{a, c\}$ or to $\{a\}$, depending on the order in which the reductions are performed. This problem does not occur with orderings, as the attributes are sorted and can be reduced back to front.

A recent paper by Wang and Cherniack [911] presented the idea of combining order optimization with the optimization of groupings. Based upon Simmen's framework, they annotated each attribute in an ordering with the information whether it is actually ordered by or grouped by. For a single attribute a , they write $O_{aO}(R)$ to denote that R is ordered by a , $O_{aG}(R)$ to denote that R is grouped by a and $O_{aO \rightarrow bG}$ to denote that R is first ordered by a and then grouped by b (within blocks of the same a value). Before checking if a required ordering or grouping is satisfied by a given plan, they use some inference rules to get all orderings and groupings satisfied by the plan. Basically, this is Simmen's reduction algorithm with two extra transformations for groupings. In their paper the check itself is just written as $\in$, however, at least one reduction on the required ordering would be needed for this to work (and even that would not be trivial, as the stated transformations on groupings are ambiguous). The promised details in the cited technical report are currently not available, as the report has not appeared yet. Also note that, as explained above, the reduction approach is fundamentally not suited for groupings. In Wang's and Cherniack's paper, this problem does not occur, as they only look at a very specialized kind of grouping: As stated in their Axiom 3.6, they assume that a grouping

$O_{aG \rightarrow bG}$ is first grouped by a and then (within the block of tuples with the same a value) grouped by b . However, this is a very strong condition that is usually not satisfied by a hash-based grouping operator. Therefore, their work is not general enough to capture the full functionality offered by a state-of-the-art query execution engine.

In this chapter, we followed [644, 643].

Chapter 24

Cardinality and Cost Estimation

24.1 Introduction

The plan generator relies on a cost function to evaluate the different plans and to determine the cheapest one. This chapter is concerned with the development of cost functions. The main input to cost functions are cardinalities. For example, assume a scan of a relation, which also applies a selection predicate. Clearly, the cost of scanning the relation depends on the physical layout of the relation on disk. Further, the CPU cost for evaluating the predicate depends on the number of tuples in the relation. Note that the cardinality of a relation is independent of its physical layout.

In general, the cost of an algebraic operator is estimated by using a *profile* of the database. The profile must be small, e.g., a couple of kilobytes per relation¹. We distinguish between the *logical* and the *physical* profile. For each database item and its constituents, there exist specialized logical and physical profiles. They exist for relations, indices, attributes, and sets of attributes. Consider a relation R . Its cardinality $|R|$ belongs to its logical profile, whereas the number of pages $||R||$ it occupies belongs to its physical profile. In Chapter 4, we saw more advanced physical profiles.

The DBMS must be capable to perform several operations to derive profiles and to deal with them. Fig. 24.1 gives an overview. This figure roughly follows the approach of Mannino et al. [583, 582]. The first operation is the *build* operation, that takes as input a specification of the profiles to be build (because there are many different alternatives, as we will see) and the database. From that, it builds the according profiles for all database items of all the different granularities. When updates arrive, the profiles must be updated. This can either be done by a complete recalculation or by an incremental update operation on the profiles themselves. The latter is reflected in the operation *update*. Unfortunately, not all profiles can have an update operation. Within this book, we will not be too concerned with building and updating profiles. At the end

¹Given today's cost for main memory, it may also be reasonable to use a couple of megabytes.

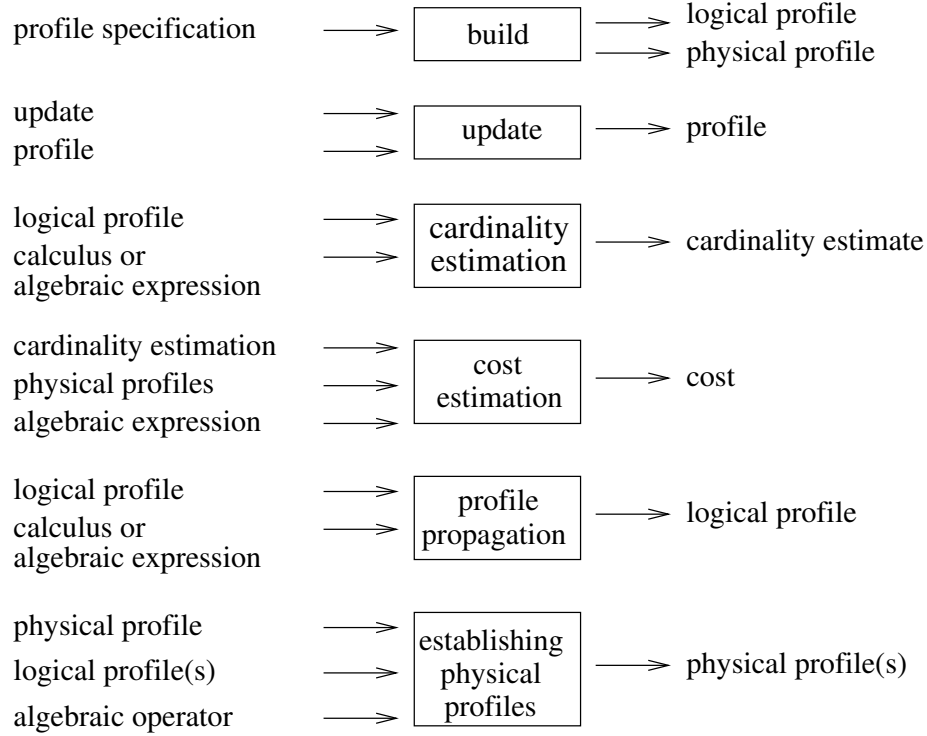


Figure 24.1: Overview of operations for cardinality and cost estimations

of this chapter, we will provide some references (See [210] for an overview).

The main operations this chapter deals with are among the remaining ones. The first of them is *cardinality estimation*. Given an algebraic expression or a calculus expression together with a logical profile of the database, we estimate the output/result cardinality of the expression. Why do we say algebraic or calculus expression? Remember that plan generators generate plans for plan classes. Each plan class corresponds to a set of equivalent plans. They all produce the same result and, hence, the same number of output tuples. Thus, in theory, one arbitrary representative of the class of equivalent algebraic expressions should suffice to calculate the logical profile, as a logical profile depends only on the outcome. On the other hand, the plan class more directly corresponds to a calculus expression. Hence, estimating the result cardinality of a calculus expression is a viable alternative. In the literature, most papers deal with the first approach while only a few deal with the latter (e.g., [227]).

The second operation we will be concerned with is *cost estimation*. Given logical and physical profiles for all inputs and an algebraic operator (tree), this operation calculates the actual costs. Chapter 4 contains a detailed discussion about disk access cost calculation. Hence, this part is considered done for building blocks and access paths.

The third major task is profile propagation. Given a logical or physical profile and an expression, we must be able to calculate the profile of the result, since this may be the input to other expressions and thus be needed for further cardinality estimates. The estimation of a physical profile occurs mostly in

$ \chi_{a:e_2}(e_1) $	$=$	$ e_1 $
$ \Gamma_{g,f}(e) $	$=$	$ \Pi_g^D(e) $
$ e_1 \bowtie_{g,f} e_2 $	$=$	$ e_1 $
$ e_1 \triangleright e_2 $	$=$	$ e_1 - e_1 \bowtie e_2 $
$ e_1 \bowtie e_2 $	$=$	$ e_1 \bowtie e_2 + e_1 \triangleright e_2 $
$ e_1 \bowtie e_2 $	$=$	$ e_1 \bowtie e_2 + e_1 \triangleright e_2 + e_2 \triangleright e_1 $
$ e_1 \bowtie e_2 $	$=$	$ \Pi_{\mathcal{A}(e_1) \cup \mathcal{A}(e_2)}^D(e_1 \bowtie e_2) $
$ \text{Sort}(e) $	$=$	$ e $
$ \text{Tmp}(e) $	$=$	$ e $
$ e_1 \times e_2 $	$=$	$ e_1 * e_2 $
$ \Pi_A(e) $	$=$	$ e $ (bag semantics)
$ e_1 \cup_s e_2 $	$=$	$ \Pi_{\mathcal{A}(e_1)}^D(e_1 \cup_b e_2) $ (bag vs. set semantics)
$ e_1 \cap_s e_2 $	$=$	$ e_1 \bowtie e_2 $ equijoin over all attributes
$ e_1 \setminus_s e_2 $	$=$	$ e_1 - e_1 \cap_s e_2 $
$ e_1 \cup_b e_2 $	$=$	$ e_1 + e_2 $ bag semantics
$ \Pi_{\alpha \cup \beta}^D(R) $	$=$	$ \Pi_\alpha^D(R) $ if there is an FD $\alpha \rightarrow \beta$

Table 24.1: Observations on cardinalities of different algebraic operators

cases where operators write to disk. Given Chapter 4, this task is easy enough to be left to the reader.

Since we follow the algebraic approach, we must be able to calculate the output cardinality of every operator occurring in the algebra. This task is vastly simplified by the observations contained in Table 24.1.

This shows that we can go a far way if we are able to estimate the output cardinality for duplicate eliminating projections, selections, (bag) joins, and semijoins. For a certain class of profiles, Richard shows that a profile consisting ‘only’ of the sizes of all duplicate eliminating projections on all subsets of attributes of all relations is a complete profile under certain assumptions [730]. Since the set of subsets of a set of attributes can be quite large, Richard exploits functional dependencies to reduce this set by exploiting the fact that $|\Pi_{\alpha \cup \beta}^D(R)| = |\Pi_\alpha^D(R)|$ if there exists a functional dependency $\alpha \rightarrow \beta$.

A major differentiator for logical attribute profiles is the kind of the domain of the attribute. We distinguish between categorical attributes (e.g., color), discrete ordered domains (e.g., integer attributes, decimals, strings), and continuous ordered domains (e.g., float). Categorical domains may be ordered or unordered. In the first case they are called *ordinal*, in the latter *nominal*. We will be mainly concerned with integer attributes. Strings are special, and we discuss some approaches in Sec. 24.13.6. Continuous domains are also special. The probability of occurrence of any value in a continuous domain in a finite set is zero. The techniques developed in this section can often easily be adopted to continuous

domains, even if we do not mention this explicitly.

24.2 A First Approach

The first approach to cost and cardinality estimation integrated into a dynamic programming-based plan generator was presented by Selinger et al. [784]. We will use it as the basis for this section.

24.2.1 Top-Most Cost Formula (Overall Costs)

Their top-most cost formula states that the total cost of a query evaluation plan equals the weighted sum of the I/O and CPU costs:

$$\mathcal{C} = \mathcal{C}_{I/O} + w\mathcal{C}_{cpu} \quad (24.1)$$

where w is the weight which can be adapted to different situations. If, for example, the system is CPU bound, we should increase w and if it is I/O bound, we decrease w .

However, it is not totally clear what we are going to optimize under this cost formula. One interpretation could be the following. Assume $w = 0.5$. Then, we could interpret the total costs as response time under the assumption that fifty percent of the CPU time can be executed in parallel with I/O. Accordingly, we find other top-most cost formulas. For example, the weight is sometimes dropped [407]:

$$\mathcal{C} = \mathcal{C}_{I/O} + \mathcal{C}_{cpu} \quad (24.2)$$

Under the above interpretation, this would mean that concurrency is totally absent. The opposite, total concurrency between I/O and CPU, can also be found [170]:

$$\mathcal{C} = \max(\mathcal{C}_{I/O}, \mathcal{C}_{cpu}) \quad (24.3)$$

In these green days, an alternative is to calculate the power consumption during query execution. Therefore, we convert CPU time to Watts consumed by the CPU and disk time to Watts consumed by the disks and simply add up these number to get an estimate of the power consumption of the plan.

24.2.2 Summation of Operator Costs

Given a query evaluation plan, the task is to calculate its I/O and CPU costs. This can be done by calculating the costs for each operator (op) occurring in the query evaluation plan (QEP) and adding up the according costs:

$$\begin{aligned} \mathcal{C}_{I/O} &= \sum_{op \in QEP} \mathcal{C}_{I/O}(op) \\ \mathcal{C}_{cpu} &= \sum_{op \in QEP} \mathcal{C}_{cpu}(op) \end{aligned}$$

However, these formulae sometimes raise a problem. For example, the nested loop join method requires multiple evaluations of its inner part.

Further, in order to count the I/O cost correctly, it is necessary to make some assumptions when intermediate results are written to and read from disk. We will use the following assumption: Every operator is responsible for passing its result to the next operator via main memory. For example, a sort merge join may require sorting its inputs. The sort operators are then responsible for handing over their result to the merge join via main memory. This means that the merge join may not require any I/O if the merge can be done purely in main memory.

24.2.3 CPU Cost

To get an estimate of the CPU costs, Selinger et al. simply count the number of calls to the tuple oriented interface (called RSI). This roughly corresponds to the number of `next` calls in an iterator-based implementation of algebraic operators (for details on the Research Storage Engine (RSS) and its interface RSI see [42]). Hence, what needs to be known is the output cardinality of each operator in the query plan.

Given the input and output cardinalities, it is often quite straightforward to calculate the CPU costs. Take, for example, a selection operator. Clearly, the selection predicate is called n times if n is the input cardinality. The selection predicate itself consists of several calls to comparison functions, Boolean operators, arithmetic operators and the like. The CPU cost of each of these operators can easily be determined (by counting CPU cycles or measurements), and thus the total CPU cost of a selection operator can be determined easily. Other operators are also straightforward. A problem only arises if functions are called whose CPU costs can not easily be determined since they depend on their parameters. A typical example are string comparisons, where the CPU costs depend on the length of the string. Another example are user-defined functions. The framework presented in [410] can be used for all more complex functions. Another possibility is to use simplifying assumptions. The functions we are talking about are executed on a per tuple basis. As there are typically many tuples, using the average execution time for cost calculations is not a bad idea.

24.2.4 Abbreviations

We need some abbreviations to state our cost formulas. A first bunch of them is summarized in Table 24.2. There, we assume that an index is always a B^+ tree.

24.2.5 I/O Costs

Selinger et al. measure I/O costs in the *number of pages read*. Let us first discuss the different possible access paths to a single relation. Clearly, for a simple scan of a relation R , $\|R\|$ pages have to be read. The next access path is composed of an access to a non-clustered index I to retrieve the tuple identifiers of those tuples that satisfy a predicate p followed by an access to the base relation R . Let $F(p)$ be the fraction of tuples satisfying a certain predicate p . $F(p)$ is called

R, S, T	relations
I	index
A, B, C	attributes or sets of attributes
D_A	$\Pi_A^D(R)$
d_A	$ D_A $
$\min_A$	$\min \Pi_A^D(R)$ for an attribute A of R
$\max_A$	$\max \Pi_A^D(R)$ for an attribute A of R
$ R $	number of tuples of R
$ R $	number of pages on which R is stored
$ A _B$	average length of a value of attribute A of R (in bytes)
$ \mathcal{A}(R) _B$	average length of a tuple in bytes
$ I $	number of leaf pages of an index
$H(I)$	depth of the index I minus 1

Table 24.2: Notational conventions

the *selectivity* of p . It is the main focus of the next subsection. Selinger et al. distinguish two cases. In the first case, all pages containing qualifying tuples fit into main memory. For this case, they estimate the number of pages accessed by

$$H(T) + F(p) * (||I|| + ||R||).$$

EXC

Note that with the help of Chapter 4, we can already do better. In the second case, where the pages containing qualifying tuples do not fit into main memory, they give the estimate

$$H(T) + F(p) * (||I|| + |R|)$$

for the number of pages read.

In case of a clustered index, they estimate the number of pages read by

$$H(T) + F(p) * (||I|| + ||R||).$$

Next, we have to discuss the costs of different join methods. Selinger et al. propose cost formulas for the simple nested loop join ($\bowtie^{nl}$) and the sort merge join ($\bowtie^{sm}$). Since summing up the costs of all operators in a tree results in some problems for nested loop joins, they adhere to a recursive computation of total costs. Let e_1 and e_2 be two algebraic expressions. Then they estimate the cost of the simple nested loop join as follows:

$$C_{I/O}(e_1 \bowtie^{nl} e_2) = C_{I/O}(e_1) + |e_1| * C_{I/O}(e_2)$$

where $|e_1|$ denotes the number of tuples produced by the expression e_1 .

As the cost calculation for the sort merge join is not convincing, we follow our own very simple approach here (see also Sec. 24.14). We split the costs into the sort costs and the merge costs. Given today's memory sizes, it is not unlikely that we need a single merge phase. Hence, the I/O cost for sorting

consists of writing and reading the result of e_i if it needs to be sorted. This can be estimated as

$$C_{I/O}(\text{sort}(e_i)) = C_{I/O}(e_i) + 2 * \lceil 1.2 ||\mathcal{A}(e_i)||_B * |e_i| / \text{pagesize} \rceil$$

where **pagesize** is the page size in bytes. The factor 1.2 is called the *universal fudge factor*. In the above case, it takes care of the storage overhead incurred by using slotted pages. If we assume that the merge phase of the sort merge join can be performed in main memory, no additional I/O costs occur and we are done.

Clearly, in the light of Chapter 4, counting the numbers of pages read is not sufficient as the discrepancy between random and sequential I/O is tremendous. Thus, better cost functions should use a more elaborate I/O cost model along the lines of Chapter 4. In any case, note that the calculation of the I/O or CPU costs of any operator highly depends on its input and output cardinalities.

24.2.6 Cardinality Estimates

Given a predicate p , we want to estimate its selectivity, which is defined as the fraction of qualifying tuples. If p is a selection predicate applied to a relation R , the selectivity of p is defined as

$$s(p) = \frac{|\sigma_p(R)|}{|R|}.$$

If we know the selectivity of p , we can then easily calculate the result size of a selection:

$$|\sigma_p(R)| = s(p) |R|$$

Similarly for joins. Given a join predicate p and two relations R and S , we define the selectivity of p as

$$s(p) = \frac{|R \bowtie_p S|}{|R \times S|} = \frac{|R \bowtie_p S|}{|R| * |S|}$$

and can calculate the result size of a join by

$$|R \bowtie_p S| = s(p) |R| |S|.$$

The idea of the approach of Selinger et al. is to calculate the result cardinality for a plan class by the following procedure. First, the sizes of all relations represented by the plan class are multiplied. This is the result of their cross product. In a second step, they take a look at the predicate p applied to the relations in the plan class. For p they calculate a selectivity estimate $s(p)$ and multiply it with the result of the first step. This then gives the result. Hence, if a plan class represents the algebraic expression

$$\sigma_p(\times_{i=1}^n R_i),$$

the cardinality estimate is

$$s(p) \prod_{i=1}^n |R_i|.$$

predicate	$s(p)$	comment
$\text{not}(p_1)$	$1 - s(p_1)$	
$p_1 \wedge p_2$	$s(p_1) * s(p_2)$	independence
$p_1 \vee p_2$	$s(p_1) + s(p_2) - s(p_1)s(p_2)$	
$A = c$	$1/d_A$	if d_A is known, uniformity
	$1/10$	else
$A = B$	$1/\max(d_A, d_B)$	if d_A and d_B are known, uniformity
	$1/d_X$	if only d_X , $X \in \{A, B\}$ is known
	$1/10$	else
$A > c$	$\frac{\max_A - c}{\max_A - \min_A}$	if min and max are known, uniformity
	$1/3$	else
$c_1 \leq A \leq c_2$	$\frac{c_2 - c_1}{\max_A - \min_A}$	if min and max are known, uniformity
	$1/4$	else
$A \text{ IN } L$	$\min(1/2, s(A = c) L)$	
$A \text{ IN } Q$	$ Q / X $	X is cross product of all relations in Q 's from clause

Table 24.3: Selectivity estimation as proposed by Selinger et al.[784]

Since p can be a complex predicate involving boolean operators, they have to deal with them. Table 24.3 summarizes the proposed selectivity estimation. A and B denote attributes, c , c_1 , c_2 denote constants, L denotes a list of values, Q denotes a subquery. In System R, the number of distinct values for an attribute (d_A, d_B) is only known if there exists an according index on the attribute. Let us give some rational for the selectivity estimation of $A \text{ IN } Q$ for an attribute A and a subquery Q . Assume that A is an attribute of relation R and the subquery Q is of the form **select B from S . . .**. Further assume that $\Pi_A(R) \subseteq \Pi_B(S)$, i.e., referential integrity holds. Clearly, if all tuples of S are in the result of Q , the selectivity is equal to 1. If the output cardinality of Q is restricted by a factor $s' = |Q|/|S|$, then we may assume that the number of distinct values in Q 's result is restricted by the same factor. Hence, the selectivity factor of the total predicate is also s' . Selinger et al. now continue as follows: “With a little optimism, we can extend this reasoning to include subqueries which are joins and subqueries in which column $[B]$ is replaced by an arithmetic expression involving column names. This leads to the formula given above.”

Discussion Taking a broad view at the above model, we see that

- the estimates for CPU and I/O times are quite rough,
- the approach is not complete, especially projection and semijoin are not treated,
- profile propagation is not discussed.

Further, the uniformity and independence assumptions are applied. This has been shown to be quite inaccurate in many cases. More specifically, apply-

ing these and other assumptions often leads to an overestimate of real result cardinalities [177, 179].

How bad is it in terms of plan generation if we under- or overestimate the cardinalities of intermediate results? As Ioannidis and Christodoulakis pointed out, errors propagate multiplicatively through joins [451]. Assume we want to join eight relations $R_1, \dots, R_8$ and that the cardinality estimates of R_i are each a factor of 5 off. Then the cardinality estimation of $R_1 \bowtie R_2 \bowtie R_3$ will be a factor of 125 off. Clearly, this can affect the subsequent join ordering. If we were only a factor of 2 off, the cardinality estimation of $R_1 \bowtie R_2 \bowtie R_3$ could be only a factor of eight off. This shows that minimizing the multiplicative error is a serious intention.

The effect of misestimating cardinalities on plan quality has not been thoroughly investigated. There exists a study by Kumar and Stonebraker, which concludes that it does not matter [521]. However, we do not trust this conclusion. Swami and Schiefer give a query and its profiles for which bad cardinality estimates lead to a very bad plan [864]. A very impressive example query is presented in [884]. The plan produced for the query under cardinality estimation errors runs 40 minutes while the plan produced with better cardinality estimates takes less than 2 seconds. Later, we will give two further examples showing that good cardinality estimation is vital for generation of good plans. Hence, we are very sure that accurate estimation is vital for plan generation. We suggest to the reader to find examples, using the simple C_{out} cost function, where wrong cardinality estimates lead to bad plans.

EXC

24.3 The Simple Profile: A First Logical Profile and its Propagation

We call a logical profile *complete* if it allows us to perform cardinality estimation and logical profile propagation for all algebraic operators. In this section, we present an almost complete logical profile and describe the procedure of profile propagation. The main components are the cumulated frequency, i.e., the number of tuples, and the number of distinct values for each attribute in a relation. It is easy to see that we cannot do without either of them. Further, an upper and lower bound for values of an attribute is needed. Again, we will see that we cannot do without them. Hence, the following profile is minimal.

24.3.1 The Logical Profile

For every attribute A of a relation, we define its *logical profile* as a four tuple

$$b_A = [l_A, u_A, f_A, d_A]$$

where l_A is a lower and u_A is an upper bound for the values of A . Further, f_A is the *cumulated frequency*, i.e., the number of tuples with an A value within the bounds, and d_A is the *number of distinct values* occurring as A 's values within the given bounds.

For the purpose of this section, we can define

$$\begin{aligned} l_A &= \min(\Pi_A(R)) \\ u_A &= \max(\Pi_A(R)) \\ f_A &= |R| \\ d_A &= |\Pi_A^D(R)| \end{aligned}$$

If the attribute A is implicit from the context or does not matter, we may omit it.

24.3.2 Assumptions

The first two assumptions we make are:

1. All attribute values are uniformly distributed, and
2. the values of all attributes are drawn independently.

Other assumptions will follow.

Often in the formulas developed below, we talk about the *universe* ($\mathcal{U}$) or *domain* of the attributes. This is the potential set of values from which a given attribute takes its values. In case of integer attributes, it is easy to see that the domain of attribute A is $[l_A, u_A]$. The size of the domain, denoted by n_A , then is $n_A = u_A - l_A + 1$. For real values, the size of the domain is (almost) infinite. Thus, only some of the formulas given below may carry over to attributes whose type is real. Please do not confuse the universe/domain of an attribute A with the *active domain* of an attribute A , which contains the actual values $D_A = \Pi_A^D(R)$ occurring for A in relation R .

Let us take a closer look at the assumptions. The uniform distribution assumption means that every value occurs about the same number of times. However, this cannot mean that every value of the domain does so, since d_A may be much smaller than n_A . Hence, we refine the *uniform distribution assumption* (UDA) by assuming that every distinct value occurs about f_A/d_A times.

The second assumption is called *attribute value independence assumption* (AVI) or simply *independence assumption*. Assume we have two predicates p_1 and p_2 and wish to calculate the selectivity of $p_1 \wedge p_2$. Independence tells us that we can do so by multiplying the selectivities of p_1 and p_2 . Thus, under independence $\text{sel}(p_1 \wedge p_2) = \text{sel}(p_1) * \text{sel}(p_2)$.

We still need another assumption: the *equal spread assumption* (ESA), also called *uniform spread assumption* (USA) [210, 700]. It is used to answer the question where the occurring distinct values are in the domain. The equal spread assumption states that they occur at equal distance. Let us elaborate a little on this.

For integers, we know the number n_A of possible values from which A can be drawn. It is $n_A = u_A - l_A + 1$. Let us assume that we have only a few distinct values, that is $d_A \ll n_A$. This is not strictly necessary but is good for our intuition. We can now define the spread between two values occurring in A . Let $D_A = \Pi_A^D(R) = \{x_1, \dots, x_{d_A}\}$ where $x_i < x_{i+1}$ be the sorted set of values

occurring for attribute A , also known as *active domain*. Then we can define the *spread* as

$$\Delta_i = x_{i+1} - x_i$$

The *equal spread assumption* (ESA) states that $\Delta_i = \Delta_j$ for all $1 \leq i, j < d_A$. Denote this value by Δ_A .

There are three subtypes of the equal spread assumption, depending on whether we assume the lower and upper bounds l_A and u_A belong to D_A . Type I assumes $l_A, u_A \in D_A$. Then Δ_A becomes $(u_A - l_A)/(d_A - 1)$. In case of type II, where $l_A \in D_A$ and $u_A \notin D_A$ holds, we have $\Delta_A = (u_A - l_A)/d_A$. For type III, where $l_A \notin D_A$ and $u_A \notin D_A$ we get $\Delta_A = (u_A - l_A)/(d_A + 1)$. As an example, take $l_A = 1$, $u_A = 13$, and $d_A = 3$. Then for the three types we have the different values $12/2 = 6$, $12/3 = 4$, and $12/4 = 3$. It should be clear that the difference is small if d_A is sufficiently large. If d_A is small, we can store the frequency of each value explicitly. Otherwise, it is large, and it does not matter which type we use. In case of integers, the above numbers may result in non-integers. Thus, we prefer to define in this case

$$\Delta_A = \lfloor \frac{u_q - l_q + 1}{d_A} \rfloor.$$

An alternative to the uniform distribution assumption and the equal spread assumption is the *continuous-value assumption*. Here, we assume that all values in the (discrete and finite) domain occur with frequency f_A/n_A .

Different assumptions can lead to different estimates. To see this, we first fix some notation. Then, we provide estimation procedures under the continuous value assumption and under the equal spread assumption. Afterwards, we present an example. Assume we are given a relation R and one of its attributes A . The possible values for attribute A as implied by its type is called *universe* and abbreviated by $\mathcal{U}_A$. The set of possible values is the active domain D_A , which we already saw. The total number of tuples in R is typically called its cardinality and denoted by $|R|$. However, in this chapter we prefer to call this value *cumulated frequency* and denote it by f_A . Remember that we denote the minimum of D_A by l_A and the maximum by u_A .

For attribute A , we consider range queries and try to estimate the result cardinality thereof. Thus, we are interested in queries Q of the form

select count(*) from R where $l_q \leq A \leq u_q$.

We denote the result of this range query by f_q .

We describe frequency densities of some attribute A by sets of points (x_i, f_i) , where x_i is a domain value and f_i is the frequency of the domain value. Thus, the frequency density is the result of the query

select A , count(*) from R group by A .

Here is our example for a frequency density:

(1, 7), (5, 4), (7, 2), (8, 1).

Thus, the integer value 1 occurs 7 times and the value 7 occurs 2 times.

To estimate the result cardinality of a range query Q with bounds l_q and u_q under the continuous value assumption, we use the formula

$$\hat{f}_q(\text{cva}) := \frac{u_q - l_q + 1}{u_a - l_a + 1} * f_A.$$

Let us first recall the spread under the equal spread assumptions. For integer values, we defined

$$\Delta_A := \lfloor \frac{u_q - l_q + 1}{d_A} \rfloor.$$

Using this definition, we provide an estimate for $\hat{f}_q(\text{esa})$ by applying the following formula:

$$\hat{f}_q(\text{esa}) := \lfloor \frac{q_u - q_l + 1}{\Delta_A} \rfloor * \frac{f_A}{d_A}.$$

Note that if the active domain is dense, i.e., all possible values within $[l_A, u_A]$ occur in the database, then the estimation under cva and esa coincide.

EXC

Fig. 24.2 shows the results for 28 different range queries specified by their lower bound (l_q) and upper bound (u_q) for the frequency density given above. The true cumulated frequency within the given query range is given in the column f_q . The estimates determined under CVA and ESA are presented as well as a column indicating the better assumption for that particular query. As we can see, in most cases ESA wins. However, experiments by Wang and Sevcik [906] came to the conclusion that the opposite is true and CVA is superior to ESA. (We can follow this claim at least for some of their data sets). Since estimates using CVA are easier to calculate and easily extendible to continuous domains, we prefer them.

Given the above assumptions (and one more to come), the task is to establish the operations *cardinality estimation* and *logical profile propagation*. The latter implies that we can calculate the logical profile of all attributes of any result relation established by applying some algebraic operator. Assume we have solved this task. Then it is clear that the cumulated frequency f_A , which equals $|R|$ in this section, solves the task of cardinality estimation. Hence, we will not mention the cardinality estimation task explicitly any more. The use of the cumulated frequency f_A instead of the seemingly simpler cardinality notation $|R|$ is motivated by the fact that a single attribute will have multiple (small, piecewise) profiles if histograms are applied. To make the formulas of this section readily available for histogram use is the main motivation for using the cumulated frequency.

24.3.3 Profile Propagation for Selection

We start with the selection operation. Let R be a relation and $A, C \in \mathcal{A}(R)$ be two attributes of R . We are given the profiles $b_A = [l_A, u_A, f_A, d_A]$ and $b_C = [l_C, u_C, f_C, d_C]$ and have to calculate the profiles $b'_A = [l'_A, u'_A, f'_A, d'_A]$ and $b'_C = [l'_C, u'_C, f'_C, d'_C]$ for $\sigma_{p(A)}(R)$ for various selection predicates $p(A)$ in attribute A . We assume that the attribute values of A and C are uniformly distributed and that A and C are independent. If a selection predicate uses

no	l_q	u_q	f_q	$\hat{f}_q(cva)$	$\hat{f}_q(esa)$	winner
1	1	2	7	3.5	3.5	
2	1	3	7	5.25	3.5	cva
3	1	4	7	7	7	
4	1	5	11	8.75	7	cva
5	1	6	11	10.5	10.5	
6	1	7	13	12.25	10.5	cva
7	1	8	14	14	14	
8	2	3	0	3.5	3.5	
9	2	4	0	5.25	3.5	esa
10	2	5	4	7	7	
11	2	6	4	8.75	7	esa
12	2	7	6	10.5	10.5	
13	2	8	7	12.25	10.5	esa
14	3	4	0	3.5	3.5	
15	3	5	4	5.25	3.5	esa
16	3	6	4	7	7	
17	3	7	6	8.75	7	esa
18	3	8	7	10.5	10.5	
19	4	5	4	3.5	3.5	
20	4	6	4	5.25	3.5	esa
21	4	7	6	7	7	
22	4	8	7	8.75	7	esa
23	5	6	4	3.5	3.5	
24	5	7	6	5.25	3.5	cva
25	5	8	7	7	7	
26	6	7	2	3.5	3.5	
27	6	8	3	5.25	3.5	esa
28	7	8	3	3.5	3.5	

Figure 24.2: Sample for range query result estimation under CVA and ESA.

two attributes A and B , we again need to give the profile propagation for all attributes C , which are different from them.

Exact match queries The first case we consider is $\sigma_{A=c}$ for a constant c . Clearly, $l'_A = c$, $u'_A = c$. Further,

$$d'_A = \begin{cases} 1 & \text{if } c \in \Pi_A(R) \\ 0 & \text{else} \end{cases}$$

We cannot be sure whether the first or second case occurs. Since no reasonable cardinality estimation should ever return zero, we always assume $c \in \Pi_A(R)$. More generally, we assume that all constants in a query are contained in the database in the according attributes.

As every distinct value occurs about f_A/d_A times, we conclude that $f'_A =$

f_A/d_A . A special case occurs if A is the key. Then, we can immediately conclude that $f'_A = 1$.

Let us now consider another attribute $C \in \mathcal{A}(R)$, $C \neq A$. Since $f'_C = f'_A$, we only need to establish d'_C . For the lack of any further knowledge, we keep the lower and upper bounds, i.e. $l'_C = l_C$ and $u'_C = u_C$. To derive the number of distinct values remaining for attribute B , we can use the formula by Yao/Waters (see Sec. 4.16.1) Denote by $s(p) = |\sigma_{A=c}(R)|/|R| = f'_A/f_A$ the fraction of tuples that survives the selection with predicate $p \equiv A = c$. Fix a distinct value for C . Using the uniform distribution assumption, it occurs in f_C/d_C tuples of R . Then, for this value we have $\binom{f_A - f_C/d_C}{f'_A}$ possibilities to chose f'_A tuples without it. The total number of possibilities to chose f'_A tuples is $\binom{f_A}{f'_A}$. Thus, we may conclude that

$$d'_C = d_C \mathcal{Y}_{f_C/d_C}^{f'_A}(f'_A)$$

Alternatively, we could use

$$d'_C = d_C * (1 - (1 - s(p))^{f_C/d_C})$$

or any other good approximation (see Section 4.16.1).

Range queries Let us now turn to range queries, i.e. selection predicates of the form $c_1 \leq A \leq c_2$, where $l_A \leq c_1 < c_2 \leq u_A$. In all of them, the lower and upper bounds are given by the range, i.e. $l'_A = c_1$ and $u'_A = c_2$. Using the System R approach, we can estimate

$$\begin{aligned} f'_A &= \frac{c_2 - c_1}{u_A - l_A} * f_A \\ d'_A &= \frac{c_2 - c_1}{u_A - l_A} * d_A \end{aligned}$$

This estimate is good for real values.

We could also rewrite the above estimate for the number of distinct values d'_A to

$$d'_A = \frac{c_2 - c_1}{\Delta_A}$$

As soon as we have estimated the number of distinct values in a given range, we can easily derive the cumulated frequency, as every distinct value occurs as often as it did in R . Thus $f'_A = f_A * (d'_A/d_A)$.

For another attribute C , $C \neq A$, the profile propagation is the same as in the case for $A = c$. We only need to define $s(p) = |\sigma_{c_1 \leq A \leq c_2}(R)|/|R|$.

Equality-based correlation The next case we consider is a predicate of the form $A = B$. If $u_A < l_B$ or $u_B < l_A$, the result is empty. If $l_A \neq l_B$ or $u_A \neq u_B$, we first apply a selection with predicate $\max(l_A, l_B) \leq A \leq \min(u_A, u_B)$ and $\max(l_A, l_B) \leq B \leq \min(u_A, u_B)$. So assume w.l.o.g. that $l_A = l_B$ and $u_A = u_B$. Note that $f_A = f_B$. Denote this number by f . Define n to be the number of values in the domain of attributes A and B . For integers, this number is

$n = u_A - l_A + 1$. To refer to the elements of the domain, we assume that it is $\{x_1, \dots, x_n\}$ with $x_i < x_{i+1}$.

Let x be a value in the domain. Then we say that R has a *hole* at x in attribute A , if $x \notin \Pi_A(R)$. Consider a value x in the domain. The probability of not having a hole at x in A is

$$p(x \in A) = \frac{\binom{n-1}{d_A-1}}{\binom{n}{d_A}} = \frac{d_A}{n}$$

In general, we have

$$f'_A = f'_B = \sum_{i=1}^n \overline{f_A} p(x_i = A) p(x_i = B | x_i = A) \quad (24.4)$$

where $\overline{f_A} = f/d_A$ is the average frequency of a distinct value in $\Pi_A(R)$, $p(x_i = A) = d_A/n$ is the probability that a tuple has x_i as its value for attribute A , and $p(x_i = B | x_i = A)$ is the conditional probability that a tuple has x_i in its B value if it is known that it has an A value x_i .

Let us first consider the special case where $\Pi_A(R) \subseteq \Pi_B(R)$. Then $p(x_i = B | x_i = A)$ becomes $1/d_B$. Hence,

$$f'_A = f'_B = \sum_{i=1}^n \frac{f}{d_A} \frac{d_A}{n} \frac{1}{d_B} = f/d_B$$

For $\Pi_B(R) \subseteq \Pi_A(R)$, we get $f'_A = f'_B = f/d_A$. Summarizing these cases, we may conclude that

$$f'_A = f'_B = \frac{f}{\max(d_A, d_B)}$$

which is the formula applied in System R if indices exist on A and B . Clearly, we can calculate an upper bound on the number of distinct values as

$$d'_A = d'_B = \min(d_A, d_B).$$

Let us estimate the cumulated frequency after the selection if none of the above conditions hold and independence of A and B holds. Then, the conditional probability $p(x_i = B | x_i = A)$ becomes $p(x_i = B) = 1/n$. Thus

$$f'_A = f'_B = \sum_{i=1}^n \frac{f}{d_A} \frac{d_A}{n} \frac{1}{n} = \frac{f}{n}$$

If A and B are independent and uniformly distributed, the number of distinct values $d'_A = d'_B$ can be estimated as follows. According to Section 4.16.1, we can estimate the number of distinct values in $\Pi_{AB}(R)$ as $\mathcal{D}(n * n, |R|)$, where $|R| = f_A = f_B$. Since out of the $n * n$ possible pairs of values only n are of the form (x_i, x_i) , only $n/(n * n) = 1/n$ tuples are of the qualifying form. Using this factor, we derive

$$d'_A = d'_B = \frac{\mathcal{D}(n * n, f_A)}{n}$$

In case of $\Pi_A(R) \subseteq \Pi_B(R)$, only d_A such pairs out of $d_A * d_B$ exist. Thus, the factor becomes $d_A/(d_A * d_B) = 1/d_B$. For $\Pi_B(R) \subseteq \Pi_A(R)$, we have the factor $1/d_A$. Both cases can be summarized as in

$$d'_A = d'_B = \frac{\mathcal{D}(n * n, f_A)}{\max(d_A, d_B)}$$

In case the domain size n is not available, we could estimate it by $|\Pi_A^D(R) \cup \Pi_B^D(S)|$. If this number is not available either, we could hesitatingly use $d_A d_B$.

An alternative is to use

$$d'_A = d'_B = d_A * \mathcal{Y}_{f_A/d_A}^{f_A}(f'_A)$$

or some of its approximations like

$$d'_A = d'_B = d_A * (1 - (1 - s(A = B))^{f_A/d_A}),$$

where $s(A = B) = f'_A/f_A$.

Inequality-based correlation As a last exercise, let us calculate the profile for selections of the form $\sigma_{A \leq B}(R)$. For simplicity, we assume $l_A = l_B$ and $u_A = u_B$. Thus, $l'_A = l'_B = l_A$ and $u'_A = u'_B = u_A$. To calculate the cumulative frequency of the result under independence of A and B , we apply the type I equal spread assumption, with $\Delta_A = (u_A - l_A)/(d_A - 1)$. Hence, we assume that $x_i = l_A + (i - 1)\Delta_A$. This gives us

$$\begin{aligned} f'_A &= \sum_{i=1}^{d_A} \overline{f_A} p(x_i \leq B | x_i = A) \\ &= \sum_{i=1}^{d_A} \overline{f_A} p(x_i \leq B) \\ &= \overline{f_A} \sum_{i=1}^{d_A} \frac{x_i - l_B}{u_B - l_B} \\ &= \overline{f_A} \frac{1}{u_B - l_B} \left(\left(\sum_{i=1}^{d_A} x_i \right) - d_A l_B \right) \\ &= \overline{f_A} \frac{d_A}{u_B - l_B} \left(l_A - l_B + \Delta_A \frac{d_A - 1}{2} \right) \\ &= \overline{f_A} \frac{d_A}{u_B - l_B} \frac{(u_A - l_A)}{(d_A - 1)} \frac{(d_A - 1)}{2} \\ &= \overline{f_A} \frac{u_A - l_A}{u_B - l_B} \frac{d_A}{2} \\ &= \frac{\overline{f_A} d_A}{d_A 2} \\ &= \frac{\overline{f_A}}{2} \end{aligned}$$

predicate	f'	d'	comment
$A = c$	$f'_A = f_A/d_A$	$d'_A = 1$	
$c_1 \leq A \leq c_2$	$f'_A = \frac{c_2 - c_1}{u_A - l_A} * f_A$ $f'_A = d'_A * (f_A/d_A)$	$d'_A = \frac{c_2 - c_1}{u_A - l_A} * d_A$ $d'_A = \frac{(c_2 - c_1)}{\Delta_A}$	
$A = B$	$f'_A = \frac{f}{\max(d_A, d_B)}$ $f'_A = f'_B = \frac{f_A}{n}$	$d'_A = d_A * \mathcal{Y}_{f_A/d_A}^{f_A}(f'_A)$ $d'_A = d_A * \mathcal{Y}_{f_A/d_A}^{f_A}(f'_A)$	$\Pi_A(R) \stackrel{\subseteq}{=} \Pi_B(R)$ else
$A \leq B$	$f'_A = f'_B = \frac{f_A}{2}$	$d'_A = d_A * \mathcal{Y}_{f_A/d_A}^{f_A}(f'_A)$	
$p(\mathcal{A})$	$f'_C = f_A$ $C \notin \mathcal{A} = \mathcal{F}(p)$	$d'_C = d_C * \mathcal{Y}_{f_C/d_C}^{f_A}(f'_A)$	

Table 24.4: Profile propagation for selection

As an exercise the reader may verify that $f'_A = (d_A - 1)f_A/(2d_A)$ under the type II equal spread assumption. As an additional exercise the reader should derive d'_A and d'_B . We conjecture that

EXC

$$d'_A = \mathcal{D}(n_A, f'_A)$$

or

$$d'_A = d_A * \mathcal{Y}_{f_A/d_A}^{f_A}(f'_A).$$

The following observation is crucial: Even if in the original relations the values of A and B are uniformly distributed, which typically is not the case, the distribution of the values A and B after the selection with $A \leq B$ is non-uniform. For example,

$$p(x_i \leq B) = \frac{x_i - l_B}{u_B - l_B}$$

for $l_B \leq x_i \leq u_B$. Table 24.4 summarizes our findings about profile propagation for selections.

Open ranges and functions There are plenty of other cases for selection predicates, which we have not discussed. Let us briefly mention a few of them.

Clearly, we have:

$$\begin{aligned}
|\sigma_{A \neq c}(R)| &= |R| - |\sigma_{A=c}(R)| \\
|\sigma_{c_1 < A \leq c_2}(R)| &= |\sigma_{c_1 \leq A \leq c_2}(R)| - |\sigma_{A=c_1}(R)| \\
|\sigma_{c_1 \leq A < c_2}(R)| &= |\sigma_{c_1 \leq A \leq c_2}(R)| - |\sigma_{A=c_2}(R)| \\
|\sigma_{c_1 < A < c_2}(R)| &= |\sigma_{c_1 \leq A \leq c_2}(R)| - |\sigma_{A=c_1}(R)| - |\sigma_{A=c_2}(R)| \\
|\sigma_{c_1 \leq A}(R)| &= |\sigma_{c_1 \leq A \leq u_A}(R)| \\
|\sigma_{c_1 < A}(R)| &= |\sigma_{c_1 \leq A < u_A}(R)| \\
|\sigma_{A \leq c_2}(R)| &= |\sigma_{l_A \leq A \leq c_2}(R)| \\
|\sigma_{A < c_2}(R)| &= |\sigma_{l_A \leq A < c_2}(R)|
\end{aligned}$$

EXC

This helps to estimate the f'_A . The d'_A are left to the reader.

Estimating selectivities for (user defined) functions and expressions can be done by using *computed attributes*. For example, cardinalities for selections with predicates like $g(A) = c$ for a function g can be treated by introducing an additional attribute g_A for which a profile can be established.

24.3.4 Profile Propagation for Join

As for equality-based correlation, we assume for this subsection that the upper and lower bounds of the join attributes A and B are the same. If this is not the case, we apply the according selections first.

Semijoin Let us now turn to the join operator and its variants. We start with the left-semijoin and consider expressions of the type $R \bowtie_{A=B} S$. If $\Pi_A(R) \subseteq \Pi_B(S)$, then $R \bowtie_{A=B} S = R$, and no profiles change. If $\Pi_A(R) \supseteq \Pi_B(S)$, then $f'_A = f_A d_B / d_A$ and $d'_A = d_B$. If A and B are independent, we calculate

$$f'_A = \sum_{i=1}^n \frac{f_A}{d_A} p(x_i = A) p(x_i \in B) = \sum_{i=1}^n \frac{f_A}{d_A} \frac{d_A}{n} \frac{d_B}{n} = \frac{f_A d_B}{n}.$$

and

$$d'_A = \frac{d_A d_B}{n}.$$

For an attribute $C \in \mathcal{A}(R) \setminus \{A, B\}$, we have $f'_C = f'_A$ and

$$d'_C = d_C * \mathcal{Y}_{f_C/d_C}^{f'_A}(f'_A).$$

Regular Join For the regular join $R \bowtie_{A=B} S$. Let us start with an attribute $C \in \mathcal{A}(R) \setminus \{A, B\}$. We can apply the formulas for the semijoin because $\Pi_C^D(R \bowtie_{A=B} S) = \Pi_C^D(R \bowtie S)$. For attributes $C \in \mathcal{A}(S) \setminus \{A, B\}$ remember that the join commutes.

We turn to the case where $\Pi_A(R) \subseteq \Pi_B(S)$. In this case, it is easy to see that $f'_A = f'_B = f_A f_B / d_B$ and $d'_A = d'_B = d_A$. If $\Pi_A(R) \supseteq \Pi_B(S)$, we have just to exchange the roles of R and S .

If A and B are independent, we proceed as follows. Denote again by n the domain size of A and B and the values of $D_A = \Pi_A(R)$ by $\{x_1, \dots, x_{d_A}\}$. Then, we can argue that

$$f'_A = f'_B = \sum_{i=1}^n \frac{f_A}{d_A} \frac{f_B}{d_B} p(x_i = A) p(x_i \in B) = \sum_{i=1}^n \frac{f_A}{d_A} \frac{f_B}{d_B} \frac{d_A}{n} \frac{d_B}{n} = \frac{f_A f_B}{n}.$$

Rosenthal showed that this result also holds if the condition of *fairness* holds for at least one relation [734]. A relation is called *fair* with respect to an attribute A if for the expected value $E(|\sigma_{A=x}(R)|) = |R|/n_A$ holds. In this case, the expected value for the result of the join is $(|R||S|)/n$. Note that $\Pi_A^D(R \bowtie_{A=B} S) = \Pi^D(R \bowtie_{A=B} S)$. Thus, we can estimate the number of distinct values as

$$d'_A = d'_B = \frac{d_A d_B}{n}.$$

Selfjoin The above formulas only apply if we are not dealing with a selfjoin. Of course, $R \bowtie_{A=B} R$ does not pose any problems. However, $R \bowtie_{A=A} R$ does, because all tuples find a join partner. The estimates are easy to derive:

$$\begin{aligned} f'_A &= \frac{f_A f_A}{d_A} \\ d'_A &= d_A \end{aligned}$$

For all attributes C other than A , $f'_C = f'_A$ and $d'_C = d_C$.

As pointed out by [23], selfjoin sizes can be used to derive an upper bound for general joins:

$$|R \bowtie_{A=B} S| \leq \frac{|R \bowtie_{A=A} R| + |S \bowtie_{B=B} S|}{2}.$$

This bound, which is an immediate consequence of the Cauchy Schwarz inequality, can be used as a sanity check. Table 24.5 summarizes our findings for joins.

24.3.5 Profile Propagation for Projection

The next operator we deal with is the duplicate eliminating projection. Given our work from Chapter 4, it is not surprising, that we simply have to apply the $\mathcal{D}$ function. For single attributes A , we have

$$f'_A = d'_A = d_A$$

For a set of attributes $\mathcal{A} = \{A_1, \dots, A_n\}$, the result cardinality of $\Pi_{\mathcal{A}}^D(R)$ is

$$\mathcal{D}\left(\prod_{i=1}^n n_{A_i}, |R|\right),$$

if n_{A_i} , the size of the domain of A_i , is known. Otherwise, we can use the estimate

$$\mathcal{D}\left(\prod_{i=1}^n d_{A_i}, |R|\right)$$

join	f'	d'	comment
$R \bowtie_{A=B} S$	$f'_A = f_A$	$d'_A = d_A$	$\Pi_A(R) \subseteq \Pi_B(S)$
	$f'_A = \frac{f_A d_B}{d_A}$	$d'_A = d_B$	$\Pi_A(R) \supseteq \Pi_B(S)$
	$f'_A = \frac{f_A d_B}{n}$	$d'_A = \frac{d_A d_B}{n}$	else
	$f'_C = f'_A$	$d'_C = d_C * \mathcal{Y}_{d_C/f_C}^{f'_A}(f'_A)$	for $C \in \mathcal{A}(R) \setminus \{A, B\}$
$R \bowtie_{A=B} S$	$f'_A = \frac{f_A f_B}{d_B}$	$d'_A = d_A$	$\Pi_A(R) \subseteq \Pi_B(S)$
	$f'_A = \frac{f_A f_B}{n}$	$d'_A = \frac{d_A d_B}{n}$	else
$R \bowtie_{A=A} R$	$f'_A = \frac{f_A f_A}{d_A}$	$d'_A = d_A$	

Table 24.5: Profile propagation for joins

The number of distinct values in any attribute does not change, i.e. $d'_{A_i} = d_{A_i}$.

If we have a functional dependencies and $\kappa \rightarrow \mathcal{A}$ for a set of attributes $\mathcal{A}$ and $\kappa \subset \mathcal{A}$, then

$$\Pi_{\mathcal{A}}^D(R) = \Pi_{\kappa}^D(R).$$

Further, if $|\Pi_{\mathcal{A}}^D(R)| = |R|$, we have $|\Pi_{\mathcal{A}'}^D(R)| = |R|$ for all A' with $\mathcal{A}' \supseteq \mathcal{A}$.

The above estimates for the result size of a duplicate eliminating projection assumes that the attribute values are uniformly distributed, i.e., every distinct value occurs with the same probability. As we will not deal with projections any more in this part of the book, let us complete the subject by giving an approach where each attribute value can have its own probability of occurrence. This is not unlikely, and for attributes with few possible values the following approach proposed by Yu, Zuzarte, and Sevcik is quite reasonable [966]. The assumptions are that the attributes are independent and the values of each of them are drawn by independent Bernoulli trials. Under these assumptions, they derive the following three results: a lower bound, an upper bound, and an estimate for the expected number of distinct values in the projection. In order to state these results, we need some additional notation. Let R be a relation and define $N = |R|$. Further let $G = \{A_1, \dots, A_n\}$ be a subset of the attributes of R . Define $d_i = |\Pi_{A_i}^D(R)|$ to be the number of distinct values occurring in attribute A_i . We denote values of A_i by $a_{i,1}, \dots, a_{i,d_i}$.

We wish to derive an estimate for $D_G = |\Pi_G^D(R)|$. Therefore, we model each attribute A_i by a frequency vector $f_i = (f_{i,1}, \dots, f_{i,d_i})$ where $f_{i,j}$ is the number of occurrences of the j -th distinct value $a_{i,j}$ of A_i divided by N . If, for example, A_1 has three distinct values which occur 90, 9, and 1 times in a relation with $N = 100$ elements, then f_1 becomes $(0.9, 0.09, 0.01)$.

Let us first look at bounds for D_G . Trivially, D_G is bounded from above by

$$D_G \leq \min\{N, \prod_{i=1}^n d_i\}$$

and from below by

$$D_G \geq \max_{i=1}^n d_i.$$

These bounds are very rough. This motivated Yu et al. to derive better ones.

Before we proceed, let us consider another example. Assume we have three attributes A_1 , A_2 , and A_3 all with frequency vectors $f_i = (0.9, 0.09, 0.01)$ for a relation of size $N = 100$. Since we assume attribute independence, the probability of $(a_{1,3}, a_{2,3}, a_{3,3})$ is $0.01 * 0.01 * 0.01$. Thus, its occurrence in a relation of size 100 is highly unlikely. Hence, we expect D_G to be less than $27 = 3*3*3$. In general, we observe that the probability of occurrence of a tuple $(a_{1,j_1}, \dots, a_{n,j_n})$ is the product of the relative frequencies $f_{1,j_1} * \dots * f_{n,j_n}$. From this, the basic idea of the approach of Yu et al. becomes clear: we have to systematically consider all the different possibilities to multiply relative frequencies. This is nicely captured by the Kronecker product (tensor product).

Before we proceed, let us state the upper and lower bounds in case of two attributes by giving two theorems developed by Yu et al. [966].

Theorem 24.3.1 (lower bound) *For a set of attributes $\{A_1, A_2\}$ of a relation R and its frequency vectors, we define $l_{i,j}$ for $i = 1, 2$ and $1 \leq j \leq d_i$ as the minimum number of different values that have to be combined with $f_{i,j}$ given the marginals, i.e.*

$$l_{i,j} = \min\{|F| \mid F \subseteq \{1, \dots, d_{i'}\}, \forall q \notin F \ S(F) \leq f_{i,j} < S(F) + f_{i',q}\}$$

where $i' = 3 - i$ and $S(F) = \sum_{p \in F} f_{i',p}$. Further define

$$D_G^\perp = \max_{i=1,2} \sum_{j=1}^{d_i} l_{i,j}.$$

Then $D_G^\perp$ is a lower bound for D_G and $D_G^\perp \geq \max_{d_1, d_2}$.

Theorem 24.3.2 (upper bound) *For a set of attributes $\{A_1, A_2\}$ of a relation R and its frequency vectors, we define $u_{i,j}$ for $i = 1, 2$ and $1 \leq j \leq d_i$ as the maximum number of different values that can be combined with $f_{i,j}$ given the marginals, i.e.*

$$u_{i,j} = \min\{d_{i'}, f_{i,j}N\}$$

where $i' = 3 - i$. Further define

$$D_G^\top = \min_{i=1,2} \sum_{j=1}^{d_i} u_{i,j}.$$

Then $D_G^\top$ is a lower bound for D_G and $D_G^\top \leq \min\{N, \prod_{i=1}^n d_i\}$.

The algorithm in Fig. 24.3 calculates the lower bound $D_G^\perp$. Calculating the upper bound $D_G^\top$ is much easier. For each $f_{i,j}$, we compute $u_{i,j}$ by simply comparing $f_{i,j}N$ and $d_{i'}$. Adding up the $u_{i,j}$ for each attribute and taking the lesser of the two sums gives the desired result.

```

CalculateLowerBoundForNumberOfDistinctValues( $f_1, f_2$ )
/* frequency vectors  $f_1$  and  $f_2$  */
sort  $f_i$  ( $i = 1, 2$ ) in descending order;
for  $i = 1, 2$  {
     $i' = 3 - i$ ;
    for  $j = 1, \dots, d_i$  {
         $k = 1$ ;
        while ( $f_{i,j} > \sum_{l=1}^k f_{i',l}$ )
            ++ $k$ ;
         $l_{i,j} = k$ ;
    }
     $lb_i = \sum_{j=1}^{d_i} l_{i,j}$ ;
}
 $D_G^\perp = \max_{i=1,2} lb_i$ ;
return  $D_G^\perp$ 

```

Figure 24.3: Calculating the lower bound $D_G^\perp$

Let us start by repeating the definition of the Kronecker product of two matrices $A = (a_{i,j})$ and $B = (b_{i,j})$ of dimension $n \times m$ and $n' \times m'$. The result $A \otimes B$ is a matrix of dimension $nn' \times mm'$. The general definition is

$$A \otimes B = \begin{pmatrix} a_{1,1}B & a_{1,2}B & \dots & a_{1,m}B \\ a_{2,1}B & a_{2,2}B & \dots & a_{2,m}B \\ \dots & \dots & \dots & \dots \\ a_{n,1}B & a_{n,2}B & \dots & a_{n,m}B \end{pmatrix}.$$

The estimate can not be calculated easily. First, we calculate the Kronecker product $f_G = f_1 \otimes \dots \otimes f_n$ of all frequency vectors. Note that to every value combination $v \in \Pi_{A_1}^D(R) \times \dots \times \Pi_{A_n}^D(R)$ there corresponds exactly one component in f_G , which contains its probability of occurrence. With this observation, it is easy to derive the following theorem, in which we denote by $f_{G,i}$ the i -th component of f_G and by M its length, i.e. $M = \prod_{i=1}^n d_i$. Further remember that $N = |R|$.

Theorem 24.3.3 (estimate) *Let the following assumptions hold:*

1. *The data distributions of individual attributes in G are independent.*
2. *For the value combinations v_i , its occurrence is the result of an independent Bernoulli trial, with the success (occurrence) probability $f_{G,i}$.*
3. *The occurrences of individual possible value combinations are independent.*

Then, the expected number of distinct values D_G is

$$E[D_G] = M - \sum_{i=1}^M (1 - f_{G,i})^N.$$

```

EstimateNumberOfDistinctValues( $f_1, \dots, f_n$ )
  /* frequency vectors  $f_i$  */
  /* step 1: calculate  $f_G = f_1 \otimes \dots \otimes f_n$  */
   $f_G = f_1$ ;
  for ( $i = 2$ ;  $i \leq n$ ;  $++i$ ) {
     $f^{\text{old}} = f_G$ ;
     $f_G = \epsilon$ ; // empty vector
    for ( $j = 1$ ;  $j \leq |f^{\text{old}}|$ ;  $++j$ ) {
      for ( $k = 1$ ;  $k \leq d_i$ ;  $++k$ ) {
         $f_G = \text{push\_back}(f_G, f_j^{\text{old}} \times f_{i,j})$ ; // append a value to a vector
      }
    }
  }
  /* step 2: compute the expected number of distinct value combinations */
   $S = 0$ ;
  for ( $j = 1$ ,  $j \leq M$ ;  $++j$ ) { //  $M = \text{length}(f_G)$ 
     $S += (1 - f_j)^N$ ;
  }
   $\hat{D}_G = M - S$ ;
  return  $\hat{D}_G$ ;

```

Figure 24.4: Calculating the estimate for D_G

The algorithm for computing the estimate is given in Fig. 24.4. In the first, most expensive phase, it constructs the Kronecker product. Then, the simple calculations according to the theorem follow. A more efficient implementation would calculate the Kronecker product only implicitly. Further, the frequency vectors may not be completely known but only a part of it via some histogram. As was also shown by Yu et al., end-biased histograms (coming soon) are optimal under the following error metrics. Let $\hat{D}_{G,\text{hist}}$ be the estimate derived for a histogram. The error function they consider is

$$\mathcal{E}_{\text{abs}} = |\hat{D}_G - \hat{D}_{G,\text{hist}}|.$$

24.3.6 Profile Propagation for Division

As a starting point, we use an observation made by Merrett and Otoo [603]. Assume we are given two sets X and Y , which are both subsets of a finite domain D with $|D| = n$ elements. Then $|X| < |Y|$ implies that $X \not\supseteq Y$. Otherwise, we can calculate the probability of $X \supseteq Y$ as

$$p(X \supseteq Y) = \binom{|X|}{|Y|} / \binom{n}{|Y|}$$

Now let R and S be two relations with $\mathcal{A}(R) = \{A, B\}$ and $\mathcal{A}(S) = \{B\}$. A value $a \in \Pi_A^D(R)$ is contained in the result of $R \div B$ if and only if $\Pi_B(\sigma_{A=a}(R)) \supseteq$

S . Hence, for any such a , $\overline{f_A} = f_A/d_A$ and n_B equal to the size of the common domain of $R.B$ and $S.B$, we can calculate the survival probability as

$$\binom{\overline{f_A}}{|S|} / \binom{n_B}{|S|}$$

provided that $\overline{f_A} \geq |S|$ and R is a set. Denote by f'_A and d'_A the cumulated frequency and the number of distinct values for attribute A in the result of $R \div S$. Then we have the estimate

$$f'_A = d'_A = d_A * \binom{\overline{f_A}}{|S|} / \binom{n_B}{|S|}$$

in case R is a set.

If R is a bag, we must be prepared to see duplicates in $\sigma_{A=a}(R)$. In this case we can adjust the above formula to

$$f'_A = d'_A = d_A * \binom{x_A}{|S|} / \binom{n}{|S|}$$

where $x_A = \mathcal{D}(x_A, n_A)$, and n_A is the size of the domain of $R.A$.

If there is some variance among the number of distinct values associated with the $a \in \Pi_A^D(R)$, the estimate will be rough. To cure this, we need better information. Define for each $a \in \Pi_A^D(R)$ the number h_a to be the number of distinct b values occurring for it, i.e. $h_a = |\Pi_B^D(\sigma_{A=a}(R))|$. Then we could estimate f'_A and d'_A as follows:

$$f'_A = d'_A = \sum_{a \in \Pi_A^D(R)} \binom{h_a}{|S|} / \binom{n}{|S|}$$

Keeping h_a for every possible a may not be practical. However, if the number of distinct values in $H = \{h_a | a \in \Pi_A^D(R)\}$ is small, we can keep the number of distinct a values for each possible h_a . Assume $H = \{h_1, \dots, h_k\}$ and define

$$g_i = |\{a \in \Pi_A^D(R) | h_a = h_i\}|,$$

then we have the estimate

$$f'_A = d'_A = \sum_{i=0, h_i \geq |S|}^k g_i \binom{h_i}{|S|} / \binom{n}{|S|}.$$

24.3.7 Remarks

NULL Values Our profile is not really complete for attributes which can have NULL values. To deal with these, we need to extend our profiles by the frequency $d_A^\perp$ with which NULL occurs in an attribute A of some relation. It is straightforward to extend the above profile to deal with this additional count.

Name	Definition	Error minimized
$\text{median}(\tilde{x})$	$\begin{cases} x_{(n+1)/2} & n \text{ odd} \\ (x_{n/2} + x_{n/2+1})/2 & n \text{ even} \end{cases}$	$\mathcal{E}_1 = \sum_{i=1}^n x_i - \hat{x} $
$\text{mean}(\bar{x})$	$1/n \sum_{i=1}^n x_i$	$\mathcal{E}_2 = \sqrt{\sum_{i=1}^n (x_i - \hat{x})^2}$
middle	$(\max(x) + \min(x))/2$	$\mathcal{E}_\infty = \max_{i=1}^n x_i - \hat{x} $
q-value	$\sqrt{\max(X) \min(X)}$	$\mathcal{E}_q = \max_{i=1}^n \max\{x_i/\hat{x}, \hat{x}/x_i\}$

Table 24.6: Approximations of a set of numbers by a single number

Uniformity is not sufficient As we have seen, even if all attributes are uniformly distributed, which is rarely the case in practice, the result of algebraic operators may no longer be uniformly distributed. As a consequence, we need to be concerned with the approximation of the true distribution of values.

Sets of Attributes Note that nothing prevents us to use the formulas developed above for selections and joins if A and B are attribute sets instead of single attributes. We just have to know or calculate d_A for sets of attributes $\mathcal{A}$.

24.4 Approximation of a Set of Values

24.4.1 Approximations and Error Metrics

Assume we have a set of values $x = \{x_1, \dots, x_n\}$. The task we want to tackle is to approximate this set of values by a single value. The left two columns of Table 24.6 show the names and definitions of some possible approximations. Whereas mean and median are well known, the other two may be not. The middle is defined as the value exactly between the minimum and maximum of X . Hence, the distance from the middle to either extreme is the same. The q-value needs some further restriction: the values in X must be larger than zero. For our purposes, this restriction is not bad since execution costs are typically larger than zero and frequencies are mostly larger than zero if they are not exactly zero. The latter case needs some special attention if we use something like the q-value, which we could also term *geometric* or *multiplicative middle*.

Let us take a look at a simple example. Assume $X = \{1, 2, 9\}$. Then we can easily calculate the approximations summarized in the following table:

median	mean	middle	q-value
2	4	5	3

Which of these approximations is the best one? The answer depends on the error function we wish to minimize. Therefore, the rightmost column of Table 24.6 shows some error functions, which are minimized by the approximation defined in the same line. The variable $\hat{x}$ denotes the estimate whose error is to be

calculated. For $\mathcal{E}_2$ there exist plenty of equivalent formulations, where we think of two error measures as being equivalent, if and only if they result in the same minimum. Some important alternatives are $1/n \sum (x_i - \hat{x})^2$, $1/(n-1) \sum (x_i - \hat{x})^2$ (empirical variance), and simply $\sum (x_i - \hat{x})^2$.

A nice property half of the approximations give us are error bounds. These are $\mathcal{E}_\infty$ and $\mathcal{E}_q$. Define the *spread* s of x as $\max(x) - \min(x)$. Then, given the middle m of x , we have for every $x_i \in x$ that

$$m - s/2 \leq x_i \leq m + s/2.$$

Thus, we have a symmetric, additive error bound for all elements in x . Define the *geometric spread* as $s = \sqrt{\max(x)/\min(x)}$. Then we have a symmetric, multiplicative error bound for all elements x_i in x given by

$$(1/s)q \leq x_i \leq sq$$

if q is the geometric middle. The following table shows the possible errors for all approximations of our example set $X = \{1, 2, 9\}$:

	median	mean	middle	geo. mean
	2	4	5	3
$\mathcal{E}_1$	8	10	11	9
$\mathcal{E}_2$	7.1	6.2	6.4	6.4
$\mathcal{E}_\infty$	7	5	4	6
$\mathcal{E}_q$	4.5	4	5	3

Which of these error metrics and, hence, which approximation is the best? Obviously, this depends on the application. In the query compiler context, $\mathcal{E}_1$ plays no role that we are aware of. $\mathcal{E}_2$ plays a predominant role as it is used to approximate values in a given histogram bucket. This has not come by sharp reasoning about the best possibility but merely by the existence of a huge body of literature in this area. Currently, the other two error metrics, $\mathcal{E}_\infty$ and $\mathcal{E}_q$, play minor roles. But this will change.

24.4.2 Example Applications

Let us discuss some example applications relevant to building a query compiler. Assume we have to come up with the execution time (CPU usage) for some function. This could be a simple arithmetic function built into our system, a hash function executed for a hash-join, the CPU time used to dereference a TID if the according page is in memory, the CPU time needed to search a B-tree page residing in the buffer, or the CPU time needed to load a page from secondary storage into the buffer. Careful as we are, we measure the function's execution time several times. Almost certainly, the numbers will not be same for every execution, except maybe for the simplest functions. To come up within a single number, we need to approximate the set of numbers derived from our measurements. If the function is going to be executed many times within a query execution plan (in which it occurs), we need to cost the average case and the mean is the approximation of choice. We will see more applications in Section 24.5.2.

24.5 Approximation with Linear Models

24.5.1 Linear Models

In this section, we want to approximate a given set of points (x_i, y_i) ($1 \leq i \leq m$) by a linear combination $\hat{f}$ of given functions Φ_j , $1 \leq j \leq n$. The general assumption is that $m > n$. We define the estimation function $\hat{f}$ as

$$\hat{f}(x) := \sum_{j=1}^n c_j \Phi_j(x)$$

for coefficients $c_j \in \mathbb{R}$. The estimates $\hat{y}_i$ for y_i are then derived from $\hat{f}$ by

$$\hat{y}_i := \hat{f}(x_i) = \sum_{j=1}^n c_j \Phi_j(x_i).$$

Note that the functions Φ_j are not necessarily linear functions. For example, we could use polynomials $\Phi_j(x) = x^{j-1}$. Further, there is no need for x to be a single number. It could as well be a vector $\vec{x}$.

It is convenient to state our approximation problem in terms of vectors and matrices. Let (x_i, y_i) be the points we want to approximate and Φ_j , $1 \leq j \leq n$ be some functions. We define the *design matrix* $A \in \mathbb{R}^{m \times n}$, $A = (a_{i,j})$ by

$$a_{i,j} = \Phi_j(x_i)$$

or, equivalently, as an explicit matrix

$$A = \begin{pmatrix} \Phi_1(x_1) & \Phi_2(x_1) & \Phi_3(x_1) & \dots & \Phi_n(x_1) \\ \Phi_1(x_2) & \Phi_2(x_2) & \Phi_3(x_2) & \dots & \Phi_n(x_2) \\ \dots & \dots & \dots & \dots & \dots \\ \Phi_1(x_m) & \Phi_2(x_m) & \Phi_3(x_m) & \dots & \Phi_n(x_m) \end{pmatrix} \quad (24.5)$$

Assume we wish to approximate the points by a polynomial of degree $n - 1$. Then, $\Phi_i(x) = x^{i-1}$ and the design matrix becomes

$$A = \begin{pmatrix} 1 & (x_1)^1 & (x_1)^2 & \dots & (x_1)^{n-1} \\ 1 & (x_2)^1 & (x_2)^2 & \dots & (x_2)^{n-1} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & (x_m)^1 & (x_m)^2 & \dots & (x_m)^{n-1} \end{pmatrix}.$$

In the simplest case, where we want to use a linear function $\hat{f}(x) = c_1 + c_2x$ to approximate the points, the design matrix becomes

$$A = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \dots & \dots \\ 1 & x_m \end{pmatrix}.$$

As an example consider the three points

$$(1, 20), (2, 10), (3, 60).$$

The design matrix becomes

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \quad (24.6)$$

For every column vector $\vec{c} = (c_1, c_2)^T$

$$A\vec{c}$$

gives the result of $\hat{f}$ for all points. Clearly, $\vec{c}$ should be determined such that the deviation of $A\vec{c}$ from $\vec{y} = (y_1, \dots, y_m)^T$ becomes minimal.

The deviation could be zero, that is $A\vec{c} = \vec{y}$. However, remember our assumption that $m > n$. This means that we have more equations than variables. Thus, we have an overdetermined system of equations and it is quite unlikely that a solution to this system of equations exists. This motivates our goal to find an approximation as good as possible. Next, we formalize this goal.

Often used measures for deviations or distances of two vectors are based on norms.

Definition 24.5.1 (norm) *Let S be a linear space. Then a function $\|x\| : S \rightarrow \mathbb{R}$ is called a norm if and only if it has the following three properties:*

1. $\|x\| > 0$ unless $x = 0$
2. $\|\lambda x\| = |\lambda| \|x\|$
3. $\|x + y\| \leq \|x\| + \|y\|$

Various norms, called p norms can be found in the literature. Let $x \in \mathbb{R}^n$ and $p \geq 1$ where $p = \infty$ is possible. Then

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}.$$

The most important norms are the l_1 , l_2 , and l_∞ norms:

$$\begin{aligned} \|x\|_1 &= |x_1| + \dots + |x_n| \\ \|x\|_2 &= \sqrt{(x_1)^2 + \dots + (x_n)^2} \\ \|x\|_\infty &= \max_{i=1}^n |x_i| \end{aligned}$$

Using these norms, we can define distance functions d_1 , d_2 , and d_∞ . For two vectors x and y in $\mathbb{R}^n$, we define

$$\begin{aligned} d_1(x, y) &= \|x - y\|_1 \\ d_2(x, y) &= \|x - y\|_2 \\ d_\infty(x, y) &= \|x - y\|_\infty \end{aligned}$$

It should be clear, that these define the error measures $\mathcal{E}_1$, $\mathcal{E}_2$, and $\mathcal{E}_\infty$, which we used in Sec. 24.4. The only missing error function is $\mathcal{E}_q$. We immediately fill this gap, and start with the one dimensional case.

Definition 24.5.2 (Q-paranorm in R) Define for $x \in R$,

$$\|x\|_Q = \begin{cases} \infty & \text{if } x \leq 0 \\ 1/x & \text{if } 0 < x \leq 1 \\ x & \text{if } 1 \leq x \end{cases}$$

$\|\cdot\|_Q$ is called Q-paranorm.

Note that for $x > 0$, $\|x\|_Q = \max(x, 1/x)$. The multivariate case is a straightforward extension using the maximum over all components:

Definition 24.5.3 (Q-paranorm in R^n) For $x \in R^n$, define

$$\|x\|_Q = \max_{i=1}^n \|x_i\|_Q.$$

We denote this paranorm by l_q .

Definition 24.5.4 (paranorm) Let S be a linear space. Then a function $\|x\| : S \rightarrow R$ is called a paranorm if and only if the following two properties hold:

1. $\|x\| \geq 0$
2. $\|x + y\| \leq \|x\| + \|y\|$

The Q-paranorm is a norm, hence the name. The only missing part is the distance function stated next. Let x and y be two vectors in $\mathbb{R}^n$, where $y = (y_1, \dots, y_n)^T$ with $y_i > 0$. Then we define

$$d_q(x, y) = \|x/y\|_Q$$

where we define x/y for two column vectors $x, y \in \mathbb{R}^n$ as follows:

$$x/y = (x_1/y_1, \dots, x_n/y_n)^T.$$

Between norms there exist some inequalities. For all vectors $x \in \mathbb{R}^n$, we have

$$\begin{aligned} \|x\|_2 &\leq \|x\|_1 \leq \sqrt{n}\|x\|_2 \\ \|x\|_\infty &\leq \|x\|_2 \leq \sqrt{n}\|x\|_\infty \\ \|x\|_\infty &\leq \|x\|_1 \leq n\|x\|_\infty \end{aligned}$$

For l_q , no such inequality exists as $\|x\|_Q$ approaches infinity as x approaches zero.

We can now formally state the approximation problem. Let $A \in \mathbb{R}^{m \times n}$ be the design matrix and (x_i, y_i) , $1 \leq i \leq m$ be a set of points, and $\vec{y} = (y_1, \dots, y_m)$. The goal is to find a vector $\vec{a}^* \in \mathbb{R}^n$ minimizing $d(A\vec{a}, \vec{y})$. That is, we look for $\vec{a}^* \in \mathbb{R}^n$ such that

$$d(A\vec{a}^*, \vec{y}) = \min_{\vec{a} \in \mathbb{R}^n} d(A\vec{a}, \vec{y}) \quad (24.7)$$

$\vec{a}^*$ is then called *solution* of the approximation problem or *best approximation*.

For different l (d), we get different problems. For l_1 the problem is called *quantile regression*. We will not deal with it here, since we do not know of any application of it in the database context. The solutions for the problems for l_2 , l_∞ , and l_q are discussed in subsequent sections, after we have given some example applications of what needs to be approximated in a DBMS. Before we proceed, let us give the solutions for approximating the points $(1, 20)$, $(2, 10)$, $(3, 60)$ with a linear function $\alpha + \beta x$. The following table shows the values of x , y and estimates for y produced by the best approximations $\hat{f}_{l_2}$, $\hat{f}_{l_\infty}$, $\hat{f}_{l_q}$, which minimize l_2 , l_∞ , and l_q , resp. Additionally, we give the α and β of the best approximations as well as their quality measured by l_1 , l_2 and l_q .

x	y	$\hat{f}_{l_2}$	$\hat{f}_{l_\infty}$	$\hat{f}_{l_q}$
1	20	10	5	10
2	10	30	25	20
3	60	50	45	30
α		20	20	10
β		-10	-15	0
l_2		14.1421	15	19.1485
l_∞		20	15	30
l_q		3	4	2

Let us repeat some general insights into approximation problems as defined above. Thereby, we follow the exposition of Watson [914]. We start with stating theorems on the existence of a solution. The following two theorems only apply to norms. That is, they do not apply to l_q . However, as we will see later, solutions under l_q exist.

Theorem 24.5.5 (Existence 1) *Let M denote a compact set in a normed linear space. Then to each point g of the space there exists a point of M closest to g .*

Compactness is a sufficient but not a necessary condition.

Theorem 24.5.6 (Existence 2) *Let M be a finite dimensional subspace of a normed linear space S . Then there exists a best approximation in M to any point of S .*

The next point to consider is the uniqueness of a solution. Proving the uniqueness of a solution is easy, if the norm is *strictly convex*.

Definition 24.5.7 ((strictly) convex) *Let $f(x)$ be a function on the elements x of a linear space S . Then $f(x)$ is convex if*

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

for all $x_1, x_2 \in S$ and $0 \leq \lambda \leq 1$.

If $0 < \lambda < 1$ implies strict inequality in the above inequality, $f(x)$ is called strictly convex.

It is easy to show that all l_p norms for $p \neq \infty$ are strictly convex and that l_∞ and l_q are convex, but not strictly convex. For strictly convex norms, it is easy to show that a solution is unique.

Theorem 24.5.8 *In a strictly convex normed linear space S , a finite dimensional subspace M contains a unique best approximation to any point of S .*

Although l_∞ and l_q are not strictly convex, under certain circumstances a unique best approximation exists for them. This is discussed in subsequent sections.

Considering the above, one might conjecture that l_2 approximation is much simpler than l_∞ or l_q approximation. This is indeed the case. We will not repeat all the findings from approximation theory and the algorithms developed. There are plenty of excellent textbooks on this matter. We highly recommend the excellent book of Golub and van Loan [336], which discusses l_2 approximation and several algorithms to solve them (e.g. QR factorization and SVD). Other good books to refresh one's knowledge on matrix algebra are [389, 408, 432, 777]. Überhuber wrote another good book discussing l_2 approximation, QR factorization and SVD [885]. In the context of statistics, many different regression models exist to approximate a given set of data. An excellent overview is provided by [?]. Another good reading, not only in this context, is the book by Press, Teukolsky, Vetterling, and Flannery [702]. Before reading these books, it might be helpful to repeat some linear algebra and some basics of matrices. An excellent book for doing so was written by Schmidt and Trenkler [777]. The only book we know of that discusses approximation under l_∞ , is the one by Watson, already cited above [914]. Approximation under l_q is not discussed in any textbook. Hence, we must refer to the original articles [617, 623]. In any case, since mathematics is quite involved at times, we give explicit algorithms only for the approximation by a linear function. For all other cases, we refer to the literature. ToDo

24.5.2 Example Applications

In this section, we give some examples of approximation problems occurring in the database context. As we will see, different problems demand different norms. Additionally, we sketch how to use approximations. The details are left to the reader as an exercise.

Disk seek times

There exist small benchmarks, which measure the disk seek time for travelling n cylinders (see Sec. 4.1). To cover for random errors, many measurements are taken. The task is to find the parameters d and c_i , $1 \leq i \leq 4$, for the disk seek time formula from Sec. 4.1:

$$\text{seektime}(d) = \begin{cases} c_1 + c_2\sqrt{d} & d \leq c_0 \\ c_3 + c_4d & d > c_0 \end{cases}$$

Since many seeks occur during the processing of a single query, l_2 is the appropriate norm. On the surface, we seem to have a problem using a simple linear model. However, we can approximate the parts $c_1 + c_2\sqrt{d}$ and $c_3 + c_4d$ for several distinct c_0 either by trying a full range of values for c_0 or by a binary search. The solution for c_0 we then favor is the one in which the maximum of the errors on both parts becomes minimal. A second problem is the occurrence of $\sqrt{d}$,

since this does not look linear. However, choosing $\Phi_1 = 1$ and $\Phi_2(x) = \sqrt{x}$ will work fine.

Another method is to transform a set of points (x_i, y_i) with two (injective) transformation functions t_x and t_y into the set of points $(t_x(x_i), t_y(y_i))$. Then this set is approximated and the result is transformed back. While using this approach, special attention has to be paid to the norm, as it can change due to the transformation. We see examples of this later on in Sec. 24.5.6.

Functions sensitive to parameter size

Another example is to approximate the execution time of a hash function on string values. As its calculation depends on the length of the input string, measurements can be taken for various lengths. Using l_2 as a norm is perfect, because the hash function is typically executed many times during a hash join or hash teams [352].

Approximation of frequency densities and distributions

We start by demonstrating the usage of approximating functions for cardinality estimation. Then, we look at the choice of error metrics for estimating selectivity results and the influence of cardinality estimation errors on joins.

Let R be a relation and A one of its attributes. Let (x_i, f_i) denote the frequency f_i with which the value x_i occurs in attribute A . Typically, only those values x_i are written down and approximated for which $f_i \neq 0$. We further assume that the x_i are sorted, i.e., $x_i < x_{i+1}$. Using the methods to come, we can approximate this set of points by a function $\hat{f}(x)$. To calculate the output cardinality of a selection $\sigma_{A=c}(R)$, we can simply return $\hat{f}(c)$ as an estimate. Hence it is a good choice to use l_q (see below for strong arguments).

To calculate the result cardinality for a range query of the form $\sigma_{c_1 \leq A \leq c_2}(R)$, we distinguish several cases. First, assume that the domain of A is discrete and the number of values between c_1 and c_2 is small. Then, we can calculate the result quite efficiently by

$$\sum_{c_1 \leq x \leq c_2} \hat{f}(x)$$

if the active domain of the attribute under consideration is dense, which assume in this subsection. In Section ??, we present estimation formulas without this assumption. If the number of values between c_1 and c_2 is too large for an explicit summation, we can apply speed-up techniques if the function $\hat{f}$ has a simple form. For example, if $\hat{f}$ is a linear function $\hat{f}(x) = \alpha + \beta x$, the above sum can be calculated very efficiently. If the number of values between c_1 and c_2 is very large and no efficient form for the above sum can be found, or if we do not have a discrete domain, we can use the integral to approximate the sum. Thus, we use the right-hand side of

$$\sum_{c_1 \leq x \leq c_2} \hat{f}(x) \approx \int_{c_1}^{c_2} \hat{f}(x) dx$$

x_i	1	2	3	4	5	6	7	8	9	10
f_i	10	10	0	1	1	1	0	1	40	36
f_i^+	10	20	20	21	22	23	23	24	64	100
$5x$	5	10	15	20	25	30	35	40	45	50

Figure 24.5: Example frequency density and cumulated frequency

to approximate the sum by evaluating an expression which is hopefully less expensive to evaluate.

Yet another solution is the following. Instead of approximating (x_i, f_i) directly, we approximate (x_i, f_i^+) where $f_i^+ = \sum_{j \leq i} (f_j)$. Let us denote the approximation of this *cumulated frequency distribution* by $\hat{f}^+$. Then the result cardinality of a range query of the form $\sigma_{c_1 \leq A \leq c_2}(R)$ can be simply calculated by

$$\hat{f}^+(c_2) - \hat{f}^+(c_1).$$

However, this can be very dangerous since even if the approximation of the cumulated frequency function is rather precise, the difference can be vastly off the true value as we will see next.

An example for a frequency density is shown in Figure 24.5. Further define the cumulated frequency $f^+(c_1, c_2)$ as

$$f^+(c_1, c_2) := \sum_{c_1 \leq x_i \leq c_2} f_i.$$

Then, $f^+(c_1, c_2)$ gives the result for above query.

Define the cumulated frequency $f^+(c_2) := \sum_{x_i \leq c_2} f_i$. With the help of $f^+(c_2)$ we can calculate $f^+(c_1, c_2)$ by observing that $f^+(c_1, c_2) = f^+(c_2) - f^+(c_1 - 1)$. An idea often found in the literature is to approximate $f^+(c_2)$ by some function $\hat{f}^+(c_2)$ and provide an estimate for $f^+(c_1, c_2)$ by defining

$$\hat{f}^+(c_1, c_2) := \hat{f}^+(c_2) - \hat{f}^+(c_1 - 1).$$

For our example, we let us define $\hat{f}^+(x) = 5x$. Note that this is a linear approximation. Then, we see that $\hat{f}^+(x_i)$ is never more than a factor of 2 away from $f^+(x_i)$. Thus, it is a pretty good approximation. This is illustrated in Figure 24.6. However, we see that

$$\begin{aligned} \hat{f}^+(8, 10) &= 15 \\ f^+(8, 10) &= 77 \\ \hat{f}^+(4, 7) &= 20 \\ f^+(4, 7) &= 3 \end{aligned}$$

The estimates differ by far more than a factor of 2 from their true values. Thus, we have to look for a different solution.

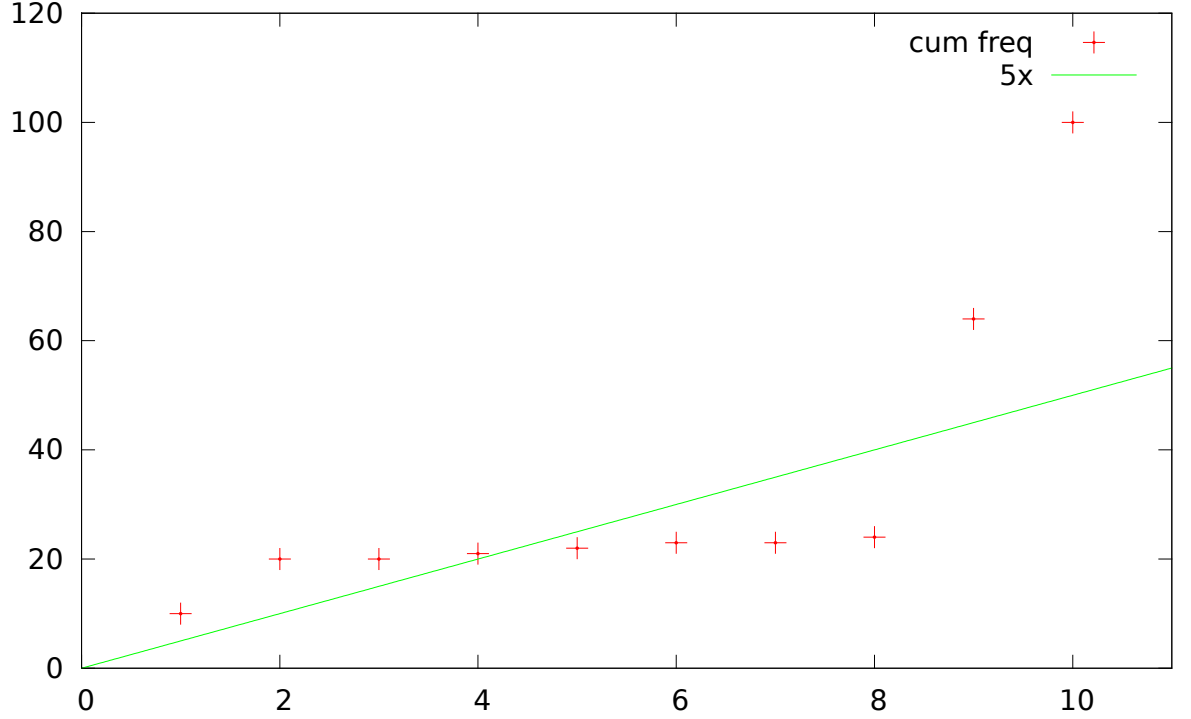


Figure 24.6: Cumulated frequency and its approximation

Assume for a change, that we are interested in half-open intervals. Thus, we would like to provide estimates for $f^-(c_1, c_2) := \sum_{c_1 \leq x_i < c_2} f_i$. A rather simple method is to directly approximate $f^-(c_1, c_2)$ by a linear function $\hat{f}^-(c_1, c_2) = ac_1 + bc_2 + c$, where we apply the two constraints that $\hat{f}^-(x, x) = 0$ and $\hat{f}^-(l_A, u_A) = f_A$. Remember that $l_A = \min(\Pi_A(R))$, $u_A = \max(\Pi_A(R))$, and $f_A = |R|$. With these constraints, $\hat{f}$ simplifies to $\hat{f}^-(c_1, c_2) = \frac{c_2 - c_1}{u_A - l_A}$, which should look familiar to the reader. No error bounds come with this approximation. Even worse, we do not know, whether it minimizes the q-errors or not. Thus, it seems to be better to control the q-error more directly.

Number of Distinct Values Let A be an attribute of some relation R . Consider the problem to provide estimates for

```
select count(distinct A) from R where c1 ≤ A ≤ c2
```

That is, we are interested in the number of distinct values of A in the interval $[c_1, c_2]$. As above, we can approximate the function

$$d^+(c_1, c_2) = |\{a | a \in \Pi_A(R), c_1 \leq a \leq c_2\}|$$

by some linear function.

Positions of interesting values

ELIM?

Consider an integer attribute A of a relation R with values in $[l, u]$. Imagine a

situation where almost all values in $[l, u]$ occur with some frequency in A . Only a few holes exist. Denote by W the set of values in $[l, u]$ that do not occur in A . Again, the selection is of the form $\sigma_{c_1 \leq A \leq c_2}(R)$. In the case of holes, the above summation for range queries leads to a wrong result. It would be better to calculate the result cardinality as in

$$\sum_{c_1 \leq x \leq c_2} \hat{f}(x) - \sum_{c_1 \leq x \leq c_2, x \in W} \hat{f}(x)$$

For this to work, we have to know where the holes in $[l, u]$ are. If there are only a few of them, we can memorize them. If they are too many to be stored, we can approximate them as follows. Let $W = \{w_1, \dots, w_m\}$. Then, we can use approximation techniques to approximate the set of points (i, w_i) , $1 \leq i \leq m$. Depending on the size of the interval $[l, u]$ either l_∞ or l_q is the appropriate norm. Similarly, the *peaks*, i.e., the distinct values occurring in the attribute A of R , can be approximated if there are only a few of them in the domain of A .

Why Q?

Minimizing error propagation. Let us assume that the purpose of our approximation is to estimate the output cardinalities of selections on relations R_i , i.e. $\sigma_{p_i}(R_i)$ for $i = 1, \dots, n$. The results of these cardinality estimations are then used to find the optimal order of subsequent joins. More specifically, assume we have to find the optimal query execution plan for the following expression:

$$\sigma_{p_1}(R_1) \bowtie \dots \bowtie \sigma_{p_n}(R_n), \quad (24.8)$$

where we intentionally left out all the join predicates. Ioanidis and Christodoulakis pointed out that errors propagate exponentially through joins [451]. Denote by s_i the cardinality of $\sigma_{p_i}(R_i)$ and by $\hat{s}_i$ its estimate. Further assume that independence holds. This means, that s_i can be written as $f_i |R_i|$, where f_i is the selectivity of p_i . Denote by $f_{i,j}$ the selectivity of the join predicate between R_i and R_j , if it exists. Otherwise we define $f_{i,j} = 1$. The result of joining a subset $x \subseteq \{R_1, \dots, R_n\}$ has cardinality

$$s_x = \left(\prod_{R_i \in x} f_i \right) \left(\prod_{R_i, R_j \in x} f_{i,j} \right) \left(\prod_{R_i \in x} |R_i| \right)$$

Denote by $\hat{f}_i$ the estimate for the selectivities of the p_i and assume that the join selectivities have been estimated correctly (which, of course, is difficult in practice). Then, the estimated cardinality of the result of joining the relations

in x is

$$\begin{aligned}
\hat{s}_x &= \left(\prod_{R_i \in x} \hat{f}_i \right) \left(\prod_{R_i, R_j \in x} f_{i,j} \right) \left(\prod_{R_i \in x} |R_i| \right) \\
&= \left(\prod_{R_i \in x} f_i / f_i \right) \left(\prod_{R_i \in x} \hat{f}_i \right) \left(\prod_{R_i, R_j \in x} f_{i,j} \right) \left(\prod_{R_i \in x} |R_i| \right) \\
&= \left(\prod_{R_i \in x} \hat{f}_i / f_i \right) \left(\prod_{R_i \in x} f_i \right) \left(\prod_{R_i, R_j \in x} f_{i,j} \right) \left(\prod_{R_i \in x} |R_i| \right) \\
&= \left(\prod_{R_i \in x} \hat{f}_i / f_i \right) s_x
\end{aligned}$$

where some i belong to the category with $\hat{f}_i / f_i < 1$ and others to the one with $\hat{f}_i / f_i > 1$. Remember that during dynamic programming, all subsets of relations are considered. Especially those subsets occur in which all relations belong to one category only. Hence, building on the cancellation of errors by mixing them from different categories is not a true option. Instead, we should minimize

$$\prod_{R_i \in x} \max\{f_i / \hat{f}_i, \hat{f}_i / f_i\}$$

in order to minimize errors and error propagation. This product can be minimized by minimizing each of its factors. This means that if we want to minimize error propagation, we have to minimize the multiplicative error $\mathcal{E}_q$ for estimating the cardinalities of selections based on equality. This finding can obviously be generalized to any kind of selections. Thus, for cardinality estimations for selections (and joins, or cardinality estimation in general) the q-error is the error metrics of choice.

Error bounds guaranteeing plan optimality. Let us give another strong argument for minimizing the multiplicative error $\mathcal{E}_q$. Let us consider again the join expression given in 24.8. Further, denote by f_i the correct selectivity of $\sigma_{A_i=c_i}$ and by $\hat{f}_i$ some estimate. If the plan generator uses the correct cardinalities, it produces the optimal plan. Given the estimates, it might produce another plan. The question is, how far can the cardinality estimates deviate from the true cardinalities such that the optimal plan still remains the same. More formally, denote by P the optimal plan under the correct cardinalities f and by $\hat{P}$ the optimal plan under the estimates $\hat{f}$. Then, we can restate the above question to whether there exists a condition on $\hat{f}$ such that if this condition holds then $\hat{P} = P$. The nice truth is that such conditions exist and they involve the Q paranorm.

In the simplest case, let us assume that the expression given in 24.8 is used to evaluate a star query under an ASI cost function without considering cross products. From Sec. 3.2.2, we can conclude that the optimal join order for star queries starts with the center relation and orders the satellite relations according to their rank. This holds if the a symmetric cost function is used like C_{out} . The rank of a relation R is defined as $\text{rank}(R_i) = (T(R_i) - 1) / C(R_i)$, where $C(S)$ are some fixed per tuple costs and $T(R_i) = f_{0,i} f_i |R|$, if $f_{0,i}$ is the

join selectivity of the join of R_i with the center relation R_0 . Thus, $\hat{P} = P$ if f and $\hat{f}$ result in the same ordering of the relations. Since $f(x) = (x - 1)/c$ is monotonically increasing for constants c , we can conclude that the ordering is indeed the same as long as for all $i \neq j$ we have

$$f_{0,i}f_i|R_i| < f_{0,j}f_j|R_j| \iff f_{0,i}\hat{f}_i|R_i| < f_{0,j}\hat{f}_j|R_j|$$

which is equivalent to

$$\frac{f_i r_i}{f_j r_j} < 1 \iff \frac{\hat{f}_i r_i}{\hat{f}_j r_j} < 1$$

for $r_i = f_{0,i}|R_i|$. We now show that if

$$\left\| \frac{f_i}{\hat{f}_i} \right\|_Q < \min_{i \neq j} \sqrt{\left\| \frac{f_i r_i}{f_j r_j} \right\|_Q} \quad (24.9)$$

for all i , then $P = \hat{P}$. This condition implies the much weaker condition that for all $i \neq j$

$$\left\| \frac{\hat{f}_i}{f_i} \right\|_Q \left\| \frac{\hat{f}_j}{f_j} \right\|_Q < \left\| \frac{f_i r_i}{f_j r_j} \right\|_Q \quad (24.10)$$

To show the claim, it suffices to show that $(f_i r_i)/(f_j r_j) < 1$ implies $(\hat{f}_i r_i)/(\hat{f}_j r_j) < 1$. This follows from

$$\begin{aligned} \frac{\hat{f}_i r_i}{\hat{f}_j r_j} &= \frac{\hat{f}_i f_j f_i r_i}{f_i \hat{f}_j f_j r_j} \\ &= \left(\frac{\hat{f}_i f_j}{f_i \hat{f}_j} \right) / \left(\left\| \frac{f_i r_i}{f_j r_j} \right\|_Q \right) \quad (*) \\ &\leq \left(\left\| \frac{\hat{f}_i}{f_i} \right\|_Q \left\| \frac{\hat{f}_j}{f_j} \right\|_Q \right) / \left(\left\| \frac{f_i r_i}{f_j r_j} \right\|_Q \right) \\ &< 1 \end{aligned}$$

where $(*)$ follows from $(f_i r_i)/(f_j r_j) < 1$. Thus, we have shown that if the q-error is limited as in condition 24.9, the produced plan is still optimal.

From cardinality estimation error bounds to cost error bounds. If there are cardinality estimation errors, the plan generator can accidentally produce the wrong plan. This plan may be suboptimal under the true cardinalities but is optimal under the estimated cardinalities. The question is, how bad is the plan? To clarify this, assume that the optimal plan under the true cardinalities is P . The optimal plan under the estimated cardinalities is $\hat{P}$. Then, we are interested in the factor by which the $\hat{P}$ is worse than P . The following theorem answers this question [622]:

Theorem 24.5.9 *Let $C = C_{SMJ}$ or $C = C_{GHJ}$ be the cost function of the sort-merge or the Grace hash join. For a given query in n relations, let P be the optimal plan under the true cardinalities, $\hat{P}$ be the optimal plan under the*

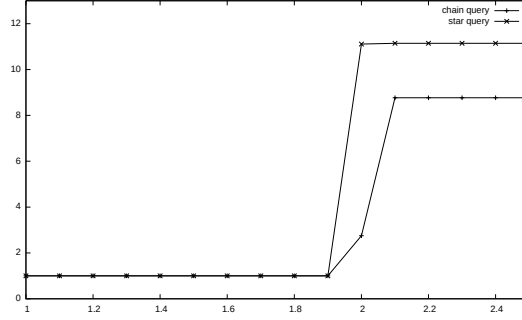


Figure 24.7: Q-error and plan optimality

estimated cardinalities, $C(P)$ be the true costs under C of the optimal plan, and $C(\hat{P})$ be the true costs under C of the plan produced under the estimated cardinalities. Then

$$C(\hat{P}) \leq q^4 C(P),$$

where q is defined as

$$q = \max_{x \subseteq X} \|\hat{s}_x / s_x\|_Q,$$

with X being the set of relations to be joined, and s_x ($\hat{s}_x$) is the true (estimated) size of the join of the relations in x . That is, q is the maximum estimation error taken over all intermediate results.

This bound is rather tight, as is demonstrated by the example shown in Fig. 24.7 (taken from [622]). This figure shows for a chain and a star query with four relations the quotient $\text{cost}(\hat{P})/\text{cost}(P)$ for increasing q-errors. For the star query, we see that this ratio is about 11.11, which is about $2^{3.46}$. Thus, a bound of the form $q^3 C(P)$ would fail.

24.5.3 Linear Models Under l_2

Now that we know that the solution to our problem exists and is unique, we continue by characterizing it. Let S be a linear space (say $\mathbb{R}^2$) and $s \in S$ some point. Further denote by G some linear subspace of S (say a straight line). For any $g \in G$, we can define the residual vector $g - f$. Using residuals, we can characterize the unique solution quite easily and intuitively. Exactly the vector $g^* \in G$ is closest to f whose residual $f - g$ is orthogonal to G . Remember that two vectors are orthogonal if and only if their scalar product is zero. Now we can characterize the solution to our approximation problem under l_2 .

Theorem 24.5.10 (Characterization) *Let S be a linear space and G a subspace. An element g^* is the best approximation of a point $s \in S$ if and only if*

$$\langle g^* - f, g \rangle = 0$$

holds. That is, if the error is orthogonal to all elements in G .

Since we are used to solve equations for x , we rewrite our problem to $A\vec{x} = b$. That is, the vector $\vec{x}$ replaces the coefficient vector c . Using Theorem 24.5.10, we must have that $A\vec{x}^* - b$ is orthogonal to the range of A . The range of a matrix $A \in \mathbb{R}^{m \times n}$ is defined as $\mathcal{R}(A) = \{Ax | x \in \mathbb{R}^n\}$. Let a_i be i -th column vector of A and $\vec{x} = (x_1, \dots, x_n)^T$. Then, the best approximation can be found by solving the following system of linear equations, which is called (*Gauß*) *normal equations*:

$$\begin{array}{ccccccccc} \langle a_1, a_1 \rangle x_1 & + & \langle a_2, a_1 \rangle x_2 & + & \dots & + & \langle a_n, a_1 \rangle x_n & = & \langle b, a_1 \rangle \\ \langle a_1, a_2 \rangle x_1 & + & \langle a_2, a_2 \rangle x_2 & + & \dots & + & \langle a_n, a_2 \rangle x_n & = & \langle b, a_2 \rangle \\ \vdots & & \vdots & & \vdots & & \vdots & & \vdots \\ \langle a_1, a_n \rangle x_1 & + & \langle a_2, a_n \rangle x_2 & + & \dots & + & \langle a_n, a_n \rangle x_n & = & \langle b, a_n \rangle \end{array}$$

or, using matrix notation, we get

$$A^T A \vec{x} = A^T \vec{b} \quad (24.11)$$

This system of linear equations can be solved by many different approaches. Some fast and numerically stable approaches are QR decomposition and singular value decomposition (SVD). Both leave the conditioning of the problem unchanged. QR decomposition can only be applied if the matrix has full rank (see below). Otherwise, one has to keep up with variants of QR decomposition or SVD. Hence, we will briefly discuss SVD. We will not give any algorithms. The interested reader is referred to [336].

Before we proceed with SVD, let us repeat some basics on matrices. A special matrix is the identity matrix $I \in \mathbb{R}^{n \times n}$ with $I = (\delta_{i,j})_{i,j}$, $1 \leq i \leq n$. Matrices can have plenty of properties. Here are some of them.

Definition 24.5.11 (rank) The rank of a matrix A , denoted by $\text{rank}(A)$, is the rank of the subspace $\mathcal{R}(A)$.

Definition 24.5.12 (full rank) A matrix $A \in \mathbb{R}^{m \times n}$, $m > n$ has full rank if its rank is n .

Definition 24.5.13 (symmetric) A matrix $A \in \mathbb{R}^n$ is symmetric if and only if $A^T = A$.

Note that for all matrices $A \in \mathbb{R}^{m \times n}$, we always have that AA^T and $A^T A$ are symmetric.

Definition 24.5.14 (idempotent) A matrix $A \in \mathbb{R}^{n \times n}$ is idempotent if and only if $AA = A$.

Definition 24.5.15 (inverse) A Matrix $A^{-1} \in \mathbb{R}^{n \times n}$ is the inverse of a matrix $A \in \mathbb{R}^{n \times n}$ if and only if $A^{-1}A = AA^{-1} = I$.

A matrix for which the uniquely determined inverse exists is called *regular*.

Definition 24.5.16 (orthogonal) A matrix $A \in \mathbb{R}^{n \times n}$ is orthogonal if and only if $AA^T = A^T A = I$.

Let us use a simple, operational, recursive definition of the determinant.

Definition 24.5.17 (determinant) Let $A \in \mathbb{R}^{n \times n}$ be a matrix. We define the determinant of A as $\det(A) = a_{1,1}$ if $n = 1$. Otherwise, we define

$$\det(A) = \sum_{j=1}^n (-1)^{i+j} a_{i,j} \det(A_{i,j})$$

where $A_{i,j} \in \mathbb{R}^{(n-1) \times (n-1)}$ results from A by eliminating the i -th row and the j -th column.

Definition 24.5.18 (characteristic polynomial) Let $A \in \mathbb{R}^{n \times n}$ be a matrix. The characteristic polynomial is defined as

$$P_n(z; A) := \det(A - zI) = \begin{vmatrix} (a_{1,1} - z) & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & (a_{2,2} - z) & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & (a_{n,n} - z) \end{vmatrix}$$

Definition 24.5.19 (Eigenvalue) Let $A \in \mathbb{R}^{n \times n}$ be a matrix and $P_n(z; A)$ its characteristic polynomial. Any root λ_i of $P_n(z; A)$, i.e. $P_n(z; A)(\lambda_i) = 0$ is called Eigenvalue. The set of Eigenvalues is denoted by

$$\lambda(A) := \{\lambda_1, \dots, \lambda_k\}$$

and is called spectrum of A .

Definition 24.5.20 (similar) Two matrices $A, B \in \mathbb{R}^{n \times n}$ are similar if and only if there exists a regular matrix $X \in \mathbb{R}^{n \times n}$ such that $B = X^{-1}AX$.

Two similar matrices have the same Eigenvalues, as can be seen from the following theorem.

Theorem 24.5.21 Let $A, B \in \mathbb{R}^{n \times n}$ be two similar matrices. Then they have the same characteristic polynomial.

Definition 24.5.22 (generalized inverse) A matrix $A^- \in \mathbb{R}^{n \times m}$ is the generalized inverse, or g-inverse, of a matrix $A \in \mathbb{R}^{m \times n}$, if $AA^-A = A$ holds.

Every matrix and, hence, every vector has a g-inverse. For regular matrices, the g-inverse and the inverse coincide. In general, the g-inverse is not uniquely determined. Adding some additional properties makes it unique.

Definition 24.5.23 (Moore-Penrose inverse) A matrix $A^+ \in \mathbb{R}^{n \times m}$ is the Moore-Penrose inverse of a matrix $A \in \mathbb{R}^{m \times n}$ if the following conditions hold:

1. $AA^+A = A$
2. $A^+AA^+ = A^+$

$$3. (A^+A)^T = A^+A$$

$$4. (AA^+)^T = AA^+$$

For every matrix and, hence, every vector there exists a uniquely determined Moore-Penrose inverse. In case A is regular, $A^+ = A^{-1}$ holds. If A is symmetric, then $A^+A = AA^+$. If A is symmetric and idempotent, then $A^+ = A$. Further, all of A^+A , AA^+ , $I - A^+A$, and $I - AA^+$ are idempotent. Here are some equalities holding for the Moore-Penrose inverse:

$$(A^+)^+ = A \quad (24.12)$$

$$(A^T)^+ = (A^+)^T \quad (24.13)$$

$$(A^T A)^+ = A^+ (A^T)^+ \quad (24.14)$$

$$(AA^T)^+ = (A^T)^+ A^+ \quad (24.15)$$

$$A^T AA^+ = A^T \quad (24.16)$$

$$A^+ AA^T = A^+ \quad (24.17)$$

The following theorem states the existence of a decomposition of any matrix into regular/orthogonal submatrices.

Theorem 24.5.24 (singular value decomposition) *Let $A \in \mathbb{R}^{m \times n}$ be a matrix. Then there exist an orthogonal matrix $U \in \mathbb{R}^{m \times m}$ and an orthogonal matrix $V \in \mathbb{R}^{n \times n}$ such that*

$$U^T AV = S$$

such that $S \in \mathbb{R}^{m \times n}$ is of the form

$$S = \text{diag}(s_1, \dots, s_k)$$

with $k = \min(m, n)$ and, further

$$s_1 \geq s_2 \geq \dots \geq s_r > s_{r+1} = \dots = s_k = 0$$

holds where $r = \text{rank}(A)$.

For a proof and algorithms to calculate the SVD of an arbitrary matrix see the book by Golub and Loan [336]. Another proof can be found in the book by Harville [408]. The diagonal elements s_i of S , which is orthogonal equivalent to A , are called *singular values*. From

$$S^T S = (U^T AV)^T (U^T AV) = V^T A^T U U^T AV = V^{-1} A^T AV$$

it follows that $S^T S$ and $A^T A$ are similar. Since $S^T S = \text{diag}(s_1^2, \dots, s_r^2, 0, \dots, 0)$ and similar matrices have the same spectrum, it follows that

$$s_i = \sqrt{\lambda_i}$$

for $\lambda_i \in \lambda(A^T A)$, $1 \leq i \leq n$.

Define $S^{-1} = \text{diag}(1/s_1, \dots, 1/s_r, 0, \dots, 0)$ and $A^+ = VS^{-1}U^T$. From

$$\begin{aligned} AA^+A &= (USV^T)(VS^{-1}U^T)(USV^T) \\ &= USS^{-1}SV^T \\ &= USV^T \\ &= A \end{aligned}$$

and

$$\begin{aligned} A^+AA^+ &= (VS^{-1}U^T)(USV^T)(VS^{-1}U^T) \\ &= VS^{-1}SS^{-1}U^T \\ &= VS^{-1}U^T \\ &= A^+ \end{aligned}$$

we see that $A^+ = VS^{-1}U^T$ is a g-inverse of A . The reader is advised to check the remaining conditions of the Moore-Penrose inverse.

Remember that we have to solve $A^T A \vec{x} = A^T \vec{b}$ for $\vec{x}$ in order to find the best approximation for our set of data points. Set $\vec{x} = A^+ \vec{b}$. Then

$$\begin{aligned} A^T A \vec{x} &= A^T A A^+ \vec{b} \\ &= A^T \vec{b} \end{aligned}$$

where we used Eqn. 24.16. Hence, the Moore-Penrose inverse² solves our problem. Moreover, the solution can be obtained easily from the singular value decomposition.

Approximation by a linear function

Assume we are given m points (x_i, y_i) , $1 \leq i \leq m$ and wish to approximate them by a linear function $f(x) = \alpha + \beta x$. The design matrix, b and $\vec{x}$ then are

$$A = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{pmatrix} \quad x = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}, \quad b = \begin{pmatrix} y_1 \\ \vdots \\ y_m \end{pmatrix}$$

The resulting system of normal equations

$$\begin{pmatrix} m & \sum_{i=1}^m x_i \\ \sum_{i=1}^m x_i & \sum_{i=1}^m (x_i)^2 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^m y_i \\ \sum_{i=1}^m x_i y_i \end{pmatrix}$$

has the solution

$$\alpha = \frac{\sum_{i=1}^m (x_i)^2 \sum_{i=1}^m y_i - \sum_{i=1}^m x_i y_i \sum_{i=1}^m x_i}{m \sum_{i=1}^m (x_i)^2 - (\sum_{i=1}^m x_i)^2}, \quad \beta = \frac{m \sum_{i=1}^m x_i y_i - \sum_{i=1}^m x_i \sum_{i=1}^m y_i}{m \sum_{i=1}^m (x_i)^2 - (\sum_{i=1}^m x_i)^2}$$

Note that this is a very nice formula as new points arrive or are deleted, only the sums have to be updated and the quotients to be calculated. There is no need to look at the other points again.

²The Greville algorithm to calculate the Moore-Penrose inverse directly is described in [777].

24.5.4 Linear Models Under l_∞

Let $A \in \mathbb{R}^{m \times n}$ be a matrix, where $m > n$, and $b \in \mathbb{R}^m$ a vector. The problem we solve in this section is to

$$\text{find } \vec{a} \in \mathbb{R}^n \text{ to minimize } \|r(\vec{a})\|_\infty \quad (24.18)$$

where

$$r(\vec{a}) = \vec{b} - A\vec{a} \quad (24.19)$$

The components of the vector $r(\vec{a})$ are denoted by $r_i(\vec{a})$.

As pointed out earlier, l_∞ is a convex norm. Hence, a solution exists. Since l_∞ is not strictly convex, the uniqueness of the solution is not guaranteed. To solve problem 24.18 by following the approach proposed by Watson [914]. We start by characterizing the solution, continue with the conditions under which uniqueness holds, make some more observations, and finally derive an algorithm for the case $n = 2$, i.e. we find a best approximation by a linear function. Although only few applications in databases exist for l_∞ , it is very useful to find a best approximation under l_q if we want to approximate by a function $e^{\beta + \alpha x}$ (see Sec. 24.5.6).

Assume we have a best solution $\vec{a}$. Then, for some indices i , $r_i(\vec{a})$ attains the maximum, i.e. $r_i(\vec{a}) = \|r(\vec{a})\|_\infty$. Otherwise, a better solution would exist. We denote the set of indices where the maximum is attained by $\bar{I}(\vec{a})$. We further denote by $\theta_i(\vec{a})$ the sign of $r_i(\vec{a})$. Thus $r_i(\vec{a}) = \theta_i(\vec{a})\|r(\vec{a})\|_\infty$ for all $i \in \bar{I}$. The following theorem gives a characterization of the solution.

Theorem 24.5.25 *A vector $\vec{a} \in \mathbb{R}^n$ solves problem 24.18 if and only if there exists a subset I of $\bar{I}$ with $|I| \leq n + 1$ and a vector $\vec{\lambda} \in \mathbb{R}^m$ such that*

1. $\lambda_i = 0$ for all $i \notin I$,
2. $\lambda_i \theta_i \geq 0$ for all $i \in I$, and
3. $A^T \vec{\lambda} = \vec{0}$.

The set I in the theorem is called an *extremal subset* of a solution $\vec{a}$.

There are two important corollaries to this theorem.

Corollary 24.5.26 *Let $\vec{a}$ solve problem 24.18. Then $\vec{a}$ solves an l_∞ approximation problem in $\mathbb{R}^{n+1}$ obtained by restricting the components of $r(\vec{a})$ to some particular $n + 1$ components. If A has rank t , then the components of $r(\vec{a})$ may be restricted to a particular $t + 1$ components.*

Corollary 24.5.27 *Let $\vec{a}$ solve problem 24.18 and let I be chosen according to Theorem 24.5.25 such that $\lambda_i \neq 0$ for all $i \in I$. Further let $\vec{d}$ be another solution to 24.18. Then*

$$r_i(\vec{d}) = r_i(\vec{a}).$$

Hence, not surprisingly, any two solutions have the same residuals for components where the maximum is attained. The theorem and its first corollary state that we need at most $t + 1$ solutions for a matrix A of rank t . The next theorem shows that at least $t + 1$ indices exist where the maximum is attained.

Theorem 24.5.28 *If A has rank t , a solution to problem 24.18 exists for which $|\bar{I}| \geq t + 1$.*

Thus, any submatrix of A consisting of a subset of the rows of A , which correspond to the indices contained in $\bar{I}(\bar{a})$, must have rank t for some solution $\bar{a}$ to problem 24.18.

The above theorems and corollaries indicate that the clue to uniqueness is the rank of subsets of rows of A . The following definition captures this intuition.

Definition 24.5.29 (Haar condition) *A matrix $R \in \mathbb{R}^{m \times n}$, where $m \geq n$ satisfies the Haar condition if and only if every submatrix consisting of n rows of A is nonsingular.*

Finally, we can derive uniqueness for those A which satisfy the Haar condition:

Theorem 24.5.30 *If A satisfies the Haar condition, the solution to problem 24.18 is unique.*

Obviously, we need to know, whether the Haar condition holds for a matrix A . Remember that we want to approximate a set of points by a linear combination of functions Φ_j , $1 \leq j \leq n$. From the points (x_i, y_i) , $1 \leq i \leq m$, and the Φ_j , the design matrix A is derived as shown in Equation 24.5. If the Φ_j form a Chebyshev set, the design matrix will fulfill the Haar condition.

Definition 24.5.31 (Chebyshev set) *Let X be a closed interval of $\mathbb{R}$. A set of continuous function $\Phi_1(x), \dots, \Phi_n(x)$, $\Phi_i : X \rightarrow \mathbb{R}$, is called a Chebyshev set, if every non-trivial linear combination of these functions has at most $n - 1$ zeros in X .*

Assume the x_i are ordered, that is $x_i < x_{i+1}$ for $1 \leq i < m$. Further, it is well-known that the set of polynomials $\Phi_j = x^{j-1}$, $1 \leq j \leq n$, forms a Chebyshev set on any interval X . From now on, we assume that our x_i are ordered, that is $x_1 < \dots < x_m$. Further, we define $X = [x_1, x_m]$. We also assume that the matrix A of Problem 24.18 is defined as given in Equation 24.5, where the Φ_j are continuous functions from X to $\mathbb{R}$.

We still need some more knowledge in order to build an algorithm. The next definition will help to derive a solution for subsets I of $\{1, \dots, m\}$ with $|I| = n + 1$.

Definition 24.5.32 (alternating set) *Let $\vec{a}$ be a vector in $\mathbb{R}^n$. We say that $r(\vec{a})$ alternates s times, if there exists points $x_{i_1}, \dots, x_{i_s} \in \{x_1, \dots, x_m\}$ such that*

$$r_{i_k}(\vec{a}) = -r_{i_{k+1}}(\vec{a})$$

for $1 \leq k < s$. The set $\{x_{i_1}, \dots, x_{i_s}\}$ is called an alternating set for $\vec{a}$.

Theorem 24.5.33 *Let (x_i, y_i) , $1 \leq i \leq m$, be an ordered set of points with $x_i < x_{i+1}$ for $1 \leq i < m$. Define $X = [x_1, x_m]$. Further let Φ_j , $1 \leq j \leq n$ be a Chebyshev set on X . Define $A = (a_{i,j})$, where $1 \leq i \leq m$, $1 \leq j \leq n$, and $a_{i,j} = \Phi_j(x_i)$. Then, a vector $\vec{a} \in \mathbb{R}^n$ solves Problem 24.18 if and only if there exists an alternating set with $n + 1$ points for $\vec{a}$.*

Consider again the example where we want to approximate the three points $(1, 20)$, $(2, 10)$, and $(3, 60)$ by a linear function. We saw that the solution to our problem is $\hat{f}_{l_\infty}(x) = -15 + 20x$. The following table gives the points, the value of $\hat{f}_{l_\infty}$, the residuals, including their signs.

x	y	$\hat{f}_{l_\infty}$	r_i
1	20	5	+15
2	10	25	-15
3	60	45	+15

As Theorem 24.5.33 predicts, the signs of the residuals alternate.

The proof of Lemma 24.5.33 uses the following lemma (see [914]).

Lemma 24.5.34 *Let (x_i, y_i) , $1 \leq i \leq m$, be an ordered set of points with $x_i \leq x_{i+1}$ for $1 \leq i < m$. Define $X = [x_1, x_m]$. Further let Φ_j , $1 \leq j \leq n$ be a Chebyshev set on X . Define the $n \times n$ determinant*

$$\Delta_i = \Delta(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_{n+1})$$

where

$$\Delta(x_1, \dots, x_n) = \det \begin{vmatrix} \Phi_1(x_1) & \dots & \Phi_n(x_1) \\ \vdots & & \vdots \\ \Phi_1(x_n) & \dots & \Phi_n(x_n) \end{vmatrix} \quad (24.20)$$

Then

$$\text{sign}(\Delta_i) = \text{sign}(\Delta_{i+1}), \quad \forall 1 \leq i \leq n.$$

Let us take a closer look at Theorem 24.5.33 in the special case where $m = 3$, i.e. we have exactly three points (x_{i_1}, y_{i_1}) , (x_{i_2}, y_{i_2}) , and (x_{i_3}, y_{i_3}) . We find the best linear approximation $\hat{f}(x) = \alpha + \beta x$ under l_∞ by solving the following equations:

$$\begin{aligned} y_{i_1} - (\alpha + \beta x_{i_1}) &= -1 * \lambda \\ y_{i_2} - (\alpha + \beta x_{i_2}) &= +1 * \lambda \\ y_{i_3} - (\alpha + \beta x_{i_3}) &= -1 * \lambda \end{aligned}$$

where λ represents the value of $\|r(\vec{a})\|_\infty$ for the solution $\vec{a}$ to be found. Solving these equations results in

$$\begin{aligned} \lambda &= \frac{y_{i_2} - y_{i_1}}{2} - \frac{(y_{i_3} - y_{i_1})(x_{i_2} - x_{i_1})}{2(x_{i_3} - x_{i_1})} \\ \beta &= \frac{y_{i_2} - y_{i_1}}{x_{i_2} - x_{i_1}} - \frac{2\lambda}{x_{i_2} - x_{i_1}} \\ \alpha &= y_{i_1} + \lambda - x_{i_1}\beta \end{aligned}$$

The algorithm to find the best approximation under l_∞ starts with three arbitrary points with indices i_1 , i_2 , and i_3 with $x_{i_1} < x_{i_2} < x_{i_3}$. Next, it derives α , β , and λ using the solutions to our equations 24.21-24.21. Then, the algorithm tries find new indices j_1, j_2, j_3 by exchanging one of the i_j with

some k such that λ will be increased. Obviously, we use a k that maximizes the deviation from the best approximation $\hat{f}$ of i_1, i_2, i_3 , i.e.

$$\|y_k - \hat{f}(x_k)\|_\infty = \max_{i=1,\dots,m} \|y_i - \hat{f}(x_i)\|_\infty.$$

Depending on the position of x_k in the sequence i_1, i_2, i_3 and the signs of the residuals we determine the i_j to be exchanged with k .

- $x_k < x_{i_1}$
if $(\text{sign}(y_k - \hat{f}_k) == \text{sign}(y_{i_1} - \hat{f}_{i_1}))$
then $j_1 = k, j_2 = i_2, j_3 = i_3$
else $j_1 = k, j_2 = i_1, j_3 = i_2$
- $x_{i_1} < x_k < x_{i_2}$
if $(\text{sign}(y_k - \hat{f}_k) == \text{sign}(y_{i_1} - \hat{f}_{i_1}))$
then $j_1 = k, j_2 = i_2, j_3 = i_3$
else $j_1 = i_1, j_2 = k, j_3 = i_2$
- $x_{i_2} < x_k < x_{i_3}$
if $(\text{sign}(y_k - \hat{f}_k) == \text{sign}(y_{i_2} - \hat{f}_{i_2}))$
then $j_1 = i_1, j_2 = k, j_3 = i_2$
else $j_1 = i_1, j_2 = i_2, j_3 = k$
- $x_k > x_{i_3}$
if $(\text{sign}(y_k - \hat{f}_k) == \text{sign}(y_{i_3} - \hat{f}_{i_3}))$
then $j_1 = i_1, j_2 = i_2, j_3 = k$
else $j_1 = i_2, j_2 = i_3, j_3 = k$

The above rules are called *exchange rules*. In general, they state that if k falls between two indices, the one with the same sign as r_k is replaced by k . If k is smaller than the smallest index (larger than the largest index), we consider two cases. If the smaller (largest) index has the same sign of its residue as k , we exchange it with k ; otherwise we exchange it with the largest (smallest) index. Stated this way, we can use the exchange rules for cases where $n > 2$.

Algorithm 24.8 summarizes the above considerations.

In case $n > 2$, the above algorithm remains applicable. We just have to use the general exchange rule and provide a routine solving the following system of equations for x_i and λ :

$$\begin{aligned} a_{1,1}x_1 + a_{1,2}x_2 + a_{1,n}x_n &= -\lambda \\ a_{2,1}x_1 + a_{2,2}x_2 + a_{2,n}x_n &= +\lambda \\ &\dots \quad \dots \\ a_{n+1,1}x_1 + a_{n+1,2}x_2 + a_{n+1,n}x_n &= (-1)^{n+1}\lambda \end{aligned}$$

24.5.5 Linear Models Under l_q

Let (x_i, y_i) for $1 \leq i \leq m$ be a set of points with $y_i > 0$, which we again want to approximate by a linear combination of a given set of functions Φ_j , $1 \leq j \leq n$.

EXC

BestLinearApproximationUnderChebyshevNorm

1. Choose arbitrary i_1, i_2, i_3 with $x_{i_1} < x_{i_2} < x_{i_3}$.
(e.g. equi-distant i_j .)
2. Calculate the solution for the system of equations.
This gives us an approximation function $\hat{f}(x) = \alpha + \beta x$ and λ .
3. Find an x_k for which the deviation of $\hat{f}$ from the given data is maximized.
Call this maximal deviation $\lambda_{\max}$.
4. If $\lambda_{\max} - \lambda > \epsilon$ for some small ϵ
then apply the exchange rule using x_k and go to step 2.
(The ϵ is mainly needed for rounding problems with floating point numbers.)
5. Return α, β, λ .

Figure 24.8: Algorithm for best linear approximation under l_∞

This time, we measure the deviation by applying l_q . That is, we want to find a coefficients a_j such that the function

$$\hat{f}(x) = \sum_{j=1}^n a_j \Phi_j(x)$$

minimizes

$$\max_{i=1, \dots, m} \max \left\{ \frac{y_i}{\hat{f}(x_i)}, \frac{\hat{f}(x_i)}{y_i} \right\}.$$

Let $\vec{a}$ and $\vec{b}$ be two vectors in R^n with $b_i > 0$. Then, we define $\vec{a}/\vec{b} = \frac{\vec{a}}{\vec{b}} = (a_1/b_1, \dots, a_n/b_n)^t T$.

Let $A \in \mathbb{R}^{m \times n}$ be a matrix, where $m > n$ and $\vec{b} = (b_1, \dots, b_m)^t T$ be a vector in $\mathbb{R}^m$ with $b_i > 0$. Then we can state the problem as

$$\text{find } \vec{a} \in \mathbb{R}^n \text{ that minimizes } \|A\vec{a}/\vec{b}\|_Q \quad (24.21)$$

under the constraint that $\alpha_i^t T > 0$, $1 \leq i \leq m$, for all row vectors α_i of A .

Alternatively, we can modify A by “dividing” it by $\vec{b}$. We need some notations to do so. Let $\vec{b} = (b_1, \dots, b_m)^t T$ be a vector in $\mathbb{R}^m$. Define $\text{diag}(\vec{b})$ to be the $m \times m$ diagonal matrix which contains the b_i in its diagonal and is zero outside the diagonal. For vectors $\vec{b}$ with $b_i > 0$, we can define $\vec{b}^{-1} = (1/b_1, \dots, 1/b_m)^t T$.

Using these notations, we can define

$$A' = \text{diag}(\vec{b}^{-1})A$$

In the special case of univariate polynomial approximation with $\hat{f}(x) = a_1 +$

$a_2x + \dots + a_nx^{n-1}$ the matrix A' has the form

$$A' = \begin{pmatrix} 1/y_1 & x_1/y_1 & \dots & x_1^{n-1}/y_1 \\ 1/y_2 & x_2/y_2 & \dots & x_2^{n-1}/y_2 \\ \vdots & \vdots & \dots & \vdots \\ 1/y_m & x_m/y_m & \dots & x_m^{n-1}/y_m \end{pmatrix}. \quad (24.22)$$

Keeping the trick with A' in mind, it is easy to see that Problem 24.21 can be solved, if we can solve the general problem

$$\text{find } \vec{a} \in \mathbb{R}^n \text{ that minimizes } \|A\vec{a}\|_Q. \quad (24.23)$$

The following proposition ensures that a solution to this general problem exists. Further, since $\|A\vec{a}\|_Q$ is convex, the minimum is a global one.

Proposition 24.5.1 *Let $A \in \mathbb{R}^{m,n}$ such that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^m \neq \emptyset$. Then $\|A \cdot\|_Q$ attains its minimum.*

Recall that l_q is subadditive and convex. Further it is lower semi-continuous (see also [732, p. 52]). However, it is not strictly convex. Hence, as with l_∞ , we expect uniqueness to hold only under certain conditions.

We need some more notation. Let $A \in \mathbb{R}^{m,n}$. We denote by $\mathcal{R}(A) = \{A\vec{a} \mid \vec{a} \in \mathbb{R}^n\}$ the *range* of A and by $\mathcal{N}(A) = \{\vec{a} \in \mathbb{R}^n \mid A\vec{a} = 0\}$ the *nullspace* of A .

Problem (24.23) can be rewritten as the following constrained minimization problem:

$$\min_{(\vec{a}, q) \in \mathbb{R}^n \times \mathbb{R}} q \quad \text{subject to} \quad \frac{1}{q} \leq A\vec{a} \leq q \quad \text{and} \quad q \geq 1. \quad (24.24)$$

The Lagrangian of (24.24) is given by

$$L(\vec{a}, q, \lambda^+, \lambda^-, \mu) := q - (\lambda^+)^T(q - A\vec{a}) - (\lambda^-)^T(A\vec{a} - \frac{1}{q}) - \mu(q - 1).$$

Assume that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^m \neq \emptyset$. Then the set $\{(\vec{a}, q) : \frac{1}{q} \leq A\vec{a} \leq q \text{ and } q \geq 1\}$ is non-empty and closed and there exists $(\vec{a}, q)$ for which we have strong inequality in all conditions. Then the following *Karush-Kuhn-Tucker* conditions are necessary and sufficient for $(\hat{\vec{a}}, \hat{q})$ to be a minimizer of (24.24), see, e.g., [825, p. 62]: there exist $\hat{\lambda}^+, \hat{\lambda}^- \in \mathbb{R}_{\geq 0}^m$ and $\hat{\mu} \geq 0$ such that

$$\nabla_{\vec{a}} L(\hat{\vec{a}}, \hat{q}, \hat{\lambda}^+, \hat{\lambda}^-, \hat{\mu}) = A^T \lambda^+ - A^T \lambda^- = 0 \quad (24.25)$$

$$\frac{\partial}{\partial q} L(\hat{\vec{a}}, \hat{q}, \hat{\lambda}^+, \hat{\lambda}^-, \hat{\mu}) = 1 - \sum_{i=1}^m \hat{\lambda}_i^+ - \frac{1}{\hat{q}^2} \sum_{i=1}^m \hat{\lambda}_i^- - \mu = 0 \quad (24.26)$$

and for $i = 1, \dots, m$,

$$\hat{\lambda}_i^+ \left(\hat{a} - (\hat{A}\hat{a})_i \right) = 0, \quad (24.27)$$

$$\hat{\lambda}_i^- \left((\hat{A}\hat{a})_i - \frac{1}{\hat{q}} \right) = 0, \quad (24.28)$$

$$\hat{\mu}(\hat{q} - 1) = 0.$$

Assume that $1_m \notin \mathcal{R}(A)$, where 1_m is the vector with all components 1. Then $\hat{q} > 1$ and consequently $\hat{\mu} = 0$. Furthermore, it is clear that not both $\hat{\lambda}_i^+$ and $\hat{\lambda}_i^-$ can be positive because the conditions $\hat{q} = (A\hat{a})_i$ and $\frac{1}{\hat{q}} = (A\hat{a})_i$ cannot be fulfilled at the same time, since $\hat{q} > 1$.

Setting $\hat{\lambda} := \hat{\lambda}^+ - \hat{\lambda}^-$, we can summarize our findings (24.25) - (24.28) in the following theorem.

Theorem 24.5.35 *Let $A \in \mathbb{R}^{m,n}$ such that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^m \neq \emptyset$ and $1_m \notin \mathcal{R}(A)$. Then $(\hat{a}, \hat{q})$ solves (24.24) if and only if there exists $\hat{\lambda} \in \mathbb{R}^m$ such that*

i) $A^T \hat{\lambda} = 0$.

ii) $q = q \sum_{\hat{\lambda}_i > 0} \hat{\lambda}_i + \frac{1}{q} \sum_{\hat{\lambda}_i < 0} \hat{\lambda}_i$.

iii) $\hat{\lambda}_i = 0$ if $\frac{1}{\hat{q}} < (A\hat{a})_i < q$.

iv) if $\hat{\lambda}_i > 0$ then $(A\hat{a})_i = \hat{q}$ and if $\hat{\lambda}_i < 0$ then $(A\hat{a})_i = 1/\hat{q}$.

Remark. We see that $1 < \hat{q} = (A\hat{a})_i$ implies $\text{sign}((A\hat{a})_i - 1) = 1$ and that $1 > 1/\hat{q} = (A\hat{a})_i$ implies $\text{sign}((A\hat{a})_i - 1) = -1$; whence $\hat{\lambda}_i ((A\hat{a})_i - 1) \geq 0$. For our approximation problem (24.21) this means that the residuum $\hat{f}(x_i) - b_i$ fulfills $\hat{\lambda}_i (\hat{f}(x_i) - b_i) \geq 0$.

Under certain conditions, problem (24.23) has a unique solution which can be simply characterized. Let us start with some straightforward considerations in this direction. If $\mathcal{N}(A) \neq \{\vec{0}\}$, then we have for any minimizer $\hat{a}$ of $\|A \cdot\|_Q$ that $\hat{a} + \beta$, $\beta \in \mathcal{N}(A)$ is also a minimizer. In particular, we have that $\mathcal{N}(A) \neq \{\vec{0}\}$ if

- $m < n$,
- $m \geq n$ and A is not of full range, i.e., $\text{rank}(A) < n$.

In these cases, we cannot have a unique minimizer. Note further, that if $1_m \in \mathcal{R}(A)$, then the minimum of $\|A \cdot\|_Q$ is 1 and the set of minimizers is given by

$$A^+ 1_m + \mathcal{N}(A),$$

where A^+ denotes the Moore-Penrose inverse of A . Of course, this can easily be checked using the methods of Sec. 24.5.3.

In the following, we restrict our attention to the case $m > n$ and $\text{rank}(A) = n$.

The following proposition considers $(n+1, n)$ -matrices.

Proposition 24.5.2 *Let $A \in \mathbb{R}^{n+1,n}$ such that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^{n+1} \neq \emptyset$, $1_m \notin \mathcal{R}(A)$ and $\text{rank}(A) = n$. Then $\|A \cdot\|_Q$ has a unique minimizer if and only if the Lagrange multipliers $\hat{\lambda}_i$, $i = 1, \dots, n+1$ are not zero.*

By $\text{spark}(A)$ we denote the smallest number of rows of A which are linearly dependent. In other words, any $\text{spark}(A) - 1$ rows of A are linearly independent. For the 'spark' notation we also refer to [27].

Examples. 1. We obtain for the matrix

$$A := \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}, \quad \text{rg}(A) = 3, \quad \text{spark}(A) = 3.$$

2. The matrix (m, n) -matrix A in (24.22) is the product of the diagonal matrix $\text{diag}(1/b_i)_{i=1}^m$ with positive diagonal entries and a Vandermonde matrix. Hence, it can easily be seen that $\text{spark}(A) = n + 1$. If an (m, n) -matrix A has $\text{spark}(A) = n + 1$, then A fulfills the Haar condition.

Proposition 24.5.2 can be reformulated as follows:

Corollary 24.5.36 *Let $A \in \mathbb{R}^{n+1, n}$ such that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^{n+1} \neq \emptyset$ and $1_m \notin \mathcal{R}(A)$. Then $\|A \cdot\|_Q$ has a unique minimizer if and only if $\text{spark}(A) = n + 1$.*

The result can be generalized by the following theorem.

Theorem 24.5.37 *Let $A \in \mathbb{R}^{m, n}$ such that $\mathcal{R}(A) \cap \mathbb{R}_{>0}^m \neq \emptyset$. Suppose that $\text{spark}(A) = n + 1$. Then $\|A \cdot\|_Q$ has a unique minimizer which is determined by $n + 1$ rows of A , i.e., there exists an index set $J \subset \{1, \dots, m\}$ of cardinality $|J| = n + 1$ such that $\|A \cdot\|_Q$ and $\|A|_J \cdot\|_Q$ have the same minimum and the same minimizer. Here $A|_J$ denotes the restriction of A to the rows which are contained in the index set J . We call such index set J an extremal set.*

Of course the condition $\text{spark}(A) = n + 1$ is not necessary for $\|A \cdot\|_Q$ to have a unique minimizer as the following example shows.

Example. The matrices

$$A := \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & \frac{1}{2} \\ -4 & 2 \end{pmatrix}, \quad \text{and} \quad A := \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -4 & 4 \\ -1 & 1 \end{pmatrix}$$

have both $\text{spark}(A) = 2$. By some following considerations, we obtain for both problems that the minimum of $\|A \cdot\|_Q$ is $\hat{q} = 2$. However, in the first problem the minimizer is uniquely determined by $\hat{a} = (\frac{1}{2}, 2)^T$ while the whole line $c(\frac{1}{2}, 1)^T + (1 - c)(\frac{3}{2}, 2)^T$, $c \in [0, 1]$ minimizes the functional in the second case. For $(\frac{1}{2}, 1)^T$ we have $\text{sign}(\hat{\lambda}_1, \hat{\lambda}_2, \hat{\lambda}_3, \hat{\lambda}_4) = (-1, 0, 1, -1)$ while the pattern is $(0, 1, 1, -1)$ for $(\frac{3}{2}, 2)^T$ and $(0, 0, 1, -1)$ within the line bounded by these points.

By Theorem 24.5.37, a method for finding the minimizer of $\|A \cdot\|_Q$ would be to compute the unique minimizers of the $\binom{m}{n+1}$ subproblems $\|A|_J \cdot\|_Q$ for all index sets J of cardinality $n + 1$ and to take the largest minimum $\hat{a}$ and the corresponding $\hat{a}$ as minimizer of the original problem. For our line problem there exist $\binom{m}{3} = \mathcal{O}(m^3)$ of these subproblems. In the following section, we give another algorithm which is also based on Theorem 24.5.37, but ensures

that the value a enlarges for each new choice of the subset J . Since there is only a finite number of such subsets we must reach a stage where no further increase is possible and J is an extremal set. In normed spaces such methods are known as *ascent methods*, see [914].

In this section, we suggest a detailed algorithm for minimizing $\|A \cdot\|_Q$, where we restrict our attention to the line problem

$$\max_{i=1,\dots,m} \max \left\{ \frac{b_i}{\beta + \alpha x_i}, \frac{\beta + \alpha x_i}{b_i} \right\}. \quad (24.29)$$

i.e., to the matrix A in (24.22) with $n = 2$.

Corollary 24.5.38 *Let (x_i, b_i) , $i = 1, 2, 3$ be given points with pairwise distinct $x_i \in \mathbb{R}$ and positive b_i , $i = 1, 2, 3$. Then the minimum $\hat{q}$ and the minimizer $\hat{a} \in \mathbb{R}^2$ of (24.29) are given by $\hat{q} = \|\hat{q}_1\|_Q$ and*

$$\begin{pmatrix} \hat{\beta} \\ \hat{\alpha} \end{pmatrix} = \frac{1}{x_2 - x_1} \begin{pmatrix} x_2 & -x_1 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} b_1 \hat{q}_1 \\ b_2 \hat{q}_2 \end{pmatrix},$$

where

$$\hat{q}_1 := \begin{cases} \sqrt{\frac{r_2}{1-r_1}} & \text{if } r_1 < 0 \text{ and } r_2 > 0, \\ \sqrt{\frac{1-r_2}{r_1}} & \text{if } r_1 > 0 \text{ and } r_2 < 0, \\ \sqrt{\frac{1}{r_1+r_2}} & \text{if } r_1 > 0 \text{ and } r_2 > 0, \end{cases} \quad (24.30)$$

$$\hat{q}_2 := \begin{cases} 1/\hat{q}_1 & \text{if } \frac{r_1}{r_2} < 0, \\ \hat{x}_1 & \text{if } \frac{r_1}{r_2} > 0 \end{cases}$$

and

$$r_1 := \frac{b_1(x_2 - x_3)}{b_3(x_2 - x_1)}, \quad r_2 := \frac{b_2(x_3 - x_1)}{b_3(x_2 - x_1)}.$$

Remark. If the points are ordered, i.e., $x_1 < x_2 < x_3$ (or alternatively in descending order), then either $A\hat{a} = (\hat{q}, 1/\hat{q}, \hat{q})^T$ or $A\hat{a} = (1/\hat{q}, \hat{q}, 1/\hat{q})^T$. This means that $\hat{\lambda}$ in Theorem 24.5.35 has alternating signs. In other words, the points $f(x_1)$, $f(x_3)$ lie above b_1 , b_3 and $f(x_2)$ lies below b_2 or conversely.

Later we will show that the alternating sign condition is true for general best polynomial approximation with respect to the Q -paranorm.

Corollary 24.5.38 is the basis of the Algorithm 24.9, which finds the optimal line with respect to three points in each step and chooses the next three points if the minimum corresponding to their line becomes larger.

Proposition 24.5.3 *The algorithm computes the line $f(x) = \hat{\beta} + \hat{\alpha}x$ which minimizes (24.29).*

Remark. Alternatively, one can deal with ordered points $b_1 < b_2 < b_3$ which restricts the effort in (24.30) to $\hat{q}_1 = \frac{r_2}{1-r_1}$ but requires an ascending ordering of the points x_{i_1}, x_{i_2}, x_j in each step of the algorithm.

Finally, we want to generalize the remark on the signs of the Lagrange multipliers given after Corollary 24.5.38. Remember that the set of polynomials $\Phi_i(x) = x^{i-1}$, $i = 1, \dots, n$ forms a Chebyshev set (see Def. 24.5.31). Applying again Lemma 24.5.34, one can easily prove the following result.

Algorithm. (Best line approximation with respect to l_q)

Input: (x_i, b_i) , $i = 1, \dots, m$ of pairwise distinct points $x_i \in \mathbb{R}$ and $b_i > 0$

Set $i_1 := 1$, $i_2 := 2$ and $stop\text{signal} := -1$.

While $stop\text{signal} = -1$ do

1. For $i = 1, \dots, m$; $i \neq i_1, i_2$ compute

$$r_{1,i} := \frac{b_{i_1}(x_{i_2} - x_i)}{b_i(x_{i_2} - x_{i_1})}, \quad r_{2,i} := \frac{b_{i_2}(x_i - x_{i_1})}{b_i(x_{i_2} - x_{i_1})}.$$

2. Compute $\hat{a}_j = \max_i \{|\hat{x}_1(r_{1,i}, r_{2,i})|_Q\}$ by (24.30). Let $j \neq i_1, i_2$ be an index where the maximum is attained and $\hat{x}_1 = \hat{x}_1(r_{1,j}, r_{2,j})$.

3. Compute $a := \max_i \{|r_{1,i}\hat{x}_1 + r_{2,i}\hat{x}_2|_Q\}$.
Let k be an index, where the maximum is attained.

4. If $a \leq \hat{a}_j$ then $stop\text{signal} = 1$ and $\hat{a} = \hat{a}_j$,

$$\begin{pmatrix} \hat{\beta} \\ \hat{\alpha} \end{pmatrix} = \frac{1}{x_{i_2} - x_{i_1}} \begin{pmatrix} x_{i_2} & -x_{i_1} \\ -1 & 1 \end{pmatrix} \begin{pmatrix} b_{i_1} \hat{q}_1 \\ b_{i_2} / \hat{q}_1 \end{pmatrix},$$

otherwise set $i_1 := j$ and $i_2 := k$ and return to 1.

Figure 24.9: Algorithm finding best linear approximation under l_q .

Theorem 24.5.39 Let $\Phi_i : I \rightarrow \mathbb{R}$, $i = 1, \dots, n$ be a Chebyshev set and let $x_1 < \dots < x_{n+1}$ be points in I . Then, for

$$\Phi := (\Phi_j(x_i))_{i,j=1}^{n+1,n},$$

the Lagrange multipliers $\hat{\lambda}_i$, $i = 1, \dots, n+1$ corresponding to the minimizer of $\|\Phi \cdot\|_Q$ have alternating signs.

For our polynomial approximation problem $\arg\min_{\vec{a} \in \mathbb{R}^n} \|A\vec{a}\|_Q$ with $A \in \mathbb{R}^{n+1,n}$ defined by (24.22) and ordered points $x_1 < \dots < x_{n+1}$ we see that $A = \text{diag}(1/b_i)_{i=1}^{n+1} \Phi$, where Φ is the matrix belonging to the Chebyshev set $\Phi_i(x) = x^{i-1}$. Since the b_i are positive, we obtain immediately that the Lagrange multipliers $\hat{\lambda}_i$ have alternating signs. Again, this means that $\hat{f}(x_i) - b_i$ has alternating signs.

Using Theorem 24.5.39, we not only simplify Algorithm 24.9 but also use Algorithm 24.8 even for approximation by arbitrary Chebyshev sets of size $n > 2$. We only have to provide a routine which solves the following system of equations:

$$\begin{aligned}
a_{1,1}x_1 + a_{1,2}x_2 + a_{1,n}x_n &= \lambda^{+1} \\
a_{2,1}x_1 + a_{2,2}x_2 + a_{2,n}x_n &= \lambda^{-1} \\
&\dots \quad \dots \\
a_{n+1,1}x_1 + a_{n+1,2}x_2 + a_{n+1,n}x_n &= \lambda^{(-1)^n}
\end{aligned}$$

Let us illustrate this for $n = 2$. In this case, we can write

$$\begin{aligned}
\frac{1}{\lambda}(\alpha + \beta x_1) &= y_1 \\
\lambda(\alpha + \beta x_2) &= y_2 \\
\frac{1}{\lambda}(\alpha + \beta x_3) &= y_3
\end{aligned}$$

If we number the above equations from 1 to 3, then we may conclude that

$$\begin{aligned}
3 &\Rightarrow & \alpha &= \lambda y_3 - \beta x_3 & (*) \\
1, (*) &\Rightarrow & \lambda y_3 - \beta x_3 + \beta x_1 &= \lambda y_1 \\
&\Rightarrow & (y_3 - y_1)\lambda &= (x_3 - x_1)\beta \\
&\Rightarrow & \lambda &= \frac{x_3 - x_1}{y_3 - y_1} \beta & (**) \\
&\Rightarrow & \lambda &= q_{13} \beta & (**) \\
2, (*), (**) &\Rightarrow & q_{13} \beta (q_{13} y_3 - \beta x_3 + \beta x_2) &= y_2 \\
&\Rightarrow & \beta^2 (q_{13} y_3 - x_3 + x_2) &= y_2 q_{13}^{-1} \\
&\Rightarrow & \beta &= \sqrt{g^{-1} y_2 q_{13}^{-1}}
\end{aligned}$$

where

$$\begin{aligned}
q_{13} &:= \frac{x_3 - x_1}{y_3 - y_1} \\
g &:= q_{13} y_3 - x_3 + x_2
\end{aligned}$$

Caution is necessary, if $\beta = 0$. Then:

$$\begin{aligned}
\beta &= 0 \\
\alpha &= \lambda y_1 \\
\lambda &= \sqrt{y_2 / y_1}
\end{aligned}$$

24.5.6 Non-Linear Models under l_q

In general there is a lot to say about this subject and we refer the reader to the literature. However, let us consider two simple problems, which we can solve using algorithms we already know:

1. Find the best approximation using $e^{p(x)}$ under l_q , and
2. find the best approximation using $\ln(p(x))$ under l_∞ ,

where $p(x)$ is a linear combination of a set of Chebyshev functions.

Let us start with the first problem. That is, we ask for an exponential function

$$\hat{f} = e^{\sum_{j=1}^n \alpha_j \Phi_j}$$

which best approximates under l_q a given set of points (x_i, y_i) , $i = 1, \dots, m$ with pairwise distinct $x_i \in \mathbb{R}^d$ and $y_i > 0$, $1 \leq i \leq m$. Note that $\hat{f} > 0$ by definition. Since the $\ln$ function increases strictly monotone this is equivalent to minimizing

$$\begin{aligned} \ln \left(\max_{i=1, \dots, m} \max \left\{ \frac{y_i}{\hat{f}(x_i)}, \frac{\hat{f}(x_i)}{y_i} \right\} \right) &= \max_{i=1, \dots, m} \max \{ \ln y_i - \ln \hat{f}(x_i), \ln \hat{f}(x_i) - \ln y_i \} \\ &= \max_{i=1, \dots, m} \left| \ln y_i - \sum_{j=1}^n \alpha_j \Phi_j(x_i) \right| \\ &= \|(\ln y_i)_{i=1}^m - \Phi \alpha\|_{\infty}. \end{aligned}$$

Thus, it remains to find the best function $\sum_{j=1}^n \alpha_j \Phi_j(x_i)$ with respect to the l_{∞} norm.

EXC It is now easy to see that we can solve the second problem as follows. Let (x_i, y_i) be the data we want to approximate by a function of the form $\ln(p(x))$ while minimizing the Chebyshev norm. We can do so by finding the best approximation of (x_i, e^{y_i}) under l_q .

24.5.7 Multidimensional Models under l_q

In this section, we show that it is possible to find the best approximation under l_q in the multidimensional setting. The idea is to reduce the problem to *second order cone programming* (SOCP).

In general, SOCP can be used to solve problems of the form

$$\min_{x \in \mathbb{R}^s} \langle c, x \rangle \text{ subject to } Mx + b \in K$$

where $c \in \mathbb{R}^s$, $b \in \mathbb{R}^t$, $M \in \mathbb{R}^{t,s}$, and K is a convex cone in $\mathbb{R}^t$. For details on SOCP, we refer to [564]. For us, it is important that software packages like MOSEK solve SOCP problems quite efficiently.

We now show how our problem can be reduced to SOCP. Thereby, we follow the approach of Setzer et al. [792]. Assume our set of d -dimensional points is $X = \{x_1, \dots, x_m\} \subset \mathbb{R}^d$. For each point x_i , we have a measurement (e.g., its frequency) $f_i > 0$. Further, we want to find a linear model in functions Φ_j , $1 \leq j \leq n$. Then, the model can be represented by a matrix $A := (\Phi_j / f_i)_{i,j=1}^{m,n}$. We assume that $n < m$ and that A has full rank. The problem to find a best approximation under l_q can then be formulated as

$$\hat{\alpha} = \operatorname{argmin}_{\alpha \in \mathbb{R}^n} \|A\alpha\|_Q \quad (24.31)$$

This is equivalent to the constraint problem

$$\min_{u \in \mathbb{R}^m, \alpha \in \mathbb{R}^n} \|u\|_Q \text{ subject to } A\alpha = u$$

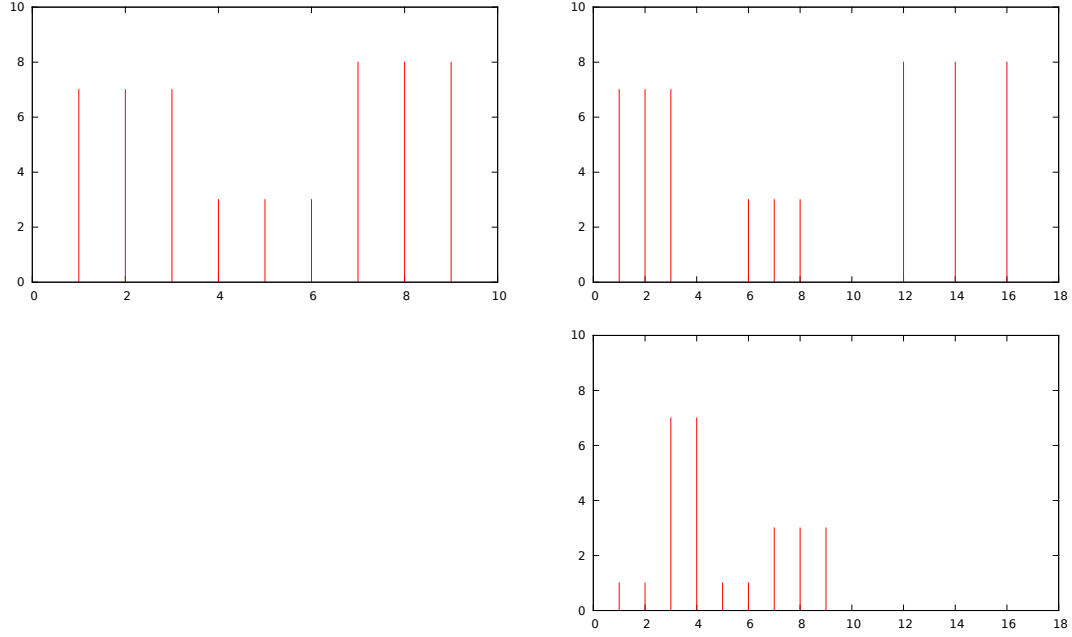


Figure 24.10: Sample data sets

which in turn can be rewritten as

$$\min_{a \in \mathbb{R}, u \in \mathbb{R}^m, \alpha \in \mathbb{R}^n} a \text{ subject to } A\alpha = u, 1 \leq a, 1/a \leq u \leq a \quad (24.32)$$

The first two constraints and $u \leq a$ are already cone constraints. The remaining constraints $1 \leq au_i$ can be rewritten to

$$\begin{pmatrix} \sqrt{2} & 0 \\ 0 & \sqrt{2} \\ 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} u_i \\ a \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ \sqrt{2} \\ -\sqrt{2} \end{pmatrix} \in L_r^4$$

because the following inequalities are equivalent:

$$\begin{aligned} (\sqrt{2}u_i)^2 + (\sqrt{2}a)^2 &\leq 2(u_i + a + \sqrt{2})(u_i + a - \sqrt{2}) \\ u_i^2 + a^2 &\leq (u_i + a)^2 - 2 \\ 1 &\leq u_i a \end{aligned}$$

An alternative formulation of the last constraint can be found in [792].

24.6 Traditional Histograms

In the simple profile of Section 24.3, we approximated the frequency density of the whole domain by two numbers, the cumulated frequency f_A and the number of distinct values d_A . The idea behind histograms is that a piecewise approximation of the active domain may result in better estimates. Therefore, the

active domain $[l_A, u_A]$ is partitioned into subsets called *buckets*. Traditionally, for each bucket B_i , the cumulated frequency f_i^+ of the values falling within the bucket as well as the number of distinct values d_i^+ of A within the bucket is stored. A *histogram* then consists of a sequence of buckets.

24.6.1 Bucketization

Assume we wish to partition the active domain $D_A (= \Pi_A(R))$ of some attribute A of some relation R into β buckets B_i ($1 \leq i \leq \beta$). Then, each bucket contains a subset of values of the universe of A . That is $B_i \subseteq [l_A, u_A]$. Not any subset is used in practice. Since it is too memory consuming to store the values in each bucket explicitly, buckets always comprise subintervals of the active domain. Further, these are typically non-overlapping. That is $B_i \cap B_j = \emptyset$ for $i \neq j$.

Such a partitioning of the active domain can be achieved in two steps. In a first step, we fix a set of *bucket boundaries* $b_i \in [l_A, u_A]$ such that $l_A = b_0 \leq b_1 \leq \dots \leq b_\alpha = u_A$. In order to decrease the search space, the b_i are typically chosen from the active domain, that is $b_i \in D_A$.

In a second step, we use these values as bucket boundaries to determine the buckets. Here, there are several alternatives. Let us first consider the case of an integer-valued attribute A . If we use closed intervals for buckets, we can define a bucket as comprising the values in $[b_{i-1} + 1, b_i]$. Note that $[b_i, b_{i+1}]$ does not work, since it overlaps with $[b_{i+1}, b_{i+2}]$. We could also build a bucket $[b_{i-1}, b_i - 1]$. But with proper choices of the b_i , these two are equivalent. A not equivalent alternative is to use half-open intervals. In this case, we can define a bucket as $[b_i, b_{i+1}[$.

Another issue is whether the buckets completely cover the active domain, that is whether $\bigcup_{i=1}^{\beta} B_i = [l_A, u_A]$ holds or not. In the latter case, we can define buckets comprising closed intervals as $[b_i, b_{i+1}]$ if we do not define buckets for $[b_{i-1}, b_i]$ and $[b_{i+1}, b_{i+2}]$. Thus, our histogram (the set of buckets we define) contains holes. This is typically only the case if no value from D_A falls into the hole.

Summarizing, we have the following three alternatives for attribute with a discrete, ordered domain:

1. closed-interval histogram without holes
2. closed-interval histogram with holes
3. half-open interval histogram without holes

Note that independent of the kind of histogram constructed, all the values b_i must be stored.

The literature is sparse on investigations of whether holes are good or not. Most papers don't even specify which kind of histogram they are talking about. As an exception, Wang and Sevcik consider 1) and 2) [906]. They propose to treat these two possibilities as alternatives during the construction of the histogram.

For a continuous domain, e.g. floating point numbers, alternative 1) is not (easily) possible. Thus, in this case, we can only choose between 2) and 3).

If most range queries use close ranges, 1) and 2) should be the preferred options. To see this, note that we must add or subtract the frequencies of the boundaries to convert a half-open or open interval into a closed interval or vice versa. Further, even if we have a good approximation of the exact frequencies of single values, we still face the problem of values not occurring in the active domain if it is not dense. Nonetheless, the subsequent discussion applies with minor modifications to all three alternatives.

24.6.2 Heuristics to Determine Bucket Boundaries

In the sequel, we discuss some heuristics to determine the bucket boundaries. All these algorithms have a parameter β for the number of buckets to construct. Subsequently, we assume that we are given a set d of value/frequency pairs $X = \{(x_i, f_i) | 1 \leq i \leq d\}$. We assume that the x_i are sorted in increasing order, i.e., $x_i < x_{i+1}$ for $1 \leq i < d$. By F^+ or n , we denote the cumulated frequency $F^+ = n = \sum_{i=0}^d f_i$. The bucket boundaries will be denoted by $b_0, \dots, b_\beta$, where $b_0 = x_0$ and $b_\beta = x_d$. Of course, we assume that $b < d$.

Equi-Width Histograms

Kooi was the first to propose histograms for selectivity estimation [512]. The first type of histograms he proposed were equi-width histograms. In an *equi-width* histogram, the bucket boundaries are determined by

$$b_i = x_0 + i\delta$$

where $\delta = (x_d - x_0)/\beta$.

Equi-Width Histograms

Kooi [512] also proposed the alternative of equi-width histograms [512]. There, all buckets have about the same cumulated frequency. This can be achieved by a single scan through X and starting a new bucket as soon as the current bucket's cumulated frequency exceeds F^+/β .

Another interesting approach to equi-width histograms has been described by Piatetsky-Shapiro and Connell [688]. There, buckets can overlap. The construction is quite simple, though expensive. First, from X we construct a bag Y of cardinality n such that each value x_i occurs exactly f_i times in Y . Then, Y is sorted by increasing values. A parameter S is used to determine the number of values to be stored. S determines the distance in the sorted vector Y between two stored values via $N = (n - 1)/S$. From the sorted vector we then pick every value at a position $1 + iN$ for $i = 0, \dots, S$. Hence, we store $S + 1$ values. If $(n - 1)/S$ is not an integer, the distance between the last two elements can be smaller.

This approach is called *distribution steps*, but could also be termed *quantiles*. If some values occur very frequently (more often than N times), then they are stored more than once. Thus, the buckets overlap. Besides the values nothing else is stored. Piatetsky-Shapiro and Connell then continue to give selectivity estimation formulas [688], which we do not repeat here.

The Heuristics Zoo

Besides the classic equi-width and equi-depth heuristics to find bucket boundaries, a whole zoo of heuristics has been proposed. Fortunately, this zoo has been classified by Poosala, Ioannidis, Haas, and Shekita [700].

Before we begin, let us recall some basic definitions. For every x_i except the last one, we define the *spread* s_i as $s_i = x_{i+1} - x_i$ and the *area* a_i as $a_i = f_i * s_i$. The motivation behind these definitions is based on the two major problems we face when approximating a given data distribution X . The problems are

1. largely varying f_i and
2. largely varying s_i .

Any of the x_i , f_i , s_i , or a_i is a parameter to the heuristics. We denote which one of these the heuristics should apply by V (value), F (frequency), S (spread), A (area), respectively.

The first partitioning heuristics is Equi-Sum. It gives rise to Equi-Sum(V), Equi-Sum(F), Equi-Sum(S), and Equi-Sum(A) and tries to balance the sum of its parameter such that all buckets exhibit about the same value for the sum. Thus, Equi-Sum(V) corresponds to equi-width (here, we count the number of x_i falling into a bucket) and Equi-Sum(F) corresponds to equi-depth histograms.

The next bunch of heuristics are max-diff histograms. They come in the flavors of max-diff(F), max-diff(S), and max-diff(A). They put bucket bounds between those $\beta - 1$ values x_i and x_{i+1} where the $\beta - 1$ highest values of the differences $|f_{i+1} - f_i|$, $|s_{i+1} - s_i|$, $|a_{i+1} - a_i|$ are found.

End-biased histograms store those values together with their frequencies that exhibit the β_1 lowest and β_2 highest frequencies with $\beta = \beta_1 + \beta_2$. The remaining values are put into a single bucket. High-biased histograms only store the β highest values and use a single histogram for the remaining values. Compressed histograms additionally use a regular histogram (e.g. equi-depth) to approximate the frequency distribution of the remaining values.

Note that none of these histograms comes with a guarantee regarding the maximal possible q-error for range queries. Histograms that exhibit this property will be discussed in Sec. ???. But before that, we need to prepare ourselves a little more.

24.7 More on Q

24.7.1 Properties of the Q-Error

Definition of the Q-Error

Let $f \geq 0$ be a number and $\hat{f} \geq 0$ be an estimate for f . Then, we define the q-error of $\hat{f}$ as

$$\|\hat{f}/f\|_Q,$$

where $\|x\|_Q := \max(x, 1/x)$.

If for some value $q \geq 1$ $\|\hat{f}/f\|_Q \leq q$, we say that the estimate is q -acceptable.

Let R be a relation and A be one of its attributes. Let $\Pi_A^D(R) = \{x_1, \dots, x_d\}$ with $x_i \leq x_{i+1}$. Denote by f_i the frequency of x_i and by $f^+(c_1, c_2) := \sum_{c_1 \leq x < c_2} f_i$ the cumulated frequency. Let $\hat{f}^+(x, y)$ be an estimation function for f^+ . Let $q \geq 1$ be some number. We say that $\hat{f}^+$ is q -acceptable, if for all $x_1 \leq c_1 \leq c_2 \leq x_d$ the estimate $\hat{f}^+(c_1, c_2)$ is q -acceptable.

Assume we know that

$$\frac{1}{a}x \leq x' \leq bx$$

for some numbers $a, b \geq 1$ and $x, x' > 0$. Then, it is easy to see that

$$\|x/x'\|_Q \leq \max(a, b)$$

holds.

Sums

For $1 \leq i \leq n$, let f_i be true values and $\hat{f}_i$ be estimates with $\|\hat{f}_i/f_i\|_Q \leq q$ for all $1 \leq i \leq n$. Then

$$1/q * \sum_{i=1}^n f_i \leq \sum_{i=1}^n \hat{f}_i \leq q * \sum_{i=1}^n f_i$$

holds, i.e.,

$$\left\| \frac{\sum_{i=1}^n \hat{f}_i}{\sum_{i=1}^n f_i} \right\|_Q \leq q.$$

Products

For $1 \leq i \leq n$, let f_i be true values and $\hat{f}_i$ be estimates with $\|\hat{f}_i/f_i\|_Q \leq q_i$ for all $1 \leq i \leq n$. Then

$$\prod_{i=1}^n (1/q_i) * \prod_{i=1}^n f_i \leq \prod_{i=1}^n \hat{f}_i \leq \prod_{i=1}^n q_i * \prod_{i=1}^n f_i$$

holds, i.e.,

$$\left\| \frac{\prod_{i=1}^n \hat{f}_i}{\prod_{i=1}^n f_i} \right\|_Q \leq \prod_{i=1}^n q_i.$$

Note that division behaves like multiplication.

Differences

Assume we are given a total value $t > 0$, which is the sum of a large values $l > 0$ and a small value $s > 0$. Thus, $t = l + s$ and $s \leq l$. The latter implies that $s \leq t/2 \leq l$.

If we know t and an estimate $\hat{l}$ of l , we can get an estimate $\hat{s} = t - \hat{l}$. This kind of estimation is not a good idea, as we see from the following example. Assume that $t = 100$, $l = 90$, $s = 10$, and $\hat{l} = 99$. Although $\|\hat{l}/l\|_Q = 1.1$, we have $\hat{s} = t - \hat{l} = 1$ and, thus, $\|\hat{s}/s\|_Q = 10$.

The situation is different, if we use an estimate $\hat{s}$ of (the smaller) s to derive an estimate $\hat{l}$ for (the larger) l as the following theorem shows.

Theorem 24.7.1 *Let $t, l, s > 0$ be three numbers such that $t = l + s$ and $s \leq l$. Let $\hat{s}$ be an estimate for s with $\|\hat{s}/s\|_Q \leq q$ for some $q \geq 1$. Define the estimate $\hat{l}$ for l as $\hat{l} = \max(t/2, t - \hat{s})$. Then $\|\hat{l}/l\|_Q \leq \min(2, q)$.*

Proof: First, observe that if $q = 1$ the theorem trivially holds. Second, observe that $\|\hat{l}/l\|_Q \leq 2$ always holds. Thus, if for some q we have $q = \|\hat{s}/s\|_Q > 2$ then $\|\hat{l}/l\|_Q \leq q$. Finally, we have to show that $\|\hat{l}/l\|_Q \leq q$ for $1 < q < 2$.

Due to $t = l + s$ and $s \leq l$, we have

$$t/2 \leq l = t - s \quad (*),$$

and, thus $t \leq 2(t - s)$.

Further remember our assumption $2 > q = \|\hat{s}/s\|_Q$. According to the definition of $\hat{l}$, we have to distinguish two cases.

Case 1: Assume $t - \hat{s} < t/2$. Then

$$\begin{aligned} t - \hat{s} &< t/2 \\ \hat{s} &> t - \frac{t}{2} \\ qs &> t - \frac{t}{2} \\ qs &> \frac{t}{2} \\ 2q &> \frac{t}{s} \\ \frac{1}{2q} &< \frac{s}{t} \quad (**) \end{aligned}$$

Thus,

$$\begin{aligned} \|\frac{\hat{l}}{l}\|_Q &= \|\frac{t/2}{l}\|_Q \\ &= \|\frac{t}{2(t-s)}\|_Q \\ &\stackrel{(*)}{=} \frac{2(t-s)}{t} \\ &= 2(1 - \frac{s}{t}) \\ &\stackrel{(**)}{\leq} 2(1 - \frac{1}{2q}) \\ &= 2 - \frac{1}{q} \\ &\leq q \end{aligned}$$

To see why the last inequality holds, we first observe that $2 - \frac{1}{q}$ is strongly increasing in q . Further, remember that $1 \leq q$ always holds and observe that (1) $q - (2 - \frac{1}{q})$ is strongly decreasing in q and (b) it attains its minimum in

the interval $[1, 2]$ at $q = 1$, for which $q - (2 - \frac{1}{q}) = 1 - (2 - \frac{1}{1}) = 0$. Thus, the difference $q - (2 - \frac{1}{q})$ is always greater or equal to zero. This fact together with

$$\begin{aligned} q - (2 - \frac{1}{q}) &\geq 0 \\ q &\geq 2 - \frac{1}{q} \end{aligned}$$

finishes Case 1.

Case 2: We have to show that

$$(1/q)l \leq t - \hat{s} \leq ql$$

under the assumptions that $t/2 \leq t - \hat{s}$ and $1 < q < 2$.

We start by showing $t - \hat{s} \leq ql$.

$$\begin{aligned} t - \hat{s} &\leq ql \\ \prec \succ \quad t - \hat{s} &\leq q(t - s) \\ \prec \quad t - \frac{1}{q}s &\leq q(t - s) \\ \prec \succ \quad t - \frac{1}{q}s &\leq qt - qs \\ \prec \succ \quad qs - \frac{1}{q}s &\leq qt - t \\ \prec \succ \quad (q - \frac{1}{q})s &\leq (q - 1)t \\ \prec \succ \quad \frac{(q - \frac{1}{q})}{(q - 1)} &\leq \frac{t}{s} \end{aligned}$$

We now observe that

$$\begin{aligned} \frac{(q - \frac{1}{q})}{(q - 1)} &= \frac{q - 1 + 1 - \frac{1}{q}}{q - 1} \\ &= 1 + \frac{1 - \frac{1}{q}}{q - 1} \\ &\leq 2 \\ &\leq \frac{t}{s} \end{aligned}$$

To see that $\frac{1 - \frac{1}{q}}{q - 1} \leq 1$ consider

$$\begin{aligned} \frac{1 - \frac{1}{q}}{q - 1} &\leq 1 \\ \prec \succ \quad 1 - \frac{1}{q} &\leq q - 1 \\ \prec \succ \quad 2 &\leq q + \frac{1}{q} \end{aligned}$$

The function $f(x) = x + 1/x$ is monotonically increasing on $x \in [1, 2]$, our interval of interest. Thus, since $1 + 1/1 = 2$, the claim follows.

We now show that $(1/q)l \leq t - \hat{s}$ under the assumptions that $t/2 \leq t - \hat{s}$ and $1 < q < 2$. If $(1/q)(t - s) \leq t/2$ then $(1/q)(t - s) \leq t/2 \leq t - \hat{s}$ and we are done. Thus, assume $t/2 \leq (1/q)(t - s)$.

Consider

$$\begin{aligned}
 (1/q)l &\leq t - \hat{s} \\
 \Leftrightarrow 1/q(t-s) &\leq t - \hat{s} \\
 \Leftarrow 1/q(t-s) &\leq t - qs \\
 \Leftrightarrow \frac{1}{q}t - \frac{1}{q}s &\leq t - qs \\
 \Leftrightarrow qs - \frac{1}{q}s &\leq t - \frac{1}{q}t \\
 \Leftrightarrow (q - \frac{1}{q})s &\leq (1 - \frac{1}{q})t \\
 \Leftrightarrow \frac{q - \frac{1}{q}}{1 - \frac{1}{q}} &\leq \frac{t}{s} \quad (*)
 \end{aligned}$$

Observe that

$$\begin{aligned}
 \frac{q - \frac{1}{q}}{1 - \frac{1}{q}} &= \frac{q^2 - 1}{q - 1} \\
 &= \frac{(q+1)(q-1)}{q-1} \\
 &= q+1 \quad (**)
 \end{aligned}$$

Summarizing (*) and (**), it suffices to show

$$q+1 \leq \frac{t}{s}.$$

From our assumption $t/2 \leq \frac{1}{q}(t-s)$, we can derive

$$\begin{aligned}
 t/2 &\leq \frac{1}{q}(t-s) \\
 \Leftrightarrow qt &\leq 2t - 2s \\
 \Leftrightarrow q &\leq 2\frac{t-s}{t} \\
 \Leftrightarrow q &\leq 2(1 - \frac{s}{t}) \\
 \Leftrightarrow q &\leq 2 - 2\frac{s}{t} \\
 \Leftrightarrow q+1 &\leq 3 - 2\frac{s}{t}
 \end{aligned}$$

Now, it suffices to show that

$$3 - 2\frac{s}{t} \leq \frac{t}{s}.$$

The following inequalities are equivalent:

$$\begin{aligned}
 3 - 2\frac{s}{t} &\leq \frac{t}{s} \\
 3ts - 2s^2 &\leq t^2 \\
 0 &\leq t^2 - 3ts + 2s^2 \\
 0 &\leq t^2 - 2ts + s^2 - ts + s^2 \\
 0 &\leq (t-s)^2 - s(t-s) \\
 0 &\leq (t-s) - s \\
 0 &\leq t - 2s
 \end{aligned}$$

Since the latter holds, we are done with the case $t/2 \leq \frac{1}{q}(t-s)$ and, thus, with Case 2.

□

Theorem 24.7.2 *Let $t, l, s > 0$ be three numbers such that $t = l + s$ and $s \leq l$. Let $\hat{t}$ be an estimate for t with $\|\hat{t}/t\|_Q \leq q$ for some $q \geq 1$. Define the estimate $\hat{l}$ for l as $\hat{l} = \max(\hat{t}/2, t - s)$. Then $\|\hat{l}/l\|_Q \leq 2q$.*

Proof

Case 1. $\hat{l} = \hat{t}/2$. Define

$$q^* := \left\| \frac{\hat{l}}{l} \right\|_Q = \left\| \frac{\hat{t}/2}{t-s} \right\|_Q.$$

Case 1.1 $\hat{t}/2 \geq t - s$. Then

$$\begin{aligned} q^* &= \frac{\hat{t}/2}{t-s} \\ &\leq \frac{\hat{t}/2}{t/2} \\ &\leq q \end{aligned}$$

Case 1.2 $t - s \geq \hat{t}/2$. Then

$$\begin{aligned} q^* &= \frac{t-s}{\hat{t}/2} \\ &\leq \frac{t}{\hat{t}/2} \\ &\leq 2q \end{aligned}$$

Case 2. $\hat{l} = \hat{t} - s$, $\hat{t} - s \geq \hat{t}/2$. Define

$$q^* := \left\| \frac{\hat{l}}{l} \right\|_Q = \left\| \frac{\hat{t} - s}{t - s} \right\|_Q.$$

Case 2.1 $\hat{t} - s \geq t - s$. Then

$$\begin{aligned} q^* &= \frac{\hat{t} - s}{t - s} \\ &\leq \frac{\hat{t} - t/2}{t - t/2} \\ &\leq \frac{\hat{t} - t/2}{t/2} \\ &\leq 2q - 1 \end{aligned}$$

Case 2.2 $t - s \geq \hat{t} - s$. Then

$$\begin{aligned} q^* &= \frac{t-s}{\hat{t}-s} \\ &\leq \frac{t-s}{\hat{t}/2} \\ &\leq \frac{t}{\hat{t}/2} \\ &\leq 2q \end{aligned}$$

□

Note that if we make sure that we overestimate t , i.e., $\hat{t} \geq t$, then only Case 2.1 applies and $\hat{t}$ is quite precise.

Theorem 24.7.3 *Let $t, l, s > 0$ be three numbers such that $t = l + s$ and $s \leq l$. Let $\hat{t}$ be an estimate for t with $\|\hat{t}/t\|_Q \leq q_t$ for some $q_t \geq 1$. Let $\hat{s}$ be an estimate for s with $\|\hat{s}/s\|_Q \leq q_s$ for some $q_s \geq 1$. Define $\hat{l} := \max(\hat{t}/2, \hat{t} - \hat{s})$. Additionally, assume that $(1/q_t) - q_s s > 0$. Then, $\|\hat{l}/l\|_Q \leq \max(2q_t, q_t^2 q_s)$.*

ToDo

Proof

Case 1. Consider the case where $\hat{t}/2 > \hat{t} - \hat{s}$ and, thus, $\hat{l} = \hat{t}/2$. The first condition implies that $\hat{s} > \hat{t}/2$. Also, by our preconditions, $t/2 \leq t - s$. Define

$$q^* := \left\| \frac{\hat{l}}{l} \right\|_Q = \left\| \frac{\hat{t}/2}{t-s} \right\|_Q.$$

Case 1.1 Assume $\hat{l} \geq l$. Then

$$\begin{aligned} q^* &= \frac{\hat{t}/2}{t-s} \\ &\leq \frac{q_t t/2}{t/2} \\ &\leq q_t \end{aligned}$$

Case 1.2 Assume $\hat{l} < l$. Then

$$\begin{aligned} q^* &= \frac{t-s}{\hat{t}/2} \\ &= \frac{t-s}{1/q_t t/2} \\ &= q_t \frac{t-s}{t/2} \\ &\leq 2q_t \end{aligned}$$

Case 2. Consider the case where $\hat{t}/2 \leq \hat{t} - \hat{s}$ and, thus, $\hat{l} = \hat{t} - \hat{s}$. Define

$$q^* := \left\| \frac{\hat{l}}{l} \right\|_Q = \left\| \frac{\hat{t} - \hat{s}}{t-s} \right\|_Q.$$

Case 2.1 Assume $\hat{l} \geq l$. Then

$$\begin{aligned} q^* &= \frac{\hat{t} - \hat{s}}{t-s} \\ &\leq q_t \frac{t}{t-s} - \frac{1}{q_s} \frac{s}{t-s} \\ &\leq 2q_t - \frac{1}{q_s} \frac{s}{t-s} \\ &\leq 2q_t \end{aligned}$$

Also, we have that

$$\begin{aligned}
 q^* &= \frac{\hat{t} - \hat{s}}{t - s} \\
 &\leq q_t \frac{t - (1/q_t)\hat{s}}{t - s} \\
 &\leq q_t \frac{t - \hat{s}}{t - s} \\
 &\leq q_t q_s
 \end{aligned}$$

This is not too bad since q_t will be close to 1 in our applications.

Also

$$\begin{aligned}
 q^* &= \frac{\hat{t} - \hat{s}}{t - s} \\
 &\leq \frac{t - \hat{s} + (q_t - 1)t}{t - s} \\
 &\leq q_s + \frac{(q_t - 1)t}{t - s} \\
 &\leq q_s + 2(q_t - 1)
 \end{aligned}$$

This is not too bad since q_t will be close to 1 in our applications.

Case 2.2 Assume $\hat{l} < l$. Then

$$\begin{aligned}
 q^* &= \frac{t - s}{\hat{t} - \hat{s}} \\
 &\leq \frac{t - s}{(1/q_t)t - \hat{s}} \\
 &\leq q_t \frac{t - s}{t - q_t \hat{s}} \\
 &\leq q_t^2 q_s
 \end{aligned}$$

This holds if $t - q_t q_s s \geq t/2$.

If $q_t q_s < 2$ then

$$\begin{aligned}
 q^* &= \frac{t - s}{\hat{t} - \hat{s}} \\
 &\leq \frac{t - s}{(1/q_t)t - \hat{s}} \\
 &\leq \frac{t/2}{(1/q_t)t - q_s(t/2)} \\
 &\leq \frac{t}{(2/q_t)t - q_s(t)} \\
 &\leq \frac{1}{(2/q_t) - q_s} \\
 &\leq q_t \frac{1}{2 - q_t q_s}
 \end{aligned}$$

das ist doof.

□

24.7.2 Properties of Estimation Functions

Let R be a relation and A one of its attributes. We assume that $\Pi_A^D(R) = \{x_1, \dots, x_d\}$, where $d := \Pi_A^D(R)$ and $x_i \leq x_j$ for all $1 \leq i \leq j \leq d$.

We only treat range queries since exact match queries are simpler than range queries and distinct value queries are similar.

An estimation function $\hat{f}^+$ is called *monotonic* on $[l, u]$, if and only if for all $l \leq c_1 \leq c'_1 \leq c'_2 \leq c_2 \leq u$

$$\hat{f}^+(c'_1, c'_2) \leq \hat{f}^+(c_1, c_2)$$

holds.

An estimation function $\hat{f}^+$ is called *additive* on $[l, u]$, if and only if for all $l = c_1 \leq \dots \leq c_k = u$

$$\hat{f}^+(c_1, c_k) = \sum_{i=1}^{k-1} \hat{f}^+(c_i, c_{i+1})$$

holds. Note that every additive estimation function is monotonic.

Assume we have an additive linear estimation function

$$\hat{f}^+(x, y) = \alpha x + \beta y + \gamma.$$

Then, we must have for all x, y, z with $x \leq y \leq z$:

$$\begin{aligned} \alpha x + \beta z + \gamma &= (\alpha x + \beta y + \gamma) + (\alpha y + \beta z + \gamma) \\ \text{<> } 0 &= \alpha y + \beta y + \gamma \end{aligned}$$

This can only be achieved if $\gamma = 0$ and $\alpha = -\beta$. Thus, every linear and additive estimation function is of the form

$$\hat{f}^+(x, y) = \alpha(y - x).$$

We typically wish the bucket's estimation function to be precise for the whole bucket. Thus, we demand that

$$\|\hat{f}^+(\text{lb}, \text{ub})/f^+(\text{lb}, \text{ub})\|_Q \leq q$$

for some error bound q . With $f^+ := f^+(\text{lb}, \text{ub})$, we have

$$\begin{aligned} (1/q)f^+ &\leq \hat{f}^+(\text{lb}, \text{ub}) \leq qf^+ \\ (1/q)f^+ &\leq \alpha(\text{ub} - \text{lb}) \leq qf^+ \\ (1/q)\frac{f^+}{\text{ub} - \text{lb}} &\leq \alpha \leq q\frac{f^+}{\text{ub} - \text{lb}} \end{aligned}$$

and the above holds if

$$\left\| \frac{\alpha}{\frac{f^+}{\text{ub} - \text{lb}}} \right\|_Q = \left\| \frac{\alpha(\text{ub} - \text{lb})}{f^+} \right\|_Q \leq q.$$

This clearly holds, if we use the usual estimation function

$$\hat{f}_{\text{avg}}^+(x, y) = \frac{y - x}{\text{ub} - \text{lb}} f^+$$

but we have the possibility to choose α within certain bounds.

24.7.3 θ, q -Acceptability

One problem occurs if the cardinality estimate for some query is $\hat{f} \geq 1$ and the true cardinality is zero. This happens, since we should never return an estimate of zero, because this leads to query simplifications which may be wrong or in reorderings which may not be appropriate. To solve this dilemma, there is only a single solution: during query optimization time, we execute building blocks and even access paths until the first tuple has been delivered. From there on, we know for sure, whether the result will be empty or not. If there is a tuple delivered, we buffer it, since we want to avoid its recalculation at runtime. The overhead of this method should therefore be low. Now, assume that we are willing to buffer more tuples (say 1000). Then, if there are less than 1000 qualifying tuples, we know the exact answer after fetching them. If we have to halt the evaluation of the build block since the buffer is full, we know that there will be ≥ 1000 qualifying tuples. Let us denote by θ_{buf} the number of tuples we are willing to buffer. Since we interleave query optimization and query execution, this can be considered a small step in the direction of *adaptive query optimization* [232].

However, before we can evaluate a building block or access paths, we have to determine an optimal one, which in turn requires cardinality estimates! Before we proceed note that cardinality estimates may be imprecise as long as they do not influence the decisions of the query optimizer badly. This means, as long as the query optimizer produces the best plan, any estimate is o.k. Let's for example take the decision whether to exploit an index or not. Assume, an index is better than a scan if less than 10% of the tuples qualify (This is a typical value [591, 367]). If the relation has 10000 tuples, the threshold is at 1000 tuples. Thus, assume that for a given range query both, the estimate and the true value do not exceed 500. Then, no matter what the estimate is, we should use the index. Note that the q -error can be 500 (e.g., the estimate is 1 and the true value is 500). Still it does not have any bad influence on our decision. The important thing is that the estimate has to be precise around 1000. For a given relation and one of its indices, we denote by θ_{idx} the number of tuples that, if exceeded make a table scan more efficient than the index scan.

Let us now combine these two things. Assume we want to have a maximal q -error of q . Define $\theta = \min(\theta_{\text{buf}} - 1, (1/q)\theta_{\text{idx}})$. Assume that $\hat{f}$ is an estimate for the true cardinality f . Further assume that if $\hat{f}$ or f exceeds θ , then $\|\hat{f}/f\|_Q \leq q$. Now let's go through the optimizer. In a first step, we define our building blocks and access paths, which requires to decide on index usage. Clearly, the estimate will be precise above $(1/q)\theta_{\text{idx}}$, which includes the critical part. After evaluating a building block or access path, we have precise cardinality estimates if fewer than θ_{buf} tuples are retrieved. Otherwise, our estimate will obey the given q -error. Thus, we are as precise as necessary under all circumstances.

These simple observations motivate us to introduce the notion of θ, q -acceptability. Let $f \geq 0$ be a number and $\hat{f} \geq 0$ be an estimate for f . Let $q \geq 1$ and $\theta \geq 1$ be numbers. We say that $\hat{f}$ is θ, q -acceptable if

1. $f \leq \theta \wedge \hat{f} \leq \theta$ or

$$2. \|\hat{f}/f\|_Q \leq q.$$

Let R be a relation and A be one of its attributes. Let $\Pi_A^D(R) = \{x_1, \dots, x_d\}$ with $x_i \leq x_{i+1}$. Denote by f_i the frequency of x_i and by $f^+(c_1, c_2) := \sum_{c_1 \leq x < c_2} f_i$ the cumulated frequency. Let $\hat{f}^+(x, y)$ be an estimation function for f^+ . We say that $\hat{f}^+$ is θ, q -acceptable, if for all $x_1 \leq c_1 \leq c_2 \leq x_d$ the estimate $\hat{f}^+(c_1, c_2)$ is θ, q -acceptable.

Another way to look at θ is that it is a 'Bagatellgrenze'. If the cardinality is below it, we do not really care how large it really is. The reason is that query execution will be fast anyway, even if we pick the wrong plan.

24.7.4 Testing θ, q -Acceptability for Buckets

Let R be a relation and A one of its attributes. We assume that $\Pi_A^D(R) = \{x_1, \dots, x_d\}$, where $d := \Pi_A^D(R)$ and $x_i \leq x_j$ for all $1 \leq i \leq j \leq d$.

We only treat range queries since exact match queries are simpler than range queries and distinct value queries are similar.

We assume left-open bucket boundaries and range queries of the form $[a, b[$.

Discretization

Testing θ, q -acceptability for a given bucket for a continuous domain directly is impossible since it would involve testing θ, q -acceptability of $\hat{f}^+(c_1, c_2)$ for all c_1, c_2 within the bucket. In this section, we show that a test quadratic in the number of distinct values in the bucket suffices.

Let c_1, c_2 be a query interval. Assume i, j are chosen such that $[x_i, x_j] \subseteq [c_1, c_2] \subset [x_{i-1}, x_{j+1}]$. Since there is no distinct value between x_i and x_{i-1} and between x_j and x_{j+1} , we have that $f^+(c_1, c_2) = f^+(x_i, x_j) < f^+(x_{i-1}, x_{j+1})$. Assume the following conditions hold:

1. $\hat{f}^+$ is monotonic.
2. $\|\frac{\hat{f}^+(x_i, x_j)}{f^+(c_1, c_2)}\|_Q \leq q$
3. $\|\frac{\hat{f}^+(x_{i-1}, x_{j+1})}{f^+(c_1, c_2)}\|_Q \leq q$

Since $\hat{f}^+(x_i, x_j) = \hat{f}^+(c_1, c_2) \leq \hat{f}^+(x_{i-1}, x_{j+1})$, we then have $\|\frac{\hat{f}^+(c_1, c_2)}{f^+(c_1, c_2)}\|_Q \leq q$.

Exploiting this fact, we can develop the following quadratic test for some given θ and q . If for all i, j such that x_i and x_j are in the bucket we have that

$$\hat{f}^+(x_{i-1}, x_{j+1}) \leq \theta \wedge f^+(x_{i-1}, x_{j+1}) \leq \theta$$

or

$$\|\frac{\hat{f}^+(x_i, x_j)}{f^+(x_i, x_j)}\|_Q \leq q \wedge \|\frac{\hat{f}^+(x_{i-1}, x_{j+1})}{f^+(x_i, x_j)}\|_Q \leq q$$

then the bucket is θ, q -acceptable.

Subtests

Still, after discretization, the number of tests is quadratic in the number of distinct values contained in a bucket. We can restrict this even further for monotonic and additive estimators $\hat{f}^+$. For a given, fixed θ and for any i ($1 \leq i < d$), we define i' to be the index such that

1. $f^+(x_i, x_{i'}) \leq \theta$
2. $\hat{f}^+(x_i, x_{i'}) \leq \theta$
3. $f^+(x_i, x_{i'+1}) > \theta$ or $\hat{f}^+(x_i, x_{i'+1}) > \theta$

This index i' can be found by binary search.

For a given L , assume that for all l with $1 \leq l \leq L$

- $\|\hat{f}^+(x_i, x_{i'+l})/f^+(x_i, x_{i'+l})\|_Q \leq q$

and

- $f^+(x_i, x_{i'+L}) \geq k\theta$ and
- $\hat{f}^+(x_i, x_{i'+L}) \geq k\theta$.

That is, we stop after L tests.

Then, we will show that the bucket is $\theta, (q + \frac{1}{k})$ -acceptable.

Consider the range query $[x_i, x_j]$. If $\hat{f}^+(x_i, x_j) \leq k\theta$, then it is θ, q -acceptable for $f^+(x_i, x_j)$.

Otherwise, we can find $i_1, \dots, i_m$ such that

- $x_i = x_{i_1}$ and
- $x_j = x_{i_m}$.

Also, we can achieve that (a)

- $\forall i_j < m - 1 \quad f^+(x_{i_j}, x_{i_{j+1}}) \geq k\theta$ and
- $f^+(x_{i_{m-1}}, x_{i_m}) < \theta$

or (b)

- $\forall i_j < m - 1 \quad \hat{f}^+(x_{i_j}, x_{i_{j+1}}) \geq k\theta$.
- $\hat{f}^+(x_{i_{m-1}}, x_{i_m}) < \theta$.

In the worst case, we have $m = 3$.

Case 1. $f^+(x_i, x_j) \leq \hat{f}^+(x_i, x_j)$ implies

$$\begin{aligned}
\left\| \frac{\hat{f}^+(x_i, x_j)}{f^+(x_i, x_j)} \right\|_Q &= \frac{\hat{f}^+(x_i, x_j)}{f^+(x_i, x_j)} \\
&= \frac{\hat{f}^+(x_{i_1}, x_{i_{l-1}}) + \hat{f}^+(x_{i_l}, x_j)}{f^+(x_{i_1}, x_{i_{l-1}}) + f^+(x_{i_{l-1}}, x_{i_l})} \\
&\leq \frac{qf^+(x_{i_1}, x_{i_{l-1}}) + \theta}{f^+(x_{i_1}, x_{i_{l-1}}) + 1} \\
&\leq \frac{qf^+(x_{i_1}, x_{i_{l-1}}) + \theta}{f^+(x_{i_1}, x_{i_{l-1}})} \\
&\leq q + \frac{\theta}{f^+(x_{i_1}, x_{i_{l-1}})} \\
&\leq q + \frac{\theta}{k\theta} \\
&\leq q + \frac{1}{k}
\end{aligned}$$

Case2. $\hat{f}^+(x_i, x_j) < f^+(x_i, x_j)$ implies

$$\begin{aligned}
\left\| \frac{\hat{f}^+(x_i, x_j)}{f^+(x_i, x_j)} \right\|_Q &= \frac{f^+(x_i, x_j)}{\hat{f}^+(x_i, x_j)} \\
&= \frac{f^+(x_{i_1}, x_{i_{l-1}}) + f^+(x_{i_l}, x_j)}{\hat{f}^+(x_{i_1}, x_{i_{l-1}}) + \hat{f}^+(x_{i_{l-1}}, x_{i_l})} \\
&\leq \frac{f^+(x_{i_1}, x_{i_{l-1}}) + \theta}{\hat{f}^+(x_{i_1}, x_{i_{l-1}}) + 1} \\
&\leq \frac{f^+(x_{i_1}, x_{i_{l-1}}) + \theta}{\hat{f}^+(x_{i_1}, x_{i_{l-1}})} \\
&\leq q + \frac{\theta}{\hat{f}^+(x_{i_1}, x_{i_{l-1}})} \\
&\leq q + \frac{1}{k}
\end{aligned}$$

Summarizing, we are able to trade in accuracy for performance when testing the θ, q -acceptability of some bucket.

A Cheap Pretest for Dense Buckets

If the domain of the attribute is discrete and every domain value within the bucket has a frequency larger than zero, the bucket is *dense*. This is always the case if dictionaries are used as in systems like Blink or Hana. In this case, θ, q -acceptability is implied by either of the following conditions:

1. The cumulated frequency of the bucket is less than or equal to θ or
2. $\max_i f_i / \min_i f_i \leq q^2$.

The first condition also holds for non-dense buckets. The last condition only holds if we use our flexibility concerning the α in our approximation function. If we use $\hat{f}_{\text{avg}}^+$, we need to exchange it against

$$q\bar{f} \geq \max_i f_i \wedge (1/q)\bar{f} \leq \min_i f_i,$$

where $\bar{f}$ is the average frequency of the bucket.

If this cheap pretest fails, we need to apply the quadratic test or the subtest.

24.7.5 From Buckets To Histograms

As usual, let R be some relation and A be some of its attributes with $\Pi_A^D = \{x_1, \dots, x_d\}$, where $d := |\Pi_A^D(R)|$ and $x_i \leq x_j$ for $1 \leq i \leq j \leq d$.

In general, θ, q -acceptability does not carry over from buckets to histograms. That is, even though all buckets maybe *theta, q*-acceptable, the histogram must not be. Consider a histogram in which each bucket has the true cumulated frequency θ and the estimate for each bucket is 1. Then, the estimate for a range query comprising n buckets is n and the true value is $n\theta$. Clearly, the histogram is not θ, q -acceptable if $q < \theta$.

Theorem 24.7.4 *Let H be a histogram. Consider two neighbored buckets B_1 and B_2 spanning the intervals $[b_i, b_{i+1}[$ for $i = 0, 1$. Let $k \geq 2$ be a number. If both buckets B_1 and B_2 are θ, q -acceptable then the histogram is $k\theta, q + \frac{q}{k-1}$ -acceptable.*

Proof: Assume we have two buckets $B_1 = [b_0, b_1[$ and $B_2 = [b_1, b_2]$ and a range query asking for the cumulated frequency in $[c_1, c_2[$ with

$$b_0 \leq c_1 \leq b_1 \leq c_2 \leq b_2.$$

For each bucket B_i , we denote by

$$f_i^+(x, y) = \sum_{x \leq x_i < y} f_i$$

the true cumulated frequency within an interval contained in the bucket. Further, we assume that the approximation function $\hat{f}_i^+(x, y)$ of every bucket B_i is θ, q -acceptable.

We introduce the following abbreviations:

$$\begin{aligned} f_1 &:= f^+(c_1, b_1) \\ f_2 &:= f^+(b_1, c_2) \\ f &:= f_1 + f_2 \\ \hat{f}_1 &:= \hat{f}_1^+(c_1, b_1) \\ \hat{f}_2 &:= \hat{f}_2^+(b_1, c_2) \\ \hat{f} &:= \hat{f}_1 + \hat{f}_2 \end{aligned}$$

Now, we investigate the estimation error for our range query. We distinguish several cases.

Case 1. In the first case, we assume $f \leq k\theta$ and $\hat{f} \leq k\theta$. In this case, the estimate is $k\theta, q$ -acceptable.

Case 2. In the second case, we assume that $(f_1 > \theta \vee \hat{f}_1 > \theta) \wedge (f_2 > \theta \vee \hat{f}_2 > \theta)$. It follows that $\|\hat{f}/f\|_Q \leq q$.

Case 3. We now assume that neither the condition of Case 1 nor the condition of Case 2 holds. Thus,

$$\neg(f \leq k\theta \wedge \hat{f} \leq k\theta) \wedge \neg((f_1 > \theta \vee \hat{f}_1 > \theta) \wedge (f_2 > \theta \vee \hat{f}_2 > \theta)),$$

which is equivalent to

$$(f > k\theta \vee \hat{f} > k\theta) \wedge ((f_1 \leq \theta \wedge \hat{f}_1 \leq \theta) \vee (f_2 \leq \theta \wedge \hat{f}_2 \leq \theta)).$$

We consider four subcases, where we denote by q^* the q-error of $\hat{f}$, i.e.,

$$q^* := \|\frac{\hat{f}}{f}\|_Q = \|\frac{\hat{f}_1 + \hat{f}_2}{f_1 + f_2}\|_Q.$$

Case 3.1 Assume

$$f > k\theta, f_1 \leq \theta, \hat{f}_1 \leq \theta.$$

From this, it follows that

$$k\theta < f = f_1 + f_2 \leq \theta + f_2$$

and thus $(k-1)\theta < f_2$ and, since $k \geq 2$

$$\|\frac{\hat{f}_2}{f_2}\|_Q \leq q.$$

Case 3.1.1 Assume $\hat{f}_1 + \hat{f}_2 \geq f_1 + f_2$. A simple calculation gives us

$$\begin{aligned} q^* &= \frac{\hat{f}_1 + \hat{f}_2}{f_1 + f_2} \\ &= \frac{\hat{f}_1}{f_1 + f_2} + \frac{\hat{f}_2}{f_1 + f_2} \\ &< \frac{\theta}{k\theta} + \frac{\hat{f}_2}{f_2} \\ &\leq q + \frac{1}{k} \end{aligned}$$

Case 3.1.2 Assume $f_1 + f_2 > \hat{f}_1 + \hat{f}_2$. A simple calculation gives us

$$\begin{aligned}
 q^* &= \frac{f_1 + f_2}{\hat{f}_1 + \hat{f}_2} \\
 &\leq \frac{\theta + q\hat{f}_2}{\hat{f}_2} \\
 &\leq q + \frac{\theta}{\hat{f}_2} \\
 &\leq q + \frac{\theta}{(1/q)f_2} \\
 &\leq q + \frac{\theta}{(1/q)(k-1)\theta} \\
 &\leq q + \frac{q}{k-1}
 \end{aligned}$$

Case 3.2 Assume

$$f > k\theta, f_2 \leq \theta, \hat{f}_2 \leq \theta.$$

This implies

$$k\theta < f = f_1 + f_2 \leq f_1 + \theta$$

and thus $(k-1)\theta < f_1$ and, since $k \geq 2$

$$\left\| \frac{\hat{f}_1}{f_1} \right\|_Q \leq q.$$

Case 3.2.1 Assume $\hat{f}_1 + \hat{f}_2 \geq f_1 + f_2$. Then, $\hat{f}_1 + \hat{f}_2 > k\theta$.

A simple calculation gives us

$$\begin{aligned}
 q^* &= \frac{\hat{f}_1 + \hat{f}_2}{f_1 + f_2} \\
 &= \frac{\hat{f}_1}{f_1 + f_2} + \frac{\hat{f}_2}{f_1 + f_2} \\
 &\leq \frac{\hat{f}_1}{f_1} + \frac{\theta}{k\theta} \\
 &\leq q + \frac{1}{k}
 \end{aligned}$$

‘ **Case 3.2.2** Assume $f_1 + f_2 > \hat{f}_1 + \hat{f}_2$. A simple calculation gives us

$$\begin{aligned}
 q^* &= \frac{f_1 + f_2}{\hat{f}_1 + \hat{f}_2} \\
 &\leq \frac{q\hat{f}_1 + \theta}{\hat{f}_2} \\
 &\leq q + \frac{\theta}{\hat{f}_2} \\
 &\leq q + \frac{\theta}{(1/q)f_2} \\
 &\leq q + \frac{\theta}{(1/q)(k-1)\theta} \\
 &\leq q + \frac{q}{k-1}
 \end{aligned}$$

Case 3.3 Assume

$$\hat{f} > k\theta, f_1 \leq \theta, \hat{f}_1 \leq \theta.$$

From this, it follows that

$$k\theta < \hat{f} = \hat{f}_1 + \hat{f}_2 \leq \theta + \hat{f}_2$$

and thus $(k-1)\theta < \hat{f}_2$ and, since $k \geq 2$

$$\left\| \frac{\hat{f}_2}{f_2} \right\|_Q \leq q.$$

Case 3.3.1 Assume $\hat{f}_1 + \hat{f}_2 \geq f_1 + f_2$. A simple calculation gives us

$$\begin{aligned}
 q^* &= \frac{\hat{f}_1 + \hat{f}_2}{f_1 + f_2} \\
 &\leq \frac{\hat{f}_1}{f_1 + f_2} + \frac{\hat{f}_2}{f_1 + f_2} \\
 &\leq \frac{\theta}{f_2} + \frac{\hat{f}_2}{f_2} \\
 &\leq \frac{\theta}{(1/q)\hat{f}_2} + q \\
 &\leq \frac{\theta}{(1/q)(k-1)\theta} + q \\
 &\leq q + \frac{q}{k-1}
 \end{aligned}$$

Case 3.3.2 Assume $f_1 + f_2 > \hat{f}_1 + \hat{f}_2$. A simple calculation gives us

$$\begin{aligned} q^* &= \frac{f_1 + f_2}{\hat{f}_1 + \hat{f}_2} \\ &\leq \frac{f_1}{\hat{f}_1 + \hat{f}_2} + \frac{f_2}{\hat{f}_1 + \hat{f}_2} \\ &\leq \frac{\theta}{k\theta} + \frac{f_2}{\hat{f}_2} \\ &\leq q + \frac{1}{k} \end{aligned}$$

Case 3.4 Assume

$$\hat{f} > k\theta, f_2 \leq \theta, \hat{f}_2 \leq \theta.$$

From this, it follows that

$$k\theta < \hat{f} = \hat{f}_1 + \hat{f}_2 \leq \hat{f}_1 + \theta$$

and thus $(k-1)\theta < \hat{f}_1$ and, since $k \geq 2$

$$\left\| \frac{\hat{f}_1}{f_1} \right\|_Q \leq q.$$

Case 3.4.1 Assume $\hat{f}_1 + \hat{f}_2 \geq f_1 + f_2$.

$$\begin{aligned} q^* &= \frac{\hat{f}_1 + \hat{f}_2}{f_1 + f_2} \\ &\leq \frac{\hat{f}_1}{f_1 + f_2} + \frac{\hat{f}_2}{f_1 + f_2} \\ &\leq \frac{\hat{f}_1}{f_1} + \frac{\theta}{f_1} \\ &\leq q + \frac{\theta}{(1/q)\hat{f}_1} \\ &\leq q + \frac{\theta}{(1/q)(k-1)\theta} \\ &\leq q + \frac{q}{k-1} \end{aligned}$$

Case 3.4.2 Assume $f_1 + f_2 > \hat{f}_1 + \hat{f}_2$. A simple calculation gives us

$$\begin{aligned} q^* &= \frac{f_1 + f_2}{\hat{f}_1 + \hat{f}_2} \\ &\leq \frac{f_1}{\hat{f}_1 + \hat{f}_2} + \frac{f_2}{\hat{f}_1 + \hat{f}_2} \\ &\leq \frac{f_1}{\hat{f}_1} + \frac{\theta}{k\theta} \\ &\leq q + \frac{1}{k} \end{aligned}$$

□

Theorem 24.7.5 *Let H be a histogram. Consider $n \geq 3$ consecutive buckets B_i in H spanning the intervals $[b_i, b_{i+1}[$ for $i = 0, \dots, n$. Let $k \geq 3$ be a number. If every estimate for a range query spanning a whole bucket is q -acceptable and every bucket B_i is θ, q -acceptable then the histogram is $k\theta, q + \frac{2q}{k-2}$ -acceptable.*

Proof: Assume a query interval $[c_1, c_2[$ spanning the n buckets of H . That is $b_0 \leq c_1 \leq b_1$ and $b_{n-1} \leq c_2 \leq b_n$.

We introduce the following abbreviations:

$$\begin{aligned} f_1 &:= f^+(c_1, b_1) \\ f_2 &:= f^+(b_1, b_{n-1}) \\ f_3 &:= f^+(b_{n-1}, c_2) \\ f &:= f_1 + f_2 + f_3 \\ \hat{f}_1 &:= \hat{f}_1^+(c_1, b_1) \\ \hat{f}_2 &:= \hat{f}_2^+(b_1, b_{n-1}) \\ \hat{f}_3 &:= \hat{f}_3^+(b_{n-1}, c_2) \\ \hat{f} &:= \hat{f}_1^+ + \hat{f}_2^+ + \hat{f}_3^+ \end{aligned}$$

By assumption, we have $\|\hat{f}_2^+/f_2^+\|_Q \leq q$.

We distinguish several cases.

Case 1. If $f \leq k\theta$ and $\hat{f} \leq k\theta$, then the estimate is $k\theta, q$ -acceptable.

Case 2. If $(f_1 > \theta \vee \hat{f}_1 > \theta) \wedge (f_3 > \theta \vee \hat{f}_3 > \theta)$, the estimate is q -acceptable.

Case 3. We now assume that neither the condition of Case 1 nor the condition of Case 2 holds. Thus

$$\neg(f \leq k\theta \wedge \hat{f} \leq k\theta) \wedge \neg((f_1 > \theta \vee \hat{f}_1 > \theta) \wedge (f_3 > \theta \vee \hat{f}_3 > \theta)),$$

which is equivalent to

$$(f > k\theta \vee \hat{f} > k\theta) \wedge ((f_1 \leq \theta \wedge \hat{f}_1 \leq \theta) \vee (f_3 \leq \theta \wedge \hat{f}_3 \leq \theta)).$$

We denote by q^* the q -error of $\hat{f}$, i.e.,

$$q^* := \|\frac{\hat{f}}{f}\|_Q = \|\frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{f_1 + f_2 + f_3}\|_Q.$$

Case 3.1 Assume $f_1 \leq \theta$ and $\hat{f}_1 \leq \theta$ and $f_3 \leq \theta$ and $\hat{f}_3 \leq \theta$.

Case 3.1.1 Assume $f > k\theta$. From $f = f_1 + f_2 + f_3 > k\theta$ and $f_1 \leq \theta$ and $f_3 \leq \theta$, we get

$$f_2 > (k-2)\theta$$

and

$$q\hat{f}_2 > (k-2)\theta.$$

If $f \leq \hat{f}$ we get

$$\begin{aligned}
 q^* &= \frac{\hat{f}}{f} \\
 &= \frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{f_1 + f_2 + f_3} \\
 &= \frac{\hat{f}_1 + \hat{f}_3}{f_1 + f_2 + f_3} + \frac{\hat{f}_2}{f_1 + f_2 + f_3} \\
 &\leq \frac{2\theta}{k\theta} + q \\
 &\leq q + \frac{2}{k}
 \end{aligned}$$

If $\hat{f} \leq f$ we get

$$\begin{aligned}
 q^* &= \frac{f}{\hat{f}} \\
 &= \frac{f_1 + f_2 + f_3}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\
 &\leq \frac{2\theta + f_2}{\hat{f}_2} \\
 &\leq q + \frac{2\theta}{\hat{f}_2} \\
 &\leq q + \frac{2\theta}{(1/q)(k-2)\theta} \\
 &\leq q + \frac{2q}{k-2}
 \end{aligned}$$

Case 3.1.2 Assume $\hat{f} > k\theta$. From $\hat{f} = \hat{f}_1 + \hat{f}_2 + \hat{f}_3 > k\theta$ and $\hat{f}_1 \leq \theta$ and $\hat{f}_3 \leq \theta$, we get

$$\hat{f}_2 > (k-2)\theta$$

and

$$q\hat{f}_2 > (k-2)\theta.$$

If $f \leq \hat{f}$ we get

$$\begin{aligned}
 q^* &= \frac{\hat{f}}{f} \\
 &= \frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{f_1 + f_2 + f_3} \\
 &\leq q + \frac{2\theta}{\hat{f}_2} \\
 &\leq q + \frac{2\theta}{(1/q)(k-2)\theta} \\
 &\leq q + \frac{2q}{k-2}
 \end{aligned}$$

If $\hat{f} \leq f$ we get

$$\begin{aligned}
 q^* &= \frac{f}{\hat{f}} \\
 &= \frac{f_1 + f_2 + f_3}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\
 &\leq \frac{f_2 + 2\theta}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\
 &\leq q + \frac{2\theta}{k\theta} \\
 &\leq q + \frac{2}{k}
 \end{aligned}$$

Case 3.2 Assume $f_1 \leq \theta$ and $\hat{f}_1 \leq \theta$ and $f_3 > \theta \vee \hat{f}_3 > \theta$.

Case 3.2.1 Assume $f > k\theta$.

From $f = f_1 + f_2 + f_3 > k\theta$ and $f_1 \leq \theta$, we get

$$f_2 + f_3 > (k-1)\theta$$

and

$$q(\hat{f}_2 + \hat{f}_3) > (k-1)\theta$$

since $\|\frac{\hat{f}_2 + \hat{f}_3}{f_2 + f_3}\|_Q \leq q$.

If $f \leq \hat{f}$ we get

$$\begin{aligned}
 q^* &= \frac{\hat{f}}{f} \\
 &= \frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{f_1 + f_2 + f_3} \\
 &\leq q + \frac{\theta}{k\theta} \\
 &\leq q + \frac{1}{k}
 \end{aligned}$$

If $\hat{f} \leq f$ we get

$$\begin{aligned}
 q^* &= \frac{f}{\hat{f}} \\
 &= \frac{f_1 + f_2 + f_3}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\
 &\leq q + \frac{\theta}{\hat{f}_2 + \hat{f}_3} \\
 &\leq q + \frac{\theta}{(1/q)(k-1)\theta} \\
 &\leq q + \frac{q}{k-1}
 \end{aligned}$$

Case 3.2.2 Assume $\hat{f} > k\theta$.

From $\hat{f} = \hat{f}_1 + \hat{f}_2 + \hat{f}_3 > k\theta$ and $\hat{f}_1 \leq \theta$, we get

$$\hat{f}_2 + \hat{f}_3 > (k-1)\theta$$

and

$$q(f_2 + f_3) > (k-1)\theta.$$

since $\|\frac{\hat{f}_2 + \hat{f}_3}{f_2 + f_3}\|_Q \leq q$.

If $f \leq \hat{f}$ we get

$$\begin{aligned} q^* &= \frac{\hat{f}}{f} \\ &= \frac{\hat{f}_1 + \hat{f}_2 + \hat{f}_3}{f_1 + f_2 + f_3} \\ &\leq q + \frac{\theta}{f_2 + f_3} \\ &\leq q + \frac{\theta}{(1/q)(k-1)\theta} \\ &\leq q + \frac{q}{k-1} \end{aligned}$$

If $\hat{f} \leq f$ we get

$$\begin{aligned} q^* &= \frac{f}{\hat{f}} \\ &= \frac{f_1 + f_2 + f_3}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\ &\leq q + \frac{\theta}{\hat{f}_1 + \hat{f}_2 + \hat{f}_3} \\ &\leq q + \frac{\theta}{k\theta} \\ &\leq q + \frac{1}{k} \end{aligned}$$

Case 3.3 Assume $f_1 > \theta \vee \hat{f}_1 > \theta$ and $f_3 \leq \theta$ and $\hat{f}_3 \leq \theta$.

Case 3.3.1 Assume $f > k\theta$. By Symmetry.

Case 3.3.2 Assume $\hat{f} > k\theta$. By Symmetry.

□

In case the estimates for a whole bucket are precise, e.g., if we use $\hat{f}_{\text{avg}}^+$, we can refine the bounds.

Corollary 24.7.6 *Let H be a histogram. Consider $n \geq 3$ consecutive buckets B_i in H spanning the intervals $[b_i, b_{i+1}[$ for $i = 0, \dots, n$. Let $k \geq 3$ be a number. If every estimate for a range query spanning a whole bucket is 1-acceptable and every bucket B_i is θ, q -acceptable then the histogram is $k\theta, q'$ -acceptable, where $q' := \frac{2}{k-2}q + 1$.*

To see that the corollary holds, simply reconsider the above proof. Let us mention that for $k \geq 3$, we never saw a q -error larger than $q + 1/k$.

```

qcompressb(x, b)
  return (0 == x) ? 0 :  $\lceil \log_b(x) \rceil + 1$ 

qdecompressb(y, b)
  return (0 == y) ? 0 :  $b^{y-1+0.5}$ 

qcompressbase(x, k)
  // x is the largest number to be compressed
  // k is the number of bits used to store a compressed value
  return  $x^{1/((1 < k) - 1)}$ 

```

Figure 24.11: Q-compression, $\log_b$ -based

24.7.6 Q-Compression

General Q-Compression

The goal of q-compression is to approximate a number $x \geq 1$ with a small q-error. Given some $b > 0$, let x be some number in the interval $[b^{2l}, b^{2(l+1)})$. If we approximate x by b^{2l+1} then $\|b^{2l+1}/x\|_Q \leq b$. Let $x_{\max}$ be the largest number to be compressed. If $x_{\max} \leq b^{2(k+1)}$ for some k is the maximal occurring number, we can approximate any x in $[1, x_{\max}]$ with $\lceil \log_2(k) \rceil$ bits obeying a maximal q-error of b . We can extend q-compression to allow for the compression of 0 as in the code in Fig. 24.11. There, we use the base b instead of b^2 as above. Thus, the error is at most $\sqrt{b}$. Let us consider a concrete example. Let $b = 1.1$. Assume we use 8 bits to store a number. Then, since $1.1^{254} \approx 32.6 \cdot 10^9$, we can approximate even huge numbers with a small q-error of at most $\sqrt{1.1} = 1.0488$. Other examples are given in Table 24.7.

There exists a small disadvantage of q-compression with a general base. Though calculating the logarithm is quite cheap, since typically machine instructions to do so exist, calculating the power during decompression is quite expensive. On our machine, compression takes roughly 54 ns whereas decompression takes 158 ns. This is bad since in the context of cardinality estimation, decompression is used far more often than compression. Thus, we introduce an alternative called *binary q-compression*.

Binary Q-Compression

The idea of binary q-compression is simple. Let x be the number we want to compress. If we take the base $b = 2$ then $\lceil \log_2(x) \rceil = k$, where k is the index of the highest bit set. This calculation can be done by a rather efficient machine instruction. This gives us a maximum q-error of $\sqrt{2}$. We can go below this, by remembering not only the highest bit set, but the k highest bits set. Additionally, we store the position of them (their shift) in s bits. The pseudocode is given in Fig. 24.12, where we extended the scheme to allow for the compression of zero. So far, this resembles a special floating point

#Bits	Base	Largest compressable number	q-Error
4	2.5	372529	1.58
4	2.6	645099	1.61
4	2.7	1094189	1.64
5	1.7	8193465	1.30
5	1.8	45517159	1.34
5	1.9	230466617	1.38
6	1.2	81140	1.10
6	1.3	11600797	1.14
6	1.4	1147990282	1.18
7	1.1	164239	1.05
7	1.2	9480625727	1.10
8	1.1	32639389743	1.05

Table 24.7: Examples for q-compression

representation with only positiv mantissa and exponent.

The q-middle of 2^n and $2^{n+1} - 1$ is $\sqrt{2^n * (2^{n+1} - 1)}$. This is the estimate we should return for n . We do not want to compute the square root during decompression, since this is too expensive. A little calculation helps.

$$\begin{aligned}
\sqrt{2^n * (2^{n+1} - 1)} &\approx \sqrt{2^n * 2^{n+1}} \\
&= \sqrt{2^{2n} * 2} \\
&= \sqrt{2} * 2^n \\
&= 2^n + (\sqrt{2} - 1) * 2^n
\end{aligned}$$

The second part can be calculated by a constant $(\sqrt{2} - 1)$ shifted by n to the left. The pseudocode in Fig. 24.12 gives the calculation of this constant C in C. The best theoretical q-error achievable with storing k bits is $\sqrt{1 + 2^{1-k}}$. With our fast approximation, we get pretty close as the following table shows. The observed maximal q-error column was obtained experimentally. The deviation from the observed maximal q-error to the theoretical maximal q-error is due to the fact that only a small portion of the digits of C are used. Further, compression (2.7 ns) and decompression (2.8 ns) are fast.

```

qcompress2(x, k, s)
  if  $2^s > x$ 
  then
    bits = x
    shift = 0
  else
    shift = index-of-highest-bit-set(x) - k + 1;
    bits = (x >> shift)
  return (bits << shift) | shift

qdecompress2(y, k, s)
  shift = y & ( $2^s - 1$ )
  bits = y >> shift
  x = bits << shift
  – assume  $C = (\text{int}) ((\text{sqrt}((\text{double}) 2.0) - 1.0) * 4 * (1 << 30))$ 
  x |= (C >> (32 - shift))
  return x

```

Figure 24.12: Binary Q-compression

k	max q-error observed	max q-error theoretical ($\sqrt{1 + 2^{1-k}}$)
1	1.5	1.41
2	1.25	1.22
3	1.13	1.12
4	1.07	1.06
5	1.036	1.03
6	1.018	1.016
7	1.0091	1.0078
8	1.0045	1.0039
9	1.0023	1.00195
10	1.0011	1.00098
11	1.00056	1.00048
12	1.00027	1.00024

Incremental Updates

It might come as a surprise that q-compressed numbers can be incrementally updated. Already in 1978, Morris observed this fact [628]. Later, Flajolet analyzed the probabilistic counting method thoroughly [284]. The main idea is rather simple. For binary q-compressed numbers, the incrementing procedure is defined as follows:

```

RandomIncrement(int& c)
  // c: the counter

```

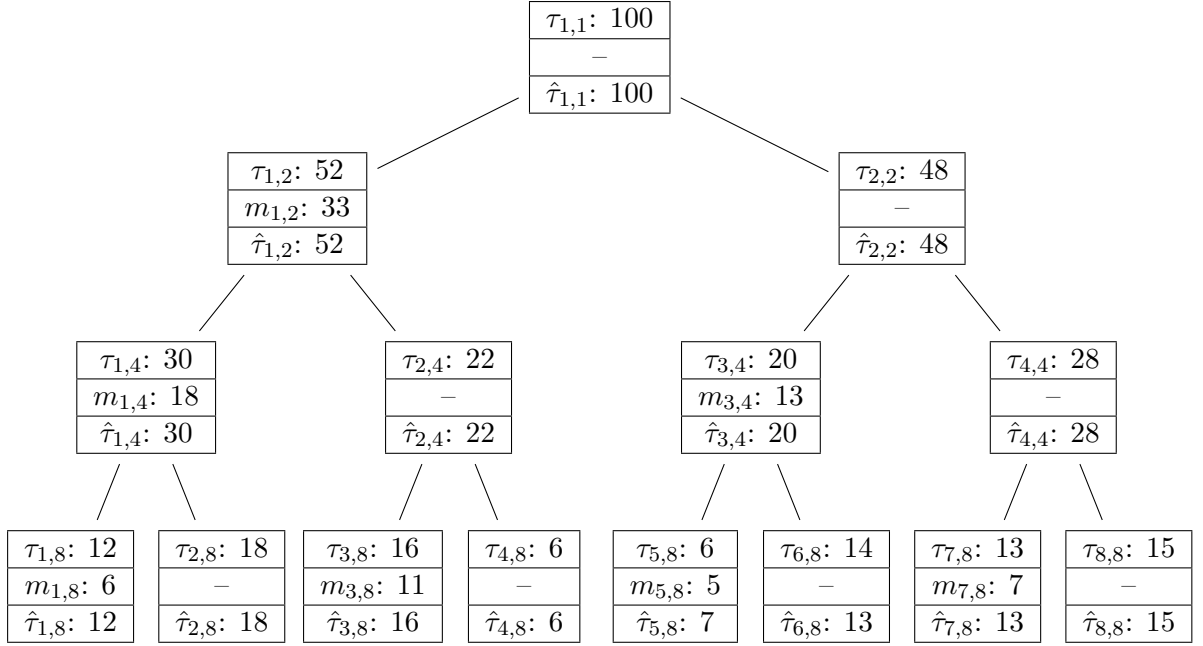


Figure 24.13: FLT example 1

let δ be a binary random variable which takes
 value 1 with probability 2^{-c} and
 value 0 with probability $1 - 2^{-c}$.
 $c += \delta$

To go to an arbitrary base, we have to modify the random variable δ such that it takes the value 1 with probability a^{-c} and 0 with probability $1 - a^{-c}$.

24.8 One Dimensional Synopses

24.8.1 Four Level Tree and Variants

The Original Four Level Tree

Four level trees were introduced by Buccafurri, Pontieri, Rosaci, and Sacca [117]. Later, Buccafurri, Lax, Sacca, Pontieri, and Rosaci discussed three, five, and N-Level level trees [115, 116]. A concise description can also be found in [210].

The basic idea is to divide a bucket into eight subbuckets (called bucklets) of equal width. Consider the following sample bucket [116]:

x_i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
f_i	7	5	18	0	6	10	0	6	0	6	9	5	13	0	8	7

This bucket is divided into 8 bucklets of width $16/8 = 2$. Every bucklet $\tau_{i,8}$ summarizes the values in bucket i , $1 \leq i \leq 8$. The next higher level of the four

level tree contains four values $\tau_{i,4}$ ($1 \leq i \leq 4$) summing the frequencies in the i -th quarter of the bucket. Thus, $\tau_{i,4} = \tau_{2i-1,8} + \tau_{2i,8}$ for $1 \leq i \leq 4$. The third level of the four level tree defines the values $\tau_{i,2}$ for $i = 1, 2$ summing up the frequencies in each half of the bucket. The last level, $\tau_{1,1}$ contains the sum of all frequencies f_i in the bucket. This scheme is illustrated in Fig. 24.13 and formally defined as

$$\tau_{i,2^k} := \tau_{2i-1,2^{k+1}} + \tau_{2i,2^{k+1}}$$

for $k = 0, \dots, 3$.

The four level tree in Fig. 24.13 is compressed into 64 bits as follows. $\tau_{1,1}$ is stored in the first 32 bits. Next, the $\tau_{j,2^k}$ for $k > 0$ are only stored if j is odd. For even $j = 2i$, $\tau_{2i,2^{k+1}}$ can be calculated given $\tau_{i,2^k}$:

$$\tau_{2i,2^{k+1}} := \tau_{i,2^k} - \tau_{2i-1,2^{k+1}}$$

for $k = 1, \dots, 3$. Further, since 7 numbers have to be compressed into 32 bits, only an approximation thereof is stored. The number of bits b_k used to store the approximation of some $\tau_{2i-1,2^{k+1}}$ decreases from top to bottom:

k	0	1	2	3
b_k	32	6	5	4

The intention is that if we make a mistake at a higher level, all lower levels are affected. Thus, we want to be precise at higher levels.

Instead of storing $\tau_{2i-1,2^{k+1}}$ directly, the ratio $\tau_{2i-1,2^{k+1}}/\tau_{i,2^k}$ is approximated using b_k bits:

$$m_{2i-1,2^{k+1}} := \text{round}\left(\frac{\tau_{2i-1,2^{k+1}}}{\tau_{i,2^k}}(2^{b_k} - 1)\right). \quad (24.33)$$

The 7 $m_{i,j}$ values are stored in the second 32 bits:

$m_{1,2}$	$m_{1,4}$	$m_{3,4}$	$m_{1,8}$	$m_{3,8}$	$m_{5,8}$	$m_{7,8}$
33	18	13	6	11	5	7
100001	10010	01101	0110	1011	0101	0111

The number of zeros and ones in the last line is $1 * 6 + 2 * 5 + 4 * 4 = 32$.

From $m_{2i-1,2^k}$, we can restore an estimate for $\hat{\tau}_{2i,2^{2k}}$ by calculating

$$\hat{\tau}_{2i,2^{2k}} := \text{round}\left(\frac{m_{2i-1,2^k}}{2^{b_k} - 1} * \hat{\tau}_{i,2^k}\right). \quad (24.34)$$

This recursion is possible, since we store $\tau_{1,1}$ explicitly. The $\hat{\tau}$ are also given in Fig. 24.13.

Now, consider the example in Fig. 24.14. It shows the four level tree for a frequency density where the eight buckets have the following cumulated frequencies:

i	1	2	3	4	5	6	7	8
f_i^+	1.000.000	100.000	10.000	1000	100	10	1	10.000

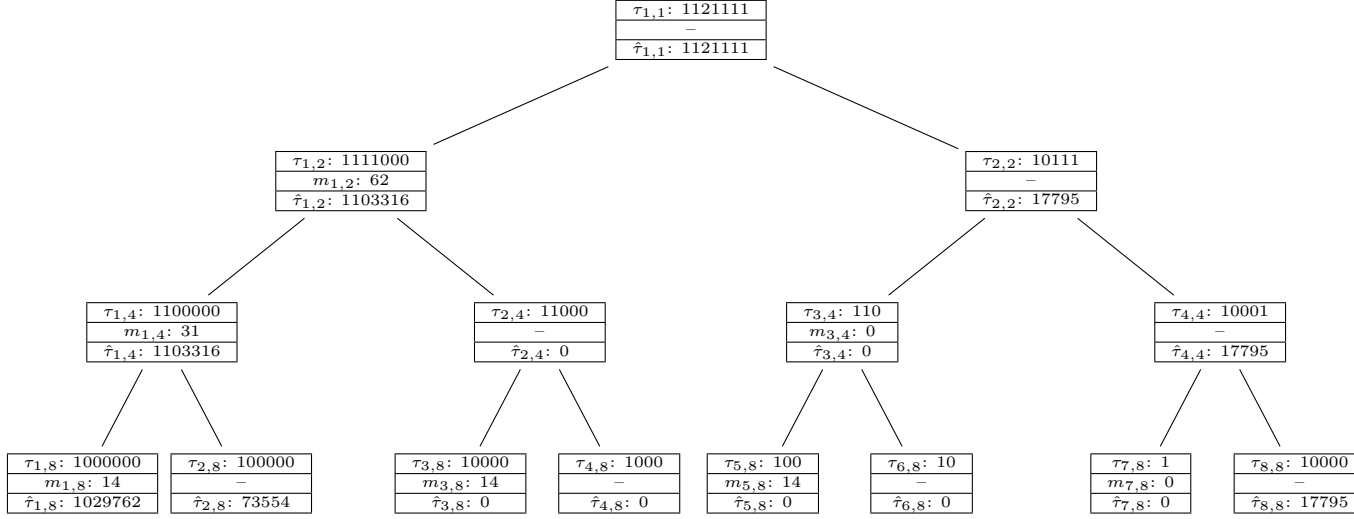


Figure 24.14: FLT example 2

As we can see, the error for last bucketlet 8,8 is quite large. The reason is that we subtract an estimate of larger number from a smaller number, which is not a good idea (see Sec. 24.7.1). Although, the four level tree is an excellent idea, it has two major problems:

1. Whenever the fraction in Formula 24.33 is smaller than $1/2^{b_k+1}$, rounding takes place towards zero.
2. Always the left child's τ is subtracted from the right child's τ . This results in uncontrollable errors if the right child's τ is smaller than the left child's τ (see Sec. 24.7.1).

Thus, we will modify the four level tree.

Variants of the Four Level Tree

Exploiting the techniques of (binary) q-compression, we can easily come up with several variants of the four level tree. All variants we discuss here use 7 indicator bits to remember whether the left or the right child node contained the smaller $\tau_{i,j}$.

The variant FLT2 stores $\tau_{1,1}$ in 11 bits using binary q-compression. For the other $\tau_{i,j}$, the original compression scheme is used. At level 2, 8 instead of 6 bits are used, at level 3, 7 bits instead of 5 bits are used and at level 4, 6 instead of 4 bits are used.

The variant qFLT also stores $\tau_{1,1}$ in 11 bits using binary q-compression. However, instead of deriving the other $\tau_{i,j}$ from estimates of their parents, it directly stores these values in q-compressed form. The number of bits used at each level is the same as in FLT2. The base used is derived from the estimate $\hat{\tau}_{1,1}$ for $\tau_{1,1}$. At level i , the minimal base for the number $\lceil \hat{\tau}_{1,1}/2^{i-1} \rceil$ is chosen.

24.8.2 Q-Histograms (Type I)

So far, every bucket contains the same information. Loosening this restriction results in *heterogeneous histograms*. This leads to *heterogeneous histograms* which contain different kinds of buckets. However, there are a few disadvantages: higher cpu and memory consumption. Further, with every bucket type considered, histogram construction costs increase. Besides the bucket boundaries and the bucket contents a bucket header has to be stored. Since this is typically only a single byte per bucket, the increased flexibility by using different bucket types by far outways the price of this byte, leading to more compact and precise histograms.

Simple Bucket Types

We now briefly summarize some possible bucket types.

standard bucket A standard bucket contains the cumulated frequency and the number of distinct values.

standard bucket with boundary frequency Microsoft SQL Server stores for every bucket the frequency of the lower bucket boundary [226, p208].

poly2dim, exppoly2dim Assume (x_i, f_i) is the frequency density of our attribute A. For given bucket boundaries b_1, b_2 , we can approximate the set

$$\text{RGE} := \{(x_i, x_j, f^+(x_i, x_j)) | b_1 \leq x_i \leq x_j \leq b_2\}$$

by a 2-dimensional polynomial. It is advantageous to use only low degrees (one or two) and find the best approximation under l_q . Approximation by a e^p for a polynomial p leads to another alternative.

poly1dim, exppoly1dim

Histogram Construction by Dynamic Programming

A Heuristic for Histogram Construction

24.8.3 Q-Histogram (Type II)

24.9 Sketches For Counting The Number of Distinct Values

Histograms are used to overcome deficiencies of the uniform distribution assumption. So far, the only way we discussed to combine selectivities derived for several predicates from different attributes has to be done under the attribute value independence assumption. In this section, we discuss an approach to avoid the attribute value independence assumption. This can be done by providing precise estimates on the number of distinct values in a set of attributes

(sometimes called *column group*). Although the number of distinct values for a set of attributes is rather interesting at different points in cardinality estimation (see our simple profile), we show how to provide selectivity estimates under the uniform distribution assumption but without relying on the attribute value independence assumption. The number of distinct values for a column group can be calculated by sorting or hashing, but this is often far too expensive. Thus, we provide a set of techniques based on *sketches*. Sketches are small summaries of data typically calculated online, i.e., with a single scan over the data. We concentrate on sketches for the count distinct case. A general introduction and an overview is contained in [210] and a recent evaluation of different sketches can be found in [406].

The problem we look at in this section is to approximately answer a query of the form

```
select count(distinct  $A_1, \dots, A_n$ )  
from  $R$ 
```

Standard techniques like hashtables or red-black trees can be used to collect the distinct values contained in the attributes $A_1, \dots, A_n$ of R . However, the space consumption is linear in the number of distinct values. Sketches require far less space.

Before we delve into the details of the algorithms, let us recall an observation made by Ilyas, Markl, Haas, Brown, and Aboulnaga [442]. They observed that sometimes assuming uniformity is not as bad as assuming attribute value independence. In this case, keeping the number of distinct values for a set of attributes helps to make estimates more precise. Consider their example of a car database repeated in Fig. 24.15. Assume that we want to provide an estimate for

```
select count(*)  
from  $Car$   
where  $Make = Honda$  and  $Model = Accord$ 
```

Denote by p_1 (p_2) the first (second) predicate in the query. The selectivities are $s(p_1) = 1/7$ and $s(p_2) = 1/8$. Assuming AVI yields $\hat{s}(p_1 \wedge p_2) = s(p_1) * s(p_2) = 1/56$. The true selectivity is $1/10$. The number of distinct values in the attribute group (Make, Model) is calculated by the following query

```
select count(distinct  $Make, Model$ )  
from  $Car$ 
```

and results in $\#DV = 9$. Assuming that all distinct values occur equally often (uniformity assumption) results in the selectivity estimate $\hat{s}(p_1 \wedge p_2) = 1/9$, which is much better.

Throughout the rest of this section, we assume that we want to produce an estimate $\hat{d}$ for the number of distinct values d of a multiset (bag) $X = \{x_1, \dots, x_d\}$.

Car		
ID	Make	Model
1	Honda	Accord
2	Honda	Civic
3	Toyota	Camry
4	Nissan	Sentra
5	Toyota	Corolla
6	BMW	323
7	Mazda	323
8	Saab	95i
9	Ford	F150
10	Mazda	323

Figure 24.15: Car database example

24.9.1 Linear Counting

The first algorithm we look at is **Linear Counting** by Astrahan, Schkolnick, and Whang [40]. Later, Whang, Vander-Zanden, and Taylor analyzed the algorithm and proposed a slight improvement [924]. Further, they showed analytically and experimentally, that linear counting still yields good results even if the fill factor goes up to 5-12. The algorithm is rather simple: Initialize a bitvector B of length b to contain only zeros. Then, for each member x in the given multiset X , set $B[h(x)]$ to one, where h is a hash function. Finally, the number of zeros z in B is counted and the resulting estimate for the number of distinct values in X is $\hat{d} = -b \ln(z/b)$ (or $\hat{d} = b \ln(b/z)$). If the number of ones in the bitvector is rather small, this number can be used as the estimate. The pseudocode of **LinearCounting** is given in Fig. 24.16. A problem occurs, if the bitvector becomes full, i.e., all bits are set. In this case, Whang et al. propose to run the algorithm a second time with another hash function. Further, they show how to keep the probability of running into a full bitvector a second time low. However, it is better to integrate **LinearCounting** into other algorithms that are capable of counting large numbers of distinct values in far less than linear space.

24.9.2 DvByKMinVal

Assume the hash function h hashes the elements of our set X to the interval $[0, 1[$. Further, let $H = \{h_i | h_i = h(x_i), x_i \in X\}$ and assume that $0 \leq h_i \leq h_{i+1} < 1$. If the hash function spreads out the elements evenly, we expect an average distance of $\delta = 1/(d+1) \approx 1/d$ between two neighbored hash values. For some given k , consider h_k , i.e., the k -th smallest value in H . This value can easily be calculated exploiting a heap to keep the lowest k distinct values while scanning X . Clearly, we expect the value of h_k to be around $k\delta$. Thus, $\delta = h_k/k$. If we plug this into the former equation, we get $h_k/k = 1/\hat{d}$ and hence $\hat{d} = k/h_k$. This very simple algorithm (see Fig.24.17), which we

```

LinearCounting( $X, h, B$ )
   $X$ : bag of elements
   $h$ : hash function in  $[0, b - 1]$ 
   $B$ : bitvector of length  $bk$ 
  initialize  $B$  with zeros
  for all  $x \in X$  do  $B[h(x)] = 1$ 
  for all  $i \in [0, m[$  do
    if  $0 = M[i]$  then  $z := z + 1$ 
  if  $0 = z$  then  $z := 1$ 
   $o := m - z$  // number of ones in the bitvector corresponding to  $M$ 
  if  $o < \sqrt{m}$ 
    then return  $o$ 
    else return  $m * \ln(m/z)$ 

```

Figure 24.16: Linear Counting

```

DvByKMinVal( $X, h$ )
  input: a bag  $X$ , a hashfunction  $h : X \rightarrow [0, 1]$ 
  output: estimate for the number  $\hat{d}$  for the number of distinct values in  $X$ 
  using, e.g., a heap calculate the  $k$ -th minimal value  $h_k$  in  $\{h(x) | x \in X\}$ 
   $\hat{d} := (k - 1) / h_k$ 
  return  $\hat{d}$ 

```

Figure 24.17: Algorithm DvByKMinVal

call **DvByKMinVal**, was developed and analyzed by Bar-Yossef, Jayram, Kumar, Sivikumar, Trevisan [52]. Later Beyer, Haas, Reinwald, Sismanis, and Gemulla showed that this estimator is biased. They found that an unbiased estimator is $\hat{d} = (k-1)/h_k$ [84]. As the hash function they recommend using the golden-ratio multiplicative hashing method [505].

24.9.3 Logarithmic Counting

In a series of papers, Flajolet and Martin introduced three different probabilistic counting algorithms [286, 287, 288]. For these algorithms, the hash function must map the elements of X to some bit pattern of a fixed length (say 32 or 64 bits).

Let us start with the simplest one called **LogarithmicCounting**. The idea behind this algorithm is the observation that the probability that the first bit in a bit pattern produced by the hash function being zero is $1/2$. The probability that the first two bits are zero is $1/4$, and so on. The algorithm calculates now the smallest index R , such that among the bitvectors $h(x_i)$ the R -th bit is never set. The estimate then roughly is 2^R . However, this estimate is biased.

```

LogarithmicCounting( $X, b$ )
  //  $X$ : bag of elements
  //  $h$ : hash function
  //  $b$ : length of bitvector
  // constant  $\phi = 0.7735162909$ 
  // indices for bitvectors start with 0
  let  $B$  be a bitvector of length  $b$ , with all bits set to zero
  for each  $x \in X$  do    $B \mid = \text{lowest-bit-set}(h(x))$ 
   $R = \text{index-of-lowest-zero-bit in } B$ 
  return  $(1/\phi) * 2^R$ 

```

Figure 24.18: Algorithm *LogarithmicCounting*

A factor $1/\phi$ corrects this. Fig. 24.18 shows the full algorithm including ϕ .

LogarithmicCounting produces rather rough estimates. This is remedied by a first alternative called *Multiple Probabilistic Counting*. The idea is to calculate m estimates for m independent hash functions and average these. However, using m different hash function is expensive and it may prove difficult to find these [24]. As a variant, Flajolet and Martin suggest to use several predetermined permutations and only one hash function [287]. However, both alternatives are still quite expensive. Hence, we don't detail on this algorithm.

The third variant, *Probabilistic Counting with Stochastic Averaging* (PC-SA), also averages several estimates, but it does so without applying multiple hash functions. The idea is to split the bitvector of the hashed value into two parts. The first k bits give an index into an array of bitvectors of size $b - k$. Then, for every input value x_i , only the bitvector determined by the first k bits of $h(x_i)$ is manipulated by remembering its lowest bit set. Instead of one R , we now have several R_j for $1 \leq j \leq 2^k$. These are averaged and the resulting estimate is produced. Fig. 24.19 shows the full pseudocode, where we also integrated the unbiasing presented in [287]. The standard deviation of PCSA is $0.78/\sqrt{m}$.

24.9.4 SuperLogLog Counting

Durand and Flajolet introduce two more space efficient probabilistic counting algorithms called **LogLogCounting** and **SuperLogLogCounting** [250]. The core idea of both algorithms is to remember the maximum of all indices i such that i corresponds to the lowest bit set for some $x \in X$. Note that this requires far less space: less than a byte suffices. As before, not only one such maximum is retained but $m = 2^k$ for some k . Fig. 24.20 shows how to fill an array M of m such maxima. The filled array M is the basis for **LogLogCounting** and **SuperLogLogCounting**. The algorithms just differ in how they produce their estimates of $\hat{d}$ from M .

In order to produce the **LogLogCounting** estimate, the maxima in M are averaged. Raising 2 by the power of this average and multiplying it by m yields

```

PCSA( $X, h, b, k$ )
  //  $X$ : bag of elements
  //  $h$ : hash function
  //  $b$ : length of bitvector produced by  $h$ 
  //  $k$ : length of prefix used to index array
  // constant  $\phi = 0.7735162909$ 
  // constant  $\psi = 1 + (0.31/m)$ 
  // indices for bitvectors start with zero
   $m := 2^k$ 
  let  $B$  be an array of size  $m$  containing bitvectors of size  $b - k$ 
  for each  $x \in X$  do
    //  $i, r$  = split  $h(x)$  into  $k$  and  $b - k$  bits:
     $i = h(x) \& ((1 \ll k) - 1)$ 
     $r = h(x) \gg k$ 
     $B[i] \mid = \text{lowest-bit-set}(r)$ 
  for each  $B[j], (1 \leq j \leq m)$  do
     $R_j = \text{index-of-lowest-zero-bit}(B[j])$ 
   $S = \sum_j R_j$ 
  return  $(m/(\phi * \psi)) * 2^{(S/m)}$ 

```

Figure 24.19: Algorithm PCSA

the estimate. Again, it is biased. To unbiased it, a constant α_m , for a given m , is used. Summarizing, the **LogLogCounting** estimate is calculated as follows:

```

LogLogCounting( $M$ )
   $\alpha_m := (\Gamma(-1/m) \frac{1-2^{1/m}}{\ln 2})^{-m}$  //  $\Gamma$  is the gamma function
   $\hat{d}_{\log\log} := \alpha_m m 2^{(1/m) \sum_j M[j]}$ 
  return  $\hat{d}$ 

```

The standard deviation of **LogLogCounting** is $\frac{1.3}{\sqrt{m}}$.

There are three major improvements in **SuperLogLogCounting**. For small numbers, **LogLogCounting** yields bad estimates. Thus, **SuperLogLogCounting** includes **LinearCounting** for this case. Second, instead of averaging all maxima in M , only a fraction of size $\lfloor 0.7m \rfloor$ is averaged, where the highest 30% are left out. Thus, we average only the $\lfloor 0.7m \rfloor$ smallest maxima in M . Third, some correction for hash collisions is performed.

The integration of **LinearCounting** uses M as its 'bitmap'. First, the number of zeros z is determined by counting the entries in M which are zero. The estimate is then produced as in **LinearCounting** (see Fig. 24.16).

As the details of **SuperLogLogCounting** cannot be found in [250], we refer the reader to Durand's thesis [249]. We review **SuperLogLogCounting** here to allow the reader to implement it. First, as already mentioned, the average of the $\lfloor 0.7m \rfloor$ smallest maxima in M is calculated. Let us call this value a_{partial} . This


```

FillM(X, h, M)
  // X: bag of elements
  // h: hash function
  // M: array of integers of size  $m = 2^k$ 
  // k: length of prefix used to index array M of maxima
  // indices for bitvectors start with one
  initialize M to 0
  for each  $x \in X$  do
     $y = h(x)$ 
    if  $0 == y$ 
      then
         $M[0] := \max(M[0], 33 - k)$  // if the length of hash value is 32 bits
      else
         $i := y \& ((1 \ll k) - 1)$ 
         $j := \text{idx-lowest-bit-set}(y \gg k)$ 
         $M[i] = \max(M[i], j)$ 

```

Figure 24.20: Filling M for LogLogCounting, SuperLogLogCounting, and HyperLogLogCounting

```

SuperLogLog(M)
   $\hat{d}_{\text{linc}} := \text{LinearCounting}(M)$ 
   $\hat{d}_{\text{loglog}} := \text{LogLogCounting}(M)$ 
   $\hat{d}_{\text{supll}} := \tilde{\alpha}(\hat{d}_{\text{loglog}})m2^{a_{\text{partial}}}$ 
   $L := 10m/5$ 
  case
    when  $\hat{d}_{\text{supll}} < L \wedge \hat{d}_{\text{linc}} < L$  do  $N := \hat{d}_{\text{linc}}$ 
    when  $\hat{d}_{\text{supll}} > L \wedge \hat{d}_{\text{linc}} > L$  do  $N := \hat{d}_{\text{supll}}$ 
    else  $N := (\hat{d}_{\text{linc}} + \hat{d}_{\text{supll}})/2$ 
  esac
   $H = 2^{32}$  // if 32 bit is the length of a hash value
  return  $-H \ln(1 - \frac{N}{H})$  // correction of hash collisions

```

Figure 24.21: SuperLogLog Counting

eliminates bad accidental outliers. Then, three estimates are calculated. The first is $\hat{d}_{\text{linc}}$ produced by **LinearCounting**. The second is $\hat{d}_{\text{loglog}}$ produced by **LogLogCounting**. This estimate is only used to produce the next estimate via some function $\tilde{\alpha}$. The third is $\hat{d}_{\text{supll}}$. The estimate produced by **SuperLogLog** is then calculated as shown in Fig. 24.21.

The only missing piece is the calculation of the unbiasing function $\tilde{\alpha}$ given in Fig. 24.22. As one can see, a polynomial of degree 4 is evaluated if k exceeds

```

 $\tilde{\alpha}(x)$ 
  //  $x$  is the estimate produced by LogLogCounting
  // remember:  $k$  is the number of bits used for indexing  $M$ 
   $\kappa := \lfloor \ln(x/m)/\ln(2) + 1.48 \rfloor + 1 - \ln(x/m)/\ln(2)$ 
  if  $k < 4$ 
    then  $r := 0.74$ 
  else  $r := c_4\kappa^4 + c_3\kappa^3 + c_2\kappa^2 + c_1\kappa^1 + c_0$ 
  return  $r$ 

```

Coefficients c_i :

	c_4	c_3	c_2	c_1	c_0
$k = 4$	0.003497	-0.03555	0.1999	-0.4812	1.139000
$k = 5$	0.00324250	-0.0346687	0.19794194	-0.47555735320	1.140732
$k = 6$	0.0031390489	-0.0343776755	0.197295	-0.4730536	1.141759
$k = 7$	0.0030924632	-0.0342657653	0.197045	-0.4718622	1.142318
$k = 8$	0.0030709	-0.034219	0.19694	-0.47129	1.142600
$k \in [9, 12]$	0.0030517	-0.034180	0.19685	-0.47077	1.142870
$k > 12$	0.0030504	-0.034177	0.19685	-0.47073	1.142880

Figure 24.22: Calculation of $\tilde{\alpha}$

3. The coefficients differ for different k . They are also given in the figure. The standard deviation of **SuperLogLogCounting** is $1.05/\sqrt{m}$. For hashing strings, Flajolet and co-workers suggest to use the hash function proposed by Lum, Yuen, Dodd [572].

24.9.5 HyperLogLog Counting

The algorithm **HyperLogLog Counting** developed by Flajolet, Fusy, Gandouet, and Meunier uses the same procedure **FillM** but, instead of using the geometric means as **LogLogCounting** and successors, it relies on harmonic means [285]. Its pseudocode is given in Fig. 24.23. Again, for few entries in M , the linear counting estimate is returned. For large ranges, a correction for hash collisions is performed. The unbiasing factor α_m is dependent on m and is calculated as follows: $\alpha_{16} = 0.673$, $\alpha_{32} = 0.697$, $\alpha_{64} = 0.709$, and $\alpha_m = 0.7213/(1 + 1.079/m)$ if $m \geq 128$.

24.9.6 DvByMinAvg

Whereas **DvByKMinVal** calculates the k -th smallest value, Lumbroso proposed to calculate m minima and average them [573]. This is done by splitting the values in the bag X into m partitions using the first l bits of the hash values. The remaining bits are then used to calculate the minima. The code of **DvByMinAvg** is shown in Fig. 24.24. The average of the minima contained in M is then calculated as the estimate E . As before, linear counting is used to estimate small numbers of distinct values. For the medium range, Lumbroso showed that the expected value of the estimate $\hat{d}$ of the algorithm is (see Theorem 4 in

```

HyperLogLog( $X, h, m$ )
   $X$ : bag of elements
   $h$ : hash function to  $\{0, 1\}^{32}$ 
   $m$ : number of entries in matrix  $M$ ,  $m = 2^l$  for some  $l$ 
  FillM( $X, h, M$ )
   $E := \alpha_m m^2 (\sum_{i=0} m^{2^{-M[i]}})^{-1}$  // 'raw' estimate
  if  $E < \frac{5/2}{m}$ 
    then  $V :=$  number of empty entries in  $M$ 
          $E^* := (V = 0) ? E^* : m \log(m/V)$ 
  else if  $E \leq \frac{1}{30} 2^{32}$ 
    then  $E^* := E$ 
  else  $E^* := -2^{32} \log(1 - E/2^{32})$ 
  return  $E^*$ 

```

Figure 24.23: HyperLogLog Counting

[573]):

$$E(\hat{d}) \approx \frac{d}{1 - e^{-\lambda}},$$

where d is the true number of distinct values in X and $\lambda = d/m$. In order to correct this bias, we set $y = \hat{d}/m$ and solve

$$y = \frac{\lambda}{\lambda - e^{-\lambda}}$$

for λ . Let us denote this inverse function by $f^{-1}(\lambda)$. The best quadratic approximation under l_q is $f^{-1}(x) \approx -0.0329046x^2 + 1.34703x - 0.932685$ with a maximal q-error of 1.0035.

24.9.7 DvByKMinAvg

Giroire proposed an algorithm we call **DvByKMinAvg** [331]. Although older than the approach by Lumbroso, **DvByKMinAvg** can easily be understood as a combination of **DvByKMin** and **DvByMinAvg**. As can be seen in Fig. 24.25, we maintain an array M of buckets. Each bucket holds the k minimal values assigned to it, where k is a parameter pragmatically chosen to be 3 [331]. This combines relatively low overhead with relatively high precision. After the array M has been filled with the minimal values of the actual estimate is calculated in two steps. First, the sum of the negative logarithms of the k -th minimal values is calculated. In the algorithm, we denote by $M^k[i]$ the k -th smallest value in bucket i . Then, the actual estimate is calculated from this sum. The estimate found in the algorithm corresponds to the *logarithm family* algorithm. Giroire presented two more estimators, namely the *inverse family* algorithm and the *square root family* algorithm [331].

```

DvByMinAvg( $X, h, m$ )
   $X$ : bag of elements
   $h$ : hash function to  $[0, 1]$ .
   $m$ : number of entries in array matrix  $M$ ,  $m = 2^l$  for some  $l$ 
  // calculate  $m$  minima
  for all  $x \in X$  do
     $a := h(x)$ 
     $i := \lfloor am \rfloor$ 
     $M[i] := \min(M[i], am - \lfloor am \rfloor)$ 
  od
   $\hat{d} := \frac{m(m-1)}{M[0] + \dots + M[m-1]}$ 
   $V :=$  number of empty entries in  $M$ 
  if  $V \leq 0.86m$ 
    then  $E^* := m \log(m/V)$ 
  else if  $V < m$ 
    then  $E^* := mf^{-a}(\hat{d}/m)$ 
  else
     $E^* := \hat{d}$ 
  return  $E^*$ 

```

Figure 24.24: DvByMinAvg

24.9.8 Pointers to the Literature

A general introduction and an overview is contained in [210]. A recent evaluation of different sketches can be found in [406]. Gelenbe and Gardy discuss a direct estimation approach to estimate the size of a projection [325] (see also the simple profile). In another paper, they do so in the presence of functional dependencies [324], which is an issue also investigated by Richard [730]. Oracle's approach to counting distinct values is described in [137]. Beyer, Haas, Reinwald, Sismanis, Gemulla show how to augment DvByKMinVal sketches with counters such that sketches for different bags can be combined such that estimates for unions/intersections/difference of these bags can be derived [84]. It is left as an exercise to the reader to show that any of the algorithms presented here can be used to efficiently estimate the number of distinct elements of disjoint unions of bags. This issue is important since large relations are often partitioned.

24.10 Multidimensional Synopsis

In the headerline we cheated a little. We only discuss 2-dimensional synopsis. This has the advantages that on the one hand it is already sufficiently complex but on the other hand 2-dimensional figures are still drawable. Nonetheless, most of the approaches presented here can be elevated to more than two di-

```

DvByKMinAvg( $X, h, m$ )
   $X$ : bag of elements
   $h$ : hash function to  $[0, 1]$ .
   $m$ : number of entries in array  $M$  of buckets,  $m = 2^l$  for some  $l$ 
  every bucket in  $M$  holds the  $k$  smallest values assigned to this bucket
  // calculate  $m$  times  $k$  smallest values
  for all  $x \in X$  do
    if  $\frac{i-1}{m} \leq h(x) \leq \frac{1}{m}$ 
      then actualize the  $k$  minima of bucket  $i$  with  $h(x)$ 
  od
   $s := \sum_{i=1}^m (\ln(M^k[i]))$ 
   $\hat{d} := m \cdot \left(\frac{\Gamma(k-\frac{1}{m})}{\Gamma(k)}\right)^1 \cdot e^{-\frac{s}{m}}$ 
  return  $\hat{d}$ 

```

Figure 24.25: DvByKMinAvg

mensions.

24.10.1 Introductory Example

To see why correlations happen, consider a very simple example with just one relation named **Orders**, which contains orders a sample company processes. We are only interested in two attributes: **orderdate** (**od**) and **shipdate** (**sd**). Assume every day 10 orders arrive. 5 are shipped the same day, 4 are shipped the next day and 1 is shipped the day after. Our database contains the orders for days 1 to 9. Thus, the cardinality of **Orders** is 90. The orders that are not yet shipped contain a NULL value in **shipdate**. Hence, there exist 6 tuples with null values in **shipdate**.

Define the frequency matrix F as

$$F(i, j) := |\sigma_{\text{od}=i \wedge \text{sd}=j}(\text{Orders})|.$$

Then, for our example we get the frequency matrix

	1	2	3	4	5	6	7	8	9
1	5	4	1	0	0	0	0	0	0
2	0	5	4	1	0	0	0	0	0
3	0	0	5	4	1	0	0	0	0
4	0	0	0	5	4	1	0	0	0
5	0	0	0	0	5	4	1	0	0
6	0	0	0	0	0	5	4	1	0
7	0	0	0	0	0	0	5	4	1
8	0	0	0	0	0	0	0	5	4
9	0	0	0	0	0	0	0	0	5

This frequency matrix is highly correlated. Let us look at the consequences.

Example 1 Assume we have a query

$$\sigma_{\text{od} \leq 4 \wedge \text{sd} \leq 4}(\text{Orders})$$

and we wish to estimate the result cardinality using the independence assumption. Since the selectivity of $\text{od} \leq 4$ is $40/84$ and the selectivity of $\text{sd} \leq 4$ is $34/84$, the total selectivity under independence is $40/84 * 34/84 = 0.19$, and thus an estimate of $0.19 * 84 \approx 16$ for our result cardinality. The true result cardinality is 34.

Example 2 Assume we have a query

$$\sigma_{\text{od} \leq 4 \wedge \text{sd} \geq 6}(\text{Orders})$$

and we wish to estimate the result cardinality using the independence assumption. Since the selectivity of $\text{od} \leq 4$ is $40/84$ and the selectivity of $\text{sd} \geq 6$ is $40/84$, we get that the total selectivity is $40/84 * 40/84 = 0.23$, and thus an estimate of $0.23 * 84 = 19$ for our result cardinality. The true result cardinality is 1.

Two dimensional synopses are meant to avoid these inaccuracies.

24.10.2 Solving the Introductory Problem without 2-Dimensional Synopsis

For the example above, a special solution exploiting one dimensional histograms exists. Instead of building a two-dimensional histogram on the attributes od and sd , we build a one-dimensional histogram on the difference $(\text{sd} - \text{od})$. The exact histogram for the introductory example looks as follows:

$(\text{sd} - \text{od})$	frequency
0	45
1	32
2	7

To see why this is a useful statistics to calculate estimates for the result sizes of our example queries, consider the general case of a conjunction of two range predicates

$$(c_1 \leq A \leq c_2) \wedge (d_1 \leq B \leq d_2) \quad (24.35)$$

on attributes A and B . This predicate implies

$$\begin{aligned} A - B &\leq c_2 - d_1 \\ B - A &\leq d_2 - c_1 \end{aligned}$$

which is equivalent to

$$\begin{aligned} A - B &\geq c_1 - d_2 \\ A - B &\leq c_2 - d_1 \end{aligned}$$

and thus

$$(c_1 - d_2) \leq (A - B) \leq (c_2 - d_1). \quad (24.36)$$

Using the one-dimensional histogram, we can derive an estimate for the selectivity of Eq. 24.36. Call this selectivity $s(Eq.24.36)$. Additionally, denote by $s(c_1 \leq A \leq c_2)$ and $s(d_1 \leq B \leq c_2)$ the selectivities of the two range predicates. Under the independence assumption, we would calculate the selectivity of Eq.24.35 as

$$s(Eq.24.35) = s(c_1 \leq A \leq c_2) * s(d_1 \leq B \leq c_2),$$

which results in the problems illustrated by the introductory example. Now, let us take the minimum of the two terms and multiply it with the selectivity of the predicate in Eq. 24.36. Thus, the estimate for the conjunct in Eq.24.35 becomes

$$s(Eq.24.35) = \min(s(c_1 \leq A \leq c_2), s(d_1 \leq B \leq c_2)) * s(Eq.24.36).$$

Let us see how this works for our example queries. In order to determine $s(od \leq 4 \wedge sd \leq 4)$, we have to determine the selectivities of the single predicates, which are $s(od \leq 4) = 40/84$ and $s(sd \leq 4) = 34/84$. Instantiating Eq.24.36 with $c_1 = d_1 = 0$ and $c_2 = d_2 = 4$ gives us $-4 \leq (sd - od) \leq 4$. Thus, all tuples qualify and the selectivity of this predicate is 1. Hence, we derive the cardinality estimate

$$\min(40/84, 34/84) * 1 * 84 = 34,$$

which is accidentally perfect. Now consider the predicate $(od \leq 4 \wedge sd \geq 6)$. The selectivity of $(sd \geq 6)$ is $40/84$. The selectivity of $2 \leq (sd - od) \leq 4$ is $7/84$. Thus, the cardinality estimate is

$$\min(40/84, 40/84) * 7/84 * 84 = 3,$$

which is closer to the truth than the estimate produced under independence.

24.10.3 Statistical Views

The above histogram is easily created, since both attributes come from the same relation. In reality, things can be a little more complex. Consider for example the following query against the TPC-H schema:

```
SELECT count(*)
FROM   Lineitem l, Orders o
WHERE  o.orderdate >= 1995.03.01 AND
       l.shipdate  <= 1995.03.07 AND
       l.orderno = o.orderno
```

Here, the two date attributes come from different relations. The solution to this problem is rather simple: define a *statistical view*. Although the exact syntax may differ, it is simply a view definition as in

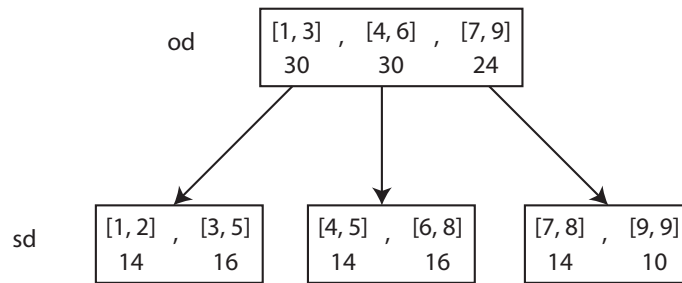


Figure 24.26: Example for Equi-Depth Tree

```
CREATE STATISTICAL VIEW statview_lo_date AS
SELECT o.shipdate - o.orderdate
FROM   Lineitem l, Orders o
WHERE  l.orderno = o.orderno
```

together with some specification what kind of synopsis should be created on the projected attributes. If more than a single attribute is projected, any of the following multi-dimensional synopsis can be used. One major advantage of this approach is that it covers the correlations introduced by the join predicate.

24.10.4 Regular Partitioning: equi-width

[603]

	[1, 3]	[4, 6]	[7, 9]
[1, 3]	24	6	0
[4, 6]	0	24	6
[7, 9]	0	0	24

24.10.5 Equi-Depth Histogram

[634]

24.10.6 2-Dimensional Synopsis based on SVD

24.10.7 PHASED

24.10.8 MHIST

24.10.9 GENHIST

24.10.10 HiRed

24.10.11 VI Histograms

24.10.12 Grid Trees

24.10.13 More

STHoles to organize query feedback.

24.11 Iterative Selectivity Combination

In this approach, independence is assumed and selectivities are simply multiplied. There is only one minor complication. Consider the query

```
select *
from   R, S, T
where  R.A = S.B and S.B = T.C
```

The query compiler uses transitivity to derive more predicates to increase the search space and make it more independent of the actual query formulation chosen by the user (Sec. 11.2.2). Thus, the query is rewritten to

```
select *
from   R, S, T
where  R.A = S.B and S.B = T.C and R.A = T.C
```

and all of $\{R.A, S.B, T.C\}$ are within the same equivalence class. All of the three equality predicates have an associated selectivity. However, after two of the predicates have been applied and their selectivities have been multiplied, the third predicate is implied by the other two and, accordingly its selectivity should not be used. This can easily be prevented by using a *union find* datastructure [209] associated with each plan class. It contains only those variables that contain in equivalence classes with cardinality greater than two. Initially, each of these variables is in its own class. Then, whenever an equality predicate is about to be applied, we check whether the two variables on the left and right are already in the same equivalence class. If so, we ignore the predicate. Otherwise, we apply the predicate and union the two equivalence classes of the variables. There remains only one open question. Assume the plan generator has generated the partial plan $R \bowtie_{R.A=S.B} S$. Then, there are two predicates left to join T : $S.B = T.C$ and $R.A = T.C$. For this case, where several predicates that can be applied, Swami and Schiefer [864] showed that the following rule (called *LS*) is the correct way to do it:

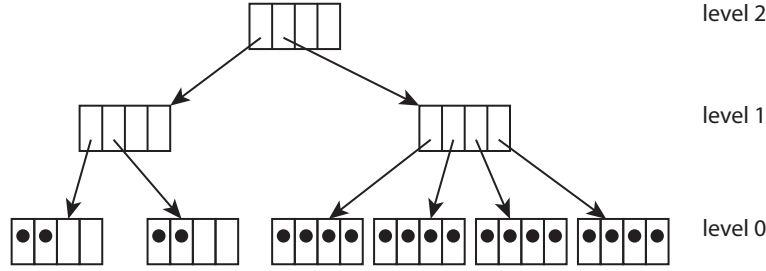
”Given a choice of join selectivities for a single equivalence class,
always pick the largest join selectivity.

Thus, to make things more efficient, we sort the equality predicates whose variables belong to equivalence classes with more than two elements by decreasing selectivity. Then, we can proceed as indicated above.

24.12 Maximum Entropy

[589, 590]

(useless) theoretical discussions: [480, 721, 722]

Figure 24.27: Sample B⁺-Tree

24.13 Selected Issues

24.13.1 Exploiting and Augmenting Existing DBMS Data Structures

Index Structures

Assume we have an attribute A of some relation R and a B⁺- or B*-Tree on A . The goal of this subsection is to show that we can use as simple procedure to produce a cardinality estimate for a given range query $A \in I_q$, where $I_q = [a, b]$ is some query interval.

Let us start with some notation. Denote a node in the B⁺-Tree by $\mathfrak{N}$. We assign a level to every node in the B⁺-Tree, increasing level numbers from leaf nodes up to the root whereby we start with level 0 for the leaf nodes (see Fig. 24.27). By $\mathfrak{N}$, we denote an arbitrary node in the B⁺-Tree. An arbitrary node at level l is denoted by $\mathfrak{N}[l]$. By $\mathfrak{N}[l].I[j]$ we denote the j -th interval within which all tuples R fall and by $\mathfrak{N}[l].S[j]$ we denote the correspond child (subtree root). For an interval $I = [a, b]$, we denote by $\text{len}(I) := b - a$ its length.

Roughly, there are two alternatives. In the first alternative, we maintain as little extra information in the or about the B-tree as possible. In the second alternative, we can maintain counters $\mathfrak{N}[l].C$ to remember the number of elements in the subtree. The resulting enhanced B⁺-Tree is called a *ranked tree* [32]. Reading out these counters for a given query and producing an estimate is rather simple and we will not detail on it here. However, maintaining these counters may not always be affordable in a transactional system. Thus, we concentrate on the first approach.

The leaf nodes store either tuples (e.g., for an index only table) or tuple identifiers (TIDs). In either case, there is a certain number of tuples stored in every leaf node. For indices on non-key attributes, and values with high frequency, overflow pages may exist. We ignore this fact by simply assuming that we are given the minimum and the maximum number of (referenced) tuples in a leaf page (possibly including overflow pages. We denote these numbers by $\text{min}[0]$ and $\text{max}[0]$. Similarly, we denote the minimum and maximum fanout at level $l > 0$ by $\text{min}[l]$ and $\text{max}[l]$. For fixed length keys, the B⁺-Trees guarantee that these two numbers are at most a factor of two apart, i.e., $\|\frac{\text{min}[l]}{\text{max}[l]}\|_Q \leq 2$ and that these numbers are the same at all internal nodes except the root node.

Further, these numbers can be derived from the sizes of the nodes, keys, and page pointers. For B⁺-Trees on attributes with domains of variable size (e.g., varchar), these numbers have to be maintained explicitly or estimates have to be produced. The same is true for the min[0] and max[0] values of the leaf nodes. Let us first assume that the min[i] and max[i] values are given. This then results in *pseudo-ranked trees* [32].

For the true number of tuples $f[0]$ in a leaf node $\min[0] \leq f[0] \leq \max[0]$ holds. For an arbitrary node $\mathfrak{N}[1]$ at level 1 the number of tuples $f[1]$ in any of its subtrees j satisfies $\min[0] * \min[1] \leq f[1] \leq \max[0] * \max[1]$. In general, for an arbitrary non-root node at level l , the number of tuples $f[l]$ stored in its subtree satisfies $\prod_{i=1}^l \min[i] \leq f[l] \leq \prod_{i=1}^l \max[i]$. Denote by MIN[l] the first product and by MAX[l] the second. Then, the most accurate estimate we can return is

$$\text{q-middle}(\text{MIN}[i], \text{MAX}[i])$$

with a maximal q-error of $\sqrt{(2^l)}$ if $\|\frac{\min[i]}{\max[i]}\|_Q \leq 2$ holds at all levels including the leaf node level.

Given a node $\mathfrak{N}[l, k]$ at an arbitrary level $l > 0$ with J child nodes, we can estimate its contribution to a range query with query interval I_q as in

$$\sum_{j=1}^J \frac{\text{len}(I_q \cap \mathfrak{N}[l, k].I[j])}{\text{len}(\mathfrak{N}[l, k].I[j])} \text{q-middle}(\text{MIN}[l], \text{MAX}[l]).$$

This procedure can now be applied to the root node only. However, it maybe beneficial to descend into child nodes for better estimates. This is especially true for those child nodes, that are not fully contained in the query interval. Thus, the questions arise (1) which nodes to descend and (2) when to stop. Several traversal strategies have been defined (see [36]).

This is not too bad, but there are certain problems. As indicated above, variable length keys and overflow pages due to high skew result in certain problems. Concerning the former problem. One possibility to overcome the former problem is to explicitly maintain the minimal and maximal fanout for each level. If this is too expensive, we could maintain the number of nodes $n[l]$ at every level l and use this to calculate the average fanout at level l as $n[l+1]/n[l]$ and use this number instead of the minimal and maximal fanout. Definitely, we loose any error bounds in this case. Consider the latter problem. The simplest solution is to maintain the number of leaf nodes explicitly and to derive an average number $\text{avg}[0]$ of tuples in the leaf nodes, which is then used instead of $\min[0]$ and $\max[0]$. Obviously, we lose precision, which can only be restored by maintaining explicit cardinality counters.

Dictionaries

Introduction Many main memory database management systems designed for OLAP are column stores. Further, they often use ordered dictionaries to facilitate compression of columns. Two commercial systems following these lines are Hana [?], DB2 Blue [?] SQL Server [?].

Let A be an attribute of some relation R . Assume that the active domain $D_A = \{x_1, \dots, x_n\}$ with $x_i < x_j$ if $i < j$. A *dictionary* then comprises two mappings:

1. a mapping of i to x_i and
2. a mapping from x_i to i .

We call the i *dictionary indexes* and the x_i *dictionary values*. No matter whether the domain of A is discrete or continuous, the dictionary indexes are positive integers. In a column store, the column for A then contains the (compressed) dictionary indexes of the original values.

If the dictionary is ordered, a range query

$$Q := \sigma_{l_q \leq A \leq u_q}(R) \quad (24.37)$$

with values l_q and u_q can be mapped to a range query on dictionary indexes. Depending on the use of $\leq$ vs. $<$, the lower and upper query bounds are mapped to lower (l_{idx}) and upper (u_{idx}) bounds on dictionary indexes as follows:

$$\begin{aligned} l_q \leq A &\rightarrow l_{\text{idx}} := \max(\{i | x_i \geq l_q\}) \\ l_q < A &\rightarrow l_{\text{idx}} := \max(\{i | x_i > l_q\}) \\ A \leq u_q &\rightarrow u_{\text{idx}} := \min(\{i | x_i \leq u_q\}) \\ A < u_q &\rightarrow u_{\text{idx}} := \min(\{i | x_i < u_q\}) \end{aligned}$$

Any range query (open or half-open or open) is then mapped to the *closed* range query

$$Q_{\text{idx}} := \sigma_{l_{\text{idx}} \leq A \leq u_{\text{idx}}}(R). \quad (24.38)$$

The mapping itself can be carried out by rather efficiently by a binary search within the dictionary.

Since Q and Q_{idx} are equivalent, estimation problems can now be carried out on Q_{idx} . This task is simplified by the very structure of a dictionary.

Distinct Values Since the dictionary is typically dense, that is no values that do not occur in the active domain are stored, the number of distinct values of A in Q can be calculated exactly:

$$|\Pi_A^D(\sigma_{l_{\text{idx}} \leq A \leq u_{\text{idx}}}(R))| = u_{\text{idx}} - l_{\text{idx}} + 1. \quad (24.39)$$

Cardinality Assume that every dictionary value x_i occurs with frequency f_i . Then, we have

$$|\Pi_A(\sigma_{l_{\text{idx}} \leq A \leq u_{\text{idx}}}(R))| = \sum_{i=l_{\text{idx}}}^{u_{\text{idx}}} f_i. \quad (24.40)$$

This requires that the f_i (4 bytes) are stored for every dictionary entry. At the expense of CPU time, we can use q-compression on the f_i to diminish memory consumption to one byte per dictionary entry. Thereby, we can be very precise, since, e.g., $1.1^{255} \approx 36 * 10^9$.

If $I := (u_{\text{idx}} - l_{\text{idx}} + 1)$ is small, the above summation yields acceptable performance. Assume, that we are willing to add 2δ frequencies. (If $\delta = 50$, this means we are willing to add 100 frequencies.) If $I > 2\delta$, we have to rely on alternative mechanisms. We have several options. Among the most obvious are:

- build a tree-like structure with fan out δ and height $\lceil \log_{\delta}(n) \rceil$, where n is the number of dictionary entries, or
- build some kind of histogram on the dictionary index, where, within every bucket, we have to be precise only for ranges comprising more than δ values (see Sec. 24.8.3).

24.13.2 Sampling

[210]

24.13.3 Query Feedback

24.13.4 Combining Data Summaries with Sampling

24.13.5 Wavelets

24.13.6 Selectivity of String-Valued Attributes

24.14 Cost Functions

24.14.1 Disk-based Joins

24.14.2 Main Memory Joins

24.14.3 Additional Pointers to the Literature

[963] bit-valued attributes, top-k queries.

[227] Cardinality estimation at the calculus level.

[177] Estimating Block Transfers and Join Sizes

[178] parametric: Pearson Type 2 and 7 for symmetric, unimodal distributions.

[274] Inverted files, multiple regression for Zipf distributions.

[275] also gives details cost model for inverted file retrieval.

[479] models relations as arrays of bits. Defines similarity function between bitmaps, lusters homogeneous rectangles using a pyramidal scheme.

[583] overview article summarizing the eighties up to 1986.

[318] uses generating functions.

[54] overview article (New Jersey data reduction report), several techniques.

[447] Overviews article, several techniques.

[699] application of histograms to load balancing for parallel joins.

[698] M-dimension Histograms (MHIST)

[461] optimal histograms with quality guarantees: minimum error under given space, minimal space under given maximal error, construction algo: $O(n^2)$ if n is the number of distinct values contained in the histogram

- [313] cost model for parallel query optimization
- [513] application of SVD to time series data
- [508] piecewise approximation, rough, with linear functions
- [96] kernel estimators
- [448] applies histograms to approximate query answers.
- [516] optimal histograms for hierarchical range queries (OLAP)
- [245] split and merge buckets to capture changes in variance (gives dynamic v-optimal histograms (DVO histograms)
- [110, 111] STHoles multidimension histograms constructed from query feedback
- [460] constructs many histograms at once to meet global storage bounds. this allows to give more memory to histograms for more skewed (less easy approximatable) attributes
- [376] histograms for data streams
- [374] discovery and application of check constraints
- [332] exploiting soft constraints
- [377] fast algorithm for histogram construction for hierarchical range queries (OLAP)
- [874] dynamic multidimensional histograms (for data streams)
- [117] 4byte encoding of 4-level tree to allow refinement estimates within a histogram bucket. (very nice idea)
- [509] automatic tuning of data synopsis
- [115] uses N-Level Tree Histograms (again bit encodings) to estimate range queries
- [442, 443, 441] automatic relationship discovery: correlations and soft functional dependencies
- [965] HASE: combines synopses-based selectivity estimation with sampling-based sel. est.
- [30] fast comp. of approx. statistics
- [262] just-in-time statistics (todo)
- [642] approximation of CDF with splines
- Teorey, Das: [870] physical database design
- Spyratos: [826] operational approach, database updates, views
- Yu, Luk, Siu: [963] Estimation Number of Desired Records with Respect to a given query
- Piatetsky-Shapiro [688]: Distribution Steps [635]: DDSM
- Architecture of Cardinality and Cost Estimation: Parameter systems [730]; [582]

Part V

Implementation

Chapter 25

Architecture of a Query Compiler

25.1 Compilation process

25.2 Architecture

Figure 25.1 a path of a query through the optimizer. For every step, a single component is responsible. Providing a facade for the components results in the overall architecture (Fig. 25.2). Every component is reentrant and stateless. The information necessary for a component to process a query is passed via references to control blocks. Control blocks are discussed next, then we discuss memory management. Subsequent sections describe the components in some detail.

25.3 Control Blocks

It is very convenient to have a hierarchy of control blocks within the optimizer. Figure 25.3 shows some of the control blocks. For simplification, those blocks concerned with session handling and transaction handling are omitted. Every routine call within the optimizer has a control block pointer as a parameter. The routines belonging to a specific phase have a pointer to the phase' specific control block as a parameter. For example, the routines in NFST have a `NFST_CB` pointer as a parameter. We now discuss the purpose of the different control blocks.

The global control block governs the behavior of the query compiler. It contains boolean variables indicating which phases to perform and which phases of the compilation process are to be traced. It also contains indicators for the individual phases. For example, for the first rewrite phase it contains indicators which rules to apply, which rules to trace and so on. These control indicators are manipulated by the driver which also allows to step through the different phases. This is very important for debugging purposes. Besides this overall control of the query compilers behavior, the global control block also contains

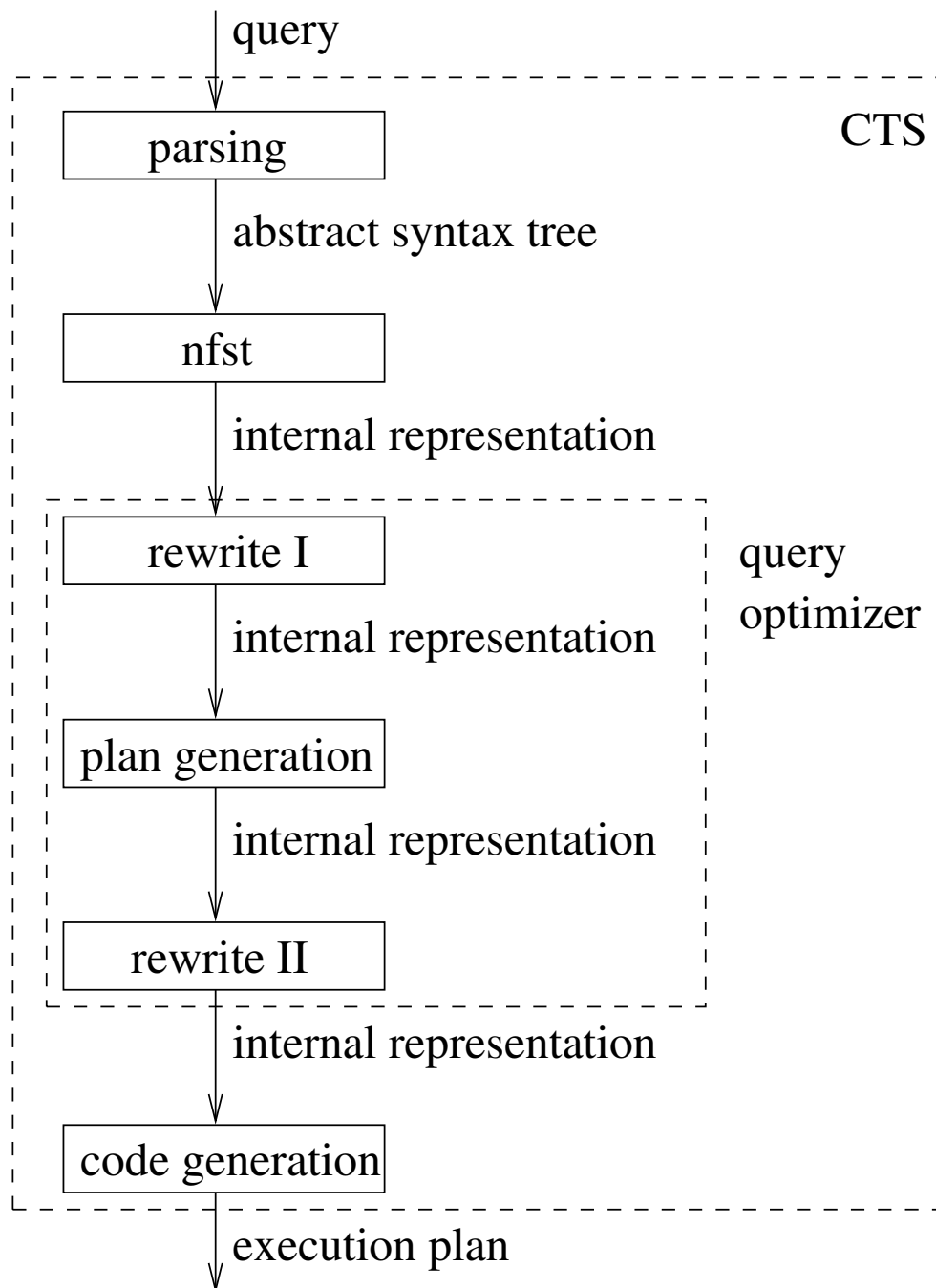


Figure 25.1: The compilation process

a pointer to the schema cache. The schema cache itself allows to look up type names, relations, extensions, indexes, and so on.

The query control block contains all the information gathered for the current query so far. It contains the abstract syntax tree, after its construction, the analyzed and translated query after NFST has been applied, the rewritten plan

after the `Rewrite_I` phase, and so on. It also contains a link to the memory manager that manages memory for this specific query. After the control block for a query is created, the memory manager is initialized. During the destructor call, the memory manager is destroyed and memory released.

Some components need helpers. These are also associated with the control blocks. We discuss them together with the components.

25.4 Memory Management

There are three approaches to memory management in query optimizers. The first approach is to use an automatic garbage collector if the language provides one. This is not necessarily the most efficient approach but by far the most convenient one. This approach can be imitated by an implementation based on smart pointers. I would not recommend doing so since the treatment of cycles can be quite tricky and it is inefficient. Another approach would be to collect all references to newly created objects and release these after the query has been processed. This approach is easy to implement, very convenient (transparent to the implementor), but inefficient. A better approach is to allocate bigger areas of memory by a memory manager. Factories¹ then use these memory chunks to generate objects as necessary. After the query has been processed, the chunks are freed.

Here, we consider only memory whose duration lasts for the processing of a single query. In general, we have more kinds of memory whose validity conforms to sessions and transactions.

25.5 Tracing and Plan Visualization

25.6 Driver

25.7 Bibliography

¹Design pattern.

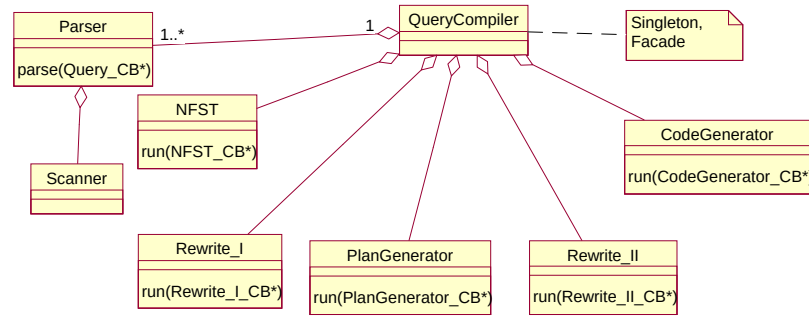


Figure 25.2: Class Architecture of the Query Compiler

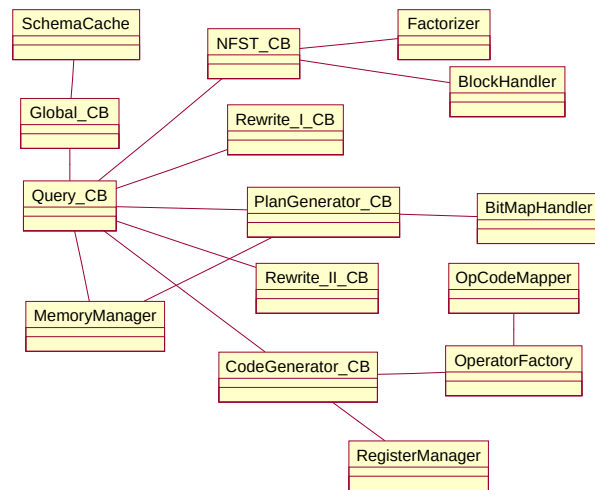


Figure 25.3: Control Block Structure

Chapter 26

Internal Representations

26.1 Requirements

easy access to information

query representation: overall design goal: methods/functions with semantic meaning, not only syntactic meaning.

relationships: consumer/producer (occurrence) precedence order information equivalence of expressions (transitivity of equality) see also `expr.h` fuer andere funktionen/beziehungen die gebraucht werden

2-ebenen repraesentation. 2. ebene materialisiert einige beziehungen und funktionen, die haeufig gebraucht werden und kompliziert zu berechnen sind anderer grund fuer materialisierung: vermeide zuviele geschachtelte forschleifen. bsp: keycheck: gegeben eine menge von attributen und eine menge von schluesseln, ist die menge ein schluessel? teste jeden schluessel, kommt jedes element in schluessel in menge von attributen vor? (schon drei schleifen!!!)

modellierungsdetail: ein grosser struct mit dicken case oder feine klassenhierarchie. wann splitten: nur wenn innerhalb des optimierers verschiedene abarbeitung erfordert.

Representation: info captured: 1) 1st class information (information obvious in original query+(standard)semantic analysis) 2) 2nd class information (derived information) 3) historic information (during query optimization itself) - modified (original expression, modifier) - copied (original expression, copier) 4) information about the expression itself: (e.g.: `is_function_call`, `is_select`) 5) specific representations for specific purposes (optimization algorithms, code generation, semantic analysis) beziehungen zwischen diesen repraesentationen

info captured for 1) different parts of the optimizer

syntactic/semantic information

garbage collection: 1) manually 2) automatic 3) semi-automatic (collect references, free at end of query)

26.2 Algebraic Representations

relational algebra in: [197].

26.2.1 Graph Representations

26.2.2 Query Graph

also called *object graph*: [79, 962]

26.2.3 Operator Graph

used in: [823], [957]

enhanced to represent physical properties: [739]

with outerjoins: [736], [309]

graph representation and equivalence to calculus: [661]

26.3 Query Graph Model (QGM)

26.4 Classification of Predicates

klassifikation von praedikaten

- nach stelligkeit, wertigkeit (selektion, join, nasty)
- nach funktor (=, i, ..., between, oder boolsche funktion)
- nach funktion: fuer keys in index: start/stop/range/exact/enum range(in-predicate)
- nach sel-wert: simple (col = const), komplex (col = expr) cheap/expensive
- nach join wert: fuer hj, smj, hbnlj, ...
- korrelationspraedikate

26.5 Treatment of Distinct

26.6 Query Analysis and Materialization of Analysis Results

Questions:

1. was materialisieren wir
 - bsp: properties: zeiger auf property oder besser inline properties
 - bsp: unique number: entweder in expr oder getrennter dictionary struktur
2. was packen wir in die 1. repraesentation?

```

query analysis (purpose, determine optimization algorithm)
#input relations, #predicates, #ex-quantifiers, #all-quantifiers,
#conjunctions, #disjunctions, #joininggraphkind(star,chain,tree,cyclic)
#strongly-connected-components (for crossproduct indication)
#false aggregates in projection list clause (implies grouping required)
/* remark: typical query optimizes should at least have two algorithms:
   - exhaustive (for large queries)
   - heuristic (for small queries)
*/

```

for blocks: indicator whether they should produce a null-tuple, in case they do not produce any tuple. this is nice for some rewrite rules. other possibility: if-statement in algebra.

26.7 Query and Plan Properties

Logical and Physical Properties of Plans

Ausführungsplänen können eine Reihe von Eigenschaften zugeordnet werden. Diese Eigenschaften fallen in drei Klassen

1. logische Eigenschaften, also beispielsweise
 - (a) beinhaltetete Relationen
 - (b) beinhaltetete Attribute
 - (c) angewendete Prädikate
2. physische Eigenschaften, also beispielsweise
 - (a) Ordnung der Tupel
 - (b) Strom oder Materialisierung des Ergebnisses
 - (c) Materialisierung im Hauptspeicher oder Hintergrundspeicher
 - (d) Zugriffspfade auf das Ergebnis
 - (e) Rechnerknoten des Ergebnis (im verteilten Fall)
 - (f) Kompression
3. quantitative Eigenschaften, also beispielsweise
 - (a) Anzahl der Elemente im Ergebnis
 - (b) Größe des Ergebnisses oder eines Ergebniselementes
 - (c) Auswertungskosten aufgeschlüsselt nach I/O, CPU und Kommunikationskosten

kosten: diese sind zu berechnen und dienen als grundlage fuer die planbewertung
 ges-kosten /* gesamt kosten (ressourcenverbrauch) */
 ges-kosten += cpu-instr / inst/sek
 ges-kosten += seek-kosten * overhead (waiting/cpu)
 ges-kosten += i/o-kosten * io-weight
 cpu-kosten /* reine cpu-kosten */
 i/o-kosten /* hintergrundspeicherzugriff (warten auf platte + cpu fuer seitenzugriffe) */

com-kosten /* kommunikation */ com-init /* initialisierungskosten fuer kommunikationsvorgang */ com-exit /* exitkosten fuer kommunikationsvorgang */ com-cptu /* kosten fuer jede transfereinheit (z.b. byte) waehrend eines kommunikationsvorgangs */

kostenstruktur koennte etwas sein, dass ges/cpu/io kosten enthaelt. ausserdem waeren kosten fuer rescanning interessant, falls dies notwendig ist (pufferprobleme, indexscan und dann faellt seite raus) weiteres interessantes kostenmass sind die kosten, bis das erste tupel berechnet wird.

dies sind die konstanten, die system-abhaengig sind. am besten sind, sie werden gemessen. Hardware: #cpu-instruktionen pro sekunde #cpu-instruktionen fuer block zugriff/transfer lesen/schreiben #cpu-instruktionen pro transfer init/send/exit init/receive/exit ms fuer seek/latency/transfer pro nK block

RTS-kosten #cpu-instruktionen fuer open/next/close fuer scan operatoren unter verschiedenen voraussetzungen:mit/ohne praedikat, mit/ohne projektion (entsprechend den avm programmen) #cpu-instruktionen fuer open/next/close fuer jeden alg operator, #cpu-instruktionen fuer funktionen/operationen/praedikate/avm-befehle

statistics: first/large physical page of a relation number of pages of a relation -i to estimate scan cost measured sequential scan cost (no interference/plenty interference)

-properties:

- menge der quns
- menge der attribute
- menge der praedikate
- ordnung
- boolean properties
- globale menge der gepipelineten quns
- kostenvektor
- cardinalitaeten bewiesen/geschaetzt
- gewuenschter puffer
- schluessel, fds
- #seiten, die durch ein fetch gelesen werden sollen
- menge der objekte, von denen der plan (der ja teilplan sein kann) abhaengt
- eigenschaften fuer parallele plaene
- eigenschaften fuer smp plaene

das folgende ist alles blabla. aber es weist auf den punkt hin,
das in dieser beziehung etwas getan werden muss.

--index: determine degree of clustering

- lese_rate = #gelesene_seiten / seiten_fuer_relation

ein praedikate erniedrigt die lesen_rate, ein erneutes lesen aufgrund einer
falls TIDs sortiert werden, muss fetch_ration erneut berechnet werden

- seiten koennen in gruppen z.b. auf einem zylinder zusammengefasst werden
und mit einem prefetch befehl geholt werden. anzahl seeks abschaetzen

- cluster_ration(CR)

CR = P(read(t) ohne page read) = (card - anzahl pagefetch)/card
= (card - (#pagefetch - #page))/card

das ist besonderer quark

- cluster_factor(CF)

CF = P(avoid unnecessary pagefetch) = (pagefetch/maxpagefetch)
= card - #fetch / card - #pageinrel

das ist besonderer quark

index retrieval on full key => beide faktoren auf 100% setzen, da
innerhalb eines index die TIDs pro key-eintrag sortiert werden.

Speicherung von Properties unter dynamischem Programmieren und Mem-
oization: Kosten und andere Eigenschaften, die nicht vom Plan abhängen,
können pro Planklasse gespeichert werden und brauchen nicht pro Plan gespe-
ichert zu werden.

26.8 Conversion to the Internal Representation

26.8.1 Preprocessing

26.8.2 Translation into the Internal Representation

26.9 Bibliography

Chapter 27

Details on the Phases of Query Compilation

27.1 Parsing

Lexical analysis is pretty much the same as for traditional compilers. However, it is convenient to treat keywords as soft. This allows for example for relation names like *order* which is a keyword in SQL. This might be very convenient for users since SQL has plenty (several hundreds) of keywords. For some keywords like *select* there is less danger of it being a relation name. A solution for *group* and *order* would be to lex them as a single token together with the following *by*.

Parsing again is very similar to parsing in compiler construction. For both, lexing and parsing, generators can be used to generate these components. The parser specification of SQL is quite lengthy while the one for OQL is pretty compact. In both cases, a LALR(2) grammar suffices. The outcome of the parser should be an abstract syntax tree. Again the data structure for abstract syntax trees (ast) as well as operations to deal with them (allocation, deletion, traversal) can be generated from an according ast specification.

During parsing already some of the basic rewriting techniques can be applied. For example, **between** can be eliminated.

In BD II, there are currently four parsers (for SQL, OQL, NQL (a clean version of XQuery), XQuery). The driver allows to step through the query compiler and allows to influence its overall behavior. For example, several trace levels can be switched on and off while within the driver. Single rewrites can be enabled and disabled. Further, the driver allows to switch to a different query language. This is quite convenient for debugging purposes. We used the Cocktail tools to generate the lexer, parser, ast, and NFST component.

27.2 Semantic Analysis, Normalization, Factorization, Constant Folding, and Translation

The NFST component performs (at least) four different tasks:

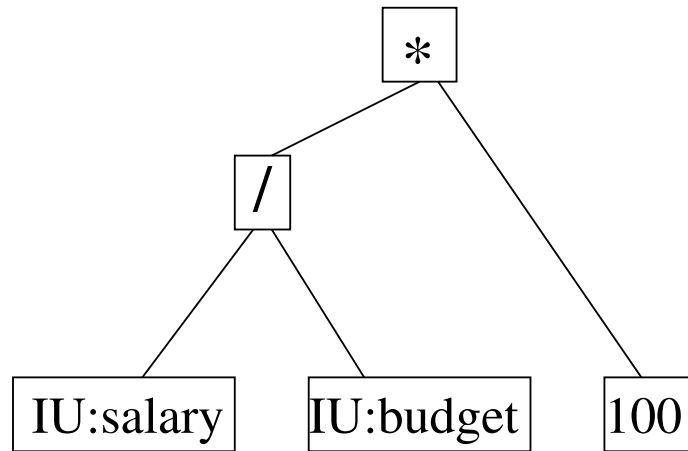


Figure 27.1: Expression

1. normalization of expressions,
2. factorization of common subexpressions,
3. semantic analysis, and
4. translation into the internal algebra-based query representation.

Although these are different tasks, a single pass over the abstract syntax tree suffices to perform all these tasks in one step.

Consider the following example query:

```

select  e.name, (d.salary / d.budget) * 100
from    Employee e, Department d
where   e.salary > 100000 and e.dno = d.dno
  
```

The internal representation of the expression $(d.salary / d.budget) * 100$ in the query is shown in Fig. 27.1. It contains two operator nodes for the operations “*” and “/”. At the bottom, we find IU nodes. IU stands for Information Unit. A single IU corresponds to a variable that can be bound to a value. Sample IUs are attributes of a relation or, as we will see, intermediate results. In the query representation, there are three IUs. The first two IUs are bound to attribute values for the attributes *salary* and *budget*. The third IU is bound to the constant 100.

NFST routines can be implemented using a typical compiler generator tool. It is implemented in a rule-based language. Every rule matches a specific kind of AST nodes and performs an action. The ast tree is processed in post order.

The hierarchy for organizing different kinds of expressions is shown in Fig 27.2. Here is a list of useful functions:

- occurrence of expressions in another expression

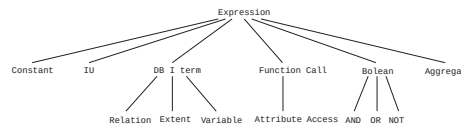


Figure 27.2: Expression hierarchy

- for a given expression: compute the set of occurring (consumed, free) IUs
- for a given expression: compute the set of produced IUs
- for a given IU, retrieve the block producing the IU
- determine whether some block returns a single value only
- computation of the transitivity of predicates, especially equality to derive its equivalence classes.
- determine whether some expression produces a subset of another expression
- constant folding
- merge and/or (from e.g. binary to n-ary) and push not operations
- replace a certain expression by another one
- deep and shallow copy

These functions can be implemented either as member functions of expressions or according to visitor/collector/mutator patterns. For more complex functions (consumer/producer) we recommend the latter.

Some of these functions will be called quite frequently, e.g. the consumer/producer, precedence ordering, equivalence (transitivity of equality) functions. So it might be convenient to compute these relationships only once and then materialize them. Since some transformation in the rewrite phases are quite complex, a recomputation of these materialized functions should be possible since their direct maintenance might be too complex.

27.3 Normalization

Fig. 27.3 shows the result after normalization. The idea of normalization is to introduce intermediate IUs such that all operators take only IUs as arguments. This representation is quite useful.

27.4 Factorization

Common subexpressions are factorized by replacing them with references to some IU. For the expressions in TPCD query 1, the result is shown in Fig. 27.4.

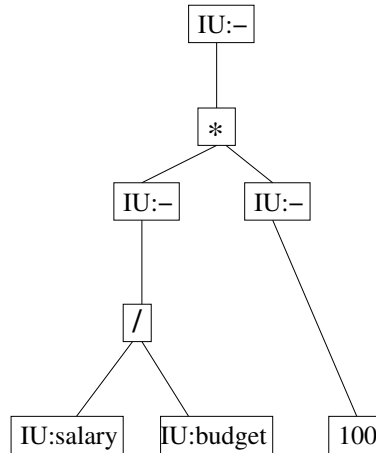


Figure 27.3: Expression

Factorization is enabled by a factorization component that takes care of all expressions seen so far and the IUs representing these expressions. Every expression encountered by some NFST routine is passed to the factorization. The result is a reference to an IU. This IU can be a new IU in case of a new expression, or an existing IU in case of a common subexpression. The factorization component is available to the NFST routines via the NFST control block which is associated with a factorization component (Fig.25.3).

27.5 Constant Folding

27.6 Semantic analysis

The main purpose of semantic analysis is to attach a type to every expression. For simple expressions it is very similar to traditional semantic analysis in compiler construction. The only difference occurs for references to schema constructs. The schema is persistence and references to e.g. relations or named objects have to be looked up there. For performance reasons it is convenient to have a schema cache in order to cache frequently used references. Another aspect complicating semantic analysis a little is that collection types are frequently used in the database context. Their incorporation is rather straight forward but the different collection types should be handled with care.

As programming languages, query languages provide a block structure. Consider for example the SQL query

```

...
select  a, b, c
from    A, B
where   d > e and f = g
...

```

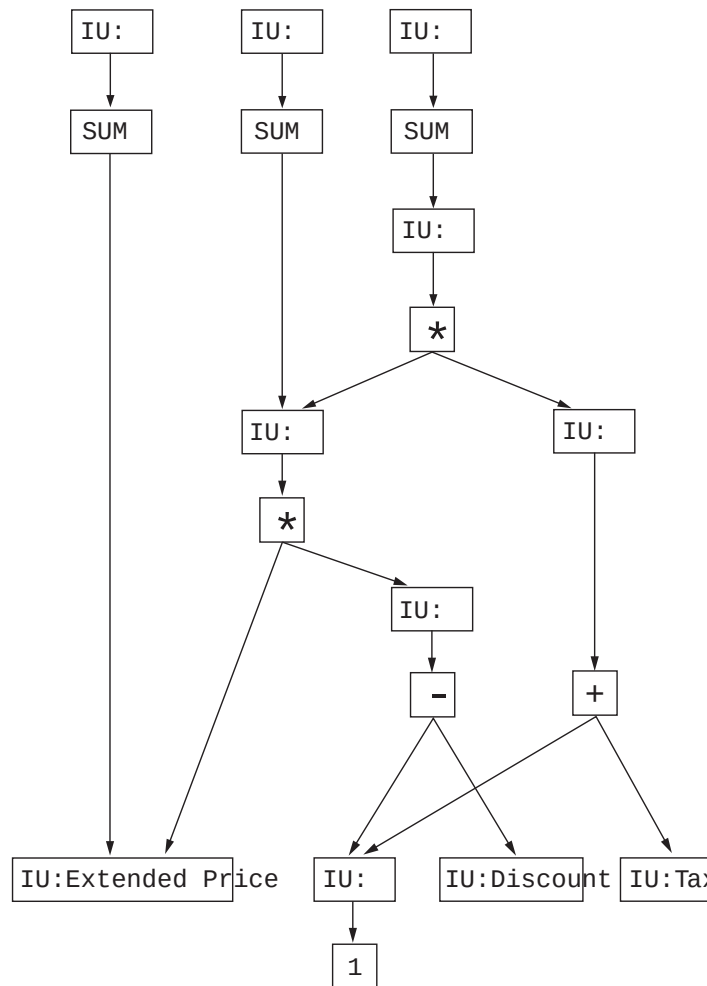


Figure 27.4: Query 1

Consider the semantic analysis of d . Since SQL provides *implicit name look up*, we have to check (formerly analyzed) relations A and B whether they provide an attribute called d . If none of them provides an attribute d , then we must check the next upper SFW-block. If at least one of the relations A or B provides an attribute d , we just check that only one of them provides such an attribute. Otherwise, there would be an unallowed ambiguity. The blockwise look up is handled by block handler. For every newly encountered block (e.g. SFW block), a new block is opened. All identifiers analyzed within that block are pushed into the list of identifiers for that block. In case the query language allows for implicit name resolution, it might also be convenient to push all the attributes of an analyzed relation into the blocks list. The lookup is then performed blockwise. Within every block, we have to check for ambiguities. If the lookup fails, we have to proceed looking up the identifier in the schema. The handling of blocks and lookups is performed by the BlockHandler component attached to the control block of the NFST component (Fig. 25.3).

Another departure from standard semantic analysis are *false aggregates* as provided by SQL.

```
select  avg(age)
from    Students
```

I call *count(age)* a *false aggregate* since a true aggregate function operators on a collection of values and returns a single value. Here, the situation is different. The attribute *age* is of type integer. Hence, for the average function with signature $avg : \{int\} \longrightarrow int$ the semantic analysis would detect a typing error. The result is that we have to treat these false aggregates as special cases. This is (mostly) not necessary for query languages like OQL.

27.7 Translation

The translation step translates the original AST representation into an internal representation. There are as many internal query representations as there are query compiler. They all build on calculus expressions, operator graphs build over some algebra, or tableaux representations [886, 887]. A very powerful representation that also captures the subtleties of duplicate handling is the query graph model (QGM) [689].

The representation we use here is a mixture of a typed algebra and calculus. Algebraic expressions are simple operator trees with algebraic operators like selection, join, etc. as nodes. These operator trees must be correctly typed. For example, we are very picky about whether a selection operator returns a set or a bag. The expression that more resemble a calculus representation than an algebraic expression is the *SFWD block* used in the internal representation. We first clarify our notion of block within the query representation described here and then give an example of an SFWD block. A block is everything that produces variable bindings. For example a SFWD-block that pretty directly corresponds to a SFW-block in SQL or OQL. Other examples of blocks are quantifier expressions and grouping operators. A block has the following ingredients:

- a list of inputs of type collection of tuples¹ (labeled *from*)
- a set of expressions whose top is an IU (labeled *define*)
- a selection predicate of type bool (labeled *where*)

For *quantifier blocks* and *group blocks*, the list of inputs is restricted to length one. The SFWD-block and the grouping block additionally have a projection list (labeled *select*) that indicates which IUs are to be projected (i.e. passed to subsequent operators). Blocks are typed (algebraic) expressions and can thus be mixed with other expressions and algebraic operator trees.

An example of a SFWD-block is shown in Fig. 27.5 where dashed lines indicate the *produced-by* relationship. The graph corresponds to the internal

¹We use a quite general notion of tuple: a tuple is a set of variable (IU) bindings.

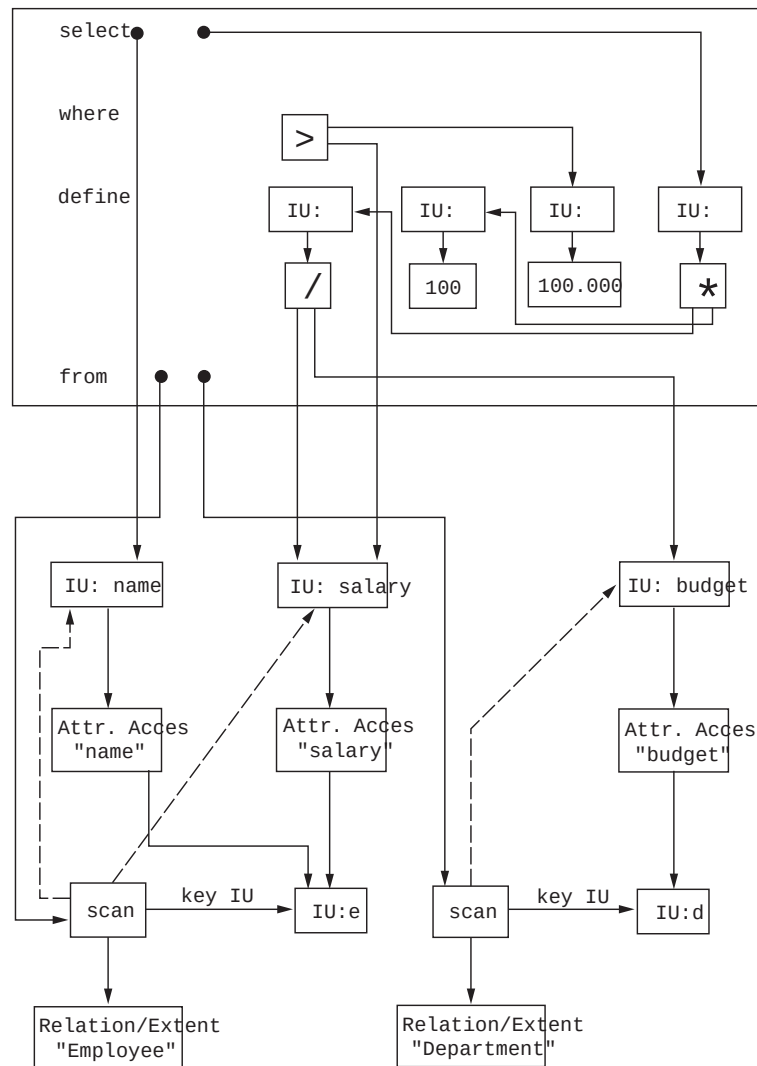


Figure 27.5: Internal representation

representation of our example query. The semantics of a SFWD-block can be described as follows. First, take the cross product of the collections of tuples found in the list of inputs. (If this is not possible, due to dependencies, d-joins have to be used.) Then, for every resulting tuple, compute the bindings for all the IUs mentioned in the *define* clause, apply the selection predicate and return all the bindings for the IUs mentioned in the *select* clause.

Although the SFWD-block looks neat, it lacks certain information that must be represented. This information concerns the role of the entries in the *from* clause and duplicate elimination. Let us start with the latter. There are three views relevant to duplicate processing:

1. the user view: did the user specify distinct?
2. the context view: does the occurrence or elimination of duplicates make

a difference for the query result?

3. the processing view: does the block produce duplicates?

All this information is attached to a block. This information can then be summarized to one of three values representing

- eliminate duplicates
- preserve duplicates
- don't care about duplicates
(The optimizer can feel free to do whatever is more efficient.)

This summary is also attached to every block. Let us illustrate this by a simple example:

```
select  distinct ssno
from    Employee
where   ... and
        exists( select ... from ... where )
```

For the inner block, the user specifies that duplicates are to be preserved. However, duplicates or not does not modify the outcome of *exists*. Hence, the contextual information indicates that the outcome for the inner block is a don't care. The processing view can determine whether the block produces duplicates. If for all the entries in the *from* clause, a key is projected in the *select* clause, then the query does not produce duplicates. Hence, no special care has to be taken to remove duplicates produced by the outer block if we assume that *ssno* is the key of *Employee*.

Now let us consider the annotations for the arguments in the *from* clause. The query

```
select  distinct e.name
from    Employee e, Department d
where   e.dno = d.dno
```

retrieves only *Employee* attributes. Such a query is most efficiently evaluated by a semi-join. Hence, we can add a semi-join (SJ) annotation to the *Department d* clause.

For queries without a **distinct**, the result may be wrong (e.g. in case an employee works in several departments) since a typical semi-join just checks for existence. A special semi-join that preserves duplicates should be used. The according annotation is (SJ,PD). Another annotation occurs whenever an outer-join is used. Outer joins can (in SQL) be part of the *from* clause. Typically they have to be fully parenthesized since outer joins and regular joins not always commute. But under special circumstances, they commute and hence a list of entries in the *from* clause suffices [308]. Then, the entry to be preserved (the

outer part) should be annotated by (OJ). We use (AJ) as the anti-join annotation, and (DJ) for a d-join. To complete annotation, the case of a regular join can be annotated by (J). If the query language also supports all-quantifications, that translate to divisions, then the annotation (D) should be supported.

Since the graphical representation of a query is quite complex, we also use text representations of the result of the NFST phase. Consider the following OQL query:

```
select distinct s.name, s.age, s.supervisor.name, s.supervisor.age
from           s in Student
where         s.gpa > 8 and s.supervisor.age < 30
```

The annotated result (without duplicate annotations) of the normalization and factorization steps is

```
select distinct sn, sa, ssn, ssa
from           s in Student (J)
where         sg > 8 and ssa < 30
define        sn = s.name
                sg = s.gpa
                sa = s.age
                ss = s.supervisor
                ssn= ss.name
                ssa= ss.age
```

Semantic analysis just adds type information (which we never show).

In standard relational query processing multiple entries in the **from** clause are translated into a cross product. This is not always possible in object-oriented query processing. Consider the following query

```
select distinct s
from           s in Student, c in s.courses
where         c.name = "Database"
```

which after normalization yields

```
select distinct s
from           s in Student, c in s.courses
where         cn = "Database"
define        cn = c.name
```

The evaluation of *c in s.courses* is dependent on *s* and cannot be evaluated if no *s* is given. Hence, a cross product would not make much sense. To deal with this situation, the *d-join* has been introduced [189]. It is a binary operator that evaluates for every input tuple from its left input its right input and flattens the result. Consider the algebraic expression given in Fig. 27.6. For every student

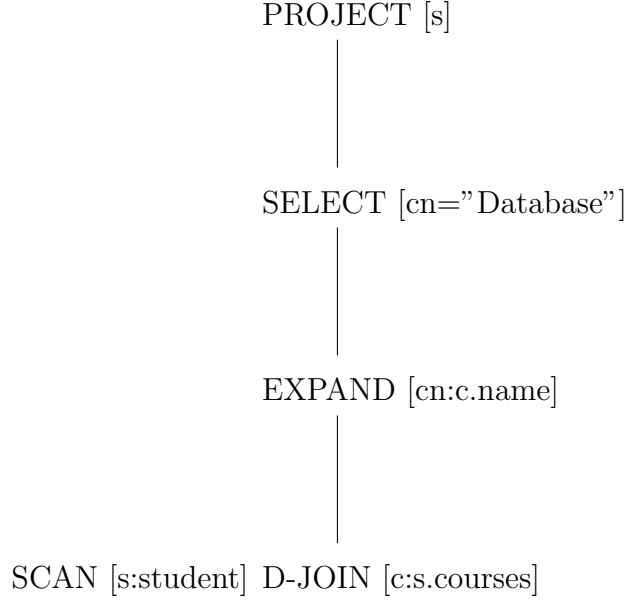


Figure 27.6: An algebraic operator tree with a d-join

s from its left input, the d-join computes the set $s.courses$. For every course c in $s.courses$ an output tuple containing the original student s and a single course c is produced. If the evaluation of the right argument of the d-join is not dependent on the left argument, the d-join is equivalent with a cross product. The first optimization is to replace d-joins by cross products whenever possible.

Queries with a **group by** clause must be translated using the *unary grouping* operator GROUP which we denote by Γ . It is defined as

$$\begin{aligned} \Gamma_{g;\theta A;f}(e) &= \{y.A \circ [g : G] | y \in e, \\ &\quad G = f(\{x | x \in e, x.A\theta y.A\})\} \end{aligned}$$

where the subscripts have the following semantics: (i) g is a new attribute that will hold the elements of the group (ii) θA is the grouping criterion for a sequence of comparison operators θ and a sequence of attribute names A , and (iii) the function f will be applied to each group after it has been formed. We often use some abbreviations. If the comparison operator θ is equal to "=", we don't write it. If the function f is identity, we omit it. Hence, $\Gamma_{g;A}$ abbreviates $\Gamma_{g;=A;id}$.

Let us complete the discussion on internal query representation. We already mentioned algebraic operators like selection and join. These are called logical algebraic operators. Their implementations are called physical algebraic operators. Typically, there exist several possible implementations for a single logical algebraic operator. The most prominent example being the join operator with implementations like Grace join, sort-merge join, nested-loop join etc. All the operators can be modelled as objects. To do so, we extend the expression hierarchy by an algebra hierarchy. Although not shown in Fig 27.7, the algebra class should be a subclass of the expression class. This is not necessary for SQL but is a requirement for more orthogonal query languages like OQL.

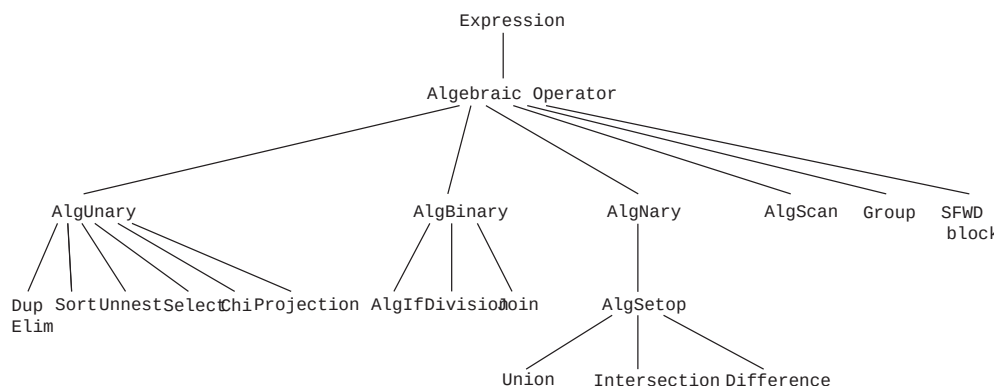


Figure 27.7: Algebra

27.8 Rewrite I

27.9 Plan Generation

27.10 Rewrite II

27.11 Code generation

In order to discuss the tasks of code generation, it is necessary to have a little understanding of the interface to the runtime system that interpretes the execution plan. I have chosen AODB as an example runtime system since this is one I know. The interface to AODB is defined by the AODB Virtual Machine (AVM). For simple operations, like arithmetic operations, comparisons and so on, AVM provides assembler-like operations that are interpreted at runtime. Simple AVM operations work on *registers*. A single register is able to hold the contents of exactly one IU. Additionally, AVM provides physical algebraic operators. These operators take AVM programs (possibly with algebraic operators) as arguments. There is one specialty about AVM programs though. In order to efficiently support factorization of common subexpressions involving arithmetic operations (as needed in aggregations like avg, sum), arithmetic operators in AVM can have two side effects. They are able to store the result of the operation into a register and they are able to add the result of the operation to the contents of another register. This is denoted by the result mode. If the result mode is A, they just add the result to some register, if it is C, they copy (store) the result to some register, if it is B, they do both. This is explored in the code for Query 1 of the TPC-D benchmark (Fig. 1.6).

Code generation has the following tasks. First it must map the physical operators in a plan to the operators of the AVM code. This mapping is a straight forward 1:1 mapping. Then, the code for the subscripts of the operators has to be generated. Subscripts are for example the predicate expressions for the selection and join operators. For grouping, several AVM programs have to be

generated. First program is the *init* program. It initializes the registers that will hold the results for the aggregate functions. For example, for an average operation, the register is initialized with 0. The *advance* program is executed once for every tuple to advance the aggregate computation. For example, for an average operations, the value of some register of the input tuple is added to the result register holding the average. The *finalize* program performs post-processing for aggregate functions. For example for the average, it divides the sum by the number of tuples. For hash-based grouping, the last two programs (see Fig.1.6) compute the hash value of the input register set and compare the group-by attributes of the input registers with those of every group in the hash bucket.

During the code generation for the subscripts factorization of common subexpression has to take place. Another task is register allocation and deallocation. This task is performed by the register manager. It uses subroutines to determine whether some registers are no longer needed. The register manager must also keep track in which register some IU is stored (if at all). Another component used during code generation is a factory that generates new AVM operations. This factory is associated with a table driven component that maps the operations used in the internal query representation to AVM opcodes.

27.12 Bibliography

Chapter 28

Hard-Wired Algorithms

28.1 Hard-wired Dynamic Programming

28.1.1 Introduction

Plan generation is performed block-wise. The goal is to generate a plan for every block. Typically, not all possible plans are generated. For example, the group operator (if necessary for the query) is mostly performed last (see also Sec. ??). This mainly leaves ordering joins and selections as the task of plan generation. A plan is an operator tree whose node consist of physical algebraic operators, e.g. selection, sort-operator, sort-merge and other joins, relation and index scans. The process of plan generation has received a lot of attention. Often, the term query optimization is used synonymous for the plan generation phase.

Figure 28.1 shows a plan for the block

```
select   e.name
from     Employee e, Department d
where    e.dno = d.dno and d.name = "shoe"
```

The bottom level contains two table scans that scan the base tables *Employee* and *Department*. Then, a selection operator is applied to restrict the departments to those named “shoe”. A nested-loop join is used to select those employees that work in the selected departments. The projection restricts the output to the name of the employees, as required by the query block. For such a plan, a *cost function* is used to estimate its cost. The goal of plan generation is to generate the cheapest possible plan. Costing is briefly sketched in Section ??.

The foundation of plan generation are algebraic equivalences. For $e, e_1, e_2, \dots$ being algebraic expressions and p, q predicates, here are some example equiva-

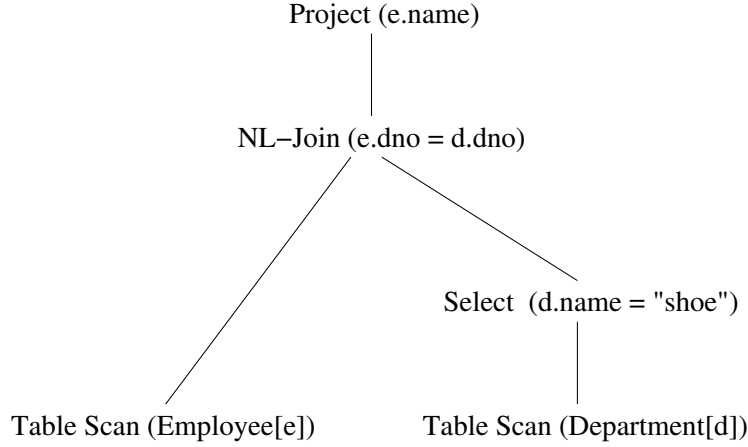


Figure 28.1: A sample execution plan

lences:

$$\begin{aligned}
 \sigma_p(\sigma_q(e)) &\equiv \sigma_q(\sigma_p(e)) \\
 \sigma_p(e_1 \bowtie_q e_2) &\equiv (\sigma_p(e_1)) \bowtie_q e_2 \quad \text{if } p \text{ is applicable to } e_1 \\
 e_1 \bowtie_p e_2 &\equiv e_2 \bowtie_p e_1 \\
 (e_1 \bowtie_p e_2) \bowtie_q e_3 &\equiv e_1 \bowtie_p (e_2 \bowtie_q e_3) \\
 e_1 \cup e_2 &\equiv e_2 \cup e_1 \\
 (e_1 \cup e_2) \cup e_3 &\equiv e_1 \cup (e_2 \cup e_3) \\
 e_1 \cap e_2 &\equiv e_2 \cap e_1 \\
 (e_1 \cap e_2) \cap e_3 &\equiv e_1 \cap (e_2 \cap e_3) \\
 \sigma_p(e_1 \cap e_2) &\equiv \sigma_p(e_1) \cap e_2
 \end{aligned}$$

For more equivalences and conditions that ought to be attached to the equivalences see the appendix ?? . Note that commutativity and associativity of the join operator allow an arbitrary ordering. Since the join operator is the most expensive operation, ordering joins is the most prominent problem in plan generation.

These equivalences are of course independent of the actual implementation of the algebraic operators. The total number of plans equivalent to the original query block is called the *potential search space*. However, not always is the total search space considered. The set of plans equivalent to the original query considered by the plan generator is the *actual search space*. Since the System R plan generator [784], certain restrictions are applied. The most prominent are:

- Generate only plans where selections are pushed down as far as possible.
- Do not consider cross products if not absolutely necessary.
- Generate only left-deep trees.

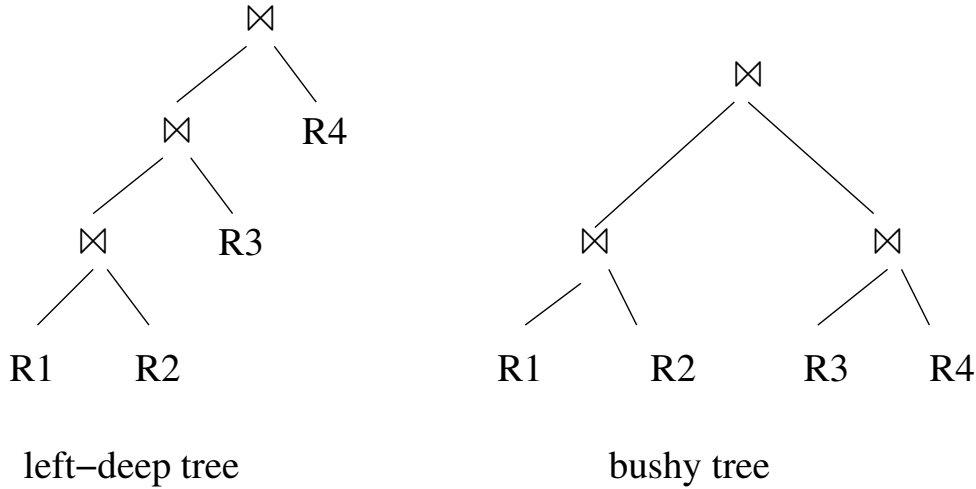


Figure 28.2: Different join operator trees

- If the query block contains a grouping operation, the group operator is performed last.

Some comments are in order. Cross products are only necessary, if the query graph is unconnected where a query graph is defined as follows: the nodes are the relations and the edges correspond to the predicates (*boolean factors*¹) found in the **where** clause. More precisely, the query graph is a hypergraph, since a boolean factor may involve more than two relations. A left-deep tree is an operator tree where the right argument of a join operator always is a base relation. A plan with join operators whose both arguments are derived by other join operators is called *bushy tree*. Figure 28.2 gives an example of a left-deep tree and a bushy tree.

If we take all the above restrictions together, the problem boils down to ordering the join operators or relations. This problem has been studied extensively. The complexity of finding the best (according to some cost function) ordering (operator tree) was first studied by Ibaraki and Kameda [438]. They proved that the problem of generating optimal left-deep trees with no cross products is NP-hard for a special block-wise nested loop join cost function. This cost function applied in the proof is quite complex. Later it was shown that even if the cost function is very simple, the problem remains NP-hard [194]. The cost function (C_{out}) used there just adds up intermediate results sizes. This cost function is interesting in that it is the kernel of many other cost functions and it fulfills the ASI property of which we now the following: If the cost function fulfills the *ASI property* and the query graph is acyclic, then the problem can be solved in polynomial time [438, 520]. Ono and Lohman gave examples that considering cross products can substantially improve performance [653]. However, generating optimal left-deep trees with cross products even for C_{out} makes the problem NP-hard [194]. Generating optimal bushy trees is

¹A *boolean factor* is a disjunction of basic predicates in a conjunctive normal form.

even harder. Even if there is no predicate, that is only cross products have to be used, the problem is NP-hard [768]. This is surprising since generating left-deep trees with cross products as the only operation is very simple: just sort the relations by increasing sizes.

Given the complexity of the problem, there are only two alternatives to generate plans: either explore the total search space or use heuristics. The former can be quite expensive. This is the reason why the above mentioned restrictions to the search space have traditionally been applied. The latter approach risks missing good plans. The best-known heuristics is to join the relation next, that results in the smallest next intermediate result. Estimating the cardinality of such results is discussed in Section ??.

Traditionally, selections were pushed as far down as possible. However, for expensive selection predicates (e.g. user defined predicates, those involving user-defined functions, predicates with subqueries) this does not suffice. For example, if a computer vision application has to compute the percentage of snow coverage for a given set of satellite images, this is not going to be cheap. In fact, it can be more expensive than a join operation. In these cases, pushing the expensive selection down misses good plans. That is why lately research started to take expensive predicates into account. However, some of the proposed solutions do not guarantee to find the optimal plans. Some approaches and their bugs are discussed in [156, 415, 413, 767, 769]. Although we will subsequently give an algorithm that incorporates correct predicate placement, not all plan generators do so. An alternative approach (though less good) is to pull-up expensive predicates in the Rewrite-II-phase.

There are several approaches to explore the search space. The original approach is to use dynamic programming [784]. The dynamic programming algorithm is typically hard-coded. Figure 28.3 illustrates the principle of bottom-up plan generation as applied in dynamic programming. The bottom level consists of the original relations to be joined. The next level consists of all plans that join a subset of cardinality two of the original relations. The next level contains all plans for subsets of cardinality three, and so on. With the advent of new query optimization techniques, new data models, extensible database systems, researchers were no longer satisfied with the hard-wired approach. Instead, they aimed for rule-based plan generation. There exist two different approaches for rule-based query optimizers. In the first approach, the algebraic equivalences that span the search space are used to transform some initial query plan derived from the query block into alternatives. As search strategies either exhaustive search is used or some stochastic approach such as simulated annealing, iterative improvement, genetic algorithms and the like [74, 446, 452, 453, 834, 860, 859, 862]. This is the *transformation-based* approach. This approach is quite inefficient. Another approach is to generate plans by rules in a bottom-up fashion. This is the *generation-based* approach. In this approach, either a dynamic programming algorithm [565] is used or memoization [360]. It is convenient to classify the rules used into logical and physical rules. The logical rules directly reflect the algebraic equivalences. The physical rules or implementation rules transform a logical algebraic operator into a physical algebraic operator. For example, a join-node becomes a nested-

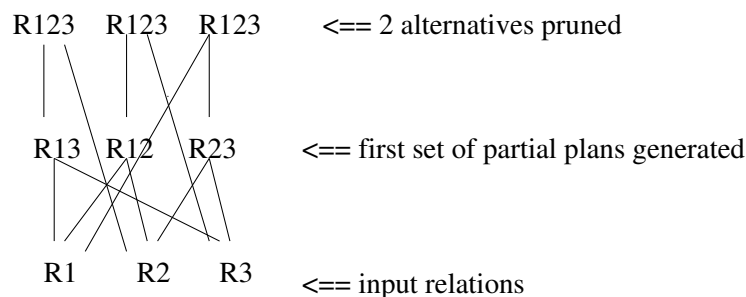


Figure 28.3: Bottom up plan generation

loop join node.

28.1.2 A plan generator for bushy trees

Within the brief discussion in the last subsection, we enumerated plans such that first all 1-relation plans are generated, then all 2-relation plans and so on. This enumeration order is not the most efficient one. Let us consider the simple problem where we have to generate exactly one best plan for the subsets of the n element set of relations to be joined. The empty subset is not meaningful, leaving the number of subsets to be investigated at $2^n - 1$. Enumerating these subsets can be done most efficient by enumerating them in *counting order*. That is, we initialize a n bit counter with 1 and count until have reached $2^n - 1$. The n bits represent the subsets. Note that with this enumeration order, plans are still generated bottom up. For a given subset R of the relations (encoded as the bit pattern a), we have to generate a plan from subsets of this subset (encoded as the bit pattern s). For example, if we only want to generate left-deep trees, then we must consider 1 element subsets and their complements. If we want to generate bushy trees, all subsets must be considered. We can generate these subsets by a very fast algorithm developed by Vance and Maier [898]:

```

s = a & -a;
while(s) {
    s = a & (s - a);
    process(s);
}

```

The meaning of *process(s)* depends on the kinds of plans we generate. If we concentrate on join ordering neglecting selection operations (i.e. pushing them) This step essentially looks up the plans for s and its complement $\bar{s}$ and then joins the plans found there. Lookup is best implemented via an array with s as an index.

28.1.3 A plan generator for bushy trees and expensive selections

Figure 28.4 shows the pseudocode of a dynamic programming algorithm that generates plans with cross products, selections, and joins. It generates optimal bushy trees. Efficient implementation technique for the algorithm can be found in [898, 769]. As input parameters, the algorithm takes a set of relations R and a set of predicates P . The set of relations for which a selection predicate exists is denoted by R_S . We identify relations and predicates that apply to these relations. For all subsets M_k of the relations and subsets P_l of the predicates, an optimal plan is constructed and entered into the table T . The loops range over all M_k and P_l . Thereby, the set M_k is split into two disjoint subsets L and L' , and the set P_l is split into three parts (line 7). The first part (V) contains those predicates that apply to relations in L only. The second part (V') contains those predicates that apply to relations in L' only. The third part (p) is a conjunction of all the join predicates connecting relations in L and L' (line 8). Line 9 constructs a plan by joining the two plans found for the pairs $[L, V]$ and $[L', V']$ in the table T . If this plan has so far the best costs, it is memoized in the table (lines 10-12). Last, different possibilities of not pushing predicates in P_l are investigated (lines 15-19).

Another issue that complicates the application of dynamic programming are certain properties of plans. The most prominent such properties are *interesting orders* [784, 818, 819]. Take a look at the following query:

```

select    d.no, e.name
from      Employee e, Department d
where     e.dno = d.dno
order by d.dno

```

Here, the user requests the result to be order on $d.dno$. Incidentally, this is also a join attribute. During bottom up plan generation, we might think that a Grace hash join is more efficient than a sort-merge join since the cost of sorting the relations is too high. However, the result has to be sorted anyway so that this sort may pay off. Hence, we have to keep both plans. The approach is the following. In the example, an ordering on $d.dno$ is called an interesting order. In general, any order that is helpful for ordering the output as requested by the user, for a join operator, for a grouping operator, or for duplicate elimination is called an *interesting order*. The dynamic programming algorithm is then modified such that plans are not pruned, if they produce different interesting orders.

28.1.4 A plan generator for bushy trees, expensive selections and functions

28.2 Bibliography

```

proc Optimal-Bushy-Tree( $R, P$ )
1  for  $k = 1$  to  $n$  do
2      for all  $k$ -subsets  $M_k$  of  $R$  do
3          for  $l = 0$  to  $\min(k, m)$  do
4              for all  $l$ -subsets  $P_l$  of  $M_k \cap R_S$  do
5                   $best\_cost\_so\_far = \infty$ ;
6                  for all subsets  $L$  of  $M_k$  with  $0 < |L| < k$  do
7                       $L' = M_k \setminus L, V = P_l \cap L, V' = P_l \cap L'$ ;
8                       $p = \bigwedge \{p_{i,j} \mid p_{i,j} \in P, R_i \in V, R_j \in V'\}$ ; //  $p=true$  might hold
9                       $T = (T[L, V] \bowtie_p T[L', V'])$ ;
10                     if  $Cost(T) < best\_cost\_so\_far$  then
11                          $best\_cost\_so\_far = Cost(T)$ ;
12                          $T[M_k, P_l] = T$ ;
13                     fi;
14                 od;
15                 for all  $R \in P_l$  do
16                      $T = \sigma_R(T[M_k, P_l \setminus \{R\}])$ ;
17                     if  $Cost(T) < best\_cost\_so\_far$  then
18                          $best\_cost\_so\_far = Cost(T)$ ;
19                          $T[M_k, P_l] = T$ ;
20                     fi;
21                 od;
22             od;
23         od;
24     od;
25 od;
26 return  $T[R, S]$ ;

```

Figure 28.4: A Dynamic Programming Optimization Algorithm

Chapter 29

Rule-Based Algorithms

29.1 Rule-based Dynamic Programming

The section is beyond the scope of the paper and the reader is referred to the starburst papers, especially [388, 541, 540, 565, 567].

29.2 Rule-based Memoization

This section is beyond the scope of the paper and the reader is referred to the Volcano and Cascade papers [345, 350, 356, 359, 360]. Both optimizer frameworks derived from the earlier Exodus query optimizer generator [343, 357].

29.3 Bibliography

Chapter 30

Example Query Compiler

30.1 Research Prototypes

30.1.1 AQUA and COLA

30.1.2 Black Dahlia II

30.1.3 Epoq

Für das objektorientierte Datenmodell *Encore* [969] wurde die Anfragesprache Equal [802, 801, 803], eine objektorientierte Algebra, die die Erzeugung von Objekten erlaubt, entwickelt. Zur Optimierung von Equal-Algebra-Ausdrücken soll der Optimierer Epoq dienen. Eine Realisierung von Epoq steht noch aus. Konkretisiert wurden jedoch bereits der Architekturansatz [612] und die Kontrolle der Alternativenerzeugung [611] innerhalb dieser Architektur. Einen Gesamtüberblick gibt die Dissertation von Mitchell [610].

Der Architekturvorschlag besteht aus einer generischen Architektur, die an einem Beispielloptimierer konkretisiert wurde [610, 611]. Die elementaren Bausteine der Architektur sind Regionen. Sie bestehen aus einer Kontrollkomponente und wiederum Regionen beziehungsweise Transformationen. Die einfachste Region ist dabei eine Transformation/Regel, die einen Algebraausdruck in einen äquivalenten Algebraausdruck umformt. Jede Region selbst wird wiederum als eine Transformation aufgefaßt. Innerhalb der Architektur werden nun diese Regionen in einer Hierarchie oder auch einem gerichteten azyklischen Graphen, organisiert. Abbildung 30.1 zeigt eine solche Beispielorganisation. Regionen selbst können bis auf die Kontrolle als Module im Sinne von Sciore und Sieg [780] aufgefaßt werden. Sie weisen sehr ähnliche Parameter und Schnittstellen auf. Während jedoch bei Sciore und Sieg die Kontrollstrategie eines Moduls aus einer festen Menge von gegebenen Kontrollstrategien ausgewählt werden muß, kann sie hier freier spezifiziert werden.

Unabhängig davon, ob die Transformationen einer Region wiederum Regionen sind oder elementare Transformationen, wird ihre Anwendung einheitlich von der Kontrolle der Region bestimmt. Die Aufgabe dieser Kontrolle besteht darin, eine Folge von Transformationen zu finden, die die gegebene Anfrage in eine äquivalente überführen. Sinngebend ist hierbei ein gewisses Ziel, das es zu erreichen gilt. Beispielsweise kann dieses Ziel lauten: Optimierte eine

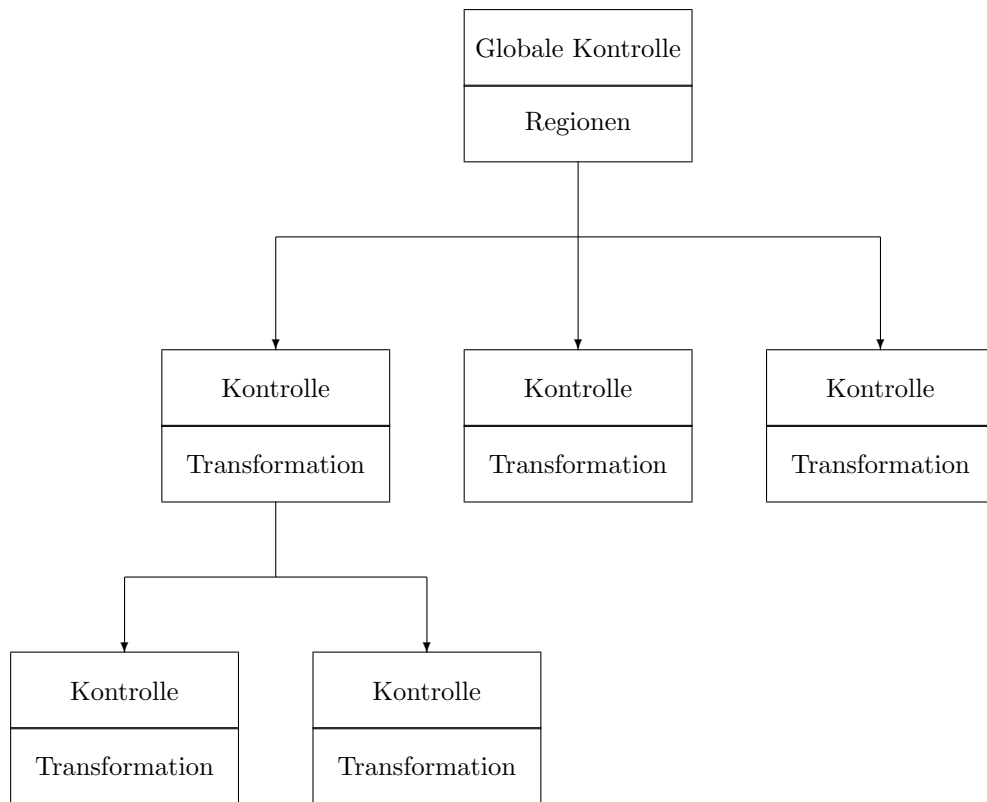


Figure 30.1: Beispiel einer Epoq-Architektur

geschachtelte Anfrage. Um dieses Ziel zu erreichen, sind zwei grobe Schritte notwendig. Zunächst muß die Anfrage entschachtelt werden und als nächstes die entschachtelte Anfrage optimiert werden. Man sieht sofort, daß die Folge der Transformationen, die die Kontrolle auszuwählen hat, sowohl von den Eigenschaften der Anfrage selbst wie auch vom zu erfüllenden Ziel abhängt. Basierend auf dieser Beobachtung wird die Kontrolle nicht als Suchfunktion implementiert, sondern es wird das Planungsparadigma zur Realisierung gewählt. Die Kontrolle selbst wird mit Hilfe eines Satzes von Regeln spezifiziert, die aus Vorbedingung und Aktion bestehen.

Da es nicht möglich ist, im Vorfeld einen Plan, also eine Sequenz von Transformationen/Regionen, zu erstellen, der in garantierter Weise das Ziel erreicht, wird erlaubt, daß die Ausführung einer Transformation/Region fehlschlägt. In diesem Fall kann dann ein alternativer Plan erzeugt werden, der aber auf dem bisher Erreichten aufsetzt. Hierzu werden die Regeln, die die Kontrolle spezifizieren in Gruppen eingeteilt, wobei jeder Gruppe eine einheitliche Vorbedingung zugeordnet ist. Zu jeder Gruppe gehört dann eine Sequenz von Aktionen, die der Reihe nach ausprobiert werden. Schlägt eine vorangehende Aktion fehl, so wird die nächste in der Reihe der Aktionen angewendet. Schlagen alle Ak-

tionen fehl, so schlägt auch die Anwendung der Region fehl.

Jede Aktion selbst ist wiederum eine Sequenz von elementaren Aktionen. Jede dieser elementaren Aktionen ist entweder die Anwendung einer elementaren Transformation, der Aufruf einer Region oder der rekursive Aufruf des Planers mit einem neuformulierten Ziel, dessen Teilplan dann an entsprechender Stelle in die Aktion eingebaut wird.

Die Erweiterbarkeit dieses Ansatzes um neue Regionen scheint einfach möglich, da die Schnittstelle der Regionen genormt ist. Probleme könnte es lediglich bei den Kontrollstrategien geben, da nicht klar ist, ob die benutzte Regelsprache mächtig genug ist, um alle wünschenswerten Kontrollstrategien zu verwirklichen.

Die Frage, ob die einzelnen Komponenten des Optimierers, also die Regionen, evaluiert werden können, ist schwierig zu beantworten. Dafür spricht jedoch, daß jede Region in einem gewissen Kontext aufgerufen wird, also zur Erreichung eines bestimmten Zieles bei der Optimierung einer Anfrage mit ebenso bestimmten Eigenschaften. Beurteilen kann man daher die Erfolgsquote einer Region innerhalb ihrer verschiedenen Anwendungen. Da jede Region lediglich eine Alternative erzeugen darf, aufgrund des *eine Region ist eine Transformation*-Paradigmas, ist schwer zu sagen, in wieweit sich die durch die beschriebene Bewertung gewonnene Information zur Verbesserung der Regionen oder des Gesamtoptimierers einsetzen läßt.

Da auch hier der transformierende Ansatz zugrunde liegt, treffen die bereits diskutierten Probleme auch für den Optimierer für Straube zu.

Einen stetigen Leistungsabfall könnte man durch die Realisierung von alternativen Regionen erreichen, indem man ein Ziel *OptimiereSchnell* einführt, das dann entsprechend weniger sorgfältige, aber schnellere Regionen aufruft. Vorhersagen über der Güte (bei gegebener Optimierungszeit) scheinen aber schwerlich möglich.

30.1.4 Ereq

A primary goal of the EREQ project is to define a common architecture for the next generation of database managers. This architecture now includes

- * the query language OQL (a la ODMG), * the logical algebra AQUA (a la Brown), and * the physical algebra OPA (a la OGI/PSU).

It also includes

- * software to parse OQL into AQUA (a la Bolo)

and query optimizers:

- * OPT++ (Wisconsin), * EPOQ (Brown), * Cascades (PSU/OGI), and * Reflective Optimizer (OGI).

In order to test this architecture, we hope to conduct a "bakeoff" in which the four query optimizers will participate. The primary goal of this bakeoff is to determine whether optimizers written in different contexts can accommodate the architecture we have defined. Secondly, we hope to collect enough performance statistics to draw some conclusions about the four optimizers, which have been written using significantly different paradigms.

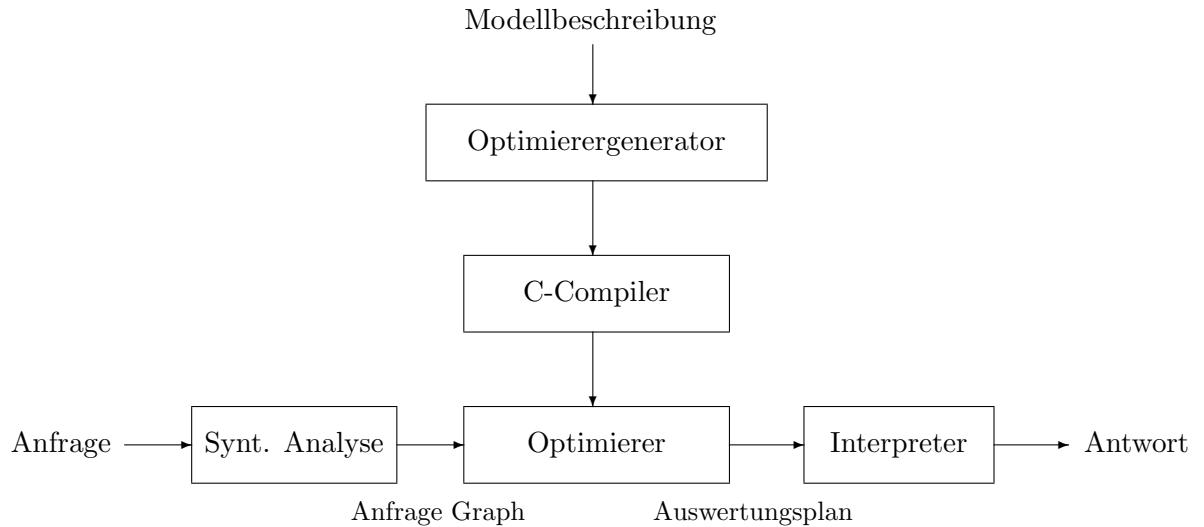


Figure 30.2: Exodus Optimierer Generator

At present, OGI and PSU are testing their optimizers on the bakeoff queries. Here is the prototype bakeoff optimizer developed at OGI. This set of Web pages is meant to report on the current progress of their effort, and to define the bakeoff rules. Please email your suggestions for improvement to Leo Fegaras fegaras@cse.ogi.edu. Leo will route comments to the appropriate author.

<http://www.cse.ogi.edu/DISC/projects/ereq/bakeoff/bakeoff.html>

30.1.5 Exodus/Volcano/Cascade

Im Rahmen des Exodus-Projektes wurde ein Optimierergenerator entwickelt [357]. Einen Überblick über den Exodus-Optimierergenerator gibt Abbildung 30.2. Ein Model description file enthält alle Angaben, die für einen Optimierer nötig sind. Da der Exodus-Optimierergenerator verschiedene Datenmodelle unterstützen soll, enthält dieses File zunächst einmal die Definition der verfügbaren *Operatoren* und *Methoden*. Dabei werden mit *Operatoren* die Operatoren der logischen Algebra bezeichnet und mit *Methoden* diejenigen der physischen Algebra, also die Implementierungen der Operatoren. Das Model description file enthält weiterhin zwei Klassen von Regeln. *Transformationen* basieren auf algebraischen Gleichungen und führen einen Operatorbaum in einen anderen über. *Implementierungsregeln* wählen für einen gegebenen Operator eine Methode aus. Beide Klassen von Regeln haben einen linken Teil, der mit einem Teil des aktuellen Operatorgraphen übereinstimmen muß, einen rechten Teil, der den Operatorgraphen nach Anwendung der Regel beschreibt, und eine Bedingung, die erfüllt sein muß, damit die Regel angewendet werden kann. Während die linke und rechte Seite der Regel als Muster angegeben werden, wird die Bedingung durch C-Code beschrieben. Auch für die Transformation lassen sich C-Routinen verwenden. In einer abschließenden Sektion des Model description files finden sich dann die benötigten C-Routinen.

Aus dem Model description file wird durch den Optimierergenerator ein C-Programm erzeugt, das anschließend übersetzt und gebunden wird. Das Ergebnis ist dann der Anfrageoptimierer, der in der herkömmlichen Art und Weise verwendet werden kann. Es wurde ein übersetzender Ansatz für die Regeln gewählt und kein interpretierender, da in einem von den Autoren vorher durchgeführten Experiment sich die Regelinterpretation als zu langsam erwiesen hat.

Die Regelarbeitung im generierten Optimierer verwaltet eine Liste OPEN, in der alle anwendbaren Regeln gehalten werden. Ein Auswahlmechanismus bestimmt dann die nächste anzuwendende Regel und entfernt sie aus OPEN. Nach deren Anwendung werden die hierdurch ermöglichten Regelanwendungen detektiert und in OPEN vermerkt. Zur Implementierung des Auswahlmechanismus werden sowohl die Kosten eines aktuellen Ausdrucks als auch eine Abschätzung des Potentials einer Regel in Betracht gezogen. Diese Abschätzung des Potentials berechnet sich aus dem Quotienten der Kosten für einen Operatorbaum vor und nach Regelanwendung für eine Reihe von vorher durchgeführten Regelanwendungen. Mit Hilfe dieser beiden Angaben, den Kosten des aktuellen Operatorgraphen, auf den die Regel angewendet werden soll, und ihres Potentials können dann Abschätzungen über die Kosten des erzeugten Operatorgraphen berechnet werden. Die Suchstrategie ist Hill climbing.

Der von den Autoren vermerkte Hauptnachteil ihres Optimierergenerators, den sie jedoch für alle transformierenden regelbasierten Optimierer geltend machen, ist die Unmöglichkeit der Abschätzung der absoluten Güte eines Operatorbaumes und des Potentials eines Operatorbaumes im Hinblick auf zukünftige Optimierungen. Dadurch kann niemals abgeschätzt werden, ob der optimale Operatorbaum bereits erreicht wurde. Erst nach Generierung aller Alternativen ist die Auswahl des optimalen Operatorbaumes möglich. Weiter bedauern es die Autoren, daß es nicht möglich ist, den A*-Algorithmus als Suchfunktion zu verwenden, da die Abschätzung des Potentials oder der Distanz zum optimalen Operatorgraphen nicht möglich ist.

Zumindest kritisch gegenüberstehen sollte man auch der Bewertung einzelner Regeln, da diese, basierend auf algebraischen Gleichungen, von zu feiner Granularität sind, als daß eine allgemeine Bewertung möglich wäre. Die erfolgreiche Verwendung des Vertauschens zweier Verbundoperationen in einer Anfrage bedeutet noch lange nicht, daß diese Vertauschung auch in der nächsten Anfrage die Kosten verringert. Die Hauptursache für die kritische Einstellung gegenüber dieser recht ansprechenden Idee ist, daß eine Regelanwendung zu wenig Information/Kontext berücksichtigt. Würde dieses Manko beseitigt, wären Regeln also von entschieden größerer Granularität, so erschiene dieser Ansatz vielversprechend. Ein Beispiel wäre eine Regel, die alle Verbundoperationen gemäß einer gegebenen Heuristik ordnet, also ein komplexer Algorithmus, der mehr Wissen in seine Entscheidungen einbezieht.

Graefe selbst führt einige weitere Nachteile des Exodus-Optimierergenerators an, die dann zur Entwicklung des Volcano-Optimierergenerators führten [359, 360]. Unzureichend unterstützt werden

- nicht-triviale Kostenmodelle,

- Eigenschaften,
- Heuristiken und
- Transformationen von Subskripten von algebraischen Operatoren in algebraische Operatoren.

Der letzte Punkt ist insbesondere im Bereich der Objektbanken wesentlich, um beispielsweise Pfadausdrücke in eine Folge von Verbundoperationen umwandeln zu können.

Im Volcano-Optimierergenerator werden algebraische Ausdrücke wieder in einen Operatorbaum umgewandelt. Wie im Exodus-Optimierergenerator wird der Optimierer wieder mit einer Menge von transformierenden und implementierenden Regeln beschrieben. Die Nachteile des transformierenden Ansatz werden somit geerbt. Eine Trennung in zwei Phasen, wie bei vielen Optimierern anzutreffen, ist für den Volcano-Optimierergenerator nicht notwendig. Der Entwickler des Optimierers hat die Freiheit, die Phasen selbst festzulegen. Die Probleme, die sonst bei der Kopplung der algebraischen mit der nicht-algebraischen Optimierung auftreten, können also vermieden werden. Die Behandlung der Eigenschaften erfolgt zielorientiert. Die in der Anfrage geforderten Eigenschaften (bspw. Sortierung), werden der Suchfunktion als Parameter übergeben, damit gezielt Pläne erstellt werden, die diese erfüllen. Wenn ein Operator oder eine Methode eingebaut wird, so wird darauf geachtet, daß diese noch nicht erfüllten Eigenschaften durch den Operator oder die Methode erzielt werden. Die geforderten Eigenschaften dienen wieder als Zielbeschreibung für die nachfolgenden Aufrufe der Suchfunktion. Zu diesen Eigenschaften gehören auch Kostengrenzen, mit denen die Suchfunktion dann einen Branch-and-bound-Algorithmus implementiert. Bevor ein Plan für einen algebraischen Ausdruck generiert wird, wird in einer Hash-Tabelle nachgeschaut, ob ein entsprechender Ausdruck mit den geforderten Eigenschaften bereits existiert. Dadurch wird Doppeltarbeit vermieden. Bei beiden Optimierergeneratoren werden die Forderungen nach stetigem Leistungsabfall, früher Bewertung von Alternativen und Evaluierbarkeit einzelner Komponenten nicht erfüllt.

30.1.6 Freytags regelbasierte System R-Emulation

[295] zeigt, wie man mit Hilfe eines regelbasierten Ansatzes den Optimierer von System R [784] emulieren kann. Die Eingabe besteht aus einem Lisp-ähnlichen Ausdruck:

```
(select <proj-list>
      <sel-pred-list>
      <join-pred-list>
      <table-list>)
```

Die Projektionsliste besteht aus Attributspezifikationen der Form

```
<rel-name>.<attr-name>
```

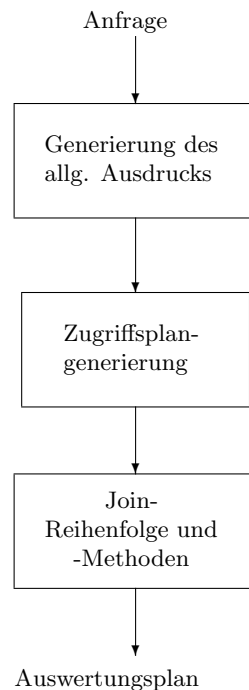


Figure 30.3: Organisation der Optimierung

Diese werden auch für die Selektionsprädikate und Joinprädikate verwendet. Die Algebra beinhaltet sowohl Operatoren der logischen als auch der physischen Algebra. Im einzelnen gibt es Scan-, Sort-, Projektions, Verbundoperatoren in einer logischen und verschiedenen physischen Ausprägungen. Die Erzeugung der Auswertungspläne wird in verschiedene Schritte unterteilt, die wiederum in Teilschritte zerlegt sind (siehe Abb. 30.3). Zunächst erfolgt die Übersetzung in die logische Algebra. Hier werden Scan-Operatoren um die Relationen gebaut und Selektionen, die nur eine Relation betreffen, in die Scan-Operatoren eingebaut. Der zweite Schritt generiert Zugriffspläne, indem der Scan-Operator durch einen einfachen File-Scan (FSCAN) ersetzt wird, oder falls möglich, durch einen Index-Scan (ISCAN). Der dritte Schritt generiert zunächst verschiedene Verbund-Reihenfolgen und bestimmt anschließend die Verbund-Methoden. Sie in System R wird zwischen Sort-merge- und Nested-loop-join unterschieden.

Es werden keinerlei Aussagen über die Auswahl einer Suchstrategie gemacht. Ziel ist es vielmehr, durch die Modellierung des System R Optimierers mit Hilfe eines Regelsystems die prinzipielle Brauchbarkeit des regelbasierten Ansatzes nachzuweisen.

30.1.7 Genesis

Das globale Ziel des Genesisprojektes [59, 60, 61, 64] war es, die gesamte Datenbanksoftware zu modularisieren und eine erhöhte Wiederverwendbarkeit von Datenbankmodulen zu erreichen. Zwei Teilziele wurden hierbei angestrebt:

1. Standardisierung der Schnittstellen und
2. Formulierung der Algorithmen unabhängig von der DBMS-Implementierung.

Wir interessieren uns hier lediglich für die Erreichung der Ziele beim Bau von Optimierern [57, 62].

Die Standardisierung der Schnittstellen wird durch eine Verallgemeinerung von Anfragegraphen erreicht. Die Algorithmen selbst werden durch Transformationen auf Anfragegraphen beschrieben. Man beachte, daß dies nicht bedeutet, daß die Algorithmen auch durch Transformationsregeln implementiert werden. Regeln werden lediglich als Beschreibungsmittel benutzt, um die Natur der Wiederverwendbarkeit von Optimierungsalgorithmen zu verstehen.

Die Optimierung wird in zwei Phasen eingeteilt, die Reduktionsphase und die Verbundphase. Die Reduktionsphase bildet Anfragegraphen, die auf nicht reduzierten Datenmengen arbeiten, auf solche ab, die auf reduzierten Datenmengen arbeiten. Die Reduktionsphase orientiert sich also deutlich an den Heuristiken zum Durchschieben von Selektionen und Projektionen. Die zweite Phase bestimmt Verbundordnungen. Damit ist die in den Papieren beschriebene Ausprägung des Ansatzes sehr konservativ in dem Sinne, daß nur klassische Datenmodelle betrachtet werden. Eine Anwendung der Methodik auf objektorientierte oder deduktive Datenmodelle steht noch aus.

Folglich lassen sich nur die existierenden klassischen Optimierungsansätze mit diesen Mitteln hinreichend gut beschreiben. Ebenso lassen sich die existierenden klassischen Optimierer mit den vorgestellten Mitteln als Zusammensetzung der ebenfalls im Formalismus erfaßten Algorithmen beschreiben. Die Zusammensetzung selbst wird mit algebraischen Termersetzungen beschrieben. Durch neue Kompositionsregeln lassen sich dann auch neue Optimierer beschreiben, die andere Kombinationen von Algorithmen verwenden.

Durch die formale, implementierungsunabhängige Beschreibung sowohl der einzelnen Optimierungsalgorithmen als auch der Zusammensetzung eines Optimierers wird die Wiederverwendbarkeit von bestehenden Algorithmen optimal unterstützt. Wichtig dabei ist auch die Verwendung der standardisierten Anfragegraphen. Dieser Punkt wird allerdings aufgeweicht, da auch vorgesehen ist, verschiedene Darstellungen von Anfragegraphen zu verwenden [60]. Hierdurch wird die Wiederverwendung von Implementierungen von Optimierungsalgorithmen natürlich in Frage gestellt, da diese üblicherweise nur auf einer bestimmten Darstellung der Anfragegraphen arbeiten.

Wenn neue Optimierungsansätze entwickelt werden, so lassen sie sich ebenfalls im vorgestellten Formalismus beschreiben. Gleiches gilt auch für neue Indexstrukturen, da auch diese formal beschrieben werden [58, 63]. Nicht abzusehen ist, in wieweit der standardisierte Anfragegraph Erweiterungen standhält. Dies ist jedoch kein spezifisches Problem des Genesisansatzes, sondern gilt für alle Optimierer. Es ist noch offen, ob es gelingt, die Optimierungsalgorithmen so zu spezifizieren und zu implementieren, daß sie unabhängig von der konkreten Darstellung oder Implementierung der Anfragegraphen arbeiten. Der objektorientierte Ansatz kann hier nützlich sein. Es erhebt sich jedoch die Frage, ob bei Einführung eines neuen Operators die bestehenden Algorithmen so implemen-

tierbar sind, daß sie diesen ignorieren können und trotzdem sinnvolle Arbeit leisten.

Die Beschränkung auf zwei Optimierungsphasen, die Reduktions- und die Verbundphase, ist keine Einschränkung, da auch sie mittels Termersetzungsregeln festgelegt wurde, und somit leicht geändert werden kann.

Da die Beschreibungen des Optimierers und der einzelnen Algorithmen unabhängig von der tatsächlichen Implementierung sind, sind auch die globale Kontrolle des Optimierers und die lokalen Kontrollen der einzelnen Algorithmen voneinander losgelöst. Dieses ist eine wichtige Forderung, um Erweiterbarkeit zu erreichen. Sie wird oft bei regelbasierten Optimierern verletzt und schränkt somit deren Erweiterbarkeit ein.

Die Evaluierbarkeit, die Vorhersagbarkeit und die frühe Bewertung von Alternativen sind mit dem vorgestellten Ansatz nicht möglich, da die einzelnen Algorithmen als Transformationen auf dem Anfragegraphen aufgefaßt werden. Dieser Nachteil gilt jedoch nicht allein für den hier vorgestellten Genesisansatz, sondern generell für alle bis auf einen Optimierer. Es ist allerdings nicht absehbar, ob dieser Nachteil aus dem verwendeten Formalismus resultiert oder lediglich aus deren Konkretisierung bei der Modellierung bestehender Optimierer. Es ist durchaus möglich, daß der Formalismus mit leichten Erweiterungen auch andere Ansätze, insbesondere den generierenden, beschreiben kann.

Insgesamt handelt es sich beim Genesisansatz um einen sehr brauchbaren Ansatz. Leider hat er, im Gegensatz zur Regelbasierung, nicht genug Widerhall gefunden. Er hat höchst wahrscheinlich mehr Möglichkeiten, die Anforderungen zu erfüllen, als bisher ausgelotet wurde.

30.1.8 GOMbgo

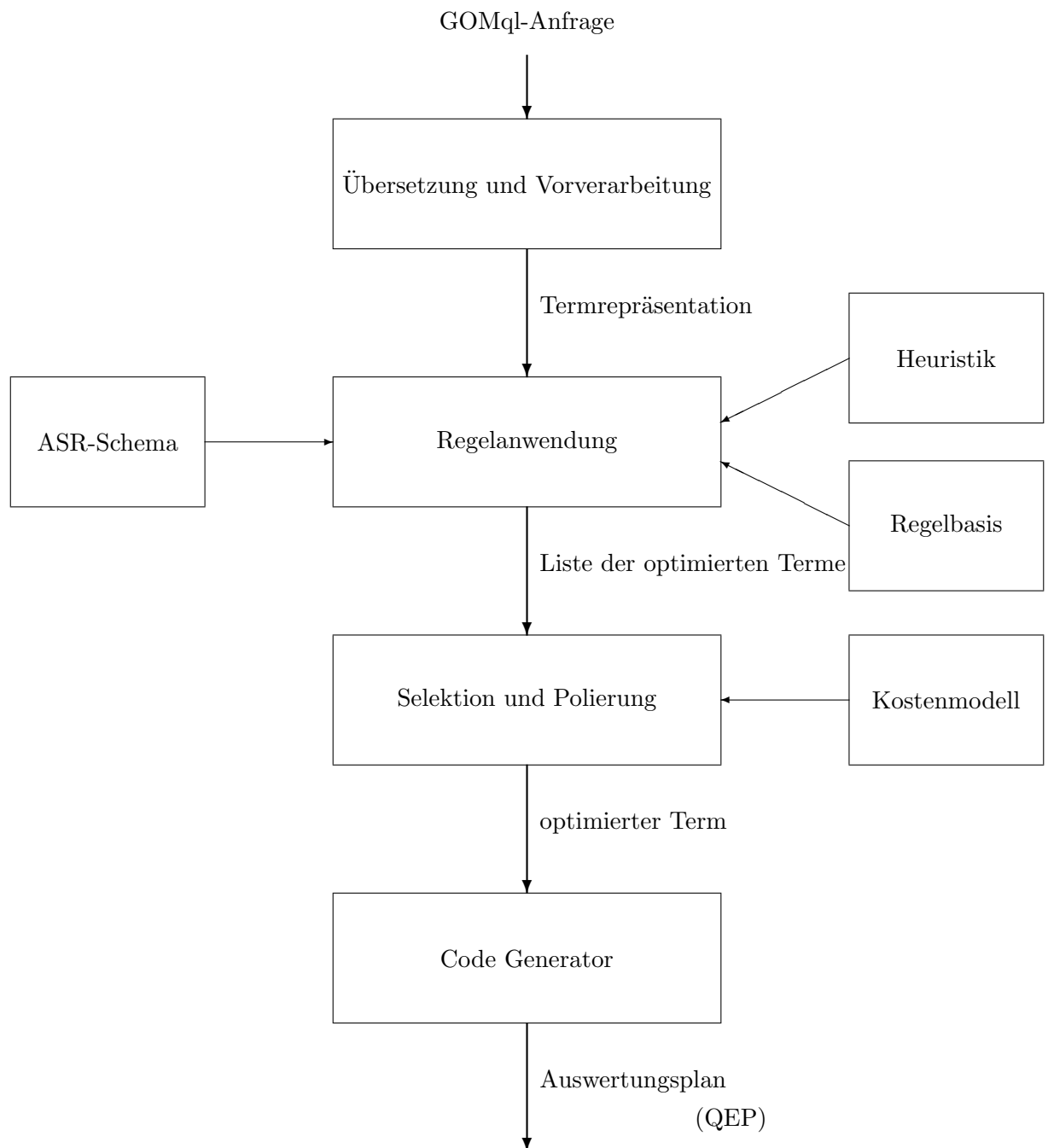


Figure 30.4: Ablauf der Optimierung

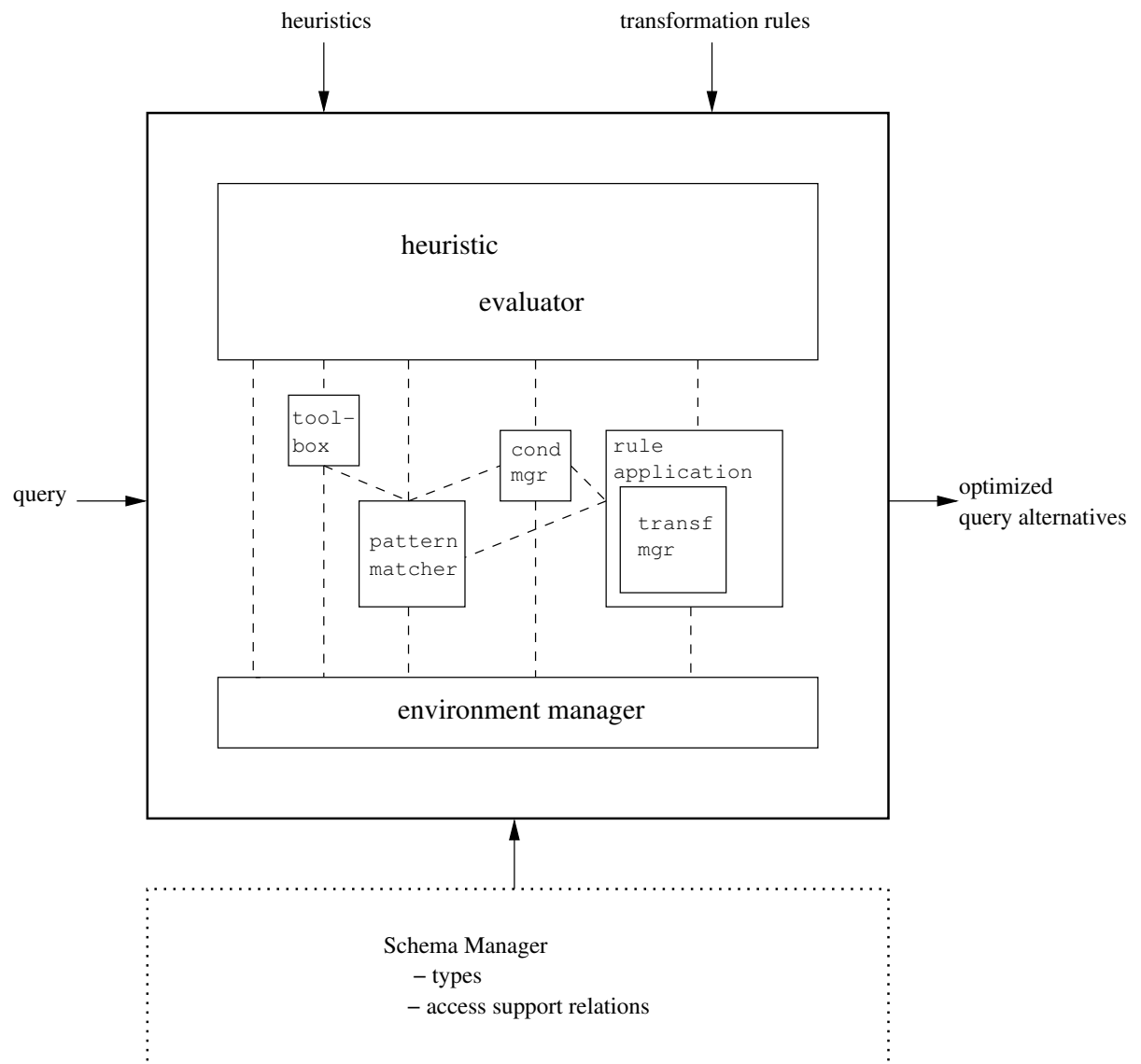


Figure 30.5: Architektur von GOMrbo

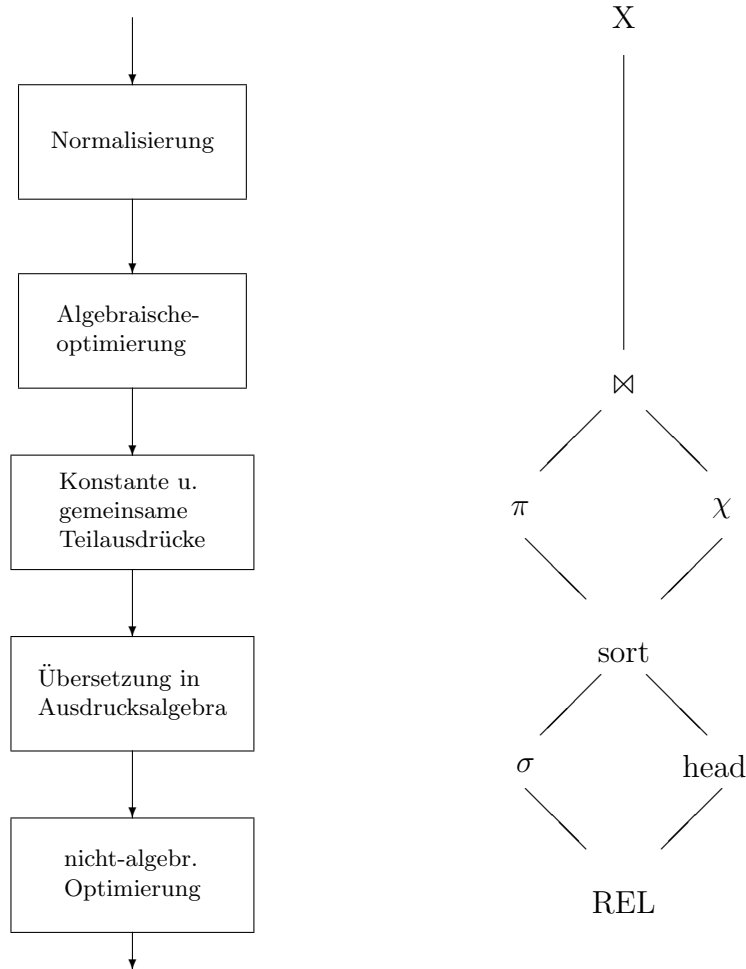


Figure 30.6: a) Architektur des Gral-Optimierers; b) Operatorhierarchie nach Kosten

30.1.9 Gral

Gral ist ein erweiterbares geometrisches Datenbanksystem. Der für dieses System entwickelte Optimierer, ein regelbasierter Optimierer in Reinkultur, erzeugt aus einer gegebenen Anfrage in fünf Schritten einen Ausführungsplan (s. Abb. 30.6 a) [67]. Die Anfragesprache ist gleich der verwendeten deskriptiven Algebra (*descriptive algebra*). Diese ist eine um geometrische Operatoren erweiterte relationale Algebra. Als zusätzliche Erweiterung enthält sie die Möglichkeit, Ausdrücke an Variablen zu binden. Ein Auswertungsplan wird durch einen Ausdruck der Ausführungsalgebra (*executable algebra*) dargestellt. Die Ausführungsalgebra beinhaltet im wesentlichen verschiedene Implementierungen der deskriptiven Algebra und Scan-Operationen. Die Trennung zwischen deskriptiver Algebra und Ausführungsalgebra ist strikt, das heißt, es kommen keine gemischten Ausdrücke vor (außer während der expliziten Konvertierung (Schritt 4)).

Die Schritte 1 und 3 sind durch feste Algorithmen implementiert. Während

der Normalisierung (Schritt 1) werden Variablenvorkommen durch die an sie gebundenen Ausdrücke ersetzt. Dies ist notwendig, um das Optimierungspotential vollständig erschließen zu können. Schritt 3 führt für konstante Ausdrücke Variablen ein. Die entspricht der Entschachtelung von Anfragen vom Typ N und A (s. Kapitel ?? und [494]). Die Behandlung von gemeinsamen Teilausdrücken ist noch nicht implementiert, aber für Schritt 3 vorgesehen.

Die Schritte 2, 4 und 5 sind regelbasiert. Zur Formulierung der Regeln wird eine Regelbeschreibungssprache (*rule description language*) verwendet. Die Beschreibungen der Regeln werden in einer Datei abgelegt. Innerhalb der Datei werden Regeln zu Gruppen (*sections*) zusammengefaßt. Diese Gruppen werden nacheinander angewandt. Daraus ergeben sich auch für einen Schritt mehrere kleinere Schritte. Beispielsweise ist der Schritt 2 im OPTEX-Optimierer für Gral in vier Teilschritte unterteilt:

1. Dekomposition von Selektionen mit komplexen Selektionsprädikaten in eine Folge von Selektionen mit einfachen Selektionsprädikaten und Zerlegung von Verbundoperationen in eine Folge von Selektionen und Kreuzprodukten.
2. Eigentlicher IMPROVING Schritt (siehe unten).
3. Teilausdrücke bestehend aus einer Selektion und einem unmittelbar folgenden Kreuzprodukt werden in Verbundoperationen umgewandelt.
4. Bestimmung einer Ordnung zwischen den Verbundoperationen und Kreuzprodukten. Dabei werden Kreuzprodukte zum Schluß ausgeführt und kleine Relationen zuerst verbunden.

Jeder Gruppe wird eine von drei in Gral implementierten Suchstrategien zugeordnet.

STANDARD Führt solange alle Regeln einer Gruppe aus, bis keine Regel mehr anwendbar ist. Es werden keine Vorkehrungen getroffen, um Endlosschleifen zu verhindern. Die Regeln müssen also dementsprechend formuliert werden. Diese Strategie kann für Schritte 2 und 5 verwendet werden.

IMPROVING Diese Strategie unterstützt algebraische Optimierung in der deskriptiven Algebra (Schritt 2). Das Ziel ist hierbei eine gute Ordnung der algebraischen Operatoren zu erlangen. Hierzu wird eine partielle Ordnung der algebraischen Operatoren gemäß ihrer Kosten definiert (s. Abb. 30.6 b) für ein Beispiel). Die IMPROVING-Strategie versucht dann die hierdurch definierte Ordnung in einem gegebenen Ausdruck zu erreichen. Hierzu wird sie zunächst rekursiv auf alle Teilausdrücke eines Ausdrucks angewendet. Regeln zur Umformung werden dann angewendet, wenn dadurch eine höhere Kohärenz der Operatorfolge im Ausdruck mit der der Operatorkostenhierarchie erreicht werden kann. Dies entspricht einem Bubble-sort auf dem Ausdruck. Ausdrücke mit der kleinsten Anzahl von *runs* werden bevorzugt. Dabei ist ein *run* eine Folge von Operatoren innerhalb des zu optimierenden Ausdrucks, dessen Operatoren gemäß der Operatorkostenhierarchie geordnet sind.

TRANSLATION Regelgruppen mit dieser Strategie werden während der Übersetzung von der deskriptiven Algebra in die Ausführungsalgebra angewendet (Schritt 4). Jede Regel beschreibt dabei die Übersetzung eines einzelnen deskriptiven Operators in einen Ausdruck der Ausführungsalgebra, also einen, der keine deskriptiven Operatoren enthalten darf. Die Übersetzung erfolgt lokal. Für Parameter, also beispielsweise Selektions- und Verbundprädikate, können Regeln angegeben werden, die einen Suchraum für die Reorganisation des Parameters erlauben. Hiermit kann man beispielsweise alle Permutationen einer Konjunktion erzeugen. Die Suchstrategie für die Parameterbestimmung ist erschöpfend und trägt Vorsorge, daß keine Zyklen auftreten. Eine Auswahl kann mittels des *valuation*-Eintrags in den Regeln getroffen werden. Dieser kann beispielsweise Kosten repräsentieren. Dementsprechend werden dann Regeln mit der kleinsten *valuation* bevorzugt. Jede für einen Parameter generierte Darstellung wird übersetzt.

Die Syntax für eine Regel ist

```
specification
definition
RULE
    pattern
    → result1 valuation1 if condition1
    ...
    → resultn valuationn if conditionn
```

wobei

specification von der Form

$$\text{SPEC spec}_1, \dots, \text{spec}_n$$

ist. Dabei sind die spec_i Range-Spezifikationen wie beispielsweise *op_i in < OpSet >*.

definition Variablen definiert (bspw. für Attributsequenzen). In Gral existieren verschiedene Sorten von Variablen für Attribute, Operationen, Relationen etc.

pattern ein Muster in Form eines Ausdrucks ist, der Variablen und Konstanten enthalten kann. Der Ausdruck kann ein Ausdruck der deskriptiven Algebra oder der Ausführungsalgebra sein.

condition_i eine Bedingung ist. Diese Bedingung ist ein allgemeiner boolescher Ausdruck. Spezielle Prädikate wie *ExistsIndex* (existiert ein Index für eine Relation?) werden von Gral zur Verfügung gestellt.

result_i wiederum ein Ausdruck ist, der das Ergebnis der Regel beschreibt.

valuation_i ist ein arithmetischer Ausdruck, der einen numerischen Wert zurückliefert. Dieser kann in einer (Gral unterstützt mehrere) Auswahlstrategie herangezogen werden: Es wird die Regel mit der kleinsten *valuation* bevorzugt.

Die Auswertung einer Regel erfolgt standardmäßig. Sei E der Ausdruck auf den die Regel angewendet werden soll.

```

if  $\exists$  Substitution  $\sigma$ , Unterausdruck  $E'$  von  $E$  mit  $E'\sigma = pattern$ 
  and  $\forall 1 \leq i \leq j: \neg condition_i$ 
  and  $condition_j$ 
then ersetze  $E'$  in  $E$  durch  $result_j\sigma$ 

```

Der Gral-Optimierer ist ein reiner regelbasierter Optimierer, der den Transformationsansatz verfolgt. Dementsprechend treffen alle vorher identifizierten Nachteile derselben zu.

Zu bemängeln sind im einzelnen folgende Punkte:

- Es erfolgt keine frühzeitige Bewertung der Alternativen.
- Die Suchstrategien sind fest eingebaut und nicht sonderlich ausgefeilt.
- Der Einbau von hochspezialisierten Algorithmen, die besondere Optimierungstechniken repräsentieren, ist schwierig, wenn nicht unmöglich.
- Eine Bestimmung der Verbundreihenfolge gemäß eines komplexeren Algorithmus ist nicht möglich.
- Da die Übersetzung in die Ausführungsalgebra lokal ist und keine Annotationen zugelassen sind, können vorhandene Sortierreihenfolgen nur schwer ausgenutzt werden.

Es wird nur eine Alternative der algebraischen Optimierung zur physischen Optimierung übergeben. Das kann zu Fällen führen, in denen der Optimierer niemals das Optimum finden kann. Wenngleich dies auch im allgemeinen nicht immer möglich ist, so sollte jedoch diese Eigenschaft nicht inhärent sein.

Positiv zu vermerken ist, daß für IMPROVING und TRANSLATION der Aufwand für das Pattern-matching vermutlich gering gehalten werden kann.

30.1.10 Lambda-DB

<http://lambda.uta.edu/lambda-DB/manual/overview.html>

30.1.11 Lanzelotte in short

Query Language Der Lanzelotte-Optimierer verwendet keine spezielle Anfragesprache. Ausgangspunkt der Betrachtungen sind sog. Anfragegraphen (request graphs, query graphs). Einzelheiten stehen in meiner Ausarbeitung. In einem Papier ([529]) wird gezeigt wie man von einer Regelsprache (RDL) zu Anfragegraphen kommt.

Internal Representation Die interne Repraesentation einer Anfrage ist der Class Connection Graph. Dort enthalten sind die Datenbankobjekte (Extensionen), die in der Anfrage referenziert werden aus der Sicht des physikalischen Schemas und die in der Anfrage bedeutsamen Beziehungen zwischen diesen Extensionen (Joins, Attributpfade, Selektionen).

Query Execution Plans QEPs werden als (deep) processing trees repräsentiert.

Architecture Der Lancelotte-Optimierer ist regelbasiert.

Transformation versus generation Lancelotte bietet Regeln fuer beide Spielarten. Sie unterscheidet enumerative search (Generierung), randomized search (Transformation) und genetic search (Transformation).

Control/Search-Strategy Lancelotte versucht von den Einzelheiten der verwendeten Strategien zu abstrahieren und stellt eine erweiterbare Optimierung vor, die die Einzelheiten ueberdeckt. Die tatsaechlich zu einem bestimmten Zeitpunkt verwendete Strategie wird durch “assertions” bestimmt. (Dazu steht nicht viel in den Papieren, vielleicht meint sie auch die Bedingungsteile der Regeln)

Cost Model Ziemlich aehnlich dem, das wir verwenden. Sie benutzt auch solche Sachen wie $card(C)$, $size(C)$, $ndist(A_i)$, $fan(A_i)$, $share(A_i)$. Einzelheiten stehen in meiner Ausarbeitung.

30.1.12 Opt++

wisconsin

30.1.13 Postgres

Postgres ist kein Objektbanksystem sondern fällt in die Klasse der erweiterten relationalen Systeme [844]. Die wesentlichen Erweiterungen sind

- berechenbare Attribute, die als Quel-Anfragen formuliert werden [842],
- Operationen [840],
- abstrakte Datentypen [839] und
- Regeln [843].

Diese beiden Punkte sollen uns jedoch an dieser Stelle nicht interessieren. Die dort entwickelten Optimierungstechniken, insbesondere die Materialisierung der berechenbaren Attribute, sind in der Literatur beschrieben [469, 400, 398, 399]. Unser Interesse richtet sich vielmehr auf eine neuere Publikation, in der eine Vorschlag für die Reihenfolgebestimmung von Selektionen und Verbundoperationen unterbreitet wird [415]. Diese soll im folgenden kurz vorgestellt werden. Zunächst jedoch einige Vorbemerkungen.

Wenn man eine Selektion verzögert, also nach einem Verbund ausführt, obwohl dies nicht notwendig wäre, so kann es passieren, daß das Selektionsprädikat auf mehr Tupeln ausgewertet werden muß. Es kann jedoch nicht passieren, daß es auf mehr verschiedenen Werten ausgeführt werden muß. Im Gegenteil, die Anzahl der Argumentewerte wird durch einen Verbund im allgemeinen verkleinert. Cached man also die bereits errechneten Werte des Selektionsprädikates, so wird die Anzahl der Auswertungen des Selektionsprädikates

nach einem Verbund zumindest nicht größer. Die Auswertung wird dann durch ein Nachschlagen ersetzt. Da wir hier nur teure Selektionsprädikate betrachten, ist ein Nachschlagen sehr billig gegenüber der Auswertung. Die Kosten für das Nachschlagen können sogar vernachlässigt werden. Es bleibt das Problem der Größe des Caches. Liegt Eingabe sortiert nach den Argumenten des Selektionsprädikates vor, so kann der die Größe des Caches unter Umständen auf 1 reduziert werden. Er erübrigt sich ganz, wenn man eine indirekte Repräsentation des Verbundergebnisses verwendet. Eine mögliche indirekte Repräsentation ist in Abbildung ?? dargestellt, wobei die linke der abgebildeten Relationen die Argumente für das betrachtete Selektionsprädikat enthalte.

Für jedes Selektionsprädikat $p(a_1, \dots, a_n)$ mit Argumenten a_i bezeichne c_p die Kosten der Auswertung auf einem Tupel. Diese setzen sich aus CPU- und I/O-Kosten zusammen (s. [415]). Ein *Plan* ist ein Baum, dessen Blätter *scan*-Knoten enthalten und dessen innere Knoten mit Selektions- und Verbundprädikaten markiert sind. Ein *Strom* in einem Plan ist ein Pfad von einem Blatt zur Wurzel. Die zentrale Idee ist nun die Selektions- und Verbundprädikate nicht zu unterscheiden, sondern gleich zu behandeln. Dabei wird angenommen, daß alle diese Prädikate auf dem Kreuzprodukt aller Relationen der betrachteten Anfrage arbeiten. Dies erfordert eine Anpassung der Kosten. Seien $a_1, \dots, a_n$ die Relationen der betrachteten Anfrage und p ein Prädikat über den Relationen $a_1, \dots, a_k$. Dann sind die *globalen Kosten* von p wie folgt definiert:

$$C(p) = \frac{c_p}{\prod_{i=k+1}^n |a_i|}$$

Die globalen Kosten berechnen die Kosten der Auswertung des Prädikates über der gesamten Anfrage. Hierbei müssen natürlich diejenigen Relationen herausgenommen werden, die das Prädikat nicht beeinflussen. Zur Illustration nehme man an, p sei ein Selektionsprädikat auf nur einer Relation a_1 . Wendet man p direkt auf a_1 an, so entstehen die Kosten $c_p * |a_1|$. Im vereinheitlichten Modell wird angenommen, daß jedes Prädikat auf dem Kreuzprodukt aller in der Anfrage beteiligten Relationen ausgewertet wird. Es entstehen also die Kosten $C(p) * |a_1| * |a_2| * \dots * |a_n|$. Diese sind aber gleich $c_p * |a_1|$. Dies ist natürlich nur unter der Verwendung eines Caches für die Werte der Selektionsprädikate korrekt. Man beachte weiter, daß die Selektivität $s(p)$ eines Prädikates p unabhängig von der Lage innerhalb eines Stroms ist.

Der *globale Rang* eines Prädikates p ist definiert als

$$rank(p) = \frac{s(p)}{C(p)}$$

Man beachte, daß die Prädikate innerhalb eines Stroms nicht beliebig umordbar sind, da wir gewährleisten müssen, daß die von einem Prädikat benutzten Argumente auch vorhanden sein müssen. In [415] wird noch eine weitere Einschränkung vorgenommen: Die Verbundreihenfolge darf nicht angetastet werden. Es wird also vorausgesetzt, daß eine optimale Verbundreihenfolge bereits bestimmt wurde und nur noch die reinen Selektionsprädikate verschoben werden dürfen.

Betrachtet man zunächst einmal nur die Umordnung der Prädikate auf einem Strom, so erhält man bedingt durch die Umordbarkeitseinschränkungen

das *Sequentialisierungsproblem mit Vorrangbedingungen* für das Algorithmus mit Laufzeit $O(n \log n)$ (n ist die Stromlänge) eine optimale Lösung bekannt ist [627].

Das in [415] vorgeschlagene Verfahren wendet diesen Algorithmus solange auf jeden Strom an, bis keine Verbesserung mehr erzielt werden kann. Das Ergebnis ist ein polynomialer Algorithmus, der die optimale Lösung garantiert. Dies jedoch nur unter der Einschränkung, daß die Kosten des Joins linear sind.

Damit sind wir bereits bei einem der Nachteile des Verfahrens: Die Kosten der Verbundoperation nicht mitunter nicht linear sondern sogar quadratisch. Ein weiterer Nachteil liegt in der Voraussetzung, daß die optimale Verbundreihenfolge schon bestimmt wurde, denn diese hängt wesentlich davon ab, an welcher Stelle die Selektionen eingebaut werden. Üblicherweise wird bei der Bestimmung der optimalen Verbundreihenfolge vorausgesetzt, daß alle Selektionsprädikate soweit wie möglich nach unten verschoben werden. Dies ist jedoch jetzt nicht mehr der Fall. Es ist also notwendig die Selektionsprädikatmigration in die Joinreihenfolgebestimmung zu integrieren. Nur dann kann man auf gute Ergebnisse hoffen. Die Integration mit einem Ansatz des dynamischen Programmierens ist problematisch, da dort Lösungen verworfen werden, die unter Umständen zur Optimalen Lösung führen, wenn ein Selektionsprädikat nicht ganz nach unten durchgeschoben wird [415].

Eine Teillösung wird dort auch angedeutet. Ist der Rang eines Selektionsprädikates größer als jeder Rang jedes Plans einer Menge von Verbunden, so ist das Selektionsprädikat in einem optimalen Baum oberhalb all dieser Verbundoperationen platziert. Ein entsprechender Algorithmus hat aber, wenn er beispielsweise nur Left-deep-trees erzeugt, eine Worst-case-Komplexität von $O(n^4 n!)$.

30.1.14 Sciore & Sieg

Die Hauptidee von Sciore und Sieg ist es, die Regelmenge in Module zu organisieren und jedem Modul eine eigene Suchstrategie, Kostenberechnung und Regelmenge zuzuordnen. Module können andere Module explizit aufrufen, oder implizit ihre Ausgabemenge an das nächste Modul weiterleiten.

30.1.15 Secondo

Gueting

30.1.16 Squirrel

Der erste Ansatz eines regelbasierten Optimierers, Squirrel, kann auf das Jahr 1975 zurückgeführt werden [823]. Man beachte, daß dieses Papier vier Jahre älter ist als das vielleicht am häufigsten zitierte Papier über den System R Optimierer [784], der jedoch nicht regelbasiert, sondern fest verdrahtet ist.

Abbildung 30.7 gibt einen Überblick über den Aufbau von Squirrel. Nach der syntaktischen Analyse liegt ein Operatorgraph vor. Dieser ist in Squirrel zunächst auf einen Operatorbaum beschränkt. Zur Behandlung von gemeinsamen Teilausdrücken wird das Anlegen von temporären Relationen, die den

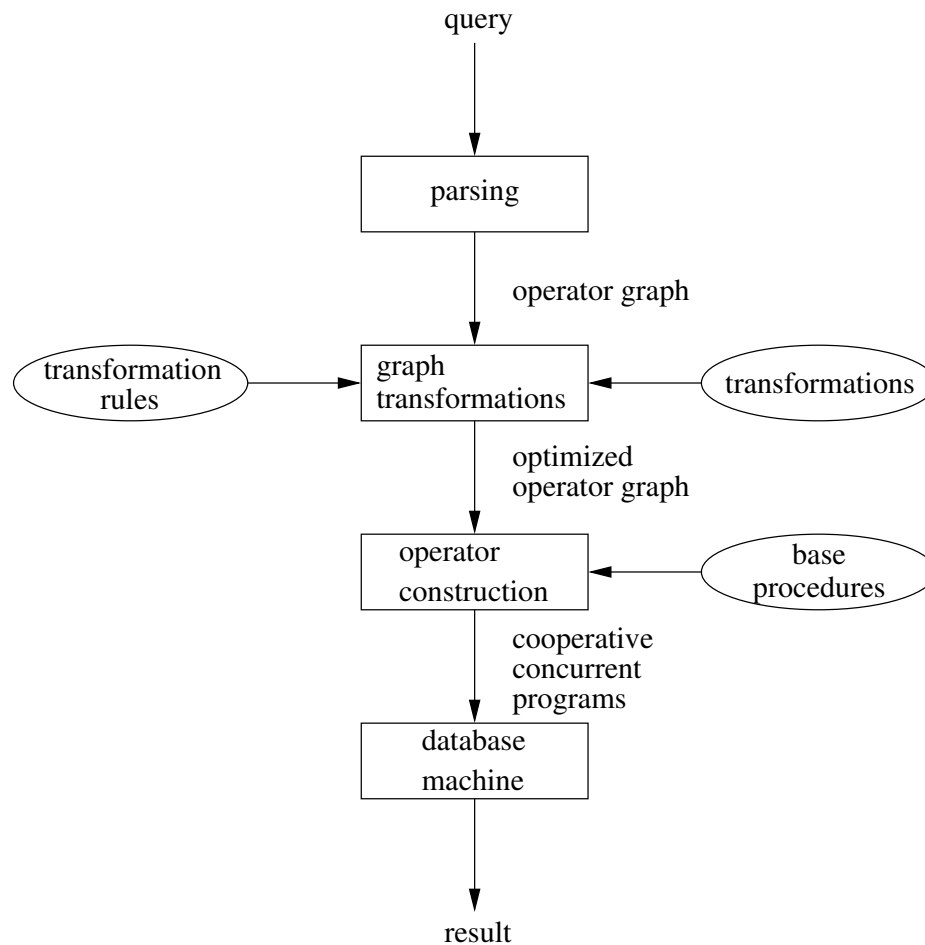


Figure 30.7: Die Squirrelarchitektur

gemeinsamen Teilausdrücken entsprechen, vorgeschlagen. Diese temporären Relationen ersetzen dann die gemeinsamen Teilausdrücke. Dadurch ist es möglich, sich auf Operatorbäume zu beschränken.

Der Operatorbaum wird dann in einen optimierten Operatorbaum transformiert. Hierzu werden Regeln, die den algebraischen Gleichungen entsprechen, verwendet. Die Anwendung dieser Transformationsregeln ist rein heuristisch gesteuert. Die Heuristik selber ist in den Transformationsanwendungsregeln abgelegt. Eine dieser Regeln sagt beispielsweise, daß Projektionen nur dann nach unten geschoben werden, wenn die Operation, über die die Projektion als nächstes geschoben werden soll, keine Verbundoperation ist. Neben den Standardregeln, die das Vertauschen von relationalen Operatoren ermöglichen, gibt es Regeln, die es erlauben, relationale Ausdrücke in komplexe boolesche Ausdrücke, die dann als Selektionsprädikate Verwendung finden, zu überführen. Dies ist der erste Vorschlag, nicht nur primitive Selektionsprädikate in Form von Literalen, sondern auch komplexere Ausdrücke mit booleschen Verknüpfungen zu verwenden. Auf die Optimierung dieser Ausdrücke wird jedoch nicht weiter

eingegangen.

Die wesentliche Aufgabe der Operatorkonstruktion ist die Auswahl der tatsächlichen Implementierungen der Operatoren im Operatorgraph unter optimaler Ausnutzung gegebener Sortierreihenfolgen. Auch diese Phase der Optimierung ist in Squirrel nicht kostenbasiert. Sie wird durch zwei Durchläufe durch den Operatorgraphen realisiert. Der erste Durchlauf berechnet von unten nach oben die möglichen Sortierungen, die ohne zusätzlichen Aufwand möglich sind, da beispielsweise Relationen schon sortiert sind, und vorhandene Sortierungen durch Operatoren nicht zerstört werden. Im zweiten Durchlauf, von oben nach unten, werden Umsortierungen nur dann vorgenommen, wenn keine der im ersten Durchlauf berechneten Sortierungen eine effiziente Implementierung des zu konvertierenden Operators erlaubt. Beide Durchläufe sind mit Regelsätzen spezifiziert. Es ist bemerkenswert, daß die Anzahl der Regeln, 32 für den Aufwärtspass und 34 für den Abwärtspass, die Anzahl der Regeln für die Transformationsphase (insgesamt 7 Regeln), bei weitem übertrifft. Auch die Komplexität der Regeln ist erheblich höher.

Beide für uns interessante Phasen, die Operatorgraphtransformation und Operatorkonstruktion, sind mit Regeln spezifiziert. Es ist jedoch in beiden Phasen kein Suchprozeß nötig, da die Regeln alle Fälle sehr gezielt auflisten und somit einen eindeutigen Entscheidungsbaum beschreiben. Eine noch minutiösere Unterscheidung für die Erzeugung von Ausführungsplänen in der Operatorkonstruktionsphase gibt es nur noch bei Yao [957]. Diese haben auch den Vorteil, durch Kostenrechnungen belegt zu sein.

Da die Regeln in ihren Prämissen die Heuristik ihrer Anwendung mit kodieren und keine eigene Suchfunktion zur Anwendung der Regeln existiert, ist die Erweiterbarkeit sehr schwierig. Das Fehlen jeglicher Kostenbewertung macht eine Evaluation der Alternativen unmöglich. Daher ist es auch schwer, die einzelnen Komponenten des Optimierers, nämlich die Regeln, zu bewerten, zumal der transformierende Ansatz gewählt wurde. Der Forderung nach Vorhersagbarkeit und stetiger Leistungsabfall wird in diesem Ansatz ebenfalls nicht nachgegangen.

30.1.17 System R and System R*

30.1.18 Starburst and DB2

Starburst [267, 387] liegt ein erweiterbares relationales Datenmodell zugrunde. Die Anfragebearbeitung ist wie in System R und System R* in die zwei Schritte Anfrageübersetzung und -ausführung zergliedert [388]. Wir interessieren uns für den ersten Schritt, die Anfrageübersetzung. Einen Überblick gibt Abbildung 30.8. Nach der standardmäßigen Zerteilung liegt die Anfrage in der internen Darstellung QGM (Query Graph Model) vor. QGM ist an die Anfragesprache Hydrogen (ähnlich SQL) von Starburst angelehnt. Der wichtigste Grundbaustein von QGM ist der *select*-Operator. Dieser enthält eine Projektionsliste und das Anfrageprädikat in Graphform. Die Knoten sind markiert und referenzieren (gespeicherte) Relationen oder weitere QGM-Operatoren. Die Markierung ist entweder ein Quantor ($\forall$, $\exists$) oder die Mengenerzeugermarkierung

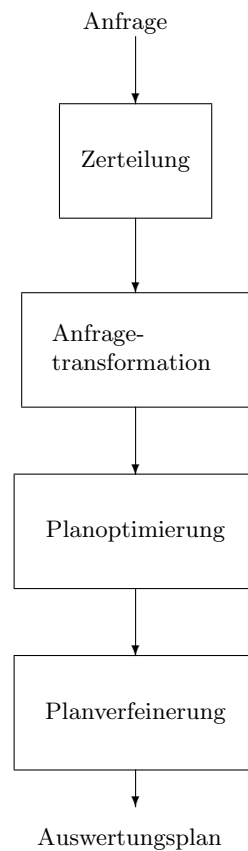


Figure 30.8: Starburst Optimierer

(F). Knoten, die mit F markiert sind, tragen zur Erzeugung des Ergebnisses eines Operators bei, die Quantorenmarkierungen zu dessen Einschränkung. Die Kanten sind mit den Prädikaten markiert. Es ergeben sich also Schleifen für nur eine Relation betreffende Prädikate. Weitere Operatoren sind *insert*, *update*, *intersection*, *union* und *group-by*. Daneben wird die QGM-Repräsentation einer Anfrage mit Schemainformation und statistischen Daten angereichert. Sie dient also auch als Sammelbecken für alle die Anfrage betreffende Information.

Die QGM-Repräsentation dient der Anfragetransformation (Abb. 30.8) als Ausgangspunkt. Die Anfragetransformation generiert zu einer QGM-Repräsentation verschiedene äquivalente QGM-Repräsentationen. Die Anfragetransformation läßt sich, abgesehen von den Darstellungsunterschieden von QGM und Hydrogen, als eine Variante der Source-level-Transformationen ansehen. Sie wird regelbasiert implementiert, wobei C die Regelsprache ist. Eine Regel besteht aus 2 Teilen, einer Bedingung und einer Aktion. Jeder Teil wird durch eine C-Prozedur beschrieben. Dadurch erübrigt sich die Implementierung eines allgemeinen Regelinterpreters mit Pattern-matching. Regeln können in Gruppen zusammengefaßt werden. Der aktuelle Optimierer umfaßt drei Klassen von Regeln:

1. Migration von Prädikaten

2. Migration von Projektionen
3. Verschmelzung von Operationen

Für die Ausführung der Regeln stehen drei verschiedene Suchstrategien zur Verfügung:

1. sequentiell,
2. prioritätsgesteuert und
3. zufällig, gemäß einer gegebenen Verteilung.

Die Teilgraphen der QGM-Repräsentation, auf die Regeln anwendbar sind, können entweder durch eine depth-first oder eine breadth-first Suche bestimmt werden. Falls mehrere alternative QGM-Repräsentationen existieren (was meistens der Fall ist), wird ein *Choose*-Operator [361] verwendet, der die verschiedenen QGMs in einen QGM zusammenbaut. Die nachfolgende Phase wählt dann kostenbasiert einen dieser alternativen QGMs aus. Dies ist nicht zwingend, die Auswahl kann auch erst zur Auswertungszeit stattfinden. Begründet wird dieses Vorgehen damit, daß keine Kosten für QGMs berechnet werden können, und somit keine Bewertung eines QGMs stattfinden kann. Wie die Autoren selbst anmerken, ist dieser Umstand sehr mißlich, da keine Alternativen verworfen werden können. Sie kündigen daher Untersuchungen an, die Transformation (Schritt 2) mit der Planoptimierung (Schritt 3) zu verschmelzen. Um eine gewisse Kontrolle über das Verhalten der Transformation zu haben, kann diesem Schritt ein "budget" mitgegeben werden, nach dessen Ablauf der Schritt beendet wird. Die genaue Funktionsweise des "budget" ist leider nicht erläutert.

Der Schritt der Planoptimierung (s. Abb. 30.8) kann mit der bisherigen Optimierung verglichen werden. Sie arbeitet regelbasiert, benutzt aber nicht den transformierenden, sondern den generierenden Ansatz [567]. Aus Basisoperationen – LOLEPOPs (LOW-LEVEL Plan OPERator) genannt – werden mit (grammatischen) Regeln – STARS (strategy alternative rules) genannt – (alternative) Auswertungspläne erzeugt. LOLEPOPs entstammen der um SCAN, SORT und ähnliche physische Operatoren angereicherten relationalen Algebra. Ein Auswertungsplan ist dann ein Ausdruck von geschachtelten Funktionsaufrufen, wobei die Funktionen den LOLEPOPs entsprechen.

Ein STAR definiert ein benanntes parametrisiertes Objekt, das einem Nicht-terminalsymbol entspricht. Er besteht aus einer Menge von alternativen Definitionen, die jede aus einer Bedingung für die Anwendbarkeit und der Definition eines Plans bestehen. Der generierte Plan kann LOLEPOPs (entsprechen Terminalsymbolen) und STARS referenzieren. Ein rootSTAR entspricht dem Startsymbol der Grammatik. STARS ähneln den Regeln, die in Genesis nicht nur für den Optimierer, sondern für das ganze DBMS eingesetzt werden, um alternative Implementierungen zu erhalten [59, 57, 61, 60, 62]. Um erzeugte Alternativen für einen Plan zusammenzusetzen und zu verhindern, daß diese Alternativen die Anzahl der Pläne in denen diese vorkommen, vervielfachen, wird ein Glue-Mechanismus eingesetzt. Dieser hat den *Choose*-Operator als Wurzel. Darunter hängen dann Alternativen, die beispielsweise einen Strom

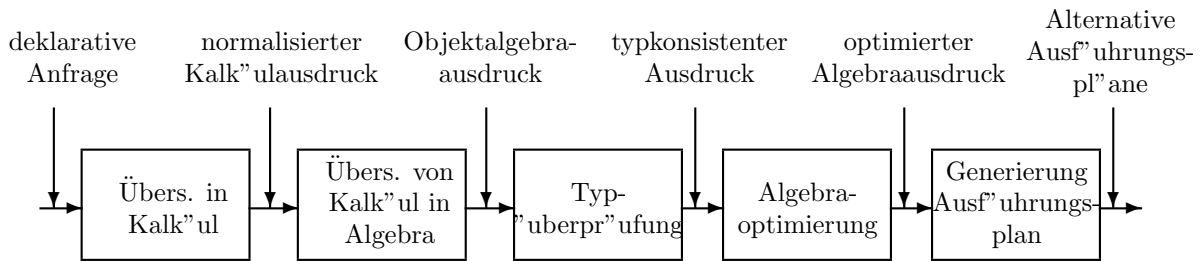


Figure 30.9: Der Optimierer von Straube

mit gewissen Eigenschaften (Sortierung, Lokation) erzeugen. Von diesen Alternativen werden nur diejenigen betrachtet, die die geringsten Kosten bei gleichen Eigenschaften haben [541]. Die Kosten beziehen sich dabei immer nur auf den bisher erreichten Teilplan.

Der Aufbau eines Auswertungsplanes erfolgt Bottom-up. Die Menge der anwendbaren STARs wird in einer ToDo-Liste gehalten. Diese ist eine sortierte Liste. Hiermit können dann verschiedene Suchstrategien implementiert werden, indem verschiedene Sortierungen für die ToDo-Liste Verwendung finden [541]. Ein Vorteil des STAR-Ansatzes ist die Vermeidung von Pattern-matching. Dies erlaubt es, die STARs zu interpretieren [541].

Die Beurteilung der Erweiterbarkeit ist sehr schwierig. Zum einen handelt es sich um einen erweiterbaren Optimierer, da sowohl LOLEPOPs als auch STARs hinzugefügt werden können. Der Glue-Mechanismus kann ebenfalls spezifiziert werden, ohne in die Implementierung einzugreifen. Das Problem ist lediglich die Komplexität dieser Änderungen. Man kann diesen Ansatz daher vielleicht als bedingt erweiterbar kennzeichnen.

Eine Trennung der Optimierung in verschiedene Phasen wirft die damit verbundenen Probleme auf. Wie oben bereits angeführt, kündigen die Autoren weitere Untersuchungen an, um eine Verschmelzung der Phasen zu ermöglichen. Da keine Alternativen verworfen werden, ist es potentiell möglich, den optimalen Auswertungsplan zu errechnen. Schwer zu sehen ist jedoch, wie ein stetiger Leistungsabfall zu realisieren ist. Gleiches gilt für die Evaluierbarkeit der einzelnen Komponenten (STARs).

More on Starburst can be found in [689, 690].

30.1.19 Der Optimierer von Straube

In seiner Dissertation stellt Straube den von ihm entwickelten Optimierer dar [850]. Die Ergebnisse dieser Arbeit flossen in eine Reihe von Veröffentlichungen ein [889, 847, 848, 846, 849]. Der Aufbau des Optimierers ist in Abbildung 30.9 skizziert. Eine Anfrage wird zunächst in den Objektkalkül übersetzt und von dort in die Objektalgebra. Hier findet dann zunächst eine Typüberprüfung statt. Danach beginnt die eigentliche Optimierung. Diese besteht aus zwei Phasen, der algebraischen Optimierung und der Generierung des Ausführungsplans.

Die erste Phase, die algebraische Optimierung, folgt dem Transformationssparadigma. Die algebraischen Ausdrücke werden mit Hilfe von Regeln in

äquivalente algebraische Ausdrücke transformiert. Straube beschränkt sich dabei im wesentlichen auf die Formulierung der Regeln. Für die Abarbeitung der Regeln schlägt er lediglich die Verwendung des Exodus-Optimierergenerators [357] oder des Anfrageumformers von Starburst [409] vor.

Die zweite Phase, die Generierung der Ausführungspläne, ist nicht regelbasiert. Ihr liegt eine sogenannte Ausführungsplanschablone zu Grunde. Sie ist vergleichbar mit einem Und/Oder-Baum, der alle möglichen Ausführungspläne implizit beinhaltet. Zur Generierung eines konkreten Ausführungsplans wird der durch die Ausführungsschablone aufgespannte Suchraum vollständig durchsucht. Der billigste Ausführungsplan kommt dann zur Abarbeitung.

Da ein regelbasierter Ansatz für die erste Phase gewählt wurde und die Verwendung des Exodus-Optimierergenerators oder des Starburst-Anfrageumformers vorgeschlagen wird, verweisen wir für die Bewertung dieser Phase auf die entsprechenden Abschnitte.

Die zweite Phase ist voll auskodiert und damit schlecht erweiterbar. Eine frühe Bewertung der Alternativen ist nicht ausgeschlossen, wird aber nicht vorgenommen. Ein vollständiges Durchsuchen verhindert natürlich auch den stetigen Leistungsabfall des Optimierers.

Erschwerend für den gewählten Ansatz kommt die Zweiphasigkeit hinzu. Es ist schwierig zu sehen, wie eine phasenübergreifende Kontrolle auszusehen hat, die zumindest potentiell Optimalität gewährleistet. Die Evaluierbarkeit einzelner Komponenten des Optimierers ist nicht möglich.

30.1.20 Other Query Optimizer

Neben den in den vorangehenden Abschnitten erwähnten Optimierern gibt es noch eine ganze Reihe anderer, die aber nicht im einzelnen vorgestellt werden sollen. Erwähnt werden sollen noch die Systeme Probe [223, 222, 655] und Prima [404, 402]. Der Schwerpunkt bei der Optimierung liegt im Primasystem auf dem dynamischen Zusammenbau von Molekülen. Es wäre zu untersuchen, ob ein Assembly-Operator (s. [483]) hier von Nutzen wäre. Besonders erwähnenswert ist noch eine Arbeit, die Optimierungsmöglichkeiten für die Datenbankprogrammiersprache FAD vorstellt [894]. Diese Arbeit stellt einen ersten Schritt in Richtung eines Optimierers für eine General-purpose-Programmiersprache dar. Ein wesentlicher Punkt ist dabei, daß auch ändernde Operationen optimiert werden. Der Optimierer ist in zwei Module (RWR and OPT) eingeteilt. RWR ist ein Sprachmodul, das die Übersetzung von FAD in ein internes FAD vornimmt. Immer wenn RWR einen Ausdruck erkennt, der in der vom Optimierer OPT bearbeitbaren Sprache ausgedrückt werden kann, so wird dieser an den Optimierer weitergegeben und dort optimiert. Es wird Exhaustive search als Suchstrategie für den Optimierer vorgeschlagen.

Im erweiterten O_2 -Kontext wurde das Zerteilen von Pfadausdrücken weiter untersucht [187]. Es werden die Vorteile einer typisierten Algebra für diese Zwecke herausgearbeitet. Eine graphische Notation sorgt für eine anschauliche Darstellung. Ihr besonderes Augenmerk richten die Autoren auf die Faktorisierung gemeinsamer Teilausdrücke. Einige der Ersetzungsregeln sind aus [802] und [801] entnommen und werden gewinnbringend eingesetzt, so beispielsweise die

Ersetzung von Selektionen durch Verbundoperatoren.

Ebenfalls erwähnt wurden bereits die Arbeiten im Orion-Kontext [50, 51, 493], die sich auf die Behandlung von Pfadausdrücken konzentrieren. Auch hier wurde ein funktionsfähiger Optimierer entwickelt.

Wie bereits erwähnt, stammt der erste regelbasierte Optimierer von Smith und Chang [823]. Doch erst die neueren Arbeiten führten zu einer Blüte des regelbasierten Ansatzes. Hier ist insbesondere die Arbeit von Freytag zu erwähnen, die diese Blüte mit initiierte [295]. Dort wird gezeigt, wie man mit Hilfe eines regelbasierten Ansatzes den Optimierer von System R [784] emulieren kann. Die Eingabe besteht aus einem Lisp-ähnlichen Ausdruck:

```
(select <proj-list>
      <sel-pred-list>
      <join-pred-list>
      <table-list>)
```

Die Projektionsliste besteht aus Attributspezifikationen der Form

```
<rel-name>.<attr-name>
```

Diese werden auch für die Selektionsprädikate und Joinprädikate verwendet. Die Algebra beinhaltet sowohl Operatoren der logischen als auch der physischen Algebra. Im einzelnen gibt es Scan-, Sort-, Projektions, Verbundoperatoren in einer logischen und verschiedenen physischen Ausprägungen. Die Erzeugung der Auswertungspläne wird in verschiedene Schritte unterteilt, die wiederum in Teilschritte zerlegt sind (siehe Abb. 30.3).

Zunächst erfolgt die Übersetzung in die logische Algebra. Hier werden Scan-Operatoren um die Relationen gebaut und Selektionen, die nur eine Relation betreffen, in die Scan-Operatoren eingebaut. Der zweite Schritt generiert Zugriffspläne, indem der Scan-Operator durch einen einfachen File-Scan (FSCAN) ersetzt wird oder falls möglich, durch einen Index-Scan (ISCAN). Der dritte Schritt generiert zunächst verschiedene Verbundreihenfolgen und bestimmt anschließend die Verbundmethoden. Wie in System R wird zwischen dem Sortiere- und Mische-Verbund und dem Verbund durch geschachtelte Schleifen unterschieden. Es werden keinerlei Aussagen über die Auswahl einer Suchstrategie gemacht. Ziel ist es vielmehr, durch die Modellierung des System R Optimierers mit Hilfe eines Regelsystems die prinzipielle Brauchbarkeit des regelbasierten Ansatzes nachzuweisen.

Man beachte auch die erwähnte Arbeit von Sciore und Sieg zur Modularisierung von regelbasierten Optimierern [780]. Die Hauptidee von Sciore und Sieg ist es, die Regelmengen in Module zu organisieren und jedem Modul eine eigene Suchstrategie, Kostenberechnung und Regelmengen zuzuordnen. Module können andere Module explizit aufrufen oder implizit ihre Ausgabemenge an das nächste Modul weiterleiten. Der erste Optimierer des GOM-Systems ist ebenfalls regelbasiert [486, 485]. Die gesamte Regelmengen wurde hier in Teilmengen ähnlich zu den Modulen organisiert. Die Steuerung zwischen den Teilmengen erfolgt durch ein heuristisches Netz, das angibt in welchen Fällen zu welcher weiteren Teilmenge von Regeln zu verzweigen ist. Die Strukturierung des Optimiererwissens steht auch in [580] im Vordergrund.

In diesem Zusammenhang, der Strukturierung von Optimierern und der Wiederverwendbarkeit einzelner Teile, sei noch einmal ausdrücklich auf die Arbeiten von Batory [60] aus dem Genesiskontext hingewiesen (s. auch Abschnitt 30.1.7). Der dort leider ein wenig zu kurz kommende Aspekt der Wiederverwendbarkeit von Suchfunktionen wird in einer Arbeit von Lancelotte und Valduriez [530] ausführlicher behandelt. Hier wurde eine Typhierarchie existierender Suchfunktionen entworfen und deren Schnittstellen vereinheitlicht. Die Suchfunktionen selbst wurden modularisiert. Weitere Arbeiten aus derselben Gruppe beschäftigen sich mit der Optimierung von objektorientierten Anfragen [529, 533], wobei hier die Behandlung von Pfaden im Vordergrund steht. Eine neuere Arbeit beschäftigt sich mit der Optimierung von rekursiven Anfragen im objektorientierten Kontext [531].

Viele kommerzielle Systeme besitzen eine Anfragesprache und einen Optimierer. Einer der wenigen Optimierer, die auch in der Literatur beschrieben werden, ist der von ObjectStore [654]. Durch die einfache Anfragesprache, die nur Teilmengenbestimmung erlaubt, und die strikte Verwendung von *C*-Semantik für boolesche Ausdrücke sind die meisten Optimierungsmöglichkeiten jedoch ausgeschlossen, und der “Optimierer” ist daher sehr einfach.

30.2 Commercial Query Compiler

30.2.1 The DB 2 Query Compiler

30.2.2 The Oracle Query Compiler

Oracle still provides two modes for its optimizer. Dependent on the user specified optimizer mode, a query is optimized either by the rule-based optimizer (RBO) or by the cost-based optimizer (CBO). The RBO is a heuristic optimizer that resembles the simple optimizer of chapter 2. Here we concentrate on the more powerful CBO. The user can also determine whether the optimizer should optimize for throughput or response time.

- nested loop join, nested loop outer join, index nested loop joins, sort merge join, sort merge outer join, hash joins, hash outer join, cartesian join, full outer join, cluster join, anti-joins, semi-joins, uses bitmap indexes for star queries
- sort group-by,
- bitmap indexes, bitmap join indexes
- index skip scans
- partitioned tables and indexes
- index-organized tables
- reverse key indexes
- function-based indexes

- SAMPLE clause in SELECT statement
- parallel query and parallel DML
- star transformations and star joins
- query rewrite with materialized views
- cost: considers CPU, I/O, memory
- access path: table scan, fast full index scan, index scan, ROWID scans (access ROW by ROWID), cluster scans, hash scans. [former two with prefetching] index scans:
 - index unique scan (UNIQUE or PRIMARY KEY constraints)
 - index range scan (one or more leading columns or key)
 - index range scan descending
 - index skip scan (> 1 leading key values not given)
 - index full scan, index fast full scan
 - index joins (joins indexes with hash join, resembles index anding)
 - bitmap joins (index anding/oring)
 - cluster scan: for indexed cluster to retrieve rows with the same cluster id
 - hash scan: to locate rows in a hash cluster

CBO: parsed quer $\rightarrow$ [query transformer] $\rightarrow$ [estimator] $\rightarrow$ [plan generator]
1-16.

after parser: nested query blocks

simple rewrites:

- eliminate between
- eliminate x in (c1 ... cn) (also uses IN-LIST iterator as outer table constructor in a d-join or nested-loop join like operation.

query transformer:

- view merging
- predicate pushing
- subquery unnesting
- query rewrite using materialized views (cost based)

remaining subplans for nested query blocks are ordered in an efficient manner
plan generator:

- choose access path, join order (upper limit on number of permutations considered), join method.

- generate subplan for every block in a bottom-up fashion
- (> 1 for still nested queries and unmerged views)
- stop generating more plans if there already exists a cheap plan
- starting plan: order by their effective cardinality
- considers normally only left-deep (zig-zag) trees.
- single row joins are placed first (based on unique and key constraints).
- join statement with outer join: table with outer join operator must come after the other table in the condition in the join order. optimizer does not consider join orders that violate this rule.
- NOT IN (SELECT ...) becomes a anti-join that is executed as a nested-loop join by default unless hints are given and various conditions are met which allow the transformation of the NOT IN uncorrelated subquery into a sort-merge or hash anti-join.
- EXISTS (SELECT ...) becomes a semi-join. execution as index nested loops, if there is an index. otherwise a nested-loop join is used by default for EXISTS and IN subqueries that cannot be merged with the containing query unless a hint specifies otherwise and conditions are met to allow the transformation of the subquery into a sort-merge or hash semi-join.
- star query detection

cost:

- takes unique/key constraints into consideration
- low/high values and uniform distribution
- host variables: guess small selectivity value to favor index access
- histograms
- common subexpression optimization
- complex view merging
- push-join predicate
- bitmap access paths for tables with only B-tree indexes
- subquery unnesting
- index joins

rest:

- Oracle allows user hints in SQL statements to influence the Optimizer. for example join methods can be given explicitly

parameters:

- HASH_AREA_SIZE
- SORT_AREA_SIZE
- DB_FILE_MULTIBLOCK_READ_COUNT (number of prefetched pages)

statistics:

- table statistics
number of rows, number of blocks, average row length
- column statistics
number of distinct values, number of nulls, data distribution
- index statistics
number of keys, (from column statistics?) number of leaf blocks, levels, clustering factor (collocation amount of the index block/data blocks, 3-17)
- system statistics
I/O performance and utilization, cpu performance and utilization

generating statistics:

- estimation based on random data sampling
(row sampling, block sampling)
- exact computation
- user-defined statistics collection methods

histograms:

- height-based histograms (approx. equal number of values per bucket)
- value-based histograms
used for number of distinct values $\leq$ number of buckets
- support of index-only queries
- index-organized tables
- bitmap indexes (auch fuer null-werte `x <> const`)
- convert b-tree result RID lists to bitmaps for further bitmap anding
- bitmaps and count
- bitmap join index
- cluster tables (cluster rows of different tables on the same block)
- hash clusters

- hint: `USE_CONCAT`: `OR ::= UNION ALL`
- hint: `STAR_TRANSFORMATION`: see Oracle9i Database Concepts
- `NOT IN ::= anti-join`
- `EXISTS ::= special join preserving duplicates and adding no phantom duplicates (semi-join) (5-27)`
- continue 5-35

30.2.3 The SQL Server Query Compiler

Part VI

Selected Topics

Chapter 31

Generating Plans for Top-N-Queries?

31.1 Motivation and Introduction

motivation:

- first by user (ordered)
- optimize for n rows (user/cursor)
- exist(subquery) optimize for 1 row
- having count(*) <= n

31.2 Optimizing for the First Tuple

31.3 Optimizing for the First N Tuples

- nl-join instead of sm/hash join
- index access over table scan
- disable prefetching

[127, 128, 129] [150, 246] [268, 269, 433] [554]

[379] (also contains inverted list algorithms under frequent updates)

[554]

Chapter 32

Recursive Queries

Chapter 33

Issues Introduced by OQL

33.1 Type-Based Rewriting and Pointer Chasing Elimination

The first rewrite technique especially tailored for the object-oriented context is *type-based rewriting*. Consider the query

```
select distinct sn, ssn, ssa
from           s in Student
where          sg > 8 and ssa < 30
```

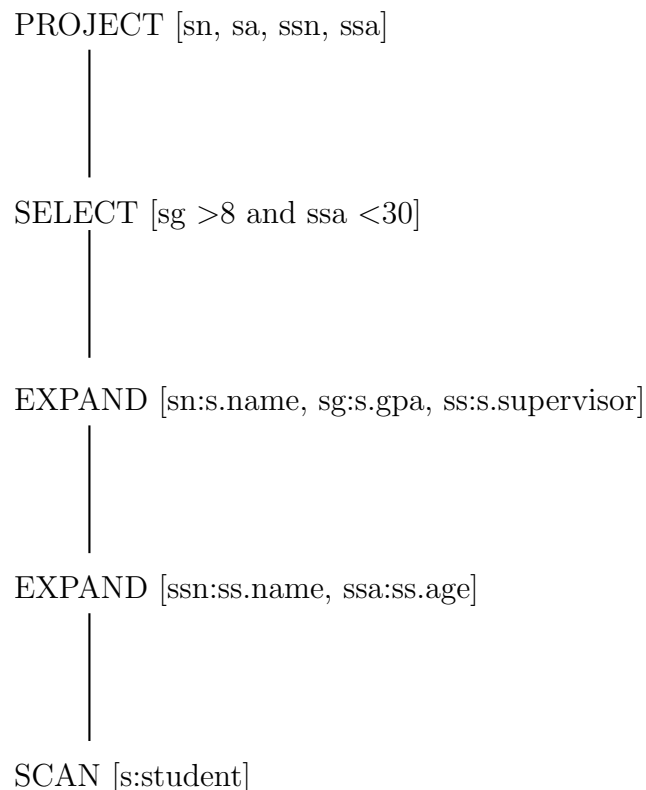


Figure 33.1: Algebraic representation of a query


```

define      sn = s.name
              sg = s.gpa
              ss = s.supervisor
              ssn= ss.name
              ssa= ss.age

```

The algebraic expression in Fig. 33.1 implies a scan of all students and a subsequent dereferentiation of the *supervisor* attribute in order to access the supervisors. If not all supervisors fit into main memory, this may result in many page accesses. Further, if there exists an index on the supervisor's *age*, and the selection condition $ssa < 30$ is highly selective, the index should be applied in order to retrieve only those supervisors required for answering the query. Type-based rewriting enables this kind of optimization. For any expression of certain type with an associated extent, the extent is introduced in the *from* clause. For our query this results in

```

select distinct sn, pn, pa
from            s in Student, p in Professor
where          sg > 8 and pa < 30 and ss = p
define        sn = s.name
              sg = s.gpa
              ss = s.supervisor
              pn = ss.name
              pa = ss.age

```

As a side-effect, the attribute traversal from students via supervisor to professor is replaced by a join. Now, join-ordering allows for several new plans that could not be investigated otherwise. For example, we could exploit the above mentioned index to retrieve the young professors and join them with the students having a *gpa* greater than 8. The according plan is given in Fig. 33.2. Turning implicit joins or pointer chasing into explicit joins which can be freely reordered is an original query optimization technique for object-oriented queries. Note that the plan generation component is still allowed to turn the explicit join into an implicit join again.

Consider the query

```

select distinct p
from            p in Professor
where          p.room.number = 209

```

Straight forward evaluation of this query would scan all professors. For every professor, the *room* relationship would be traversed to find the room where the professor resides. Last, the room's number would be retrieved and tested to be 209. Using the *inverse relationship*, the query could as well be rewritten to

```

select distinct r.occupiedBy

```

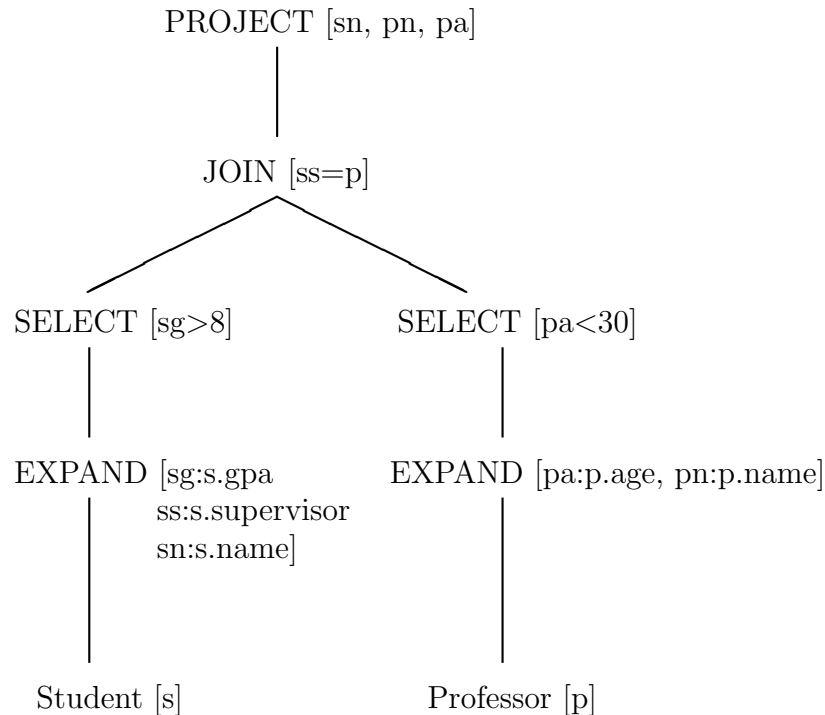


Figure 33.2: A join replacing pointer chasing

```

from      r in Room
where     r.number = 209
  
```

The evaluation of this query can be much more efficient, especially if there exists an index on the room number. Rewriting queries by exploiting inverse relationships is another rewrite technique to be applied during Rewrite Phase I.

33.2 Class Hierarchies

Another set of equivalences known from the relational context involves the UNION operator ($\cup$) and plays a vital role in dealing with class/extent hierarchies. Consider the simple class hierarchy given in Figure 33.3. Obviously, for the user, it must appear that the extent of *Employee* contains all *Managers*. However, the system has different alternatives to implement extents. Most OBMSs organize an object base into areas or volumes. Each area or volume is then further organized into several files. A file is a logical grouping of objects not necessarily consisting of subsequent physical pages on disk. Files don't share pages.

The simplest possible implementation to scan all objects belonging to a certain extent is to perform an area scan and select those objects belonging to the extent in question. Obviously, this is far to expensive. Therefore, some more sophisticated possibilities to realize extents and scans over them are needed. The

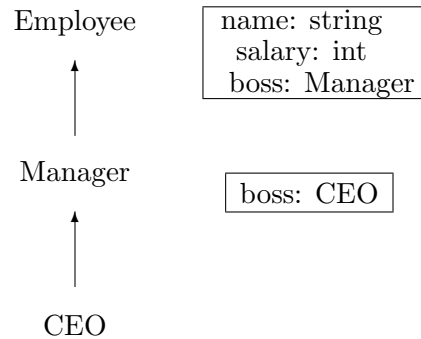


Figure 33.3: A Sample Class Hierarchy

different possible implementations can be classified along two dimensions. The first dimension distinguishes between logical and physical extents, the second distinguishes between strict and (non-strict) extents.

Logical *vs.* Physical Extents

An extent can be realized as a collection of object identifiers. A scan over the extent is then implemented by a scan over all the object identifiers contained in the collection. Subsequently, the object identifiers are dereferenced to yield the objects themselves. This approach leads to logical extents. Another possibility is to implement extent membership by physical containment. The best alternative is to store all objects of an extent in a file. This results in physical extents. A scan over a physical extent is then implemented by a file scan.

Extents *vs.* Strict Extents

A strict extent contains the objects (or their OIDs) of a class excluding those of its subclasses. A non-strict extent contains the objects of a class and all objects of its subclasses.

Given a class C , any strict extent of a subclass C' of C is called a subextent of C .

Obviously, the two classifications are orthogonal. Applying them both results in the four possibilities presented graphically in Fig. 33.4. [195] strongly argues that strict extents are the method of choice. The reason is that only this way the query optimizer might exploit differences for extents. For example, there might be an index on the *age* of *Manager* but not for *Employee*. This difference can only be exploited for a query including a restriction on *age*, if we have strict extents.

However, strict extents result in initial query plans including UNION operators. Consider the query

```

select  e
from    e in Employee
where   e.salary > 100.000
  
```

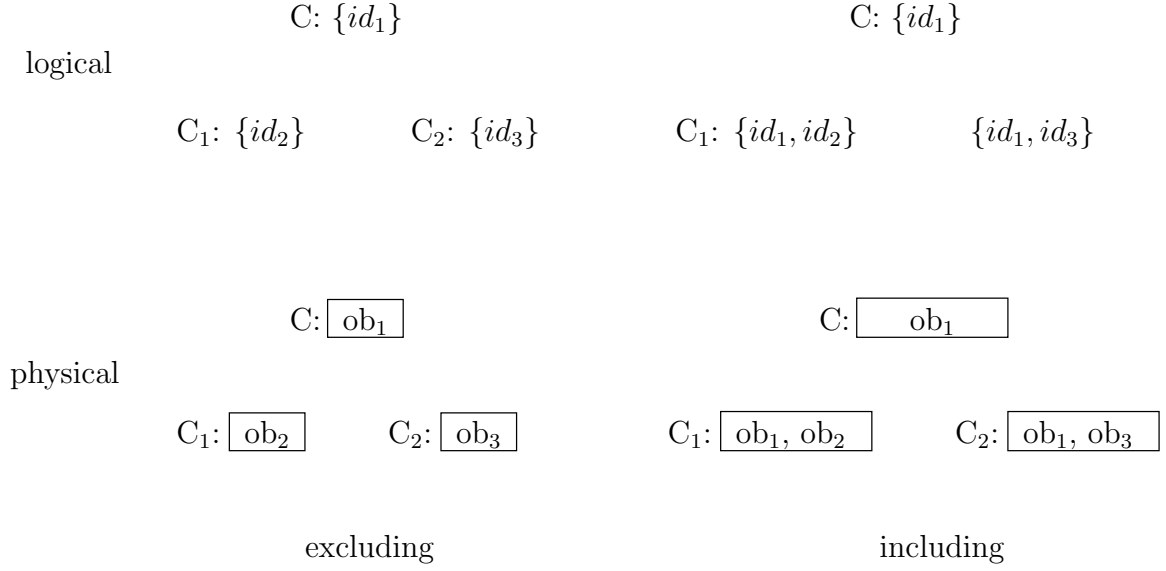


Figure 33.4: Implementation of Extents

The initial plan is

$$\sigma_{sa>100.000}(\chi_{sa:x.salary}((Employee[x] \cup Manager[x]) \cup CEO[x]))$$

Hence, algebraic equivalences are needed to reorder UNION operators with other algebraic operators. The most important equivalences are

$$e_1 \cup e_2 \equiv e_2 \cup e_1 \quad (33.1)$$

$$e_1 \cup (e_2 \cup e_3) \equiv (e_1 \cup e_2) \cup e_3 \quad (33.2)$$

$$\sigma_p(e_1 \cup e_2) \equiv \sigma_p(e_1) \cup \sigma_p(e_2) \quad (33.3)$$

$$\chi_{a:e}(e_1 \cup e_2) \equiv \chi_{a:e}(e_1) \cup \chi_{a:e}(e_2) \quad (33.4)$$

$$(e_1 \cup e_2) \bowtie_p e_3 \equiv (e_1 \bowtie_p e_3) \cup (e_2 \bowtie_p e_3) \quad (33.5)$$

Equivalences containing the UNION operator sometimes involve tricky typing constraints. These go beyond the current chapter and the reader is referred to [613].

33.3 Cardinalities and Cost Functions

Chapter 34

Issues Introduced by XPath

34.1 A Naive XPath-Interpreter and its Problems

34.2 Dynamic Programming and Memoization

[338, 340, 339]

34.3 Naive Translation of XPath to Algebra

34.4 Pushing Duplicate Elimination

34.5 Avoiding Duplicate Work

34.6 Avoiding Duplicate Generation

[416]

34.7 Index Usage and Materialized Views

[48]

34.8 Cardinalities and Costs

34.9 Bibliography

Chapter 35

Issues Introduced by XQuery

35.1 Reordering in Ordered Context

35.2 Result Construction

[281, 282] [797]

35.3 Unnesting Nested XQueries

Unnesting with error: [673]
[593, 595, 594, 596]

35.4 Cardinalities and Cost Functions

cardinality: [168, 941, 942, 761] [7]
XPathLearner: [560]
Polyzotis et al (XSKETCH): [695, 696, 693], [697]

35.5 Bibliography

- [599] [883] [233]
Numbering: [280] Timber [459] TAX Algebra [462], physical algebra of Timber [674]
Structural Joins [20, 827]
SAL: [70], TAX: [462], XAL: [293]
- XML Statistics for hidden web: [8]
 - XPath selectivity for internet scale: [7]
 - StatiX: [294]
 - IMAX: incremental statistics [711]
 - Metrics for XML Document Collections: [499]

- output size containment join: [909]
- Bloom Histogram: [910]

View and XML: [2]

Quilt: [141]

Timber: [459] Monet: [774] Natix: NoK: [970]

Correlated XPath: [971]

Wood: [933, 934, 935]

Path based approach to Storage (XRel): [961]

Grust: [370, 372, 371, 373, 873]

Liefke: Loop fusion etc.: [559]

Benchmarking: XMach-1: [98], MBench: [751] XBench: [667, 952, 953],

XMark: [775] XOO7: [108]

Rewriting: [237, 363, 364]

[236, 428]

Incremental Schema Validation: [102, 672]

Franklin (filtering): [240]

Chapter 36

Outlook

What we did not talk about: multiple query optimization, semantic query optimization, special techniques for optimization in OBMSs, multi-media data bases, object-relational databases, spatial databases, temporal databases, and query optimization for parallel and distributed database systems.

Multi Query Optimization? [787]

Parametric/Dynamic/Adaptive Query Optimization? [33, 34, 35, 29, 361, 354] [455, 456, 474, 890]
[45]

Parallel Database Systems?

Distributed Database Systems? [514]

Recursive Queries?

Multi Database Systems?

Temporal Database Systems?

Spatial Database Systems?

Translation of Triggers and Updates?

Online Queries (Streams)?

Approximate Answers? [330]

Appendix A

Query Languages?

A.1 Designing a query language

requirements

design principles for object-oriented query languages: [427] [83]

A.2 SQL

A.3 OQL

A.4 XPath

A.5 XQuery

A.6 Datalog

Appendix B

Query Execution Engine (?)

- Overview Books: [403, 316]
- Overview: Graefe [347, 348]
- Implementation of Division [344, 353, 355]
- Implementation of Division and set-containment joins [708]
- Hash vs. Sort: [349, 358]
- Heap-Filter Merge Join: [346]
- Hash-Teams

Appendix C

Glossary of Rewrite and Optimization Techniques

trivopt Triviale Auswertungen bspw. solche für widersprüchliche Prädikate werden sofort vorgenommen. Dies ist eine Optimierungstechnik, die oft bereits auf der Quellebene durchgeführt wird.

pareval Falls ein Glied einer Konjunktion zu *false* evaluiert, werden die restlichen Glieder nicht mehr evaluiert. Dies ergibt sich automatisch durch die Verwendung von hintereinanderausgeführten Selektionen.

pushnot Falls ein Prädikat die Form $\neg(p_1 \wedge p_2)$ hat, so ist *pareval* nicht anwendbar. Daher werden Negationen nach innen gezogen. Auf $\neg p_1 \vee \neg p_2$ ist *pareval* dann wieder anwendbar. Das Durchschieben von Negationen ist auch im Kontext von NULL-Werten unabdingbar für die Korrektheit. Dies ist eine Optimierungstechnik, die oft bereits auf der Quellebene durchgeführt wird.

bxp Verallgemeinert man die in *pareval* und *notpush* angesprochene Problematik, so führt dies auf die Optimierung von allgemeinen booleschen Prädikaten.

trans Durch Ausnutzen der Transitivität von Vergleichsoperationen können neue Selektionsprädikate gewonnen und Konstanten propagiert werden. Diese Optimierungstechnik erweitert den Suchraum und wird ebenfalls auf der Quellebene durchgeführt. Bei manchen Systemen wird dieser Schritt nicht durchgeführt, falls sehr viele Relationen zu joinen sind, um den Suchraum nicht noch weiter zu vergrößern [322, 323].

selpush Selektionen werden so früh wie möglich durchgeführt. Diese Technik führt nicht immer zu optimalen Auswertungsplänen und stellt somit eine Heuristik dar. Diese Optimierungstechnik schränkt den Suchraum ein.

projpush Die Technik zur Behandlung von Projektionen ist nicht ganz so einfach wie die der Selektion. Zu unterscheiden ist hier, ob es sich um eine Projektion mit Duplikateliminierung handelt oder nicht. Je nach dem

ist es sinnvoll, die Projektion zur Wurzel des Operatorgraphen zu verschieben oder zu den Blättern hin. Die Projektion verringert den Speicherbedarf von Zwischenergebnissen, da die Tupel weniger Attribute enthalten. Handelt es sich um eine duplikateliminiierende Projektion, so wird möglicherweise auch die Anzahl der Tupel verringert. Duplikatelimination als solche ist aber eine sehr teure Operation. Diese wird üblicherweise durch Sortieren implementiert. Bei großen Datenmengen gibt es allerdings bessere Alternativen. Auch Hash-basierte Verfahren eignen sich zur Duplikateliminierung. Diese Optimierungstechnik schränkt den Suchraum ein.

grouppush Pushing a grouping operation past a join can lead to better plans.

crossjoin Ein Kreuzprodukt, das von einer Selektion gefolgt wird, wird wenn immer möglich in eine Verbundoperation umgewandelt. Diese Optimierungstechnik schränkt den Suchraum ein, da Pläne mit Kreuzprodukten vermieden werden.

nocross Kreuzprodukte werden wenn immer möglich vermieden oder, wenn dies nicht möglich ist, erst so spät wie möglich durchgeführt. Diese Technik verringert den Suchraum, führt aber nicht immer zu optimalen Auswertungsplänen.

semjoin Eine Verbundoperation kann durch eine Semiverbundoperation ersetzt werden, wenn nur die Attribute einer Relation weitere Verwendung finden.

joinor Die Auswertungsreihenfolge von Verbundoperationen ist kritisch. Daher wurden eine Reihe von Verfahren entwickelt, die optimale oder quasi-optimale Reihenfolge von Verbundoperationen zu bestimmen. Oft wird dabei der Suchraum auf Listen von Verbundoperationen beschränkt. Die Motivation hierbei ist das Verkleinern des Suchraums und die Beschränkung auf nur eine zu erzeugenden Zwischenrelation. Dieses Verfahren garantiert nicht mehr ein optimales Ergebnis.

joinpush Tables that are guaranteed to produce a single tuple are always pushed to be joined first. This reduces the search space. The single tuple condition can be evaluated by determining whether all key attributes of a relation are fully qualified. [322, 323].

elimredjoin Eliminate redundant join operations. See Sections. . . XXX

indnest Eine direkte Evaluierung von geschachtelten Anfragen wird durch geschachtelte Schleifen vorgenommen. Dabei wird eine Unteranfrage für jede erzielte Bindung der äußeren Anfrage evaluiert. Dies erfordert quadratischen Aufwand und ist deshalb sehr ineffizient. Falls die innere Anfrage unabhängig von der äußeren Anfrage evaluiert werden kann, so wird diese herausgezogen und getrennt evaluiert. Weitere Optimierungen geschachtelter Anfragen sind möglich.

unnest Entschachtelung von Anfragen [189, 191, 314, 494, 500, 501, 689, 830, 832, 833]

compop Oft ist es sinnvoll, mehrere Operationen zu einer komplexeren zusammenzufassen. Beispielsweise können zwei hintereinander ausgeführte Selektionen durch eine Selektion mit einem komplexeren Prädikat ersetzt werden. Ebenso kann auch das Zusammenfassen von Verbundoperationen, Selektionen und Projektionen sinnvoll sein.

comsubexpr Gemeinsame Teilausdrücke werden nur einfach evaluiert. Hierunter fallen zum einen Techniken, die das mehrmalige Lesen vom Hintergrundspeicher verhindern, und zum anderen Techniken, die Zwischenergebnisse von Teilausdrücken materialisieren. Letzteres sollte nur dann angewendet werden, falls die k-malige Auswertung teurer ist als das einmalige Auswerten und das Erzeugen des Ergebnisses mit k-maligem Lesen, wobei k die Anzahl der Vorkommen im Plan ist.

dynminmax Dynamisch gewonnene Minima und Maxima von Attributwerten können für die Erzeugung von zusätzlichen Restriktionen herangezogen werden. Diese Technik funktioniert auch sehr gut für unkorrelierte Anfragen. Dabei werden min- und max-Werte herangezogen um zusätzliche Restriktionen für die Anfrage zu gewinnen. [500, 322, 323]

pma Predicate Move around moves predicates between queries and subqueries. Mostly they are duplicated in order to yield as many restrictions in a block as possible [551]. As a special case, predicates will be pushed into view definitions if they have to be materialized temporarily [322, 323].

exproj For subqueries with exist prune unnecessary entries in the **select** clause. The intention behind is that attributes projected unnecessarily might influence the optimizer's decision on the optimal access path [322, 323].

vm View merging expands the view definition within the query such that it can be optimized together with the query. Thereby, duplicate accesses to the view are resolved by different copies of the views definition in order to facilitate unnesting [322, 323, 689].

inConstSet2Or A predicate of the form $x \in \{a_1, \dots, a_n\}$ is transformed into a sequence of disjunctions $x = a_1 \vee \dots \vee x = a_n$ if the a_i are constants in order to allow index or-ing (TID list operations or bitvector operations) [322, 323].

like1 If the like predicate does not start with %, then a prefix index can be used.

like2 The pattern is analyzed to see whether a range of values can be extracted such that the pattern does not have to be evaluated on all tuples. The result is either a pretest or an index access. [322, 323].

like3 Special indexes supporting like predicates are introduced.

sort Vorhandene Sortierungen können für verschiedene Operatoren ausgenutzt werden. Falls keine Sortierung vorhanden ist, kann es sinnvoll sein, diese zu erzeugen [818]. Z.B. aufeinanderfolgende joins, joins und gruppierungen. Dabei kann man die Gruppierungsattribute permutieren, um sie mit einer gegebenen Sortierreihenfolge in Einklang zu bringen [322, 323].

aps Zugriffspfade werden eingesetzt, wann immer dies gewinnbringend möglich ist. Beispielsweise kann die Anfrage

select count(*) **from** R;

durch einen Indexscan effizient ausgewertet werden [171].

tmpidx Manchmal kann es sinnvoll sein, temporäre Zugriffspfade anzulegen.

optimpl Für algebraische Operatoren existieren im allgemeinen mehrere Implementierungen. Es sollte hier immer die für einen Operator im vorliegenden Fall billigste Lösung ausgewählt werden. Ebenfalls von Bedeutung ist die Darstellung des Zwischenergebnisses. Beispielsweise können Relationen explizit oder implizit dargestellt werden, wobei letztere Darstellung nur Zeiger auf Tupel oder Surrogate der Tupel enthält. Weitergedacht führt diese Technik zu den TID-Listen-basierten Operatoren.

setpipe Die Evaluation eines algebraischen Ausdrucks kann entweder mengenorientiert oder nebenläufig (pipelining) erfolgen. Letzteres erspart das Erzeugen von großen Zwischenergebnissen.

tmpplay Das temporäre Ändern eines Layouts eines Objektes kann durchaus sinnvoll sein, wenn die Kosten, die durch diese Änderung entstehen, durch den Gewinn der mehrmaligen Verwendung dieses Layouts mehr als kompensiert werden. Ein typisches Beispiel ist *Pointer-swizzling*.

matSubQ If a query is not unnested, then for every argument combination passed to the subquery, the result is materialized in order to avoid duplicate computation of the same subquery expression for the same argument combination [322, 323]. This technique is favorable for detachment [845, 931, 962]

AggrJoin Joins with non-equi join predicates based on $\leq$ or $<$, can be processed more efficiently than by a cross product with a subsequent selection [193].

ClassHier Class hierarchies involve the computation of queries over a union of extents (if implemented that way). Pushing algebraic operations past unions allows often for more efficient plans [195].

AggrIDX Use an index to determine aggregate values like min/max/avg/count.

rid/tidsort When several tuples qualify during an index scan, the resulting TIDs can be sorted in order to guarantee sequential access to the base relation.

multIDX Perform operations like union and disjunction on the outcome of an index scan.

multIDXsplit If two ranges are queried within the same query ([1-10],[20-30]) consider multIDX or use a single scan through the index [1-30] with an additional qualification predicate.

multIDXor Queries with more conditions on indexed attributes can be evaluated by more complex combinations of index scans and tid-list/bitvector operations. ($A = 5$ and ($B = 3$ or $B = 4$)).

scanDirChange During multiple sequential scans of relation (e.g. for a block-wise nested loop join), the direction of the scan can be changed in order to reuse as much of the pages in the buffer as possible.

lock The optimizer should chose the correct locks to set on tables. For example, if a whole table is scanned, a table lock should be set.

expFunMat Expensive functions can be cached during query evaluation in order to avoid their multiple evaluation for the same arguments [414].

expFunFil Easier to evaluate predicates that are implied by more expensive predicates can serve as filters in order to avoid the evaluation of the expensive predicate on all tuples.

stop Stop evaluation after the first tuple qualifies. This is good for existential subqueries, universal subqueries (disqualify), semi-joins for distinct results and the like.

expensive projections

1. zum Schluss, da dort am wenigsten verschiedene Werte
2. durchschieben, falls cache fuer Funktionsergebnisse dadurch vermieden werden kann

OO-Kontext: problematisch: objekte muessen fuer funktionen/methoden als ganzes vorhanden sein. daher ist eine einfache strategie nicht moeglich.

distinct/sorting `select distinct a,b,c`
 `...`
 `order by a,b`

kann auch nach a,b,c sortiert werden. stoert gar nicht, vereinfacht aber die duplikateliminierung. nur ein sortieren notwendig.

index access • by key

- by key range
- by dashed key range (set of keys/key ranges)
- index anding/oring

alternative operator implementations e.g. join: nlj bnlj hj grace-hash
hybrid-hash smj diag-join star-join

distpd Push-down or Pull-up distinct.

aggregate with distinct `select a, agg(distinct b)`

```
...
group by a
====>
sort on a,b
dup elim
group a,sum(b)
```

alternative: `aggr(distinct *)` is implemented such that it uses a hashtable to eliminate duplicates this is only good, if the number of groups is small and the number of distinct values in each group is small.

XXX - use keys, inclusion dependencies, fds etc. (all user specified and derived) (propagate keys over joins as fds), (for a function call: derived IU is functional dependent on arguments of the function call if function is deterministic) (keys can be represented as sets of IUs or as bitvectors (given numbering of IUs)) (numbering imprecise: bitvectors can be used as filters (like for signatures))

Appendix D

Useful Formulas

The following identities can be found in the book by Graham, Knuth, and Patashnik [362].

We use the following definition of binomial coefficients:

$$\binom{n}{k} = \begin{cases} \frac{n!}{k!(n-k)!} & \text{if } 0 \leq k \leq n \\ 0 & \text{else} \end{cases} \quad (\text{D.1})$$

We start with some simple identities.

$$\binom{n}{k} = \binom{n}{n-k} \quad (\text{D.2})$$

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1} \quad (\text{D.3})$$

$$k \binom{n}{k} = n \binom{n-1}{k-1} \quad (\text{D.4})$$

$$(n-k) \binom{n}{k} = n \binom{n-1}{k} \quad (\text{D.5})$$

$$(n-k) \binom{n}{k} = n \binom{n-1}{n-k-1} \quad (\text{D.6})$$

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1} \quad (\text{D.7})$$

$$\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k} \quad (\text{D.8})$$

The following identities are good for sums of binomial coefficients.

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad (\text{D.9})$$

$$\sum_{k=0}^n \binom{k}{m} = \binom{n+1}{m+1} \quad (\text{D.10})$$

$$\sum_{k=0}^n \binom{m+k}{k} = \binom{m+n+1}{m+1} = \binom{m+n+1}{n} \quad (\text{D.11})$$

$$\sum_{k=0}^n \binom{m-n+k}{k} = \binom{m+1}{n} \quad (\text{D.12})$$

From Identities D.2 and D.11 it follows that

$$\sum_{k=0}^m \binom{k+r}{r} = \binom{m+r+1}{r+1} \quad (\text{D.13})$$

For sums of products, we have

$$\sum_{k=0}^n \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n} \quad (\text{D.14})$$

$$\sum_{k=0}^n \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1} \quad (\text{D.15})$$

$$\sum_{k=0}^n \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n} \quad (\text{D.16})$$

Last,

$$\sum_{k=0}^n k \binom{n}{k} = n2^{n-1}$$

Bibliography

- [1] K. Aberer and G. Fischer. Semantic query optimization for methods in object-oriented database systems. In *Proc. IEEE Conference on Data Engineering*, pages 70–79, 1995.
- [2] S. Abiteboul. On Views and XML. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 1–9, 1999.
- [3] S. Abiteboul, C. Beeri, M. Gyssens, and D. Van Gucht. An introduction to the completeness of languages for complex objects and nested relations. In S. Abiteboul, P.C. Fischer, and H.-J. Schek, editors, *Nested Relations and Complex Objects in Databases*, pages 117–138. Lecture PAGESs in Computer Science 361, Springer, 1987.
- [4] S. Abiteboul and N. Bidoit. Non first normal form relations: An algebra allowing restructuring. *Journal of Computer Science and Systems*, 33(3):361, 1986.
- [5] S. Abiteboul, S. Cluet, V. Christophides, T. Milo, G. Moerkotte, and J. Simeon. Querying documents in object databases. *International Journal on Digital Libraries*, 1(1):5–19, April 1997.
- [6] S. Abiteboul and O. Duschka. Complexity of answering queries using materialized views. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 254–263, 1998.
- [7] A. Abounaga, A. Alameldeen, and J. Naughton. Estimating the selectivity of XML path expressions for internet scale applications. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 591–600, 2001.
- [8] A. Abounaga and J. Naughton. Building XML statistics for the hidden web. In *Int. Conference on Information and Knowledge Management (CIKM)*, pages 358–365, 2003.
- [9] W. Abu-Sufah, D. J. Kuch, and D. H. Lawrie. On the performance enhancement of paging systems through program analysis and transformations. *IEEE Trans. on Computers*, C-50(5):341–355, 1981.
- [10] B. Adelberg, H. Garcia-Molina, and J. Widom. The STRIP rule system for efficiently maintaining derived data. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 147–158, 1997.

- [11] F. Afrati, M. Gergatsoulis, and T. Kavalieros. Answering queries using materialized views with disjunctions. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 435–452, 1999.
- [12] F. Afrati, C. Li, and J. Ullman. Generating efficient plans for queries using views. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 319–330, 2001.
- [13] F. Afrati and C. Papadimitriou. The parallel complexity of simple chain queries. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 210–?, 1987.
- [14] V. Aggelis and S. Cosmadakis. Optimization of nested sql queries by tableau equivalence. In *Int. Workshop on Database Programming Languages*, pages 31–42, 1999.
- [15] D. Agrawal, A. Abbadi, A. Singh, and T. Yurek. Efficient view maintenance at data warehouses. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 417–427, 1997.
- [16] A. Aho, P. Denning, and J. Ullman. Principles of optimal page replacement. *Journal of the ACM*, 18(1):80–93, 1971.
- [17] A. Aho, Y. Sagiv, and J. Ullman. Efficient optimization of a class of relational expressions. *ACM Trans. on Database Systems*, 4(4):435–454, 1979.
- [18] A. Aho, Y. Sagiv, and J. Ullman. Equivalence of relational expressions. *SIAM J. on Computing*, 8(2):218–246, 1979.
- [19] A. V. Aho, C. Beeri, and J. D. Ullman. The theory of joins in relational databases. *ACM Trans. on Database Systems*, 4(3):297–314, 1979.
- [20] S. Al-Khalifa, H. Jagadish, N. Koudas, J. Patel, D. Srivastava, and Y. Wu. Structural joins: A primitive for efficient XML query pattern matching. In *Proc. IEEE Conference on Data Engineering*, pages 141–152, 2002.
- [21] J. Albert. Algebraic properties of bag data types. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 211–219, 1991.
- [22] F. E. Allen and J. Cocke. A catalogue of optimizing transformations. In R. Rustin, editor, *Design and optimization of compilers*, pages 1–30. Prentice Hall, 1971.
- [23] N. Alon, P. Gibbons, Y. Matias, and M. Szegedy. Tracking join and self-join sizes in limited storage. *J. Comput System Sciences*, 35(4):391–432, 2002.
- [24] N. Alon, Y. Matias, and M. Szegedy. The space complexity of approximating the frequency moments. *J. of Computer and System Sciences*, 58(1):137–147, 1999.

- [25] J. Alsabbagh and V. Rahavan. Analysis of common subexpression exploitation models in multiple-query processing. In *Proc. IEEE Conference on Data Engineering*, pages 488–497, 1994.
- [26] P. Alsberg. Space and time savings through large database compression and dynamic restructuring. In *Proc IEEE 63,8*, Aug. 1975.
- [27] D. Donoho and M. Elad. Optimally sparse representation in general (non-orthogonal) dictionaries via ℓ_1 minimization. *Proc. of the National Academy of Sciences*, 100(5):2197–2202, 2003.
- [28] S. Amer-Yahia, S. Cho, L. Lakshmanan, and D. Srivastava. Efficient algorithms for minimizing tree pattern queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 497–508, 2001.
- [29] L. Amsaleg, M. Franklin, A. Tomasic, and T. Urhan. Scrambling query plans to cope with unexpected delay. In *4th Int. Conference on Parallel and Distributed Information Systems (PDIS)*, Palm Beach, FL, 1996.
- [30] C. Zuzarte and X. Yu. Fast approximate computation of statistics on views. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, page 724, 2006.
- [31] O. Anfinsen. A study of access path selection in DB2. Technical report, Norwegian Telecommunication Administration and University of Oslo, Norway, Oct. 1989.
- [32] G. Antoshenkov. Random sampling from pseudo-ranked b^+ -trees. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 375–382, 1992.
- [33] G. Antoshenkov. Dynamic query optimization in RDB/VMS. In *Proc. IEEE Conference on Data Engineering*, pages 538–547, Vienna, Apr. 1993.
- [34] G. Antoshenkov. Query processing in DEC Rdb: Major issues and future challenges. *IEEE Data Engineering Bulletin*, 16:42–52, Dec. 1993.
- [35] G. Antoshenkov. Dynamic optimization of index scans restricted by booleans. In *Proc. IEEE Conference on Data Engineering*, pages 430–440, 1996.
- [36] P. Aoki. Algorithms for index-assisted selectivity estimation. Technical Report UCB/CSD-98-1021, University of California, Berkeley, Oct 1998.
- [37] P.M.G. Apers, A.R. Hevner, and S.B. Yao. Optimization algorithms for distributed queries. *IEEE Trans. on Software Eng.*, 9(1):57–68, 1983.
- [38] P.M.G. Apers, A.R. Hevner, and S.B. Yao. Optimization algorithms for distributed queries. *IEEE Trans. on Software Eng.*, 9(1):57–68, 1983.
- [39] R. Ashenurst. Acm forum. *Communications of the ACM*, 20(8):609–612, 1977.

- [40] M. Astrahan, M. Schkolnick, and K. Whang. Counting unique values of an attribute without sorting. *Information Systems*, 12(1):11–15, 1987.
- [41] M. M. Astrahan and D. D. Chamberlin. Implementation of a structured English query language. *Communications of the ACM*, 18(10):580–588, 1975.
- [42] M.M. Astrahan, M.W. Blasgen, D.D. Chamberlin, K.P. Eswaran, J.N. Gray, P.P. Griffiths, W.F. King, R.A. Lorie, P.R. Mc Jones, J.W. Mehl, G.R. Putzolu, I.L. Traiger, B.W. Wade, and V. Watson. System R: relational approach to database management. *ACM Transactions on Database Systems*, 1(2):97–137, June 1976.
- [43] R. Avnur and J. Hellerstein. Eddies: Continuously adaptive query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, 2000.
- [44] B. Babcock and S. Chaudhuri. Towards a robust query optimizer: A principled and practical approach. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 119–130, 2005.
- [45] S. Babu, P. Bizarro, and D. DeWitt. Proactive re-optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 107–118, 2005.
- [46] L. Baekgaard and L. Mark. Incremental computation of nested relational query expressions. *ACM Trans. on Database Systems*, 20(2):111–148, 1995.
- [47] T. Baer. Iperfex: A hardware performance monitor for Linux/IA32 systems. perform internet search for this or similar tools.
- [48] A. Balmin, F. Özcan, K. Beyer, R. Cochrane, and H. Pirahesh. A framework for using materialized XPath views in XML query processing. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 60–71, 2004.
- [49] F. Bancilhon and K. Koshafian. A calculus for complex objects. In *ACM Symp. on Principles of Database Systems*, pages 53–59, 1986.
- [50] J. Banerjee, W. Kim, and K.-C. Kim. Queries in object-oriented databases. MCC Technical Report DB-188-87, MCC, Austin, TX 78759, June 1987.
- [51] J. Banerjee, W. Kim, and K.-C. Kim. Queries in object-oriented databases. In *Proc. IEEE Conference on Data Engineering*, pages 31–38, 1988.
- [52] Z. Bar-Yossef, T. Jayram, R. Kumar, D. Sivikumar, and L. Trevisan. Counting distinct elements in a data stream. In *6th Int. Workshop RAN-DOM 2002*, pages 1–10, 2002.

- [53] E. Baralis, S. Paraboschi, and E. Teniente. Materialized views selection in a multidimensional database. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 156–165, 1997.
- [54] D. Barbar'a, W. DuMouchel, C. Faloutsos, P. Haas, J. Hellerstein, Y. Ioannidis, H. Jagadish, T. Johnson, R. Ng, V. Poosala, K. Ross, and K. Sevcik. The New Jersey data reduction report. *IEEE Data Engineering Bulletin*, 20(4):3–45, 1997.
- [55] M. Bassiouni. Data compression in scientific and statistical databases. *IEEE Trans. on Software Eng.*, 11(10):1047–1058, 1985.
- [56] D. Batory. On searching transposed files. *ACM Trans. on Database Systems*, 4(4):531–544, 1979.
- [57] D. Batory. Extensible cost models and query optimization in Genesis. *IEEE Database Engineering*, 9(4), Nov 1986.
- [58] D. S. Batory. Modeling the storage architecture of commercial database systems. *ACM Trans. on Database Systems*, 10(4):463–528, Dec. 1985.
- [59] D. S. Batory. A molecular database systems technology. Tech. Report TR-87-23, University of Austin, 1987.
- [60] D. S. Batory. Building blocks of database management systems. Technical Report TR-87-23, University of Texas, Austin, TX, Feb. 1988.
- [61] D. S. Batory. Concepts for a database system compiler. In *Proc. of the 17th ACM SIGMOD*, pages 184–192, 1988.
- [62] D. S. Batory. On the reusability of query optimization algorithms. *Information Sciences*, 49:177–202, 1989.
- [63] D. S. Batory and C. Gotlieb. A unifying model of physical databases. *ACM Trans. on Database Systems*, 7(4):509–539, Dec. 1982.
- [64] D. S. Batory, T. Y. Leung, and T. E. Wise. Implementation concepts for an extensible data model and data language. *ACM Trans. on Database Systems*, 13(3):231–262, Sep 1988.
- [65] L. Baxter. *The Complexity of Unification*. PhD thesis, University of Waterloo, 1976.
- [66] L. Becker and R. H. Güting. Rule-based optimization and query processing in an extensible geometric database system. *ACM Trans. on Database Systems (to appear)*, 1991.
- [67] L. Becker and R. H. Güting. Rule-based optimization and query processing in an extensible geometric database system. *ACM Trans. on Database Systems*, 17(2):247–303, June 1992.

- [68] C. Beeri and Y. Kornatzky. Algebraic optimization of object-oriented query languages. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 72–88, 1990.
- [69] C. Beeri and Y. Kornatzky. Algebraic optimization of object-oriented query languages. *Theoretical Computer Science*, 116(1):59–94, 1993.
- [70] C. Beeri and Y. Tzaban. SAL: An algebra for semistructured data and XML. In *ACM SIGMOD Workshop on the Web and Databases (WebDB)*, 1999.
- [71] L. A. Belady. A study of replacement algorithms for virtual storage computers. *IBM Systems Journal*, 5(2):78–101, 1966.
- [72] S. Bellamkonda, R. Ahmed, A. Witkowski, A. Amor, M. Zait, and C.-C. Lin. Enhanced subquery optimization in Oracle. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 1366–1377, 2009.
- [73] D. Beneventano, S. Bergamaschi, and C. Sartori. Description logic for semantic query optimization in object-oriented database systems. *ACM Trans. on Database Systems*, 28(1):1–50, 2003.
- [74] K. Bennett, M. Ferris, and Y. Ioannidis. A genetic algorithm for database query optimization. Technical Report Tech. Report 1004, University of Wisconsin, 1990.
- [75] K. Bennett, M. Ferris, and Y. Ioannidis. A genetic algorithm for database query optimization. In *Proc. 4th Int. Conf. on Genetic Algorithms*, pages 400–407, 1991.
- [76] J. L. Bentley and A. C.-C. Yao. An almost optimal algorithm for unbounded searching. *Inf. Proc. Lett.*, 5(3):82–87, 1976.
- [77] A. Bernasconi and B. Codenetti. Measures of boolean function complexity based on harmonic analysis. In M. Bonuccelli, P. Crescenzi, and R. Petreschi, editors, *Algorithms and Complexity (2nd. Italian Conference)*, pages 63–72, Rome, Feb. 1994. Springer (Lecture Notes in Computer Science 778).
- [78] P. Bernstein, E. Wong, C. Reeve, and J. Rothnie. Query processing in a system for distributed databases (sdd-1). *ACM Trans. on Database Systems*, 6(4):603–625, 1981.
- [79] P. A. Bernstein and D. M. W. Chiu. Using semi-joins to solve relational queries. *Journal of the ACM*, 28(1):25–40, 1981.
- [80] P. A. Bernstein and N. Goodman. The power of inequality semijoin. *Information Systems*, 6(4):255–265, 1981.
- [81] P. A. Bernstein and N. Goodman. The power of natural semijoin. *SIAM J. Comp.*, 10(4):751–771, 1981.

- [82] E. Bertino and D. Musto. Query optimization by using knowledge about data semantics. *Data & Knowledge Engineering*, 9(2):121–155, 1992.
- [83] E. Bertino, M. Negri, G. Pelagatti, and L. Sbattella. Object-oriented query languages: The notion and the issues. *IEEE Trans. on Knowledge and Data Eng.*, 4(3):223–237, June 1992.
- [84] K. Beyer, P. Haas, B. Reinwald, Y. Sismanis, and R. Gemulla. On synopses for distinct-value estimation under multiset operations. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 199–210, 2007.
- [85] G. Bhargava, P. Goel, and B. Iyer. Hypergraph based reorderings of outer join queries with complex predicates. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 304–315, 1995.
- [86] G. Bhargava, P. Goel, and B. Iyer. No regression algorithm for the enumeration of projections in SQL queries with joins and outer joins. In *IBM Centre for Advanced Studies Conference (CASCOM)*, 1995.
- [87] G. Bhargava, P. Goel, and B. Iyer. Simplification of outer joins. In *IBM Centre for Advanced Studies Conference (CASCOM)*, 1995.
- [88] G. Bhargava, P. Goel, and B. Iyer. Efficient processing of outer joins and aggregate functions. In *Proc. IEEE Conference on Data Engineering*, pages 441–449, 1996.
- [89] A. Biliris. An efficient database storage structure for large dynamic objects. In *Proc. IEEE Conference on Data Engineering*, pages 301–308, 1992.
- [90] J. Biskup. A formal approach to null values in database relations. In *Advances in Database Theory*, 1981.
- [91] D. Bitton and D. DeWitt. Duplicate record elimination in large data files. *ACM Trans. on Database Systems*, 8(2):255–265, 1983.
- [92] S. Bitzer. Design and implementation of a query unnesting module in natix. Master’s thesis, University of Mannheim, 2007.
- [93] J. Blakeley and N. Martin. Join index, materialized view, and hybrid hash-join: a performance analysis. In *Proc. IEEE Conference on Data Engineering*, pages 256–236, 1990.
- [94] J. Blakeley, W. McKenna, and G. Graefe. Experiences building the Open OODB query optimizer. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 287–295, 1993.
- [95] J. A. Blakeley, P. A. Larson, and F. W. Tompa. Efficiently updating materialized views. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 61–71, Washington, D.C., 1986.

- [96] B. Blohsfeld, D. Korus, and B. Seeger. A comparison of selectivity estimators for range queries on metric attributes. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 239–250, 1999.
- [97] P. Bodorik and J.S. Riordon. Distributed query processing optimization objectives. In *Proc. IEEE Conference on Data Engineering*, pages 320–329, 1988.
- [98] T. Böhme and E. Rahm. Xmach-1: A benchmark for XML data management. In *BTW*, pages 264–273, 2001.
- [99] A. Bolour. Optimal retrieval for small range queries. *SIAM J. of Comput.*, 10(4):721–741, 1981.
- [100] P. Bonatti. On the decidability of containment of recursive datalog queries - preliminary report. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 297–306, 2004.
- [101] P. Boncz, A. Wilschut, and M. Kersten. Flattening an object algebra to provide performance. In *Proc. IEEE Conference on Data Engineering*, pages 568–577, 1998.
- [102] B. Bouchou, M. Halfeld, and F. Alves. Updates and incremental validation of XML documents. In *Int. Workshop on Database Programming Languages*, pages 216–232, 2003.
- [103] M. Brantner, S. Helmer, C.-C. Kanne, and G. Moerkotte. Full-fledged algebraic XPath processing in Natix. In *Proc. IEEE Conference on Data Engineering*, pages 705–716, 2005.
- [104] M. Brantner, N. May, and G. Moerkotte. Unnesting SQL queries in the presence of disjunction. Technical Report TR-2006-013, University of Mannheim, 2006.
- [105] M. Brantner, N. May, and G. Moerkotte. Unnesting scalar SQL queries in the presence of disjunction. In *Proc. IEEE Conference on Data Engineering*, 2007. 46–55.
- [106] K. Bratbergsengen and K. Norvag. Improved and optimized partitioning techniques in database query processing. In *Advances in Databases, 15th British National Conference on Databases*, pages 69–83, 1997.
- [107] Y. Breitbart and A. Reiter. Algorithms for fast evaluation of boolean expressions. *Acta Informatica*, 4:107–116, 1975.
- [108] S. Bressan, M. Lee, Y. Li, Z. Lacroix, and U. Nambiar. The XOO7 XML Management System Benchmark. Technical Report TR21/00, National University of Singapore, 2001.
- [109] K. Brown, M. Carey, and M. Livny. Goal-oriented buffer management revisited. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 353–364, Montreal, Canada, Jun 1996.

- [110] N. Bruno, S. Chaudhuri, and L. Gravano. STHoles: a multidimensional workload-aware histogram. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 211–222, 2001.
- [111] N. Bruno, S. Chaudhuri, and L. Gravano. STHoles: a multidimensional workload-aware histogram. Technical Report MSR-TR-2001-36, Microsoft Research, 2001.
- [112] N. Bruno, C. Galindo-Legaria, and M. Joshi. Polynomial heuristics for query optimization. In *Proc. IEEE Conference on Data Engineering*, pages 589–600, 2010.
- [113] F. Bry. Logical rewritings for improving the evaluation of quantified queries. In *2nd. Symp. on Mathematical Fundamentals of Database Systems*, pages 100–116, June 1989, Visegrad, Hungary, 1989.
- [114] F. Bry. Towards an efficient evaluation of general queries: Quantifiers and disjunction processing revisited. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 193–204, 1989.
- [115] F. Buccafurri and G. Lax. Fast range query estimation by n-level tree histograms. *Data & Knowledge Engineering*, 51:257–275, 2004.
- [116] F. Buccafurri, G. Lax, D. Sacca, L. Pontieri, and D. Rosaci. Enhancing histograms by tree-like bucket indices. *The VLDB Journal*, 17:1041–1061, 2008.
- [117] F. Buccafurri, L. Pontieri, D. Rosaci, and D. Sacca. Improving range query estimation on histograms. In *Proc. IEEE Conference on Data Engineering*, pages 628–638, 2002.
- [118] P. Buneman, S. Davidson, W. Fan, C. Hara, and W. Tan. Keys for XML. In *WWW Conference*, pages 201–210, 2001.
- [119] P. Buneman, L. Libkin, D. Suciu, V. Tannen, and L. Wong. Comprehension syntax. *SIGMOD Record*, 23(1):87–96, 1994.
- [120] L. Cabibbo and R. Torlone. A framework for the investigation of aggregate functions in database queries. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 383–397, 1999.
- [121] J.-Y. Cai, V. Chakaravarthy, R. Kaushik, and J. Naughton. On the complexity of join predicates. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 207–214, 2001.
- [122] D. Calvanese, G. DeGiacomo, M. Lenzerini, and M. Vardi. View-based query answering and query containment over semistructured data. In *Int. Workshop on Database Programming Languages*, pages 40–61, 2001.
- [123] B. Cao and A. Badia. A nested relational approach to processing sql subqueries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 191–202, 2005.

- [124] L. Cardelli and P. Wegner. On understanding types, data abstraction, and polymorphism. *JOURNAL of the ACM*, Computing Surveys:471–522, 1985.
- [125] A. F. Cardenas. Analysis and performance of inverted data base structures. *Communications of the ACM*, 18(5):253–263, 1975.
- [126] M. Carey, D. DeWitt, J. Richardson, and E. Shikita. Object and file management in the EXODUS extensible database system. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 91–100, 1986.
- [127] M. Carey and D. Kossmann. On saying “enough already!” in SQL. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 219–230, 1997.
- [128] M. Carey and D. Kossmann. Processing top N and bottom N queries. *IEEE Data Engineering Bulletin*, 20(3):12–19, 1997.
- [129] M. Carey and D. Kossmann. Reducing the braking distance of an SQL query engine. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 158–169, 1998.
- [130] J. Carlis. HAS: A relational algebra operator, or divide is not to conquer. In *Proc. IEEE Conference on Data Engineering*, pages 254–261, 1986.
- [131] L. Carlitz, D. Roselle, and R. Scoville. Some remarks on ballot-type sequences of positive integers. *Journal of Combinatorial Theory*, 11:258–271, 1971.
- [132] C. R. Carlson and R. S. Kaplan. A generalized access path model and its application to a relational database system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 143–154, 1976.
- [133] R. Cattell, D. Barry, M. Berler, J. Eastman, D. Jordan, C. Russel, O. Schadow, T. Stanienda, and F. Velez, editors. *The Object Database Standard: ODMG 3.0*. Morgan Kaufmann, 1999. Release 3.0.
- [134] S. Ceri and G. Gottlob. Translating SQL into relational algebra: Optimization, semantics and equivalence of SQL queries. *IEEE Trans. on Software Eng.*, 11(4):324–345, Apr 1985.
- [135] S. Ceri and G. Pelagatti. Correctness of query execution strategies in distributed databases. *ACM Trans. on Database Systems*, 8(4):577–607, Dec. 1983.
- [136] S. Ceri and G. Pelagatti. *Distributed Databases: Principles and Systems*. McGraw-Hill, 1985.
- [137] S. Chakkappen, T. Curanes, B. Dageville, L. Jiang, U. Shaft, H. Su, and M. Zait. Efficient statistics gathering for large databases in Oracle 11g. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 1053–1064, 2008.

- [138] S. Chakravarthy. Devide and conquer: a basis for augmenting a conventional query optimizer with multiple query processing capabilities. In *Proc. IEEE Conference on Data Engineering*, 1991.
- [139] U. S. Chakravarthy, J. Grant, and J. Minker. Logic-based approach to semantic query optimization. *ACM Trans. on Database Systems*, 15(2):162–207, 1990.
- [140] U. S. Chakravarthy and J. Minker. Multiple query processing in deductive databases using query graphs. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages ?–?, 1986.
- [141] D. Chamberlin, J. Robie, and D. Florescu. Quilt: An XML query language for heterogeneous data sources. In *ACM SIGMOD Workshop on the Web and Databases (WebDB)*, 2000.
- [142] C. Chan and B. Ooi. Efficient scheduling of page accesses in index-based join processing. *IEEE Trans. on Knowledge and Data Eng.*, 9(6):1005–1011, 1997.
- [143] A. Chandra and P. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proc. ACM SIGACT Symp. on the Theory of Computing*, pages 77–90, 1977.
- [144] A. K. Chandra and P. M. Merlin. Optimal implementation of conjunctive queries in relational data bases. In *Proc. 9th ACM Symposium on Theory of Computing*, pages 77–90, 1976.
- [145] S. Chatterji, S. Evani, S. Ganguly, and M. Yemmanuru. On the complexity of approximate query optimization. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 282–292, 2002.
- [146] D. Chatziantoniou, M. Akinde, T. Johnson, and S. Kim. The MD-Join: An Operator for Complex OLAP. In *Proc. IEEE Conference on Data Engineering*, pages 524–533, 2001.
- [147] D. Chatziantoniou and K. Ross. Querying multiple features in relational databases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 295–306, 1996.
- [148] D. Chatziantoniou and K. Ross. Groupwise processing of relational queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 476–485, 1997.
- [149] S. Chaudhuri, P. Ganesan, and S. Sarawagi. Factorizing complex predicates in queries to exploit indexes. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 361–372, 2003.
- [150] S. Chaudhuri and L. Gravano. Evaluating top-k selection queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 397–410, 1999.

- [151] S. Chaudhuri, R. Krishnamurthy, S. Potamianos, and K. Shim. Optimizing queries with materialized views. In *Proc. IEEE Conference on Data Engineering*, pages 190–200, 1995.
- [152] S. Chaudhuri, R. Krishnamurthy, S. Potamianos, and K. Shim. *Optimizing Queries with Materialized Views*, pages 77–92. MIT Press, 1999.
- [153] S. Chaudhuri, V. Narasayya, and R. Ramamurthy. Estimating progress of long running sql queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 803–814, 2004.
- [154] S. Chaudhuri and K. Shim. Query optimization in the presence of foreign functions. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 529–542, 1993.
- [155] S. Chaudhuri and K. Shim. Including group-by in query optimization. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 354–366, 1994.
- [156] S. Chaudhuri and K. Shim. Optimization of queries with user-defined predicates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 87–98, 1996.
- [157] S. Chaudhuri and K. Shim. Optimizing queries with aggregate views. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 167–182, 1996.
- [158] S. Chaudhuri and M. Vardi. On the equivalence of recursive and nonrecursive datalog programs. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 55–66, 1992.
- [159] S. Chaudhuri and M. Vardi. Optimization of real conjunctive queries. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 59–70, 1993.
- [160] S. Chaudhuri and M. Vardi. Optimization of real conjunctive queries. Technical Report HPL-93-26, HP Software Technology Laboratory, 1993.
- [161] S. Chaudhuri and M. Vardi. On the complexity of equivalence between recursive and nonrecursive datalog programs. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 55–66, 1994.
- [162] P. Cheeseman, B. Kanefsky, and W. Taylor. Where the *really* hard problems are. In *Int. Joint Conf. on Artificial Intelligence (IJCAI)*, pages 331–337, 1991.
- [163] C. Chekuri and A. Rajaraman. Conjunctive query containment revisited. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 56–70, 1997.
- [164] C. Chekuri and A. Rajaraman. Conjunctive query containment revisited. *Theoretical Computer Science*, 239:211–229, 2000.

- [165] A.L.P. Chen. Outerjoin optimization in multidatabase systems. In *Proc. 2nd. Int. Symp. on Databases in Parallel and Distributed Systems*, pages 211–218, 1990.
- [166] C. Chen and N. Roussopoulos. The implementation and performance evaluation of the ADMS query optimizer: Integrating query result caching and matching. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 323–336, 1994.
- [167] Z. Chen, J. Gehrke, and F. Korn. Query optimization in compressed database systems. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 271–282, 2001.
- [168] Z. Chen, H. V. Jagadish, F. Korn, N. Koudas, S. Muthukrishnan, R. Ng, and D. Srivastava. Counting twig matches in a tree. In *Proc. IEEE Conference on Data Engineering*, pages 595–604, 2001.
- [169] Z. Chen and V. Narasayya. Efficient computation of multiple group by queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 263–274, 2005.
- [170] J. Cheng, D. Haderle, R. Hedges, B. Iyer, T. Messenger, C. Mohan, and Y. Wang. An efficient hybrid join algorithm: A DB2 prototype. In *Proc. IEEE Conference on Data Engineering*, pages 171–180, 1991.
- [171] J. Cheng, C. Loosley, A. Shibamiya, and P. Worthington. IBM DB2 Performance: design, implementation, and tuning. *IBM Sys. J.*, 23(2):189–210, 1984.
- [172] Q. Cheng, J. Gryz, F. Koo, T. Y. Cliff Leung, L. Liu, X. Quian, and B. Schiefer. Implementation of two semantic query optimization techniques in DB2 universal database. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 687–698, 1999.
- [173] M. Cherniack and S. Zdonik. Rule languages and internal algebras for rule-based optimizers. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 401–412, 1996.
- [174] T.-Y. Cheung. Estimating block accesses and number of records in file management. *Communications of the ACM*, 25(7):484–487, 1982.
- [175] D. M. Chiu and Y. C. Ho. A methodology for interpreting tree queries into optimal semi-join expressions. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 169–178, 1980.
- [176] H.-T. Chou and D. DeWitt. An evaluation of buffer management strategies for relational database systems. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 127–141, 1985.
- [177] S. Christodoulakis. Estimating block transfers and join sizes. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 40–54, 1983.

- [178] S. Christodoulakis. Estimating record selectivities. *Information Systems*, 8(2):105–115, 1983.
- [179] S. Christodoulakis. Implications of certain assumptions in database performance evaluation. *ACM Trans. on Database Systems*, 9(2):163–186, 1984.
- [180] S. Christodoulakis. Analysis of retrieval performance for records and objects using optical disk technology. *ACM Trans. on Database Systems*, 12(2):137–169, 1987.
- [181] V. Christophides, S. Cluet, and G. Moerkotte. Evaluating queries with generalized path expressions. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 413–422, 1996.
- [182] C. Clarke, G. Cormack, and F. Burkowski. An algebra for structured text search and a framework for its implementation. *The Computer Journal*, 38(1):43–56, 1995.
- [183] J. Claussen, A. Kemper, and D. Kossmann. Order-preserving hash joins: Sorting (almost) for free. Technical Report MIP-9810, University of Passau, 1998.
- [184] J. Claussen, A. Kemper, G. Moerkotte, and K. Peithner. Optimizing queries with universal quantification in object-oriented and object-relational databases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 286–295, 1997.
- [185] J. Claussen, A. Kemper, G. Moerkotte, and K. Peithner. Optimizing queries with universal quantification in object-oriented and object-relational databases. Technical Report MIP-9706, University of Passau, Fak. f. Mathematik u. Informatik, Mar 1997.
- [186] J. Claussen, A. Kemper, G. Moerkotte, K. Peithner, and M. Steinbrunn. Optimization and evaluation of disjunctive queries. *IEEE Trans. on Knowledge and Data Eng.*, 12(2):238–260, 2000.
- [187] S. Cluet and C. Delobel. A general framework for the optimization of object-oriented queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 383–392, 1992.
- [188] S. Cluet, C. Delobel, C. Lecluse, and P. Richard. Reloop, an algebra based query language for an object-oriented database system. In *Proc. Int. Conf. on Deductive and Object-Oriented Databases (DOOD)*, 1989.
- [189] S. Cluet and G. Moerkotte. Nested queries in object bases. In *Proc. Int. Workshop on Database Programming Languages*, pages 226–242, 1993.
- [190] S. Cluet and G. Moerkotte. Classification and optimization of nested queries in object bases. In *BDA*, pages 331–349, 1994.

- [191] S. Cluet and G. Moerkotte. Classification and optimization of nested queries in object bases. Technical Report 95-6, RWTH Aachen, 1995.
- [192] S. Cluet and G. Moerkotte. Efficient evaluation of aggregates on bulk types. Technical Report 95-5, RWTH-Aachen, 1995.
- [193] S. Cluet and G. Moerkotte. Efficient evaluation of aggregates on bulk types. In *Proc. Int. Workshop on Database Programming Languages*, 1995.
- [194] S. Cluet and G. Moerkotte. On the complexity of generating optimal left-deep processing trees with cross products. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 54–67, 1995.
- [195] S. Cluet and G. Moerkotte. Query optimization techniques exploiting class hierarchies. Technical Report 95-7, RWTH-Aachen, 1995.
- [196] E. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [197] E. Codd. *Database Systems - Courant Computer Science Symposium*. Prentice Hall, 1972.
- [198] E. F. Codd. A database sublanguage founded on the relational calculus. In *Proc. ACM-SIGFIDET Workshop, Datadescription, Access, and Control*, pages 35–68, San Diego, Calif., 1971. ACM.
- [199] E. F. Codd. Relational completeness of data base sublanguages. In *Courant Computer Science Symposia No. 6: Data Base Systems*, pages 67–101, New York, 1972. Prentice Hall.
- [200] E. F. Codd. Extending the relational database model to capture more meaning. *ACM Trans. on Database Systems*, 4(4):397–434, Dec. 1979.
- [201] S. Cohen, W. Nutt, and Y. Sagiv. Equivalences among aggregate queries with negation. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 215–226, 2001.
- [202] S. Cohen, W. Nutt, and Y. Sagiv. Containment of aggregate queries (extended version). Technical report, Hebrew University of Jerusalem, 2002. available at www.cs.huji.ac.il/~sarina/papers/agg-containment-long.ps.
- [203] S. Cohen, W. Nutt, and Y. Sagiv. Containment of aggregate queries. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 111–125, 2003.
- [204] L. Colby. A recursive algebra and query optimization for nested relational algebra. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 273–283, 1989.
- [205] L. Colby, A. Kawaguchi, D. Lieuwen, I. Mumick, and K. Ross. Supporting multiple view maintenance policies. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 405–416, 1997.

- [206] G. Copeland and S. Khoshafian. A decomposition storage model. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 268–279, Austin, TX, 1985.
- [207] G. Cormack. Data compression on a database system. *Communications of the ACM*, 28(12):1336–1342, 1985.
- [208] T. Cormen, C. Leiserson, and R. Rivest. *Introduction to Algorithms*. MIT Press, 1990.
- [209] T. Cormen, C. Leiserson, R. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 2001. 2nd Edition.
- [210] G. Cormode, M. Garofalakis, P. Haas, and C. Jermaine. *Synopses for Massive Data: Samples, Histograms, Wavelets, Sketches*. NOW Press, 2012.
- [211] D. Cornell and P. Yu. Integration of buffer management and query optimization in relational database environments. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 247–255, 1989.
- [212] J. Crawford and L. Auton. Experimental results on the crossover point in satisfiability problems. In *Proc. National Conference on Artificial Intelligence*, pages 21–27, 1993.
- [213] K. Culik, T. Ottmann, and D. Wood. Dense multiway trees. *ACM Trans. on Database Systems*, 6(3):486–512, 1981.
- [214] C. Cunningham, G. Graefe, and C. Galindo-Legaria. Pivot and unpivot: Optimization and execution strategies in an rdbms. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 998–1009, 2004.
- [215] M. Dadashzadeh. An improved division operator for relational algebra. *Information Systems*, 14(5):431–437, 1989.
- [216] D. Daniels. Query compilation in a distributed database system. Technical Report RJ 3432, IBM Research Laboratory, San Jose, CA, 1982.
- [217] D. Das and D. Batory. Praire: A rule specification framework for query optimizers. In *Proc. IEEE Conference on Data Engineering*, pages 201–210, 1995.
- [218] C. J. Date. The outer join. In *Proc. of the Int. Conf. on Databases*, Cambridge, England, 1983.
- [219] U. Dayal. Processing queries with quantifiers: A horticultural approach. In *ACM Symp. on Principles of Database Systems*, pages 125–136, 1983.
- [220] U. Dayal. Of nests and trees: A unified approach to processing queries that contain nested subqueries, aggregates, and quantifiers. In *VLDB*, pages 197–208, 1987.

- [221] U. Dayal, N. Goodman, and R.H. Katz. An extended relational algebra with control over duplicate elimination. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 117–123, 1982.
- [222] U. Dayal, F. Manola, A. Buchman, U. Chakravarthy, D. Goldhirsch, S. Heiler, J. Orenstein, and A. Rosenthal. Simplifying complex object: The PROBE approach to modelling and querying them. In *H.J. Schek and G. Schlageter (eds.) Proc. BTW*, pages 17–37, 1987.
- [223] U. Dayal and J. Smith. PROBE: A knowledge-oriented database management system. In *Proc. Islamorada Workshop on Large Scale Knowledge Base and Reasoning Systems*, 1985.
- [224] G. de Balbine. Note on random permutations. *Mathematics of Computation*, 21:710–712, 1967.
- [225] D. DeHaan, P.-A. Larson, and J. Zhou. Stacked indexed views in Microsoft SQL server. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 179–190, 2005.
- [226] K. Delaney. *Inside Microsoft SQL Server 2005: Query Tuning and Optimization*. Microsoft Press, 2008.
- [227] R. Demolombe. Estimation of the number of tuples satisfying a query expressed in predicate calculus language. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 55–63, 1980.
- [228] P. Denning. Effects of scheduling on file memory operations. In *Proc. AFIPS*, pages 9–21, 1967.
- [229] N. Derrett and M.-C. Shan. Rule-based query optimization in IRIS. Technical report, Hewlett-Packard Laboratories, 1501 Page Mill Road, Palo Alto, CA94303, 1990.
- [230] B. Desai. Performance of a composite attribute and join index. *IEEE Trans. on Software Eng.*, 15(2):142–152, Feb. 1989.
- [231] A. Deshpande, M. Garofalakis, and R. Rastogi. Independence is good: Dependency-based histogram synopses for high-dimensional data. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 199–210, 2001.
- [232] A. Deshpande, Z. Ives, and V. Raman. *Adaptive Query Optimization*. NOW, 2007.
- [233] A. Deutsch, M. Fernandez, D. Florescu, A. Levy, D. Maier, and D. Suciu. Querying XML data. *IEEE Data Engineering Bulletin*, 22(3):10–18, 1999.
- [234] A. Deutsch, B. Ludäscher, and A. Nash. Rewriting queries using views with access patterns under integrity constraints. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 352–367, 2005.

- [235] A. Deutsch and V. Tannen. Containment and integrity constraints for XPath. In *Int. Workshop on Knowledge Representation meets Databases (KRDB)*, 2001.
- [236] A. Deutsch and V. Tannen. Optimization properties for classes of conjunctive regular path queries. In *Int. Workshop on Database Programming Languages*, pages 21–39, 2001.
- [237] A. Deutsch and V. Tannen. Reformulation of XML queries and constraints. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 225–241, 2003.
- [238] D. DeWitt, R. Katz, F. Olken, L. Shapiro, M. Stonebraker, and D. Wood. Implementation techniques for main memory database systems. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 151–1??, 1984.
- [239] D. DeWitt, J. Naughton, and D. Schneider. An evaluation of non-equijoin algorithms. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 443–452, 1991.
- [240] Y. Diao, M. Altinel, M. Franklin, H. Zhang, and P. Fischer. Path sharing and predicate evaluation for high-performance xml filtering. *ACM Trans. on Database Systems*, 28(4):367–516, 2003.
- [241] G. Diehr and A. Saharia. Estimating block accesses in database organizations. *IEEE Trans. on Knowledge and Data Engineering*, 6(3):497–499, 1994.
- [242] P. Dietz. Optimal algorithms for list indexing and subset ranking. In *Workshop on Algorithms and Data Structures (LNCS 382)*, pages 39–46, 1989.
- [243] Z. Dimitrijevic, R. Rangaswami, E. Chang, D. Watson, and A. Acharya. Diskbench: User-level disk feature extraction tool. Technical report, University of California, Santa Barbara, 2004.
- [244] J. Dongarra, K. London, S. Moore, P. Mucci, and D. Terpstra. Using PAPI for hardware performance monitoring on Linux systems. perform internet search for this or similar tools.
- [245] D. Donjerkovic, Y. Ioannidis, and R. Ramakrishnan. Dynamic histograms: Capturing evolving data sets. In *Proc. IEEE Conference on Data Engineering*, page 86, 2000.
- [246] D. Donjerkovic and R. Ramakrishnan. Probabilistic optimization of Top n queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 411–422, 1999.
- [247] J. Donovan. Database system approach to management of decision support. *ACM Trans. on Database Systems*, 1(4):344–368, 1976.

- [248] M. Drmota, D. Gardy, and B. Gittenberger. A unified presentation of some urn models. *Algorithmica*, 29:120–147, 2001.
- [249] M. Durand. *Combinatoire analytique et algorithmique des ensembles de données*. PhD thesis, Ecole Polytechnique, 2004.
- [250] M. Durand and P. Flajolet. Loglog counting of large cardinalities. In *Algorithms - ESA 2003, Annual European Symposium*, pages 605–617. Springer LNCS 2832, 2003.
- [251] R. Durstenfeld. Algorithm 235: Random permutation. *Communications of the ACM*, 7(7):420, 1964.
- [252] O. Duschka. *Query Planning and Optimization in Information Integration*. PhD thesis, Stanford University, 1997.
- [253] O. Duschka and M. Genesereth. Answering recursive queries using views. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 109–116, 1997.
- [254] O. Duschka and M. Genesereth. Query planning with disjunctive sources. In *AAAI Workshop on AI and Information Integration*, 1998.
- [255] O. Duschka and M. Gensereth. Answering recursive queries using views. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 109–116, 1997.
- [256] O. Duschka and A. Levy. Recursive plans for information gathering. In *Int. Joint Conf. on Artificial Intelligence (IJCAI)*, 1997.
- [257] W. Effelsberg and T. Härder. Principles of database buffer management. *ACM Trans. on Database Systems*, 9(4):560–595, 1984.
- [258] S. Eggers, F. Olken, and A. Shoshani. A compression technique for large statistical data bases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 424–434, 1981.
- [259] S. Eggers and A. Shoshani. Efficient access of compressed data. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 205–211, 1980.
- [260] J. F. Egler. A procedure for converting logic table conditions into an efficient sequence of test instructions. *Communications of the ACM*, 6(9):510–514, 1963.
- [261] M. Eisner and D. Severance. Mathematical techniques for efficient record segmentation in large shared databases. *Journal of the ACM*, 23(4):619–635, 1976.
- [262] A. El-Helw, I. Ilyas, W. Lau, V. Markl, and C. Zuzarte. Collecting and maintaining just-in-time statistics. In *Proc. IEEE Conference on Data Engineering*, pages 516–525, 2007.

- [263] Ml Elhemali, C. Galindo-Legaria, T. Grabs, and M. Joshi. Execution strategies for SQL subqueries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 993–1003, 2007.
- [264] R. Elmasri and S. Navathe. *Fundamentals of Database Systems*. Addison-Wesley, 2000. 3rd Edition.
- [265] G. Lohman et al. Optimization of nested queries in a distributed relational database. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, 1984.
- [266] N. Roussopoulos et al. The maryland ADMS project: Views R Us. *IEEE Data Engineering Bulletin*, 18(2), 1995.
- [267] P. Schwarz et al. Extensibility in the starburst database system. In *Proc. Int. Workshop on Object-Oriented Database Systems*, 1986.
- [268] R. Fagin. Combining fuzzy information from multiple systems. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 216–226, 1996.
- [269] R. Fagin, A. Lotem, and M. Naor. Optimal aggregation algorithms for middleware. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 102–113, 2001.
- [270] C. Farr’e, E. Teniente, and T. Urp’i. Query containment checking as a view updating problem. In *Int. Conf. on Database and Expert Systems Applications (DEXA)*, pages 310–321, 1998.
- [271] C. Farr’e, E. Teniente, and T. Urp’i. The constructive method for query containment checking. In *Int. Conf. on Database and Expert Systems Applications (DEXA)*, pages 583–593, 1999.
- [272] C. Farr’e, E. Teniente, and T. Urp’i. Query containment with negated IDB predicates. In *ABDIS*, pages 583–593, 2003.
- [273] C. Farr’e, E. Teniente, and T. Urp’i. Checking query containment with the cq method. *Data & Knowledge Engineering*, 53:163–223, 2005.
- [274] J. Fedorowicz. Database evaluation using multiple regression techniques. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 70–76, 1984.
- [275] J. Fedorowicz. Database performance evaluation in an indexed file environment. *ACM Trans. on Database Systems*, 12(1):85–110, 1987.
- [276] L. Fegaras. Optimizing large OODB queries. In *Proc. Int. Conf. on Deductive and Object-Oriented Databases (DOOD)*, pages 421–422, 1997.
- [277] L. Fegaras. A new heuristic for optimizing large queries. In *DEXA*, pages 726–735, 1998.

- [278] L. Fegaras and D. Maier. Towards an effective calculus for object query languages. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 47–58, 1995.
- [279] L. Fegaras and D. Maier. Optimizing object queries using an effective calculus. *ACM Trans. on Database Systems*, 25(4):457–516, 2000.
- [280] T. Fiebig and G. Moerkotte. Evaluating Queries on Structure with eXtended Access Support Relations. In *WebDB 2000*, 2000.
- [281] T. Fiebig and G. Moerkotte. Algebraic XML construction in Natix. In *Proc. Int. Conf. on Web Information Systems Engineering (WISE)*, pages 212–221, 2001.
- [282] T. Fiebig and G. Moerkotte. Algebraic XML construction and its optimization in Natix. *World Wide Web Journal*, 4(3):167–187, 2002.
- [283] S. Finkelstein. Common expression analysis in database applications. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 235–245, 1982.
- [284] P. Flajolet. Approximate counting: A detailed analysis. *BIT*, 25(1):113–134, 1985.
- [285] P. Flajolet, E. Fusy, O. Gandouet, and F. Meunier. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *Conf. on Analysis of Algorithms (AofA). Discrete Mathematics and Theoretical Computer Science*, pages 127–146, 2007.
- [286] P. Flajolet and G. Martin. Probabilistic counting. In *Annual Symposium on Foundations of Computer Science (FOCS)*, pages 76j–82, 1983.
- [287] P. Flajolet and G. Martin. Probabilistic counting algorithms for data base applications. *Rapports de Recherche 313*, INRIA Rocquencourt, 1984.
- [288] P. Flajolet and G. Martin. Probabilistic counting algorithms for data base applications. *J. Comput. Syst. Sci.*, 31(2):182–209, 1985.
- [289] D. Florescu. *Espaces de Recherche pour l’Optimisation de Requêtes Objet (Search Spaces for Query Optimization)*. PhD thesis, Université de Paris VI, 1996. in French.
- [290] D. Florescu, A. Levy, and D. Suciu. Query containment for conjunctive queries with regular expressions. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 139–148, 1998.
- [291] P. Fortier. *SQL-3, Implementing the SQL Foundation Standard*. McGraw Hill, 1999.
- [292] F. Fotouhi and S. Pramanik. Optimal secondary storage access sequence for performing relational join. *IEEE Trans. on Knowledge and Data Eng.*, 1(3):318–328, 1989.

- [293] F. Frasincar, G.-J. Houben, and C. Pau. XAL: An algebra for XML query optimization. In *Australasian Database Conference (ADC)*, 2002.
- [294] J. Freire, J. Haritsa, M. Ramanath, P. Roy, and J. Simeon. StatiX: making XML count. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 181–191, 2002.
- [295] J. C. Freytag. A rule-based view of query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 173–180, 1987.
- [296] J. C. Freytag and N. Goodman. On the translation of relational queries into iterative programs. *ACM Trans. on Database Systems*, 14(1):1–27, 1989.
- [297] C. Galindo-Legaria. *Outerjoin Simplification and Reordering for Query Optimization*. PhD thesis, Harvard University, 1992.
- [298] C. Galindo-Legaria. Outerjoins as disjunctions. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 348–358, 1994.
- [299] C. Galindo-Legaria. Outerjoins as disjunctions. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 348–538, 1994.
- [300] C. Galindo-Legaria and M. Joshi. Orthogonal optimization of subqueries and aggregation. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 571–581, 2001.
- [301] C. Galindo-Legaria, M. Joshi, F. Waas, and M.-C. Wu. Statistics on views. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 952–962, 2003.
- [302] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. Cost distribution of search spaces in query optimization. Technical Report CS-R9432, CWI, Amsterdam, NL, 1994.
- [303] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. Fact, randomized join-order selection — why use transformations? In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 85–95, 1994.
- [304] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. Fast, randomized join-order selection — why use transformations? Technical Report CS-R-9416, CWI, Amsterdam, NL, 1994.
- [305] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. The impact of catalogs and join algorithms on probabilistic query optimization. Technical Report CS-R9459, CWI, Amsterdam, NL, 1994.
- [306] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. Uniformly-distributed random generation of join orders. Technical Report CS-R9431, CWI, Amsterdam, NL, 1994.

- [307] C. Galindo-Legaria, A. Pellenkoff, and M. Kersten. Cost distribution of search spaces in query optimization. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 280–293, 1995.
- [308] C. Galindo-Legaria and A. Rosenthal. Outerjoin simplification and re-ordering for query optimization. *ACM Trans. on Database Systems*, 22(1):43–73, Marc 1997.
- [309] C. Galindo-Legaria, A. Rosenthal, and E. Kortright. Expressions, graphs, and algebraic identities for joins, 1991. working paper.
- [310] S. Ganapathy and V. Rajaraman. Information theory applied to the conversion of decision tables to computer programs. *Communications of the ACM*, 16:532–539, 1973.
- [311] S. Gandeharizadeh, J. Stone, and R. Zimmermann. Techniques to quantify SCSI-2 disk subsystem specifications for multimedia. Technical Report 95-610, University of Southern California, 1995.
- [312] S. Ganguly. On the complexity of finding optimal join order sequence for star queries without cross products. personal correspondance, 2000.
- [313] S. Ganguly, A. Goel, and A. Silberschatz. Efficient and accurate cost model for parallel query optimization. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 172–181, 1996.
- [314] R. Ganski and H. Wong. Optimization of nested SQL queries revisited. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 23–33, 1987.
- [315] G. Garani and R. Johnson. Joining nested relations and subrelations. *Information Systems*, 25(4):287–307, 2000.
- [316] H. Garcia-Molina, J. Ullman, and J. Widom. *Database System Implementation*. Prentice Hall, 2000.
- [317] D. Gardy and L. Nemirovski. Urn models and yao’s formula. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 100–112, 1999.
- [318] D. Gardy and C. Puech. On the effect of join operations on relation sizes. *ACM Trans. on Database Systems*, 14(4):574–603, 1989.
- [319] M. Garey and D. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1978.
- [320] M. R. Garey and D. S. Johnson. *Computers and Intractability: a Guide to the Theory of NP-Completeness*. Freeman, San Francisco, 1979.
- [321] M. Garofalakis and P. Gibbons. Wavelet synopses with error guarantees. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 476–487, 2002.

- [322] P. Gassner, G. Lohman, and K. Schiefer. Query optimization in the IBM DB2 family. *IEEE Data Engineering Bulletin*, 16:4–18, Dec. 1993.
- [323] P. Gassner, G. Lohman, K. Schiefer, and Y. Wang. Query optimization in the IBM DB2 family. Technical report rj 9734, IBM, 1994.
- [324] E. Gelenbe and D. Gardy. On the size of projections: I. *Information Processing Letters*, 14:1, 1982.
- [325] E. Gelenbe and D. Gardy. The size of projections of relations satisfying a functional dependency. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 325–333, 1982.
- [326] I. Gent and T. Walsh. Towards an understanding of hill-climbing procedures for SAT. In *Proc. National Conference on Artificial Intelligence*, pages 28–33, 1993.
- [327] A. Gheazal, R. Bhashyam, and A. Crolotte. Block optimization in the Teradata RDBMS. In *Int. Conf. on Database and Expert Systems Applications (DEXA)*, pages 782–791, 2003.
- [328] A. Gheazal, A. Crolotte, and R. Bhashyam. Outer join elimination in the Teradata RDBMS. In *Int. Conf. on Database and Expert Systems Applications (DEXA)*, pages 730–740, 2004.
- [329] L. Giakoumakis and C. Galinda-Legaria. Testing sql server’s query optimizer: Challenges, techniques and experiences. *IEEE Data Engineering Bulletin*, 31(1):36–43, 2008.
- [330] P. Gibbons and Y. Matias. New sampling-based summary statistics for improving approximate query answers. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 331–342, 1998.
- [331] F. Giroire. Order statistics and estimating cardinalities of massive data sets. *Discrete Applied Mathematics*, 157:406–427, 2009.
- [332] P. Godfrey, J. Gryz, and C. Zuzarte. Exploiting constraint-like data characterizations in query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 582–592, 2001.
- [333] D. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [334] J. Goldstein and P. Larson. Optimizing queries using materialized views: A practical, scalable solution. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 331–342, 2001.
- [335] J. Goldstein, R. Ramakrishnan, and U. Shaft. Compressing relations and indexes. In *Proc. IEEE Conference on Data Engineering*, 1998. to appear.
- [336] G. Golub and C. van Loan. *Matrix Computations*. The John Hopkins University Press, 1996. Third Edition.

- [337] G. Gorry and S. Morton. A framework for management information systems. *Sloan Management Review*, 13(1):55–70, 1971.
- [338] G. Gottlob, C. Koch, and R. Pichler. Efficient algorithms for processing XPath queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 95–106, 2002.
- [339] G. Gottlob, C. Koch, and R. Pichler. XPath processing in a nutshell. *SIGMOD Record*, 2003.
- [340] G. Gottlob, C. Koch, and R. Pichler. XPath query evaluation: Improving time and space efficiency. In *Proc. IEEE Conference on Data Engineering*, page to appear, 2003.
- [341] M. Gouda and U. Dayal. Optimal semijoin schedules for query processing in local distributed database systems. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 164–175, 1981.
- [342] P. Goyal. Coding methods for text string search on compressed databases. *Information Systems*, 8(3):231–233, 1983.
- [343] G. Graefe. Software modularization with the exodus optimizer generator. *IEEE Database Engineering*, 9(4):37–45, 1986.
- [344] G. Graefe. Relational division: Four algorithms and their performance. In *Proc. IEEE Conference on Data Engineering*, pages 94–101, 1989.
- [345] G. Graefe. Encapsulation of parallelism in the Volcano query processing system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages ?–?, 1990.
- [346] G. Graefe. Heap-filter merge join: A new algorithm for joining medium-size inputs. *IEEE Trans. on Software Eng.*, 17(9):979–982, 1991.
- [347] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2), June 1993.
- [348] G. Graefe. Query evaluation techniques for large databases. Shortened version: [347], July 1993.
- [349] G. Graefe. Sort-merge-join: An idea whose time has(h) passed? In *Proc. IEEE Conference on Data Engineering*, pages 406–417, 1994.
- [350] G. Graefe. The cascades framework for query optimization. *IEEE Data Engineering Bulletin*, 18(3):19–29, Sept 1995.
- [351] G. Graefe. Executing nested queries. In *BTW*, pages 58–77, 2003.
- [352] G. Graefe, R. Bunker, and S. Cooper. Hash joins and hash teams in Microsoft SQL Server. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 86–97, 1998.

- [353] G. Graefe and R. Cole. Fast algorithms for universal quantification in large databases. Internal report, Portland State University and University of Colorado at Boulder, 19??
- [354] G. Graefe and R. Cole. Dynamic query evaluation plans. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages ?–?, 1994.
- [355] G. Graefe and R. Cole. Fast algorithms for universal quantification in large databases. *ACM Trans. on Database Systems*, ?(?):?–?, ? 1995?
- [356] G. Graefe, R. Cole, D. Davison, W. McKenna, and R. Wolniewicz. Extensible query optimization and parallel execution in Volcano. In *Dagstuhl Query Processing Workshop*, pages 337–380, 1991.
- [357] G. Graefe and D. DeWitt. The EXODUS optimizer generator. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 160–172, 1987.
- [358] G. Graefe, A. Linville, and L. Shapiro. Sort versus hash revisited. *IEEE Trans. on Knowledge and Data Eng.*, 6(6):934–944, Dec. 1994.
- [359] G. Graefe and W. McKenna. The Volcano optimizer generator. Tech. Report 563, University of Colorado, Boulder, 1991.
- [360] G. Graefe and W. McKenna. Extensibility and search efficiency in the volcano optimizer generator. In *Proc. IEEE Conference on Data Engineering*, pages 209–218, 1993.
- [361] G. Graefe and K. Ward. Dynamic query evaluation plans. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 358–366, 1989.
- [362] R. Graham, D. Knuth, and O. Patashnik. *Concrete Mathematics*. Addison-Wesley, 2002.
- [363] G. Grahne and A. Thomo. Algebraic rewritings for optimizing regular path queries. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 301–315, 2001.
- [364] G. Grahne and A. Thomo. New rewritings and optimizations for regular path queries. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 242–258, 2003.
- [365] J. Grant, J. Gryz, J. Minker, and L. Raschid. Semantic query optimization for object databases. In *Proc. IEEE Conference on Data Engineering*, pages 444–453, 1997.
- [366] J. Gray, editor. *The Benchmark Handbook*. Morgan Kaufmann Publishers, San Mateo, CA, 1991.
- [367] J. Gray and G. Graefe. The five-minute rule ten years later, and other computer storage rules of thumb. *ACM SIGMOD Record*, 26(4):63–68, 1997.

- [368] J. Gray and F. Putzolu. The 5 minute rule for trading memory for disk accesses and the 10 byte rule for trading memory for CPU time. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 395–398, 1987.
- [369] P. Grefen and R. de By. A multi-set extended relational algebra – a formal approach to a practical issue. In *Proc. IEEE Conference on Data Engineering*, pages 80–88, 1994.
- [370] T. Grust. Accelerating XPath location steps. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 109–120, 2002.
- [371] T. Grust and M. Van Keulen. Tree awareness for relational database kernels: Staircase join. In *Intelligent Search on XML Data*, pages 231–245, 2003.
- [372] T. Grust, M. Van Keulen, and J. Teubner. Staircase join: Teach a relational dbms to watch its (axis) steps. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 524–525, 2003.
- [373] T. Grust, M. Van Keulen, and J. Teubner. Accelerating XPath evaluation in any RDBMS. *ACM Trans. on Database Systems*, 29(1):91–131, 2004.
- [374] J. Gryz, B. Schiefer, J. Zheng, and C. Zuzarte. Discovery and application of check constraints in DB2. In *Proc. IEEE Conference on Data Engineering*, pages 551–556, 2001.
- [375] E. Gudes and A. Reiter. On evaluating boolean expression. *Software Practice and Experience*, 3:345–350, 1973.
- [376] S. Guha, N. Koudas, and K. Shim. Data-streams and histograms. In *Annual ACM Symposium on Theory of Computing (STOC)*, pages 471–475, 2001.
- [377] S. Guha, N. Koudas, and D. Srivastava. Fast algorithms for hierarchical range histogram construction. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 180–187, 2002.
- [378] H. Gunadhi and A. Segev. Query processing algorithms for temporal intersection joins. In *Proc. IEEE Conference on Data Engineering*, pages 336–344, 1991.
- [379] L. Guo, K. Beyer, J. Shanmugasundaram, and E. Shekita. Efficient inverted lists and query algorithms for structured value ranking in update-intense relational databases. In *Proc. IEEE Conference on Data Engineering*, pages 298–309, 2005.
- [380] M. Guo, S. Y. W. Su, and H. Lam. An association algebra for processing object-oriented databases. In *Proc. IEEE Conference on Data Engineering*, pages ?–?, 1991.

- [381] A. Gupta, V. Harinarayan, and D. Quass. Aggregate-query processing in data warehousing environments. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 358–369, 1995.
- [382] A. Gupta, V. Harinarayan, and D. Quass. Generalized projections: A powerful approach to aggregation. Technical Report, 1995.
- [383] A. Gupta and I. Mumick. Maintenance of materialized views: problems, techniques and applications. *IEEE Data Engineering Bulletin*, 18(2):3–19, 1995.
- [384] R. Güting, R. Zicari, and D. Choy. An algebra for structured office documents. *ACM Trans. on Information Systems*, 7(4):123–157, 1989.
- [385] R. H. Güting. Geo-relational algebra: A model and query language for geometric database systems. In J. W. Schmidt, S. Ceri, and M. Missikoff, editors, *Proc. of the Intl. Conf. on Extending Database Technology*, pages 506–527, Venice, Italy, Mar 1988. Springer-Verlag, Lecture Notes in Computer Science No. 303.
- [386] R. H. Güting. Second-order signature: A tool for specifying data models, query processing, and optimization. Informatik-Bericht 12/1992, ETH Zürich, 1992.
- [387] L. Haas, W. Chang, G. Lohman, J. McPherson, P. Wilms, G. Lapis, B. Lindsay, H. Pirahesh, M. Carey, and E. Shekita. Starburst mid-flight: As the dust clears. *IEEE Trans. on Knowledge and Data Eng.*, 2(1):143–160, 1990.
- [388] L. Haas, J. Freytag, G. Lohman, and H. Pirahesh. Extensible query processing in starburst. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 377–388, 1989.
- [389] A. Hadi. *Matrix Algebra as a Tool*. Duxbury Press, 1996.
- [390] A. Halevy. Answering queries using views: A survey. *The VLDB Journal*, 10(4):270–294, Dec. 2001.
- [391] P. A. V. Hall. Common subexpression identification in general algebraic systems. Tech. rep. uksc 0060, IBM UK Scientific Center, Peterlee, England, 1974.
- [392] P. A. V. Hall. Optimization of single expressions in a relational database system. *IBM J. Res. Devel.*, 20(3):244–257, 1976.
- [393] P. A. V. Hall and S. Todd. Factorization of algebraic expressions. Tech. Report UKSC 0055, IBM UK Scientific Center, Peterlee, England, 1974.
- [394] C. Hamalainen. Complexity of query optimisation and evaluation. Master’s thesis, Griffith University, Queensland, Australia, 2002.

- [395] M. Hammer and B. Niamir. A heuristic approach to attribute partitioning. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 93–101, 1979.
- [396] J. Han. Smallest-first query evaluation for database systems. In *Australian Database Conference*, pages ?–?, Christchurch, New Zealand, Jan. 1994.
- [397] M. Z. Hanani. An optimal evaluation of boolean expressions in an online query system. *Communications of the ACM*, 20(5):344–347, 1977.
- [398] E. Hanson. A performance analysis of view materialization strategies. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 440–453, 1987.
- [399] E. Hanson. Processing queries against database procedures. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages ?–?, 1988.
- [400] E.N. Hanson, M. Chaabouni, C.-H. Kim, and Y.-W. Wang. A predicate matching algorithm for database rule systems. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 271–?, 1990.
- [401] T. Härder. Implementing a generalized access path structure for a relational database system. *ACM Trans. on Database Systems*, 3(3):285–298, 1978.
- [402] T. Härder, B. Mitschang, and H. Schöning. Query processing for complex objects. *Data and Knowledge Engineering*, 7(3):181–200, 1992.
- [403] T. Härder and E. Rahm. *Datenbanksysteme*. Springer, 1999.
- [404] T. Härder, H. Schöning, and A. Sikeler. Parallelism in processing queries on complex objects. In *International Symposium on Databases in Parallel and Distributed Systems*, Ausgin, TX, August 1988.
- [405] V. Harinarayan and A. Gupta. Generalized projections: a powerful query optimization technique. Technical Report STAN-CS-TN-94-14, Stanford University, 1994.
- [406] H. Harmouch and F. Naumann. Cardinality estimation: An experimental survey. *Proc. of the VLDB Endowment (PVLDB)*, 11(4):499–512, 2017.
- [407] E. Harris and K. Ramamohanarao. Join algorithm costs revisited. *The VLDB Journal*, 5(1):?–?, Jan 1996.
- [408] D. Harville. *Matrix Algebra from a Statistician's Perspective*. Springer, 2008.
- [409] W. Hasan and H. Pirahesh. Query rewrite optimization in starburst. Research Report RJ6367, IBM, 1988.
- [410] Z. He, B. Lee, and R. Snapp. Self-tuning cost modeling of user-defined functions in an object-relational DBMS. *ACM Trans. on Database Systems*, 30(3):812–853, 2005.

- [411] M. Heimel, M. Kiefer, and V. Markl. Self-tuning, GPU-accelerated kernel density models for multidimensional selectivity estimation. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 1477–1492, 2015.
- [412] Heller. Rabbit: A performance counter library for Intel/AMD processors and Linux. perform internet search for this or similar tools.
- [413] J. Hellerstein. Practical predicate placement. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 325–335, 1994.
- [414] J. Hellerstein and J. Naughton. Query execution techniques for caching expensive methods. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 423–434, 1996.
- [415] J. Hellerstein and M. Stonebraker. Predicate migration: Optimizing queries with expensive predicates. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 267–277, 1993.
- [416] S. Helmer, C.-C. Kanne, and G. Moerkotte. Optimized translation of XPath expressions into algebraic expressions parameterized by programs containing navigational primitives. In *Proc. Int. Conf. on Web Information Systems Engineering (WISE)*, 2002. 215-224.
- [417] S. Helmer, C.-C. Kanne, and G. Moerkotte. Optimized translation of XPath expressions into algebraic expressions parameterized by programs containing navigational primitives. Technical Report 11, University of Mannheim, 2002.
- [418] S. Helmer, B. König-Ries, and G. Moerkotte. The relational difference calculus and applications. Technical report, Universität Karlsruhe, 1993. (unpublished manuscript).
- [419] S. Helmer and G. Moerkotte. Evaluation of main memory join algorithms for joins with set comparison join predicates. Technical Report 13/96, University of Mannheim, Mannheim, Germany, 1996.
- [420] S. Helmer and G. Moerkotte. Evaluation of main memory join algorithms for joins with set comparison join predicates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 386–395, 1997.
- [421] S. Helmer and G. Moerkotte. Index structures for databases containing data items with set-valued attributes. Technical Report 2/97, University of Mannheim, 1997.
- [422] S. Helmer and G. Moerkotte. A study of four index structures for set-valued attributes of low cardinality. Technical Report 02/99, University of Mannheim, 1999.
- [423] S. Helmer and G. Moerkotte. Compiling away set containment and intersection joins. Technical Report 4, University of Mannheim, 2002.

- [424] S. Helmer and G. Moerkotte. A performance study of four index structures for set-valued attributes of low cardinality. *VLDB Journal*, 12(3):244–261, 2003.
- [425] S. Helmer, T. Neumann, and G. Moerkotte. Early grouping gets the skew. Technical Report 9, University of Mannheim, 2002.
- [426] M. Henderson and R. Lawrence. An evaluation of multi-way joins for relational database systems. In *ICEIS*, pages 37–50, 2013.
- [427] A. Heuer and M. H. Scholl. Principles of object-oriented query languages. In *Proc. der GI-Fachtagung Datenbanksysteme für Büro, Technik und Wissenschaft (BTW)*. Springer, 1991.
- [428] J. Hidders and P. Michiels. Avoiding unnecessary ordering operations in XPath. In *Int. Workshop on Database Programming Languages*, pages 54–70, 2003.
- [429] D. Hirschberg. On the complexity of searching a set of vectors. *SIAM J. Computing*, 9(1):126–129, 1980.
- [430] T. Hogg and C. Williams. Solving the really hard problems with cooperative search. In *Proc. National Conference on Artificial Intelligence*, pages 231–236, 1993.
- [431] L. Hong-Cheu and K. Ramamohanarao. Algebraic equivalences among nested relational expressions. In *CIKM*, pages 234–243, 1994.
- [432] R. Horn and C. Johnson. *Matrix Analysis*. Cambridge University Press, 2007.
- [433] V. Hristidis, N. Koudas, and Y. Papakonstantinou. PREFER: A system for the efficient execution of multi-parametric ranked queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages ?–?, 2001.
- [434] N. Huyn. Multiple-view self-maintenance in data warehousing environments. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 26–35, 1997.
- [435] F. Hwang and G. Chang. Enumerating consecutive and nested partitions for graphs. Technical Report DIMACS Technical Report 93-15, Rutgers University, 1993.
- [436] H.-Y. Hwang and Y.-T. Yu. An analytical method for estimating and interpreting query time. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 347–358, 1987.
- [437] L. Hyafil and R. Rivest. Constructing optimal binary decision trees is NP-complete. *Information Processing Letters*, 5(1):15–17, 1976.
- [438] T. Ibaraki and T. Kameda. Optimal nesting for computing n-relational joins. *ACM Trans. on Database Systems*, 9(3):482–502, 1984.

- [439] O. Ibarra and J. Su. On the containment and equivalence of database queries with linear constraints. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 32–43, 1997.
- [440] A. IJbema and H. Blanken. Estimating bucket accesses: A practical approach. In *Proc. IEEE Conference on Data Engineering*, pages 30–37, 1986.
- [441] I. Ilyas, V. Markl, P. Haas, P. Brown, and A. Aboulnaga. Automatic relationship discovery in self-managing database systems. In *Proc. Int. Conf. on Automatic Computing (ICAC)*, pages 340–341, 2004.
- [442] I. Ilyas, V. Markl, P. Haas, P. Brown, and A. Aboulnaga. Cords: Automatic discovery of correlations and soft functional dependencies. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 647–658, 2004.
- [443] I. Ilyas, V. Markl, P. Haas, P. Brown, and A. Aboulnaga. Cords: Automatic generation of correlation statistics in db2. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, 2004.
- [444] I. Ilyas, J. Rao, G. Lohman, D. Gao, and E. Lin. Estimating compilation time of a query optimizer. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 373–384, 2003.
- [445] Y. Ioannidis. Query optimization. *ACM Computing Surveys*, 28(1):121–123, 1996.
- [446] Y. Ioannidis. *A. Tucker (ed.): The Computer Science and Engineering Handbook*, chapter Query Optimization, pages 1038–1057. CRC Press, 1997.
- [447] Y. Ioannidis. The history of histograms (abridged). In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 19–30, 2003.
- [448] Y. Ioannidis and V. Poosala. Histogram-based approximation of set-valued query answers. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 174–185, 1999.
- [449] Y. Ioannidis and R. Ramakrishnan. Generalized containment of conjunctive queries. Technical report, U. Wisconsin, Madison, 1992.
- [450] Y. Ioannidis and R. Ramakrishnan. Containment of conjunctive queries: Beyond relations and sets. *ACM Trans. on Database Systems*, 20(3):288–324, 1995.
- [451] Y. E. Ioannidis and S. Christodoulakis. On the propagation of errors in the size of join results. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 268–277, 1991.

- [452] Y. E. Ioannidis and Y. C. Kang. Randomized algorithms for optimizing large join queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 312–321, 1990.
- [453] Y. E. Ioannidis and Y. C. Kang. Left-deep vs. bushy trees: An analysis of strategy spaces and its implications for query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 168–177, 1991.
- [454] Y. E. Ioannidis, Y. C. Kang, and T. Zhang. Cost wells in random graphs. personal communication, Dec. 1996.
- [455] Y. E. Ioannidis, R. T. Ng, K. Shim, and T. K. Sellis. Parametric query optimization. Tech. report, University of Wisconsin, Madison, 1992.
- [456] Y. E. Ioannidis, R. T. Ng, K. Shim, and T. K. Sellis. Parametric query optimization. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 103–114, 1992.
- [457] Y. E. Ioannidis and E. Wong. Query optimization by simulated annealing. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 9–22, 1987.
- [458] D. Jacobsen and J. Wilkes. Disk scheduling algorithms based on rotational position. Technical Report HPL-CSP-91-7, Hewlett-Packard Laboratories, 1991.
- [459] H. V. Jagadish, S. Al-Khalifa, A. Chapman, L.V.S. Lakshmanan, A. Nierman, S. Paparizos, J. Patel, D. Srivastava, N. Wiwatwattana, Y. Wu, and C. Yu. TIMBER: A Native XML Database. *VLDB Journal*, 2003. to appear.
- [460] H. V. Jagadish, H. Jin, B. C. Ooi, and K.-L. Tan. Global optimization of histograms. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 223–234, 2001.
- [461] H. V. Jagadish, N. Koudas, S. Muthukrishnan, V. Poosala, K. C. Sevcik, and T. Suel. Optimal histograms with quality guarantees. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 275–286, 1998.
- [462] H. V. Jagadish, L. V. S. Lakshmanan, D. Srivastava, and K. Thompson. TAX: A tree algebra for XML. In *Proc. Int. Workshop on Database Programming Languages*, pages 149–164, 2001.
- [463] C. Janssen. The visual profiler. perform internet search for this or similar tools.
- [464] M. Jarke. Common subexpression isolation in multiple query optimization. In W. Kim, D. Reiner, and D. Batory, editors, *Topics in Information Systems. Query Processing in Database Systems*, pages 191–205, 1985.

- [465] M. Jarke. Common subexpression isolation in multiple query optimization. In *Query Processing in Database Systems*, W. Kim, D. Reiner, D. Batory (Eds.), pages 191–205, 1985.
- [466] M. Jarke, J. Clifford, and Y. Vassiliou. An optimizing PROLOG front-end to a relational query system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 296–306, 1984.
- [467] M. Jarke and J. Koch. Range nesting: A fast method to evaluate quantified queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 196–206, 1983.
- [468] M. Jarke and J. Koch. Query optimization in database systems. *ACM Computing Surveys*, pages 111–152, Jun 1984.
- [469] A. Jhingran. A performance study of query optimization algorithms on a database system supporting procedures. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 88–99, 1988.
- [470] D. Johnson and A. Klug. Testing containment of conjunctive queries under functional and inclusion dependencies. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 164–168, 1982.
- [471] D. Johnson and A. Klug. Testing containment of conjunctive queries under functional and inclusion dependencies. *J. Comp. Sys. Sci.*, 28(1):167–189, 1984.
- [472] D. S. Johnson and A. Klug. Optimizing conjunctive queries that contain untyped variables. *SIAM J. Comput.*, 12(4):616–640, 1983.
- [473] B. Jonsson, M. Franklin, and D. Srivastava. Interaction of query evaluation and buffer management for information retrieval. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 118–129, 1998.
- [474] N. Kabra and D. DeWitt. Efficient mid-query re-optimization of sub-optimal query execution plans. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 106–117, 1998.
- [475] J. Kahn, G. Kalai, and N. Linial. The influence of variables on boolean functions. In *IEEE ???*, pages 68–80, 1988.
- [476] M. Kamath and K. Ramamritham. Bucket skip merge join: A scalable algorithm for join processing in very large databases using indexes. Technical Report 20, University of Massachusetts at Amherst, Amherst, MA, 1996.
- [477] Y. Kambayashi. Processing cyclic queries. In W. Kim, D. Reiner, and D. Batory, editors, *Query Processing in Database Systems*, pages 62–78, 1985.

- [478] Y. Kambayashi and M. Yoshikawa. Query processing utilizing dependencies and horizontal decomposition. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 55–68, 1983.
- [479] N. Kamel and R. King. A model of data distribution based on texture analysis. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 319–325, 1985.
- [480] R. Kaushik, C. R'è, and D. Suciu. General database statistics using entropy maximization. In *Proc. Int. Workshop on Database Programming Languages*, pages 84–99, 2009.
- [481] A. Kawaguchi, D. Lieuwen, I. Mumick, and K. Ross. Implementing incremental view maintenance in nested data models. In *Proc. Int. Workshop on Database Programming Languages*, 1997.
- [482] A. Keller and J. Basu. A predicate-based caching scheme for client-server database architectures. In *PDIS*, pages 229–238, 1994.
- [483] T. Keller, G. Graefe, and D. Maier. Efficient assembly of complex objects. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 148–157, 1991.
- [484] A. Kemper and A. Eickler. *Datenbanksysteme*. Oldenbourg, 2001. 4th Edition.
- [485] A. Kemper and G. Moerkotte. Advanced query processing in object bases: A comprehensive approach to access support, query transformation and evaluation. Technical Report 27/90, University of Karlsruhe, 1990.
- [486] A. Kemper and G. Moerkotte. Advanced query processing in object bases using access support relations. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 294–305, 1990.
- [487] A. Kemper and G. Moerkotte. Query optimization in object bases: Exploiting relational techniques. In *Proc. Dagstuhl Workshop on Query Optimization (J.-C. Freytag, D. Maier und G. Vossen (eds.))*. Morgan-Kaufman, 1993.
- [488] A. Kemper, G. Moerkotte, and K. Peithner. A blackboard architecture for query optimization in object bases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 543–554, 1993.
- [489] A. Kemper, G. Moerkotte, K. Peithner, and M. Steinbrunn. Optimizing disjunctive queries with expensive predicates. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 336–347, 1994.
- [490] A. Kemper, G. Moerkotte, and M. Steinbrunn. Optimierung Boolescher Ausdrücke in Objektbanken. In *Grundlagen von Datenbanken (Eds. U. Lipeck, R. Manthey)*, pages 91–95, 1992.

- [491] A. Kemper, G. Moerkotte, and M. Steinbrunn. Optimization of boolean expressions in object bases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 79–90, 1992.
- [492] W. Kiessling. On semantic reefs and efficient processing of correlation queries with aggregates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 241–250, 1985.
- [493] K. C. Kim, W. Kim, D. Woelk, and A. Dale. Acyclic query processing in object-oriented databases. In *Proc. of the Entity Relationship Conf.*, 1988.
- [494] W. Kim. On optimizing an SQL-like nested query. *ACM Trans. on Database Systems*, 7(3):443–469, Sep 82.
- [495] J. J. King. Exploring the use of domain knowledge for query processing efficiency. Technical Report STAN-CS-79-781, Computer Science Department, Stanford University, 1979.
- [496] J. J. King. Quist: A system for semantik query optimization in relational databases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 510–517, 1981.
- [497] A. Klausner. *Multirelations in Relational Databases*. PhD thesis, Harvard University, Cambridge, 1986.
- [498] A. Klausner and N. Goodman. Multirelations – semantics and languages. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 251–258, 1985.
- [499] M. Klettke, L. Schneider, and A. Heuer. Metrics for XML Document Collections. In *EDBT Workshop XML-Based Data Management (XMLDM)*, pages 15–28, 2002.
- [500] A. Klug. Calculating constraints on relational expressions. *ACM Trans. on Database Systems*, 5(3):260–290, 1980.
- [501] A. Klug. Access paths in the “ABE” statistical query facility. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 161–173, 1982.
- [502] A. Klug. Equivalence of relational algebra and relational calculus query languages having aggregate functions. *Journal of the ACM*, 29(3):699–717, 1982.
- [503] A. Klug. On conjunctive queries containing inequalities. *Journal of the ACM*, 35(1):146–160, 1988. Written 1982 and published posthumously.
- [504] D. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 1997.
- [505] D. Knuth. *The Art of Computer Programming; Volume 3: Sorting and Searching*. Addison Wesley, 2000.

- [506] J. Koch. *Relationale Anfragen: Zerlegung und Optimierung*. Informatik-Fachberichte 101. Springer-Verlag, 1985.
- [507] J. Kollias. An estimate for seek time for batched searching of random or index sequential structured files. *The Computer Journal*, 21(2):132–133, 1978.
- [508] A. König and G. Weikum. Combining histograms and parametric curve fitting for feedback-driven query result-size estimation. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 423–434, 1999.
- [509] A. König and G. Weikum. Automatic tuning of data synopsis. *Information Systems*, 28:85–109, 2003.
- [510] B. König-Ries, S. Helmer, and G. Moerkotte. An experimental study on the complexity of left-deep join ordering problems for cyclic queries. Working Draft, 1994.
- [511] B. König-Ries, S. Helmer, and G. Moerkotte. An experimental study on the complexity of left-deep join ordering problems for cyclic queries. Technical Report 95-4, RWTH-Aachen, 1995.
- [512] R. Kooi. *The Optimization of Queries in Relational Databases*. PhD thesis, Case Western Reserve University, 1980.
- [513] F. Korn, H.V. Jagadish, and C. Faloutsos. Efficiently supporting ad hoc queries in large datasets of time sequences. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 289–300, 1997.
- [514] D. Kossmann. The state of the art in distributed query processing. *ACM Computing Surveys*, 32(4):422–469, 2000.
- [515] D. Kossmann and K. Stocker. Iterative dynamic programming: a new class of query optimization algorithms. *ACM Trans. on Database Systems*, 25(1):43–82, 2000.
- [516] N. Koudas, S. Muthukrishnan, and D. Srivastava. Optimal histograms for hierarchical range queries. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 196–204, 2000.
- [517] W. Kowarschick. Semantic optimization: What are disjunctive residues useful for? *SIGMOD Record*, 21(3):26–32, September 1992.
- [518] R. Krauthgamer, A. Mehta, V. Raman, and A. Rudra. Greedy list intersection. In *Proc. IEEE Conference on Data Engineering*, pages 1033–1042, 2008.
- [519] D. Kreher and D. Stinson. *Combinatorial Algorithms: Generation, Enumeration, and Search*. CRC Press, 1999.
- [520] R. Krishnamurthy, H. Boral, and C. Zaniolo. Optimization of nonrecursive queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 128–137, 1986.

- [521] A. Kumar and M. Stonebraker. The effect of join selectivities on optimal nesting order. *SIGMOD Record*, 16(1):28–41, 1987.
- [522] I. Kunen and D. Suciu. A scalable algorithm for query minimization. ask Dan for more information, year.
- [523] S. Kwan and H. Strong. Index path length evaluation for the research storage system of system r. Technical Report RJ2736, IBM Research Laboratory, San Jose, 1980.
- [524] M. Lacroix and A. Pirotte. Generalized joins. *SIGMOD Record*, 8(3):14–15, 1976.
- [525] L. Lakshman and R. Missaoui. Pushing semantics inside recursion: A general framework for semantic optimization of recursive queries. In *Proc. IEEE Conference on Data Engineering*, pages 211–220, 1995.
- [526] S. Lang and Y. Manolopoulos. Efficient expressions for completely and partly unsuccessful batched search of tree-structured files. *IEEE Trans. on Software Eng.*, 16(12):1433–1435, 1990.
- [527] S.-D. Lang, J. Driscoll, and J. Jou. A unified analysis of batched searching of sequential and tree-structured files. *ACM Trans. on Database Systems*, 14(4):604–618, 1989.
- [528] T. Lang, C. Wood, and I. Fernandez. Database buffer paging in virtual storage systems. *ACM Trans. on Database Systems*, 2(4):339–351, 1977.
- [529] R. Lancelotte and J.-P. Cheiney. Adapting relational optimisation technology for deductive and object-oriented declarative database languages. In *Proc. Int. Workshop on Database Programming Languages*, pages 322–336, 1991.
- [530] R. Lancelotte and P. Valduriez. Extending the search strategy in a query optimizer. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 363–373, 1991.
- [531] R. Lancelotte, P. Valduriez, and M. Zait. Optimization of object-oriented recursive queries using cost-controlled strategies. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 256–265, 1992.
- [532] R. Lancelotte, P. Valduriez, and M. Zäit. On the effectiveness of optimization search strategies for parallel execution. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 493–504, 1993.
- [533] R. Lancelotte, P. Valduriez, M. Ziane, and J.-P. Cheiney. Optimization of nonrecursive queries in OODBMs. In *Proc. Int. Conf. on Deductive and Object-Oriented Databases (DOOD)*, pages 1–21, 1991.
- [534] P.-A. Larson. Data reduction by partial preaggregation. In *Proc. IEEE Conference on Data Engineering*, pages 706–715, 2002.

- [535] P.-Å. Larson and H. Yang. Computing queries from derived relations. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 259–269, 1985.
- [536] Y.-N. Law, H. Wang, and C. Zaniolo. Query languages and data models for database sequences and data streams. In *VLDB*, pages 492–503, 2004.
- [537] E. Lawler. Sequencing jobs to minimize total weighted completion time subject to precedence constraints. *Ann. Discrete Math.*, 2:75–90, 1978.
- [538] B. S. Lee and G. Wiederhold. Outer joins and filters for instantiating objects from relational databases through views. *IEEE Trans. on Knowledge and Data Engineering*, 6(1):108–119, 1994.
- [539] C. Lee, C.-S. Shih, and Y.-H. Chen. Optimizing large join queries using a graph-based approach. *IEEE Trans. on Knowledge and Data Eng.*, 13(2):298–315, 2001.
- [540] M. K. Lee, J. C. Freytag, and G. M. Lohman. Implementing an interpreter for functional rules in a query optimizer. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 218–239, 1988.
- [541] M. K. Lee, J. C. Freytag, and G. M. Lohman. Implementing an optimizer for functional rules in a query optimizer. Technical Report RJ 6125, IBM Almaden Research Center, San Jose, CA, 1988.
- [542] T. Lehman and B. Lindsay. The Starburst long field manager. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 375–383, 1989.
- [543] K. Lehnert. *Regelbasierte Beschreibung von Optimierungsverfahren für relationale Datenbankabfragesprachen*. PhD thesis, Technische Universität München, 8000 München, West Germany, Dec 1988.
- [544] A. Lerner and D. Shasha. AQuery: query language for ordered data, optimization techniques, and experiments. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 345–356, 2003.
- [545] H. Leslie, R. Jain, D. Birdsall, and H. Yaghmai. Efficient search of multi-dimensional B-trees. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 710–719, 1995.
- [546] M. Levene and G. Loizou. Correction to null values in nested relational databases by m. roth and h. korth and a. silberschatz. *Acta Informatica*, 28(6):603–605, 1991.
- [547] M. Levene and G. Loizou. A fully precise null extended nested relational algebra. *Fundamenta Informaticae*, 19(3/4):303–342, 1993.
- [548] A. Levy, A. Mendelzon, and Y. Sagiv. Answering queries using views. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages ?–?, 1995.

- [549] A. Levy, A. Mendelzon, Y. Sagiv, and D. Srivastava. *Answering Queries Using Views*, pages 93–106. MIT Press, 1999.
- [550] A. Levy, A. Mendelzon, D. Srivastava, and Y. Sagiv. Answering queries using views. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 95–104, 1995.
- [551] A. Levy, I. Mumick, and Y. Sagiv. Query optimization by predicate move-around. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 96–107, 1994.
- [552] A.Y. Levy and I.S. Mumick. Reasoning with aggregation constraints. In P. Apers, M. Bouzeghoub, and G. Gardarin, editors, *Proc. European Conf. on Extending Database Technology (EDBT)*, Lecture Notes in Computer Science, pages 514–534. Springer, March 1996.
- [553] H. Lewis and C. Papadimitriou. *Elements of the Theory of Computation*. Prentice Hall, 1981.
- [554] C. Li, K. Chang, I. Ilyas, and S. Song. Ranksql: Query algebra and optimization for relational top-k queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 131–142, 2005.
- [555] D. Lichtenstein. Planar formulae and their uses. *SIAM J. Comp.*, 11(2):329–343, 1982.
- [556] J. Liebehenschel. Ranking and unranking of lexicographically ordered words: An average-case analysis. *J. of Automata, Languages, and Combinatorics*, 2:227–268, 1997.
- [557] J. Liebehenschel. Lexicographical generation of a generalized dyck language. Technical Report 5/98, University of Frankfurt, 1998.
- [558] J. Liebehenschel. *Lexikographische Generierung, Ranking und Unranking kombinatorischer Objekt: Eine Average-Case Analyse*. PhD thesis, University of Frankfurt, 2000.
- [559] H. Liefke. Horizontal query optimization on ordered semistructured data. In *ACM SIGMOD Workshop on the Web and Databases (WebDB)*, 1999.
- [560] L. Lim, M. Wang, S. Padmanabhan, J. Vitter, and R. Parr. XPathLearner: An on-line self-tuning Markov histogram for XML path selectivity estimation. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 442–453, 2002.
- [561] J. Lin and M. Ozsoyoglu. Processing OODB queries by O-algebra. In *Int. Conference on Information and Knowledge Management (CIKM)*, pages 134–142, 1996.
- [562] J. W. S. Liu. Algorithms for parsing search queries in systems with inverted file organization. *ACM Trans. on Database Systems*, 1(4):299–316, 1976.

- [563] M.-L. Lo and C. Ravishankar. Towards eliminating random I/O in hash joins. In *Proc. IEEE Conference on Data Engineering*, pages 422–429, 1996.
- [564] M. Lobo, L. Vandenberghe, S. Boyd, and H. Lebrete. Applications of second-order cone programming. *Linear Algebra and its Applications*, 284:192–228, 1998.
- [565] G. Lohman. Grammar-like functional rules for representing query optimization alternatives. Research report rj 5992, IBM, 1987.
- [566] G. Lohman. Heuristic method for joining relational data base tables. *IBM Technical Disclosure Bulletin*, 30(9):8–10, 1988.
- [567] G. M. Lohman. Grammar-like functional rules for representing query optimization alternatives. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 18–27, 1988.
- [568] D. Lomet. B-tree page size when caching is considered. *ACM SIGMOD Record*, 27(3):28–32, 1998.
- [569] R. Lorie. XRM - an extended (N-ary) relational model. Technical Report 320-2096, IBM Cambridge Scientific Center, 1974.
- [570] H. Lu and K.-L. Tan. On sort-merge algorithms for band joins. *IEEE Trans. on Knowledge and Data Eng.*, 7(3):508–510, Jun 1995.
- [571] W. S. Luk. On estimating block accesses in database organizations. *Communications of the ACM*, 26(11):945–947, 1983.
- [572] V. Lum, P. Yuen, and M. Dodd. Key-to-address transform techniques. *Communications of the ACM*, 14:228–239, 1971.
- [573] J. Lumbroso. An optimal cardinality estimation algorithm based on order statistics and its full analysis. In *Conf. on Analysis of Algorithms (AofA). Discrete Mathematics and Theoretical Computer Science*, pages 491–506, 2010.
- [574] G. Luo, J. Naughton, C. Ellmann, and M. Watzke. Toward a progress indicator for database queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 791–802, 2004.
- [575] G. Luo, J. Naughton, C. Ellmann, and M. Watzke. Increasing the accuracy and coverage of SQL progress indicators. In *Proc. IEEE Conference on Data Engineering*, pages 853–864, 2005.
- [576] D. Maier and D. S. Warren. Incorporating computed relations in relational databases. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 176–187, 1981.
- [577] M. Majster-Cederbaum. Elimination of redundant operations in relational queries with general selection operators. *Computing*, 34(4):303–323, 1984.

- [578] A. Makinouchi, M. Tezuka, H. Kitakami, and S. Adachi. The optimization strategy for query evaluation in RDB/V1. In *Proc. IEEE Conference on Data Engineering*, pages 518–529, 1981.
- [579] T. Malkemus, S. Padmanabhan, and B. Bhattacharjee. Predicate derivation and monotonicity detection in DB2 UDB. In *Proc. IEEE Conference on Data Engineering*, pages ?–?, 2005.
- [580] C. V. Malley and S. B. Zdonik. A knowledge-based approach to query optimization. In *Proc. Int. Conf. on Expert Database Systems*, pages 329–344, 1987.
- [581] N. Mamoulis. Efficient processing of joins on set-valued attributes. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 157–168, 2003.
- [582] M. Mannino and A. Rivera. An extensible model of selectivity estimation. *Information Sciences*, 49:225–247, 1989.
- [583] M. V. Mannino, P. Chu, and T. Sager. Statistical profile estimation in database systems. *ACM Computing Surveys*, 20(3):191–221, 1988.
- [584] Y. Manolopoulos and J. Kollias. Estimating disk head movement in batched searching. *BIT*, 28:27–36, 1988.
- [585] Y. Manolopoulos, J. Kollias, and M. Hatzopoulos. Sequential vs. binary batched search. *The Computer Journal*, 29(4):368–372, 1986.
- [586] Y. Manopoulos and J. Kollias. Performance of a two-headed disk system when serving database queries under the scan policy. *ACM Trans. on Database Systems*, 14(3):425–442, 1989.
- [587] S. March and D. Severence. The determination of efficient record segmentation and blocking factors for shared data files. *ACM Trans. on Database Systems*, 2(3):279–296, 1977.
- [588] R. Marek and E. Rahm. TID hash joins. In *Int. Conference on Information and Knowledge Management (CIKM)*, pages 42–49, 1994.
- [589] V. Markl, P. Haas, M. Kutsch, N. Meggido, U. Srivastava, and T. Tran. Consistent selectivity estimation via maximum entropy. *The VLDB Journal*, 16:55–76, 2007.
- [590] V. Markl, N. Megiddo, M. Kutsch, T. Tran, P. Haas, and U. Srivastava. Consistently estimating the selectivity of conjuncts of predicates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 373–384, 2005.
- [591] N. May, M. Brantner, A. Böhm, C.-C. Kanne, and G. Moerkotte. Index vs. navigation in XPath evaluation. In *Int. XML Database Symp. (XSym)*, pages 16–30, 2006.

- [592] N. May, S. Helmer, C.-C. Kanne, and G. Moerkotte. Xquery processing in natix with an emphasis on join ordering. In *Int. Workshop on XQuery Implementation, Experience and Perspectives (XIME-P)*, pages 49–54, 2004.
- [593] N. May, S. Helmer, and G. Moerkotte. Nested queries and quantifiers in an ordered context. Technical report, University of Mannheim, 2003.
- [594] N. May, S. Helmer, and G. Moerkotte. Quantifiers in XQuery. In *Proc. Int. Conf. on Web Information Systems Engineering (WISE)*, pages 313–316, 2003.
- [595] N. May, S. Helmer, and G. Moerkotte. Three Cases for Query Decorrelation in XQuery. In *Int. XML Database Symp. (XSym)*, pages 70–84, 2003.
- [596] N. May, S. Helmer, and G. Moerkotte. Nested queries and quantifiers in an ordered context. In *Proc. IEEE Conference on Data Engineering*, pages 239–250, 2004.
- [597] N. May, S. Helmer, and G. Moerkotte. Strategies for query unnesting in XML databases. *ACM Trans. on Database Systems*, 31(3):968–1013, 2006.
- [598] N. May and G. Moerkotte. Main memory implementations for binary grouping. In *Int. XML Database Symp. (XSym)*, pages 162–176, 2005.
- [599] J. McHugh and J. Widom. Query optimization for XML. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 315–326, 1999.
- [600] N. Megiddo and D. Modha. Outperforming LRU with an adaptive replacement cache algorithm. *IEEE Computer*, 37(4):58–65, 2004.
- [601] S. Melnik and H. Garcia-Molina. Divide-and-conquer algorithm for computing set containment joins. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 427–444, 2002.
- [602] S. Melnik and H. Garcia-Molina. Adaptive algorithms for set containment joins. *ACM Trans. on Database Systems*, 28(1):56–99, 2003.
- [603] T. Merrett and E. Otoo. Distribution models of relations. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 418–425, 1979.
- [604] T. H. Merrett, Y. Kambayashi, and H. Yasuura. Scheduling of page-fetches in join operations. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 488–498, 1981.
- [605] R. Van Meter. Observing the effects of multi-zone disks. In *USENIX Annual Technical Conference*, 1997.
- [606] G. Miklau and D. Suciu. Containment and equivalence of XPath expressions. *Journal of the ACM*, 51(1):2–45, 2002.

- [607] T. Milo and D. Suciu. Index structures for path expressions. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 277–295, 1999.
- [608] M. Minoux. *Mathematical Programming. Theory and Algorithms*. Wiley, 1986.
- [609] D. Mitchell, B. Selman, and H. Levesque. Hard and easy distributions of SAT problems. In *Proc. National Conference on Artificial Intelligence*, pages 459–465, 1992.
- [610] G. Mitchell. *Extensible Query Processing in an Object-Oriented Database*. PhD thesis, Brown University, Providence, RI 02912, 1993.
- [611] G. Mitchell, U. Dayal, and S. Zdonik. Control of an extensible query optimizer: A planning-based approach. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages ?–?, 1993.
- [612] G. Mitchell, S. Zdonik, and U. Dayal. An architecture for query processing in persistent object stores. In *Proc. of the Hawaiian Conf. on Computer and System Sciences*, pages 787–798, 1992.
- [613] G. Mitchell, S. Zdonik, and U. Dayal. *A. Dogac and M. T. Özsu and A. Biliris, and T. Sellis: Object-Oriented Database Systems*, chapter Optimization of Object-Oriented Queries: Problems and Applications, pages 119–146. NATO ASI Series F: Computer and Systems Sciences, Vol. 130. Springer, 1994.
- [614] G. Moerkotte. Small materialized aggregates: A light weight index structure for data warehousing. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 476–487, 1998.
- [615] G. Moerkotte. Constructing Optimal Bushy Trees Possibly Containing Cross Products for Order Preserving Joins is in P. Technical Report 12, University of Mannheim, 2003.
- [616] G. Moerkotte. Dp-counter analytics. Technical Report 2, University of Mannheim, 2006.
- [617] G. Moerkotte. Best approximation under a convex paranorm. Technical Report MA-08-07, University of Mannheim, 2008.
- [618] G. Moerkotte and T. Neumann. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy trees without cross products. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 930–941, 2006.
- [619] G. Moerkotte and T. Neumann. Dynamic programming strikes back. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 539–552, 2008.

- [620] G. Moerkotte and T. Neumann. Faster join enumeration for complex queries. In *Proc. IEEE Conference on Data Engineering*, pages 1430–1432, 2008.
- [621] G. Moerkotte and T. Neumann. Accelerating queries with group-by and join by groupjoin. In *Proc. of the VLDB Endowment (PVLDB)*, pages 843–851, 2011.
- [622] G. Moerkotte, T. Neumann, and G. Steidl. Preventing bad plans by bounding the impact of cardinality estimation errors. *Proc. of the VLDB Endowment (PVLDB)*, 2(1):982–993, 2009.
- [623] G. Moerkotte and G. Steidl. Best approximation with respect to a quotient functional. Technical Report X, University of Mannheim, 2008.
- [624] C. Mohan. Interactions between query optimization and concurrency control. In *Int. Workshop on RIDE*, 1992.
- [625] C. Mohan, D. Haderle, Y. Wang, and J. Cheng. Single table access using multiple indexes: Optimization, execution, and concurrency control techniques. In *Int. Conf. on Extended Database Technology (EDBT)*, pages 29–43, 1990.
- [626] J. Monk and R. Bonnett, editors. *Handbook of Boolean Algebras*. North Holland, 1989.
- [627] C. Monma and J. Sidney. Sequencing with series-parallel precedence constraints. *Math. Oper. Res.*, 4:215–224, 1979.
- [628] R. Morris. Counting large numbers of events in small registers. *Communications of the ACM*, 21(10):840–842, 1978.
- [629] T. Morzy, M. Matyasiak, and S. Salza. Tabu search optimization of large join queries. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 309–322, 1994.
- [630] L. Moses and R. Oakland. *Tables of Random Permutations*. Stanford University Press, 1963.
- [631] I. Mumick, S. Finkelstein, H. Pirahesh, and R. Ramakrishnan. Magic is relevant. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 247–258, 1990.
- [632] I. Mumick and H. Pirahesh. Implementation of magic sets in a relational database system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 103–114, 1994.
- [633] I. Mumick, H. Pirahesh, and R. Ramakrishnan. The magic of duplicates and aggregates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 264–277, 1990.

- [634] M. Muralikrishna and D.J. DeWitt. Equi-depth histograms for estimating selectivity factors for multi-dimensional queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 28–36, 1988.
- [635] B. Muthuswamy and L. Kerschberg. A detailed statistical model for relational query optimization. In *ACM Annual Conference - The range of computing: mid-80's perspective*, pages 439–447, 1985.
- [636] W. Myrvold and F. Ruskey. Ranking and unranking permutations in linear time. *Information Processing Letters*, 79(6):281–284, 2001.
- [637] R. Nakano. Translation with optimization from relational calculus to relational algebra having aggregate funktions. *ACM Trans. on Database Systems*, 15(4):518–557, 1990.
- [638] K. Seppia nd J. Barnes and C. Morris. A bayesian approach to database query optimization. *ORSA J. on Computing*, 5(4):410–419, 1993.
- [639] T. Neumann. Query simplification: graceful degradation for join-order optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 403–414, 2009.
- [640] T. Neumann and C. Galindo-Legaria. Taking the edge off cardinality estimation errors using incremental execution. In *Proc. der GI-Fachtagung Datenbanksysteme für Büro, Technik und Wissenschaft (BTW)*, pages 73–92, 2013.
- [641] T. Neumann and A. Kemper. Unnesting arbitrary queries. In *BTW*, pages 383–402, 2015.
- [642] T. Neumann and S. Michel. Smooth interpolation histograms with error guarantees. In *British National Conference on Databases (BNCOD)*, pages ?–?, 2008.
- [643] T. Neumann and G. Moerkotte. A combined framework for grouping and order optimization. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 960–971, 2004.
- [644] T. Neumann and G. Moerkotte. An efficient framework for order optimization. In *Proc. IEEE Conference on Data Engineering*, pages 461–472, 2004.
- [645] F. Neven and T. Schwentick. XPath containment in the presence of disjunction, dtlds, and variables. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 315–329, 2003.
- [646] S. Ng. Advances in disk technology: Performance issues. *IEEE Computer*, 31(5):75–81, 1998.
- [647] W. Ng and C. Ravishankar. Relational database compression using augmented vector quantization. In *Proc. IEEE Conference on Data Engineering*, pages 540–549, 1995.

- [648] S. Nigam and K. Davis. A semantic query optimization algorithm for object-oriented databases. In *Second International Workshop on Constraint Database Systems*, pages 329–344, 1997.
- [649] E. Omicienski. Heuristics for join processing using nonclustered indexes. *IEEE Trans. on Software Eng.*, 15(1):18–25, Feb. 1989.
- [650] P. O’Neil. *Database Principles, Programming, Performance*. Morgan Kaufmann, 1994.
- [651] P. O’Neil and D. Quass. Improved query performance with variant indexes. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 38–49, 1997.
- [652] K. Ono and G. Lohman. Extensible enumeration of feasible joins for relational query optimization. Technical Report RJ 6625, IBM Almaden Research Center, 1988.
- [653] K. Ono and G. Lohman. Measuring the complexity of join enumeration in query optimization. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 314–325, 1990.
- [654] J. A. Orenstein, S. Haradhvala, B. Margulies, and D. Sakahara. Query processing in the ObjectStore database system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 403–412, 1992.
- [655] J. A. Orenstein and F. A. Manola. PROBE spatial data modeling and query processing in an image database application. *IEEE Trans. on Software Eng.*, 14(5):611–629, 1988.
- [656] M. Ortega-Binderberger, K. Chakrabarti, and S. Mehrotra. An approach to integrating query refinement in sql. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 15–33, 2002.
- [657] S. Osborn. Identity, equality and query optimization. In *Proc. OODB*, 1989.
- [658] N. Ott. On the problem of removing redundant join operations. Technical Report TR 80.01.002, IBM Scientific Center, Heidelberg, 1980.
- [659] N. Ott and K. Horlaender. Removing redundant joins in queries involving views. Technical Report TR-82.03.003, IBM Scientific Center, Heidelberg, 1982.
- [660] G. Ozsoyoglu, V. Matos, and Z. M. Ozsoyoglu. Query processing techniques in the Summary-Table-by-Example database query language. *ACM Trans. on Database Systems*, 14(4):526–573, 1989.
- [661] G. Ozsoyoglu and H. Wang. A relational calculus with set operators, its safety and equivalent graphical languages. *IEEE Trans. on Software Eng.*, SE-15(9):1038–1052, 1989.

- [662] T. Özsu and J. Blakeley. *W. Kim (ed.): Modern Database Systems*, chapter Query Processing in Object-Oriented Database Systems, pages 146–174. Addison Wesley, 1995.
- [663] T. Özsu and D. Meechan. Finding heuristics for processing selection queries in relational database systems. *Information Systems*, 15(3):359–373, 1990.
- [664] T. Özsu and D. Meechan. Join processing heuristics in relational database systems. *Information Systems*, 15(4):429–444, 1990.
- [665] T. Özsu and P. Valduriez. *Principles of Distributed Database Systems*. Prentice-Hall, 1999.
- [666] T. Özsu and P. Valduriez. *Principles of Distributed Database Systems*. Springer, 2011.
- [667] T. Özsu and B. Yao. Evaluation of DBMSs using Xbench benchmark. Technical Report CS-2003-24, University of Waterloo, 2003.
- [668] P. Palvia. Expressions for batched searching of sequential and hierarchical files. *ACM Trans. on Database Systems*, 10(1):97–106, 1985.
- [669] P. Palvia and S. March. Approximating block accesses in database organizations. *Information Processing Letters*, 19:75–79, 1984.
- [670] S. Papadimitriou, H. Kitagawa, P. Gibbons, and C. Faloutsos. LOCI: Fast outlier detection using local correlation integral. In *ICDE*, pages 315–, 2003.
- [671] V. Papadimos and D. Maier. Mutant query plans. *Information & Software Technology*, 44(4):197–206, 2002.
- [672] Y. Papakonstantinou and V. Vianu. Incremental validation of XML documents. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 47–63, 2003.
- [673] S. Paparizos, S. Al-Khalifa, H. V. Jagadish, L. V. S. Lakshmanan, A. Nierman, D. Srivastava, and Y. Wu. Grouping in XML. In *EDBT Workshops*, pages 128–147, 2002.
- [674] S. Paparizos, S. Al-Khalifa, H. V. Jagadish, A. Niermann, and Y. Wu. A physical algebra for XML. Technical report, University of Michigan, 2002.
- [675] J. Paredaens and D. Van Gucht. Converting nested algebra expressions into flat algebra expressions. *ACM Trans. on Database Systems*, 17(1):65–93, Mar 1992.
- [676] C.-S. Park, M. Kim, and Y.-J. Lee. Rewriting OLAP queries using materialized views and dimension hierarchies in data. In *Proc. IEEE Conference on Data Engineering*, pages 515–523, 2001.

- [677] J. Patel, M. Carey, and M. Vernon. Accurate modeling of the hybrid hash join algorithm. In *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 56–66, 1994.
- [678] R. Patterson, G. Gibson, E. Ginting, D. Stodolsky, and J. Zelenka. Informed prefetching and caching. Technical Report CMU-CS-95-134, Carnegie Mellon University, 1995.
- [679] R. Patterson, G. Gibson, and M. Sayanarayanan. A status report on research in transparent informed prefetching. Technical Report CMU-CS-93-113, Carnegie Mellon University, 1993.
- [680] G. Paulley. *Exploiting Functional Dependence in Query Optimization*. PhD thesis, University of Waterloo, 2000.
- [681] G. Paulley and P.-A. Larson. Exploiting uniqueness in query optimization. In *Proc. IEEE Conference on Data Engineering*, pages 68–79, 1994.
- [682] J. Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison Wesley, 1984.
- [683] A. Pellenkoff, C. Galindo-Legaria, and M. Kersten. Complexity of transformation-based optimizers and duplicate-free generation of alternatives. Technical Report CS-R9639, CWI, 1996.
- [684] A. Pellenkoff, C. Galindo-Legaria, and M. Kersten. The complexity of transformation-based join enumeration. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 306–315, 1997.
- [685] A. Pellenkoff, C. Galindo-Legaria, and M. Kersten. Duplicate-free generation of alternatives in transformation-based optimizers. In *Proceedings of the International Conference on Database Systems for Advanced Applications (DASFAA)*, pages 117–124, 1997.
- [686] M. Pettersson. Linux x86 performance monitoring counters driver. perform internet search for this or similar tools.
- [687] M. Pezarro. A note on estimating hit ratios for direct-access storage devices. *The Computer Journal*, 19(3):271–272, 1976.
- [688] B. Piatetsky-Shapiro and C. Connell. Accurate estimation of the number of tuples satisfying a condition. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 256–276, 1984.
- [689] H. Pirahesh, J. Hellerstein, and W. Hasan. Extensible/rule-based query rewrite optimization in Starburst. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 39–48, 1992.
- [690] H. Pirahesh, T. Leung, and W. Hassan. A rule engine for query transformation in Starburst and IBM DB2 C/S DBMS. In *Proc. IEEE Conference on Data Engineering*, pages 391–400, 1997.

- [691] A. Pirotte. Fundamental and secondary issues in the design of non-procedural relational languages. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 239–250, 1979.
- [692] M. Piwowarski. Comments on batched searching of sequential and tree-structured files. *ACM Trans. on Database Systems*, 10(2):285–287, 1985.
- [693] N. Plyzotis and M. Garofalakis. XSKETCH synopsis for XML. In *Hellenic Data Management Symposium 02*, 2002.
- [694] S. L. Pollack. Conversion of limited entry decision tables to computer programs. *Communications of the ACM*, 8(11):677–682, 1965.
- [695] N. Polyzotis and M. Garofalakis. Statistical synopses for graph-structured XML databases. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 358–369, 2002.
- [696] N. Polyzotis and M. Garofalakis. Structure and value synopsis for XML data graphs. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 466–477, 2002.
- [697] N. Polyzotis, M. Garofalakis, and Y. Ioannidis. Selectivity estimation for XML twigs. In *Proc. IEEE Conference on Data Engineering*, pages 264–275, 2002.
- [698] V. Poosala and Y. Ioannidis. Selectivity estimation without the attribute value independence assumption. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 486–495, 1997.
- [699] V. Poosola and Y. Ioannidis. Estimation of query-result distribution and its application to parallel-join load balancing. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 448–459, 1996.
- [700] V. Poosola, Y. Ioannidis, P. Haas, and E. Shekita. Improved histograms for selectivity estimates of range predicates. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 294–305, 1996.
- [701] S. Pramanik and D. Ittner. Use of graph-theoretic models for optimal relational database accesses to perform joins. *ACM Trans. on Database Systems*, 10(1):57–74, 1985.
- [702] W. Press, S. Teukolsky, W. Vetterling, and B. Flannery. *Numerical Recipes*. Cambridge University Press, 2007. Third Edition.
- [703] X. Qian. Query folding. In *Proc. IEEE Conference on Data Engineering*, pages 48–55, 1996.
- [704] D. Quass and J. Widom. On-line warehouse view maintenance. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 393–404, 1997.

- [705] Y.-J. Qyang. A tight upper bound for the lumped disk seek time for the SCAN disk scheduling policy. *Information Processing Letters*, 54:355–358, 1995.
- [706] E. Rahm. *Mehrrechner-Datenbanksysteme: Grundlagen der verteilten und parallelen Datenbankverwaltung*. Addison-Wesley, 1994.
- [707] A. Rajaraman, Y. Sagiv, and J.D. Ullman. Answering queries using templates with binding patterns. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, PODS, 1995.
- [708] Bernhard Mitschang Ralf Rantza, Leonard D. Shapiro and Quan Wang. Algorithms and applications for universal quantification in relational databases. *Information Systems*, 28(1-2):3–32, 2003.
- [709] R. Ramakrishnan and J. Gehrke. *Database Management Systems*. McGraw Hill, 2000. 2nd Edition.
- [710] K. Ramamohanarao, J. Lloyd, and J. Thom. Partial-match retrieval using hashing descriptors. *ACM Trans. on Database Systems*, 8(4):552–576, 1983.
- [711] M. Ramanath, L. Zhang, J. Freire, and J. Haritsa. IMAX: Incremental maintenance of schema-based xXML statistics. In *Proc. IEEE Conference on Data Engineering*, pages 273–284, 2005.
- [712] K. Ramasamy, J. Naughton, and D. Maier. High performance implementation techniques for set-valued attributes. Technical report, University of Wisconsin, Wisconsin, 2000.
- [713] K. Ramasamy, J. Patel, J. Naughton, and R. Kaushik. Set containment joins: The good, the bad, and the ugly. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 351–362, 2000.
- [714] S. Ramaswamy and P. Kanellakis. OODB indexing by class division. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 139–150, 1995.
- [715] R. Rantza, L. Shapiro, B. Mitschang, and Q. Wang. Universal quantification in relational databases: A classification of data and algorithms. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 445–463, 2002.
- [716] J. Rao, B. Lindsay, G. Lohman, H. Pirahesh, and D. Simmen. Using EELs: A practical approach to outerjoin and antijoin reordering. Technical Report RJ 10203, IBM, 2000.
- [717] J. Rao, B. Lindsay, G. Lohman, H. Pirahesh, and D. Simmen. Using EELs: A practical approach to outerjoin and antijoin reordering. In *Proc. IEEE Conference on Data Engineering*, pages 595–606, 2001.

- [718] J. Rao and K. Ross. Reusing invariants: A new strategy for correlated queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 37–48, Seattle, WA, 1998.
- [719] S. Rao, A. Badia, and D. Van Gucht. Providing better support for a class of decision support queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 217–227, 1996.
- [720] G. Ray, J. Haritsa, and S. Seshadri. Database compression: A performance enhancement tool. In *COMAD*, 1995.
- [721] C. R’e and D. Suciu. Understanding cardinality estimation using entropy maximization. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 53–64, 2010.
- [722] C. R’e and D. Suciu. Understanding cardinality estimation using entropy maximization. *ACM Trans. on Database Systems*, 37(1):6, 2012.
- [723] D. Reiner and A. Rosenthal. Strategy spaces and abstract target machines for query optimization. *Database Engineering*, 5(3):56–60, Sept. 1982.
- [724] D. Reiner and A. Rosenthal. Querying relational views of networks. In W. Kim, D. Reiner, and D. Batory, editors, *Query Processing in Database Systems*, pages 109–124, 1985.
- [725] E. Reingold, J. Nievergelt, and N. Deo. *Combinatorial Algorithms: Theory and Practice*. Prentice Hall, 1977.
- [726] L. T. Reinwald and R. M. Soland. Conversion of limited entry decision tables to optimal computer programs I: minimum average processing time. *Journal of the ACM*, 13(3):339–358, 1966.
- [727] F. Reiss and T. Kanungo. A characterization of the sensitivity of query optimization to storage access cost parameters. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 385–396, 2003.
- [728] A. Reiter, A. Clute, and J. Tenenbaum. Representation and execution of searches over large tree-structured data bases. In *Proc. IFIP Congress, Booklet TA-3*, pages 134–144, 1971.
- [729] C. Rich, A. Rosenthal, and M. Scholl. Reducing duplicate work in relational join(s): A unified approach. In *CISMOD*, pages 87–102, 1993.
- [730] P. Richard. Evaluation of the size of a query expressed in relational algebra. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 155–163, 1981.
- [731] R. Van De Riet, A. Wassermann, M. Kersten, and W. De Jonge. High-level programming features for improving the efficiency of a relational database system. *ACM Trans. on Database Systems*, 6(3):464–485, 1981.
- [732] R. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.

- [733] D.J. Rosenkrantz and M.B. Hunt. Processing conjunctive predicates and queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 64–74, 1980.
- [734] A. Rosenthal. Note on the expected size of a join. *SIGMOD Record*, 11(4):19–25, 1981.
- [735] A. Rosenthal and U. S. Chakravarthy. Anatomy of a modular multiple query optimizer. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 230–239, 1988.
- [736] A. Rosenthal and C. Galindo-Legaria. Query graphs, implementing trees, and freely-reorderable outerjoins. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 291–299, 1990.
- [737] A. Rosenthal, S. Heiler, U. Dayal, and F. Manola. Traversal recursion: a practical approach to supporting recursive applications. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 166–167, 1986.
- [738] A. Rosenthal and P. Helman. Understanding and extending transformation-based optimizers. *IEEE Data Engineering*, 9(4):44–51, 1986.
- [739] A. Rosenthal and D. Reiner. An architecture for query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 246–255, 1982.
- [740] A. Rosenthal and D. Reiner. Extending the algebraic framework of query processing to handle outerjoins. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 334–343, 1984.
- [741] A. Rosenthal and D. Reiner. Querying relational views of networks. In W. Kim, D. Reiner, and D. Batory, editors, *Query Processing in Database Systems*, New York, 1984. Springer.
- [742] A. Rosenthal, C. Rich, and M. Scholl. Reducing duplicate work in relational join(s): a modular approach using nested relations. Technical report, ETH Zürich, 1991.
- [743] K. Ross. Conjunctive selection conditions in main memory. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 108–120, 2002.
- [744] K. Ross, D. Srivastava, and D. Chatziantoniou. Complex aggregation at multiple granularities. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 263–278, 1998.
- [745] M. Roth and S. Horn. Database compression. *SIGMOD Record*, 22(3):31–39, 1993.
- [746] M. Roth, H. Korth, and A. Silberschatz. Extended algebra and calculus for nested relational databases. *ACM Trans. on Database Systems*, 13(4):389–417, 1988. see also [868].

- [747] M. Roth, H. Korth, and A. Silberschatz. Null values in nested relational databases. *Acta Informatica*, 26(7):615–642, 1989.
- [748] M. Roth, H. Korth, and A. Silberschatz. Addendum to null values in nested relational databases. *Acta Informatica*, 28(6):607–610, 1991.
- [749] N. Roussopoulos. View indexing in relational databases. *ACM Trans. on Database Systems*, 7(2):258–290, 1982.
- [750] C. Ruemmler and J. Wilkes. An introduction to disk drive modeling. *IEEE Computer*, 27(3):17–29, 1994.
- [751] K. Runapongsa, J. Patel, H. Jagadish, and S. AlKhalifa. The michigan benchmark. Technical report, University of Michigan, 2002.
- [752] G. Sacco. Index access with a finite buffer. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 301–309, 1987.
- [753] G. Sacco and M. Schkolnick. A technique for managing the buffer pool in a relational system using the hot set model. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 257–262, 1982.
- [754] G. Sacco and M. Schkolnick. Buffer management in relational database systems. *ACM Trans. on Database Systems*, 11(4):473–498, 1986.
- [755] G. M. Sacco. Fragmentation: A technique for efficient query processing. *ACM Trans. on Database Systems*, 11(2):?–?, June 1986.
- [756] Y. Sagiv. *Optimization of queries in relational databases*. PhD thesis, Princeton University, 1978.
- [757] Y. Sagiv. *Optimization of Queries in Relational Databases*. UMI Research Press, Ann Arbor, Michigan, 1981.
- [758] Y. Sagiv. Quadratic algorithms for minimizing joins in restricted relational expressions. *SIAM J. Comput.*, 12(2):321–346, 1983.
- [759] Y. Sagiv and M. Yannakakis. Equivalence among expressions with the union and difference operators. *Journal of the ACM*, 27(4):633–655, 1980.
- [760] V. Sarathy, L. Saxton, and D. Van Gucht. Algebraic foundation and optimization for object based query languages. In *Proc. IEEE Conference on Data Engineering*, pages 113–133, 1993.
- [761] C. Sartiani. A general framework for estimating XML query cardinality. In *Int. Workshop on Database Programming Languages*, pages 257–277, 2003.
- [762] S. Savage. *The Flaw of Average*. John Wiley & Sons, 2009.
- [763] F. Scarcello, G. Greco, and N. Leone. Weighted hypertree decomposition and optimal query plans. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 210–221, 2004.

- [764] J. Scheible. A survey of storage options. *IEEE Computer*, 35(12):42–46, 2002.
- [765] H.-J. Schek and M. Scholl. The relational model with relation-valued attributes. *Information Systems*, 11(2):137–147, 1986.
- [766] W. Scheufele. *Algebraic Query Optimization in Database Systems*. PhD thesis, Universität Mannheim, 1999.
- [767] W. Scheufele and G. Moerkotte. Optimal ordering of selections and joins in acyclic queries with expensive predicates. Technical Report 96-3, RWTH-Aachen, 1996.
- [768] W. Scheufele and G. Moerkotte. On the complexity of generating optimal plans with cross products. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 238–248, 1997.
- [769] W. Scheufele and G. Moerkotte. Efficient dynamic programming algorithms for ordering expensive joins and selections. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 201–215, 1998.
- [770] J. Schindler, A. Ailamaki, and G. Ganger. Lachesis: Robust database storage management based on device-specific performance characteristics. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 706–717, 2003.
- [771] J. Schindler and G. Ganger. Automated disk drive characterization. Technical Report CMU-CS-99-176, Carnegie Mellon University, 1999.
- [772] J. Schindler, J. Griffin, C. Lumb, and G. Ganger. Track-aligned extents: Matching access patterns to disk drive characteristics. Technical Report CMU-CS-01-119, Carnegie Mellon University, 2001.
- [773] J. Schindler, J. Griffin, C. Lumb, and G. Ganger. Track-aligned extents: Matching access patterns to disk drive characteristics. In *Conf. on File and Storage Technology (FAST)*, pages 259–274, 2002.
- [774] A. Schmidt, M. Kersten, M. Windhouwer, and F. Waas. Efficient relational storage and retrieval of XML documents. In *ACM SIGMOD Workshop on the Web and Databases (WebDB)*, 2000.
- [775] A. Schmidt, F. Waas, M. Kersten, D. Florescu, I. Manolescu, M. Carey, and R. Busse. The XML Benchmark Project. Technical Report INS-R0103, CWI, Amsterdam, 2001.
- [776] J. W. Schmidt. Some high level language constructs for data of type relation. *ACM Trans. on Database Systems*, 2(3):247–261, 1977.
- [777] K. Schmidt and G. Trenkler. *Moderne Matrix Algebra*. Springer, 2006. Second Edition.

- [778] M. Scholl. Theoretical foundation of algebraic optimization utilizing unnormalized relations. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages ?–?, 1986.
- [779] T. Schwentick. XPath query containment. *ACM SIGMOD Record*, 33(1):101–109, 2004.
- [780] E. Sciore and J. Sieg. A modular query optimizer generator. In *Proc. IEEE Conference on Data Engineering*, pages 146–153, 1990.
- [781] B. Seeger. An analysis of schedules for performing multi-page requests. *Information Systems*, 21(4):387–407, 1996.
- [782] B. Seeger, P.-A. Larson, and R. McFadyen. Reading a set of disk pages. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 592–603, 1993.
- [783] A. Segev. Optimization of join operations in horizontally partitioned database systems. *ACM Trans. on Database Systems*, 11(1):48–80, 1986.
- [784] P. Selinger, M. Astrahan, D. Chamberlin, R. Lorie, and T. Price. Access path selection in a relational database management system. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 23–34, 1979.
- [785] T. Sellis. Intelligent caching and indexing techniques for relational database systems. *Information Systems*, 13(2):175–185, 1988.
- [786] T. Sellis. Intelligent caching and indexing techniques for relational database systems. *Information Systems*, 13(2):175–186, 1988.
- [787] T. Sellis. Multiple-query optimization. *ACM Trans. on Database Systems*, 13(1):23–52, 1988.
- [788] T. K. Sellis. Global query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 191–205, 1986.
- [789] M. Seltzer, P. Chen, and J. Ousterhout. Disk scheduling revisited. In *USENIX*, pages 313–323, 1990.
- [790] V. Sengar and J. Haritsa. PLASTIC: Reducing query optimization overheads through plan recycling. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, page 676, 2003.
- [791] P. Seshadri, J. Hellerstein, H. Pirahesh, T. Leung, R. Ramakrishnan, D. Srivastava, P. Stuckey, and S. Sudarshan. Cost-based optimization for magic: Algebra and implementation. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 435–446, 1996.
- [792] S. Setzer, G. Steidl, T. Teuber, and G. Moerkotte. Approximation related to quotient functionals. *Journal of Approximation Theory*, 162(3):545–558, 2010.

- [793] K. Sevcik. Data base system performance prediction using an analytical model. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 182–198, 1981.
- [794] D. Severance. A practitioner’s guide to data base compression. *Information Systems*, 8(1):51–62, 1983.
- [795] D. Severance and G. Lohman. Differential files: their application to the maintenance of large databases. *ACM Trans. on Database Systems*, 1(3):256–267, Sep 1976.
- [796] M. C. Shan. Optimal plan search in a rule-based query optimizer. In J. W. Schmidt, S. Ceri, and M. Missikoff, editors, *Proc. of the Intl. Conf. on Extending Database Technology*, pages 92–112, Venice, Italy, Mar 1988. Springer-Verlag, Lecture Notes in Computer Science No. 303.
- [797] J. Shanmugasundaram, R. Barr E. J. Shekita, M. J. Carey, B. G. Lindsay, H. Pirahesh, and B. Reinwald. Efficiently Publishing Relational Data as XML Documents. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 65–76, 2000.
- [798] L. Shapiro, D. Maier, P. Benninghoff, K. Billings, Y. Fan, K. Hatwal, Q. Wang, Y. Zhang, H.-M. Wu, and B. Vance. Exploiting upper and lower bounds in top-down query optimization. In *IDEAS*, pages 20–33, 2001.
- [799] L. Shapiro and A. Stephens. Bootstrap percolation, the schröder numbers and the n -kings problem. *SIAM J. Discr. Math.*, 4(2):275–280, 1991.
- [800] G. M. Shaw and S.B. Zdonik. Object-oriented queries: Equivalence and optimization. In *1st Int. Conf. on Deductive and Object-Oriented Databases*, pages 264–278, 1989.
- [801] G. M. Shaw and S.B. Zdonik. A query algebra for object-oriented databases. Tech. report no. cs-89-19, Department of Computer Science, Brown University, 1989.
- [802] G.M. Shaw and S.B. Zdonik. An object-oriented query algebra. In *2nd Int. Workshop on Database Programming Languages*, pages 111–119, 1989.
- [803] G.M. Shaw and S.B. Zdonik. A query algebra for object-oriented databases. In *Proc. IEEE Conference on Data Engineering*, pages 154–162, 1990.
- [804] E. Shekita and M. Carey. A performance evaluation of pointer-based joins. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 300–311, 1990.
- [805] E. Shekita, K.-L. Tan, and H. Young. Multi-join optimization for symmetric multiprocessors. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 479–492, 1993.

- [806] E. Shekita, H. Young, and K.-L. Tan. Multi-join optimization for symmetric multiprocessors. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 479–492, 1993.
- [807] P. Shenoy and H. Cello. A disk scheduling framework for next generation operating systems. In *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 44–55, 1998.
- [808] S. T. Shenoy and Z. M. Ozsoyoglu. A system for semantic query optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 181–195, 1987.
- [809] S. Sherman and R. Brice. Performance of a database manager in a virtual memory system. *ACM Trans. on Database Systems*, 1(4):317–343, 1976.
- [810] B. Shneiderman and V. Goodman. Batched searching of sequential and tree structured files. *ACM Trans. on Database Systems*, 1(3):208–222, 1976.
- [811] E. Shriver. *Performance Modeling for Realistic Storage Devices*. PhD thesis, University of New York, 1997.
- [812] E. Shriver, A. Merchant, and J. Wilkes. An analytical behavior model for disk drives with readahead caches and request reordering. In *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 182–191, 1998.
- [813] A. Shrufi and T. Topaloglou. Query processing for knowledge bases using join indices. In *Int. Conference on Information and Knowledge Management (CIKM)*, 1995.
- [814] K. Shwayder. Conversion of limited entry decision tables to computer programs — a proposed modification to Pollack’s algorithm. *Communications of the ACM*, 14(2):69–73, 1971.
- [815] M. Siegel, E. Sciore, and S. Salveter. A method for automatic rule derivation to support semantic query optimization. *ACM Trans. on Database Systems*, 17(4):53–600, 1992.
- [816] A. Silberschatz, H. Korth, and S. Sudarshan. *Database System Concepts*. McGraw Hill, 1997. 3rd Edition.
- [817] D. Simmen, C. Leung, and H. Pirahesh. Exploitation of uniqueness properties for the optimization of SQL queries using a 1-tuple condition. Research Report RJ 10008 (89098), IBM Almaden Research Division, Feb. 1996.
- [818] D. Simmen, E. Shekita, and T. Malkemus. Fundamental techniques for order optimization. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 57–67, 1996.

- [819] D. Simmen, E. Shekita, and T. Malkemus. Fundamental techniques for order optimization. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 625–62, 1996.
- [820] G. Slivinskas, C. Jensen, and R. Snodgrass. Bringing order to query optimization. *SIGMOD Record*, 13(2):5–14, 2002.
- [821] D. Smith and M. Genesereth. Ordering conjunctive queries. *Artificial Intelligence*, 26:171–215, 1985.
- [822] J. A. Smith. Sequentiality and prefetching in database systems. *ACM Trans. on Database Systems*, 3(3):223–247, 1978.
- [823] J. M. Smith and P. Y.-T. Chang. Optimizing the performance of a relational algebra database interface. *Communications of the ACM*, 18(10):568–579, 1975.
- [824] R. Sasic, J. Gu, and R. Johnson. The Unison algorithm: Fast evaluation of boolean expressions. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 1:456 – 477, 1996.
- [825] P. Spellucci. *Numerische Verfahren der Nichtlinearen Optimierung*. Birkhäuser, 1993.
- [826] N. Spyratos. An operational approach to data bases. In *Proc. ACM SIGMOD/SIGACT Conf. on Princ. of Database Syst. (PODS)*, pages 212–220, 1982.
- [827] D. Srivastava, S. Al-Khalifa, H. V. Jagadish, N. Koudas, J. Patel, and Y. Wu. Structural joins: A primitive for efficient XML query pattern matching. In *Proc. IEEE Conference on Data Engineering*, 2002.
- [828] D. Srivastava, S. Dar, J. Jagadish, and A. Levy. Answering queries with aggregation using views. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 318–329, 1996.
- [829] R. Stanley. *Enumerative Combinatorics, Volume I*, volume 49 of *Cambridge Studies in Advanced Mathematics*. Cambridge University Press, 1997.
- [830] H. Steenhagen. *Optimization of Object Query Languages*. PhD thesis, University of Twente, 1995.
- [831] H. Steenhagen, P. Apers, and H. Blanken. Optimization of nested queries in a complex object model. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 337–350, 1994.
- [832] H. Steenhagen, P. Apers, H. Blanken, and R. de By. From nested-loop to join queries in oodb. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 618–629, 1994.

- [833] H. Steenhagen, R. de By, and H. Blanken. Translating OSQL queries into efficient set expressions. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 183–197, 1996.
- [834] M. Steinbrunn, G. Moerkotte, and A. Kemper. Heuristic and randomized optimization for the join ordering problem. *The VLDB Journal*, 6(3):191–208, Aug. 1997.
- [835] M. Steinbrunn, K. Peithner, G. Moerkotte, and A. Kemper. Bypassing joins in disjunctive queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 228–238, 1995.
- [836] K. Stocker, D. Kossmann, R. Braumandl, and A. Kemper. Integrating semi-join reducers into state-of-the-art query processors. In *Proc. IEEE Conference on Data Engineering*, pages 575–584, 2001.
- [837] L. Stockmeyer and C. Wong. On the number of comparisons to find the intersection of two relations. Technical report, IBM Watson Research Center, 1978.
- [838] H. Stone and H. Fuller. On the near-optimality of the shortest-latency-time-first drum scheduling discipline. *Communications of the ACM*, 16(6):352–353, 1973.
- [839] M. Stonebraker. Inclusion of new types in relational database systems. In *Proc. IEEE Conference on Data Engineering*, pages ?–?, 1986.
- [840] M. Stonebraker, J. Anton, and E. Hanson. Extending a database system with procedures. *ACM Trans. on Database Systems*, 12(3):350–376, Sep 1987.
- [841] M. Stonebraker and P. Brown. *Object-Relational DBMSs, Tracking the Next Great Wave*. Morgan Kaufman, 1999.
- [842] M. Stonebraker et al. QUEL as a data type. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, Boston, MA, June 1984.
- [843] M. Stonebraker, A. Jhingran, J. Goh, and S. Potamios. On rules, procedures, caching and views in data base systems. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 281–290, 1990.
- [844] M. Stonebraker and L. A. Rowe. The design of postgres. In *Proc. of the 15th ACM SIGMOD*, pages 340–355, 1986.
- [845] M. Stonebraker, E. Wong, P. Kreps, and G. Held. The design and implementation of INGRES. *ACM Trans. on Database Systems*, 1(3):189–222, 1976.
- [846] D. Straube and T. Özsu. Access plan generation for an object algebra. Technical Report TR 90-20, Department of Computing Science, University of Alberta, June 1990.

- [847] D. Straube and T. Özsu. Queries and query processing in object-oriented database systems. Technical report, Department of Computing Science, University of Alberta, Edmonton, Alberta, Canada, 1990.
- [848] D. Straube and T. Özsu. Queries and query processing in object-oriented database systems. *ACM Trans. on Information Systems*, 8(4):387–430, 1990.
- [849] D. Straube and T. Özsu. Execution plan generation for an object-oriented data model. In *Proc. Int. Conf. on Deductive and Object-Oriented Databases (DOOD)*, pages 43–67, 1991.
- [850] D. D. Straube. *Queries and Query Processing in Object-Oriented Database Systems*. PhD thesis, The University of Alberta, Edmonton, Alberta, Canada, Dec 1990.
- [851] S. Subramanian and S. Venkataraman. Cost-based optimization of decision support queries using transient views. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 319–330, Seattle, WA, 1998.
- [852] D. Suciu. Query decomposition and view maintenance for query languages for unconstrained data. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 227–238, 1996.
- [853] N. Südkamp and V. Linnemann. Elimination of views and redundant variables in an SQL-like database language for extended NF² structures. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 302–313, 1990.
- [854] Wei Sun and Clement T. Yu. Automatic knowledge acquisition and maintenance for semantic query optimization. *IEEE Trans. on Knowledge and Data Engineering*, 1(3):362–375, 1989.
- [855] Wei Sun and Clement T. Yu. Semantic query optimization for tree and chain queries. *IEEE Trans. on Knowledge and Data Engineering*, 6(1):136–151, 1994.
- [856] K. Sutner, A. Satyanarayana, and C. Suffel. The complexity of the residual node connectedness reliability problem. *SIAM J. Comp.*, 20(1):149–155, 1991.
- [857] P. Svensson. On search performance for conjunctive queries in compressed, fully transposed ordered files. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 155–163, 1979.
- [858] A. Swami. *Optimization of Large Join Queries*. PhD thesis, Stanford University, 1989. Technical Report STAN-CS-89-1262.
- [859] A. Swami. Optimization of large join queries: Combining heuristics and combinatorial techniques. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 367–376, 1989.

- [860] A. Swami and A. Gupta. Optimization of large join queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 8–17, 1988.
- [861] A. Swami and B. Iyer. A polynomial time algorithm for optimizing join queries. Technical Report RJ 8812, IBM Almaden Research Center, 1992.
- [862] A. Swami and B. Iyer. A polynomial time algorithm for optimizing join queries. In *Proc. IEEE Conference on Data Engineering*, pages 345–354, 1993.
- [863] A. Swami and B. Schiefer. Estimating page fetches for index scans with finite LRU buffers. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 173–184, 1994.
- [864] A. Swami and B. Schiefer. On the estimation of join result sizes. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 287–300, 1994.
- [865] M. Switakowski, P. Boncz, and M. Zukowski. From cooperative scans to predictive buffer management. *Proc. of the VLDB Endowment (PVLDB)*, 5(12):1759–1770, 2012.
- [866] N. Talagala, R. Arpaci-Dusseau, and D. Patterson. Microbenchmark-based extraction of local and global disk characteristics. Technical Report UCB-CSD-99-1063, University of Berkeley, 2000.
- [867] K.-L. Tan and H. Lu. A note on the strategy space of multiway join query optimization problem in parallel systems. *SIGMOD Record*, 20(4):81–82, 1991.
- [868] A. Tansel and L. Garnett. On roth, korth, and silberschatz’s extended algebra and calculus for nested relational databases. *ACM Trans. on Database Systems*, 17(2):374–383, 1992.
- [869] Y. C. Tay. On the optimality of strategies for multiple joins. *Journal of the ACM*, 40(5):1067–1086, 1993.
- [870] T. Teorey and K. Das. Application of an analytical model to evaluate storage structures. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 9–19, 1976.
- [871] T. Teorey and T. Pinkerton. A comparative analysis of disk scheduling policies. In *Proc. of the AFIPS Fall Joint Computer Conference*, pages 1–11, 1972.
- [872] T. Teorey and T. Pinkerton. A comparative analysis of disk scheduling policies. *Communications of the ACM*, 15(3):177–184, 1972.
- [873] J. Teubner, T. Grust, and M. Van Keulen. Bridging the gap between relational and native XML storage with staircase join. *Grundlagen von Datenbanken*, pages 85–89, 2003.

- [874] N. Thaper, S. Guha, P. Indyk, and N. Koudas. Dynamic multidimensional histograms. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 428–439, 2002.
- [875] H. To, K. Chiang, and C. Shahabi. Entropy-based histograms for selectivity estimation. In *CIKM*, pages 1939–1948, 2013.
- [876] C. Tompkins. Machine attacks on problems whose variables are permutations. *Numerical Analysis (Proc. of Symposia in Applied Mathematics)*, 6, 1956.
- [877] R. Topor. Join-ordering is NP-complete. Draft, personal communication, 1998.
- [878] Transaction Processing Council (TPC). TPC Benchmark D. <http://www.tpc.org>, 1995.
- [879] Transaction Processing Performance Council, 777 N. First Street, Suite 600, San Jose, CA, USA. *TPC Benchmark R*, 1999. Revision 1.2.0. <http://www.tpc.org>.
- [880] P. Triantafillou, S. Christodoulakis, and C. Georgiadis. A comprehensive analytical performance model for disk devices under random workloads. *IEEE Trans. on Knowledge and Data Eng.*, 14(1):140–155, 2002.
- [881] O. Tsatalos, M. Solomon, and Y. Ioannidis. The GMAP: A versatile tool for physical data independence. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 367–378, 1994.
- [882] A. Tsois and T. Sellis. The generalized pre-grouping transformation: Aggregate-query optimization in the presence of dependencies. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 644–655, 2003.
- [883] K. Tufté and D. Maier. Aggregation and accumulation of XML data. *IEEE Data Engineering Bulletin*, 24(2):34–39, 2001.
- [884] K. Tzoumas, A. Deshpande, and C. Jensen. Efficiently adapting graphical models for selectivity estimation. *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, 22:3–27, 2013.
- [885] Überhuber. *Computer Numerik 2*. Springer, 1995.
- [886] J.D. Ullman. *Database and Knowledge Base Systems*, volume Volume 1. Computer Science Press, 1989.
- [887] J.D. Ullman. *Database and Knowledge Base Systems*, volume Volume 2. Computer Science Press, 1989.
- [888] J.D. Ullman. *Database and Knowledge Base Systems*. Computer Science Press, 1989.

- [889] D. Straube und T. Özsu. Query transformation rules for an object algebra. Technical Report TR 89-23, Department of Computing Science, University of Alberta, Sept. 1989.
- [890] T. Urhan, M. Franklin, and L. Amsaleg. Cost based query scrambling for initial delays. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 130–141, 1998.
- [891] M. Uysal, G. Alvarez, and A. Merchant. A modular analytical throughput model for modern disk arrays. In *MASCOTS*, pages 183–192, 2001.
- [892] P. Valduriez. Join indices. *ACM Trans. on Database Systems*, 12(2):218–246, 1987.
- [893] P. Valduriez and H. Boral. Evaluation of recursive queries using join indices. In *Proc. Int. Conf. on Expert Database Systems (EDS)*, pages 197–208, 1986.
- [894] P. Valduriez and S. Danforth. Query optimization in database programming languages. In *Proc. Int. Conf. on Deductive and Object-Oriented Databases (DOOD)*, pages 516–534, 1989.
- [895] L. Valiant. The complexity of computing the permanent. *Theoretical Comp. Science*, 8:189–201, 1979.
- [896] L. Valiant. The complexity of enumeration and reliability problems. *SIAM J. Comp.*, 8(3):410–421, 1979.
- [897] B. Vance. *Join-order Optimization with Cartesian Products*. PhD thesis, Oregon Graduate Institute of Science and Technology, 1998.
- [898] B. Vance and D. Maier. Rapid bushy join-order optimization with cartesian products. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 35–46, 1996.
- [899] S. L. Vandenberg and D. DeWitt. An algebra for complex objects with arrays and identity. Internal report, Computer Sciences Department, University of Wisconsin, Madison, WI 53706, USA, 1990.
- [900] S. L. Vandenberg and D. DeWitt. Algebraic support for complex objects with arrays, identity, and inheritance. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 158–167, 1991.
- [901] G. von Bülzingsloewen. Optimizing SQL queries for parallel execution. *ACM SIGMOD Record*, 1989.
- [902] G. von Bülzingsloewen. *Optimierung von SQL-Anfragen für parallele Bearbeitung (Optimization of SQL-queries for parallel processing)*. PhD thesis, University of Karlsruhe, 1990. in German.
- [903] G. von Bülzingsloewen. *SQL-Anfragen: Optimierung für parallele Bearbeitung*. FZI-Berichte Informatik. Springer, 1991.

- [904] F. Waas and A. Pellenkoft. Probabilistic bottom-up join order selection – breaking the curse of NP-completeness. Technical Report INS-R9906, CWI, 1999.
- [905] F. Waas and A. Pellenkoft. Join order selection - good enough is easy. In *BNCOD*, pages 51–67, 2000.
- [906] H. Wang and K. Sevcik. Histograms based on the minimum description length principle. *The VLDB Journal*, 17:419–442, 2008.
- [907] J. Wang, J. Li, and G. Butler. Implementing the PostgreSQL query optimizer within the OPT++ framework. In *Asia-Pacific Software Engineering Conference (APSEC)*, pages 262–272, 2003.
- [908] J. Wang, M. Maher, and R. Topor. Rewriting unions of general conjunctive queries using views. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 52–69, 2002.
- [909] W. Wang, H. Jiang, H. Lu, and J. Yu. Containment join size estimation: Models and methods. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 145–156, 2003.
- [910] W. Wang, H. Jiang, H. Lu, and J. Yu. Bloom histogram: Path selectivity estimation for xml data with updates. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 240–251, 2004.
- [911] X. Wang and M. Cherniack. Avoiding ordering and grouping in query processing. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 826–837, 2003.
- [912] S. Waters. File design fallacies. *The Computer Journal*, 15(1):1–4, 1972.
- [913] S. Waters. Hit ratio. *Computer Journal*, 19(1):21–24, 1976.
- [914] G. Watson. *Approximation Theory and Numerical Methods*. Addison-Wesley, 1980.
- [915] H. Wedekind and G. Zörntlein. Prefetching in realtime database applications. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 215–226, 1986.
- [916] M. Wedekind. On the selection of access paths in a database system. In J. Klimbie and K. Koffeman, editors, *IFIP Working Conference Data Base Management*, pages 385–397, Amsterdam, 1974. North-Holland.
- [917] G. Weikum. Set-oriented disk access to large complex objects. In *Proc. IEEE Conference on Data Engineering*, pages 426–433, 1989.
- [918] G. Weikum, B. Neumann, and H.-B. Paul. Konzeption und Realisierung einer mengenorientierten Seitenschnittstelle zum effizienten Zugriff auf komplexe Objekte. In *Proc. der GI-Fachtagung Datenbanksysteme für Büro, Technik und Wissenschaft (BTW)*, pages 212–230, 1987.

- [919] T. Westmann. *Effiziente Laufzeitsysteme für Datenlager*. PhD thesis, University of Mannheim, 2000.
- [920] T. Westmann, D. Kossmann, S. Helmer, and G. Moerkotte. The implementation and performance of compressed databases. Technical Report 03/98, University of Mannheim, 1998.
- [921] T. Westmann, D. Kossmann, S. Helmer, and G. Moerkotte. The implementation and performance of compressed databases. *SIGMOD Record*, 29(3):55–67, 2000.
- [922] T. Westmann and G. Moerkotte. Variations on grouping and aggregations. Technical Report 11/99, University of Mannheim, 1999.
- [923] K.-Y. Whang, A. Malhotra, G. Sockut, and L. Burns. Supporting universal quantification in a two-dimensional database query language. In *Proc. IEEE Conference on Data Engineering*, pages 68–75, 1990.
- [924] K.-Y. Whang, B. Vander-Zanden, and H. Taylor. A linear-time probabilistic counting algorithm for database applications. *ACM Trans. on Database Systems*, 15(2):208–229, 1990.
- [925] K.-Y. Whang, G. Wiederhold, and D. Sagalowicz. Estimating block accesses in database organizations: A closed noniterative formula. *Communications of the ACM*, 26(11):940–944, 1983.
- [926] N. Wilhelm. A general model for the performance of disk systems. *Journal of the ACM*, 24(1):14–31, 1977.
- [927] D. E. Willard. Efficient processing of relational calculus queries using range query theory. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 164–175, 1984.
- [928] C. Williams and T. Hogg. Using deep structure to locate hard problems. In *Proc. National Conference on Artificial Intelligence*, pages 472–477, 1992.
- [929] J. Wolf, R. Iyer, K. Pattipati, and J. Turek. Optimal buffer partitioning for the nested block join algorithm. In *Proc. IEEE Conference on Data Engineering*, pages 510–519, 1991.
- [930] C. Wong. Minimizing expected head movement in one-dimensional and two-dimensional mass storage systems. *ACM Computing Surveys*, 12(2):167–177, 1980.
- [931] F. Wong and K. Youssefi. Decomposition – a strategy for query processing. *ACM Trans. on Database Systems*, 1(3):223–241, 1976.
- [932] H. Wong and J. Li. Transposition algorithms on very large compressed databases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 304–311, 1986.

- [933] P. Wood. On the equivalence of XML patterns. In *CL 2000*, 2000.
- [934] P. Wood. Minimizing simple XPath expressions. In *Int. Workshop on Database Programming Languages*, pages 13–18, 2001.
- [935] P. Wood. Containment for XPath fragments under dtd constraints. In *Proc. Int. Conf. on Database Theory (ICDT)*, pages 300–314, 2003.
- [936] W. A. Woods. Procedural semantics for question-answering systems. In *FJCC (AFIPS Vol. 33 Part I)*, pages 457–471, 1968.
- [937] B. Worthington, G. Ganger, Y. Patt, and J. Wilkes. Scheduling algorithms for modern disk drives. In *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 241–251, 1994.
- [938] B. Worthington, G. Ganger, Y. Patt, and J. Wilkes. On-line extraction of SCSI disk drive parameters. In *Proc. ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 146–156, 1995.
- [939] B. Worthington, G. Ganger, Y. Patt, and J. Wilkes. On-line extraction of SCSI disk drive parameters. Technical Report CSE-TR-323-96, University of Michigan, 1996.
- [940] M.-C. Wu. Query optimization for selections using bitmaps. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 227–238, 1999.
- [941] Y. Wu, J. Patel, and H.V. Jagadish. Estimating answer sizes for XML queries. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 590–608, 2002.
- [942] Y. Wu, J. Patel, and H.V. Jagadish. Estimating answer sizes for XML queries. *Information Systems*, 28(1-2):33–59, 2003.
- [943] Z. Xie. Optimization of object queries containing encapsulated methods. In *Proc. 2nd. Int. Conf. on Information and Knowledge Management*, pages 451–460, 1993.
- [944] Z. Xie and J. Han. Join index hierarchies for supporting efficient navigation in object-oriented databases. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 522–533, 1994.
- [945] G. D. Xu. Search control in semantic query optimization. Technical Report 83-09, COINS, University of Massachusetts, Amherst, MA, 1983.
- [946] W. Yan and P.-A. Larson. Performing group-by before join. In *Proc. IEEE Conference on Data Engineering*, pages 89–100, 1994.
- [947] W. Yan and P.-A. Larson. Eager aggregation and lazy aggregation. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 345–357, 1995.
- [948] H. Yang and P.-A. Larson. Query transformation for PSJ-queries. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 245–254, 1987.

- [949] J. Yang, K. Karlapalem, and Q. Li. Algorithms for materialized view design in data warehousing environment. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 136–145, 1997.
- [950] Qi Yang. Computation of chain queries in distributed database systems. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 348–355, 1994.
- [951] M. Yannakakis. Algorithms for acyclic database schemes. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 82–94, 1981.
- [952] B. Yao and T. Özsu. XBench – A Family of Benchmarks for XML DBMSs. Technical Report CS-2002-39, University of Waterloo, 2002.
- [953] B. Yao, T. Özsu, and N. Khandelwal. Xbench benchmark and performance testing of XML DBMSs. In *Proc. IEEE Conference on Data Engineering*, pages 621–632, 2004.
- [954] S. B. Yao. Approximating block accesses in database organizations. *Communications of the ACM*, 20(4):260–261, 1977.
- [955] S. B. Yao. An attribute based model for database access cost analysis. *ACM Trans. on Database Systems*, 2(1):45–67, 1977.
- [956] S. B. Yao and D. DeJong. Evaluation of database access paths. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 66–77, 1978.
- [957] S.B. Yao. Optimization of query evaluation algorithms. *ACM Trans. on Database Systems*, 4(2):133–155, 1979.
- [958] S.B. Yao, A.R. Hevner, and H. Young-Myers. Analysis of database system architectures using benchmarks. *IEEE Trans. on Software Eng.*, SE-13(6):709–725, 1987.
- [959] J. Yiannis and J. Zobel. Compression techniques for fast external sorting. *VLDB Journal*, 16(2):269–291, 2007.
- [960] Y. Yoo and S. Lafortune. An intelligent search method for query optimization by semijoins. *IEEE Trans. on Knowledge and Data Eng.*, 1(2):226–237, June 1989.
- [961] M. Yoshikawa, T. Amagasa, T. Shimura, and S. Uemura. Xrel: A path-based approach to storage and retrieval of XML documents using relational databases. *ACM Transactions on Internet Technology*, 1(1):110–141, June 2001.
- [962] K. Youssefi and E. Wong. Query processing in a relational database management system. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 409–417, 1979.
- [963] C. T. Yu, W. S. Luk, and M. K. Siu. On the estimation of the number of desired records with respect to a given query. *ACM Trans. on Database Systems*, 3(1):41–56, 1978.

- [964] L. Yu and S. L. Osborn. An evaluation framework for algebraic object-oriented query models. In *Proc. IEEE Conference on Data Engineering*, 1991.
- [965] X. Yu, N. Koudas, and C. Zuzarte. HASE: a hybrid approach to selectivity estimation for conjunctive predicates. In *Proc. of the Int. Conf. on Extending Database Technology (EDBT)*, pages 460–477, 2006.
- [966] X. Yu, C. Zuzarte, and K. Sevcik. Towards estimating the number of distinct value combinations for a set of attributes. In *CIKM*, pages 656–663, 2005.
- [967] J. Zahorjan, B. Bell, and K. Sevcik. Estimating block transfers when record access probabilities are non-uniform. *Information Processing Letters*, 16(5):249–252, 1983.
- [968] B. T. Vander Zander, H. M. Taylor, and D. Bitton. Estimating block accesses when attributes are correlated. In *Proc. Int. Conf. on Very Large Data Bases (VLDB)*, pages 119–127, 1986.
- [969] S. Zdonik and G. Mitchell. ENCORE: An object-oriented approach to database modelling and querying. *IEEE Data Engineering Bulletin*, 14(2):53–57, June 1991.
- [970] N. Zhang, V. Kacholia, and T. Özsu. A succinct physical storage scheme for efficient evaluation of path queries in XML. In *Proc. IEEE Conference on Data Engineering*, pages 54–65, 2004.
- [971] N. Zhang and T. Özsu. Optimizing correlated path expressions in XML languages. Technical Report CS-2002-36, University of Waterloo, 2002.
- [972] Y. Zhao, P. Deshpande, J. Naughton, and A. Shukla. Simultaneous optimization and evaluation of multiple dimensional queries. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, pages 271–282, 1998.
- [973] Y. Zhuge, H. Garcia-Molina, J. Hammer, and J. Widom. View maintenance in a warehouse environment. In *Proc. of the ACM SIGMOD Conf. on Management of Data*, 1995.

Appendix E

ToDo

- size of a query in rel alg: [730]
- [908]
- Integrating Buffer Issues into Query Optimization: [211, 473]
- Integrating concurrency control issues into query optimization: [624, 625]
- [101]
- where do we put "counting page accesses"?
- control, A^* , ballooning: [611, 610]
- Bypass Plans
- Properties (rather complete list, partial ordering, plan independent properties: store them somewhere else (dpstructure or memostructure))
- describe prep-phase of plan generator
- reuse plans: [790]
- estimating query compilation time: [444]
- cost model [793]
- sensitivity of QO to storage access cost parameters [727] (and join selectivities on join order: [521] [papier ist nicht ernst zu nehmen])
- magic set and semi join reducers [79, 81, 80, 175, 341, 633, 631, 633, 632, 791, 836, 960]
- join indexes and clustering tuples of different relations with 1:n relationship [230, 401, 892, 893, 813]
- B-Trees with space filling curves (Bayer)
- Prefetching [915]

- feedback to optimizer [508]
- compression [26, 55, 167, 207, 259, 258, 335, 342] [647, 720, 745, 794, 795, 857, 920, 932]
- semantic QO SQO: [1, 82, 172, 332, 365, 495, 496, 517, 525] [648, 656, 657, 681, 808, 815, 817, 945] [552]
- join processing with nonclustered indexes: [649]
- join+buffer: [929]
- removal/elimination of redundant joins [659, 853]
- benchmark(ing): Gray Book: [366]; papers: [98, 108, 667, 751, 775, 958, 952, 953]
- dynamic qo: [671] [29, 890] [43] [474]
- unnesting: [675, 718, 641]
- prefetching: [679, 678, 822, 915]
- Starburst: [689, 690]
- BXP: [107, 260, 310, 375, 397, 437, 491, 694, 726, 814, 821, 824, 490]
BXP complexity: [77] BXP var infl: [475]
- joins: [701]
- query folding: [703]
- quantification: [114, 113, 185, 186, 715, 708, 923] [467]
- outerjoins: [85, 88, 218, 308, 299, 298, 717, 736]
- partial match + hashing: [710]
- OODB indexing by class division: [193, 714]
- decision support [719]
- tree structured databases: [728]
- Rosenthal: [739, 723, 740, 741, 724, 735, 738, 737]
- conj. queries [733]
- aggregation/(generalized proj): [120, 193, 300, 744] [381, 382, 405]
- do nest/unnest to optimize duplicate work: [742]
 $e_1 \bowtie_{A_1=A_2} e_2 \equiv \mu_g(e_1 \bowtie_{A_1=A_2} \Gamma_{g:=A_2;id}(e_2))$
- join size: [734]
- fragmentation: [755]

- eqv: [17, 18]
- alg eqvs union/difference: [759] [908]
- other sagiv: [757, 758]
- bayesian approach to QO: [638]
- cache query plans: [790]
- joins for horizontally fragmentation: [783]
- partitioning: [56, 106, 395, 478, 651]
- MQO: [25, 140, 138, 788, 787, 972]
- indexing+caching: [786]
- rule-based QO: [796, 66, 67, 295]
- rule-based IRIS: [229]
- cost: [837] [891]
- search space: [867], join ordering: [869]
- access path: [132, 916, 956, 99]
- eff aggr: [296] [922]
- misc: [927] [9] [13]
- access paths: bitmaps [940]
- dist db: [37, 38, 97, 216, 950] Donald's state of the art: [514]
- [147, 148]
- eqv: bags [21, 221]
- eqvs old: [22]
- DB2: norwegian analysis: [31]
- nested: [46]
- Genesis/Praire/Batory: [57, 61, 60, 62, 217]
- eqvs OO: [68, 69]
- dupelim: [91]
- (generalized) division: [130, 215, 355, 344]
- early aggregation
- chunks-wise processing [231, 351]

- temporal intersection join: [378]
- 2nd ord sig: Güting: [386]
- classics: [392]
- smallest first: [396]
- Hwang/Yu: [436]
- Kambayashi: [477]
- Koch [506], Lehnert [543]
- I/O cost reduction for (hash) joins: [563, 604]
- dist nest: [265]
- band join: [570]
- Donovan (TODS 76,1,4) Decision Support: [247]
- whenever materialize something (sort, hash join, etc) compute min/max of some attributes and use these as additional selection predicates
- determine optimal page access sequence and buffer size to access pairs (x,y) of pages where join partners of one relation lie on x and of the other on y (Fotouhi, Pramanik [292], Merret, Kambayashi, Yasuura [604], Omiecinski [649], Pramanik, Ittner [701], Chan, Ooi [142])
- buffer mgmt: [865]
- Scarcello, Greco, Leone: [763]
- Sigmod05:
 - proactive reoptimization [45]
 - robust query optimizer [44]
 - stacked indexed views [225]
 - NF²-approach to processing nested sql queries [123]
 - efficient computatio of multiple groupby queries [169]
- LOCI: [670]
- Wavelet synopses: [321]
- Progress Indicators: [153, 574, 575]
- PostgresExperience: [907]
- Chaudhuri sigmod 05: [44]
- Cesar on testing sql server [329]

- Bruno, Galindo-Legaria, Joshi [112] which is like GOO with the two additional techniques of pushing partial plans down and pulling partial plans up whenever a new join is added.
- incremental evaluation to justify θ, q -acceptability [640]
- Execution Strategies for sql subqueries by Cesar [263]
- PIVOT, UNPIVOT: optimization/execution in sql server by Cesar [214]
- statistical views by cesar [301]
- multiway joins [426]
- [518]: heuristics to order index anding.
- [875]: Entropy-based histograms for selectivity estimation
- [884]: efficiently adapting graphical models for selectivity estimation
- [411]: multidim selectivity estimation via kernel density
- [?]: effective and complete discovery of order-dependencies via set-based axiomatization
- section joinorder/top-down: pit, pruning [798]
- chapter plan generation
- chapter unnesting Oracle: Coalescing [72];
- benchmarking query optimizers: [?]
- Herodotou, Borisov, Babu: Query Optimization Techniques for Partitioned Tables
- Al-Kateb, Sinclair, Au, Ballinger: Hybrid Row-Column Partitioning in Teradata
- Antova, El-Helw, Soliman, Gu, ZPetrooulos, Waas: Optimizing Queries over Partitioned Tables in MPP Systems.
- Chen, Yi: Two-Level Sampling for Join Size Estimation [?]
- check: joinorder chapter: ccp: Fig3: numbers for #ccp for chain for n=20, formel: $n^n - i2^n$
- cardinality estimation: Shekelyan [?]
- cardinality estimation: Kyuseok Shim [?]
- heuristics join ordering [805]
- predicate inference [854]
- merge min/max subqueries if possible using tableau equivalence [14]
- Union-All-Duplicate Operator (UAD) [16]